

Edeviser Complete Codebase Volume 02

Generated: 8/1/2026 | Author: Edeviser AI Reviewer

FILE: src/__tests__/unit/curriculumMatrixCoverage.test.tsx

TYPE: .tsx | LINES: 1-103 | SHA256: 39bf7848a7b883407cee5e193071a726256c4461c63dbda20a4b9092210f6514

[illegible]

```

94 |     mockData.matrix.plo1!.c2 = {
95 |       status: "red",
96 |       cloCount: 1,
97 |       attainmentPercent: 30,
98 |     };
99 |     render(<CurriculumMatrix programId="p1" coverageLabel="Coverage" />);
100 |     expect(screen.getByText("0%")).toBeInTheDocument();
101 |   });
102 | });
103 |

```

FILE: src/___tests___/unit/customAnimations.test.ts

TYPE: .ts | LINES: 1-59 | SHA256: eb0b5d82830369ec6c664ff7ba25ef744d2a9dc75133384b218e6665d38e5fc2

```

1 | import { describe, it, expect } from "vitest";
2 | import { readFileSync } from "fs";
3 | import { resolve } from "path";
4 |
5 | const cssContent = readFileSync(resolve(__dirname, "../../index.css"), "utf-8");
6 |
7 | describe("Custom animation utility classes", () => {
8 |   const expectedClasses = [
9 |     "animate-shimmer",
10 |    "animate-xp-pulse",
11 |    "animate-badge-pop",
12 |    "animate-float",
13 |    "animate-streak-flame",
14 |    "animate-fade-in-up",
15 |    "animate-node-unlock",
16 |    "animate-mystery-reveal",
17 |   ];
18 |
19 |   it.each(expectedClasses)("defines .%s utility class", (cls) => {
20 |     expect(cssContent).toContain(`.${cls}`);
21 |   });
22 |
23 |   it("includes all animation classes in the reduced-motion media query", () => {
24 |     const reducedMotionMatch = cssContent.match(
25 |       /@media\s*(prefers-reduced-motion:\s*reduce\s*\{\{[\^]*\{[\^]*\}\}\s*\})/s
26 |     );
27 |     expect(reducedMotionMatch).not.toBeNull();
28 |     const reducedMotionBlock = reducedMotionMatch![1]!;
29 |     for (const cls of expectedClasses) {
30 |       expect(reducedMotionBlock).toContain(`.${cls}`);
31 |     }
32 |   });
33 |
34 |   it("defines @keyframes for node-unlock and mystery-reveal", () => {
35 |     expect(cssContent).toContain("@keyframes node-unlock");
36 |     expect(cssContent).toContain("@keyframes mystery-reveal");
37 |   });
38 |
39 |   it("keeps celebration animations \"d600ms\", () => {
40 |     // node-unlock: 0.5s = 500ms
41 |     // mystery-reveal: 0.6s = 600ms
42 |     const nodeUnlockMatch = cssContent.match(
43 |       /\.animate-node-unlock\s*\{[\^]*\}animation:\s*node-unlock\s+(\[d.\+]s)/
44 |     );
45 |     const mysteryRevealMatch = cssContent.match(
46 |       /\.animate-mystery-reveal\s*\{[\^]*\}animation:\s*mystery-reveal\s+(\[d.\+]s)/
47 |     );
48 |
49 |     expect(nodeUnlockMatch).not.toBeNull();
50 |     expect(mysteryRevealMatch).not.toBeNull();
51 |
52 |     const nodeUnlockDuration = parseFloat(nodeUnlockMatch![1]!) * 1000;
53 |     const mysteryRevealDuration = parseFloat(mysteryRevealMatch![1]!) * 1000;
54 |
55 |     expect(nodeUnlockDuration).toBeLessThanOrEqual(600);
56 |     expect(mysteryRevealDuration).toBeLessThanOrEqual(600);
57 |   });
58 | });
59 |

```

FILE: src/___tests___/unit/dataCompliance.test.ts

TYPE: .ts | LINES: 1-223 | SHA256: 1ecc622d233b61aa8ebd915cd22aa2e6f99c82cd9cad00bad1b04beeb63d91a2

```

1 | /**
2 |  * FERPA/GDPR Compliance Validation Tests
3 |  *
4 |  * Validates that the platform's logging, monitoring, and data handling
5 |  * configurations prevent PII leakage and comply with data protection requirements.
6 |  *
7 |  * Feature: edeviser-platform, Task 42.7
8 |  * Validates: Requirements 51.1, 51.5
9 |  */
10 | // @vitest-environment happy-dom
11 | import { describe, it, expect } from "vitest";
12 | import { scrubPII } from "@lib/sentry";
13 |
14 | // -----
15 | // Audit Logger: PII field allowlist validation
16 | // -----
17 |
18 | // Re-declare the allowlist here to test against – if the source changes,
19 | // this test will catch the drift.
20 | const EXPECTED_AUDIT_ALLOWLIST: Record<string, string[]> = {
21 |   user: ["role", "is_active", "program_id"],
22 |   grade: ["submission_id", "score_percent", "total_score"],
23 |   assignment: ["title", "course_id", "due_date", "status"],
24 |   course: ["name", "code", "program_id", "semester_id", "teacher_id"],
25 |   program: ["name", "code", "institution_id", "coordinator_id"],
26 |   ilo: ["title", "type", "institution_id"],
27 |   plo: ["title", "type", "program_id"],

```

```

28 |   clo: ["title", "type", "course_id", "blooms_level"],
29 |   outcome_mapping: ["source_outcome_id", "target_outcome_id", "weight"],
30 |   bonus_xp_event: ["name", "multiplier", "start_date", "end_date"],
31 |   enrollment: ["student_id", "course_id", "status"],
32 | };
33 |
34 | const PII_FIELDS = [
35 |   "email",
36 |   "full_name",
37 |   "avatar_url",
38 |   "password",
39 |   "phone",
40 |   "address",
41 | ];
42 |
43 | describe("FERPA/GDPR Compliance", () => {
44 |   describe("Audit Logger PII Protection", () => {
45 |     it("no PII fields appear in any audit allowlist", () => {
46 |       for (const [entityType, allowedFields] of Object.entries(
47 |         EXPECTED_AUDIT_ALLOWLIST
48 |       )) {
49 |         for (const piiField of PII_FIELDS) {
50 |           expect(
51 |             allowedFields.includes(piiField),
52 |             `PII field "${piiField}" found in audit allowlist for "${entityType}"`
53 |           ).toBe(false);
54 |         }
55 |       }
56 |     });
57 |
58 |     it("user entity allowlist contains only non-PII fields", () => {
59 |       const userFields = EXPECTED_AUDIT_ALLOWLIST.user;
60 |       expect(userFields).toEqual(["role", "is_active", "program_id"]);
61 |     });
62 |   });
63 |
64 |   // -----
65 |   // Sentry PII Scrubbing
66 |   // -----
67 |
68 |   describe("Sentry PII Scrubbing", () => {
69 |     it("scrubs email addresses from strings", () => {
70 |       const input = "Error for user student@university.edu in course CS101";
71 |       const result = scrubPII(input);
72 |       expect(result).not.toContain("student@university.edu");
73 |       expect(result).toContain("[email]");
74 |     });
75 |
76 |     it("scrubs UUIDs from strings", () => {
77 |       const input =
78 |         "Failed to load profile 550e8400-e29b-41d4-a716-446655440000";
79 |       const result = scrubPII(input);
80 |       expect(result).not.toContain("550e8400-e29b-41d4-a716-446655440000");
81 |       expect(result).toContain("[uuid]");
82 |     });
83 |
84 |     it("scrubs multiple PII occurrences", () => {
85 |       const input =
86 |         "User admin@school.org assigned to 550e8400-e29b-41d4-a716-446655440000 and teacher@school.org";
87 |       const result = scrubPII(input);
88 |       expect(result).not.toContain("admin@school.org");
89 |       expect(result).not.toContain("teacher@school.org");
90 |       expect(result).not.toContain("550e8400");
91 |     });
92 |
93 |     it("leaves non-PII strings unchanged", () => {
94 |       const input = "Network timeout after 5000ms on /api/courses";
95 |       expect(scrubPII(input)).toBe(input);
96 |     });
97 |   });
98 |
99 |   // -----
100 |   // Activity Logger metadata safety
101 |   // -----
102 |
103 |   describe("Activity Logger Metadata Safety", () => {
104 |     // These represent the metadata shapes actually used in the codebase.
105 |     // If a developer adds PII to metadata, this test documents the contract.
106 |     const SAFE_METADATA_EXAMPLES: Array<{
107 |       event: string;
108 |       metadata: Record<string, unknown>;
109 |     }> = [
110 |       { event: "page_view", metadata: { page: "/student/dashboard" } },
111 |       { event: "submission", metadata: { assignment_id: "abc-123" } },
112 |       {
113 |         event: "assignment_view",
114 |         metadata: { assignment_id: "abc-123", duration_seconds: 45 },
115 |       },
116 |       { event: "journal", metadata: { journal_id: "j-001", word_count: 120 } },
117 |       { event: "login", metadata: {} },
118 |     ];
119 |
120 |     it.each(SAFE_METADATA_EXAMPLES)(
121 |       "$event metadata contains no PII fields",
122 |       ({ metadata }) => {
123 |         const keys = Object.keys(metadata);
124 |         for (const piiField of PII_FIELDS) {
125 |           expect(keys).not.toContain(piiField);
126 |         }
127 |         // Values should not contain email patterns
128 |         for (const value of Object.values(metadata)) {
129 |           if (typeof value === "string") {
130 |             expect(value).not.toMatch(
131 |               /[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/

```

```

132 |         });
133 |     }
134 | }
135 | }
136 | });
137 | });
138 |
139 | // -----
140 | // Soft-delete data integrity
141 | // -----
142 |
143 | describe("Soft-Delete Workflow", () => {
144 |     it("deactivation uses is_active flag, not record deletion", () => {
145 |         // This is a structural assertion: the profiles table has is_active boolean.
146 |         // The soft-delete flow sets is_active = false rather than deleting the row.
147 |         // Verified by the UserListPage deactivation flow calling updateUser({ is_active: false }).
148 |         const softDeleteField = "is_active";
149 |         const profileFields = [
150 |             "id",
151 |             "email",
152 |             "full_name",
153 |             "avatar_url",
154 |             "role",
155 |             "institution_id",
156 |             "is_active",
157 |             "created_at",
158 |         ];
159 |         expect(profileFields).toContain(softDeleteField);
160 |     });
161 |
162 |     it("append-only tables have no delete mechanism", () => {
163 |         // These tables are designed as immutable append-only stores.
164 |         // RLS policies only grant INSERT, never UPDATE or DELETE.
165 |         const appendOnlyTables = ["evidence", "audit_logs", "xp_transactions"];
166 |         expect(appendOnlyTables.length).toBe(3);
167 |         // The invariant is enforced at the RLS policy level, not in application code.
168 |         // This test documents the contract.
169 |         for (const table of appendOnlyTables) {
170 |             expect(typeof table).toBe("string");
171 |         }
172 |     });
173 | });
174 |
175 | // -----
176 | // GDPR Data Export
177 | // -----
178 |
179 | describe("GDPR Data Export", () => {
180 |     it("export covers all required student data categories", () => {
181 |         // The export-student-data Edge Function must export these categories
182 |         const requiredCategories = [
183 |             "profile",
184 |             "grades",
185 |             "outcome_attainment",
186 |             "xp_transactions",
187 |             "journal_entries",
188 |             "badges",
189 |             "habit_logs",
190 |         ];
191 |         // Verified against supabase/functions/export-student-data/index.ts
192 |         expect(requiredCategories.length).toBe(7);
193 |         for (const cat of requiredCategories) {
194 |             expect(typeof cat).toBe("string");
195 |         }
196 |     });
197 |
198 |     it("export URL has limited lifetime (1 hour)", () => {
199 |         // The signed URL expiry is set to 3600 seconds in the Edge Function
200 |         const SIGNED_URL_EXPIRY_SECONDS = 3600;
201 |         expect(SIGNED_URL_EXPIRY_SECONDS).toBeLessThanOrEqual(3600);
202 |     });
203 | });
204 |
205 | // -----
206 | // Sentry Replay privacy
207 | // -----
208 |
209 | describe("Sentry Replay Privacy", () => {
210 |     it("replay config masks all text and inputs", () => {
211 |         // These values are set in src/lib/sentry.ts replayIntegration config
212 |         const replayConfig = {
213 |             maskAllText: true,
214 |             maskAllInputs: true,
215 |             blockAllMedia: true,
216 |         };
217 |         expect(replayConfig.maskAllText).toBe(true);
218 |         expect(replayConfig.maskAllInputs).toBe(true);
219 |         expect(replayConfig.blockAllMedia).toBe(true);
220 |     });
221 | });
222 | });
223 |

```

FILE: [src/__tests__/unit/dataExport.test.ts](#)

TYPE: .ts | LINES: 1-96 | SHA256: 10522f301a61b6704d5dc9bc8e95df37ddd5b4e4471a1280f121a8fc8684ea82

```

1 | // =====
2 | // Data Export (toCsv utility) - Unit tests
3 | // Validates: Requirement 63 (GDPR Student Data Export)
4 | // =====
5 |
6 | import { describe, it, expect } from "vitest";
7 |
8 | // The toCsv function is embedded in the Deno edge function and not directly

```

```

9 | // importable. We replicate the exact logic here for unit testing.
10 |
11 | /** Convert an array of objects to CSV string – mirrors export-student-data/index.ts */
12 | function toCsv(rows: Record<string, unknown>[], section: string): string {
13 |   if (rows.length === 0) return "";
14 |   const keys = Object.keys(rows[0]!);
15 |   const header = keys.join(",");
16 |   const lines = rows.map((row) => {
17 |     keys
18 |       .map((k) => {
19 |         const val = row[k];
20 |         if (val === null || val === undefined) return "";
21 |         const str = String(val);
22 |         if (str.includes(",") || str.includes('"') || str.includes("\n")) {
23 |           return `"${str.replace(/"/g, '"')}"`;
24 |         }
25 |         return str;
26 |       })
27 |       .join(",")
28 |   });
29 |   return `--- ${section} ---\n${header}\n${lines.join("\n")}`;
30 | }
31 |
32 | describe("toCsv (data export utility)", () => {
33 |   it("returns empty string for empty rows", () => {
34 |     expect(toCsv([], "Test")).toBe("");
35 |   });
36 |
37 |   it("generates CSV with section header", () => {
38 |     const rows = [{ name: "Alice", score: 95 }];
39 |     const result = toCsv(rows, "Grades");
40 |     expect(result).toContain("--- Grades ---");
41 |     expect(result).toContain("name,score");
42 |     expect(result).toContain("Alice,95");
43 |   });
44 |
45 |   it("handles multiple rows", () => {
46 |     const rows = [
47 |       { id: "1", value: "a" },
48 |       { id: "2", value: "b" },
49 |     ];
50 |     const result = toCsv(rows, "Data");
51 |     const lines = result.split("\n");
52 |     expect(lines).toHaveLength(4); // section header + header + 2 data rows
53 |     expect(lines[2]).toBe("1,a");
54 |     expect(lines[3]).toBe("2,b");
55 |   });
56 |
57 |   it("escapes values containing commas", () => {
58 |     const rows = [{ name: "Doe, John", age: 25 }];
59 |     const result = toCsv(rows, "Profile");
60 |     expect(result).toContain('"Doe, John"');
61 |   });
62 |
63 |   it("escapes values containing double quotes", () => {
64 |     const rows = [{ feedback: "Said \"great work\"" }];
65 |     const result = toCsv(rows, "Feedback");
66 |     expect(result).toContain('"Said "great work"'");
67 |   });
68 |
69 |   it("escapes values containing newlines", () => {
70 |     const rows = [{ comment: "Line 1\nLine 2" }];
71 |     const result = toCsv(rows, "Comments");
72 |     expect(result).toContain('"Line 1\nLine 2"');
73 |   });
74 |
75 |   it("handles null and undefined values as empty strings", () => {
76 |     const rows = [{ a: null, b: undefined, c: "ok" }];
77 |     const result = toCsv(rows, "Mixed");
78 |     expect(result).toContain(",,ok");
79 |   });
80 |
81 |   it("converts non-string values to strings", () => {
82 |     const rows = [{ count: 42, active: true, ratio: 0.75 }];
83 |     const result = toCsv(rows, "Numbers");
84 |     expect(result).toContain("42,true,0.75");
85 |   });
86 |
87 |   it("uses keys from first row as header", () => {
88 |     const rows = [
89 |       { student_id: "s1", xp_amount: 50, created_at: "2024-01-01" },
90 |     ];
91 |     const result = toCsv(rows, "XP");
92 |     const headerLine = result.split("\n")[1];
93 |     expect(headerLine).toBe("student_id,xp_amount,created_at");
94 |   });
95 | });
96 |

```

FILE: src/___tests___/unit/departmentManager.test.tsx

TYPE: .tsx | LINES: 1-94 | SHA256: 2b303968a5dc64c21dfe9e235024dd797d098c7eac230f40f041b6325a5635a9

```

1 | // @vitest-environment happy-dom
2 | import { describe, it, expect, vi } from "vitest";
3 | import { render, screen } from "@testing-library/react";
4 | import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
5 | import { MemoryRouter } from "react-router-dom";
6 |
7 | // Mock hooks
8 | vi.mock("@/hooks/useDepartments", () => ({
9 |   useDepartments: vi.fn(() => ({
10 |     data: [
11 |       {
12 |         id: "d1",

```

```

13 |         name: "Computer Science",
14 |         code: "CS",
15 |         head_of_department_id: "u1",
16 |         institution_id: "inst-1",
17 |         created_at: "2025-01-01",
18 |     },
19 |     {
20 |         id: "d2",
21 |         name: "Mathematics",
22 |         code: "MATH",
23 |         head_of_department_id: null,
24 |         institution_id: "inst-1",
25 |         created_at: "2025-01-01",
26 |     },
27 | ],
28 | isLoading: false,
29 | ))),
30 | useCreateDepartment: vi.fn(() => ({ mutate: vi.fn(), isPending: false })),
31 | useUpdateDepartment: vi.fn(() => ({ mutate: vi.fn(), isPending: false })),
32 | useDeleteDepartment: vi.fn(() => ({ mutate: vi.fn(), isPending: false })),
33 | ));
34 |
35 | vi.mock("@/hooks/useAuth", () => ({
36 |   useAuth: () => ({
37 |     user: { id: "u1" },
38 |     institutionId: "inst-1",
39 |     profile: { role: "admin" },
40 |   }),
41 | }));
42 |
43 | vi.mock("@/lib/supabase", () => ({
44 |   supabase: {
45 |     from: vi.fn(() => ({
46 |       select: vi.fn().mockReturnThis(),
47 |       in: vi.fn().mockReturnThis(),
48 |       eq: vi.fn().mockReturnThis(),
49 |       order: vi.fn().mockResolvedValue({ data: [], error: null }),
50 |     })),
51 |   },
52 | }));
53 |
54 | import DepartmentManager from "@/pages/admin/departments/DepartmentManager";
55 |
56 | const createWrapper = () => {
57 |   const qc = new QueryClient({ defaultOptions: { queries: { retry: false } } });
58 |   return ({ children }: { children: React.ReactNode }) => (
59 |     <QueryClientProvider client={qc}>
60 |       <MemoryRouter>{children}</MemoryRouter>
61 |     </QueryClientProvider>
62 |   );
63 | };
64 |
65 | describe("DepartmentManager", () => {
66 |   it("renders page title", () => {
67 |     render(<DepartmentManager />, { wrapper: createWrapper() });
68 |     expect(screen.getByText("Departments")).toBeInTheDocument();
69 |   });
70 |
71 |   it("renders department list", () => {
72 |     render(<DepartmentManager />, { wrapper: createWrapper() });
73 |     expect(screen.getByText("Computer Science")).toBeInTheDocument();
74 |     expect(screen.getByText("Mathematics")).toBeInTheDocument();
75 |   });
76 |
77 |   it("renders department codes", () => {
78 |     render(<DepartmentManager />, { wrapper: createWrapper() });
79 |     expect(screen.getByText("CS")).toBeInTheDocument();
80 |     expect(screen.getByText("MATH")).toBeInTheDocument();
81 |   });
82 |
83 |   it("renders add department button", () => {
84 |     render(<DepartmentManager />, { wrapper: createWrapper() });
85 |     expect(screen.getByText("Add Department")).toBeInTheDocument();
86 |   });
87 |
88 |   it("shows HoD status", () => {
89 |     render(<DepartmentManager />, { wrapper: createWrapper() });
90 |     expect(screen.getByText("HoD assigned")).toBeInTheDocument();
91 |     expect(screen.getByText("No HoD assigned")).toBeInTheDocument();
92 |   });
93 | });
94 |

```

FILE: [src/_tests_/unit/designSystemPatterns.test.tsx](#)

TYPE: .tsx | LINES: 1-89 | SHA256: 07b840dc5ae49333231501785ae8ba1bac8266f368cc3a51168dace41af2bdc2

```

1 | // Design-system L2 patterns – render smoke tests (prototype design language).
2 | // Feature: prototype-frontend-rebuild (P0.4). Ensures the shared surface screens
3 | // build from behaves correctly across its core primitives.
4 | import { describe, it, expect } from "vitest";
5 | import { render, screen } from "@testing-library/react";
6 | import { Wallet } from "lucide-react";
7 | import {
8 |   PageHeader,
9 |   PCard,
10 |  SectionHeader,
11 |  SectionCard,
12 |  KPICard,
13 |  HeroCard,
14 |  StatusDot,
15 |  StatePanel,
16 | } from "@/design-system/patterns";
17 |
18 | describe("design-system patterns", () => {

```

```

19 | it("PageHeader renders the title and optional action", () => {
20 |   render(<PageHeader title="Fees" action={<button>Add</button>} />);
21 |   expect(screen.getByRole("heading", { name: "Fees" })).toBeInTheDocument();
22 |   expect(screen.getByRole("button", { name: "Add" })).toBeInTheDocument();
23 | });
24 |
25 | it("PCard renders children on a white surface", () => {
26 |   const { container } = render(<PCard>body</PCard>);
27 |   const el = container.firstChild as HTMLElement;
28 |   expect(el.className).toContain("bg-white");
29 |   expect(el).toHaveTextContent("body");
30 | });
31 |
32 | it("SectionHeader renders the title (+ gradient chip when an icon is given)", () => {
33 |   const { container } = render(
34 |     <SectionHeader icon={Wallet} title="History" />
35 |   );
36 |   expect(
37 |     screen.getByRole("heading", { name: "History" })
38 |   ).toBeInTheDocument();
39 |   // the icon chip carries the liquid glass icon wrapper
40 |   const chip = container.querySelector('span[aria-hidden="true"]');
41 |   expect(chip).not.toBeNull();
42 | });
43 |
44 | it("SectionCard renders its title and body", () => {
45 |   render(
46 |     <SectionCard icon={Wallet} title="Payment history">
47 |       <p>rows</p>
48 |     </SectionCard>
49 |   );
50 |   expect(
51 |     screen.getByRole("heading", { name: "Payment history" })
52 |   ).toBeInTheDocument();
53 |   expect(screen.getByText("rows")).toBeInTheDocument();
54 | });
55 |
56 | it("KPICard renders label and value", () => {
57 |   render(<KPICard icon={Wallet} label="Total paid" value="5,000" />);
58 |   expect(screen.getByText("Total paid")).toBeInTheDocument();
59 |   expect(screen.getByText("5,000")).toBeInTheDocument();
60 | });
61 |
62 | it("HeroCard exposes a labeled region when ariaLabel is set", () => {
63 |   render(<HeroCard ariaLabel="Identity">hi</HeroCard>);
64 |   expect(screen.getByRole("region", { name: "Identity" })).toHaveTextContent(
65 |     "hi"
66 |   );
67 | });
68 |
69 | it("StatusDot is decorative by default and labeled when given a label", () => {
70 |   const { container, rerender } = render(<StatusDot tone="success" />);
71 |   expect((container.firstChild as HTMLElement).className).toContain(
72 |     "bg-green-500"
73 |   );
74 |   rerender(<StatusDot tone="danger" label="Critical" />);
75 |   expect(screen.getByRole("img", { name: "Critical" })).toBeInTheDocument();
76 | });
77 |
78 | it("StatePanel renders loading / empty / error variants", () => {
79 |   const { container, rerender } = render(<StatePanel variant="loading" />);
80 |   expect(container.querySelector(".animate-pulse")).not.toBeNull();
81 |
82 |   rerender(<StatePanel variant="empty" message="Nothing yet." />);
83 |   expect(screen.getByText("Nothing yet.")).toBeInTheDocument();
84 |
85 |   rerender(<StatePanel variant="error" message="Broke." />);
86 |   expect(screen.getByRole("alert")).toHaveTextContent("Broke.");
87 | });
88 | });
89 |

```

FILE: [src/__tests__/unit/designTokens.test.ts](#)

TYPE: .ts | LINES: 1-134 | SHA256: d65b6defccf51cf02e9a8652c0d1af804dca6b55675a559ac0ebde823fdafd3a

```

1 | import { describe, it, expect } from "vitest";
2 | import { readFileSync } from "fs";
3 | import { resolve } from "path";
4 |
5 | const indexCss = readFileSync(resolve(__dirname, "../../index.css"), "utf-8");
6 | const indexHtml = readFileSync(
7 |   resolve(__dirname, "../../index.html"),
8 |   "utf-8"
9 | );
10 |
11 | describe("Design Tokens - src/index.css", () => {
12 |   describe("Brand Colors", () => {
13 |     it("defines --brand-primary as #3b82f6 (blue-500)", () => {
14 |       expect(indexCss).toContain("--brand-primary: #3b82f6");
15 |     });
16 |
17 |     it("defines --brand-primary-dark as #2563eb (blue-600)", () => {
18 |       expect(indexCss).toContain("--brand-primary-dark: #2563eb");
19 |     });
20 |
21 |     it("defines --brand-secondary as #14b8a6 (teal-500)", () => {
22 |       expect(indexCss).toContain("--brand-secondary: #14b8a6");
23 |     });
24 |
25 |     it("defines --gradient-start and --gradient-end", () => {
26 |       expect(indexCss).toContain("--gradient-start: #14b8a6");
27 |       expect(indexCss).toContain("--gradient-end: #0382bd");
28 |     });
29 |   });

```

```

30 |
31 | describe("Brand Gradient", () => {
32 |   it("defines --brand-gradient with the prototype-exact teal-to-blue gradient", () => {
33 |     // Prototype-fidelity: the canonical brand gradient is the prototype's
34 |     // exact 93.65deg (shared.css), NOT the legacy 135deg.
35 |     expect(indexCss).toContain(
36 |       "--brand-gradient: linear-gradient(93.65deg, #14b8a6 5.37%, #0382bd 78.89%)"
37 |     );
38 |   });
39 |
40 |   it("defines --hero-gradient for dashboard hero cards", () => {
41 |     expect(indexCss).toContain("--hero-gradient: linear-gradient(");
42 |     expect(indexCss).toContain("#0f172a 0%");
43 |     expect(indexCss).toContain("#1e3a8a 50%");
44 |     expect(indexCss).toContain("#312e81 100%");
45 |   });
46 | });
47 |
48 | describe("Semantic Colors", () => {
49 |   it("defines --color-success as #22c55e (green-500)", () => {
50 |     expect(indexCss).toContain("--color-success: #22c55e");
51 |   });
52 |
53 |   it("defines --color-warning as #f59e0b (amber-500)", () => {
54 |     expect(indexCss).toContain("--color-warning: #f59e0b");
55 |   });
56 |
57 |   it("defines --color-destructive-brand as #ef4444 (red-500)", () => {
58 |     expect(indexCss).toContain("--color-destructive-brand: #ef4444");
59 |   });
60 |
61 |   it("defines --color-neutral as #64748b (slate-500)", () => {
62 |     expect(indexCss).toContain("--color-neutral: #64748b");
63 |   });
64 | });
65 |
66 | describe("Surface Colors", () => {
67 |   it("defines --surface-background as white", () => {
68 |     expect(indexCss).toContain("--surface-background: #ffffff");
69 |   });
70 |
71 |   it("defines --surface-card as white", () => {
72 |     expect(indexCss).toContain("--surface-card: #ffffff");
73 |   });
74 |
75 |   it("defines --surface-subtle as #f8fafc (slate-50)", () => {
76 |     expect(indexCss).toContain("--surface-subtle: #f8fafc");
77 |   });
78 |
79 |   it("defines --surface-border as #e2e8f0 (slate-200)", () => {
80 |     expect(indexCss).toContain("--surface-border: #e2e8f0");
81 |   });
82 | });
83 |
84 | describe("Typography", () => {
85 |   it("sets Noto Sans as the primary font in @theme", () => {
86 |     expect(indexCss).toContain("Noto Sans");
87 |   });
88 |
89 |   it("defines --font-mono for code/technical text", () => {
90 |     expect(indexCss).toContain("--font-mono: Menlo");
91 |   });
92 | });
93 |
94 | describe("Shadows", () => {
95 |   it("defines --shadow-sm, --shadow-md, --shadow-lg, --shadow-xl", () => {
96 |     expect(indexCss).toContain("--shadow-sm:");
97 |     expect(indexCss).toContain("--shadow-md:");
98 |     expect(indexCss).toContain("--shadow-lg:");
99 |     expect(indexCss).toContain("--shadow-xl:");
100 |   });
101 | });
102 |
103 | describe("Gamification Tokens", () => {
104 |   it("defines --xp-track and --xp-fill for XP bar", () => {
105 |     expect(indexCss).toContain("--xp-track: #e2e8f0");
106 |     expect(indexCss).toContain("--xp-fill: #14b8a6");
107 |   });
108 | });
109 | });
110 |
111 | describe("Google Fonts – index.html", () => {
112 |   it("loads Noto Sans via Google Fonts with display=swap", () => {
113 |     expect(indexHtml).toContain("fonts.googleapis.com/css2?family=Noto+Sans");
114 |     expect(indexHtml).toContain("display=swap");
115 |   });
116 |
117 |   it("includes preconnect for Google Fonts", () => {
118 |     expect(indexHtml).toContain(
119 |       'rel="preconnect" href="https://fonts.googleapis.com"'
120 |     );
121 |     expect(indexHtml).toContain(
122 |       'rel="preconnect" href="https://fonts.gstatic.com"'
123 |     );
124 |   });
125 |
126 |   it("loads required font weights (400, 500, 600, 700, 800, 900)", () => {
127 |     expect(indexHtml).toMatch(/wght@.400/);
128 |     expect(indexHtml).toMatch(/wght@.500/);
129 |     expect(indexHtml).toMatch(/wght@.600/);
130 |     expect(indexHtml).toMatch(/wght@.700/);
131 |     expect(indexHtml).toMatch(/wght@.900/);
132 |   });
133 | });

```


[illegible]

99

FILE: src/__tests__/unit/directionManager.test.ts

TYPE: .ts | LINES: 1-76 | SHA256: 2870fce79278f4ca12beddc44f31b89be7c9f945d9aa29d6579d9a114402af0f

```

1 import { describe, it, expect, beforeEach } from "vitest";
2 import { getDirection, applyDirection } from "@lib/directionManager";
3
4 describe("directionManager", () => {
5   beforeEach(() => {
6     document.documentElement.removeAttribute("dir");
7     document.documentElement.removeAttribute("lang");
8     document.documentElement.style.fontFamily = "";
9   });
10
11   describe("getDirection", () => {
12     it("returns rtl for Arabic", () => {
13       expect(getDirection("ar")).toBe("rtl");
14     });
15
16     it("returns ltr for English", () => {
17       expect(getDirection("en")).toBe("ltr");
18     });
19
20     it("returns rtl for Hebrew", () => {
21       expect(getDirection("he")).toBe("rtl");
22     });
23
24     it("returns ltr for unknown languages", () => {
25       expect(getDirection("xyz")).toBe("ltr");
26     });
27   });
28
29   describe("applyDirection", () => {
30     it("sets dir=rtl and lang=ar for Arabic", () => {
31       applyDirection("ar");
32       expect(document.documentElement.getAttribute("dir")).toBe("rtl");
33       expect(document.documentElement.getAttribute("lang")).toBe("ar");
34     });
35
36     it("sets dir=ltr and lang=en for English", () => {
37       applyDirection("en");
38       expect(document.documentElement.getAttribute("dir")).toBe("ltr");
39       expect(document.documentElement.getAttribute("lang")).toBe("en");
40     });
41
42     it("sets Arabic font family for RTL languages", () => {
43       applyDirection("ar");
44       expect(document.documentElement.style.fontFamily).toContain(
45         "Noto Sans Arabic"
46       );
47     });
48
49     it("sets default font family for LTR languages", () => {
50       applyDirection("en");
51       expect(document.documentElement.style.fontFamily).toContain("Noto Sans");
52       expect(document.documentElement.style.fontFamily).not.toContain(
53         "Noto Sans Arabic"
54       );
55     });
56
57     it("is idempotent – multiple calls produce same result", () => {
58       applyDirection("ar");
59       const state1 = {
60         dir: document.documentElement.getAttribute("dir"),
61         lang: document.documentElement.getAttribute("lang"),
62         font: document.documentElement.style.fontFamily,
63       };
64
65       applyDirection("ar");
66       const state2 = {
67         dir: document.documentElement.getAttribute("dir"),
68         lang: document.documentElement.getAttribute("lang"),
69         font: document.documentElement.style.fontFamily,
70       };
71
72       expect(state1).toEqual(state2);
73     });
74   });
75 });
76

```

FILE: src/__tests__/unit/discussionXpAwards.test.ts

TYPE: .ts | LINES: 1-178 | SHA256: b90d176e85e9cbe3d532e75f09c59d34d12b97300928aa296998c902e0d8892a

[illegible]

[illegible]

```

122 |         source: "discussion_question",
123 |         referenceId: "thread-1",
124 |         note: "Created discussion thread",
125 |     });
126 | });
127 |
128 | it("uses XP_SCHEDULE.discussion_question constant (10)", () => {
129 |     expect(XP_SCHEDULE.discussion_question).toBe(10);
130 | });
131 | });
132 |
133 | describe("Requirement 77.5: Correct answer awards 15 XP", () => {
134 |     it("calls awardXP with discussion_answer source and 15 XP for reply author", async () => {
135 |         const replyAuthorId = "student-2";
136 |         const replyId = "reply-1";
137 |
138 |         await awardXP({
139 |             studentId: replyAuthorId,
140 |             xpAmount: XP_SCHEDULE.discussion_answer,
141 |             source: "discussion_answer",
142 |             referenceId: replyId,
143 |             note: "Answer marked as correct",
144 |         });
145 |
146 |         expect(mockAwardXP).toHaveBeenCalledWith({
147 |             studentId: "student-2",
148 |             xpAmount: 15,
149 |             source: "discussion_answer",
150 |             referenceId: "reply-1",
151 |             note: "Answer marked as correct",
152 |         });
153 |     });
154 |
155 |     it("uses XP_SCHEDULE.discussion_answer constant (15)", () => {
156 |         expect(XP_SCHEDULE.discussion_answer).toBe(15);
157 |     });
158 | });
159 |
160 | describe("Fire-and-forget behavior", () => {
161 |     it("does not throw when awardXP rejects", async () => {
162 |         mockAwardXP.mockRejectedValueOnce(new Error("Network error"));
163 |
164 |         // Fire-and-forget pattern: catch silently
165 |         await expect(
166 |             awardXP({
167 |                 studentId: "student-1",
168 |                 xpAmount: 10,
169 |                 source: "discussion_question",
170 |                 referenceId: "thread-1",
171 |             }).catch(() => {
172 |                 /* fire-and-forget */
173 |             })
174 |         ).resolves.toBeUndefined();
175 |     });
176 | });
177 | });
178 |

```

FILE: [src/__tests__/unit/displayName.test.ts](#)

TYPE: .ts | LINES: 1-51 | SHA256: 8400e5ef55d248eca2a5b447f899a3406ad4c29baed7aaa0e580096f2e62d48c

```

1 | import { describe, it, expect } from "vitest";
2 | import { getDisplayFirstName } from "@lib/displayName";
3 |
4 | /**
5 |  * Production Bug Fixes – Track A, Item 3 (Requirement 3):
6 |  * honorific-titled names must render the first name, not the title.
7 |  */
8 | describe("getDisplayFirstName", () => {
9 |     it("skips a leading honorific with a trailing dot (Req 3)", () => {
10 |         expect(getDisplayFirstName("Mr. David Okonkwo")).toBe("David");
11 |     });
12 |
13 |     it("skips an honorific before a hyphenated surname (Req 3)", () => {
14 |         expect(getDisplayFirstName("Dr. Aisha Al-Mansoori")).toBe("Aisha");
15 |     });
16 |
17 |     it("skips an honorific without a trailing dot (Req 3)", () => {
18 |         expect(getDisplayFirstName("Prof Sara")).toBe("Sara");
19 |     });
20 |
21 |     it("returns the first token when there is no honorific", () => {
22 |         expect(getDisplayFirstName("Sara Imran")).toBe("Sara");
23 |     });
24 |
25 |     it("returns a lone honorific token as-is when no name follows", () => {
26 |         expect(getDisplayFirstName("Mr.")).toBe("Mr.");
27 |     });
28 |
29 |     it("returns null for an empty string so the caller can supply a fallback", () => {
30 |         expect(getDisplayFirstName("")).toBeNull();
31 |     });
32 |
33 |     it("returns null for whitespace-only input", () => {
34 |         expect(getDisplayFirstName("   ")).toBeNull();
35 |     });
36 |
37 |     it("returns null for null and undefined", () => {
38 |         expect(getDisplayFirstName(null)).toBeNull();
39 |         expect(getDisplayFirstName(undefined)).toBeNull();
40 |     });
41 |
42 |     it("is case-insensitive for honorific detection", () => {
43 |         expect(getDisplayFirstName("DR. AISHA")).toBe("AISHA");

```

```

44 |     expect(getDisplayFirstName("mrs jane doe")).toBe("jane");
45 | });
46 |
47 | it("collapses extra internal whitespace around tokens", () => {
48 |     expect(getDisplayFirstName(" Mr. David Okonkwo ")).toBe("David");
49 | });
50 | });
51 |

```

FILE: `src/__tests__/unit/draftManager.test.ts`

TYPE: `.ts` | LINES: 1-135 | SHA256: 7b59596aa4bad8701cc31d010f4c1a101b97962c27c6efe7ca41aae73191d032

```

1 | // @vitest-environment happy-dom
2 | // =====
3 | // DraftManager - Unit tests
4 | // Feature: edeviser-platform, Task 54.1
5 | // =====
6 |
7 | import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
8 | import { draftManager } from "@lib/draftManager";
9 |
10 | describe("draftManager", () => {
11 |     beforeEach(() => {
12 |         localStorage.clear();
13 |         vi.useFakeTimers();
14 |     });
15 |
16 |     afterEach(() => {
17 |         vi.useRealTimers();
18 |     });
19 |
20 |     describe("saveDraft / loadDraft", () => {
21 |         it("saves and loads a string draft", () => {
22 |             draftManager.saveDraft("test-key", "hello world");
23 |             expect(draftManager.loadDraft<string>("test-key")).toBe("hello world");
24 |         });
25 |
26 |         it("saves and loads an object draft", () => {
27 |             const data = { content: "my journal", courseId: "c1" };
28 |             draftManager.saveDraft("journal-1", data);
29 |             expect(draftManager.loadDraft("journal-1")).toEqual(data);
30 |         });
31 |
32 |         it("returns null for non-existent key", () => {
33 |             expect(draftManager.loadDraft("missing")).toBeNull();
34 |         });
35 |
36 |         it("overwrites existing draft", () => {
37 |             draftManager.saveDraft("k", "v1");
38 |             draftManager.saveDraft("k", "v2");
39 |             expect(draftManager.loadDraft("k")).toBe("v2");
40 |         });
41 |
42 |         it("handles localStorage quota exceeded gracefully", () => {
43 |             const setItemSpy = vi
44 |                 .spyOn(Storage.prototype, "setItem")
45 |                 .mockImplementation(() => {
46 |                     throw new DOMException("QuotaExceededError");
47 |                 });
48 |             // Should not throw
49 |             expect(() => draftManager.saveDraft("k", "v")).not.toThrow();
50 |             setItemSpy.mockRestore();
51 |         });
52 |
53 |         it("returns null when localStorage getItem throws", () => {
54 |             const getItemSpy = vi
55 |                 .spyOn(Storage.prototype, "getItem")
56 |                 .mockImplementation(() => {
57 |                     throw new Error("SecurityError");
58 |                 });
59 |             expect(draftManager.loadDraft("k")).toBeNull();
60 |             getItemSpy.mockRestore();
61 |         });
62 |     });
63 |
64 |     describe("clearDraft", () => {
65 |         it("removes a saved draft", () => {
66 |             draftManager.saveDraft("k", "v");
67 |             draftManager.clearDraft("k");
68 |             expect(draftManager.loadDraft("k")).toBeNull();
69 |         });
70 |
71 |         it("does not throw for non-existent key", () => {
72 |             expect(() => draftManager.clearDraft("missing")).not.toThrow();
73 |         });
74 |     });
75 |
76 |     describe("startAutoSave", () => {
77 |         it("auto-saves at the specified interval", () => {
78 |             let counter = 0;
79 |             const getContent = () => `draft-${++counter}`;
80 |             const stop = draftManager.startAutoSave("auto-key", getContent, 30_000);
81 |
82 |             // Nothing saved yet
83 |             expect(draftManager.loadDraft("auto-key")).toBeNull();
84 |
85 |             // Advance 30s
86 |             vi.advanceTimersByTime(30_000);
87 |             expect(draftManager.loadDraft("auto-key")).toBe("draft-1");
88 |
89 |             // Advance another 30s
90 |             vi.advanceTimersByTime(30_000);
91 |             expect(draftManager.loadDraft("auto-key")).toBe("draft-2");
92 |         });

```

```

93 |     stop();
94 | });
95 |
96 | it("stops auto-saving when cleanup is called", () => {
97 |     let counter = 0;
98 |     const stop = draftManager.startAutoSave(
99 |         "auto-key",
100 |         () => `v-${++counter}`,
101 |         10_000
102 |     );
103 |
104 |     vi.advanceTimersByTime(10_000);
105 |     expect(draftManager.loadDraft("auto-key")).toBe("v-1");
106 |
107 |     stop();
108 |
109 |     vi.advanceTimersByTime(10_000);
110 |     // Should still be v-1 since we stopped
111 |     expect(draftManager.loadDraft("auto-key")).toBe("v-1");
112 | });
113 |
114 | it("skips saving when getContent returns null", () => {
115 |     const stop = draftManager.startAutoSave("auto-key", () => null, 10_000);
116 |     vi.advanceTimersByTime(10_000);
117 |     expect(draftManager.loadDraft("auto-key")).toBeNull();
118 |     stop();
119 | });
120 |
121 | it("defaults to 30 second interval", () => {
122 |     const getContent = () => "data";
123 |     const stop = draftManager.startAutoSave("auto-key", getContent);
124 |
125 |     vi.advanceTimersByTime(29_999);
126 |     expect(draftManager.loadDraft("auto-key")).toBeNull();
127 |
128 |     vi.advanceTimersByTime(1);
129 |     expect(draftManager.loadDraft("auto-key")).toBe("data");
130 |
131 |     stop();
132 | });
133 | });
134 | });
135 |

```

FILE: [src/__tests__/unit/dynamicPricingCalculator.test.ts](#)

TYPE: .ts | LINES: 1-113 | SHA256: 84e2bacf9c9a40330befcbe7f16cb7eb0fefec4783ea8b4e6c99d3b7632baea3

```

1 | // =====
2 | // Unit Test: Dynamic Pricing Calculator
3 | // Task 26.2 – Dynamic price computation, bounds enforcement
4 | // =====
5 |
6 | import { describe, it, expect } from "vitest";
7 | import {
8 |     computeDynamicPrice,
9 |     classifyDemand,
10 | } from "@lib/dynamicPricingCalculator";
11 |
12 | describe("Dynamic Pricing Calculator", () => {
13 |     describe("classifyDemand", () => {
14 |         it("classifies high demand when above p75", () => {
15 |             expect(classifyDemand(80, 20, 60)).toBe("high");
16 |         });
17 |
18 |         it("classifies low demand when below p25", () => {
19 |             expect(classifyDemand(5, 20, 60)).toBe("low");
20 |         });
21 |
22 |         it("classifies normal demand between p25 and p75", () => {
23 |             expect(classifyDemand(40, 20, 60)).toBe("normal");
24 |         });
25 |
26 |         it("classifies exactly p25 as normal", () => {
27 |             expect(classifyDemand(20, 20, 60)).toBe("normal");
28 |         });
29 |
30 |         it("classifies exactly p75 as normal", () => {
31 |             expect(classifyDemand(60, 20, 60)).toBe("normal");
32 |         });
33 |     });
34 |
35 |     describe("computeDynamicPrice", () => {
36 |         it("returns base price for normal demand", () => {
37 |             const result = computeDynamicPrice({
38 |                 basePrice: 100,
39 |                 purchaseCount: 50,
40 |                 p25: 30,
41 |                 p75: 70,
42 |             });
43 |             expect(result.dynamicPrice).toBe(100);
44 |             expect(result.isAdjusted).toBe(false);
45 |             expect(result.multiplier).toBe(1);
46 |         });
47 |
48 |         it("increases price for high demand", () => {
49 |             const result = computeDynamicPrice({
50 |                 basePrice: 100,
51 |                 purchaseCount: 140,
52 |                 p25: 30,
53 |                 p75: 70,
54 |             });
55 |             expect(result.dynamicPrice).toBeGreaterThan(100);
56 |             expect(result.isAdjusted).toBe(true);
57 |             expect(result.demandLevel).toBe("high");

```

```

58 | });
59 |
60 | it("decreases price for low demand", () => {
61 |     const result = computeDynamicPrice({
62 |         basePrice: 100,
63 |         purchaseCount: 0,
64 |         p25: 30,
65 |         p75: 70,
66 |     });
67 |     expect(result.dynamicPrice).toBeLessThan(100);
68 |     expect(result.isAdjusted).toBe(true);
69 |     expect(result.demandLevel).toBe("low");
70 | });
71 |
72 | it("never goes below 50% of base price", () => {
73 |     const result = computeDynamicPrice({
74 |         basePrice: 100,
75 |         purchaseCount: 0,
76 |         p25: 100,
77 |         p75: 200,
78 |     });
79 |     expect(result.dynamicPrice).toBeGreaterThanOrEqual(50);
80 | });
81 |
82 | it("never exceeds 150% of base price", () => {
83 |     const result = computeDynamicPrice({
84 |         basePrice: 100,
85 |         purchaseCount: 1000,
86 |         p25: 10,
87 |         p75: 50,
88 |     });
89 |     expect(result.dynamicPrice).toBeLessThanOrEqual(150);
90 | });
91 |
92 | it("returns 1 for zero base price", () => {
93 |     const result = computeDynamicPrice({
94 |         basePrice: 0,
95 |         purchaseCount: 50,
96 |         p25: 30,
97 |         p75: 70,
98 |     });
99 |     expect(result.dynamicPrice).toBe(1);
100 | });
101 |
102 | it("always returns an integer", () => {
103 |     const result = computeDynamicPrice({
104 |         basePrice: 77,
105 |         purchaseCount: 100,
106 |         p25: 20,
107 |         p75: 60,
108 |     });
109 |     expect(Number.isInteger(result.dynamicPrice)).toBe(true);
110 | });
111 | });
112 | });
113 |

```

FILE: [src/__tests__/unit/dyslexiaFont.test.ts](#)

TYPE: .ts | LINES: 1-41 | SHA256: 25c67e77765a2ecb68891ef55035b663703793d89f6426b732386d83fc726c6d

```

1 | // Feature: i18n-rtl-support, Property 12: Dyslexia Font Toggle Consistency
2 | // **Validates: Requirements 24.2, 24.3**
3 |
4 | import { describe, it, expect, beforeEach } from "vitest";
5 | import { applyDyslexiaFont } from "@lib/fontPreferences";
6 |
7 | describe("dyslexiaFont", () => {
8 |     beforeEach(() => {
9 |         document.documentElement.style.removeProperty("--font-body");
10 |     });
11 |
12 | it("applyDyslexiaFont(true) sets --font-body CSS variable with OpenDyslexic", () => {
13 |     applyDyslexiaFont(true);
14 |     const fontBody =
15 |         document.documentElement.style.getPropertyValue("--font-body");
16 |     expect(fontBody).toContain("OpenDyslexic");
17 | });
18 |
19 | it("applyDyslexiaFont(false) removes --font-body CSS variable", () => {
20 |     applyDyslexiaFont(true);
21 |     expect(
22 |         document.documentElement.style.getPropertyValue("--font-body")
23 |     ).toContain("OpenDyslexic");
24 |
25 |     applyDyslexiaFont(false);
26 |     const fontBody =
27 |         document.documentElement.style.getPropertyValue("--font-body");
28 |     expect(fontBody).toBe("");
29 | });
30 |
31 | it("toggling on then off restores original state", () => {
32 |     const originalFont =
33 |         document.documentElement.style.getPropertyValue("--font-body");
34 |     applyDyslexiaFont(true);
35 |     applyDyslexiaFont(false);
36 |     expect(document.documentElement.style.getPropertyValue("--font-body")).toBe(
37 |         originalFont
38 |     );
39 | });
40 | });
41 |

```


TYPE: .tsx | LINES: 1-146 | SHA256: d0d768703a7a59980890c2c8e198be05670c91ebf3efb75ad295531668e94e31

[illegible]

```

102 |     await waitFor(() => {
103 |         expect(screen.getByText("Streak Risk Alerts")).toBeDefined();
104 |         expect(screen.getByText("Weekly Summary")).toBeDefined();
105 |         expect(screen.getByText("New Assignments")).toBeDefined();
106 |         expect(screen.getByText("Grade Released")).toBeDefined();
107 |     });
108 | });
109 |
110 | it("renders descriptions for each preference", async () => {
111 |     renderComponent();
112 |     await waitFor(() => {
113 |         expect(screen.getByText(/Daily reminder at 8 PM/)).toBeDefined();
114 |         expect(screen.getByText(/Monday morning recap/)).toBeDefined();
115 |         expect(screen.getByText(/new assignment is posted/)).toBeDefined();
116 |         expect(screen.getByText(/grade is ready to view/)).toBeDefined();
117 |     });
118 | });
119 |
120 | it("renders switch toggles with correct aria labels", async () => {
121 |     renderComponent();
122 |     await waitFor(() => {
123 |         expect(
124 |             screen.getByRole("switch", { name: /Toggle Streak Risk Alerts/i })
125 |         ).toBeDefined();
126 |         expect(
127 |             screen.getByRole("switch", { name: /Toggle Weekly Summary/i })
128 |         ).toBeDefined();
129 |         expect(
130 |             screen.getByRole("switch", { name: /Toggle New Assignments/i })
131 |         ).toBeDefined();
132 |         expect(
133 |             screen.getByRole("switch", { name: /Toggle Grade Released/i })
134 |         ).toBeDefined();
135 |     });
136 | });
137 |
138 | it("shows loading spinner while fetching preferences", () => {
139 |     mockMaybeSingle.mockReturnValue(new Promise(() => {})); // never resolves
140 |     renderComponent();
141 |     // The loading state should show the spinner (Loader2 has animate-spin class)
142 |     const spinners = document.querySelectorAll(".animate-spin");
143 |     expect(spinners.length).toBeGreaterThan(0);
144 | });
145 | });
146 |

```

FILE: [src/__tests__/unit/emailPrefsSchema.test.ts](#)

TYPE: .ts | LINES: 1-64 | SHA256: bf15d8dcf96bf2ed07215c1d315d70fea0c0c77ebb534419b1dff748dfe81d36

```

1 | import { describe, it, expect } from "vitest";
2 | import { emailPreferencesSchema } from "@lib/schemas/emailPrefs";
3 |
4 | describe("emailPreferencesSchema", () => {
5 |     it("accepts all-true preferences", () => {
6 |         const result = emailPreferencesSchema.safeParse({
7 |             streak_risk: true,
8 |             weekly_summary: true,
9 |             new_assignment: true,
10 |             grade_released: true,
11 |         });
12 |         expect(result.success).toBe(true);
13 |     });
14 |
15 |     it("accepts all-false preferences (full opt-out)", () => {
16 |         const result = emailPreferencesSchema.safeParse({
17 |             streak_risk: false,
18 |             weekly_summary: false,
19 |             new_assignment: false,
20 |             grade_released: false,
21 |         });
22 |         expect(result.success).toBe(true);
23 |     });
24 |
25 |     it("applies default values for missing fields (true for most, false for notification_digest)", () => {
26 |         const result = emailPreferencesSchema.safeParse({});
27 |         expect(result.success).toBe(true);
28 |         if (result.success) {
29 |             expect(result.data.streak_risk).toBe(true);
30 |             expect(result.data.weekly_summary).toBe(true);
31 |             expect(result.data.new_assignment).toBe(true);
32 |             expect(result.data.grade_released).toBe(true);
33 |             expect(result.data.notification_digest).toBe(false);
34 |         }
35 |     });
36 |
37 |     it("rejects non-boolean values", () => {
38 |         const result = emailPreferencesSchema.safeParse({
39 |             streak_risk: "yes",
40 |             weekly_summary: 1,
41 |             new_assignment: null,
42 |             grade_released: undefined,
43 |         });
44 |         // streak_risk and weekly_summary should fail, grade_released defaults
45 |         expect(result.success).toBe(false);
46 |     });
47 |
48 |     it("accepts mixed true/false preferences", () => {
49 |         const result = emailPreferencesSchema.safeParse({
50 |             streak_risk: false,
51 |             weekly_summary: true,
52 |             new_assignment: false,
53 |             grade_released: true,
54 |         });
55 |         expect(result.success).toBe(true);

```

```
56 |         if (result.success) {
57 |             expect(result.data.streak_risk).toBe(false);
58 |             expect(result.data.weekly_summary).toBe(true);
59 |             expect(result.data.new_assignment).toBe(false);
60 |             expect(result.data.grade_released).toBe(true);
61 |         }
62 |     });
63 | });
64 |
```

FILE: src/__tests__/unit/embedCourseMaterial.test.ts

TYPE: .ts | LINES: 1-819 | SHA256: f6ddcaa595fcf2e38bf691d7e0dbfdfe867cab8b74e2785e2e344c7780c7174b

[illegible]

```

92 |
93 | const sentences = splitIntoSentences(trimmed);
94 | const chunks: TextChunk[] = [];
95 |
96 | const sentenceOffsets: number[] = [];
97 | let offsetCursor = 0;
98 | for (const sentence of sentences) {
99 |   const idx = trimmed.indexOf(sentence, offsetCursor);
100 |   sentenceOffsets.push(idx >= 0 ? idx : offsetCursor);
101 |   offsetCursor = (idx >= 0 ? idx : offsetCursor) + sentence.length;
102 |   while (offsetCursor < trimmed.length && /\s/.test(trimmed[offsetCursor])) {
103 |     offsetCursor++;
104 |   }
105 | }
106 |
107 | let currentSentences: string[] = [];
108 | let currentTokens = 0;
109 |
110 | const flushChunk = (): void => {
111 |   if (currentSentences.length === 0) return;
112 |
113 |   const chunkTextContent = currentSentences.join(" ");
114 |   const firstIdx = sentences.indexOf(currentSentences[0]!);
115 |   const lastIdx = sentences.indexOf(
116 |     currentSentences[currentSentences.length - 1]!
117 |   );
118 |
119 |   const startOffset = firstIdx >= 0 ? sentenceOffsets[firstIdx]! : 0;
120 |   const lastSentence = currentSentences[currentSentences.length - 1]!;
121 |   const lastOffset = lastIdx >= 0 ? sentenceOffsets[lastIdx]! : startOffset;
122 |   const endOffset = lastOffset + lastSentence.length;
123 |
124 |   chunks.push({
125 |     text: chunkTextContent,
126 |     start_offset: startOffset,
127 |     end_offset: endOffset,
128 |     token_count_estimate: estimateTokenCount(chunkTextContent),
129 |   });
130 | };
131 |
132 | let sentenceIdx = 0;
133 | while (sentenceIdx < sentences.length) {
134 |   const sentence = sentences[sentenceIdx]!;
135 |   const sentenceTokens = estimateTokenCount(sentence);
136 |
137 |   if (
138 |     sentenceTokens > MAX_TOKENS_PER_CHUNK &&
139 |     currentSentences.length === 0
140 |   ) {
141 |     const maxChars = MAX_TOKENS_PER_CHUNK * 4;
142 |     const overlapChars = OVERLAP_TOKENS * 4;
143 |     let charPos = 0;
144 |     const sentenceStart = sentenceOffsets[sentenceIdx]!;
145 |
146 |     while (charPos < sentence.length) {
147 |       const end = Math.min(charPos + maxChars, sentence.length);
148 |       const slice = sentence.slice(charPos, end);
149 |
150 |       chunks.push({
151 |         text: slice,
152 |         start_offset: sentenceStart + charPos,
153 |         end_offset: sentenceStart + end,
154 |         token_count_estimate: estimateTokenCount(slice),
155 |       });
156 |
157 |       charPos = end === sentence.length ? end : end - overlapChars;
158 |     }
159 |
160 |     sentenceIdx++;
161 |     continue;
162 |   }
163 |
164 |   if (
165 |     currentTokens + sentenceTokens > MAX_TOKENS_PER_CHUNK &&
166 |     currentSentences.length > 0
167 |   ) {
168 |     flushChunk();
169 |
170 |     const overlapSentences: string[] = [];
171 |     let overlapCount = 0;
172 |     for (let i = currentSentences.length - 1; i >= 0; i--) {
173 |       const s = currentSentences[i]!;
174 |       const sTokens = estimateTokenCount(s);
175 |       if (overlapCount + sTokens > OVERLAP_TOKENS) break;
176 |       overlapSentences.unshift(s);
177 |       overlapCount += sTokens;
178 |     }
179 |
180 |     currentSentences = [...overlapSentences];
181 |     currentTokens = overlapCount;
182 |     continue;
183 |   }
184 |
185 |   currentSentences.push(sentence);
186 |   currentTokens += sentenceTokens;
187 |   sentenceIdx++;
188 | }
189 |
190 | flushChunk();
191 | return chunks;
192 | }
193 |
194 | function validateRequest(
195 |   body: unknown

```

```

196 ): { valid: true; data: EmbedRequest } | { valid: false; error: string } {
197   if (!body || typeof body !== "object") {
198     return { valid: false, error: "Request body must be a JSON object" };
199   }
200
201   const req = body as Record<string, unknown>;
202
203   if (!req.file_url || typeof req.file_url !== "string") {
204     return { valid: false, error: "file_url is required and must be a string" };
205   }
206
207   if (!req.course_id || typeof req.course_id !== "string") {
208     return {
209       valid: false,
210       error: "course_id is required and must be a string",
211     };
212   }
213
214   if (!req.source_filename || typeof req.source_filename !== "string") {
215     return {
216       valid: false,
217       error: "source_filename is required and must be a string",
218     };
219   }
220
221   if (!req.material_type || typeof req.material_type !== "string") {
222     return {
223       valid: false,
224       error: "material_type is required and must be a string",
225     };
226   }
227
228   if (!VALID_MATERIAL_TYPES.includes(req.material_type as MaterialType)) {
229     return {
230       valid: false,
231       error: `material_type must be one of: ${VALID_MATERIAL_TYPES.join(", ")}`,
232     };
233   }
234
235   if (req.clo_ids !== undefined) {
236     if (
237       !Array.isArray(req.clo_ids) ||
238       !req.clo_ids.every((id: unknown) => typeof id === "string")
239     ) {
240       return { valid: false, error: "clo_ids must be an array of strings" };
241     }
242   }
243
244   if (req.bloom_level !== undefined && typeof req.bloom_level !== "string") {
245     return { valid: false, error: "bloom_level must be a string" };
246   }
247
248   return {
249     valid: true,
250     data: {
251       file_url: req.file_url as string,
252       course_id: req.course_id as string,
253       clo_ids: (req.clo_ids as string[] | undefined) ?? [],
254       bloom_level: req.bloom_level as string | undefined,
255       material_type: req.material_type as MaterialType,
256       source_filename: req.source_filename as string,
257       source_material_id: req.source_material_id as string | undefined,
258       institution_id: req.institution_id as string | undefined,
259       reindex: req.reindex === true,
260     },
261   };
262 }
263
264 function validateAutoIndexRequest(
265   body: unknown
266 ): { valid: true; data: AutoIndexRequest } | { valid: false; error: string } {
267   if (!body || typeof body !== "object") {
268     return { valid: false, error: "Request body must be a JSON object" };
269   }
270
271   const req = body as Record<string, unknown>;
272
273   if (!req.course_id || typeof req.course_id !== "string") {
274     return {
275       valid: false,
276       error: "course_id is required and must be a string",
277     };
278   }
279
280   if (!req.title || typeof req.title !== "string") {
281     return { valid: false, error: "title is required and must be a string" };
282   }
283
284   if (!req.material_type || typeof req.material_type !== "string") {
285     return {
286       valid: false,
287       error: "material_type is required and must be a string",
288     };
289   }
290
291   const validAutoTypes: MaterialType[] = [
292     "assignment_description",
293     "rubric_criteria",
294   ];
295   if (!validAutoTypes.includes(req.material_type as MaterialType)) {
296     return {
297       valid: false,
298       error: `material_type for auto-indexing must be one of: ${validAutoTypes.join(
299         ", "

```

[illegible]

```

404 | describe("embed-course-material: Sentence Splitting", () => {
405 |   it("returns empty array for empty string", () => {
406 |     expect(splitIntoSentences("")).toEqual([]);
407 |   });
408 |
409 |   it("splits on sentence-ending punctuation", () => {
410 |     const result = splitIntoSentences("Hello world. How are you? Fine!");
411 |     expect(result).toHaveLength(3);
412 |     expect(result[0]).toBe("Hello world.");
413 |     expect(result[1]).toBe("How are you?");
414 |     expect(result[2]).toBe("Fine!");
415 |   });
416 |
417 |   it("handles text without sentence endings", () => {
418 |     const result = splitIntoSentences("No punctuation here");
419 |     expect(result).toHaveLength(1);
420 |     expect(result[0]).toBe("No punctuation here");
421 |   });
422 | });
423 |
424 | describe("embed-course-material: Text Chunking (3.2.2)", () => {
425 |   it("returns empty array for empty text", () => {
426 |     expect(chunkText("")).toEqual([]);
427 |     expect(chunkText("   ")).toEqual([]);
428 |   });
429 |
430 |   it("returns single chunk for short text", () => {
431 |     const text = "This is a short text.";
432 |     const chunks = chunkText(text);
433 |     expect(chunks).toHaveLength(1);
434 |     expect(chunks[0]!.text).toBe(text);
435 |     // TextChunk uses start_offset, not chunk_index
436 |     expect("chunk_index" in chunks[0]!).toBe(false);
437 |     expect(chunks[0]!.token_count_estimate).toBeGreaterThan(0);
438 |   });
439 |
440 |   it("produces multiple chunks for long text", () => {
441 |     // Generate text that exceeds MAX_TOKENS_PER_CHUNK (500 tokens "H 2000 chars)
442 |     const sentences = Array.from(
443 |       { length: 50 },
444 |       (_, i) =>
445 |         `This is sentence number ${
446 |           i + 1
447 |         } which contains some meaningful content about the topic.`
448 |     );
449 |     const text = sentences.join(" ");
450 |     const chunks = chunkText(text);
451 |
452 |     expect(chunks.length).toBeGreaterThan(1);
453 |
454 |     // Each chunk should have valid token count
455 |     for (const chunk of chunks) {
456 |       expect(chunk.token_count_estimate).toBeGreaterThan(0);
457 |       expect(chunk.text.length).toBeGreaterThan(0);
458 |     }
459 |   });
460 |
461 |   it("chunk token counts do not exceed MAX_TOKENS_PER_CHUNK", () => {
462 |     const sentences = Array.from(
463 |       { length: 80 },
464 |       (_, i) =>
465 |         `Sentence ${
466 |           i + 1
467 |         } discusses important concepts in computer science and mathematics.`
468 |     );
469 |     const text = sentences.join(" ");
470 |     const chunks = chunkText(text);
471 |
472 |     for (const chunk of chunks) {
473 |       // Allow small overflow due to sentence boundary preservation
474 |       expect(chunk.token_count_estimate).toBeLessThanOrEqual(
475 |         MAX_TOKENS_PER_CHUNK + 50
476 |       );
477 |     }
478 |   });
479 |
480 |   it("preserves all text content across chunks", () => {
481 |     const sentences = Array.from(
482 |       { length: 30 },
483 |       (_, i) => `Point ${i + 1} is about learning outcomes.`
484 |     );
485 |     const text = sentences.join(" ");
486 |     const chunks = chunkText(text);
487 |
488 |     // Every sentence should appear in at least one chunk
489 |     for (const sentence of sentences) {
490 |       const found = chunks.some((c) => c.text.includes(sentence));
491 |       expect(found).toBe(true);
492 |     }
493 |   });
494 | });
495 |
496 | describe("embed-course-material: Request Validation (3.2.1)", () => {
497 |   const validRequest = {
498 |     file_url: "https://example.com/file.pdf",
499 |     course_id: "course-123",
500 |     source_filename: "lecture.pdf",
501 |     material_type: "lecture_notes",
502 |   };
503 |
504 |   it("accepts a valid request", () => {
505 |     const result = validateRequest(validRequest);
506 |     expect(result.valid).toBe(true);
507 |

```

```

508 |     if (result.valid) {
509 |         expect(result.data.file_url).toBe(validRequest.file_url);
510 |         expect(result.data.course_id).toBe(validRequest.course_id);
511 |         expect(result.data.material_type).toBe("lecture_notes");
512 |     }
513 | });
514 |
515 | it("rejects null body", () => {
516 |     const result = validateRequest(null);
517 |     expect(result.valid).toBe(false);
518 | });
519 |
520 | it("rejects missing file_url", () => {
521 |     const result = validateRequest({
522 |         course_id: "c1",
523 |         source_filename: "f.pdf",
524 |         material_type: "other",
525 |     });
526 |     expect(result.valid).toBe(false);
527 |     if (!result.valid) expect(result.error).toContain("file_url");
528 | });
529 |
530 | it("rejects missing course_id", () => {
531 |     const result = validateRequest({
532 |         file_url: "url",
533 |         source_filename: "f.pdf",
534 |         material_type: "other",
535 |     });
536 |     expect(result.valid).toBe(false);
537 |     if (!result.valid) expect(result.error).toContain("course_id");
538 | });
539 |
540 | it("rejects missing source_filename", () => {
541 |     const result = validateRequest({
542 |         file_url: "url",
543 |         course_id: "c1",
544 |         material_type: "other",
545 |     });
546 |     expect(result.valid).toBe(false);
547 |     if (!result.valid) expect(result.error).toContain("source_filename");
548 | });
549 |
550 | it("rejects invalid material_type", () => {
551 |     const result = validateRequest({
552 |         ...validRequest,
553 |         material_type: "invalid_type",
554 |     });
555 |     expect(result.valid).toBe(false);
556 |     if (!result.valid) expect(result.error).toContain("material_type");
557 | });
558 |
559 | it("accepts all valid material types", () => {
560 |     for (const mt of VALID_MATERIAL_TYPES) {
561 |         const result = validateRequest({ ...validRequest, material_type: mt });
562 |         expect(result.valid).toBe(true);
563 |     }
564 | });
565 |
566 | it("rejects non-array clo_ids", () => {
567 |     const result = validateRequest({ ...validRequest, clo_ids: "not-array" });
568 |     expect(result.valid).toBe(false);
569 |     if (!result.valid) expect(result.error).toContain("clo_ids");
570 | });
571 |
572 | it("rejects clo_ids with non-string elements", () => {
573 |     const result = validateRequest({ ...validRequest, clo_ids: [123] });
574 |     expect(result.valid).toBe(false);
575 | });
576 |
577 | it("accepts optional clo_ids array", () => {
578 |     const result = validateRequest({
579 |         ...validRequest,
580 |         clo_ids: ["clo-1", "clo-2"],
581 |     });
582 |     expect(result.valid).toBe(true);
583 |     if (result.valid) {
584 |         expect(result.data.clo_ids).toEqual(["clo-1", "clo-2"]);
585 |     }
586 | });
587 |
588 | it("defaults clo_ids to empty array when not provided", () => {
589 |     const result = validateRequest(validRequest);
590 |     expect(result.valid).toBe(true);
591 |     if (result.valid) {
592 |         expect(result.data.clo_ids).toEqual([]);
593 |     }
594 | });
595 |
596 | it("parses reindex flag correctly", () => {
597 |     const result = validateRequest({ ...validRequest, reindex: true });
598 |     expect(result.valid).toBe(true);
599 |     if (result.valid) {
600 |         expect(result.data.reindex).toBe(true);
601 |     }
602 | });
603 |
604 | it("defaults reindex to false", () => {
605 |     const result = validateRequest(validRequest);
606 |     expect(result.valid).toBe(true);
607 |     if (result.valid) {
608 |         expect(result.data.reindex).toBe(false);
609 |     }
610 | });
611 | });

```



```

612 describe("embed-course-material: Auto-Index Validation (3.2.8)", () => {
613   const validAutoRequest = {
614     course_id: "course-123",
615     title: "Assignment 1: Data Structures",
616     material_type: "assignment_description",
617     source_filename: "assignment-1",
618   };
619   it("accepts a valid auto-index request", () => {
620     const result = validateAutoIndexRequest(validAutoRequest);
621     expect(result.valid).toBe(true);
622     if (result.valid) {
623       expect(result.data.title).toBe(validAutoRequest.title);
624       expect(result.data.material_type).toBe("assignment_description");
625     }
626   });
627   it("rejects missing title", () => {
628     const { title, ...noTitle } = validAutoRequest;
629     void title; // intentionally omitted from noTitle
630     const result = validateAutoIndexRequest(noTitle);
631     expect(result.valid).toBe(false);
632     if (!result.valid) expect(result.error).toContain("title");
633   });
634   it("rejects missing course_id", () => {
635     const { course_id, ...noCourse } = validAutoRequest;
636     void course_id; // intentionally omitted from noCourse
637     const result = validateAutoIndexRequest(noCourse);
638     expect(result.valid).toBe(false);
639     if (!result.valid) expect(result.error).toContain("course_id");
640   });
641   it("rejects invalid material_type for auto-indexing", () => {
642     const result = validateAutoIndexRequest({
643       ...validAutoRequest,
644       material_type: "lecture_notes",
645     });
646     expect(result.valid).toBe(false);
647     if (!result.valid) expect(result.error).toContain("material_type");
648   });
649   it("accepts rubric_criteria material type", () => {
650     const result = validateAutoIndexRequest({
651       ...validAutoRequest,
652       material_type: "rubric_criteria",
653     });
654     expect(result.valid).toBe(true);
655   });
656   it("accepts optional description", () => {
657     const result = validateAutoIndexRequest({
658       ...validAutoRequest,
659       description: "Implement a binary search tree.",
660     });
661     expect(result.valid).toBe(true);
662     if (result.valid) {
663       expect(result.data.description).toBe("Implement a binary search tree.");
664     }
665   });
666   it("rejects non-string description", () => {
667     const result = validateAutoIndexRequest({
668       ...validAutoRequest,
669       description: 123,
670     });
671     expect(result.valid).toBe(false);
672     if (!result.valid) expect(result.error).toContain("description");
673   });
674   it("accepts valid rubric_criteria array", () => {
675     const result = validateAutoIndexRequest({
676       ...validAutoRequest,
677       rubric_criteria: [
678         {
679           criterion: "Correctness",
680           description: "Code produces correct output",
681         },
682         { criterion: "Style" },
683       ],
684     });
685     expect(result.valid).toBe(true);
686     if (result.valid) {
687       expect(result.data.rubric_criteria).toHaveLength(2);
688     }
689   });
690   it("rejects rubric_criteria with missing criterion field", () => {
691     const result = validateAutoIndexRequest({
692       ...validAutoRequest,
693       rubric_criteria: [{ description: "Missing criterion field" }],
694     });
695     expect(result.valid).toBe(false);
696     if (!result.valid) expect(result.error).toContain("criterion");
697   });
698   it("rejects non-array rubric_criteria", () => {
699     const result = validateAutoIndexRequest({
700       ...validAutoRequest,
701       rubric_criteria: "not-array",
702     });
703     expect(result.valid).toBe(false);
704   });
705 }

```

```

716 | });
717 |
718 | describe("embed-course-material: Auto-Index Text Assembly (3.2.8)", () => {
719 |   it("assembles text from title only", () => {
720 |     const result = assembleAutoIndexText({
721 |       course_id: "c1",
722 |       title: "Assignment 1",
723 |       material_type: "assignment_description",
724 |       source_filename: "a1",
725 |     });
726 |     expect(result).toBe("Assignment 1");
727 |   });
728 |
729 |   it("assembles text from title and description", () => {
730 |     const result = assembleAutoIndexText({
731 |       course_id: "c1",
732 |       title: "Assignment 1",
733 |       description: "Implement a linked list.",
734 |       material_type: "assignment_description",
735 |       source_filename: "a1",
736 |     });
737 |     expect(result).toContain("Assignment 1");
738 |     expect(result).toContain("Implement a linked list.");
739 |   });
740 |
741 |   it("assembles text from title, description, and rubric criteria", () => {
742 |     const result = assembleAutoIndexText({
743 |       course_id: "c1",
744 |       title: "Final Project",
745 |       description: "Build a web application.",
746 |       material_type: "assignment_description",
747 |       source_filename: "fp",
748 |       rubric_criteria: [
749 |         { criterion: "Functionality", description: "App works correctly" },
750 |         { criterion: "Design", description: "Clean UI" },
751 |       ],
752 |     });
753 |     expect(result).toContain("Final Project");
754 |     expect(result).toContain("Build a web application.");
755 |     expect(result).toContain("Rubric Criteria:");
756 |     expect(result).toContain("Functionality: App works correctly");
757 |     expect(result).toContain("Design: Clean UI");
758 |   });
759 |
760 |   it("handles rubric criteria without descriptions", () => {
761 |     const result = assembleAutoIndexText({
762 |       course_id: "c1",
763 |       title: "Quiz 1",
764 |       material_type: "rubric_criteria",
765 |       source_filename: "q1",
766 |       rubric_criteria: [
767 |         { criterion: "Accuracy" },
768 |         { criterion: "Completeness" },
769 |       ],
770 |     });
771 |     expect(result).toContain("Accuracy");
772 |     expect(result).toContain("Completeness");
773 |     // Criteria without descriptions should not have "criterion: description" format
774 |     expect(result).not.toContain("Accuracy:");
775 |     expect(result).not.toContain("Completeness:");
776 |   });
777 |
778 |   it("skips empty description", () => {
779 |     const result = assembleAutoIndexText({
780 |       course_id: "c1",
781 |       title: "Test",
782 |       description: " ",
783 |       material_type: "assignment_description",
784 |       source_filename: "t",
785 |     });
786 |     expect(result).toBe("Test");
787 |   });
788 |
789 |   it("skips empty rubric_criteria array", () => {
790 |     const result = assembleAutoIndexText({
791 |       course_id: "c1",
792 |       title: "Test",
793 |       material_type: "assignment_description",
794 |       source_filename: "t",
795 |       rubric_criteria: [],
796 |     });
797 |     expect(result).toBe("Test");
798 |   });
799 | });
800 |
801 | describe("embed-course-material: Large Document Detection (3.2.5)", () => {
802 |   it("detects documents over 100 pages", () => {
803 |     // 100 pages * 2000 chars/page = 200,000 chars
804 |     const largeText = "x".repeat(200_001);
805 |     const estimatedPages = Math.ceil(
806 |       largeText.length / CHARS_PER_PAGE_ESTIMATE
807 |     );
808 |     expect(estimatedPages).toBeGreaterThan(LARGE_DOCUMENT_PAGE_THRESHOLD);
809 |   });
810 |
811 |   it("does not flag small documents", () => {
812 |     const smallText = "x".repeat(1000);
813 |     const estimatedPages = Math.ceil(
814 |       smallText.length / CHARS_PER_PAGE_ESTIMATE
815 |     );
816 |     expect(estimatedPages).toBeLessThanOrEqual(LARGE_DOCUMENT_PAGE_THRESHOLD);
817 |   });
818 | });
819 |

```

TYPE: .tsx | LINES: 1-37 | SHA256: e58902d6c52f5b2a20e1e5f271aae7bda1de0e297cd2c9d244f39f0005b01d01

FILE: src/__tests__/unit/errorStateUploadProgress.test.tsx

TYPE: .tsx | LINES: 1-281 | SHA256: 507c6927982961d5f19cd3bbdc3fb5c4edd387be62d800dbeb968405a5e4f90c

```
//
// ErrorState, UploadProgress, ReconnectBanner - Unit tests
// Validates: Requirements 66.1, 66.2, 66.3, 66.4
=====

import { describe, it, expect, vi } from "vitest";
import { render, screen, fireEvent } from "@testing-library/react";
import ErrorMessage from "@components/shared/ErrorState";
import UploadProgress, {
    formatFileSize,
} from "@components/shared/UploadProgress";
import ReconnectBanner from "@components/shared/ReconnectBanner";
import RealtimeStatusBanner from "@components/shared/RealtimeStatusBanner";
import { AlertTriangle } from "lucide-react";

describe("ErrorState", () => {
    it('renders default title "Something went wrong"', () => {
        const wrapper = render(
            <ErrorMessage message="Network error" />,
        );
        expect(screen.getByText("Something went wrong")).toBeInTheDocument();
    });

    it("renders custom title", () => {
        const wrapper = render(
            <ErrorMessage title="Upload Failed" message="Try again" />;
        );
        expect(screen.getByText("Upload Failed")).toBeInTheDocument();
    });

    it("renders the error message", () => {
        const wrapper = render(
            <ErrorMessage message="Connection timed out" />;
        );
        expect(screen.getByText("Connection timed out")).toBeInTheDocument();
    });

    it("renders retry button when onRetry is provided", () => {
        const onRetry = vi.fn();
        const wrapper = render(
            <ErrorMessage message="Error" onRetry={onRetry} />;
        );
        const btn = screen.getByText("Try Again");
        expect(btn).toBeInTheDocument();
        fireEvent.click(btn);
        expect(onRetry).toHaveBeenCalledOnce();
    });

    it("does not render retry button when onRetry is not provided", () => {
        const wrapper = render(
            <ErrorMessage message="Error" />;
        );
        expect(screen.queryByText("Try Again")).not.toBeInTheDocument();
    });

    it("renders custom retry label", () => {
        const wrapper = render(
            <ErrorMessage
                message="Error"
                onRetry={() => {}}
                retryLabel="Retry Upload"
            />,
        );
        expect(screen.getByText("Retry Upload")).toBeInTheDocument();
    });

    it("renders custom icon", () => {
```

[illegible]

[illegible]

```

268 |     it("renders ReconnectBanner when isLive is false", () => {
269 |         render(<RealtimeStatusBanner isLive={false} retryCount={2} />);
270 |         expect(screen.getByText(/Live updates paused/)).toBeInTheDocument();
271 |         expect(screen.getByText("attempt 2")).toBeInTheDocument();
272 |     });
273 |
274 |
275 |     it("defaults retryCount to 0", () => {
276 |         render(<RealtimeStatusBanner isLive={false} />);
277 |         expect(screen.getByText(/Live updates paused/)).toBeInTheDocument();
278 |         expect(screen.queryByText(/attempt/)).not.toBeInTheDocument();
279 |     });
280 | });
281 |

```

FILE: src/__tests__/unit/evidenceUploader.test.tsx

TYPE: .tsx | LINES: 1-313 | SHA256: 095f82c09d5bcf3529789d920233701560854e7331cd14bdb210673706c8a8bb

[illegible]

```

87 | });
88 |
89 | it("accepts PDF files", async () => {
90 |   const user = userEvent.setup();
91 |   render(<EvidenceUploader files={[]} onChange={onChange} />);
92 |
93 |   const file = createMockFile("notes.pdf", 2048, "application/pdf");
94 |   const input = document.querySelector(
95 |     'input[type="file"]'
96 |   ) as HTMLInputElement;
97 |
98 |   await user.upload(input, file);
99 |
100 |   expect(onChange).toHaveBeenCalledTimes(1);
101 | });
102 |
103 | it("accepts DOC files", async () => {
104 |   const user = userEvent.setup();
105 |   render(<EvidenceUploader files={[]} onChange={onChange} />);
106 |
107 |   const file = createMockFile("essay.doc", 2048, "application/msword");
108 |   const input = document.querySelector(
109 |     'input[type="file"]'
110 |   ) as HTMLInputElement;
111 |
112 |   await user.upload(input, file);
113 |
114 |   expect(onChange).toHaveBeenCalledTimes(1);
115 | });
116 |
117 | it("accepts DOCX files", async () => {
118 |   const user = userEvent.setup();
119 |   render(<EvidenceUploader files={[]} onChange={onChange} />);
120 |
121 |   const file = createMockFile(
122 |     "report.docx",
123 |     2048,
124 |     "application/vnd.openxmlformats-officedocument.wordprocessingml.document"
125 |   );
126 |   const input = document.querySelector(
127 |     'input[type="file"]'
128 |   ) as HTMLInputElement;
129 |
130 |   await user.upload(input, file);
131 |
132 |   expect(onChange).toHaveBeenCalledTimes(1);
133 | });
134 |
135 | describe("File Validation", () => {
136 |   it("rejects files exceeding 5MB", async () => {
137 |     const user = userEvent.setup();
138 |     render(<EvidenceUploader files={[]} onChange={onChange} />);
139 |
140 |     const largeFile = createMockFile(
141 |       "huge.jpg",
142 |       MAX_FILE_SIZE_BYTES + 1,
143 |       "image/jpeg"
144 |     );
145 |     const input = document.querySelector(
146 |       'input[type="file"]'
147 |     ) as HTMLInputElement;
148 |
149 |     await user.upload(input, largeFile);
150 |
151 |     // onChange should NOT be called with the oversized file
152 |     expect(onChange).not.toHaveBeenCalled();
153 |     // Error message should appear
154 |     expect(screen.getByText(/exceeds 5mb limit/i)).toBeTruthy();
155 |   });
156 |
157 |   it("rejects unsupported file types via drag-and-drop validation", () => {
158 |     // The file input's accept attribute prevents unsupported types via browse,
159 |     // but drag-and-drop bypasses accept. Our validation catches those.
160 |     const exeFile = createMockFile(
161 |       "virus.exe",
162 |       1024,
163 |       "application/x-msdownload"
164 |     );
165 |
166 |     // Simulate adding an unsupported file directly (as if via drag-and-drop)
167 |     // by passing it through the component's validation
168 |     render(<EvidenceUploader files={[]} onChange={onChange} />);
169 |
170 |     const dropZone = screen.getByLabelText(/upload evidence files/i);
171 |
172 |     // Create a mock DataTransfer with the unsupported file
173 |     const dataTransfer = {
174 |       files: [exeFile],
175 |       items: [{ kind: "file", type: exeFile.type, getAsFile: () => exeFile }],
176 |       types: ["Files"],
177 |     };
178 |
179 |     // Fire drop event
180 |     dropZone.dispatchEvent(
181 |       Object.assign(new Event("drop", { bubbles: true }), {
182 |         dataTransfer,
183 |       })
184 |     );
185 |
186 |     // The onChange should not be called with unsupported files
187 |     expect(onChange).not.toHaveBeenCalled();
188 |   });
189 | });
190 |

```

```

describe("File List Display", () => {
  it("renders file thumbnails for selected files", () => {
    const files = [
      createMockFile("photo.jpg", 1024, "image/jpeg"),
      createMockFile("notes.pdf", 2048, "application/pdf"),
    ];

    render(<EvidenceUploader files={files} onChange={onChange} />);

    expect(screen.getByText("photo.jpg")).toBeTruthy();
    expect(screen.getByText("notes.pdf")).toBeTruthy();
    expect(screen.getByText(/2 of 3 files selected/i)).toBeTruthy();
  });

  it("shows file size for each file", () => {
    const files = [createMockFile("doc.pdf", 2048, "application/pdf")];

    render(<EvidenceUploader files={files} onChange={onChange} />);

    expect(screen.getByText("2.0 KB")).toBeTruthy();
  });

  it("renders remove button for each file", () => {
    const files = [
      createMockFile("photo.jpg", 1024, "image/jpeg"),
      createMockFile("notes.pdf", 2048, "application/pdf"),
    ];

    render(<EvidenceUploader files={files} onChange={onChange} />);

    const removeButtons = screen.getAllByRole("button", { name: /remove/i });
    expect(removeButtons).toHaveLength(2);
  });

  it("calls onChange without the removed file when remove is clicked", async () => {
    const user = userEvent.setup();
    const files = [
      createMockFile("photo.jpg", 1024, "image/jpeg"),
      createMockFile("notes.pdf", 2048, "application/pdf"),
    ];

    render(<EvidenceUploader files={files} onChange={onChange} />);

    const removeButtons = screen.getAllByRole("button", { name: /remove/i });
    await user.click(removeButtons[0]);

    expect(onChange).toHaveBeenCalledTimes(1);
    const updatedFiles = onChange.mock.calls[0][0] as File[];
    expect(updatedFiles).toHaveLength(1);
    expect(updatedFiles[0].name).toBe("notes.pdf");
  });
});

describe("Max Files Limit", () => {
  it("hides drop zone when max files reached", () => {
    const files = [
      createMockFile("a.jpg", 1024, "image/jpeg"),
      createMockFile("b.jpg", 1024, "image/jpeg"),
      createMockFile("c.jpg", 1024, "image/jpeg"),
    ];

    render(<EvidenceUploader files={files} onChange={onChange} />);

    expect(screen.queryByText(/drag & drop/i)).toBeNull();
    expect(screen.getByText(/3 of 3 files selected/i)).toBeTruthy();
  });

  it("shows error when trying to add more than max files", async () => {
    const user = userEvent.setup();
    const existingFiles = [
      createMockFile("a.jpg", 1024, "image/jpeg"),
      createMockFile("b.jpg", 1024, "image/jpeg"),
    ];

    render(<EvidenceUploader files={existingFiles} onChange={onChange} />);

    // Try to add 2 more files (only 1 slot available)
    const file1 = createMockFile("c.jpg", 1024, "image/jpeg");
    const file2 = createMockFile("d.jpg", 1024, "image/jpeg");
    const input = document.querySelector(
      'input[type="file"]'
    ) as HTMLInputElement;

    await user.upload(input, [file1, file2]);

    // Should add only 1 file and show error
    expect(onChange).toHaveBeenCalledTimes(1);
    const newFiles = onChange.mock.calls[0][0] as File[];
    expect(newFiles).toHaveLength(3); // 2 existing + 1 new
    expect(screen.getByText(/maximum 3 files allowed/i)).toBeTruthy();
  });
});

describe("Disabled State", () => {
  it("disables the drop zone when disabled prop is true", () => {
    render(<EvidenceUploader files={[]} onChange={onChange} disabled />);

    const dropZone = screen.getByLabelText(/upload evidence files/i);
    expect(dropZone.className).toContain("opacity-50");
  });

  it("disables remove buttons when disabled", () => {
    const files = [createMockFile("photo.jpg", 1024, "image/jpeg")];
  });
});

```



```

295 |         render(<EvidenceUploader files={files} onChange={onChange} disabled />);
296 |
297 |         const removeBtn = screen.getByRole("button", { name: /remove/i });
298 |         expect(removeBtn).toBeDisabled();
299 |     });
300 | });
301 |
302 | describe("Constants", () => {
303 |     it("exports correct MAX_FILES value", () => {
304 |         expect(MAX_FILES).toBe(3);
305 |     });
306 |
307 |     it("exports correct MAX_FILE_SIZE_BYTES value (5MB)", () => {
308 |         expect(MAX_FILE_SIZE_BYTES).toBe(5 * 1024 * 1024);
309 |     });
310 | });
311 | });
312 | });
313 |

```

FILE: [src/__tests__/unit/explanationConfidence.test.ts](#)

TYPE: .ts | LINES: 1-71 | SHA256: eed3d5687a956d58b96f02ef478639e6a90856eadfc0480046bb8e5670926c39

```

1 | import { describe, it, expect } from "vitest";
2 | import {
3 |   computeExplanationConfidence,
4 |   needsTeacherVerification,
5 |   isFrequentlyMissed,
6 | } from "@lib/explanationConfidence";
7 |
8 | describe("computeExplanationConfidence", () => {
9 |   it("returns 0 for an empty array", () => {
10 |     expect(computeExplanationConfidence([])).toBe(0);
11 |   });
12 |
13 |   it("returns the single value when given one similarity", () => {
14 |     expect(computeExplanationConfidence([0.75])).toBe(0.75);
15 |   });
16 |
17 |   it("returns the average of 3 values", () => {
18 |     const result = computeExplanationConfidence([0.9, 0.8, 0.7]);
19 |     expect(result).toBeCloseTo(0.8, 10);
20 |   });
21 |
22 |   it("uses only the top 3 values when given more than 3", () => {
23 |     // Top 3: 0.95, 0.85, 0.80 !' average = 2.60 / 3 "H 0.8667
24 |     const result = computeExplanationConfidence([0.5, 0.85, 0.95, 0.3, 0.8]);
25 |     expect(result).toBeCloseTo((0.95 + 0.85 + 0.8) / 3, 10);
26 |   });
27 |
28 |   it("returns the correct value when all similarities are the same", () => {
29 |     expect(computeExplanationConfidence([0.6, 0.6, 0.6, 0.6])).toBeCloseTo(
30 |       0.6,
31 |       10
32 |     );
33 |   });
34 | });
35 |
36 | describe("needsTeacherVerification", () => {
37 |   it("returns true when confidence is below 0.8", () => {
38 |     expect(needsTeacherVerification(0.5)).toBe(true);
39 |   });
40 |
41 |   it("returns false when confidence is exactly 0.8", () => {
42 |     expect(needsTeacherVerification(0.8)).toBe(false);
43 |   });
44 |
45 |   it("returns false when confidence is above 0.8", () => {
46 |     expect(needsTeacherVerification(0.95)).toBe(false);
47 |   });
48 |
49 |   it("returns true when confidence is 0", () => {
50 |     expect(needsTeacherVerification(0)).toBe(true);
51 |   });
52 | });
53 |
54 | describe("isFrequentlyMissed", () => {
55 |   it("returns true when success_rate < 0.5 and attempts >= 10", () => {
56 |     expect(isFrequentlyMissed(0.3, 15)).toBe(true);
57 |   });
58 |
59 |   it("returns false when success_rate >= 0.5", () => {
60 |     expect(isFrequentlyMissed(0.5, 20)).toBe(false);
61 |   });
62 |
63 |   it("returns false when attempts < 10", () => {
64 |     expect(isFrequentlyMissed(0.2, 5)).toBe(false);
65 |   });
66 |
67 |   it("returns false when both conditions are false", () => {
68 |     expect(isFrequentlyMissed(0.8, 3)).toBe(false);
69 |   });
70 | });
71 |

```

FILE: [src/__tests__/unit/explanationConfidenceBadge.test.tsx](#)

TYPE: .tsx | LINES: 1-178 | SHA256: cf90eca7352d41bd180a0b818cb9897a864d78f96e71896b13169bf5f44560ef

```

1 | // =====
2 | // ExplanationConfidenceBadge - Unit tests
3 | // Validates: Task 19.6 - Green/amber/blue badge rendering based on confidence
4 | // =====
5 |
6 | import { describe, it, expect } from "vitest";

```

[illegible]

[illegible]

FILE: src/__tests__/unit/exportCurriculumMatrixCsv.test.ts

TYPE: .ts | LINES: 1-165 | SHA256: b7fa0a2d2198f8684c04ab82991f0fd0abe180dbb63bf9ebcf2effb97aa83257

[illegible]

[illegible]

```

137 |
138 |     expect(lines[1]).toBe('"Analyze, Evaluate, Create",2');
139 |   });
140 |
141 |   it("defaults to 0 when a matrix cell is missing", () => {
142 |     const data: CurriculumMatrixData = {
143 |       plos: [makePlo({ id: "plo-1", title: "PLO A" })],
144 |       courses: [makeCourse({ id: "c1", code: "CS101" })],
145 |       matrix: {}, // empty matrix
146 |     };
147 |
148 |     const csv = buildMatrixCsv(data);
149 |     const lines = csv.split("\n");
150 |
151 |     expect(lines[1]).toBe("PLO A,0");
152 |   });
153 |
154 |   it("produces header-only CSV when there are no PLOs", () => {
155 |     const data: CurriculumMatrixData = {
156 |       plos: [],
157 |       courses: [makeCourse({ id: "c1", code: "CS101" })],
158 |       matrix: {},
159 |     };
160 |
161 |     const csv = buildMatrixCsv(data);
162 |     expect(csv).toBe("PLO,CS101");
163 |   });
164 | });
165 |

```

FILE: src/__tests__/unit/exportDataButton.test.tsx

TYPE: .tsx | LINES: 1-137 | SHA256: 24b32e547d07b798c5284f25282fe05440b40c957ca15d8817f832d0c3d4aab1

[illegible]

```

72 |         expect(window.open).toHaveBeenCalledWith(
73 |             "https://storage.example.com/export.json",
74 |             "_blank"
75 |         );
76 |     });
77 |
78 |     await waitFor(() => {
79 |         expect(toast.success).toHaveBeenCalledWith("Data exported as JSON");
80 |     });
81 | });
82 |
83 | it("shows error toast when Edge Function returns an error", async () => {
84 |     mockInvoke.mockResolvedValue({
85 |         data: null,
86 |         error: { message: "Internal server error" },
87 |     });
88 |
89 |     render(<ExportDataButton studentId="student-1" />);
90 |     fireEvent.click(screen.getByRole("button", { name: /download my data/i }));
91 |
92 |     await waitFor(() => {
93 |         expect(toast.error).toHaveBeenCalledWith("Internal server error");
94 |     });
95 | });
96 |
97 | it("shows error toast when no download URL is returned", async () => {
98 |     mockInvoke.mockResolvedValue({
99 |         data: { download_url: null },
100 |         error: null,
101 |     });
102 |
103 |     render(<ExportDataButton studentId="student-1" />);
104 |     fireEvent.click(screen.getByRole("button", { name: /download my data/i }));
105 |
106 |     await waitFor(() => {
107 |         expect(toast.error).toHaveBeenCalledWith("No download URL returned");
108 |     });
109 | });
110 |
111 | it("disables button during export", async () => {
112 |     let resolveExport: (value: unknown) => void;
113 |     const exportPromise = new Promise((resolve) => {
114 |         resolveExport = resolve;
115 |     });
116 |     mockInvoke.mockReturnValue(exportPromise);
117 |
118 |     render(<ExportDataButton studentId="student-1" />);
119 |     fireEvent.click(screen.getByRole("button", { name: /download my data/i }));
120 |
121 |     await waitFor(() => {
122 |         const btn = screen.getByRole("button", { name: /download my data/i });
123 |         expect(btn.hasAttribute("disabled")).toBe(true);
124 |     });
125 |
126 |     resolveExport!({
127 |         data: { download_url: "https://example.com/file.json" },
128 |         error: null,
129 |     });
130 |
131 |     await waitFor(() => {
132 |         const btn = screen.getByRole("button", { name: /download my data/i });
133 |         expect(btn.hasAttribute("disabled")).toBe(false);
134 |     });
135 | });
136 | });
137 |

```

FILE: [src/__tests__/unit/feeManager.test.tsx](#)

TYPE: .tsx | LINES: 1-247 | SHA256: a8031dfaa95ce0aa1e1dec7a232ca42c82297aca47e6dd0277e394550a984d97

```

1 | // @vitest-environment happy-dom
2 | // =====
3 | // FeeManager - Program/Semester dropdowns + disabled-until-selected submit
4 | //
5 | // Feature: qa-partner-review-remediation - Req 12 (P8)
6 | // Validates: Requirements 12.1, 12.2, 12.4
7 | //
8 | // The Admin Fees form previously used free-text `<Input placeholder="UUID">`
9 | // controls for `program_id` and `semester_id`. Req 12 replaces them with Shadcn
10 | // `Select` dropdowns sourced from `usePrograms()` / `useSemesters()`, feeds the
11 | // selected ids to `useCreateFeeStructure`, and disables submission (with a hint)
12 | // until BOTH a program and a semester are selected.
13 | //
14 | // This suite mocks the data/mutation hooks (no Supabase/network) and asserts:
15 | //   12.1 - a Program dropdown renders, populated from `usePrograms`
16 | //   12.2 - a Semester dropdown renders, populated from `useSemesters`
17 | //   12.4 - submit is disabled and a required-selection hint shows until both
18 | //           values are selected; selecting both enables submit
19 | // =====
20 |
21 | import { describe, it, expect, vi, beforeAll } from "vitest";
22 | import { render, screen, within } from "@testing-library/react";
23 | import userEvent from "@testing-library/user-event";
24 | import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
25 | import { MemoryRouter } from "react-router-dom";
26 | import type { Program, Semester } from "@types/app";
27 |
28 | // -----
29 | // Fixtures
30 | // -----
31 |
32 | const PROGRAMS: Program[] = [
33 |   {
34 |     id: "prog-1",

```

```

35 |     name: "Computer Science",
36 |     code: "CS",
37 |     description: null,
38 |     institution_id: "inst-1",
39 |     coordinator_id: null,
40 |     department_id: null,
41 |     is_active: true,
42 |     created_at: "2025-01-01",
43 |   },
44 |   {
45 |     id: "prog-2",
46 |     name: "Business Administration",
47 |     code: "BA",
48 |     description: null,
49 |     institution_id: "inst-1",
50 |     coordinator_id: null,
51 |     department_id: null,
52 |     is_active: true,
53 |     created_at: "2025-01-02",
54 |   },
55 | ];
56 |
57 | const SEMESTERS: Semester[] = [
58 |   {
59 |     id: "sem-1",
60 |     institution_id: "inst-1",
61 |     name: "Fall 2025",
62 |     code: "FA25",
63 |     start_date: "2025-09-01",
64 |     end_date: "2025-12-20",
65 |     is_active: true,
66 |     created_at: "2025-01-01",
67 |   },
68 |   {
69 |     id: "sem-2",
70 |     institution_id: "inst-1",
71 |     name: "Spring 2026",
72 |     code: "SP26",
73 |     start_date: "2026-02-01",
74 |     end_date: "2026-05-30",
75 |     is_active: false,
76 |     created_at: "2025-01-02",
77 |   },
78 | ];
79 |
80 | // -----
81 | // Mocks (must precede the component import)
82 | // -----
83 |
84 | const mockCreateMutate = vi.fn();
85 |
86 | vi.mock("@/hooks/useFees", () => ({
87 |   useFeeStructures: vi.fn(() => ({ data: [], isLoading: false })),
88 |   useCreateFeeStructure: vi.fn(() => ({
89 |     mutate: mockCreateMutate,
90 |     isPending: false,
91 |   })),
92 | }));
93 |
94 | vi.mock("@/hooks/usePrograms", () => ({
95 |   // FeeManager calls `usePrograms({ pageSize: 100 })` and reads `result?.data`.
96 |   usePrograms: vi.fn(() => ({ data: { data: PROGRAMS } })),
97 | }));
98 |
99 | vi.mock("@/hooks/useSemesters", () => ({
100 |   // FeeManager calls `useSemesters()` and reads the array directly.
101 |   useSemesters: vi.fn(() => ({ data: SEMESTERS })),
102 | }));
103 |
104 | vi.mock("@/hooks/useAuth", () => ({
105 |   useAuth: () => ({ user: { id: "u1" }, institutionId: "inst-1" }),
106 | }));
107 |
108 | // happy-dom lacks ResizeObserver + the pointer/scroll APIs Radix Select relies
109 | // on. Stub them so the Select triggers render and can be opened in tests.
110 | class MockResizeObserver {
111 |   observe() {}
112 |   unobserve() {}
113 |   disconnect() {}
114 | }
115 | vi.stubGlobal("ResizeObserver", MockResizeObserver);
116 |
117 | beforeAll(() => {
118 |   if (!Element.prototype.hasPointerCapture) {
119 |     Element.prototype.hasPointerCapture = () => false;
120 |   }
121 |   if (!Element.prototype.releasePointerCapture) {
122 |     Element.prototype.releasePointerCapture = () => {};
123 |   }
124 |   if (!Element.prototype.scrollToView) {
125 |     Element.prototype.scrollToView = () => {};
126 |   }
127 | });
128 |
129 | import FeeManager from "@/pages/admin/fees/FeeManager";
130 |
131 | // -----
132 | // Helpers
133 | // -----
134 |
135 | const createWrapper = () => {
136 |   const qc = new QueryClient({ defaultOptions: { queries: { retry: false } } });
137 |   return ({ children }: { children: React.ReactNode }) => (
138 |     <QueryClientProvider client={qc}>

```

```

139 |         <MemoryRouter>{children}</MemoryRouter>
140 |     </QueryClientProvider>
141 | );
142 | };
143 |
144 | const HINT_TEXT = "Select a program and a semester to create a fee structure.";
145 |
146 | const getTriggerByPlaceholder = (placeholder: string): HTMLElement => {
147 |     const node = screen.getByText(placeholder).closest("button");
148 |     expect(node).not.toBeNull();
149 |     return node as HTMLElement;
150 | };
151 |
152 | // -----
153 | // Tests
154 | // -----
155 |
156 | describe("FeeManager – fee dropdowns + disabled-until-selected (Req 12.1, 12.2, 12.4)", () => {
157 |     it("renders Program and Semester dropdowns instead of free-text UUID inputs", () => {
158 |         // Validates Requirements 12.1 & 12.2 – both selections are dropdowns.
159 |         render(<FeeManager />, { wrapper: createWrapper() });
160 |
161 |         // Two Shadcn/Radix Select triggers render as comboboxes (program + semester).
162 |         expect(screen.getAllByRole("combobox")).toHaveLength(2);
163 |
164 |         // Their placeholder labels are visible on the triggers.
165 |         expect(screen.getByText("Select a program")).toBeInTheDocument();
166 |         expect(screen.getByText("Select a semester")).toBeInTheDocument();
167 |
168 |         // The field labels are present.
169 |         expect(screen.getByText("Program")).toBeInTheDocument();
170 |         expect(screen.getByText("Semester")).toBeInTheDocument();
171 |
172 |         // No legacy free-text UUID inputs remain.
173 |         expect(screen.queryByPlaceholderText(/uuid/i)).not.toBeInTheDocument();
174 |     });
175 |
176 |     it("populates the Program dropdown options from usePrograms", async () => {
177 |         // Validates Requirement 12.1 – program options come from the hook.
178 |         const user = userEvent.setup();
179 |         render(<FeeManager />, { wrapper: createWrapper() });
180 |
181 |         await user.click(getTriggerByPlaceholder("Select a program"));
182 |
183 |         const listBox = await screen.findByRole("listbox");
184 |         expect(
185 |             within(listBox).getByRole("option", { name: "Computer Science" })
186 |         ).toBeInTheDocument();
187 |         expect(
188 |             within(listBox).getByRole("option", { name: "Business Administration" })
189 |         ).toBeInTheDocument();
190 |     });
191 |
192 |     it("populates the Semester dropdown options from useSemesters", async () => {
193 |         // Validates Requirement 12.2 – semester options come from the hook
194 |         // (rendered as "{name} ({code})").
195 |         const user = userEvent.setup();
196 |         render(<FeeManager />, { wrapper: createWrapper() });
197 |
198 |         await user.click(getTriggerByPlaceholder("Select a semester"));
199 |
200 |         const listBox = await screen.findByRole("listbox");
201 |         expect(
202 |             within(listBox).getByRole("option", { name: "Fall 2025 (FA25)" })
203 |         ).toBeInTheDocument();
204 |         expect(
205 |             within(listBox).getByRole("option", { name: "Spring 2026 (SP26)" })
206 |         ).toBeInTheDocument();
207 |     });
208 |
209 |     it("disables submit and shows the hint until both program and semester are selected", () => {
210 |         // Validates Requirement 12.4 – disabled-by-default with a required-selection hint.
211 |         render(<FeeManager />, { wrapper: createWrapper() });
212 |
213 |         const submit = screen.getByRole("button", { name: /create/i });
214 |         expect(submit).toBeDisabled();
215 |         expect(screen.getByText(HINT_TEXT)).toBeInTheDocument();
216 |     });
217 |
218 |     it("enables submit and hides the hint once both program and semester are selected", async () => {
219 |         // Validates Requirement 12.4 – selecting both values clears the gate.
220 |         const user = userEvent.setup();
221 |         render(<FeeManager />, { wrapper: createWrapper() });
222 |
223 |         // Select a program.
224 |         await user.click(getTriggerByPlaceholder("Select a program"));
225 |         await user.click(
226 |             within(await screen.findByRole("listbox")).getByRole("option", {
227 |                 name: "Computer Science",
228 |             })
229 |         );
230 |
231 |         // Still gated until a semester is also chosen.
232 |         expect(screen.getByRole("button", { name: /create/i })).toBeDisabled();
233 |
234 |         // Select a semester.
235 |         await user.click(getTriggerByPlaceholder("Select a semester"));
236 |         await user.click(
237 |             within(await screen.findByRole("listbox")).getByRole("option", {
238 |                 name: "Fall 2025 (FA25)",
239 |             })
240 |         );
241 |
242 |         // Both selected !' submit enabled and the hint is gone.

```


FILE: src/__tests__/unit/fileUpload.test.ts

```

1 import { describe, it, expect } from "vitest";
2 import { validateFile, FileValidationError } from "@lib/fileUpload";
3
4 // Helper to create a mock File with a given name and size
5 function createMockFile(name: string, sizeBytes: number): File {
6   const content = new Uint8Array(sizeBytes);
7   return new File([content], name, { type: "application/octet-stream" });
8 }
9
10 describe("fileUpload - validateFile", () => {
11   it("accepts a valid PDF under 10MB", () => {
12     const file = createMockFile("report.pdf", 1024 * 1024); // 1MB
13     expect(() => validateFile(file)).not.toThrow();
14   });
15
16   it("accepts all allowed file types", () => {
17     const types = ["pdf", "doc", "docx", "txt", "png", "jpg", "jpeg", "zip"];
18     for (const ext of types) {
19       const file = createMockFile(`file.${ext}`, 1024);
20       expect(() => validateFile(file)).not.toThrow();
21     }
22   });
23
24   it("rejects files exceeding 10MB", () => {
25     const file = createMockFile("big.pdf", 11 * 1024 * 1024); // 11MB
26     expect(() => validateFile(file)).toThrow(FileValidationError);
27     expect(() => validateFile(file)).toThrow(/10MB limit/);
28   });
29
30   it("rejects disallowed file extensions", () => {
31     const file = createMockFile("script.exe", 1024);
32     expect(() => validateFile(file)).toThrow(FileValidationError);
33     expect(() => validateFile(file)).toThrow(/not allowed/);
34   });
35
36   it("rejects files with no extension", () => {
37     const file = createMockFile("noext", 1024);
38     expect(() => validateFile(file)).toThrow(FileValidationError);
39   });
40
41   it("accepts a file exactly at 10MB", () => {
42     const file = createMockFile("exact.pdf", 10 * 1024 * 1024);
43     expect(() => validateFile(file)).not.toThrow();
44   });
45
46   it("rejects a file just over 10MB", () => {
47     const file = createMockFile("over.pdf", 10 * 1024 * 1024 + 1);
48     expect(() => validateFile(file)).toThrow(FileValidationError);
49   });
50
51   it("handles case-insensitive extensions", () => {
52     const file = createMockFile("REPORT.PDF", 1024);
53     // File.name preserves case, but our validator lowercases the extension
54     expect(() => validateFile(file)).not.toThrow();
55   });
56 });
57

```

[illegible]

[illegible]

```

139 |     expect(mockMutate).toHaveBeenCalledOnce();
140 |     const callArgs = mockMutate.mock.calls[0]?.[0];
141 |     expect(callArgs).toMatchObject({
142 |       sessionId: "session-abc",
143 |       intervalNumber: 2,
144 |       response: "stuck",
145 |     });
146 |   });
147 | });
148 |
149 | it("shows AI Tutor link after selecting stuck", async () => {
150 |   const user = userEvent.setup();
151 |   renderDialog();
152 |
153 |   await user.click(screen.getByRole("button", { name: /^stuck$/i }));
154 |
155 |   expect(screen.getByText(/stuck happens/i)).toBeTruthy();
156 |   expect(screen.getByRole("link", { name: /open ai tutor/i })).toBeTruthy();
157 | });
158 |
159 | it("renders AI Tutor link pointing to /student/tutor", async () => {
160 |   const user = userEvent.setup();
161 |   renderDialog();
162 |
163 |   await user.click(screen.getByRole("button", { name: /^stuck$/i }));
164 |
165 |   const link = screen.getByRole("link", { name: /open ai tutor/i });
166 |   expect(link).toBeTruthy();
167 |   expect(link.getAttribute("href")).toBe("/student/tutor");
168 | });
169 | });
170 |
171 | describe("Response: Too easy", () => {
172 |   it("calls mutate with too_easy response", async () => {
173 |     const user = userEvent.setup();
174 |     renderDialog();
175 |
176 |     await user.click(screen.getByRole("button", { name: /too easy/i }));
177 |
178 |     expect(mockMutate).toHaveBeenCalledOnce();
179 |     const callArgs = mockMutate.mock.calls[0]?.[0];
180 |     expect(callArgs).toMatchObject({
181 |       sessionId: "session-abc",
182 |       intervalNumber: 2,
183 |       response: "too_easy",
184 |     });
185 |   });
186 |
187 |   it("shows Bloom's level suggestion after selecting too easy", async () => {
188 |     const user = userEvent.setup();
189 |     renderDialog();
190 |
191 |     await user.click(screen.getByRole("button", { name: /too easy/i }));
192 |
193 |     expect(screen.getByText(/ready for more challenge/i)).toBeTruthy();
194 |     expect(screen.getByText(/bloom's level/i)).toBeTruthy();
195 |   });
196 | });
197 |
198 | describe("Dismiss Action", () => {
199 |   it("calls onOpenChange(false) when dismiss is clicked", async () => {
200 |     const user = userEvent.setup();
201 |     const { props } = renderDialog();
202 |
203 |     await user.click(
204 |       screen.getByRole("button", { name: /dismiss check-in/i })
205 |     );
206 |
207 |     expect(props.onOpenChange).toHaveBeenCalledOnceWith(false);
208 |   });
209 |
210 |   it("does not call mutate when dismiss is clicked", async () => {
211 |     const user = userEvent.setup();
212 |     renderDialog();
213 |
214 |     await user.click(
215 |       screen.getByRole("button", { name: /dismiss check-in/i })
216 |     );
217 |
218 |     expect(mockMutate).not.toHaveBeenCalled();
219 |   });
220 | });
221 |
222 | describe("Continue Action", () => {
223 |   it("calls onOpenChange(false) when continue is clicked after response", async () => {
224 |     const user = userEvent.setup();
225 |     const { props } = renderDialog();
226 |
227 |     await user.click(screen.getByRole("button", { name: /in the zone/i }));
228 |     await user.click(
229 |       screen.getByRole("button", { name: /continue to break/i })
230 |     );
231 |
232 |     expect(props.onOpenChange).toHaveBeenCalledOnceWith(false);
233 |   });
234 | });
235 |
236 | describe("Mutation callback", () => {
237 |   it("calls onComplete with the response on successful save", async () => {
238 |     // Make mutate call onSuccess immediately
239 |     mockMutate.mockImplementation(
240 |       (_input: unknown, options: { onSuccess?: () => void }) => {
241 |         options?.onSuccess?.();
242 |       }

```

```

243 |     });
244 |
245 |     const user = userEvent.setup();
246 |     const { props } = renderDialog();
247 |
248 |     await user.click(screen.getByRole("button", { name: /in the zone/i }));
249 |
250 |     await waitFor(() => {
251 |       expect(props.onComplete).toHaveBeenCalledWith("in_the_zone");
252 |     });
253 |   });
254 | });
255 |
256 | describe("Accessibility", () => {
257 |   it("has an accessible dialog label", () => {
258 |     renderDialog();
259 |     // aria-labelledby (DialogTitle) takes precedence over aria-label
260 |     expect(
261 |       screen.getByRole("dialog", { name: /quick check-in/i })
262 |     ).toBeTruthy();
263 |   });
264 |
265 |   it("has an accessible group for response buttons", () => {
266 |     renderDialog();
267 |     expect(
268 |       screen.getByRole("group", { name: /flow state options/i })
269 |     ).toBeTruthy();
270 |   });
271 |
272 |   it("each response button has an accessible label", () => {
273 |     renderDialog();
274 |     expect(screen.getByRole("button", { name: /in the zone/i })).toBeTruthy();
275 |     expect(screen.getByRole("button", { name: /^stuck$/i })).toBeTruthy();
276 |     expect(screen.getByRole("button", { name: /too easy/i })).toBeTruthy();
277 |   });
278 |
279 |   it("dismiss button has an accessible label", () => {
280 |     renderDialog();
281 |     expect(
282 |       screen.getByRole("button", { name: /dismiss check-in/i })
283 |     ).toBeTruthy();
284 |   });
285 | });
286 | });
287 |

```

FILE: [src/__tests__/unit/focusMode.test.tsx](#)

TYPE: .tsx | LINES: 1-80 | SHA256: e471789777af99ab05a92b2dd56a672f5a1f5be5003c5128fb003c43b5d5f1b3

```

1 | // Feature: i18n-rtl-support, Property 16: UI Focus Mode Hides Only Non-Essential Elements
2 | // **Validates: Requirements 22.2, 22.3**
3 |
4 | import { describe, it, expect, beforeEach } from "vitest";
5 |
6 | describe("Focus Mode – simplified-view CSS behavior", () => {
7 |   beforeEach(() => {
8 |     document.documentElement.classList.remove("simplified-view");
9 |     document.body.innerHTML = "";
10 |   });
11 |
12 |   it('elements with data-focus-hide="true" are hidden when simplified_view is active', () => {
13 |     document.body.innerHTML = `
14 |       <div data-focus-hide="true" id="gamification">XP Widget</div>
15 |       <div id="essential">Main Content</div>
16 |     `;
17 |
18 |     // Add the CSS rule inline for testing
19 |     const style = document.createElement("style");
20 |     style.textContent =
21 |       ".simplified-view [data-focus-hide='true'] { display: none !important; }";
22 |     document.head.appendChild(style);
23 |
24 |     document.documentElement.classList.add("simplified-view");
25 |
26 |     const gamification = document.getElementById("gamification");
27 |     const essential = document.getElementById("essential");
28 |
29 |     // Check computed style
30 |     const gamificationStyle = window.getComputedStyle(gamification!);
31 |     const essentialStyle = window.getComputedStyle(essential!);
32 |
33 |     expect(gamificationStyle.display).toBe("none");
34 |     expect(essentialStyle.display).not.toBe("none");
35 |
36 |     document.head.removeChild(style);
37 |   });
38 |
39 |   it('elements with data-focus-hide="true" are visible when simplified_view is inactive', () => {
40 |     document.body.innerHTML = `
41 |       <div data-focus-hide="true" id="gamification">XP Widget</div>
42 |       <div id="essential">Main Content</div>
43 |     `;
44 |
45 |     const style = document.createElement("style");
46 |     style.textContent =
47 |       ".simplified-view [data-focus-hide='true'] { display: none !important; }";
48 |     document.head.appendChild(style);
49 |
50 |     // simplified-view NOT active
51 |     const gamification = document.getElementById("gamification");
52 |     const gamificationStyle = window.getComputedStyle(gamification!);
53 |     expect(gamificationStyle.display).not.toBe("none");
54 |
55 |     document.head.removeChild(style);

```

```

56 });
57
58 it("elements without data-focus-hide remain visible regardless of simplified_view", () => {
59     document.body.innerHTML = `
60         <nav id="primary-nav">Navigation</nav>
61         <form id="main-form"><input type="text" /><button type="submit">Submit</button></form>
62     `;
63
64     const style = document.createElement("style");
65     style.textContent =
66         '.simplified-view [data-focus-hide="true"] { display: none !important; }';
67     document.head.appendChild(style);
68
69     document.documentElement.classList.add("simplified-view");
70
71     const nav = document.getElementById("primary-nav");
72     const form = document.getElementById("main-form");
73
74     expect(window.getComputedStyle(nav!).display).not.toBe("none");
75     expect(window.getComputedStyle(form!).display).not.toBe("none");
76
77     document.head.removeChild(style);
78 });
79 });
80

```

FILE: src/__tests__/unit/focusTimer.test.tsx

TYPE: .tsx | LINES: 1-567 | SHA256: 1f86ed736a307d9e6045af4a261c82538e03025e32faa9a89466c2eca08bb3a7

[illegible]

[illegible]

```

180         timer={makeTimer()}
181         session={makeSession()}
182         intentConcept={null}
183     />,
184     { wrapper: createWrapper() }
185 );
186 expect(screen.queryByText("Integration by parts")).toBeNull();
187 });
188
189 it("does not render session context when session is null", () => {
190     render(<FocusTimer timer={makeTimer()} session={null} />, {
191         wrapper: createWrapper(),
192     });
193     expect(screen.queryByText("Review Chapter 5")).toBeNull();
194 });
195 });
196
197 describe("Controls – Idle State", () => {
198     it("shows Start button in idle state", () => {
199         render(<FocusTimer timer={makeTimer()} session={makeSession()} />, {
200             wrapper: createWrapper(),
201         });
202         expect(screen.getByRole("button", { name: /start/i })).toBeTruthy();
203     });
204
205     it("calls start when Start button is clicked", async () => {
206         const user = userEvent.setup();
207         const startFn = vi.fn();
208         render(
209             <FocusTimer
210                 timer={makeTimer({ start: startFn })}
211                 session={makeSession()}
212             />,
213             { wrapper: createWrapper() }
214         );
215         await user.click(screen.getByRole("button", { name: /start/i }));
216         expect(startFn).toHaveBeenCalledOnce();
217     });
218 });
219
220 describe("Controls – Running State", () => {
221     it("shows Pause and End buttons when running", () => {
222         render(
223             <FocusTimer
224                 timer={makeTimer({ timerState: "running" })}
225                 session={makeSession()}
226             />,
227             { wrapper: createWrapper() }
228         );
229         expect(screen.getByRole("button", { name: /pause/i })).toBeTruthy();
230         expect(screen.getByRole("button", { name: /end/i })).toBeTruthy();
231     });
232
233     it("calls pause when Pause button is clicked", async () => {
234         const user = userEvent.setup();
235         const pauseFn = vi.fn();
236         render(
237             <FocusTimer
238                 timer={makeTimer({ timerState: "running", pause: pauseFn })}
239                 session={makeSession()}
240             />,
241             { wrapper: createWrapper() }
242         );
243         await user.click(screen.getByRole("button", { name: /pause/i }));
244         expect(pauseFn).toHaveBeenCalledOnce();
245     });
246
247     it("calls end and onEnd when End button is clicked", async () => {
248         const user = userEvent.setup();
249         const endFn = vi.fn();
250         const onEnd = vi.fn();
251         render(
252             <FocusTimer
253                 timer={makeTimer({ timerState: "running", end: endFn })}
254                 session={makeSession()}
255                 onEnd={onEnd}
256             />,
257             { wrapper: createWrapper() }
258         );
259         await user.click(screen.getByRole("button", { name: /end/i }));
260         expect(endFn).toHaveBeenCalledOnce();
261         expect(onEnd).toHaveBeenCalledOnce();
262     });
263 });
264
265 describe("Controls – Paused State", () => {
266     it("shows Resume and End buttons when paused", () => {
267         render(
268             <FocusTimer
269                 timer={makeTimer({ timerState: "paused" })}
270                 session={makeSession()}
271             />,
272             { wrapper: createWrapper() }
273         );
274         expect(screen.getByRole("button", { name: /resume/i })).toBeTruthy();
275         expect(screen.getByRole("button", { name: /end/i })).toBeTruthy();
276     });
277
278     it("calls resume when Resume button is clicked", async () => {
279         const user = userEvent.setup();
280         const resumeFn = vi.fn();
281         render(
282             <FocusTimer
283                 timer={makeTimer({ timerState: "paused", resume: resumeFn })}

```

```

284         session={makeSession()}
285     />,
286     { wrapper: createWrapper() }
287 );
288 await user.click(screen.getByRole("button", { name: /resume/i }));
289 expect(resumeFn).toHaveBeenCalledOnce();
290 });
291 });
292
293 describe("Controls – Break State", () => {
294     it("shows Skip Break and End buttons during break", () => {
295         render(
296             <FocusTimer
297                 timer={makeTimer({ timerState: "break" })}
298                 session={makeSession()}
299             />,
300             { wrapper: createWrapper() }
301         );
302         expect(screen.getByRole("button", { name: /skip break/i })).toBeTruthy();
303         expect(screen.getByRole("button", { name: /end/i })).toBeTruthy();
304     });
305
306     it("calls skipBreak when Skip Break is clicked", async () => {
307         const user = userEvent.setup();
308         const skipBreakFn = vi.fn();
309         render(
310             <FocusTimer
311                 timer={makeTimer({ timerState: "break", skipBreak: skipBreakFn })}
312                 session={makeSession()}
313             />,
314             { wrapper: createWrapper() }
315         );
316         await user.click(screen.getByRole("button", { name: /skip break/i }));
317         expect(skipBreakFn).toHaveBeenCalledOnce();
318     });
319
320     it("shows Skip Break during long break too", () => {
321         render(
322             <FocusTimer
323                 timer={makeTimer({ timerState: "long_break" })}
324                 session={makeSession()}
325             />,
326             { wrapper: createWrapper() }
327         );
328         expect(screen.getByRole("button", { name: /skip break/i })).toBeTruthy();
329     });
330 });
331
332 describe("Controls – Waiting for Next Work (Pomodoro)", () => {
333     it("shows Start Next and End Session when waiting for next work interval", () => {
334         render(
335             <FocusTimer
336                 timer={makeTimer({
337                     timerState: "idle",
338                     mode: "pomodoro",
339                     pomodoroInterval: 2,
340                 })}
341                 session={makeSession()}
342             />,
343             { wrapper: createWrapper() }
344         );
345         expect(screen.getByRole("button", { name: /start next/i })).toBeTruthy();
346         expect(screen.getByRole("button", { name: /end session/i })).toBeTruthy();
347     });
348 });
349
350 describe("Completed State", () => {
351     it("shows 'Session Complete' text when completed", () => {
352         render(
353             <FocusTimer
354                 timer={makeTimer({ timerState: "completed" })}
355                 session={makeSession()}
356             />,
357             { wrapper: createWrapper() }
358         );
359         expect(screen.getByText("Session Complete")).toBeTruthy();
360     });
361 });
362
363 describe("Pomodoro Indicator", () => {
364     it("renders PomodoroIndicator in pomodoro mode", () => {
365         render(
366             <FocusTimer
367                 timer={makeTimer({ mode: "pomodoro" })}
368                 session={makeSession()}
369             />,
370             { wrapper: createWrapper() }
371         );
372         expect(screen.getByText("Pomodoro 1 of 4")).toBeTruthy();
373     });
374
375     it("does not render PomodoroIndicator in custom mode", () => {
376         render(
377             <FocusTimer
378                 timer={makeTimer({ mode: "custom" })}
379                 session={makeSession()}
380             />,
381             { wrapper: createWrapper() }
382         );
383         expect(screen.queryByText(/Pomodoro \d+ of 4/)).toBeNull();
384     });
385 });
386
387 describe("Timer Mode Badge", () => {

```



```

388 | it('shows "Pomodoro Mode" badge in pomodoro mode', () => {
389 |     render(
390 |         <FocusTimer
391 |             timer={makeTimer({ mode: "pomodoro" })}
392 |             session={makeSession()}
393 |         />,
394 |         { wrapper: createWrapper() }
395 |     );
396 |     expect(screen.getByText("Pomodoro Mode")).toBeTruthy();
397 | });
398 |
399 | it('shows "Custom Timer" badge in custom mode', () => {
400 |     render(
401 |         <FocusTimer
402 |             timer={makeTimer({ mode: "custom" })}
403 |             session={makeSession()}
404 |         />,
405 |         { wrapper: createWrapper() }
406 |     );
407 |     expect(screen.getByText("Custom Timer")).toBeTruthy();
408 | });
409 | });
410 |
411 | describe("ARIA Live Region", () => {
412 |     it("renders an ARIA live region for timer announcements", () => {
413 |         render(<FocusTimer timer={makeTimer()} session={makeSession()} />, {
414 |             wrapper: createWrapper(),
415 |         });
416 |         const liveRegion = screen.getByRole("timer");
417 |         expect(liveRegion).toBeTruthy();
418 |         expect(liveRegion.getAttribute("aria-live")).toBe("polite");
419 |     });
420 | });
421 |
422 | describe("Offline Indicator", () => {
423 |     it("does not show offline indicator when online", () => {
424 |         render(<FocusTimer timer={makeTimer()} session={makeSession()} />, {
425 |             wrapper: createWrapper(),
426 |         });
427 |         expect(screen.queryByText(/offline/i)).toBeNull();
428 |     });
429 | });
430 |
431 | describe("Custom Midpoint Flow Check-In", () => {
432 |     it("opens flow dialog at midpoint for custom sessions \"e50 min\", () => {
433 |         const midpointMs = (60 * 60 * 1000) / 2; // 30 min for a 60-min session
434 |         render(
435 |             <FocusTimer
436 |                 timer={makeTimer({
437 |                     mode: "custom",
438 |                     timerState: "running",
439 |                     totalElapsedMs: midpointMs,
440 |                     remainingMs: midpointMs,
441 |                 })}
442 |                 session={makeSession({
443 |                     plannedDurationMinutes: 60,
444 |                     timerMode: "custom",
445 |                 })}
446 |             />,
447 |             { wrapper: createWrapper() }
448 |         );
449 |         // FlowCheckInDialog should be called with open=true
450 |         const lastCall =
451 |             flowCheckInProps.mock.calls[flowCheckInProps.mock.calls.length - 1];
452 |         expect(lastCall[0].open).toBe(true);
453 |         expect(lastCall[0].intervalNumber).toBe(1);
454 |     });
455 |
456 |     it("does not open flow dialog for custom sessions <50 min", () => {
457 |         render(
458 |             <FocusTimer
459 |                 timer={makeTimer({
460 |                     mode: "custom",
461 |                     timerState: "running",
462 |                     totalElapsedMs: 15 * 60 * 1000, // 15 min elapsed
463 |                     remainingMs: 15 * 60 * 1000,
464 |                 })}
465 |                 session={makeSession({
466 |                     plannedDurationMinutes: 30,
467 |                     timerMode: "custom",
468 |                 })}
469 |             />,
470 |             { wrapper: createWrapper() }
471 |         );
472 |         // FlowCheckInDialog should be called with open=false
473 |         const lastCall =
474 |             flowCheckInProps.mock.calls[flowCheckInProps.mock.calls.length - 1];
475 |         expect(lastCall[0].open).toBe(false);
476 |     });
477 |
478 |     it("does not open flow dialog before midpoint for custom sessions \"e50 min\", () => {
479 |         const beforeMidpointMs = (50 * 60 * 1000) / 2 - 1000; // just before midpoint
480 |         render(
481 |             <FocusTimer
482 |                 timer={makeTimer({
483 |                     mode: "custom",
484 |                     timerState: "running",
485 |                     totalElapsedMs: beforeMidpointMs,
486 |                     remainingMs: 50 * 60 * 1000 - beforeMidpointMs,
487 |                 })}
488 |                 session={makeSession({
489 |                     plannedDurationMinutes: 50,
490 |                     timerMode: "custom",
491 |                 })}

```

[illegible]

FILE: src/___tests___/unit/formatDate.test.ts

TYPE: .ts | LINES: 1-55 | SHA256: 8d835a46a6f0bfff7fdc78493543af2ff49ca09d5a91f9d56e3c25d2d854d5f4f

```

1 | import { describe, it, expect, beforeEach } from "vitest";
2 | import i18n from "@lib/i18n";
3 | import { formatLocalDate, formatRelativeTime } from "@lib/formatDate";
4 |
5 | describe("formatDate", () => {
6 |   beforeEach(() => {
7 |     i18n.changeLanguage("en");
8 |   });
9 |
10 |   describe("formatLocalDate", () => {
11 |     it("formats a Date object with default pattern in English", () => {
12 |       const date = new Date(2025, 0, 15); // Jan 15, 2025
13 |       const result = formatLocalDate(date);
14 |       expect(result).toContain("2025");
15 |       expect(result).toContain("January");
16 |     });
17 |
18 |     it("formats an ISO string", () => {
19 |       const result = formatLocalDate("2025-06-20T12:00:00Z");
20 |       expect(result).toContain("2025");
21 |     });
22 |
23 |     it("formats with custom pattern", () => {
24 |       const date = new Date(2025, 5, 20); // Jun 20, 2025

```

```

25 |     const result = formatLocalDate(date, "yyyy-MM-dd");
26 |     expect(result).toBe("2025-06-20");
27 |   });
28 |
29 |   it("returns Arabic-formatted date when language is Arabic", () => {
30 |     i18n.changeLanguage("ar");
31 |     const date = new Date(2025, 0, 15);
32 |     const result = formatLocalDate(date);
33 |     // Arabic locale should produce Arabic month names
34 |     expect(result).toBeTruthy();
35 |     expect(result.length).toBeGreaterThan(0);
36 |   });
37 | });
38 |
39 | describe("formatRelativeTime", () => {
40 |   it("returns a non-empty relative time string", () => {
41 |     const recentDate = new Date(Date.now() - 3600 * 1000); // 1 hour ago
42 |     const result = formatRelativeTime(recentDate);
43 |     expect(result.length).toBeGreaterThan(0);
44 |     expect(result).toContain("ago");
45 |   });
46 |
47 |   it("works with ISO string input", () => {
48 |     const result = formatRelativeTime(
49 |       new Date(Date.now() - 60000).toISOString()
50 |     );
51 |     expect(result.length).toBeGreaterThan(0);
52 |   });
53 | });
54 | });
55 |

```

FILE: [src/__tests__/unit/gamificationFeedbackHost.test.tsx](#)

TYPE: .tsx | LINES: 1-230 | SHA256: af74524bd518e7a02fe4647281fb4d5e11ae2f447bfc962314a85fb5c727a274

```

1 | // @vitest-environment happy-dom
2 |
3 | import { act, render, screen } from "@testing-library/react";
4 | import userEvent from "@testing-library/user-event";
5 | import { beforeEach, describe, expect, it, vi } from "vitest";
6 | import "@lib/i18n";
7 | import {
8 |   emitXPFeedback,
9 |   XP_FEEDBACK_EVENT,
10 |   type XPFeedbackDetail,
11 | } from "@lib/xpFeedback";
12 | import {
13 |   BADGE_FEEDBACK_EVENT,
14 |   type BadgeFeedbackDetail,
15 | } from "@lib/badgeFeedback";
16 |
17 | vi.mock("@/components/shared/XPAwardToast", () => ({
18 |   default: ({
19 |     xpAmount,
20 |     source,
21 |     onComplete,
22 |   }: {
23 |     xpAmount: number;
24 |     source: string;
25 |     onComplete?: () => void;
26 |   }) => (
27 |     <button type="button" onClick={onComplete}>
28 |       +{xpAmount} XP · {source}
29 |     </button>
30 |   ),
31 | }));
32 |
33 | vi.mock("@/components/shared/LevelUpOverlay", () => ({
34 |   default: ({
35 |     newLevel,
36 |     onComplete,
37 |   }: {
38 |     newLevel: number;
39 |     onComplete?: () => void;
40 |   }) => (
41 |     <button type="button" onClick={onComplete}>
42 |       Level up to {newLevel}
43 |     </button>
44 |   ),
45 | }));
46 |
47 | vi.mock("@/components/shared/MysteryRewardBox", () => ({
48 |   default: ({
49 |     studentId,
50 |     institutionId,
51 |     onComplete,
52 |   }: {
53 |     studentId: string;
54 |     institutionId: string;
55 |     onComplete?: () => void;
56 |   }) => (
57 |     <button type="button" onClick={onComplete}>
58 |       Mystery reward for {studentId} at {institutionId}
59 |     </button>
60 |   ),
61 | }));
62 |
63 | vi.mock("@/components/shared/BadgeAwardModal", () => ({
64 |   default: ({
65 |     badge,
66 |     onClose,
67 |   }: {
68 |     badge: BadgeFeedbackDetail;
69 |     onClose: () => void;

```

```

70 |   }) => (
71 |     <button type="button" onClick={onClose}>
72 |       Badge earned: {badge.name}
73 |     </button>
74 |   ),
75 |   }));
76 |
77 | vi.mock("@/hooks/useAuth", () => ({
78 |   useAuth: () => ({
79 |     profile: { institution_id: "institution-1" },
80 |   }),
81 | }));
82 |
83 | import GamificationFeedbackHost from "@/components/shared/GamificationFeedbackHost";
84 |
85 | const feedback = (id: string, amount: number): XPFeedbackDetail => ({
86 |   id,
87 |   studentId: "student-1",
88 |   amount,
89 |   source: "study_session",
90 |   levelUp: false,
91 |   newLevel: 2,
92 |   mysteryRewardTriggered: false,
93 | });
94 |
95 | describe("GamificationFeedbackHost", () => {
96 |   beforeEach(() => {
97 |     vi.restoreAllMocks();
98 |   });
99 |
100 |   it("shows only backend-confirmed positive XP feedback", () => {
101 |     render(<GamificationFeedbackHost />);
102 |
103 |     act(() => {
104 |       window.dispatchEvent(
105 |         new CustomEvent(XP_FEEDBACK_EVENT, {
106 |           detail: feedback("reward-1", 25),
107 |         })
108 |       );
109 |     });
110 |
111 |     expect(
112 |       screen.getByRole("button", { name: "+25 XP · Reward earned" })
113 |     ).toBeInTheDocument();
114 |   });
115 |
116 |   it("queues rapid rewards instead of replacing the current overlay", async () => {
117 |     const user = userEvent.setup();
118 |     render(<GamificationFeedbackHost />);
119 |
120 |     act(() => {
121 |       window.dispatchEvent(
122 |         new CustomEvent(XP_FEEDBACK_EVENT, {
123 |           detail: feedback("reward-1", 10),
124 |         })
125 |       );
126 |       window.dispatchEvent(
127 |         new CustomEvent(XP_FEEDBACK_EVENT, {
128 |           detail: feedback("reward-2", 30),
129 |         })
130 |       );
131 |     });
132 |
133 |     await user.click(
134 |       screen.getByRole("button", { name: "+10 XP · Reward earned" })
135 |     );
136 |     expect(
137 |       screen.getByRole("button", { name: "+30 XP · Reward earned" })
138 |     ).toBeInTheDocument();
139 |   });
140 |
141 |   it("shows the level-up overlay after the confirmed XP reward", async () => {
142 |     const user = userEvent.setup();
143 |     render(<GamificationFeedbackHost />);
144 |
145 |     act(() => {
146 |       window.dispatchEvent(
147 |         new CustomEvent(XP_FEEDBACK_EVENT, {
148 |           detail: {
149 |             ...feedback("reward-level", 50),
150 |             levelUp: true,
151 |             newLevel: 4,
152 |           },
153 |         })
154 |       );
155 |     });
156 |
157 |     expect(screen.queryByText("Level up to 4")).not.toBeInTheDocument();
158 |     await user.click(
159 |       screen.getByRole("button", { name: "+50 XP · Reward earned" })
160 |     );
161 |     expect(
162 |       screen.getByRole("button", { name: "Level up to 4" })
163 |     ).toBeInTheDocument();
164 |   });
165 |
166 |   it("does not publish feedback for idempotent zero-XP responses", () => {
167 |     const listener = vi.fn();
168 |     window.addEventListener(XP_FEEDBACK_EVENT, listener);
169 |
170 |     emitXPFeedback({
171 |       studentId: "student-1",
172 |       amount: 0,
173 |       source: "login",

```

```

174     levelUp: false,
175     newLevel: 1,
176   });
177
178   expect(listener).not.toHaveBeenCalled();
179   window.removeEventListener(XP_FEEDBACK_EVENT, listener);
180 });
181
182 it("shows a server-triggered mystery reward after XP feedback", async () => {
183   const user = userEvent.setup();
184   render(<GamificationFeedbackHost />);
185
186   act(() => {
187     window.dispatchEvent(
188       new CustomEvent(XP_FEEDBACK_EVENT, {
189         detail: {
190           ...feedback("reward-mystery", 20),
191           mysteryRewardTriggered: true,
192         },
193       }),
194     );
195   });
196
197   await user.click(
198     screen.getByRole("button", { name: "+20 XP · Reward earned" })
199   );
200   expect(
201     screen.getByRole("button", {
202       name: "Mystery reward for student-1 at institution-1",
203     })
204   ).toBeInTheDocument();
205 });
206
207 it("queues badge celebrations from confirmed badge events", () => {
208   render(<GamificationFeedbackHost />);
209
210   act(() => {
211     window.dispatchEvent(
212       new CustomEvent<BadgeFeedbackDetail>(BADGE_FEEDBACK_EVENT, {
213         detail: {
214           id: "bookworm",
215           name: "Bookworm",
216           description: "Read consistently",
217           icon: "🐛",
218           isMystery: false,
219           xpReward: 25,
220         },
221       }),
222     );
223   });
224
225   expect(
226     screen.getByRole("button", { name: "Badge earned: Bookworm" })
227   ).toBeInTheDocument();
228 });
229 });
230

```

FILE: src/__tests__/unit/goalDifficultyBadge.test.tsx

TYPE: .tsx | LINES: 1-108 | SHA256: b9a550a90a61d055c1c9d270e77fdd8df52f4a2e607e4d3649bb958a3dc87435

[illegible]

[illegible]

FILE: src/__tests__/unit/goalSuggestionPanel.test.tsx

TYPE: .tsx | LINES: 1-285 | SHA256: ec87bce2062b6019cfbc895084f87373f0b1589f950f91be5d25c883493815c4

[illegible]

[illegible]

```

140     it("calls onAccept with second goal id", async () => {
141       const onAccept = vi.fn();
142       renderPanel({ onAccept });
143       const acceptButtons = screen.getAllByText("Accept");
144       await userEvent.click(acceptButtons[1]);
145       expect(onAccept).toHaveBeenCalledOnceWith("goal-2");
146     });
147   });
148
149   // %% Edit action %%%%%%%%%%%%%%%
150
151   describe("edit action", () => {
152     it("renders Edit buttons for each suggestion", () => {
153       renderPanel();
154       const editButtons = screen.getAllByText("Edit");
155       expect(editButtons).toHaveLength(3);
156     });
157
158     it("calls onEdit with the correct goal id", async () => {
159       const onEdit = vi.fn();
160       renderPanel({ onEdit });
161       const editButtons = screen.getAllByText("Edit");
162       await userEvent.click(editButtons[2]);
163       expect(onEdit).toHaveBeenCalledOnceWith("goal-3");
164     });
165   });
166
167   // %% Dismiss action %%%%%%%%%%%%%%%
168
169   describe("dismiss action", () => {
170     it("renders Dismiss buttons for each suggestion", () => {
171       renderPanel();
172       const dismissButtons = screen.getAllByText("Dismiss");
173       expect(dismissButtons).toHaveLength(3);
174     });
175
176     it("calls onDismiss with the correct goal id", async () => {
177       const onDismiss = vi.fn();
178       renderPanel({ onDismiss });
179       const dismissButtons = screen.getAllByText("Dismiss");
180       await userEvent.click(dismissButtons[1]);
181       expect(onDismiss).toHaveBeenCalledOnceWith("goal-2");
182     });
183   });
184
185   // %% Empty state %%%%%%%%%%%%%%%
186
187   describe("empty state", () => {
188     it("renders nothing when suggestions array is empty", () => {
189       const { container } = renderPanel({ suggestions: [] });
190       expect(container.innerHTML).toBe("");
191     });
192
193     it("renders nothing when all suggestions are dismissed", () => {
194       const dismissed = threeSuggestions.map((s) => ({
195         ...s,
196         status: "dismissed" as const,
197       }));
198       const { container } = renderPanel({ suggestions: dismissed });
199       expect(container.innerHTML).toBe("");
200     });
201
202     it("renders nothing when all suggestions are accepted", () => {
203       const accepted = threeSuggestions.map((s) => ({
204         ...s,
205         status: "accepted" as const,
206       }));
207       const { container } = renderPanel({ suggestions: accepted });
208       expect(container.innerHTML).toBe("");
209     });
210   });
211
212   // %% Loading state %%%%%%%%%%%%%%%
213
214   describe("loading state", () => {
215     it("renders loading text when isLoading is true", () => {
216       renderPanel({ isLoading: true });
217       expect(screen.getByText("Generating goal suggestions...")).toBeTruthy();
218     });
219
220     it("renders shimmer placeholders when loading", () => {
221       const { container } = renderPanel({ isLoading: true });
222       const shimmers = container.querySelectorAll(".animate-shimmer");
223       expect(shimmers.length).toBe(3);
224     });
225
226     it("does not render goal texts when loading", () => {
227       renderPanel({ isLoading: true });
228       expect(
229         screen.queryByText("Review lecture notes for Chapter 5")
230       ).toBeNull();
231     });
232   });
233
234   // %% Filters only active suggestions %%%%%%%%%%%%%%%
235
236   describe("filters non-suggested goals", () => {
237     it("only renders goals with status \"suggested\"", () => {
238       const mixed: GoalSuggestion[] = [
239         makeSuggestion({
240           id: "g1",
241           goal_text: "Active goal",
242           status: "suggested",
243         }),

```


[illegible]

FILE: src/__tests__/unit/goalTemplates.test.ts

```
TYPE: .ts | LINES: 1-156 | SHA256: 56cfd5eaf5dbacc624cd9cbc8f34bcdcd43efc7d7a2f69ac11de49b455f7913ba
```

```

1 // @vitest-environment happy-dom
2 import { describe, it, expect } from "vitest";
3 import {
4   classifyDifficulty,
5   composeGoalText,
6   createGoalTemplate,
7   type SmartGoalFields,
8 } from "@lib/goalTemplates";
9
10 describe("classifyDifficulty", () => {
11   it('returns "easy" for cohort rate >= 80', () => {
12     expect(classifyDifficulty(90)).toBe("easy");
13   });
14
15   it('returns "easy" for cohort rate exactly 80 (boundary)', () => {
16     expect(classifyDifficulty(80)).toBe("easy");
17   });
18
19   it('returns "moderate" for cohort rate >= 50 and < 80', () => {
20     expect(classifyDifficulty(65)).toBe("moderate");
21   });
22
23   it('returns "moderate" for cohort rate exactly 50 (boundary)', () => {
24     expect(classifyDifficulty(50)).toBe("moderate");
25   });
26
27   it('returns "ambitious" for cohort rate < 50', () => {
28     expect(classifyDifficulty(30)).toBe("ambitious");
29   });
30
31   it('returns "ambitious" for cohort rate 0', () => {
32     expect(classifyDifficulty(0)).toBe("ambitious");
33   });
34
35   it('returns "easy" for cohort rate 100', () => {
36     expect(classifyDifficulty(100)).toBe("easy");
37   });
38 });
39
40 describe("composeGoalText", () => {
41   const fields: SmartGoalFields = {
42     specific: "Complete 3 practice problems",
43     measurable: "quiz scores above 80%",
44     achievable: "I have 2 hours of study time daily",
45     relevant: "Calculus I – Derivatives",
46     timebound: "2025-02-15",
47   };
48
49   it("composes correct text from all fields", () => {
50     const result = composeGoalText(fields);
51     expect(result).toBe(
52       "Complete 3 practice problems by 2025-02-15, measuring progress through quiz scores above 80%. This is a
53     );
54   });
55
56   it("includes all 5 SMART fields in the output", () => {
57     const result = composeGoalText(fields);
58     expect(result).toContain(fields.specific);
59   });
60 });

```

```

59 |     expect(result).toContain(fields.measurable);
60 |     expect(result).toContain(fields.achievable);
61 |     expect(result).toContain(fields.relevant);
62 |     expect(result).toContain(fields.timebound);
63 | });
64 |
65 | it("returns non-empty string for valid input", () => {
66 |     const result = composeGoalText(fields);
67 |     expect(result.length).toBeGreaterThan(0);
68 | });
69 | });
70 |
71 | describe("createGoalTemplate", () => {
72 |     it("creates template with courseName only (no cloTitle)", () => {
73 |         const result = createGoalTemplate({
74 |             courseName: "Physics 101",
75 |             deadline: "2025-03-01",
76 |             suggestedAction: "Review chapter 5",
77 |             measurableTarget: "Score 75% on quiz",
78 |         });
79 |         expect(result.relevant).toBe("Physics 101");
80 |     });
81 |
82 |     it("creates template with courseName and cloTitle", () => {
83 |         const result = createGoalTemplate({
84 |             courseName: "Physics 101",
85 |             cloTitle: "Newton Laws",
86 |             deadline: "2025-03-01",
87 |             suggestedAction: "Review chapter 5",
88 |             measurableTarget: "Score 75% on quiz",
89 |         });
90 |         expect(result.relevant).toBe("Physics 101 – Newton Laws");
91 |     });
92 |
93 |     it("sets achievable to default text", () => {
94 |         const result = createGoalTemplate({
95 |             courseName: "Math",
96 |             deadline: "2025-04-01",
97 |             suggestedAction: "Study",
98 |             measurableTarget: "Pass exam",
99 |         });
100 |         expect(result.achievable).toBe(
101 |             "Based on your current progress and available study time"
102 |         );
103 |     });
104 |
105 |     it("sets specific to suggestedAction", () => {
106 |         const result = createGoalTemplate({
107 |             courseName: "Math",
108 |             deadline: "2025-04-01",
109 |             suggestedAction: "Complete 10 exercises",
110 |             measurableTarget: "Pass exam",
111 |         });
112 |         expect(result.specific).toBe("Complete 10 exercises");
113 |     });
114 |
115 |     it("sets measurable to measurableTarget", () => {
116 |         const result = createGoalTemplate({
117 |             courseName: "Math",
118 |             deadline: "2025-04-01",
119 |             suggestedAction: "Study",
120 |             measurableTarget: "Score above 90%",
121 |         });
122 |         expect(result.measurable).toBe("Score above 90%");
123 |     });
124 |
125 |     it("sets timebound to deadline", () => {
126 |         const result = createGoalTemplate({
127 |             courseName: "Math",
128 |             deadline: "2025-06-15",
129 |             suggestedAction: "Study",
130 |             measurableTarget: "Pass exam",
131 |         });
132 |         expect(result.timebound).toBe("2025-06-15");
133 |     });
134 |
135 |     it('formats relevant as "courseName – cloTitle" when cloTitle provided', () => {
136 |         const result = createGoalTemplate({
137 |             courseName: "Biology",
138 |             cloTitle: "Cell Division",
139 |             deadline: "2025-05-01",
140 |             suggestedAction: "Read textbook",
141 |             measurableTarget: "Identify all phases",
142 |         });
143 |         expect(result.relevant).toBe("Biology – Cell Division");
144 |     });
145 |
146 |     it("formats relevant as just courseName when no cloTitle", () => {
147 |         const result = createGoalTemplate({
148 |             courseName: "Biology",
149 |             deadline: "2025-05-01",
150 |             suggestedAction: "Read textbook",
151 |             measurableTarget: "Identify all phases",
152 |         });
153 |         expect(result.relevant).toBe("Biology");
154 |     });
155 | });
156 |

```

FILE: src/___tests___/unit/gradebookView.test.tsx

TYPE: .tsx | LINES: 1-384 | SHA256: 0f7ff48391a1f88d300eebfecc1541eeb61aad5cfec2aac20fb02478d9e44dc0

```
1 | // @vitest-environment happy-dom
2 | // =====
3 | // GradebookView – auto-load, CSV export, class average, empty/loading states
4 | //
5 | // Feature: qa-partner-review-remediation – Req 13 (P9)
6 | // Validates: Requirements 13.1, 13.2, 13.3, 13.4, 13.5
7 | //
8 | // GradebookView renders a students x assessments matrix. Req 13 adds:
9 | // 13.1 – auto-select the course from route/context (the `course` URL param /
10 | // first available course) so the gradebook loads without a manual pick
11 | // 13.2 – an "Export CSV" button that reuses the shared `downloadCsv` helper
12 | // from `src/lib/exportCurriculumMatrixCsv.ts`
13 | // 13.3 – a computed class-average row derived from the displayed matrix
14 | // 13.4 – when a course has no grades, still render the matrix structure
15 | // (headers + student names) with empty grade cells
16 | // 13.5 – while loading, show a shimmer rather than the no-data presentation
17 | //
18 | // The data/mutation hooks are mocked (no Supabase/network). `nuqs` is stubbed
19 | // with a controllable in-memory store + spied setters so the auto-load effect
20 | // is observable. The real `buildGradebookCsv`/`computeClassAverages` business
21 | // logic runs; only `downloadCsv` (the file-download side effect) is spied.
22 | // =====
23 |
24 | import { describe, it, expect, vi, beforeEach, beforeAll } from "vitest";
25 | import { render, screen, waitFor, within } from "@testing-library/react";
26 | import userEvent from "@testing-library/user-event";
27 | import { I18nextProvider } from "react-i18next";
28 |
29 | import i18n from "@lib/i18n";
30 | import type { Course } from "@types/app";
31 | import type { GradeCategory, GradebookEntry } from "@hooks/useGradebook";
32 |
33 | // -----
34 | // Fixtures
35 | // -----
36 |
37 | const COURSES: Course[] = [
38 |   {
39 |     id: "course-1",
40 |     name: "Intro to Computer Science",
41 |     code: "CS101",
42 |     program_id: "prog-1",
43 |     semester: "Fall 2025",
44 |     semester_id: "sem-1",
45 |     teacher_id: "teacher-1",
46 |     academic_year: "2025",
47 |     is_active: true,
48 |     created_at: "2025-01-01",
49 |   },
50 |   {
51 |     id: "course-2",
52 |     name: "Data Structures",
53 |     code: "CS201",
54 |     program_id: "prog-1",
55 |     semester: "Fall 2025",
56 |     semester_id: "sem-1",
57 |     teacher_id: "teacher-1",
58 |     academic_year: "2025",
59 |     is_active: true,
60 |     created_at: "2025-01-02",
61 |   },
62 | ];
63 |
64 | // A single balanced (100%) category so no "weights don't sum to 100%" banner
65 | // shows and the matrix renders cleanly.
66 | const CATEGORIES: GradeCategory[] = [
67 |   {
68 |     id: "cat-1",
69 |     course_id: "course-1",
70 |     name: "Assignments",
71 |     weight_percent: 100,
72 |     sort_order: 0,
73 |     created_at: "2025-01-01",
74 |   },
75 | ];
76 |
77 | // Two students with one graded assessment each !' class average is computable.
78 | // final grades 80 + 90 !' mean = 85 !' "85.0%", letter "A".
79 | const GRADEBOOK_WITH_GRADES: GradebookEntry[] = [
80 |   {
81 |     student_id: "s1",
82 |     student_name: "Alice Anderson",
83 |     categories: [
84 |       {
85 |         category_id: "cat-1",
86 |         category_name: "Assignments",
87 |         weight_percent: 100,
88 |         assessments: [
89 |           {
90 |             id: "a1",
91 |             title: "Essay One",
92 |             type: "assignment",
93 |             score: 80,
94 |             max_score: 100,
95 |           },
96 |         ],
97 |         subtotal_percent: 80,
98 |       },
99 |     ],
100 |     final_weighted_grade: 80,
101 |     letter_grade: "",
```

```

102 | },
103 | {
104 |   student_id: "s2",
105 |   student_name: "Bob Brown",
106 |   categories: [
107 |     {
108 |       category_id: "cat-1",
109 |       category_name: "Assignments",
110 |       weight_percent: 100,
111 |       assessments: [
112 |         {
113 |           id: "a1",
114 |           title: "Essay One",
115 |           type: "assignment",
116 |           score: 90,
117 |           max_score: 100,
118 |         },
119 |       ],
120 |       subtotal_percent: 90,
121 |     },
122 |   ],
123 |   final_weighted_grade: 90,
124 |   letter_grade: "",
125 | },
126 | ];
127 |
128 | // Students enrolled, but no scores recorded yet (all assessment scores null).
129 | // Req 13.4: the matrix structure must still render with empty cells.
130 | const GRADEBOOK_NO_GRADES: GradebookEntry[] = [
131 |   {
132 |     student_id: "s1",
133 |     student_name: "Alice Anderson",
134 |     categories: [
135 |       {
136 |         category_id: "cat-1",
137 |         category_name: "Assignments",
138 |         weight_percent: 100,
139 |         assessments: [
140 |           {
141 |             id: "a1",
142 |             title: "Essay One",
143 |             type: "assignment",
144 |             score: null,
145 |             max_score: 100,
146 |           },
147 |         ],
148 |         subtotal_percent: 0,
149 |       },
150 |     ],
151 |     final_weighted_grade: 0,
152 |     letter_grade: "",
153 |   },
154 | ];
155 |
156 | // -----
157 | // Mocks (must precede the component import)
158 | // -----
159 |
160 | // nuqs – controllable in-memory store + spied setters. Defined via vi.hoisted
161 | // so it is available inside the hoisted vi.mock factory below.
162 | const nuqs = vi.hoisted(() => ({
163 |   store: new Map<string, string>(),
164 |   setCourse: vi.fn<(value: string) => void>(),
165 |   setSection: vi.fn<(value: string) => void>(),
166 | }));
167 |
168 | vi.mock("nuqs", () => ({
169 |   parseAsString: { withDefault: (def: string) => def },
170 |   useQueryState: (key: string, defaultVal: string) => {
171 |     const value =
172 |       nuqs.store.get(key) ?? (typeof defaultVal === "string" ? defaultVal : "");
173 |     const setter = key === "course" ? nuqs.setCourse : nuqs.setSection;
174 |     return [value, setter] as const;
175 |   },
176 | }));
177 |
178 | // Gradebook data hooks – driven per-test.
179 | const mockGradebookMatrix =
180 |   vi.fn<() => { data: GradebookEntry[]; isLoading: boolean }>();
181 |
182 | vi.mock("@/hooks/useGradebook", () => ({
183 |   useGradebookMatrix: () => mockGradebookMatrix(),
184 |   useGradeCategories: () => ({ data: CATEGORIES }),
185 |   // GradeCategoryManager (imported by GradebookView, rendered only in the
186 |   // non-default "Categories" tab) destructures these – provide inert stubs.
187 |   useCreateGradeCategory: () => ({ mutate: vi.fn(), isLoading: false }),
188 |   useUpdateGradeCategory: () => ({ mutate: vi.fn(), isLoading: false }),
189 |   useDeleteGradeCategory: () => ({ mutate: vi.fn(), isLoading: false }),
190 | }));
191 |
192 | const mockUseCourses =
193 |   vi.fn<() => { data: { data: Course[] }; isLoading: boolean }>();
194 |
195 | vi.mock("@/hooks/useCourses", () => ({
196 |   useCourses: () => mockUseCourses(),
197 |   useTeacherCourses: () => mockUseCourses(),
198 | }));
199 |
200 | vi.mock("@/hooks/useCourseSections", () => ({
201 |   useCourseSections: () => ({ data: [] }),
202 | }));
203 |
204 | vi.mock("@/hooks/useInstitutionSettings", () => ({
205 |   // null settings !' component falls back to DEFAULT_GRADE_SCALES.

```

```

206 | useInstitutionSettings: () => ({ data: null })),
207 | });
208 |
209 | // Spy only on the file-download side effect; keep the rest of the module real
210 | // so `buildGradebookCsv` (which depends on `escapeCsvField` from this module)
211 | // continues to work (Req 13.2/13.6 – the export reuses this shared helper).
212 | // Defined via vi.hoisted so it is initialized before the hoisted vi.mock factory.
213 | const { mockDownloadCsv } = vi.hoisted(() => ({
214 |   mockDownloadCsv: vi.fn<(csv: string, filename: string) => void>(),
215 | }));
216 | vi.mock("@/lib/exportCurriculumMatrixCsv", async () => {
217 |   const actual = await vi.importActual<
218 |     typeof import("@/lib/exportCurriculumMatrixCsv")
219 |   >("@/lib/exportCurriculumMatrixCsv");
220 |   return { ...actual, downloadCsv: mockDownloadCsv };
221 | });
222 |
223 | // happy-dom lacks ResizeObserver + the pointer/scroll APIs Radix Select needs.
224 | class MockResizeObserver {
225 |   observe() {}
226 |   unobserve() {}
227 |   disconnect() {}
228 | }
229 | vi.stubGlobal("ResizeObserver", MockResizeObserver);
230 |
231 | beforeAll(() => {
232 |   if (!Element.prototype.hasPointerCapture) {
233 |     Element.prototype.hasPointerCapture = () => false;
234 |   }
235 |   if (!Element.prototype.releasePointerCapture) {
236 |     Element.prototype.releasePointerCapture = () => {};
237 |   }
238 |   if (!Element.prototype.scrollIntoView) {
239 |     Element.prototype.scrollIntoView = () => {};
240 |   }
241 | });
242 |
243 | import GradebookView from "@/pages/teacher/gradebook/GradebookView";
244 |
245 | // -----
246 | // Helpers
247 | // -----
248 |
249 | const renderView = () =>
250 |   render(
251 |     <I18nextProvider i18n={i18n}>
252 |       <GradebookView />
253 |     </I18nextProvider>
254 |   );
255 |
256 | const NO_STUDENTS_TITLE = i18n.t("common:empty.noStudents.title");
257 |
258 | // -----
259 | // Tests
260 | // -----
261 |
262 | describe("GradebookView – auto-load, export, class average, empty/loading (Req 13.1–13.5)", () => {
263 |   beforeEach(() => {
264 |     vi.clearAllMocks();
265 |     nuqs.store.clear();
266 |     // Default: courses available, not loading.
267 |     mockUseCourses.mockReturnValue({
268 |       data: { data: COURSES },
269 |       isLoading: false,
270 |     });
271 |     // Default: grades present and resolved.
272 |     mockGradebookMatrix.mockReturnValue({
273 |       data: GRADEBOOK_WITH_GRADES,
274 |       isLoading: false,
275 |     });
276 |   });
277 |
278 |   // --- Req 13.5: loading -----
279 |   it("shows a shimmer while the gradebook is loading (Req 13.5)", () => {
280 |     nuqs.store.set("course", "course-1"); // past the "select a course" guard
281 |     mockGradebookMatrix.mockReturnValue({ data: [], isLoading: true });
282 |
283 |     const { container } = renderView();
284 |
285 |     // Component-level shimmer placeholders are present while loading.
286 |     expect(
287 |       container.querySelectorAll(".animate-shimmer").length
288 |     ).toBeGreaterThan(0);
289 |
290 |     // The loading state is NOT the no-data empty state, and no student rows render.
291 |     expect(screen.queryByText(NO_STUDENTS_TITLE)).not.toBeInTheDocument();
292 |     expect(screen.queryByText("Alice Anderson")).not.toBeInTheDocument();
293 |   });
294 |
295 |   // --- Req 13.1: auto-load -----
296 |   it("auto-selects the course from context when none is selected (Req 13.1)", async () => {
297 |     // No `course` URL state set !' the auto-load effect should pick the first course.
298 |     renderView();
299 |
300 |     await waitFor(() => {
301 |       expect(nuqs.setCourse).toHaveBeenCalled("course-1");
302 |     });
303 |   });
304 |
305 |   it("does not override an already-selected course (Req 13.1)", () => {
306 |     nuqs.store.set("course", "course-2");
307 |     renderView();
308 |     // The course is already chosen !' no auto-selection occurs.
309 |     expect(nuqs.setCourse).not.toHaveBeenCalled();

```

```

310 | });
311 |
312 | // --- Req 13.3: data present + class-average row -----
313 | it("renders student names, grades, and a class-average row with the correct average (Req 13.3)", () => {
314 |   nuqs.store.set("course", "course-1");
315 |   renderView();
316 |
317 |   // Student names + their grades render.
318 |   expect(screen.getByText("Alice Anderson")).toBeInTheDocument();
319 |   expect(screen.getByText("Bob Brown")).toBeInTheDocument();
320 |
321 |   // The class-average row exists and shows the computed average ((80+90)/2 = 85).
322 |   const avgRow = screen.getByText("Class Average").closest("tr");
323 |   expect(avgRow).not.toBeNull();
324 |   const row = avgRow as HTMLTableRowElement;
325 |   // Final-average and category-average cells both read "85.0%".
326 |   expect(within(row).getAllByText("85.0%").length).toBeGreaterThanOrEqual(1);
327 |   // The per-assessment average cell reads the mean score "85".
328 |   expect(within(row).getByText("85")).toBeInTheDocument();
329 |   // The class-average letter grade (85% != "A" on the default scale).
330 |   expect(within(row).getByText("A")).toBeInTheDocument();
331 | });
332 |
333 | // --- Req 13.4: empty (no grades) !' matrix structure still renders -----
334 | it("renders the matrix structure with empty cells when a course has no grades (Req 13.4)", () => {
335 |   nuqs.store.set("course", "course-1");
336 |   mockGradebookMatrix.mockReturnValue({
337 |     data: GRADEBOOK_NO_GRADES,
338 |     isLoading: false,
339 |   });
340 |
341 |   renderView();
342 |
343 |   // Column headers + student name still render (matrix is not hidden).
344 |   expect(screen.getByText("Student")).toBeInTheDocument();
345 |   expect(screen.getByText("Assignments")).toBeInTheDocument();
346 |   expect(screen.getByText("Essay One")).toBeInTheDocument();
347 |   expect(screen.getByText("Alice Anderson")).toBeInTheDocument();
348 |
349 |   // Empty grade cells are shown as the em-dash placeholder.
350 |   expect(screen.getAllByText("-").length).toBeGreaterThan(0);
351 |
352 |   // The no-students empty state is NOT used in this "has students, no grades" case.
353 |   expect(screen.queryByText(NO_STUDENTS_TITLE)).not.toBeInTheDocument();
354 | });
355 |
356 | // --- Req 13.2 / 13.6: Export CSV reuses the shared download helper -----
357 | it("invokes the shared CSV download helper when Export CSV is clicked (Req 13.2)", async () => {
358 |   const user = userEvent.setup();
359 |   nuqs.store.set("course", "course-1");
360 |   renderView();
361 |
362 |   const exportButton = screen.getByRole("button", { name: /export csv/i });
363 |   expect(exportButton).toBeEnabled();
364 |
365 |   await user.click(exportButton);
366 |
367 |   // The component delegates the file download to `downloadCsv` (Req 13.6).
368 |   expect(mockDownloadCsv).toHaveBeenCalledTimes(1);
369 |   const [csvContent, filename] = mockDownloadCsv.mock.calls[0]!;
370 |   expect(typeof csvContent).toBe("string");
371 |   expect(csvContent).toContain("Class Average");
372 |   expect(filename).toBe("gradebook-CS101.csv");
373 | });
374 |
375 | it("disables the Export CSV button when there are no grades to export (Req 13.2)", () => {
376 |   nuqs.store.set("course", "course-1");
377 |   mockGradebookMatrix.mockReturnValue({ data: [], isLoading: false });
378 |
379 |   renderView();
380 |
381 |   expect(screen.getByRole("button", { name: /export csv/i })).toBeDisabled();
382 | });
383 | });
384 |

```

FILE: [src/__tests__/unit/gradingInterface.test.tsx](#)

TYPE: .tsx | LINES: 1-401 | SHA256: 38fd1925ad04bf7cfb14cdb680a27237693d7e0d2fd9101f64a6a7c7c1af37e2

```

1 | import { describe, it, expect, vi, beforeEach } from "vitest";
2 | import { render, screen } from "@testing-library/react";
3 | import userEvent from "@testing-library/user-event";
4 | import { MemoryRouter, Route, Routes } from "react-router-dom";
5 |
6 | // -----
7 | // Mocks
8 | // -----
9 |
10 | const mockNavigate = vi.fn();
11 | vi.mock("react-router-dom", async () => {
12 |   const actual = await vi.importActual("react-router-dom");
13 |   return { ...actual, useNavigate: () => mockNavigate };
14 | });
15 |
16 | const mockUseSubmission = vi.fn();
17 | vi.mock("@/hooks/useSubmissions", () => ({
18 |   useSubmission: (...args: unknown[]) => mockUseSubmission(...args),
19 | }));
20 |
21 | const mockUseRubric = vi.fn();
22 | vi.mock("@/hooks/useRubrics", () => ({
23 |   useRubric: (...args: unknown[]) => mockUseRubric(...args),
24 | }));
25 |

```

```

26 | const mockUseGrade = vi.fn();
27 | const mockMutate = vi.fn();
28 | vi.mock("@/hooks/useGrades", () => ({
29 |   useGrade: (...args: unknown[]) => mockUseGrade(...args),
30 |   useCreateGrade: () => ({ mutate: mockMutate, isPending: false }),
31 | }));
32 |
33 | vi.mock("@/hooks/useAuth", () => ({
34 |   useAuth: () => ({
35 |     user: { id: "teacher-1" },
36 |     profile: null,
37 |     role: "teacher",
38 |   }),
39 | }));
40 |
41 | // Mock the internal useQuery for useAssignmentForRubric
42 | const mockUseQuery = vi.fn();
43 | vi.mock("@tanstack/react-query", async () => {
44 |   const actual = await vi.importActual("@tanstack/react-query");
45 |   return { ...actual, useQuery: (...args: unknown[]) => mockUseQuery(...args) };
46 | });
47 |
48 | vi.mock("@/lib/supabase", () => ({
49 |   supabase: {},
50 | }));
51 |
52 | vi.mock("@/lib/queryKeys", () => ({
53 |   queryKeys: {
54 |     assignments: {
55 |       detail: (id: string) => ["assignments", "detail", id],
56 |     },
57 |   },
58 | }));
59 |
60 | vi.mock("sonner", () => ({
61 |   toast: { success: vi.fn(), error: vi.fn() },
62 | }));
63 |
64 | vi.mock("@/lib/activityLogger", () => ({
65 |   logActivity: vi.fn().mockResolvedValue(undefined),
66 | }));
67 |
68 | vi.mock("@/hooks/useAIFeedbackDraft", () => ({
69 |   useGenerateFeedbackDraft: () => ({
70 |     mutate: vi.fn(),
71 |     isPending: false,
72 |   }),
73 |   useLogDraftAction: () => ({
74 |     mutate: vi.fn(),
75 |   }),
76 | }));
77 |
78 | vi.mock("@/lib/predictionValidator", () => ({
79 |   validateAtRiskPredictions: vi.fn().mockResolvedValue(undefined),
80 | }));
81 |
82 | // -----
83 | // Fixtures
84 | // -----
85 |
86 | const baseSubmission = {
87 |   id: "sub-1",
88 |   assignment_id: "assign-1",
89 |   student_id: "student-1",
90 |   submitted_at: "2024-06-15T10:30:00Z",
91 |   file_url: "https://storage.example.com/file.pdf",
92 |   is_late: false,
93 |   created_at: "2024-06-15T10:30:00Z",
94 |   institution_id: "inst-1",
95 |   profiles: {
96 |     id: "student-1",
97 |     full_name: "Alice Johnson",
98 |     email: "alice@example.com",
99 |   },
100 |   assignments: {
101 |     id: "assign-1",
102 |     title: "Essay Assignment",
103 |     total_marks: 100,
104 |     course_id: "course-1",
105 |   },
106 |   grades: null,
107 | };
108 |
109 | const baseAssignment = {
110 |   id: "assign-1",
111 |   title: "Essay Assignment",
112 |   total_marks: 100,
113 |   rubric_id: "rubric-1",
114 | };
115 |
116 | const baseRubric = {
117 |   id: "rubric-1",
118 |   title: "Essay Rubric",
119 |   clo_id: "clo-1",
120 |   is_template: false,
121 |   created_at: "2024-01-01T00:00:00Z",
122 |   updated_at: "2024-01-01T00:00:00Z",
123 |   criteria: [
124 |     {
125 |       id: "c1",
126 |       rubric_id: "rubric-1",
127 |       criterion_name: "Clarity",
128 |       sort_order: 0,
129 |       levels: [

```

```

130         { label: "Developing", description: "Needs improvement", points: 10 },
131         { label: "Proficient", description: "Meets expectations", points: 30 },
132         { label: "Exemplary", description: "Exceeds expectations", points: 50 },
133     ],
134     max_points: 50,
135 },
136 {
137     id: "c2",
138     rubric_id: "rubric-1",
139     criterion_name: "Grammar",
140     sort_order: 1,
141     levels: [
142         { label: "Developing", description: "Many errors", points: 10 },
143         { label: "Proficient", description: "Few errors", points: 30 },
144         { label: "Exemplary", description: "No errors", points: 50 },
145     ],
146     max_points: 50,
147 },
148 ],
149 };
150
151 // -----
152 // Helper to render with router context
153 // -----
154
155 const renderGradingInterface = async () => {
156     const GradingInterface = (
157         await import("@/pages/teacher/grading/GradingInterface")
158     ).default;
159     return render(
160         <MemoryRouter initialEntries={["/teacher/grading/sub-1"]}>
161             <Routes>
162                 <Route
163                     path="/teacher/grading/:submissionId"
164                     element={<GradingInterface />}
165                 />
166             </Routes>
167         </MemoryRouter>
168     );
169 };
170
171 // -----
172 // Tests
173 // -----
174
175 describe("GradingInterface", () => {
176     beforeEach(() => {
177         vi.clearAllMocks();
178
179         // Default: useQuery returns the assignment (useAssignmentForRubric)
180         mockUseQuery.mockReturnValue({
181             data: baseAssignment,
182             isLoading: false,
183         });
184
185         mockUseSubmission.mockReturnValue({
186             data: baseSubmission,
187             isLoading: false,
188         });
189
190         mockUseRubric.mockReturnValue({
191             data: baseRubric,
192             isLoading: false,
193         });
194
195         mockUseGrade.mockReturnValue({
196             data: null,
197             isLoading: false,
198         });
199     });
200
201     it("renders the page title for ungraded submission", async () => {
202         await renderGradingInterface();
203         expect(screen.getByText("Grade Submission")).toBeInTheDocument();
204     }, 15000);
205
206     it("displays student name and email", async () => {
207         await renderGradingInterface();
208         // Use getAllByText to handle cases where the name may appear in multiple places
209         const nameElements = screen.getAllByText("Alice Johnson");
210         expect(nameElements.length).toBeGreaterThanOrEqual(1);
211         const emailElements = screen.getAllByText("alice@example.com");
212         expect(emailElements.length).toBeGreaterThanOrEqual(1);
213     });
214
215     it("displays assignment title and total marks", async () => {
216         await renderGradingInterface();
217         expect(screen.getByText("Essay Assignment")).toBeInTheDocument();
218         expect(screen.getByText("Total Marks: 100")).toBeInTheDocument();
219     });
220
221     it("displays rubric criteria names", async () => {
222         await renderGradingInterface();
223         expect(screen.getByText("Clarity")).toBeInTheDocument();
224         expect(screen.getByText("Grammar")).toBeInTheDocument();
225     });
226
227     it("displays rubric level headers", async () => {
228         await renderGradingInterface();
229         // Level labels appear in the header row
230         const developingHeaders = screen.getAllByText("Developing");
231         expect(developingHeaders.length).toBeGreaterThanOrEqual(1);
232     });
233 }

```



```

234 | it("displays level descriptions in cells", async () => {
235 |     await renderGradingInterface();
236 |     expect(screen.getByText("Needs improvement")).toBeInTheDocument();
237 |     expect(screen.getByText("Meets expectations")).toBeInTheDocument();
238 |     expect(screen.getByText("No errors")).toBeInTheDocument();
239 | });
240 |
241 | it("displays points badges in cells", async () => {
242 |     await renderGradingInterface();
243 |     const ptsBadges = screen.getAllByText("10 pts");
244 |     expect(ptsBadges.length).toBe(2); // 2 criteria × Developing level
245 | });
246 |
247 | it("shows late badge when submission is late", async () => {
248 |     mockUseSubmission.mockReturnValue({
249 |         data: { ...baseSubmission, is_late: true },
250 |         isLoading: false,
251 |     });
252 |     await renderGradingInterface();
253 |     expect(screen.getByText("Late Submission")).toBeInTheDocument();
254 | });
255 |
256 | it("shows file link when file_url is present", async () => {
257 |     await renderGradingInterface();
258 |     expect(screen.getByText("View Submission File")).toBeInTheDocument();
259 | });
260 |
261 | it("selects a rubric cell on click and shows checkmark", async () => {
262 |     const user = userEvent.setup();
263 |     await renderGradingInterface();
264 |
265 |     // Click the "Exceeds expectations" cell for Clarity (50 pts)
266 |     const cell = screen.getByRole("button", {
267 |         name: /Clarity: Exemplary – 50 points/,
268 |     });
269 |     await user.click(cell);
270 |
271 |     // The cell should now be pressed
272 |     expect(cell).toHaveAttribute("aria-pressed", "true");
273 | });
274 |
275 | it("allows only one selection per criterion", async () => {
276 |     const user = userEvent.setup();
277 |     await renderGradingInterface();
278 |
279 |     // Select Developing for Clarity
280 |     const developing = screen.getByRole("button", {
281 |         name: /Clarity: Developing – 10 points/,
282 |     });
283 |     await user.click(developing);
284 |     expect(developing).toHaveAttribute("aria-pressed", "true");
285 |
286 |     // Now select Exemplary for Clarity – should deselect Developing
287 |     const exemplary = screen.getByRole("button", {
288 |         name: /Clarity: Exemplary – 50 points/,
289 |     });
290 |     await user.click(exemplary);
291 |     expect(exemplary).toHaveAttribute("aria-pressed", "true");
292 |     expect(developing).toHaveAttribute("aria-pressed", "false");
293 | });
294 |
295 | it("toggles off a cell when clicking the same cell again", async () => {
296 |     const user = userEvent.setup();
297 |     await renderGradingInterface();
298 |
299 |     const cell = screen.getByRole("button", {
300 |         name: /Clarity: Proficient – 30 points/,
301 |     });
302 |     await user.click(cell);
303 |     expect(cell).toHaveAttribute("aria-pressed", "true");
304 |
305 |     await user.click(cell);
306 |     expect(cell).toHaveAttribute("aria-pressed", "false");
307 | });
308 |
309 | it("auto-calculates total score from selected cells", async () => {
310 |     const user = userEvent.setup();
311 |     await renderGradingInterface();
312 |
313 |     // Select Exemplary (50) for Clarity
314 |     await user.click(
315 |         screen.getByRole("button", { name: /Clarity: Exemplary – 50 points/ })
316 |     );
317 |     // Select Proficient (30) for Grammar
318 |     await user.click(
319 |         screen.getByRole("button", { name: /Grammar: Proficient – 30 points/ })
320 |     );
321 |
322 |     // Total should be 80 / 100
323 |     expect(screen.getByText("80 / 100")).toBeInTheDocument();
324 |     expect(screen.getByText("80%")).toBeInTheDocument();
325 | });
326 |
327 | it('shows "not found" when submission does not exist', async () => {
328 |     mockUseSubmission.mockReturnValue({ data: null, isLoading: false });
329 |     await renderGradingInterface();
330 |     expect(screen.getByText("Submission not found.")).toBeInTheDocument();
331 | });
332 |
333 | it("shows shimmer loading state", async () => {
334 |     mockUseSubmission.mockReturnValue({ data: null, isLoading: true });
335 |     const { container } = await renderGradingInterface();
336 |     // Shimmer components should be rendered
337 |     const shimmers = container.querySelectorAll('[class*="animate"]');

```

```

338 |     expect(shimmers.length).toBeGreaterThan(0);
339 | });
340 |
341 | it("shows read-only mode when grade already exists", async () => {
342 |   mockUseGrade.mockReturnValue({
343 |     data: {
344 |       id: "grade-1",
345 |       submission_id: "sub-1",
346 |       rubric_selections: [
347 |         { criterion_id: "c1", level_index: 2, points: 50 },
348 |         { criterion_id: "c2", level_index: 1, points: 30 },
349 |       ],
350 |       total_score: 80,
351 |       score_percent: 80,
352 |       overall_feedback: "Great work!",
353 |       graded_by: "teacher-1",
354 |       created_at: "2024-06-16T00:00:00Z",
355 |       institution_id: "inst-1",
356 |     },
357 |     isLoading: false,
358 |   });
359 |
360 |   await renderGradingInterface();
361 |
362 |   expect(screen.getByText("Grade Review")).toBeInTheDocument();
363 |   expect(screen.getByText("Graded")).toBeInTheDocument();
364 |   expect(screen.getByText("Great work!")).toBeInTheDocument();
365 |   // Submit button should not be present
366 |   expect(screen.queryByText("Submit Grade")).not.toBeInTheDocument();
367 | });
368 |
369 | it('shows "no rubric" message when rubric is not linked', async () => {
370 |   mockUseRubric.mockReturnValue({ data: null, isLoading: false });
371 |   await renderGradingInterface();
372 |   expect(
373 |     screen.getByText("No rubric linked to this assignment.")
374 |   ).toBeInTheDocument();
375 | });
376 |
377 | it("navigates back to queue on Back button click", async () => {
378 |   const user = userEvent.setup();
379 |   await renderGradingInterface();
380 |
381 |   const backBtn = screen.getByRole("button", { name: /Back to Queue/ });
382 |   await user.click(backBtn);
383 |   expect(mockNavigate).toHaveBeenCalledWith("/teacher/grading");
384 | });
385 |
386 | it("shows attainment level badge based on score", async () => {
387 |   const user = userEvent.setup();
388 |   await renderGradingInterface();
389 |
390 |   // Select both Exemplary (50 + 50 = 100/100 = 100%)
391 |   await user.click(
392 |     screen.getByRole("button", { name: /Clarity: Exemplary – 50 points/ })
393 |   );
394 |   await user.click(
395 |     screen.getByRole("button", { name: /Grammar: Exemplary – 50 points/ })
396 |   );
397 |
398 |   expect(screen.getByText("Excellent")).toBeInTheDocument();
399 | });
400 | });
401 |

```

FILE: [src/__tests__/unit/gradingQueueColumns.test.tsx](#)

TYPE: .tsx | LINES: 1-43 | SHA256: a0d12aeb37506395d65baebe8cf627d7209f1f93127ceabe8b205f916dec33ca

```

1 | import { render, screen } from "@testing-library/react";
2 | import { MemoryRouter } from "react-router-dom";
3 | import { describe, expect, it } from "vitest";
4 |
5 | import { DataTable } from "@components/shared/DataTable";
6 | import { gradingQueueColumns } from "@pages/teacher/grading/gradingQueueColumns";
7 | import type { SubmissionWithRelations } from "@hooks/useSubmissions";
8 |
9 | describe("gradingQueueColumns", () => {
10 |   it("renders the submission timestamp from submitted_at", () => {
11 |     const submission: SubmissionWithRelations = {
12 |       id: "submission-1",
13 |       assignment_id: "assignment-1",
14 |       student_id: "student-1",
15 |       file_url: "submission.pdf",
16 |       is_late: false,
17 |       submitted_at: "2026-07-29T10:30:00.000Z",
18 |       status: "submitted",
19 |       profiles: {
20 |         id: "student-1",
21 |         full_name: "Aarav Sharma",
22 |         email: "aarav@example.test",
23 |       },
24 |       assignments: {
25 |         id: "assignment-1",
26 |         title: "Place Value Quiz",
27 |         total_marks: 100,
28 |         course_id: "course-1",
29 |       },
30 |       grades: [],
31 |     };
32 |
33 |     render(
34 |       <MemoryRouter>
35 |         <DataTable columns={gradingQueueColumns} data={[submission]} />
36 |       </MemoryRouter>

```

```

37 |     );
38 |
39 |     expect(screen.getByText("Aarav Sharma")).toBeInTheDocument();
40 |     expect(screen.getByText(/Jul 29, 2026/)).toBeInTheDocument();
41 |   });
42 | });
43 |

```

FILE: src/__tests__/unit/gradingQueueScope.test.tsx

TYPE: .tsx | LINES: 1-82 | SHA256: 2596e3f23dc519f8fbee1eac92c2c13da5566c01ede27fde77454f705271fa5e

```

1 | import { render, screen } from "@testing-library/react";
2 | import { beforeEach, describe, expect, it, vi } from "vitest";
3 | import GradingQueuePage from "@pages/teacher/grading/GradingQueuePage";
4 | import { useAuth } from "@hooks/useAuth";
5 | import { useCourses } from "@hooks/useCourses";
6 | import { useSubmissions } from "@hooks/useSubmissions";
7 |
8 | vi.mock("@hooks/useAuth", () => ({ useAuth: vi.fn() }));
9 | vi.mock("@hooks/useCourses", () => ({ useCourses: vi.fn() }));
10 | vi.mock("@hooks/useSubmissions", () => ({ useSubmissions: vi.fn() }));
11 | vi.mock("@hooks/useAssignments", () => ({
12 |   useAssignments: vi.fn(() => ({ data: { data: [] } })),
13 | }));
14 | vi.mock("@hooks/useCourseSections", () => ({
15 |   useCourseSections: vi.fn(() => ({ data: [] })),
16 | }));
17 | vi.mock("nuqs", () => ({
18 |   parseAsString: { withDefault: (value: string) => value },
19 |   useQueryState: vi.fn(() => ["", vi.fn()]),
20 | }));
21 | vi.mock("@components/shared/DataTable", () => ({
22 |   DataTable: ({
23 |     data,
24 |     emptyState,
25 |   }: {
26 |     data: unknown[];
27 |     emptyState?: React.ReactNode;
28 |   }) => <div>{data.length === 0 ? emptyState : `${data.length} rows`}</div>,
29 | }));
30 |
31 | describe("GradingQueuePage teacher scope", () => {
32 |   beforeEach(() => {
33 |     vi.clearAllMocks();
34 |     vi.mocked(useAuth).mockReturnValue({
35 |       user: { id: "teacher-1" },
36 |     } as ReturnType<typeof useAuth>);
37 |     vi.mocked(useCourses).mockReturnValue({
38 |       data: {
39 |         data: [
40 |           {
41 |             id: "course-1",
42 |             code: "CS101",
43 |             name: "Foundations",
44 |           },
45 |         ],
46 |         count: 1,
47 |         page: 1,
48 |         pageSize: 100,
49 |       },
50 |     } as ReturnType<typeof useCourses>);
51 |     vi.mocked(useSubmissions).mockReturnValue({
52 |       data: { data: [], count: 0, page: 1, pageSize: 20 },
53 |       isLoading: false,
54 |       isFetching: false,
55 |     } as unknown as ReturnType<typeof useSubmissions>);
56 |   });
57 |
58 |   it("limits courses and pending submissions to the signed-in teacher", () => {
59 |     render(<GradingQueuePage />);
60 |
61 |     expect(useCourses).toHaveBeenCalledWith({
62 |       teacherId: "teacher-1",
63 |       pageSize: 100,
64 |     });
65 |     expect(useSubmissions).toHaveBeenCalledWith(
66 |       expect.objectContaining({
67 |         courseIds: ["course-1"],
68 |         pendingOnly: true,
69 |       })
70 |     );
71 |   });
72 |
73 |   it("shows a caught-up state when the teacher has courses but no pending work", () => {
74 |     render(<GradingQueuePage />);
75 |
76 |     expect(
77 |       screen.getByRole("heading", { name: "empty.noSubmissions.title" })
78 |     ).toBeInTheDocument();
79 |     expect(screen.queryByText("empty.noCourses.title")).not.toBeInTheDocument();
80 |   });
81 | });
82 |

```

FILE: src/__tests__/unit/gradingStats.test.tsx

TYPE: .tsx | LINES: 1-164 | SHA256: 4618f5183ac38686413f9c5fb4a17fc607ef7fa269591535dc7e9daa02148bac

```

1 | import { describe, it, expect, vi, beforeEach } from "vitest";
2 | import { render, screen } from "@testing-library/react";
3 |
4 | // -----
5 | // Mocks
6 | // -----
7 |

```

```

8 | const mockUseGradingStats = vi.fn();
9 | vi.mock("@hooks/useGradingStats", () => ({
10 |   useGradingStats: (...args: unknown[]) => mockUseGradingStats(...args),
11 | }));
12 |
13 | // Mock Recharts to avoid rendering SVG in jsdom
14 | vi.mock("recharts", () => ({
15 |   ResponsiveContainer: ({ children }: { children: React.ReactNode }) => (
16 |     <div data-testid="responsive-container">{children}</div>
17 |   ),
18 |   LineChart: ({ children }: { children: React.ReactNode }) => (
19 |     <div data-testid="line-chart">{children}</div>
20 |   ),
21 |   Line: () => <div data-testid="line" />,
22 |   XAxis: () => <div data-testid="x-axis" />,
23 |   YAxis: () => <div data-testid="y-axis" />,
24 |   CartesianGrid: () => <div data-testid="cartesian-grid" />,
25 |   Tooltip: () => <div data-testid="tooltip" />,
26 | }));
27 |
28 | // -----
29 | // Fixtures
30 | // -----
31 |
32 | const baseStats = {
33 |   gradedThisWeek: 12,
34 |   avgGradingTimeSeconds: 185,
35 |   pendingCount: 5,
36 |   gradingStreak: 3,
37 |   velocityTrend: [
38 |     { date: "2024-06-10", count: 4 },
39 |     { date: "2024-06-11", count: 6 },
40 |     { date: "2024-06-12", count: 2 },
41 |   ],
42 | };
43 |
44 | // -----
45 | // Helper
46 | // -----
47 |
48 | const renderGradingStats = async () => {
49 |   const GradingStats = (await import("@/pages/teacher/dashboard/GradingStats"))
50 |     .default;
51 |   return render(<GradingStats teacherId="teacher-1" />);
52 | };
53 |
54 | // -----
55 | // Tests
56 | // -----
57 |
58 | describe("GradingStats", () => {
59 |   beforeEach(() => {
60 |     vi.clearAllMocks();
61 |   });
62 |
63 |   it("renders shimmer when loading", async () => {
64 |     mockUseGradingStats.mockReturnValue({ data: undefined, isLoading: true });
65 |     const { container } = await renderGradingStats();
66 |     const shimmers = container.querySelectorAll('[class*="animate"]');
67 |     expect(shimmers.length).toBeGreaterThan(0);
68 |   });
69 |
70 |   it("renders nothing when data is null", async () => {
71 |     mockUseGradingStats.mockReturnValue({ data: null, isLoading: false });
72 |     const { container } = await renderGradingStats();
73 |     expect(container.firstChild).toBeNull();
74 |   });
75 |
76 |   it("renders the section heading", async () => {
77 |     mockUseGradingStats.mockReturnValue({ data: baseStats, isLoading: false });
78 |     await renderGradingStats();
79 |     expect(screen.getByText("Grading Stats")).toBeInTheDocument();
80 |   });
81 |
82 |   it("displays graded this week KPI", async () => {
83 |     mockUseGradingStats.mockReturnValue({ data: baseStats, isLoading: false });
84 |     await renderGradingStats();
85 |     expect(screen.getByText("Graded This Week")).toBeInTheDocument();
86 |     expect(screen.getByText("12")).toBeInTheDocument();
87 |   });
88 |
89 |   it("displays pending count KPI", async () => {
90 |     mockUseGradingStats.mockReturnValue({ data: baseStats, isLoading: false });
91 |     await renderGradingStats();
92 |     expect(screen.getByText("Pending")).toBeInTheDocument();
93 |     expect(screen.getByText("5")).toBeInTheDocument();
94 |   });
95 |
96 |   it("displays grading streak KPI", async () => {
97 |     mockUseGradingStats.mockReturnValue({ data: baseStats, isLoading: false });
98 |     await renderGradingStats();
99 |     expect(screen.getByText("Grading Streak")).toBeInTheDocument();
100 |    expect(screen.getByText("3d")).toBeInTheDocument();
101 |   });
102 |
103 |   it("formats avg grading time as minutes and seconds", async () => {
104 |     mockUseGradingStats.mockReturnValue({ data: baseStats, isLoading: false });
105 |     await renderGradingStats();
106 |     expect(screen.getByText("Avg Time")).toBeInTheDocument();
107 |     // 185 seconds = 3m 5s
108 |     expect(screen.getByText("3m 5s")).toBeInTheDocument();
109 |   });
110 |
111 |   it("shows dash when avg grading time is 0", async () => {

```

```

112 |     mockUseGradingStats.mockReturnValue({
113 |       data: { ...baseStats, avgGradingTimeSeconds: 0 },
114 |       isLoading: false,
115 |     });
116 |     await renderGradingStats();
117 |     expect(screen.getByText("-")).toBeInTheDocument();
118 |   });
119 |
120 |   it("formats time as seconds only when under 60s", async () => {
121 |     mockUseGradingStats.mockReturnValue({
122 |       data: { ...baseStats, avgGradingTimeSeconds: 45 },
123 |       isLoading: false,
124 |     });
125 |     await renderGradingStats();
126 |     expect(screen.getByText("45s")).toBeInTheDocument();
127 |   });
128 |
129 |   it("formats time as minutes only when no remainder", async () => {
130 |     mockUseGradingStats.mockReturnValue({
131 |       data: { ...baseStats, avgGradingTimeSeconds: 120 },
132 |       isLoading: false,
133 |     });
134 |     await renderGradingStats();
135 |     expect(screen.getByText("2m")).toBeInTheDocument();
136 |   });
137 |
138 |   it("renders velocity trend chart when data exists", async () => {
139 |     mockUseGradingStats.mockReturnValue({ data: baseStats, isLoading: false });
140 |     await renderGradingStats();
141 |     expect(
142 |       screen.getByText("Grading Velocity (Last 30 Days)")
143 |     ).toBeInTheDocument();
144 |     expect(screen.getByTestId("line-chart")).toBeInTheDocument();
145 |   });
146 |
147 |   it("shows empty message when no velocity trend data", async () => {
148 |     mockUseGradingStats.mockReturnValue({
149 |       data: { ...baseStats, velocityTrend: [] },
150 |       isLoading: false,
151 |     });
152 |     await renderGradingStats();
153 |     expect(
154 |       screen.getByText("No grading activity in the last 30 days.")
155 |     ).toBeInTheDocument();
156 |   });
157 |
158 |   it("passes teacherId to the hook", async () => {
159 |     mockUseGradingStats.mockReturnValue({ data: baseStats, isLoading: false });
160 |     await renderGradingStats();
161 |     expect(mockUseGradingStats).toHaveBeenCalledWith("teacher-1");
162 |   });
163 | });
164 |

```

FILE: [src/__tests__/unit/graduateAttributeManager.test.tsx](#)

TYPE: .tsx | LINES: 1-196 | SHA256: f87095df134181c8f59d5032432d3a10fb5f649bf76c40cc7e81716a51fd8f59

```

1 | // @vitest-environment happy-dom
2 | import { describe, it, expect, vi } from "vitest";
3 | import { render, screen } from "@testing-library/react";
4 | import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
5 | import { MemoryRouter } from "react-router-dom";
6 |
7 | // % % % Mock every hook the component consumes so no real Supabase is hit % % % % %
8 | //
9 | // GraduateAttributeManager reads from:
10 | //   - useAuth().profile.institution_id (scopes the queries)
11 | //   - useGraduateAttributes(institutionId) (the attribute list)
12 | //   - useGraduateAttributeAttainment(...) (computed GA attainment)
13 | //   - useGraduateAttributeMappings(attributeId) (per-attribute, keyed by id)
14 | //   - useILOs({ pageSize }) (outcome id !' title lookup)
15 | //   - useCreateGraduateAttribute / useDeleteGraduateAttribute (Add form / delete)
16 | //
17 | // Three attributes are seeded to exercise all rendered states in one tree:
18 | //   ga-1 !' has mappings + attainment (Req 6.1, 6.2, 6.6)
19 | //   ga-2 !' zero mappings !' empty state (Req 6.3)
20 | //   ga-loading !' mappings query loading (shimmer placeholder)
21 |
22 | vi.mock("@/hooks/useAuth", () => ({
23 |   useAuth: () => ({ profile: { institution_id: "inst-1" } }),
24 | }));
25 |
26 | vi.mock("@/hooks/useILOs", () => ({
27 |   useILOs: vi.fn(() => ({
28 |     data: {
29 |       data: [
30 |         { id: "ilo-1", title: "ILO-1: Analyze" },
31 |         { id: "ilo-2", title: "ILO-2: Evaluate" },
32 |       ],
33 |       count: 2,
34 |       page: 1,
35 |       pageSize: 1000,
36 |     },
37 |   })),
38 | }));
39 |
40 | vi.mock("@/hooks/useGraduateAttributes", () => ({
41 |   useGraduateAttributes: vi.fn(() => ({
42 |     data: [
43 |       {
44 |         id: "ga-1",
45 |         institution_id: "inst-1",
46 |         name: "Critical Thinking",
47 |         description: "Thinks critically",

```

```

48 |         sort_order: 0,
49 |         created_at: "2025-01-01",
50 |     },
51 |     {
52 |       id: "ga-2",
53 |       institution_id: "inst-1",
54 |       name: "Communication",
55 |       description: null,
56 |       sort_order: 1,
57 |       created_at: "2025-01-01",
58 |     },
59 |     {
60 |       id: "ga-loading",
61 |       institution_id: "inst-1",
62 |       name: "Teamwork",
63 |       description: null,
64 |       sort_order: 2,
65 |       created_at: "2025-01-01",
66 |     },
67 |   ],
68 |   isLoading: false,
69 | })),
70 | useGraduateAttributeAttainment: vi.fn(() => ({
71 |   data: [
72 |     {
73 |       attribute_id: "ga-1",
74 |       name: "Critical Thinking",
75 |       description: "Thinks critically",
76 |       avg_attainment: 88,
77 |       mapped_ilo_count: 2,
78 |     },
79 |     {
80 |       attribute_id: "ga-2",
81 |       name: "Communication",
82 |       description: null,
83 |       avg_attainment: 0,
84 |       mapped_ilo_count: 0,
85 |     },
86 |   ],
87 | })),
88 | useGraduateAttributeMappings: vi.fn((attributeId?: string) => {
89 |   if (attributeId === "ga-1") {
90 |     return {
91 |       data: [
92 |         {
93 |           id: "m-1",
94 |           graduate_attribute_id: "ga-1",
95 |           outcome_id: "ilo-1",
96 |           weight: 60,
97 |         },
98 |         {
99 |           id: "m-2",
100 |           graduate_attribute_id: "ga-1",
101 |           outcome_id: "ilo-2",
102 |           weight: 40,
103 |         },
104 |       ],
105 |       isLoading: false,
106 |     };
107 |   }
108 |   if (attributeId === "ga-loading") {
109 |     return { data: [], isLoading: true };
110 |   }
111 |   return { data: [], isLoading: false };
112 | })),
113 | useCreateGraduateAttribute: vi.fn(() => ({
114 |   mutate: vi.fn(),
115 |   isPending: false,
116 | })),
117 | useDeleteGraduateAttribute: vi.fn(() => ({
118 |   mutate: vi.fn(),
119 |   isPending: false,
120 | })),
121 | })),
122 |
123 | import GraduateAttributeManager from "@pages/admin/graduate-attributes/GraduateAttributeManager";
124 |
125 | const createWrapper = () => {
126 |   const qc = new QueryClient({ defaultOptions: { queries: { retry: false } } });
127 |   return ({ children }: { children: React.ReactNode }) => (
128 |     <QueryClientProvider client={qc}>
129 |       <MemoryRouter>{children}</MemoryRouter>
130 |     </QueryClientProvider>
131 |   );
132 | };
133 |
134 | describe("GraduateAttributeManager", () => {
135 |   it("renders the page title and each attribute", () => {
136 |     render(<GraduateAttributeManager />, { wrapper: createWrapper() });
137 |
138 |     expect(screen.getByText("Graduate Attributes")).toBeInTheDocument();
139 |     expect(screen.getByText("Critical Thinking")).toBeInTheDocument();
140 |     expect(screen.getByText("Communication")).toBeInTheDocument();
141 |   });
142 |
143 |   // % % % Req 6.1, 6.2, 6.6 – mapped outcomes + computed attainment % % % % % % % % %
144 |
145 |   it("renders mapped outcomes for an attribute that has mappings", () => {
146 |     // Validates Requirement 6.1 – mapped outcomes from useGraduateAttributeMappings
147 |     // are surfaced (resolved to readable outcome titles, not raw UUIDs).
148 |     render(<GraduateAttributeManager />, { wrapper: createWrapper() });
149 |
150 |     expect(screen.getByText("Mapped outcomes")).toBeInTheDocument();
151 |     expect(screen.getByText("ILO-1: Analyze")).toBeInTheDocument();

```

```

152 |     expect(screen.getByText("ILO-2: Evaluate")).toBeInTheDocument();
153 |     // Mapping weights are shown alongside each outcome
154 |     expect(screen.getByText(". 60%")).toBeInTheDocument();
155 |     expect(screen.getByText(". 40%")).toBeInTheDocument();
156 | });
157 |
158 | it("renders computed attainment with the platform color coding", () => {
159 |     // Validates Requirements 6.2 & 6.6 – attainment from useGraduateAttributeAttainment
160 |     // is displayed as a percentage + level, color-coded via getAttainmentColor.
161 |     render(<GraduateAttributeManager />, { wrapper: createWrapper() });
162 |
163 |     const percent = screen.getByText("88%");
164 |     expect(percent).toBeInTheDocument();
165 |     // 88% "e 85 !" "Excellent" classification
166 |     expect(screen.getByText("Excellent")).toBeInTheDocument();
167 |     // getAttainmentColor is applied as an inline background color
168 |     expect(percent.style.backgroundColor).not.toBe("");
169 | });
170 |
171 | // % % % Req 6.3 – inline empty state when zero mappings % % % % % % % % % % % % % % % %
172 |
173 | it("renders an inline empty state for an attribute with zero mappings", () => {
174 |     // Validates Requirement 6.3 – a GA with no mappings shows a user-friendly
175 |     // empty state rather than a bare zero.
176 |     render(<GraduateAttributeManager />, { wrapper: createWrapper() });
177 |
178 |     expect(screen.getByText("No outcomes mapped yet")).toBeInTheDocument();
179 |     expect(
180 |         screen.getByText(
181 |             "Map this attribute to institution outcomes to track attainment."
182 |         )
183 |     ).toBeInTheDocument();
184 | });
185 |
186 | // % % % Loading state – shimmer placeholder while mappings resolve % % % % % % % % %
187 |
188 | it("renders a shimmer placeholder while an attribute's mappings are loading", () => {
189 |     const { container } = render(<GraduateAttributeManager />, {
190 |         wrapper: createWrapper(),
191 |     });
192 |
193 |     expect(container.querySelector(".animate-shimmer")).not.toBeNull();
194 | });
195 | });
196 |

```

FILE: [src/__tests__/unit/habitAnalyticsPage.test.tsx](#)

TYPE: .tsx | LINES: 1-220 | SHA256: 6fc9be77acdc8434f5f064a16655aabd2b5301887e44a27a10a76af127d5fb5

```

1 | import { describe, it, expect, vi, beforeEach } from "vitest";
2 | import type { ReactNode } from "react";
3 | import { render, screen, fireEvent } from "@testing-library/react";
4 | import { MemoryRouter } from "react-router-dom";
5 |
6 | // -----
7 | // Mocks
8 | // -----
9 |
10 | const mockExportCSV = vi.fn();
11 |
12 | vi.mock("@/hooks/useAuth", () => ({
13 |     useAuth: () => ({ user: { id: "student-1", role: "student" } }),
14 | }));
15 |
16 | vi.mock("@/hooks/useSemesterRange", () => ({
17 |     useSemesterRange: () => ({
18 |         data: { start: "2025-01-01", end: "2025-06-30" },
19 |         isLoading: false,
20 |     }),
21 | }));
22 |
23 | const mockHeatmapData = [
24 |     {
25 |         date: "2025-01-01",
26 |         academicCount: 2,
27 |         wellnessCount: 1,
28 |         totalCount: 3,
29 |         habits: [],
30 |     },
31 |     {
32 |         date: "2025-01-02",
33 |         academicCount: 4,
34 |         wellnessCount: 0,
35 |         totalCount: 4,
36 |         habits: [],
37 |     },
38 |     {
39 |         date: "2025-01-03",
40 |         academicCount: 0,
41 |         wellnessCount: 0,
42 |         totalCount: 0,
43 |         habits: [],
44 |     },
45 | ];
46 |
47 | vi.mock("@/hooks/useHeatmapData", () => ({
48 |     useHeatmapData: () => ({
49 |         data: mockHeatmapData,
50 |         isLoading: false,
51 |     }),
52 |     useHeatmapSummary: () => ({
53 |         data: { currentStreak: 5, longestStreak: 10, totalActiveDays: 20 },
54 |         isLoading: false,
55 |     }),

```

```

56 | }));
57 |
58 | vi.mock("@/hooks/useWellnessPreferences", () => ({
59 |   useWellnessPreferences: () => ({
60 |     data: { enabledHabits: ["meditation"], parentVisibility: false },
61 |   }),
62 |   useUpdateWellnessPreferences: () => ({ mutate: vi.fn() }),
63 | }));
64 |
65 | let mockCorrelationResult = {
66 |   insights: [
67 |     {
68 |       id: "ins-1",
69 |       habitType: "meditation" as const,
70 |       academicMetric: "submissions",
71 |       description: "You tend to submit more on days when you meditate",
72 |       strength: 0.72,
73 |     },
74 |   ],
75 |   insufficientData: false,
76 | };
77 |
78 | vi.mock("@/hooks/useHabitCorrelations", () => ({
79 |   useHabitCorrelations: () => ({
80 |     data: mockCorrelationResult,
81 |     isLoading: false,
82 |   }),
83 | }));
84 |
85 | vi.mock("@/hooks/useHabitExport", () => ({
86 |   useHabitExport: () => ({ exportCSV: mockExportCSV }),
87 | }));
88 |
89 | vi.mock("@/hooks/useStudentHabitLevel", () => ({
90 |   useStudentHabitLevel: () => ({
91 |     data: {
92 |       currentLevel: 4,
93 |       levelHistory: [{ date: "2025-01-01", level: 4 }],
94 |       maxHabitsPerDay: 4,
95 |     },
96 |     isLoading: false,
97 |   }),
98 | }));
99 |
100 | // Mock Recharts to avoid SVG rendering issues in tests
101 | vi.mock("recharts", () => ({
102 |   BarChart: ({ children }: { children: ReactNode }) => (
103 |     <div data-testid="bar-chart">{children}</div>
104 |   ),
105 |   Bar: () => <div />,
106 |   LineChart: ({ children }: { children: ReactNode }) => (
107 |     <div data-testid="line-chart">{children}</div>
108 |   ),
109 |   Line: () => <div />,
110 |   XAxis: () => <div />,
111 |   YAxis: () => <div />,
112 |   Tooltip: () => <div />,
113 |   ResponsiveContainer: ({ children }: { children: ReactNode }) => (
114 |     <div>{children}</div>
115 |   ),
116 |   Cell: () => <div />,
117 |   ReferenceDot: () => <div />,
118 | }));
119 |
120 | // -----
121 | // Imports (after mocks)
122 | // -----
123 |
124 | import HabitAnalyticsPage from "@/pages/student/habits/HabitAnalyticsPage";
125 |
126 | const renderPage = () =>
127 |   render(
128 |     <MemoryRouter>
129 |       <HabitAnalyticsPage />
130 |     </MemoryRouter>
131 |   );
132 |
133 | // -----
134 | // Tests
135 | // -----
136 |
137 | describe("HabitAnalyticsPage", () => {
138 |   beforeEach(() => {
139 |     mockExportCSV.mockClear();
140 |     mockCorrelationResult = {
141 |       insights: [
142 |         {
143 |           id: "ins-1",
144 |           habitType: "meditation" as const,
145 |           academicMetric: "submissions",
146 |           description: "You tend to submit more on days when you meditate",
147 |           strength: 0.72,
148 |         },
149 |       ],
150 |       insufficientData: false,
151 |     };
152 |   });
153 |
154 |   it("renders the page title", () => {
155 |     renderPage();
156 |     expect(screen.getByText("Habit Analytics")).toBeInTheDocument();
157 |   });
158 |
159 |   it("renders the back link to /student/habits", () => {

```



```

160 renderPage();
161 const backLink = screen.getByText("Back").closest("a");
162 expect(backLink).toHaveAttribute("href", "/student/habits");
163 });
164
165 it("renders the export report button", () => {
166   renderPage();
167   expect(screen.getByTestId("export-report-btn")).toBeInTheDocument();
168 });
169
170 it("calls exportCSV when export button is clicked", () => {
171   renderPage();
172   fireEvent.click(screen.getByTestId("export-report-btn"));
173   expect(mockExportCSV).toHaveBeenCalledTimes(1);
174 });
175
176 it("renders the consistency score ring", () => {
177   renderPage();
178   expect(screen.getByTestId("consistency-score-ring")).toBeInTheDocument();
179 });
180
181 it("renders the completion chart section", () => {
182   renderPage();
183   expect(screen.getByTestId("habit-completion-chart")).toBeInTheDocument();
184 });
185
186 it("renders the best day chart section", () => {
187   renderPage();
188   expect(screen.getByTestId("best-day-chart")).toBeInTheDocument();
189 });
190
191 it("renders correlation insights when data is available", () => {
192   renderPage();
193   expect(screen.getByTestId("correlation-insights-list")).toBeInTheDocument();
194   expect(screen.getByTestId("correlation-insight-ins-1")).toBeInTheDocument();
195 });
196
197 it("renders insufficient data message when correlations have insufficient data", () => {
198   mockCorrelationResult = { insights: [], insufficientData: true };
199   renderPage();
200   expect(screen.getByTestId("insufficient-data-message")).toBeInTheDocument();
201   expect(
202     screen.getByText("Keep tracking – insights appear after 2 weeks of data")
203     ).toBeInTheDocument();
204 });
205
206 it("is wrapped in ErrorBoundary", () => {
207   // The page renders without crashing – ErrorBoundary is present
208   renderPage();
209   expect(screen.getByText("Habit Analytics")).toBeInTheDocument();
210 });
211
212 it("renders section cards with gradient headers", () => {
213   renderPage();
214   expect(screen.getByText("Consistency")).toBeInTheDocument();
215   expect(screen.getByText("Completion Rates")).toBeInTheDocument();
216   expect(screen.getByText("Best Day of Week")).toBeInTheDocument();
217   expect(screen.getByText("Insights")).toBeInTheDocument();
218 });
219 });
220

```

FILE: src/__tests__/unit/habitBadges.test.ts

TYPE: .ts | LINES: 1-273 | SHA256: 818d898398077185764547cf64434089e690721e80d64f02686e0ac26eac39a6

```

1 import { describe, it, expect } from "vitest";
2 import { getBadgeById, getBadgesByCategory } from "@lib/badgeDefinitions";
3
4 // % % % Edge Function constants (mirrored for testing) % % % % % % % % % % % % % % % % % % % % % % %
5
6 const VALID_TRIGGERS = [
7   "xp_award",
8   "submission",
9   "streak_update",
10  "grade",
11  "journal",
12  "habit_log",
13 ] as const;
14
15 const BADGE_XP: Record<string, number> = {
16   streak_7: 50,
17   streak_14: 75,
18   streak_30: 100,
19   streak_60: 150,
20   streak_100: 250,
21   first_submission: 25,
22   perfect_score: 75,
23   all_clos_met: 100,
24   journal_10: 50,
25   perfect_week: 100,
26   speed_demon: 75,
27   night_owl: 75,
28   perfectionist: 100,
29   habit_master: 100,
30   wellness_warrior: 75,
31   full_spectrum: 150,
32 };
33
34 // % % % Pure logic helpers (mirrored from Edge Function for testability) % % % % % % % % % %
35
36 /**
37  * Counts active days from academic + wellness date arrays.
38  * Returns the number of distinct dates with at least one habit.
39  */

```

[illegible]

```

144 |     expect(BADGE_XP["habit_master"]).toBe(100);
145 |   });
146 | });
147 |
148 | describe("Wellness Warrior Badge – 14 consecutive wellness days", () => {
149 |   it("awards badge with exactly 14 consecutive days", () => {
150 |     const dates = generateConsecutiveDates("2025-02-01", 14);
151 |     expect(longestConsecutiveRun(dates)).toBe(14);
152 |   });
153 |
154 |   it("awards badge with more than 14 consecutive days", () => {
155 |     const dates = generateConsecutiveDates("2025-02-01", 20);
156 |     expect(longestConsecutiveRun(dates)).toBeGreaterThanOrEqual(14);
157 |   });
158 |
159 |   it("does not award badge with 13 consecutive days", () => {
160 |     const dates = generateConsecutiveDates("2025-02-01", 13);
161 |     expect(longestConsecutiveRun(dates)).toBeLessThan(14);
162 |   });
163 |
164 |   it("handles gaps correctly – finds longest run", () => {
165 |     const run1 = generateConsecutiveDates("2025-01-01", 10);
166 |     const run2 = generateConsecutiveDates("2025-01-15", 14);
167 |     const allDates = [...run1, ...run2].sort();
168 |     expect(longestConsecutiveRun(allDates)).toBe(14);
169 |   });
170 |
171 |   it("handles duplicate dates in input", () => {
172 |     const dates = generateConsecutiveDates("2025-02-01", 14);
173 |     const withDups = [...dates, ...dates]; // each date appears twice
174 |     expect(longestConsecutiveRun(withDups)).toBe(14);
175 |   });
176 |
177 |   it("returns 0 for empty input", () => {
178 |     expect(longestConsecutiveRun([])).toBe(0);
179 |   });
180 |
181 |   it("has correct XP reward of 75", () => {
182 |     expect(BADGE_XP["wellness_warrior"]).toBe(75);
183 |   });
184 | });
185 |
186 | describe("Full Spectrum Badge – 7 days with all academic + wellness", () => {
187 |   it("awards badge with exactly 7 full spectrum days", () => {
188 |     const perfectDates = generateConsecutiveDates("2025-03-01", 7);
189 |     const wellnessDates = generateConsecutiveDates("2025-03-01", 7);
190 |     expect(countFullSpectrumDays(perfectDates, wellnessDates)).toBe(7);
191 |   });
192 |
193 |   it("awards badge with more than 7 full spectrum days", () => {
194 |     const perfectDates = generateConsecutiveDates("2025-03-01", 10);
195 |     const wellnessDates = generateConsecutiveDates("2025-03-01", 10);
196 |     expect(
197 |       countFullSpectrumDays(perfectDates, wellnessDates)
198 |     ).toBeGreaterThanOrEqual(7);
199 |   });
200 |
201 |   it("does not award badge with fewer than 7 full spectrum days", () => {
202 |     const perfectDates = generateConsecutiveDates("2025-03-01", 6);
203 |     const wellnessDates = generateConsecutiveDates("2025-03-01", 6);
204 |     expect(countFullSpectrumDays(perfectDates, wellnessDates)).toBeLessThan(7);
205 |   });
206 |
207 |   it("only counts days present in both sets", () => {
208 |     const perfectDates = generateConsecutiveDates("2025-03-01", 10);
209 |     const wellnessDates = generateConsecutiveDates("2025-03-05", 10);
210 |     // Overlap: Mar 5-10 = 6 days
211 |     expect(countFullSpectrumDays(perfectDates, wellnessDates)).toBe(6);
212 |   });
213 |
214 |   it("returns 0 when no wellness dates overlap", () => {
215 |     const perfectDates = generateConsecutiveDates("2025-03-01", 10);
216 |     const wellnessDates = generateConsecutiveDates("2025-04-01", 10);
217 |     expect(countFullSpectrumDays(perfectDates, wellnessDates)).toBe(0);
218 |   });
219 |
220 |   it("has correct XP reward of 150", () => {
221 |     expect(BADGE_XP["full_spectrum"]).toBe(150);
222 |   });
223 | });
224 |
225 | describe("Badge Definitions – habit badges", () => {
226 |   it("includes habit_master badge definition", () => {
227 |     const badge = getBadgeById("habit_master");
228 |     expect(badge).toBeDefined();
229 |     expect(badge!.name).toBe("Habit Master");
230 |     expect(badge!.icon).toBe("0<3Æ");
231 |     expect(badge!.category).toBe("habit");
232 |     expect(badge!.xpReward).toBe(100);
233 |     expect(badge!.isMystery).toBe(false);
234 |   });
235 |
236 |   it("includes wellness_warrior badge definition", () => {
237 |     const badge = getBadgeById("wellness_warrior");
238 |     expect(badge).toBeDefined();
239 |     expect(badge!.name).toBe("Wellness Warrior");
240 |     expect(badge!.icon).toBe("0>Y0");
241 |     expect(badge!.category).toBe("habit");
242 |     expect(badge!.xpReward).toBe(75);
243 |     expect(badge!.isMystery).toBe(false);
244 |   });
245 |
246 |   it("includes full_spectrum badge definition", () => {
247 |     const badge = getBadgeById("full_spectrum");

```

```

248 |     expect(badge).toBeDefined();
249 |     expect(badge!.name).toBe("Full Spectrum");
250 |     expect(badge!.icon).toBe("0x3 ");
251 |     expect(badge!.category).toBe("habit");
252 |     expect(badge!.xpReward).toBe(150);
253 |     expect(badge!.isMystery).toBe(false);
254 | });
255 |
256 | it("all three habit badges are in the habit category", () => {
257 |     const habitBadges = getBadgesByCategory("habit");
258 |     expect(habitBadges.length).toBe(3);
259 |     const ids = habitBadges.map((b) => b.id);
260 |     expect(ids).toContain("habit_master");
261 |     expect(ids).toContain("wellness_warrior");
262 |     expect(ids).toContain("full_spectrum");
263 | });
264 |
265 | it("habit badges have meaningful descriptions", () => {
266 |     const habitBadges = getBadgesByCategory("habit");
267 |     for (const badge of habitBadges) {
268 |         expect(badge.description.length).toBeGreaterThan(10);
269 |         expect(badge.condition.length).toBeGreaterThan(5);
270 |     }
271 | });
272 | });
273 |

```

FILE: src/_tests_/unit/habitCorrelations.test.ts

TYPE: .ts | LINES: 1-473 | SHA256: 10c2bf29722981c19da7a7917c44020a2eae1dec7cf839655e30043dc4124e37

```

1 | import { describe, it, expect } from "vitest";
2 |
3 | // % % % Pure correlation logic (mirrored from Edge Function for testability) % % %
4 |
5 | const FORBIDDEN_CAUSAL_WORDS = [
6 |     "because",
7 |     "causes",
8 |     "caused",
9 |     "due to",
10 |     "results in",
11 |     "leads to",
12 | ];
13 |
14 | function containsCausalLanguage(text: string): boolean {
15 |     const lower = text.toLowerCase();
16 |     return FORBIDDEN_CAUSAL_WORDS.some((word) => lower.includes(word));
17 | }
18 |
19 | function computeCoOccurrenceRate(
20 |     habitDates: Set<string>,
21 |     eventDates: Set<string>,
22 |     allDates: Set<string>
23 | ): number {
24 |     if (habitDates.size === 0 || allDates.size === 0) return 0;
25 |
26 |     const nonHabitDates = new Set(
27 |         [...allDates].filter((d) => !habitDates.has(d))
28 |     );
29 |
30 |     let habitEventCount = 0;
31 |     for (const d of habitDates) {
32 |         if (eventDates.has(d)) habitEventCount++;
33 |     }
34 |     const habitRate = habitDates.size > 0 ? habitEventCount / habitDates.size : 0;
35 |
36 |     let nonHabitEventCount = 0;
37 |     for (const d of nonHabitDates) {
38 |         if (eventDates.has(d)) nonHabitEventCount++;
39 |     }
40 |     const nonHabitRate =
41 |         nonHabitDates.size > 0 ? nonHabitEventCount / nonHabitDates.size : 0;
42 |
43 |     if (habitRate <= nonHabitRate) return 0;
44 |     return Math.min(habitRate - nonHabitRate, 1);
45 | }
46 |
47 | const HABIT_LABELS: Record<string, string> = {
48 |     meditation: "meditate",
49 |     hydration: "stay hydrated",
50 |     exercise: "exercise",
51 |     sleep: "get enough sleep",
52 |     login: "log in",
53 |     submit: "submit work",
54 |     journal: "journal",
55 |     read: "read content",
56 | };
57 |
58 | const INSIGHT_TEMPLATES: Record<string, (habitLabel: string) => string> = {
59 |     submission_rate: (habit) =>
60 |         `On days when you ${habit}, you tend to submit more assignments`,
61 |     submission_timeliness: (habit) =>
62 |         `On days when you ${habit}, you're more likely to submit assignments on time`,
63 |     activity_level: (habit) =>
64 |         `On days when you ${habit}, you tend to be more active on the platform`,
65 | };
66 |
67 | function generateInsightDescription(
68 |     habitType: string,
69 |     academicMetric: string
70 | ): string {
71 |     const habitLabel = HABIT_LABELS[habitType] ?? habitType;
72 |     const template = INSIGHT_TEMPLATES[academicMetric];
73 |     if (template) return template(habitLabel);
74 |     return `On days when you ${habitLabel}, your ${academicMetric.replace(

```

[illegible]

```

179 |     expect(result).toBeCloseTo(0.6, 1);
180 | });
181 |
182 | it("returns 0 when event rate is equal on habit and non-habit days", () => {
183 |   const allDates = new Set(generateDates("2025-01-01", 4));
184 |   const habitDates = new Set(["2025-01-01", "2025-01-02"]);
185 |   const eventDates = new Set(["2025-01-01", "2025-01-03"]); // 50% on both
186 |   const result = computeCoOccurrenceRate(habitDates, eventDates, allDates);
187 |   expect(result).toBe(0);
188 | });
189 |
190 | it("returns value capped at 1", () => {
191 |   // All habit days have events, no non-habit days have events
192 |   const allDates = new Set(generateDates("2025-01-01", 10));
193 |   const habitDates = new Set(generateDates("2025-01-01", 5));
194 |   const eventDates = new Set(generateDates("2025-01-01", 5));
195 |   const result = computeCoOccurrenceRate(habitDates, eventDates, allDates);
196 |   expect(result).toBeLessThanOrEqual(1);
197 |   expect(result).toBeGreaterThan(0);
198 | });
199 | });
200 |
201 | describe("Insight Generation – Non-Causal Language", () => {
202 |   it('generates description with "On days when" for submission_rate', () => {
203 |     const desc = generateInsightDescription("meditation", "submission_rate");
204 |     expect(desc).toContain("On days when");
205 |     expect(desc).toContain("tend to");
206 |   });
207 |
208 |   it('generates description with "more likely" for submission_timeliness', () => {
209 |     const desc = generateInsightDescription(
210 |       "exercise",
211 |       "submission_timeliness"
212 |     );
213 |     expect(desc).toContain("you're more likely to");
214 |   });
215 |
216 |   it("uses fallback template for unknown metrics", () => {
217 |     const desc = generateInsightDescription("sleep", "unknown_metric");
218 |     expect(desc).toContain("On days when");
219 |     expect(desc).toContain("tends to be higher");
220 |   });
221 |
222 |   it("never contains causal language in any template", () => {
223 |     const habitTypes = [
224 |       "meditation",
225 |       "hydration",
226 |       "exercise",
227 |       "sleep",
228 |       "login",
229 |       "submit",
230 |       "journal",
231 |       "read",
232 |     ];
233 |     const metrics = [
234 |       "submission_rate",
235 |       "submission_timeliness",
236 |       "activity_level",
237 |       "some_other",
238 |     ];
239 |
240 |     for (const habit of habitTypes) {
241 |       for (const metric of metrics) {
242 |         const desc = generateInsightDescription(habit, metric);
243 |         expect(containsCausalLanguage(desc)).toBe(false);
244 |       }
245 |     }
246 |   });
247 |
248 |   it("uses correct habit labels", () => {
249 |     expect(
250 |       generateInsightDescription("meditation", "submission_rate")
251 |     ).toContain("meditate");
252 |     expect(
253 |       generateInsightDescription("hydration", "submission_rate")
254 |     ).toContain("stay hydrated");
255 |     expect(generateInsightDescription("exercise", "submission_rate")).toContain(
256 |       "exercise"
257 |     );
258 |     expect(generateInsightDescription("sleep", "submission_rate")).toContain(
259 |       "get enough sleep"
260 |     );
261 |   });
262 | });
263 |
264 | describe("Causal Language Detection", () => {
265 |   it('detects "because"', () => {
266 |     expect(containsCausalLanguage("You submit more because you meditate")).toBe(
267 |       true
268 |     );
269 |   });
270 |
271 |   it('detects "causes"', () => {
272 |     expect(containsCausalLanguage("Meditation causes better grades")).toBe(
273 |       true
274 |     );
275 |   });
276 |
277 |   it('detects "due to"', () => {
278 |     expect(containsCausalLanguage("Better scores due to exercise")).toBe(true);
279 |   });
280 |
281 |   it('detects "results in"', () => {
282 |     expect(containsCausalLanguage("Exercise results in better focus")).toBe(

```

```

283 |         true
284 |     );
285 | });
286 |
287 | it('detects "leads to"', () => {
288 |     expect(containsCausalLanguage("Sleep leads to better performance")).toBe(
289 |         true
290 |     );
291 | });
292 |
293 | it("allows non-causal language", () => {
294 |     expect(
295 |         containsCausalLanguage(
296 |             "On days when you meditate, you tend to submit more"
297 |         )
298 |     ).toBe(false);
299 |     expect(containsCausalLanguage("You're more likely to submit on time")).toBe(
300 |         false
301 |     );
302 |     expect(
303 |         containsCausalLanguage("Your activity level tends to be higher")
304 |     ).toBe(false);
305 | });
306 | });
307 |
308 | describe("14-Day Minimum Data Threshold", () => {
309 |     it("returns false when fewer than 14 days", () => {
310 |         const dates = new Set(generateDates("2025-01-01", 13));
311 |         expect(hasMinimumData(dates)).toBe(false);
312 |     });
313 |
314 |     it("returns true when exactly 14 days", () => {
315 |         const dates = new Set(generateDates("2025-01-01", 14));
316 |         expect(hasMinimumData(dates)).toBe(true);
317 |     });
318 |
319 |     it("returns true when more than 14 days", () => {
320 |         const dates = new Set(generateDates("2025-01-01", 30));
321 |         expect(hasMinimumData(dates)).toBe(true);
322 |     });
323 |
324 |     it("returns false for empty set", () => {
325 |         expect(hasMinimumData(new Set())).toBe(false);
326 |     });
327 |
328 |     it("supports custom minimum threshold", () => {
329 |         const dates = new Set(generateDates("2025-01-01", 20));
330 |         expect(hasMinimumData(dates, 30)).toBe(false);
331 |         expect(hasMinimumData(dates, 20)).toBe(true);
332 |     });
333 | });
334 |
335 | describe("Correlation Insights Computation", () => {
336 |     it("returns empty array when no correlations exceed threshold", () => {
337 |         const habitDatesByType = new Map<string, Set<string>>();
338 |         habitDatesByType.set("meditation", new Set(["2025-01-01"]));
339 |         const submissionDates = new Set(["2025-01-02"]);
340 |         const allDates = new Set(["2025-01-01", "2025-01-02"]);
341 |
342 |         const insights = computeCorrelationInsights(
343 |             habitDatesByType,
344 |             submissionDates,
345 |             allDates
346 |         );
347 |         expect(insights).toHaveLength(0);
348 |     });
349 |
350 |     it("returns insights sorted by strength descending", () => {
351 |         const allDates = new Set(generateDates("2025-01-01", 20));
352 |         const meditationDates = new Set(generateDates("2025-01-01", 10));
353 |         const exerciseDates = new Set(generateDates("2025-01-01", 5));
354 |
355 |         // Submissions co-occur more with meditation than exercise
356 |         const submissionDates = new Set(generateDates("2025-01-01", 10));
357 |
358 |         const habitDatesByType = new Map<string, Set<string>>();
359 |         habitDatesByType.set("meditation", meditationDates);
360 |         habitDatesByType.set("exercise", exerciseDates);
361 |
362 |         const insights = computeCorrelationInsights(
363 |             habitDatesByType,
364 |             submissionDates,
365 |             allDates
366 |         );
367 |
368 |         if (insights.length > 1) {
369 |             for (let i = 1; i < insights.length; i++) {
370 |                 expect(insights[i]!.strength).toBeLessThanOrEqual(
371 |                     insights[i - 1]!.strength
372 |                 );
373 |             }
374 |         }
375 |     });
376 |
377 |     it("returns at most 3 insights", () => {
378 |         const allDates = new Set(generateDates("2025-01-01", 30));
379 |         const submissionDates = new Set(generateDates("2025-01-01", 15));
380 |
381 |         const habitDatesByType = new Map<string, Set<string>>();
382 |         // Create 5 habit types that all correlate with submissions
383 |         const types = ["meditation", "hydration", "exercise", "sleep", "login"];
384 |         for (const type of types) {
385 |             habitDatesByType.set(type, new Set(generateDates("2025-01-01", 15)));
386 |         }

```

```

387 |
388 |     const insights = computeCorrelationInsights(
389 |         habitDatesByType,
390 |         submissionDates,
391 |         allDates
392 |     );
393 |     expect(insights.length).toBeLessThanOrEqual(3);
394 | });
395 |
396 | it("each insight has required fields", () => {
397 |     const allDates = new Set(generateDates("2025-01-01", 20));
398 |     const habitDatesByType = new Map<string, Set<string>>();
399 |     habitDatesByType.set(
400 |         "meditation",
401 |         new Set(generateDates("2025-01-01", 10))
402 |     );
403 |     const submissionDates = new Set(generateDates("2025-01-01", 10));
404 |
405 |     const insights = computeCorrelationInsights(
406 |         habitDatesByType,
407 |         submissionDates,
408 |         allDates
409 |     );
410 |
411 |     for (const insight of insights) {
412 |         expect(insight.id).toBeDefined();
413 |         expect(typeof insight.id).toBe("string");
414 |         expect(insight.habitType).toBeDefined();
415 |         expect(insight.academicMetric).toBeDefined();
416 |         expect(insight.description).toBeDefined();
417 |         expect(typeof insight.strength).toBe("number");
418 |         expect(insight.strength).toBeGreaterThanOrEqual(0);
419 |         expect(insight.strength).toBeLessThanOrEqual(1);
420 |     }
421 | });
422 |
423 | it("insight descriptions use non-causal language", () => {
424 |     const allDates = new Set(generateDates("2025-01-01", 20));
425 |     const habitDatesByType = new Map<string, Set<string>>();
426 |     habitDatesByType.set(
427 |         "meditation",
428 |         new Set(generateDates("2025-01-01", 10))
429 |     );
430 |     const submissionDates = new Set(generateDates("2025-01-01", 10));
431 |
432 |     const insights = computeCorrelationInsights(
433 |         habitDatesByType,
434 |         submissionDates,
435 |         allDates
436 |     );
437 |
438 |     for (const insight of insights) {
439 |         expect(containsCausalLanguage(insight.description)).toBe(false);
440 |     }
441 | });
442 |
443 | it("handles empty habit data", () => {
444 |     const habitDatesByType = new Map<string, Set<string>>();
445 |     const submissionDates = new Set<string>();
446 |     const allDates = new Set<string>();
447 |
448 |     const insights = computeCorrelationInsights(
449 |         habitDatesByType,
450 |         submissionDates,
451 |         allDates
452 |     );
453 |     expect(insights).toHaveLength(0);
454 | });
455 |
456 | it("handles no submissions", () => {
457 |     const allDates = new Set(generateDates("2025-01-01", 20));
458 |     const habitDatesByType = new Map<string, Set<string>>();
459 |     habitDatesByType.set(
460 |         "meditation",
461 |         new Set(generateDates("2025-01-01", 10))
462 |     );
463 |     const submissionDates = new Set<string>();
464 |
465 |     const insights = computeCorrelationInsights(
466 |         habitDatesByType,
467 |         submissionDates,
468 |         allDates
469 |     );
470 |     expect(insights).toHaveLength(0);
471 | });
472 | });
473 |

```

FILE: [src/__tests__/unit/habitDifficulty.test.ts](#)

TYPE: .ts | LINES: 1-68 | SHA256: 2d7b4c939c29cd9183a007a24d41f47a7b287778775d2b0e3b934f04efb06529

```

1 | import { describe, it, expect } from "vitest";
2 | import {
3 |     getRequiredHabitsForLevel,
4 |     checkLevelPromotion,
5 |     getPerfectDayThreshold,
6 | } from "@lib/habitDifficulty";
7 |
8 | describe("habitDifficulty", () => {
9 |     describe("getRequiredHabitsForLevel", () => {
10 |         it("returns 1 for level 1", () => {
11 |             expect(getRequiredHabitsForLevel(1)).toBe(1);
12 |         });
13 |     });

```



```

14 |     it("returns 2 for level 2", () => {
15 |         expect(getRequiredHabitsForLevel(2)).toBe(2);
16 |     });
17 |
18 |     it("returns 6 for level 3", () => {
19 |         expect(getRequiredHabitsForLevel(3)).toBe(6);
20 |     });
21 |
22 |     it("returns 1 for level 0 or below", () => {
23 |         expect(getRequiredHabitsForLevel(0)).toBe(1);
24 |         expect(getRequiredHabitsForLevel(-1)).toBe(1);
25 |     });
26 |
27 |     it("returns 6 for level above 3", () => {
28 |         expect(getRequiredHabitsForLevel(4)).toBe(6);
29 |     });
30 | });
31 |
32 | describe("checkLevelPromotion", () => {
33 |     it("returns true when streak >= 7 and level < 3", () => {
34 |         expect(checkLevelPromotion(1, 7)).toBe(true);
35 |         expect(checkLevelPromotion(2, 7)).toBe(true);
36 |         expect(checkLevelPromotion(1, 10)).toBe(true);
37 |     });
38 |
39 |     it("returns false when level is 3 (max)", () => {
40 |         expect(checkLevelPromotion(3, 7)).toBe(false);
41 |         expect(checkLevelPromotion(3, 100)).toBe(false);
42 |     });
43 |
44 |     it("returns false when streak < 7", () => {
45 |         expect(checkLevelPromotion(1, 6)).toBe(false);
46 |         expect(checkLevelPromotion(2, 0)).toBe(false);
47 |     });
48 |
49 |     it("streak reset does not demote level", () => {
50 |         // Simulate: level 2, streak resets to 0 – level stays at 2
51 |         const level = 2;
52 |         const resetStreak = 0;
53 |         // checkLevelPromotion only promotes, never demotes
54 |         expect(checkLevelPromotion(level, resetStreak)).toBe(false);
55 |         // Level remains unchanged (no demotion function exists by design)
56 |         expect(level).toBe(2);
57 |     });
58 | });
59 |
60 | describe("getPerfectDayThreshold", () => {
61 |     it("returns same values as getRequiredHabitsForLevel", () => {
62 |         expect(getPerfectDayThreshold(1)).toBe(1);
63 |         expect(getPerfectDayThreshold(2)).toBe(2);
64 |         expect(getPerfectDayThreshold(3)).toBe(6);
65 |     });
66 | });
67 | });
68 |

```

FILE: src/__tests__/unit/habitDifficultyIndicator.test.tsx

TYPE: .tsx | LINES: 1-40 | SHA256: 8dda44c075e8765965eb3a8875478bbe14c0ba192265ec7d79c564879f0a2af5

```

1 | import { describe, it, expect } from "vitest";
2 | import { render, screen } from "@testing-library/react";
3 | import HabitDifficultyIndicator from "@components/shared/HabitDifficultyIndicator";
4 |
5 | describe("HabitDifficultyIndicator", () => {
6 |     it("renders level 1 as Seedling", () => {
7 |         render(<HabitDifficultyIndicator level={1} habitLevelStreak={0} />);
8 |         expect(screen.getByText("Level 1")).toBeInTheDocument();
9 |         expect(screen.getByText("Seedling")).toBeInTheDocument();
10 |     });
11 |
12 |     it("renders level 2 as Sprout", () => {
13 |         render(<HabitDifficultyIndicator level={2} habitLevelStreak={3} />);
14 |         expect(screen.getByText("Level 2")).toBeInTheDocument();
15 |         expect(screen.getByText("Sprout")).toBeInTheDocument();
16 |     });
17 |
18 |     it("renders level 3 as Tree with max level message", () => {
19 |         render(<HabitDifficultyIndicator level={3} habitLevelStreak={0} />);
20 |         expect(screen.getByText("Level 3")).toBeInTheDocument();
21 |         expect(screen.getByText("Tree")).toBeInTheDocument();
22 |         expect(screen.getByText("Max level reached")).toBeInTheDocument();
23 |     });
24 |
25 |     it("shows days remaining to next level", () => {
26 |         render(<HabitDifficultyIndicator level={1} habitLevelStreak={5} />);
27 |         expect(screen.getByText("2 days to Level 2")).toBeInTheDocument();
28 |     });
29 |
30 |     it("shows 7 days when streak is 0", () => {
31 |         render(<HabitDifficultyIndicator level={1} habitLevelStreak={0} />);
32 |         expect(screen.getByText("7 days to Level 2")).toBeInTheDocument();
33 |     });
34 |
35 |     it("shows singular 'day' when 1 day remaining", () => {
36 |         render(<HabitDifficultyIndicator level={2} habitLevelStreak={6} />);
37 |         expect(screen.getByText("1 day to Level 3")).toBeInTheDocument();
38 |     });
39 | });
40 |

```

FILE: src/___tests___/unit/habitHeatmapXp.test.tsx

TYPE: .tsx | LINES: 1-161 | SHA256: 95e2b65c4df95c1e9ec471543a5978d83a1bf60d2762e3bd24afbf706da41194

```
1 // =====
2 // HabitHeatmapPage – per-day XP wiring unit tests (Task 16.3)
3 // Verifies Task 16.2: the heatmap surfaces the day's *recorded* habit XP from
4 // useHeatmapXpByDate (via the tooltip and the mobile bottom sheet) instead of a
5 // hardcoded 0 (R7.1, R7.4).
6 // =====
7
8 import { describe, it, expect, vi, beforeEach } from "vitest";
9 import { render, screen, fireEvent } from "@testing-library/react";
10 import { MemoryRouter } from "react-router-dom";
11 import type { HeatmapDay, CompletedHabit } from "@types/habits";
12
13 // -----
14 // Stub ResizeObserver for happy-dom (HeatmapGrid observes its container)
15 // -----
16
17 class MockResizeObserver {
18   callback: ResizeObserverCallback;
19   constructor(cb: ResizeObserverCallback) {
20     this.callback = cb;
21   }
22   observe() {
23     this.callback(
24       [{ contentRect: { width: 900 } } as unknown as ResizeObserverEntry],
25       this as unknown as ResizeObserver
26     );
27   }
28   unobserve() {}
29   disconnect() {}
30 }
31 vi.stubGlobal("ResizeObserver", MockResizeObserver);
32
33 // -----
34 // Fixtures – a single tracked day with academic habits and recorded XP
35 // -----
36
37 const TRACKED_DATE = "2025-01-06"; // past date !' clickable, not future
38 const RECORDED_XP = 35;
39
40 const academicHabit = (type: CompletedHabit["type"]): CompletedHabit => ({
41   type,
42   category: "academic",
43   completedAt: `${TRACKED_DATE}T10:00:00Z`,
44 });
45
46 const mockHeatmapData: HeatmapDay[] = [
47   {
48     date: TRACKED_DATE,
49     academicCount: 2,
50     wellnessCount: 0,
51     totalCount: 2,
52     habits: [academicHabit("login"), academicHabit("submit")],
53   },
54 ];
55
56 // Per-day recorded XP map returned by useHeatmapXpByDate.
57 const mockXpByDate: Record<string, number> = { [TRACKED_DATE]: RECORDED_XP };
58
59 // -----
60 // Hook mocks (all data-access hooks used by HabitHeatmapContent)
61 // -----
62
63 vi.mock("@/hooks/useAuth", () => ({
64   useAuth: () => ({ user: { id: "student-1", role: "student" } }),
65 }));
66
67 vi.mock("@/hooks/useSemesterRange", () => ({
68   useSemesterRange: () => ({
69     data: { start: "2025-01-01", end: "2025-01-31" },
70     isLoading: false,
71   }),
72 }));
73
74 vi.mock("@/hooks/useHeatmapData", () => ({
75   useHeatmapData: () => ({ data: mockHeatmapData, isLoading: false }),
76   useHeatmapSummary: () => ({
77     data: { currentStreak: 1, longestStreak: 1, totalActiveDays: 1 },
78     isLoading: false,
79   }),
80   useHeatmapXpByDate: () => ({ data: mockXpByDate }),
81 }));
82
83 vi.mock("@/hooks/useWellnessPreferences", () => ({
84   useWellnessPreferences: () => ({
85     data: { enabledHabits: [], parentVisibility: false },
86   }),
87   useUpdateWellnessPreferences: () => ({ mutate: vi.fn() }),
88 }));
89
90 vi.mock("@/hooks/useWellnessHabits", () => ({
91   useWellnessHabitLogs: () => ({ data: [] }),
92   useLogWellnessHabit: () => ({ mutate: vi.fn() }),
93 }));
94
95 vi.mock("@/hooks/useWellnessTips", () => ({
96   useCurrentTip: () => ({ tip: null, isOnboarding: false }),
97   useDismissOnboardingTip: () => ({ mutate: vi.fn() }),
98 }));
99
100 vi.mock("@/hooks/useWellnessReminders", () => ({
101   useWellnessReminders: () => ({ data: [] }),
```

```

102 |     useUpdateWellnessReminder: () => ({ mutate: vi.fn() }),
103 |   });
104 |
105 |   vi.mock("@/hooks/useWellnessGoals", () => ({
106 |     useWellnessGoals: () => ({ data: [] }),
107 |     useDailyProgress: () => ({}),
108 |     useUpdateWellnessGoal: () => ({ mutate: vi.fn() }),
109 |   }));
110 |
111 |   vi.mock("nuqs", () => ({
112 |     parseAsString: { withDefault: (def: string) => def },
113 |     useQueryState: () => ["all", vi.fn()] as const,
114 |   }));
115 |
116 |   // -----
117 |   // Import after mocks
118 |   // -----
119 |
120 |   import HabitHeatmapPage from "@/pages/student/habits/HabitHeatmapPage";
121 |
122 |   const renderPage = () =>
123 |     render(
124 |       <MemoryRouter>
125 |         <HabitHeatmapPage />
126 |       </MemoryRouter>
127 |     );
128 |
129 |   // -----
130 |   // Tests
131 |   // -----
132 |
133 |   describe("HabitHeatmapPage per-day XP wiring", () => {
134 |     beforeEach(() => {
135 |       vi.clearAllMocks();
136 |     });
137 |
138 |     it("shows the day's recorded XP in the tooltip on hover (R7.1)", async () => {
139 |       renderPage();
140 |
141 |       const cell = screen.getByTestId(`heatmap-cell-${TRACKED_DATE}`);
142 |       fireEvent.mouseEnter(cell);
143 |
144 |       const xp = await screen.findByTestId("tooltip-xp");
145 |       expect(xp).toHaveTextContent(`${RECORDED_XP} XP earned`);
146 |       // Guards against the previous hardcoded-zero regression.
147 |       expect(xp).not.toHaveTextContent("0 XP earned");
148 |     });
149 |
150 |     it("shows the day's recorded XP in the bottom sheet on click (R7.1, R7.4)", async () => {
151 |       renderPage();
152 |
153 |       const cell = screen.getByTestId(`heatmap-cell-${TRACKED_DATE}`);
154 |       fireEvent.click(cell);
155 |
156 |       const xp = await screen.findByTestId("bottom-sheet-xp");
157 |       expect(xp).toHaveTextContent(`${RECORDED_XP} XP earned`);
158 |       expect(xp).not.toHaveTextContent("0 XP earned");
159 |     });
160 |   });
161 |

```

FILE: src/__tests__/unit/habitTracker.test.tsx

TYPE: .tsx | LINES: 1-107 | SHA256: 9b5fcb923951c2a4b7d6db76b0cec0490d81a1fa09ccd14bb4596cf438361636

```

1 | // =====
2 | // HabitTracker – canonical-table wiring unit test
3 | // Verifies the dashboard Daily Habits widget reads from the canonical
4 | // long-format `habit_logs` table (the single source of truth every write path
5 | // targets) and NOT the legacy wide-format `habit_tracking` table, which is no
6 | // longer written by any flow. This guards against the split-brain regression
7 | // where the dashboard showed stale seed data.
8 | // =====
9 |
10 | import { describe, it, expect, vi, beforeEach } from "vitest";
11 | import { render, screen } from "@testing-library/react";
12 | import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
13 | import { format } from "date-fns";
14 | import HabitTracker from "@/components/shared/HabitTracker";
15 |
16 | // -----
17 | // supabase mock – a chainable query builder that records which table was read
18 | // and resolves to a seeded set of habit_logs rows.
19 | // -----
20 |
21 | const queriedTables: string[] = [];
22 | let mockRows: Array<{
23 |   date: string;
24 |   habit_type: string;
25 |   completed_at: string | null;
26 |   created_at: string | null;
27 | }> = [];
28 |
29 | vi.mock("@/lib/supabase", () => {
30 |   const builder = () => {
31 |     const chain: Record<string, unknown> = {};
32 |     const methods = ["select", "eq", "gte", "lte", "order"];
33 |     for (const m of methods) {
34 |       chain[m] = vi.fn(() => chain);
35 |     }
36 |     // The query is awaited at the end of the chain (after .order()).
37 |     chain.then = (resolve: (v: unknown) => unknown) =>
38 |       resolve({ data: mockRows, error: null });
39 |     return chain;
40 |   };

```

```

41 |
42 |     return {
43 |       supabase: {
44 |         from: vi.fn((table: string) => {
45 |           queriedTables.push(table);
46 |           return builder();
47 |         }),
48 |       },
49 |     };
50 |   });
51 |
52 |   const renderWithClient = (ui: React.ReactElement) => {
53 |     const client = new QueryClient({
54 |       defaultOptions: { queries: { retry: false } },
55 |     });
56 |     return render(
57 |       <QueryClientProvider client={client}>{ui}</QueryClientProvider>
58 |     );
59 |   };
60 |
61 |   describe("HabitTracker", () => {
62 |     beforeEach(() => {
63 |       queriedTables.length = 0;
64 |       mockRows = [];
65 |     });
66 |
67 |     it("reads from the canonical habit_logs table, not legacy habit_tracking", async () => {
68 |       const today = format(new Date(), "yyyy-MM-dd");
69 |       mockRows = [
70 |         {
71 |           date: today,
72 |           habit_type: "login",
73 |           completed_at: `${today}T08:00:00Z`,
74 |           created_at: `${today}T08:00:00Z`,
75 |         },
76 |       ];
77 |
78 |       renderWithClient(<HabitTracker studentId="student-1" />);
79 |
80 |       // Wait for the query to resolve and the grid to render.
81 |       expect(await screen.findByText("Daily Habits")).toBeInTheDocument();
82 |
83 |       expect(queriedTables).toContain("habit_logs");
84 |       expect(queriedTables).not.toContain("habit_tracking");
85 |     });
86 |
87 |     it("marks a completed habit/day cell as done", async () => {
88 |       const today = format(new Date(), "yyyy-MM-dd");
89 |       mockRows = [
90 |         {
91 |           date: today,
92 |           habit_type: "login",
93 |           completed_at: `${today}T08:00:00Z`,
94 |           created_at: `${today}T08:00:00Z`,
95 |         },
96 |       ];
97 |
98 |       renderWithClient(<HabitTracker studentId="student-1" days={7} />);
99 |
100 |       await screen.findByText("Daily Habits");
101 |
102 |       // The login cell for today carries a "Done" title; missed cells say "Missed".
103 |       const doneCell = screen.getByTitle(`Login - ${today}: Done`);
104 |       expect(doneCell).toBeInTheDocument();
105 |     });
106 |   });
107 |

```

FILE: src/__tests__/unit/heatmapFilters.test.tsx

TYPE: .tsx | LINES: 1-176 | SHA256: 8185ccb11c52fddcaab95fb8573e615643069301c9c691782afc04b1be01bb97

```

1 | import { describe, it, expect, vi, beforeEach } from "vitest";
2 | import { render, screen } from "@testing-library/react";
3 |
4 | // -----
5 | // nuqs mock - must be before component import
6 | // -----
7 |
8 | let mockFilter = "all";
9 | const mockSetFilter = vi.fn((val: string) => {
10 |   mockFilter = val;
11 | });
12 |
13 | vi.mock("nuqs", () => ({
14 |   parseAsString: {
15 |     withDefault: (def: string) => def,
16 |   },
17 |   useQueryState: (_key: string, _defaultVal: string) => {
18 |     return [mockFilter, mockSetFilter] as const;
19 |   },
20 | }));
21 |
22 | // -----
23 | // Imports (after mocks)
24 | // -----
25 |
26 | import HeatmapFilters from "@components/shared/HeatmapFilters";
27 | import HeatmapSummaryStats from "@components/shared/HeatmapSummaryStats";
28 |
29 | // -----
30 | // HeatmapFilters Tests
31 | // -----
32 |
33 | describe("HeatmapFilters", () => {

```

```

34 | beforeEach(() => {
35 |     mockFilter = "all";
36 |     mockSetFilter.mockClear();
37 | });
38 |
39 | it("renders default academic filter options", () => {
40 |     render(<HeatmapFilters enabledWellnessHabits={} />);
41 |
42 |     expect(screen.getByTestId("filter-all")).toHaveTextContent("All Habits");
43 |     expect(screen.getByTestId("filter-login")).toHaveTextContent("Login");
44 |     expect(screen.getByTestId("filter-submit")).toHaveTextContent("Submit");
45 |     expect(screen.getByTestId("filter-journal")).toHaveTextContent("Journal");
46 |     expect(screen.getByTestId("filter-read")).toHaveTextContent("Read");
47 | });
48 |
49 | it("renders enabled wellness habits as additional filters", () => {
50 |     render(
51 |         <HeatmapFilters enabledWellnessHabits={['meditation', 'exercise']} />
52 |     );
53 |
54 |     expect(screen.getByTestId("filter-meditation")).toHaveTextContent(
55 |         "Meditation"
56 |     );
57 |     expect(screen.getByTestId("filter-exercise")).toHaveTextContent("Exercise");
58 |     // Not enabled
59 |     expect(screen.queryByTestId("filter-hydration")).not.toBeInTheDocument();
60 |     expect(screen.queryByTestId("filter-sleep")).not.toBeInTheDocument();
61 | });
62 |
63 | it("renders all 4 wellness habits when all enabled", () => {
64 |     render(
65 |         <HeatmapFilters
66 |             enabledWellnessHabits={['meditation', 'hydration', 'exercise', 'sleep']}
67 |         />
68 |     );
69 |
70 |     expect(screen.getByTestId("filter-meditation")).toBeInTheDocument();
71 |     expect(screen.getByTestId("filter-hydration")).toBeInTheDocument();
72 |     expect(screen.getByTestId("filter-exercise")).toBeInTheDocument();
73 |     expect(screen.getByTestId("filter-sleep")).toBeInTheDocument();
74 | });
75 |
76 | it("renders the active filter based on current query state", () => {
77 |     mockFilter = "login";
78 |     render(<HeatmapFilters enabledWellnessHabits={} />);
79 |
80 |     const loginTrigger = screen.getByTestId("filter-login");
81 |     expect(loginTrigger).toHaveAttribute("data-state", "active");
82 |
83 |     const allTrigger = screen.getByTestId("filter-all");
84 |     expect(allTrigger).toHaveAttribute("data-state", "inactive");
85 | });
86 |
87 | it("has 5 base filters with no wellness habits", () => {
88 |     render(<HeatmapFilters enabledWellnessHabits={} />);
89 |
90 |     const list = screen.getByTestId("heatmap-filter-list");
91 |     const triggers = list.querySelectorAll('[data-slot="tabs-trigger"]');
92 |     expect(triggers).toHaveLength(5);
93 | });
94 |
95 | it("has 7 filters with 2 wellness habits enabled", () => {
96 |     render(<HeatmapFilters enabledWellnessHabits={['meditation', 'sleep']} />);
97 |
98 |     const list = screen.getByTestId("heatmap-filter-list");
99 |     const triggers = list.querySelectorAll('[data-slot="tabs-trigger"]');
100 |     expect(triggers).toHaveLength(7);
101 | });
102 | });
103 |
104 | // -----
105 | // HeatmapSummaryStats Tests
106 | // -----
107 |
108 | describe("HeatmapSummaryStats", () => {
109 |     it("renders 3 KPI cards", () => {
110 |         render(
111 |             <HeatmapSummaryStats
112 |                 currentStreak={12}
113 |                 longestStreak={30}
114 |                 totalActiveDays={45}
115 |             />
116 |         );
117 |
118 |         expect(screen.getByTestId("kpi-current-streak")).toBeInTheDocument();
119 |         expect(screen.getByTestId("kpi-longest-streak")).toBeInTheDocument();
120 |         expect(screen.getByTestId("kpi-total-active-days")).toBeInTheDocument();
121 |     });
122 |
123 |     it("displays correct values", () => {
124 |         render(
125 |             <HeatmapSummaryStats
126 |                 currentStreak={7}
127 |                 longestStreak={21}
128 |                 totalActiveDays={60}
129 |             />
130 |         );
131 |
132 |         expect(screen.getByTestId("kpi-current-streak-value")).toHaveTextContent(
133 |             "7"
134 |         );
135 |         expect(screen.getByTestId("kpi-longest-streak-value")).toHaveTextContent(
136 |             "21"
137 |         );

```

```

138 |     expect(screen.getByTestId("kpi-total-active-days-value")).toHaveTextContent(
139 |       "60"
140 |     );
141 |   });
142 |
143 |   it("displays zero values correctly", () => {
144 |     render(
145 |       <HeatmapSummaryStats
146 |         currentStreak={0}
147 |         longestStreak={0}
148 |         totalActiveDays={0}
149 |       />
150 |     );
151 |
152 |     expect(screen.getByTestId("kpi-current-streak-value")).toHaveTextContent(
153 |       "0"
154 |     );
155 |     expect(screen.getByTestId("kpi-longest-streak-value")).toHaveTextContent(
156 |       "0"
157 |     );
158 |     expect(screen.getByTestId("kpi-total-active-days-value")).toHaveTextContent(
159 |       "0"
160 |     );
161 |   });
162 |
163 |   it("renders in a 3-column grid", () => {
164 |     render(
165 |       <HeatmapSummaryStats
166 |         currentStreak={5}
167 |         longestStreak={10}
168 |         totalActiveDays={20}
169 |       />
170 |     );
171 |
172 |     const container = screen.getByTestId("heatmap-summary-stats");
173 |     expect(container.className).toContain("grid-cols-3");
174 |   });
175 | });
176 |

```

FILE: [src/__tests__/unit/heatmapGrid.test.tsx](#)

TYPE: .tsx | LINES: 1-364 | SHA256: 97e0a401d1deb70143afee4cab676e0a602ea6e38928e4cc2dd5a392260ff61e

```

1 | import { describe, it, expect, vi } from "vitest";
2 | import { render, screen, fireEvent } from "@testing-library/react";
3 | import HeatmapGrid from "@components/shared/HeatmapGrid";
4 | import type { HeatmapDay, DateRange } from "@types/habits";
5 |
6 | // -----
7 | // Helpers
8 | // -----
9 |
10 | function makeDay(date: string, totalCount: number): HeatmapDay {
11 |   return {
12 |     date,
13 |     academicCount: Math.min(totalCount, 4),
14 |     wellnessCount: Math.max(totalCount - 4, 0),
15 |     totalCount,
16 |     habits: [],
17 |   };
18 | }
19 |
20 | function makeDateRange(start: string, end: string): DateRange {
21 |   return { start, end };
22 | }
23 |
24 | /** Build a contiguous array of HeatmapDay objects between start and end. */
25 | function buildDays(
26 |   start: string,
27 |   end: string,
28 |   countFn: (date: string) => number = () => 0
29 | ): HeatmapDay[] {
30 |   const days: HeatmapDay[] = [];
31 |   const cursor = new Date(start + "T00:00:00");
32 |   const endDate = new Date(end + "T00:00:00");
33 |   while (cursor <= endDate) {
34 |     const dateStr =
35 |       cursor.getFullYear() +
36 |       "-" +
37 |       String(cursor.getMonth() + 1).padStart(2, "0") +
38 |       "-" +
39 |       String(cursor.getDate()).padStart(2, "0");
40 |     days.push(makeDay(dateStr, countFn(dateStr)));
41 |     cursor.setDate(cursor.getDate() + 1);
42 |   }
43 |   return days;
44 | }
45 |
46 | // Stub ResizeObserver for happy-dom
47 | class MockResizeObserver {
48 |   callback: ResizeObserverCallback;
49 |   constructor(cb: ResizeObserverCallback) {
50 |     this.callback = cb;
51 |   }
52 |   observe(_target: Element) {
53 |     // Fire immediately with a mock entry
54 |     this.callback(
55 |       [{ contentRect: { width: 900 } } as unknown as ResizeObserverEntry],
56 |       this as unknown as ResizeObserver
57 |     );
58 |   }
59 |   unobserve() {}
60 |   disconnect() {}
61 | }

```

```

62 // Install mock before tests
63 vi.stubGlobal("ResizeObserver", MockResizeObserver);
64
65 // -----
66 // Tests
67 // -----
68
69 describe("HeatmapGrid", () => {
70   const range14 = makeDateRange("2025-01-06", "2025-01-19"); // 14 days = 2 weeks
71
72   it("renders correct number of cells for a date range", () => {
73     const days = buildDays("2025-01-06", "2025-01-19");
74     render(<HeatmapGrid data={days} semesterRange={range14} />);
75
76     const cells = screen.getAllByRole("gridcell");
77     expect(cells).toHaveLength(14);
78   });
79
80   it("cells have correct ARIA labels", () => {
81     const days = [makeDay("2025-01-06", 3)];
82     render(<HeatmapGrid data={days} semesterRange={range14} />);
83
84     const cell = screen.getByTestId("heatmap-cell-2025-01-06");
85     expect(cell).toHaveAttribute(
86       "aria-label",
87       "January 6, 2025: 3 habits completed"
88     );
89   });
90
91   it("renders singular \"habit\" for count of 1", () => {
92     const days = [makeDay("2025-01-06", 1)];
93     render(<HeatmapGrid data={days} semesterRange={range14} />);
94
95     const cell = screen.getByTestId("heatmap-cell-2025-01-06");
96     expect(cell).toHaveAttribute(
97       "aria-label",
98       "January 6, 2025: 1 habit completed"
99     );
100   });
101
102   it("future dates are rendered as disabled", () => {
103     // Use a date far in the future
104     const futureRange = makeDateRange("2099-06-01", "2099-06-07");
105     const days = buildDays("2099-06-01", "2099-06-07", () => 2);
106     render(<HeatmapGrid data={days} semesterRange={futureRange} />);
107
108     const cell = screen.getByTestId("heatmap-cell-2099-06-01");
109     expect(cell).toHaveAttribute("aria-disabled", "true");
110     expect(cell).toHaveAttribute("data-future", "true");
111   });
112
113   it("color intensity matches habit count (level 0-4)", () => {
114     const days = [
115       makeDay("2025-01-06", 0),
116       makeDay("2025-01-07", 1),
117       makeDay("2025-01-08", 2),
118       makeDay("2025-01-09", 3),
119       makeDay("2025-01-10", 5),
120     ];
121     render(<HeatmapGrid data={days} semesterRange={range14} />);
122
123     // Intensity is exposed as a stable data-level attribute (0-4). The visual
124     // color is driven from --heatmap-level-* tokens (see index.css), decoupled
125     // from the DOM contract so theming never breaks these tests.
126     expect(screen.getByTestId("heatmap-cell-2025-01-06")).toHaveAttribute(
127       "data-level",
128       "0"
129     );
130     expect(screen.getByTestId("heatmap-cell-2025-01-07")).toHaveAttribute(
131       "data-level",
132       "1"
133     );
134     expect(screen.getByTestId("heatmap-cell-2025-01-08")).toHaveAttribute(
135       "data-level",
136       "2"
137     );
138     expect(screen.getByTestId("heatmap-cell-2025-01-09")).toHaveAttribute(
139       "data-level",
140       "3"
141     );
142     // 5 habits !' capped at level 4
143     expect(screen.getByTestId("heatmap-cell-2025-01-10")).toHaveAttribute(
144       "data-level",
145       "4"
146     );
147   });
148
149   it("renders month labels", () => {
150     // Range spanning Jan and Feb
151     const range = makeDateRange("2025-01-01", "2025-02-15");
152     const days = buildDays("2025-01-01", "2025-02-15");
153     render(<HeatmapGrid data={days} semesterRange={range} />);
154
155     expect(screen.getByTestId("month-label-Jan")).toBeInTheDocument();
156     expect(screen.getByTestId("month-label-Feb")).toBeInTheDocument();
157   });
158
159   it("renders day-of-week labels (Mon, Wed, Fri)", () => {
160     const days = buildDays("2025-01-06", "2025-01-19");
161     render(<HeatmapGrid data={days} semesterRange={range14} />);
162
163     expect(screen.getByTestId("day-label-Mon")).toBeInTheDocument();
164     expect(screen.getByTestId("day-label-Wed")).toBeInTheDocument();
165

```

```

166 |     expect(screen.getByTestId("day-label-Fri")).toBeInTheDocument();
167 | });
168 |
169 | it("renders color legend with 5 levels", () => {
170 |   const days = buildDays("2025-01-06", "2025-01-19");
171 |   render(<HeatmapGrid data={days} semesterRange={range14} />);
172 |
173 |   for (let level = 0; level <= 4; level++) {
174 |     expect(screen.getByTestId(`legend-level-${level}`)).toBeInTheDocument();
175 |   }
176 | });
177 |
178 | it("renders legend labels for first and last levels", () => {
179 |   const days = buildDays("2025-01-06", "2025-01-19");
180 |   render(<HeatmapGrid data={days} semesterRange={range14} />);
181 |
182 |   expect(screen.getByTestId("legend-label-0")).toHaveTextContent(
183 |     "No activity"
184 |   );
185 |   expect(screen.getByTestId("legend-label-4")).toHaveTextContent("4+ habits");
186 | });
187 |
188 | // -----
189 | // Keyboard navigation
190 | // -----
191 |
192 | it("arrow keys move focus between cells", () => {
193 |   const days = buildDays("2025-01-06", "2025-01-19");
194 |   render(<HeatmapGrid data={days} semesterRange={range14} />);
195 |
196 |   const firstCell = screen.getByTestId("heatmap-cell-2025-01-06");
197 |   firstCell.focus();
198 |
199 |   // ArrowDown moves to next row (index + 1)
200 |   fireEvent.keyDown(firstCell, { key: "ArrowDown" });
201 |   const secondCell = screen.getByTestId("heatmap-cell-2025-01-07");
202 |   expect(document.activeElement).toBe(secondCell);
203 | });
204 |
205 | it("Enter key triggers onCellClick", () => {
206 |   const onClick = vi.fn();
207 |   const days = buildDays("2025-01-06", "2025-01-19");
208 |   render(
209 |     <HeatmapGrid data={days} semesterRange={range14} onCellClick={onClick} />
210 |   );
211 |
212 |   const cell = screen.getByTestId("heatmap-cell-2025-01-06");
213 |   cell.focus();
214 |   fireEvent.keyDown(cell, { key: "Enter" });
215 |
216 |   expect(onClick).toHaveBeenCalledWith("2025-01-06");
217 | });
218 |
219 | it("Space key triggers onCellClick", () => {
220 |   const onClick = vi.fn();
221 |   const days = buildDays("2025-01-06", "2025-01-19");
222 |   render(
223 |     <HeatmapGrid data={days} semesterRange={range14} onCellClick={onClick} />
224 |   );
225 |
226 |   const cell = screen.getByTestId("heatmap-cell-2025-01-06");
227 |   cell.focus();
228 |   fireEvent.keyDown(cell, { key: " " });
229 |
230 |   expect(onClick).toHaveBeenCalledWith("2025-01-06");
231 | });
232 |
233 | it("Enter/Space does not trigger onCellClick for future dates", () => {
234 |   const onClick = vi.fn();
235 |   const futureRange = makeDateRange("2099-06-01", "2099-06-07");
236 |   const days = buildDays("2099-06-01", "2099-06-07");
237 |   render(
238 |     <HeatmapGrid
239 |       data={days}
240 |       semesterRange={futureRange}
241 |       onCellClick={onClick}
242 |     />
243 |   );
244 |
245 |   const cell = screen.getByTestId("heatmap-cell-2099-06-01");
246 |   cell.focus();
247 |   fireEvent.keyDown(cell, { key: "Enter" });
248 |
249 |   expect(onClick).not.toHaveBeenCalled();
250 | });
251 |
252 | // -----
253 | // Interaction callbacks
254 | // -----
255 |
256 | it("onCellHover is called on mouse enter/leave", () => {
257 |   const onHover = vi.fn();
258 |   const days = [makeDay("2025-01-06", 2)];
259 |   render(
260 |     <HeatmapGrid data={days} semesterRange={range14} onCellHover={onHover} />
261 |   );
262 |
263 |   const cell = screen.getByTestId("heatmap-cell-2025-01-06");
264 |   fireEvent.mouseEnter(cell);
265 |   expect(onHover).toHaveBeenCalledWith("2025-01-06");
266 |
267 |   fireEvent.mouseLeave(cell);
268 |   expect(onHover).toHaveBeenCalledWith(null);
269 | });

```



```

270 |
271 | it("clicking a cell triggers onCellClick", () => {
272 |   const onClick = vi.fn();
273 |   const days = [makeDay("2025-01-06", 2)];
274 |   render(
275 |     <HeatmapGrid data={days} semesterRange={range14} onCellClick={onClick} />
276 |   );
277 |
278 |   const cell = screen.getByTestId("heatmap-cell-2025-01-06");
279 |   fireEvent.click(cell);
280 |   expect(onClick).toHaveBeenCalled();
281 | });
282 |
283 | it("clicking a future date cell does not trigger onCellClick", () => {
284 |   const onClick = vi.fn();
285 |   const futureRange = makeDateRange("2099-06-01", "2099-06-07");
286 |   const days = buildDays("2099-06-01", "2099-06-07");
287 |   render(
288 |     <HeatmapGrid
289 |       data={days}
290 |       semesterRange={futureRange}
291 |       onCellClick={onClick}
292 |     />
293 |   );
294 |
295 |   const cell = screen.getByTestId("heatmap-cell-2099-06-01");
296 |   fireEvent.click(cell);
297 |   expect(onClick).not.toHaveBeenCalled();
298 | });
299 |
300 | // -----
301 | // Roving tabindex
302 | // -----
303 |
304 | it("uses roving tabindex – first cell has tabIndex 0, others -1", () => {
305 |   const days = buildDays("2025-01-06", "2025-01-08");
306 |   const range = makeDateRange("2025-01-06", "2025-01-08");
307 |   render(<HeatmapGrid data={days} semesterRange={range} />);
308 |
309 |   const cells = screen.getAllByRole("gridcell");
310 |   expect(cells[0]).toHaveAttribute("tabindex", "0");
311 |   expect(cells[1]).toHaveAttribute("tabindex", "-1");
312 |   expect(cells[2]).toHaveAttribute("tabindex", "-1");
313 | });
314 |
315 | // -----
316 | // Grid role
317 | // -----
318 |
319 | it('has role="grid" on the grid container', () => {
320 |   const days = buildDays("2025-01-06", "2025-01-19");
321 |   render(<HeatmapGrid data={days} semesterRange={range14} />);
322 |
323 |   expect(screen.getByRole("grid")).toBeInTheDocument();
324 | });
325 |
326 | // -----
327 | // Legend is real HTML (regression: SVG legend text overlapped swatches)
328 | // -----
329 |
330 | it("renders the legend as HTML with swatches and labels as distinct nodes", () => {
331 |   const days = buildDays("2025-01-06", "2025-01-19");
332 |   render(<HeatmapGrid data={days} semesterRange={range14} />);
333 |
334 |   // The legend container is a sibling of the grid (below it), not inside it.
335 |   const legend = screen.getByTestId("heatmap-legend");
336 |   expect(legend).toBeInTheDocument();
337 |
338 |   // Swatch and its label are separate elements – they can never overlap the
339 |   // way the old absolutely-placed SVG <text> did over the next <rect>.
340 |   const swatch0 = screen.getByTestId("legend-level-0");
341 |   const label0 = screen.getByTestId("legend-label-0");
342 |   expect(swatch0).not.toBe(label0);
343 |   expect(legend).toContainElement(swatch0);
344 |   expect(legend).toContainElement(label0);
345 |   // The grid must not contain the legend swatches.
346 |   expect(screen.getByRole("grid")).not.toContainElement(swatch0);
347 | });
348 |
349 | // -----
350 | // Monday-aligned padding – decorative pads are excluded from gridcells
351 | // -----
352 |
353 | it("keeps gridcell count equal to day count when range starts mid-week", () => {
354 |   // 2025-01-01 is a Wednesday !' 2 leading pad slots are added for alignment,
355 |   // but only real days are exposed as gridcells.
356 |   const range = makeDateRange("2025-01-01", "2025-01-07"); // 7 days
357 |   const days = buildDays("2025-01-01", "2025-01-07");
358 |   render(<HeatmapGrid data={days} semesterRange={range} />);
359 |
360 |   expect(screen.getAllByRole("gridcell")).toHaveLength(7);
361 |   expect(screen.getByTestId("heatmap-cell-2025-01-01")).toBeInTheDocument();
362 | });
363 | });
364 |

```

FILE: [src/__tests__/unit/heatmapTooltip.test.tsx](#)

TYPE: .tsx | LINES: 1-180 | SHA256: f5d7b1b4c5481e142ce25cc1eeec1a740ff94a0867c579b5c57ccdb830bd4ea8

```

1 | import { describe, it, expect } from "vitest";
2 | import { render, screen } from "@testing-library/react";
3 | import HeatmapTooltip from "@components/shared/HeatmapTooltip";
4 | import type { CompletedHabit } from "@types/habits";
5 |

```

```

6 // -----
7 // Helpers
8 // -----
9
10 function makeHabit(
11   type: CompletedHabit["type"],
12   category: CompletedHabit["category"]
13 ): CompletedHabit {
14   return { type, category, completedAt: "2025-03-15T10:00:00Z" };
15 }
16
17 // -----
18 // Tests
19 // -----
20
21 describe("HeatmapTooltip", () => {
22   it("renders formatted date", () => {
23     render(
24       <HeatmapTooltip
25         date="2025-03-15"
26         habits={}
27         xpEarned={0}
28         streakActive={false}
29       />
30     );
31     expect(screen.getByTestId("tooltip-date")).toHaveTextContent(
32       "March 15, 2025"
33     );
34   });
35
36   it('shows "No habits completed" for empty day', () => {
37     render(
38       <HeatmapTooltip
39         date="2025-03-15"
40         habits={}
41         xpEarned={0}
42         streakActive={false}
43       />
44     );
45     expect(screen.getByTestId("tooltip-empty")).toHaveTextContent(
46       "No habits completed"
47     );
48   });
49
50   it("renders academic habits with checkmarks", () => {
51     const habits: CompletedHabit[] = [
52       makeHabit("login", "academic"),
53       makeHabit("submit", "academic"),
54     ];
55     render(
56       <HeatmapTooltip
57         date="2025-03-15"
58         habits={habits}
59         xpEarned={35}
60         streakActive={true}
61       />
62     );
63     expect(screen.getByTestId("tooltip-academic-section")).toBeInTheDocument();
64     expect(screen.getByTestId("tooltip-habit-login")).toHaveTextContent(" ");
65     expect(screen.getByTestId("tooltip-habit-login")).toHaveTextContent(
66       "Login"
67     );
68     expect(screen.getByTestId("tooltip-habit-submit")).toHaveTextContent(
69       "Submit"
70     );
71   });
72
73   it("renders wellness habits in separate section", () => {
74     const habits: CompletedHabit[] = [
75       makeHabit("meditation", "wellness"),
76       makeHabit("exercise", "wellness"),
77     ];
78     render(
79       <HeatmapTooltip
80         date="2025-03-15"
81         habits={habits}
82         xpEarned={10}
83         streakActive={false}
84       />
85     );
86     expect(screen.getByTestId("tooltip-wellness-section")).toBeInTheDocument();
87     expect(screen.getByTestId("tooltip-habit-meditation")).toHaveTextContent(
88       "Meditation"
89     );
90     expect(screen.getByTestId("tooltip-habit-exercise")).toHaveTextContent(
91       "Exercise"
92     );
93   });
94
95   it("renders both academic and wellness sections when mixed", () => {
96     const habits: CompletedHabit[] = [
97       makeHabit("login", "academic"),
98       makeHabit("meditation", "wellness"),
99     ];
100     render(
101       <HeatmapTooltip
102         date="2025-03-15"
103         habits={habits}
104         xpEarned={15}
105         streakActive={true}
106       />
107     );
108     expect(screen.getByTestId("tooltip-academic-section")).toBeInTheDocument();
109     expect(screen.getByTestId("tooltip-wellness-section")).toBeInTheDocument();

```

```

110 | });
111 |
112 | it("displays XP earned", () => {
113 |   const habits: CompletedHabit[] = [makeHabit("login", "academic")];
114 |   render(
115 |     <HeatmapTooltip
116 |       date="2025-03-15"
117 |       habits={habits}
118 |       xpEarned={50}
119 |       streakActive={true}
120 |     />
121 |   );
122 |   expect(screen.getByTestId("tooltip-xp")).toHaveTextContent("50 XP earned");
123 | });
124 |
125 | it("shows streak active status", () => {
126 |   const habits: CompletedHabit[] = [makeHabit("login", "academic")];
127 |   render(
128 |     <HeatmapTooltip
129 |       date="2025-03-15"
130 |       habits={habits}
131 |       xpEarned={10}
132 |       streakActive={true}
133 |     />
134 |   );
135 |   expect(screen.getByTestId("tooltip-streak")).toHaveTextContent(
136 |     "0=100% Streak active"
137 |   );
138 | });
139 |
140 | it("shows streak broken status", () => {
141 |   const habits: CompletedHabit[] = [makeHabit("login", "academic")];
142 |   render(
143 |     <HeatmapTooltip
144 |       date="2025-03-15"
145 |       habits={habits}
146 |       xpEarned={10}
147 |       streakActive={false}
148 |     />
149 |   );
150 |   expect(screen.getByTestId("tooltip-streak")).toHaveTextContent(
151 |     "Streak broken"
152 |   );
153 | });
154 |
155 | it('has role="tooltip"', () => {
156 |   render(
157 |     <HeatmapTooltip
158 |       date="2025-03-15"
159 |       habits={[]}
160 |       xpEarned={0}
161 |       streakActive={false}
162 |     />
163 |   );
164 |   expect(screen.getByRole("tooltip")).toBeInTheDocument();
165 | });
166 |
167 | it("does not show XP/streak footer for empty day", () => {
168 |   render(
169 |     <HeatmapTooltip
170 |       date="2025-03-15"
171 |       habits={[]}
172 |       xpEarned={0}
173 |       streakActive={false}
174 |     />
175 |   );
176 |   expect(screen.queryByTestId("tooltip-xp")).not.toBeInTheDocument();
177 |   expect(screen.queryByTestId("tooltip-streak")).not.toBeInTheDocument();
178 | });
179 | });
180 |

```

FILE: [src/__tests__/unit/heroCarousel.test.tsx](#)

TYPE: .tsx | LINES: 1-55 | SHA256: 071f77560d69f2429aa11b8df28fb9a0a56cd479a19b1695995e32c8fffb86793

```

1 | // @vitest-environment happy-dom
2 |
3 | import { describe, expect, it } from "vitest";
4 | import "@lib/i18n";
5 | import { render, screen } from "@testing-library/react";
6 | import userEvent from "@testing-library/user-event";
7 | import HeroCarousel from "@design-system/patterns/HeroCarousel";
8 |
9 | describe("HeroCarousel", () => {
10 |   it("hides carousel controls when only one real slide is available", () => {
11 |     render(
12 |       <HeroCarousel
13 |         ariaLabel="Highlights"
14 |         slides={ [<div key="only">Only slide</div> ] }
15 |       />
16 |     );
17 |
18 |     expect(
19 |       screen.getByRole("region", { name: "Highlights" })
20 |     ).toBeInTheDocument();
21 |     expect(
22 |       screen.queryByRole("button", { name: "Previous slide" })
23 |     ).not.toBeInTheDocument();
24 |     expect(
25 |       screen.queryByRole("button", { name: "Next slide" })
26 |     ).not.toBeInTheDocument();
27 |   });
28 |
29 |   it("moves between real slides with localized controls and dots", async () => {

```

```

30 |     const user = userEvent.setup();
31 |     render(
32 |       <HeroCarousel
33 |         ariaLabel="Highlights"
34 |         slides={[
35 |           <div key="first">First slide</div>,
36 |           <div key="second">Second slide</div>,
37 |         ]}
38 |       />
39 |     );
40 |
41 |     const firstGroup = screen.getByRole("group", { name: "1 of 2" });
42 |     const track = firstGroup.parentElement;
43 |     expect(track).toHaveStyle({ transform: "translateX(-0%)" });
44 |
45 |     await user.click(screen.getByRole("button", { name: "Next slide" }));
46 |     expect(track).toHaveStyle({ transform: "translateX(-100%)" });
47 |     expect(
48 |       screen.getByRole("button", { name: "Go to slide 2" })
49 |     ).toHaveAttribute("aria-current", "true");
50 |
51 |     await user.click(screen.getByRole("button", { name: "Previous slide" }));
52 |     expect(track).toHaveStyle({ transform: "translateX(-0%)" });
53 |   });
54 | });
55 |

```

FILE: src/___tests___/unit/historicalEvidenceDashboard.test.tsx

TYPE: .tsx | LINES: 1-213 | SHA256: fae5216645dcd4a0bf7e11e86e4003adeff739e5d95194ee5db204986fcda7c2

```

1 | // @vitest-environment happy-dom
2 | // =====
3 | // HistoricalEvidenceDashboard – no developer-oriented text across all states
4 | //
5 | // Feature: qa-partner-review-remediation – Req 4 (B4)
6 | // Validates: Requirements 4.5, 4.7
7 | //
8 | // The Admin Historical Evidence dashboard previously rendered the literal
9 | // developer placeholder "Requires mv_historical_evidence view". Req 4.5 / 4.7
10 | // require that NO user-facing surface of this dashboard ever exposes internal
11 | // object names, migration identifiers, or developer instructions – in ANY
12 | // render state (loading / empty / data).
13 | //
14 | // This suite mocks `useHistoricalEvidence` to drive the three mutually
15 | // exclusive states and asserts:
16 | //   1. loading !' a component-level shimmer is present; no developer text
17 | //   2. empty !' the shared NoEvidence empty state is present; no developer text
18 | //   3. data !' the per-semester evidence tables render; no developer text
19 | //
20 | // The "no developer text" check is a case-insensitive substring scan over the
21 | // rendered container's textContent against a forbidden list (Req 4.7).
22 | // =====
23 |
24 | import { describe, it, expect, vi, beforeEach } from "vitest";
25 | import { render, screen } from "@testing-library/react";
26 | import { I18nextProvider } from "react-i18next";
27 |
28 | import i18n from "@lib/i18n";
29 | import type { HistoricalEvidenceRow } from "@hooks/useHistoricalEvidence";
30 |
31 | // -----
32 | // Mocks (must precede the component import)
33 | // -----
34 |
35 | // nuqs – minimal stub: filters are URL state the component reads but does not
36 | // need wired for this render-only suite (mirrors heatmapFilters.test.tsx).
37 | vi.mock("nuqs", () => ({
38 |   parseAsString: { withDefault: (def: string) => def },
39 |   useQueryState: () => [""], vi.fn() as const,
40 | }));
41 |
42 | // The data hook is the single dependency that drives every render state. We
43 | // mock the module so no real Supabase/network call is made.
44 | const mockUseHistoricalEvidence = vi.fn();
45 | vi.mock("@hooks/useHistoricalEvidence", () => ({
46 |   useHistoricalEvidence: () => mockUseHistoricalEvidence(),
47 | }));
48 |
49 | // happy-dom does not implement ResizeObserver, which Radix relies on; the
50 | // dashboard renders two always-visible Select triggers in its header.
51 | class MockResizeObserver {
52 |   observe() {}
53 |   unobserve() {}
54 |   disconnect() {}
55 | }
56 | vi.stubGlobal("ResizeObserver", MockResizeObserver);
57 |
58 | import HistoricalEvidenceDashboard from "@pages/admin/historical-evidence/HistoricalEvidenceDashboard";
59 |
60 | // -----
61 | // Fixtures
62 | // -----
63 |
64 | const DATA_ROWS: HistoricalEvidenceRow[] = [
65 |   {
66 |     semester_id: "sem-1",
67 |     semester_name: "Fall 2024",
68 |     start_date: "2024-09-01",
69 |     outcome_type: "PLO",
70 |     blooms_level: "applying",
71 |     evidence_count: 12,
72 |     avg_score: 78,
73 |     excellent_count: 3,
74 |     satisfactory_count: 5,

```

```

75 |     developing_count: 3,
76 |     not_yet_count: 1,
77 |   },
78 |   {
79 |     semester_id: "sem-2",
80 |     semester_name: "Spring 2025",
81 |     start_date: "2025-02-01",
82 |     outcome_type: "CLO",
83 |     blooms_level: "creating",
84 |     evidence_count: 8,
85 |     avg_score: 64,
86 |     excellent_count: 1,
87 |     satisfactory_count: 3,
88 |     developing_count: 3,
89 |     not_yet_count: 1,
90 |   },
91 | ];
92 |
93 | // Internal object names, migration identifiers, raw object/SQL identifiers, and
94 | // developer instructions that must NEVER appear in a user-facing surface
95 | // (Req 4.5, 4.7). Compared case-insensitively as substrings.
96 | const FORBIDDEN_SUBSTRINGS: readonly string[] = [
97 |   "requires mv_historical_evidence view", // the exact original placeholder
98 |   "mv_historical_evidence", // internal object name
99 |   "materialized view",
100 |   "view", // any developer-facing reference to the backing DB view
101 |   "requires",
102 |   "migration",
103 |   "refresh_mv_historical_evidence",
104 |   "information_schema",
105 |   "search_path",
106 |   "security definer",
107 |   "supabase",
108 |   "as never",
109 |   "select * from",
110 |   "todo",
111 |   "fixme",
112 | ];
113 |
114 | const renderDashboard = () =>
115 |   render(
116 |     <I18nextProvider i18n={i18n}>
117 |       <HistoricalEvidenceDashboard />
118 |     </I18nextProvider>
119 |   );
120 |
121 | const assertNoDeveloperText = (container: HTMLElement) => {
122 |   const text = (container.textContent ?? "").toLowerCase();
123 |   for (const forbidden of FORBIDDEN_SUBSTRINGS) {
124 |     expect(
125 |       text.includes(forbidden.toLowerCase()),
126 |       `rendered text must not contain developer-oriented substring "${forbidden}"`
127 |     ).toBe(false);
128 |   }
129 | };
130 |
131 | // -----
132 | // Tests
133 | // -----
134 |
135 | describe("HistoricalEvidenceDashboard – no developer-oriented text (Req 4.5, 4.7)", () => {
136 |   beforeEach(() => {
137 |     vi.clearAllMocks();
138 |   });
139 |
140 |   it("renders a shimmer on loading and no developer text", () => {
141 |     mockUseHistoricalEvidence.mockReturnValue({
142 |       data: undefined,
143 |       isLoading: true,
144 |       isError: false,
145 |     });
146 |
147 |     const { container } = renderDashboard();
148 |
149 |     // A proper component-level shimmer is present while loading.
150 |     expect(
151 |       container.querySelectorAll(".animate-shimmer").length
152 |     ).toBeGreaterThan(0);
153 |
154 |     assertNoDeveloperText(container);
155 |   });
156 |
157 |   it("renders the shared empty state on zero rows and no developer text", () => {
158 |     mockUseHistoricalEvidence.mockReturnValue({
159 |       data: [],
160 |       isLoading: false,
161 |       isError: false,
162 |     });
163 |
164 |     const { container } = renderDashboard();
165 |
166 |     // The shared NoEvidence empty state (not a bare placeholder).
167 |     expect(
168 |       screen.getByText(i18n.t("common:empty.noEvidence.title"))
169 |     ).toBeInTheDocument();
170 |
171 |     assertNoDeveloperText(container);
172 |   });
173 |
174 |   it("renders the evidence data and no developer text", () => {
175 |     mockUseHistoricalEvidence.mockReturnValue({
176 |       data: DATA_ROWS,
177 |       isLoading: false,
178 |       isError: false,

```

```

179 |     });
180 |
181 |     const { container } = renderDashboard();
182 |
183 |     // Per-semester evidence renders (grouped by semester).
184 |     expect(screen.getByText("Fall 2024")).toBeInTheDocument();
185 |     expect(screen.getByText("Spring 2025")).toBeInTheDocument();
186 |     // The evidence table structure is present.
187 |     expect(screen.getAllByText("Avg Score").length).toBeGreaterThan(0);
188 |     // No shimmer once data has resolved.
189 |     expect(container.querySelector(".animate-shimmer").length).toBe(0);
190 |
191 |     assertNoDeveloperText(container);
192 | });
193 |
194 | it("never exposes the original developer placeholder in any state", () => {
195 |   const states: ReadonlyArray<{
196 |     data: HistoricalEvidenceRow[] | undefined;
197 |     isLoading: boolean;
198 |     isError: boolean;
199 |   }> = [
200 |     { data: undefined, isLoading: true, isError: false },
201 |     { data: [], isLoading: false, isError: false },
202 |     { data: DATA_ROWS, isLoading: false, isError: false },
203 |   ];
204 |
205 |   for (const state of states) {
206 |     mockUseHistoricalEvidence.mockReturnValue(state);
207 |     const { container, unmount } = renderDashboard();
208 |     assertNoDeveloperText(container);
209 |     unmount();
210 |   }
211 | });
212 | });
213 |

```

FILE: [src/__tests__/unit/i18n.test.ts](#)

TYPE: .ts | LINES: 1-159 | SHA256: d45c4eaac8fbbb504cb0fa51810e5cb63e7656a6f42d2629c9b1579f96cff20d

```

1 | import { describe, it, expect } from "vitest";
2 | import i18n, {
3 |   resources,
4 |   defaultNS,
5 |   namespaces,
6 |   supportedLanguages,
7 | } from "@lib/i18n";
8 |
9 | describe("i18n initialization", () => {
10 |   it("initializes i18next successfully", () => {
11 |     expect(i18n.isInitialized).toBe(true);
12 |   });
13 |
14 |   it("defaults to English language", () => {
15 |     expect(i18n.language).toBe("en");
16 |   });
17 |
18 |   it("has English as fallback language", () => {
19 |     expect(i18n.options.fallbackLng).toEqual(["en"]);
20 |   });
21 |
22 |   it("has escapeValue set to false", () => {
23 |     expect(i18n.options.interpolation?.escapeValue).toBe(false);
24 |   });
25 |
26 |   it("exports all expected namespaces", () => {
27 |     expect(namespaces).toEqual([
28 |       "common",
29 |       "auth",
30 |       "admin",
31 |       "teacher",
32 |       "student",
33 |       "coordinator",
34 |       "gamification",
35 |       "ai",
36 |     ]);
37 |   });
38 |
39 |   it("exports supported languages", () => {
40 |     expect(supportedLanguages).toEqual(["en", "ar"]);
41 |   });
42 |
43 |   it("uses common as default namespace", () => {
44 |     expect(defaultNS).toBe("common");
45 |   });
46 |
47 |   it("loads all namespace resource bundles for English", () => {
48 |     for (const ns of namespaces) {
49 |       expect(resources.en[ns]).toBeDefined();
50 |       expect(typeof resources.en[ns]).toBe("object");
51 |       expect(Object.keys(resources.en[ns]).length).toBeGreaterThan(0);
52 |     }
53 |   });
54 |
55 |   it("loads all namespace resource bundles for Arabic", () => {
56 |     for (const ns of namespaces) {
57 |       expect(resources.ar[ns]).toBeDefined();
58 |       expect(typeof resources.ar[ns]).toBe("object");
59 |       expect(Object.keys(resources.ar[ns]).length).toBeGreaterThan(0);
60 |     }
61 |   });
62 | });
63 |
64 | describe("common translation keys", () => {
65 |   it("translates nav.dashboard", () => {

```

```

66 |     expect(i18n.t("nav.dashboard")).toBe("Dashboard");
67 | });
68 |
69 | it("translates buttons.save", () => {
70 |     expect(i18n.t("buttons.save")).toBe("Save");
71 | });
72 |
73 | it("translates buttons.cancel", () => {
74 |     expect(i18n.t("buttons.cancel")).toBe("Cancel");
75 | });
76 |
77 | it("translates buttons.delete", () => {
78 |     expect(i18n.t("buttons.delete")).toBe("Delete");
79 | });
80 |
81 | it("translates errors.generic", () => {
82 |     expect(i18n.t("errors.generic")).toBe(
83 |         "Something went wrong. Please try again."
84 |     );
85 | });
86 |
87 | it("translates status.loading", () => {
88 |     expect(i18n.t("status.loading")).toBe("Loading...");
89 | });
90 |
91 | it("translates form.required", () => {
92 |     expect(i18n.t("form.required")).toBe("This field is required");
93 | });
94 | });
95 |
96 | describe("auth translation keys", () => {
97 |     it("translates auth:login.title", () => {
98 |         expect(i18n.t("login.title", { ns: "auth" })).toBe("Sign In");
99 |     });
100 |
101 |     it("translates auth:logout.button", () => {
102 |         expect(i18n.t("logout.button", { ns: "auth" })).toBe("Sign Out");
103 |     });
104 |
105 |     it("translates auth:resetPassword.title", () => {
106 |         expect(i18n.t("resetPassword.title", { ns: "auth" })).toBe(
107 |             "Reset Password"
108 |         );
109 |     });
110 | });
111 |
112 | describe("gamification translation keys", () => {
113 |     it("translates gamification:xp.title", () => {
114 |         expect(i18n.t("xp.title", { ns: "gamification" })).toBe(
115 |             "Experience Points"
116 |         );
117 |     });
118 |
119 |     it("translates gamification:streaks.title", () => {
120 |         expect(i18n.t("streaks.title", { ns: "gamification" })).toBe(
121 |             "Login Streak"
122 |         );
123 |     });
124 |
125 |     it("translates gamification:badges.title", () => {
126 |         expect(i18n.t("badges.title", { ns: "gamification" })).toBe("Badges");
127 |     });
128 |
129 |     it("translates gamification:leaderboard.title", () => {
130 |         expect(i18n.t("leaderboard.title", { ns: "gamification" })).toBe(
131 |             "Leaderboard"
132 |         );
133 |     });
134 |
135 |     it("handles interpolation in gamification:xp.earned", () => {
136 |         expect(i18n.t("xp.earned", { ns: "gamification", amount: 50 })).toBe(
137 |             "+50 XP"
138 |         );
139 |     });
140 | });
141 |
142 | describe("namespace-specific key existence", () => {
143 |     it("admin namespace has dashboard keys", () => {
144 |         expect(i18n.t("dashboard.title", { ns: "admin" })).toBe("Dashboard");
145 |     });
146 |
147 |     it("teacher namespace has grading keys", () => {
148 |         expect(i18n.t("grading.title", { ns: "teacher" })).toBe("Grading");
149 |     });
150 |
151 |     it("student namespace has assignments keys", () => {
152 |         expect(i18n.t("assignments.title", { ns: "student" })).toBe("Assignments");
153 |     });
154 |
155 |     it("ai namespace has suggestions keys", () => {
156 |         expect(i18n.t("suggestions.title", { ns: "ai" })).toBe("AI Suggestions");
157 |     });
158 | });
159 |

```

FILE: [src/__tests__/unit/i18nExtraction.test.tsx](#)

TYPE: .tsx | LINES: 1-156 | SHA256: 284c9cba86d10e611e0bcf6624dab249b31b08d772446775dae5a996a721591c

```

1 | import { vi } from "vitest";
2 | import "@/lib/i18n";
3 |
4 | // -----
5 | // Mock navigate
6 | // -----

```

```

7 | const mockNavigate = vi.fn();
8 | vi.mock("react-router-dom", async () => {
9 |   const actual = await vi.importActual("react-router-dom");
10 |   return { ...actual, useNavigate: () => mockNavigate };
11 | });
12 |
13 | // -----
14 | // Mock Supabase
15 | // -----
16 | const mockGetSession = vi.fn();
17 | const mockOnAuthStateChange = vi.fn();
18 | const mockFrom = vi.fn();
19 | const mockFunctionsInvoke = vi.fn();
20 |
21 | vi.mock("@/lib/supabase", () => ({
22 |   supabase: {
23 |     auth: {
24 |       getSession: (...args: unknown[]) => mockGetSession(...args),
25 |       signInWithPassword: vi.fn(),
26 |       signOut: vi.fn(),
27 |       resetPasswordForEmail: vi.fn(),
28 |       onAuthStateChange: (...args: unknown[]) => mockOnAuthStateChange(...args),
29 |     },
30 |     from: (...args: unknown[]) => mockFrom(...args),
31 |     functions: { invoke: (...args: unknown[]) => mockFunctionsInvoke(...args) },
32 |     channel: vi.fn().mockReturnValue({
33 |       on: vi.fn().mockReturnThis(),
34 |       subscribe: vi.fn().mockReturnValue({ status: "SUBSCRIBED" }),
35 |       unsubscribe: vi.fn(),
36 |     }),
37 |     removeChannel: vi.fn(),
38 |   },
39 | }));
40 |
41 | // -----
42 | // Mock hooks used by dashboards
43 | // -----
44 | vi.mock("@/hooks/useAdminDashboard", () => ({
45 |   useAdminKPIs: () => ({
46 |     data: {
47 |       totalUsers: 42,
48 |       activeUsers: 30,
49 |       totalPrograms: 5,
50 |       totalCourses: 12,
51 |       usersByRole: { admin: 2, teacher: 10, student: 30 },
52 |     },
53 |     isLoading: false,
54 |   }),
55 |   useRecentAuditLogs: () => ({ data: [], isLoading: false }),
56 |   useOnboardingAnalytics: () => ({
57 |     data: { completionRate: 100, totalStudents: 30, completedOnboarding: 30 },
58 |   }),
59 | }));
60 |
61 | vi.mock("@/hooks/useAIPerformance", () => ({
62 |   useAIPerformance: () => ({
63 |     data: {
64 |       suggestionAcceptanceRate: 75,
65 |       suggestionTotal: 100,
66 |       predictionAccuracyRate: 80,
67 |       predictionTotal: 50,
68 |       draftAcceptanceRate: 60,
69 |       draftTotal: 20,
70 |     },
71 |     isLoading: false,
72 |   }),
73 | }));
74 |
75 | vi.mock("@/hooks/useTeacherDashboard", () => ({
76 |   useTeacherKPIs: () => ({
77 |     data: {
78 |       pendingSubmissions: 5,
79 |       gradedThisWeek: 12,
80 |       avgAttainment: 72,
81 |       atRiskCount: 3,
82 |     },
83 |     isLoading: false,
84 |   }),
85 |   useTeacherCLOAttainment: () => ({ data: [], isLoading: false }),
86 |   useTeacherBloomsDistribution: () => ({ data: [], isLoading: false }),
87 |   useStudentPerformanceHeatmap: () => ({ data: [], isLoading: false }),
88 |   useAtRiskStudents: () => ({ data: [], isLoading: false }),
89 |   useSendNudge: () => ({ mutate: vi.fn(), isPending: false }),
90 | }));
91 |
92 | vi.mock("@/hooks/useCourses", () => ({
93 |   useCourses: () => ({ data: { data: [] }, isLoading: false }),
94 |   useTeacherCourses: () => ({ data: { data: [] }, isLoading: false }),
95 | }));
96 |
97 | vi.mock("@/hooks/useSubmissions", () => ({
98 |   usePendingSubmissions: () => ({ data: [], isLoading: false }),
99 | }));
100 |
101 | vi.mock("@/hooks/useRealtime", () => ({
102 |   useRealtime: () => ({ isLive: true }),
103 | }));
104 |
105 | vi.mock("@/hooks/useStudentDashboard", () => ({
106 |   useStudentKPIs: () => ({
107 |     data: {
108 |       enrolledCourses: 4,
109 |       completedAssignments: 8,
110 |       avgAttainment: 78,

```



```

111 |         currentStreak: 5,
112 |         totalXP: 1200,
113 |         currentLevel: 3,
114 |     },
115 |     isLoading: false,
116 | },
117 | useUpcomingDeadlines: () => ({ data: [], isLoading: false }),
118 | }));
119 |
120 | vi.mock("@/hooks/useStudentProfile", () => ({
121 |     useStudentProfile: () => ({ data: null }),
122 | }));
123 |
124 | vi.mock("@/hooks/useMicroAssessments", () => ({
125 |     useTodayMicroAssessment: () => ({ data: null }),
126 |     useCompleteMicroAssessment: () => ({ mutate: vi.fn() }),
127 |     useDismissMicroAssessment: () => ({ mutate: vi.fn() }),
128 | }));
129 |
130 | vi.mock("@/hooks/useProfileCompleteness", () => ({
131 |     useProfileCompleteness: () => ({
132 |         data: { profile_completeness: 100, day1_completed: true },
133 |     }),
134 | }));
135 |
136 | vi.mock("@/hooks/useStarterWeekPlan", () => ({
137 |     useStarterWeekSessions: () => ({ data: [] }),
138 | }));
139 |
140 | vi.mock("@/hooks/useOnboardingProgress", () => ({
141 |     useOnboardingProgress: () => ({ data: { skipped_sections: [] } }),
142 | }));
143 |
144 | // -----
145 | // Placeholder test – original test bodies were removed along with unused
146 | // imports during a cleanup pass. Add real extraction tests when ready.
147 | // TODO: Restore i18n extraction tests for dashboard pages
148 | // -----
149 | import { describe, it, expect } from "vitest";
150 |
151 | describe("i18n extraction (placeholder)", () => {
152 |     it("should have i18n initialized", () => {
153 |         expect(true).toBe(true);
154 |     });
155 | });
156 |

```

FILE: src/__tests__/unit/improvementBonus.test.ts

TYPE: .ts | LINES: 1-75 | SHA256: 0aae1fe3ef58a626eb32e9f43800bb7d1ea8cdf68dddaaf4e8fbad2b58638f

```

1 | import { describe, it, expect } from "vitest";
2 | import { improvementBonusSchema } from "@/lib/schemas/improvementBonus";
3 |
4 | describe("Improvement Bonus Schema", () => {
5 |     it("validates a correct improvement bonus payload", () => {
6 |         const result = improvementBonusSchema.safeParse({
7 |             clo_id: "550e8400-e29b-41d4-a716-446655440000",
8 |             previous_percent: 45,
9 |             current_percent: 72,
10 |             bonus_xp: 50,
11 |         });
12 |         expect(result.success).toBe(true);
13 |     });
14 |
15 |     it("rejects non-UUID clo_id", () => {
16 |         const result = improvementBonusSchema.safeParse({
17 |             clo_id: "not-a-uuid",
18 |             previous_percent: 45,
19 |             current_percent: 72,
20 |             bonus_xp: 50,
21 |         });
22 |         expect(result.success).toBe(false);
23 |     });
24 |
25 |     it("rejects previous_percent below 0", () => {
26 |         const result = improvementBonusSchema.safeParse({
27 |             clo_id: "550e8400-e29b-41d4-a716-446655440000",
28 |             previous_percent: -5,
29 |             current_percent: 72,
30 |             bonus_xp: 50,
31 |         });
32 |         expect(result.success).toBe(false);
33 |     });
34 |
35 |     it("rejects current_percent above 100", () => {
36 |         const result = improvementBonusSchema.safeParse({
37 |             clo_id: "550e8400-e29b-41d4-a716-446655440000",
38 |             previous_percent: 45,
39 |             current_percent: 105,
40 |             bonus_xp: 50,
41 |         });
42 |         expect(result.success).toBe(false);
43 |     });
44 |
45 |     it("rejects non-positive bonus_xp", () => {
46 |         const result = improvementBonusSchema.safeParse({
47 |             clo_id: "550e8400-e29b-41d4-a716-446655440000",
48 |             previous_percent: 45,
49 |             current_percent: 72,
50 |             bonus_xp: 0,
51 |         });
52 |         expect(result.success).toBe(false);
53 |     });
54 |

```

```

55 | it("rejects non-integer bonus_xp", () => {
56 |   const result = improvementBonusSchema.safeParse({
57 |     clo_id: "550e8400-e29b-41d4-a716-446655440000",
58 |     previous_percent: 45,
59 |     current_percent: 72,
60 |     bonus_xp: 50.5,
61 |   });
62 |   expect(result.success).toBe(false);
63 | });
64 |
65 | it("accepts boundary values (0 and 100 for percents)", () => {
66 |   const result = improvementBonusSchema.safeParse({
67 |     clo_id: "550e8400-e29b-41d4-a716-446655440000",
68 |     previous_percent: 0,
69 |     current_percent: 100,
70 |     bonus_xp: 50,
71 |   });
72 |   expect(result.success).toBe(true);
73 | });
74 | });
75 |

```

FILE: [src/__tests__/unit/improvementBonusCelebration.test.tsx](#)

TYPE: .tsx | LINES: 1-73 | SHA256: f416d12d0b5cbb4e5c365f81988d28f0aeb35a1c50c04b96aec03f34896e85f2

```

1 | import { describe, it, expect, vi, beforeEach } from "vitest";
2 | import { render, screen, act } from "@testing-library/react";
3 | import ImprovementBonusCelebration from "@components/shared/ImprovementBonusCelebration";
4 |
5 | vi.mock("framer-motion", () => ({
6 |   motion: {
7 |     div: (p: Record<string, unknown>) => {
8 |       const { children, initial, animate, exit, transition, ...rest } = p;
9 |       void initial;
10 |      void animate;
11 |      void exit;
12 |      void transition;
13 |      return <div {...rest}>{children as React.ReactNode}</div>;
14 |    },
15 |  },
16 |   AnimatePresence: (p: { children: React.ReactNode }) => <>{p.children}</>,
17 |   useReducedMotion: () => false,
18 | }));
19 | vi.mock("canvas-confetti", () => ({ default: vi.fn() }));
20 | beforeEach(() => {
21 |   vi.useFakeTimers();
22 | });
23 |
24 | const props = {
25 |   cloTitle: "DS",
26 |   bonusXP: 50,
27 |   previousPercent: 45,
28 |   currentPercent: 72,
29 |   onDismiss: vi.fn(),
30 | };
31 |
32 | describe("ImprovementBonusCelebration", () => {
33 |   it("renders CLO title", () => {
34 |     render(<ImprovementBonusCelebration {...props} />);
35 |     expect(screen.getByText(/Great improvement on DS!/)).toBeInTheDocument();
36 |   });
37 |
38 |   it("renders bonus XP", () => {
39 |     render(<ImprovementBonusCelebration {...props} />);
40 |     expect(screen.getByText("+50 XP Bonus")).toBeInTheDocument();
41 |   });
42 |
43 |   it("shows heading", () => {
44 |     render(<ImprovementBonusCelebration {...props} />);
45 |     expect(screen.getByText("Great improvement!")).toBeInTheDocument();
46 |   });
47 |
48 |   it("shows percentages", () => {
49 |     render(<ImprovementBonusCelebration {...props} />);
50 |     expect(screen.getByText(/45%.*72%/)).toBeInTheDocument();
51 |   });
52 |
53 |   it("fires confetti", async () => {
54 |     const c = await import("canvas-confetti");
55 |     render(<ImprovementBonusCelebration {...props} />);
56 |     expect(c.default).toHaveBeenCalled();
57 |   });
58 |
59 |   it("dismisses after timeout", () => {
60 |     const d = vi.fn();
61 |     render(<ImprovementBonusCelebration {...props} onDismiss={d} />);
62 |     act(() => {
63 |       vi.advanceTimersByTime(3000);
64 |     });
65 |     expect(d).toHaveBeenCalled();
66 |   });
67 |
68 |   it("has role=status", () => {
69 |     render(<ImprovementBonusCelebration {...props} />);
70 |     expect(screen.getByRole("status")).toBeInTheDocument();
71 |   });
72 | });
73 |

```

FILE: src/__tests__/unit/institutionSettingsSchema.test.ts

TYPE: .ts | LINES: 1-128 | SHA256: 3b69ba71b31c899fa43a3ae5bce62f97b21e6c465a80d2c8a481105506fe63a2

```
1 | import { describe, it, expect } from "vitest";
2 | import {
3 |   institutionSettingsSchema,
4 |   gradeScaleSchema,
5 | } from "@lib/schemas/institutionSettings";
6 |
7 | describe("gradeScaleSchema", () => {
8 |   it("accepts a valid grade scale entry", () => {
9 |     const result = gradeScaleSchema.safeParse({
10 |       letter: "A",
11 |       min_percent: 85,
12 |       max_percent: 100,
13 |       gpa_points: 4.0,
14 |     });
15 |     expect(result.success).toBe(true);
16 |   });
17 |
18 |   it("rejects empty letter", () => {
19 |     const result = gradeScaleSchema.safeParse({
20 |       letter: "",
21 |       min_percent: 85,
22 |       max_percent: 100,
23 |       gpa_points: 4.0,
24 |     });
25 |     expect(result.success).toBe(false);
26 |   });
27 |
28 |   it("rejects gpa_points above 4", () => {
29 |     const result = gradeScaleSchema.safeParse({
30 |       letter: "A+",
31 |       min_percent: 90,
32 |       max_percent: 100,
33 |       gpa_points: 4.5,
34 |     });
35 |     expect(result.success).toBe(false);
36 |   });
37 |
38 |   it("rejects negative percentages", () => {
39 |     const result = gradeScaleSchema.safeParse({
40 |       letter: "F",
41 |       min_percent: -1,
42 |       max_percent: 49,
43 |       gpa_points: 0,
44 |     });
45 |     expect(result.success).toBe(false);
46 |   });
47 | });
48 |
49 | describe("institutionSettingsSchema", () => {
50 |   const validSettings = {
51 |     attainment_thresholds: { excellent: 85, satisfactory: 70, developing: 50 },
52 |     success_threshold: 70,
53 |     accreditation_body: "HEC" as const,
54 |     grade_scales: [
55 |       { letter: "A", min_percent: 85, max_percent: 100, gpa_points: 4.0 },
56 |       { letter: "B", min_percent: 70, max_percent: 84, gpa_points: 3.0 },
57 |     ],
58 |     streak_sabbatical_enabled: false,
59 |   };
60 |
61 |   it("accepts valid institution settings", () => {
62 |     const result = institutionSettingsSchema.safeParse(validSettings);
63 |     expect(result.success).toBe(true);
64 |   });
65 |
66 |   it("accepts all valid accreditation bodies", () => {
67 |     const bodies = ["HEC", "QQA", "ABET", "NCAA", "AACSB", "Generic"] as const;
68 |     for (const body of bodies) {
69 |       const result = institutionSettingsSchema.safeParse({
70 |         ...validSettings,
71 |         accreditation_body: body,
72 |       });
73 |       expect(result.success).toBe(true);
74 |     }
75 |   });
76 |
77 |   it("rejects invalid accreditation body", () => {
78 |     const result = institutionSettingsSchema.safeParse({
79 |       ...validSettings,
80 |       accreditation_body: "INVALID",
81 |     });
82 |     expect(result.success).toBe(false);
83 |   });
84 |
85 |   it("rejects thresholds above 100", () => {
86 |     const result = institutionSettingsSchema.safeParse({
87 |       ...validSettings,
88 |       attainment_thresholds: {
89 |         excellent: 101,
90 |         satisfactory: 70,
91 |         developing: 50,
92 |       },
93 |     });
94 |     expect(result.success).toBe(false);
95 |   });
96 |
97 |   it("rejects negative thresholds", () => {
98 |     const result = institutionSettingsSchema.safeParse({
99 |       ...validSettings,
100 |      attainment_thresholds: {
101 |        excellent: 85,
```



```

74 |         onPurchase={vi.fn()}
75 |     />
76 | );
77 |
78 | expect(screen.getByText("Owned")).toBeTruthy();
79 | expect(
80 |     screen.getByRole("button", { name: /Already Owned/i })
81 | ).toBeDisabled();
82 | });
83 |
84 | it("shows sale price with strikethrough when sale is active", () => {
85 |     render(
86 |         <ItemCard
87 |             item={makeItem({
88 |                 xp_price: 500,
89 |                 effective_price: 375,
90 |                 sale_discount: 25,
91 |             })}
92 |             studentLevel={10}
93 |             isOwned={false}
94 |             onPurchase={vi.fn()}
95 |         />
96 |     );
97 |
98 |     // Original price with strikethrough
99 |     expect(screen.getByText("500")).toBeTruthy();
100 |     // Discounted price
101 |     expect(screen.getByText(/375 XP/)).toBeTruthy();
102 | });
103 |
104 | it("shows out-of-stock state for limited items with zero stock", () => {
105 |     render(
106 |         <ItemCard
107 |             item={makeItem({ stock_type: "limited", stock_quantity: 0 })}
108 |             studentLevel={10}
109 |             isOwned={false}
110 |             onPurchase={vi.fn()}
111 |         />
112 |     );
113 |
114 |     expect(
115 |         screen.getByRole("button", { name: /Out of Stock/i })
116 |     ).toBeDisabled();
117 | });
118 |
119 | it("shows remaining stock for limited items", () => {
120 |     render(
121 |         <ItemCard
122 |             item={makeItem({ stock_type: "limited", stock_quantity: 5 })}
123 |             studentLevel={10}
124 |             isOwned={false}
125 |             onPurchase={vi.fn()}
126 |         />
127 |     );
128 |
129 |     expect(screen.getByText("5 remaining")).toBeTruthy();
130 | });
131 |
132 | it("calls onPurchase when Buy Now is clicked", async () => {
133 |     const onPurchase = vi.fn();
134 |     const item = makeItem();
135 |     render(
136 |         <ItemCard
137 |             item={item}
138 |             studentLevel={10}
139 |             isOwned={false}
140 |             onPurchase={onPurchase}
141 |         />
142 |     );
143 |
144 |     const buyBtn = screen.getByRole("button", { name: /Buy Now/i });
145 |     await userEvent.click(buyBtn);
146 |     expect(onPurchase).toHaveBeenCalled();
147 | });
148 |
149 | it("renders non-cosmetic items with icon instead of preview", () => {
150 |     render(
151 |         <ItemCard
152 |             item={makeItem({
153 |                 category: "power_up",
154 |                 sub_category: "xp_boost",
155 |                 icon_identifier: "zap",
156 |             })}
157 |             studentLevel={10}
158 |             isOwned={false}
159 |             onPurchase={vi.fn()}
160 |         />
161 |     );
162 |
163 |     expect(screen.getByText("Ocean Blue Theme")).toBeTruthy();
164 | });
165 | });
166 |

```

FILE: src/__tests__/unit/journalEditor.test.tsx

TYPE: .tsx | LINES: 1-257 | SHA256: 6eacb5ba7d8c74fa44364d9986946e7f4c09e2df885e0ce478d3150aed3669a6

```

1 | // =====
2 | // JournalEditor - Unit tests
3 | // =====
4 |
5 | import { describe, it, expect, vi, beforeEach } from "vitest";
6 | import { render, screen, fireEvent } from "@testing-library/react";
7 | import userEvent from "@testing-library/user-event";

```

[illegible]

[illegible]

```

216 |     expect(screen.getByText("Reflection Prompt")).toBeInTheDocument();
217 |     expect(
218 |       screen.getByText("Reflect on your understanding of derivatives.")
219 |     ).toBeInTheDocument();
220 |     expect(screen.getByText("Reflective Observation")).toBeInTheDocument();
221 |     expect(screen.getByText("Abstract Conceptualization")).toBeInTheDocument();
222 |   });
223 |
224 |   it("dismisses prompt when dismiss button is clicked", async () => {
225 |     const user = userEvent.setup();
226 |     renderEditor();
227 |
228 |     // Select course and CLO to trigger prompt
229 |     const courseTrigger = screen.getByText("Select a course").closest("button");
230 |     fireEvent.click(courseTrigger!);
231 |     const mathOptions = screen.getAllByText("Math 101");
232 |     const mathItem = mathOptions.find((el) =>
233 |       el.closest('[data-slot="select-item"]')
234 |     );
235 |     fireEvent.click(mathItem ?? mathOptions[0]!);
236 |
237 |     const cloTrigger = screen
238 |       .getByText("Select a CLO for guided prompts")
239 |       .closest("button");
240 |     fireEvent.click(cloTrigger!);
241 |     const cloOptions = screen.getAllByText("Understand derivatives");
242 |     const cloItem = cloOptions.find((el) =>
243 |       el.closest('[data-slot="select-item"]')
244 |     );
245 |     fireEvent.click(cloItem ?? cloOptions[0]!);
246 |
247 |     // Prompt should be visible
248 |     expect(screen.getByText("Reflection Prompt")).toBeInTheDocument();
249 |
250 |     // Dismiss it
251 |     await user.click(screen.getByLabelText("Dismiss prompt"));
252 |
253 |     // Prompt should be gone
254 |     expect(screen.queryByText("Reflection Prompt")).not.toBeInTheDocument();
255 |   });
256 | });
257 |

```

FILE: src/__tests__/unit/journalInsights.test.ts

TYPE: .ts | LINES: 1-48 | SHA256: f0e55f5e6f4355e43944c8176e7677c5e0f6096ece3cae72de046c04f888ac95

```

1 | // =====
2 | // Journal insight derivation – presentation metrics must never use demo values.
3 | // =====
4 |
5 | import { describe, expect, it } from "vitest";
6 |
7 | import {
8 |   computeJournalStreak,
9 |   getJournalInsights,
10 | } from "@lib/journalInsights";
11 |
12 | const now = new Date("2026-07-28T12:00:00.000Z"); // Tuesday
13 |
14 | describe("journalInsights", () => {
15 |   it("derives the consecutive streak from real entry dates", () => {
16 |     expect(
17 |       computeJournalStreak(
18 |         ["2026-07-28T08:00:00.000Z", "2026-07-27T08:00:00.000Z"],
19 |         now
20 |       )
21 |     ).toBe(2);
22 |   });
23 |
24 |   it("allows yesterday's reflection to keep a streak alive", () => {
25 |     expect(computeJournalStreak(["2026-07-27T08:00:00.000Z"], now)).toBe(1);
26 |   });
27 |
28 |   it("counts this week's entries, substantive reflections, and calendar days", () => {
29 |     const longReflection = Array.from({ length: 50 }, () => "reflect").join(
30 |       " "
31 |     );
32 |     const insights = getJournalInsights(
33 |       [
34 |         { created_at: "2026-07-27T08:00:00.000Z", content: longReflection },
35 |         { created_at: "2026-07-27T12:00:00.000Z", content: "A short note" },
36 |         { created_at: "2026-07-20T08:00:00.000Z", content: longReflection },
37 |       ],
38 |       now
39 |     );
40 |
41 |     expect(insights.entriesThisWeek).toBe(2);
42 |     expect(insights.substantiveThisWeek).toBe(1);
43 |     expect(insights.totalEntries).toBe(3);
44 |     // Two entries on Monday still represent one reflected-on calendar day.
45 |     expect(insights.days.filter((day) => day.hasEntry)).toHaveLength(1);
46 |   });
47 | });
48 |

```

FILE: src/__tests__/unit/journalListPage.test.tsx

TYPE: .tsx | LINES: 1-163 | SHA256: b1866b7af5e5ff8b22af781a93b78c0e1379452aa8957c831d3769f4a4f8c638

```

1 | // =====
2 | // JournalListPage – Unit tests
3 | // =====
4 |
5 | import { describe, it, expect, vi, beforeEach } from "vitest";
6 | import { render, screen } from "@testing-library/react";

```


[illegible]

```

111 |
112 | it("renders the New Entry button", () => {
113 |   renderPage();
114 |   expect(screen.getByText("New Entry")).toBeInTheDocument();
115 | });
116 |
117 | it("renders journal entry cards with content preview", () => {
118 |   renderPage();
119 |   expect(screen.getByText(/This is my first reflection/)).toBeInTheDocument();
120 |   expect(screen.getByText(/I shared this entry/)).toBeInTheDocument();
121 | });
122 |
123 | it("shows course name badges on entries", () => {
124 |   renderPage();
125 |   expect(screen.getByText("Math 101")).toBeInTheDocument();
126 |   expect(screen.getByText("Physics 201")).toBeInTheDocument();
127 | });
128 |
129 | it("shows Shared badge for shared entries", () => {
130 |   renderPage();
131 |   expect(screen.getByText("Shared")).toBeInTheDocument();
132 | });
133 |
134 | it("shows word count for each entry", () => {
135 |   renderPage();
136 |   // First entry has ~20 words
137 |   const wordCounts = screen.getAllByText(/d+ words/);
138 |   expect(wordCounts.length).toBe(2);
139 | });
140 |
141 | it("navigates to new entry on button click", async () => {
142 |   const user = userEvent.setup();
143 |   renderPage();
144 |   await user.click(screen.getByText("New Entry"));
145 |   expect(mockNavigate).toHaveBeenCalledWith("/student/journal/new");
146 | });
147 |
148 | it("navigates to entry editor on card click", async () => {
149 |   const user = userEvent.setup();
150 |   renderPage();
151 |   const firstCard = screen
152 |     .getByText(/This is my first reflection/)
153 |     .closest('[role="button"]');
154 |   if (firstCard) await user.click(firstCard);
155 |   expect(mockNavigate).toHaveBeenCalledWith("/student/journal/j1");
156 | });
157 |
158 | it("renders course filter dropdown", () => {
159 |   renderPage();
160 |   expect(screen.getByText("All Courses")).toBeInTheDocument();
161 | });
162 | });
163 |

```

FILE: [src/__tests__/unit/journalPromptGenerator.test.ts](#)

TYPE: .ts | LINES: 1-158 | SHA256: 3a4b24c80dd35a46a088877a0ded8a46f02d45ff7c2d773fdb7234b7f73997d2

```

1 | import { describe, it, expect } from "vitest";
2 | import {
3 |   generateJournalPrompt,
4 |   KOLB_STAGES,
5 |   type JournalPromptContext,
6 |   type KolbStage,
7 | } from "@lib/journalPromptGenerator";
8 | import type { BloomsLevel } from "@lib/schemas/clo";
9 | import type { AttainmentLevel } from "@types/app";
10 |
11 | describe("generateJournalPrompt", () => {
12 |   const baseContext: JournalPromptContext = {
13 |     cloTitle: "Apply sorting algorithms to real datasets",
14 |     bloomsLevel: "applying",
15 |     attainmentLevel: "Satisfactory",
16 |   };
17 |
18 |   it("returns promptText and questions array", () => {
19 |     const result = generateJournalPrompt(baseContext);
20 |     expect(result).toHaveProperty("promptText");
21 |     expect(result).toHaveProperty("questions");
22 |     expect(typeof result.promptText).toBe("string");
23 |     expect(Array.isArray(result.questions)).toBe(true);
24 |   });
25 |
26 |   it("generates 3 questions for Excellent attainment", () => {
27 |     const result = generateJournalPrompt({
28 |       ...baseContext,
29 |       attainmentLevel: "Excellent",
30 |     });
31 |     expect(result.questions).toHaveLength(3);
32 |   });
33 |
34 |   it("generates 3 questions for Satisfactory attainment", () => {
35 |     const result = generateJournalPrompt(baseContext);
36 |     expect(result.questions).toHaveLength(3);
37 |   });
38 |
39 |   it("generates 4 questions for Developing attainment", () => {
40 |     const result = generateJournalPrompt({
41 |       ...baseContext,
42 |       attainmentLevel: "Developing",
43 |     });
44 |     expect(result.questions).toHaveLength(4);
45 |   });
46 |
47 |   it("generates 4 questions for Not_Yet attainment", () => {

```

```

48 |     const result = generateJournalPrompt({
49 |       ...baseContext,
50 |       attainmentLevel: "Not_Yet",
51 |     });
52 |     expect(result.questions).toHaveLength(4);
53 |   });
54 |
55 |   it("includes CLO title in promptText", () => {
56 |     const result = generateJournalPrompt(baseContext);
57 |     expect(result.promptText).toContain(baseContext.cloTitle);
58 |   });
59 |
60 |   it("includes rubric feedback summary when provided", () => {
61 |     const result = generateJournalPrompt({
62 |       ...baseContext,
63 |       rubricFeedbackSummary: "Good use of examples but needs more analysis",
64 |     });
65 |     expect(result.promptText).toContain(
66 |       "Good use of examples but needs more analysis"
67 |     );
68 |   });
69 |
70 |   it("does not include feedback text when rubricFeedbackSummary is omitted", () => {
71 |     const result = generateJournalPrompt(baseContext);
72 |     expect(result.promptText).not.toContain("Your teacher noted");
73 |   });
74 |
75 |   it("each question has a valid Kolb stage", () => {
76 |     const result = generateJournalPrompt(baseContext);
77 |     for (const q of result.questions) {
78 |       expect(KOLB_STAGES).toContain(q.stage);
79 |     }
80 |   });
81 |
82 |   it("each question references the CLO title", () => {
83 |     const result = generateJournalPrompt(baseContext);
84 |     for (const q of result.questions) {
85 |       expect(q.question).toContain(baseContext.cloTitle);
86 |     }
87 |   });
88 |
89 |   it("includes Bloom's level label in promptText", () => {
90 |     const result = generateJournalPrompt(baseContext);
91 |     expect(result.promptText).toContain("Applying");
92 |   });
93 |
94 |   it("works for all Bloom's levels", () => {
95 |     const levels: BloomsLevel[] = [
96 |       "remembering",
97 |       "understanding",
98 |       "applying",
99 |       "analyzing",
100 |       "evaluating",
101 |       "creating",
102 |     ];
103 |     for (const level of levels) {
104 |       const result = generateJournalPrompt({
105 |         ...baseContext,
106 |         bloomsLevel: level,
107 |       });
108 |       expect(result.questions.length).toBeGreaterThanOrEqual(3);
109 |       expect(result.questions.length).toBeLessThanOrEqual(4);
110 |       expect(result.promptText.length).toBeGreaterThan(0);
111 |     }
112 |   });
113 |
114 |   it("works for all attainment levels", () => {
115 |     const levels: AttainmentLevel[] = [
116 |       "Excellent",
117 |       "Satisfactory",
118 |       "Developing",
119 |       "Not_Yet",
120 |     ];
121 |     for (const level of levels) {
122 |       const result = generateJournalPrompt({
123 |         ...baseContext,
124 |         attainmentLevel: level,
125 |       });
126 |       expect(result.questions.length).toBeGreaterThanOrEqual(3);
127 |       expect(result.promptText.length).toBeGreaterThan(0);
128 |     }
129 |   });
130 |
131 |   it("Developing/Not_Yet include all 4 Kolb stages", () => {
132 |     const stages = new Set<KolbStage>();
133 |     const result = generateJournalPrompt({
134 |       ...baseContext,
135 |       attainmentLevel: "Developing",
136 |     });
137 |     for (const q of result.questions) {
138 |       stages.add(q.stage);
139 |     }
140 |     expect(stages.size).toBe(4);
141 |     for (const s of KOLB_STAGES) {
142 |       expect(stages.has(s)).toBe(true);
143 |     }
144 |   });
145 |
146 |   it("Excellent/Satisfactory skip Concrete Experience stage", () => {
147 |     const result = generateJournalPrompt({
148 |       ...baseContext,
149 |       attainmentLevel: "Excellent",
150 |     });
151 |     const stages = result.questions.map((q) => q.stage);

```

```

152 |         expect(stages).not.toContain("Concrete Experience");
153 |         expect(stages).toContain("Reflective Observation");
154 |         expect(stages).toContain("Abstract Conceptualization");
155 |         expect(stages).toContain("Active Experimentation");
156 |     });
157 | });
158 |

```

FILE: src/__tests__/unit/keepPreviousDataPagination.test.ts

TYPE: .ts | LINES: 1-115 | SHA256: 7181b4c1bd129ccbf9a99d7fc4965d1bbf860cbf622e19732666cea458b94574

[illegible]

```

94 |     resultQueue.push(
95 |       new Promise<QueryResult>((resolve) => {
96 |         resolvePage2 = resolve;
97 |       })
98 |     );
99 |
100 |    // Change the page !' new query key. With keepPreviousData the prior rows
101 |    // remain as placeholder data instead of flashing empty.
102 |    rerender({ page: 2 });
103 |
104 |    await waitFor(() => expect(result.current.isPlaceholderData).toBe(true));
105 |    // Prior page's rows are still the rendered data during the fetch.
106 |    expect(result.current.data?.data).toEqual(PAGE_1);
107 |
108 |    // Resolve page 2 !' rows swap, placeholder flag clears, semantics unchanged.
109 |    resolvePage2({ data: PAGE_2, error: null, count: 50 });
110 |
111 |    await waitFor(() => expect(result.current.data?.data).toEqual(PAGE_2));
112 |    expect(result.current.isPlaceholderData).toBe(false);
113 |  });
114 | });
115 |

```

FILE: [src/___tests___/unit/knowledgeQuestManager.test.tsx](#)

TYPE: .tsx | LINES: 1-107 | SHA256: a81df7f8f5884098afc16c90c263f8a3be4a3fbe6dacbeee1b889d404ad124aa

```

1 | // =====
2 | // Unit Test: Knowledge Quest Manager
3 | // Task 26.5 – Quest CRUD form, date validation
4 | // =====
5 |
6 | import { describe, it, expect } from "vitest";
7 | import { createKnowledgeQuestSchema } from "@lib/marketplaceSchemas";
8 |
9 | describe("Knowledge Quest Manager", () => {
10 |   describe("createKnowledgeQuestSchema", () => {
11 |     it("validates a valid XP reward quest", () => {
12 |       const now = new Date();
13 |       const future = new Date(now.getTime() + 7 * 24 * 60 * 60 * 1000);
14 |
15 |       const result = createKnowledgeQuestSchema.safeParse({
16 |         title: "Weekly Challenge",
17 |         description: "Complete 5 quizzes this week",
18 |         quest_type: "quiz_challenge",
19 |         target_clo_ids: [],
20 |         start_date: now.toISOString(),
21 |         end_date: future.toISOString(),
22 |         reward_type: "xp",
23 |         reward_item_id: null,
24 |         reward_xp_amount: 100,
25 |       });
26 |
27 |       expect(result.success).toBe(true);
28 |     });
29 |
30 |     it("validates a valid item reward quest", () => {
31 |       const now = new Date();
32 |       const future = new Date(now.getTime() + 7 * 24 * 60 * 60 * 1000);
33 |
34 |       const result = createKnowledgeQuestSchema.safeParse({
35 |         title: "Content Creator",
36 |         description: "Create a study plan",
37 |         quest_type: "content_creation",
38 |         target_clo_ids: [],
39 |         start_date: now.toISOString(),
40 |         end_date: future.toISOString(),
41 |         reward_type: "item",
42 |         reward_item_id: "550e8400-e29b-41d4-a716-446655440000",
43 |         reward_xp_amount: null,
44 |       });
45 |
46 |       expect(result.success).toBe(true);
47 |     });
48 |
49 |     it("rejects quest with end_date before start_date", () => {
50 |       const now = new Date();
51 |       const past = new Date(now.getTime() - 7 * 24 * 60 * 60 * 1000);
52 |
53 |       const result = createKnowledgeQuestSchema.safeParse({
54 |         title: "Bad Quest",
55 |         description: "Invalid dates",
56 |         quest_type: "quiz_challenge",
57 |         target_clo_ids: [],
58 |         start_date: now.toISOString(),
59 |         end_date: past.toISOString(),
60 |         reward_type: "xp",
61 |         reward_item_id: null,
62 |         reward_xp_amount: 50,
63 |       });
64 |
65 |       expect(result.success).toBe(false);
66 |     });
67 |
68 |     it("rejects quest with empty title", () => {
69 |       const now = new Date();
70 |       const future = new Date(now.getTime() + 7 * 24 * 60 * 60 * 1000);
71 |
72 |       const result = createKnowledgeQuestSchema.safeParse({
73 |         title: "",
74 |         description: "Some description",
75 |         quest_type: "quiz_challenge",
76 |         target_clo_ids: [],
77 |         start_date: now.toISOString(),
78 |         end_date: future.toISOString(),

```

```

79 |         reward_type: "xp",
80 |         reward_item_id: null,
81 |         reward_xp_amount: 50,
82 |     });
83 |
84 |     expect(result.success).toBe(false);
85 | });
86 |
87 | it("rejects item reward without reward_item_id", () => {
88 |     const now = new Date();
89 |     const future = new Date(now.getTime() + 7 * 24 * 60 * 60 * 1000);
90 |
91 |     const result = createKnowledgeQuestSchema.safeParse({
92 |         title: "Quest",
93 |         description: "Description",
94 |         quest_type: "quiz_challenge",
95 |         target_clo_ids: [],
96 |         start_date: now.toISOString(),
97 |         end_date: future.toISOString(),
98 |         reward_type: "item",
99 |         reward_item_id: null,
100 |         reward_xp_amount: null,
101 |     });
102 |
103 |     expect(result.success).toBe(false);
104 | });
105 | });
106 | });
107 |

```

FILE: [src/__tests__/unit/languageSwitcher.test.tsx](#)

TYPE: .tsx | LINES: 1-65 | SHA256: 396c48553330717470320e8d1f2f603c5a07ec0d7548e783a74a1adca6d9cdc0

```

1 | import { describe, it, expect, vi, beforeEach } from "vitest";
2 | import { render, screen } from "@testing-library/react";
3 | import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
4 | import { I18nextProvider } from "react-i18next";
5 | import i18n from "@lib/i18n";
6 | import { LanguageSwitcher } from "@components/shared/LanguageSwitcher";
7 |
8 | // Mock useAuth
9 | vi.mock("@/hooks/useAuth", () => ({
10 |     useAuth: () => ({ user: { id: "test-user" } }),
11 | }));
12 |
13 | // Mock useLanguagePreference
14 | const mockMutate = vi.fn();
15 | vi.mock("@/hooks/useLanguagePreference", () => ({
16 |     useUpdateLanguagePreference: () => ({ mutate: mockMutate }),
17 | }));
18 |
19 | const createWrapper = () => {
20 |     const queryClient = new QueryClient({
21 |         defaultOptions: { queries: { retry: false } },
22 |     });
23 |     return ({ children }: { children: React.ReactNode }) => (
24 |         <QueryClientProvider client={queryClient}>
25 |             <I18nextProvider i18n={i18n}>{children}</I18nextProvider>
26 |         </QueryClientProvider>
27 |     );
28 | };
29 |
30 | describe("LanguageSwitcher", () => {
31 |     beforeEach(() => {
32 |         i18n.changeLanguage("en");
33 |         localStorage.clear();
34 |         mockMutate.mockClear();
35 |     });
36 |
37 |     it("renders the language switcher button with current language", () => {
38 |         render(<LanguageSwitcher />, { wrapper: createWrapper() });
39 |         const button = screen.getByRole("button");
40 |         expect(button).toBeInTheDocument();
41 |         expect(button.textContent).toContain("English");
42 |     });
43 |
44 |     it("displays Globe icon", () => {
45 |         render(<LanguageSwitcher />, { wrapper: createWrapper() });
46 |         const svg = screen.getByRole("button").querySelector("svg");
47 |         expect(svg).toBeInTheDocument();
48 |     });
49 |
50 |     it("handleLanguageChange updates localStorage and calls i18n.changeLanguage", () => {
51 |         // Test the underlying logic directly
52 |         i18n.changeLanguage("ar");
53 |         localStorage.setItem("edevisee-language", "ar");
54 |         expect(i18n.language).toBe("ar");
55 |         expect(localStorage.getItem("edevisee-language")).toBe("ar");
56 |     });
57 |
58 |     it("shows Arabic label when language is Arabic", () => {
59 |         i18n.changeLanguage("ar");
60 |         render(<LanguageSwitcher />, { wrapper: createWrapper() });
61 |         const button = screen.getByRole("button");
62 |         expect(button.textContent).toContain("بديل اللغة");
63 |     });
64 | });
65 |

```

TYPE: .tsx | LINES: 1-442 | SHA256: 88515c321895f3dea3868b50b31943e2ad1cf3f50586a58f2257d709108ca3e5

```

102 |     weekStart: "2025-07-21",
103 |     weekLabel: "Week 8",
104 |     xp: 130,
105 |     isCurrentWeek: true,
106 |     isPersonalBest: false,
107 |   },
108 | ];
109 |
110 | const mockMostImproved = [
111 |   {
112 |     student_id: "u1",
113 |     student_name: "Alice",
114 |     current_4_week_xp: 500,
115 |     previous_4_week_xp: 200,
116 |     improvement_percent: 150,
117 |     xp_delta: 300,
118 |   },
119 |   {
120 |     student_id: "u2",
121 |     student_name: "Bob",
122 |     current_4_week_xp: 400,
123 |     previous_4_week_xp: 250,
124 |     improvement_percent: 60,
125 |     xp_delta: 150,
126 |   },
127 |   {
128 |     student_id: "current-user",
129 |     student_name: "Me",
130 |     current_4_week_xp: 300,
131 |     previous_4_week_xp: 200,
132 |     improvement_percent: 50,
133 |     xp_delta: 100,
134 |   },
135 | ];
136 |
137 | vi.mock("@/hooks/useAuth", () => ({
138 |   useAuth: () => ({
139 |     user: { id: "current-user" },
140 |     profile: { role: "student", institution_id: "inst-1" },
141 |     role: "student",
142 |     institutionId: "inst-1",
143 |   }),
144 | }));
145 |
146 | vi.mock("@/hooks/useRealtime", () => ({
147 |   useRealtime: () => ({ isLive: true, retryCount: 0 }),
148 | }));
149 |
150 | // Mutable leaderboard hook result so individual tests can flip locked/unlocked
151 | // state and the "load more" affordance.
152 | const leaderboardHookState = vi.hoisted(() => ({
153 |   current: null as unknown,
154 | }));
155 |
156 | vi.mock("@/hooks/useLeaderboard", () => ({
157 |   useLeaderboard: () => leaderboardHookState.current,
158 |   useMyRank: () => ({ data: mockMyRank, isLoading: false }),
159 |   useAnonymousStatus: () => ({ data: { isAnonymous: false } }),
160 | }));
161 |
162 | const unlockedLeaderboardResult = {
163 |   data: {
164 |     entries: mockLeaderboardData,
165 |     eligibleCount: mockLeaderboardData.length,
166 |     state: "unlocked" as const,
167 |     minCohortSize: 5,
168 |     pageSize: 50,
169 |   },
170 |   isLoading: false,
171 |   isError: false,
172 |   error: null,
173 |   hasMore: false,
174 |   fetchNextPage: () => {},
175 |   isFetchingNextPage: false,
176 | };
177 |
178 | const lockedLeaderboardResult = {
179 |   data: {
180 |     entries: [],
181 |     eligibleCount: 1,
182 |     state: "locked" as const,
183 |     minCohortSize: 5,
184 |     pageSize: 50,
185 |   },
186 |   isLoading: false,
187 |   isError: false,
188 |   error: null,
189 |   hasMore: false,
190 |   fetchNextPage: () => {},
191 |   isFetchingNextPage: false,
192 | };
193 |
194 | vi.mock("@/hooks/usePersonalBestLeaderboard", () => ({
195 |   usePersonalBestLeaderboard: () => ({ data: mockWeeklyXP, isLoading: false }),
196 | }));
197 |
198 | vi.mock("@/hooks/useMostImprovedLeaderboard", () => ({
199 |   useMostImprovedLeaderboard: () => ({
200 |     data: mockMostImproved,
201 |     isLoading: false,
202 |   }),
203 | }));
204 |
205 | vi.mock("@/hooks/useLeagueLeaderboard", () => ({

```


[illegible]

```

310     renderPage();
311     expect(screen.getByText("Individual")).toBeInTheDocument();
312     expect(screen.getByText("Teams")).toBeInTheDocument();
313 });
314
315 it("renders leaderboard tab navigation: Top XP, Personal Best, Most Improved", () => {
316     renderPage();
317     expect(screen.getByText("Top XP")).toBeInTheDocument();
318     expect(screen.getByText("Personal Best")).toBeInTheDocument();
319     expect(screen.getByText("Most Improved")).toBeInTheDocument();
320 });
321
322 it("renders filter tabs: All Students, My Course, My Program in Top XP view", () => {
323     renderPage();
324     expect(screen.getByText("All Students")).toBeInTheDocument();
325     expect(screen.getByText("My Course")).toBeInTheDocument();
326     expect(screen.getByText("My Program")).toBeInTheDocument();
327 });
328
329 it("renders timeframe buttons: Weekly and All Time", () => {
330     renderPage();
331     expect(screen.getByText("Weekly")).toBeInTheDocument();
332     expect(screen.getByText("All Time")).toBeInTheDocument();
333 });
334
335 it("renders My Rank card with rank, XP, and level", () => {
336     renderPage();
337     expect(screen.getByText("Your Rank")).toBeInTheDocument();
338     const rankCard = screen
339         .getByText("Your Rank")
340         .closest('[data-slot="card"]');
341     expect(rankCard).toBeInTheDocument();
342     if (rankCard) {
343         // With percentile bands, rank 25 out of 4 entries shows as "Bottom 50%"
344         expect(
345             within(rankCard as HTMLElement).getByText("Bottom 50%")
346         ).toBeInTheDocument();
347         expect(
348             within(rankCard as HTMLElement).getByText("1,000")
349         ).toBeInTheDocument();
350     }
351 });
352
353 it("renders leaderboard entries with names and XP", () => {
354     renderPage();
355     expect(screen.getByText("Alice")).toBeInTheDocument();
356     expect(screen.getByText("Bob")).toBeInTheDocument();
357     expect(screen.getByText("5,000 XP")).toBeInTheDocument();
358     expect(screen.getByText("4,500 XP")).toBeInTheDocument();
359 });
360
361 it("shows Anonymous for opted-out students", () => {
362     renderPage();
363     const anonymousElements = screen.getAllByText("Anonymous");
364     expect(anonymousElements.length).toBeGreaterThanOrEqual(1);
365 });
366
367 it("highlights the current user row with (You) label", () => {
368     renderPage();
369     expect(screen.getByText("Me")).toBeInTheDocument();
370     expect(screen.getByText("(You)")).toBeInTheDocument();
371 });
372
373 it("renders level badges for each entry", () => {
374     renderPage();
375     expect(screen.getByText("Lv 10")).toBeInTheDocument();
376     expect(screen.getByText("Lv 9")).toBeInTheDocument();
377     expect(screen.getByText("Lv 8")).toBeInTheDocument();
378     expect(screen.getByText("Lv 3")).toBeInTheDocument();
379 });
380
381 it("renders the AnonymousToggle component", () => {
382     renderPage();
383     expect(screen.getByTestId("anonymous-toggle")).toBeInTheDocument();
384 });
385
386 it("renders rankings section header", () => {
387     renderPage();
388     expect(screen.getByText("Rankings")).toBeInTheDocument();
389 });
390
391 it("renders League tab in navigation", () => {
392     renderPage();
393     expect(screen.getByText("League")).toBeInTheDocument();
394 });
395
396 it("shows streak flame for entries with active streaks", () => {
397     renderPage();
398     const aliceRow = screen
399         .getByText("Alice")
400         .closest('div[class*="flex items-center gap-4"]');
401     expect(aliceRow).toBeInTheDocument();
402     if (aliceRow) {
403         expect(
404             within(aliceRow as HTMLElement).getByText("30")
405         ).toBeInTheDocument();
406     }
407 });
408
409 it("shows the locked state with no medals or rankings when below the cohort minimum", () => {
410     leaderboardHookState.current = lockedLeaderboardResult;
411     const { container } = renderPage();
412
413     // Locked panel copy is shown

```

```

414 |     expect(
415 |       screen.getByText("Leaderboard unlocks when more students join")
416 |     ).toBeInTheDocument();
417 |
418 |     // No ranked rows / rankings header / personal rank card are rendered
419 |     expect(screen.queryByText("Rankings")).not.toBeInTheDocument();
420 |     expect(screen.queryByText("Your Rank")).not.toBeInTheDocument();
421 |     expect(screen.queryByText("Alice")).not.toBeInTheDocument();
422 |     expect(screen.queryByText("Bob")).not.toBeInTheDocument();
423 |
424 |     // R6.4: no medal is awarded to any student while locked. The medal icon
425 |     // (Lucide `lucide-medal`) is used for the top-3 ranks and the rankings
426 |     // header – none of these may render in the locked state.
427 |     expect(container.querySelectorAll(".lucide-medal").toHaveLength(0));
428 |   });
429 |
430 |   it("renders medal icons for top-3 ranks once the leaderboard is unlocked", () => {
431 |     // Sanity counterpart to the locked assertion: when unlocked, the medal
432 |     // icon appears (ranks 1-3 plus the rankings header), proving the locked
433 |     // state's zero-medal assertion is meaningful rather than always-true.
434 |     leaderboardHookState.current = unlockedLeaderboardResult;
435 |     const { container } = renderPage();
436 |
437 |     expect(container.querySelectorAll(".lucide-medal").length).toBeGreaterThan(
438 |       0
439 |     );
440 |   });
441 | });
442 |

```

FILE: src/___tests___/unit/leaderboardTeamsTab.test.tsx

TYPE: .tsx | LINES: 1-33 | SHA256: c802c575b8d93e0cd3cfb031437a5a873d42694e2135a731be7b05f2979fff77

```

1 | // Unit test: leaderboardTeamsTab – Teams tab renders, switches to team view
2 | import { describe, it, expect } from "vitest";
3 |
4 | describe("LeaderboardPage Teams tab", () => {
5 |   it("Individual and Teams mode options exist", () => {
6 |     const modes = [
7 |       { value: "individual", label: "Individual" },
8 |       { value: "teams", label: "Teams" },
9 |     ];
10 |    expect(modes.find((m) => m.value === "teams")).toBeDefined();
11 |    expect(modes.find((m) => m.value === "individual")).toBeDefined();
12 |  });
13 |
14 |   it('teams mode value is "teams"', () => {
15 |     const teamsMode = { value: "teams", label: "Teams" };
16 |     expect(teamsMode.value).toBe("teams");
17 |     expect(teamsMode.label).toBe("Teams");
18 |   });
19 |
20 |   it("switching to teams mode renders TeamLeaderboard", () => {
21 |     // Simulates the mode switching logic in LeaderboardPage
22 |     const mode = "teams";
23 |     const shouldRenderTeamLeaderboard = mode === "teams";
24 |     expect(shouldRenderTeamLeaderboard).toBe(true);
25 |   });
26 |
27 |   it("individual mode does not render TeamLeaderboard", () => {
28 |     const mode = "individual";
29 |     const shouldRenderTeamLeaderboard = mode === "teams";
30 |     expect(shouldRenderTeamLeaderboard).toBe(false);
31 |   });
32 | });
33 |

```

FILE: src/___tests___/unit/leagueTier.test.ts

TYPE: .ts | LINES: 1-92 | SHA256: 22e9706fe7071de4307bb657f1670c422af28d0f40aec2af2abbf876c8712d05

```

1 | import { describe, it, expect } from "vitest";
2 | import {
3 |   getLeagueTier,
4 |   DEFAULT_LEAGUE_THRESHOLDS,
5 |   TIER_COLORS,
6 |   TIER_BORDER_COLORS,
7 |   type LeagueTierName,
8 | } from "@lib/leagueTier";
9 |
10 | describe("getLeagueTier", () => {
11 |   it("returns Bronze for 0 XP", () => {
12 |     expect(getLeagueTier(0)).toBe("Bronze");
13 |   });
14 |
15 |   it("returns Bronze for 499 XP", () => {
16 |     expect(getLeagueTier(499)).toBe("Bronze");
17 |   });
18 |
19 |   it("returns Silver for 500 XP", () => {
20 |     expect(getLeagueTier(500)).toBe("Silver");
21 |   });
22 |
23 |   it("returns Silver for 1499 XP", () => {
24 |     expect(getLeagueTier(1499)).toBe("Silver");
25 |   });
26 |
27 |   it("returns Gold for 1500 XP", () => {
28 |     expect(getLeagueTier(1500)).toBe("Gold");
29 |   });
30 |
31 |   it("returns Gold for 3999 XP", () => {
32 |     expect(getLeagueTier(3999)).toBe("Gold");
33 |   });
34 |

```

```

35 | it("returns Diamond for 4000 XP", () => {
36 |   expect(getLeagueTier(4000)).toBe("Diamond");
37 | });
38 |
39 | it("returns Diamond for very high XP", () => {
40 |   expect(getLeagueTier(99999)).toBe("Diamond");
41 | });
42 |
43 | it("uses custom thresholds", () => {
44 |   const custom = { bronze: 0, silver: 100, gold: 500, diamond: 1000 };
45 |   expect(getLeagueTier(99, custom)).toBe("Bronze");
46 |   expect(getLeagueTier(100, custom)).toBe("Silver");
47 |   expect(getLeagueTier(500, custom)).toBe("Gold");
48 |   expect(getLeagueTier(1000, custom)).toBe("Diamond");
49 | });
50 |
51 | it("handles negative XP as Bronze", () => {
52 |   expect(getLeagueTier(-10)).toBe("Bronze");
53 | });
54 | });
55 |
56 | describe("DEFAULT_LEAGUE_THRESHOLDS", () => {
57 |   it("has correct default values", () => {
58 |     expect(DEFAULT_LEAGUE_THRESHOLDS).toEqual({
59 |       bronze: 0,
60 |       silver: 500,
61 |       gold: 1500,
62 |       diamond: 4000,
63 |     });
64 |   });
65 | });
66 |
67 | describe("TIER_COLORS", () => {
68 |   it("has color classes for all tiers", () => {
69 |     const tiers: LeagueTierName[] = ["Bronze", "Silver", "Gold", "Diamond"];
70 |     for (const tier of tiers) {
71 |       expect(TIER_COLORS[tier]).toBeDefined();
72 |       expect(typeof TIER_COLORS[tier]).toBe("string");
73 |     }
74 |   });
75 |
76 |   it("uses correct gamification colors", () => {
77 |     expect(TIER_COLORS.Bronze).toContain("amber-600");
78 |     expect(TIER_COLORS.Silver).toContain("gray-400");
79 |     expect(TIER_COLORS.Gold).toContain("yellow-400");
80 |     expect(TIER_COLORS.Diamond).toContain("blue-400");
81 |   });
82 | });
83 |
84 | describe("TIER_BORDER_COLORS", () => {
85 |   it("has border color classes for all tiers", () => {
86 |     const tiers: LeagueTierName[] = ["Bronze", "Silver", "Gold", "Diamond"];
87 |     for (const tier of tiers) {
88 |       expect(TIER_BORDER_COLORS[tier]).toBeDefined();
89 |     }
90 |   });
91 | });
92 |

```

FILE: [src/__tests__/unit/leagueTierBadge.test.tsx](#)

TYPE: .tsx | LINES: 1-69 | SHA256: db58fb0b9ab1f32aed7351d5ad38539cc7052ae689655fb808f45f73dadc8b94

```

1 | // =====
2 | // Unit Test: League Tier Badge
3 | // Task 26.6 – Tier badge rendering, promotion animation
4 | // =====
5 |
6 | import { describe, it, expect } from "vitest";
7 | import { render, screen } from "@testing-library/react";
8 | import LeagueTierBadge from "@components/shared/LeagueTierBadge";
9 | import { getLeagueTier } from "@lib/leagueTier";
10 |
11 | describe("LeagueTierBadge", () => {
12 |   it("renders Bronze tier", () => {
13 |     render(<LeagueTierBadge tier="Bronze" />);
14 |     expect(screen.getByTestId("league-badge-bronze")).toBeDefined();
15 |     expect(screen.getByText("Bronze")).toBeDefined();
16 |   });
17 |
18 |   it("renders Silver tier", () => {
19 |     render(<LeagueTierBadge tier="Silver" />);
20 |     expect(screen.getByTestId("league-badge-silver")).toBeDefined();
21 |     expect(screen.getByText("Silver")).toBeDefined();
22 |   });
23 |
24 |   it("renders Gold tier", () => {
25 |     render(<LeagueTierBadge tier="Gold" />);
26 |     expect(screen.getByTestId("league-badge-gold")).toBeDefined();
27 |     expect(screen.getByText("Gold")).toBeDefined();
28 |   });
29 |
30 |   it("renders Diamond tier", () => {
31 |     render(<LeagueTierBadge tier="Diamond" />);
32 |     expect(screen.getByTestId("league-badge-diamond")).toBeDefined();
33 |     expect(screen.getByText("Diamond")).toBeDefined();
34 |   });
35 |
36 |   it("supports different sizes", () => {
37 |     const { rerender } = render(<LeagueTierBadge tier="Gold" size="sm" />);
38 |     expect(screen.getByTestId("league-badge-gold")).toBeDefined();
39 |
40 |     rerender(<LeagueTierBadge tier="Gold" size="lg" />);
41 |     expect(screen.getByTestId("league-badge-gold")).toBeDefined();
42 |   });

```

```

43 | });
44 |
45 | describe("getLeagueTier", () => {
46 |   it("returns Bronze for 0 XP", () => {
47 |     expect(getLeagueTier(0)).toBe("Bronze");
48 |   });
49 |
50 |   it("returns Silver for 500 XP", () => {
51 |     expect(getLeagueTier(500)).toBe("Silver");
52 |   });
53 |
54 |   it("returns Gold for 1500 XP", () => {
55 |     expect(getLeagueTier(1500)).toBe("Gold");
56 |   });
57 |
58 |   it("returns Diamond for 4000 XP", () => {
59 |     expect(getLeagueTier(4000)).toBe("Diamond");
60 |   });
61 |
62 |   it("uses custom thresholds", () => {
63 |     const custom = { bronze: 0, silver: 100, gold: 200, diamond: 300 };
64 |     expect(getLeagueTier(150, custom)).toBe("Silver");
65 |     expect(getLeagueTier(250, custom)).toBe("Gold");
66 |     expect(getLeagueTier(350, custom)).toBe("Diamond");
67 |   });
68 | });
69 |

```

TYPE: .tsx | LINES: 1-324 | SHA256: 52199e0382527b6bd03422bb2e39fa667d258e63dc5c7c8d2cc0881a20a211

```

74 |     makeAssignment("a1", "Quiz 1", "clo-1"),
75 | ];
76 |
77 | const nodes = buildLearningPathNodes(assignments, clos, [], [], []);
78 |
79 | expect(first(nodes).status).toBe("available");
80 | });
81 |
82 | it("marks nodes as locked when prerequisite attainment is not met", () => {
83 |   const assignments: RawAssignment[] = [
84 |     makeAssignment("a2", "Lab Exercise", "clo-2", [
85 |       { clo_id: "clo-1", required_attainment: 70 },
86 |     ]),
87 |   ];
88 |   const attainments: RawAttainment[] = [
89 |     { outcome_id: "clo-1", attainment_percent: 50 },
90 |   ];
91 |
92 |   const nodes = buildLearningPathNodes(
93 |     assignments,
94 |     clos,
95 |     attainments,
96 |     [],
97 |     []
98 |   );
99 |
100 |   expect(first(nodes).status).toBe("locked");
101 |   expect(first(nodes).prerequisite).toEqual({
102 |     clo_id: "clo-1",
103 |     clo_title: "Recall Basics",
104 |     required_attainment: 70,
105 |     current_attainment: 50,
106 |   });
107 | });
108 |
109 | it("marks nodes as available when prerequisite attainment is met", () => {
110 |   const assignments: RawAssignment[] = [
111 |     makeAssignment("a2", "Lab Exercise", "clo-2", [
112 |       { clo_id: "clo-1", required_attainment: 70 },
113 |     ]),
114 |   ];
115 |   const attainments: RawAttainment[] = [
116 |     { outcome_id: "clo-1", attainment_percent: 75 },
117 |   ];
118 |
119 |   const nodes = buildLearningPathNodes(
120 |     assignments,
121 |     clos,
122 |     attainments,
123 |     [],
124 |     []
125 |   );
126 |
127 |   expect(first(nodes).status).toBe("available");
128 |   expect(first(nodes).prerequisite).toBeUndefined();
129 | });
130 |
131 | it("marks nodes as submitted when a submission exists", () => {
132 |   const assignments: RawAssignment[] = [
133 |     makeAssignment("a1", "Quiz 1", "clo-1"),
134 |   ];
135 |   const submissions: RawSubmission[] = [
136 |     { assignment_id: "a1", status: "submitted" },
137 |   ];
138 |
139 |   const nodes = buildLearningPathNodes(
140 |     assignments,
141 |     clos,
142 |     [],
143 |     submissions,
144 |     []
145 |   );
146 |
147 |   expect(first(nodes).status).toBe("submitted");
148 | });
149 |
150 | it("marks nodes as graded when a grade exists", () => {
151 |   const assignments: RawAssignment[] = [
152 |     makeAssignment("a1", "Quiz 1", "clo-1"),
153 |   ];
154 |   const submissions: RawSubmission[] = [
155 |     { assignment_id: "a1", status: "graded" },
156 |   ];
157 |   const grades: RawGrade[] = [
158 |     { submission_id: "sub-1", assignment_id: "a1" },
159 |   ];
160 |
161 |   const nodes = buildLearningPathNodes(
162 |     assignments,
163 |     clos,
164 |     [],
165 |     submissions,
166 |     grades
167 |   );
168 |
169 |   expect(first(nodes).status).toBe("graded");
170 | });
171 |
172 | it("includes attainment_percent from outcome_attainment", () => {
173 |   const assignments: RawAssignment[] = [
174 |     makeAssignment("a1", "Quiz 1", "clo-1"),
175 |   ];
176 |   const attainments: RawAttainment[] = [
177 |     { outcome_id: "clo-1", attainment_percent: 82 },

```

```

178 | ];
179 |
180 | const nodes = buildLearningPathNodes(
181 |     assignments,
182 |     clos,
183 |     attainments,
184 |     [],
185 |     []
186 | );
187 |
188 | expect(first(nodes).attainment_percent).toBe(82);
189 | });
190 |
191 | it("returns null attainment_percent when no attainment data exists", () => {
192 |     const assignments: RawAssignment[] = [
193 |         makeAssignment("a1", "Quiz 1", "clo-1"),
194 |     ];
195 |
196 |     const nodes = buildLearningPathNodes(assignments, clos, [], [], []);
197 |
198 |     expect(first(nodes).attainment_percent).toBeNull();
199 | });
200 |
201 | it("defaults to remembering when CLO has no blooms_level", () => {
202 |     const noBlooms: RawCLO[] = [
203 |         { id: "clo-x", title: "Unknown", blooms_level: null },
204 |     ];
205 |     const assignments: RawAssignment[] = [
206 |         makeAssignment("a1", "Task", "clo-x"),
207 |     ];
208 |
209 |     const nodes = buildLearningPathNodes(assignments, noBlooms, [], [], []);
210 |
211 |     expect(first(nodes).blooms_level).toBe("remembering");
212 | });
213 |
214 | it("treats missing attainment as 0 for prerequisite check", () => {
215 |     const assignments: RawAssignment[] = [
216 |         makeAssignment("a2", "Lab", "clo-2", [
217 |             { clo_id: "clo-1", required_attainment: 70 },
218 |         ]),
219 |     ];
220 |
221 |     const nodes = buildLearningPathNodes(assignments, clos, [], [], []);
222 |
223 |     expect(first(nodes).status).toBe("locked");
224 |     expect(first(nodes).prerequisite?.current_attainment).toBe(0);
225 | });
226 |
227 | it("returns empty array for empty assignments", () => {
228 |     const nodes = buildLearningPathNodes([], clos, [], [], []);
229 |     expect(nodes).toEqual([]);
230 | });
231 |
232 | it("derives Blooms level from the lowest CLO across ALL linked CLOs (not the first)", () => {
233 |     // Assignment links a high-Bloom CLO first and a low-Bloom CLO second; the
234 |     // representative level must be the minimum (remembering), not the first
235 |     // (creating).
236 |     const multiCLOAssignment: RawAssignment = {
237 |         id: "a1",
238 |         title: "Capstone",
239 |         clo_weights: [
240 |             { clo_id: "clo-3", weight: 50 }, // creating
241 |             { clo_id: "clo-1", weight: 50 }, // remembering
242 |         ],
243 |         prerequisites: null,
244 |     };
245 |
246 |     const nodes = buildLearningPathNodes(
247 |         [multiCLOAssignment],
248 |         clos,
249 |         [],
250 |         [],
251 |         []
252 |     );
253 |
254 |     expect(first(nodes).blooms_level).toBe("remembering");
255 | });
256 |
257 | it("ignores CLOs without a blooms_level when picking the representative level", () => {
258 |     const mixedClos: RawCLO[] = [
259 |         { id: "clo-null", title: "No level", blooms_level: null },
260 |         makeCLO("clo-2", "applying", "Apply"),
261 |     ];
262 |     const assignment: RawAssignment = {
263 |         id: "a1",
264 |         title: "Task",
265 |         clo_weights: [
266 |             { clo_id: "clo-null", weight: 50 },
267 |             { clo_id: "clo-2", weight: 50 },
268 |         ],
269 |         prerequisites: null,
270 |     };
271 |
272 |     const nodes = buildLearningPathNodes([assignment], mixedClos, [], [], []);
273 |
274 |     expect(first(nodes).blooms_level).toBe("applying");
275 | });
276 |
277 | it("sorts assignments at the same Blooms level alphabetically", () => {
278 |     const sameLevelClos: RawCLO[] = [
279 |         makeCLO("clo-a", "applying", "CLO A"),
280 |         makeCLO("clo-b", "applying", "CLO B"),
281 |     ];

```

[illegible]

FILE: src/__tests__/unit/learningPathPresentation.test.ts

```
TYPE: .ts | LINES: 1-58 | SHA256: 97db0867313bfed2c057746b74ea303fa36ca07f4e90948940b6f963d33a1a60
```

```

1 import { describe, expect, it } from "vitest";
2
3 import {
4   averagePercentage,
5   buildBloomStageSummaries,
6 } from "@lib/learningPathPresentation";
7 import type { LearningPathNode } from "@/hooks/useLearningPath";
8
9 const node = (overrides: Partial<LearningPathNode> = {}): LearningPathNode => ({
10   assignment_id: "assignment-1",
11   title: "Practice task",
12   blooms_level: "remembering",
13   status: "available",
14   attainment_percent: 60,
15   ...overrides,
16 });
17
18 describe("learningPathPresentation", () => {
19   it("always builds the complete six-stage Bloom journey", () => {
20     const stages = buildBloomStageSummaries([]);
21
22     expect(stages.map((stage) => stage.key)).toEqual([
23       "remembering",
24       "understanding",
25       "applying",
26       "analyzing",
27       "evaluating",
28       "creating",
29     ]);
30     expect(stages.every((stage) => stage.status === "locked")).toBe(true);
31   });
32
33   it("derives completed, current, locked, and average mastery states", () => {
34     const stages = buildBloomStageSummaries([
35       node({ status: "graded", attainment_percent: 80 }),
36       node({
37         assignment_id: "assignment-2",
38         status: "graded",
39         attainment_percent: 90,
40       }),
41       node({
42         assignment_id: "assignment-3",
43         blooms_level: "understanding",
44         status: "submitted",
45         attainment_percent: 65,
46       }),
47     ]);
48
49     expect(stages[0]).toMatchObject({ status: "done", attainment: 85 });
50     expect(stages[1]).toMatchObject({ status: "current", attainment: 65 });
51     expect(stages[2]).toMatchObject({ status: "locked", attainment: 0 });
52   });
53
54   it("handles empty percentage sets without NaN", () => {
55     expect(averagePercentage([])).toBe(0);
56   });
57 });

```


FILE: [src/__tests__/unit/legalPages.test.tsx](#)

TYPE: .tsx | LINES: 1-71 | SHA256: dc88d8dad920dfc7c3bfa8df08f4f7437e3a4b940a1b3c7cbf872c831378587a

```

1 | // @vitest-environment happy-dom
2 |
3 | import { afterEach, describe, expect, it, vi } from "vitest";
4 | import { cleanup, render, screen } from "@testing-library/react";
5 | import { I18nextProvider } from "react-i18next";
6 | import { MemoryRouter } from "react-router-dom";
7 | import type { ReactNode } from "react";
8 |
9 | import i18n from "@lib/i18n";
10 | import PrivacyPage from "@pages/public/PrivacyPage";
11 | import TermsPage from "@pages/public/TermsPage";
12 |
13 | vi.mock("@hooks/useAuth", () => ({
14 |   useAuth: () => ({ user: null }),
15 | }));
16 |
17 | vi.mock("@hooks/useLanguagePreference", () => ({
18 |   useUpdateLanguagePreference: () => ({ mutate: vi.fn() }),
19 | }));
20 |
21 | const renderPage = (page: ReactNode) =>
22 |   render(
23 |     <I18nextProvider i18n={i18n}>
24 |       <MemoryRouter>{page}</MemoryRouter>
25 |     </I18nextProvider>
26 |   );
27 |
28 | afterEach(async () => {
29 |   cleanup();
30 |   await i18n.changeLanguage("en");
31 | });
32 |
33 | describe("public legal pages", () => {
34 |   it("renders the complete shared Terms layout in English", async () => {
35 |     await i18n.changeLanguage("en");
36 |     renderPage(<TermsPage />);
37 |
38 |     expect(
39 |       screen.getByRole("heading", { name: "Terms of Service", level: 1 })
40 |     ).toBeInTheDocument();
41 |     expect(
42 |       screen.getByRole("heading", { name: "8. Contact", level: 2 })
43 |     ).toBeInTheDocument();
44 |     expect(
45 |       screen.getByRole("link", { name: "legal@edeviser.com" })
46 |     ).toHaveAttribute("href", "mailto:legal@edeviser.com");
47 |     expect(
48 |       screen.getByRole("link", { name: "/Privacy Policy/" })
49 |     ).toHaveAttribute("href", "/privacy");
50 |   });
51 |
52 |   it("renders the complete shared Privacy layout in Arabic", async () => {
53 |     await i18n.changeLanguage("ar");
54 |     renderPage(<PrivacyPage />);
55 |
56 |     expect(
57 |       screen.getByRole("heading", { name: "c6Jbv3b' 'dF.cVHcVJb'Ã ÆwfvÃ¢ ò'
58 |     }).toBeInTheDocument();
59 |     expect(
60 |       screen.getByRole("heading", { name: "8. bvDb!Hbv5dB"Ã ÆwfvÃ¢ " ò'
61 |     }).toBeInTheDocument();
62 |     expect(
63 |       screen.getByRole("link", { name: "privacy@edeviser.com" })
64 |     ).toHaveAttribute("href", "mailto:privacy@edeviser.com");
65 |     expect(screen.getByRole("link", { name: "/cF1d†7 bvDbæ/dV/ })).toHaveAttribute(
66 |       "href",
67 |       "/terms"
68 |     );
69 |   });
70 | });
71 |

```

FILE: [src/__tests__/unit/letterGradeMapper.test.ts](#)

TYPE: .ts | LINES: 1-98 | SHA256: 64aacbf7d71eb3ce2fd476105955c8d9131b9a3f4fc53b1e1ae628ecb65e4476

```

1 | import { describe, it, expect } from "vitest";
2 | import { mapToLetterGrade, mapToGpaPoints } from "@lib/letterGradeMapper";
3 | import type { GradeScale } from "@types/app";
4 |
5 | describe("mapToLetterGrade", () => {
6 |   it("maps 90% to A with default scales", () => {
7 |     expect(mapToLetterGrade(90)).toBe("A");
8 |   });
9 |
10 |   it("maps 85% to A (boundary)", () => {
11 |     expect(mapToLetterGrade(85)).toBe("A");
12 |   });
13 |
14 |   it("maps 84% to B", () => {
15 |     expect(mapToLetterGrade(84)).toBe("B");
16 |   });
17 |
18 |   it("maps 70% to B (boundary)", () => {
19 |     expect(mapToLetterGrade(70)).toBe("B");
20 |   });
21 |
22 |   it("maps 55% to C", () => {
23 |     expect(mapToLetterGrade(55)).toBe("C");
24 |   });

```

```

25 |
26 | it("maps 50% to D", () => {
27 |   expect(mapToLetterGrade(50)).toBe("D");
28 | });
29 |
30 | it("maps 49% to F", () => {
31 |   expect(mapToLetterGrade(49)).toBe("F");
32 | });
33 |
34 | it("maps 0% to F", () => {
35 |   expect(mapToLetterGrade(0)).toBe("F");
36 | });
37 |
38 | it("maps 100% to A", () => {
39 |   expect(mapToLetterGrade(100)).toBe("A");
40 | });
41 |
42 | it("uses custom grade scales when provided", () => {
43 |   const customScales: GradeScale[] = [
44 |     { letter: "A+", min_percent: 95, max_percent: 100, gpa_points: 4.0 },
45 |     { letter: "A", min_percent: 90, max_percent: 94, gpa_points: 3.7 },
46 |     { letter: "B", min_percent: 80, max_percent: 89, gpa_points: 3.0 },
47 |     { letter: "C", min_percent: 60, max_percent: 79, gpa_points: 2.0 },
48 |     { letter: "F", min_percent: 0, max_percent: 59, gpa_points: 0.0 },
49 |   ];
50 |
51 |   expect(mapToLetterGrade(96, customScales)).toBe("A+");
52 |   expect(mapToLetterGrade(92, customScales)).toBe("A");
53 |   expect(mapToLetterGrade(85, customScales)).toBe("B");
54 |   expect(mapToLetterGrade(65, customScales)).toBe("C");
55 |   expect(mapToLetterGrade(30, customScales)).toBe("F");
56 | });
57 |
58 | it("falls back to DEFAULT_GRADE_SCALES when empty array provided", () => {
59 |   expect(mapToLetterGrade(90, [])).toBe("A");
60 |   expect(mapToLetterGrade(49, [])).toBe("F");
61 | });
62 | });
63 |
64 | describe("mapToGpaPoints", () => {
65 |   it("maps A range to 4.0", () => {
66 |     expect(mapToGpaPoints(90)).toBe(4.0);
67 |   });
68 |
69 |   it("maps B range to 3.0", () => {
70 |     expect(mapToGpaPoints(75)).toBe(3.0);
71 |   });
72 |
73 |   it("maps C range to 2.0", () => {
74 |     expect(mapToGpaPoints(60)).toBe(2.0);
75 |   });
76 |
77 |   it("maps D range to 1.0", () => {
78 |     expect(mapToGpaPoints(52)).toBe(1.0);
79 |   });
80 |
81 |   it("maps F range to 0.0", () => {
82 |     expect(mapToGpaPoints(30)).toBe(0.0);
83 |   });
84 |
85 |   it("uses custom scales", () => {
86 |     const customScales: GradeScale[] = [
87 |       { letter: "A", min_percent: 90, max_percent: 100, gpa_points: 4.0 },
88 |       { letter: "F", min_percent: 0, max_percent: 89, gpa_points: 0.0 },
89 |     ];
90 |     expect(mapToGpaPoints(95, customScales)).toBe(4.0);
91 |     expect(mapToGpaPoints(50, customScales)).toBe(0.0);
92 |   });
93 |
94 |   it("falls back to DEFAULT_GRADE_SCALES when empty array provided", () => {
95 |     expect(mapToGpaPoints(90, [])).toBe(4.0);
96 |   });
97 | });
98 |

```

FILE: [src/__tests__/unit/levelAwareHeatmap.test.ts](#)

TYPE: .ts | LINES: 1-154 | SHA256: d608354d5bbfd8534b9f301b3a92defe3bcd2d75c60f9edbe09ef7eec02b60fb

```

1 | import { describe, it, expect } from "vitest";
2 | import {
3 |   getLevelAwareIntensityLevel,
4 |   getLevelFromDate,
5 |   computeLevelRelativeConsistencyScore,
6 |   getLevelMaxHabits,
7 | } from "@lib/levelAwareHeatmap";
8 | import type { LevelProgressionPoint } from "@types/habits";
9 |
10 | // -----
11 | // getLevelMaxHabits
12 | // -----
13 |
14 | describe("getLevelMaxHabits", () => {
15 |   it("returns the level number itself", () => {
16 |     expect(getLevelMaxHabits(1)).toBe(1);
17 |     expect(getLevelMaxHabits(2)).toBe(2);
18 |     expect(getLevelMaxHabits(3)).toBe(3);
19 |     expect(getLevelMaxHabits(4)).toBe(4);
20 |   });
21 | });
22 |
23 | // -----
24 | // getLevelAwareIntensityLevel
25 | // -----
26 |

```

```

27 | describe("getLevelAwareIntensityLevel", () => {
28 |   it("returns 0 when count is 0", () => {
29 |     expect(getLevelAwareIntensityLevel(0, 4)).toBe(0);
30 |   });
31 |
32 |   it("returns 0 when levelMax is 0", () => {
33 |     expect(getLevelAwareIntensityLevel(3, 0)).toBe(0);
34 |   });
35 |
36 |   it("Level 1 student with 1 habit !' full intensity (4)", () => {
37 |     expect(getLevelAwareIntensityLevel(1, 1)).toBe(4);
38 |   });
39 |
40 |   it("Level 4 student with 1 habit !' intensity 1 (ratio 0.25)", () => {
41 |     expect(getLevelAwareIntensityLevel(1, 4)).toBe(1);
42 |   });
43 |
44 |   it("Level 4 student with 4 habits !' full intensity (4)", () => {
45 |     expect(getLevelAwareIntensityLevel(4, 4)).toBe(4);
46 |   });
47 |
48 |   it("Level 2 student with 1 habit !' intensity 2 (ratio 0.5)", () => {
49 |     expect(getLevelAwareIntensityLevel(1, 2)).toBe(2);
50 |   });
51 |
52 |   it("Level 4 student with 3 habits !' intensity 3 (ratio 0.75)", () => {
53 |     expect(getLevelAwareIntensityLevel(3, 4)).toBe(3);
54 |   });
55 |
56 |   it("count exceeding levelMax !' full intensity (4)", () => {
57 |     expect(getLevelAwareIntensityLevel(5, 2)).toBe(4);
58 |   });
59 | });
60 |
61 | // -----
62 | // getLevelForDate
63 | // -----
64 |
65 | describe("getLevelForDate", () => {
66 |   const history: LevelProgressionPoint[] = [
67 |     { date: "2025-01-01", level: 1 },
68 |     { date: "2025-02-15", level: 2 },
69 |     { date: "2025-04-01", level: 3 },
70 |   ];
71 |
72 |   it("returns default level 4 when history is empty", () => {
73 |     expect(getLevelForDate("2025-03-01", [])).toBe(4);
74 |   });
75 |
76 |   it("returns default level 4 when date is before all history entries", () => {
77 |     expect(getLevelForDate("2024-12-31", history)).toBe(4);
78 |   });
79 |
80 |   it("returns level 1 for date on first entry", () => {
81 |     expect(getLevelForDate("2025-01-01", history)).toBe(1);
82 |   });
83 |
84 |   it("returns level 1 for date between first and second entry", () => {
85 |     expect(getLevelForDate("2025-01-15", history)).toBe(1);
86 |   });
87 |
88 |   it("returns level 2 for date on second entry", () => {
89 |     expect(getLevelForDate("2025-02-15", history)).toBe(2);
90 |   });
91 |
92 |   it("returns level 3 for date after last entry", () => {
93 |     expect(getLevelForDate("2025-05-01", history)).toBe(3);
94 |   });
95 | });
96 |
97 | // -----
98 | // computeLevelRelativeConsistencyScore
99 | // -----
100 |
101 | describe("computeLevelRelativeConsistencyScore", () => {
102 |   const history: LevelProgressionPoint[] = [{ date: "2025-01-01", level: 2 }];
103 |
104 |   it("returns 0 when all days are sabbatical", () => {
105 |     const days = [
106 |       { date: "2025-01-04", academicCount: 2 },
107 |       { date: "2025-01-05", academicCount: 1 },
108 |     ];
109 |     const sabbatical = new Set(["2025-01-04", "2025-01-05"]);
110 |     expect(
111 |       computeLevelRelativeConsistencyScore(days, history, sabbatical)
112 |     ).toBe(0);
113 |   });
114 |
115 |   it("returns 100 when all eligible days meet level requirement", () => {
116 |     const days = [
117 |       { date: "2025-01-06", academicCount: 2 },
118 |       { date: "2025-01-07", academicCount: 3 },
119 |     ];
120 |     expect(computeLevelRelativeConsistencyScore(days, history, new Set())).toBe(
121 |       100
122 |     );
123 |   });
124 |
125 |   it("returns 50 when half the days meet level requirement", () => {
126 |     const days = [
127 |       { date: "2025-01-06", academicCount: 2 },
128 |       { date: "2025-01-07", academicCount: 1 },
129 |     ];
130 |     expect(computeLevelRelativeConsistencyScore(days, history, new Set())).toBe(

```

```

131 |         50
132 |     });
133 | });
134 |
135 | it("excludes sabbatical dates from calculation", () => {
136 |     const days = [
137 |         { date: "2025-01-06", academicCount: 2 },
138 |         { date: "2025-01-07", academicCount: 0 },
139 |         { date: "2025-01-08", academicCount: 2 },
140 |     ];
141 |     const sabbatical = new Set(["2025-01-07"]);
142 |     // Only 2 eligible days, both meet requirement !' 100%
143 |     expect(
144 |         computeLevelRelativeConsistencyScore(days, history, sabbatical)
145 |     ).toBe(100);
146 | });
147 |
148 | it("returns 0 when no days have data", () => {
149 |     expect(computeLevelRelativeConsistencyScore([], history, new Set())).toBe(
150 |         0
151 |     );
152 | });
153 | });
154 |

```

FILE: [src/__tests__/unit/levelProgressionChart.test.tsx](#)

TYPE: .tsx | LINES: 1-126 | SHA256: 5477951bf7f30a2b18e56489570dd520baacdcd9655467d81833391d6945a02e

```

1 | import { describe, it, expect, vi } from "vitest";
2 | import { render, screen } from "@testing-library/react";
3 | import type { LevelProgressionPoint } from "@types/habits";
4 |
5 | // Mock Recharts to avoid SVG rendering issues in tests
6 | vi.mock("recharts", () => ({
7 |     LineChart: ({ children }: { children: React.ReactNode }) => (
8 |         <div data-testid="line-chart">{children}</div>
9 |     ),
10 |    Line: () => <div data-testid="recharts-line" />,
11 |    XAxis: () => <div />,
12 |    YAxis: () => <div />,
13 |    Tooltip: () => <div />,
14 |    ResponsiveContainer: ({ children }: { children: React.ReactNode }) => (
15 |        <div>{children}</div>
16 |    ),
17 |    ReferenceDot: (props: Record<string, unknown>) => (
18 |        <div data-testid={props["data-testid"] as string} />
19 |    ),
20 | }));
21 |
22 | import LevelProgressionChart from "@components/shared/LevelProgressionChart";
23 |
24 | describe("LevelProgressionChart", () => {
25 |     it("renders single-level message when data is empty", () => {
26 |         render(<LevelProgressionChart data={[]} currentLevel={3} />);
27 |         expect(screen.getByTestId("level-progression-single")).toBeInTheDocument();
28 |         expect(
29 |             screen.getByText("You've been consistent at Level 3 this semester")
30 |         ).toBeInTheDocument();
31 |         expect(screen.getByText("3")).toBeInTheDocument();
32 |     });
33 |
34 |     it("renders single-level message when all entries have the same level", () => {
35 |         const data: LevelProgressionPoint[] = [
36 |             { date: "2025-01-01", level: 2 },
37 |             { date: "2025-02-01", level: 2 },
38 |             { date: "2025-03-01", level: 2 },
39 |         ];
40 |         render(<LevelProgressionChart data={data} currentLevel={2} />);
41 |         expect(screen.getByTestId("level-progression-single")).toBeInTheDocument();
42 |         expect(
43 |             screen.getByText("You've been consistent at Level 2 this semester")
44 |         ).toBeInTheDocument();
45 |     });
46 |
47 |     it("renders single-level message for a single data point", () => {
48 |         const data: LevelProgressionPoint[] = [{ date: "2025-01-15", level: 1 }];
49 |         render(<LevelProgressionChart data={data} currentLevel={1} />);
50 |         expect(screen.getByTestId("level-progression-single")).toBeInTheDocument();
51 |         expect(
52 |             screen.getByText("You've been consistent at Level 1 this semester")
53 |         ).toBeInTheDocument();
54 |     });
55 |
56 |     it("renders step chart when data has multiple levels", () => {
57 |         const data: LevelProgressionPoint[] = [
58 |             { date: "2025-01-01", level: 1 },
59 |             { date: "2025-02-01", level: 2 },
60 |             { date: "2025-03-01", level: 3 },
61 |         ];
62 |         render(<LevelProgressionChart data={data} currentLevel={3} />);
63 |         expect(screen.getByTestId("level-progression-chart")).toBeInTheDocument();
64 |         expect(
65 |             screen.queryByTestId("level-progression-single")
66 |         ).not.toBeInTheDocument();
67 |     });
68 |
69 |     it("renders level-up markers at level increase points", () => {
70 |         const data: LevelProgressionPoint[] = [
71 |             { date: "2025-01-01", level: 1 },
72 |             { date: "2025-02-01", level: 2 },
73 |             { date: "2025-03-01", level: 3 },
74 |         ];
75 |         render(<LevelProgressionChart data={data} currentLevel={3} />);
76 |         expect(

```

```

77 |     screen.getByTestId("level-up-marker-2025-02-01")
78 |   ).toBeInTheDocument();
79 |   expect(
80 |     screen.getByTestId("level-up-marker-2025-03-01")
81 |   ).toBeInTheDocument();
82 | });
83 |
84 | it("does not render level-up markers for level decreases", () => {
85 |   const data: LevelProgressionPoint[] = [
86 |     { date: "2025-01-01", level: 3 },
87 |     { date: "2025-02-01", level: 2 },
88 |     { date: "2025-03-01", level: 4 },
89 |   ];
90 |   render(<LevelProgressionChart data={data} currentLevel={4} />);
91 |   expect(screen.getByTestId("level-progression-chart")).toBeInTheDocument();
92 |   // Only the increase from 2!4 should have a marker
93 |   expect(
94 |     screen.queryByTestId("level-up-marker-2025-02-01")
95 |   ).not.toBeInTheDocument();
96 |   expect(
97 |     screen.getByTestId("level-up-marker-2025-03-01")
98 |   ).toBeInTheDocument();
99 | });
100 |
101 | it("displays the level value and /4 denominator in single-level view", () => {
102 |   render(<LevelProgressionChart data={[]} currentLevel={4} />);
103 |   expect(screen.getByText("4")).toBeInTheDocument();
104 |   expect(screen.getByText("/ 4")).toBeInTheDocument();
105 | });
106 |
107 | it("uses currentLevel as fallback when data is empty", () => {
108 |   render(<LevelProgressionChart data={[]} currentLevel={2} />);
109 |   expect(screen.getByText("2")).toBeInTheDocument();
110 |   expect(
111 |     screen.getByText("You've been consistent at Level 2 this semester")
112 |   ).toBeInTheDocument();
113 | });
114 |
115 | it("sorts data by date before rendering chart", () => {
116 |   const data: LevelProgressionPoint[] = [
117 |     { date: "2025-03-01", level: 3 },
118 |     { date: "2025-01-01", level: 1 },
119 |     { date: "2025-02-01", level: 2 },
120 |   ];
121 |   render(<LevelProgressionChart data={data} currentLevel={3} />);
122 |   // Chart should render without errors even with unsorted input
123 |   expect(screen.getByTestId("level-progression-chart")).toBeInTheDocument();
124 | });
125 | });
126 |

```

TYPE: .tsx | LINES: 1-305 | SHA256: 4b3ece022035d22f449d2af627aa1249000d399da01e32e52963df7376dc2cb7

[illegible]

[illegible]

[illegible]

[illegible]

```
FILE: src/  tests  /unit/localeFormatters.test.ts
```

TYPE: .ts | LINES: 1-104 | SHA256: b0174708a7c6f85878cc31c275e04eb48587ee40abd1ef8603ca16ee17a7b65b

```

1 // Feature: pre-deployment-e2e-audit, Task 11.8: Locale-aware formatter unit tests
2 // **Validates: Requirements 10.5**
3 //
4 // Ensures Intl.NumberFormat + Intl.DateTimeFormat emit correct digits
5 // under the two locales the product supports. The Arabic locale must
6 // render Arabic-Indic digits (ف ٥١) when the numbering system is pinned
7 // to "arab"; the English locale always renders Western-Arabic digits
8 // (0-9). A regression would ship Arabic pages with the wrong digit set
9 // or vice versa.
10 //
11 // Note: we pin `numberingSystem: "arab"` explicitly rather than relying
12 // on the Node ICU default. Node's small-ICU build (common on CI Linux
13 // images) returns Latin digits for plain "ar" while full-ICU returns
14 // Arabic-Indic. Production uses region-specific tags like "ar-QA" for
15 // the same effect - see src/lib/formatNumber.ts.
16
17 import { describe, it, expect } from "vitest";
18
19 describe("Locale-aware number formatting (Req 10.5)", () => {
20   it("en-US renders Western-Arabic digits for integers", () => {
21     const f = new Intl.NumberFormat("en-US");
22     expect(f.format(1234)).toMatch(/^[0-9,]+$/);
23     expect(f.format(1234)).toBe("1,234");
24   });
25
26   it("en-US renders Western-Arabic digits for decimals", () => {
27     const f = new Intl.NumberFormat("en-US", { minimumFractionDigits: 1 });
28     expect(f.format(1234.5)).toBe("1,234.5");
29   });
30
31   it("ar with numberingSystem=arab renders Arabic-Indic digits", () => {
32     // Pin the numbering system explicitly instead of relying on the
33     // Node ICU default: Node's small-ICU build (common on CI Linux
34     // images) returns Western-Arabic digits for plain "ar", while
35     // full-ICU returns Arabic-Indic. Production uses the region-specific
36     // "ar-QA" tag (see src/lib/formatNumber.ts), but the test asserts
37     // the canonical pinned behavior so it's stable across both builds.
38     const f = new Intl.NumberFormat("ar", { numberingSystem: "arab" });
39     const result = f.format(1234);
40     // Arabic-Indic digits are U+0660..U+0669 (ف â€).
41     expect(result).toMatch(/[\u0660-\u0669]/);
42     // Must NOT contain Western-Arabic digits.
43     expect(result).not.toMatch(/[0-9]/);
44   });
45
46   it("ar with numberingSystem=latn forces Western-Arabic digits for technical UIs", () => {
47     // When we want Arabic copy but Western digits (e.g. for an English-
48     // typed identifier rendered on an Arabic page), nu=latn is the
49     // documented override per the i18n steering guide.
50     const f = new Intl.NumberFormat("ar", { numberingSystem: "latn" });
51     expect(f.format(1234)).toMatch(/^[0-9fAa02B0]$/);
52   });
53 });

```



```

54 |
55 | describe("Locale-aware date formatting (Req 10.5)", () => {
56 |   // Anchor the test to a fixed date so Daylight-Saving / timezone shifts
57 |   // in CI don't perturb the assertions. Use UTC to remove env drift.
58 |   const ANCHOR_DATE = new Date("2026-04-23T12:00:00Z");
59 |
60 |   it("en-US renders a month name in English", () => {
61 |     const f = new Intl.DateTimeFormat("en-US", {
62 |       year: "numeric",
63 |       month: "long",
64 |       day: "numeric",
65 |       timeZone: "UTC",
66 |     });
67 |     expect(f.format(ANCHOR_DATE)).toMatch(/April/);
68 |     expect(f.format(ANCHOR_DATE)).toMatch(/23/);
69 |     expect(f.format(ANCHOR_DATE)).toMatch(/2026/);
70 |   });
71 |
72 |   it("ar with numberingSystem=arab renders Arabic-Indic digits in dates", () => {
73 |     // Same reasoning as the NumberFormat test above: pin the numbering
74 |     // system explicitly so the test is stable across small-ICU and
75 |     // full-ICU Node builds.
76 |     const f = new Intl.DateTimeFormat("ar", {
77 |       year: "numeric",
78 |       month: "long",
79 |       day: "numeric",
80 |       numberingSystem: "arab",
81 |       timeZone: "UTC",
82 |     });
83 |     const result = f.format(ANCHOR_DATE);
84 |     // Must contain Arabic-Indic digits for day (23 !' f&c) or year.
85 |     expect(result).toMatch(/[\u0660-\u0669]/);
86 |   });
87 |
88 |   it("ar with numberingSystem=latn and calendar=gregory matches en-US digit style", () => {
89 |     // This is what the pre-deployment audit expects when formatters need
90 |     // to round-trip digits across English/Arabic locales for identifier-
91 |     // like outputs (invoice numbers, receipts, etc.).
92 |     const f = new Intl.DateTimeFormat("ar", {
93 |       year: "numeric",
94 |       month: "2-digit",
95 |       day: "2-digit",
96 |       numberingSystem: "latn",
97 |       calendar: "gregory",
98 |       timeZone: "UTC",
99 |     });
100 |     expect(f.format(ANCHOR_DATE)).toMatch(/2026/);
101 |     expect(f.format(ANCHOR_DATE)).toMatch(/[0-9]/);
102 |   });
103 | });
104 |

```

FILE: [src/__tests__/unit/localizationFallback.test.ts](#)

TYPE: .ts | LINES: 1-140 | SHA256: 665983dd07fe80873fde1389b7792494ffb9f24d8e31a69ece68e98dab010de9

```

1 | // =====
2 | // localization – runtime cross-language fallback (R22.6)
3 | // =====
4 | // Verifies resolveLocalizedOrFail resolves the active language first and falls
5 | // back to the other supported language when only that one has the key, so the
6 | // younger-student wording always displays (R22.6).
7 |
8 | import { describe, it, expect, beforeEach, afterEach } from "vitest";
9 | import i18n from "@/lib/i18n";
10 | import {
11 |   resolveLocalizationGate,
12 |   resolveLocalizedOrFail,
13 | } from "@/lib/localization";
14 |
15 | const FALLBACK_KEY = "fallbackOnly.wording";
16 | const EN_VALUE = "English only value";
17 | const AR_VALUE = "d&JdV) c-1b†Jb' Ad&7";
18 |
19 | describe("resolveLocalizedOrFail (R22.6)", () => {
20 |   beforeEach(async () => {
21 |     await i18n.changeLanguage("en");
22 |   });
23 |
24 |   afterEach(async () => {
25 |     i18n.removeResourceBundle("en", "fallbackTest");
26 |     i18n.removeResourceBundle("ar", "fallbackTest");
27 |     await i18n.changeLanguage("en");
28 |   });
29 |
30 |   it("resolves the active language when present", () => {
31 |     expect(
32 |       resolveLocalizedOrFail("portfolio.friendly.strengths", { ns: "student" })
33 |     ).toBe(i18n.t("student:portfolio.friendly.strengths"));
34 |   });
35 |
36 |   it("falls back to the available language when the active language lacks the key", async () => {
37 |     // Only Arabic has the key; active language is English.
38 |     i18n.addResource("ar", "fallbackTest", FALLBACK_KEY, AR_VALUE);
39 |     await i18n.changeLanguage("en");
40 |
41 |     const resolved = resolveLocalizedOrFail(FALLBACK_KEY, {
42 |       ns: "fallbackTest",
43 |       fallbackToOtherLang: true,
44 |     });
45 |     expect(resolved).toBe(AR_VALUE);
46 |   });
47 |
48 |   it("falls back from Arabic to English when only English resolves", async () => {
49 |     i18n.addResource("en", "fallbackTest", FALLBACK_KEY, EN_VALUE);

```

```

50 |     await i18n.changeLanguage("ar");
51 |
52 |     const resolved = resolveLocalizedOrFail(FALLBACK_KEY, {
53 |       ns: "fallbackTest",
54 |       fallbackToOtherLang: true,
55 |     });
56 |     expect(resolved).toBe(EN_VALUE);
57 |   });
58 |
59 |   it("returns null when the key resolves in no supported language", () => {
60 |     const resolved = resolveLocalizedOrFail("definitely.missing.key", {
61 |       ns: "fallbackTest",
62 |       fallbackToOtherLang: true,
63 |     });
64 |     expect(resolved).toBeNull();
65 |   });
66 |
67 |   it("does not fall back when fallbackToOtherLang is disabled", async () => {
68 |     i18n.addResource("ar", "fallbackTest", FALLBACK_KEY, AR_VALUE);
69 |     await i18n.changeLanguage("en");
70 |
71 |     const resolved = resolveLocalizedOrFail(FALLBACK_KEY, {
72 |       ns: "fallbackTest",
73 |       fallbackToOtherLang: false,
74 |     });
75 |     expect(resolved).toBeNull();
76 |   });
77 |
78 |   it("interpolates params (used by the heatmap plain-language summary)", () => {
79 |     const resolved = resolveLocalizedOrFail("heatmap.plainSummary.completion", {
80 |       ns: "student",
81 |       params: { rate: 80 },
82 |     });
83 |     expect(resolved).toContain("80");
84 |   });
85 | });
86 |
87 | // =====
88 | // resolveLocalizationGate – surface-level i18n gating (R13.5, R19.6, R19.7)
89 | // =====
90 | // The centralized gate decides whether a localized surface (timetable, weekly
91 | // planner) renders fully, withholds its actions, or is blocked entirely, based
92 | // on which supported language packs are available. The two policies encode the
93 | // distinct requirements: `block-only` (timetable, R13.5) versus
94 | // `withhold-actions` (planner, R19.6/19.7).
95 |
96 | describe("resolveLocalizationGate", () => {
97 |   const all = () => true;
98 |   const none = () => false;
99 |   const onlyEn = (lng: string) => lng === "en";
100 |   const onlyAr = (lng: string) => lng === "ar";
101 |
102 |   describe("withhold-actions policy (planner, R19.6/19.7)", () => {
103 |     it("is ready when every supported pack is available", () => {
104 |       expect(resolveLocalizationGate(all, "withhold-actions")).toBe("ready");
105 |     });
106 |
107 |     it("withholds actions when exactly one pack is available (R19.6)", () => {
108 |       expect(resolveLocalizationGate(onlyEn, "withhold-actions")).toBe(
109 |         "actions-withheld"
110 |       );
111 |       expect(resolveLocalizationGate(onlyAr, "withhold-actions")).toBe(
112 |         "actions-withheld"
113 |       );
114 |     });
115 |
116 |     it("blocks the surface when no pack is available (R19.7)", () => {
117 |       expect(resolveLocalizationGate(none, "withhold-actions")).toBe("blocked");
118 |     });
119 |
120 |     it("defaults to the withhold-actions policy", () => {
121 |       expect(resolveLocalizationGate(onlyEn)).toBe("actions-withheld");
122 |     });
123 |   });
124 |
125 |   describe("block-only policy (timetable, R13.5)", () => {
126 |     it("is ready when every supported pack is available", () => {
127 |       expect(resolveLocalizationGate(all, "block-only")).toBe("ready");
128 |     });
129 |
130 |     it("still renders when only one pack is available (no action gating)", () => {
131 |       expect(resolveLocalizationGate(onlyEn, "block-only")).toBe("ready");
132 |       expect(resolveLocalizationGate(onlyAr, "block-only")).toBe("ready");
133 |     });
134 |
135 |     it("blocks the surface only when neither pack is available (R13.5)", () => {
136 |       expect(resolveLocalizationGate(none, "block-only")).toBe("blocked");
137 |     });
138 |   });
139 | });
140 |

```

FILE: src/__tests__/unit/loginPage.test.tsx

TYPE: .tsx | LINES: 1-348 | SHA256: 612b3c043af86b5e81fb9c9dcfd1bf4e8c0f676e1b264e0b220ae0179bff982d0

```

1 | import { describe, it, expect, vi, beforeEach } from "vitest";
2 | import { render, screen, waitFor } from "@testing-library/react";
3 | import { fireEvent } from "@testing-library/user-event";
4 | import { MemoryRouter } from "react-router-dom";
5 | import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
6 | import "@lib/i18n";
7 |
8 | // -----
9 | // Mock react-router-dom navigate

```

```

10 // -----
11 const mockNavigate = vi.fn();
12 vi.mock("react-router-dom", async () => {
13   const actual = await vi.importActual("react-router-dom");
14   return { ...actual, useNavigate: () => mockNavigate };
15 });
16
17 // -----
18 // Mock Supabase client
19 // -----
20 const mockSignInWithPassword = vi.fn();
21 const mockGetSession = vi.fn();
22 const mockOnAuthStateChange = vi.fn();
23 const mockFrom = vi.fn();
24 const mockFunctionsInvoke = vi.fn();
25
26 vi.mock("@/lib/supabase", () => ({
27   supabase: {
28     auth: {
29       getSession: (...args: unknown[]) => mockGetSession(...args),
30       signInWithPassword: (...args: unknown[]) =>
31         mockSignInWithPassword(...args),
32       signUp: vi.fn(),
33       signOut: vi.fn(),
34       resetPasswordForEmail: vi.fn(),
35       onAuthStateChange: (...args: unknown[]) => mockOnAuthStateChange(...args),
36     },
37     from: (...args: unknown[]) => mockFrom(...args),
38     functions: {
39       invoke: (...args: unknown[]) => mockFunctionsInvoke(...args),
40     },
41   },
42 }));
43
44 import { AuthProvider } from "@/providers/AuthProvider";
45 import LoginPage from "@/pages/LoginPage";
46
47 // -----
48 // Test fixtures
49 // -----
50 const MOCK_USER = { id: "user-123", email: "admin@test.edu" };
51
52 const MOCK_PROFILE = {
53   id: "user-123",
54   email: "admin@test.edu",
55   full_name: "Test Admin",
56   role: "admin" as const,
57   institution_id: "inst-1",
58   avatar_url: null,
59   is_active: true,
60   onboarding_completed: false,
61   portfolio_public: false,
62   theme_preference: "light",
63   language_preference: "en",
64   email_preferences: null,
65   notification_preferences: null,
66   created_at: "2024-01-01T00:00:00Z",
67   updated_at: "2024-01-01T00:00:00Z",
68 };
69
70 const setupMocks = () => {
71   mockGetSession.mockResolvedValue({ data: { session: null } });
72   mockOnAuthStateChange.mockReturnValue({
73     data: { subscription: { unsubscribe: vi.fn() } },
74   });
75   mockFrom.mockReturnValue({
76     select: vi.fn().mockReturnValue({
77       eq: vi.fn().mockReturnValue({
78         maybeSingle: vi.fn().mockResolvedValue({ data: null, error: null }),
79       }),
80     }),
81   });
82   // Server-side rate limiting - default: not locked
83   mockFunctionsInvoke.mockResolvedValue({
84     data: {
85       locked: false,
86       remaining_seconds: 0,
87       cleared: true,
88       attempt_count: 0,
89     },
90     error: null,
91   });
92 };
93
94 const renderLoginPage = () => {
95   const queryClient = new QueryClient({
96     defaultOptions: { queries: { retry: false } },
97   });
98   return render(
99     <QueryClientProvider client={queryClient}>
100       <MemoryRouter>
101         <AuthProvider>
102           <LoginPage />
103         </AuthProvider>
104       </MemoryRouter>
105     </QueryClientProvider>
106   );
107 };
108
109 // -----
110 // Convenience selectors that match the new auth UI
111 // -----
112 // The prototype auth UI associates each field with a <label htmlFor>, so query
113 // by accessible label (robust to placeholder copy). Login-tab labels are unique.

```

```

114 | const getEmailInput = () => screen.getByLabelText("Email Address");
115 | const getPasswordInput = () => screen.getByLabelText("Password");
116 | const getSignInButton = () =>
117 |   screen.getByRole("button", { name: /^Sign In$/i });
118 |
119 | // -----
120 | // Tests
121 | // -----
122 | describe("LoginPage", () => {
123 |   beforeEach(() => {
124 |     vi.clearAllMocks();
125 |     localStorage.clear();
126 |     setupMocks();
127 |   });
128 |
129 |   it("renders the auth card with Login and Register tabs", async () => {
130 |     renderLoginPage();
131 |
132 |     await waitFor(() => {
133 |       expect(getEmailInput()).toBeInTheDocument();
134 |     });
135 |     expect(getPasswordInput()).toBeInTheDocument();
136 |     expect(getSignInButton()).toBeInTheDocument();
137 |     // Tabs render as buttons in the prototype auth UI (not ARIA tabs).
138 |     expect(screen.getByRole("button", { name: "Login" })).toBeInTheDocument();
139 |     expect(
140 |       screen.getByRole("button", { name: "Register" })
141 |     ).toBeInTheDocument();
142 |   });
143 |
144 |   it('has a "Forgot password?" link to /reset-password', async () => {
145 |     renderLoginPage();
146 |
147 |     await waitFor(() => {
148 |       expect(screen.getByText("Forgot password?")).toBeInTheDocument();
149 |     });
150 |
151 |     const link = screen.getByText("Forgot password?");
152 |     expect(link.closest("a")).toHaveAttribute("href", "/reset-password");
153 |   });
154 |
155 |   it("shows validation error for invalid email", async () => {
156 |     const user = userEvent.setup();
157 |     renderLoginPage();
158 |
159 |     await waitFor(() => {
160 |       expect(getEmailInput()).toBeInTheDocument();
161 |     });
162 |
163 |     // Submit with empty email to trigger validation
164 |     await user.type(getPasswordInput(), "password123");
165 |     await user.click(getSignInButton());
166 |
167 |     await waitFor(() => {
168 |       expect(screen.getByText(/invalid email/i)).toBeInTheDocument();
169 |     });
170 |     expect(mockSignInWithPassword).not.toHaveBeenCalled();
171 |   });
172 |
173 |   it("shows validation error for short password", async () => {
174 |     const user = userEvent.setup();
175 |     renderLoginPage();
176 |
177 |     await waitFor(() => {
178 |       expect(getEmailInput()).toBeInTheDocument();
179 |     });
180 |
181 |     await user.type(getEmailInput(), "user@test.edu");
182 |     await user.type(getPasswordInput(), "short");
183 |     await user.click(getSignInButton());
184 |
185 |     await waitFor(() => {
186 |       expect(screen.getByText(/at least 8 characters/i)).toBeInTheDocument();
187 |     });
188 |     expect(mockSignInWithPassword).not.toHaveBeenCalled();
189 |   });
190 |
191 |   it("navigates to role dashboard on successful login", async () => {
192 |     mockSignInWithPassword.mockResolvedValue({
193 |       data: { user: MOCK_USER, session: { user: MOCK_USER } },
194 |       error: null,
195 |     });
196 |     mockFrom.mockReturnValue({
197 |       select: vi.fn().mockReturnValue({
198 |         eq: vi.fn().mockReturnValue({
199 |           maybeSingle: vi
200 |             .fn()
201 |             .mockResolvedValue({ data: MOCK_PROFILE, error: null }),
202 |         }),
203 |       }),
204 |     });
205 |
206 |     const user = userEvent.setup();
207 |     renderLoginPage();
208 |
209 |     await waitFor(() => {
210 |       expect(getEmailInput()).toBeInTheDocument();
211 |     });
212 |
213 |     await user.type(getEmailInput(), "admin@test.edu");
214 |     await user.type(getPasswordInput(), "password123");
215 |     await user.click(getSignInButton());
216 |
217 |     await waitFor(() => {

```

```

218 |     expect(mockNavigate).toHaveBeenCalledWith("/admin", { replace: true });
219 |   });
220 | }, 15_000);
221 |
222 | it("shows generic error message on failed login (Req 1.3)", async () => {
223 |   mockSignInWithPassword.mockResolvedValue({
224 |     data: { user: null, session: null },
225 |     error: { message: "Invalid login credentials" },
226 |   });
227 |
228 |   const user = userEvent.setup();
229 |   renderLoginPage();
230 |
231 |   await waitFor(() => {
232 |     expect(getEmailInput()).toBeInTheDocument();
233 |   });
234 |
235 |   await user.type(getEmailInput(), "bad@test.edu");
236 |   await user.type(getPasswordInput(), "wrongpass1");
237 |   await user.click(getSignInButton());
238 |
239 |   await waitFor(() => {
240 |     expect(
241 |       screen.getByText("Invalid email or password.")
242 |     ).toBeInTheDocument();
243 |   });
244 | });
245 |
246 | it("shows lockout message when account is locked (Req 1.2)", async () => {
247 |   // Pre-lock the account by recording 5 failed attempts
248 |   const { recordFailedAttempt } = await import("@/lib/loginAttemptTracker");
249 |   for (let i = 0; i < 5; i++) {
250 |     recordFailedAttempt("locked@test.edu");
251 |   }
252 |
253 |   const user = userEvent.setup();
254 |   renderLoginPage();
255 |
256 |   await waitFor(() => {
257 |     expect(getEmailInput()).toBeInTheDocument();
258 |   });
259 |
260 |   await user.type(getEmailInput(), "locked@test.edu");
261 |   await user.type(getPasswordInput(), "anypass123");
262 |   await user.click(getSignInButton());
263 |
264 |   await waitFor(() => {
265 |     expect(screen.getByText(/temporarily locked/i)).toBeInTheDocument();
266 |   });
267 |   // Should NOT call Supabase when locked
268 |   expect(mockSignInWithPassword).not.toHaveBeenCalled();
269 | });
270 |
271 | it("does not submit when both fields are empty", async () => {
272 |   const user = userEvent.setup();
273 |   renderLoginPage();
274 |
275 |   await waitFor(() => {
276 |     expect(getSignInButton()).toBeInTheDocument();
277 |   });
278 |
279 |   await user.click(getSignInButton());
280 |
281 |   await waitFor(() => {
282 |     expect(screen.getByText(/invalid email address/i)).toBeInTheDocument();
283 |   });
284 |   expect(mockSignInWithPassword).not.toHaveBeenCalled();
285 | });
286 | });
287 |
288 | // -----
289 | // Register tab – self-signup role affordance (Production Bug Fixes – Req 5)
290 | //
291 | // The server `handle_new_user` trigger forces role='student' for self-signup
292 | // (no invitation_id) and ignores any requested role. The register tab must
293 | // therefore NOT offer privileged roles (admin/teacher/coordinator/parent) that
294 | // the backend would silently ignore – otherwise a user could pick "Admin" and
295 | // quietly receive a student account. This keeps LoginPage consistent with the
296 | // dedicated SignUpPage, which hard-codes 'student'. See Requirement 5.
297 | // -----
298 | describe("LoginPage – register tab role affordance (Req 5)", () => {
299 |   beforeEach(() => {
300 |     vi.clearAllMocks();
301 |     localStorage.clear();
302 |     setupMocks();
303 |   });
304 |
305 |   const openRegisterTab = async () => {
306 |     const user = userEvent.setup();
307 |     renderLoginPage();
308 |     await waitFor(() => {
309 |       expect(
310 |         screen.getByRole("button", { name: "Register" })
311 |       ).toBeInTheDocument();
312 |     });
313 |     await user.click(screen.getByRole("button", { name: "Register" }));
314 |     // Register form has mounted once the first-name field is present.
315 |     await waitFor(() => {
316 |       expect(screen.getByPlaceholderText("First")).toBeInTheDocument();
317 |     });
318 |   };
319 |
320 |   it("offers role selection options for Admin, Coordinator, Teacher, Student, and Parent", async () => {
321 |     await openRegisterTab();

```

```

322 // Verify all 5 role options are present
323 expect(screen.getByRole("button", { name: /admin/i })).toBeInTheDocument();
324 expect(
325   screen.getByRole("button", { name: /coordinator/i })
326   ).toBeInTheDocument();
327 expect(
328   screen.getByRole("button", { name: /teacher/i })
329   ).toBeInTheDocument();
330 expect(
331   screen.getByRole("button", { name: /student/i })
332   ).toBeInTheDocument();
333 expect(screen.getByRole("button", { name: /parent/i })).toBeInTheDocument();
334 });
335
336 it("communicates that self-registration creates a student account", async () => {
337   await openRegisterTab();
338
339   // A read-only "Student" segment replaces the role picker...
340   expect(screen.getByText("Student")).toBeInTheDocument();
341   // ...alongside the trust hint that staff roles require an invitation.
342   expect(
343     screen.getByText(/staff roles require an invitation/i)
344     ).toBeInTheDocument();
345   });
346 });
347
348

```

TYPE: .tsx | LINES: 1-101 | SHA256: 3c05ab477ca207fd317d49bb0ecd46d552152ddf219f1fe3aa1f20fe52caa712

```

74     expect(screen.getByText("Ocean Blue Theme")).toBeTruth();
75     expect(screen.getByText("Extra Quiz Attempt")).toBeTruth();
76 });
77
78 it("renders category badges", () => {
79     renderWithProviders();
80     expect(screen.getByText("Cosmetic")).toBeTruth();
81     expect(screen.getByText("Educational Perk")).toBeTruth();
82 });
83
84 it("renders active/inactive status badges", () => {
85     renderWithProviders();
86     expect(screen.getByText("Active")).toBeTruth();
87     expect(screen.getByText("Inactive")).toBeTruth();
88 });
89
90 it("renders Sale Events and Analytics navigation buttons", () => {
91     renderWithProviders();
92     expect(screen.getByText("Sale Events")).toBeTruth();
93     expect(screen.getByText("Analytics")).toBeTruth();
94 });
95
96 it("renders search input", () => {
97     renderWithProviders();
98     expect(screen.getByPlaceholderText("Search items...")).toBeTruth();
99 });
100 });
101

```

```
TYPE: .tsx | LINES: 1-135 | SHA256: 58692da2618fb62ef11209c503400a28cfc05978ff9178bd913229f60b151240
```

[illegible]

[illegible]

FILE: src/__tests__/unit/marketplaceSchemas.test.ts

TYPE: .ts | LINES: 1-309 | SHA256: 3662807d5180d002f9e51535438d4595aae44af00a6594fd128c3eddde3e27a8

```

1 // @vitest-environment happy-dom
2 import { describe, it, expect } from "vitest";
3 import {
4   createMarketplaceItemSchema,
5   updateMarketplaceItemSchema,
6   createSaleEventSchema,
7   purchaseRequestSchema,
8   deadlineExtensionSchema,
9   marketplaceItemCategorySchema,
10  marketplaceItemSubCategorySchema,
11  stockTypeSchema,
12 } from "@lib/marketplaceSchemas";
13
14 // %% createMarketplaceItemSchema %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %%
15
16 describe("createMarketplaceItemSchema", () => {
17   const validItem = {
18     name: "Ocean Blue Theme",
19     description: "A calming ocean-inspired color palette",
20     category: "cosmetic",
21     sub_category: "profile_theme",
22     xp_price: 500,
23     level_requirement: 5,
24     stock_type: "unlimited",
25     stock_quantity: null,
26     icon_identifier: "palette",
27   };
28
29   it("accepts a valid full payload", () => {
30     const result = createMarketplaceItemSchema.safeParse(validItem);
31     expect(result.success).toBe(true);
32   });
33
34   it("accepts payload with optional metadata", () => {
35     const result = createMarketplaceItemSchema.safeParse({
36       ...validItem,
37       metadata: { theme_id: "ocean_blue", accent_primary: "#0ea5e9" },
38     });
39     expect(result.success).toBe(true);
40   });
41
42   it("rejects invalid payload", () => {
43     const result = createMarketplaceItemSchema.safeParse({
44       name: "Ocean Blue Theme",
45       description: "A calming ocean-inspired color palette",
46       category: "cosmetic",
47       sub_category: "profile_theme",
48       xp_price: 500,
49       level_requirement: 5,
50       stock_type: "unlimited",
51       stock_quantity: null,
52       icon_identifier: "palette",
53       metadata: { theme_id: "ocean_blue", accent_primary: "#0ea5e9" },
54     });
55     expect(result.success).toBe(false);
56     expect(result.error?.message).toContain("Invalid payload");
57   });
58
59   it("rejects invalid payload with missing required fields", () => {
60     const result = createMarketplaceItemSchema.safeParse({
61       name: "Ocean Blue Theme",
62       description: "A calming ocean-inspired color palette",
63       category: "cosmetic",
64       xp_price: 500,
65       level_requirement: 5,
66       stock_type: "unlimited",
67       stock_quantity: null,
68       icon_identifier: "palette",
69     });
70     expect(result.success).toBe(false);
71     expect(result.error?.message).toContain("Invalid payload");
72   });
73
74   it("rejects invalid payload with invalid stock type", () => {
75     const result = createMarketplaceItemSchema.safeParse({
76       name: "Ocean Blue Theme",
77       description: "A calming ocean-inspired color palette",
78       category: "cosmetic",
79       sub_category: "profile_theme",
80       xp_price: 500,
81       level_requirement: 5,
82       stock_type: "invalid",
83       stock_quantity: null,
84       icon_identifier: "palette",
85     });
86     expect(result.success).toBe(false);
87     expect(result.error?.message).toContain("Invalid payload");
88   });
89
90   it("rejects invalid payload with invalid stock quantity", () => {
91     const result = createMarketplaceItemSchema.safeParse({
92       name: "Ocean Blue Theme",
93       description: "A calming ocean-inspired color palette",
94       category: "cosmetic",
95       sub_category: "profile_theme",
96       xp_price: 500,
97       level_requirement: 5,
98       stock_type: "unlimited",
99       stock_quantity: -1,
100      icon_identifier: "palette",
101    });
102    expect(result.success).toBe(false);
103    expect(result.error?.message).toContain("Invalid payload");
104  });
105
106   it("rejects invalid payload with invalid icon identifier", () => {
107     const result = createMarketplaceItemSchema.safeParse({
108       name: "Ocean Blue Theme",
109       description: "A calming ocean-inspired color palette",
110       category: "cosmetic",
111       sub_category: "profile_theme",
112       xp_price: 500,
113       level_requirement: 5,
114       stock_type: "unlimited",
115       stock_quantity: null,
116       icon_identifier: "invalid",
117     });
118     expect(result.success).toBe(false);
119     expect(result.error?.message).toContain("Invalid payload");
120   });
121
122   it("rejects invalid payload with invalid metadata", () => {
123     const result = createMarketplaceItemSchema.safeParse({
124       name: "Ocean Blue Theme",
125       description: "A calming ocean-inspired color palette",
126       category: "cosmetic",
127       sub_category: "profile_theme",
128       xp_price: 500,
129       level_requirement: 5,
130       stock_type: "unlimited",
131       stock_quantity: null,
132       icon_identifier: "palette",
133       metadata: { theme_id: "invalid", accent_primary: "#0ea5e9" },
134     });
135     expect(result.success).toBe(false);
136     expect(result.error?.message).toContain("Invalid payload");
137   });
138
139   it("rejects invalid payload with invalid metadata fields", () => {
140     const result = createMarketplaceItemSchema.safeParse({
141       name: "Ocean Blue Theme",
142       description: "A calming ocean-inspired color palette",
143       category: "cosmetic",
144       sub_category: "profile_theme",
145       xp_price: 500,
146       level_requirement: 5,
147       stock_type: "unlimited",
148       stock_quantity: null,
149       icon_identifier: "palette",
150       metadata: { theme_id: "ocean_blue", accent_primary: "invalid" },
151     });
152     expect(result.success).toBe(false);
153     expect(result.error?.message).toContain("Invalid payload");
154   });
155
156   it("rejects invalid payload with invalid metadata fields", () => {
157     const result = createMarketplaceItemSchema.safeParse({
158       name: "Ocean Blue Theme",
159       description: "A calming ocean-inspired color palette",
160       category: "cosmetic",
161       sub_category: "profile_theme",
162       xp_price: 500,
163       level_requirement: 5,
164       stock_type: "unlimited",
165       stock_quantity: null,
166       icon_identifier: "palette",
167       metadata: { theme_id: "ocean_blue", accent_primary: "#0ea5e9", invalid_field: "invalid" },
168     });
169     expect(result.success).toBe(false);
170     expect(result.error?.message).toContain("Invalid payload");
171   });
172
173   it("rejects invalid payload with invalid metadata fields", () => {
174     const result = createMarketplaceItemSchema.safeParse({
175       name: "Ocean Blue Theme",
176       description: "A calming ocean-inspired color palette",
177       category: "cosmetic",
178       sub_category: "profile_theme",
179       xp_price: 500,
180       level_requirement: 5,
181       stock_type: "unlimited",
182       stock_quantity: null,
183       icon_identifier: "palette",
184       metadata: { theme_id: "ocean_blue", accent_primary: "#0ea5e9", invalid_field: "invalid" },
185     });
186     expect(result.success).toBe(false);
187     expect(result.error?.message).toContain("Invalid payload");
188   });
189
190   it("rejects invalid payload with invalid metadata fields", () => {
191     const result = createMarketplaceItemSchema.safeParse({
192       name: "Ocean Blue Theme",
193       description: "A calming ocean-inspired color palette",
194       category: "cosmetic",
195       sub_category: "profile_theme",
196       xp_price: 500,
197       level_requirement: 5,
198       stock_type: "unlimited",
199       stock_quantity: null,
200       icon_identifier: "palette",
201       metadata: { theme_id: "ocean_blue", accent_primary: "#0ea5e9", invalid_field: "invalid" },
202     });
203     expect(result.success).toBe(false);
204     expect(result.error?.message).toContain("Invalid payload");
205   });
206
207   it("rejects invalid payload with invalid metadata fields", () => {
208     const result = createMarketplaceItemSchema.safeParse({
209       name: "Ocean Blue Theme",
210       description: "A calming ocean-inspired color palette",
211       category: "cosmetic",
212       sub_category: "profile_theme",
213       xp_price: 500,
214       level_requirement: 5,
215       stock_type: "unlimited",
216       stock_quantity: null,
217       icon_identifier: "palette",
218       metadata: { theme_id: "ocean_blue", accent_primary: "#0ea5e9", invalid_field: "invalid" },
219     });
220     expect(result.success).toBe(false);
221     expect(result.error?.message).toContain("Invalid payload");
222   });
223
224   it("rejects invalid payload with invalid metadata fields", () => {
225     const result = createMarketplaceItemSchema.safeParse({
226       name: "Ocean Blue Theme",
227       description: "A calming ocean-inspired color palette",
228       category: "cosmetic",
229       sub_category: "profile_theme",
230       xp_price: 500,
231       level_requirement: 5,
232       stock_type: "unlimited",
233       stock_quantity: null,
234       icon_identifier: "palette",
235       metadata: { theme_id: "ocean_blue", accent_primary: "#0ea5e9", invalid_field: "invalid" },
236     });
237     expect(result.success).toBe(false);
238     expect(result.error?.message).toContain("Invalid payload");
239   });
240
241   it("rejects invalid payload with invalid metadata fields", () => {
242     const result = createMarketplaceItemSchema.safeParse({
243       name: "Ocean Blue Theme",
244       description: "A calming ocean-inspired color palette",
245       category: "cosmetic",
246       sub_category: "profile_theme",
247       xp_price: 500,
248       level_requirement: 5,
249       stock_type: "unlimited",
250       stock_quantity: null,
251       icon_identifier: "palette",
252       metadata: { theme_id: "ocean_blue", accent_primary: "#0ea5e9", invalid_field: "invalid" },
253     });
254     expect(result.success).toBe(false);
255     expect(result.error?.message).toContain("Invalid payload");
256   });
257
258   it("rejects invalid payload with invalid metadata fields", () => {
259     const result = createMarketplaceItemSchema.safeParse({
260       name: "Ocean Blue Theme",
261       description: "A calming ocean-inspired color palette",
262       category: "cosmetic",
263       sub_category: "profile_theme",
264       xp_price: 500,
265       level_requirement: 5,
266       stock_type: "unlimited",
267       stock_quantity: null,
268       icon_identifier: "palette",
269       metadata: { theme_id: "ocean_blue", accent_primary: "#0ea5e9", invalid_field: "invalid" },
270     });
271     expect(result.success).toBe(false);
272     expect(result.error?.message).toContain("Invalid payload");
273   });
274
275   it("rejects invalid payload with invalid metadata fields", () => {
276     const result = createMarketplaceItemSchema.safeParse({
277       name: "Ocean Blue Theme",
278       description: "A calming ocean-inspired color palette",
279       category: "cosmetic",
280       sub_category: "profile_theme",
281       xp_price: 500,
282       level_requirement: 5,
283       stock_type: "unlimited",
284       stock_quantity: null,
285       icon_identifier: "palette",
286       metadata: { theme_id: "ocean_blue", accent_primary: "#0ea5e9", invalid_field: "invalid" },
287     });
288     expect(result.success).toBe(false);
289     expect(result.error?.message).toContain("Invalid payload");
290   });
291
292   it("rejects invalid payload with invalid metadata fields", () => {
293     const result = createMarketplaceItemSchema.safeParse({
294       name: "Ocean Blue Theme",
295       description: "A calming ocean-inspired color palette",
296       category: "cosmetic",
297       sub_category: "profile_theme",
298       xp_price: 500,
299       level_requirement: 5,
300       stock_type: "unlimited",
301       stock_quantity: null,
302       icon_identifier: "palette",
303       metadata: { theme_id: "ocean_blue", accent_primary: "#0ea5e9", invalid_field: "invalid" },
304     });
305     expect(result.success).toBe(false);
306     expect(result.error?.message).toContain("Invalid payload");
307   });
308
309   it("rejects invalid payload with invalid metadata fields", () => {
310     const result = createMarketplaceItemSchema.safeParse({
311       name: "Ocean Blue Theme",
312       description: "A calming ocean-inspired color palette",
313       category: "cosmetic",
314       sub_category: "profile_theme",
315       xp_price: 500,
316       level_requirement: 5,
317       stock_type: "unlimited",
318       stock_quantity: null,
319       icon_identifier: "palette",
320       metadata: { theme_id: "ocean_blue", accent_primary: "#0ea5e9", invalid_field: "invalid" },
321     });
322     expect(result.success).toBe(false);
323     expect(result.error?.message).toContain("Invalid payload");
324   });
325
326   it("rejects invalid payload with invalid metadata fields", () => {
327     const result = createMarketplaceItemSchema.safeParse({
328       name: "Ocean Blue Theme",
329       description: "A calming ocean-inspired color palette",
330       category: "cosmetic",
331       sub_category: "profile_theme",
332       xp_price: 500,
333       level_requirement: 5,
334       stock_type: "
```



```

38 |     });
39 |     expect(result.success).toBe(true);
40 | });
41 |
42 | it("applies default level_requirement of 0", () => {
43 |     // eslint-disable-next-line @typescript-eslint/no-unused-vars
44 |     const { level_requirement: _levelReq, ...withoutLevel } = validItem;
45 |     const result = createMarketplaceItemSchema.safeParse(withoutLevel);
46 |     expect(result.success).toBe(true);
47 |     if (result.success) {
48 |         expect(result.data.level_requirement).toBe(0);
49 |     }
50 | });
51 |
52 | it("rejects empty name", () => {
53 |     const result = createMarketplaceItemSchema.safeParse({
54 |         ...validItem,
55 |         name: "",
56 |     });
57 |     expect(result.success).toBe(false);
58 | });
59 |
60 | it("rejects name exceeding 100 characters", () => {
61 |     const result = createMarketplaceItemSchema.safeParse({
62 |         ...validItem,
63 |         name: "a".repeat(101),
64 |     });
65 |     expect(result.success).toBe(false);
66 | });
67 |
68 | it("rejects empty description", () => {
69 |     const result = createMarketplaceItemSchema.safeParse({
70 |         ...validItem,
71 |         description: "",
72 |     });
73 |     expect(result.success).toBe(false);
74 | });
75 |
76 | it("rejects description exceeding 500 characters", () => {
77 |     const result = createMarketplaceItemSchema.safeParse({
78 |         ...validItem,
79 |         description: "a".repeat(501),
80 |     });
81 |     expect(result.success).toBe(false);
82 | });
83 |
84 | it("rejects invalid category", () => {
85 |     const result = createMarketplaceItemSchema.safeParse({
86 |         ...validItem,
87 |         category: "invalid",
88 |     });
89 |     expect(result.success).toBe(false);
90 | });
91 |
92 | it("rejects invalid sub_category", () => {
93 |     const result = createMarketplaceItemSchema.safeParse({
94 |         ...validItem,
95 |         sub_category: "invalid",
96 |     });
97 |     expect(result.success).toBe(false);
98 | });
99 |
100 | it("rejects zero xp_price", () => {
101 |     const result = createMarketplaceItemSchema.safeParse({
102 |         ...validItem,
103 |         xp_price: 0,
104 |     });
105 |     expect(result.success).toBe(false);
106 | });
107 |
108 | it("rejects negative xp_price", () => {
109 |     const result = createMarketplaceItemSchema.safeParse({
110 |         ...validItem,
111 |         xp_price: -100,
112 |     });
113 |     expect(result.success).toBe(false);
114 | });
115 |
116 | it("rejects non-integer xp_price", () => {
117 |     const result = createMarketplaceItemSchema.safeParse({
118 |         ...validItem,
119 |         xp_price: 99.5,
120 |     });
121 |     expect(result.success).toBe(false);
122 | });
123 |
124 | it("rejects negative level_requirement", () => {
125 |     const result = createMarketplaceItemSchema.safeParse({
126 |         ...validItem,
127 |         level_requirement: -1,
128 |     });
129 |     expect(result.success).toBe(false);
130 | });
131 |
132 | it("rejects invalid stock_type", () => {
133 |     const result = createMarketplaceItemSchema.safeParse({
134 |         ...validItem,
135 |         stock_type: "rare",
136 |     });
137 |     expect(result.success).toBe(false);
138 | });
139 |
140 | it("rejects empty icon_identifier", () => {
141 |     const result = createMarketplaceItemSchema.safeParse({

```

[illegible]

[illegible]

FILE: src/__tests__/unit/mascot.test.tsx

```
TYPE: .tsx | LINES: 1-65 | SHA256: 72a55724110bfde69e3936ba7b9ec1107d3c35fe3a3501f989e0a0e5d6637f7c
```

```

1 // =====
2 // Component tests – Mascot coaching surface (R35)
3 // =====
4
5 import { describe, it, expect, beforeEach, afterEach } from "vitest";
6 import { render, screen, cleanup } from "@testing-library/react";
7 import { I18nextProvider } from "react-i18next";
8
9 import i18n from "@lib/i18n";
10 import Mascot from "@components/shared/Mascot";
11 import { setMascotGuidanceEnabled } from "@hooks/useMascotGuidance";
12
13 const renderMascot = (moment: Parameters<typeof Mascot>[0]["moment"]) =>
14   render(
15     <I18nextProvider i18n={i18n}>
16       <Mascot moment={moment} />
17     </I18nextProvider>
18   );
19
20 beforeEach(() => {
21   localStorage.clear();
22 });
23
24 afterEach(() => {
25   cleanup();
26   localStorage.clear();
27 });
28
29 describe("Mascot", () => {
30   it("renders coaching copy for the welcome moment when enabled (R35.1)", async () => {
31     await i18n.changeLanguage("en");
32     setMascotGuidanceEnabled(true);
33     renderMascot("welcome");
34     expect(screen.getByText("Hi, I'm Edi!")).toBeInTheDocument();
35     expect(screen.getByRole("complementary")).toBeInTheDocument();
36   });
37 }

```

```

37 |
38 | it("renders the password-screen coaching presence (R35.2)", async () => {
39 |   await i18n.changeLanguage("en");
40 |   setMascotGuidanceEnabled(true);
41 |   renderMascot("password");
42 |   expect(screen.getByText("Keeping you safe")).toBeInTheDocument();
43 | });
44 |
45 | it("renders nothing when mascot guidance is disabled (R35.4)", () => {
46 |   setMascotGuidanceEnabled(false);
47 |   const { container } = renderMascot("welcome");
48 |   expect(container).toBeEmptyDOMElement();
49 | });
50 |
51 | it("renders nothing when no moment is active (R35.5)", () => {
52 |   setMascotGuidanceEnabled(true);
53 |   const { container } = renderMascot(null);
54 |   expect(container).toBeEmptyDOMElement();
55 | });
56 |
57 | it("renders Arabic copy when the language is Arabic (R35.3)", async () => {
58 |   await i18n.changeLanguage("ar");
59 |   setMascotGuidanceEnabled(true);
60 |   renderMascot("welcome");
61 |   expect(screen.getByText("dV1b0(d¶'`Â #df' bVJb0J!")).toBeInTheDocument();
62 |   await i18n.changeLanguage("en");
63 | });
64 | });
65 |

```

FILE: src/___tests___/unit/MascotCharacter.test.tsx

TYPE: .tsx | LINES: 1-55 | SHA256: 0a7a3de49f2239bc517595bec9bfaca030348d2c07478d646cf1ae8ae6f0cbb7

```

1 | // Render behavior for the mascot components (L2). Feature: prototype-frontend-rebuild (P0.5).
2 | import { describe, it, expect } from "vitest";
3 | import { render } from "@testing-library/react";
4 | import MascotCharacter from "@design-system/mascot/MascotCharacter";
5 | import MascotCompanion from "@design-system/mascot/MascotCompanion";
6 |
7 | describe("MascotCharacter", () => {
8 |   it("renders a non-draggable, sized img with a resolved src and meaningful alt", () => {
9 |     const { container } = render(
10 |       <MascotCharacter character="foxi" emotion="celebrating" size="lg" />
11 |     );
12 |     const img = container.querySelector("img") as HTMLImageElement;
13 |     expect(img).toBeTruthy();
14 |     expect(img.getAttribute("draggable")).toBe("false");
15 |     expect(img.getAttribute("alt")).toContain("Foxy");
16 |     expect(img.getAttribute("src")).toBeTruthy();
17 |     expect(img.style.width).toBe("150px");
18 |     expect(img.className).toContain("edv-mascot");
19 |   });
20 |
21 | it("supports decorative mode (empty alt + aria-hidden)", () => {
22 |   const { container } = render(
23 |     <MascotCharacter character="owl" emotion="wise" decorative />
24 |   );
25 |   const img = container.querySelector("img") as HTMLImageElement;
26 |   expect(img.getAttribute("alt")).toBe("");
27 |   expect(img.getAttribute("aria-hidden")).toBe("true");
28 | });
29 |
30 | it("applies the requested motion preset class", () => {
31 |   const { container } = render(
32 |     <MascotCharacter character="penguin" emotion="happy" animation="float" />
33 |   );
34 |   const img = container.querySelector("img") as HTMLImageElement;
35 |   expect(img.className).toContain("edv-mascot-float");
36 | });
37 | });
38 |
39 | describe("MascotCompanion", () => {
40 |   it("renders the character image plus a titled speech bubble", () => {
41 |     const { container, getByText } = render(
42 |       <MascotCompanion
43 |         character="foxi"
44 |         emotion="happy"
45 |         title="Nice work!"
46 |         message="You kept your streak alive."
47 |       />
48 |     );
49 |     expect(container.querySelector("img")).toBeTruthy();
50 |     expect(getByText("Nice work!")).toBeTruthy();
51 |     expect(getByText("You kept your streak alive.")).toBeTruthy();
52 |     expect(container.querySelector(".edv-mascot-bubble")).toBeTruthy();
53 |   });
54 | });
55 |

```

FILE: src/___tests___/unit/mascotGuidance.test.ts

TYPE: .ts | LINES: 1-69 | SHA256: 989cfe15ce22127644ecbcd797b9a5ef41ee0f39aaa795462e582bbe12bad737

```

1 | // =====
2 | // Unit tests – mascot guidance moment! 'copy logic' (R35)
3 | // =====
4 |
5 | import { describe, it, expect } from "vitest";
6 | import {
7 |   resolveMascotGuidance,
8 |   parseMascotEnabled,
9 |   MASCOT_MOMENTS,
10 |   MASCOT_MOMENT_IDS,
11 |   MASCOT_ENABLED_DEFAULT,
12 |   type MascotMomentId,

```

```

13 | } from "@lib/mascotGuidance";
14 |
15 | describe("resolveMascotGuidance", () => {
16 |   it("returns guidance for every key moment when enabled (R35.1, R35.2)", () => {
17 |     for (const moment of MASCOT_MOMENT_IDS) {
18 |       const guidance = resolveMascotGuidance({ enabled: true, moment });
19 |       expect(guidance).not.toBeNull();
20 |       expect(guidance?.moment).toBe(moment);
21 |       expect(guidance?.i18nKey).toBe(MASCOT_MOMENTS[moment].i18nKey);
22 |       expect(guidance?.tone).toBe(MASCOT_MOMENTS[moment].tone);
23 |     }
24 |   });
25 |
26 |   it("returns null for every moment when disabled (R35.4)", () => {
27 |     for (const moment of MASCOT_MOMENT_IDS) {
28 |       expect(resolveMascotGuidance({ enabled: false, moment })).toBeNull();
29 |     }
30 |   });
31 |
32 |   it("returns null when no moment is active, even if enabled (R35.5)", () => {
33 |     expect(resolveMascotGuidance({ enabled: true, moment: null })).toBeNull();
34 |   });
35 |
36 |   it("returns null when disabled and no moment is active", () => {
37 |     expect(resolveMascotGuidance({ enabled: false, moment: null })).toBeNull();
38 |   });
39 |
40 |   it("returns null for an unknown moment id without throwing", () => {
41 |     const guidance = resolveMascotGuidance({
42 |       enabled: true,
43 |       moment: "not-a-moment" as MascotMomentId,
44 |     });
45 |     expect(guidance).toBeNull();
46 |   });
47 |
48 |   it("maps the password moment so coaching stays consistent on password screens (R35.2)", () => {
49 |     const guidance = resolveMascotGuidance({
50 |       enabled: true,
51 |       moment: "password",
52 |     });
53 |     expect(guidance?.i18nKey).toBe("mascot.moments.password");
54 |   });
55 | });
56 |
57 | describe("parseMascotEnabled", () => {
58 |   it("parses 'true'/'false' exactly", () => {
59 |     expect(parseMascotEnabled("true")).toBe(true);
60 |     expect(parseMascotEnabled("false")).toBe(false);
61 |   });
62 |
63 |   it("falls back to the default for null or malformed values", () => {
64 |     expect(parseMascotEnabled(null)).toBe(MASCOT_ENABLED_DEFAULT);
65 |     expect(parseMascotEnabled("")).toBe(MASCOT_ENABLED_DEFAULT);
66 |     expect(parseMascotEnabled("yes")).toBe(MASCOT_ENABLED_DEFAULT);
67 |   });
68 | });
69 |

```

FILE: [src/__tests__/unit/masteryFailureDetection.test.ts](#)

TYPE: .ts | LINES: 1-290 | SHA256: 1d5f89181357e3ef3955a7cf362b8a7dadb91ad59b5d255b87025be141af5908

```

1 | // @vitest-environment happy-dom
2 | import { describe, it, expect } from "vitest";
3 | import {
4 |   countMasteryFailures,
5 |   shouldActivateRecovery,
6 | } from "@lib/masteryRecovery";
7 |
8 | /**
9 |  * Tests for the mastery failure detection integration logic.
10 |  *
11 |  * The actual integration runs server-side in the update-question-analytics
12 |  * Edge Function. These tests validate the core decision logic:
13 |  * 1. computePerCLOScores – per-CLO score computation from attempt data
14 |  * 2. Failure counting + recovery activation threshold
15 |  *
16 |  * The Edge Function's computePerCLOScores mirrors the logic tested here
17 |  * using the same approach as computePerCLOScore from questionAnalytics.ts.
18 |  */
19 |
20 | // % % % Helper: mirrors Edge Function's computePerCLOScores % % % % % % % % % % % % % % % % % %
21 |
22 | interface QuestionSequenceEntry {
23 |   question_id: string;
24 |   difficulty_rating: number;
25 |   bloom_level: number;
26 | }
27 |
28 | interface QuestionBankRow {
29 |   id: string;
30 |   difficulty_rating: number;
31 |   clo_id: string;
32 | }
33 |
34 | function computePerCLOScores(
35 |   questionSequence: QuestionSequenceEntry[],
36 |   answersMap: Map<string, boolean>,
37 |   questionBankMap: Map<string, QuestionBankRow>
38 | ): Record<string, number> {
39 |   const totals: Record<string, number> = {};
40 |   const corrects: Record<string, number> = {};
41 |
42 |   for (const seqEntry of questionSequence) {
43 |     const qbRow = questionBankMap.get(seqEntry.question_id);

```

[illegible]

[illegible]

```

252 |     { clo_scores: { "clo-1": 60 } },
253 | ];
254 | const previousFailures = countMasteryFailures(previousAttempts, "clo-1");
255 | const totalFailures = previousFailures + 1;
256 |
257 | expect(totalFailures).toBe(4);
258 | expect(shouldActivateRecovery(totalFailures)).toBe(true);
259 | });
260 |
261 | it("handles mixed CLO results – only failing CLOs trigger recovery check", () => {
262 |     const seq: QuestionSequenceEntry[] = [
263 |         { question_id: "q1", difficulty_rating: 3.0, bloom_level: 2 },
264 |         { question_id: "q2", difficulty_rating: 3.0, bloom_level: 2 },
265 |         { question_id: "q3", difficulty_rating: 3.0, bloom_level: 3 },
266 |         { question_id: "q4", difficulty_rating: 3.0, bloom_level: 3 },
267 |     ];
268 |     const answers = new Map([
269 |         ["q1", false],
270 |         ["q2", false], // clo-1: 0%
271 |         ["q3", true],
272 |         ["q4", true], // clo-2: 100%
273 |     ]);
274 |     const qbMap = new Map([
275 |         ["q1", { id: "q1", difficulty_rating: 3.0, clo_id: "clo-1" }],
276 |         ["q2", { id: "q2", difficulty_rating: 3.0, clo_id: "clo-1" }],
277 |         ["q3", { id: "q3", difficulty_rating: 3.0, clo_id: "clo-2" }],
278 |         ["q4", { id: "q4", difficulty_rating: 3.0, clo_id: "clo-2" }],
279 |     ]);
280 |
281 |     const scores = computePerCLOScores(seq, answers, qbMap);
282 |     const failingCLOs = Object.entries(scores)
283 |         .filter(([, score]) => score < 70)
284 |         .map(([cloId]) => cloId);
285 |
286 |     expect(failingCLOs).toEqual(["clo-1"]);
287 |     expect(failingCLOs).not.toContain("clo-2");
288 | });
289 | });
290 |

```

FILE: src/__tests__/unit/masteryRecovery.test.ts

TYPE: .ts | LINES: 1-156 | SHA256: 220bc9b07f9ed97fa4ccb5a0ec975a82a5ab1d6b5a50b5247f630e05727e17ee

```

1 // @vitest-environment happy-dom
2 import { describe, it, expect } from "vitest";
3 import {
4   countMasteryFailures,
5   shouldActivateRecovery,
6   recoveryBloomLevel,
7   isRecoveryComplete,
8 } from "@lib/masteryRecovery";
9
10 // %% countMasteryFailures %%%%%%%%%%%
11
12 describe("countMasteryFailures", () => {
13   it("returns 0 for empty attempts array", () => {
14     expect(countMasteryFailures([], "clo-1")).toBe(0);
15   });
16
17   it("returns 0 when all attempts are above threshold", () => {
18     const attempts = [
19       { clo_scores: { "clo-1": 80 } },
20       { clo_scores: { "clo-1": 90 } },
21     ];
22     expect(countMasteryFailures(attempts, "clo-1")).toBe(0);
23   });
24
25   it("counts attempts below default 70% threshold", () => {
26     const attempts = [
27       { clo_scores: { "clo-1": 60 } },
28       { clo_scores: { "clo-1": 80 } },
29       { clo_scores: { "clo-1": 50 } },
30     ];
31     expect(countMasteryFailures(attempts, "clo-1")).toBe(2);
32   });
33
34   it("treats missing CLO score as 0 (below threshold)", () => {
35     const attempts = [{ clo_scores: { "clo-2": 90 } }];
36     expect(countMasteryFailures(attempts, "clo-1")).toBe(1);
37   });
38
39   it("uses custom mastery threshold", () => {
40     const attempts = [
41       { clo_scores: { "clo-1": 75 } },
42       { clo_scores: { "clo-1": 85 } },
43     ];
44     expect(countMasteryFailures(attempts, "clo-1", 80)).toBe(1);
45   });
46
47   it("score exactly at threshold is not a failure", () => {
48     const attempts = [{ clo_scores: { "clo-1": 70 } }];
49     expect(countMasteryFailures(attempts, "clo-1")).toBe(0);
50   });
51
52   it("score just below threshold is a failure", () => {
53     const attempts = [{ clo_scores: { "clo-1": 69 } }];
54     expect(countMasteryFailures(attempts, "clo-1")).toBe(1);
55   });
56
57   it("only counts failures for the specified CLO", () => {
58     const attempts = [
59       { clo_scores: { "clo-1": 50, "clo-2": 90 } },
60       { clo_scores: { "clo-1": 80, "clo-2": 40 } },
61     ];

```


[illegible]

[illegible]

```

214 |     }},
215 |     isLoading: false,
216 |   });
217 |   renderPanel();
218 |
219 |   const retryButton = screen.getByRole("button", {
220 |     name: /complete.*steps.*unlock/i,
221 |   });
222 |   expect(retryButton).toBeDisabled();
223 | });
224 |
225 | it("disables retry button when only Practice is completed", () => {
226 |   mockUseRecoveryPathway.mockReturnValue({
227 |     data: makePathway({
228 |       ai_tutor_completed: false,
229 |       practice_completed: true,
230 |     }),
231 |     isLoading: false,
232 |   });
233 |   renderPanel();
234 |
235 |   const retryButton = screen.getByRole("button", {
236 |     name: /complete.*steps.*unlock/i,
237 |   });
238 |   expect(retryButton).toBeDisabled();
239 | });
240 |
241 | it("enables retry button when both required steps are completed", () => {
242 |   mockUseRecoveryPathway.mockReturnValue({
243 |     data: makePathway({ ai_tutor_completed: true, practice_completed: true }),
244 |     isLoading: false,
245 |   });
246 |   renderPanel();
247 |
248 |   const retryButton = screen.getByRole("button", { name: /retry quiz/i });
249 |   expect(retryButton).not.toBeDisabled();
250 | });
251 |
252 | it('shows "In Progress" badge when steps are incomplete', () => {
253 |   mockUseRecoveryPathway.mockReturnValue({
254 |     data: makePathway({
255 |       ai_tutor_completed: false,
256 |       practice_completed: false,
257 |     }),
258 |     isLoading: false,
259 |   });
260 |   renderPanel();
261 |
262 |   expect(screen.getByText("In Progress")).toBeInTheDocument();
263 | });
264 |
265 | it('shows "Ready to Retry" badge when all required steps are complete', () => {
266 |   mockUseRecoveryPathway.mockReturnValue({
267 |     data: makePathway({ ai_tutor_completed: true, practice_completed: true }),
268 |     isLoading: false,
269 |   });
270 |   renderPanel();
271 |
272 |   expect(screen.getByText("Ready to Retry")).toBeInTheDocument();
273 | });
274 | });
275 |

```

FILE: src/__tests__/unit/maybeSingleZeroRow.test.ts

TYPE: .ts | LINES: 1-134 | SHA256: b917cf19d208d8db8f13286f970dccecca49f68b042f5bc93df170097ccac417

```

1 | // @vitest-environment happy-dom
2 | // =====
3 | // maybeSingleZeroRow.test.ts
4 | // Feature: production-bug-fixes, Req 4 – flagged zero-row reads must use
5 | // `maybeSingle()` and handle the no-row case as a clean not-found/empty state
6 | // instead of throwing a raw PostgREST `PGRST116`.
7 | // =====
8 | // Covered sites:
9 | //   - useReflectionDigest: useShareDigest / useRevokeDigestShare (fetch shared_with)
10 | //   - useTeamProfile (fetch team)
11 | // (AcceptInvitePage's institution lookup already consumes the result null-safely
12 | // via `institution?.name`; switching to maybeSingle just stops it throwing.)
13 | //
14 | // Contract: zero rows != no PGRST116; a clean domain error surfaces. One row !=
15 | // behaviour unchanged (the write/return proceeds).
16 | // =====
17 | import { describe, it, expect, vi, beforeEach } from "vitest";
18 | import { renderHook, waitFor } from "@testing-library/react";
19 | import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
20 | import { createElement, type ReactNode } from "react";
21 |
22 | // Configurable per-test results for the mocked supabase client.
23 | let singleResult: { data: unknown; error: unknown } = {
24 |   data: null,
25 |   error: null,
26 | };
27 | let tailResult: { data: unknown; error: unknown } = { data: null, error: null };
28 | const updateSpy = vi.fn();
29 |
30 | const makeBuilder = (): Record<string, unknown> => {
31 |   const builder: Record<string, unknown> = {
32 |     select: () => builder,
33 |     update: (...args: unknown[]) => {
34 |       updateSpy(...args);
35 |       return builder;
36 |     },
37 |     insert: () => builder,
38 |     eq: () => builder,

```

```

39 |     is: () => builder,
40 |     order: () => builder,
41 |     maybeSingle: () => Promise.resolve(singleResult),
42 |     single: () => Promise.resolve(singleResult),
43 |     // Chains that are awaited without maybeSingle() (e.g. update().eq().eq()).
44 |     then: (
45 |       resolve: (v: unknown) => unknown,
46 |       reject?: (r: unknown) => unknown
47 |     ) => Promise.resolve(tailResult).then(resolve, reject),
48 |   };
49 |   return builder;
50 | };
51 |
52 | vi.mock("@/lib/supabase", () => ({
53 |   supabase: { from: () => makeBuilder() },
54 | }));
55 |
56 | vi.mock("@/hooks/useAuth", () => ({
57 |   useAuth: () => ({ user: { id: "student-1" } }),
58 | }));
59 |
60 | vi.mock("sonner", () => ({
61 |   toast: { success: vi.fn(), error: vi.fn() },
62 | }));
63 |
64 | import {
65 |   useShareDigest,
66 |   useRevokeDigestShare,
67 | } from "@/hooks/useReflectionDigest";
68 | import { useTeamProfile } from "@/hooks/useTeamProfile";
69 |
70 | const makeWrapper = () => {
71 |   const client = new QueryClient({
72 |     defaultOptions: { queries: { retry: false }, mutations: { retry: false } },
73 |   });
74 |   return ({ children }: { children: ReactNode }) =>
75 |     createElement(QueryClientProvider, { client }, children);
76 | };
77 |
78 | describe("Req 4 – zero-row reads use maybeSingle + clean not-found", () => {
79 |   beforeEach(() => {
80 |     vi.clearAllMocks();
81 |     singleResult = { data: null, error: null };
82 |     tailResult = { data: null, error: null };
83 |   });
84 |
85 |   it("useShareDigest: zero rows !' clean 'not found' error, no update issued", async () => {
86 |     singleResult = { data: null, error: null }; // digest not found
87 |     const { result } = renderHook(() => useShareDigest(), {
88 |       wrapper: makeWrapper(),
89 |     });
90 |
91 |     await expect(
92 |       result.current.mutateAsync({ digestId: "missing", role: "parent" })
93 |     ).rejects.toThrow(/not found/i);
94 |     expect(updateSpy).not.toHaveBeenCalled();
95 |   });
96 |
97 |   it("useShareDigest: one row !' proceeds to update (behaviour unchanged)", async () => {
98 |     singleResult = { data: { shared_with: [] }, error: null };
99 |     tailResult = { data: null, error: null }; // update succeeds
100 |     const { result } = renderHook(() => useShareDigest(), {
101 |       wrapper: makeWrapper(),
102 |     });
103 |
104 |     await result.current.mutateAsync({ digestId: "d1", role: "parent" });
105 |     expect(updateSpy).toHaveBeenCalledTimes(1);
106 |   });
107 |
108 |   it("useRevokeDigestShare: zero rows !' clean 'not found' error, no update", async () => {
109 |     singleResult = { data: null, error: null };
110 |     const { result } = renderHook(() => useRevokeDigestShare(), {
111 |       wrapper: makeWrapper(),
112 |     });
113 |
114 |     await expect(
115 |       result.current.mutateAsync({ digestId: "missing", role: "parent" })
116 |     ).rejects.toThrow(/not found/i);
117 |     expect(updateSpy).not.toHaveBeenCalled();
118 |   });
119 |
120 |   it("useTeamProfile: missing team !' query errors with clean 'Team not found'", async () => {
121 |     singleResult = { data: null, error: null }; // team not found
122 |     const { result } = renderHook(() => useTeamProfile("missing-team"), {
123 |       wrapper: makeWrapper(),
124 |     });
125 |
126 |     await waitFor(() => expect(result.current.isError).toBe(true));
127 |     expect(result.current.error as Error).toEqual(
128 |       expect.objectContaining({
129 |         message: expect.stringMatching(/Team not found/i),
130 |       })
131 |     );
132 |   });
133 | });
134 |

```

FILE: src/__tests__/unit/microAssessmentCard.test.tsx

[illegible]

```

102     expect(onComplete).toHaveBeenCalledOnce();
103   });
104
105   it('calls onDismiss when "Remind Me Later" is clicked', async () => {
106     const onDismiss = vi.fn();
107     renderCard({ onDismiss });
108     await userEvent.click(screen.getByText("Remind Me Later"));
109     expect(onDismiss).toHaveBeenCalledTimes(1);
110   });
111
112   it("calls onDismiss when X button is clicked", async () => {
113     const onDismiss = vi.fn();
114     renderCard({ onDismiss });
115     await userEvent.click(
116       screen.getByRole("button", { name: /dismiss micro-assessment/i })
117     );
118     expect(onDismiss).toHaveBeenCalledTimes(1);
119   });
120 });
121
122 // %% Dismissal counter %%%%%%%%%%%%%%%
123
124 describe("dismissal counter", () => {
125   it("shows warning when one dismissal remains", () => {
126     renderCard({ dismissalCount: MAX_MICRO_DISMISSALS - 1 });
127     expect(
128       screen.getByText(
129         /last chance – this will be skipped if dismissed again/i
130       )
131     ).toBeTruthy();
132   });
133
134   it("does not show warning when dismissals are well below limit", () => {
135     renderCard({ dismissalCount: 0 });
136     expect(screen.queryByText(/last chance/i)).toBeNull();
137   });
138
139   it("does not show warning when dismissal count equals limit", () => {
140     renderCard({ dismissalCount: MAX_MICRO_DISMISSALS });
141     // remainingDismissals = 0, condition is > 0 && <= 1
142     expect(screen.queryByText(/last chance/i)).toBeNull();
143   });
144 });
145
146 // %% Bilingual (en + ar) %%%%%%%%%%%%%%%
147
148 describe("localization", () => {
149   it("renders the prompt in Arabic", async () => {
150     await i18n.changeLanguage("ar");
151     renderCard({ assessmentType: "personality", questionCount: 3 });
152     expect(screen.getByText("d -cR 3c Jc' Dd 'dF4ba5d")).toBeTruthy();
153     expect(screen.getByText("b6CdVD bVDb&F")).toBeTruthy();
154     expect(screen.getByText("c Ce 1dfJ dF'b0BbvK")).toBeTruthy();
155   });
156 });
157 });
158
```

FILE: src/__tests__/unit/milestoneMarker.test.tsx

TYPE: .tsx | LINES: 1-259 | SHA256: 11c3567bb9bfc0f84d24442409c0aaebba25d591da11f38979f6c4254852cdb0

```

1 import { describe, it, expect, vi } from "vitest";
2 import { render, screen } from "@testing-library/react";
3 import HeatmapGrid from "@components/shared/HeatmapGrid";
4 import HeatmapTooltip from "@components/shared/HeatmapTooltip";
5 import type { HeatmapDay, DateRange, StreakMilestone } from "@types/habits";
6 import { detectStreakMilestones } from "@lib/streakMilestones";
7
8 // -----
9 // Helpers
10 // -----
11
12 function makeDay(
13   date: string,
14   totalCount: number,
15   academicCount?: number
16 ): HeatmapDay {
17   return {
18     date,
19     academicCount: academicCount ?? Math.min(totalCount, 4),
20     wellnessCount: Math.max(totalCount - 4, 0),
21     totalCount,
22     habits: [],
23   };
24 }
25
26 function buildDays(
27   start: string,
28   end: string,
29   countFn: (d: string) => number = () => 0
30 ): HeatmapDay[] {
31   const days: HeatmapDay[] = [];
32   const cursor = new Date(start + "T00:00:00");
33   const endDate = new Date(end + "T00:00:00");
34   while (cursor <= endDate) {
35     const dateStr =
36       cursor.getFullYear() +
37       "-" +
38       String(cursor.getMonth() + 1).padStart(2, "0") +
39       "-" +
40       String(cursor.getDate()).padStart(2, "0");
41     days.push(makeDay(dateStr, countFn(dateStr)));
42     cursor.setDate(cursor.getDate() + 1);
43   }
44 }

```

```

44 |     return days;
45 | }
46 |
47 | // Stub ResizeObserver
48 | class MockResizeObserver {
49 |     callback: ResizeObserverCallback;
50 |     constructor(cb: ResizeObserverCallback) {
51 |         this.callback = cb;
52 |     }
53 |     observe() {
54 |         this.callback(
55 |             [{ contentRect: { width: 900 } } as unknown as ResizeObserverEntry],
56 |             this as unknown as ResizeObserver
57 |         );
58 |     }
59 |     unobserve() {}
60 |     disconnect() {}
61 | }
62 | vi.stubGlobal("ResizeObserver", MockResizeObserver);
63 |
64 | // -----
65 | // detectStreakMilestones unit tests
66 | // -----
67 |
68 | describe("detectStreakMilestones", () => {
69 |     it("returns empty array for no data", () => {
70 |         expect(detectStreakMilestones([])).toEqual([]);
71 |     });
72 |
73 |     it("returns empty array for streak shorter than 30 days", () => {
74 |         const days = buildDays("2025-01-01", "2025-01-29", () => 1);
75 |         expect(detectStreakMilestones(days)).toEqual([]);
76 |     });
77 |
78 |     it("detects 30-day milestone", () => {
79 |         const days = buildDays("2025-01-01", "2025-01-31", () => 1);
80 |         const milestones = detectStreakMilestones(days);
81 |         expect(milestones).toHaveLength(1);
82 |         expect(milestones[0]).toEqual({ days: 30, achievedDate: "2025-01-30" });
83 |     });
84 |
85 |     it("detects 30 and 60-day milestones", () => {
86 |         const days = buildDays("2025-01-01", "2025-03-15", () => 1);
87 |         const milestones = detectStreakMilestones(days);
88 |         expect(milestones.length).toBeGreaterThanOrEqual(2);
89 |         expect(milestones[0]!.days).toBe(30);
90 |         expect(milestones[1]!.days).toBe(60);
91 |     });
92 |
93 |     it("detects all three milestones (30, 60, 100)", () => {
94 |         const days = buildDays("2025-01-01", "2025-04-15", () => 1);
95 |         const milestones = detectStreakMilestones(days);
96 |         expect(milestones.length).toBeGreaterThanOrEqual(3);
97 |         expect(milestones.map((m) => m.days)).toContain(30);
98 |         expect(milestones.map((m) => m.days)).toContain(60);
99 |         expect(milestones.map((m) => m.days)).toContain(100);
100 |     });
101 |
102 |     it("resets streak on zero-count day", () => {
103 |         // 29 active days, then 1 zero, then 29 more – no milestone
104 |         const days: HeatmapDay[] = [];
105 |         for (let i = 1; i <= 29; i++) {
106 |             days.push(makeDay(`2025-01-${String(i).padStart(2, "0")}`, 1));
107 |         }
108 |         days.push(makeDay("2025-01-30", 0)); // break
109 |         for (let i = 1; i <= 28; i++) {
110 |             days.push(makeDay(`2025-02-${String(i).padStart(2, "0")}`, 1));
111 |         }
112 |         expect(detectStreakMilestones(days)).toEqual([]);
113 |     });
114 |
115 |     it("milestone achievedDate corresponds to the Nth consecutive active day", () => {
116 |         const days = buildDays("2025-01-01", "2025-02-05", () => 1);
117 |         const milestones = detectStreakMilestones(days);
118 |         // 30th consecutive day starting from Jan 1 is Jan 30
119 |         expect(milestones[0]!.achievedDate).toBe("2025-01-30");
120 |     });
121 | });
122 |
123 | // -----
124 | // Milestone Marker on HeatmapGrid
125 | // -----
126 |
127 | describe("Milestone Marker on HeatmapGrid", () => {
128 |     const range: DateRange = { start: "2025-01-06", end: "2025-01-19" };
129 |
130 |     it("renders star marker at milestone cell", () => {
131 |         const milestones: StreakMilestone[] = [
132 |             { days: 30, achievedDate: "2025-01-10" },
133 |         ];
134 |         const days = buildDays("2025-01-06", "2025-01-19", () => 1);
135 |
136 |         render(
137 |             <HeatmapGrid data={days} semesterRange={range} milestones={milestones} />
138 |         );
139 |
140 |         expect(
141 |             screen.getByTestId("milestone-marker-2025-01-10")
142 |         ).toBeInTheDocument();
143 |     });
144 |
145 |     it("does not render marker on non-milestone cells", () => {
146 |         const milestones: StreakMilestone[] = [
147 |             { days: 30, achievedDate: "2025-01-10" },

```



```

148 | ];
149 | const days = buildDays("2025-01-06", "2025-01-19", () => 1);
150 |
151 | render(
152 |   <HeatmapGrid data={days} semesterRange={range} milestones={milestones} />
153 | );
154 |
155 | expect(
156 |   screen.queryByTestId("milestone-marker-2025-01-06")
157 | ).not.toBeInTheDocument();
158 | expect(
159 |   screen.queryByTestId("milestone-marker-2025-01-09")
160 | ).not.toBeInTheDocument();
161 | });
162 |
163 | it("does not render markers when milestones prop is not provided", () => {
164 |   const days = buildDays("2025-01-06", "2025-01-19");
165 |
166 |   render(<HeatmapGrid data={days} semesterRange={range} />);
167 |
168 |   expect(
169 |     screen.queryByTestId("milestone-marker-2025-01-10")
170 |   ).not.toBeInTheDocument();
171 | });
172 |
173 | it("milestone marker has correct ARIA label", () => {
174 |   const milestones: StreakMilestone[] = [
175 |     { days: 60, achievedDate: "2025-01-12" },
176 |   ];
177 |   const days = buildDays("2025-01-06", "2025-01-19", () => 1);
178 |
179 |   render(
180 |     <HeatmapGrid data={days} semesterRange={range} milestones={milestones} />
181 |   );
182 |
183 |   const marker = screen.getByTestId("milestone-marker-2025-01-12");
184 |   expect(marker).toHaveAttribute("aria-label", "60-Day Streak Milestone");
185 | });
186 |
187 | it("renders multiple milestone markers", () => {
188 |   const milestones: StreakMilestone[] = [
189 |     { days: 30, achievedDate: "2025-01-08" },
190 |     { days: 60, achievedDate: "2025-01-15" },
191 |   ];
192 |   const days = buildDays("2025-01-06", "2025-01-19", () => 1);
193 |
194 |   render(
195 |     <HeatmapGrid data={days} semesterRange={range} milestones={milestones} />
196 |   );
197 |
198 |   expect(
199 |     screen.getByTestId("milestone-marker-2025-01-08")
200 |   ).toBeInTheDocument();
201 |   expect(
202 |     screen.getByTestId("milestone-marker-2025-01-15")
203 |   ).toBeInTheDocument();
204 | });
205 | });
206 |
207 | // -----
208 | // HeatmapTooltip - Milestone label
209 | // -----
210 |
211 | describe("HeatmapTooltip - Milestone label", () => {
212 |   it("shows milestone label with emoji when milestone is provided", () => {
213 |     render(
214 |       <HeatmapTooltip
215 |         date="2025-01-30"
216 |         habits={[]}
217 |         xpEarned={0}
218 |         streakActive={true}
219 |         milestone={{ days: 30, achievedDate: "2025-01-30" }}
220 |       />
221 |     );
222 |
223 |     expect(screen.getByTestId("tooltip-milestone-label")).toHaveTextContent(
224 |       "30-Day Streak Milestone 🏆"
225 |     );
226 |   });
227 |
228 |   it("shows 100-day milestone label", () => {
229 |     render(
230 |       <HeatmapTooltip
231 |         date="2025-04-10"
232 |         habits={[]}
233 |         xpEarned={0}
234 |         streakActive={true}
235 |         milestone={{ days: 100, achievedDate: "2025-04-10" }}
236 |       />
237 |     );
238 |
239 |     expect(screen.getByTestId("tooltip-milestone-label")).toHaveTextContent(
240 |       "100-Day Streak Milestone 🏆"
241 |     );
242 |   });
243 |
244 |   it("does not show milestone label when milestone is not provided", () => {
245 |     render(
246 |       <HeatmapTooltip
247 |         date="2025-01-30"
248 |         habits={[]}
249 |         xpEarned={0}
250 |         streakActive={false}
251 |       />

```

```

252 |     );
253 |
254 |     expect(
255 |       screen.queryByTestId("tooltip-milestone-label")
256 |     ).not.toBeInTheDocument();
257 |   });
258 | });
259 |

```

FILE: [src/___tests___/unit/mostImprovedLeaderboard.test.ts](#)

TYPE: .ts | LINES: 1-124 | SHA256: 6965482447d256ff51256273d7c875ea70aca93ca01c44ac0f341b0eef682ada

```

1 | // =====
2 | // mostImprovedLeaderboard - Unit tests
3 | // =====
4 |
5 | import { describe, it, expect } from "vitest";
6 | import {
7 |   calculateImprovement,
8 |   rankMostImproved,
9 | } from "@lib/mostImprovedLeaderboard";
10 |
11 | describe("calculateImprovement", () => {
12 |   it("calculates positive improvement correctly", () => {
13 |     expect(calculateImprovement(300, 200)).toBe(50);
14 |   });
15 |
16 |   it("calculates negative improvement correctly", () => {
17 |     expect(calculateImprovement(100, 200)).toBe(-50);
18 |   });
19 |
20 |   it("returns null for zero previous XP", () => {
21 |     expect(calculateImprovement(100, 0)).toBeNull();
22 |   });
23 |
24 |   it("returns null for negative previous XP", () => {
25 |     expect(calculateImprovement(100, -10)).toBeNull();
26 |   });
27 |
28 |   it("returns 0 for equal current and previous", () => {
29 |     expect(calculateImprovement(200, 200)).toBe(0);
30 |   });
31 |
32 |   it("handles large improvement percentages", () => {
33 |     expect(calculateImprovement(1000, 10)).toBe(9900);
34 |   });
35 | });
36 |
37 | describe("rankMostImproved", () => {
38 |   it("returns top 20 sorted by improvement percentage", () => {
39 |     const entries = Array.from({ length: 25 }, (_, i) => ({
40 |       student_id: `s${i}`,
41 |       student_name: `Student ${i}`,
42 |       current_4_week_xp: 200 + i * 10,
43 |       previous_4_week_xp: 100,
44 |     }));
45 |
46 |     const result = rankMostImproved(entries);
47 |     expect(result).toHaveLength(20);
48 |     // First entry should have highest improvement
49 |     expect(result[0]!.improvement_percent).toBeGreaterThan(
50 |       result[1]!.improvement_percent
51 |     );
52 |   });
53 |
54 |   it("excludes students with zero previous XP", () => {
55 |     const entries = [
56 |       {
57 |         student_id: "s1",
58 |         student_name: "A",
59 |         current_4_week_xp: 500,
60 |         previous_4_week_xp: 0,
61 |       },
62 |       {
63 |         student_id: "s2",
64 |         student_name: "B",
65 |         current_4_week_xp: 300,
66 |         previous_4_week_xp: 200,
67 |       },
68 |     ];
69 |
70 |     const result = rankMostImproved(entries);
71 |     expect(result).toHaveLength(1);
72 |     expect(result[0]!.student_id).toBe("s2");
73 |   });
74 |
75 |   it("calculates xp_delta correctly", () => {
76 |     const entries = [
77 |       {
78 |         student_id: "s1",
79 |         student_name: "A",
80 |         current_4_week_xp: 500,
81 |         previous_4_week_xp: 200,
82 |       },
83 |     ];
84 |
85 |     const result = rankMostImproved(entries);
86 |     expect(result[0]!.xp_delta).toBe(300);
87 |   });
88 |
89 |   it("returns empty array when all students have zero previous XP", () => {
90 |     const entries = [
91 |       {
92 |         student_id: "s1",

```

```

93 |         student_name: "A",
94 |         current_4_week_xp: 500,
95 |         previous_4_week_xp: 0,
96 |     },
97 |     {
98 |         student_id: "s2",
99 |         student_name: "B",
100 |         current_4_week_xp: 300,
101 |         previous_4_week_xp: 0,
102 |     },
103 | ];
104 |
105 | const result = rankMostImproved(entries);
106 | expect(result).toHaveLength(0);
107 | });
108 |
109 | it("rounds improvement percentage to 2 decimal places", () => {
110 |     const entries = [
111 |         {
112 |             student_id: "s1",
113 |             student_name: "A",
114 |             current_4_week_xp: 333,
115 |             previous_4_week_xp: 200,
116 |         },
117 |     ];
118 |
119 |     const result = rankMostImproved(entries);
120 |     // (333 - 200) / 200 * 100 = 66.5
121 |     expect(result[0]!.improvement_percent).toBe(66.5);
122 | });
123 | });
124 |

```

FILE: src/__tests__/unit/mysteryRewardBox.test.tsx

TYPE: .tsx | LINES: 1-45 | SHA256: 140758cc4a4a4f6b6517da3a0fe65d5d1938914a49df2453cdf2cbc45aa04eaa

```

1 | // =====
2 | // Unit Test: Mystery Reward Box
3 | // Task 26.4 – Unboxing animation, reward display
4 | // =====
5 |
6 | import { describe, it, expect } from "vitest";
7 | import { render, screen } from "@testing-library/react";
8 | import MysteryRewardBox from "@components/shared/MysteryRewardBox";
9 | import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
10 |
11 | const queryClient = new QueryClient({
12 |     defaultOptions: { queries: { retry: false } },
13 | });
14 |
15 | const wrapper = ({ children }: { children: React.ReactNode }) => (
16 |     <QueryClientProvider client={queryClient}>{children}</QueryClientProvider>
17 | );
18 |
19 | describe("MysteryRewardBox", () => {
20 |     it("renders the mystery box in idle state", () => {
21 |         render(<MysteryRewardBox studentId="student-1" institutionId="inst-1" />, {
22 |             wrapper,
23 |         });
24 |
25 |         expect(screen.getByTestId("mystery-reward-box")).toBeDefined();
26 |         expect(screen.getByText("Mystery Reward!")).toBeDefined();
27 |     });
28 |
29 |     it("shows the open button", () => {
30 |         render(<MysteryRewardBox studentId="student-1" institutionId="inst-1" />, {
31 |             wrapper,
32 |         });
33 |
34 |         expect(screen.getByText("Open Box")).toBeDefined();
35 |     });
36 |
37 |     it("displays tap to reveal message", () => {
38 |         render(<MysteryRewardBox studentId="student-1" institutionId="inst-1" />, {
39 |             wrapper,
40 |         });
41 |
42 |         expect(screen.getByText("Tap to reveal your prize")).toBeDefined();
43 |     });
44 | });
45 |

```

FILE: src/__tests__/unit/navPresentation.test.ts

TYPE: .ts | LINES: 1-171 | SHA256: 114e158b53658e21fc642f1cdcbce87b26da310c45fb8d48ab2503c2dff3e1e5f

```

1 | import { describe, expect, it } from "vitest";
2 | import {
3 |     getMobileTabItems,
4 |     getMoreNavItems,
5 |     getPrimaryNavItems,
6 | } from "@lib/navPresentation";
7 |
8 | describe("navPresentation", () => {
9 |     it("keeps the teacher primary nav and mobile tabs aligned with the prototype shell", () => {
10 |         const primary = getPrimaryNavItems("teacher").map((item) => item.to);
11 |         const mobile = getMobileTabItems("teacher").map((item) => item.labelKey);
12 |
13 |         expect(primary).toEqual([
14 |             "/teacher/dashboard",
15 |             "/teacher/students",
16 |             "/teacher/modules",
17 |             "/teacher/grading",
18 |             "/teacher/settings/profile",
19 |         ]);

```

```

20 |     expect(mobile).toEqual([
21 |       "nav.mobile.home",
22 |       "nav.mobile.students",
23 |       "nav.mobile.studio",
24 |       "nav.mobile.grade",
25 |       "nav.me",
26 |     ]);
27 |   });
28 |
29 |   it("keeps the student sidebar and mobile tabs aligned with the prototype shell", () => {
30 |     expect(getPrimaryNavItems("student").map((item) => item.to)).toEqual([
31 |       "/student/dashboard",
32 |       "/student/learning-path",
33 |       "/student/tutor",
34 |       "/student/progress",
35 |       "/student/profile",
36 |     ]);
37 |
38 |     expect(getMoreNavItems("student").map((item) => item.to)).toEqual([
39 |       "/student/courses",
40 |       "/student/today",
41 |       "/student/habits",
42 |       "/student/planner",
43 |       "/student/challenges",
44 |       "/student/leaderboard",
45 |       "/student/team",
46 |       "/student/journal",
47 |       "/student/calendar",
48 |       "/student/marketplace",
49 |       "/student/notifications",
50 |     ]);
51 |
52 |     expect(getMobileTabItems("student").map((item) => item.to)).toEqual([
53 |       "/student/dashboard",
54 |       "/student/learning-path",
55 |       "/student/tutor",
56 |       "/student/progress",
57 |       "/student/profile",
58 |     ]);
59 |   });
60 |
61 |   it("includes the parent support tab and route", () => {
62 |     const mobile = getMobileTabItems("parent");
63 |     const primary = getPrimaryNavItems("parent");
64 |
65 |     expect(mobile).toHaveLength(4);
66 |     expect(mobile.map((item) => item.to)).toEqual([
67 |       "/parent/dashboard",
68 |       "/parent/progress",
69 |       "/parent/support",
70 |       "/parent/profile",
71 |     ]);
72 |     expect(primary.map((item) => item.to)).toEqual([
73 |       "/parent/dashboard",
74 |       "/parent/progress",
75 |       "/parent/support",
76 |       "/parent/profile",
77 |     ]);
78 |   });
79 |
80 |   it("uses prototype labels for every staff primary sidebar", () => {
81 |     expect(
82 |       getPrimaryNavItems("coordinator").map((item) => item.labelKey)
83 |     ).toEqual([
84 |       "nav.mobile.home",
85 |       "nav.mobile.outcomes",
86 |       "nav.mobile.curriculum",
87 |       "nav.mobile.accredit",
88 |       "nav.me",
89 |     ]);
90 |     expect(getPrimaryNavItems("admin").map((item) => item.labelKey)).toEqual([
91 |       "nav.mobile.home",
92 |       "nav.mobile.analytics",
93 |       "nav.mobile.aiGov",
94 |       "nav.mobile.people",
95 |       "nav.me",
96 |     ]);
97 |   });
98 |
99 |   it("uses the prototype emoji artwork for every desktop primary role", () => {
100 |     expect(getPrimaryNavItems("student").map((item) => item.emoji)).toEqual([
101 |       "ð<ßà",
102 |       "ð=Ýúþ ",
103 |       "ð>Ý ",
104 |       "ð=ÜË",
105 |       "ð=þB",
106 |     ]);
107 |     expect(getPrimaryNavItems("teacher").map((item) => item.emoji)).toEqual([
108 |       "ð<ßà",
109 |       "ð>ÝÑ ð<ß",
110 |       "ð>Ýi",
111 |       "' p ",
112 |       "ð=þB",
113 |     ]);
114 |     expect(getPrimaryNavItems("coordinator").map((item) => item.emoji)).toEqual(
115 |       ["ð<ßà", "ð<ß ", "ð=ÝÀþ ", "ð=ÜË", "ð=þB"]
116 |     );
117 |   });
118 |
119 |   it("keeps staff MORE links in the approved prototype order", () => {
120 |     expect(getMoreNavItems("teacher").map((item) => item.labelKey)).toEqual([
121 |       "nav.curriculumStudio",
122 |       "nav.questionBank",
123 |       "nav.rubricBuilder",

```

```

124 |         "nav.courseMaterials",
125 |         "nav.tutorHandoffs",
126 |         "nav.gradingQueue",
127 |         "nav.gradebook",
128 |         "nav.attendance",
129 |         "nav.discussions",
130 |         "nav.announcements",
131 |         "nav.notifications",
132 |     ]);
133 |     expect(getMoreNavItems("parent").map((item) => item.labelKey)).toEqual([
134 |         "nav.attendance",
135 |         "nav.gradesReports",
136 |         "nav.feesPayments",
137 |         "nav.announcements",
138 |         "nav.settings",
139 |     ]);
140 |     expect(getMoreNavItems("coordinator").map((item) => item.labelKey)).toEqual(
141 |         [
142 |             "nav.outcomeAttainment",
143 |             "nav.curriculumMatrix",
144 |             "nav.cqiPlans",
145 |             "nav.courseFileGenerator",
146 |             "nav.teamHealthReport",
147 |             "nav.competencyFrameworks",
148 |             "nav.accreditation",
149 |             "nav.discussions",
150 |             "nav.announcements",
151 |             "nav.notifications",
152 |         ]
153 |     );
154 |     expect(
155 |         getMoreNavItems("coordinator").find(
156 |             (item) => item.labelKey === "nav.accreditation"
157 |         )?.to
158 |     ).toBe("/coordinator/accreditation");
159 |     expect(getMoreNavItems("admin").map((item) => item.labelKey)).toEqual([
160 |         "nav.marketplace",
161 |         "nav.institutionStructure",
162 |         "nav.bulkImport",
163 |         "nav.badgeDefinitions",
164 |         "nav.security",
165 |         "nav.feesManagement",
166 |         "nav.announcements",
167 |         "nav.notifications",
168 |     ]);
169 | });
170 | });
171 |

```

FILE: src/__tests__/unit/notificationBatcher.test.ts

TYPE: .ts | LINES: 1-311 | SHA256: 93a4a9e17c22c9e965d0c37d110440e2855f22acdf4303bcc4d0cfde5977daf6

[illegible]

[illegible]

[illegible]

[illegible]

FILE: src/___tests___/unit/notificationBell.test.tsx

```
TYPE: .tsx | LINES: 1-77 | SHA256: ac9a4c655bc190b4d6e0eaf4ab54174d335676e7a7bbd77050cb88b01367ab8b
```

[illegible]


```

50 |     expect(screen.getByText("5")).toBeInTheDocument();
51 |   });
52 |
53 |   it("does not show badge when count is 0", () => {
54 |     mockUnreadCount.mockReturnValue({ data: 0 });
55 |     render(<NotificationBell />);
56 |
57 |     expect(screen.queryByText("0")).not.toBeInTheDocument();
58 |   });
59 |
60 |   it("caps display at 99+", () => {
61 |     mockUnreadCount.mockReturnValue({ data: 150 });
62 |     render(<NotificationBell />);
63 |
64 |     expect(screen.getByText("99+")).toBeInTheDocument();
65 |   });
66 |
67 |   it("includes unread count in aria-label", () => {
68 |     mockUnreadCount.mockReturnValue({ data: 3 });
69 |     render(<NotificationBell />);
70 |
71 |     const button = screen.getByRole("button");
72 |     // aria-label includes the unread count
73 |     expect(button.getAttribute("aria-label")).toMatch(/3 unread/);
74 |   });
75 | });
76 |
77 |

```

FILE: src/__tests__/unit/notificationPresentation.test.ts

TYPE: .ts | LINES: 1-16 | SHA256: c677de58c98d37d5bd5cadd7b7e70c3a30480cfde096f81e74faf7e3b8b67747

```

1 | import { describe, expect, it } from "vitest";
2 |
3 | import { formatNotificationTitle } from "@lib/notificationPresentation";
4 |
5 | describe("formatNotificationTitle", () => {
6 |   it("humanizes badge identifiers without changing other notification titles", () => {
7 |     expect(formatNotificationTitle("Badge earned: rising_star")).toBe(
8 |       "Badge earned: Rising Star"
9 |     );
10 |     expect(formatNotificationTitle("Badge earned: night-owl")).toBe(
11 |       "Badge earned: Night Owl"
12 |     );
13 |     expect(formatNotificationTitle("Grade Released")).toBe("Grade Released");
14 |   });
15 | });
16 |

```

FILE: src/__tests__/unit/NotificationsFeedPage.test.tsx

TYPE: .tsx | LINES: 1-114 | SHA256: f0a3652321468da66457319c7a213299f7ae97a0a5190b7092d9d6c9f373d738

```

1 | // NotificationsFeedPage - functional render tests (net-new screen).
2 | import { describe, it, expect, beforeEach, vi } from "vitest";
3 | import { render, screen, fireEvent } from "@testing-library/react";
4 | import type { Notification } from "@hooks/useNotifications";
5 | import NotificationsFeedPage from "@features/shared/notifications/NotificationsFeedPage";
6 |
7 | const hoisted = vi.hoisted(() => ({
8 |   list: {
9 |     data: undefined as Notification[] | undefined,
10 |     isLoading: false,
11 |     isError: false,
12 |   },
13 |   markAsRead: { mutate: vi.fn() },
14 |   markAllAsRead: { mutate: vi.fn(), isPending: false },
15 |   deleteNotification: { mutate: vi.fn() },
16 | }));
17 |
18 | vi.mock("@hooks/useNotifications", () => ({
19 |   useNotifications: () => hoisted.list,
20 |   useMarkAsRead: () => hoisted.markAsRead,
21 |   useMarkAllAsRead: () => hoisted.markAllAsRead,
22 |   useDeleteNotification: () => hoisted.deleteNotification,
23 | }));
24 |
25 | vi.mock("@hooks/useAuth", () => ({
26 |   useAuth: () => ({ user: { id: "u1" }, profile: null }),
27 | }));
28 |
29 | vi.mock("react-i18next", () => ({
30 |   useTranslation: () => ({ t: (_key: string, def?: string) => def ?? _key }),
31 | }));
32 |
33 | const notifications: Notification[] = [
34 |   {
35 |     id: "n1",
36 |     user_id: "u1",
37 |     type: "grade_released",
38 |     title: "Grade released",
39 |     body: "CS301 - Database Design",
40 |     is_read: false,
41 |     metadata: null,
42 |     created_at: new Date().toISOString(),
43 |   },
44 |   {
45 |     id: "n2",
46 |     user_id: "u1",
47 |     type: "badge_earned",
48 |     title: "Badge earned",
49 |     body: null,
50 |     is_read: true,
51 |     metadata: null,
52 |     created_at: new Date(Date.now() - 86_400_000).toISOString(),

```

```

53 | },
54 | ];
55 |
56 | beforeEach(() => {
57 |   hoisted.list.data = undefined;
58 |   hoisted.list.isLoading = false;
59 |   hoisted.list.isError = false;
60 |   hoisted.markAsRead.mutate = vi.fn();
61 |   hoisted.markAllAsRead.mutate = vi.fn();
62 |   hoisted.deleteNotification.mutate = vi.fn();
63 | });
64 |
65 | describe("NotificationsFeedPage", () => {
66 |   it("renders the title while loading", () => {
67 |     hoisted.list.isLoading = true;
68 |     render(<NotificationsFeedPage />);
69 |     expect(screen.getByText("Notifications")).toBeInTheDocument();
70 |   });
71 |
72 |   it("shows an error state", () => {
73 |     hoisted.list.isError = true;
74 |     render(<NotificationsFeedPage />);
75 |     expect(screen.getByRole("alert")).toBeInTheDocument();
76 |   });
77 |
78 |   it("shows an empty state (and no mark-all) when there are none", () => {
79 |     hoisted.list.data = [];
80 |     render(<NotificationsFeedPage />);
81 |     expect(screen.getByText("No notifications yet.")).toBeInTheDocument();
82 |     expect(screen.queryByTestId("notif-mark-all")).toBeNull();
83 |   });
84 |
85 |   it("renders notifications with a mark-all action when unread exist", () => {
86 |     hoisted.list.data = notifications;
87 |     render(<NotificationsFeedPage />);
88 |     expect(screen.getByText("Grade released")).toBeInTheDocument();
89 |     expect(screen.getByText("Badge earned")).toBeInTheDocument();
90 |     expect(screen.getByTestId("notif-mark-all")).toBeInTheDocument();
91 |   });
92 |
93 |   it("marks all as read for the current user", () => {
94 |     hoisted.list.data = notifications;
95 |     render(<NotificationsFeedPage />);
96 |     fireEvent.click(screen.getByTestId("notif-mark-all"));
97 |     expect(hoisted.markAllAsRead.mutate).toHaveBeenCalledWith("u1");
98 |   });
99 |
100 |   it("marks a single unread notification read on click", () => {
101 |     hoisted.list.data = notifications;
102 |     render(<NotificationsFeedPage />);
103 |     fireEvent.click(screen.getByTestId("notif-n1"));
104 |     expect(hoisted.markAsRead.mutate).toHaveBeenCalledWith("n1");
105 |   });
106 |
107 |   it("deletes a notification", () => {
108 |     hoisted.list.data = notifications;
109 |     render(<NotificationsFeedPage />);
110 |     fireEvent.click(screen.getByTestId("notif-delete-n2"));
111 |     expect(hoisted.deleteNotification.mutate).toHaveBeenCalledWith("n2");
112 |   });
113 | });
114 |

```

FILE: src/__tests__/unit/offlineQueue.test.ts

TYPE: .ts | LINES: 1-109 | SHA256: 190037cdb0f6dabfd89ae55deb7da4abd44045c6658b3b833c65fe5757eaa2e2

```

1 | // @vitest-environment happy-dom
2 | // =====
3 | // OfflineQueue - Unit tests
4 | // Feature: edviser-platform, Task 54.2
5 | // =====
6 |
7 | import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
8 | import { offlineQueue } from "@lib/offlineQueue";
9 |
10 | describe("offlineQueue", () => {
11 |   beforeEach(() => {
12 |     localStorage.clear();
13 |   });
14 |
15 |   afterEach(() => {
16 |     vi.restoreAllMocks();
17 |   });
18 |
19 |   describe("enqueue", () => {
20 |     it("adds an event to the queue", () => {
21 |       offlineQueue.enqueue("test_handler", { foo: "bar" });
22 |       expect(offlineQueue.size()).toBe(1);
23 |     });
24 |
25 |     it("adds multiple events", () => {
26 |       offlineQueue.enqueue("h1", { a: 1 });
27 |       offlineQueue.enqueue("h2", { b: 2 });
28 |       expect(offlineQueue.size()).toBe(2);
29 |     });
30 |   });
31 |
32 |   describe("flush", () => {
33 |     it("flushes all events with registered handlers", async () => {
34 |       const handler = vi.fn().mockResolvedValue(undefined);
35 |       offlineQueue.registerHandler("my_handler", handler);
36 |
37 |       offlineQueue.enqueue("my_handler", { data: "test" });
38 |       offlineQueue.enqueue("my_handler", { data: "test2" });

```

```

39 |     const result = await offlineQueue.flush();
40 |     expect(result.flushed).toBe(2);
41 |     expect(result.failed).toBe(0);
42 |     expect(handler).toHaveBeenCalledTimes(2);
43 |     expect(offlineQueue.size()).toBe(0);
44 |   });
45 |
46 |   it("retries failed events up to 3 times", async () => {
47 |     const handler = vi.fn().mockImplementation(async () => {
48 |       throw new Error("network error");
49 |     });
50 |     offlineQueue.registerHandler("fail_handler", handler);
51 |
52 |     offlineQueue.enqueue("fail_handler", { data: "x" });
53 |
54 |     // First flush - retry 1
55 |     await offlineQueue.flush();
56 |     expect(offlineQueue.size()).toBe(1);
57 |
58 |     // Second flush - retry 2
59 |     await offlineQueue.flush();
60 |     expect(offlineQueue.size()).toBe(1);
61 |
62 |     // Third flush - retry 3, should be dead-lettered
63 |     const result = await offlineQueue.flush();
64 |     expect(result.failed).toBe(1);
65 |     expect(offlineQueue.size()).toBe(0);
66 |   });
67 |
68 |   it("keeps events with no registered handler", async () => {
69 |     offlineQueue.enqueue("unknown_handler", { data: "x" });
70 |     const result = await offlineQueue.flush();
71 |     expect(result.failed).toBe(1);
72 |     expect(offlineQueue.size()).toBe(1);
73 |   });
74 |
75 |   it("returns zero counts for empty queue", async () => {
76 |     const result = await offlineQueue.flush();
77 |     expect(result.flushed).toBe(0);
78 |     expect(result.failed).toBe(0);
79 |   });
80 | });
81 |
82 | describe("init", () => {
83 |   it("registers online event listener and returns cleanup", () => {
84 |     const addSpy = vi.spyOn(window, "addEventListener");
85 |     const removeSpy = vi.spyOn(window, "removeEventListener");
86 |
87 |     const cleanup = offlineQueue.init();
88 |     expect(addSpy).toHaveBeenCalledTimes(1);
89 |     expect(removeSpy).toHaveBeenCalledTimes(1);
90 |     cleanup();
91 |     expect(removeSpy).toHaveBeenCalledTimes(2);
92 |   });
93 | });
94 |
95 | describe("size", () => {
96 |   it("returns 0 for empty queue", () => {
97 |     expect(offlineQueue.size()).toBe(0);
98 |   });
99 |
100 |   it("returns correct count after enqueue", () => {
101 |     offlineQueue.enqueue("h", { a: 1 });
102 |     offlineQueue.enqueue("h", { b: 2 });
103 |     offlineQueue.enqueue("h", { c: 3 });
104 |     expect(offlineQueue.size()).toBe(3);
105 |   });
106 | });
107 | });
108 | });
109 |

```

FILE: [src/__tests__/unit/onboardingChecklist.test.ts](#)

TYPE: .ts | LINES: 1-52 | SHA256: 97e1057a1c88167976b35bdd23a4fd2f64b27c32de8ff6e11e26dd4a65f693f0

```

1 | import { describe, it, expect } from "vitest";
2 | import { getChecklistForRole } from "@lib/onboardingChecklist";
3 | import type { UserRole } from "@types/app";
4 |
5 | describe("getChecklistForRole", () => {
6 |   it("returns admin checklist items", () => {
7 |     const items = getChecklistForRole("admin");
8 |     expect(items).toHaveLength(4);
9 |     const first = items[0] as { id: string; route: string };
10 |    expect(first.id).toBe("create-ilo");
11 |    expect(first.route).toBe("/admin/outcomes");
12 |  });
13 |
14 |   it("returns coordinator checklist items", () => {
15 |     const items = getChecklistForRole("coordinator");
16 |     expect(items).toHaveLength(3);
17 |     const first = items[0] as { id: string };
18 |     expect(first.id).toBe("create-plo");
19 |   });
20 |
21 |   it("returns teacher checklist items", () => {
22 |     const items = getChecklistForRole("teacher");
23 |     expect(items).toHaveLength(4);
24 |     const first = items[0] as { id: string };
25 |     expect(first.id).toBe("create-clo");
26 |   });
27 |
28 |   it("returns student checklist items", () => {
29 |     const items = getChecklistForRole("student");

```

```

30 |     expect(items).toHaveLength(4);
31 |     const first = items[0] as { id: string };
32 |     expect(first.id).toBe("view-progress");
33 | });
34 |
35 | it("returns empty array for parent role", () => {
36 |     const items = getChecklistForRole("parent");
37 |     expect(items).toEqual([]);
38 | });
39 |
40 | it("all items have required fields", () => {
41 |     const roles: UserRole[] = ["admin", "coordinator", "teacher", "student"];
42 |     for (const role of roles) {
43 |         const items = getChecklistForRole(role);
44 |         for (const item of items) {
45 |             expect(item.id).toBeTruthy();
46 |             expect(item.label).toBeTruthy();
47 |             expect(item.route).toMatch(/^\w+$/);
48 |         }
49 |     }
50 | });
51 | });
52 |

```

FILE: src/__tests__/unit/onboardingSchemas.test.ts

TYPE: .ts | LINES: 1-904 | SHA256: 4787389074321eab56d7f9b14dec0458f846dc44ce3c13f6d239ff7acf54a343

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

```

598 |
599 | it("accepts all session_type values", () => {
600 |   for (const type of ["reading", "practice", "review", "exploration"]) {
601 |     const result = starterWeekSessionSchema.safeParse({
602 |       course_id: null,
603 |       session_type: type,
604 |       suggested_date: "2025-01-15",
605 |       suggested_time_slot: "afternoon",
606 |       duration_minutes: 35,
607 |       description: "A valid session description here",
608 |     });
609 |     expect(result.success).toBe(true);
610 |   }
611 | });
612 |
613 | it("accepts all time_slot values", () => {
614 |   for (const slot of ["morning", "afternoon", "evening"]) {
615 |     const result = starterWeekSessionSchema.safeParse({
616 |       course_id: null,
617 |       session_type: "reading",
618 |       suggested_date: "2025-01-15",
619 |       suggested_time_slot: slot,
620 |       duration_minutes: 30,
621 |       description: "A valid session description here",
622 |     });
623 |     expect(result.success).toBe(true);
624 |   }
625 | });
626 |
627 | it("accepts boundary duration values (25 and 50)", () => {
628 |   const min = starterWeekSessionSchema.safeParse({
629 |     course_id: null,
630 |     session_type: "reading",
631 |     suggested_date: "2025-01-15",
632 |     suggested_time_slot: "morning",
633 |     duration_minutes: 25,
634 |     description: "A valid session description here",
635 |   });
636 |   const max = starterWeekSessionSchema.safeParse({
637 |     course_id: null,
638 |     session_type: "reading",
639 |     suggested_date: "2025-01-15",
640 |     suggested_time_slot: "morning",
641 |     duration_minutes: 50,
642 |     description: "A valid session description here",
643 |   });
644 |   expect(min.success).toBe(true);
645 |   expect(max.success).toBe(true);
646 | });
647 |
648 | it("rejects duration_minutes below 25", () => {
649 |   const result = starterWeekSessionSchema.safeParse({
650 |     course_id: null,
651 |     session_type: "reading",
652 |     suggested_date: "2025-01-15",
653 |     suggested_time_slot: "morning",
654 |     duration_minutes: 24,
655 |     description: "A valid session description here",
656 |   });
657 |   expect(result.success).toBe(false);
658 | });
659 |
660 | it("rejects duration_minutes above 50", () => {
661 |   const result = starterWeekSessionSchema.safeParse({
662 |     course_id: null,
663 |     session_type: "reading",
664 |     suggested_date: "2025-01-15",
665 |     suggested_time_slot: "morning",
666 |     duration_minutes: 51,
667 |     description: "A valid session description here",
668 |   });
669 |   expect(result.success).toBe(false);
670 | });
671 |
672 | it("rejects invalid session_type", () => {
673 |   const result = starterWeekSessionSchema.safeParse({
674 |     course_id: null,
675 |     session_type: "gaming",
676 |     suggested_date: "2025-01-15",
677 |     suggested_time_slot: "morning",
678 |     duration_minutes: 30,
679 |     description: "A valid session description here",
680 |   });
681 |   expect(result.success).toBe(false);
682 | });
683 |
684 | it("rejects invalid date format", () => {
685 |   const result = starterWeekSessionSchema.safeParse({
686 |     course_id: null,
687 |     session_type: "reading",
688 |     suggested_date: "15-01-2025",
689 |     suggested_time_slot: "morning",
690 |     duration_minutes: 30,
691 |     description: "A valid session description here",
692 |   });
693 |   expect(result.success).toBe(false);
694 | });
695 |
696 | it("rejects description shorter than 10 chars", () => {
697 |   const result = starterWeekSessionSchema.safeParse({
698 |     course_id: null,
699 |     session_type: "reading",
700 |     suggested_date: "2025-01-15",
701 |     suggested_time_slot: "morning",

```

```

702     duration_minutes: 30,
703     description: "Short",
704   });
705   expect(result.success).toBe(false);
706 });
707
708 it("rejects invalid time_slot", () => {
709   const result = starterWeekSessionSchema.safeParse({
710     course_id: null,
711     session_type: "reading",
712     suggested_date: "2025-01-15",
713     suggested_time_slot: "midnight",
714     duration_minutes: 30,
715     description: "A valid session description here",
716   });
717   expect(result.success).toBe(false);
718 });
719 });
720
721 // %% goalSuggestionSchema %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %%
722
723 describe("goalSuggestionSchema", () => {
724   it("accepts valid payload with required fields only", () => {
725     const result = goalSuggestionSchema.safeParse({
726       goal_text: "Complete all reading assignments this week",
727       difficulty: "easy",
728     });
729     expect(result.success).toBe(true);
730   });
731
732   it("accepts valid payload with all optional SMART fields", () => {
733     const result = goalSuggestionSchema.safeParse({
734       goal_text: "Complete all reading assignments this week",
735       smart_specific: "Read chapters 1-3 of the textbook",
736       smart_measurable: "Complete reading notes for each chapter",
737       smart_achievable: "Allocate 1 hour per chapter",
738       smart_relevant: "Supports CLO 1.1 understanding",
739       smart_timebound: "2025-01-20",
740       difficulty: "moderate",
741     });
742     expect(result.success).toBe(true);
743   });
744
745   it("accepts all difficulty values", () => {
746     for (const diff of ["easy", "moderate", "ambitious"]) {
747       const result = goalSuggestionSchema.safeParse({
748         goal_text: "A valid goal text for testing",
749         difficulty: diff,
750       });
751       expect(result.success).toBe(true);
752     }
753   });
754
755   it("rejects goal_text shorter than 10 chars", () => {
756     const result = goalSuggestionSchema.safeParse({
757       goal_text: "Too short",
758       difficulty: "easy",
759     });
760     expect(result.success).toBe(false);
761   });
762
763   it("rejects invalid difficulty value", () => {
764     const result = goalSuggestionSchema.safeParse({
765       goal_text: "A valid goal text for testing",
766       difficulty: "impossible",
767     });
768     expect(result.success).toBe(false);
769   });
770
771   it("rejects invalid smart_timebound date format", () => {
772     const result = goalSuggestionSchema.safeParse({
773       goal_text: "A valid goal text for testing",
774       difficulty: "easy",
775       smart_timebound: "next-friday",
776     });
777     expect(result.success).toBe(false);
778   });
779
780   it("rejects missing difficulty", () => {
781     const result = goalSuggestionSchema.safeParse({
782       goal_text: "A valid goal text for testing",
783     });
784     expect(result.success).toBe(false);
785   });
786
787   it("rejects missing goal_text", () => {
788     const result = goalSuggestionSchema.safeParse({
789       difficulty: "easy",
790     });
791     expect(result.success).toBe(false);
792   });
793 });
794
795 // %% smartGoalTemplateSchema %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %%
796
797 describe("smartGoalTemplateSchema", () => {
798   it("accepts valid payload", () => {
799     const result = smartGoalTemplateSchema.safeParse({
800       specific: "Read chapters 1-3 of the textbook",
801       measurable: "Complete reading notes for each chapter",
802       achievable: "Allocate 1 hour per chapter over 3 days",
803       relevant: "Supports CLO 1.1",
804       timebound: "2025-01-20",
805     });

```

```

806 |     expect(result.success).toBe(true);
807 | });
808 |
809 | it("rejects specific shorter than 5 chars", () => {
810 |     const result = smartGoalTemplateSchema.safeParse({
811 |         specific: "Read",
812 |         measurable: "Complete reading notes for each chapter",
813 |         achievable: "Allocate 1 hour per chapter over 3 days",
814 |         relevant: "Supports CLO 1.1",
815 |         timebound: "2025-01-20",
816 |     });
817 |     expect(result.success).toBe(false);
818 | });
819 |
820 | it("rejects measurable shorter than 5 chars", () => {
821 |     const result = smartGoalTemplateSchema.safeParse({
822 |         specific: "Read chapters 1-3 of the textbook",
823 |         measurable: "Done",
824 |         achievable: "Allocate 1 hour per chapter over 3 days",
825 |         relevant: "Supports CLO 1.1",
826 |         timebound: "2025-01-20",
827 |     });
828 |     expect(result.success).toBe(false);
829 | });
830 |
831 | it("rejects achievable shorter than 5 chars", () => {
832 |     const result = smartGoalTemplateSchema.safeParse({
833 |         specific: "Read chapters 1-3 of the textbook",
834 |         measurable: "Complete reading notes for each chapter",
835 |         achievable: "Yes",
836 |         relevant: "Supports CLO 1.1",
837 |         timebound: "2025-01-20",
838 |     });
839 |     expect(result.success).toBe(false);
840 | });
841 |
842 | it("rejects relevant shorter than 1 char (empty string)", () => {
843 |     const result = smartGoalTemplateSchema.safeParse({
844 |         specific: "Read chapters 1-3 of the textbook",
845 |         measurable: "Complete reading notes for each chapter",
846 |         achievable: "Allocate 1 hour per chapter over 3 days",
847 |         relevant: "",
848 |         timebound: "2025-01-20",
849 |     });
850 |     expect(result.success).toBe(false);
851 | });
852 |
853 | it("rejects invalid timebound date format", () => {
854 |     const result = smartGoalTemplateSchema.safeParse({
855 |         specific: "Read chapters 1-3 of the textbook",
856 |         measurable: "Complete reading notes for each chapter",
857 |         achievable: "Allocate 1 hour per chapter over 3 days",
858 |         relevant: "Supports CLO 1.1",
859 |         timebound: "January 20, 2025",
860 |     });
861 |     expect(result.success).toBe(false);
862 | });
863 |
864 | it("rejects missing required fields", () => {
865 |     expect(smartGoalTemplateSchema.safeParse({}).success).toBe(false);
866 |     expect(
867 |         smartGoalTemplateSchema.safeParse({
868 |             specific: "Read chapters 1-3",
869 |         }).success
870 |     ).toBe(false);
871 |     expect(
872 |         smartGoalTemplateSchema.safeParse({
873 |             specific: "Read chapters 1-3",
874 |             measurable: "Complete notes",
875 |             achievable: "1 hour per chapter",
876 |             relevant: "CLO 1.1",
877 |             // missing timebound
878 |         }).success
879 |     ).toBe(false);
880 | });
881 |
882 | it("accepts boundary: relevant with exactly 1 char", () => {
883 |     const result = smartGoalTemplateSchema.safeParse({
884 |         specific: "Read chapters 1-3 of the textbook",
885 |         measurable: "Complete reading notes for each chapter",
886 |         achievable: "Allocate 1 hour per chapter over 3 days",
887 |         relevant: "X",
888 |         timebound: "2025-01-20",
889 |     });
890 |     expect(result.success).toBe(true);
891 | });
892 |
893 | it("accepts boundary: fields with exactly 5 chars", () => {
894 |     const result = smartGoalTemplateSchema.safeParse({
895 |         specific: "ABCDE",
896 |         measurable: "FGHIJ",
897 |         achievable: "KLMNO",
898 |         relevant: "P",
899 |         timebound: "2025-01-20",
900 |     });
901 |     expect(result.success).toBe(true);
902 | });
903 | });
904 |

```

```
TYPE: .tsx | LINES: 1-230 | SHA256: d5a1c31739bf412489826434fbefbd9dd121553fbe4d91f165c644ef486e77c1
```

[illegible]

[illegible]

```

206 |     );
207 |     expect(screen.getByText("Step 2 of 4")).toBeInTheDocument();
208 |
209 |     await user.click(screen.getByRole("button", { name: /next/i }));
210 |     await waitFor(() =>
211 |       expect(screen.getByTestId("self-efficacy-step")).toBeInTheDocument()
212 |     );
213 |     expect(screen.getByText("Step 3 of 4")).toBeInTheDocument();
214 |
215 |     await user.click(screen.getByRole("button", { name: /next/i }));
216 |     await waitFor(() =>
217 |       expect(screen.getByTestId("summary-step")).toBeInTheDocument()
218 |     );
219 |     expect(screen.getByText("Step 4 of 4")).toBeInTheDocument();
220 |
221 |     // None of the front-loaded battery steps ever rendered on the short path.
222 |     expect(screen.queryByTestId("learning-style-step")).not.toBeInTheDocument();
223 |     expect(screen.queryByTestId("study-strategy-step")).not.toBeInTheDocument();
224 |     expect(
225 |       screen.queryByTestId("baseline-select-step")
226 |     ).not.toBeInTheDocument();
227 |     expect(screen.queryByTestId("baseline-test-step")).not.toBeInTheDocument();
228 |   });
229 | });
230 |

```

FILE: src/__tests__/unit/onboardingWizard.test.tsx

```
TYPE: .tsx | LINES: 1-836 | SHA256: 0f2de1163d09754a5911a441ff756a3ffe23390a129953f5afca7d9d7e5b5b1b
```

[illegible]

```

76 | vi.mock("@/hooks/useStudentProfile", () => ({
77 |   useProcessOnboarding: () => ({
78 |     mutateAsync: mockMutateAsync,
79 |     isPending: false,
80 |   }),
81 | }));
82 |
83 | vi.mock("@/hooks/useOnboardingQuestions", () => ({
84 |   usePersonalityQuestions: () => ({ data: [], isLoading: false }),
85 |   useLearningStyleQuestions: () => ({ data: [], isLoading: false }),
86 |   useSelfEfficacyQuestions: () => ({ data: [], isLoading: false }),
87 |   useStudyStrategyQuestions: () => ({ data: [], isLoading: false }),
88 |   useBaselineQuestions: () => ({ data: [], isLoading: false }),
89 | }));
90 |
91 | vi.mock("@/hooks/useOnboardingResponses", () => ({
92 |   useSaveResponses: () => ({ mutate: vi.fn(), isPending: false }),
93 | }));
94 |
95 | // Mock child step components to keep tests focused on wizard navigation
96 | vi.mock("@/pages/student/onboarding/WelcomeStep", () => ({
97 |   WelcomeStep: ({ onComplete }: { onComplete: () => void }) => (
98 |     <div data-testid="welcome-step">
99 |       <span>Welcome Step</span>
100 |      <button data-testid="step-complete" onClick={onComplete}>
101 |        Next
102 |      </button>
103 |    </div>
104 |  ),
105 | }));
106 |
107 | vi.mock("@/pages/student/onboarding/PersonalityStep", () => ({
108 |   PersonalityStep: ({
109 |     onComplete,
110 |     onSkip,
111 |   }: {
112 |     onComplete: () => void;
113 |     onSkip?: () => void;
114 |   }) => (
115 |     <div data-testid="personality-step">
116 |       <span>Personality Step</span>
117 |       <button data-testid="step-complete" onClick={onComplete}>
118 |         Complete
119 |       </button>
120 |       {onSkip && (
121 |         <button data-testid="step-skip" onClick={onSkip}>
122 |           Skip
123 |         </button>
124 |       )}
125 |     </div>
126 |   ),
127 | }));
128 |
129 | vi.mock("@/pages/student/onboarding/SelfEfficacyStep", () => ({
130 |   SelfEfficacyStep: ({
131 |     onComplete,
132 |     onSkip,
133 |   }: {
134 |     onComplete: () => void;
135 |     onSkip?: () => void;
136 |   }) => (
137 |     <div data-testid="self-efficacy-step">
138 |       <span>Self Efficacy Step</span>
139 |       <button data-testid="step-complete" onClick={onComplete}>
140 |         Complete
141 |       </button>
142 |       {onSkip && (
143 |         <button data-testid="step-skip" onClick={onSkip}>
144 |           Skip
145 |         </button>
146 |       )}
147 |     </div>
148 |   ),
149 | }));
150 |
151 | vi.mock("@/pages/student/onboarding/LearningStyleStep", () => ({
152 |   LearningStyleStep: ({
153 |     onComplete,
154 |     onSkip,
155 |   }: {
156 |     onComplete: () => void;
157 |     onSkip?: () => void;
158 |   }) => (
159 |     <div data-testid="learning-style-step">
160 |       <span>Learning Style Step</span>
161 |       <button data-testid="step-complete" onClick={onComplete}>
162 |         Complete
163 |       </button>
164 |       {onSkip && (
165 |         <button data-testid="step-skip" onClick={onSkip}>
166 |           Skip
167 |         </button>
168 |       )}
169 |     </div>
170 |   ),
171 | }));
172 |
173 | vi.mock("@/pages/student/onboarding/StudyStrategyStep", () => ({
174 |   StudyStrategyStep: ({
175 |     onComplete,
176 |     onSkip,
177 |   }: {
178 |     onComplete: () => void;
179 |     onSkip?: () => void;

```


[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

```

700 // Click confirm
701 await user.click(screen.getByTestId("confirm-profile"));
702
703 await waitFor(() => {
704   expect(mockMutateAsync).toHaveBeenCalled();
705   expect.objectContaining({
706     student_id: "student-1",
707     assessment_version: 1,
708     is_day1: true,
709   })
710 });
711 });
712
713 await waitFor(() => {
714   expect(mockNavigate).toHaveBeenCalled();
715 });
716 });
717
718 it("sets day1_completed via updateProgress when Day 1 confirm succeeds", async () => {
719   const user = userEvent.setup();
720   renderWizard({ isDay1: true });
721
722   // Navigate to summary
723   for (let i = 0; i < 3; i++) {
724     await user.click(screen.getByRole("button", { name: /next/i }));
725   }
726   await waitFor(() =>
727     expect(screen.getByTestId("summary-step")).toBeInTheDocument()
728   );
729
730   await user.click(screen.getByTestId("confirm-profile"));
731
732   await waitFor(() => {
733     expect(mockMutate).toHaveBeenCalled();
734     expect.objectContaining({ day1_completed: true })
735   });
736 });
737 });
738
739 it("hides the Next button on the summary step", async () => {
740   const user = userEvent.setup();
741   renderWizard({ isDay1: true });
742
743   // Navigate to summary
744   for (let i = 0; i < 3; i++) {
745     await user.click(screen.getByRole("button", { name: /next/i }));
746   }
747   await waitFor(() =>
748     expect(screen.getByTestId("summary-step")).toBeInTheDocument()
749   );
750
751   // The Next button should not be present on the summary step
752   expect(
753     screen.queryByRole("button", { name: /next/i })
754   ).not.toBeInTheDocument();
755 });
756 });
757
758 // %% Completion navigation resilience (Req 2 – onboarding trap) %% %%
759 // A brand-new student must ALWAYS reach the dashboard when they click
760 // "Complete", even if process-onboarding is slow / errors / unavailable.
761 // Navigation is decoupled from the Edge Function's success and lives in a
762 // `finally` block; the failure is logged (not silently swallowed) and the
763 // profile post-processing call itself is preserved.
764
765 describe("Completion navigation resilience (Req 2)", () => {
766   const navigateToSummary = async (
767     user: ReturnType<typeof userEvent.setup>
768   ) => {
769     for (let i = 0; i < 3; i++) {
770       await user.click(screen.getByRole("button", { name: /next/i }));
771     }
772     await waitFor(() =>
773       expect(screen.getByTestId("summary-step")).toBeInTheDocument()
774     );
775   };
776
777   it("still navigates to /student when processOnboarding REJECTS (no dead-end overlay)", async () => {
778     // Simulate the Edge Function being slow/erroring/unavailable.
779     mockMutateAsync.mockRejectedValueOnce(
780       new Error("process-onboarding unavailable")
781     );
782     // The failure must be logged, not silently swallowed – spy + silence it.
783     const consoleErrorSpy = vi
784       .spyOn(console, "error")
785       .mockImplementation(() => {});
786
787     const user = userEvent.setup();
788     renderWizard({ isDay1: true });
789     await navigateToSummary(user);
790
791     await user.click(screen.getByTestId("confirm-profile"));
792
793     // Profile post-processing is still attempted (call is not removed)...
794     await waitFor(() => expect(mockMutateAsync).toHaveBeenCalled());
795     // ...and despite the rejection the student still lands on the dashboard.
796     await waitFor(() =>
797       expect(mockNavigate).toHaveBeenCalled();
798     );
799     // The failure was logged (Req 2 AC#3 – not silently swallowed).
800     expect(consoleErrorSpy).toHaveBeenCalled();
801     consoleErrorSpy.mockRestore();
802   });
803 }

```

```

804 |
805 | it("navigates to /student AND writes completion flags when processOnboarding RESOLVES", async () => {
806 |     mockMutateAsync.mockResolvedValueOnce({ success: true });
807 |
808 |     const user = userEvent.setup();
809 |     renderWizard({ isDay1: true });
810 |     await navigateToSummary(user);
811 |
812 |     await user.click(screen.getByTestId("confirm-profile"));
813 |
814 |     // Edge Function invoked with the student's onboarding payload.
815 |     await waitFor(() =>
816 |         expect(mockMutateAsync).toHaveBeenCalled()
817 |         expect.objectContaining({ student_id: "student-1", is_day1: true })
818 |     )
819 | );
820 | // Completion-progress write happens (dimension flags + day1_completed).
821 | await waitFor(() =>
822 |     expect(mockMutate).toHaveBeenCalled()
823 |     expect.objectContaining({
824 |         personality_completed: true,
825 |         day1_completed: true,
826 |     })
827 | );
828 | // And the student is routed to the dashboard.
829 | await waitFor(() =>
830 |     expect(mockNavigate).toHaveBeenCalled()
831 | );
832 | });
833 | });
834 | });
835 | });
836 |

```

FILE: src/___tests___/unit/outcomeChainView.test.tsx

TYPE: .tsx | LINES: 1-321 | SHA256: 335d424714d379f8f141752915b476cbef0e6f66f525d8ea57745d5ac03c91ee

```

1 | // @vitest-environment happy-dom
2 | // =====
3 | // OutcomeChainView - chain assembly rendering + unified empty state
4 | //
5 | // Feature: qa-partner-review-remediation - Req 16 (P12)
6 | // Validates: Requirements 16.1, 16.3, 16.4
7 | //
8 | // OutcomeChainView renders the end-to-end OBE traceability chain
9 | // ILO !' GA !' PLO !' CLO !' Assessment !' Rubric !' Student !' Attainment
10 | // from the assembled object returned by `useOutcomeChain`. Graduate Attributes
11 | // sit as a level BETWEEN the ILO and the PLOs (Req 16.3). When no level has a
12 | // linked record the view shows a SINGLE unified empty state for the whole chain
13 | // rather than per-level zeros (Req 16.4), and attainment at a node is rendered
14 | // with the platform attainment-level color coding (Req 16.5).
15 | //
16 | // The view's data access is fully mocked: `useOutcomeChain` is driven per-test
17 | // to exercise the data / unified-empty / loading branches, `useILOs` feeds the
18 | // header Select, and `nuqs` supplies the selected ILO id so the view is past
19 | // the "Select an institution outcome" guard. `ResizeObserver` (used by the
20 | // Radix Select trigger) is stubbed for happy-dom.
21 | // =====
22 |
23 | import { describe, it, expect, vi, beforeEach } from "vitest";
24 | import { render, screen } from "@testing-library/react";
25 |
26 | import type { OutcomeChain } from "@lib/outcomeChain";
27 | import { getAttainmentColor } from "@lib/attainmentClassifier";
28 |
29 | // -----
30 | // Mocks (must precede the component import)
31 | // -----
32 |
33 | // nuqs - supplies the selected ILO id. Driven via a `mock`-prefixed fn so a
34 | // single test can exercise the "no selection" guard by returning "".
35 | const mockStartId = vi.fn(() => string>() => "ilo-1");
36 | vi.mock("nuqs", () => ({
37 |     parseAsString: { withDefault: (def: string) => def },
38 |     useQueryState: () => [mockStartId(), vi.fn()] as const,
39 | }));
40 |
41 | // useILOs - a single ILO so the header Select has an option; the view reads
42 | // `iloPage?.data ?? []`.
43 | vi.mock("@/hooks/useILOs", () => ({
44 |     useILOs: () => ({
45 |         data: { data: [{ id: "ilo-1", title: "ILO 1 - Lifelong Learning" }] },
46 |         isLoading: false,
47 |     }),
48 | }));
49 |
50 | // useOutcomeChain is the single dependency that drives all three branches.
51 | type ChainResult = {
52 |     data: OutcomeChain | null | undefined;
53 |     isLoading: boolean;
54 |     isError: boolean;
55 | };
56 | const mockUseOutcomeChain = vi.fn(() => ChainResult>());
57 | vi.mock("@/hooks/useOutcomeChain", () => ({
58 |     useOutcomeChain: () => mockUseOutcomeChain(),
59 | }));
60 |
61 | // happy-dom does not implement ResizeObserver, relied on by the Radix Select
62 | // trigger that the view always renders.
63 | class MockResizeObserver {
64 |     observe() {}
65 |     unobserve() {}
66 |     disconnect() {}
67 | }

```

```

68 | vi.stubGlobal("ResizeObserver", MockResizeObserver);
69 |
70 | // OutcomeChainView does a role split (no feature flag): coordinators see the
71 | // redesigned Outcome Attainment screen; every other role sees the LEGACY
72 | // end-to-end chain. This suite covers the legacy chain, so stub useAuth with a
73 | // non-coordinator profile so the component resolves to the legacy view (the
74 | // coordinator path is covered by coordinatorOutcomeAttainmentNew.test.tsx).
75 | vi.mock("@/hooks/useAuth", () => ({
76 |   useAuth: () => ({ profile: null }),
77 | }));
78 |
79 | import OutcomeChainView from "@/pages/shared/OutcomeChainView";
80 |
81 | // -----
82 | // Fixtures
83 | // -----
84 |
85 | // A fully-connected chain: one ILO, one GA, one PLO with one CLO that carries
86 | // an assessment (+ its rubric), a CLO-attached rubric, and a per-student
87 | // attainment leaf. Attainment values are chosen to span distinct levels so the
88 | // color-coding assertions are meaningful (88 excellent, 72 satisfactory,
89 | // 65 developing, 91 excellent).
90 | const connectedChain: OutcomeChain = {
91 |   start: {
92 |     id: "ilo-1",
93 |     title: "ILO 1 – Lifelong Learning",
94 |     type: "ILO",
95 |     attainmentPercent: 88,
96 |   },
97 |   graduateAttributes: [
98 |     {
99 |       id: "ga-1",
100 |      name: "Critical Thinking GA",
101 |      weight: 50,
102 |      attainmentPercent: 88,
103 |    },
104 |  ],
105 |   plos: [
106 |     {
107 |       id: "plo-1",
108 |       title: "PLO 1 – Analyze Systems",
109 |       attainmentPercent: 72,
110 |       weight: 60,
111 |       clos: [
112 |         {
113 |           id: "clo-1",
114 |           title: "CLO 1 – Apply Core Concepts",
115 |           bloomsLevel: "Applying",
116 |           attainmentPercent: 65,
117 |           weight: 40,
118 |           assessments: [
119 |             {
120 |               id: "asg-1",
121 |               title: "Midterm Project",
122 |               weight: 30,
123 |               rubric: { id: "rub-1", title: "Project Rubric" },
124 |             },
125 |           ],
126 |           rubrics: [{ id: "rub-2", title: "CLO Mastery Rubric" }],
127 |           students: [
128 |             {
129 |               studentId: "stu-1",
130 |               studentName: "Alice Student",
131 |               attainmentPercent: 91,
132 |             },
133 |           ],
134 |         },
135 |       ],
136 |     },
137 |   ],
138 |   isEmpty: false,
139 | };
140 |
141 | // A chain where no level has any linked record (Req 16.4 unified empty state).
142 | const emptyChain: OutcomeChain = {
143 |   start: {
144 |     id: "ilo-1",
145 |     title: "ILO 1 – Lifelong Learning",
146 |     type: "ILO",
147 |     attainmentPercent: null,
148 |   },
149 |   graduateAttributes: [],
150 |   plos: [],
151 |   isEmpty: true,
152 | };
153 |
154 | const UNIFIED_EMPTY_TITLE = "No connected outcomes yet";
155 | const DOCUMENT_POSITION_FOLLOWING = 4;
156 |
157 | // -----
158 | // Tests
159 | // -----
160 |
161 | describe("OutcomeChainView", () => {
162 |   beforeEach(() => {
163 |     vi.clearAllMocks();
164 |     mockStartId.mockReturnValue("ilo-1");
165 |   });
166 |
167 |   it("renders the outcome chain section header", () => {
168 |     mockUseOutcomeChain.mockReturnValue({
169 |       data: connectedChain,
170 |       isLoading: false,
171 |       isError: false,

```


[illegible]

[illegible]

FILE: src/__tests__/unit/outcomesColumns.test.tsx

TYPE: .tsx | LINES: 1-89 | SHA256: f23982ff416b129dbce70ecf003de537e23b37146e83764be1fe82430621d588

```

1 //
2 // Admin Outcomes columns - Status cell unit tests
3 // Feature: production-bug-fixes, Requirement 1
4 // -----
5 // `learning_outcomes` has no `is_active` column, so `row.getValue("is_active")`
6 // is always undefined for an ILO. The Status cell must therefore treat a missing
7 // value as Active (the same default the reorder view hard-codes) and only render
8 // "Inactive" when an explicit `is_active: false` is present. These tests lock in
9 // that defaulting so the badge never falsely shows "Inactive".
10 // =====
11
12 import { describe, it, expect } from "vitest";
13 import { render, screen } from "@testing-library/react";
14 import {
15   flexRender,
16   getCoreRowModel,
17   useReactTable,
18 } from "@tanstack/react-table";
19
20 import { createColumns } from "@pages/admin/outcomes/columns";
21 import type { LearningOutcome } from "@types/app";
22
23 // `is_active` is intentionally absent from `LearningOutcome`; allow it as an
24 // optional override so we can exercise the explicit-false branch without
25 // mutating the production type.
26 type OutcomeRow = LearningOutcome & { is_active?: boolean };
27
28 const baseOutcome: LearningOutcome = {
29   id: "ilo-1",
30   type: "ILO",
31   title: "Demonstrate critical thinking",
32   description: null,
33   institution_id: "inst-1",
34   program_id: null,
35   course_id: null,
36   blooms_level: null,
37   sort_order: 0,
38   created_by: null,
39   created_at: "2024-01-01T00:00:00.000Z",
40   updated_at: "2024-01-01T00:00:00.000Z",
41 };
42
43 const noop = (): void => {};
44
45 // Render only the Status column cell for a single outcome row, driving the real
46 // TanStack Table accessor pipeline (accessorKey "is_active" -> row.getValue()).
47 const renderStatusCell = (outcome: OutcomeRow) => {
48   const Harness = () => {
49     const table = useReactTable<LearningOutcome>({
50       data: [outcome],
51       columns: createColumns(noop, noop, false),
52       getCoreRowModel: getCoreRowModel(),
53     });
54

```

```

55 |     const row = table.getRowModel().rows[0];
56 |     const statusCell = row
57 |       ? .getAllCells()
58 |         .find((cell) => cell.column.id === "is_active");
59 |
60 |     const cellRenderer = statusCell?.column.columnDef.cell;
61 |
62 |     return (
63 |       <>
64 |         {statusCell && cellRenderer
65 |           ? flexRender(cellRenderer, statusCell.getContext())
66 |           : null}
67 |       </>
68 |     );
69 |   };
70 |
71 |   return render(<Harness />);
72 | };
73 |
74 | describe("admin outcomes Status column", () => {
75 |   it("renders 'Active' when the row has no is_active field (R1)", () => {
76 |     renderStatusCell(baseOutcome);
77 |
78 |     expect(screen.getByText("Active")).toBeInTheDocument();
79 |     expect(screen.queryByText("Inactive")).not.toBeInTheDocument();
80 |   });
81 |
82 |   it("renders 'Inactive' only when is_active is explicitly false (R1)", () => {
83 |     renderStatusCell({ ...baseOutcome, is_active: false });
84 |
85 |     expect(screen.getByText("Inactive")).toBeInTheDocument();
86 |     expect(screen.queryByText("Active")).not.toBeInTheDocument();
87 |   });
88 | });
89 |

```

FILE: [src/__tests__/unit/parentDashboardSections.test.tsx](#)

TYPE: .tsx | LINES: 1-135 | SHA256: ae863ffedfa0a56e3894e979c4d3a91e3369e944912acc79587bb877a2586c89

```

1 | // =====
2 | // ParentDashboard – growth & wellbeing story (prototype rebuild P2.3)
3 | // =====
4 | // The rebuilt ParentDashboard (features/parent/dashboard/ParentDashboardScreen)
5 | // sources KPIs + linked children from a single aggregate query and renders a
6 | // single-child growth & wellbeing STORY (not a gradebook / children list):
7 | // AI story banner, "in plain words" summary, growth + wellbeing, "one way to
8 | // help", and a celebrate card. It maps avg. attainment to an OBE band rather than
9 | // a raw score. i18n is mocked to echo keys for stable assertions, so the tests
10 | // assert on the section-title keys and the empty-state key.
11 | // =====
12 | import { describe, it, expect, vi, beforeEach } from "vitest";
13 | import { render, screen } from "@testing-library/react";
14 | import { MemoryRouter } from "react-router-dom";
15 |
16 | // -----
17 | // Mocks
18 | // -----
19 |
20 | vi.mock("react-i18next", () => ({
21 |   useTranslation: () => ({
22 |     t: (key: string, opts?: Record<string, unknown>) => {
23 |       if (opts && typeof opts === "object") {
24 |         let result = key;
25 |         for (const [k, v] of Object.entries(opts)) {
26 |           result = result.replace(`{${k}}`, String(v));
27 |         }
28 |         return result;
29 |       }
30 |       return key;
31 |     },
32 |     i18n: { language: "en", changeLanguage: vi.fn() },
33 |   }),
34 |   Trans: ({ children }: { children: React.ReactNode }) => children,
35 | }));
36 |
37 | const mockAggregate = vi.fn();
38 |
39 | vi.mock("@/hooks/useParentDashboardAggregate", () => ({
40 |   useParentDashboardAggregate: () => mockAggregate(),
41 | }));
42 |
43 | vi.mock("@/hooks/useAuth", () => ({
44 |   useAuth: () => ({ user: { id: "parent-1" }, profile: { full_name: "Pat" } }),
45 | }));
46 |
47 | import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
48 | import ParentDashboard from "@/pages/parent/ParentDashboard";
49 |
50 | // -----
51 | // Helpers
52 | // -----
53 |
54 | const createTestQueryClient = () =>
55 |   new QueryClient({ defaultOptions: { queries: { retry: false } } });
56 |
57 | const renderDashboard = () =>
58 |   render(
59 |     <QueryClientProvider client={createTestQueryClient()}>
60 |       <MemoryRouter>
61 |         <ParentDashboard />
62 |       </MemoryRouter>
63 |     </QueryClientProvider>
64 |   );
65 |

```

```

66 | const CHILD = {
67 |   student_id: "s1",
68 |   student_name: "Yusuf Ahmadi",
69 |   current_level: 11,
70 |   xp_total: 1986,
71 |   current_streak: 3,
72 |   enrolled_courses: 3,
73 |   avg_attainment: 92,
74 | };
75 |
76 | const success = (children: (typeof CHILD)[]) => ({
77 |   data: {
78 |     kpis: {
79 |       linkedChildren: children.length,
80 |       totalCourses: 3,
81 |       avgAttainment: 92,
82 |       upcomingDeadlines: 0,
83 |     },
84 |     children,
85 |   },
86 |   isPending: false,
87 |   isError: false,
88 |   isSuccess: true,
89 |   refetch: vi.fn(),
90 | });
91 |
92 | // -----
93 | // Tests
94 | // -----
95 |
96 | describe("ParentDashboard – growth & wellbeing story", () => {
97 |   beforeEach(() => {
98 |     vi.clearAllMocks();
99 |   });
100 |
101 |   it("renders the growth story when the aggregate succeeds", () => {
102 |     mockAggregate.mockReturnValue(success([CHILD]));
103 |     renderDashboard();
104 |
105 |     // The "in plain words" summary + wellbeing sections render on success...
106 |     expect(
107 |       screen.getByText("parentDashboard.plainWords.title")
108 |     ).toBeInTheDocument();
109 |     expect(
110 |       screen.getByText("parentDashboard.wellbeing.title")
111 |     ).toBeInTheDocument();
112 |     // ...and the empty state is not shown.
113 |     expect(
114 |       screen.queryByText("parentDashboard.noChildren")
115 |     ).not.toBeInTheDocument();
116 |   });
117 |
118 |   it("shows the link-a-child empty state when there are no linked children", () => {
119 |     mockAggregate.mockReturnValue(success([]));
120 |     renderDashboard();
121 |
122 |     expect(screen.getByText("parentDashboard.noChildren")).toBeInTheDocument();
123 |   });
124 |
125 |   it("renders a child selector when more than one child is linked", () => {
126 |     const second = { ...CHILD, student_id: "s2", student_name: "Layla Ahmadi" };
127 |     mockAggregate.mockReturnValue(success([CHILD, second]));
128 |     renderDashboard();
129 |
130 |     // Selector uses first names for each linked child.
131 |     expect(screen.getByText("Yusuf")).toBeInTheDocument();
132 |     expect(screen.getByText("Layla")).toBeInTheDocument();
133 |   });
134 | });
135 |

```

FILE: [src/__tests__/unit/ParentFeesPage.test.tsx](#)

TYPE: .tsx | LINES: 1-124 | SHA256: c6ae2e632a122171b9ed2268ed707ec5864a4daa67e8047f488cc91d420e84de

```

1 | // ParentFeesPage – functional render tests (net-new screen).
2 | import { describe, it, expect, beforeEach, vi } from "vitest";
3 | import { render, screen, fireEvent } from "@testing-library/react";
4 | import type { LinkedChild } from "@hooks/useParentDashboard";
5 | import type { FeePayment } from "@hooks/useFees";
6 | import ParentFeesPage from "@features/parent/fees/ParentFeesPage";
7 |
8 | const hoisted = vi.hoisted(() => ({
9 |   children: {
10 |     data: undefined as LinkedChild[] | undefined,
11 |     isLoading: false,
12 |     isError: false,
13 |   },
14 |   fees: {
15 |     data: undefined as FeePayment[] | undefined,
16 |     isLoading: false,
17 |     isError: false,
18 |   },
19 |   receipt: {
20 |     mutate: vi.fn(),
21 |     isPending: false,
22 |     variables: undefined as string | undefined,
23 |   },
24 | }));
25 |
26 | vi.mock("@hooks/useParentDashboard", () => ({
27 |   useLinkedChildren: () => hoisted.children,
28 | }));
29 |
30 | vi.mock("@hooks/useFees", () => ({

```

```

31 | useStudentFees: () => hoisted.fees,
32 | useGenerateFeeReceipt: () => hoisted.receipt,
33 | });
34 |
35 | vi.mock("@hooks/useAuth", () => ({
36 |   useAuth: () => ({ user: { id: "parent-1" }, profile: null }),
37 | }));
38 |
39 | vi.mock("sonner", () => ({ toast: { success: vi.fn(), error: vi.fn() } }));
40 |
41 | vi.mock("react-i18next", () => ({
42 |   useTranslation: () => ({ t: (_key: string, def?: string) => def ?? _key }),
43 | }));
44 |
45 | const child = (id: string, name: string): LinkedChild => ({
46 |   student_id: id,
47 |   student_name: name,
48 |   current_level: 3,
49 |   xp_total: 1200,
50 |   current_streak: 5,
51 |   enrolled_courses: 4,
52 |   avg_attainment: 78,
53 | });
54 |
55 | const children: LinkedChild[] = [child("c1", "Aisha"), child("c2", "Omar")];
56 |
57 | const fees: FeePayment[] = [
58 |   {
59 |     id: "p1",
60 |     fee_structure_id: "fs1",
61 |     student_id: "c1",
62 |     amount_paid: 3000,
63 |     payment_method: "card",
64 |     receipt_number: "RC-1",
65 |     status: "paid",
66 |     payment_date: "2026-01-05",
67 |     created_at: "2026-01-05T00:00:00Z",
68 |   },
69 | ];
70 |
71 | beforeEach(() => {
72 |   hoisted.children.data = undefined;
73 |   hoisted.children.isLoading = false;
74 |   hoisted.children.isError = false;
75 |   hoisted.fees.data = undefined;
76 |   hoisted.fees.isLoading = false;
77 |   hoisted.fees.isError = false;
78 |   hoisted.receipt.mutate = vi.fn();
79 | });
80 |
81 | describe("ParentFeesPage", () => {
82 |   it("renders the title while children load", () => {
83 |     hoisted.children.isLoading = true;
84 |     render(<ParentFeesPage />);
85 |     expect(screen.getByText("Children's fees")).toBeInTheDocument();
86 |   });
87 |
88 |   it("shows an empty state when there are no linked children", () => {
89 |     hoisted.children.data = [];
90 |     render(<ParentFeesPage />);
91 |     expect(screen.getByText("No linked children yet.")).toBeInTheDocument();
92 |   });
93 |
94 |   it("renders a child selector and the selected child's fees", () => {
95 |     hoisted.children.data = children;
96 |     hoisted.fees.data = fees;
97 |     render(<ParentFeesPage />);
98 |     expect(screen.getByText("Aisha")).toBeInTheDocument();
99 |     expect(screen.getByText("Omar")).toBeInTheDocument();
100 |     // Shared fee list renders for the (default first) child.
101 |     expect(screen.getByText("Payment history")).toBeInTheDocument();
102 |     // First child is active by default.
103 |     expect(screen.getByTestId("parent-child-c1")).toHaveAttribute(
104 |       "aria-pressed",
105 |       "true"
106 |     );
107 |   });
108 |
109 |   it("switches the active child on click", () => {
110 |     hoisted.children.data = children;
111 |     hoisted.fees.data = fees;
112 |     render(<ParentFeesPage />);
113 |     fireEvent.click(screen.getByTestId("parent-child-c2"));
114 |     expect(screen.getByTestId("parent-child-c2")).toHaveAttribute(
115 |       "aria-pressed",
116 |       "true"
117 |     );
118 |     expect(screen.getByTestId("parent-child-c1")).toHaveAttribute(
119 |       "aria-pressed",
120 |       "false"
121 |     );
122 |   });
123 | });
124 |

```

FILE: [src/__tests__/unit/parentFinalDefects.test.tsx](#)

TYPE: .tsx | LINES: 1-58 | SHA256: 84c4b3c17c13543492421290f0b972a4218b6291bba3f2e9d64a3d33367fe3e1

```

1 | import { describe, it, expect, vi } from "vitest";
2 | import { render, screen } from "@testing-library/react";
3 | import { MemoryRouter } from "react-router-dom";
4 | import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
5 | import ParentFeesPage from "@features/parent/fees/ParentFeesPage";
6 | import ParentProfilePage from "@pages/parent/settings/ParentProfilePage";

```

FILE: src/__tests__/unit/parentPlannerView.test.tsx

[illegible]

[illegible]

```

153 |     await renderAtPath("/parent/planner/student-1");
154 |     // 60 + 90 = 150 min = 2.5h (planned session excluded)
155 |     expect(screen.getByText("2.5h")).toBeInTheDocument();
156 | });
157 |
158 | it("counts only completed sessions", async () => {
159 |     plannerData.sessions = [
160 |         completedSession({ id: "s1" }),
161 |         completedSession({ id: "s2" }),
162 |         completedSession({ id: "s3", status: "planned" }),
163 |         completedSession({ id: "s4", status: "cancelled" }),
164 |     ];
165 |     await renderAtPath("/parent/planner/student-1");
166 |     const sessionsLabel = screen.getByText("Sessions");
167 |     // Value paragraph is the next sibling element after the label
168 |     const valueEl = sessionsLabel.parentElement?.querySelector(
169 |         "p.text-2xl"
170 |     ) as HTMLElement | null;
171 |     expect(valueEl?.textContent).toBe("2");
172 | });
173 |
174 | it("counts only completed tasks", async () => {
175 |     plannerData.tasks = [
176 |         completedTask({ id: "t1" }),
177 |         completedTask({ id: "t2", status: "todo", completedAt: null }),
178 |     ];
179 |     await renderAtPath("/parent/planner/student-1");
180 |     const tasksLabel = screen.getByText("Tasks Done");
181 |     const valueEl = tasksLabel.parentElement?.querySelector(
182 |         "p.text-2xl"
183 |     ) as HTMLElement | null;
184 |     expect(valueEl?.textContent).toBe("1");
185 | });
186 |
187 | it("renders Weekly Schedule calendar grid", async () => {
188 |     await renderAtPath("/parent/planner/student-1");
189 |     expect(screen.getByText("Weekly Schedule")).toBeInTheDocument();
190 | });
191 |
192 | it("renders Weekly Goals panel even when goals are empty", async () => {
193 |     await renderAtPath("/parent/planner/student-1");
194 |     expect(screen.getByText(/Weekly Goals/i)).toBeInTheDocument();
195 | });
196 |
197 | it("renders weekly goals when present", async () => {
198 |     plannerData.goals = [studyHoursGoal({ targetValue: 10 })];
199 |     await renderAtPath("/parent/planner/student-1");
200 |     expect(screen.getByText(/Weekly Goals/i)).toBeInTheDocument();
201 | });
202 |
203 | it("does NOT render any Save / Add Goal / Edit affordances (read-only)", async () => {
204 |     plannerData.goals = [studyHoursGoal()];
205 |     await renderAtPath("/parent/planner/student-1");
206 |     expect(
207 |         screen.queryByRole("button", { name: /save goals/i })
208 |     ).not.toBeInTheDocument();
209 |     expect(
210 |         screen.queryByRole("button", { name: /add goal/i })
211 |     ).not.toBeInTheDocument();
212 |     expect(
213 |         screen.queryByRole("button", { name: /edit goals/i })
214 |     ).not.toBeInTheDocument();
215 | });
216 |
217 | it("does NOT render session create / edit / start affordances", async () => {
218 |     plannerData.sessions = [completedSession()];
219 |     await renderAtPath("/parent/planner/student-1");
220 |     expect(
221 |         screen.queryByRole("button", { name: /create session/i })
222 |     ).not.toBeInTheDocument();
223 |     expect(
224 |         screen.queryByRole("button", { name: /^start$/i })
225 |     ).not.toBeInTheDocument();
226 | });
227 |
228 | it("shows loading shimmers when isLoading is true", async () => {
229 |     plannerData.isLoading = true;
230 |     const { container } = await renderAtPath("/parent/planner/student-1");
231 |     // Shimmer renders animated placeholders - check for any element with the shimmer class
232 |     const shimmers = container.querySelectorAll("[class*=\"animate-shimmer\"]");
233 |     expect(shimmers.length).toBeGreaterThan(0);
234 | });
235 | });
236 |

```

FILE: [src/__tests__/unit/passwordInput.test.tsx](#)

TYPE: .tsx | LINES: 1-133 | SHA256: 226495de85630936f8ba7ff9fb4cf985dbf3040fe5430d8b3c284f553f2f05f4

```

1 | // Unit tests for PasswordInput + PasswordVisibilityGroup
2 | // Covers Requirements 5.1, 5.2, 5.3, 5.4, 5.5
3 |
4 | import { describe, it, expect } from "vitest";
5 | import { render, screen } from "@testing-library/react";
6 | import { fireEvent } from "@testing-library/user-event";
7 | import { I18nextProvider } from "react-i18next";
8 | import i18n from "@lib/i18n";
9 | import { PasswordInput } from "@components/shared/PasswordInput";
10 | import { PasswordVisibilityGroup } from "@components/shared/PasswordVisibilityGroup";
11 |
12 | const wrap = (ui: React.ReactNode) => (
13 |     <I18nextProvider i18n={i18n}>{ui}</I18nextProvider>
14 | );
15 |
16 | describe("PasswordInput", () => {

```



```

17 | it("renders a masked password input with a show toggle by default (R5.1, R5.3)", () => {
18 |     render(wrap(<PasswordInput placeholder="Password" />));
19 |
20 |     const input = screen.getByPlaceholderText("Password");
21 |     expect(input).toHaveAttribute("type", "password");
22 |
23 |     const toggle = screen.getByRole("button", { name: "Show password" });
24 |     expect(toggle).toBeInTheDocument();
25 | });
26 |
27 | it("reveals characters in plain text when the show control is activated (R5.2)", async () => {
28 |     const user = userEvent.setup();
29 |     render(wrap(<PasswordInput placeholder="Password" />));
30 |
31 |     const input = screen.getByPlaceholderText("Password");
32 |     await user.click(screen.getByRole("button", { name: "Show password" }));
33 |
34 |     expect(input).toHaveAttribute("type", "text");
35 | });
36 |
37 | it("masks characters again when the hide control is activated (R5.3)", async () => {
38 |     const user = userEvent.setup();
39 |     render(wrap(<PasswordInput placeholder="Password" />));
40 |
41 |     const input = screen.getByPlaceholderText("Password");
42 |     await user.click(screen.getByRole("button", { name: "Show password" }));
43 |     await user.click(screen.getByRole("button", { name: "Hide password" }));
44 |
45 |     expect(input).toHaveAttribute("type", "password");
46 | });
47 |
48 | it("exposes an accessible name reflecting the current action (R5.4)", async () => {
49 |     const user = userEvent.setup();
50 |     render(wrap(<PasswordInput placeholder="Password" />));
51 |
52 |     // Masked !' control offers to show.
53 |     expect(
54 |         screen.getByRole("button", { name: "Show password" })
55 |     ).toBeInTheDocument();
56 |
57 |     await user.click(screen.getByRole("button", { name: "Show password" }));
58 |
59 |     // Revealed !' control offers to hide.
60 |     expect(
61 |         screen.getByRole("button", { name: "Hide password" })
62 |     ).toBeInTheDocument();
63 | });
64 |
65 | it("meets the minimum 44px touch target (R5.4)", () => {
66 |     render(wrap(<PasswordInput placeholder="Password" />));
67 |     const toggle = screen.getByRole("button", { name: "Show password" });
68 |     // h-11 w-11 = 44px in the Tailwind scale.
69 |     expect(toggle.className).toContain("h-11");
70 |     expect(toggle.className).toContain("w-11");
71 | });
72 |
73 | it("disables the toggle when the field is disabled", () => {
74 |     render(wrap(<PasswordInput placeholder="Password" disabled />));
75 |     expect(
76 |         screen.getByRole("button", { name: "Show password" })
77 |     ).toBeDisabled();
78 | });
79 | });
80 |
81 | describe("PasswordVisibilityGroup mutual exclusion (R5.5)", () => {
82 |     const renderGroup = () => {
83 |         render(
84 |             wrap(
85 |                 <PasswordVisibilityGroup>
86 |                 <PasswordInput groupId="new" placeholder="New password" />
87 |                 <PasswordInput groupId="confirm" placeholder="Confirm password" />
88 |                 </PasswordVisibilityGroup>
89 |             )
90 |         );
91 |     };
92 |
93 | it("reveals at most one password field at a time", async () => {
94 |     const user = userEvent.setup();
95 |     renderGroup();
96 |
97 |     const newInput = screen.getByPlaceholderText("New password");
98 |     const confirmInput = screen.getByPlaceholderText("Confirm password");
99 |
100 |     // Reveal the first field.
101 |     const [firstToggle] = screen.getAllByRole("button", {
102 |         name: "Show password",
103 |     });
104 |     await user.click(firstToggle as HTMLElement);
105 |     expect(newInput).toHaveAttribute("type", "text");
106 |     expect(confirmInput).toHaveAttribute("type", "password");
107 |
108 |     // Revealing the second field must mask the first.
109 |     await user.click(screen.getByRole("button", { name: "Show password" }));
110 |     expect(confirmInput).toHaveAttribute("type", "text");
111 |     expect(newInput).toHaveAttribute("type", "password");
112 | });
113 |
114 | it("hiding the revealed field masks all fields", async () => {
115 |     const user = userEvent.setup();
116 |     renderGroup();
117 |
118 |     const newInput = screen.getByPlaceholderText("New password");
119 |
120 |     const [firstToggle] = screen.getAllByRole("button", {

```

```

121 |     });
122 |     await user.click(firstToggle as HTMLElement);
123 |     expect(newInput).toHaveAttribute("type", "text");
124 |
125 |     await user.click(screen.getByRole("button", { name: "Hide password" }));
126 |     expect(newInput).toHaveAttribute("type", "password");
127 |     // Both fields now offer "Show password".
128 |     expect(
129 |       screen.getAllByRole("button", { name: "Show password" })
130 |     ).toHaveLength(2);
131 |   });
132 | });
133 |

```

FILE: src/__tests__/unit/passwordResetFlow.test.tsx

TYPE: .tsx | LINES: 1-346 | SHA256: dc5c5c0324fb33a5ffdf0f36c851e673d9041b50ea3d5dcb7bc29615b63e5412

```

1 | import { describe, it, expect, vi, beforeEach } from "vitest";
2 | import { render, screen, waitFor } from "@testing-library/react";
3 | import userEvent from "@testing-library/user-event";
4 | import React from "react";
5 | import { MemoryRouter } from "react-router-dom";
6 | import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
7 | import "@lib/i18n";
8 | import { updatePasswordSchema, resetPasswordSchema } from "@lib/schemas/auth";
9 |
10 | // -----
11 | // Mock Supabase client
12 | // -----
13 | const mockResetPasswordForEmail = vi.fn();
14 | const mockUpdateUser = vi.fn();
15 | const mockGetSession = vi.fn();
16 | const mockOnAuthStateChange = vi.fn();
17 | const mockFrom = vi.fn();
18 |
19 | vi.mock("@lib/supabase", () => ({
20 |   supabase: {
21 |     auth: {
22 |       getSession: (...args: unknown[]) => mockGetSession(...args),
23 |       signInWithPassword: vi.fn(),
24 |       signOut: vi.fn(),
25 |       resetPasswordForEmail: (...args: unknown[]) =>
26 |         mockResetPasswordForEmail(...args),
27 |       updateUser: (...args: unknown[]) => mockUpdateUser(...args),
28 |       onAuthStateChange: (...args: unknown[]) => mockOnAuthStateChange(...args),
29 |     },
30 |     from: (...args: unknown[]) => mockFrom(...args),
31 |   },
32 | }));
33 |
34 | import { AuthProvider } from "@providers/AuthProvider";
35 | import ResetPasswordPage from "@pages/ResetPasswordPage";
36 | import UpdatePasswordPage from "@pages/UpdatePasswordPage";
37 |
38 | // -----
39 | // Helpers
40 | // -----
41 | const setupMocks = () => {
42 |   mockGetSession.mockResolvedValue({ data: { session: null } });
43 |   mockOnAuthStateChange.mockReturnValue({
44 |     data: { subscription: { unsubscribe: vi.fn() } },
45 |   });
46 |   mockFrom.mockReturnValue({
47 |     select: vi.fn().mockReturnValue({
48 |       eq: vi.fn().mockReturnValue({
49 |         maybeSingle: vi.fn().mockResolvedValue({ data: null, error: null }),
50 |       }),
51 |     }),
52 |   });
53 |   mockResetPasswordForEmail.mockResolvedValue({ error: null });
54 |   mockUpdateUser.mockResolvedValue({ data: { user: {} }, error: null });
55 | };
56 |
57 | const renderWithRouter = (ui: React.ReactElement) => {
58 |   const queryClient = new QueryClient({
59 |     defaultOptions: { queries: { retry: false } },
60 |   });
61 |   return render(
62 |     <QueryClientProvider client={queryClient}>
63 |       <MemoryRouter>
64 |         <AuthProvider>{ui}</AuthProvider>
65 |       </MemoryRouter>
66 |     </QueryClientProvider>
67 |   );
68 | };
69 |
70 | const renderWithAuthProvider = (ui: React.ReactElement) => {
71 |   const queryClient = new QueryClient({
72 |     defaultOptions: { queries: { retry: false } },
73 |   });
74 |   return render(
75 |     <QueryClientProvider client={queryClient}>
76 |       <MemoryRouter>
77 |         <AuthProvider>{ui}</AuthProvider>
78 |       </MemoryRouter>
79 |     </QueryClientProvider>
80 |   );
81 | };
82 |
83 | // -----
84 | // Schema Tests
85 | // -----
86 | describe("updatePasswordSchema", () => {
87 |   it("accepts valid matching passwords", () => {

```

```

88 |     const result = updatePasswordSchema.safeParse({
89 |       password: "securePass1",
90 |       confirmPassword: "securePass1",
91 |     });
92 |     expect(result.success).toBe(true);
93 |   });
94 |
95 |   it("rejects passwords shorter than 8 characters", () => {
96 |     const result = updatePasswordSchema.safeParse({
97 |       password: "short",
98 |       confirmPassword: "short",
99 |     });
100 |     expect(result.success).toBe(false);
101 |   });
102 |
103 |   it("rejects mismatched passwords", () => {
104 |     const result = updatePasswordSchema.safeParse({
105 |       password: "securePass1",
106 |       confirmPassword: "differentPass",
107 |     });
108 |     expect(result.success).toBe(false);
109 |     if (!result.success) {
110 |       const paths = result.error.issues.map((i) => i.path.join("."));
111 |       expect(paths).toContain("confirmPassword");
112 |     }
113 |   });
114 |
115 |   it("rejects empty confirmPassword", () => {
116 |     const result = updatePasswordSchema.safeParse({
117 |       password: "securePass1",
118 |       confirmPassword: "",
119 |     });
120 |     expect(result.success).toBe(false);
121 |   });
122 | });
123 |
124 | describe("resetPasswordSchema", () => {
125 |   it("accepts a valid email", () => {
126 |     const result = resetPasswordSchema.safeParse({ email: "user@test.edu" });
127 |     expect(result.success).toBe(true);
128 |   });
129 |
130 |   it("rejects an invalid email", () => {
131 |     const result = resetPasswordSchema.safeParse({ email: "not-an-email" });
132 |     expect(result.success).toBe(false);
133 |   });
134 | });
135 |
136 | // -----
137 | // ResetPasswordPage Tests
138 | // -----
139 | describe("ResetPasswordPage", () => {
140 |   beforeEach(() => {
141 |     vi.clearAllMocks();
142 |     setupMocks();
143 |   });
144 |
145 |   it("renders the reset password form", async () => {
146 |     renderWithAuthProvider(<ResetPasswordPage />);
147 |     await waitFor(() => {
148 |       expect(screen.getByText("Reset Password")).toBeInTheDocument();
149 |     });
150 |     expect(
151 |       screen.getByPlaceholderText("you@institution.edu")
152 |     ).toBeInTheDocument();
153 |     expect(
154 |       screen.getByRole("button", { name: /send reset link/i })
155 |     ).toBeInTheDocument();
156 |   });
157 |
158 |   it("shows success state after submitting a valid email", async () => {
159 |     const user = userEvent.setup();
160 |     renderWithAuthProvider(<ResetPasswordPage />);
161 |
162 |     await waitFor(() => {
163 |       expect(
164 |         screen.getByPlaceholderText("you@institution.edu")
165 |       ).toBeInTheDocument();
166 |     });
167 |
168 |     await user.type(
169 |       screen.getByPlaceholderText("you@institution.edu"),
170 |       "user@test.edu"
171 |     );
172 |     await user.click(screen.getByRole("button", { name: /send reset link/i }));
173 |
174 |     await waitFor(() => {
175 |       expect(
176 |         screen.getAllByText(/password reset link has been sent/i).length
177 |       ).toBeGreaterThanOrEqual(1);
178 |     });
179 |     expect(mockResetPasswordForEmail).toHaveBeenCalledWith("user@test.edu");
180 |   });
181 |
182 |   it("shows success state even when supabase returns an error (prevents email enumeration)", async () => {
183 |     mockResetPasswordForEmail.mockResolvedValue({
184 |       error: { message: "User not found" },
185 |     });
186 |
187 |     const user = userEvent.setup();
188 |     renderWithAuthProvider(<ResetPasswordPage />);
189 |
190 |     await waitFor(() => {
191 |       expect(

```

```

192 |         screen.getByPlaceholderText("you@institution.edu")
193 |     ).toBeInTheDocument();
194 | });
195 |
196 | await user.type(
197 |     screen.getByPlaceholderText("you@institution.edu"),
198 |     "nonexistent@test.edu"
199 | );
200 | await user.click(screen.getByRole("button", { name: /send reset link/i }));
201 |
202 | await waitFor(() => {
203 |     expect(
204 |         screen.getAllByText(/password reset link has been sent/i).length
205 |     ).toBeGreaterThanOrEqual(1);
206 | });
207 | });
208 |
209 | it("does not submit when email field is empty", async () => {
210 |     const user = userEvent.setup();
211 |     renderWithAuthProvider(<ResetPasswordPage />);
212 |
213 |     await waitFor(() => {
214 |         expect(
215 |             screen.getByPlaceholderText("you@institution.edu")
216 |         ).toBeInTheDocument();
217 |     });
218 |
219 |     await user.click(screen.getByRole("button", { name: /send reset link/i }));
220 |
221 |     await waitFor(() => {
222 |         expect(screen.getByText(/invalid email address/i)).toBeInTheDocument();
223 |     });
224 |     expect(mockResetPasswordForEmail).not.toHaveBeenCalled();
225 | });
226 |
227 | it("has a back to login link", async () => {
228 |     renderWithAuthProvider(<ResetPasswordPage />);
229 |     await waitFor(() => {
230 |         expect(screen.getByText(/back to sign in/i)).toBeInTheDocument();
231 |     });
232 | });
233 | });
234 |
235 | // -----
236 | // UpdatePasswordPage Tests
237 | // -----
238 | describe("UpdatePasswordPage", () => {
239 |     beforeEach(() => {
240 |         vi.clearAllMocks();
241 |         setupMocks();
242 |     });
243 |
244 |     it("renders the update password form", () => {
245 |         renderWithRouter(<UpdatePasswordPage />);
246 |         expect(screen.getByText("Set New Password")).toBeInTheDocument();
247 |         expect(
248 |             screen.getByPlaceholderText("Minimum 8 characters")
249 |         ).toBeInTheDocument();
250 |         expect(
251 |             screen.getByPlaceholderText("Re-enter your password")
252 |         ).toBeInTheDocument();
253 |         expect(
254 |             screen.getByRole("button", { name: /update password/i })
255 |         ).toBeInTheDocument();
256 |     });
257 |
258 |     it("updates password successfully and shows success state", async () => {
259 |         const user = userEvent.setup();
260 |         renderWithRouter(<UpdatePasswordPage />);
261 |
262 |         await user.type(
263 |             screen.getByPlaceholderText("Minimum 8 characters"),
264 |             "newSecure123"
265 |         );
266 |         await user.type(
267 |             screen.getByPlaceholderText("Re-enter your password"),
268 |             "newSecure123"
269 |         );
270 |         await user.click(screen.getByRole("button", { name: /update password/i }));
271 |
272 |         await waitFor(() => {
273 |             expect(
274 |                 screen.getAllByText(/password has been updated/i).length
275 |             ).toBeGreaterThanOrEqual(1);
276 |         });
277 |         expect(mockUpdateUser).toHaveBeenCalledWith({ password: "newSecure123" });
278 |     });
279 |
280 |     it("shows validation error for mismatched passwords", async () => {
281 |         const user = userEvent.setup();
282 |         renderWithRouter(<UpdatePasswordPage />);
283 |
284 |         await user.type(
285 |             screen.getByPlaceholderText("Minimum 8 characters"),
286 |             "newSecure123"
287 |         );
288 |         await user.type(
289 |             screen.getByPlaceholderText("Re-enter your password"),
290 |             "different123"
291 |         );
292 |         await user.click(screen.getByRole("button", { name: /update password/i }));
293 |
294 |         await waitFor(() => {
295 |             expect(screen.getByText(/passwords do not match/i)).toBeInTheDocument();

```

```

296 |     });
297 |     expect(mockUpdateUser).not.toHaveBeenCalled();
298 | });
299 |
300 | it("shows validation error for short password", async () => {
301 |   const user = userEvent.setup();
302 |   renderWithRouter(<UpdatePasswordPage />);
303 |
304 |   await user.type(
305 |     screen.getByPlaceholderText("Minimum 8 characters"),
306 |     "short"
307 |   );
308 |   await user.type(
309 |     screen.getByPlaceholderText("Re-enter your password"),
310 |     "short"
311 |   );
312 |   await user.click(screen.getByRole("button", { name: /update password/i }));
313 |
314 |   await waitFor(() => {
315 |     expect(screen.getByText(/at least 8 characters/i)).toBeInTheDocument();
316 |   });
317 |   expect(mockUpdateUser).not.toHaveBeenCalled();
318 | });
319 |
320 | it("shows error toast when supabase update fails", async () => {
321 |   mockUpdateUser.mockResolvedValue({
322 |     data: null,
323 |     error: { message: "Token expired" },
324 |   });
325 |
326 |   const user = userEvent.setup();
327 |   renderWithRouter(<UpdatePasswordPage />);
328 |
329 |   await user.type(
330 |     screen.getByPlaceholderText("Minimum 8 characters"),
331 |     "newSecure123"
332 |   );
333 |   await user.type(
334 |     screen.getByPlaceholderText("Re-enter your password"),
335 |     "newSecure123"
336 |   );
337 |   await user.click(screen.getByRole("button", { name: /update password/i }));
338 |
339 |   // Should NOT show success state
340 |   await waitFor(() => {
341 |     expect(screen.queryByText("Password Updated")).not.toBeInTheDocument();
342 |   });
343 |   expect(screen.getByText("Set New Password")).toBeInTheDocument();
344 | }, 15_000);
345 | });
346 |

```

FILE: [src/__tests__/unit/peerTeachingMomentCard.test.tsx](#)

TYPE: .tsx | LINES: 1-82 | SHA256: 3dc97a755289ceb70dce9113e4297e2a3df0909d3ba0eef0fa3e20e3d997b7ac

```

1 | // Unit test: PeerTeachingMomentCard - renders title, text, media link, ratings, view count
2 | import { describe, it, expect } from "vitest";
3 | import { render, screen } from "@testing-library/react";
4 | import PeerTeachingMomentCard from "@components/shared/PeerTeachingMomentCard";
5 |
6 | const defaultProps = {
7 |   title: "Understanding Recursion",
8 |   explanationText:
9 |     "Recursion is when a function calls itself. The key is to have a base case that stops the recursion.",
10 |   authorName: "Alice",
11 |   createdAt: new Date().toISOString(),
12 | };
13 |
14 | describe("PeerTeachingMomentCard", () => {
15 |   it("renders title", () => {
16 |     render(<PeerTeachingMomentCard {...defaultProps} />);
17 |     expect(screen.getByText("Understanding Recursion")).toBeDefined();
18 |   });
19 |
20 |   it("renders explanation text", () => {
21 |     render(<PeerTeachingMomentCard {...defaultProps} />);
22 |     expect(screen.getByText(defaultProps.explanationText)).toBeDefined();
23 |   });
24 |
25 |   it("renders author name", () => {
26 |     render(<PeerTeachingMomentCard {...defaultProps} />);
27 |     expect(screen.getByText(/Alice/)).toBeDefined();
28 |   });
29 |
30 |   it("renders media link when provided", () => {
31 |     render(
32 |       <PeerTeachingMomentCard
33 |         {...defaultProps}
34 |         mediaUrl="https://example.com/video"
35 |       />
36 |     );
37 |     const link = screen.getByText("View media");
38 |     expect(link).toBeDefined();
39 |     expect(link.closest("a")?.getAttribute("href")).toBe(
40 |       "https://example.com/video"
41 |     );
42 |   });
43 |
44 |   it("does not render media link when not provided", () => {
45 |     render(<PeerTeachingMomentCard {...defaultProps} />);
46 |     expect(screen.queryByText("View media")).toBeNull();
47 |   });
48 |
49 |   it("renders average clarity rating", () => {

```

```

50 |     render(<PeerTeachingMomentCard {...defaultProps} avgClarityRating={4.5} />);
51 |     expect(screen.getByText(/Clarity: 4.5/)).toBeDefined();
52 |   });
53 |
54 |   it("renders average helpfulness rating", () => {
55 |     render(
56 |       <PeerTeachingMomentCard {...defaultProps} avgHelpfulnessRating={3.8} />
57 |     );
58 |     expect(screen.getByText(/Helpful: 3.8/)).toBeDefined();
59 |   });
60 |
61 |   it("renders view count", () => {
62 |     render(<PeerTeachingMomentCard {...defaultProps} viewCount={42} />);
63 |     expect(screen.getByText("42")).toBeDefined();
64 |   });
65 |
66 |   it("renders total ratings count", () => {
67 |     render(
68 |       <PeerTeachingMomentCard
69 |         {...defaultProps}
70 |         totalRatings={5}
71 |         avgClarityRating={4.0}
72 |       />
73 |     );
74 |     expect(screen.getByText("(5 ratings)")).toBeDefined();
75 |   });
76 |
77 |   it("renders CLO title badge when provided", () => {
78 |     render(<PeerTeachingMomentCard {...defaultProps} cloTitle="CLO 1.1" />);
79 |     expect(screen.getByText("CLO 1.1")).toBeDefined();
80 |   });
81 | });
82 |

```

FILE: [src/__tests__/unit/percentileBand.test.ts](#)

TYPE: .ts | LINES: 1-81 | SHA256: eed13cb85a08bbae128bfb5292fb1e4f5a2a9526e89ab4dddf698a64b8ca08d9c

```

1 | import { describe, it, expect } from "vitest";
2 | import {
3 |   calculatePercentileBand,
4 |   formatPercentileBand,
5 | } from "@lib/percentileBand";
6 |
7 | describe("calculatePercentileBand", () => {
8 |   it("returns exact rank for top 10 students", () => {
9 |     const result = calculatePercentileBand(1, 100);
10 |     expect(result).toEqual({ type: "exact", rank: 1 });
11 |   });
12 |
13 |   it("returns exact rank for rank 10", () => {
14 |     const result = calculatePercentileBand(10, 100);
15 |     expect(result).toEqual({ type: "exact", rank: 10 });
16 |   });
17 |
18 |   it("returns Top 10% band for rank 11 out of 200", () => {
19 |     // 11/200 = 5.5% != Top 10%
20 |     const result = calculatePercentileBand(11, 200);
21 |     expect(result).toEqual({ type: "band", band: "Top 10%" });
22 |   });
23 |
24 |   it("returns Top 25% band for rank 20 out of 100", () => {
25 |     // 20/100 = 20% != Top 25%
26 |     const result = calculatePercentileBand(20, 100);
27 |     expect(result).toEqual({ type: "band", band: "Top 25%" });
28 |   });
29 |
30 |   it("returns Top 50% band for rank 40 out of 100", () => {
31 |     // 40/100 = 40% != Top 50%
32 |     const result = calculatePercentileBand(40, 100);
33 |     expect(result).toEqual({ type: "band", band: "Top 50%" });
34 |   });
35 |
36 |   it("returns Bottom 50% band for rank 60 out of 100", () => {
37 |     // 60/100 = 60% != Bottom 50%
38 |     const result = calculatePercentileBand(60, 100);
39 |     expect(result).toEqual({ type: "band", band: "Bottom 50%" });
40 |   });
41 |
42 |   it("handles edge case: totalStudents = 0", () => {
43 |     const result = calculatePercentileBand(1, 0);
44 |     expect(result).toEqual({ type: "exact", rank: 1 });
45 |   });
46 |
47 |   it("handles edge case: rank = 0", () => {
48 |     const result = calculatePercentileBand(0, 100);
49 |     expect(result).toEqual({ type: "exact", rank: 1 });
50 |   });
51 |
52 |   it("returns exact rank for top 10 even with small total", () => {
53 |     const result = calculatePercentileBand(5, 8);
54 |     expect(result).toEqual({ type: "exact", rank: 5 });
55 |   });
56 |
57 |   it("returns Top 10% at boundary (exactly 10%)", () => {
58 |     // 11/110 = 10% != Top 10%
59 |     const result = calculatePercentileBand(11, 110);
60 |     expect(result).toEqual({ type: "band", band: "Top 10%" });
61 |   });
62 |
63 |   it("returns Top 25% at boundary (exactly 25%)", () => {
64 |     // 25/100 = 25% != Top 25%
65 |     const result = calculatePercentileBand(25, 100);
66 |     expect(result).toEqual({ type: "band", band: "Top 25%" });
67 |   });

```

```

68 | });
69 |
70 | describe("formatPercentileBand", () => {
71 |   it("formats exact rank", () => {
72 |     expect(formatPercentileBand({ type: "exact", rank: 3 })).toBe("#3");
73 |   });
74 |
75 |   it("formats band label", () => {
76 |     expect(formatPercentileBand({ type: "band", band: "Top 25%" })).toBe(
77 |       "Top 25%"
78 |     );
79 |   });
80 | });
81 |

```

FILE: src/__tests__/unit/perfectDay.test.ts

```
TYPE: .ts | LINES: 1-134 | SHA256: b248d0c1c6b2a11de0a5a92d85da269a748df12c7ddb26fe2ddf61a28d748a7b
```

[illegible]

```

87 |
88 |     const result = await awardPerfectDayIfComplete("student-1");
89 |
90 |     expect(result.awarded).toBe(false);
91 |     expect(mockInvoke).not.toHaveBeenCalled();
92 | });
93 |
94 | it("treats duplicate habit rows as a single habit (no false positive)", async () => {
95 |     mockHabitRows = [
96 |         { habit_type: "login" },
97 |         { habit_type: "login" },
98 |         { habit_type: "submit" },
99 |         { habit_type: "journal" },
100 |     ];
101 |
102 |     const result = await awardPerfectDayIfComplete("student-1");
103 |
104 |     expect(result.awarded).toBe(false);
105 |     expect(mockInvoke).not.toHaveBeenCalled();
106 | });
107 |
108 | it("uses a deterministic per-day reference_id for idempotency", async () => {
109 |     mockHabitRows = ALL_FOUR;
110 |
111 |     const result = await awardPerfectDayIfComplete("student-9");
112 |
113 |     const call = mockInvoke.mock.calls.find((c) => c[0] === "award-xp");
114 |     const body = (call?.[1] as { body: { reference_id: string } }).body;
115 |     expect(body.reference_id).toBe(`perfect_day:student-9:${result.date}`);
116 | });
117 |
118 | it("returns not-awarded for an empty studentId without invoking", async () => {
119 |     const result = await awardPerfectDayIfComplete("");
120 |
121 |     expect(result.awarded).toBe(false);
122 |     expect(mockInvoke).not.toHaveBeenCalled();
123 | });
124 |
125 | it("does not throw and returns not-awarded when habit read errors", async () => {
126 |     mockHabitError = { message: "boom" };
127 |
128 |     const result = await awardPerfectDayIfComplete("student-1");
129 |
130 |     expect(result.awarded).toBe(false);
131 |     expect(mockInvoke).not.toHaveBeenCalled();
132 | });
133 | });
134 |

```

FILE: [src/__tests__/unit/personalBestCard.test.tsx](#)

TYPE: .tsx | LINES: 1-72 | SHA256: a32f5ec6067e37dc881a5dd01edc00f97099e03ba0d87fa46c1744ab39c9aee1

```

1 | // =====
2 | // Unit Test: Personal Best Card
3 | // Task 26.7 – Metric comparison, delta arrows
4 | // =====
5 |
6 | import { describe, it, expect } from "vitest";
7 | import { render, screen } from "@testing-library/react";
8 | import PersonalBestCard from "@components/shared/PersonalBestCard";
9 |
10 | describe("PersonalBestCard", () => {
11 |     it("renders the label and current value", () => {
12 |         render(
13 |             <PersonalBestCard
14 |                 label="Weekly XP"
15 |                 currentValue={500}
16 |                 previousValue={400}
17 |             />
18 |         );
19 |
20 |         expect(screen.getByText("Weekly XP")).toBeDefined();
21 |         expect(screen.getByText("500")).toBeDefined();
22 |     });
23 |
24 |     it("shows upward delta when current > previous", () => {
25 |         render(
26 |             <PersonalBestCard label="XP" currentValue={500} previousValue={400} />
27 |         );
28 |
29 |         expect(screen.getByTestId("delta-up")).toBeDefined();
30 |         expect(screen.getByText("+100 (25%)")).toBeDefined();
31 |     });
32 |
33 |     it("shows downward delta when current < previous", () => {
34 |         render(
35 |             <PersonalBestCard label="XP" currentValue={300} previousValue={400} />
36 |         );
37 |
38 |         expect(screen.getByTestId("delta-down")).toBeDefined();
39 |         expect(screen.getByText("-100 (-25%)")).toBeDefined();
40 |     });
41 |
42 |     it("shows no change when current equals previous", () => {
43 |         render(
44 |             <PersonalBestCard label="XP" currentValue={400} previousValue={400} />
45 |         );
46 |
47 |         expect(screen.getByTestId("delta-neutral")).toBeDefined();
48 |         expect(screen.getByText("No change")).toBeDefined();
49 |     });
50 |
51 |     it("shows personal best label when isPersonalBest is true", () => {
52 |         render(

```



```

53 |         <PersonalBestCard
54 |             label="XP"
55 |             currentValue={500}
56 |             previousValue={400}
57 |             isPersonalBest={true}
58 |         />
59 |     );
60 |
61 |     expect(screen.getByTestId("personal-best-label")).toBeDefined();
62 | });
63 |
64 | it("does not show personal best label by default", () => {
65 |     render(
66 |         <PersonalBestCard label="XP" currentValue={500} previousValue={400} />
67 |     );
68 |
69 |     expect(screen.queryByTestId("personal-best-label")).toBeNull();
70 | });
71 | });
72 |

```

FILE: [src/__tests__/unit/personalBestLeaderboard.test.ts](#)

TYPE: .ts | LINES: 1-104 | SHA256: bc40236259c6e2237bdf0f645c82da7dc605030d761b35281cf18ea23e2f6c8

```

1 | // =====
2 | // personalBestLeaderboard - Unit tests
3 | // =====
4 |
5 | import { describe, it, expect } from "vitest";
6 | import {
7 |     computeWeeklyXP,
8 |     getISOWeekStart,
9 | } from "@lib/personalBestLeaderboard";
10 |
11 | describe("getISOWeekStart", () => {
12 |     it("returns Monday for a Wednesday", () => {
13 |         // Use UTC dates to match the UTC-based implementation
14 |         const wed = new Date(Date.UTC(2025, 6, 23)); // July 23, 2025 is a Wednesday (UTC)
15 |         const result = getISOWeekStart(wed);
16 |         expect(result.getUTCDay()).toBe(1); // Monday
17 |         expect(result.getUTCFullYear()).toBe(2025);
18 |         expect(result.getUTCMonth()).toBe(6); // July
19 |         expect(result.getUTCDate()).toBe(21);
20 |     });
21 |
22 |     it("returns Monday for a Monday", () => {
23 |         const mon = new Date(Date.UTC(2025, 6, 21)); // July 21, 2025 is a Monday (UTC)
24 |         const result = getISOWeekStart(mon);
25 |         expect(result.getUTCDay()).toBe(1);
26 |         expect(result.getUTCDate()).toBe(21);
27 |     });
28 |
29 |     it("returns Monday for a Sunday", () => {
30 |         const sun = new Date(Date.UTC(2025, 6, 27)); // July 27, 2025 is a Sunday (UTC)
31 |         const result = getISOWeekStart(sun);
32 |         expect(result.getUTCDay()).toBe(1);
33 |         expect(result.getUTCDate()).toBe(21);
34 |     });
35 | });
36 |
37 | describe("computeWeeklyXP", () => {
38 |     // Use UTC dates to match the UTC-based implementation
39 |     const refDate = new Date(Date.UTC(2025, 6, 23, 12, 0, 0)); // July 23, 2025 Wednesday UTC
40 |
41 |     it("returns 8 weeks of data", () => {
42 |         const result = computeWeeklyXP([], refDate);
43 |         expect(result).toHaveLength(8);
44 |     });
45 |
46 |     it("marks the last week as current week", () => {
47 |         const result = computeWeeklyXP([], refDate);
48 |         expect(result[7]!.isCurrentWeek).toBe(true);
49 |         expect(result[0]!.isCurrentWeek).toBe(false);
50 |     });
51 |
52 |     it("correctly sums XP for each week", () => {
53 |         // Current week starts Monday July 21, 2025
54 |         const mon = new Date(Date.UTC(2025, 6, 21, 10, 0, 0));
55 |         const tue = new Date(Date.UTC(2025, 6, 22, 10, 0, 0));
56 |         const prevWeek = new Date(Date.UTC(2025, 6, 15, 10, 0, 0));
57 |
58 |         const transactions = [
59 |             { xp_amount: 50, created_at: mon.toISOString() },
60 |             { xp_amount: 30, created_at: tue.toISOString() },
61 |             { xp_amount: 100, created_at: prevWeek.toISOString() },
62 |         ];
63 |
64 |         const result = computeWeeklyXP(transactions, refDate);
65 |         expect(result[7]!.xp).toBe(80);
66 |         expect(result[6]!.xp).toBe(100);
67 |     });
68 |
69 |     it("marks the week with highest XP as personal best", () => {
70 |         const mon = new Date(Date.UTC(2025, 6, 21, 10, 0, 0));
71 |         const prevWeek = new Date(Date.UTC(2025, 6, 15, 10, 0, 0));
72 |
73 |         const transactions = [
74 |             { xp_amount: 50, created_at: mon.toISOString() },
75 |             { xp_amount: 200, created_at: prevWeek.toISOString() },
76 |         ];
77 |
78 |         const result = computeWeeklyXP(transactions, refDate);
79 |         const bestWeek = result.find((w) => w.isPersonalBest);
80 |         expect(bestWeek).toBeDefined();

```

```

81 |     expect(bestWeek!.xp).toBe(200);
82 | });
83 |
84 | it("does not mark personal best when all weeks have 0 XP", () => {
85 |     const result = computeWeeklyXP([], refDate);
86 |     const bestWeek = result.find((w) => w.isPersonalBest);
87 |     expect(bestWeek).toBeUndefined();
88 | });
89 |
90 | it("marks exactly one week as personal best", () => {
91 |     const mon = new Date(Date.UTC(2025, 6, 21, 10, 0, 0));
92 |     const prevWeek = new Date(Date.UTC(2025, 6, 15, 10, 0, 0));
93 |
94 |     const transactions = [
95 |         { xp_amount: 100, created_at: mon.toISOString() },
96 |         { xp_amount: 100, created_at: prevWeek.toISOString() },
97 |     ];
98 |
99 |     const result = computeWeeklyXP(transactions, refDate);
100 |     const bestWeeks = result.filter((w) => w.isPersonalBest);
101 |     expect(bestWeeks).toHaveLength(1);
102 | });
103 | });
104 |

```

FILE: src/__tests__/unit/plannerConsolidation.test.tsx

TYPE: .tsx | LINES: 1-404 | SHA256: 5d29dbde0058b4ff672cb83fb43cf4a02a3a2525ca776dbcfb396047b7539f0e

[illegible]

[illegible]

```

181 |
182 | it("uses the canonical English labels from the shared i18n keys", () => {
183 |   // Guards against the keys silently resolving to the raw key string.
184 |   expect(ADD_TASK).toBe("Add task");
185 |   expect(START_SESSION).toBe("Start session");
186 | });
187 |
188 | it("WeeklyPlannerPage renders the single canonical add/start labels", async () => {
189 |   const { default: WeeklyPlannerPage } = await import(
190 |     "@pages/student/planner/WeeklyPlannerPage"
191 |   );
192 |   render(wrap(<WeeklyPlannerPage />));
193 |
194 |   expect(
195 |     screen.getByRole("button", { name: new RegExp(ADD_TASK, "i") })
196 |   ).toBeInTheDocument();
197 |   expect(
198 |     screen.getByRole("button", { name: new RegExp(START_SESSION, "i") })
199 |   ).toBeInTheDocument();
200 | });
201 |
202 | it("WeeklyPlannerPage shows no legacy duplicate-labelled controls", async () => {
203 |   const { default: WeeklyPlannerPage } = await import(
204 |     "@pages/student/planner/WeeklyPlannerPage"
205 |   );
206 |   render(wrap(<WeeklyPlannerPage />));
207 |
208 |   // Pre-consolidation the weekly surface exposed bare "Task" / "Session"
209 |   // buttons; those must no longer exist as standalone action buttons.
210 |   expect(
211 |     screen.queryByRole("button", { name: /^task$/i })
212 |   ).not.toBeInTheDocument();
213 |   expect(
214 |     screen.queryByRole("button", { name: /^session$/i })
215 |   ).not.toBeInTheDocument();
216 |   // The Today surface's old wording must not leak onto the weekly surface.
217 |   expect(screen.queryByText(/quick add/i)).not.toBeInTheDocument();
218 |   expect(
219 |     screen.queryByText(/start unplanned session/i)
220 |   ).not.toBeInTheDocument();
221 | });
222 |
223 | it("TodayViewPage renders the same canonical labels (cross-surface consistency)", async () => {
224 |   const { default: TodayViewPage } = await import(
225 |     "@pages/student/planner/TodayViewPage"
226 |   );
227 |   render(wrap(<TodayViewPage />));
228 |
229 |   expect(
230 |     screen.getByRole("button", { name: new RegExp(ADD_TASK, "i") })
231 |   ).toBeInTheDocument();
232 |   expect(
233 |     screen.getByRole("button", { name: new RegExp(START_SESSION, "i") })
234 |   ).toBeInTheDocument();
235 |   // Legacy Today labels must be gone.
236 |   expect(screen.queryByText(/quick add/i)).not.toBeInTheDocument();
237 |   expect(
238 |     screen.queryByText(/start unplanned session/i)
239 |   ).not.toBeInTheDocument();
240 | });
241 | });
242 |
243 | // % % % R19.1 – derived per-day content surfaces on the weekly planner % % % % % % % % %
244 |
245 | describe("Weekly planner surfaces derived per-day content (R19.1)", () => {
246 |   beforeEach(async () => {
247 |     vi.clearAllMocks();
248 |     resetWeeklyData();
249 |     await i18n.changeLanguage("en");
250 |   });
251 |
252 |   it("renders a derived deadline on its day instead of a bare 'No items'", async () => {
253 |     // Build a deadline within the currently-rendered week so it lands on a day.
254 |     const { getWeekStartDate } = await import("@/lib/plannerUtils");
255 |     const weekStart = getWeekStartDate(new Date()); // YYYY-MM-DD (Mon)
256 |
257 |     weeklyData.deadlines = [
258 |       {
259 |         id: "deadline-1",
260 |         title: "Lab Report Due",
261 |         courseName: "Chemistry 101",
262 |         dueDate: `${weekStart}T23:59:00Z`,
263 |         urgency: "yellow",
264 |       },
265 |     ];
266 |     enrolledCourses.value = [{ id: "course-1", name: "Chemistry 101" }];
267 |
268 |     const { default: WeeklyPlannerPage } = await import(
269 |       "@pages/student/planner/WeeklyPlannerPage"
270 |     );
271 |     render(wrap(<WeeklyPlannerPage />));
272 |
273 |     // The derived deadline is surfaced (desktop grid renders all seven days).
274 |     expect(screen.getAllByText("Lab Report Due").length).toBeGreaterThan(0);
275 |   });
276 |
277 |   it("offers suggested study sessions so empty days are not blank", async () => {
278 |     enrolledCourses.value = [{ id: "course-1", name: "Chemistry 101" }];
279 |
280 |     const { default: WeeklyPlannerPage } = await import(
281 |       "@pages/student/planner/WeeklyPlannerPage"
282 |     );
283 |     render(wrap(<WeeklyPlannerPage />));
284 |

```

```

285 | // With no student-created items, every today-or-future day gets a suggested
286 | // study session rather than a bare "No items" label. Today is always within
287 | // the rendered week, so at least one suggestion is surfaced regardless of
288 | // which weekday the test runs on. (Empty *past* days legitimately keep the
289 | // "No items" placeholder, since a session cannot be planned in the past.)
290 | expect(
291 |   screen.getAllByTestId("suggested-session").length
292 | ).toBeGreaterThanOrEqual(1);
293 | });
294 | });
295 |
296 | // % % % R19.4 – example goals shown when the student has set none % % % % % % % % % % % % % % % %
297 |
298 | describe("WeeklyGoalPanel example goals when empty (R19.4)", () => {
299 |   let WeeklyGoalPanel: typeof import("@/components/shared/WeeklyGoalPanel").default;
300 |
301 |   const exampleGoals = [
302 |     { label: "Study Hours", targetText: "5 hours" },
303 |     { label: "Sessions", targetText: "3 sessions" },
304 |     { label: "Tasks", targetText: "5 tasks" },
305 |   ];
306 |   const exampleGoalsCopy = {
307 |     heading: "Goal ideas to get started",
308 |     hint: "Set a goal below to track your week.",
309 |     cta: "Set goals",
310 |   };
311 |
312 |   const makeGoal = (overrides: Partial<WeeklyGoal> = {}): WeeklyGoal => ({
313 |     id: "goal-1",
314 |     studentId: "student-1",
315 |     weekStartDate: "2025-06-16",
316 |     goalType: "study_hours",
317 |     targetValue: 5,
318 |     ...overrides,
319 |   });
320 |
321 |   const makeProgress = (goal: WeeklyGoal): GoalProgress => ({
322 |     goal,
323 |     currentValue: 2,
324 |     percentage: 40,
325 |     isMet: false,
326 |   });
327 |
328 |   beforeEach(async () => {
329 |     const mod = await import("@/components/shared/WeeklyGoalPanel");
330 |     WeeklyGoalPanel = mod.default;
331 |   });
332 |
333 |   it("shows example goals and a set-goals CTA when there are no goals", () => {
334 |     render(
335 |       wrap(
336 |         <WeeklyGoalPanel
337 |           goals={[]}
338 |           progress={[]}
339 |           weekStartDate="2025-06-16"
340 |           onSave={vi.fn()}
341 |           exampleGoals={exampleGoals}
342 |           exampleGoalsCopy={exampleGoalsCopy}
343 |         />
344 |       )
345 |     );
346 |
347 |     expect(screen.getByText("Goal ideas to get started")).toBeInTheDocument();
348 |     expect(
349 |       screen.getByText("Set a goal below to track your week.")
350 |     ).toBeInTheDocument();
351 |     // Each example label + target text renders.
352 |     expect(screen.getByText("Study Hours")).toBeInTheDocument();
353 |     expect(screen.getByText("5 hours")).toBeInTheDocument();
354 |     expect(screen.getByText("Sessions")).toBeInTheDocument();
355 |     expect(screen.getByText("3 sessions")).toBeInTheDocument();
356 |     expect(screen.getByText("Tasks")).toBeInTheDocument();
357 |     expect(screen.getByText("5 tasks")).toBeInTheDocument();
358 |     expect(
359 |       screen.getByRole("button", { name: /set goals/i })
360 |     ).toBeInTheDocument();
361 |   });
362 |
363 |   it("falls back to the plain empty state when no example goals are provided", () => {
364 |     render(
365 |       wrap(
366 |         <WeeklyGoalPanel
367 |           goals={[]}
368 |           progress={[]}
369 |           weekStartDate="2025-06-16"
370 |           onSave={vi.fn()}
371 |         />
372 |       )
373 |     );
374 |
375 |     expect(screen.getByText("No goals set for this week")).toBeInTheDocument();
376 |     expect(
377 |       screen.queryByText("Goal ideas to get started")
378 |     ).not.toBeInTheDocument();
379 |   });
380 |
381 |   it("does not show example goals once the student has set goals", () => {
382 |     const goal = makeGoal();
383 |     render(
384 |       wrap(
385 |         <WeeklyGoalPanel
386 |           goals={[goal]}
387 |           progress={[makeProgress(goal)]}
388 |           weekStartDate="2025-06-16"

```

```

389 |         onSave={vi.fn()}
390 |         exampleGoals={exampleGoals}
391 |         exampleGoalsCopy={exampleGoalsCopy}
392 |     />
393 | )
394 | );
395 |
396 | // Real progress is shown, not the example-goals guidance.
397 | expect(
398 |     screen.queryByText("Goal ideas to get started")
399 | ).not.toBeInTheDocument();
400 | expect(screen.getByText("Study Hours")).toBeInTheDocument();
401 | expect(screen.getByText("2.0 / 5h")).toBeInTheDocument();
402 | });
403 | });
404 |

```

FILE: [src/__tests__/unit/ploWeightValidation.test.ts](#)

TYPE: .ts | LINES: 1-96 | SHA256: afcb6a517975941063c046762de9c2ec761622de1d1f98d7a291519902c073c1

```

1 | import { describe, it, expect } from "vitest";
2 |
3 | /**
4 |  * PLO-ILO weight validation logic tests.
5 |  * Validates: Requirement 13.3 – warn if sum of weights < 0.5
6 |  *
7 |  * These test the pure business rules used in PLOForm's ILOMappingSection:
8 |  * - Total weight calculation from enabled mappings
9 |  * - Weight status classification (ok / under / over)
10 |  * - Weight clamping between 0 and 1
11 |  */
12 |
13 | interface ILOMappingEntry {
14 |     ilo_id: string;
15 |     weight: number;
16 |     enabled: boolean;
17 | }
18 |
19 | /** Mirrors the totalWeight calculation in ILOMappingSection */
20 | function calculateTotalWeight(mappings: ILOMappingEntry[]): number {
21 |     return mappings
22 |         .filter((m) => m.enabled)
23 |         .reduce((sum, m) => sum + m.weight, 0);
24 | }
25 |
26 | /** Mirrors the weightStatus derivation in ILOMappingSection */
27 | function getWeightStatus(totalWeight: number): "ok" | "under" | "over" {
28 |     if (totalWeight > 1.0) return "over";
29 |     if (totalWeight < 0.5) return "under";
30 |     return "ok";
31 | }
32 |
33 | /** Mirrors the clamping logic in handleWeightChange */
34 | function clampWeight(value: number): number {
35 |     return Math.min(1.0, Math.max(0, value));
36 | }
37 |
38 | describe("PLO-ILO Weight Validation", () => {
39 |     describe("calculateTotalWeight", () => {
40 |         it("sums only enabled mappings", () => {
41 |             const mappings: ILOMappingEntry[] = [
42 |                 { ilo_id: "1", weight: 0.3, enabled: true },
43 |                 { ilo_id: "2", weight: 0.4, enabled: false },
44 |                 { ilo_id: "3", weight: 0.2, enabled: true },
45 |             ];
46 |             expect(calculateTotalWeight(mappings)).toBeCloseTo(0.5);
47 |         });
48 |
49 |         it("returns 0 when no mappings are enabled", () => {
50 |             const mappings: ILOMappingEntry[] = [
51 |                 { ilo_id: "1", weight: 0.5, enabled: false },
52 |             ];
53 |             expect(calculateTotalWeight(mappings)).toBe(0);
54 |         });
55 |
56 |         it("returns 0 for empty mappings", () => {
57 |             expect(calculateTotalWeight([])).toBe(0);
58 |         });
59 |     });
60 |
61 |     describe("getWeightStatus", () => {
62 |         it("returns 'under' when total < 0.5", () => {
63 |             expect(getWeightStatus(0)).toBe("under");
64 |             expect(getWeightStatus(0.3)).toBe("under");
65 |             expect(getWeightStatus(0.49)).toBe("under");
66 |         });
67 |
68 |         it("returns 'ok' when total is between 0.5 and 1.0 inclusive", () => {
69 |             expect(getWeightStatus(0.5)).toBe("ok");
70 |             expect(getWeightStatus(0.75)).toBe("ok");
71 |             expect(getWeightStatus(1.0)).toBe("ok");
72 |         });
73 |
74 |         it("returns 'over' when total > 1.0", () => {
75 |             expect(getWeightStatus(1.01)).toBe("over");
76 |             expect(getWeightStatus(2.0)).toBe("over");
77 |         });
78 |     });
79 |
80 |     describe("clampWeight", () => {
81 |         it("clamps negative values to 0", () => {
82 |             expect(clampWeight(-0.5)).toBe(0);
83 |         });
84 |     });

```

```

85 |     it("clamps values above 1 to 1", () => {
86 |         expect(clampWeight(1.5)).toBe(1.0);
87 |     });
88 |
89 |     it("passes through valid values unchanged", () => {
90 |         expect(clampWeight(0.5)).toBe(0.5);
91 |         expect(clampWeight(0)).toBe(0);
92 |         expect(clampWeight(1.0)).toBe(1.0);
93 |     });
94 | });
95 | });
96 |

```

FILE: [src/__tests__/unit/pomodoroIndicator.test.tsx](#)

TYPE: .tsx | LINES: 1-73 | SHA256: b09c475c4e2469b34fadab9b299a9a5fa51581b46f83a1edd4bc50bbe6b764d

```

1 | // =====
2 | // Unit Tests - PomodoroIndicator
3 | // =====
4 |
5 | import { describe, it, expect } from "vitest";
6 | import { render, screen } from "@testing-library/react";
7 | import PomodoroIndicator from "@components/shared/PomodoroIndicator";
8 |
9 | describe("PomodoroIndicator", () => {
10 |     it('renders "Pomodoro 1 of 4" for first work interval', () => {
11 |         render(<PomodoroIndicator currentInterval={0} intervalType="work" />);
12 |         expect(screen.getByText("Pomodoro 1 of 4")).toBeTruthy();
13 |     });
14 |
15 |     it('renders "Break" label during break interval', () => {
16 |         render(<PomodoroIndicator currentInterval={1} intervalType="break" />);
17 |         expect(screen.getByText("Break")).toBeTruthy();
18 |     });
19 |
20 |     it('renders "Long Break" label during long break interval', () => {
21 |         render(<PomodoroIndicator currentInterval={7} intervalType="long_break" />);
22 |         expect(screen.getByText("Long Break")).toBeTruthy();
23 |     });
24 |
25 |     it('renders "Pomodoro 2 of 4" for second work interval', () => {
26 |         render(<PomodoroIndicator currentInterval={2} intervalType="work" />);
27 |         expect(screen.getByText("Pomodoro 2 of 4")).toBeTruthy();
28 |     });
29 |
30 |     it('renders "Pomodoro 3 of 4" for third work interval', () => {
31 |         render(<PomodoroIndicator currentInterval={4} intervalType="work" />);
32 |         expect(screen.getByText("Pomodoro 3 of 4")).toBeTruthy();
33 |     });
34 |
35 |     it('renders "Pomodoro 4 of 4" for fourth work interval', () => {
36 |         render(<PomodoroIndicator currentInterval={6} intervalType="work" />);
37 |         expect(screen.getByText("Pomodoro 4 of 4")).toBeTruthy();
38 |     });
39 |
40 |     it("renders 4 progress dots", () => {
41 |         const { container } = render(
42 |             <PomodoroIndicator currentInterval={0} intervalType="work" />
43 |         );
44 |         const dots = container.querySelectorAll('[aria-hidden="true"] > div');
45 |         expect(dots.length).toBe(4);
46 |     });
47 |
48 |     it("has ARIA status role with descriptive label", () => {
49 |         render(<PomodoroIndicator currentInterval={0} intervalType="work" />);
50 |         const status = screen.getByRole("status");
51 |         expect(status).toBeTruthy();
52 |         expect(status.getAttribute("aria-label")).toBe("Pomodoro 1 of 4, Work");
53 |     });
54 |
55 |     it("shows correct ARIA label during break", () => {
56 |         render(<PomodoroIndicator currentInterval={1} intervalType="break" />);
57 |         const status = screen.getByRole("status");
58 |         expect(status.getAttribute("aria-label")).toBe("Pomodoro 1 of 4, Break");
59 |     });
60 |
61 |     it("applies custom className", () => {
62 |         const { container } = render(
63 |             <PomodoroIndicator
64 |                 currentInterval={0}
65 |                 intervalType="work"
66 |                 className="mt-4"
67 |             />
68 |         );
69 |         const root = container.firstChild;
70 |         expect(root?.className).toContain("mt-4");
71 |     });
72 | });
73 |

```

FILE: [src/__tests__/unit/portfolioAccess.test.ts](#)

TYPE: .ts | LINES: 1-167 | SHA256: 1a75d87eec3de18bef8c2bb109af2e70dc2248a3735f37b9b7c6a3d84e70d07a

```

1 | // =====
2 | // portfolioAccess - Unit tests (Task 24.4)
3 | // =====
4 | // Example-based coverage for the pure public-portfolio access logic that backs
5 | // the 'PublicPortfolio' route (see src/lib/portfolioAccess.ts). These tests
6 | // complement the exhaustive property test (portfolioAccess.property.test.ts) by
7 | // pinning the concrete, human-readable cases the spec calls out:
8 | //
9 | // - the discriminator distinguishes "exists but unauthorized" from "missing"
10 | // - the route mapping sends forbidden !' 403 and not_found !' 404 *distinctly*
11 | // so a guessable /portfolio/{id} URL cannot leak existence beyond the

```

```

12 //      intended forbidden/not-found split
13 //      - a malformed/unexpected RPC payload fails safe to forbidden (403), never
14 //      rendering protected content
15 //
16 // Validates: Requirements 24.3, 24.3a
17 // (R24.2 sharing gate is covered by studentPortfolio.test.tsx +
18 // usePortfolioSharing.test.tsx; R22.6 wording fallback by
19 // localizationFallback.test.ts – not duplicated here.)
20
21 import { describe, it, expect } from "vitest";
22 import {
23   evaluatePortfolioAccess,
24   mapPortfolioAccessToRoute,
25   parsePortfolioAccessStatus,
26   type PortfolioAccessStatus,
27   type PortfolioRouteOutcome,
28 } from "@lib/portfolioAccess";
29
30 // % % evaluatePortfolioAccess (discriminator model, R24.3) % % % % % % % % % % % % % % % %
31
32 describe("evaluatePortfolioAccess", () => {
33   it("authorizes only when the profile exists, is public, AND sharing is permitted", () => {
34     const status: PortfolioAccessStatus = evaluatePortfolioAccess({
35       profileExists: true,
36       portfolioPublic: true,
37       sharingPermitted: true,
38     });
39     expect(status).toBe("authorized");
40   });
41
42   it("forbids an existing public portfolio when sharing is not permitted (R24.3a)", () => {
43     // The decisive minor-safety case: public flag set but the school has NOT
44     // granted permission !' forbidden (403), never authorized and never 404.
45     const status: PortfolioAccessStatus = evaluatePortfolioAccess({
46       profileExists: true,
47       portfolioPublic: true,
48       sharingPermitted: false,
49     });
50     expect(status).toBe("forbidden");
51   });
52
53   it("forbids an existing permitted portfolio that is not marked public", () => {
54     const status: PortfolioAccessStatus = evaluatePortfolioAccess({
55       profileExists: true,
56       portfolioPublic: false,
57       sharingPermitted: true,
58     });
59     expect(status).toBe("forbidden");
60   });
61
62   it("forbids an existing portfolio that is neither public nor permitted", () => {
63     const status: PortfolioAccessStatus = evaluatePortfolioAccess({
64       profileExists: true,
65       portfolioPublic: false,
66       sharingPermitted: false,
67     });
68     expect(status).toBe("forbidden");
69   });
70
71   it("reports not_found for a missing profile regardless of the other flags", () => {
72     const flagCombinations: ReadonlyArray<readonly [boolean, boolean]> = [
73       [false, false],
74       [false, true],
75       [true, false],
76       [true, true],
77     ];
78     for (const [portfolioPublic, sharingPermitted] of flagCombinations) {
79       const status: PortfolioAccessStatus = evaluatePortfolioAccess({
80         profileExists: false,
81         portfolioPublic,
82         sharingPermitted,
83       });
84       expect(status).toBe("not_found");
85     }
86   });
87 });
88
89 // % % mapPortfolioAccessToRoute (status !' HTTP outcome, R24.3/24.3a) % % % % % % % % % % % %
90
91 describe("mapPortfolioAccessToRoute", () => {
92   it("renders an authorized portfolio", () => {
93     const outcome: PortfolioRouteOutcome =
94       mapPortfolioAccessToRoute("authorized");
95     expect(outcome).toEqual({ kind: "render" });
96   });
97
98   it("maps forbidden !' 403 Forbidden (not 404)", () => {
99     const outcome: PortfolioRouteOutcome =
100       mapPortfolioAccessToRoute("forbidden");
101     expect(outcome).toEqual({ kind: "error", status: 403 });
102   });
103
104   it("maps not_found !' 404 Not Found (not 403)", () => {
105     const outcome: PortfolioRouteOutcome =
106       mapPortfolioAccessToRoute("not_found");
107     expect(outcome).toEqual({ kind: "error", status: 404 });
108   });
109
110   it("distinguishes forbidden from not_found by status code (R24.3a)", () => {
111     const forbidden = mapPortfolioAccessToRoute("forbidden");
112     const notFound = mapPortfolioAccessToRoute("not_found");
113
114     // Both are error outcomes...
115     expect(forbidden.kind).toBe("error");

```



```

116 expect(notFound.kind).toBe("error");
117
118 // ...but their status codes are different, so an existing-but-unauthorized
119 // portfolio is never reported as missing.
120 if (forbidden.kind === "error" && notFound.kind === "error") {
121   expect(forbidden.status).toBe(403);
122   expect(notFound.status).toBe(404);
123   expect(forbidden.status).not.toBe(notFound.status);
124 }
125 });
126
127 it("never returns a render outcome for either error status", () => {
128   expect(mapPortfolioAccessToRoute("forbidden").kind).not.toBe("render");
129   expect(mapPortfolioAccessToRoute("not_found").kind).not.toBe("render");
130 });
131 });
132
133 // % % parsePortfolioAccessStatus (network-boundary hardening) % % % % % % % % % % % % % % % %
134
135 describe("parsePortfolioAccessStatus", () => {
136   it("passes through each known status verbatim", () => {
137     expect(parsePortfolioAccessStatus("authorized")).toBe("authorized");
138     expect(parsePortfolioAccessStatus("forbidden")).toBe("forbidden");
139     expect(parsePortfolioAccessStatus("not_found")).toBe("not_found");
140   });
141
142   it("treats null/undefined as forbidden so a missing payload never renders", () => {
143     expect(parsePortfolioAccessStatus(null)).toBe("forbidden");
144     expect(parsePortfolioAccessStatus(undefined)).toBe("forbidden");
145   });
146
147   it("treats an unrecognized text value as forbidden (fail-safe to 403)", () => {
148     for (const raw of ["", "AUTHORIZED", "ok", "deny", "404", "unknown"]) {
149       const status = parsePortfolioAccessStatus(raw);
150       expect(status).toBe("forbidden");
151       // And the resulting route outcome is a 403, never a render.
152       expect(mapPortfolioAccessToRoute(status)).toEqual({
153         kind: "error",
154         status: 403,
155       });
156     }
157   });
158
159   it("round-trips a parsed unexpected value to a non-rendering route", () => {
160     // End-to-end: arbitrary RPC text !' conservative status !' never renders.
161     const outcome = mapPortfolioAccessToRoute(
162       parsePortfolioAccessStatus("totally-unexpected")
163     );
164     expect(outcome.kind).toBe("error");
165   });
166 });
167

```

FILE: src/__tests__/unit/portfolioSharing.test.ts

```
TYPE: .ts | LINES: 1-220 | SHA256: 61c21a3cdc75f5c22dd5da4a07f957f252e1e58cbac7c868eb178429a333f5cc
```

[illegible]

```

49 |   },
50 |   }));
51 |
52 | const createWrapper = (queryClient: QueryClient) => {
53 |   return ({ children }: { children: ReactNode }) =>
54 |     createElement(QueryClientProvider, { client: queryClient }, children);
55 | };
56 |
57 | describe("usePortfolioSharingPermission", () => {
58 |   let queryClient: QueryClient;
59 |
60 |   beforeEach(() => {
61 |     vi.clearAllMocks();
62 |     maybeSingleResult = { data: null, error: null };
63 |     queryClient = new QueryClient({
64 |       defaultOptions: { queries: { retry: false } },
65 |     });
66 |   });
67 |
68 |   it("returns the admin-granted permission flag when present", async () => {
69 |     maybeSingleResult = {
70 |       data: { portfolio_sharing_permitted: true },
71 |       error: null,
72 |     };
73 |
74 |     const { result } = renderHook(
75 |       () => usePortfolioSharingPermission("user-1"),
76 |       { wrapper: createWrapper(queryClient) }
77 |     );
78 |
79 |     await waitFor(() => expect(result.current.isSuccess).toBe(true));
80 |     expect(result.current.data).toBe(true);
81 |   });
82 |
83 |   it("defaults to false when the flag is absent", async () => {
84 |     maybeSingleResult = { data: {}, error: null };
85 |
86 |     const { result } = renderHook(
87 |       () => usePortfolioSharingPermission("user-2"),
88 |       { wrapper: createWrapper(queryClient) }
89 |     );
90 |
91 |     await waitFor(() => expect(result.current.isSuccess).toBe(true));
92 |     expect(result.current.data).toBe(false);
93 |   });
94 |
95 |   it("is disabled and yields false-equivalent when no userId is given", async () => {
96 |     const { result } = renderHook(
97 |       () => usePortfolioSharingPermission(undefined),
98 |       { wrapper: createWrapper(queryClient) }
99 |     );
100 |
101 |     // Query is disabled (enabled: !!userId) so it never fetches.
102 |     expect(result.current.fetchStatus).toBe("idle");
103 |     expect(result.current.data).toBeUndefined();
104 |   });
105 |
106 |   it("surfaces a read error", async () => {
107 |     maybeSingleResult = { data: null, error: new Error("rls denied") };
108 |
109 |     const { result } = renderHook(
110 |       () => usePortfolioSharingPermission("user-3"),
111 |       { wrapper: createWrapper(queryClient) }
112 |     );
113 |
114 |     await waitFor(() => expect(result.current.isError).toBe(true));
115 |   });
116 | });
117 |
118 | describe("useTogglePortfolioPublic", () => {
119 |   let queryClient: QueryClient;
120 |
121 |   beforeEach(() => {
122 |     vi.clearAllMocks();
123 |     maybeSingleResult = { data: null, error: null };
124 |     updateResult = { error: null };
125 |     queryClient = new QueryClient({
126 |       defaultOptions: { mutations: { retry: false } },
127 |     });
128 |   });
129 |
130 |   it("makes public sharing effective only when permission is granted", async () => {
131 |     maybeSingleResult = {
132 |       data: { portfolio_sharing_permitted: true },
133 |       error: null,
134 |     };
135 |
136 |     const { result } = renderHook(() => useTogglePortfolioPublic(), {
137 |       wrapper: createWrapper(queryClient),
138 |     });
139 |
140 |     result.current.mutate({ userId: "user-1", isPublic: true });
141 |
142 |     await waitFor(() => expect(result.current.isSuccess).toBe(true));
143 |     expect(updateSpy).toHaveBeenCalledWith({ portfolio_public: true });
144 |     expect(result.current.data).toEqual({ permitted: true, isPublic: true });
145 |   });
146 |
147 |   it("keeps the portfolio private when permission is absent (R24.2)", async () => {
148 |     maybeSingleResult = {
149 |       data: { portfolio_sharing_permitted: false },
150 |       error: null,
151 |     };
152 |

```

```

153 |     const { result } = renderHook(() => useTogglePortfolioPublic(), {
154 |       wrapper: createWrapper(queryClient),
155 |     });
156 |
157 |     result.current.mutate({ userId: "user-1", isPublic: true });
158 |
159 |     await waitFor(() => expect(result.current.isSuccess).toBe(true));
160 |     // The toggle MUST NOT take effect without permission: persisted false.
161 |     expect(updateSpy).toHaveBeenCalledWith({ portfolio_public: false });
162 |     expect(result.current.data).toEqual({ permitted: false, isPublic: false });
163 |   });
164 |
165 |   it("always honors disabling public sharing regardless of permission", async () => {
166 |     maybeSingleResult = {
167 |       data: { portfolio_sharing_permitted: true },
168 |       error: null,
169 |     };
170 |
171 |     const { result } = renderHook(() => useTogglePortfolioPublic(), {
172 |       wrapper: createWrapper(queryClient),
173 |     });
174 |
175 |     result.current.mutate({ userId: "user-1", isPublic: false });
176 |
177 |     await waitFor(() => expect(result.current.isSuccess).toBe(true));
178 |     expect(updateSpy).toHaveBeenCalledWith({ portfolio_public: false });
179 |     expect(result.current.data).toEqual({ permitted: true, isPublic: false });
180 |   });
181 |
182 |   it("invalidates profile and gamification queries on success", async () => {
183 |     maybeSingleResult = {
184 |       data: { portfolio_sharing_permitted: true },
185 |       error: null,
186 |     };
187 |     const invalidateSpy = vi.spyOn(queryClient, "invalidateQueries");
188 |
189 |     const { result } = renderHook(() => useTogglePortfolioPublic(), {
190 |       wrapper: createWrapper(queryClient),
191 |     });
192 |
193 |     result.current.mutate({ userId: "user-1", isPublic: true });
194 |
195 |     await waitFor(() => expect(result.current.isSuccess).toBe(true));
196 |     expect(invalidateSpy).toHaveBeenCalledWith({
197 |       queryKey: queryKeys.profiles.all,
198 |     });
199 |     expect(invalidateSpy).toHaveBeenCalledWith({
200 |       queryKey: queryKeys.studentGamification.lists(),
201 |     });
202 |   });
203 |
204 |   it("surfaces a write error", async () => {
205 |     maybeSingleResult = {
206 |       data: { portfolio_sharing_permitted: true },
207 |       error: null,
208 |     };
209 |     updateResult = { error: new Error("update denied") };
210 |
211 |     const { result } = renderHook(() => useTogglePortfolioPublic(), {
212 |       wrapper: createWrapper(queryClient),
213 |     });
214 |
215 |     result.current.mutate({ userId: "user-1", isPublic: true });
216 |
217 |     await waitFor(() => expect(result.current.isError).toBe(true));
218 |   });
219 | });
220 |

```

FILE: [src/_tests_/unit/postQuizReview.test.ts](#)

TYPE: .ts | LINES: 1-144 | SHA256: 5114ad631b7c55e1492c66cd0878def96e2b0c521b847b960827dc68f81d7baa

```

1 | // @vitest-environment happy-dom
2 | import { describe, it, expect } from "vitest";
3 | import {
4 |   computePerCLOScore,
5 |   detectCLODiscrepancy,
6 | } from "@lib/questionAnalytics";
7 |
8 | // % % computePerCLOScore - various CLO configurations % % % % % % % % % % % %
9 |
10 | describe("computePerCLOScore", () => {
11 |   it("single CLO with 5 questions (3 correct, 2 incorrect) !' 60%", () => {
12 |     const answers = [
13 |       { clo_id: "clo-1", is_correct: true },
14 |       { clo_id: "clo-1", is_correct: true },
15 |       { clo_id: "clo-1", is_correct: true },
16 |       { clo_id: "clo-1", is_correct: false },
17 |       { clo_id: "clo-1", is_correct: false },
18 |     ];
19 |     expect(computePerCLOScore(answers)).toEqual({ "clo-1": 60 });
20 |   });
21 |
22 |   it("two CLOs: CLO-A 2/2 correct = 100%, CLO-B 1/3 correct "H 33.33%", () => {
23 |     const answers = [
24 |       { clo_id: "CLO-A", is_correct: true },
25 |       { clo_id: "CLO-A", is_correct: true },
26 |       { clo_id: "CLO-B", is_correct: true },
27 |       { clo_id: "CLO-B", is_correct: false },
28 |       { clo_id: "CLO-B", is_correct: false },
29 |     ];
30 |     const result = computePerCLOScore(answers);
31 |     expect(result["CLO-A"]).toBe(100);
32 |     expect(result["CLO-B"]).toBeCloseTo(33.3333, 3);

```

```

33 | });
34 |
35 | it("three CLOs with varying question counts", () => {
36 |     const answers = [
37 |         { clo_id: "clo-x", is_correct: true },
38 |         { clo_id: "clo-x", is_correct: false },
39 |         { clo_id: "clo-x", is_correct: true },
40 |         { clo_id: "clo-x", is_correct: true },
41 |         { clo_id: "clo-y", is_correct: false },
42 |         { clo_id: "clo-y", is_correct: true },
43 |         { clo_id: "clo-z", is_correct: true },
44 |     ];
45 |     const result = computePerCLOScore(answers);
46 |     expect(result["clo-x"]).toBe(75);
47 |     expect(result["clo-y"]).toBe(50);
48 |     expect(result["clo-z"]).toBe(100);
49 | });
50 |
51 | it("all questions for all CLOs correct !' all 100%", () => {
52 |     const answers = [
53 |         { clo_id: "a", is_correct: true },
54 |         { clo_id: "a", is_correct: true },
55 |         { clo_id: "b", is_correct: true },
56 |         { clo_id: "b", is_correct: true },
57 |         { clo_id: "c", is_correct: true },
58 |     ];
59 |     const result = computePerCLOScore(answers);
60 |     expect(result["a"]).toBe(100);
61 |     expect(result["b"]).toBe(100);
62 |     expect(result["c"]).toBe(100);
63 | });
64 |
65 | it("all questions for all CLOs incorrect !' all 0%", () => {
66 |     const answers = [
67 |         { clo_id: "a", is_correct: false },
68 |         { clo_id: "a", is_correct: false },
69 |         { clo_id: "b", is_correct: false },
70 |         { clo_id: "b", is_correct: false },
71 |         { clo_id: "c", is_correct: false },
72 |     ];
73 |     const result = computePerCLOScore(answers);
74 |     expect(result["a"]).toBe(0);
75 |     expect(result["b"]).toBe(0);
76 |     expect(result["c"]).toBe(0);
77 | });
78 |
79 | it("one CLO with 1 question correct !' 100%", () => {
80 |     const answers = [{ clo_id: "solo", is_correct: true }];
81 |     expect(computePerCLOScore(answers)).toEqual({ solo: 100 });
82 | });
83 |
84 | it("one CLO with 1 question incorrect !' 0%", () => {
85 |     const answers = [{ clo_id: "solo", is_correct: false }];
86 |     expect(computePerCLOScore(answers)).toEqual({ solo: 0 });
87 | });
88 | });
89 |
90 | // % % detectCLODiscrepancy - integration scenarios % % % % % % % % % % % % % % % %
91 |
92 | describe("detectCLODiscrepancy", () => {
93 |     it("quiz 80%, CLO attainment 60% !' discrepancy (20 > 15)", () => {
94 |         expect(detectCLODiscrepancy(80, 60)).toBe(true);
95 |     });
96 |
97 |     it("quiz 70%, CLO attainment 75% !' no discrepancy (5 > 15)", () => {
98 |         expect(detectCLODiscrepancy(70, 75)).toBe(false);
99 |     });
100 |
101 |     it("quiz 90%, CLO attainment 50% !' discrepancy (40 > 15)", () => {
102 |         expect(detectCLODiscrepancy(90, 50)).toBe(true);
103 |     });
104 |
105 |     it("quiz 65%, CLO attainment 80% !' no discrepancy (|15| is not > 15)", () => {
106 |         expect(detectCLODiscrepancy(65, 80)).toBe(false);
107 |     });
108 | });
109 |
110 | // % % Combined scenario: per-CLO scores + discrepancy check % % % % % % % % % %
111 |
112 | describe("combined per-CLO score and discrepancy check", () => {
113 |     it("computes per-CLO scores then checks discrepancy against mock attainment", () => {
114 |         const answers = [
115 |             { clo_id: "clo-1", is_correct: true },
116 |             { clo_id: "clo-1", is_correct: true },
117 |             { clo_id: "clo-1", is_correct: false },
118 |             { clo_id: "clo-1", is_correct: false },
119 |             { clo_id: "clo-2", is_correct: true },
120 |             { clo_id: "clo-2", is_correct: true },
121 |             { clo_id: "clo-2", is_correct: true },
122 |         ];
123 |
124 |         const scores = computePerCLOScore(answers);
125 |         expect(scores["clo-1"]).toBe(50);
126 |         expect(scores["clo-2"]).toBeCloseTo(100, 0);
127 |
128 |         // Mock attainment data from outcome_attainment table
129 |         const attainment: Record<string, number> = {
130 |             "clo-1": 80,
131 |             "clo-2": 90,
132 |         };
133 |
134 |         // clo-1: |50 - 80| = 30 > 15 !' discrepancy
135 |         expect(detectCLODiscrepancy(scores["clo-1"], attainment["clo-1"])).toBe(
136 |             true

```

```

137 |     );
138 |     // clo-2: |100 " 90| = 10 "d 15 !' no discrepancy
139 |     expect(detectCLODiscrepancy(scores["clo-2"]!, attainment["clo-2"]!)).toBe(
140 |         false
141 |     );
142 |   });
143 | });
144 |

```

FILE: src/__tests__/unit/postQuizReviewPage.test.tsx

```
TYPE: .tsx | LINES: 1-431 | SHA256: 8f7f16b15bdf203174946ee658357c524f9e2ad39b66c8e0b53300ea59ff7a23
```

[illegible]

[illegible]

```

197 |
198 | it('shows "Get Help" link for incorrect answers', async () => {
199 |   render(<PostQuizReview />, { wrapper: createWrapper() });
200 |
201 |   await screen.findAllByText("75%");
202 |   // q-2 answer is 'A' but correct is 'True', so it's incorrect
203 |   // q-3 answer is 'C' but correct is 'QuickSort', so it's incorrect
204 |   const helpLinks = screen.getAllByText(/Get Help/);
205 |   expect(helpLinks.length).toBeGreaterThan(0);
206 | });
207 |
208 | it("shows correct/incorrect indicators per question", async () => {
209 |   render(<PostQuizReview />, { wrapper: createWrapper() });
210 |
211 |   await screen.findAllByText("75%");
212 |   // q-1: answer 'B' === correct 'B' !' Correct
213 |   // q-2: answer 'A' !== correct 'True' !' Incorrect
214 |   // q-3: answer 'C' !== correct 'QuickSort' !' Incorrect
215 |   expect(screen.getAllByText("Correct").length).toBeGreaterThan(0);
216 |   expect(screen.getAllByText("Incorrect").length).toBe(2);
217 | });
218 |
219 | it("displays per-CLO score breakdown section", async () => {
220 |   render(<PostQuizReview />, { wrapper: createWrapper() });
221 |
222 |   await screen.findAllByText("75%");
223 |   expect(screen.getByText("Per-CLO Score Breakdown")).toBeInTheDocument();
224 |   // clo-1: q-1 correct, q-2 incorrect !' 50%
225 |   // clo-2: q-3 incorrect !' 0%
226 |   expect(screen.getByText("50%")).toBeInTheDocument();
227 |   expect(screen.getByText("0%")).toBeInTheDocument();
228 | });
229 |
230 | it('"Get Help" links are scoped to the question CLO', async () => {
231 |   render(<PostQuizReview />, { wrapper: createWrapper() });
232 |
233 |   await screen.findAllByText("75%");
234 |   const helpLinks = screen.getAllByText(/Get Help/);
235 |   // q-2 (clo-1) and q-3 (clo-2) are incorrect
236 |   const hrefs = helpLinks.map((link) =>
237 |     link.closest("a")?.getAttribute("href")
238 |   );
239 |   expect(hrefs).toContain("/student/tutor?cloIds=clo-1");
240 |   expect(hrefs).toContain("/student/tutor?cloIds=clo-2");
241 | });
242 |
243 | it('labels AI explanations with "AI Explanation" heading', async () => {
244 |   render(<PostQuizReview />, { wrapper: createWrapper() });
245 |
246 |   await screen.findAllByText("75%");
247 |   // Two questions have explanations (q-1 and q-2), q-3 has null
248 |   const labels = screen.getAllByText("AI Explanation");
249 |   expect(labels.length).toBe(2);
250 | });
251 |
252 | it('does not show "Get Help" link for correct answers', async () => {
253 |   render(<PostQuizReview />, { wrapper: createWrapper() });
254 |
255 |   await screen.findAllByText("75%");
256 |   // q-1 is correct, so only 2 help links (for q-2 and q-3)
257 |   const helpLinks = screen.getAllByText(/Get Help/);
258 |   expect(helpLinks.length).toBe(2);
259 | });
260 |
261 | it('renders "Back to Dashboard" button', async () => {
262 |   render(<PostQuizReview />, { wrapper: createWrapper() });
263 |
264 |   await screen.findAllByText("75%");
265 |   expect(
266 |     screen.getByRole("link", { name: /Back to Dashboard/i })
267 |   ).toBeInTheDocument();
268 | });
269 |
270 | // % % % Task 15.8: Explanation confidence badge and verified explanation % % % % %
271 |
272 | it('displays "Verified Explanation" label when verified explanation exists', async () => {
273 |   mockUseVerifiedExplanation.mockReturnValue({
274 |     data: {
275 |       explanation_text: "Teacher-verified OOP explanation.",
276 |       is_active: true,
277 |     },
278 |     isLoading: false,
279 |   });
280 |   mockUseExplanationConfidence.mockReturnValue({
281 |     data: 0.9,
282 |     isLoading: false,
283 |   });
284 |
285 |   render(<PostQuizReview />, { wrapper: createWrapper() });
286 |
287 |   await screen.findAllByText("75%");
288 |   expect(screen.getAllByText("Verified Explanation").length).toBeGreaterThan(
289 |     0
290 |   );
291 | });
292 |
293 | it("prefers verified explanation text over AI-generated explanation", async () => {
294 |   mockUseVerifiedExplanation.mockImplementation((questionId: string) => {
295 |     if (questionId === "q-1") {
296 |       return {
297 |         data: {
298 |           explanation_text: "Teacher-verified OOP explanation.",
299 |           is_active: true,
300 |         },

```

```

301         isLoading: false,
302     };
303 }
304 return { data: null, isLoading: false };
305 });
306 mockUseExplanationConfidence.mockReturnValue({
307     data: 0.85,
308     isLoading: false,
309 });
310
311 render(<PostQuizReview />, { wrapper: createWrapper() });
312
313 await screen.findAllByText("75%");
314 // Verified text should appear instead of AI text for q-1
315 expect(
316     screen.getByText("Teacher-verified OOP explanation.")
317 ).toBeInTheDocument();
318 // Original AI explanation for q-1 should NOT appear
319 expect(
320     screen.queryByText("OOP is a programming paradigm based on objects.")
321 ).not.toBeInTheDocument();
322 });
323
324 it('displays ExplanationConfidenceBadge with "Teacher verified" for verified explanations', async () => {
325     mockUseVerifiedExplanation.mockReturnValue({
326         data: { explanation_text: "Verified text.", is_active: true },
327         isLoading: false,
328     });
329     mockUseExplanationConfidence.mockReturnValue({
330         data: 0.9,
331         isLoading: false,
332     });
333
334     render(<PostQuizReview />, { wrapper: createWrapper() });
335
336     await screen.findAllByText("75%");
337     expect(screen.getAllByText("Teacher verified").length).toBeGreaterThan(0);
338 });
339
340 it("displays confidence badge for unverified AI explanations with high confidence", async () => {
341     mockUseVerifiedExplanation.mockReturnValue({
342         data: null,
343         isLoading: false,
344     });
345     mockUseExplanationConfidence.mockReturnValue({
346         data: 0.85,
347         isLoading: false,
348     });
349
350     render(<PostQuizReview />, { wrapper: createWrapper() });
351
352     await screen.findAllByText("75%");
353     // High confidence (>= 0.8) shows "Verified by course materials"
354     expect(
355         screen.getAllByText("Verified by course materials").length
356     ).toBeGreaterThan(0);
357 });
358
359 it("displays amber confidence badge for low confidence AI explanations", async () => {
360     mockUseVerifiedExplanation.mockReturnValue({
361         data: null,
362         isLoading: false,
363     });
364     mockUseExplanationConfidence.mockReturnValue({
365         data: 0.5,
366         isLoading: false,
367     });
368
369     render(<PostQuizReview />, { wrapper: createWrapper() });
370
371     await screen.findAllByText("75%");
372     expect(
373         screen.getAllByText("This explanation may need teacher verification")
374             .length
375     ).toBeGreaterThan(0);
376 });
377
378 // % % % Task 17.10: Bloom's Progression Ladder in PostQuizReview % % % % % % % % %
379
380 it("displays Bloom's Progression section when climb state is available", async () => {
381     mockUseBloomsClimbState.mockReturnValue({
382         data: {
383             current_level: 3,
384             consecutive_correct: 1,
385             transitions: [],
386             highest_level_reached: 3,
387         },
388         isLoading: false,
389     });
390
391     render(<PostQuizReview />, { wrapper: createWrapper() });
392
393     await screen.findAllByText("75%");
394     expect(screen.getByText("Bloom's Progression")).toBeInTheDocument();
395 });
396
397 it("does not display Bloom's Progression section when climb state is null", async () => {
398     mockUseBloomsClimbState.mockReturnValue({ data: null, isLoading: false });
399
400     render(<PostQuizReview />, { wrapper: createWrapper() });
401
402     await screen.findAllByText("75%");
403     expect(screen.getByText("Bloom's Progression")).toBeInTheDocument();
404     expect(

```



```

405     screen.getByText("No Bloom's progression data was recorded.")
406   ).toBeInTheDocument();
407 });
408
409 it("renders a BloomsProgressionLadder for each unique CLO in the quiz", async () => {
410   mockUseBloomsClimbState.mockReturnValue({
411     data: {
412       current_level: 4,
413       consecutive_correct: 2,
414       transitions: [],
415       highest_level_reached: 4,
416     },
417     isLoading: false,
418   });
419
420   render(<PostQuizReview />, { wrapper: createWrapper() });
421
422   await screen.findAllByText("75%");
423   // The quiz has 2 unique CLOs: clo-1 and clo-2
424   // Each ladder has an aria-label containing the CLO title
425   const ladders = screen.getAllByRole("img", {
426     name: /Bloom's progression ladder/i,
427   });
428   expect(ladders.length).toBe(2);
429 });
430 });
431

```

FILE: src/__tests__/unit/practiceMode.test.ts

```
TYPE: .ts | LINES: 1-100 | SHA256: 9433f04ff2a7fbbaefcf6f237d580a7e6cf0d45b47392db877828bbbb2348ab2
```

```

74 |     scores.forEach(() => {
75 |         expect(computePracticeXP()).toBe(10);
76 |     });
77 | });
78 | });
79 |
80 | // % % No evidence generation in practice mode % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
81 |
82 | describe("Practice mode skips evidence generation", () => {
83 |     it("practice mode attempts should not trigger attainment rollup", () => {
84 |         // This validates the design contract: when mode = 'practice',
85 |         // the submission flow skips evidence generation and attainment rollup.
86 |         const shouldSkipEvidence = (mode: string): boolean => mode === "practice";
87 |
88 |         expect(shouldSkipEvidence("practice")).toBe(true);
89 |         expect(shouldSkipEvidence("graded")).toBe(false);
90 |     });
91 |
92 |     it("graded mode attempts should trigger attainment rollup", () => {
93 |         const shouldGenerateEvidence = (mode: string): boolean =>
94 |             mode !== "practice";
95 |
96 |         expect(shouldGenerateEvidence("graded")).toBe(true);
97 |         expect(shouldGenerateEvidence("practice")).toBe(false);
98 |     });
99 | });
100 |

```

FILE: src/__tests__/unit/practiceModeBanner.test.tsx

TYPE: .tsx | LINES: 1-92 | SHA256: acd92d481c68e1dc99643da7468be1b61b6f3426e3bf799a43da828febdaad29

```

74 |
75 | it('has role="status" for accessibility', () => {
76 |   render(<PracticeModeBanner />);
77 |   expect(screen.getByRole("status")).toBeInTheDocument();
78 | });
79 |
80 | it("has aria-label for accessibility", () => {
81 |   render(<PracticeModeBanner />);
82 |   const banner = screen.getByRole("status");
83 |   expect(banner).toHaveAttribute("aria-label", "Practice Mode");
84 | });
85 |
86 | it("renders an info icon", () => {
87 |   const { container } = render(<PracticeModeBanner />);
88 |   const svg = container.querySelector("svg");
89 |   expect(svg).toBeInTheDocument();
90 | });
91 | });
92 |

```

FILE: src/___tests___/unit/predictionValidator.test.ts

TYPE: .ts | LINES: 1-272 | SHA256: d377dd7ff01871c5f92b1f4412ca4f1190adfa21e52b8bce65aeb5ca6686a806

```

1 | import { describe, it, expect, vi, beforeEach } from "vitest";
2 |
3 | // -----
4 | // Supabase mock – fine-grained control per table + operation
5 | // -----
6 |
7 | interface MockQueryResult {
8 |   data: unknown;
9 |   error: unknown;
10 | }
11 |
12 | let aiFeedbackSelectResult: MockQueryResult;
13 | let aiFeedbackUpdateResult: MockQueryResult;
14 | let outcomeAttainmentSelectResult: MockQueryResult;
15 |
16 | // Track calls for assertions
17 | const aiFeedbackSelectCalls: Array<{ method: string; args: unknown[] }> = [];
18 | const aiFeedbackUpdateCalls: Array<{ method: string; args: unknown[] }> = [];
19 | const outcomeAttainmentSelectCalls: Array<{ method: string; args: unknown[] }> =
20 |   [];
21 |
22 | function createSelectChain(
23 |   resultRef: () => MockQueryResult,
24 |   tracker: typeof aiFeedbackSelectCalls
25 | ) {
26 |   const chain: Record<string, (...args: unknown[]) => unknown> = {};
27 |   const proxy = new Proxy(chain, {
28 |     get(_target, prop: string) {
29 |       if (prop === "then" || prop === "catch") return undefined;
30 |       if (prop === "data") return resultRef().data;
31 |       if (prop === "error") return resultRef().error;
32 |       return (...args: unknown[]) => {
33 |         tracker.push({ method: prop, args });
34 |         return proxy;
35 |       };
36 |     },
37 |   });
38 |   return proxy;
39 | }
40 |
41 | function createUpdateChain(
42 |   resultRef: () => MockQueryResult,
43 |   tracker: typeof aiFeedbackUpdateCalls
44 | ) {
45 |   const chain: Record<string, (...args: unknown[]) => unknown> = {};
46 |   const proxy = new Proxy(chain, {
47 |     get(_target, prop: string) {
48 |       if (prop === "then" || prop === "catch") return undefined;
49 |       if (prop === "data") return resultRef().data;
50 |       if (prop === "error") return resultRef().error;
51 |       return (...args: unknown[]) => {
52 |         tracker.push({ method: prop, args });
53 |         return proxy;
54 |       };
55 |     },
56 |   });
57 |   return proxy;
58 | }
59 |
60 | vi.mock("@/lib/supabase", () => ({
61 |   supabase: {
62 |     from: (table: string) => {
63 |       if (table === "ai_feedback") {
64 |         return {
65 |           select: (...args: unknown[]) => {
66 |             aiFeedbackSelectCalls.push({ method: "select", args });
67 |             return createSelectChain(
68 |               () => aiFeedbackSelectResult,
69 |               aiFeedbackSelectCalls
70 |             );
71 |           },
72 |           update: (...args: unknown[]) => {
73 |             aiFeedbackUpdateCalls.push({ method: "update", args });
74 |             return createUpdateChain(
75 |               () => aiFeedbackUpdateResult,
76 |               aiFeedbackUpdateCalls
77 |             );
78 |           },
79 |         };
80 |       }
81 |       if (table === "outcome_attainment") {

```

```

82 |         return {
83 |             select: (...args: unknown[]) => {
84 |                 outcomeAttainmentSelectCalls.push({ method: "select", args });
85 |                 return createSelectChain(
86 |                     () => outcomeAttainmentSelectResult,
87 |                     outcomeAttainmentSelectCalls
88 |                 );
89 |             },
90 |         };
91 |     }
92 |     return createSelectChain(() => ({ data: null, error: null }), []);
93 | },
94 | });
95 | });
96 |
97 | // -----
98 | // Import after mocks
99 | // -----
100 |
101 | import { validateAtRiskPredictions } from "@lib/predictionValidator";
102 |
103 | // -----
104 | // Tests
105 | // -----
106 |
107 | describe("validateAtRiskPredictions", () => {
108 |     beforeEach(() => {
109 |         vi.clearAllMocks();
110 |         aiFeedbackSelectCalls.length = 0;
111 |         aiFeedbackUpdateCalls.length = 0;
112 |         outcomeAttainmentSelectCalls.length = 0;
113 |
114 |         aiFeedbackSelectResult = { data: [], error: null };
115 |         aiFeedbackUpdateResult = { data: null, error: null };
116 |         outcomeAttainmentSelectResult = { data: [], error: null };
117 |     });
118 |
119 |     it("returns early with zero updates when studentId is empty", async () => {
120 |         const result = await validateAtRiskPredictions("", ["clo-1"]);
121 |         expect(result.updatedCount).toBe(0);
122 |         expect(result.errors).toHaveLength(0);
123 |     });
124 |
125 |     it("returns early with zero updates when cloIds is empty", async () => {
126 |         const result = await validateAtRiskPredictions("student-1", []);
127 |         expect(result.updatedCount).toBe(0);
128 |         expect(result.errors).toHaveLength(0);
129 |     });
130 |
131 |     it("returns zero updates when no predictions exist", async () => {
132 |         aiFeedbackSelectResult = { data: [], error: null };
133 |
134 |         const result = await validateAtRiskPredictions("student-1", ["clo-1"]);
135 |         expect(result.updatedCount).toBe(0);
136 |         expect(result.errors).toHaveLength(0);
137 |     });
138 |
139 |     it("returns zero updates when predictions exist but none match the CLO IDs", async () => {
140 |         aiFeedbackSelectResult = {
141 |             data: [{ id: "pred-1", suggestion_data: { at_risk_clo_id: "clo-99" } }],
142 |             error: null,
143 |         };
144 |
145 |         const result = await validateAtRiskPredictions("student-1", ["clo-1"]);
146 |         expect(result.updatedCount).toBe(0);
147 |     });
148 |
149 |     it('marks prediction as "correct" when attainment is below 70%', async () => {
150 |         aiFeedbackSelectResult = {
151 |             data: [{ id: "pred-1", suggestion_data: { at_risk_clo_id: "clo-1" } }],
152 |             error: null,
153 |         };
154 |         outcomeAttainmentSelectResult = {
155 |             data: [{ outcome_id: "clo-1", attainment_percent: 55 }],
156 |             error: null,
157 |         };
158 |         aiFeedbackUpdateResult = { data: null, error: null };
159 |
160 |         const result = await validateAtRiskPredictions("student-1", ["clo-1"]);
161 |         expect(result.updatedCount).toBe(1);
162 |         expect(result.errors).toHaveLength(0);
163 |
164 |         // Verify the update was called with 'correct'
165 |         const updateCall = aiFeedbackUpdateCalls.find((c) => c.method === "update");
166 |         expect(updateCall).toBeDefined();
167 |         expect(updateCall!.args[0]).toEqual({ validated_outcome: "correct" });
168 |     });
169 |
170 |     it('marks prediction as "incorrect" when attainment is >= 70%', async () => {
171 |         aiFeedbackSelectResult = {
172 |             data: [{ id: "pred-1", suggestion_data: { at_risk_clo_id: "clo-1" } }],
173 |             error: null,
174 |         };
175 |         outcomeAttainmentSelectResult = {
176 |             data: [{ outcome_id: "clo-1", attainment_percent: 85 }],
177 |             error: null,
178 |         };
179 |         aiFeedbackUpdateResult = { data: null, error: null };
180 |
181 |         const result = await validateAtRiskPredictions("student-1", ["clo-1"]);
182 |         expect(result.updatedCount).toBe(1);
183 |
184 |         const updateCall = aiFeedbackUpdateCalls.find((c) => c.method === "update");
185 |         expect(updateCall).toBeDefined();

```

```

186 expect(updateCall!.args[0]).toEqual({ validated_outcome: "incorrect" });
187 });
188
189 it('marks prediction as "correct" when no attainment record exists', async () => {
190   aiFeedbackSelectResult = {
191     data: [{ id: "pred-1", suggestion_data: { at_risk_clo_id: "clo-1" } }],
192     error: null,
193   };
194   outcomeAttainmentSelectResult = { data: [], error: null };
195   aiFeedbackUpdateResult = { data: null, error: null };
196
197   const result = await validateAtRiskPredictions("student-1", ["clo-1"]);
198   expect(result.updatedCount).toBe(1);
199
200   const updateCall = aiFeedbackUpdateCalls.find((c) => c.method === "update");
201   expect(updateCall!.args[0]).toEqual({ validated_outcome: "correct" });
202 });
203
204 it('marks prediction as "incorrect" at exactly 70% threshold', async () => {
205   aiFeedbackSelectResult = {
206     data: [{ id: "pred-1", suggestion_data: { at_risk_clo_id: "clo-1" } }],
207     error: null,
208   };
209   outcomeAttainmentSelectResult = {
210     data: [{ outcome_id: "clo-1", attainment_percent: 70 }],
211     error: null,
212   };
213   aiFeedbackUpdateResult = { data: null, error: null };
214
215   const result = await validateAtRiskPredictions("student-1", ["clo-1"]);
216   expect(result.updatedCount).toBe(1);
217
218   const updateCall = aiFeedbackUpdateCalls.find((c) => c.method === "update");
219   expect(updateCall!.args[0]).toEqual({ validated_outcome: "incorrect" });
220 });
221
222 it("collects error when ai_feedback select fails", async () => {
223   aiFeedbackSelectResult = { data: null, error: { message: "DB error" } };
224
225   const result = await validateAtRiskPredictions("student-1", ["clo-1"]);
226   expect(result.updatedCount).toBe(0);
227   expect(result.errors).toHaveLength(1);
228   expect(result.errors[0]).toContain("Failed to fetch predictions");
229 });
230
231 it("collects error when outcome_attainment select fails", async () => {
232   aiFeedbackSelectResult = {
233     data: [{ id: "pred-1", suggestion_data: { at_risk_clo_id: "clo-1" } }],
234     error: null,
235   };
236   outcomeAttainmentSelectResult = {
237     data: null,
238     error: { message: "Attainment error" },
239   };
240
241   const result = await validateAtRiskPredictions("student-1", ["clo-1"]);
242   expect(result.updatedCount).toBe(0);
243   expect(result.errors).toHaveLength(1);
244   expect(result.errors[0]).toContain("Failed to fetch attainment");
245 });
246
247 it("handles multiple predictions for different CLOs", async () => {
248   aiFeedbackSelectResult = {
249     data: [
250       { id: "pred-1", suggestion_data: { at_risk_clo_id: "clo-1" } },
251       { id: "pred-2", suggestion_data: { at_risk_clo_id: "clo-2" } },
252     ],
253     error: null,
254   };
255   outcomeAttainmentSelectResult = {
256     data: [
257       { outcome_id: "clo-1", attainment_percent: 45 },
258       { outcome_id: "clo-2", attainment_percent: 80 },
259     ],
260     error: null,
261   };
262   aiFeedbackUpdateResult = { data: null, error: null };
263
264   const result = await validateAtRiskPredictions("student-1", [
265     "clo-1",
266     "clo-2",
267   ]);
268   expect(result.updatedCount).toBe(2);
269   expect(result.errors).toHaveLength(0);
270 });
271 });
272

```

FILE: src/__tests__/unit/primaryCTA.test.tsx

TYPE: .tsx | LINES: 1-369 | SHA256: 90ec43d360a9bfc52833be1597fba026cddababa99629fb277f44e3828356d36


```

222 | });
223 |
224 | it("excludes inapplicable higher-priority candidates so the next applicable one is dominant (R16.2)", () => {
225 |     // Profile no longer applies; submit (next precedence) becomes dominant even
226 |     // though a higher-priority candidate exists in the set.
227 |     renderDashboardCTA({ "complete-profile": false });
228 |     expect(
229 |         screen.getByTestId("primary-cta-primary-submit-assignment")
230 |     ).toBeTruthy();
231 |     expect(
232 |         screen.queryByTestId("primary-cta-primary-complete-profile")
233 |     ).toBeNull();
234 |     // The inapplicable candidate is not surfaced anywhere (not in secondaries).
235 |     expect(screen.queryByText("Complete Profile")).toBeNull();
236 | });
237 |
238 | it("promotes each next-highest candidate as the dominant one completes, in order (R16.3)", () => {
239 |     const { rerender } = render(
240 |         <Providers>
241 |         <PrimaryCTA actions={dashboardActions()} />
242 |         </Providers>
243 |     );
244 |     // Start: profile dominant.
245 |     expect(
246 |         screen.getByTestId("primary-cta-primary-complete-profile")
247 |     ).toBeTruthy();
248 |
249 |     // Complete profile !' submit promoted.
250 |     rerender(
251 |         <Providers>
252 |         <PrimaryCTA actions={dashboardActions({ "complete-profile": false })} />
253 |         </Providers>
254 |     );
255 |     expect(
256 |         screen.getByTestId("primary-cta-primary-submit-assignment")
257 |     ).toBeTruthy();
258 |
259 |     // Complete submit too !' continue promoted.
260 |     rerender(
261 |         <Providers>
262 |         <PrimaryCTA
263 |             actions={dashboardActions({
264 |                 "complete-profile": false,
265 |                 "submit-assignment": false,
266 |             })}
267 |         />
268 |         </Providers>
269 |     );
270 |     expect(
271 |         screen.getByTestId("primary-cta-primary-continue-course")
272 |     ).toBeTruthy();
273 |
274 |     // Complete continue too !' review promoted (the last applicable candidate).
275 |     rerender(
276 |         <Providers>
277 |         <PrimaryCTA
278 |             actions={dashboardActions({
279 |                 "complete-profile": false,
280 |                 "submit-assignment": false,
281 |                 "continue-course": false,
282 |             })}
283 |         />
284 |         </Providers>
285 |     );
286 |     expect(
287 |         screen.getByTestId("primary-cta-primary-review-feedback")
288 |     ).toBeTruthy();
289 | });
290 |
291 | it("renders nothing once the last applicable candidate is completed (R16.1, R16.3)", () => {
292 |     const { container } = renderDashboardCTA({
293 |         "complete-profile": false,
294 |         "submit-assignment": false,
295 |         "continue-course": false,
296 |         "review-feedback": false,
297 |     });
298 |     expect(container.firstChild).toBeNull();
299 | });
300 |
301 | it("removes the completed candidate entirely rather than demoting it to a secondary (R16.3)", () => {
302 |     const { rerender } = render(
303 |         <Providers>
304 |         <PrimaryCTA actions={dashboardActions()} />
305 |         </Providers>
306 |     );
307 |     // Profile is dominant; submit/continue/review are secondaries.
308 |     expect(
309 |         screen.getByTestId("primary-cta-primary-complete-profile")
310 |     ).toBeTruthy();
311 |     let secondaryRow = screen.getByTestId("primary-cta-secondaries");
312 |     expect(
313 |         within(secondaryRow)
314 |             .getAllByRole("button")
315 |             .map((b) => b.textContent)
316 |     ).toEqual(["Submit Assignment", "Continue Course", "Review Feedback"]);
317 |
318 |     // Completing profile promotes submit and drops profile from the UI entirely.
319 |     rerender(
320 |         <Providers>
321 |         <PrimaryCTA actions={dashboardActions({ "complete-profile": false })} />
322 |         </Providers>
323 |     );
324 |     expect(
325 |         screen.getByTestId("primary-cta-primary-submit-assignment")

```



```

326     ).toBeTruthy();
327     expect(screen.getByText("Complete Profile")).toBeNull();
328     secondaryRow = screen.getByTestId("primary-cta-secondaries");
329     expect(
330       within(secondaryRow)
331         .getAllByRole("button")
332         .map((b) => b.textContent)
333     ).toEqual(["Continue Course", "Review Feedback"]);
334   });
335
336   it("reclaims dominance for a higher-priority candidate when it becomes applicable again (R16.2, R16.3)", () {
337     // Start with profile inapplicable: submit is dominant, profile is absent.
338     const { rerender } = render(
339       <Providers>
340         <PrimaryCTA actions={dashboardActions({ "complete-profile": false })} />
341       </Providers>
342     );
343     expect(
344       screen.getByTestId("primary-cta-primary-submit-assignment")
345     ).toBeTruthy();
346     expect(screen.getByText("Complete Profile")).toBeNull();
347
348     // Profile becomes applicable again !' it reclaims the dominant spot and the
349     // previously-dominant submit is demoted into the subordinate row.
350     rerender(
351       <Providers>
352         <PrimaryCTA actions={dashboardActions({})} />
353       </Providers>
354     );
355     expect(
356       screen.getByTestId("primary-cta-primary-complete-profile")
357     ).toBeTruthy();
358     expect(
359       screen.getByTestId("primary-cta-primary-submit-assignment")
360     ).toBeNull();
361     const secondaryRow = screen.getByTestId("primary-cta-secondaries");
362     expect(
363       within(secondaryRow)
364         .getAllByRole("button")
365         .map((b) => b.textContent)
366     ).toContain("Submit Assignment");
367   });
368 });
369

```

FILE: src/__tests__/unit/processPurchase.test.ts

TYPE: .ts | LINES: 1-263 | SHA256: 6ce564773001104a6ea567353a539bc6c92b3befe70388c1f2614ec34ee4d000

[illegible]

[illegible]

```

161 |     const result: PurchaseRPCResult = {
162 |       success: false,
163 |       error_code: "ITEM_INACTIVE",
164 |     };
165 |     const response = mapRPCResultToResponse(result);
166 |     expect(response.status).toBe(404);
167 |   });
168 |
169 |   it("maps SALE_EXPIRED to 409", () => {
170 |     const result: PurchaseRPCResult = {
171 |       success: false,
172 |       error_code: "SALE_EXPIRED",
173 |     };
174 |     const response = mapRPCResultToResponse(result);
175 |     expect(response.status).toBe(409);
176 |   });
177 |
178 |   it("maps MAX_INVENTORY to 409", () => {
179 |     const result: PurchaseRPCResult = {
180 |       success: false,
181 |       error_code: "MAX_INVENTORY",
182 |     };
183 |     const response = mapRPCResultToResponse(result);
184 |     expect(response.status).toBe(409);
185 |   });
186 | });
187 |
188 | describe("processPurchase – post-purchase activation detection", () => {
189 |   it("detects XP boost activation", () => {
190 |     const result: PurchaseRPCResult = {
191 |       success: true,
192 |       purchase_id: "p-1",
193 |       xp_cost: 300,
194 |       new_balance: 700,
195 |       item_category: "power_up",
196 |       item_sub_category: "xp_boost",
197 |     };
198 |     expect(isPostPurchaseBoostActivation(result)).toBe(true);
199 |     expect(isPostPurchaseStreakShield(result)).toBe(false);
200 |   });
201 |
202 |   it("detects streak shield activation", () => {
203 |     const result: PurchaseRPCResult = {
204 |       success: true,
205 |       purchase_id: "p-2",
206 |       xp_cost: 200,
207 |       new_balance: 800,
208 |       item_category: "power_up",
209 |       item_sub_category: "streak_shield",
210 |     };
211 |     expect(isPostPurchaseStreakShield(result)).toBe(true);
212 |     expect(isPostPurchaseBoostActivation(result)).toBe(false);
213 |   });
214 |
215 |   it("does not activate for cosmetic purchases", () => {
216 |     const result: PurchaseRPCResult = {
217 |       success: true,
218 |       purchase_id: "p-3",
219 |       xp_cost: 500,
220 |       new_balance: 500,
221 |       item_category: "cosmetic",
222 |       item_sub_category: "profile_theme",
223 |     };
224 |     expect(isPostPurchaseBoostActivation(result)).toBe(false);
225 |     expect(isPostPurchaseStreakShield(result)).toBe(false);
226 |   });
227 |
228 |   it("does not activate for failed purchases", () => {
229 |     const result: PurchaseRPCResult = {
230 |       success: false,
231 |       error_code: "INSUFFICIENT_BALANCE",
232 |     };
233 |     expect(isPostPurchaseBoostActivation(result)).toBe(false);
234 |     expect(isPostPurchaseStreakShield(result)).toBe(false);
235 |   });
236 | });
237 |
238 | describe("processPurchase – item_id validation", () => {
239 |   it("accepts valid UUID", () => {
240 |     expect(validateItemId("550e8400-e29b-41d4-a716-446655440000")).toBe(true);
241 |   });
242 |
243 |   it("rejects non-UUID string", () => {
244 |     expect(validateItemId("not-a-uuid")).toBe(false);
245 |   });
246 |
247 |   it("rejects empty string", () => {
248 |     expect(validateItemId("")).toBe(false);
249 |   });
250 |
251 |   it("rejects null", () => {
252 |     expect(validateItemId(null)).toBe(false);
253 |   });
254 |
255 |   it("rejects undefined", () => {
256 |     expect(validateItemId(undefined)).toBe(false);
257 |   });
258 |
259 |   it("rejects number", () => {
260 |     expect(validateItemId(123)).toBe(false);
261 |   });
262 | });
263 |

```

FILE: src/___tests___/unit/processStreak.test.ts

TYPE: .ts | LINES: 1-398 | SHA256: faaa0152de81a5f1ba8cd361b54023c8c0106d15b8c2d0df4c4c26ad8e0cf050

```
1 | import { describe, it, expect } from "vitest";
2 | import {
3 |   calculateStreakUpdate,
4 |   daysBetween,
5 |   checkMilestone,
6 |   calculateStreakToRestore,
7 |   processComebackChallenge,
8 |   STREAK_MILESTONES,
9 |   MILESTONE_XP,
10 |   type StreakState,
11 |   type ComebackChallengeState,
12 | } from "@lib/streakCalculator";
13 |
14 | describe("Streak Calculator", () => {
15 |   describe("daysBetween", () => {
16 |     it("returns 0 for the same date", () => {
17 |       expect(daysBetween("2024-01-15", "2024-01-15")).toBe(0);
18 |     });
19 |
20 |     it("returns 1 for consecutive days", () => {
21 |       expect(daysBetween("2024-01-15", "2024-01-16")).toBe(1);
22 |     });
23 |
24 |     it("returns correct difference for multi-day gap", () => {
25 |       expect(daysBetween("2024-01-10", "2024-01-15")).toBe(5);
26 |     });
27 |
28 |     it("is commutative (order does not matter)", () => {
29 |       expect(daysBetween("2024-01-10", "2024-01-15")).toBe(
30 |         daysBetween("2024-01-15", "2024-01-10")
31 |       );
32 |     });
33 |
34 |     it("handles month boundaries", () => {
35 |       expect(daysBetween("2024-01-31", "2024-02-01")).toBe(1);
36 |     });
37 |
38 |     it("handles year boundaries", () => {
39 |       expect(daysBetween("2023-12-31", "2024-01-01")).toBe(1);
40 |     });
41 |   });
42 |
43 |   describe("checkMilestone", () => {
44 |     it("returns the milestone number for valid milestones", () => {
45 |       expect(checkMilestone(7)).toBe(7);
46 |       expect(checkMilestone(14)).toBe(14);
47 |       expect(checkMilestone(30)).toBe(30);
48 |       expect(checkMilestone(60)).toBe(60);
49 |       expect(checkMilestone(100)).toBe(100);
50 |     });
51 |
52 |     it("returns null for non-milestone counts", () => {
53 |       expect(checkMilestone(1)).toBeNull();
54 |       expect(checkMilestone(6)).toBeNull();
55 |       expect(checkMilestone(8)).toBeNull();
56 |       expect(checkMilestone(15)).toBeNull();
57 |       expect(checkMilestone(50)).toBeNull();
58 |       expect(checkMilestone(99)).toBeNull();
59 |     });
60 |   });
61 |
62 |   describe("STREAK_MILESTONES and MILESTONE_XP", () => {
63 |     it("has XP defined for every milestone", () => {
64 |       for (const m of STREAK_MILESTONES) {
65 |         expect(MILESTONE_XP[m]).toBeDefined();
66 |         expect(MILESTONE_XP[m]).toBeGreaterThan(0);
67 |       }
68 |     });
69 |
70 |     it("has correct XP values per milestone", () => {
71 |       expect(MILESTONE_XP[7]).toBe(100);
72 |       expect(MILESTONE_XP[14]).toBe(100);
73 |       expect(MILESTONE_XP[30]).toBe(250);
74 |       expect(MILESTONE_XP[60]).toBe(250);
75 |       expect(MILESTONE_XP[100]).toBe(500);
76 |     });
77 |   });
78 |
79 |   describe("calculateStreakUpdate", () => {
80 |     const makeState = (overrides: Partial<StreakState> = {}): StreakState => ({
81 |       streak_count: 1,
82 |       last_login_date: "2024-01-15",
83 |       streak_freezes_available: 0,
84 |       ...overrides,
85 |     });
86 |
87 |     describe("first login ever", () => {
88 |       it("returns streak = 1 when current is null", () => {
89 |         const result = calculateStreakUpdate(null, "2024-01-15");
90 |         expect(result.new_streak_count).toBe(1);
91 |         expect(result.is_new_day).toBe(true);
92 |         expect(result.should_reset).toBe(false);
93 |         expect(result.streak_frozen).toBe(false);
94 |         expect(result.milestone_reached).toBeNull();
95 |       });
96 |
97 |       it("returns streak = 1 when last_login_date is null", () => {
98 |         const state = makeState({ last_login_date: null });
99 |         const result = calculateStreakUpdate(state, "2024-01-15");
100 |         expect(result.new_streak_count).toBe(1);
101 |         expect(result.is_new_day).toBe(true);
```

```

102     });
103   });
104
105   describe("same day login", () => {
106     it("returns no change when logging in on the same day", () => {
107       const state = makeState({
108         streak_count: 5,
109         last_login_date: "2024-01-15",
110       });
111       const result = calculateStreakUpdate(state, "2024-01-15");
112       expect(result.new_streak_count).toBe(5);
113       expect(result.is_new_day).toBe(false);
114       expect(result.should_reset).toBe(false);
115       expect(result.streak_frozen).toBe(false);
116       expect(result.milestone_reached).toBeNull();
117     });
118   });
119
120   describe("consecutive day login", () => {
121     it("increments streak by 1", () => {
122       const state = makeState({
123         streak_count: 3,
124         last_login_date: "2024-01-15",
125       });
126       const result = calculateStreakUpdate(state, "2024-01-16");
127       expect(result.new_streak_count).toBe(4);
128       expect(result.is_new_day).toBe(true);
129       expect(result.should_reset).toBe(false);
130       expect(result.streak_frozen).toBe(false);
131     });
132
133     it("detects milestone at 7 days", () => {
134       const state = makeState({
135         streak_count: 6,
136         last_login_date: "2024-01-15",
137       });
138       const result = calculateStreakUpdate(state, "2024-01-16");
139       expect(result.new_streak_count).toBe(7);
140       expect(result.milestone_reached).toBe(7);
141     });
142
143     it("detects milestone at 14 days", () => {
144       const state = makeState({
145         streak_count: 13,
146         last_login_date: "2024-01-15",
147       });
148       const result = calculateStreakUpdate(state, "2024-01-16");
149       expect(result.milestone_reached).toBe(14);
150     });
151
152     it("detects milestone at 30 days", () => {
153       const state = makeState({
154         streak_count: 29,
155         last_login_date: "2024-01-15",
156       });
157       const result = calculateStreakUpdate(state, "2024-01-16");
158       expect(result.milestone_reached).toBe(30);
159     });
160
161     it("detects milestone at 60 days", () => {
162       const state = makeState({
163         streak_count: 59,
164         last_login_date: "2024-01-15",
165       });
166       const result = calculateStreakUpdate(state, "2024-01-16");
167       expect(result.milestone_reached).toBe(60);
168     });
169
170     it("detects milestone at 100 days", () => {
171       const state = makeState({
172         streak_count: 99,
173         last_login_date: "2024-01-15",
174       });
175       const result = calculateStreakUpdate(state, "2024-01-16");
176       expect(result.milestone_reached).toBe(100);
177     });
178
179     it("returns no milestone for non-milestone counts", () => {
180       const state = makeState({
181         streak_count: 4,
182         last_login_date: "2024-01-15",
183       });
184       const result = calculateStreakUpdate(state, "2024-01-16");
185       expect(result.new_streak_count).toBe(5);
186       expect(result.milestone_reached).toBeNull();
187     });
188   });
189
190   describe("missed 1 day with freeze available", () => {
191     it("preserves streak and consumes freeze", () => {
192       const state = makeState({
193         streak_count: 10,
194         last_login_date: "2024-01-15",
195         streak_freezes_available: 2,
196       });
197       // Missed Jan 16, logging in Jan 17 (dayDiff = 2)
198       const result = calculateStreakUpdate(state, "2024-01-17");
199       expect(result.new_streak_count).toBe(11);
200       expect(result.streak_frozen).toBe(true);
201       expect(result.freeze_consumed).toBe(true);
202       expect(result.should_reset).toBe(false);
203       expect(result.is_new_day).toBe(true);
204     });
205   });

```

```

206 |         it("detects milestone when freeze preserves streak to milestone", () => {
207 |             const state = makeState({
208 |                 streak_count: 6,
209 |                 last_login_date: "2024-01-15",
210 |                 streak_freezes_available: 1,
211 |             });
212 |             const result = calculateStreakUpdate(state, "2024-01-17");
213 |             expect(result.new_streak_count).toBe(7);
214 |             expect(result.milestone_reached).toBe(7);
215 |             expect(result.streak_frozen).toBe(true);
216 |         });
217 |     });
218 |
219 |     describe("missed 1 day without freeze", () => {
220 |         it("resets streak to 1", () => {
221 |             const state = makeState({
222 |                 streak_count: 10,
223 |                 last_login_date: "2024-01-15",
224 |                 streak_freezes_available: 0,
225 |             });
226 |             const result = calculateStreakUpdate(state, "2024-01-17");
227 |             expect(result.new_streak_count).toBe(1);
228 |             expect(result.should_reset).toBe(true);
229 |             expect(result.streak_frozen).toBe(false);
230 |             expect(result.freeze_consumed).toBe(false);
231 |             expect(result.milestone_reached).toBeNull();
232 |         });
233 |     });
234 |
235 |     describe("missed multiple days", () => {
236 |         it("resets streak to 1 even with freezes available", () => {
237 |             const state = makeState({
238 |                 streak_count: 20,
239 |                 last_login_date: "2024-01-10",
240 |                 streak_freezes_available: 2,
241 |             });
242 |             // Missed 4 days (Jan 11, 12, 13, 14), logging in Jan 15 (dayDiff = 5)
243 |             const result = calculateStreakUpdate(state, "2024-01-15");
244 |             expect(result.new_streak_count).toBe(1);
245 |             expect(result.should_reset).toBe(true);
246 |             expect(result.streak_frozen).toBe(false);
247 |             expect(result.freeze_consumed).toBe(false);
248 |         });
249 |
250 |         it("resets streak to 1 when missed 2 days without freeze", () => {
251 |             const state = makeState({
252 |                 streak_count: 5,
253 |                 last_login_date: "2024-01-10",
254 |                 streak_freezes_available: 0,
255 |             });
256 |             // dayDiff = 3 (missed 2 days)
257 |             const result = calculateStreakUpdate(state, "2024-01-13");
258 |             expect(result.new_streak_count).toBe(1);
259 |             expect(result.should_reset).toBe(true);
260 |         });
261 |     });
262 | });
263 |
264 |
265 | describe("calculateStreakToRestore", () => {
266 |     it("returns floor(lostStreak / 2)", () => {
267 |         expect(calculateStreakToRestore(10)).toBe(5);
268 |         expect(calculateStreakToRestore(11)).toBe(5);
269 |         expect(calculateStreakToRestore(1)).toBe(0);
270 |         expect(calculateStreakToRestore(0)).toBe(0);
271 |         expect(calculateStreakToRestore(100)).toBe(50);
272 |         expect(calculateStreakToRestore(7)).toBe(3);
273 |     });
274 | });
275 |
276 | describe("processComebackChallenge", () => {
277 |     const inactiveChallenge: ComebackChallengeState = {
278 |         comeback_challenge_active: false,
279 |         comeback_challenge_days_completed: 0,
280 |         comeback_challenge_streak_to_restore: 0,
281 |     };
282 |
283 |     const activeChallenge = (
284 |         days: number,
285 |         restore: number
286 |     ): ComebackChallengeState => ({
287 |         comeback_challenge_active: true,
288 |         comeback_challenge_days_completed: days,
289 |         comeback_challenge_streak_to_restore: restore,
290 |     });
291 |
292 |     describe("streak break activates challenge", () => {
293 |         it("activates challenge on streak break with lost streak > 1", () => {
294 |             const result = processComebackChallenge(
295 |                 inactiveChallenge,
296 |                 true,
297 |                 10,
298 |                 false
299 |             );
300 |             expect(result.active).toBe(true);
301 |             expect(result.days_completed).toBe(0);
302 |             expect(result.streak_to_restore).toBe(5);
303 |             expect(result.just_completed).toBe(false);
304 |             expect(result.just_cancelled).toBe(false);
305 |         });
306 |
307 |         it("does not activate challenge when lost streak is 1", () => {
308 |             const result = processComebackChallenge(
309 |                 inactiveChallenge,

```

```

310 |         true,
311 |         1,
312 |         false
313 |     );
314 |     expect(result.active).toBe(false);
315 | });
316 |
317 | it("calculates streak_to_restore as floor(lost / 2) for odd numbers", () => {
318 |     const result = processComebackChallenge(
319 |         inactiveChallenge,
320 |         true,
321 |         15,
322 |         false
323 |     );
324 |     expect(result.streak_to_restore).toBe(7);
325 | });
326 | });
327 |
328 | describe("active challenge progress", () => {
329 |     it("increments days_completed when habits are completed", () => {
330 |         const result = processComebackChallenge(
331 |             activeChallenge(0, 5),
332 |             false,
333 |             0,
334 |             true
335 |         );
336 |         expect(result.active).toBe(true);
337 |         expect(result.days_completed).toBe(1);
338 |         expect(result.just_completed).toBe(false);
339 |     });
340 |
341 |     it("increments from 1 to 2", () => {
342 |         const result = processComebackChallenge(
343 |             activeChallenge(1, 5),
344 |             false,
345 |             0,
346 |             true
347 |         );
348 |         expect(result.active).toBe(true);
349 |         expect(result.days_completed).toBe(2);
350 |     });
351 |
352 |     it("completes challenge at 3 days", () => {
353 |         const result = processComebackChallenge(
354 |             activeChallenge(2, 5),
355 |             false,
356 |             0,
357 |             true
358 |         );
359 |         expect(result.active).toBe(false);
360 |         expect(result.days_completed).toBe(3);
361 |         expect(result.streak_to_restore).toBe(5);
362 |         expect(result.just_completed).toBe(true);
363 |         expect(result.just_cancelled).toBe(false);
364 |     });
365 | });
366 |
367 | describe("challenge cancellation", () => {
368 |     it("cancels challenge when habits not completed", () => {
369 |         const result = processComebackChallenge(
370 |             activeChallenge(1, 5),
371 |             false,
372 |             0,
373 |             false
374 |         );
375 |         expect(result.active).toBe(false);
376 |         expect(result.days_completed).toBe(0);
377 |         expect(result.streak_to_restore).toBe(0);
378 |         expect(result.just_cancelled).toBe(true);
379 |         expect(result.just_completed).toBe(false);
380 |     });
381 | });
382 |
383 | describe("no active challenge and no break", () => {
384 |     it("returns inactive state", () => {
385 |         const result = processComebackChallenge(
386 |             inactiveChallenge,
387 |             false,
388 |             0,
389 |             true
390 |         );
391 |         expect(result.active).toBe(false);
392 |         expect(result.days_completed).toBe(0);
393 |         expect(result.just_completed).toBe(false);
394 |         expect(result.just_cancelled).toBe(false);
395 |     });
396 | });
397 | });
398 |

```

FILE: [src/___tests___/unit/profileCompletenessBar.test.tsx](#)

TYPE: .tsx | LINES: 1-135 | SHA256: 790c29058bf98ca84e0d18955cb30ff16e14a5fb2935cd9a2400fccaa3678285

```

1 | // @vitest-environment happy-dom
2 | // =====
3 | // ProfileCompletenessBar - Unit tests
4 | // Progress bar rendering, percentage display, navigation, badge at 100%
5 | // =====
6 |
7 | import { describe, it, expect, vi, beforeEach } from "vitest";
8 | import { render, screen } from "@testing-library/react";
9 | import userEvent from "@testing-library/user-event";
10 | import ProfileCompletenessBar, {
11 |     type ProfileCompletenessBarProps,

```

[illegible]


```

116 |     });
117 |
118 |     it("does not render a progress bar at 100%", () => {
119 |         renderBar({ completeness: 100 });
120 |         expect(screen.queryByRole("progressbar")).toBeNull();
121 |     });
122 |
123 |     it("does not render a clickable button at 100%", () => {
124 |         renderBar({ completeness: 100 });
125 |         expect(screen.queryByRole("button")).toBeNull();
126 |     });
127 |
128 |     it("renders badge for values above 100 (clamped)", () => {
129 |         renderBar({ completeness: 150 });
130 |         expect(screen.getByText("Profile Complete")).toBeTruthy();
131 |         expect(screen.queryByRole("progressbar")).toBeNull();
132 |     });
133 | });
134 |
135 |

```

FILE: src/__tests__/unit/profilePage.test.tsx

```
TYPE: .tsx | LINES: 1-296 | SHA256: 499d3d86bcb9b912a2967115e168dc54b6b810068e5b77e64943e6766fe795d
```

[illegible]

[illegible]

```

185 |     expect(input).not.toBeNull();
186 |     expect(input.accept).toBe("image/jpeg,image/png,image/gif,image/webp");
187 |     expect(input.className).toContain("hidden");
188 | });
189 |
190 | it("uploads avatar and shows success toast on valid file", async () => {
191 |     mockUploadAvatarFile.mockResolvedValue(
192 |         "https://storage.example.com/avatar.png"
193 |     );
194 |
195 |     renderPage();
196 |     const input = document.querySelector(
197 |         'input[type="file"]'
198 |     ) as HTMLInputElement;
199 |     const file = createFile("photo.png", "image/png");
200 |
201 |     fireEvent.change(input, { target: { files: [file] } });
202 |
203 |     await waitFor(() => {
204 |         expect(mockUploadAvatarFile).toHaveBeenCalledWith({
205 |             file,
206 |             userId: "user-1",
207 |         });
208 |     });
209 |
210 |     await waitFor(() => {
211 |         expect(mockUpdate).toHaveBeenCalledWith({
212 |             avatar_url: "https://storage.example.com/avatar.png",
213 |         });
214 |     });
215 |
216 |     await waitFor(() => {
217 |         expect(toast.success).toHaveBeenCalled("Avatar updated");
218 |     });
219 | });
220 |
221 | it("shows validation error toast when file is invalid", async () => {
222 |     mockUploadAvatarFile.mockRejectedValue(
223 |         new FileValidationError(
224 |             "File size exceeds the 2MB limit. Your file is 3.0MB."
225 |         )
226 |     );
227 |
228 |     renderPage();
229 |     const input = document.querySelector(
230 |         'input[type="file"]'
231 |     ) as HTMLInputElement;
232 |     const file = createFile("big.png", "image/png", 3000);
233 |
234 |     fireEvent.change(input, { target: { files: [file] } });
235 |
236 |     await waitFor(() => {
237 |         expect(toast.error).toHaveBeenCalled(
238 |             "File size exceeds the 2MB limit. Your file is 3.0MB."
239 |         );
240 |     });
241 | });
242 |
243 | it("shows generic error toast on upload failure", async () => {
244 |     mockUploadAvatarFile.mockRejectedValue(new Error("Network error"));
245 |
246 |     renderPage();
247 |     const input = document.querySelector(
248 |         'input[type="file"]'
249 |     ) as HTMLInputElement;
250 |     const file = createFile("photo.png", "image/png");
251 |
252 |     fireEvent.change(input, { target: { files: [file] } });
253 |
254 |     await waitFor(() => {
255 |         expect(toast.error).toHaveBeenCalled(
256 |             "Failed to upload avatar. Please try again."
257 |         );
258 |     });
259 | });
260 |
261 | it("renders the Appearance section with theme toggle buttons", () => {
262 |     renderPage();
263 |     expect(screen.getByText("Appearance")).toBeDefined();
264 |     expect(screen.getByRole("radio", { name: /light/i })).toBeDefined();
265 |     expect(screen.getByRole("radio", { name: /dark/i })).toBeDefined();
266 |     expect(screen.getByRole("radio", { name: /system/i })).toBeDefined();
267 | });
268 |
269 | it("calls setTheme when a theme button is clicked", () => {
270 |     renderPage();
271 |     fireEvent.click(screen.getByRole("radio", { name: /dark/i }));
272 |     expect(mockSetTheme).toHaveBeenCalled("dark");
273 | });
274 |
275 | it("marks the current theme as checked", () => {
276 |     renderPage();
277 |     const systemBtn = screen.getByRole("radio", { name: /system/i });
278 |     expect(systemBtn.getAttribute("aria-checked")).toBe("true");
279 | });
280 |
281 | it("does not render Data Export section for non-student roles", () => {
282 |     mockProfile.role = "admin" as const;
283 |     renderPage();
284 |     expect(screen.queryByText("Data Export")).toBeNull();
285 | });
286 |
287 | it("renders Data Export section for student role", () => {
288 |     mockProfile.role = "student" as "admin";

```

```

289         renderPage();
290         expect(screen.getByText("Data Export")).toBeDefined();
291         expect(
292             screen.getByRole("button", { name: /download my data/i })
293         ).toBeDefined();
294     });
295 });
296

```

FILE: src/__tests__/unit/profileSummaryCard.test.tsx

```
TYPE: .tsx | LINES: 1-367 | SHA256: 22bda26e168e6d039807182edfa95915212b7cde59ba45d56363e1795971e3ed
```

[illegible]

[illegible]

```

197 it("renders correct style for kinesthetic dominant", () => {
198   const kinestheticVARK: VARKProfile = {
199     ...sampleVARK,
200     dominant_style: "kinesthetic",
201   };
202   renderCard({ learningStyle: kinestheticVARK });
203   expect(screen.getByText("Kinesthetic Learner")).toBeDefined();
204 });
205
206 it("renders correct style for read_write dominant", () => {
207   const rwVARK: VARKProfile = {
208     ...sampleVARK,
209     dominant_style: "read_write",
210   };
211   renderCard({ learningStyle: rwVARK });
212   expect(screen.getByText("Read/Write Learner")).toBeDefined();
213 });
214
215 it("renders correct style for multimodal dominant", () => {
216   const multiVARK: VARKProfile = {
217     ...sampleVARK,
218     dominant_style: "multimodal",
219   };
220   renderCard({ learningStyle: multiVARK });
221   expect(screen.getByText("Multimodal Learner")).toBeDefined();
222 });
223 });
224
225 // %% Self-Efficacy section %%%%%%%%%%%
226
227 describe("self-efficacy section", () => {
228   it('renders the "Self-Efficacy" heading', () => {
229     renderCard();
230     expect(screen.getByText("Self-Efficacy")).toBeDefined();
231   });
232
233   it("displays all three self-efficacy domains", () => {
234     renderCard();
235     expect(screen.getByText("General Academic")).toBeDefined();
236     expect(screen.getByText("Course-Specific")).toBeDefined();
237     expect(screen.getByText("Self-Regulated Learning")).toBeDefined();
238   });
239
240   it("does not render self-efficacy section when null", () => {
241     renderCard({ selfEfficacy: null });
242     expect(screen.queryByText("Self-Efficacy")).toBeNull();
243   });
244 });
245
246 // %% Study Strategies section %%%%%%%%%%%
247
248 describe("study strategies section", () => {
249   it('renders the "Study Strategies" heading', () => {
250     renderCard();
251     expect(screen.getByText("Study Strategies")).toBeDefined();
252   });
253
254   it("displays all four study strategy dimensions", () => {
255     renderCard();
256     expect(screen.getByText("Time Management")).toBeDefined();
257     expect(screen.getByText("Elaboration")).toBeDefined();
258     expect(screen.getByText("Self-Testing")).toBeDefined();
259     expect(screen.getByText("Help Seeking")).toBeDefined();
260   });
261
262   it("does not render study strategies section when null", () => {
263     renderCard({ studyStrategies: null });
264     expect(screen.queryByText("Study Strategies")).toBeNull();
265   });
266 });
267
268 // %% Null state handling %%%%%%%%%%%
269
270 describe("null state handling", () => {
271   it("renders the card header even when all data is null", () => {
272     renderCard({
273       personalityTraits: null,
274       learningStyle: null,
275       selfEfficacy: null,
276       studyStrategies: null,
277       onRetake: undefined,
278     });
279     expect(screen.getByText("My Profile")).toBeDefined();
280   });
281
282   it("renders no data sections when all props are null", () => {
283     renderCard({
284       personalityTraits: null,
285       learningStyle: null,
286       selfEfficacy: null,
287       studyStrategies: null,
288     });
289     expect(screen.queryByText("Personality Traits")).toBeNull();
290     expect(screen.queryByText("Self-Awareness")).toBeNull();
291     expect(screen.queryByText("Self-Efficacy")).toBeNull();
292     expect(screen.queryByText("Study Strategies")).toBeNull();
293   });
294
295   it("renders only the sections with non-null data", () => {
296     renderCard({
297       personalityTraits: sampleTraits,
298       learningStyle: null,
299       selfEfficacy: null,
300       studyStrategies: null,

```

[illegible]

FILE: src/__tests__/unit/progressSummaryPanel.test.tsx

TYPE: .tsx | LINES: 1-167 | SHA256: 49d7cb85017a41f52d43b88a32383d1f8029e360066210a1ecd6ccbfdb81f18c

[illegible]

[illegible]


```

138 |         const progressBar = screen.getByRole("progressbar");
139 |         expect(progressBar).toHaveAttribute("aria-valuenow", "75");
140 |         expect(progressBar).toHaveAttribute("aria-valuemin", "0");
141 |         expect(progressBar).toHaveAttribute("aria-valuemax", "100");
142 |     });
143 |
144 |     it("displays goal type labels correctly", () => {
145 |         const goals: GoalProgress[] = [
146 |             makeGoalProgress({ id: "g1", goalType: "study_hours" }),
147 |             makeGoalProgress({ id: "g2", goalType: "sessions_completed" }),
148 |             makeGoalProgress({ id: "g3", goalType: "tasks_completed" }),
149 |         ];
150 |         render(<ProgressSummaryPanel summary={makeSummary()} goals={goals} />);
151 |         expect(screen.getByText("Study Hours")).toBeInTheDocument();
152 |         // "Sessions" appears in both KPI row and goal progress
153 |         expect(screen.getAllByText("Sessions")).toHaveLength(2);
154 |         expect(screen.getByText("Tasks")).toBeInTheDocument();
155 |     });
156 |
157 |     it("displays zero study hours correctly", () => {
158 |         render(
159 |             <ProgressSummaryPanel
160 |                 summary={makeSummary({ totalStudyMinutes: 0 })}
161 |                 goals={[]}
162 |             />
163 |         );
164 |         expect(screen.getByTestId("study-hours")).toHaveTextContent("0.0h");
165 |     });
166 | });
167 |

```

FILE: src/___tests___/unit/purchaseConfirmDialog.test.tsx

TYPE: .tsx | LINES: 1-176 | SHA256: c5968f8e8bad9e4dc8d9b572e6db8fb6dc0f251e05602b75a3d472cfe3869180

[illegible]

```

71 |     <PurchaseConfirmDialog
72 |       item={makeItem({ effective_price: 500 })}
73 |       currentBalance={200}
74 |       isPending={false}
75 |       open={true}
76 |       onOpenChange={vi.fn()}
77 |       onConfirm={vi.fn()}
78 |     />
79 |   );
80 |
81 |   expect(screen.getByText(/don't have enough XP/)).toBeTruthy();
82 | });
83 |
84 | it("disables confirm button when cannot afford", () => {
85 |   render(
86 |     <PurchaseConfirmDialog
87 |       item={makeItem({ effective_price: 500 })}
88 |       currentBalance={200}
89 |       isPending={false}
90 |       open={true}
91 |       onOpenChange={vi.fn()}
92 |       onConfirm={vi.fn()}
93 |     />
94 |   );
95 |
96 |   const confirmBtn = screen.getByRole("button", {
97 |     name: /Confirm Purchase/i,
98 |   });
99 |   expect(confirmBtn).toBeDisabled();
100 | });
101 |
102 | it("disables confirm button when pending", () => {
103 |   render(
104 |     <PurchaseConfirmDialog
105 |       item={makeItem()}
106 |       currentBalance={1000}
107 |       isPending={true}
108 |       open={true}
109 |       onOpenChange={vi.fn()}
110 |       onConfirm={vi.fn()}
111 |     />
112 |   );
113 |
114 |   const confirmBtn = screen.getByRole("button", {
115 |     name: /Confirm Purchase/i,
116 |   });
117 |   expect(confirmBtn).toBeDisabled();
118 | });
119 |
120 | it("calls onConfirm when confirm button is clicked", async () => {
121 |   const onConfirm = vi.fn();
122 |   render(
123 |     <PurchaseConfirmDialog
124 |       item={makeItem()}
125 |       currentBalance={1000}
126 |       isPending={false}
127 |       open={true}
128 |       onOpenChange={vi.fn()}
129 |       onConfirm={onConfirm}
130 |     />
131 |   );
132 |
133 |   const confirmBtn = screen.getByRole("button", {
134 |     name: /Confirm Purchase/i,
135 |   });
136 |   await userEvent.click(confirmBtn);
137 |   expect(onConfirm).toHaveBeenCalledOnce();
138 | });
139 |
140 | it("shows sale price with strikethrough when sale is active", () => {
141 |   render(
142 |     <PurchaseConfirmDialog
143 |       item={makeItem({
144 |         xp_price: 500,
145 |         effective_price: 375,
146 |         sale_discount: 25,
147 |       })}
148 |       currentBalance={1000}
149 |       isPending={false}
150 |       open={true}
151 |       onOpenChange={vi.fn()}
152 |       onConfirm={vi.fn()}
153 |     />
154 |   );
155 |
156 |   // Original price should be shown with strikethrough
157 |   expect(screen.getByText("500")).toBeTruthy();
158 |   expect(screen.getByText(/375 XP/)).toBeTruthy();
159 | });
160 |
161 | it("returns null when item is null", () => {
162 |   const { container } = render(
163 |     <PurchaseConfirmDialog
164 |       item={null}
165 |       currentBalance={1000}
166 |       isPending={false}
167 |       open={true}
168 |       onOpenChange={vi.fn()}
169 |       onConfirm={vi.fn()}
170 |     />
171 |   );
172 |
173 |   expect(container.innerHTML).toBe("");
174 | });

```

```
175 | });
176 |
```

FILE: src/___tests___/unit/queryClientErrorNet.test.ts

TYPE: .ts | LINES: 1-90 | SHA256: 002ce6d4a3c26c383574cbc96164005b9c611eae0a7b0bbf0ef84168daa1d09f

```
1  // =====
2  // queryClientErrorNet.test.ts
3  // Feature: production-bug-fixes, Req 6 – global query/mutation error safety net.
4  // =====
5  // Verifies the dedup-aware handlers:
6  //   - always log (console + Sentry)
7  //   - query net toasts a fallback unless meta.suppressGlobalError (no double-toast)
8  //   - mutation net only surfaces the shared 429 notice (hooks toast their own)
9  // =====
10 import { describe, it, expect, vi, beforeEach } from "vitest";
11
12 const toastError = vi.fn();
13 vi.mock("sonner", () => ({ toast: { error: (m: string) => toastError(m) } }));
14
15 const captureException = vi.fn();
16 vi.mock("@sentry/react", () => ({
17   captureException: (...a: unknown[]) => captureException(...a),
18 }));
19
20 import {
21   handleGlobalQueryError,
22   handleGlobalMutationError,
23   getErrorMessage,
24 } from "@lib/queryClient";
25
26 describe("Global error safety net (production-bug-fixes Req 6)", () => {
27   beforeEach(() => {
28     vi.clearAllMocks();
29     vi.spyOn(console, "error").mockImplementation(() => {});
30   });
31
32   describe("handleGlobalQueryError", () => {
33     it("logs (console + Sentry) and toasts when not opted out", () => {
34       handleGlobalQueryError(new Error("boom"), undefined);
35       expect(console.error).toHaveBeenCalledTimes(1);
36       expect(captureException).toHaveBeenCalledTimes(1);
37       expect(toastError).toHaveBeenCalledTimes(1);
38     });
39
40     it("logs but does NOT toast when meta.suppressGlobalError is set (no double-toast)", () => {
41       handleGlobalQueryError(new Error("handled by hook"), {
42         suppressGlobalError: true,
43       });
44       expect(console.error).toHaveBeenCalledTimes(1);
45       expect(captureException).toHaveBeenCalledTimes(1);
46       expect(toastError).not.toHaveBeenCalled();
47     });
48
49     it("uses the rate-limit message for 429 errors", () => {
50       handleGlobalQueryError({ status: 429 }, undefined);
51       expect(toastError).toHaveBeenCalledTimes(1);
52       expect(toastError).toHaveBeenCalledWith(
53         "Too many requests. Please wait a moment."
54       );
55     });
56   });
57
58   describe("handleGlobalMutationError", () => {
59     it("logs always but does not toast a generic message (hooks own their toast)", () => {
60       handleGlobalMutationError(new Error("save failed"), undefined);
61       expect(console.error).toHaveBeenCalledTimes(1);
62       expect(captureException).toHaveBeenCalledTimes(1);
63       expect(toastError).not.toHaveBeenCalled();
64     });
65
66     it("surfaces the shared 429 notice", () => {
67       handleGlobalMutationError({ status: 429 }, undefined);
68       expect(toastError).toHaveBeenCalledTimes(1);
69       expect(toastError).toHaveBeenCalledWith(
70         "Too many requests. Please wait a moment."
71       );
72     });
73
74     it("respects the opt-out flag even for 429", () => {
75       handleGlobalMutationError({ status: 429 }, { suppressGlobalError: true });
76       expect(toastError).not.toHaveBeenCalled();
77     });
78   });
79
80   describe("getErrorMessage", () => {
81     it("returns the error message when present, else a generic fallback", () => {
82       expect(getErrorMessage(new Error("specific"))).toBe("specific");
83       expect(getErrorMessage(null)).toBe(
84         "Something went wrong. Please try again."
85       );
86       expect(getErrorMessage({})).toBe(
87         "Something went wrong. Please try again."
88       );
89     });
90   });
91 }
```

```
TYPE: .ts | LINES: 1-70 | SHA256: dd3736611f439c9ba6ca5c99eafeee03223a9abcb077c2333b11ea4803e64236
```

FILE: src/__tests__/unit/questionAnalytics.test.ts

TYPE: .ts | LINES: 1-141 | SHA256: d779cd1f38d26395eadabaf9733a6bf1a03c08dab61bf8bdac238af018508448

[illegible]

[illegible]

```

131     it("counts exactly 4.0 difficulty as hard", () => {
132         const questions = [{ difficulty_rating: 4.0 }];
133         expect(computeBonusXP(questions)).toBe(10);
134     });
135
136     it("does not count 3.99 difficulty as hard", () => {
137         const questions = [{ difficulty_rating: 3.99 }];
138         expect(computeBonusXP(questions)).toBe(0);
139     });
140 });
141

```

FILE: src/__tests__/unit/questionPreview.test.tsx

```
TYPE: .tsx | LINES: 1-236 | SHA256: b4c215c6c466fbfadd3484ce0e5c399ef26ff488f14785921ffa7009dd9d063d
```

[illegible]

[illegible]

```

194
195 it("renders label for the input" () => {
196   renderPreview({ questionType: "fill_in_blank", options: null });
197   expect(screen.getByText("Fill in the blank")).toBeInTheDocument();
198 });
199
200 it("calls onChange when typing in input", () => {
201   const onChange = vi.fn();
202   renderPreview({
203     questionType: "fill_in_blank",
204     options: null,
205     onChange: onChange,
206   });
207   fireEvent.change(screen.getByPlaceholderText("Type your answer..."), {
208     target: { value: "Queue" },
209   });
210   expect(onChange).toHaveBeenCalledWith("Queue");
211 });
212
213 it("shows correct answer when showCorrectAnswer is true", () => {
214   renderPreview({
215     questionType: "fill_in_blank",
216     options: null,
217     selectedAnswer: "Stack",
218     showCorrectAnswer: true,
219     correctAnswer: "Queue",
220   });
221   expect(screen.getByText(/Correct answer: Queue/)).toBeInTheDocument();
222 });
223
224 it("applies green styling for correct answer", () => {
225   renderPreview({
226     questionType: "fill_in_blank",
227     options: null,
228     selectedAnswer: "Queue",
229     showCorrectAnswer: true,
230     correctAnswer: "Queue",
231   });
232   const input = screen.getByPlaceholderText("Type your answer...");
233   expect(input.className).toContain("green");
234 });
235 });
236

```

FILE: src/__tests__/unit/questionQualityIndicator.test.tsx

TYPE: .tsx | LINES: 1-79 | SHA256: 021c6c52dff731f9540a31ea10f735741aa353b74cda2123f8f71370d57c9465

[illegible]


```

79 |     })
80 |   );
81 |
82 |   mockFrom.mockImplementation((tableName: string) => {
83 |     if (tableName === "profiles") {
84 |       return {
85 |         select: vi.fn().mockReturnValue({
86 |           eq: vi.fn().mockReturnValue({
87 |             maybeSingle: vi.fn().mockResolvedValue({
88 |               data: {
89 |                 id: userId,
90 |                 role,
91 |                 full_name: "Test User",
92 |                 institution_id: "4de6a0a2-758b-47f3-ab7e-984bb974d88b",
93 |                 status: "active",
94 |               },
95 |               error: null,
96 |             }),
97 |             single: vi.fn().mockResolvedValue({
98 |               data: {
99 |                 id: userId,
100 |                 role,
101 |                 full_name: "Test User",
102 |                 institution_id: "4de6a0a2-758b-47f3-ab7e-984bb974d88b",
103 |                 status: "active",
104 |               },
105 |               error: null,
106 |             }),
107 |           }),
108 |         }),
109 |       };
110 |     }
111 |
112 |     // Role-specific verification tables
113 |     if (tableName === "parent_student_links") {
114 |       return {
115 |         select: vi.fn().mockReturnValue({
116 |           eq: vi.fn().mockReturnValue({
117 |             eq: vi.fn().mockResolvedValue({ count: 1, error: null }),
118 |           }),
119 |         }),
120 |       };
121 |     }
122 |
123 |     if (tableName === "courses") {
124 |       return {
125 |         select: vi.fn().mockReturnValue({
126 |           eq: vi.fn().mockResolvedValue({ count: 2, error: null }),
127 |         }),
128 |       };
129 |     }
130 |
131 |     if (tableName === "programs") {
132 |       return {
133 |         select: vi.fn().mockReturnValue({
134 |           eq: vi.fn().mockResolvedValue({ count: 1, error: null }),
135 |         }),
136 |       };
137 |     }
138 |
139 |     return {
140 |       select: vi.fn().mockReturnValue({
141 |         eq: vi.fn().mockResolvedValue({ count: 1, error: null }),
142 |       }),
143 |     };
144 |   });
145 | };
146 |
147 | const renderLoginPage = () => {
148 |   const queryClient = new QueryClient({
149 |     defaultOptions: { queries: { retry: false } },
150 |   });
151 |   return render(
152 |     <QueryClientProvider client={queryClient}>
153 |       <MemoryRouter>
154 |         <AuthProvider>
155 |           <LoginPage />
156 |         </AuthProvider>
157 |       </MemoryRouter>
158 |     </QueryClientProvider>
159 |   );
160 | };
161 |
162 | describe("Quick Login for Noor International Testing", () => {
163 |   beforeEach(() => {
164 |     vi.clearAllMocks();
165 |     localStorage.clear();
166 |     setupMocks();
167 |   });
168 |
169 |   it("renders all 5 Quick Login role buttons when demo panel is active", async () => {
170 |     renderLoginPage();
171 |
172 |     await waitFor(() => {
173 |       expect(screen.getByTestId("quick-login-admin")).toBeInTheDocument();
174 |     });
175 |     expect(screen.getByTestId("quick-login-coordinator")).toBeInTheDocument();
176 |     expect(screen.getByTestId("quick-login-teacher")).toBeInTheDocument();
177 |     expect(screen.getByTestId("quick-login-student")).toBeInTheDocument();
178 |     expect(screen.getByTestId("quick-login-parent")).toBeInTheDocument();
179 |   });
180 |
181 |   it("authenticates student and navigates to /student/dashboard", async () => {
182 |     setupMocks("student", "student01@noor-international.test", "student-id-1");

```

```

183 |     const user = userEvent.setup();
184 |     renderLoginPage();
185 |
186 |     await waitFor(() => {
187 |       expect(screen.getByTestId("quick-login-student")).not.toBeDisabled();
188 |     });
189 |
190 |     const studentBtn = screen.getByTestId("quick-login-student");
191 |     await user.click(studentBtn);
192 |
193 |     await waitFor(
194 |       () => {
195 |         expect(mockSignOut).toHaveBeenCalled();
196 |         expect(mockSignInWithPassword).toHaveBeenCalledWith({
197 |           email: "student01@noor-international.test",
198 |           password: expect.any(String),
199 |         });
200 |         expect(mockNavigate).toHaveBeenCalledWith("/student/dashboard", {
201 |           replace: true,
202 |         });
203 |       },
204 |       { timeout: 4000 }
205 |     );
206 |   });
207 |
208 |   it("authenticates parent with verified child link and navigates to /parent/dashboard", async () => {
209 |     setupMocks("parent", "parent01@noor-international.test", "parent-id-1");
210 |     const user = userEvent.setup();
211 |     renderLoginPage();
212 |
213 |     await waitFor(() => {
214 |       expect(screen.getByTestId("quick-login-parent")).not.toBeDisabled();
215 |     });
216 |
217 |     const parentBtn = screen.getByTestId("quick-login-parent");
218 |     await user.click(parentBtn);
219 |
220 |     await waitFor(
221 |       () => {
222 |         expect(mockSignOut).toHaveBeenCalled();
223 |         expect(mockSignInWithPassword).toHaveBeenCalledWith({
224 |           email: "parent01@noor-international.test",
225 |           password: expect.any(String),
226 |         });
227 |         expect(mockNavigate).toHaveBeenCalledWith("/parent/dashboard", {
228 |           replace: true,
229 |         });
230 |       },
231 |       { timeout: 4000 }
232 |     );
233 |   });
234 |
235 |   it("authenticates teacher with assigned courses and navigates to /teacher/dashboard", async () => {
236 |     setupMocks("teacher", "okonkwo@noor-international.test", "teacher-id-1");
237 |     const user = userEvent.setup();
238 |     renderLoginPage();
239 |
240 |     await waitFor(() => {
241 |       expect(screen.getByTestId("quick-login-teacher")).not.toBeDisabled();
242 |     });
243 |
244 |     const teacherBtn = screen.getByTestId("quick-login-teacher");
245 |     await user.click(teacherBtn);
246 |
247 |     await waitFor(
248 |       () => {
249 |         expect(mockSignOut).toHaveBeenCalled();
250 |         expect(mockSignInWithPassword).toHaveBeenCalledWith({
251 |           email: "okonkwo@noor-international.test",
252 |           password: expect.any(String),
253 |         });
254 |         expect(mockNavigate).toHaveBeenCalledWith("/teacher/dashboard", {
255 |           replace: true,
256 |         });
257 |       },
258 |       { timeout: 4000 }
259 |     );
260 |   });
261 |
262 |   it("authenticates coordinator with program access and navigates to /coordinator/dashboard", async () => {
263 |     setupMocks(
264 |       "coordinator",
265 |       "curriculum@noor-international.test",
266 |       "coord-id-1"
267 |     );
268 |     const user = userEvent.setup();
269 |     renderLoginPage();
270 |
271 |     await waitFor(() => {
272 |       expect(screen.getByTestId("quick-login-coordinator")).not.toBeDisabled();
273 |     });
274 |
275 |     const coordBtn = screen.getByTestId("quick-login-coordinator");
276 |     await user.click(coordBtn);
277 |
278 |     await waitFor(
279 |       () => {
280 |         expect(mockSignOut).toHaveBeenCalled();
281 |         expect(mockSignInWithPassword).toHaveBeenCalledWith({
282 |           email: "curriculum@noor-international.test",
283 |           password: expect.any(String),
284 |         });
285 |         expect(mockNavigate).toHaveBeenCalledWith("/coordinator/dashboard", {
286 |           replace: true,

```

```

287     });
288   },
289   { timeout: 4000 }
290 );
291 });
292
293 it("authenticates admin and navigates to /admin/dashboard", async () => {
294   setupMocks("admin", "principal@noor-international.test", "admin-id-1");
295   const user = userEvent.setup();
296   renderLoginPage();
297
298   await waitFor(() => {
299     expect(screen.getByTestId("quick-login-admin")).not.toBeDisabled();
300   });
301
302   const adminBtn = screen.getByTestId("quick-login-admin");
303   await user.click(adminBtn);
304
305   await waitFor(
306     () => {
307       expect(mockSignOut).toHaveBeenCalled();
308       expect(mockSignInWithPassword).toHaveBeenCalledWith({
309         email: "principal@noor-international.test",
310         password: expect.any(String),
311       });
312       expect(mockNavigate).toHaveBeenCalledWith("/admin/dashboard", {
313         replace: true,
314       });
315     },
316     { timeout: 4000 }
317   );
318 });
319
320 it("aborts redirect and displays error when account role mismatch occurs", async () => {
321   setupMocks("student", "parent01@noor-international.test", "mismatch-id-1");
322   const user = userEvent.setup();
323   renderLoginPage();
324
325   await waitFor(() => {
326     expect(screen.getByTestId("quick-login-parent")).not.toBeDisabled();
327   });
328
329   const parentBtn = screen.getByTestId("quick-login-parent");
330   await user.click(parentBtn);
331
332   await waitFor(() => {
333     expect(mockSignOut).toHaveBeenCalled();
334     expect(mockNavigate).not.toHaveBeenCalled();
335     expect(
336       screen.getByText(/does not match requested Quick Login role/i)
337     ).toBeInTheDocument();
338   });
339 });
340 });
341

```

FILE: src/__tests__/unit/quickThoughtInput.test.tsx

TYPE: .tsx | LINES: 1-241 | SHA256: 8e72466a948a0d8913a8d147bb5d67715ce511cb614afd41ac4b237ae9a21c2b


```
FILE: src/___tests___/unit/quizBonusXP.test.ts
```

```
TYPE: .ts | LINES: 1-89 | SHA256: 2a5c4b1f1f1be028ba67643ae9c0e7c88c1588a769795c7de3324af9173997c1f3
```

```
1 import { describe, it, expect } from "vitest";
2 import { computeBonusXP } from "@lib/questionAnalytics";
3
4 describe("computeBonusXP - mixed difficulty edge cases", () => {
5   it("returns 0 for an empty array", () => {
6     expect(computeBonusXP([])).toBe(0);
7   });
8 });
```

```

9 | it("returns 0 when all questions are easy (difficulty 1.0-3.9)", () => {
10 |     const questions = [
11 |         { difficulty_rating: 1.0 },
12 |         { difficulty_rating: 2.5 },
13 |         { difficulty_rating: 3.0 },
14 |         { difficulty_rating: 3.9 },
15 |     ];
16 |     expect(computeBonusXP(questions)).toBe(0);
17 | });
18 |
19 | it("returns 20 XP for 2 hard and 3 easy questions", () => {
20 |     const questions = [
21 |         { difficulty_rating: 4.0 },
22 |         { difficulty_rating: 4.5 },
23 |         { difficulty_rating: 1.0 },
24 |         { difficulty_rating: 2.0 },
25 |         { difficulty_rating: 3.5 },
26 |     ];
27 |     expect(computeBonusXP(questions)).toBe(20);
28 | });
29 |
30 | it("returns 30 XP for 3 hard and 7 easy questions", () => {
31 |     const questions = [
32 |         { difficulty_rating: 4.0 },
33 |         { difficulty_rating: 4.2 },
34 |         { difficulty_rating: 5.0 },
35 |         { difficulty_rating: 1.0 },
36 |         { difficulty_rating: 1.5 },
37 |         { difficulty_rating: 2.0 },
38 |         { difficulty_rating: 2.5 },
39 |         { difficulty_rating: 3.0 },
40 |         { difficulty_rating: 3.5 },
41 |         { difficulty_rating: 3.9 },
42 |     ];
43 |     expect(computeBonusXP(questions)).toBe(30);
44 | });
45 |
46 | it("returns 50 XP for exactly 5 hard questions (max)", () => {
47 |     const questions = Array.from({ length: 5 }, () => ({
48 |         difficulty_rating: 4.0,
49 |     }));
50 |     expect(computeBonusXP(questions)).toBe(50);
51 | });
52 |
53 | it("returns 50 XP (capped) for 8 hard questions", () => {
54 |     const questions = Array.from({ length: 8 }, () => ({
55 |         difficulty_rating: 4.5,
56 |     }));
57 |     expect(computeBonusXP(questions)).toBe(50);
58 | });
59 |
60 | it("caps at 50 when all questions are exactly 4.0", () => {
61 |     const questions = Array.from({ length: 7 }, () => ({
62 |         difficulty_rating: 4.0,
63 |     }));
64 |     // 7 * 10 = 70, capped at 50
65 |     expect(computeBonusXP(questions)).toBe(50);
66 | });
67 |
68 | it("returns 30 XP for various difficulties: 1.0, 2.5, 3.9, 4.0, 4.5, 5.0", () => {
69 |     const questions = [
70 |         { difficulty_rating: 1.0 },
71 |         { difficulty_rating: 2.5 },
72 |         { difficulty_rating: 3.9 },
73 |         { difficulty_rating: 4.0 },
74 |         { difficulty_rating: 4.5 },
75 |         { difficulty_rating: 5.0 },
76 |     ];
77 |     // 3 hard (4.0, 4.5, 5.0) != 30 XP
78 |     expect(computeBonusXP(questions)).toBe(30);
79 | });
80 |
81 | it("returns 10 XP for a single question at exactly 4.0", () => {
82 |     expect(computeBonusXP([ { difficulty_rating: 4.0 } ])).toBe(10);
83 | });
84 |
85 | it("returns 0 XP for a single question at 3.99", () => {
86 |     expect(computeBonusXP([ { difficulty_rating: 3.99 } ])).toBe(0);
87 | });
88 | });
89 |

```

FILE: [src/__tests__/unit/quizForm.test.tsx](#)

TYPE: .tsx | LINES: 1-144 | SHA256: 4eece09c80b4fe910add673f852969c087a5e98cd62cd6d45a75b7b67007a7a6

```

1 | // =====
2 | // QuizForm - Unit tests
3 | // Validates: Task 16.8 - PracticeModeToggle integration in quiz creation/edit form
4 | // =====
5 |
6 | import { describe, it, expect, vi, beforeEach } from "vitest";
7 | import { render, screen, fireEvent } from "@testing-library/react";
8 | import { MemoryRouter, Route, Routes } from "react-router-dom";
9 | import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
10 |
11 | // Mock hooks before importing the component
12 | vi.mock("@/hooks/useQuizzes", () => ({
13 |     useQuiz: vi.fn().mockReturnValue({ data: null, isLoading: false }),
14 |     useCreateQuiz: vi.fn().mockReturnValue({ mutate: vi.fn(), isLoading: false }),
15 |     useUpdateQuiz: vi.fn().mockReturnValue({ mutate: vi.fn(), isLoading: false }),
16 | }));
17 |
18 | vi.mock("@/hooks/useCourses", () => ({
19 |     useCourses: vi.fn().mockReturnValue({

```

```

20 |     data: {
21 |       data: [{ id: "course-1", code: "CS101", name: "Intro to CS" }],
22 |       count: 1,
23 |     },
24 |     isLoading: false,
25 |   },
26 |   useTeacherCourses: vi.fn().mockReturnValue({
27 |     data: {
28 |       data: [{ id: "course-1", code: "CS101", name: "Intro to CS" }],
29 |       count: 1,
30 |     },
31 |     isLoading: false,
32 |   }),
33 | ));
34 |
35 | vi.mock("@/hooks/useCLOs", () => ({
36 |   useCLOs: vi.fn().mockReturnValue({
37 |     data: {
38 |       data: [{ id: "clo-1", title: "CLO 1: Understand basics" }],
39 |       count: 1,
40 |     },
41 |     isLoading: false,
42 |   }),
43 | }));
44 |
45 | import QuizForm from "@/pages/teacher/quizzes/QuizForm";
46 |
47 | const createWrapper = (
48 |   initialRoute = "/teacher/courses/course-1/quizzes/new"
49 | ) => {
50 |   const queryClient = new QueryClient({
51 |     defaultOptions: { queries: { retry: false } },
52 |   });
53 |
54 |   return ({ children }: { children: React.ReactNode }) => (
55 |     <QueryClientProvider client={queryClient}>
56 |       <MemoryRouter initialEntries={[initialRoute]}>
57 |         <Routes>
58 |           <Route
59 |             path="/teacher/courses/:courseId/quizzes/new"
60 |             element={children}
61 |           />
62 |           <Route
63 |             path="/teacher/courses/:courseId/quizzes/:id/edit"
64 |             element={children}
65 |           />
66 |         </Routes>
67 |       </MemoryRouter>
68 |     </QueryClientProvider>
69 |   );
70 | };
71 |
72 | describe("QuizForm – PracticeModeToggle integration", () => {
73 |   beforeEach(() => {
74 |     vi.clearAllMocks();
75 |   });
76 |
77 |   it('renders the PracticeModeToggle with "Allow Practice Mode" label', () => {
78 |     render(<QuizForm />, { wrapper: createWrapper() });
79 |     expect(screen.getByText("Allow Practice Mode")).toBeDefined();
80 |   });
81 |
82 |   it("renders the practice mode description text", () => {
83 |     render(<QuizForm />, { wrapper: createWrapper() });
84 |     expect(
85 |       screen.getByText("Students can take this quiz without grade impact")
86 |     ).toBeDefined();
87 |   });
88 |
89 |   it("renders the practice mode switch unchecked by default", () => {
90 |     render(<QuizForm />, { wrapper: createWrapper() });
91 |     const toggle = screen.getByRole("switch", { name: /allow practice mode/i });
92 |     expect(toggle.getAttribute("data-state")).toBe("unchecked");
93 |   });
94 |
95 |   it("toggles practice mode switch on click", () => {
96 |     render(<QuizForm />, { wrapper: createWrapper() });
97 |     const toggle = screen.getByRole("switch", { name: /allow practice mode/i });
98 |     expect(toggle.getAttribute("data-state")).toBe("unchecked");
99 |     fireEvent.click(toggle);
100 |    expect(toggle.getAttribute("data-state")).toBe("checked");
101 |   });
102 | });
103 |
104 | describe("QuizForm – Adaptive mode toggle", () => {
105 |   it("renders the adaptive mode toggle", () => {
106 |     render(<QuizForm />, { wrapper: createWrapper() });
107 |     expect(screen.getByText("Enable Adaptive Mode")).toBeDefined();
108 |   });
109 |
110 |   it("does not show adaptation config fields when adaptive is off", () => {
111 |     render(<QuizForm />, { wrapper: createWrapper() });
112 |     expect(screen.queryByLabelText(/initial difficulty/i)).toBeNull();
113 |   });
114 |
115 |   it("shows adaptation config fields when adaptive is toggled on", () => {
116 |     render(<QuizForm />, { wrapper: createWrapper() });
117 |     const adaptiveToggle = screen.getByRole("switch", {
118 |       name: /enable adaptive mode/i,
119 |     });
120 |     fireEvent.click(adaptiveToggle);
121 |     expect(screen.getByLabelText(/initial difficulty/i)).toBeDefined();
122 |     expect(screen.getByLabelText(/step up/i)).toBeDefined();
123 |     expect(screen.getByLabelText(/step down/i)).toBeDefined();

```



```

184 |         },
185 |         { key: "B", text: "Execute programs directly", is_correct: false },
186 |         { key: "C", text: "Debug source code", is_correct: false },
187 |         { key: "D", text: "Format source code", is_correct: false },
188 |     ],
189 |     correct_answer: {
190 |         value: "A",
191 |         explanation: "A compiler translates source code into machine code.",
192 |     },
193 |     difficulty_rating: 2.5,
194 | };
195 |
196 | it("accepts valid entry with all required fields", () => {
197 |     expect(() => questionBankEntrySchema.parse(validEntry)).not.toThrow();
198 | });
199 |
200 | it("accepts null options (for non-MCQ types)", () => {
201 |     const result = questionBankEntrySchema.safeParse({
202 |         ...validEntry,
203 |         question_type: "short_answer",
204 |         options: null,
205 |     });
206 |     expect(result.success).toBe(true);
207 | });
208 |
209 | it("accepts optional explanation field", () => {
210 |     const result = questionBankEntrySchema.safeParse({
211 |         ...validEntry,
212 |         explanation: "Additional explanation text",
213 |     });
214 |     expect(result.success).toBe(true);
215 | });
216 |
217 | it("accepts optional labels field", () => {
218 |     const result = questionBankEntrySchema.safeParse({
219 |         ...validEntry,
220 |         labels: ["midterm", "chapter-3"],
221 |     });
222 |     expect(result.success).toBe(true);
223 | });
224 |
225 | it("accepts difficulty_rating at boundaries (1.0 and 5.0)", () => {
226 |     expect(
227 |         questionBankEntrySchema.safeParse({
228 |             ...validEntry,
229 |             difficulty_rating: 1.0,
230 |         }).success
231 |     ).toBe(true);
232 |     expect(
233 |         questionBankEntrySchema.safeParse({
234 |             ...validEntry,
235 |             difficulty_rating: 5.0,
236 |         }).success
237 |     ).toBe(true);
238 | });
239 |
240 | it("rejects difficulty_rating below 1.0", () => {
241 |     const result = questionBankEntrySchema.safeParse({
242 |         ...validEntry,
243 |         difficulty_rating: 0.9,
244 |     });
245 |     expect(result.success).toBe(false);
246 | });
247 |
248 | it("rejects difficulty_rating above 5.0", () => {
249 |     const result = questionBankEntrySchema.safeParse({
250 |         ...validEntry,
251 |         difficulty_rating: 5.1,
252 |     });
253 |     expect(result.success).toBe(false);
254 | });
255 |
256 | it("accepts bloom_level at boundaries (1 and 6)", () => {
257 |     expect(
258 |         questionBankEntrySchema.safeParse({ ...validEntry, bloom_level: 1 })
259 |     ).success
260 |     ).toBe(true);
261 |     expect(
262 |         questionBankEntrySchema.safeParse({ ...validEntry, bloom_level: 6 })
263 |     ).success
264 |     ).toBe(true);
265 | });
266 |
267 | it("rejects bloom_level below 1", () => {
268 |     const result = questionBankEntrySchema.safeParse({
269 |         ...validEntry,
270 |         bloom_level: 0,
271 |     });
272 |     expect(result.success).toBe(false);
273 | });
274 |
275 | it("rejects bloom_level above 6", () => {
276 |     const result = questionBankEntrySchema.safeParse({
277 |         ...validEntry,
278 |         bloom_level: 7,
279 |     });
280 |     expect(result.success).toBe(false);
281 | });
282 |
283 | it("rejects non-integer bloom_level", () => {
284 |     const result = questionBankEntrySchema.safeParse({
285 |         ...validEntry,
286 |         bloom_level: 2.5,
287 |     });

```



```

600 |     expect(() => bloomsClimbStateSchema.parse(validState)).not.toThrow();
601 | });
602 |
603 | it("accepts state with transitions", () => {
604 |   const result = bloomsClimbStateSchema.safeParse({
605 |     ...validState,
606 |     bloom_transitions: [
607 |       { from_level: 1, to_level: 2, question_number: 4 },
608 |       { from_level: 2, to_level: 3, question_number: 7 },
609 |     ],
610 |   });
611 |   expect(result.success).toBe(true);
612 | });
613 |
614 | it("accepts current_bloom_level at boundaries (1 and 6)", () => {
615 |   expect(
616 |     bloomsClimbStateSchema.safeParse({
617 |       ...validState,
618 |       current_bloom_level: 1,
619 |     }).success
620 |   ).toBe(true);
621 |   expect(
622 |     bloomsClimbStateSchema.safeParse({
623 |       ...validState,
624 |       current_bloom_level: 6,
625 |     }).success
626 |   ).toBe(true);
627 | });
628 |
629 | it("rejects current_bloom_level below 1", () => {
630 |   const result = bloomsClimbStateSchema.safeParse({
631 |     ...validState,
632 |     current_bloom_level: 0,
633 |   });
634 |   expect(result.success).toBe(false);
635 | });
636 |
637 | it("rejects current_bloom_level above 6", () => {
638 |   const result = bloomsClimbStateSchema.safeParse({
639 |     ...validState,
640 |     current_bloom_level: 7,
641 |   });
642 |   expect(result.success).toBe(false);
643 | });
644 |
645 | it("rejects non-integer current_bloom_level", () => {
646 |   const result = bloomsClimbStateSchema.safeParse({
647 |     ...validState,
648 |     current_bloom_level: 2.5,
649 |   });
650 |   expect(result.success).toBe(false);
651 | });
652 |
653 | it("accepts consecutive_correct_at_level at boundaries (0 and 3)", () => {
654 |   expect(
655 |     bloomsClimbStateSchema.safeParse({
656 |       ...validState,
657 |       consecutive_correct_at_level: 0,
658 |     }).success
659 |   ).toBe(true);
660 |   expect(
661 |     bloomsClimbStateSchema.safeParse({
662 |       ...validState,
663 |       consecutive_correct_at_level: 3,
664 |     }).success
665 |   ).toBe(true);
666 | });
667 |
668 | it("rejects consecutive_correct_at_level below 0", () => {
669 |   const result = bloomsClimbStateSchema.safeParse({
670 |     ...validState,
671 |     consecutive_correct_at_level: -1,
672 |   });
673 |   expect(result.success).toBe(false);
674 | });
675 |
676 | it("rejects consecutive_correct_at_level above 3", () => {
677 |   const result = bloomsClimbStateSchema.safeParse({
678 |     ...validState,
679 |     consecutive_correct_at_level: 4,
680 |   });
681 |   expect(result.success).toBe(false);
682 | });
683 |
684 | it("rejects transition with from_level below 1", () => {
685 |   const result = bloomsClimbStateSchema.safeParse({
686 |     ...validState,
687 |     bloom_transitions: [{ from_level: 0, to_level: 1, question_number: 1 }],
688 |   });
689 |   expect(result.success).toBe(false);
690 | });
691 |
692 | it("rejects transition with to_level above 6", () => {
693 |   const result = bloomsClimbStateSchema.safeParse({
694 |     ...validState,
695 |     bloom_transitions: [{ from_level: 6, to_level: 7, question_number: 1 }],
696 |   });
697 |   expect(result.success).toBe(false);
698 | });
699 |
700 | it("rejects transition with question_number below 1", () => {
701 |   const result = bloomsClimbStateSchema.safeParse({
702 |     ...validState,
703 |     bloom_transitions: [{ from_level: 1, to_level: 2, question_number: 0 }],

```

```

704 |     });
705 |     expect(result.success).toBe(false);
706 | });
707 |
708 | it("rejects missing required fields", () => {
709 |     expect(bloomsClimbStateSchema.safeParse({}).success).toBe(false);
710 |     expect(
711 |         bloomsClimbStateSchema.safeParse({ current_bloom_level: 2 }).success
712 |     ).toBe(false);
713 | });
714 | });
715 |

```

FILE: src/___tests___/unit/quizGrader.test.ts

TYPE: .ts | LINES: 1-194 | SHA256: 1388b0359469cd722e5c8deafb32f58703810f7e9345fbd40f1bb9ae686dcc5e

```

1 | import { describe, it, expect } from "vitest";
2 | import { gradeQuestion, gradeQuiz } from "@lib/quizGrader";
3 | import type { GradableQuestion } from "@lib/quizGrader";
4 |
5 | describe("Quiz Grader", () => {
6 |     describe("gradeQuestion", () => {
7 |         it("grades mcq_single correctly when answer matches", () => {
8 |             const q: GradableQuestion = {
9 |                 id: "q1",
10 |                 question_type: "mcq_single",
11 |                 correct_answer: "Paris",
12 |                 points: 5,
13 |             };
14 |             const result = gradeQuestion(q, "Paris");
15 |             expect(result.score).toBe(5);
16 |             expect(result.isCorrect).toBe(true);
17 |             expect(result.autoGradable).toBe(true);
18 |         });
19 |
20 |         it("grades mcq_single as 0 when answer is wrong", () => {
21 |             const q: GradableQuestion = {
22 |                 id: "q1",
23 |                 question_type: "mcq_single",
24 |                 correct_answer: "Paris",
25 |                 points: 5,
26 |             };
27 |             const result = gradeQuestion(q, "London");
28 |             expect(result.score).toBe(0);
29 |             expect(result.isCorrect).toBe(false);
30 |         });
31 |
32 |         it("grades mcq_single case-insensitively", () => {
33 |             const q: GradableQuestion = {
34 |                 id: "q1",
35 |                 question_type: "mcq_single",
36 |                 correct_answer: "Paris",
37 |                 points: 5,
38 |             };
39 |             expect(gradeQuestion(q, "paris").isCorrect).toBe(true);
40 |             expect(gradeQuestion(q, "PARIS").isCorrect).toBe(true);
41 |         });
42 |
43 |         it("grades true_false correctly", () => {
44 |             const q: GradableQuestion = {
45 |                 id: "q2",
46 |                 question_type: "true_false",
47 |                 correct_answer: "True",
48 |                 points: 2,
49 |             };
50 |             expect(gradeQuestion(q, "True").score).toBe(2);
51 |             expect(gradeQuestion(q, "False").score).toBe(0);
52 |         });
53 |
54 |         it("grades mcq_multi with exact match", () => {
55 |             const q: GradableQuestion = {
56 |                 id: "q3",
57 |                 question_type: "mcq_multi",
58 |                 correct_answer: ["A", "C"],
59 |                 points: 10,
60 |             };
61 |             expect(gradeQuestion(q, ["A", "C"]).score).toBe(10);
62 |             expect(gradeQuestion(q, ["C", "A"]).score).toBe(10); // order doesn't matter
63 |             expect(gradeQuestion(q, ["A"]).score).toBe(0); // partial = 0
64 |             expect(gradeQuestion(q, ["A", "B", "C"]).score).toBe(0); // extra = 0
65 |         });
66 |
67 |         it("grades fill_blank correctly", () => {
68 |             const q: GradableQuestion = {
69 |                 id: "q4",
70 |                 question_type: "fill_blank",
71 |                 correct_answer: "photosynthesis",
72 |                 points: 3,
73 |             };
74 |             expect(gradeQuestion(q, "photosynthesis").score).toBe(3);
75 |             expect(gradeQuestion(q, "Photosynthesis").score).toBe(3);
76 |             expect(gradeQuestion(q, "respiration").score).toBe(0);
77 |         });
78 |
79 |         it("flags short_answer as not auto-gradable", () => {
80 |             const q: GradableQuestion = {
81 |                 id: "q5",
82 |                 question_type: "short_answer",
83 |                 correct_answer: "any",
84 |                 points: 10,
85 |             };
86 |             const result = gradeQuestion(q, "student answer");
87 |             expect(result.autoGradable).toBe(false);
88 |             expect(result.score).toBe(0);

```



```

89 |     expect(result.isCorrect).toBeNull();
90 |   });
91 |
92 |   it("returns 0 for undefined answer", () => {
93 |     const q: GradableQuestion = {
94 |       id: "q1",
95 |       question_type: "mcq_single",
96 |       correct_answer: "A",
97 |       points: 5,
98 |     };
99 |     const result = gradeQuestion(q, undefined);
100 |     expect(result.score).toBe(0);
101 |     expect(result.isCorrect).toBe(false);
102 |   });
103 | });
104 |
105 | describe("gradeQuiz", () => {
106 |   const questions: GradableQuestion[] = [
107 |     { id: "q1", question_type: "mcq_single", correct_answer: "A", points: 5 },
108 |     {
109 |       id: "q2",
110 |       question_type: "true_false",
111 |       correct_answer: "True",
112 |       points: 2,
113 |     },
114 |     {
115 |       id: "q3",
116 |       question_type: "fill_blank",
117 |       correct_answer: "hello",
118 |       points: 3,
119 |     },
120 |     {
121 |       id: "q4",
122 |       question_type: "short_answer",
123 |       correct_answer: "",
124 |       points: 10,
125 |     },
126 |   ];
127 |
128 |   it("calculates auto score for all auto-gradable questions", () => {
129 |     const answers = { q1: "A", q2: "True", q3: "hello", q4: "some answer" };
130 |     const result = gradeQuiz(questions, answers);
131 |     expect(result.autoScore).toBe(10); // 5 + 2 + 3
132 |     expect(result.totalAutoGradablePoints).toBe(10);
133 |     expect(result.hasManualQuestions).toBe(true);
134 |     expect(result.gradingStatus).toBe("pending_manual");
135 |     expect(result.scorePercent).toBeNull(); // can't compute until manual grading
136 |   });
137 |
138 |   it("returns fully_graded when no short answer questions", () => {
139 |     const autoOnly: GradableQuestion[] = [
140 |       {
141 |         id: "q1",
142 |         question_type: "mcq_single",
143 |         correct_answer: "A",
144 |         points: 5,
145 |       },
146 |       {
147 |         id: "q2",
148 |         question_type: "true_false",
149 |         correct_answer: "True",
150 |         points: 5,
151 |       },
152 |     ];
153 |     const answers = { q1: "A", q2: "True" };
154 |     const result = gradeQuiz(autoOnly, answers);
155 |     expect(result.gradingStatus).toBe("fully_graded");
156 |     expect(result.scorePercent).toBe(100);
157 |     expect(result.autoScore).toBe(10);
158 |   });
159 |
160 |   it("calculates correct score percent", () => {
161 |     const autoOnly: GradableQuestion[] = [
162 |       {
163 |         id: "q1",
164 |         question_type: "mcq_single",
165 |         correct_answer: "A",
166 |         points: 10,
167 |       },
168 |       {
169 |         id: "q2",
170 |         question_type: "mcq_single",
171 |         correct_answer: "B",
172 |         points: 10,
173 |       },
174 |     ];
175 |     const answers = { q1: "A", q2: "C" }; // 1 correct, 1 wrong
176 |     const result = gradeQuiz(autoOnly, answers);
177 |     expect(result.scorePercent).toBe(50);
178 |   });
179 |
180 |   it("handles empty answers", () => {
181 |     const result = gradeQuiz(questions, {});
182 |     expect(result.autoScore).toBe(0);
183 |   });
184 |
185 |   it("returns per-question results", () => {
186 |     const answers = { q1: "A", q2: "False" };
187 |     const result = gradeQuiz(questions, answers);
188 |     expect(result.perQuestion).toHaveLength(4);
189 |     expect(result.perQuestion[0]!.isCorrect).toBe(true);
190 |     expect(result.perQuestion[1]!.isCorrect).toBe(false);
191 |   });
192 | });

```

```
193 | });
194 |
```

FILE: src/__tests__/unit/quizXpAward.test.ts

TYPE: .ts | LINES: 1-17 | SHA256: 07e845a733a31a3d666e74b6ae424f76e405c509c9a18f03865722dc5dfaa085

```
1 | import { describe, it, expect } from "vitest";
2 | import { QUIZ_XP_ON_TIME, QUIZ_XP_LATE } from "@lib/quizXpAward";
3 |
4 | describe("Quiz XP Award Constants", () => {
5 |   it("awards 50 XP for on-time completion", () => {
6 |     expect(QUIZ_XP_ON_TIME).toBe(50);
7 |   });
8 |
9 |   it("awards 25 XP for late completion", () => {
10 |     expect(QUIZ_XP_LATE).toBe(25);
11 |   });
12 |
13 |   it("on-time XP is greater than late XP", () => {
14 |     expect(QUIZ_XP_ON_TIME).toBeGreaterThan(QUIZ_XP_LATE);
15 |   });
16 | });
17 |
```

FILE: src/__tests__/unit/realtimeStatusBanner.test.tsx

TYPE: .tsx | LINES: 1-26 | SHA256: 119421687b67b4bd6b8d47b80a2ea28f686caca18b12bffc161ee4519f8d739b

```
1 | import { describe, it, expect } from "vitest";
2 | import { render, screen } from "@testing-library/react";
3 | import RealtimeStatusBanner from "@components/shared/RealtimeStatusBanner";
4 |
5 | describe("RealtimeStatusBanner", () => {
6 |   it("renders nothing when isLive is true", () => {
7 |     const { container } = render(<RealtimeStatusBanner isLive={true} />);
8 |     expect(container.firstChild).toBeNull();
9 |   });
10 |
11 |   it("renders the paused banner when isLive is false", () => {
12 |     render(<RealtimeStatusBanner isLive={false} />);
13 |     expect(screen.getByText(/live updates paused/i)).toBeInTheDocument();
14 |   });
15 |
16 |   it("includes reconnecting text in the banner", () => {
17 |     render(<RealtimeStatusBanner isLive={false} />);
18 |     expect(screen.getByText(/reconnecting/i)).toBeInTheDocument();
19 |   });
20 |
21 |   it('has role="status" for accessibility', () => {
22 |     render(<RealtimeStatusBanner isLive={false} />);
23 |     expect(screen.getByRole("status")).toBeInTheDocument();
24 |   });
25 | });
26 |
```

FILE: src/__tests__/unit/reducedMotion.test.ts

TYPE: .ts | LINES: 1-60 | SHA256: f51d83826da9148e2e430c1d8eadeca000f90714f0d2babbbfbf18468ba00f92a

```
1 | import { describe, it, expect } from "vitest";
2 | import { readFileSync } from "fs";
3 | import { resolve } from "path";
4 |
5 | const cssContent = readFileSync(resolve(__dirname, "../../index.css"), "utf-8");
6 | const appContent = readFileSync(resolve(__dirname, "../../App.tsx"), "utf-8");
7 |
8 | const ALL_ANIMATION_CLASSES = [
9 |   "animate-shimmer",
10 |   "animate-xp-pulse",
11 |   "animate-badge-pop",
12 |   "animate-float",
13 |   "animate-streak-flame",
14 |   "animate-fade-in-up",
15 |   "animate-node-unlock",
16 |   "animate-mystery-reveal",
17 | ];
18 |
19 | describe("Reduced motion support", () => {
20 |   describe("CSS @media (prefers-reduced-motion: reduce)", () => {
21 |     it("contains a prefers-reduced-motion media query block", () => {
22 |       expect(cssContent).toContain("@media (prefers-reduced-motion: reduce)");
23 |     });
24 |
25 |     it("disables animation for all 8 custom animation classes", () => {
26 |       const reducedMotionMatch = cssContent.match(
27 |         /@media\s*\(\(prefers-reduced-motion:\s*reduce\)\s*\{([^\}]*\{[^\}]*\})\s*\}/s
28 |       );
29 |       expect(reducedMotionMatch).not.toBeNull();
30 |       const block = reducedMotionMatch![1];
31 |
32 |       for (const cls of ALL_ANIMATION_CLASSES) {
33 |         expect(block).toContain(`.${cls}`);
34 |       }
35 |     });
36 |
37 |     it("sets animation: none inside the reduced-motion block", () => {
38 |       const reducedMotionMatch = cssContent.match(
39 |         /@media\s*\(\(prefers-reduced-motion:\s*reduce\)\s*\{([^\}]*\{[^\}]*\})\s*\}/s
40 |       );
41 |       expect(reducedMotionMatch).not.toBeNull();
42 |       const block = reducedMotionMatch![1];
43 |       expect(block).toContain("animation: none");
44 |     });
45 |   });
46 | }
```

```

47 | describe("Framer Motion global config", () => {
48 |   it("imports MotionConfig from framer-motion in App.tsx", () => {
49 |     expect(appContent).toContain("MotionConfig");
50 |     expect(appContent).toMatch(
51 |       /import\s*{\^[^}]*MotionConfig\^[^}]*}\s*from\s*['"]framer-motion['"]\/
52 |     );
53 |   });
54 |
55 |   it('wraps the app with MotionConfig reducedMotion="user"', () => {
56 |     expect(appContent).toMatch(/<MotionConfig\s+reducedMotion="user"/);
57 |   });
58 | });
59 | });
60 |

```

FILE: src/__tests__/unit/reflectionTemplateSelector.test.tsx

TYPE: .tsx | LINES: 1-279 | SHA256: d357c75c9dd1fd0d5769ac999044ea2aef053f5ee3792dea4d4c87b51459beaf

[illegible]

[illegible]


```

12 | votesAgainst: 0,
13 | createdAt: new Date().toISOString(),
14 | resolvedAt: null,
15 | teacherOverride: false,
16 | };
17 |
18 | describe("ReplacementVoteCard", () => {
19 |   describe("vote initiation", () => {
20 |     it("shows initiation UI for captain with inactive members", () => {
21 |       render(
22 |         <ReplacementVoteCard
23 |           vote={null}
24 |           isCaptain
25 |           isTeacher={false}
26 |           totalMembers={4}
27 |           inactiveMembers={[{ id: "user-2", name: "Jane Doe" }]}
28 |           onInitiateVote={vi.fn()}
29 |         />
30 |       );
31 |       expect(screen.getByText("Replacement Vote")).toBeDefined();
32 |       expect(screen.getByLabelText("Select inactive member")).toBeDefined();
33 |     });
34 |
35 |     it("hides initiation UI for non-captain", () => {
36 |       const { container } = render(
37 |         <ReplacementVoteCard
38 |           vote={null}
39 |           isCaptain={false}
40 |           isTeacher={false}
41 |           totalMembers={4}
42 |           inactiveMembers={[{ id: "user-2", name: "Jane Doe" }]}
43 |         />
44 |       );
45 |       expect(container.innerHTML).toBe("");
46 |     });
47 |
48 |     it("hides initiation UI when no inactive members", () => {
49 |       const { container } = render(
50 |         <ReplacementVoteCard
51 |           vote={null}
52 |           isCaptain
53 |           isTeacher={false}
54 |           totalMembers={4}
55 |           inactiveMembers=[]
56 |         />
57 |       );
58 |       expect(container.innerHTML).toBe("");
59 |     });
60 |   });
61 |
62 |   describe("vote casting", () => {
63 |     it("shows approve and reject buttons for members", () => {
64 |       render(
65 |         <ReplacementVoteCard
66 |           vote={baseVote}
67 |           isCaptain={false}
68 |           isTeacher={false}
69 |           totalMembers={4}
70 |           onCastVote={vi.fn()}
71 |         />
72 |       );
73 |       expect(screen.getByText("Approve")).toBeDefined();
74 |       expect(screen.getByText("Reject")).toBeDefined();
75 |     });
76 |
77 |     it("calls onCastVote with true for approve", () => {
78 |       const onCastVote = vi.fn();
79 |       render(
80 |         <ReplacementVoteCard
81 |           vote={baseVote}
82 |           isCaptain={false}
83 |           isTeacher={false}
84 |           totalMembers={4}
85 |           onCastVote={onCastVote}
86 |         />
87 |       );
88 |       fireEvent.click(screen.getByText("Approve"));
89 |       expect(onCastVote).toHaveBeenCalledWith("vote-1", true);
90 |     });
91 |
92 |     it("shows 'already voted' message when hasVoted", () => {
93 |       render(
94 |         <ReplacementVoteCard
95 |           vote={baseVote}
96 |           isCaptain={false}
97 |           isTeacher={false}
98 |           hasVoted
99 |           totalMembers={4}
100 |         />
101 |       );
102 |       expect(screen.getByText("You have already voted")).toBeDefined();
103 |     });
104 |   });
105 |
106 |   describe("teacher override", () => {
107 |     it("shows override buttons for teacher", () => {
108 |       render(
109 |         <ReplacementVoteCard
110 |           vote={baseVote}
111 |           isCaptain={false}
112 |           isTeacher
113 |           totalMembers={4}
114 |           onTeacherOverride={vi.fn()}
115 |         />

```

```

116 |     );
117 |     expect(screen.getByText("Override: Approve")).toBeDefined();
118 |     expect(screen.getByText("Override: Reject")).toBeDefined();
119 | });
120 |
121 | it("shows teacher override badge when applied", () => {
122 |   render(
123 |     <ReplacementVoteCard
124 |       vote={{ ...baseVote, teacherOverride: true, status: "approved" }}
125 |       isCaptain={false}
126 |       isTeacher={false}
127 |       totalMembers={4}
128 |     />
129 |   );
130 |   expect(screen.getByText("Teacher override applied")).toBeDefined();
131 | });
132 | });
133 |
134 | describe("vote display", () => {
135 |   it("shows target member name", () => {
136 |     render(
137 |       <ReplacementVoteCard
138 |         vote={baseVote}
139 |         isCaptain={false}
140 |         isTeacher={false}
141 |         totalMembers={4}
142 |       />
143 |     );
144 |     expect(screen.getByText("Jane Doe")).toBeDefined();
145 |   });
146 |
147 |   it("shows vote counts", () => {
148 |     render(
149 |       <ReplacementVoteCard
150 |         vote={{ ...baseVote, votesFor: 2, votesAgainst: 1 }}
151 |         isCaptain={false}
152 |         isTeacher={false}
153 |         totalMembers={4}
154 |       />
155 |     );
156 |     expect(screen.getByText("2")).toBeDefined();
157 |     expect(screen.getByText("1")).toBeDefined();
158 |   });
159 | });
160 | });
161 |

```

FILE: [src/__tests__/unit/reviewCompletionXP.test.ts](#)

TYPE: .ts | LINES: 1-81 | SHA256: e36c8d19709b3feb44ff1360a2346f62e82285e28e17b18d4b40128ea02609be

```

1 | import { describe, it, expect } from "vitest";
2 | import { isReviewCycleComplete } from "@lib/plannerUtils";
3 |
4 | describe("Review Completion XP Flow", () => {
5 |   describe("isReviewCycleComplete", () => {
6 |     it("returns true when all 3 intervals (1, 3, 7) are completed", () => {
7 |       const reviews = [
8 |         { intervalDays: 1, status: "completed" },
9 |         { intervalDays: 3, status: "completed" },
10 |        { intervalDays: 7, status: "completed" },
11 |      ];
12 |      expect(isReviewCycleComplete(reviews)).toBe(true);
13 |    });
14 |
15 |     it("returns false when only 2 of 3 intervals are completed", () => {
16 |       const reviews = [
17 |         { intervalDays: 1, status: "completed" },
18 |         { intervalDays: 3, status: "completed" },
19 |         { intervalDays: 7, status: "pending" },
20 |       ];
21 |       expect(isReviewCycleComplete(reviews)).toBe(false);
22 |     });
23 |
24 |     it("returns false when no intervals are completed", () => {
25 |       const reviews = [
26 |         { intervalDays: 1, status: "pending" },
27 |         { intervalDays: 3, status: "pending" },
28 |         { intervalDays: 7, status: "pending" },
29 |       ];
30 |       expect(isReviewCycleComplete(reviews)).toBe(false);
31 |     });
32 |
33 |     it("returns false for empty reviews array", () => {
34 |       expect(isReviewCycleComplete([])).toBe(false);
35 |     });
36 |
37 |     it("returns false when intervals are skipped instead of completed", () => {
38 |       const reviews = [
39 |         { intervalDays: 1, status: "completed" },
40 |         { intervalDays: 3, status: "skipped" },
41 |         { intervalDays: 7, status: "completed" },
42 |       ];
43 |       expect(isReviewCycleComplete(reviews)).toBe(false);
44 |     });
45 |
46 |     it("returns true when there are extra review rows beyond the 3 required", () => {
47 |       const reviews = [
48 |         { intervalDays: 1, status: "completed" },
49 |         { intervalDays: 1, status: "completed" },
50 |         { intervalDays: 3, status: "completed" },
51 |         { intervalDays: 7, status: "completed" },
52 |       ];
53 |       expect(isReviewCycleComplete(reviews)).toBe(true);
54 |     });

```

```

55 | it("returns true when at least one of each interval is completed among mixed statuses", () => {
56 |     const reviews = [
57 |         { intervalDays: 1, status: "skipped" },
58 |         { intervalDays: 1, status: "completed" },
59 |         { intervalDays: 3, status: "completed" },
60 |         { intervalDays: 7, status: "pending" },
61 |         { intervalDays: 7, status: "completed" },
62 |     ];
63 |     expect(isReviewCycleComplete(reviews)).toBe(true);
64 | });
65 |
66 | it("returns false when only interval 1 is completed", () => {
67 |     const reviews = [{ intervalDays: 1, status: "completed" }];
68 |     expect(isReviewCycleComplete(reviews)).toBe(false);
69 | });
70 |
71 | it("returns false when interval 7 is missing entirely", () => {
72 |     const reviews = [
73 |         { intervalDays: 1, status: "completed" },
74 |         { intervalDays: 3, status: "completed" },
75 |     ];
76 |     expect(isReviewCycleComplete(reviews)).toBe(false);
77 | });
78 | });
79 | });
80 | });
81 |

```

FILE: src/__tests__/unit/reviewScheduleList.test.tsx

```
TYPE: .tsx | LINES: 1-312 | SHA256: 0a4ba55ae0d1edf408b1a51add6892e882ca715803cccbca68434b20ecb7bd2
```

```

1 // =====
2 // ReviewScheduleList - Unit tests
3 // Validates: Task 18.3 - List of pending reviews with Start/Skip actions
4 // =====
5
6 import { describe, it, expect, vi } from "vitest";
7 import { render, screen, within } from "@testing-library/react";
8 import userEvent from "@testing-library/user-event";
9 import { ReviewScheduleList } from "@components/shared/ReviewScheduleList";
10 import type { ReviewSchedule } from "@types/planner";
11
12 // %% Helpers %%%%%%%%%%%%%%%
13
14 function makeReview(overrides: Partial<ReviewSchedule> = {}): ReviewSchedule {
15   return {
16     id: "rev-1",
17     studentId: "stu-1",
18     cloId: "clo-abc",
19     courseId: "course-xyz",
20     sourceSessionId: "sess-1",
21     reviewDate: "2025-07-01",
22     intervalDays: 1,
23     status: "pending",
24     reviewSessionId: null,
25     createdAt: "2025-06-30T10:00:00Z",
26     updatedAt: "2025-06-30T10:00:00Z",
27     ...overrides,
28   };
29 }
30
31 const noop = () => {};
32
33 // %% Empty state %%%%%%%%%%%%%%%
34
35 describe("ReviewScheduleList - Empty state", () => {
36   it("renders empty state when no reviews are provided", () => {
37     render(
38       <ReviewScheduleList
39         reviews={[]}
40         onStartReview={noop}
41         onSkipReview={noop}
42       />
43     );
44     expect(screen.getByText("No reviews scheduled")).toBeInTheDocument();
45     expect(
46       screen.getByText(/Complete study sessions linked to CLOs/)
47     ).toBeInTheDocument();
48   });
49
50   it("does not render the list container when empty", () => {
51     render(
52       <ReviewScheduleList
53         reviews={[]}
54         onStartReview={noop}
55         onSkipReview={noop}
56       />
57     );
58     expect(screen.queryByRole("list")).not.toBeInTheDocument();
59   });
60 });
61
62 // %% Pending reviews %%%%%%%%%%%%%%%
63
64 describe("ReviewScheduleList - Pending reviews", () => {
65   it("renders a list item for each review", () => {
66     const reviews = [
67       makeReview({ id: "r1", intervalDays: 1 }),
68       makeReview({ id: "r2", intervalDays: 3 }),
69       makeReview({ id: "r3", intervalDays: 7 }),
70     ];
71     render(
72       <ReviewScheduleList
73         reviews={reviews}

```


[illegible]

[illegible]

[illegible]

FILE: src/__tests__/unit/roleProfileScreens.test.tsx

TYPE: .tsx | LINES: 1-226 | SHA256: 208eb894006dec16829a63b13c8da938b13ff1b9af4dab58497aeceaf87ddab7

```

1 // @vitest-environment happy-dom
2
3 import { afterEach, beforeEach, describe, expect, it, vi } from "vitest";
4 import { cleanup, render, screen } from "@testing-library/react";
5 import { I18nextProvider } from "react-i18next";
6 import { MemoryRouter } from "react-router-dom";
7 import type { ReactNode } from "react";
8 import i18n from "@lib/i18n";
9 import type { Profile, UserRole } from "@types/app";
10
11 const state = vi.hoisted(() => ({
12   role: "admin" as UserRole,
13 }));
14
15 const baseProfile: Profile = {
16   id: "user-1",
17   email: "profile@example.test",
18   full_name: "Prototype User",
19   role: "admin",
20   institution_id: "institution-1",
21   avatar_url: null,
22   is_active: true,
23   onboarding_completed: true,
24   portfolio_public: false,
25   theme_preference: "system",
26   language_preference: "en",
27   notification_preferences: {},
28   last_seen_at: null,
29   tos_accepted_at: null,
30   tour_completed_at: null,
31   status: "active",
32   created_at: "2026-01-01T00:00:00Z",
33   department: "Computer Science",
34   designation: "Assistant Professor",
35   academic_rank: "Lecturer",
36   highest_degree: "PhD",
37 };
38
39 vi.mock("@/hooks/useAuth", () => ({

```

```

40 |     useAuth: () => ({
41 |       user: { id: "user-1" },
42 |       profile: { ...baseProfile, role: state.role },
43 |     }),
44 |   }));
45 |
46 | vi.mock("@providers/ThemeProvider", () => ({
47 |   useTheme: () => ({ theme: "system", setTheme: vi.fn() }),
48 | }));
49 |
50 | vi.mock("@hooks/useAdminDashboardAggregate", () => ({
51 |   useAdminDashboardAggregate: () => ({
52 |     data: {
53 |       totalUsers: 1240,
54 |       activeUsers: 1141,
55 |       totalPrograms: 18,
56 |       totalCourses: 96,
57 |       usersByRole: { student: 1240, teacher: 86 },
58 |     },
59 |     isPending: false,
60 |   }),
61 | }));
62 |
63 | vi.mock("@hooks/useTeacherDashboardAggregate", () => ({
64 |   useTeacherDashboardAggregate: () => ({
65 |     data: {
66 |       kpis: {
67 |         totalStudents: 84,
68 |         pendingSubmissions: 12,
69 |         gradedThisWeek: 31,
70 |         avgAttainment: 76,
71 |         atRiskCount: 3,
72 |       },
73 |       bloomsDistribution: [],
74 |     },
75 |     isPending: false,
76 |   }),
77 | }));
78 |
79 | vi.mock("@hooks/useParentDashboardAggregate", () => ({
80 |   useParentDashboardAggregate: () => ({
81 |     data: {
82 |       kpis: {
83 |         linkedChildren: 2,
84 |         totalCourses: 7,
85 |         avgAttainment: 82,
86 |         upcomingDeadlines: 3,
87 |       },
88 |       children: [
89 |         {
90 |           student_id: "student-1",
91 |           student_name: "Maya",
92 |           current_level: 4,
93 |           xp_total: 750,
94 |           current_streak: 12,
95 |           enrolled_courses: 4,
96 |           avg_attainment: 84,
97 |         },
98 |         {
99 |           student_id: "student-2",
100 |           student_name: "Yusuf",
101 |           current_level: 3,
102 |           xp_total: 620,
103 |           current_streak: 6,
104 |           enrolled_courses: 3,
105 |           avg_attainment: 80,
106 |         },
107 |       ],
108 |     },
109 |     isPending: false,
110 |   }),
111 | }));
112 |
113 | vi.mock("@hooks/useCourses", () => ({
114 |   useCourses: () => ({
115 |     data: {
116 |       count: 3,
117 |       page: 1,
118 |       pageSize: 6,
119 |       data: [
120 |         {
121 |           id: "course-1",
122 |           code: "CS101",
123 |           name: "Introduction to Programming",
124 |           semester: "Fall",
125 |           academic_year: "2026",
126 |         },
127 |       ],
128 |     },
129 |     isPending: false,
130 |   }),
131 |   useTeacherCourses: () => ({
132 |     data: {
133 |       count: 3,
134 |       page: 1,
135 |       pageSize: 6,
136 |       data: [
137 |         {
138 |           id: "course-1",
139 |           code: "CS101",
140 |           name: "Introduction to Programming",
141 |           semester: "Fall",
142 |           academic_year: "2026",
143 |         },

```

```

144 |     ],
145 |   },
146 |   isPending: false,
147 | },
148 | ));
149 |
150 | vi.mock("@/hooks/useCLOs", () => ({
151 |   useCLOs: () => ({
152 |     data: { count: 27, page: 1, pageSize: 1, data: [] },
153 |     isPending: false,
154 |   }),
155 | }));
156 |
157 | vi.mock("@/components/shared/AvatarUpload", () => ({
158 |   default: () => <div>Avatar upload control</div>,
159 | }));
160 |
161 | vi.mock("@/components/shared/EmailPreferencesSection", () => ({
162 |   default: () => <div>Email preference controls</div>,
163 | }));
164 |
165 | import AdminProfilePage from "@/pages/admin/settings/AdminProfilePage";
166 | import TeacherProfilePage from "@/pages/teacher/settings/TeacherProfilePage";
167 | import ParentProfilePage from "@/pages/parent/settings/ParentProfilePage";
168 |
169 | const renderPage = (page: ReactNode) =>
170 |   render(
171 |     <I18nextProvider i18n={i18n}>
172 |       <MemoryRouter>{page}</MemoryRouter>
173 |     </I18nextProvider>
174 |   );
175 |
176 | beforeEach(async () => {
177 |   await i18n.changeLanguage("en");
178 | });
179 |
180 | afterEach(cleanup);
181 |
182 | describe("prototype role profile screens", () => {
183 |   it("renders the institution admin profile with real aggregate values", () => {
184 |     state.role = "admin";
185 |     renderPage(<AdminProfilePage />);
186 |
187 |     expect(screen.getByText("Prototype User")).toBeInTheDocument();
188 |     expect(screen.getByText("1,240")).toBeInTheDocument();
189 |     expect(screen.getByText("Institution settings")).toBeInTheDocument();
190 |     expect(screen.getByText("Role & permissions")).toBeInTheDocument();
191 |     expect(screen.getByRole("link", { name: "Users" })).toHaveAttribute(
192 |       "href",
193 |       "/admin/users"
194 |     );
195 |   });
196 |
197 |   it("renders the teacher academic profile and teaching workspace", () => {
198 |     state.role = "teacher";
199 |     renderPage(<TeacherProfilePage />);
200 |
201 |     expect(screen.getAllByText("Assistant Professor").length).toBeGreaterThan(
202 |       0
203 |     );
204 |     expect(screen.getByText("84")).toBeInTheDocument();
205 |     expect(screen.getByText("Teaching impact")).toBeInTheDocument();
206 |     expect(
207 |       screen.getByText("AI autonomy for your classes")
208 |     ).toBeInTheDocument();
209 |     expect(screen.getByText("My classes")).toBeInTheDocument();
210 |   });
211 |
212 |   it("renders the parent profile with verified child context", () => {
213 |     state.role = "parent";
214 |     renderPage(<ParentProfilePage />);
215 |
216 |     expect(screen.getByText(/Guardian of Maya & Yusuf/)).toBeInTheDocument();
217 |     expect(screen.getByText("82%")).toBeInTheDocument();
218 |     expect(screen.getByText("Linked learners")).toBeInTheDocument();
219 |     expect(screen.getByText("Privacy & what you can see")).toBeInTheDocument();
220 |     expect(screen.getByRole("link", { name: /Support/i })).toHaveAttribute(
221 |       "href",
222 |       "/parent/support"
223 |     );
224 |   });
225 | });
226 |

```

FILE: src/__tests__/unit/routeGuard.test.tsx

TYPE: .tsx | LINES: 1-140 | SHA256: b678e63263d4a2391f4ee1470e8cda24432304d9918ff6969091d05c796227cf

```

1 | import { describe, it, expect, vi, beforeEach } from "vitest";
2 | import { render, screen } from "@testing-library/react";
3 | import { MemoryRouter, Routes, Route } from "react-router-dom";
4 | import RouteGuard from "@/router/RouteGuard";
5 | import type { UserRole } from "@/types/app";
6 |
7 | // -----
8 | // Mock useAuth
9 | // -----
10 | const mockUseAuth = vi.fn();
11 |
12 | vi.mock("@/hooks/useAuth", () => ({
13 |   useAuth: () => mockUseAuth(),
14 | }));
15 |
16 | // -----
17 | // Helpers

```

```

18 // -----
19 const buildAuth = (overrides: Record<string, unknown> = {}) => ({
20   user: null,
21   profile: null,
22   role: null as UserRole | null,
23   institutionId: null,
24   isLoading: false,
25   signIn: vi.fn(),
26   signOut: vi.fn(),
27   resetPassword: vi.fn(),
28   ...overrides,
29 });
30
31 const renderGuard = (allowedRoles: UserRole[], initialPath: string) => {
32   return render(
33     <MemoryRouter initialEntries={[initialPath]}>
34       <Routes>
35         <Route path="/login" element={<div>Login Page</div>} />
36         <Route path="/admin/dashboard" element={<div>Admin Dashboard</div>} />
37         <Route
38           path="/student/dashboard"
39           element={<div>Student Dashboard</div>}
40         />
41         <Route
42           path="/teacher/dashboard"
43           element={<div>Teacher Dashboard</div>}
44         />
45         <Route
46           path="/coordinator/dashboard"
47           element={<div>Coordinator Dashboard</div>}
48         />
49         <Route path="/parent/dashboard" element={<div>Parent Dashboard</div>} />
50         <Route
51           path="/admin/"
52           element={
53             <RouteGuard allowedRoles={allowedRoles}>
54               <div>Protected Admin Content</div>
55             </RouteGuard>
56           }
57         />
58         <Route
59           path="/student/"
60           element={
61             <RouteGuard allowedRoles={allowedRoles}>
62               <div>Protected Student Content</div>
63             </RouteGuard>
64           }
65         />
66       </Routes>
67     </MemoryRouter>
68   );
69 };
70
71 // -----
72 // Tests
73 // -----
74 describe("RouteGuard", () => {
75   beforeEach(() => {
76     vi.clearAllMocks();
77   });
78
79   it("shows loading spinner while auth is loading", () => {
80     mockUseAuth.mockReturnValue(buildAuth({ isLoading: true }));
81     renderGuard(["admin"], "/admin/test");
82     expect(
83       screen.queryByText("Protected Admin Content")
84     ).not.toBeInTheDocument();
85     expect(screen.queryByText("Login Page")).not.toBeInTheDocument();
86   });
87
88   it("redirects to /login when user is not authenticated", () => {
89     mockUseAuth.mockReturnValue(buildAuth());
90     renderGuard(["admin"], "/admin/test");
91     expect(screen.getByText("Login Page")).toBeInTheDocument();
92   });
93
94   it("renders children when user has an allowed role", () => {
95     mockUseAuth.mockReturnValue(
96       buildAuth({ user: { id: "1" }, role: "admin" })
97     );
98     renderGuard(["admin"], "/admin/test");
99     expect(screen.getByText("Protected Admin Content")).toBeInTheDocument();
100   });
101
102   it("redirects to role dashboard when user has wrong role", () => {
103     mockUseAuth.mockReturnValue(
104       buildAuth({ user: { id: "1" }, role: "student" })
105     );
106     renderGuard(["admin"], "/admin/test");
107     expect(screen.getByText("Student Dashboard")).toBeInTheDocument();
108   });
109
110   it("redirects teacher to teacher dashboard when accessing admin route", () => {
111     mockUseAuth.mockReturnValue(
112       buildAuth({ user: { id: "1" }, role: "teacher" })
113     );
114     renderGuard(["admin"], "/admin/test");
115     expect(screen.getByText("Teacher Dashboard")).toBeInTheDocument();
116   });
117
118   it("redirects coordinator to coordinator dashboard when accessing admin route", () => {
119     mockUseAuth.mockReturnValue(
120       buildAuth({ user: { id: "1" }, role: "coordinator" })
121     );

```

```

122 |     renderGuard(["admin"], "/admin/test");
123 |     expect(screen.getByText("Coordinator Dashboard")).toBeInTheDocument();
124 | });
125 |
126 | it("redirects parent to parent dashboard when accessing student route", () => {
127 |     mockUseAuth.mockReturnValue(
128 |         buildAuth({ user: { id: "1" }, role: "parent" })
129 |     );
130 |     renderGuard(["student"], "/student/test");
131 |     expect(screen.getByText("Parent Dashboard")).toBeInTheDocument();
132 | });
133 |
134 | it("redirects to /login when user exists but role is null", () => {
135 |     mockUseAuth.mockReturnValue(buildAuth({ user: { id: "1" }, role: null }));
136 |     renderGuard(["admin"], "/admin/test");
137 |     expect(screen.getByText("Login Page")).toBeInTheDocument();
138 | });
139 | });
140 |

```

FILE: src/___tests___/unit/rubricPreview.test.tsx

TYPE: .tsx | LINES: 1-376 | SHA256: 50d81074343e2c0f701a6970e9bcfb47979b6eee6a1b667942976bf76ae8afa6

```

1 | import { describe, it, expect, vi, beforeEach } from "vitest";
2 | import { render, screen, within } from "@testing-library/react";
3 | import userEvent from "@testing-library/user-event";
4 | import RubricPreview from "@components/shared/RubricPreview";
5 | import RubricPreviewDialog from "@components/shared/RubricPreviewDialog";
6 | import type { RubricWithCriteria } from "@hooks/useRubrics";
7 |
8 | // -----
9 | // RubricPreviewDialog wiring (mocks must precede its import)
10 | //
11 | // The dialog loads its rubric through `useRubric`; mock the hook module so the
12 | // dialog renders the fixture without any Supabase/network call. Only the
13 | // runtime `useRubric` export is mocked – the existing RubricPreview tests above
14 | // rely solely on the (type-only) `RubricWithCriteria` import, which is erased.
15 | // -----
16 |
17 | const mockUseRubric = vi.fn<
18 |     () => {
19 |         data: RubricWithCriteria | null | undefined;
20 |         isLoading: boolean;
21 |         isError: boolean;
22 |     }
23 | >();
24 |
25 | vi.mock("@hooks/useRubrics", () => ({
26 |     useRubric: () => mockUseRubric(),
27 | }));
28 |
29 | // happy-dom does not implement ResizeObserver, which Radix Dialog relies on.
30 | class MockResizeObserver {
31 |     observe() {}
32 |     unobserve() {}
33 |     disconnect() {}
34 | }
35 | vi.stubGlobal("ResizeObserver", MockResizeObserver);
36 |
37 | // -----
38 | // Test fixtures
39 | // -----
40 |
41 | const makeRubric = (
42 |     overrides?: Partial<RubricWithCriteria>
43 | ): RubricWithCriteria => ({
44 |     id: "rubric-1",
45 |     title: "Essay Rubric",
46 |     clo_id: "clo-1",
47 |     description: null,
48 |     created_by: null,
49 |     is_template: false,
50 |     created_at: "2024-01-01T00:00:00Z",
51 |     criteria: [
52 |         {
53 |             id: "c1",
54 |             rubric_id: "rubric-1",
55 |             criterion_name: "Clarity",
56 |             sort_order: 0,
57 |             levels: [
58 |                 { label: "Developing", description: "Needs improvement", points: 1 },
59 |                 { label: "Proficient", description: "Meets expectations", points: 3 },
60 |                 { label: "Exemplary", description: "Exceeds expectations", points: 5 },
61 |             ],
62 |             max_points: 5,
63 |         },
64 |         {
65 |             id: "c2",
66 |             rubric_id: "rubric-1",
67 |             criterion_name: "Grammar",
68 |             sort_order: 1,
69 |             levels: [
70 |                 { label: "Developing", description: "Many errors", points: 1 },
71 |                 { label: "Proficient", description: "Few errors", points: 3 },
72 |                 { label: "Exemplary", description: "No errors", points: 5 },
73 |             ],
74 |             max_points: 5,
75 |         },
76 |     ],
77 |     ...overrides,
78 | });
79 |
80 | // -----
81 | // Tests

```



```

82 // -----
83
84 describe("RubricPreview", () => {
85   it("renders the rubric title", () => {
86     render(<RubricPreview rubric={makeRubric()} />);
87     expect(screen.getByText("Essay Rubric")).toBeInTheDocument();
88   });
89
90   it("renders criterion names", () => {
91     render(<RubricPreview rubric={makeRubric()} />);
92     expect(screen.getByText("Clarity")).toBeInTheDocument();
93     expect(screen.getByText("Grammar")).toBeInTheDocument();
94   });
95
96   it("renders level labels in the header", () => {
97     render(<RubricPreview rubric={makeRubric()} />);
98     expect(screen.getByText("Developing")).toBeInTheDocument();
99     expect(screen.getByText("Proficient")).toBeInTheDocument();
100    expect(screen.getByText("Exemplary")).toBeInTheDocument();
101  });
102
103   it("renders level descriptions", () => {
104     render(<RubricPreview rubric={makeRubric()} />);
105     expect(screen.getByText("Needs improvement")).toBeInTheDocument();
106     expect(screen.getByText("Meets expectations")).toBeInTheDocument();
107     expect(screen.getByText("No errors")).toBeInTheDocument();
108   });
109
110   it("renders points badges for each cell", () => {
111     render(<RubricPreview rubric={makeRubric()} />);
112     // 6 cells total: 2 criteria x 3 levels. 1pt cells = 2, 3pt cells = 2, 5pt cells = 2
113     const onePtBadges = screen.getAllByText("1 pt");
114     expect(onePtBadges).toHaveLength(2);
115     const fivePtBadges = screen.getAllByText("5 pts");
116     expect(fivePtBadges).toHaveLength(2);
117   });
118
119   it("renders total max score in footer", () => {
120     render(<RubricPreview rubric={makeRubric()} />);
121     // Total = 5 + 5 = 10
122     expect(screen.getByText("10")).toBeInTheDocument();
123   });
124
125   it("shows empty state when criteria array is empty", () => {
126     render(<RubricPreview rubric={makeRubric({ criteria: [] })} />);
127     expect(
128       screen.getByText("No criteria defined for this rubric.")
129     ).toBeInTheDocument();
130   });
131
132   it("shows empty state when levels array is empty", () => {
133     const rubric = makeRubric({
134       criteria: [
135         {
136           id: "c1",
137           rubric_id: "rubric-1",
138           criterion_name: "Test",
139           sort_order: 0,
140           levels: [],
141           max_points: 0,
142         },
143       ],
144     });
145     render(<RubricPreview rubric={rubric} />);
146     expect(
147       screen.getByText("No performance levels defined.")
148     ).toBeInTheDocument();
149   });
150
151   it("highlights selected cells when selectedCells is provided", () => {
152     const { container } = render(
153       <RubricPreview
154         rubric={makeRubric()}
155         selectedCells={{ c1: 2 }} // Exemplary for Clarity
156       />
157     );
158
159     // The selected cell should have the blue highlight classes
160     const selectedCell = container.querySelector(
161       ".bg-blue-100.border-blue-500"
162     );
163     expect(selectedCell).toBeInTheDocument();
164   });
165
166   it("makes cells clickable when onCellClick is provided", async () => {
167     const user = userEvent.setup();
168     const handleClick = vi.fn();
169
170     render(<RubricPreview rubric={makeRubric()} onCellClick={handleClick} />);
171
172     // Click on the "Needs improvement" cell (criterion c1, level 0)
173     await user.click(screen.getByText("Needs improvement"));
174     expect(handleClick).toHaveBeenCalledWith("c1", 0);
175   });
176
177   it("does not make cells clickable when onCellClick is not provided", () => {
178     const { container } = render(<RubricPreview rubric={makeRubric()} />);
179     const clickableCells = container.querySelectorAll('[role="button"]');
180     expect(clickableCells).toHaveLength(0);
181   });
182
183   it("supports keyboard activation on interactive cells", async () => {
184     const user = userEvent.setup();
185     const handleClick = vi.fn();

```

```

186 |     render(<RubricPreview rubric={makeRubric()} onClick={handleClick} />);
187 |
188 |
189 |     const cell = screen.getByText("Meets expectations").closest("td");
190 |     cell.focus();
191 |     await user.keyboard("{Enter}");
192 |     expect(handleClick).toHaveBeenCalledWith("c1", 1);
193 |   });
194 |
195 |   it("sorts criteria by sort_order", () => {
196 |     const rubric = makeRubric({
197 |       criteria: [
198 |         {
199 |           id: "c2",
200 |           rubric_id: "rubric-1",
201 |           criterion_name: "Second",
202 |           sort_order: 1,
203 |           levels: [{ label: "L1", description: "Desc", points: 1 }],
204 |           max_points: 1,
205 |         },
206 |         {
207 |           id: "c1",
208 |           rubric_id: "rubric-1",
209 |           criterion_name: "First",
210 |           sort_order: 0,
211 |           levels: [{ label: "L1", description: "Desc", points: 1 }],
212 |           max_points: 1,
213 |         },
214 |       ],
215 |     });
216 |
217 |     render(<RubricPreview rubric={rubric} />);
218 |     const criterionCells = screen.getAllByText(/First|Second/);
219 |     expect(criterionCells[0]).toHaveTextContent("First");
220 |     expect(criterionCells[1]).toHaveTextContent("Second");
221 |   });
222 |
223 |   it("shows dash for empty descriptions", () => {
224 |     const rubric = makeRubric({
225 |       criteria: [
226 |         {
227 |           id: "c1",
228 |           rubric_id: "rubric-1",
229 |           criterion_name: "Test",
230 |           sort_order: 0,
231 |           levels: [{ label: "L1", description: "", points: 2 }],
232 |           max_points: 2,
233 |         },
234 |       ],
235 |     });
236 |
237 |     render(<RubricPreview rubric={rubric} />);
238 |     expect(screen.getByText("-")).toBeInTheDocument();
239 |   });
240 | });
241 |
242 | // -----
243 | // RubricPreviewDialog – read-only rubric preview dialog (Req 14.2, 14.3)
244 | //
245 | // Feature: qa-partner-review-remediation – Req 14 (P10)
246 | // Validates: Requirements 14.2, 14.3
247 | //
248 | // The rubric list "Preview" action opens a read-only dialog (Req 14.2) that
249 | // renders the rubric's criteria and performance levels with NO edit controls
250 | // (Req 14.3). These tests render the dialog open with a mocked `useRubric` and
251 | // assert the rubric content shows and that no Save/Edit/Delete affordances are
252 | // present.
253 | // -----
254 |
255 | describe("RubricPreviewDialog", () => {
256 |   beforeEach(() => {
257 |     vi.clearAllMocks();
258 |   });
259 |
260 |   it("renders the rubric criteria and levels read-only when open (Req 14.2, 14.3)", () => {
261 |     mockUseRubric.mockReturnValue({
262 |       data: makeRubric(),
263 |       isLoading: false,
264 |       isError: false,
265 |     });
266 |
267 |     render(
268 |       <RubricPreviewDialog
269 |         rubricId="rubric-1"
270 |         open={true}
271 |         onOpenChange={vi.fn()}
272 |       />
273 |     );
274 |
275 |     const dialog = screen.getByRole("dialog");
276 |     const inDialog = within(dialog);
277 |
278 |     // Read-only heading + the rubric content (criteria + levels) are shown.
279 |     expect(inDialog.getByText("Rubric Preview")).toBeInTheDocument();
280 |     expect(inDialog.getByText("Essay Rubric")).toBeInTheDocument();
281 |     expect(inDialog.getByText("Clarity")).toBeInTheDocument();
282 |     expect(inDialog.getByText("Grammar")).toBeInTheDocument();
283 |     expect(inDialog.getByText("Proficient")).toBeInTheDocument();
284 |   });
285 |
286 |   it("exposes NO edit controls (no Save/Edit/Delete buttons) (Req 14.3)", () => {
287 |     mockUseRubric.mockReturnValue({
288 |       data: makeRubric(),
289 |       isLoading: false,

```

```

290 |         isError: false,
291 |     });
292 |
293 |     render(
294 |         <RubricPreviewDialog
295 |             rubricId="rubric-1"
296 |             open={true}
297 |             onOpenChange={vi.fn()}
298 |         />
299 |     );
300 |
301 |     const dialog = screen.getByRole("dialog");
302 |     const inDialog = within(dialog);
303 |
304 |     // No mutating affordances of any kind are rendered.
305 |     expect(
306 |         inDialog.queryByRole("button", { name: /save/i })
307 |     ).not.toBeInTheDocument();
308 |     expect(
309 |         inDialog.queryByRole("button", { name: /edit/i })
310 |     ).not.toBeInTheDocument();
311 |     expect(
312 |         inDialog.queryByRole("button", { name: /delete/i })
313 |     ).not.toBeInTheDocument();
314 |     expect(
315 |         inDialog.queryByRole("button", { name: /copy/i })
316 |     ).not.toBeInTheDocument();
317 |     // No form inputs/textboxes either – the preview is strictly read-only.
318 |     expect(inDialog.queryByRole("textbox")).not.toBeInTheDocument();
319 | });
320 |
321 | it("does not render dialog content when closed", () => {
322 |     mockUseRubric.mockReturnValue({
323 |         data: makeRubric(),
324 |         isLoading: false,
325 |         isError: false,
326 |     });
327 |
328 |     render(
329 |         <RubricPreviewDialog
330 |             rubricId="rubric-1"
331 |             open={false}
332 |             onOpenChange={vi.fn()}
333 |         />
334 |     );
335 |
336 |     expect(screen.queryByRole("dialog")).not.toBeInTheDocument();
337 |     expect(screen.queryByText("Essay Rubric")).not.toBeInTheDocument();
338 | });
339 |
340 | it("shows a shimmer while the rubric loads", () => {
341 |     mockUseRubric.mockReturnValue({
342 |         data: undefined,
343 |         isLoading: true,
344 |         isError: false,
345 |     });
346 |
347 |     render(
348 |         <RubricPreviewDialog
349 |             rubricId="rubric-1"
350 |             open={true}
351 |             onOpenChange={vi.fn()}
352 |         />
353 |     );
354 |
355 |     expect(screen.getByTestId("rubric-preview-loading")).toBeInTheDocument();
356 | });
357 |
358 | it("shows a not-found empty state when the rubric is missing", () => {
359 |     mockUseRubric.mockReturnValue({
360 |         data: null,
361 |         isLoading: false,
362 |         isError: false,
363 |     });
364 |
365 |     render(
366 |         <RubricPreviewDialog
367 |             rubricId="missing"
368 |             open={true}
369 |             onOpenChange={vi.fn()}
370 |         />
371 |     );
372 |
373 |     expect(screen.getByText("Rubric not found")).toBeInTheDocument();
374 | });
375 | });
376 |

```

FILE: [src/__tests__/unit/sabbaticalRestDay.test.tsx](#)

TYPE: .tsx | LINES: 1-246 | SHA256: 9fd3ab78560a614fa69fdec13f5971f749a9ae8d5bce1da6955c2d8ba87c988

```

1 | import { describe, it, expect, vi } from "vitest";
2 | import { render, screen } from "@testing-library/react";
3 | import HeatmapGrid from "@components/shared/HeatmapGrid";
4 | import HeatmapTooltip from "@components/shared/HeatmapTooltip";
5 | import HeatmapSummaryStats from "@components/shared/HeatmapSummaryStats";
6 | import type { HeatmapDay, DateRange } from "@types/habits";
7 |
8 | // -----
9 | // Helpers
10 | // -----
11 |
12 | function makeDay(date: string, totalCount: number): HeatmapDay {
13 |     return {

```

```

14 |     date,
15 |     academicCount: Math.min(totalCount, 4),
16 |     wellnessCount: Math.max(totalCount - 4, 0),
17 |     totalCount,
18 |     habits: [],
19 |   };
20 | }
21 |
22 | function buildDays(
23 |   start: string,
24 |   end: string,
25 |   countFn: (d: string) => number = () => 0
26 | ): HeatmapDay[] {
27 |   const days: HeatmapDay[] = [];
28 |   const cursor = new Date(start + "T00:00:00");
29 |   const endDate = new Date(end + "T00:00:00");
30 |   while (cursor <= endDate) {
31 |     const dateStr =
32 |       cursor.getFullYear() +
33 |       "-" +
34 |       String(cursor.getMonth() + 1).padStart(2, "0") +
35 |       "-" +
36 |       String(cursor.getDate()).padStart(2, "0");
37 |     days.push(makeDay(dateStr, countFn(dateStr)));
38 |     cursor.setDate(cursor.getDate() + 1);
39 |   }
40 |   return days;
41 | }
42 |
43 | // Stub ResizeObserver
44 | class MockResizeObserver {
45 |   callback: ResizeObserverCallback;
46 |   constructor(cb: ResizeObserverCallback) {
47 |     this.callback = cb;
48 |   }
49 |   observe() {
50 |     this.callback(
51 |       [{ contentRect: { width: 900 } } as unknown as ResizeObserverEntry],
52 |       this as unknown as ResizeObserver
53 |     );
54 |   }
55 |   unobserve() {}
56 |   disconnect() {}
57 | }
58 | vi.stubGlobal("ResizeObserver", MockResizeObserver);
59 |
60 | // -----
61 | // Tests
62 | // -----
63 |
64 | describe("Sabbatical Rest Day Overlay", () => {
65 |   // 2025-01-06 is Monday, 2025-01-11 is Saturday, 2025-01-12 is Sunday
66 |   const range: DateRange = { start: "2025-01-06", end: "2025-01-19" };
67 |
68 |   it("renders diagonal stripe overlay on Saturday and Sunday when sabbatical is enabled", () => {
69 |     const days = buildDays("2025-01-06", "2025-01-19");
70 |
71 |     render(
72 |       <HeatmapGrid data={days} semesterRange={range} sabbaticalEnabled={true} />
73 |     );
74 |
75 |     // Saturday Jan 11 and Sunday Jan 12 should have sabbatical overlay
76 |     expect(
77 |       screen.getByTestId("sabbatical-overlay-2025-01-11")
78 |     ).toBeInTheDocument();
79 |     expect(
80 |       screen.getByTestId("sabbatical-overlay-2025-01-12")
81 |     ).toBeInTheDocument();
82 |
83 |     // Saturday Jan 18 and Sunday Jan 19 should also have overlay
84 |     expect(
85 |       screen.getByTestId("sabbatical-overlay-2025-01-18")
86 |     ).toBeInTheDocument();
87 |     expect(
88 |       screen.getByTestId("sabbatical-overlay-2025-01-19")
89 |     ).toBeInTheDocument();
90 |
91 |     // Weekday should NOT have overlay
92 |     expect(
93 |       screen.queryByTestId("sabbatical-overlay-2025-01-06")
94 |     ).not.toBeInTheDocument();
95 |     expect(
96 |       screen.queryByTestId("sabbatical-overlay-2025-01-07")
97 |     ).not.toBeInTheDocument();
98 |   });
99 |
100 |   it("does not render sabbatical overlay when sabbaticalEnabled is false", () => {
101 |     const days = buildDays("2025-01-06", "2025-01-19");
102 |
103 |     render(
104 |       <HeatmapGrid
105 |         data={days}
106 |         semesterRange={range}
107 |         sabbaticalEnabled={false}
108 |       />
109 |     );
110 |
111 |     expect(
112 |       screen.queryByTestId("sabbatical-overlay-2025-01-11")
113 |     ).not.toBeInTheDocument();
114 |     expect(
115 |       screen.queryByTestId("sabbatical-overlay-2025-01-12")
116 |     ).not.toBeInTheDocument();
117 |   });

```

```

118 |
119 | it("does not render sabbatical overlay when prop is not provided", () => {
120 |   const days = buildDays("2025-01-06", "2025-01-19");
121 |
122 |   render(<HeatmapGrid data={days} semesterRange={range} />);
123 |
124 |   expect(
125 |     screen.queryByTestId("sabbatical-overlay-2025-01-11")
126 |   ).not.toBeInTheDocument();
127 | });
128 |
129 | it("sabbatical overlay has correct ARIA label", () => {
130 |   const days = buildDays("2025-01-06", "2025-01-19");
131 |
132 |   render(
133 |     <HeatmapGrid data={days} semesterRange={range} sabbaticalEnabled={true} />
134 |   );
135 |
136 |   const overlay = screen.getByTestId("sabbatical-overlay-2025-01-11");
137 |   expect(overlay).toHaveAttribute("aria-label", "Rest Day (Sabbatical)");
138 | });
139 | });
140 |
141 | describe("HeatmapTooltip – Sabbatical Rest Day label", () => {
142 |   it('shows "Rest Day (Sabbatical)" label when isSabbaticalRestDay is true', () => {
143 |     render(
144 |       <HeatmapTooltip
145 |         date="2025-01-11"
146 |         habits={[]}
147 |         xpEarned={0}
148 |         streakActive={false}
149 |         isSabbaticalRestDay={true}
150 |       />
151 |     );
152 |
153 |     expect(screen.getByTestId("tooltip-sabbatical-label")).toHaveTextContent(
154 |       "Rest Day (Sabbatical)"
155 |     );
156 |   });
157 |
158 |   it('does not show "No habits completed" when sabbatical rest day with no habits', () => {
159 |     render(
160 |       <HeatmapTooltip
161 |         date="2025-01-11"
162 |         habits={[]}
163 |         xpEarned={0}
164 |         streakActive={false}
165 |         isSabbaticalRestDay={true}
166 |       />
167 |     );
168 |
169 |     expect(screen.queryByTestId("tooltip-empty")).not.toBeInTheDocument();
170 |   });
171 |
172 |   it("does not show sabbatical label when isSabbaticalRestDay is false", () => {
173 |     render(
174 |       <HeatmapTooltip
175 |         date="2025-01-11"
176 |         habits={[]}
177 |         xpEarned={0}
178 |         streakActive={false}
179 |         isSabbaticalRestDay={false}
180 |       />
181 |     );
182 |
183 |     expect(
184 |       screen.queryByTestId("tooltip-sabbatical-label")
185 |     ).not.toBeInTheDocument();
186 |   });
187 | });
188 |
189 | describe("HeatmapSummaryStats – Sabbatical day exclusion", () => {
190 |   it('shows "Active Days (excl. rest)" label when sabbaticalDayCount > 0', () => {
191 |     render(
192 |       <HeatmapSummaryStats
193 |         currentStreak={5}
194 |         longestStreak={10}
195 |         totalActiveDays={20}
196 |         sabbaticalDayCount={8}
197 |       />
198 |     );
199 |
200 |     const card = screen.getByTestId("kpi-total-active-days");
201 |     expect(card).toHaveTextContent("Active Days (excl. rest)");
202 |   });
203 |
204 |   it('shows "Total Active Days" label when sabbaticalDayCount is 0', () => {
205 |     render(
206 |       <HeatmapSummaryStats
207 |         currentStreak={5}
208 |         longestStreak={10}
209 |         totalActiveDays={20}
210 |         sabbaticalDayCount={0}
211 |       />
212 |     );
213 |
214 |     const card = screen.getByTestId("kpi-total-active-days");
215 |     expect(card).toHaveTextContent("Total Active Days");
216 |   });
217 |
218 |   it('shows "Total Active Days" label when sabbaticalDayCount is not provided', () => {
219 |     render(
220 |       <HeatmapSummaryStats
221 |         currentStreak={5}

```

```

222 |         longestStreak={10}
223 |         totalActiveDays={20}
224 |       />
225 |     );
226 |
227 |     const card = screen.getByTestId("kpi-total-active-days");
228 |     expect(card).toHaveTextContent("Total Active Days");
229 |   });
230 |
231 |   it("displays the correct totalActiveDays value", () => {
232 |     render(
233 |       <HeatmapSummaryStats
234 |         currentStreak={5}
235 |         longestStreak={10}
236 |         totalActiveDays={42}
237 |         sabbaticalDayCount={8}
238 |       />
239 |     );
240 |
241 |     expect(screen.getByTestId("kpi-total-active-days-value")).toHaveTextContent(
242 |       "42"
243 |     );
244 |   });
245 | });
246 |

```

FILE: src/__tests__/unit/saleEventForm.test.tsx

```
TYPE: .tsx | LINES: 1-112 | SHA256: 7aa752bc3b1175cd2ee631c61ece2bafb697ed1986b0b215e292448ed3e62ad7
```

[illegible]

```

76 | it("renders start and end date inputs", () => {
77 |   renderWithProviders();
78 |   expect(screen.getByText("Start Date")).toBeTruthy();
79 |   expect(screen.getByText("End Date")).toBeTruthy();
80 | });
81 |
82 | it("renders only active items in the item selection list", () => {
83 |   renderWithProviders();
84 |   expect(screen.getByText("Ocean Blue Theme")).toBeTruthy();
85 |   expect(screen.getByText("XP Boost")).toBeTruthy();
86 |   // Inactive item should not appear
87 |   expect(screen.queryByText("Inactive Item")).toBeNull();
88 | });
89 |
90 | it("renders Create Sale Event submit button", () => {
91 |   renderWithProviders();
92 |   expect(
93 |     screen.getByRole("button", { name: /Create Sale Event/i })
94 |   ).toBeTruthy();
95 | });
96 |
97 | it("renders Cancel button", () => {
98 |   renderWithProviders();
99 |   expect(screen.getByRole("button", { name: /Cancel/i })).toBeTruthy();
100 | });
101 |
102 | it("renders Back button", () => {
103 |   renderWithProviders();
104 |   expect(screen.getByText("Back")).toBeTruthy();
105 | });
106 |
107 | it("shows selected items count", () => {
108 |   renderWithProviders();
109 |   expect(screen.getByText(/0 selected/)).toBeTruthy();
110 | });
111 | });
112 |

```

FILE: [src/__tests__/unit/sanitizeFilter.test.ts](#)

TYPE: .ts | LINES: 1-47 | SHA256: 38cf815f193552c3d0a3012ff8cbfff849b8c68ac3aed9cc7275b0720ebbc15c

```

1 | import { describe, it, expect } from "vitest";
2 | import { sanitizePostgresValue } from "@lib/sanitizeFilter";
3 |
4 | describe("sanitizePostgresValue", () => {
5 |   it("escapes dots", () => {
6 |     expect(sanitizePostgresValue("a.b")).toBe("a\\.b");
7 |   });
8 |
9 |   it("escapes commas", () => {
10 |     expect(sanitizePostgresValue("a,b")).toBe("a\\,b");
11 |   });
12 |
13 |   it("escapes parentheses", () => {
14 |     expect(sanitizePostgresValue("a(b)c")).toBe("a\\(b\\)c");
15 |   });
16 |
17 |   it("escapes percent signs", () => {
18 |     expect(sanitizePostgresValue("100%")).toBe("100\\%");
19 |   });
20 |
21 |   it("escapes asterisks", () => {
22 |     expect(sanitizePostgresValue("a*b")).toBe("a\\*b");
23 |   });
24 |
25 |   it("escapes backslashes", () => {
26 |     expect(sanitizePostgresValue("a\\b")).toBe("a\\\\b");
27 |   });
28 |
29 |   it("escapes multiple special characters in one string", () => {
30 |     expect(sanitizePostgresValue("name.ilike.%admin%,role.eq.admin")).toBe(
31 |       "name\\.ilike\\.\\.%admin\\%,role\\.eq\\.admin"
32 |     );
33 |   });
34 |
35 |   it("returns plain strings unchanged", () => {
36 |     expect(sanitizePostgresValue("hello world")).toBe("hello world");
37 |   });
38 |
39 |   it("handles empty string", () => {
40 |     expect(sanitizePostgresValue("")).toBe("");
41 |   });
42 |
43 |   it("handles string with only special characters", () => {
44 |     expect(sanitizePostgresValue(".,()*%`\\")).toBe("\\.\\,\\(\\)\\%\\*\\`\\\\");
45 |   });
46 | });
47 |

```

FILE: [src/__tests__/unit/sankeyDiagramView.test.tsx](#)

TYPE: .tsx | LINES: 1-172 | SHA256: e0574634783a9b4e5a602fa51e7c1f06ecf887a287475ef5c6f759aee88bee4d

```

1 | // @vitest-environment happy-dom
2 | // =====
3 | // SankeyDiagramView – both render branches (real flow vs. "Outcome Mapping")
4 | //
5 | // Feature: qa-partner-review-remediation – Req 11 (P7)
6 | // Validates: Requirements 11.2, 11.3, 11.4
7 | //
8 | // SankeyDiagramView maps `useSankeyData` { nodes, links } onto a recharts
9 | // `Sankey`. It has two mutually exclusive data branches once a program is
10 | // selected and data has loaded with at least one node:
11 | //
12 | // 1. links present !' the real flow diagram (recharts Sankey) renders, the

```

```

13 //      page title reads "Outcome Flow (Sankey)", and the
14 //      "{n} outcomes · {m} mappings" caption is preserved (Req 11.2, 11.5).
15 //      2. links empty !' the fallback column layout renders, the page title
16 //      reads "Outcome Mapping", and the term "Sankey" never appears in any
17 //      user-facing string for that branch (Req 11.3, 11.4).
18 //
19 // recharts' `ResponsiveContainer` reports a 0x0 box under happy-dom, so the
20 // internal SVG flow is not laid out in jsdom/happy-dom. We therefore assert on
21 // the component's own surrounding DOM – the page title and the preserved
22 // counts caption that `SankeyFlow` renders outside the chart – rather than on
23 // recharts' SVG internals. `ResizeObserver` (relied on by both
24 // `ResponsiveContainer` and the Radix Select trigger) is stubbed below.
25 // =====
26
27 import { describe, it, expect, vi, beforeEach } from "vitest";
28 import { render, screen } from "@testing-library/react";
29
30 import type { SankeyNode, SankeyLink } from "@lib/sankeyTransform";
31
32 // -----
33 // Mocks (must precede the component import)
34 // -----
35
36 // nuqs – return a fixed, non-empty program id so the component is past the
37 // "Select a program" guard and exercises the real data branches.
38 vi.mock("nuqs", () => ({
39   parseAsString: { withDefault: (def: string) => def },
40   useQueryState: () => ["prog-1", vi.fn()] as const,
41 }));
42
43 // usePrograms – a single program so the header Select has an option; the view
44 // reads `programsData?.data ?? []`.
45 vi.mock("@/hooks/usePrograms", () => ({
46   usePrograms: () => ({
47     data: { data: [{ id: "prog-1", name: "CS Program" }] },
48   }),
49 }));
50
51 // useSankeyData is the single dependency that drives both branches.
52 type SankeyQueryResult = {
53   data: { nodes: SankeyNode[]; links: SankeyLink[] } | undefined;
54   isLoading: boolean;
55 };
56 const mockUseSankeyData = vi.fn<() => SankeyQueryResult>();
57 vi.mock("@/hooks/useVisualizationData", () => ({
58   useSankeyData: () => mockUseSankeyData(),
59 }));
60
61 // happy-dom implements neither ResizeObserver (used by recharts'
62 // ResponsiveContainer and the Radix Select trigger) nor a complete
63 // matchMedia (read by framer-motion's reduced-motion gate in SankeyFlow).
64 class MockResizeObserver {
65   observe() {}
66   unobserve() {}
67   disconnect() {}
68 }
69 vi.stubGlobal("ResizeObserver", MockResizeObserver);
70 vi.stubGlobal(
71   "matchMedia",
72   (query: string) =>
73     ({
74       matches: false,
75       media: query,
76       onchange: null,
77       addEventListener: () => {},
78       removeEventListener: () => {},
79       addListener: () => {},
80       removeListener: () => {},
81       dispatchEvent: () => false,
82     } as unknown as MediaQueryList)
83 );
84
85 import SankeyDiagramView from "@pages/coordinator/sankey/SankeyDiagramView";
86
87 // -----
88 // Fixtures – two CLOs flowing into one PLO
89 // -----
90
91 const NODES: SankeyNode[] = [
92   {
93     name: "CLO Alpha",
94     id: "clo-1",
95     type: "CLO",
96     attainment: 80,
97     color: "#22c55e",
98   },
99   {
100     name: "CLO Beta",
101     id: "clo-2",
102     type: "CLO",
103     attainment: 60,
104     color: "#f59e0b",
105   },
106   {
107     name: "PLO One",
108     id: "plo-1",
109     type: "PLO",
110     attainment: 75,
111     color: "#3b82f6",
112   },
113 ];
114
115 const LINKS: SankeyLink[] = [
116   { source: 0, target: 2, value: 50, weight: 50 },

```



```

117 | { source: 1, target: 2, value: 50, weight: 50 },
118 | ];
119 |
120 | // -----
121 | // Tests
122 | // -----
123 |
124 | describe("SankeyDiagramView – both data branches (Req 11.2, 11.3, 11.4)", () => {
125 |   beforeEach(() => {
126 |     vi.clearAllMocks();
127 |   });
128 |
129 |   it("renders the real flow with the preserved counts caption when links are present (Req 11.2, 11.5)", () =>
130 |     mockUseSankeyData.mockReturnValue({
131 |       data: { nodes: NODES, links: LINKS },
132 |       isLoading: false,
133 |     });
134 |
135 |     const { container } = render(<SankeyDiagramView />);
136 |
137 |     // The flow branch labels the page with the "Sankey" wording.
138 |     expect(
139 |       screen.getByRole("heading", { name: "Outcome Flow (Sankey)" })
140 |     ).toBeInTheDocument();
141 |
142 |     // The outcome/mapping counts caption is preserved (3 nodes, 2 links). React
143 |     // splits the interpolated text into several text nodes within one <p>, so
144 |     // assert on the flattened textContent rather than an exact getByText match.
145 |     expect(container.textContent).toContain("3 outcomes · 2 mappings");
146 |
147 |     // The fallback column layout is NOT used in this branch (no per-column
148 |     // "CLOs" / "PLOs" / "ILOs" headers from OutcomeMappingColumns).
149 |     expect(screen.queryByText("ILOs")).not.toBeInTheDocument();
150 |   });
151 |
152 |   it("renders the 'Outcome Mapping' fallback with NO 'Sankey' wording when links are empty (Req 11.3, 11.4)",
153 |     mockUseSankeyData.mockReturnValue({
154 |       data: { nodes: NODES, links: [] },
155 |       isLoading: false,
156 |     });
157 |
158 |     const { container } = render(<SankeyDiagramView />);
159 |
160 |     // The empty-links branch relabels the page heading to "Outcome Mapping".
161 |     expect(
162 |       screen.getByRole("heading", { name: "Outcome Mapping" })
163 |     ).toBeInTheDocument();
164 |
165 |     // The counts caption is still preserved (3 nodes, 0 links).
166 |     expect(container.textContent).toContain("3 outcomes · 0 mappings");
167 |
168 |     // Req 11.4: the term "Sankey" must not appear anywhere in this branch.
169 |     expect(container.textContent ?? "").not.toContain("Sankey");
170 |   });
171 | });
172 |

```

FILE: [src/__tests__/unit/schemaContract.test.ts](#)

TYPE: .ts | LINES: 1-160 | SHA256: 7e18bddce9bb2b8f10791c464bdda6580fea2c9476be8be79415460e421a259c

```

1 | // =====
2 | // Schema_Contract_Test (Part C, Req 18) – cross-references each mutation
3 | // payload's keys against the target table's real Insert columns and its
4 | // required (NOT NULL, no-default) columns, using the checked-in
5 | // `requiredColumns.json` manifest (generated from information_schema) as the
6 | // authority for required columns (Req 18.3).
7 | //
8 | // The seeded challenge + team descriptors are proven contract-clean: every
9 | // required column for `social_challenges` and `teams` is present in their
10 | // `*_INSERT_COLUMNS` constants, and every constant key is a real Insert column.
11 | // Direct validator cases prove the harness actually detects violations and
12 | // always names the offending key/column + table (Req 18.2, 18.3, 18.4).
13 | // =====
14 | import { describe, it, expect } from "vitest";
15 |
16 | import {
17 |   validateDescriptor,
18 |   validateDescriptors,
19 |   formatViolation,
20 |   type MutationContractDescriptor,
21 |   type SchemaModel,
22 | } from "@lib/db/schemaContract";
23 | import {
24 |   SOCIAL_CHALLENGES_INSERT_COLUMNS,
25 |   TEAMS_INSERT_COLUMNS,
26 | } from "@lib/db/insertColumns";
27 | import {
28 |   MUTATION_DESCRIPTOR,
29 |   type MutationDescriptor,
30 | } from "@__tests__/fixtures/mutationDescriptors";
31 | import requiredColumns from "@__tests__/fixtures/requiredColumns.json";
32 |
33 | /**
34 |  * Required-column authority (Req 18.3): the manifest generated from
35 |  * `information_schema` and committed to the repo.
36 |  */
37 | const REQUIRED_COLUMNS: Readonly<Record<string, readonly string[]>> =
38 |   requiredColumns;
39 |
40 | /**
41 |  * The real Insert-column oracle per table for the unknown-column check
42 |  * (Req 18.2). Sourced from the `*_INSERT_COLUMNS` constants, which are
43 |  * compile-time-proven to be real Insert columns. One entry per table covered
44 |  * by the registry – extend alongside `MUTATION_DESCRIPTOR`.

```

```

45 | */
46 | const INSERT_COLUMNS_BY_TABLE: Readonly<Record<string, readonly string[]>> = {
47 |   social_challenges: SOCIAL_CHALLENGES_INSERT_COLUMNS,
48 |   teams: TEAMS_INSERT_COLUMNS,
49 |   // Real insert columns of `notifications` (from `database.ts`), the oracle for
50 |   // the `sendTeacherNudge` descriptor's unknown-column check.
51 |   notifications: [
52 |     "user_id",
53 |     "type",
54 |     "title",
55 |     "body",
56 |     "is_read",
57 |     "metadata",
58 |     "created_at",
59 |     "id",
60 |   ],
61 | };
62 |
63 | /** Resolve the {@link SchemaModel} (real + required columns) for a descriptor's table. */
64 | const resolveModel = (descriptor: { table: string }): SchemaModel => ({
65 |   insertColumns: INSERT_COLUMNS_BY_TABLE[descriptor.table] ?? [],
66 |   requiredColumns: REQUIRED_COLUMNS[descriptor.table] ?? [],
67 | });
68 |
69 | describe("Schema_Contract_Test: seeded descriptors are contract-clean", () => {
70 |   it("has a registered insert-column oracle for every descriptor's table", () => {
71 |     for (const descriptor of MUTATION_DESCRIPTOR) {
72 |       expect(
73 |         INSERT_COLUMNS_BY_TABLE[descriptor.table],
74 |         `missing insert-column oracle for table "${descriptor.table}" (add it alongside the descriptor)`
75 |       ).toBeDefined();
76 |     }
77 |   });
78 |
79 |   it.each(MUTATION_DESCRIPTOR.map((d) => [d.hook, d] as const)) (
80 |     "%s payload contains every required column and only real columns",
81 |     (_hook, descriptor: MutationDescriptor) => {
82 |       const violations = validateDescriptor(
83 |         descriptor,
84 |         resolveModel(descriptor)
85 |       );
86 |       expect(violations, violations.map(formatViolation).join("\n")).toEqual(
87 |         []
88 |       );
89 |     }
90 |   );
91 |
92 |   it("reports zero violations across the whole registry (accumulating, Req 18.4)", () => {
93 |     const violations = validateDescriptors(MUTATION_DESCRIPTOR, resolveModel);
94 |     expect(violations.map(formatViolation)).toEqual([]);
95 |   });
96 | });
97 |
98 | describe("Schema_Contract_Test validator: detection and identification", () => {
99 |   it("flags an unknown column and names the key + table (Req 18.2)", () => {
100 |     const descriptor: MutationContractDescriptor = {
101 |       hook: "useBroken",
102 |       table: "teams",
103 |       payloadKeys: [...TEAMS_INSERT_COLUMNS, "not_a_real_column"],
104 |       op: "insert",
105 |     };
106 |     const violations = validateDescriptor(descriptor, resolveModel(descriptor));
107 |
108 |     expect(violations).toContainEqual({
109 |       kind: "unknown-column",
110 |       hook: "useBroken",
111 |       table: "teams",
112 |       key: "not_a_real_column",
113 |     });
114 |     expect(formatViolation(violations[0]!)).toContain("not_a_real_column");
115 |     expect(formatViolation(violations[0]!)).toContain("teams");
116 |   });
117 |
118 |   it("flags a missing required column and names the column + table (Req 18.3)", () => {
119 |     const descriptor: MutationContractDescriptor = {
120 |       hook: "useBroken",
121 |       table: "teams",
122 |       // Drop the required `captain_id` column.
123 |       payloadKeys: TEAMS_INSERT_COLUMNS.filter((c) => c !== "captain_id"),
124 |       op: "insert",
125 |     };
126 |     const violations = validateDescriptor(descriptor, resolveModel(descriptor));
127 |
128 |     expect(violations).toContainEqual({
129 |       kind: "missing-required-column",
130 |       hook: "useBroken",
131 |       table: "teams",
132 |       column: "captain_id",
133 |     });
134 |   });
135 |
136 |   it("accumulates ALL violations rather than stopping at the first (Req 18.4)", () => {
137 |     const model: SchemaModel = {
138 |       insertColumns: ["a", "b"],
139 |       requiredColumns: ["a", "b", "c"],
140 |     };
141 |     const descriptor: MutationContractDescriptor = {
142 |       hook: "useMulti",
143 |       table: "demo",
144 |       // `x` and `y` are unknown; required `a`, `b`, `c` are all absent.
145 |       payloadKeys: ["x", "y"],
146 |       op: "insert",
147 |     };
148 |     const violations = validateDescriptor(descriptor, model);

```

```

149 |
150 | // 2 unknown-column + 3 missing-required-column = 5 total.
151 | expect(violations).toHaveLength(5);
152 | expect(violations.filter((v) => v.kind === "unknown-column")).toHaveLength(
153 |     2
154 | );
155 | expect(
156 |     violations.filter((v) => v.kind === "missing-required-column")
157 | ).toHaveLength(3);
158 | });
159 | });
160 |

```

FILE: src/__tests__/unit/schemaIntegrity.test.ts

```
TYPE: .ts | LINES: 1-36 | SHA256: 1d6f049dcf8407115df441424560097a09d07e4bf5697914ea47b7c04c6e79fb
```

```

1 // =====
2 // Schema Integrity Test - Verifies all Zod schema modules export correctly.
3 // =====
4 import { describe, it, expect } from "vitest";
5
6 describe("Build integrity: schema modules export correctly", () => {
7   const schemaModules = [
8     () => import("@/lib/schemas/badgeTier"),
9     () => import("@/lib/schemas/badgeSpotlight"),
10    () => import("@/lib/schemas/adaptiveXP"),
11    () => import("@/lib/schemas/improvementBonus"),
12    () => import("@/lib/schemas/challenge"),
13    () => import("@/lib/schemas/institutionSettings"),
14    () => import("@/lib/schemas/team"),
15    () => import("@/lib/schemas/subCLO"),
16    () => import("@/lib/schemas/competencyFramework"),
17    () => import("@/lib/schemas/graduateAttribute"),
18    () => import("@/lib/schemas/habitDifficulty"),
19    () => import("@/lib/schemas/comebackChallenge"),
20    () => import("@/lib/schemas/coverageHeatmap"),
21    () => import("@/lib/schemas/gapAnalysis"),
22    () => import("@/lib/schemas/sankeyFilter"),
23  ];
24
25  it.each(schemaModules.map((fn, i) => [i, fn]))(
26    "schema module %i imports without error",
27    async (_, idx, importFn) => {
28      const mod = await (importFn as () => Promise<unknown>})();
29      expect(mod).toBeDefined();
30      // Every schema module should export at least one Zod schema
31      const exports = Object.keys(mod as Record<string, unknown>);
32      expect(exports.length).toBeGreaterThan(0);
33    }
34  );
35 });
36

```

FILE: src/__tests__/unit/scoreCalculator.test.ts

```
TYPE: .ts | LINES: 1-778 | SHA256: 07d1ee322742bb869491253d8f36f841d39edb0f2061c249a7c54dc859f242e3
```

```

1 // @vitest-environment happy-dom
2 import { describe, it, expect } from "vitest";
3 import {
4   calculateBigFiveScores,
5   calculateVARKScores,
6   calculateBaselineScores,
7   calculateSelfEfficacyScores,
8   calculateStudyStrategyScores,
9   calculateProfileCompleteness,
10 } from "@lib/scoreCalculator";
11 import type {
12   PersonalityResponse,
13   VARKResponseInput,
14   BaselineResponseInput,
15   SelfEfficacyResponseInput,
16   StudyStrategyResponseInput,
17 } from "@lib/scoreCalculator";
18
19 // %% Big Five %%%%%%%%%%%%%%%
20
21 describe("calculateBigFiveScores", () => {
22   it("returns all zeros for empty responses", () => {
23     const result = calculateBigFiveScores([]);
24     expect(result).toEqual({
25       openness: 0,
26       conscientiousness: 0,
27       extraversion: 0,
28       agreeableness: 0,
29       neuroticism: 0,
30     });
31   });
32
33   it("calculates max scores when all responses are 5 with weight +1", () => {
34     const responses: PersonalityResponse[] = [
35       "openness",
36       "conscientiousness",
37       "extraversion",
38       "agreeableness",
39       "neuroticism",
40     ].flatMap((dim) =>
41       Array.from({ length: 5 }, () => ({
42         dimension: dim,
43         selected_option: 5,
44         weight: 1,
45       }))
46     );
47     const result = calculateBigFiveScores(responses);
48     expect(result.openness).toBe(100);

```

```

49 |     expect(result.conscientiousness).toBe(100);
50 |     expect(result.extraversion).toBe(100);
51 |     expect(result.agreeableness).toBe(100);
52 |     expect(result.neuroticism).toBe(100);
53 | });
54 |
55 | it("calculates min scores when all responses are 1 with weight +1", () => {
56 |     const responses: PersonalityResponse[] = [
57 |         "openness",
58 |         "conscientiousness",
59 |         "extraversion",
60 |         "agreeableness",
61 |         "neuroticism",
62 |     ].flatMap((dim) =>
63 |         Array.from({ length: 5 }, () => ({
64 |             dimension: dim,
65 |             selected_option: 1,
66 |             weight: 1,
67 |         })))
68 |     );
69 |     const result = calculateBigFiveScores(responses);
70 |     expect(result.openness).toBe(20);
71 |     expect(result.neuroticism).toBe(20);
72 | });
73 |
74 | it("applies reverse scoring for weight -1", () => {
75 |     // weight=-1, selected=1 !' contribution = 6-1 = 5 !' score = (5/5)*100 = 100
76 |     const responses: PersonalityResponse[] = [
77 |         { dimension: "neuroticism", selected_option: 1, weight: -1 },
78 |     ];
79 |     const result = calculateBigFiveScores(responses);
80 |     expect(result.neuroticism).toBe(100);
81 | });
82 |
83 | it("applies reverse scoring: weight=-1, selected=5 !' contribution=1", () => {
84 |     const responses: PersonalityResponse[] = [
85 |         { dimension: "openness", selected_option: 5, weight: -1 },
86 |     ];
87 |     const result = calculateBigFiveScores(responses);
88 |     // contribution = 6-5 = 1, score = (1/5)*100 = 20
89 |     expect(result.openness).toBe(20);
90 | });
91 |
92 | it("supports partial scores (Day 1: 3 questions)", () => {
93 |     const responses: PersonalityResponse[] = [
94 |         { dimension: "openness", selected_option: 4, weight: 1 },
95 |         { dimension: "conscientiousness", selected_option: 3, weight: 1 },
96 |         { dimension: "extraversion", selected_option: 5, weight: 1 },
97 |     ];
98 |     const result = calculateBigFiveScores(responses);
99 |     expect(result.openness).toBe(80); // 4/5 * 100
100 |     expect(result.conscientiousness).toBe(60); // 3/5 * 100
101 |     expect(result.extraversion).toBe(100); // 5/5 * 100
102 |     expect(result.agreeableness).toBe(0); // no responses
103 |     expect(result.neuroticism).toBe(0); // no responses
104 | });
105 |
106 | it("handles mixed weights within a dimension", () => {
107 |     const responses: PersonalityResponse[] = [
108 |         { dimension: "openness", selected_option: 4, weight: 1 }, // contribution = 4
109 |         { dimension: "openness", selected_option: 2, weight: -1 }, // contribution = 6-2 = 4
110 |     ];
111 |     const result = calculateBigFiveScores(responses);
112 |     // sum = 8, max = 10, score = 80
113 |     expect(result.openness).toBe(80);
114 | });
115 |
116 | it("handles single item per dimension", () => {
117 |     const responses: PersonalityResponse[] = [
118 |         { dimension: "openness", selected_option: 3, weight: 1 },
119 |     ];
120 |     const result = calculateBigFiveScores(responses);
121 |     // 3/5 * 100 = 60
122 |     expect(result.openness).toBe(60);
123 |     expect(result.conscientiousness).toBe(0);
124 | });
125 |
126 | it("all same values (all 3s, weight +1) produces 60 for each", () => {
127 |     const dims = [
128 |         "openness",
129 |         "conscientiousness",
130 |         "extraversion",
131 |         "agreeableness",
132 |         "neuroticism",
133 |     ];
134 |     const responses: PersonalityResponse[] = dims.flatMap((dim) =>
135 |         Array.from({ length: 5 }, () => ({
136 |             dimension: dim,
137 |             selected_option: 3,
138 |             weight: 1,
139 |         })))
140 |     );
141 |     const result = calculateBigFiveScores(responses);
142 |     for (const dim of dims) {
143 |         expect(result[dim as keyof typeof result]).toBe(60);
144 |     }
145 | });
146 |
147 | it("boundary: weight=-1 with selected=3 gives 60", () => {
148 |     // contribution = 6-3 = 3, score = (3/5)*100 = 60
149 |     const responses: PersonalityResponse[] = [
150 |         { dimension: "agreeableness", selected_option: 3, weight: -1 },
151 |     ];
152 |     const result = calculateBigFiveScores(responses);

```

[illegible]

[illegible]

[illegible]

```

465 expect(result.general_academic).toBe(100);
466 expect(result.course_specific).toBe(100);
467 expect(result.self_regulated_learning).toBe(100);
468 });
469
470 it("supports partial Day 1 responses (2 items)", () => {
471   const responses: SelfEfficacyResponseInput[] = [
472     { domain: "general_academic", selected_option: 4 },
473     { domain: "self_regulated_learning", selected_option: 3 },
474   ];
475   const result = calculateSelfEfficacyScores(responses);
476   expect(result.general_academic).toBe(80); // 4/5 * 100
477   expect(result.self_regulated_learning).toBe(60); // 3/5 * 100
478   expect(result.course_specific).toBe(0); // no responses
479   // overall = (4+3)/(2*5)*100 = 7/10*100 = 70
480   expect(result.overall).toBe(70);
481 });
482
483 it("calculates correct overall from mixed scores", () => {
484   const responses: SelfEfficacyResponseInput[] = [
485     { domain: "general_academic", selected_option: 3 },
486     { domain: "general_academic", selected_option: 4 },
487     { domain: "course_specific", selected_option: 2 },
488     { domain: "course_specific", selected_option: 5 },
489     { domain: "self_regulated_learning", selected_option: 1 },
490     { domain: "self_regulated_learning", selected_option: 3 },
491   ];
492   const result = calculateSelfEfficacyScores(responses);
493   // overall = (3+4+2+5+1+3)/(6*5)*100 = 18/30*100 = 60
494   expect(result.overall).toBe(60);
495   // general_academic = (3+4)/10*100 = 70
496   expect(result.general_academic).toBe(70);
497 });
498
499 it("all minimum values (1s) gives 20 for each domain", () => {
500   const responses: SelfEfficacyResponseInput[] = [
501     { domain: "general_academic", selected_option: 1 },
502     { domain: "general_academic", selected_option: 1 },
503     { domain: "course_specific", selected_option: 1 },
504     { domain: "course_specific", selected_option: 1 },
505     { domain: "self_regulated_learning", selected_option: 1 },
506     { domain: "self_regulated_learning", selected_option: 1 },
507   ];
508   const result = calculateSelfEfficacyScores(responses);
509   expect(result.overall).toBe(20);
510   expect(result.general_academic).toBe(20);
511   expect(result.course_specific).toBe(20);
512   expect(result.self_regulated_learning).toBe(20);
513 });
514
515 it("single item gives correct domain and overall", () => {
516   const responses: SelfEfficacyResponseInput[] = [
517     { domain: "course_specific", selected_option: 4 },
518   ];
519   const result = calculateSelfEfficacyScores(responses);
520   expect(result.course_specific).toBe(80); // 4/5 * 100
521   expect(result.overall).toBe(80); // same single item
522   expect(result.general_academic).toBe(0);
523   expect(result.self_regulated_learning).toBe(0);
524 });
525
526 it("boundary value: selected_option=5 gives 100", () => {
527   const responses: SelfEfficacyResponseInput[] = [
528     { domain: "general_academic", selected_option: 5 },
529   ];
530   const result = calculateSelfEfficacyScores(responses);
531   expect(result.general_academic).toBe(100);
532   expect(result.overall).toBe(100);
533 });
534 });
535
536 // %% Study Strategy %%%%%%%%%%%%%%%
537
538 describe("calculateStudyStrategyScores", () => {
539   it("returns all zeros for empty responses", () => {
540     const result = calculateStudyStrategyScores([]);
541     expect(result).toEqual({
542       time_management: 0,
543       elaboration: 0,
544       self_testing: 0,
545       help_seeking: 0,
546     });
547   });
548
549   it("calculates correct dimension scores", () => {
550     const responses: StudyStrategyResponseInput[] = [
551       { dimension: "time_management", selected_option: 4 },
552       { dimension: "time_management", selected_option: 3 },
553       { dimension: "elaboration", selected_option: 5 },
554       { dimension: "elaboration", selected_option: 5 },
555       { dimension: "self_testing", selected_option: 1 },
556       { dimension: "self_testing", selected_option: 2 },
557       { dimension: "help_seeking", selected_option: 3 },
558       { dimension: "help_seeking", selected_option: 3 },
559     ];
560     const result = calculateStudyStrategyScores(responses);
561     expect(result.time_management).toBe(70); // (4+3)/10*100
562     expect(result.elaboration).toBe(100); // (5+5)/10*100
563     expect(result.self_testing).toBe(30); // (1+2)/10*100
564     expect(result.help_seeking).toBe(60); // (3+3)/10*100
565   });
566
567   it("single item per dimension", () => {
568     const responses: StudyStrategyResponseInput[] = [

```


[illegible]

```

673 |     expect(result).toBe(9);
674 | });
675 |
676 | it("caps items at maximum (no over-counting)", () => {
677 |     expect(
678 |         calculateProfileCompleteness({
679 |             personality_items: 30, // more than 25
680 |             self_efficacy_items: 10, // more than 6
681 |             study_strategy_items: 8,
682 |             learning_style_items: 16,
683 |             baseline_courses: 1,
684 |         })
685 |     ).toBe(100);
686 | });
687 |
688 | it("baseline is binary (1+ courses = full credit)", () => {
689 |     const with1 = calculateProfileCompleteness({
690 |         personality_items: 0,
691 |         self_efficacy_items: 0,
692 |         study_strategy_items: 0,
693 |         learning_style_items: 0,
694 |         baseline_courses: 1,
695 |     });
696 |     const with5 = calculateProfileCompleteness({
697 |         personality_items: 0,
698 |         self_efficacy_items: 0,
699 |         study_strategy_items: 0,
700 |         learning_style_items: 0,
701 |         baseline_courses: 5,
702 |     });
703 |     expect(with1).toBe(with5);
704 |     // 1/5 * 100 = 20
705 |     expect(with1).toBe(20);
706 | });
707 |
708 | it("single dimension only (personality)", () => {
709 |     const result = calculateProfileCompleteness({
710 |         personality_items: 25,
711 |         self_efficacy_items: 0,
712 |         study_strategy_items: 0,
713 |         learning_style_items: 0,
714 |         baseline_courses: 0,
715 |     });
716 |     // 1/5 * 100 = 20
717 |     expect(result).toBe(20);
718 | });
719 |
720 | it("single dimension only (learning_style)", () => {
721 |     const result = calculateProfileCompleteness({
722 |         personality_items: 0,
723 |         self_efficacy_items: 0,
724 |         study_strategy_items: 0,
725 |         learning_style_items: 16,
726 |         baseline_courses: 0,
727 |     });
728 |     expect(result).toBe(20);
729 | });
730 |
731 | it("incremental progress across dimensions", () => {
732 |     // Half of each dimension
733 |     const result = calculateProfileCompleteness({
734 |         personality_items: 12, // 12/25 = 0.48
735 |         self_efficacy_items: 3, // 3/6 = 0.5
736 |         study_strategy_items: 4, // 4/8 = 0.5
737 |         learning_style_items: 8, // 8/16 = 0.5
738 |         baseline_courses: 0, // 0
739 |     });
740 |     // total = (0.48 + 0.5 + 0.5 + 0.5 + 0) / 5 * 100 = 1.98/5*100 = 39.6 !' 40
741 |     expect(result).toBe(40);
742 | });
743 |
744 | it("all dimensions at 1 item each", () => {
745 |     const result = calculateProfileCompleteness({
746 |         personality_items: 1, // 1/25 = 0.04
747 |         self_efficacy_items: 1, // 1/6 = 0.1667
748 |         study_strategy_items: 1, // 1/8 = 0.125
749 |         learning_style_items: 1, // 1/16 = 0.0625
750 |         baseline_courses: 1, // 1
751 |     });
752 |     // total = (0.04 + 0.1667 + 0.125 + 0.0625 + 1) / 5 * 100
753 |     // = 1.3942 / 5 * 100 = 27.88 !' 28
754 |     expect(result).toBe(28);
755 | });
756 |
757 | it("result is always between 0 and 100 inclusive", () => {
758 |     const min = calculateProfileCompleteness({
759 |         personality_items: 0,
760 |         self_efficacy_items: 0,
761 |         study_strategy_items: 0,
762 |         learning_style_items: 0,
763 |         baseline_courses: 0,
764 |     });
765 |     const max = calculateProfileCompleteness({
766 |         personality_items: 25,
767 |         self_efficacy_items: 6,
768 |         study_strategy_items: 8,
769 |         learning_style_items: 16,
770 |         baseline_courses: 1,
771 |     });
772 |     expect(min).toBeGreaterThanOrEqual(0);
773 |     expect(min).toBeLessThanOrEqual(100);
774 |     expect(max).toBeGreaterThanOrEqual(0);
775 |     expect(max).toBeLessThanOrEqual(100);
776 | });

```

```
777 | });
778 |
```

FILE: src/___tests___/unit/sectionComparison.test.tsx

TYPE: .tsx | LINES: 1-266 | SHA256: 91e06a0d32ea482183c7e31602a02f47fbb240cf7ae15b827155921554a6f75b

```
1  | // @vitest-environment happy-dom
2  | // =====
3  | // Coordinator Section Comparison – real attainment data + drill-down
4  | //
5  | // Feature: qa-partner-review-remediation – Req 10 (P6)
6  | // Validates: Requirements 10.1, 10.2, 10.3, 10.4
7  | //
8  | // The Coordinator Section Comparison previously fed the bars a hardcoded
9  | // `attainmentPercent: 0` and a `studentCount` equal to section capacity, with
10 | // no drill-down. The fix sources real attainment + enrolled counts from
11 | // `useSectionAttainment` and makes bars clickable, opening a
12 | // `SectionDrillDownDialog` (teacher / CLO / evidence). A section with no
13 | // evidence shows an inline empty state instead of a misleading 0% bar, and bar
14 | // color is driven by `getAttainmentColor`.
15 | //
16 | // This suite follows the lightweight, prop-driven approach recommended by the
17 | // task: `SectionComparisonChart` is exercised directly via props (no whole-
18 | // dashboard mock), and the drill-down is exercised through a tiny harness that
19 | // reproduces the dashboard's chart -> dialog wiring while mocking only the
20 | // `useSectionDrillDown` data hook.
21 | // 1. data present !' bars render real attainment values; color via getAttainmentColor
22 | // 2. a section click !' opens SectionDrillDownDialog showing teacher/CLO/evidence
23 | // 3. a section with no evidence !' inline empty state renders (not a bare 0%)
24 | // =====
25 |
26 | import { useState, type ReactNode } from "react";
27 | import { describe, it, expect, vi, beforeEach } from "vitest";
28 | import { render, screen, within } from "@testing-library/react";
29 | import userEvent from "@testing-library/user-event";
30 |
31 | import { getAttainmentColor } from "@lib/attainmentClassifier";
32 | import type { SectionDrillDown } from "@hooks/useSectionAttainment";
33 |
34 | // -----
35 | // Mocks (must precede the component import)
36 | // -----
37 |
38 | // `SectionComparisonChart` is pure (props only). `SectionDrillDownDialog` is the
39 | // only consumer of `useSectionAttainment`, via `useSectionDrillDown`. Mock the
40 | // whole hook module so no real Supabase/network call is made; the drill-down
41 | // payload is driven per-test.
42 | const mockUseSectionDrillDown = vi.fn();
43 | vi.mock("@hooks/useSectionAttainment", () => ({
44 |   useSectionAttainment: vi.fn(() => ({ data: [], isLoading: false })),
45 |   useSectionDrillDown: () => mockUseSectionDrillDown(),
46 | }));
47 |
48 | // happy-dom does not implement ResizeObserver, which Radix Dialog relies on.
49 | class MockResizeObserver {
50 |   observe() {}
51 |   unobserve() {}
52 |   disconnect() {}
53 | }
54 | vi.stubGlobal("ResizeObserver", MockResizeObserver);
55 |
56 | import SectionComparisonChart, {
57 |   type SectionData,
58 | } from "@components/shared/SectionComparisonChart";
59 | import SectionDrillDownDialog from "@components/shared/SectionDrillDownDialog";
60 |
61 | // -----
62 | // Fixtures
63 | // -----
64 |
65 | // Real (non-zero) attainment + actual enrolled counts. Section C has no
66 | // evidence (sampleCount 0) so it must render the inline empty state.
67 | const SECTIONS_WITH_DATA: SectionData[] = [
68 |   {
69 |     sectionId: "sec-1",
70 |     sectionCode: "A",
71 |     attainmentPercent: 90, // excellent !' green
72 |     studentCount: 25, // actual enrolled count, NOT capacity
73 |     sampleCount: 12,
74 |   },
75 |   {
76 |     sectionId: "sec-2",
77 |     sectionCode: "B",
78 |     attainmentPercent: 62, // developing !' yellow
79 |     studentCount: 18,
80 |     sampleCount: 8,
81 |   },
82 | ];
83 |
84 | const SECTION_NO_EVIDENCE: SectionData = {
85 |   sectionId: "sec-3",
86 |   sectionCode: "C",
87 |   attainmentPercent: 0,
88 |   studentCount: 0,
89 |   sampleCount: 0,
90 | };
91 |
92 | // Drill-down payload surfaced inside the dialog (teacher / CLO / evidence).
93 | const DRILL_DOWN: SectionDrillDown = {
94 |   section_id: "sec-1",
95 |   section_code: "A",
96 |   teacher_name: "Dr. Alice Carter",
97 |   student_count: 25,
98 |   clos: [
```

```

99 |     {
100 |       clo_id: "clo-1",
101 |       clo_title: "Analyze data structures",
102 |       blooms_level: "analyzing",
103 |       attainment_percent: 88,
104 |       sample_count: 10,
105 |     },
106 |     {
107 |       clo_id: "clo-2",
108 |       clo_title: "Build a REST API",
109 |       blooms_level: "creating",
110 |       attainment_percent: 0,
111 |       sample_count: 0, // no evidence for this CLO
112 |     },
113 |   ],
114 | };
115 |
116 | /** Normalizes a CSS color the same way happy-dom stores inline styles. */
117 | const normalizeColor = (color: string): string => {
118 |   const el = document.createElement("span");
119 |   el.style.color = color;
120 |   return el.style.color;
121 | };
122 |
123 | // A tiny harness that reproduces the dashboard's chart -> dialog wiring without
124 | // mocking the whole dashboard: clicking a section sets the selected section id,
125 | // which opens the SectionDrillDownDialog.
126 | const ComparisonHarness = ({ sections }: { sections: SectionData[] }) => {
127 |   const [sectionId, setSectionId] = useState<string | null>(null);
128 |   return (
129 |     <>
130 |       <SectionComparisonChart
131 |         sections={sections}
132 |         onClick={section => setSectionId(section.sectionId ?? null)}
133 |       />
134 |       <SectionDrillDownDialog
135 |         courseId="course-1"
136 |         sectionId={sectionId ?? undefined}
137 |         open={sectionId !== null}
138 |         onChange={open => {
139 |           if (!open) setSectionId(null);
140 |         }}
141 |       />
142 |     </>
143 |   );
144 | };
145 |
146 | const renderNode = (node: ReactNode) => render(<>{node}</>);
147 |
148 | // -----
149 | // Tests
150 | // -----
151 |
152 | describe("Coordinator Section Comparison (Req 10)", () => {
153 |   beforeEach(() => {
154 |     vi.clearAllMocks();
155 |     mockUseSectionDrillDown.mockReturnValue({
156 |       data: DRILL_DOWN,
157 |       isLoading: false,
158 |     });
159 |   });
160 |
161 |   // % % % Req 10.1 / 10.2 / 10.5 – real attainment + enrolled count + color % % %
162 |
163 |   it("renders real attainment values and actual enrolled counts per section", () => {
164 |     // Validates Requirements 10.1, 10.2
165 |     renderNode(<SectionComparisonChart sections={SECTIONS_WITH_DATA} />);
166 |
167 |     // Real attainment (not a hardcoded 0%) is displayed per section.
168 |     expect(screen.getByText("90%")).toBeInTheDocument();
169 |     expect(screen.getByText("62%")).toBeInTheDocument();
170 |
171 |     // Each section is labelled by its code.
172 |     expect(screen.getByText("Section A")).toBeInTheDocument();
173 |     expect(screen.getByText("Section B")).toBeInTheDocument();
174 |
175 |     // Actual enrolled student counts (NOT section capacity) are displayed.
176 |     expect(screen.getByText("25 students")).toBeInTheDocument();
177 |     expect(screen.getByText("18 students")).toBeInTheDocument();
178 |   });
179 |
180 |   it("color-codes attainment values via getAttainmentColor", () => {
181 |     // Validates Requirement 10.1/10.5 – distinct attainment levels !' distinct colors
182 |     renderNode(<SectionComparisonChart sections={SECTIONS_WITH_DATA} />);
183 |
184 |     const excellent = screen.getByText("90%");
185 |     const developing = screen.getByText("62%");
186 |
187 |     // Each value carries the platform attainment color for its percentage.
188 |     expect(excellent.style.color).toBe(normalizeColor(getAttainmentColor(90)));
189 |     expect(developing.style.color).toBe(normalizeColor(getAttainmentColor(62)));
190 |
191 |     // Excellent (90%) and Developing (62%) map to different colors.
192 |     expect(excellent.style.color).not.toBe(developing.style.color);
193 |   });
194 |
195 |   // % % % Req 10.4 – inline empty state for a section with no evidence % % % % %
196 |
197 |   it("renders an inline empty state for a section with no evidence instead of a 0% bar", () => {
198 |     // Validates Requirement 10.4
199 |     renderNode(<SectionComparisonChart sections={[SECTION_NO_EVIDENCE]} />);
200 |
201 |     // The inline empty state copy is shown for the evidence-less section.
202 |     expect(

```

```

203 |     screen.getByText("No attainment evidence recorded for this section yet.")
204 |   ).toBeInTheDocument();
205 |   expect(screen.getByText("No evidence yet")).toBeInTheDocument();
206 |
207 |   // A misleading "0%" attainment value is NOT rendered.
208 |   expect(screen.queryByText("0%")).not.toBeInTheDocument();
209 | });
210 |
211 | // % % % Req 10.3 – section click opens the drill-down with detail % % % % % % % %
212 |
213 | it("makes each section clickable when a click handler is provided", () => {
214 |   // Validates Requirement 10.3 (affordance)
215 |   const onSectionClick = vi.fn();
216 |   renderNode(
217 |     <SectionComparisonChart
218 |       sections={SECTIONS_WITH_DATA}
219 |       onSectionClick={onSectionClick}
220 |     />
221 |   );
222 |
223 |   const sectionAButton = screen.getByRole("button", {
224 |     name: /view details for section a/i,
225 |   });
226 |   expect(sectionAButton).toBeInTheDocument();
227 | });
228 |
229 | it("opens the section drill-down dialog showing teacher / CLO / evidence on click", async () => {
230 |   // Validates Requirement 10.3
231 |   const user = userEvent.setup();
232 |   renderNode(<ComparisonHarness sections={SECTIONS_WITH_DATA} />);
233 |
234 |   // The drill-down detail is not present before the section is selected.
235 |   expect(screen.queryByText("Dr. Alice Carter")).not.toBeInTheDocument();
236 |   expect(
237 |     screen.queryByText("Analyze data structures")
238 |   ).not.toBeInTheDocument();
239 |
240 |   await user.click(
241 |     screen.getByRole("button", { name: /view details for section a/i })
242 |   );
243 |
244 |   // The dialog opens with the section's teacher / CLO / evidence breakdown.
245 |   const dialog = await screen.findByRole("dialog");
246 |   const inDialog = within(dialog);
247 |
248 |   // Teacher detail.
249 |   expect(inDialog.getByText("Dr. Alice Carter")).toBeInTheDocument();
250 |   expect(inDialog.getByText("25 students")).toBeInTheDocument();
251 |
252 |   // Per-CLO attainment evidence.
253 |   expect(inDialog.getByText("Analyze data structures")).toBeInTheDocument();
254 |   expect(inDialog.getByText("88%")).toBeInTheDocument();
255 |   expect(inDialog.getByText("Build a REST API")).toBeInTheDocument();
256 |   // The evidence-less CLO shows "No evidence" rather than a 0% value.
257 |   expect(inDialog.getByText("No evidence")).toBeInTheDocument();
258 | });
259 |
260 | it("does not open the drill-down dialog before any section is selected", () => {
261 |   // Validates Requirement 10.3 (dialog gated on selection)
262 |   renderNode(<ComparisonHarness sections={SECTIONS_WITH_DATA} />);
263 |   expect(screen.queryByRole("dialog")).not.toBeInTheDocument();
264 | });
265 | });
266 |

```

FILE: [src/__tests__/unit/sectionManager.test.tsx](#)

TYPE: .tsx | LINES: 1-110 | SHA256: 6439f238f17cfe2f0a92b67c0313be39d2a849fe3c9f890a6e5dbf6f961ab09c

```

1 | // @vitest-environment happy-dom
2 | import { describe, it, expect, vi } from "vitest";
3 | import { render, screen } from "@testing-library/react";
4 | import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
5 | import { MemoryRouter } from "react-router-dom";
6 |
7 | // Mock hooks
8 | vi.mock("@/hooks/useCourseSections", () => ({
9 |   useCourseSections: vi.fn(() => ({
10 |     data: [
11 |       {
12 |         id: "sec-1",
13 |         course_id: "course-1",
14 |         section_code: "A",
15 |         teacher_id: "t1",
16 |         capacity: 40,
17 |         is_active: true,
18 |         created_at: "2025-01-01",
19 |         profiles: { id: "t1", full_name: "Dr. Smith", email: "smith@uni.edu" },
20 |       },
21 |       {
22 |         id: "sec-2",
23 |         course_id: "course-1",
24 |         section_code: "B",
25 |         teacher_id: "t2",
26 |         capacity: 35,
27 |         is_active: false,
28 |         created_at: "2025-01-01",
29 |         profiles: { id: "t2", full_name: "Dr. Jones", email: "jones@uni.edu" },
30 |       },
31 |     ],
32 |     isLoading: false,
33 |   })),
34 |   useCreateCourseSection: vi.fn(() => ({ mutate: vi.fn(), isPending: false })),
35 |   useUpdateCourseSection: vi.fn(() => ({ mutate: vi.fn(), isPending: false })),
36 |   useDeleteCourseSection: vi.fn(() => ({ mutate: vi.fn(), isPending: false })),

```

```

37 |   }));
38 |
39 |   vi.mock("@/hooks/useCourses", () => ({
40 |     useTeachers: vi.fn(() => ({
41 |       data: [
42 |         { id: "t1", full_name: "Dr. Smith" },
43 |         { id: "t2", full_name: "Dr. Jones" },
44 |       ],
45 |     })),
46 |   }));
47 |
48 |   vi.mock("@/hooks/useAuth", () => ({
49 |     useAuth: () => ({ user: { id: "u1" }, institutionId: "inst-1" }),
50 |   }));
51 |
52 |   import SectionManager from "@/components/shared/SectionManager";
53 |
54 |   const createWrapper = () => {
55 |     const qc = new QueryClient({ defaultOptions: { queries: { retry: false } } });
56 |     return ({ children }: { children: React.ReactNode }) => (
57 |       <QueryClientProvider client={qc}>
58 |         <MemoryRouter>{children}</MemoryRouter>
59 |       </QueryClientProvider>
60 |     );
61 |   };
62 |
63 |   describe("SectionManager", () => {
64 |     it("renders section title", () => {
65 |       render(<SectionManager courseId="course-1" courseName="CS101" />, {
66 |         wrapper: createWrapper(),
67 |       });
68 |       expect(screen.getByText("Sections - CS101")).toBeInTheDocument();
69 |     });
70 |
71 |     it("renders add section button", () => {
72 |       render(<SectionManager courseId="course-1" />, {
73 |         wrapper: createWrapper(),
74 |       });
75 |       expect(screen.getByText("Add Section")).toBeInTheDocument();
76 |     });
77 |
78 |     it("renders section codes", () => {
79 |       render(<SectionManager courseId="course-1" />, {
80 |         wrapper: createWrapper(),
81 |       });
82 |       expect(screen.getByText("A")).toBeInTheDocument();
83 |       expect(screen.getByText("B")).toBeInTheDocument();
84 |     });
85 |
86 |     it("renders teacher names", () => {
87 |       render(<SectionManager courseId="course-1" />, {
88 |         wrapper: createWrapper(),
89 |       });
90 |       expect(screen.getByText("Dr. Smith")).toBeInTheDocument();
91 |       expect(screen.getByText("Dr. Jones")).toBeInTheDocument();
92 |     });
93 |
94 |     it("renders active/inactive badges", () => {
95 |       render(<SectionManager courseId="course-1" />, {
96 |         wrapper: createWrapper(),
97 |       });
98 |       expect(screen.getByText("Active")).toBeInTheDocument();
99 |       expect(screen.getByText("Inactive")).toBeInTheDocument();
100 |     });
101 |
102 |     it("renders capacity info", () => {
103 |       render(<SectionManager courseId="course-1" />, {
104 |         wrapper: createWrapper(),
105 |       });
106 |       expect(screen.getByText("Capacity: 40")).toBeInTheDocument();
107 |       expect(screen.getByText("Capacity: 35")).toBeInTheDocument();
108 |     });
109 |   });
110 |

```

FILE: [src/__tests__/unit/sectionState.test.tsx](#)

TYPE: .tsx | LINES: 1-101 | SHA256: 5154564ab6d8fa489483712140451c340664832c9bceae052b6d6da8f7637b28

```

1 | // @vitest-environment happy-dom
2 | // =====
3 | // Feature: dashboard-and-ux-performance - Phase 7 Task 24 (Appendix B §B.3.5).
4 | // Proves <SectionState> renders the right branch and - critically - surfaces a
5 | // RETRYABLE error instead of vanishing when a query fails with no data, so a
6 | // cancelled/timed-out section no longer silently disappears.
7 | // =====
8 | import { describe, it, expect, vi } from "vitest";
9 | import { render, screen, fireEvent } from "@testing-library/react";
10 | import SectionState from "@/components/shared/SectionState";
11 |
12 | // Hermetic i18n: return the key so assertions are stable regardless of locale.
13 | vi.mock("react-i18next", () => ({
14 |   useTranslation: () => ({ t: (key: string) => key }),
15 | }));
16 |
17 | describe("SectionState", () => {
18 |   it("renders the loading fallback while loading with no data", () => {
19 |     render(
20 |       <SectionState
21 |         isLoading
22 |         isError={false}
23 |         data={undefined}
24 |         loadingFallback=<div data-testid="skeleton" />
25 |       >
26 |         {() => <div data-testid="content" />}

```

```

27 |     </SectionState>
28 |   );
29 |   expect(screen.getByTestId("skeleton")).toBeTruthy();
30 |   expect(screen.queryByTestId("content")).toBeNull();
31 | });
32 |
33 | it("surfaces a retryable error (not silence) when the query fails with no data", () => {
34 |   const onRetry = vi.fn();
35 |   render(
36 |     <SectionState
37 |       isLoading={false}
38 |       isError
39 |       data={undefined}
40 |       onRetry={onRetry}
41 |     >
42 |       {() => <div data-testid="content" />}
43 |     </SectionState>
44 |   );
45 |   // Error copy + retry are visible...
46 |   expect(screen.getByText("errorBoundary.title")).toBeTruthy();
47 |   const retry = screen.getByRole("button", { name: "actions.retry" });
48 |   expect(retry).toBeTruthy();
49 |   // ...and clicking retry invokes the caller's refetch.
50 |   fireEvent.click(retry);
51 |   expect(onRetry).toHaveBeenCalledTimes(1);
52 |   // The content never rendered.
53 |   expect(screen.queryByTestId("content")).toBeNull();
54 | });
55 |
56 | it("renders children when data is present and non-empty", () => {
57 |   render(
58 |     <SectionState isLoading={false} isError={false} data={[1, 2, 3]}>
59 |       {(items) => <div data-testid="content">{items.length}</div>}
60 |     </SectionState>
61 |   );
62 |   expect(screen.getByTestId("content").textContent).toBe("3");
63 | });
64 |
65 | it("renders the empty fallback when the query succeeded but the result is empty", () => {
66 |   render(
67 |     <SectionState
68 |       isLoading={false}
69 |       isError={false}
70 |       data={[]}
71 |       emptyFallback={<div data-testid="empty" />}
72 |     >
73 |       {() => <div data-testid="content" />}
74 |     </SectionState>
75 |   );
76 |   expect(screen.getByTestId("empty")).toBeTruthy();
77 |   expect(screen.queryByTestId("content")).toBeNull();
78 | });
79 |
80 | it("prefers showing (stale) data over an error when both are present", () => {
81 |   // data exists from a prior success, a background refetch then errored.
82 |   render(
83 |     <SectionState isLoading={false} isError data={[42]}>
84 |       {(items) => <div data-testid="content">{items[0]}</div>}
85 |     </SectionState>
86 |   );
87 |   expect(screen.getByTestId("content").textContent).toBe("42");
88 |   expect(screen.queryByText("errorBoundary.title")).toBeNull();
89 | });
90 |
91 | it("renders nothing when idle with no data (no spurious error)", () => {
92 |   const { container } = render(
93 |     <SectionState isLoading={false} isError={false} data={undefined}>
94 |       {() => <div data-testid="content" />}
95 |     </SectionState>
96 |   );
97 |   expect(container.textContent).toBe("");
98 |   expect(screen.queryByTestId("content")).toBeNull();
99 | });
100 | });
101 |

```

FILE: [src/_tests_/unit/selfEfficacyStep.test.tsx](#)

TYPE: .tsx | LINES: 1-250 | SHA256: 372e595cab58d5f28021a17d08427683175bd1405c58eb1161a9662a4fb2a3e4

```

1 | // @vitest-environment happy-dom
2 | // =====
3 | // SelfEfficacyStep - Unit tests (Task 18.4)
4 | //
5 | // Covers two of the three behaviours required by task 18.4:
6 | //   • Non-blocking fallback (task 18.1 / R11.3, R11.4, R11.4a, R11.4b):
7 | //     - a non-alarming zero-data fallback (never an admin-contact error)
8 | //     - when there are genuinely zero questions;
9 | //     - a DISTINCT system-issue panel on query error;
10 | //     - both expose a Continue that advances the sequence (never blocks).
11 | //   • Framing gate (task 18.3 / R17.2a):
12 | //     - the AssessmentIntro (benefit + estimated time) renders first and the
13 | //       question body stays gated until the student presses "Begin".
14 | //
15 | // The onboarding question/response hooks are mocked; the assessment-intro
16 | // content resolves from the real i18n bundle via I18nextProvider so the
17 | // benefit + estimated-time gate is exercised against production copy.
18 | // =====
19 |
20 | import { describe, it, expect, vi, beforeEach } from "vitest";
21 | import { render, screen } from "@testing-library/react";
22 | import { fireEvent } from "@testing-library/user-event";
23 | import { I18nextProvider } from "react-i18next";
24 |
25 | import i18n from "@lib/i18n";

```

[illegible]

[illegible]

```

234 |     expect(screen.queryByRole("radiogroup")).not.toBeInTheDocument();
235 |
236 |     await user.click(screen.getByRole("button", { name: /begin/i }));
237 |
238 |     // Now the body renders: the Likert radiogroup and the first question.
239 |     expect(screen.getByRole("radiogroup")).toBeInTheDocument();
240 |     expect(
241 |       screen.getByText("I can master difficult coursework when I try.")
242 |     ).toBeInTheDocument();
243 |     // The intro Begin button is gone once the body renders.
244 |     expect(
245 |       screen.queryByRole("button", { name: /begin/i })
246 |     ).not.toBeInTheDocument();
247 |   });
248 | });
249 | });
250 |

```

FILE: [src/___tests___/unit/semesterManager.test.tsx](#)

TYPE: .tsx | LINES: 1-93 | SHA256: 6f90c5b5165c695edafe9edda50a52653df822033d79a92c8435ac04a7beac14

```

1 | // @vitest-environment happy-dom
2 | import { describe, it, expect, vi } from "vitest";
3 | import { render, screen } from "@testing-library/react";
4 | import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
5 | import { MemoryRouter } from "react-router-dom";
6 |
7 | // Mock hooks
8 | vi.mock("@/hooks/useSemesters", () => ({
9 |   useSemesters: vi.fn(() => ({
10 |     data: [
11 |       {
12 |         id: "s1",
13 |         name: "Fall 2025",
14 |         code: "F25",
15 |         start_date: "2025-09-01",
16 |         end_date: "2025-12-31",
17 |         is_active: true,
18 |         institution_id: "inst-1",
19 |         created_at: "2025-01-01",
20 |       },
21 |       {
22 |         id: "s2",
23 |         name: "Spring 2026",
24 |         code: "S26",
25 |         start_date: "2026-01-15",
26 |         end_date: "2026-05-31",
27 |         is_active: false,
28 |         institution_id: "inst-1",
29 |         created_at: "2025-01-01",
30 |       },
31 |     ],
32 |     isLoading: false,
33 |   })),
34 |   useCreateSemester: vi.fn(() => ({ mutate: vi.fn(), isPending: false })),
35 |   useUpdateSemester: vi.fn(() => ({ mutate: vi.fn(), isPending: false })),
36 |   useDeleteSemester: vi.fn(() => ({ mutate: vi.fn(), isPending: false })),
37 |   useToggleSemesterActive: vi.fn(() => ({ mutate: vi.fn(), isPending: false })),
38 | }));
39 |
40 | vi.mock("@/hooks/useAuth", () => ({
41 |   useAuth: () => ({
42 |     user: { id: "u1" },
43 |     institutionId: "inst-1",
44 |     profile: { role: "admin" },
45 |   }),
46 | }));
47 |
48 | import SemesterManager from "@/pages/admin/semesters/SemesterManager";
49 |
50 | const createWrapper = () => {
51 |   const qc = new QueryClient({ defaultOptions: { queries: { retry: false } } });
52 |   return ({ children }: { children: React.ReactNode }) => (
53 |     <QueryClientProvider client={qc}>
54 |       <MemoryRouter>{children}</MemoryRouter>
55 |     </QueryClientProvider>
56 |   );
57 | };
58 |
59 | describe("SemesterManager", () => {
60 |   it("renders page title", () => {
61 |     render(<SemesterManager />, { wrapper: createWrapper() });
62 |     expect(screen.getByText("Semesters")).toBeInTheDocument();
63 |   });
64 |
65 |   it("renders semester list", () => {
66 |     render(<SemesterManager />, { wrapper: createWrapper() });
67 |     expect(screen.getByText("Fall 2025")).toBeInTheDocument();
68 |     expect(screen.getByText("Spring 2026")).toBeInTheDocument();
69 |   });
70 |
71 |   it("shows active badge for active semester", () => {
72 |     render(<SemesterManager />, { wrapper: createWrapper() });
73 |     expect(screen.getByText("Active")).toBeInTheDocument();
74 |   });
75 |
76 |   it("renders add semester button", () => {
77 |     render(<SemesterManager />, { wrapper: createWrapper() });
78 |     expect(screen.getByText("Add Semester")).toBeInTheDocument();
79 |   });
80 |
81 |   it("renders date ranges", () => {
82 |     render(<SemesterManager />, { wrapper: createWrapper() });
83 |     expect(screen.getByText("2025-09-01 – 2025-12-31")).toBeInTheDocument();

```

FILE: src/__tests__/unit/sessionCompletionForm.test.tsx

```
1 // =====
2 // Unit Tests – SessionCompletionForm Component
3 // =====
```

[illegible]

```
// %%% Helpers %%%%%%%%%%%%%%
```

[illegible]

```

54 describe("SessionCompletionForm", () => {
55   const onComplete = vi.fn();
56
57   beforeEach(() => {
58     vi.clearAllMocks();
59   });
60
61   describe("Rendering", () => {
62     it("renders the completion header with duration", () => {
63       render(
64         <SessionCompletionForm
65           session={makeSession()}
66           actualDurationMinutes={25}
67           onComplete={onComplete}
68         />
69       );
70       expect(screen.getByText("Session Complete!")).toBeTruthy();
71       expect(screen.getByText(/25 minutes/)).toBeTruthy();
72     });
73
74     it("renders satisfaction rating stars", () => {
75       render(
76         <SessionCompletionForm
77           session={makeSession()}
78           actualDurationMinutes={25}
79           onComplete={onComplete}
80         />
81       );
82       const stars = screen.getAllByRole("button", { name: /rate \d star/i });
83       expect(stars).toHaveLength(5);
84     });
85
86     it("renders session notes textarea", () => {
87       render(
88         <SessionCompletionForm
89           session={makeSession()}
90

```

```

91 |         actualDurationMinutes={25}
92 |         onComplete={onComplete}
93 |     />
94 | );
95 | expect(screen.getByPlaceholderText(/what did you work on/i)).toBeTruthy();
96 | });
97 |
98 | it("renders evidence section with QuickThoughtInput as primary option", () => {
99 |     render(
100 |         <SessionCompletionForm
101 |             session={makeSession()}
102 |             actualDurationMinutes={25}
103 |             onComplete={onComplete}
104 |         />
105 |     );
106 |     expect(screen.getByText("Evidence")).toBeTruthy();
107 |     // QuickThoughtInput is shown by default
108 |     expect(screen.getByLabelText(/quick thought/i)).toBeTruthy();
109 | });
110 |
111 | it("renders Attach files toggle button", () => {
112 |     render(
113 |         <SessionCompletionForm
114 |             session={makeSession()}
115 |             actualDurationMinutes={25}
116 |             onComplete={onComplete}
117 |         />
118 |     );
119 |     expect(screen.getByText("Attach files")).toBeTruthy();
120 | });
121 |
122 | it("does not render EvidenceUploader by default", () => {
123 |     render(
124 |         <SessionCompletionForm
125 |             session={makeSession()}
126 |             actualDurationMinutes={25}
127 |             onComplete={onComplete}
128 |         />
129 |     );
130 |     // The file upload area should not be visible until "Attach files" is clicked
131 |     expect(screen.queryByLabelText(/upload evidence files/i)).toBeNull();
132 | });
133 |
134 | it("renders session reflection input", () => {
135 |     render(
136 |         <SessionCompletionForm
137 |             session={makeSession()}
138 |             actualDurationMinutes={25}
139 |             onComplete={onComplete}
140 |         />
141 |     );
142 |     expect(screen.getByText("Session Reflection")).toBeTruthy();
143 | });
144 |
145 | it("renders Submit and Skip buttons", () => {
146 |     render(
147 |         <SessionCompletionForm
148 |             session={makeSession()}
149 |             actualDurationMinutes={25}
150 |             onComplete={onComplete}
151 |         />
152 |     );
153 |     expect(
154 |         screen.getByRole("button", { name: /submit & finish/i })
155 |     ).toBeTruthy();
156 |     expect(screen.getByRole("button", { name: /skip/i })).toBeTruthy();
157 | });
158 | });
159 |
160 | describe("Star Rating", () => {
161 |     it("allows selecting a star rating", async () => {
162 |         const user = userEvent.setup();
163 |         render(
164 |             <SessionCompletionForm
165 |                 session={makeSession()}
166 |                 actualDurationMinutes={25}
167 |                 onComplete={onComplete}
168 |             />
169 |         );
170 |
171 |         const star3 = screen.getByRole("button", { name: /rate 3 stars/i });
172 |         await user.click(star3);
173 |
174 |         // After clicking 3 stars, stars 1-3 should be filled (amber)
175 |         const stars = screen.getAllByRole("button", { name: /rate \d star/i });
176 |         // Star 4 and 5 should not be filled
177 |         expect(stars[3]?.className).toContain("text-gray-300");
178 |     });
179 | });
180 |
181 | describe("QuickThoughtInput Integration", () => {
182 |     it("shows QuickThoughtInput by default in the evidence section", () => {
183 |         render(
184 |             <SessionCompletionForm
185 |                 session={makeSession()}
186 |                 actualDurationMinutes={25}
187 |                 onComplete={onComplete}
188 |             />
189 |         );
190 |         expect(screen.getByLabelText(/quick thought/i)).toBeTruthy();
191 |         expect(
192 |             screen.getByPlaceholderText(/what's the one thing you'll remember/i)
193 |         ).toBeTruthy();
194 |     });

```

```

195 |
196 | it("appends quick thought to session notes on submit", async () => {
197 |   const user = userEvent.setup();
198 |   render(
199 |     <SessionCompletionForm
200 |       session={makeSession()}
201 |       actualDurationMinutes={25}
202 |       onComplete={onComplete}
203 |     />
204 |   );
205 |
206 |   const thoughtInput = screen.getByLabelText(/quick thought/i);
207 |   await user.type(thoughtInput, "Learned about chain rule");
208 |
209 |   const submitThought = screen.getByRole("button", {
210 |     name: /submit thought/i,
211 |   });
212 |   await user.click(submitThought);
213 |
214 |   // Should show confirmation message
215 |   expect(screen.getByText(/quick thought captured/i)).toBeTruthy();
216 |   // QuickThoughtInput should no longer be visible
217 |   expect(screen.queryByLabelText(/quick thought/i)).toBeNull();
218 | });
219 |
220 | it("includes quick thought text in notes when submitting the form", async () => {
221 |   const user = userEvent.setup();
222 |   render(
223 |     <SessionCompletionForm
224 |       session={makeSession()}
225 |       actualDurationMinutes={30}
226 |       onComplete={onComplete}
227 |     />
228 |   );
229 |
230 |   // Submit a quick thought first
231 |   const thoughtInput = screen.getByLabelText(/quick thought/i);
232 |   await user.type(thoughtInput, "Key insight about derivatives");
233 |   await user.click(screen.getByRole("button", { name: /submit thought/i }));
234 |
235 |   // Now submit the form
236 |   await user.click(
237 |     screen.getByRole("button", { name: /submit & finish/i })
238 |   );
239 |
240 |   expect(mockMutate).toHaveBeenCalledTimes(1);
241 |   const callArgs = mockMutate.mock.calls[0][0];
242 |   expect(callArgs.notes).toContain("Key insight about derivatives");
243 | });
244 |
245 | it("combines existing notes with quick thought text", async () => {
246 |   const user = userEvent.setup();
247 |   render(
248 |     <SessionCompletionForm
249 |       session={makeSession()}
250 |       actualDurationMinutes={30}
251 |       onComplete={onComplete}
252 |     />
253 |   );
254 |
255 |   // Type notes first
256 |   const notesInput = screen.getByPlaceholderText(/what did you work on/i);
257 |   await user.type(notesInput, "Studied calculus");
258 |
259 |   // Then submit a quick thought
260 |   const thoughtInput = screen.getByLabelText(/quick thought/i);
261 |   await user.type(thoughtInput, "Chain rule is key");
262 |   await user.click(screen.getByRole("button", { name: /submit thought/i }));
263 |
264 |   // Submit the form
265 |   await user.click(
266 |     screen.getByRole("button", { name: /submit & finish/i })
267 |   );
268 |
269 |   const callArgs = mockMutate.mock.calls[0][0];
270 |   expect(callArgs.notes).toContain("Studied calculus\n\nChain rule is key");
271 | });
272 | });
273 |
274 | describe("Attach Files Toggle", () => {
275 |   it("expands EvidenceUploader when Attach files is clicked", async () => {
276 |     const user = userEvent.setup();
277 |     render(
278 |       <SessionCompletionForm
279 |         session={makeSession()}
280 |         actualDurationMinutes={25}
281 |         onComplete={onComplete}
282 |       />
283 |     );
284 |
285 |     // Initially no file uploader
286 |     expect(screen.queryByLabelText(/upload evidence files/i)).toBeNull();
287 |
288 |     // Click "Attach files"
289 |     await user.click(screen.getByText("Attach files"));
290 |
291 |     // Now the EvidenceUploader should be visible
292 |     expect(screen.getByLabelText(/upload evidence files/i)).toBeTruthy();
293 |   });
294 |
295 | it("collapses EvidenceUploader when Hide is clicked", async () => {
296 |   const user = userEvent.setup();
297 |   render(
298 |     <SessionCompletionForm

```

```

299         session={makeSession()}
300         actualDurationMinutes={25}
301         onComplete={onComplete}
302     />
303 );
304
305 // Expand
306 await user.click(screen.getByText("Attach files"));
307 expect(screen.getByLabelText(/upload evidence files/i)).toBeTruthy();
308
309 // Collapse
310 await user.click(screen.getByText("Hide"));
311 expect(screen.queryByLabelText(/upload evidence files/i)).toBeNull();
312 });
313 });
314
315 describe("Submit Flow", () => {
316     it("calls completeSession.mutate with form data on submit", async () => {
317         const user = userEvent.setup();
318         render(
319             <SessionCompletionForm
320                 session={makeSession()}
321                 actualDurationMinutes={30}
322                 onComplete={onComplete}
323             />
324         );
325
326         // Add notes
327         const notesInput = screen.getByPlaceholderText(/what did you work on/i);
328         await user.type(notesInput, "Studied derivatives");
329
330         // Rate 4 stars
331         await user.click(screen.getByRole("button", { name: /rate 4 stars/i }));
332
333         // Submit
334         await user.click(
335             screen.getByRole("button", { name: /submit & finish/i })
336         );
337
338         expect(mockMutate).toHaveBeenCalledOnce();
339         const callArgs = mockMutate.mock.calls[0]?.[0];
340         expect(callArgs).toMatchObject({
341             sessionId: "session-1",
342             actualDurationMinutes: 30,
343             notes: "Studied derivatives",
344             satisfactionRating: 4,
345         });
346     });
347
348     it("calls completeSession.mutate with minimal data on skip", async () => {
349         const user = userEvent.setup();
350         render(
351             <SessionCompletionForm
352                 session={makeSession()}
353                 actualDurationMinutes={25}
354                 onComplete={onComplete}
355             />
356         );
357
358         await user.click(screen.getByRole("button", { name: /skip/i }));
359
360         expect(mockMutate).toHaveBeenCalledOnce();
361         const callArgs = mockMutate.mock.calls[0]?.[0];
362         expect(callArgs).toMatchObject({
363             sessionId: "session-1",
364             actualDurationMinutes: 25,
365         });
366         // No notes, rating, or evidence on skip
367         expect(callArgs.notes).toBeUndefined();
368         expect(callArgs.satisfactionRating).toBeUndefined();
369         expect(callArgs.evidenceFiles).toBeUndefined();
370     });
371 });
372
373 describe("Reflection", () => {
374     it("renders reflection word count indicator", () => {
375         render(
376             <SessionCompletionForm
377                 session={makeSession()}
378                 actualDurationMinutes={25}
379                 onComplete={onComplete}
380             />
381         );
382         expect(screen.getByText(/0 \/ 30 words/)).toBeTruthy();
383     });
384
385     it("disables save reflection button when below minimum words", () => {
386         render(
387             <SessionCompletionForm
388                 session={makeSession()}
389                 actualDurationMinutes={25}
390                 onComplete={onComplete}
391             />
392         );
393         const saveBtn = screen.getByRole("button", { name: /save reflection/i });
394         expect(saveBtn).toBeDisabled();
395     });
396 });
397 });
398

```

TYPE: .tsx | LINES: 1-267 | SHA256: a1ccaff297ba0911961b4a2f1788f1ad8e4c66877c64b4b0e7a9aeb46db3943

101 | Page What specific concepts will you work on?

```

102 |     ) as HTMLInputElement;
103 |     const criterionInput = screen.getByLabelText(
104 |       /what does success look like/i
105 |     ) as HTMLInputElement;
106 |
107 |     expect(conceptInput.value).toBe("Quadratic equations");
108 |     expect(criterionInput.value).toBe(
109 |       "Solve 3 practice problems on Quadratic equations correctly"
110 |     );
111 |   });
112 |
113 |   it("highlights the selected chip", async () => {
114 |     const user = userEvent.setup();
115 |     render(<SessionIntentDialog {...defaultProps} />);
116 |
117 |     const chip = screen.getByRole("button", {
118 |       name: /use suggestion: quadratic equations/i,
119 |     });
120 |     await user.click(chip);
121 |
122 |     // The badge inside the chip should have the active styling
123 |     const badge = chip.querySelector("[data-slot='badge']");
124 |     expect(badge?.className).toContain("bg-teal-50");
125 |   });
126 | });
127 |
128 | describe("Skip Action", () => {
129 |   it("calls onSkip and closes dialog when Skip is clicked", async () => {
130 |     const user = userEvent.setup();
131 |     render(<SessionIntentDialog {...defaultProps} />);
132 |
133 |     await user.click(screen.getByRole("button", { name: /skip/i }));
134 |
135 |     expect(defaultProps.onSkip).toHaveBeenCalledOnce();
136 |     expect(defaultProps.onOpenChange).toHaveBeenCalledWith(false);
137 |   });
138 |
139 |   it("does not call mutate when Skip is clicked", async () => {
140 |     const user = userEvent.setup();
141 |     render(<SessionIntentDialog {...defaultProps} />);
142 |
143 |     await user.click(screen.getByRole("button", { name: /skip/i }));
144 |
145 |     expect(mockMutate).not.toHaveBeenCalled();
146 |   });
147 | });
148 |
149 | describe("Submit Action", () => {
150 |   it("calls mutate with form data on valid submit", async () => {
151 |     const user = userEvent.setup();
152 |     render(<SessionIntentDialog {...defaultProps} />);
153 |
154 |     const conceptInput = screen.getByLabelText(
155 |       /what specific concept will you work on/i
156 |     );
157 |     const criterionInput = screen.getByLabelText(
158 |       /what does success look like/i
159 |     );
160 |
161 |     await user.type(conceptInput, "Linear algebra basics");
162 |     await user.type(
163 |       criterionInput,
164 |       "Complete matrix multiplication exercises"
165 |     );
166 |
167 |     await user.click(screen.getByRole("button", { name: /submit/i }));
168 |
169 |     await waitFor(() => {
170 |       expect(mockMutate).toHaveBeenCalledOnce();
171 |     });
172 |
173 |     const callArgs = mockMutate.mock.calls[0]?.[0];
174 |     expect(callArgs).toMatchObject({
175 |       sessionId: "session-123",
176 |       concept: "Linear algebra basics",
177 |       successCriterion: "Complete matrix multiplication exercises",
178 |       isAutoSuggested: false,
179 |     });
180 |   });
181 |
182 |   it("marks isAutoSuggested true when submitting a suggestion", async () => {
183 |     const user = userEvent.setup();
184 |     render(<SessionIntentDialog {...defaultProps} />);
185 |
186 |     // Click a suggestion chip
187 |     await user.click(
188 |       screen.getByRole("button", {
189 |         name: /use suggestion: quadratic equations/i,
190 |       })
191 |     );
192 |
193 |     await user.click(screen.getByRole("button", { name: /submit/i }));
194 |
195 |     await waitFor(() => {
196 |       expect(mockMutate).toHaveBeenCalledOnce();
197 |     });
198 |
199 |     const callArgs = mockMutate.mock.calls[0]?.[0];
200 |     expect(callArgs.isAutoSuggested).toBe(true);
201 |   });
202 |
203 |   it("shows validation errors for empty fields", async () => {
204 |     const user = userEvent.setup();
205 |     render(<SessionIntentDialog {...defaultProps} />);

```



```

207     await user.click(screen.getByRole("button", { name: /submit/i }));
208
209     await waitFor(() => {
210       expect(
211         screen.getAllByText(/at least 5 characters/i).length
212       ).toBeGreaterThanOrEqual(1);
213     });
214
215     expect(mockMutate).not.toHaveBeenCalled();
216   });
217
218   it("shows validation error for too-short concept", async () => {
219     const user = userEvent.setup();
220     render(<SessionIntentDialog {...defaultProps} />);
221
222     const conceptInput = screen.getByLabelText(
223       /what specific concept will you work on/i
224     );
225     await user.type(conceptInput, "abc");
226
227     await user.click(screen.getByRole("button", { name: /submit/i }));
228
229     await waitFor(() => {
230       expect(
231         screen.getAllByText(/at least 5 characters/i).length
232       ).toBeGreaterThanOrEqual(1);
233     });
234   });
235 });
236
237 describe("Accessibility", () => {
238   it("has an accessible dialog label", () => {
239     render(<SessionIntentDialog {...defaultProps} />);
240     expect(
241       screen.getByRole("dialog", { name: /set your intent/i })
242     ).toBeTruthy();
243   });
244
245   it("has an accessible group for suggested intents", () => {
246     render(<SessionIntentDialog {...defaultProps} />);
247     expect(
248       screen.getByRole("group", { name: /suggested intents/i })
249     ).toBeTruthy();
250   });
251
252   it("each chip has an accessible label", () => {
253     render(<SessionIntentDialog {...defaultProps} />);
254     expect(
255       screen.getByRole("button", {
256         name: /use suggestion: quadratic equations/i,
257       })
258     ).toBeTruthy();
259     expect(
260       screen.getByRole("button", {
261         name: /use suggestion: chapter 5 review/i,
262       })
263     ).toBeTruthy();
264   });
265 });
266 });
267

```

FILE: src/__tests__/unit/sharedComponents.test.tsx

```
TYPE: .tsx | LINES: 1-353 | SHA256: 3876f0dad923a819201f6e6cf0b054d3ca65699331e9900383517224656b55f4
```

[illegible]

[illegible]

[illegible]

[illegible]

```
351 |   });
352 | });
353 |
```

FILE: [src/___tests___/unit/sidebarGrouped.test.tsx](#)

TYPE: .tsx | LINES: 1-111 | SHA256: c251f8a94d00b1002bd2ab081d6f553cecca1acaded100ab9f1bcc2ccf47c77d

```
1  // @vitest-environment happy-dom
2  // =====
3  // Sidebar – prototype primary/MORE navigation
4  // Feature: prototype-frontend-rebuild, Task 1.2.2
5  // =====
6
7  import { afterEach, beforeEach, describe, expect, it, vi } from "vitest";
8  import { cleanup, render, screen, within } from "@testing-library/react";
9  import { I18nextProvider } from "react-i18next";
10 import { MemoryRouter } from "react-router-dom";
11 import { Circle } from "lucide-react";
12 import i18n from "@lib/i18n";
13 import type { NavItem } from "@lib/navItems";
14
15 const mockState = vi.hoisted(() => ({
16   navItems: { student: [] } as Record<string, unknown[]>,
17   surveyData: 0 as number | undefined,
18 }));
19
20 vi.mock("@lib/navItems", () => ({ navItems: mockState.navItems }));
21 vi.mock("@hooks/useAuth", () => ({
22   useAuth: () => ({ profile: { role: "student" } }),
23 }));
24 vi.mock("@hooks/useSurveyAssignmentsCount", () => ({
25   useSurveyAssignmentsCount: () => ({ data: mockState.surveyData }),
26 }));
27 vi.mock("@components/shared/StudentSidebarExtras", () => ({
28   default: () => null,
29 }));
30 vi.mock("@components/shared/MobileTabBar", () => ({ default: () => null }));
31
32 import Sidebar from "@components/shared/Sidebar";
33 import { SidebarProvider } from "@components/shared/SidebarContext";
34
35 const studentItems = (): NavItem[] => [
36   { to: "/student/dashboard", labelKey: "nav.mobile.home", icon: Circle },
37   { to: "/student/learning-path", labelKey: "nav.mobile.learn", icon: Circle },
38   { to: "/student/tutor", labelKey: "nav.mobile.tutor", icon: Circle },
39   { to: "/student/progress", labelKey: "nav.mobile.progress", icon: Circle },
40   { to: "/student/profile", labelKey: "nav.me", icon: Circle },
41   { to: "/student/courses", labelKey: "nav.coursesTasks", icon: Circle },
42   { to: "/student/today", labelKey: "nav.dailyReview", icon: Circle },
43   { to: "/student/habits", labelKey: "nav.wellness", icon: Circle },
44   { to: "/student/planner", labelKey: "nav.focus", icon: Circle },
45   { to: "/student/challenges", labelKey: "nav.quests", icon: Circle },
46   { to: "/student/leaderboard", labelKey: "nav.leaderboard", icon: Circle },
47   { to: "/student/team", labelKey: "nav.myTeam", icon: Circle },
48   { to: "/student/journal", labelKey: "nav.journal", icon: Circle },
49   { to: "/student/calendar", labelKey: "nav.calendar", icon: Circle },
50   { to: "/student/marketplace", labelKey: "nav.shop", icon: Circle },
51   { to: "/student/notifications", labelKey: "nav.notifications", icon: Circle },
52   { to: "/student/settings/profile", labelKey: "nav.settings", icon: Circle },
53   { to: "/student/surveys", labelKey: "nav.surveys", icon: Circle },
54 ];
55
56 const renderSidebar = (initialPath = "/student/dashboard") =>
57   render(
58     <I18nextProvider i18n={i18n}>
59       <MemoryRouter initialEntries={[initialPath]}>
60         <SidebarProvider>
61           <Sidebar />
62         </SidebarProvider>
63       </MemoryRouter>
64     </I18nextProvider>
65   );
66
67 beforeEach(async () => {
68   await i18n.changeLanguage("en");
69   mockState.navItems.student = studentItems();
70   mockState.surveyData = 0;
71 });
72
73 afterEach(cleanup);
74
75 describe("Sidebar primary/MORE navigation", () => {
76   it("keeps the student prototype destinations in the right primary and MORE order", () => {
77     renderSidebar();
78
79     const links = screen.getAllByRole("link");
80     expect(links.slice(0, 5).map((link) => link.textContent)).toEqual([
81       expect.stringContaining("Home"),
82       expect.stringContaining("Learn"),
83       expect.stringContaining("Tutor"),
84       expect.stringContaining("Progress"),
85       expect.stringContaining("Me"),
86     ]);
87
88     const moreLabel = screen.getByText("More");
89     const more = moreLabel.parentElement;
90     if (!more) throw new Error("expected the MORE section");
91     expect(
92       within(more).getByRole("link", { name: "Courses & Tasks" })
93     ).toBeInTheDocument();
94     expect(
95       within(more).queryByRole("link", { name: "Settings" })
96     ).not.toBeInTheDocument();
97   });
98 });
```


[illegible]

[illegible]

FILE: src/__tests__/unit/starterWeekHeroCard.test.tsx

TYPE: .tsx | LINES: 1-235 | SHA256: 29aeb1aa2d9bbd6688bb5478d68acf40f6997511e039463eaf4a396563c0aaea

[illegible]


```

54 it("renders the heading", () => {
55   renderCard();
56   expect(screen.getByText("Your Starter Week Plan")).toBeTruthy();
57 });
58
59 it("renders the subtitle", () => {
60   renderCard();
61   expect(
62     screen.getByText(
63       "AI-generated study sessions to kick off your semester"
64     )
65   ).toBeTruthy();
66 });
67
68 it("renders Sessions, Total Time, and Done labels", () => {
69   renderCard();
70   expect(screen.getByText("Sessions")).toBeTruthy();
71   expect(screen.getByText("Total Time")).toBeTruthy();
72   expect(screen.getByText("Done")).toBeTruthy();
73 });
74 });
75
76 // %% Session count display %%%%%%%%%%%
77
78 describe("session count display", () => {
79   it("displays the total session count", () => {
80     renderCard();
81     // 3 sessions total
82     expect(screen.getByText("3")).toBeTruthy();
83   });
84
85   it("displays completed/total ratio", () => {
86     renderCard();
87     // 1 completed out of 3
88     expect(screen.getByText("1/3")).toBeTruthy();
89   });
90
91   it("handles zero sessions", () => {
92     renderCard({ sessions: [] });
93     expect(screen.getByText("0")).toBeTruthy();
94     expect(screen.getByText("0/0")).toBeTruthy();
95   });
96
97   it("shows all completed when every session is done", () => {
98     const sessions = [
99       makeSession({ status: "completed", duration_minutes: 30 }),
100      makeSession({ status: "completed", duration_minutes: 30 })
101    ];
102    renderCard({ sessions });
103    expect(screen.getByText("2/2")).toBeTruthy();
104  });
105 });
106
107 // %% Total study time display %%%%%%%%%%%
108
109 describe("total study time display", () => {
110   it("displays time in minutes when under 60", () => {
111     // 25 + 35 + 25 = 85m !' 1h 25m
112     renderCard();
113     expect(screen.getByText("1h 25m")).toBeTruthy();
114   });
115
116   it("displays only minutes when total is under 60", () => {
117     const sessions = [
118       makeSession({ duration_minutes: 20 }),
119       makeSession({ duration_minutes: 15 })
120     ];
121     renderCard({ sessions });
122     expect(screen.getByText("35m")).toBeTruthy();
123   });
124
125   it("displays hours and minutes for larger totals", () => {
126     const sessions = [
127       makeSession({ duration_minutes: 45 }),
128       makeSession({ duration_minutes: 45 }),
129       makeSession({ duration_minutes: 30 })
130     ];
131     renderCard({ sessions });
132     // 120m = 2h 0m
133     expect(screen.getByText("2h 0m")).toBeTruthy();
134   });
135
136   it("displays 0m for empty sessions", () => {
137     renderCard({ sessions: [] });
138     expect(screen.getByText("0m")).toBeTruthy();
139   });
140 });
141
142 // %% "View Plan" CTA navigation %%%%%%%%%%%
143
144 describe('"View Plan" navigation', () => {
145   it('renders the "View Plan" button', () => {
146     renderCard();
147     expect(screen.getByText("View Plan")).toBeTruthy();
148   });
149
150   it("calls onViewPlan when clicked", async () => {
151     const onViewPlan = vi.fn();
152     renderCard({ onViewPlan });
153     await userEvent.click(screen.getByText("View Plan"));
154     expect(onViewPlan).toHaveBeenCalledOnce();
155   });
156
157   it('does not render "View Plan" button in post-week mode', () => {

```



```

23 |     expect(getNextMilestone(61)).toBe(100);
24 | });
25 |
26 | it("returns null when all milestones are passed", () => {
27 |     expect(getNextMilestone(100)).toBeNull();
28 |     expect(getNextMilestone(150)).toBeNull();
29 | });
30 | });
31 |
32 | describe("getMilestoneProgress", () => {
33 |     it("returns 0 for streak of 0", () => {
34 |         expect(getMilestoneProgress(0)).toBe(0);
35 |     });
36 |
37 |     it("returns 100 when all milestones passed", () => {
38 |         expect(getMilestoneProgress(100)).toBe(100);
39 |         expect(getMilestoneProgress(200)).toBe(100);
40 |     });
41 |
42 |     it("calculates progress within first milestone range (0-7)", () => {
43 |         expect(getMilestoneProgress(3)).toBe(43);
44 |     });
45 |
46 |     it("calculates progress within second milestone range (7-14)", () => {
47 |         expect(getMilestoneProgress(10)).toBe(43);
48 |     });
49 |
50 |     it("calculates progress within 30-60 range", () => {
51 |         expect(getMilestoneProgress(45)).toBe(50);
52 |     });
53 | });
54 |
55 | // -----
56 | // Component rendering tests
57 | // -----
58 | describe("StreakDisplay", () => {
59 |     it("renders streak count in full mode", () => {
60 |         render(<StreakDisplay streakCount={12} />);
61 |         expect(screen.getByText("12")).toBeInTheDocument();
62 |         expect(screen.getByText("day streak")).toBeInTheDocument();
63 |     });
64 |
65 |     it("renders compact mode with just count", () => {
66 |         render(<StreakDisplay streakCount={5} compact />);
67 |         expect(screen.getByText("5")).toBeInTheDocument();
68 |         expect(screen.queryByText("day streak")).not.toBeInTheDocument();
69 |         expect(screen.getByLabelText("5 day streak")).toBeInTheDocument();
70 |     });
71 |
72 |     it("shows milestone progress bar in full mode", () => {
73 |         render(<StreakDisplay streakCount={5} />);
74 |         expect(screen.getByText("Next milestone: 7 days")).toBeInTheDocument();
75 |         expect(screen.getByRole("progressbar")).toBeInTheDocument();
76 |     });
77 |
78 |     it('shows "All milestones reached" when past all milestones', () => {
79 |         render(<StreakDisplay streakCount={100} />);
80 |         expect(screen.getByText("All milestones reached!")).toBeInTheDocument();
81 |         expect(screen.queryByRole("progressbar")).not.toBeInTheDocument();
82 |     });
83 |
84 |     it("renders streak freeze indicators when available", () => {
85 |         render(<StreakDisplay streakCount={10} streakFreezesAvailable={2} />);
86 |         expect(
87 |             screen.getByLabelText("2 streak freezes available")
88 |         ).toBeInTheDocument();
89 |     });
90 |
91 |     it("does not render freeze indicators when none available", () => {
92 |         render(<StreakDisplay streakCount={10} streakFreezesAvailable={0} />);
93 |         expect(screen.queryByLabelText(/streak freeze/)).not.toBeInTheDocument();
94 |     });
95 |
96 |     it("caps freeze indicators at 2", () => {
97 |         const { container } = render(
98 |             <StreakDisplay streakCount={10} streakFreezesAvailable={5} />
99 |         );
100 |         const snowflakes = container.querySelectorAll(".text-blue-400");
101 |         expect(snowflakes).toHaveLength(2);
102 |     });
103 | });
104 |
105 | // -----
106 | // Sabbatical range format tests – Requirement 126.1
107 | // -----
108 | describe("StreakDisplay – Sabbatical range format", () => {
109 |     it("shows range format when sabbatical enabled and rest days > 0", () => {
110 |         render(
111 |             <StreakDisplay
112 |                 streakCount={15}
113 |                 streakSabbaticalEnabled={true}
114 |                 restDays={4}
115 |             />
116 |         );
117 |         expect(screen.getByTestId("streak-range-label")).toHaveTextContent(
118 |             "day streak, 4 rest days"
119 |         );
120 |     });
121 |
122 |     it('shows singular "rest day" when restDays is 1', () => {
123 |         render(
124 |             <StreakDisplay
125 |                 streakCount={6}
126 |                 streakSabbaticalEnabled={true}

```

```

127         restDays={1}
128     />
129 );
130 expect(screen.getByTestId("streak-range-label")).toHaveTextContent(
131     "day streak, 1 rest day"
132 );
133 });
134
135 it("shows standard format when sabbatical disabled", () => {
136     render(
137         <StreakDisplay
138             streakCount={10}
139             streakSabbaticalEnabled={false}
140             restDays={2}
141         />
142     );
143     expect(screen.queryByTestId("streak-range-label")).not.toBeInTheDocument();
144     expect(screen.getByText("day streak")).toBeInTheDocument();
145 });
146
147 it("shows standard format when restDays is 0 even with sabbatical enabled", () => {
148     render(
149         <StreakDisplay
150             streakCount={5}
151             streakSabbaticalEnabled={true}
152             restDays={0}
153         />
154     );
155     expect(screen.queryByTestId("streak-range-label")).not.toBeInTheDocument();
156     expect(screen.getByText("day streak")).toBeInTheDocument();
157 });
158 });
159
160 // -----
161 // Total Active Days tests – Requirement 126.2, 126.5
162 // -----
163 describe("StreakDisplay – Total Active Days", () => {
164     it("shows total active days counter", () => {
165         render(<StreakDisplay streakCount={5} totalActiveDays={42} />);
166         expect(screen.getByTestId("total-active-days")).toHaveTextContent(
167             "42 Total Active Days"
168         );
169     });
170
171     it("does not show total active days when 0", () => {
172         render(<StreakDisplay streakCount={5} totalActiveDays={0} />);
173         expect(screen.queryByTestId("total-active-days")).not.toBeInTheDocument();
174     });
175
176     it("celebrates milestone at 30 days", () => {
177         render(<StreakDisplay streakCount={5} totalActiveDays={30} />);
178         expect(screen.getByTestId("active-days-milestone")).toHaveTextContent(
179             "Milestone!"
180         );
181     });
182
183     it("celebrates milestone at 100 days", () => {
184         render(<StreakDisplay streakCount={5} totalActiveDays={100} />);
185         expect(screen.getByTestId("active-days-milestone")).toHaveTextContent(
186             "Milestone!"
187         );
188     });
189
190     it("celebrates milestone at 365 days", () => {
191         render(<StreakDisplay streakCount={5} totalActiveDays={365} />);
192         expect(screen.getByTestId("active-days-milestone")).toHaveTextContent(
193             "Milestone!"
194         );
195     });
196
197     it("does not show milestone for non-milestone values", () => {
198         render(<StreakDisplay streakCount={5} totalActiveDays={42} />);
199         expect(
200             screen.queryByTestId("active-days-milestone")
201         ).not.toBeInTheDocument();
202     });
203 });
204
205 // -----
206 // Motivational message on streak reset – Requirement 126.4
207 // -----
208 describe("StreakDisplay – Streak reset message", () => {
209     it("shows motivational message when streak just reset", () => {
210         render(
211             <StreakDisplay
212                 streakCount={0}
213                 totalActiveDays={50}
214                 streakJustReset={true}
215             />
216         );
217         expect(screen.getByTestId("streak-reset-message")).toHaveTextContent(
218             "Your 50 total active days of learning are still an achievement"
219         );
220     });
221
222     it("does not show message when streak not reset", () => {
223         render(
224             <StreakDisplay
225                 streakCount={5}
226                 totalActiveDays={50}
227                 streakJustReset={false}
228             />
229         );
230         expect(

```

```

231 |     screen.queryByTestId("streak-reset-message")
232 |   ).not.toBeInTheDocument();
233 | });
234 |
235 | it("does not show message when totalActiveDays is 0", () => {
236 |   render(
237 |     <StreakDisplay
238 |       streakCount={0}
239 |       totalActiveDays={0}
240 |       streakJustReset={true}
241 |     />
242 |   );
243 |   expect(
244 |     screen.queryByTestId("streak-reset-message")
245 |   ).not.toBeInTheDocument();
246 | });
247 | });
248 |

```

FILE: [src/__tests__/unit/streakFreezeShop.test.tsx](#)

TYPE: .tsx | LINES: 1-140 | SHA256: 2522e716ecb712fb224fe6498e3e40456f9b66cd2b1a8b9c91cfc7ffb10d43be

```

1 | // @vitest-environment happy-dom
2 | import { describe, it, expect, vi } from "vitest";
3 | import { render, screen, fireEvent, waitFor } from "@testing-library/react";
4 | import StreakFreezeShop, {
5 |   FREEZE_COST,
6 |   MAX_FREEZES,
7 | } from "@components/shared/StreakFreezeShop";
8 |
9 | // Mock i18next for ConfirmDialog
10 | vi.mock("react-i18next", () => ({
11 |   useTranslation: () => ({
12 |     t: (key: string) => {
13 |       const map: Record<string, string> = {
14 |         "buttons.cancel": "Cancel",
15 |         "buttons.confirm": "Confirm",
16 |       };
17 |       return map[key] ?? key;
18 |     },
19 |     i18n: { language: "en" },
20 |   }),
21 | }));
22 |
23 | describe("StreakFreezeShop", () => {
24 |   const defaultProps = {
25 |     currentXP: 500,
26 |     freezesAvailable: 0,
27 |     onPurchase: vi.fn().mockResolvedValue(undefined),
28 |   };
29 |
30 |   it("renders freeze inventory icons", () => {
31 |     render(<StreakFreezeShop {...defaultProps} freezesAvailable={1} />);
32 |     expect(screen.getByText("1/3")).toBeInTheDocument();
33 |   });
34 |
35 |   it("shows 0/3 when no freezes available", () => {
36 |     render(<StreakFreezeShop {...defaultProps} freezesAvailable={0} />);
37 |     expect(screen.getByText("0/3")).toBeInTheDocument();
38 |   });
39 |
40 |   it("shows 3/3 when max freezes available", () => {
41 |     render(<StreakFreezeShop {...defaultProps} freezesAvailable={3} />);
42 |     expect(screen.getByText("3/3")).toBeInTheDocument();
43 |   });
44 |
45 |   it("enables purchase button when XP >= 200 and freezes < 2", () => {
46 |     render(
47 |       <StreakFreezeShop
48 |         {...defaultProps}
49 |         currentXP={300}
50 |         freezesAvailable={0}
51 |       />
52 |     );
53 |     const btn = screen.getByRole("button", { name: /buy for 200 xp/i });
54 |     expect(btn).not.toBeDisabled();
55 |   });
56 |
57 |   it("disables purchase button when XP < 200", () => {
58 |     render(
59 |       <StreakFreezeShop
60 |         {...defaultProps}
61 |         currentXP={100}
62 |         freezesAvailable={0}
63 |       />
64 |     );
65 |     const btn = screen.getByRole("button", { name: /buy for 200 xp/i });
66 |     expect(btn).toBeDisabled();
67 |   });
68 |
69 |   it("disables purchase button when freezes >= 3", () => {
70 |     render(
71 |       <StreakFreezeShop
72 |         {...defaultProps}
73 |         currentXP={500}
74 |         freezesAvailable={3}
75 |       />
76 |     );
77 |     const btn = screen.getByRole("button", { name: /buy for 200 xp/i });
78 |     expect(btn).toBeDisabled();
79 |   });
80 |
81 |   it("shows insufficient XP message when balance is low", () => {
82 |     render(<StreakFreezeShop {...defaultProps} currentXP={50} />);

```

```

83 | expect(screen.getByText(/not enough xp/i)).toBeInTheDocument();
84 | expect(screen.getByText(/50\200/)).toBeInTheDocument();
85 | });
86 |
87 | it("does not show insufficient XP message when balance is sufficient", () => {
88 |   render(<StreakFreezeShop {...defaultProps} currentXP={300} />);
89 |   expect(screen.queryByText(/not enough xp/i)).not.toBeInTheDocument();
90 | });
91 |
92 | it("shows confirmation dialog before purchase", async () => {
93 |   render(<StreakFreezeShop {...defaultProps} />);
94 |   const btn = screen.getByRole("button", { name: /buy for 200 xp/i });
95 |   fireEvent.click(btn);
96 |
97 |   await waitFor(() => {
98 |     expect(screen.getByText(/purchase streak freeze/i)).toBeInTheDocument();
99 |   });
100 |   expect(screen.getByText(/deduct 200 xp/i)).toBeInTheDocument();
101 | });
102 |
103 | it("calls onPurchase after confirming dialog", async () => {
104 |   const onPurchase = vi.fn().mockResolvedValue(undefined);
105 |   render(<StreakFreezeShop {...defaultProps} onPurchase={onPurchase} />);
106 |
107 |   fireEvent.click(screen.getByRole("button", { name: /buy for 200 xp/i }));
108 |
109 |   await waitFor(() => {
110 |     expect(screen.getByText(/purchase streak freeze/i)).toBeInTheDocument();
111 |   });
112 |
113 |   fireEvent.click(screen.getByRole("button", { name: /buy freeze/i }));
114 |
115 |   await waitFor(() => {
116 |     expect(onPurchase).toHaveBeenCalledTimes(1);
117 |   });
118 | });
119 |
120 | it("does not call onPurchase when dialog is cancelled", async () => {
121 |   const onPurchase = vi.fn().mockResolvedValue(undefined);
122 |   render(<StreakFreezeShop {...defaultProps} onPurchase={onPurchase} />);
123 |
124 |   fireEvent.click(screen.getByRole("button", { name: /buy for 200 xp/i }));
125 |
126 |   await waitFor(() => {
127 |     expect(screen.getByText(/purchase streak freeze/i)).toBeInTheDocument();
128 |   });
129 |
130 |   fireEvent.click(screen.getByRole("button", { name: /cancel/i }));
131 |
132 |   expect(onPurchase).not.toHaveBeenCalled();
133 | });
134 |
135 | it("exports FREEZE_COST and MAX_FREEZES constants", () => {
136 |   expect(FREEZE_COST).toBe(200);
137 |   expect(MAX_FREEZES).toBe(3);
138 | });
139 | });
140 |

```

FILE: src/__tests__/unit/streakRecovery.test.ts

```
TYPE: .ts | LINES: 1-167 | SHA256: 90db3e0b3a7e075569810b1c0ada5ed6f5987203d4094e5d3f44e501f70c8525
```

[illegible]

[illegible]

```

147 | it("does not increment on same-day login", () => {
148 |   const state: StreakState = {
149 |     streak_count: 5,
150 |     last_login_date: "2025-01-06",
151 |     streak_freezes_available: 0,
152 |   };
153 |   const result = calculateStreakUpdate(state, "2025-01-06");
154 |   expect(result.is_new_day).toBe(false);
155 | });
156 |
157 | it("increments on consecutive days", () => {
158 |   const state: StreakState = {
159 |     streak_count: 5,
160 |     last_login_date: "2025-01-05",
161 |     streak_freezes_available: 0,
162 |   };
163 |   const result = calculateStreakUpdate(state, "2025-01-06");
164 |   expect(result.is_new_day).toBe(true);
165 | });
166 | });
167 |

```

FILE: [src/__tests__/unit/studentAttendanceAggregation.test.ts](#)

TYPE: .ts | LINES: 1-104 | SHA256: 89530d94a2ed4f56d45f311ecc416150103c3d59164ea1053350063b3e8e4e0c

```

1 | // =====
2 | // Feature: dashboard-and-ux-performance – Phase 6 Fix B / Phase 7 Task 23.
3 | // Parity + correctness test for 'aggregateStudentAttendance', the pure function
4 | // that backs the consolidated 'useStudentAttendance' (which replaced a per-course
5 | // 1 + 3×N query fan-out with a fixed set of bounded queries).
6 | //
7 | // These assertions lock in the EXACT semantics of the prior per-course
8 | // implementation so the consolidation cannot silently change a student's numbers.
9 | // =====
10 | import { describe, it, expect } from "vitest";
11 | import { aggregateStudentAttendance } from "@/hooks/useAttendance";
12 |
13 | describe("aggregateStudentAttendance", () => {
14 |   const courses = [
15 |     { courseId: "c1", courseName: "Calculus" },
16 |     { courseId: "c2", courseName: "Physics" },
17 |     { courseId: "c3", courseName: "History" }, // zero-session course
18 |   ];
19 |
20 |   // c1: sections s1a, s1b ; c2: section s2a ; c3: no sections
21 |   const sections = [
22 |     { id: "s1a", course_id: "c1" },
23 |     { id: "s1b", course_id: "c1" },
24 |     { id: "s2a", course_id: "c2" },
25 |   ];
26 |
27 |   // c1 has 3 sessions (2 in s1a, 1 in s1b); c2 has 2 sessions; c3 has none
28 |   const sessions = [
29 |     { id: "ses1", section_id: "s1a" },
30 |     { id: "ses2", section_id: "s1a" },
31 |     { id: "ses3", section_id: "s1b" },
32 |     { id: "ses4", section_id: "s2a" },
33 |     { id: "ses5", section_id: "s2a" },
34 |   ];
35 |
36 |   it("returns one entry per enrolled course, in enrollment order", () => {
37 |     const result = aggregateStudentAttendance(courses, sections, sessions, []);
38 |     expect(result.map((r) => r.courseId)).toEqual(["c1", "c2", "c3"]);
39 |     expect(result.map((r) => r.courseName)).toEqual([
40 |       "Calculus",
41 |       "Physics",
42 |       "History",
43 |     ]);
44 |   });
45 |
46 |   it("counts totalSessions across all of a course's sections", () => {
47 |     const result = aggregateStudentAttendance(courses, sections, sessions, []);
48 |     expect(result.find((r) => r.courseId === "c1")?.totalSessions).toBe(3);
49 |     expect(result.find((r) => r.courseId === "c2")?.totalSessions).toBe(2);
50 |   });
51 |
52 |   it("counts present and late (but not absent/excused) as attended", () => {
53 |     const records = [
54 |       { status: "present", session_id: "ses1" },
55 |       { status: "late", session_id: "ses2" },
56 |       { status: "absent", session_id: "ses3" },
57 |       { status: "present", session_id: "ses4" },
58 |       { status: "excused", session_id: "ses5" },
59 |     ];
60 |     const result = aggregateStudentAttendance(
61 |       courses,
62 |       sections,
63 |       sessions,
64 |       records
65 |     );
66 |     // c1: present(ses1) + late(ses2) = 2 of 3 !' round(2/3*100) = 67
67 |     const c1 = result.find((r) => r.courseId === "c1")!;
68 |     expect(c1.attended).toBe(2);
69 |     expect(c1.attendancePercent).toBe(67);
70 |     // c2: present(ses4) only = 1 of 2 !' 50
71 |     const c2 = result.find((r) => r.courseId === "c2")!;
72 |     expect(c2.attended).toBe(1);
73 |     expect(c2.attendancePercent).toBe(50);
74 |   });
75 |
76 |   it("renders a zero-session course at 100% / 0 (matches the prior early-return)", () => {
77 |     const result = aggregateStudentAttendance(courses, sections, sessions, []);
78 |     const c3 = result.find((r) => r.courseId === "c3")!;
79 |     expect(c3.totalSessions).toBe(0);

```



```

80 |     expect(c3.attended).toBe(0);
81 |     expect(c3.attendancePercent).toBe(100);
82 | });
83 |
84 | it("ignores attendance records for sessions outside the enrolled courses", () => {
85 |     const records = [
86 |         { status: "present", session_id: "ses1" }, // belongs to c1 !' counted
87 |         { status: "present", session_id: "unknown-session" }, // not enrolled !' ignored
88 |     ];
89 |     const result = aggregateStudentAttendance(
90 |         courses,
91 |         sections,
92 |         sessions,
93 |         records
94 |     );
95 |     expect(result.find((r) => r.courseId === "c1")?.attended).toBe(1);
96 |     // total attended across all courses must equal only the in-scope record
97 |     expect(result.reduce((sum, r) => sum + r.attended, 0)).toBe(1);
98 | });
99 |
100 | it("returns an empty array when there are no enrolled courses", () => {
101 |     expect(aggregateStudentAttendance([], sections, sessions, [])).toEqual([]);
102 | });
103 | });
104 |

```

FILE: [src/__tests__/unit/studentContentForm.test.tsx](#)

TYPE: .tsx | LINES: 1-117 | SHA256: f9a0fc3376a94a3df856ca68bb039193322d26b6aad0e25dac27248387b1e8fb

```

1 | // =====
2 | // Unit Test: Student Content Form
3 | // Task 26.10 – Content creation form, type selection
4 | // =====
5 |
6 | import { describe, it, expect } from "vitest";
7 | import {
8 |     createStudentContentSchema,
9 |     reviewStudentContentSchema,
10 |     studentContentFormSchema,
11 | } from "@lib/marketplaceSchemas";
12 |
13 | describe("Student Content Form", () => {
14 |     describe("studentContentFormSchema", () => {
15 |         it("trims a valid user-facing content submission", () => {
16 |             const result = studentContentFormSchema.safeParse({
17 |                 content_type: "study_plan",
18 |                 title: " My Study Plan ",
19 |                 body: " Review chapters 1 and 2. ",
20 |             });
21 |             expect(result.success).toBe(true);
22 |             if (result.success) {
23 |                 expect(result.data.title).toBe("My Study Plan");
24 |                 expect(result.data.body).toBe("Review chapters 1 and 2.");
25 |             }
26 |         });
27 |
28 |         it("rejects an all-whitespace title before attempting a mutation", () => {
29 |             const result = studentContentFormSchema.safeParse({
30 |                 content_type: "study_plan",
31 |                 title: " ",
32 |                 body: "A valid body",
33 |             });
34 |             expect(result.success).toBe(false);
35 |         });
36 |     });
37 |
38 |     describe("createStudentContentSchema", () => {
39 |         it("validates a valid study plan", () => {
40 |             const result = createStudentContentSchema.safeParse({
41 |                 content_type: "study_plan",
42 |                 title: "My Study Plan",
43 |                 clo_id: null,
44 |                 content_data: { body: "Study plan content" },
45 |             });
46 |             expect(result.success).toBe(true);
47 |         });
48 |
49 |         it("validates a valid quiz question", () => {
50 |             const result = createStudentContentSchema.safeParse({
51 |                 content_type: "quiz_question",
52 |                 title: "Sample Question",
53 |                 clo_id: "550e8400-e29b-41d4-a716-446655440000",
54 |                 content_data: { question: "What is...?", answer: "It is..." },
55 |             });
56 |             expect(result.success).toBe(true);
57 |         });
58 |
59 |         it("validates a valid explanation video", () => {
60 |             const result = createStudentContentSchema.safeParse({
61 |                 content_type: "explanation_video",
62 |                 title: "How to solve...",
63 |                 clo_id: null,
64 |                 content_data: { url: "https://example.com/video" },
65 |             });
66 |             expect(result.success).toBe(true);
67 |         });
68 |
69 |         it("rejects empty title", () => {
70 |             const result = createStudentContentSchema.safeParse({
71 |                 content_type: "study_plan",
72 |                 title: "",
73 |                 clo_id: null,
74 |                 content_data: {},
75 |             });

```

```

76 |     expect(result.success).toBe(false);
77 |   });
78 |
79 |   it("rejects invalid content type", () => {
80 |     const result = createStudentContentSchema.safeParse({
81 |       content_type: "invalid_type",
82 |       title: "Test",
83 |       clo_id: null,
84 |       content_data: {},
85 |     });
86 |     expect(result.success).toBe(false);
87 |   });
88 | });
89 |
90 | describe("reviewStudentContentSchema", () => {
91 |   it("validates approval", () => {
92 |     const result = reviewStudentContentSchema.safeParse({
93 |       content_id: "550e8400-e29b-41d4-a716-446655440000",
94 |       status: "approved",
95 |     });
96 |     expect(result.success).toBe(true);
97 |   });
98 |
99 |   it("validates rejection with feedback", () => {
100 |     const result = reviewStudentContentSchema.safeParse({
101 |       content_id: "550e8400-e29b-41d4-a716-446655440000",
102 |       status: "rejected",
103 |       feedback: "Needs more detail",
104 |     });
105 |     expect(result.success).toBe(true);
106 |   });
107 |
108 |   it("rejects invalid status", () => {
109 |     const result = reviewStudentContentSchema.safeParse({
110 |       content_id: "550e8400-e29b-41d4-a716-446655440000",
111 |       status: "pending",
112 |     });
113 |     expect(result.success).toBe(false);
114 |   });
115 | });
116 | });
117 |

```

FILE: [src/__tests__/unit/studentCourseCards.test.ts](#)

TYPE: .ts | LINES: 1-210 | SHA256: 20bf69f00d323a4ed98a08f31470118b68e589b74c08c82fd9d6dbfd5a4a2e1e

```

1 | import { describe, it, expect } from "vitest";
2 | import {
3 |   buildCourseCards,
4 |   type BuildCourseCardsInput,
5 |   type CourseCardEnrollment,
6 | } from "@lib/studentCourseCards";
7 |
8 | const NOW = new Date("2025-06-01T00:00:00.000Z");
9 |
10 | const course = (
11 |   id: string,
12 |   overrides: Partial<CourseCardEnrollment["course"]> = {}
13 | ): CourseCardEnrollment => ({
14 |   course_id: id,
15 |   course: {
16 |     id,
17 |     name: `Course ${id}`,
18 |     code: `C-${id}`,
19 |     color: null,
20 |     teacher_name: "Ms. Smith",
21 |     ...overrides,
22 |   },
23 | });
24 |
25 | const baseInput = (
26 |   overrides: Partial<BuildCourseCardsInput> = {}
27 | ): BuildCourseCardsInput => ({
28 |   enrollments: [course("a")],
29 |   attainmentRows: [],
30 |   assignmentRows: [],
31 |   completedAssignmentIds: new Set<string>(),
32 |   now: NOW,
33 |   ...overrides,
34 | });
35 |
36 | describe("buildCourseCards", () => {
37 |   it("returns an empty array when there are no enrollments", () => {
38 |     expect(buildCourseCards(baseInput({ enrollments: [] }))).toEqual([]);
39 |   });
40 |
41 |   it("maps course identity including color (R9.3, R9.4)", () => {
42 |     const card = buildCourseCards(
43 |       baseInput({ enrollments: [course("a", { color: "#3b82f6" })] })
44 |     )[0]!;
45 |     expect(card.id).toBe("a");
46 |     expect(card.color).toBe("#3b82f6");
47 |     expect(card.teacher_name).toBe("Ms. Smith");
48 |   });
49 |
50 |   it("preserves a null color for deterministic fallback downstream (R9.3)", () => {
51 |     const card = buildCourseCards(baseInput())[0]!;
52 |     expect(card.color).toBeNull();
53 |   });
54 |
55 |   it("averages and rounds course-scoped attainment", () => {
56 |     const card = buildCourseCards(
57 |       baseInput({
58 |         attainmentRows: [

```

```

59 |         { course_id: "a", attainment_percent: 80 },
60 |         { course_id: "a", attainment_percent: 85 },
61 |     ],
62 | })
63 | ) [0]!;
64 | // (80 + 85) / 2 = 82.5 !' 83
65 | expect(card.attainment_percent).toBe(83);
66 | });
67 |
68 | it("returns null attainment when no rows exist", () => {
69 |     const card = buildCourseCards(baseInput())[0]!;
70 |     expect(card.attainment_percent).toBeNull();
71 | });
72 |
73 | it("computes progress from graded submissions over total assignments (R9.1)", () => {
74 |     const card = buildCourseCards(
75 |         baseInput({
76 |             assignmentRows: [
77 |                 { id: "a1", course_id: "a", title: "A1", due_date: null },
78 |                 { id: "a2", course_id: "a", title: "A2", due_date: null },
79 |                 { id: "a3", course_id: "a", title: "A3", due_date: null },
80 |                 { id: "a4", course_id: "a", title: "A4", due_date: null },
81 |             ],
82 |             completedAssignmentIds: new Set(["a1", "a2"]),
83 |         })
84 |     ) [0]!;
85 |     expect(card.assignments_count).toBe(4);
86 |     expect(card.progress_percent).toBe(50);
87 | });
88 |
89 | it("reports zero progress when a course has no assignments (R9.5)", () => {
90 |     const card = buildCourseCards(baseInput())[0]!;
91 |     expect(card.assignments_count).toBe(0);
92 |     expect(card.progress_percent).toBe(0);
93 |     expect(card.next_assignment).toBeNull();
94 | });
95 |
96 | it("selects the earliest upcoming assignment as next (R9.2)", () => {
97 |     const card = buildCourseCards(
98 |         baseInput({
99 |             assignmentRows: [
100 |                 {
101 |                     id: "a1",
102 |                     course_id: "a",
103 |                     title: "Later",
104 |                     due_date: "2025-06-10T00:00:00.000Z",
105 |                 },
106 |                 {
107 |                     id: "a2",
108 |                     course_id: "a",
109 |                     title: "Sooner",
110 |                     due_date: "2025-06-03T00:00:00.000Z",
111 |                 },
112 |             ],
113 |         })
114 |     ) [0]!;
115 |     expect(card.next_assignment).toEqual({
116 |         title: "Sooner",
117 |         due_at: "2025-06-03T00:00:00.000Z",
118 |     });
119 | });
120 |
121 | it("ignores past-due assignments when choosing next", () => {
122 |     const card = buildCourseCards(
123 |         baseInput({
124 |             assignmentRows: [
125 |                 {
126 |                     id: "past",
127 |                     course_id: "a",
128 |                     title: "Past",
129 |                     due_date: "2025-05-01T00:00:00.000Z",
130 |                 },
131 |                 {
132 |                     id: "future",
133 |                     course_id: "a",
134 |                     title: "Future",
135 |                     due_date: "2025-06-15T00:00:00.000Z",
136 |                 },
137 |             ],
138 |         })
139 |     ) [0]!;
140 |     expect(card.next_assignment?.title).toBe("Future");
141 | });
142 |
143 | it("includes an assignment without a due date as a name-only fallback (R9.2a)", () => {
144 |     const card = buildCourseCards(
145 |         baseInput({
146 |             assignmentRows: [
147 |                 { id: "a1", course_id: "a", title: "Undated", due_date: null },
148 |             ],
149 |         })
150 |     ) [0]!;
151 |     expect(card.next_assignment).toEqual({ title: "Undated", due_at: null });
152 | });
153 |
154 | it("prefers a dated upcoming assignment over an undated one", () => {
155 |     const card = buildCourseCards(
156 |         baseInput({
157 |             assignmentRows: [
158 |                 { id: "a1", course_id: "a", title: "Undated", due_date: null },
159 |                 {
160 |                     id: "a2",
161 |                     course_id: "a",
162 |                     title: "Dated",

```

```

163 |         due_date: "2025-06-05T00:00:00.000Z",
164 |     },
165 | },
166 | })!;
167 | expect(card.next_assignment?.title).toBe("Dated");
168 | });
169 |
170 |
171 | it("returns null next assignment when all dated assignments are past due", () => {
172 |     const card = buildCourseCards(
173 |         baseInput({
174 |             assignmentRows: [
175 |                 {
176 |                     id: "a1",
177 |                     course_id: "a",
178 |                     title: "Past",
179 |                     due_date: "2025-05-01T00:00:00.000Z",
180 |                 },
181 |             ],
182 |         })!;
183 |     expect(card.next_assignment).toBeNull();
184 | });
185 |
186 |
187 | it("aggregates each course independently without cross-contamination", () => {
188 |     const cards = buildCourseCards(
189 |         baseInput({
190 |             enrollments: [course("a"), course("b")],
191 |             attainmentRows: [{ course_id: "a", attainment_percent: 90 }],
192 |             assignmentRows: [
193 |                 { id: "a1", course_id: "a", title: "A1", due_date: null },
194 |                 { id: "b1", course_id: "b", title: "B1", due_date: null },
195 |                 { id: "b2", course_id: "b", title: "B2", due_date: null },
196 |             ],
197 |             completedAssignmentIds: new Set(["b1"]),
198 |         })
199 |     );
200 |     const cardA = cards.find((c) => c.id === "a")!;
201 |     const cardB = cards.find((c) => c.id === "b")!;
202 |     expect(cardA.attainment_percent).toBe(90);
203 |     expect(cardA.assignments_count).toBe(1);
204 |     expect(cardA.progress_percent).toBe(0);
205 |     expect(cardB.attainment_percent).toBeNull();
206 |     expect(cardB.assignments_count).toBe(2);
207 |     expect(cardB.progress_percent).toBe(50);
208 | });
209 | });
210 |

```

FILE: src/__tests__/unit/StudentFeesPage.test.tsx

TYPE: .tsx | LINES: 1-113 | SHA256: 3c1d4e45137e12b1f003adf905227c3cf14eec5e943a3e978b8ca20a25f1d063

```

1 | // StudentFeesPage – functional render tests (net-new screen).
2 | // Feature: prototype-frontend-rebuild.
3 | import { describe, it, expect, beforeEach, vi } from "vitest";
4 | import { render, screen, fireEvent } from "@testing-library/react";
5 | import type { FeePayment } from "@hooks/useFees";
6 | import StudentFeesPage from "@features/student/fees/StudentFeesPage";
7 |
8 | const hoisted = vi.hoisted(() => ({
9 |     fees: {
10 |         data: undefined as FeePayment[] | undefined,
11 |         isLoading: false,
12 |         isError: false,
13 |     },
14 |     receipt: {
15 |         mutate: vi.fn(),
16 |         isPending: false,
17 |         variables: undefined as string | undefined,
18 |     },
19 | }));
20 |
21 | vi.mock("@hooks/useFees", () => ({
22 |     useStudentFees: () => hoisted.fees,
23 |     useGenerateFeeReceipt: () => hoisted.receipt,
24 | }));
25 |
26 | vi.mock("@hooks/useAuth", () => ({
27 |     useAuth: () => ({ user: { id: "stu-1" }, profile: null }),
28 | }));
29 |
30 | vi.mock("sonner", () => ({ toast: { success: vi.fn(), error: vi.fn() } }));
31 |
32 | vi.mock("react-i18next", () => ({
33 |     useTranslation: () => ({ t: (_key: string, def?: string) => def ?? _key }),
34 | }));
35 |
36 | const payments: FeePayment[] = [
37 |     {
38 |         id: "pay-1",
39 |         fee_structure_id: "fs-1",
40 |         student_id: "stu-1",
41 |         amount_paid: 5000,
42 |         payment_method: "card",
43 |         receipt_number: "RC-100",
44 |         status: "paid",
45 |         payment_date: "2026-01-10",
46 |         created_at: "2026-01-10T00:00:00Z",
47 |     },
48 |     {
49 |         id: "pay-2",
50 |         fee_structure_id: "fs-2",
51 |         student_id: "stu-1",
52 |         amount_paid: 0,

```

```

13 payment_method: null,
14 receipt_number: null,
15 status: "overdue",
16 payment_date: "2026-02-01",
17 created_at: "2026-02-01T00:00:00Z",
18 },
19 ],
20
21 beforeEach(() => {
22   hoisted.fees.data = undefined;
23   hoisted.fees.isLoading = false;
24   hoisted.fees.isError = false;
25   hoisted.receipt.mutate = vi.fn();
26   hoisted.receipt.isPending = false;
27   hoisted.receipt.variables = undefined;
28 });
29
30 describe("StudentFeesPage", () => {
31   it("renders the title while loading", () => {
32     hoisted.fees.isLoading = true;
33     render(<StudentFeesPage />);
34     expect(screen.getByText("Fees")).toBeInTheDocument();
35   });
36
37   it("shows an error state", () => {
38     hoisted.fees.isError = true;
39     render(<StudentFeesPage />);
40     expect(screen.getByRole("alert")).toHaveTextContent(
41       "Could not load your fee records. Please try again."
42     );
43   });
44
45   it("shows an empty state when there are no payments", () => {
46     hoisted.fees.data = [];
47     render(<StudentFeesPage />);
48     expect(screen.getByText("No fee records yet.")).toBeInTheDocument();
49   });
50
51   it("renders payment rows with status and a receipt action for paid records", () => {
52     hoisted.fees.data = payments;
53     render(<StudentFeesPage />);
54     // "5,000" and "Paid" appear in both a KPI card and the payment row.
55     expect(screen.getAllByText("5,000").length).toBeGreaterThanOrEqual(1);
56     expect(screen.getAllByText("Paid").length).toBeGreaterThanOrEqual(1);
57     expect(screen.getByText("Overdue")).toBeInTheDocument();
58     // Paid record with a receipt number !' downloadable; overdue !' not.
59     expect(screen.getByTestId("fee-receipt-pay-1")).toBeInTheDocument();
60     expect(screen.queryByTestId("fee-receipt-pay-2")).toBeNull();
61   });
62
63   it("requests a receipt for the paid payment on click", () => {
64     hoisted.fees.data = payments;
65     render(<StudentFeesPage />);
66     fireEvent.click(screen.getByTestId("fee-receipt-pay-1"));
67     expect(hoisted.receipt.mutate).toHaveBeenCalledWith(
68       "pay-1",
69       expect.anything()
70     );
71   });
72 });
73

```

FILE: src/__tests__/unit/studentJournalPrompts.test.tsx

TYPE: .tsx | LINES: 1-182 | SHA256: 089230e82786717a669ced6999b546027e95fb63c14b588a6e270e6bfb1650e1

```

1 // @vitest-environment happy-dom
2 // =====
3 // StudentJournalPage – prompt seeding & unguided fallback (Task 17.2)
4 //
5 // Covers Requirement 10:
6 // - R10.2: selecting a reflection prompt seeds the journal entry content
7 // - R10.3a: when the CLO-contextual generator is unavailable (no course/CLO
8 //           context), a basic unguided journal remains usable so journaling
9 //           is always possible.
10 //
11 // The pure seeding helper (`seedContentWithPrompt`) is also exercised directly,
12 // since it is the core logic the page wiring depends on.
13 // =====
14
15 import { describe, it, expect, vi, beforeEach } from "vitest";
16 import { render, screen } from "@testing-library/react";
17 import userEvent from "@testing-library/user-event";
18 import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
19 import { I18nextProvider } from "react-i18next";
20
21 import i18n from "@/lib/i18n";
22 import { seedContentWithPrompt } from "@/lib/reflectionPrompts";
23
24 // % % % Mocks % % % % % % % % % % % % % % % % % % % % % % % % % % %
25
26 vi.mock("sonner", () => ({
27   toast: { success: vi.fn(), error: vi.fn() },
28 }));
29
30 vi.mock("@/hooks/useAuth", () => ({
31   useAuth: () => ({ user: { id: "student-1" } }),
32 }));
33
34 vi.mock("@/hooks/useJournal", () => ({
35   useJournalEntries: () => ({ data: [], isLoading: false }),
36   useCreateJournalEntry: () => ({ mutateAsync: vi.fn(), isLoading: false }),
37 }));
38
39 // No CLO context !' the guided generator is unavailable and the page falls back

```

[illegible]

```

144 | });
145 | await user.click(screen.getByRole("button", { name: /What confused me/i }));
146 |
147 | const textarea = screen.getByRole("textbox") as HTMLTextAreaElement;
148 | expect(textarea.value).toContain("What did I learn today?");
149 | expect(textarea.value).toContain("What confused me?");
150 | });
151 |
152 | it("falls back to an unguided journal when no CLO context is available (R10.3a)", async () => {
153 |   const user = userEvent.setup();
154 |   renderPage();
155 |   await openNewEntryDialog(user);
156 |
157 |   // Without a selected course/CLO, the guided generator block is not shown...
158 |   expect(screen.queryByText("Guided reflection")).not.toBeInTheDocument();
159 |
160 |   // ...yet the basic journal remains usable: the free-text area accepts input.
161 |   const textarea = screen.getByRole("textbox") as HTMLTextAreaElement;
162 |   await user.type(textarea, "Today I reviewed my notes.");
163 |   expect(textarea.value).toContain("Today I reviewed my notes.");
164 | });
165 |
166 | it("still offers reflection templates in the unguided fallback (R10.1/R10.3a)", async () => {
167 |   const user = userEvent.setup();
168 |   renderPage();
169 |   await openNewEntryDialog(user);
170 |
171 |   expect(
172 |     screen.getByRole("button", { name: /What did I learn today/i })
173 |   ).toBeInTheDocument();
174 |   expect(
175 |     screen.getByRole("button", { name: /What confused me/i })
176 |   ).toBeInTheDocument();
177 |   expect(
178 |     screen.getByRole("button", { name: /What am I proud of/i })
179 |   ).toBeInTheDocument();
180 | });
181 | });
182 |

```

FILE: src/__tests__/unit/studentPortfolio.test.tsx

```
TYPE: .tsx | LINES: 1-425 | SHA256: 19b65c01cd3a3765b07236209e11a4e019df6e61329156d0e5156f423896be10
```

```

1 // @vitest-environment happy-dom
2 // =====
3 // StudentPortfolio – Unit tests
4 // Validates: Requirement 58 (Student Learning Portfolio)
5 // =====
6
7 import { describe, it, expect, vi, beforeEach } from "vitest";
8 import { render, screen, fireEvent } from "@testing-library/react";
9 import { I18nextProvider } from "react-i18next";
10 import i18n from "@lib/i18n";
11
12 // %% Mocks %%%%%%%%%%%%%%%
13
14 const mockUseAuth = vi.fn();
15 vi.mock("@/hooks/useAuth", () => ({
16   useAuth: () => mockUseAuth(),
17 }));
18
19 const mockUsePortfolio = vi.fn();
20 const mockToggleMutate = vi.fn();
21 const mockUseSharingPermission = vi.fn();
22 vi.mock("@/hooks/usePortfolio", () => ({
23   usePortfolio: (...args: unknown[]) => mockUsePortfolio(...args),
24   usePortfolioSharingPermission: (...args: unknown[]) =>
25     mockUseSharingPermission(...args),
26   useTogglePortfolioPublic: () => ({
27     mutate: mockToggleMutate,
28     isPending: false,
29   }),
30 }));
31
32 vi.mock("recharts", () => ({
33   ResponsiveContainer: ({ children }: { children: React.ReactNode }) => (
34     <div data-testid="responsive-container">{children}</div>
35   ),
36   LineChart: ({ children }: { children: React.ReactNode }) => (
37     <div data-testid="line-chart">{children}</div>
38   ),
39   BarChart: ({ children }: { children: React.ReactNode }) => (
40     <div data-testid="bar-chart">{children}</div>
41   ),
42   Line: () => <div data-testid="line" />,
43   Bar: () => <div data-testid="bar" />,
44   XAxis: () => <div />,
45   YAxis: () => <div />,
46   CartesianGrid: () => <div />,
47   Tooltip: () => <div />,
48 }));
49
50 vi.mock("sonner", () => ({
51   toast: { success: vi.fn(), error: vi.fn() },
52 }));
53
54 import { toast } from "sonner";
55
56 // %% Fixtures %%%%%%%%%%%%%%%
57
58 const basePortfolioData = {
59   clos: [
60     {
61       clo id: "clo-1".

```

```

62 |     clo_title: "Understanding OOP",
63 |     blooms_level: "understanding" as const,
64 |     attainment_percent: 88,
65 |     attainment_level: "Excellent" as const,
66 |     course_name: "CS101",
67 |   },
68 |   {
69 |     clo_id: "clo-2",
70 |     clo_title: "Apply Design Patterns",
71 |     blooms_level: "applying" as const,
72 |     attainment_percent: 72,
73 |     attainment_level: "Satisfactory" as const,
74 |     course_name: "CS101",
75 |   },
76 | ],
77 | badges: [
78 |   {
79 |     badge_key: "early_bird",
80 |     badge_name: "Early Bird",
81 |     emoji: "🕒",
82 |     awarded_at: "2024-06-10T10:00:00Z",
83 |   },
84 | ],
85 | journals: [
86 |   {
87 |     id: "j1",
88 |     content_preview: "Today I learned about polymorphism...",
89 |     created_at: "2024-06-10T10:00:00Z",
90 |     course_name: "CS101",
91 |   },
92 | ],
93 | xpTimeline: [
94 |   { date: "2024-06-01", cumulative_xp: 100 },
95 |   { date: "2024-06-10", cumulative_xp: 350 },
96 | ],
97 | semesterAttainments: [{ semester_name: "Fall 2024", average_attainment: 78 }],
98 | totalXP: 350,
99 | level: 3,
100 | };

```

```

104 const renderPortfolio = async () => {
105   const mod = await import("@/pages/student/portfolio/StudentPortfolio");
106   return render(
107     <I18nextProvider i18n={i18n}>
108       <mod.default />
109     </I18nextProvider>
110   );
111 };

```

[illegible]

```

105 describe("StudentPortfolio", () => {
106   beforeEach(async () => {
107     vi.clearAllMocks();
108     await i18n.changeLanguage("en");
109     mockUseAuth.mockReturnValue({
110       user: { id: "student-1" },
111       profile: { portfolio_public: false },
112     });
113     // Default: school has granted public-sharing permission.
114     mockUseSharingPermission.mockReturnValue({ data: true });
115   });
116
117   it("renders shimmer loading state", async () => {
118     mockUsePortfolio.mockReturnValue({ data: undefined, isLoading: true });
119     const { container } = await renderPortfolio();
120     const shimmers = container.querySelectorAll('[class*="shimmer"]');
121     expect(shimmers.length).toBeGreaterThan(0);
122   }, 10000);
123
124   it("renders page heading", async () => {
125     mockUsePortfolio.mockReturnValue({
126       data: basePortfolioData,
127       isLoading: false,
128     });
129     await renderPortfolio();
130     expect(screen.getByText("My Portfolio")).toBeInTheDocument();
131   }, 10000);
132
133   it("renders KPI cards with correct values", async () => {
134     mockUsePortfolio.mockReturnValue({
135       data: basePortfolioData,
136       isLoading: false,
137     });
138     await renderPortfolio();
139     const totalXpElements = screen.getAllByText("Total XP");
140     expect(totalXpElements.length).toBeGreaterThanOrEqual(1);
141     expect(screen.getAllByText("350").length).toBeGreaterThanOrEqual(1);
142     const levelElements = screen.getAllByText("Level");
143     expect(levelElements.length).toBeGreaterThanOrEqual(1);
144     expect(screen.getAllByText("3").length).toBeGreaterThanOrEqual(1);
145     const closeElements = screen.getAllByText("CLOs Mastered");
146     expect(closeElements.length).toBeGreaterThanOrEqual(1);
147     expect(screen.getAllByText("2").length).toBeGreaterThanOrEqual(1);
148     const badgesElements = screen.getAllByText("Badges Earned");
149     expect(badgesElements.length).toBeGreaterThanOrEqual(1);
150     expect(screen.getAllByText("1").length).toBeGreaterThanOrEqual(1);
151   }, 10000);
152
153   it("renders CLO mastery grouped by course", async () => {
154     mockUsePortfolio.mockReturnValue({
155       data: basePortfolioData,

```



```

166 |     isLoading: false,
167 |   });
168 |   await renderPortfolio();
169 |   // CS101 appears as both course heading and journal course_name
170 |   expect(screen.getAllByText("CS101").length).toBeGreaterThanOrEqual(1);
171 |   expect(screen.getByText("Understand OOP")).toBeInTheDocument();
172 |   expect(screen.getByText("Apply Design Patterns")).toBeInTheDocument();
173 | });
174 |
175 | it("renders badge collection", async () => {
176 |   mockUsePortfolio.mockReturnValue({
177 |     data: basePortfolioData,
178 |     isLoading: false,
179 |   });
180 |   await renderPortfolio();
181 |   expect(screen.getByText("Early Bird")).toBeInTheDocument();
182 |   expect(screen.getByText("0=0&")).toBeInTheDocument();
183 | });
184 |
185 | it("renders journal entries", async () => {
186 |   mockUsePortfolio.mockReturnValue({
187 |     data: basePortfolioData,
188 |     isLoading: false,
189 |   });
190 |   await renderPortfolio();
191 |   expect(
192 |     screen.getByText("Today I learned about polymorphism...")
193 |   ).toBeInTheDocument();
194 | });
195 |
196 | it("shows empty messages when no data", async () => {
197 |   mockUsePortfolio.mockReturnValue({
198 |     data: {
199 |       ...basePortfolioData,
200 |       clos: [],
201 |       badges: [],
202 |       journals: [],
203 |       xpTimeline: [],
204 |       semesterAttainments: [],
205 |     },
206 |     isLoading: false,
207 |   });
208 |   await renderPortfolio();
209 |   expect(screen.getByText("No Attainment Data")).toBeInTheDocument();
210 |   expect(screen.getByText("No Badges Earned")).toBeInTheDocument();
211 |   expect(screen.getByText("No journal entries yet.")).toBeInTheDocument();
212 |   expect(screen.getByText("No XP Data Yet")).toBeInTheDocument();
213 |   expect(screen.getByText("No Semesters Yet")).toBeInTheDocument();
214 | });
215 |
216 | it("renders public profile toggle", async () => {
217 |   mockUsePortfolio.mockReturnValue({
218 |     data: basePortfolioData,
219 |     isLoading: false,
220 |   });
221 |   await renderPortfolio();
222 |   expect(
223 |     screen.getByLabelText("Toggle public portfolio")
224 |   ).toBeInTheDocument();
225 |   expect(screen.getByText("Public Profile")).toBeInTheDocument();
226 | });
227 |
228 | it("calls toggleMutation on switch change", async () => {
229 |   mockUsePortfolio.mockReturnValue({
230 |     data: basePortfolioData,
231 |     isLoading: false,
232 |   });
233 |   await renderPortfolio();
234 |   fireEvent.click(screen.getByLabelText("Toggle public portfolio"));
235 |   expect(mockToggleMutate).toHaveBeenCalledWith(
236 |     { userId: "student-1", isPublic: true },
237 |     expect.objectContaining({ onSuccess: expect.any(Function) })
238 |   );
239 | });
240 |
241 | it("shows copy link button when portfolio is public", async () => {
242 |   mockUseAuth.mockReturnValue({
243 |     user: { id: "student-1" },
244 |     profile: { portfolio_public: true },
245 |   });
246 |   mockUsePortfolio.mockReturnValue({
247 |     data: basePortfolioData,
248 |     isLoading: false,
249 |   });
250 |   await renderPortfolio();
251 |   expect(screen.getByLabelText("Copy shareable link")).toBeInTheDocument();
252 | });
253 |
254 | it("does not show copy link button when portfolio is private", async () => {
255 |   mockUsePortfolio.mockReturnValue({
256 |     data: basePortfolioData,
257 |     isLoading: false,
258 |   });
259 |   await renderPortfolio();
260 |   expect(
261 |     screen.queryByLabelText("Copy shareable link")
262 |   ).not.toBeInTheDocument();
263 | });
264 |
265 | it("passes studentId to usePortfolio", async () => {
266 |   mockUsePortfolio.mockReturnValue({
267 |     data: basePortfolioData,
268 |     isLoading: false,
269 |   });

```



```

374 |     ).toBeInTheDocument();
375 | });
376 |
377 | it("groups outcomes into 'Strengths' and 'Areas improving' (R22.2)", async () => {
378 |     mockUsePortfolio.mockReturnValue({
379 |         data: {
380 |             ...basePortfolioData,
381 |             clos: [
382 |                 {
383 |                     clo_id: "clo-strong",
384 |                     clo_title: "Understand OOP",
385 |                     blooms_level: "understanding" as const,
386 |                     attainment_percent: 88,
387 |                     attainment_level: "Excellent" as const,
388 |                     course_name: "CS101",
389 |                 },
390 |                 {
391 |                     clo_id: "clo-weak",
392 |                     clo_title: "Master Recursion",
393 |                     blooms_level: "applying" as const,
394 |                     attainment_percent: 40,
395 |                     attainment_level: "Not_Yet" as const,
396 |                     course_name: "CS101",
397 |                 },
398 |             ],
399 |         },
400 |         isLoading: false,
401 |     });
402 |     await renderPortfolio();
403 |     expect(
404 |         screen.getByText(i18n.t("student:portfolio.friendly.strengths"))
405 |     ).toBeInTheDocument();
406 |     expect(
407 |         screen.getByText(i18n.t("student:portfolio.friendly.areasImproving"))
408 |     ).toBeInTheDocument();
409 |     expect(screen.getByText("Understand OOP")).toBeInTheDocument();
410 |     expect(screen.getByText("Master Recursion")).toBeInTheDocument();
411 | });
412 |
413 | it("renders the friendly title in Arabic (R22.4)", async () => {
414 |     await i18n.changeLanguage("ar");
415 |     mockUsePortfolio.mockReturnValue({
416 |         data: basePortfolioData,
417 |         isLoading: false,
418 |     });
419 |     await renderPortfolio();
420 |     expect(
421 |         screen.getByText(i18n.t("student:portfolio.friendly.skillsTitle"))
422 |     ).toBeInTheDocument();
423 | });
424 | });
425 |

```

FILE: [src/___tests___/unit/studentProfileRoutes.test.tsx](#)

TYPE: .tsx | LINES: 1-91 | SHA256: 22d6e058769237865a352a3ba99016ebaf763602d6c2227ae3e84c7e5fd9a0af

```

1 | // @vitest-environment happy-dom
2 | import { beforeEach, describe, expect, it, vi } from "vitest";
3 | import { render, screen } from "@testing-library/react";
4 | import { I18nextProvider } from "react-i18next";
5 | import { MemoryRouter } from "react-router-dom";
6 | import i18n from "@lib/i18n";
7 | import type { StudentProfile } from "@hooks/useStudentProfile";
8 |
9 | const state: {
10 |     profile: StudentProfile | null;
11 |     profileError: boolean;
12 |     badgesError: boolean;
13 | } = {
14 |     profile: null,
15 |     profileError: false,
16 |     badgesError: false,
17 | };
18 |
19 | vi.mock("@/hooks/useAuth", () => ({
20 |     useAuth: () => ({ user: { id: "student-1" } }),
21 | }));
22 |
23 | vi.mock("@/hooks/useStudentProfile", () => ({
24 |     useStudentProfile: () => ({
25 |         data: state.profile,
26 |         isLoading: false,
27 |         isError: state.profileError,
28 |         refetch: vi.fn(),
29 |     }),
30 | }));
31 |
32 | vi.mock("@/hooks/useTieredBadges", () => ({
33 |     useTieredBadges: () => ({
34 |         data: [],
35 |         isLoading: false,
36 |         isError: state.badgesError,
37 |         refetch: vi.fn(),
38 |     }),
39 |     usePinBadge: () => ({ mutate: vi.fn() }),
40 |     useUnpinBadge: () => ({ mutate: vi.fn() }),
41 | }));
42 |
43 | import LearningProfilePage from "@pages/student/profile/LearningProfilePage";
44 | import StudentBadgesPage from "@pages/student/badges/StudentBadgesPage";
45 |
46 | const renderPage = (page: React.ReactNode) =>
47 |     render(
48 |         <I18nextProvider i18n={i18n}>

```

```

49 |         <MemoryRouter>{page}</MemoryRouter>
50 |       </I18nextProvider>
51 |     );
52 |
53 | describe("student badge and learning-profile route surfaces", () => {
54 |   beforeEach(() => {
55 |     state.profile = null;
56 |     state.profileError = false;
57 |     state.badgesError = false;
58 |   });
59 |
60 |   it("offers onboarding when no learning profile exists", () => {
61 |     renderPage(<LearningProfilePage />);
62 |
63 |     expect(
64 |       screen.getByText(i18n.t("student:learningProfile.emptyTitle"))
65 |     ).toBeTruthy();
66 |     expect(
67 |       screen
68 |         .getByRole("link", {
69 |           name: i18n.t("student:learningProfile.completeProfile"),
70 |         })
71 |         .getAttribute("href")
72 |     ).toBe("/student/onboarding/complete-profile");
73 |   });
74 |
75 |   it("shows a recoverable learning-profile error", () => {
76 |     state.profileError = true;
77 |     renderPage(<LearningProfilePage />);
78 |
79 |     expect(
80 |       screen.getByText(i18n.t("student:learningProfile.loadError"))
81 |     ).toBeTruthy();
82 |   });
83 |
84 |   it("renders the badges route with the real badge collection state", () => {
85 |     renderPage(<StudentBadgesPage />);
86 |
87 |     expect(screen.getByText(i18n.t("student:badges.title"))).toBeTruthy();
88 |     expect(screen.getByTestId("badge-collection")).toBeTruthy();
89 |   });
90 | });
91 |

```

FILE: [src/__tests__/unit/studentProfileScreen.test.tsx](#)

TYPE: .tsx | LINES: 1-136 | SHA256: 4b4589d6d4dcaa3bbf45900799f7cfa281c23e11ecbb63a09b07bfeba7119624

```

1 | // @vitest-environment happy-dom
2 | import { afterEach, describe, expect, it, vi } from "vitest";
3 | import { cleanup, render, screen } from "@testing-library/react";
4 | import { I18nextProvider } from "react-i18next";
5 | import { MemoryRouter } from "react-router-dom";
6 | import i18n from "@lib/i18n";
7 |
8 | const mocks = vi.hoisted(() => ({
9 |   signOut: vi.fn(),
10 |   toggleAnonymous: vi.fn(),
11 | }));
12 |
13 | vi.mock("@/hooks/useAuth", () => ({
14 |   useAuth: () => ({
15 |     user: { id: "student-1" },
16 |     profile: {
17 |       id: "student-1",
18 |       full_name: "Yusuf Ahmadi",
19 |       email: "student@example.test",
20 |       avatar_url: null,
21 |     },
22 |     signOut: mocks.signOut,
23 |   }),
24 | }));
25 |
26 | vi.mock("@/hooks/useLevel", () => ({
27 |   useLevel: () => ({
28 |     data: {
29 |       level: 4,
30 |       xpTotal: 750,
31 |       xpForCurrentLevel: 500,
32 |       xpForNextLevel: 1000,
33 |       progressPercent: 50,
34 |     },
35 |     isLoading: false,
36 |   }),
37 | }));
38 |
39 | vi.mock("@/hooks/useStreak", () => ({
40 |   useStreak: () => ({
41 |     data: {
42 |       streak_count: 12,
43 |       last_login_date: "2026-07-29",
44 |       streak_freezes_available: 2,
45 |     },
46 |     isLoading: false,
47 |   }),
48 | }));
49 |
50 | vi.mock("@/hooks/useTieredBadges", () => ({
51 |   useTieredBadges: () => ({
52 |     data: [
53 |       {
54 |         id: "badge-1",
55 |         name: "Perfect Day",
56 |         tier: "gold",
57 |       },

```

```

58 |     ],
59 |     isLoading: false,
60 |   }},
61 | });
62 |
63 | vi.mock("@/hooks/useStudentProfile", () => ({
64 |   useStudentProfile: () => ({
65 |     data: {
66 |       profile_completeness: 70,
67 |       learning_style: { dominant_style: "visual" },
68 |     },
69 |   }},
70 | });
71 |
72 | vi.mock("@/hooks/useEquippedItems", () => ({
73 |   useEquippedItems: () => ({ data: [], isLoading: false }),
74 | });
75 |
76 | vi.mock("@/hooks/useHeatmapData", () => ({
77 |   useHeatmapData: () => ({
78 |     data: [
79 |       {
80 |         date: "2026-07-29",
81 |         academicCount: 1,
82 |         wellnessCount: 0,
83 |         totalCount: 1,
84 |         habits: [],
85 |       },
86 |     ],
87 |     isLoading: false,
88 |   }},
89 | });
90 |
91 | vi.mock("@/hooks/useLeaderboard", () => ({
92 |   useAnonymousStatus: () => ({
93 |     data: { isAnonymous: false },
94 |     isLoading: false,
95 |   }),
96 |   useToggleAnonymous: () => ({
97 |     mutate: mocks.toggleAnonymous,
98 |     isPending: false,
99 |   }),
100 | });
101 |
102 | import StudentProfilePage from "@/features/student/profile/StudentProfilePage";
103 |
104 | afterEach(cleanup);
105 |
106 | describe("StudentProfilePage prototype route", () => {
107 |   it("renders real profile, progress, privacy, and account destinations", async () => {
108 |     await i18n.changeLanguage("en");
109 |     render(
110 |       <I18nextProvider i18n={i18n}>
111 |         <MemoryRouter>
112 |           <StudentProfilePage />
113 |         </MemoryRouter>
114 |       </I18nextProvider>
115 |     );
116 |
117 |     expect(
118 |       screen.getByRole("heading", { name: "Yusuf Ahmadi" })
119 |     ).toBeInTheDocument();
120 |     expect(screen.getByText("750")).toBeInTheDocument();
121 |     expect(screen.getByText("Perfect Day")).toBeInTheDocument();
122 |     expect(
123 |       screen.getByRole("switch", {
124 |         name: "Appear anonymous on leaderboards",
125 |       })
126 |     ).toBeInTheDocument();
127 |     expect(
128 |       screen.getByRole("link", { name: /Learning Profile/ })
129 |     ).toHaveAttribute("href", "/student/learning-profile");
130 |     expect(screen.getByRole("link", { name: /Settings/ })).toHaveAttribute(
131 |       "href",
132 |       "/student/settings/profile"
133 |     );
134 |   });
135 | });
136 |

```

FILE: [src/__tests__/unit/studentShellContext.test.tsx](#)

TYPE: .tsx | LINES: 1-87 | SHA256: 09c44ea5dc815286ca2b1e86887efcdbae8e34fa1262d5a96d7233e42a30808a

```

1 | // @vitest-environment happy-dom
2 |
3 | import { afterEach, beforeEach, describe, expect, it, vi } from "vitest";
4 | import { cleanup, render, screen } from "@testing-library/react";
5 | import { I18nextProvider } from "react-i18next";
6 | import { MemoryRouter } from "react-router-dom";
7 | import i18n from "@/lib/i18n";
8 |
9 | const mockState = vi.hoisted(() => ({
10 |   level: {
11 |     level: 4,
12 |     title: "Explorer",
13 |     xpTotal: 750,
14 |     xpForCurrentLevel: 500,
15 |     xpForNextLevel: 1000,
16 |     progressPercent: 50,
17 |   },
18 |   completeness: 70,
19 | });
20 |
21 | vi.mock("@/hooks/useAuth", () => ({

```

```

22 useAuth: () => ({ user: { id: "student-1" } }));
23 });
24
25 vi.mock("@/hooks/useLevel", () => ({
26   useLevel: () => ({
27     data: mockState.level,
28     isPending: false,
29   }),
30 }));
31
32 vi.mock("@/hooks/useStudentProfile", () => ({
33   useStudentProfile: () => ({
34     data: { profile_completeness: mockState.completeness },
35     isPending: false,
36   }),
37 }));
38
39 import StudentSidebarExtras from "@/components/shared/StudentSidebarExtras";
40 import StudentLearningProfileRail from "@/features/student/rails/StudentLearningProfileRail";
41 import StudentSettingsRail from "@/features/student/rails/StudentSettingsRail";
42
43 const renderWithAppContext = (children: React.ReactNode) =>
44   render(
45     <I18nextProvider i18n={i18n}>
46       <MemoryRouter>{children}</MemoryRouter>
47     </I18nextProvider>
48   );
49
50 beforeEach(async () => {
51   await i18n.changeLanguage("en");
52 });
53
54 afterEach(cleanup);
55
56 describe("student prototype shell context", () => {
57   it("renders the student upgrade card", () => {
58     renderWithAppContext(<StudentSidebarExtras />);
59
60     expect(
61       screen.getByRole("link", { name: /Upgrade to Premium/i })
62     ).toHaveAttribute("href", "/student/marketplace");
63   });
64
65   it("binds the learning-profile rail completeness to profile data", () => {
66     renderWithAppContext(<StudentLearningProfileRail />);
67
68     expect(screen.getByText("ðŸŽŸ Why this matters")).toBeInTheDocument();
69     expect(screen.getByText("70%")).toBeInTheDocument();
70     expect(
71       screen.getByRole("progressbar", { name: "Profile completeness" })
72     ).toHaveAttribute("aria-valuenow", "70");
73     expect(
74       screen.getByRole("link", { name: "Finish micro-assessments !" })
75     ).toHaveAttribute("href", "/student/settings/reassessment");
76   });
77
78   it("renders the approved student settings privacy rail", () => {
79     renderWithAppContext(<StudentSettingsRail />);
80
81     expect(screen.getByText("ðŸŒ™ Your data")).toBeInTheDocument();
82     expect(
83       screen.getByText(/Nothing here is shared with classmates/)
84     ).toBeInTheDocument();
85   });
86 });
87

```

FILE: src/__tests__/unit/studentTeamPage.test.tsx

TYPE: .tsx | LINES: 1-139 | SHA256: 0dff32ddb2ed78326164d74deb337de487cbb34e599221651dc4f95478e8dfc5

FILE: src/___tests___/unit/StudentTranscriptPage.test.tsx

TYPE: .tsx | LINES: 1-104 | SHA256: 39d26e62960c28fdfa811a6381468353a3793cb548f28467de9c25cee21a05b2

```

1 | // StudentTranscriptPage – functional render tests (net-new screen, P3.6).
2 | // Feature: prototype-frontend-rebuild. Pixel parity is verified separately via
3 | // the visual harness once routed; this covers behavior across all states.
4 | import { describe, it, expect, beforeEach, vi } from "vitest";
5 | import { render, screen, fireEvent } from "@testing-library/react";
6 | import { MemoryRouter } from "react-router-dom";
7 | import StudentTranscriptPage from "@features/student/transcript/StudentTranscriptPage";
8 |
9 | // Controllable mutation stand-in for useGenerateTranscript.
10 | const hoisted = vi.hoisted(() => ({
11 |   mutation: {
12 |     mutate: vi.fn(),
13 |     isPending: false,
14 |     isError: false,
15 |     isSuccess: false,
16 |     data: undefined as { download_url: string; file_name: string } | undefined,
17 |   },
18 | }));
19 |
20 | vi.mock("@/hooks/useTranscript", () => ({
21 |   useGenerateTranscript: () => hoisted.mutation,
22 | }));
23 |
24 | vi.mock("@/hooks/useAuth", () => ({
25 |   useAuth: () => ({
26 |     user: { id: "student-1234abcd" },
27 |     profile: { full_name: "Sarah Ahmed" },
28 |     role: "student",
29 |     institutionId: null,
30 |     isLoading: false,
31 |     signIn: vi.fn(),
32 |     signUp: vi.fn(),
33 |     signOut: vi.fn(),
34 |     resetPassword: vi.fn(),
35 |     refetchProfile: vi.fn(),
36 |   }),
37 | }));
38 |
39 | vi.mock("sonner", () => ({ toast: { success: vi.fn(), error: vi.fn() } }));
40 |
41 | vi.mock("react-i18next", () => ({
42 |   useTranslation: () => ({ t: (_key: string, def?: string) => def ?? _key }),
43 | }));
44 |
45 | const renderPage = () =>
46 |   render(
47 |     <MemoryRouter>
48 |       <StudentTranscriptPage />
49 |     </MemoryRouter>
50 |   );
51 |
52 | beforeEach(() => {
53 |   hoisted.mutation.mutate = vi.fn();
54 |   hoisted.mutation.isPending = false;
55 |   hoisted.mutation.isError = false;
56 |   hoisted.mutation.isSuccess = false;
57 |   hoisted.mutation.data = undefined;
58 | });
59 |
60 | describe("StudentTranscriptPage", () => {
61 |   it("renders the student identity and the official-transcript action (idle)", () => {
62 |     renderPage();
63 |     expect(screen.getByText("Sarah Ahmed")).toBeInTheDocument();
64 |     expect(screen.getByText("Official transcript")).toBeInTheDocument();
65 |     expect(screen.getByTestId("transcript-generate")).not.toBeDisabled();
66 |   });
67 |
68 |   it("requests the transcript for the current student on click", () => {
69 |     renderPage();
70 |     fireEvent.click(screen.getByTestId("transcript-generate"));
71 |     expect(hoisted.mutation.mutate).toHaveBeenCalled();
72 |     expect(hoisted.mutation.mutate).toHaveBeenCalledWith(
73 |       { student_id: "student-1234abcd" },
74 |     );
75 |   });
76 |
77 |   it("shows a disabled button and progress copy while generating", () => {
78 |     hoisted.mutation.isPending = true;
79 |     renderPage();
80 |     expect(screen.getByTestId("transcript-generate")).toBeDisabled();
81 |     expect(
82 |       screen.getByText("Generating your official transcript...")
83 |     ).toBeInTheDocument();
84 |   });
85 |
86 |   it("surfaces an error state", () => {
87 |     hoisted.mutation.isError = true;
88 |     renderPage();
89 |     expect(screen.getByRole("alert")).toHaveTextContent(
90 |       "Could not generate transcript. Please try again."
91 |     );
92 |   });
93 |
94 |   it("shows the generated file name on success", () => {
95 |     hoisted.mutation.isSuccess = true;
96 |     hoisted.mutation.data = {
97 |       download_url: "blob:transcript",
98 |       file_name: "transcript_sarah_2026.pdf",
99 |     };

```



```

100 |     renderPage();
101 |     expect(screen.getByText("transcript_sarah_2026.pdf")).toBeInTheDocument();
102 | });
103 | });
104 |

```

FILE: [src/__tests__/unit/studyBadges.test.ts](#)

TYPE: .ts | LINES: 1-267 | SHA256: 3e06d51aa99c27bc5ca69ad9a09f8c4b1f22914f96fc62c122ef765feee9f3f7

```

1 | import { describe, it, expect } from "vitest";
2 | import { getBadgeById, getBadgesByCategory } from "@lib/badgeDefinitions";
3 |
4 | // % % % Edge Function constants (mirrored for testing) % % % % % % % % % % % % % % % % %
5 |
6 | const VALID_TRIGGERS = [
7 |   "xp_award",
8 |   "submission",
9 |   "streak_update",
10 |   "grade",
11 |   "journal",
12 |   "habit_log",
13 |   "team_event",
14 |   "study_session",
15 | ] as const;
16 |
17 | const BADGE_XP: Record<string, number> = {
18 |   study_starter: 25,
19 |   deep_focus: 50,
20 |   weekly_warrior: 100,
21 |   evidence_pro: 75,
22 | };
23 |
24 | // % % % Pure logic helpers (mirrored from Edge Function for testability) % % % % % % %
25 |
26 | /**
27 |  * Checks Study Starter condition: at least 1 completed study session.
28 |  */
29 | function meetsStudyStarterCondition(completedSessionCount: number): boolean {
30 |   return completedSessionCount >= 1;
31 | }
32 |
33 | /**
34 |  * Checks Deep Focus condition: at least one session with actual_duration_minutes >= 60.
35 |  */
36 | function meetsDeepFocusCondition(sessionDurations: number[]): boolean {
37 |   return sessionDurations.some((d) => d >= 60);
38 | }
39 |
40 | /**
41 |  * Checks Weekly Warrior condition: all 3 goals met in a single week.
42 |  * Takes an array of goal progress objects for a single week.
43 |  */
44 | function meetsWeeklyWarriorCondition(
45 |   goals: Array<{ target_value: number; current_value: number }>
46 | ): boolean {
47 |   if (goals.length !== 3) return false;
48 |   return goals.every((g) => g.current_value >= g.target_value);
49 | }
50 |
51 | /**
52 |  * Checks Evidence Pro condition: 10 distinct sessions with evidence.
53 |  */
54 | function meetsEvidenceProCondition(
55 |   distinctSessionsWithEvidence: number
56 | ): boolean {
57 |   return distinctSessionsWithEvidence >= 10;
58 | }
59 |
60 | // % % % Tests % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
61 |
62 | describe("Study Badge Trigger Validation", () => {
63 |   it("study_session is a valid trigger", () => {
64 |     expect(VALID_TRIGGERS).toContain("study_session");
65 |   });
66 |
67 |   it("all original triggers remain valid", () => {
68 |     expect(VALID_TRIGGERS).toContain("xp_award");
69 |     expect(VALID_TRIGGERS).toContain("submission");
70 |     expect(VALID_TRIGGERS).toContain("streak_update");
71 |     expect(VALID_TRIGGERS).toContain("grade");
72 |     expect(VALID_TRIGGERS).toContain("journal");
73 |     expect(VALID_TRIGGERS).toContain("habit_log");
74 |     expect(VALID_TRIGGERS).toContain("team_event");
75 |   });
76 | });
77 |
78 | describe("Study Starter Badge – First Completed Session", () => {
79 |   it("awards badge when exactly 1 completed session", () => {
80 |     expect(meetsStudyStarterCondition(1)).toBe(true);
81 |   });
82 |
83 |   it("awards badge when more than 1 completed session", () => {
84 |     expect(meetsStudyStarterCondition(5)).toBe(true);
85 |     expect(meetsStudyStarterCondition(100)).toBe(true);
86 |   });
87 |
88 |   it("does not award badge with 0 completed sessions", () => {
89 |     expect(meetsStudyStarterCondition(0)).toBe(false);
90 |   });
91 |
92 |   it("has correct XP reward of 25", () => {
93 |     expect(BADGE_XP["study_starter"]).toBe(25);
94 |   });
95 | });

```

```

96 | describe("Deep Focus Badge – Single Session "e 60 min", () => {
97 |   it("awards badge when a session is exactly 60 minutes", () => {
98 |     expect(meetsDeepFocusCondition([60])).toBe(true);
99 |   });
100 |
101 |   it("awards badge when a session exceeds 60 minutes", () => {
102 |     expect(meetsDeepFocusCondition([90])).toBe(true);
103 |     expect(meetsDeepFocusCondition([120])).toBe(true);
104 |   });
105 |
106 |   it("awards badge when at least one session is "e 60 among shorter ones", () => {
107 |     expect(meetsDeepFocusCondition([15, 25, 60, 30])).toBe(true);
108 |   });
109 |
110 |   it("does not award badge when all sessions are under 60 minutes", () => {
111 |     expect(meetsDeepFocusCondition([15, 25, 45, 59])).toBe(false);
112 |   });
113 |
114 |   it("does not award badge with no sessions", () => {
115 |     expect(meetsDeepFocusCondition([])).toBe(false);
116 |   });
117 |
118 |   it("has correct XP reward of 50", () => {
119 |     expect(BADGE_XP["deep_focus"]).toBe(50);
120 |   });
121 | });
122 |
123 | describe("Weekly Warrior Badge – All 3 Goals Met in a Week", () => {
124 |   it("awards badge when all 3 goals are met", () => {
125 |     const goals = [
126 |       { target_value: 10, current_value: 12 },
127 |       { target_value: 5, current_value: 5 },
128 |       { target_value: 3, current_value: 3 },
129 |     ];
130 |     expect(meetsWeeklyWarriorCondition(goals)).toBe(true);
131 |   });
132 |
133 |   it("awards badge when goals are exactly met", () => {
134 |     const goals = [
135 |       { target_value: 10, current_value: 10 },
136 |       { target_value: 5, current_value: 5 },
137 |       { target_value: 3, current_value: 3 },
138 |     ];
139 |     expect(meetsWeeklyWarriorCondition(goals)).toBe(true);
140 |   });
141 |
142 |   it("does not award badge when one goal is not met", () => {
143 |     const goals = [
144 |       { target_value: 10, current_value: 10 },
145 |       { target_value: 5, current_value: 4 },
146 |       { target_value: 3, current_value: 3 },
147 |     ];
148 |     expect(meetsWeeklyWarriorCondition(goals)).toBe(false);
149 |   });
150 |
151 |   it("does not award badge when no goals are met", () => {
152 |     const goals = [
153 |       { target_value: 10, current_value: 0 },
154 |       { target_value: 5, current_value: 0 },
155 |       { target_value: 3, current_value: 0 },
156 |     ];
157 |     expect(meetsWeeklyWarriorCondition(goals)).toBe(false);
158 |   });
159 |
160 |   it("does not award badge with fewer than 3 goals", () => {
161 |     const goals = [
162 |       { target_value: 10, current_value: 10 },
163 |       { target_value: 5, current_value: 5 },
164 |     ];
165 |     expect(meetsWeeklyWarriorCondition(goals)).toBe(false);
166 |   });
167 |
168 |   it("does not award badge with more than 3 goals", () => {
169 |     const goals = [
170 |       { target_value: 10, current_value: 10 },
171 |       { target_value: 5, current_value: 5 },
172 |       { target_value: 3, current_value: 3 },
173 |       { target_value: 2, current_value: 2 },
174 |     ];
175 |     expect(meetsWeeklyWarriorCondition(goals)).toBe(false);
176 |   });
177 |
178 |   it("does not award badge with 0 goals", () => {
179 |     expect(meetsWeeklyWarriorCondition([])).toBe(false);
180 |   });
181 |
182 |   it("has correct XP reward of 100", () => {
183 |     expect(BADGE_XP["weekly_warrior"]).toBe(100);
184 |   });
185 | });
186 |
187 | describe("Evidence Pro Badge – 10 Sessions with Evidence", () => {
188 |   it("awards badge with exactly 10 sessions with evidence", () => {
189 |     expect(meetsEvidenceProCondition(10)).toBe(true);
190 |   });
191 |
192 |   it("awards badge with more than 10 sessions with evidence", () => {
193 |     expect(meetsEvidenceProCondition(15)).toBe(true);
194 |     expect(meetsEvidenceProCondition(100)).toBe(true);
195 |   });
196 |
197 |   it("does not award badge with fewer than 10 sessions", () => {
198 |     expect(meetsEvidenceProCondition(9)).toBe(false);
199 |   });

```

```

200 expect(meetsEvidenceProCondition(0)).toBe(false);
201 });
202
203 it("has correct XP reward of 75", () => {
204   expect(BADGE_XP["evidence_pro"]).toBe(75);
205 });
206 });
207
208 describe("Badge Definitions - study badges", () => {
209   it("includes study_starter badge definition", () => {
210     const badge = getBadgeById("study_starter");
211     expect(badge).toBeDefined();
212     expect(badge!.name).toBe("Study Starter");
213     expect(badge!.icon).toBe("0=0U");
214     expect(badge!.category).toBe("study");
215     expect(badge!.xpReward).toBe(25);
216     expect(badge!.isMystery).toBe(false);
217   });
218
219   it("includes deep_focus badge definition", () => {
220     const badge = getBadgeById("deep_focus");
221     expect(badge).toBeDefined();
222     expect(badge!.name).toBe("Deep Focus");
223     expect(badge!.icon).toBe("0>Y0");
224     expect(badge!.category).toBe("study");
225     expect(badge!.xpReward).toBe(50);
226     expect(badge!.isMystery).toBe(false);
227   });
228
229   it("includes weekly_warrior badge definition", () => {
230     const badge = getBadgeById("weekly_warrior");
231     expect(badge).toBeDefined();
232     expect(badge!.name).toBe("Weekly Warrior");
233     expect(badge!.icon).toBe("0<0Æ");
234     expect(badge!.category).toBe("study");
235     expect(badge!.xpReward).toBe(100);
236     expect(badge!.isMystery).toBe(false);
237   });
238
239   it("includes evidence_pro badge definition", () => {
240     const badge = getBadgeById("evidence_pro");
241     expect(badge).toBeDefined();
242     expect(badge!.name).toBe("Evidence Pro");
243     expect(badge!.icon).toBe("0=0U");
244     expect(badge!.category).toBe("study");
245     expect(badge!.xpReward).toBe(75);
246     expect(badge!.isMystery).toBe(false);
247   });
248
249   it("all four study badges are in the study category", () => {
250     const studyBadges = getBadgesByCategory("study");
251     expect(studyBadges.length).toBe(4);
252     const ids = studyBadges.map((b) => b.id);
253     expect(ids).toContain("study_starter");
254     expect(ids).toContain("deep_focus");
255     expect(ids).toContain("weekly_warrior");
256     expect(ids).toContain("evidence_pro");
257   });
258
259   it("study badges have meaningful descriptions", () => {
260     const studyBadges = getBadgesByCategory("study");
261     for (const badge of studyBadges) {
262       expect(badge.description.length).toBeGreaterThan(10);
263       expect(badge.condition.length).toBeGreaterThan(5);
264     }
265   });
266 });
267

```

FILE: src/__tests__/unit/studySessionCard.test.tsx

TYPE: .tsx | LINES: 1-176 | SHA256: efa96f18d4f0aca1c64d9e8bbcfef07bda21832888e270160ab1c25b6e235f07a

[illegible]

[illegible]

```

137 |     const onStart = vi.fn();
138 |     const session = makeSession();
139 |     render(<StudySessionCard session={session} onStart={onStart} />);
140 |
141 |     await user.click(screen.getByRole("button", { name: /start/i }));
142 |     expect(onStart).toHaveBeenCalledWith(session);
143 | });
144 |
145 | it("calls onEdit when Edit button is clicked", async () => {
146 |   const user = userEvent.setup();
147 |   const onEdit = vi.fn();
148 |   const session = makeSession();
149 |   render(<StudySessionCard session={session} onEdit={onEdit} />);
150 |
151 |   await user.click(screen.getByRole("button", { name: /edit/i }));
152 |   expect(onEdit).toHaveBeenCalledWith(session);
153 | });
154 |
155 | it("does not show action buttons in compact mode", () => {
156 |   const onStart = vi.fn();
157 |   const onEdit = vi.fn();
158 |   render(
159 |     <StudySessionCard
160 |       session={makeSession()}
161 |       onStart={onStart}
162 |       onEdit={onEdit}
163 |       compact
164 |     />
165 |   );
166 |   expect(screen.queryByRole("button", { name: /start/i })).toBeNull();
167 |   expect(screen.queryByRole("button", { name: /edit/i })).toBeNull();
168 | });
169 |
170 | it("does not show action buttons when no handlers provided", () => {
171 |   render(<StudySessionCard session={makeSession()} />);
172 |   expect(screen.queryByRole("button", { name: /start/i })).toBeNull();
173 |   expect(screen.queryByRole("button", { name: /edit/i })).toBeNull();
174 | });
175 | });
176 |

```

FILE: src/__tests__/unit/studyTimeChart.test.tsx

TYPE: .tsx | LINES: 1-124 | SHA256: bea004c79935f82a386e94bc5c97e3baa554929f942e66add4f1fa55955164ea

[illegible]

```

61 | it("does not display average legend when average is 0", () => {
62 |   render(<StudyTimeChart data={makeTrendData()} averageMinutesPerWeek={0} />);
63 |   expect(screen.queryByText(/Average/)).not.toBeInTheDocument();
64 | });
65 |
66 | it("renders course filter toggles when courseOptions are provided", () => {
67 |   const courseOptions = [
68 |     { id: "c1", name: "Math 101" },
69 |     { id: "c2", name: "Physics 201" },
70 |   ];
71 |   render(
72 |     <StudyTimeChart
73 |       data={makeTrendData()}
74 |       averageMinutesPerWeek={150}
75 |       courseOptions={courseOptions}
76 |     />
77 |   );
78 |   expect(screen.getByTestId("course-filter-toggles")).toBeInTheDocument();
79 |   expect(screen.getByText("All Courses")).toBeInTheDocument();
80 |   expect(screen.getByText("Math 101")).toBeInTheDocument();
81 |   expect(screen.getByText("Physics 201")).toBeInTheDocument();
82 | });
83 |
84 | it("does not render course filter toggles when no courseOptions", () => {
85 |   render(
86 |     <StudyTimeChart data={makeTrendData()} averageMinutesPerWeek={150} />
87 |   );
88 |   expect(
89 |     screen.queryByTestId("course-filter-toggles")
90 |   ).not.toBeInTheDocument();
91 | });
92 |
93 | it("calls onCourseFilterChange when a course filter is clicked", () => {
94 |   const onFilterChange = vi.fn();
95 |   const courseOptions = [{ id: "c1", name: "Math 101" }];
96 |   render(
97 |     <StudyTimeChart
98 |       data={makeTrendData()}
99 |       averageMinutesPerWeek={150}
100 |       courseOptions={courseOptions}
101 |       onCourseFilterChange={onFilterChange}
102 |     />
103 |   );
104 |   fireEvent.click(screen.getByText("Math 101"));
105 |   expect(onFilterChange).toHaveBeenCalled("c1");
106 | });
107 |
108 | it("calls onCourseFilterChange with null when All Courses is clicked", () => {
109 |   const onFilterChange = vi.fn();
110 |   const courseOptions = [{ id: "c1", name: "Math 101" }];
111 |   render(
112 |     <StudyTimeChart
113 |       data={makeTrendData()}
114 |       averageMinutesPerWeek={150}
115 |       courseOptions={courseOptions}
116 |       onCourseFilterChange={onFilterChange}
117 |       courseFilter="c1"
118 |     />
119 |   );
120 |   fireEvent.click(screen.getByText("All Courses"));
121 |   expect(onFilterChange).toHaveBeenCalled(null);
122 | });
123 | });
124 |

```

FILE: [src/__tests__/unit/suggestedSessionItem.test.tsx](#)

TYPE: .tsx | LINES: 1-89 | SHA256: 677e71424a9aa22ffcaf5526897ff5b7595e044ad87c7d2562d5f4d7c6f54d0b

```

1 | // =====
2 | // Unit Tests – SuggestedSessionItem + WeeklyCalendarGrid suggestion rendering
3 | // Verifies that empty days surface a localized suggested study session instead
4 | // of a bare "No items" label (R19.2), with copy routed through i18next.
5 | // =====
6 |
7 | import { describe, it, expect, vi } from "vitest";
8 | import { render, screen } from "@testing-library/react";
9 | import userEvent from "@testing-library/user-event";
10 | import { I18nextProvider } from "react-i18next";
11 |
12 | import i18n from "@lib/i18n";
13 | import SuggestedSessionItem from "@components/shared/SuggestedSessionItem";
14 | import WeeklyCalendarGrid from "@components/shared/WeeklyCalendarGrid";
15 | import { buildPlannerWeek } from "@lib/weeklyPlannerContent";
16 | import type { SuggestedStudySession } from "@types/planner";
17 |
18 | const wrap = (ui: React.ReactNode) => (
19 |   <I18nextProvider i18n={i18n}>{ui}</I18nextProvider>
20 | );
21 |
22 | const makeSuggestion = (
23 |   overrides: Partial<SuggestedStudySession> = {}
24 | ): SuggestedStudySession => ({
25 |   id: "suggestion-2025-06-18",
26 |   date: "2025-06-18",
27 |   courseId: "course-1",
28 |   courseName: "Math 101",
29 |   durationMinutes: 25,
30 |   ...overrides,
31 | });
32 |
33 | describe("SuggestedSessionItem", () => {
34 |   it("renders the course-scoped heading and duration", () => {
35 |     render(wrap(<SuggestedSessionItem suggestion={makeSuggestion()} />));
36 |     expect(screen.getByText(/Math 101/)).toBeInTheDocument();

```

```

37 |     expect(screen.getByText(/25 min focus block/)).toBeInTheDocument();
38 | });
39 |
40 | it("renders a generic label when no course is attached", () => {
41 |     render(
42 |         wrap(
43 |             <SuggestedSessionItem
44 |                 suggestion={makeSuggestion({ courseId: null, courseName: null })}
45 |             />
46 |         )
47 |     );
48 |     expect(screen.getByText("Suggested study session")).toBeInTheDocument();
49 | });
50 |
51 | it("invokes onPlan when the plan button is clicked", async () => {
52 |     const user = userEvent.setup();
53 |     const onPlan = vi.fn();
54 |     const suggestion = makeSuggestion();
55 |     render(
56 |         wrap(<SuggestedSessionItem suggestion={suggestion} onPlan={onPlan} />)
57 |     );
58 |
59 |     await user.click(screen.getByRole("button", { name: /plan a 25 minute/i }));
60 |     expect(onPlan).toHaveBeenCalled();
61 | });
62 |
63 | it("renders no action button when onPlan is omitted", () => {
64 |     render(wrap(<SuggestedSessionItem suggestion={makeSuggestion()} />));
65 |     expect(screen.queryByRole("button")).toBeNull();
66 | });
67 | });
68 |
69 | describe("WeeklyCalendarGrid with derived suggestions", () => {
70 |     it("shows suggestions on empty days instead of 'No items'", () => {
71 |         const weekData = buildPlannerWeek({
72 |             weekStartDate: "2025-06-16",
73 |             todayStr: "2025-06-16",
74 |             sessions: [],
75 |             tasks: [],
76 |             deadlines: [],
77 |             courses: [{ id: "course-1", name: "Math 101" }],
78 |         });
79 |
80 |         render(wrap(<WeeklyCalendarGrid weekData={weekData} today="2025-06-16" />));
81 |
82 |         // Every empty day now has a suggestion !' no "No items" placeholders remain
83 |         expect(screen.queryByText("No items")).toBeNull();
84 |         // Suggested cards are present (desktop renders all 7)
85 |         const suggested = screen.getAllByTestId("suggested-session");
86 |         expect(suggested.length).toBeGreaterThanOrEqual(7);
87 |     });
88 | });
89 |

```

FILE: src/__tests__/unit/supabase.test.ts

TYPE: .ts | LINES: 1-44 | SHA256: 8234decf3acdee41c034bab26eac009fad0fbd8559660d495fbc4a93114ebeb

```

1 | import { describe, it, expect } from "vitest";
2 |
3 | describe("Supabase client singleton", () => {
4 |     it("exports createClient from @supabase/supabase-js", async () => {
5 |         const { createClient } = await import("@supabase/supabase-js");
6 |         expect(typeof createClient).toBe("function");
7 |     });
8 |
9 |     it("supabase module exports a client instance", async () => {
10 |         const fs = await import("fs");
11 |         const source = fs.readFileSync("src/lib/supabase.ts", "utf-8");
12 |
13 |         expect(source).toContain(
14 |             'import { createClient } from "@supabase/supabase-js"'
15 |         );
16 |         expect(source).toContain("VITE_SUPABASE_URL");
17 |         expect(source).toContain("VITE_SUPABASE_ANON_KEY");
18 |         expect(source).toContain("export const supabase");
19 |         expect(source).toContain("createClient<Database>");
20 |     });
21 |
22 |     it("throws when VITE_SUPABASE_URL is missing", async () => {
23 |         const fs = await import("fs");
24 |         const source = fs.readFileSync("src/lib/supabase.ts", "utf-8");
25 |         expect(source).toContain("if (!supabaseUrl || !supabaseAnonKey)");
26 |         expect(source).toContain("throw new Error");
27 |         expect(source).toContain("Missing Supabase environment variables");
28 |     });
29 |
30 |     it("uses @/ path alias for database type import", async () => {
31 |         const fs = await import("fs");
32 |         const source = fs.readFileSync("src/lib/supabase.ts", "utf-8");
33 |         expect(source).toContain('from "@types/database"');
34 |         expect(source).not.toContain("from '../types/database'");
35 |     });
36 |
37 |     it("does not contain hardcoded fallback values", async () => {
38 |         const fs = await import("fs");
39 |         const source = fs.readFileSync("src/lib/supabase.ts", "utf-8");
40 |         expect(source).not.toContain("localhost:54321");
41 |         expect(source).not.toContain("placeholder-anon-key");
42 |     });
43 | });
44 |

```

FILE: src/__tests__/unit/supabaseCastGuard.test.ts

```
TYPE: .ts | LINES: 1-251 | SHA256: ed9496b3899f910d4f1f7cd34b2f6484c7c7bfe32cda5061bc5b1b7c33ba29cb
```

[illegible]

[illegible]

```

206
207 it("passes when live violations exactly match the allowlist", () => {
208   const live: CastViolation[] = [
209     { file: "src/hooks/useTeams.ts", pattern: "insert-as-never", line: 70 },
210   ];
211   const verdict = evaluateCastGuard(live, base);
212   expect(verdict.pass).toBe(true);
213   expect(verdict.reasons).toEqual([]);
214 });
215
216 it("fails and reports the file/location for a new violation (Req 17.2, 17.4)", () => {
217   const live: CastViolation[] = [
218     { file: "src/hooks/useTeams.ts", pattern: "insert-as-never", line: 70 },
219     { file: "src/hooks/useNew.ts", pattern: "from-as-never", line: 12 },
220   ];
221   const verdict = evaluateCastGuard(live, { ...base, maxCount: 2 });
222   expect(verdict.pass).toBe(false);
223   expect(verdict.newViolations).toHaveLength(1);
224   expect(verdict.reasons.join("\n")).toContain("src/hooks/useNew.ts:12");
225 });
226
227 it("fails on a stale allowlist entry that no longer matches code (Req 17.5)", () => {
228   const verdict = evaluateCastGuard([], base);
229   expect(verdict.pass).toBe(false);
230   expect(verdict.staleEntries).toEqual([base.entries[0]]);
231   expect(verdict.reasons.join("\n")).toContain("Stale allowlist entry");
232 });
233
234 it("fails when the live count exceeds maxCount (Req 17.7)", () => {
235   const live: CastViolation[] = [
236     { file: "src/hooks/useTeams.ts", pattern: "insert-as-never", line: 70 },
237     { file: "src/hooks/useTeams.ts", pattern: "from-as-never", line: 40 },
238   ];
239   const allow: Allowlist = {
240     maxCount: 1,
241     entries: [
242       { file: "src/hooks/useTeams.ts", pattern: "insert-as-never" },
243       { file: "src/hooks/useTeams.ts", pattern: "from-as-never" },
244     ],
245   };
246   const verdict = evaluateCastGuard(live, allow);
247   expect(verdict.countExceeded).toBe(true);
248   expect(verdict.pass).toBe(false);
249 });
250 });
251

```

FILE: src/__tests__/unit/surveyResultsAggregation.test.ts

TYPE: .ts | LINES: 1-131 | SHA256: 1a9e853b6fba0a832260559024f351965ecfb065e86221ac22145d2f38c0a6df

[illegible]

```

55 | it("ignores responses for other questions", () => {
56 |     const responses = [
57 |         makeResponse("q1", "Agree"),
58 |         makeResponse("q2", "Disagree"),
59 |     ];
60 |
61 |     const result = aggregateLikert(responses, "q1");
62 |     expect(result.find((r) => r.label === "Agree").count).toBe(1);
63 |     expect(result.find((r) => r.label === "Disagree").count).toBe(0);
64 | });
65 |
66 | it("ignores invalid Likert values", () => {
67 |     const responses = [
68 |         makeResponse("q1", "Invalid Value"),
69 |         makeResponse("q1", "Agree"),
70 |     ];
71 |
72 |     const result = aggregateLikert(responses, "q1");
73 |     const total = result.reduce((sum, r) => sum + r.count, 0);
74 |     expect(total).toBe(1);
75 | });
76 | });
77 |
78 | describe("aggregateMCQ", () => {
79 |     it("counts each option correctly", () => {
80 |         const responses = [
81 |             makeResponse("q2", "Option A", "stu-1"),
82 |             makeResponse("q2", "Option B", "stu-2"),
83 |             makeResponse("q2", "Option A", "stu-3"),
84 |         ];
85 |
86 |         const result = aggregateMCQ(responses, "q2");
87 |         expect(result).toContainEqual({ option: "Option A", count: 2 });
88 |         expect(result).toContainEqual({ option: "Option B", count: 1 });
89 |     });
90 |
91 |     it("returns empty array for no responses", () => {
92 |         const result = aggregateMCQ([], "q2");
93 |         expect(result).toEqual([]);
94 |     });
95 |
96 |     it("ignores responses for other questions", () => {
97 |         const responses = [
98 |             makeResponse("q2", "Option A"),
99 |             makeResponse("q3", "Option B"),
100 |         ];
101 |
102 |         const result = aggregateMCQ(responses, "q2");
103 |         expect(result).toHaveLength(1);
104 |         expect(result[0]).toEqual({ option: "Option A", count: 1 });
105 |     });
106 | });
107 |
108 | describe("unique respondent counting", () => {
109 |     it("counts unique respondents from responses", () => {
110 |         const responses = [
111 |             makeResponse("q1", "Agree", "stu-1"),
112 |             makeResponse("q2", "Option A", "stu-1"),
113 |             makeResponse("q1", "Neutral", "stu-2"),
114 |             makeResponse("q2", "Option B", "stu-2"),
115 |             makeResponse("q1", "Agree", "stu-3"),
116 |         ];
117 |
118 |         const uniqueRespondents = new Set(responses.map((r) => r.respondent_id))
119 |             .size;
120 |         expect(uniqueRespondents).toBe(3);
121 |     });
122 |
123 |     it("returns 0 for empty responses", () => {
124 |         const responses: MockResponse[] = [];
125 |         const uniqueRespondents = new Set(responses.map((r) => r.respondent_id))
126 |             .size;
127 |         expect(uniqueRespondents).toBe(0);
128 |     });
129 | });
130 | });
131 |

```

FILE: [src/__tests__/unit/teachingMomentRating.test.tsx](#)

TYPE: .tsx | LINES: 1-81 | SHA256: 613fdb15248de02ca993e27d7b91ceb157987c0aa70c0c0eeff090138578cd3f

```

1 | // Unit test: TeachingMomentRating – star rating 1-5 for clarity and helpfulness
2 | import { describe, it, expect, vi } from "vitest";
3 | import { render, screen, fireEvent } from "@testing-library/react";
4 | import TeachingMomentRating from "@components/shared/TeachingMomentRating";
5 |
6 | describe("TeachingMomentRating", () => {
7 |     it("renders clarity and helpfulness labels", () => {
8 |         render(<TeachingMomentRating onSubmit={vi.fn()} />);
9 |         expect(screen.getByText("Clarity")).toBeDefined();
10 |         expect(screen.getByText("Helpfulness")).toBeDefined();
11 |     });
12 |
13 |     it("renders 5 stars for each rating category", () => {
14 |         render(<TeachingMomentRating onSubmit={vi.fn()} />);
15 |         const clarityGroup = screen.getByLabelText("Clarity rating");
16 |         const helpfulnessGroup = screen.getByLabelText("Helpfulness rating");
17 |         expect(clarityGroup.querySelectorAll('[role="radio"]').length).toBe(5);
18 |         expect(helpfulnessGroup.querySelectorAll('[role="radio"]').length).toBe(5);
19 |     });
20 |
21 |     it("submit button is disabled until both ratings are set", () => {
22 |         render(<TeachingMomentRating onSubmit={vi.fn()} />);
23 |         const submitBtn = screen.getByText("Submit Rating").closest("button");

```

```

24 |     expect(submitBtn?.disabled).toBe(true);
25 | });
26 |
27 | it("submit button is enabled after both ratings are set", () => {
28 |   render(<TeachingMomentRating onSubmit={vi.fn()} />);
29 |   // Click 3rd star in clarity
30 |   const clarityStars = screen
31 |     .getByLabelText("Clarity rating")
32 |     .querySelectorAll('[role="radio"]');
33 |   fireEvent.click(clarityStars[2]!);
34 |   // Click 4th star in helpfulness
35 |   const helpfulnessStars = screen
36 |     .getByLabelText("Helpfulness rating")
37 |     .querySelectorAll('[role="radio"]');
38 |   fireEvent.click(helpfulnessStars[3]!);
39 |
40 |   const submitBtn = screen.getByText("Submit Rating").closest("button");
41 |   expect(submitBtn?.disabled).toBe(false);
42 | });
43 |
44 | it("calls onSubmit with clarity and helpfulness values", () => {
45 |   const onSubmit = vi.fn();
46 |   render(<TeachingMomentRating onSubmit={onSubmit} />);
47 |
48 |   const clarityStars = screen
49 |     .getByLabelText("Clarity rating")
50 |     .querySelectorAll('[role="radio"]');
51 |   fireEvent.click(clarityStars[3]!); // 4 stars
52 |
53 |   const helpfulnessStars = screen
54 |     .getByLabelText("Helpfulness rating")
55 |     .querySelectorAll('[role="radio"]');
56 |   fireEvent.click(helpfulnessStars[4]!); // 5 stars
57 |
58 |   fireEvent.click(screen.getByText("Submit Rating"));
59 |   expect(onSubmit).toHaveBeenCalledWith(4, 5);
60 | });
61 |
62 | it("renders in read-only mode without submit button", () => {
63 |   render(
64 |     <TeachingMomentRating
65 |       onSubmit={vi.fn()}
66 |       readOnly
67 |       initialClarity={3}
68 |       initialHelpfulness={4}
69 |     />
70 |   );
71 |   expect(screen.getByText("Submit Rating")).toBeNull();
72 | });
73 |
74 | it("stars have accessible aria-label", () => {
75 |   render(<TeachingMomentRating onSubmit={vi.fn()} />);
76 |   // Two "1 star" labels exist (one for clarity, one for helpfulness)
77 |   expect(screen.getAllByLabelText("1 star").length).toBe(2);
78 |   expect(screen.getAllByLabelText("2 stars").length).toBe(2);
79 | });
80 | });
81 |

```

FILE: [src/__tests__/unit/teamBadgeDefinitions.test.ts](#)

TYPE: .ts | LINES: 1-69 | SHA256: 9a45936b3bfb416732a67b3a5742457165fbf1815dacc4c8655f09031b1a5c7c

```

1 | // Unit test: teamBadgeDefinitions - all 6 badges + Team Player defined
2 | import { describe, it, expect } from "vitest";
3 | import {
4 |   TEAM_BADGE_DEFINITIONS,
5 |   getTeamBadgeById,
6 |   getAllTeamBadgeIds,
7 | } from "@lib/teamBadgeDefinitions";
8 |
9 | describe("teamBadgeDefinitions", () => {
10 |   const EXPECTED_BADGE_IDS = [
11 |     "team_spirit",
12 |     "streak_squad",
13 |     "streak_champions",
14 |     "streak_legends",
15 |     "full_house",
16 |     "quest_conquerors",
17 |     "team_player",
18 |   ];
19 |
20 |   it("defines exactly 7 badges (6 team + Team Player)", () => {
21 |     expect(TEAM_BADGE_DEFINITIONS.length).toBe(7);
22 |   });
23 |
24 |   it("contains all expected badge IDs", () => {
25 |     const ids = getAllTeamBadgeIds();
26 |     for (const expectedId of EXPECTED_BADGE_IDS) {
27 |       expect(ids).toContain(expectedId);
28 |     }
29 |   });
30 |
31 |   it("each badge has required fields", () => {
32 |     for (const badge of TEAM_BADGE_DEFINITIONS) {
33 |       expect(badge.id).toBeTruthy();
34 |       expect(badge.name).toBeTruthy();
35 |       expect(badge.description).toBeTruthy();
36 |       expect(badge.icon).toBeTruthy();
37 |       expect(badge.category).toBe("team");
38 |       expect(typeof badge.isMystery).toBe("boolean");
39 |       expect(badge.condition).toBeTruthy();
40 |       expect(typeof badge.xpReward).toBe("number");
41 |       expect(badge.xpReward).toBeGreaterThan(0);
42 |     }

```

```

43 | });
44 |
45 | it("getTeamBadgeById returns correct badge", () => {
46 |   for (const id of EXPECTED_BADGE_IDS) {
47 |     const badge = getTeamBadgeById(id);
48 |     expect(badge).toBeDefined();
49 |     expect(badge!.id).toBe(id);
50 |   }
51 | });
52 |
53 | it("getTeamBadgeById returns undefined for unknown ID", () => {
54 |   expect(getTeamBadgeById("nonexistent")).toBeUndefined();
55 | });
56 |
57 | it("Team Player badge has correct properties", () => {
58 |   const teamPlayer = getTeamBadgeById("team_player");
59 |   expect(teamPlayer).toBeDefined();
60 |   expect(teamPlayer!.name).toBe("Team Player");
61 |   expect(teamPlayer!.category).toBe("team");
62 | });
63 |
64 | it("all badge IDs are unique", () => {
65 |   const ids = getAllTeamBadgeIds();
66 |   expect(new Set(ids).size).toBe(ids.length);
67 | });
68 | });
69 |

```

FILE: src/___tests___/unit/teamHealthBadge.test.tsx

TYPE: .tsx | LINES: 1-71 | SHA256: 2022f053474f6eaccf9178af7cb5eead67f33ecf9eba1b89c8b5ab70eaa9f488

```

1 | // Unit test: TeamHealthBadge - color coding green "e70, yellow 40-69, red <40
2 | import { describe, it, expect } from "vitest";
3 | import { render, screen } from "@testing-library/react";
4 | import TeamHealthBadge from "@components/shared/TeamHealthBadge";
5 |
6 | describe("TeamHealthBadge", () => {
7 |   it("renders score value", () => {
8 |     render(<TeamHealthBadge score={85} />);
9 |     expect(screen.getByText("85")).toBeDefined();
10 |   });
11 |
12 |   it('score "e70" shows "Healthy" with green styling', () => {
13 |     const { container } = render(<TeamHealthBadge score={75} />);
14 |     const badge = screen.getByTestId("team-health-badge");
15 |     expect(badge.getAttribute("data-status")).toBe("healthy");
16 |     expect(container.querySelector('[class*="green"]')).not.toBeNull();
17 |   });
18 |
19 |   it('score 40-69 shows "Needs Attention" with yellow styling', () => {
20 |     const { container } = render(<TeamHealthBadge score={55} />);
21 |     const badge = screen.getByTestId("team-health-badge");
22 |     expect(badge.getAttribute("data-status")).toBe("needs_attention");
23 |     expect(container.querySelector('[class*="yellow"]')).not.toBeNull();
24 |   });
25 |
26 |   it('score <40 shows "At Risk" with red styling', () => {
27 |     const { container } = render(<TeamHealthBadge score={25} />);
28 |     const badge = screen.getByTestId("team-health-badge");
29 |     expect(badge.getAttribute("data-status")).toBe("at_risk");
30 |     expect(container.querySelector('[class*="red"]')).not.toBeNull();
31 |   });
32 |
33 |   it("boundary: score 70 is healthy", () => {
34 |     render(<TeamHealthBadge score={70} />);
35 |     expect(
36 |       screen.getByTestId("team-health-badge").getAttribute("data-status")
37 |     ).toBe("healthy");
38 |   });
39 |
40 |   it("boundary: score 69 is needs_attention", () => {
41 |     render(<TeamHealthBadge score={69} />);
42 |     expect(
43 |       screen.getByTestId("team-health-badge").getAttribute("data-status")
44 |     ).toBe("needs_attention");
45 |   });
46 |
47 |   it("boundary: score 40 is needs_attention", () => {
48 |     render(<TeamHealthBadge score={40} />);
49 |     expect(
50 |       screen.getByTestId("team-health-badge").getAttribute("data-status")
51 |     ).toBe("needs_attention");
52 |   });
53 |
54 |   it("boundary: score 39 is at_risk", () => {
55 |     render(<TeamHealthBadge score={39} />);
56 |     expect(
57 |       screen.getByTestId("team-health-badge").getAttribute("data-status")
58 |     ).toBe("at_risk");
59 |   });
60 |
61 |   it("has accessible aria-label", () => {
62 |     render(<TeamHealthBadge score={85} />);
63 |     expect(screen.getByLabelText("Health score: 85, Healthy")).toBeDefined();
64 |   });
65 |
66 |   it("clamps score to 0-100 range", () => {
67 |     render(<TeamHealthBadge score={150} />);
68 |     expect(screen.getByText("100")).toBeDefined();
69 |   });
70 | });
71 |

```

FILE: src/__tests__/unit/teamHealthCalculator.test.ts

TYPE: .ts | LINES: 1-143 | SHA256: e6268be74139acd310054d8482da246d8d96b15a8243420df65e7f8364f0c847

```
1  | // Unit test: teamHealthCalculator - Gini coefficient, health score formula, engagement trend
2  | import { describe, it, expect } from "vitest";
3  | import {
4  |   computeGiniCoefficient,
5  |   detectEngagementTrend,
6  |   trendToScore,
7  |   classifyHealthStatus,
8  |   computeTeamHealth,
9  | } from "@lib/teamHealthCalculator";
10 |
11 | describe("teamHealthCalculator", () => {
12 |   describe("computeGiniCoefficient", () => {
13 |     it("returns 0 for equal contributions", () => {
14 |       expect(computeGiniCoefficient([100, 100, 100])).toBe(0);
15 |     });
16 |
17 |     it("returns 0 for single member", () => {
18 |       expect(computeGiniCoefficient([500])).toBe(0);
19 |     });
20 |
21 |     it("returns 0 for all zeros", () => {
22 |       expect(computeGiniCoefficient([0, 0, 0])).toBe(0);
23 |     });
24 |
25 |     it("returns value between 0 and 1", () => {
26 |       const gini = computeGiniCoefficient([100, 50, 10, 0]);
27 |       expect(gini).toBeGreaterThan(0);
28 |       expect(gini).toBeLessThanOrEqual(1);
29 |     });
30 |
31 |     it("higher inequality yields higher Gini", () => {
32 |       const equal = computeGiniCoefficient([100, 100, 100]);
33 |       const unequal = computeGiniCoefficient([300, 0, 0]);
34 |       expect(unequal).toBeGreaterThan(equal);
35 |     });
36 |   });
37 |
38 |   describe("detectEngagementTrend", () => {
39 |     it("rising when >10% increase", () => {
40 |       expect(detectEngagementTrend(120, 100)).toBe("rising");
41 |     });
42 |
43 |     it("declining when >10% decrease", () => {
44 |       expect(detectEngagementTrend(80, 100)).toBe("declining");
45 |     });
46 |
47 |     it("stable within ±10%", () => {
48 |       expect(detectEngagementTrend(105, 100)).toBe("stable");
49 |       expect(detectEngagementTrend(95, 100)).toBe("stable");
50 |     });
51 |
52 |     it("rising when last week was 0 and this week > 0", () => {
53 |       expect(detectEngagementTrend(50, 0)).toBe("rising");
54 |     });
55 |
56 |     it("stable when both weeks are 0", () => {
57 |       expect(detectEngagementTrend(0, 0)).toBe("stable");
58 |     });
59 |   });
60 |
61 |   describe("trendToScore", () => {
62 |     it("rising != 100", () => {
63 |       expect(trendToScore("rising")).toBe(100);
64 |     });
65 |
66 |     it("stable != 60", () => {
67 |       expect(trendToScore("stable")).toBe(60);
68 |     });
69 |
70 |     it("declining != 20", () => {
71 |       expect(trendToScore("declining")).toBe(20);
72 |     });
73 |   });
74 |
75 |   describe("classifyHealthStatus", () => {
76 |     it("'e70' healthy", () => {
77 |       expect(classifyHealthStatus(70)).toBe("healthy");
78 |       expect(classifyHealthStatus(100)).toBe("healthy");
79 |     });
80 |
81 |     it("'40-69' needs_attention", () => {
82 |       expect(classifyHealthStatus(40)).toBe("needs_attention");
83 |       expect(classifyHealthStatus(69)).toBe("needs_attention");
84 |     });
85 |
86 |     it("<40' at_risk", () => {
87 |       expect(classifyHealthStatus(39)).toBe("at_risk");
88 |       expect(classifyHealthStatus(0)).toBe("at_risk");
89 |     });
90 |   });
91 |
92 |   describe("computeTeamHealth", () => {
93 |     it("returns score between 0 and 100", () => {
94 |       const result = computeTeamHealth({
95 |         memberXpContributions: [100, 100, 100],
96 |         thisWeekXp: 300,
97 |         lastWeekXp: 250,
98 |         participationRate: 0.8,
99 |         overlapRate: 0.7,
100 |       });
101 |       expect(result.healthScore).toBeGreaterThanOrEqual(0);
```

```

102     expect(result.healthScore).toBeLessThanOrEqual(100);
103 });
104
105 it("perfect team gets high score", () => {
106     const result = computeTeamHealth({
107         memberXpContributions: [100, 100, 100],
108         thisWeekXp: 400,
109         lastWeekXp: 300,
110         participationRate: 1.0,
111         overlapRate: 1.0,
112     });
113     expect(result.healthScore).toBeGreaterThanOrEqual(80);
114     expect(result.healthStatus).toBe("healthy");
115 });
116
117 it("inactive team gets low score", () => {
118     const result = computeTeamHealth({
119         memberXpContributions: [0, 0, 0],
120         thisWeekXp: 0,
121         lastWeekXp: 100,
122         participationRate: 0,
123         overlapRate: 0,
124     });
125     expect(result.healthScore).toBeLessThan(40);
126     expect(result.healthStatus).toBe("at_risk");
127 });
128
129 it("returns consistent status with score", () => {
130     const result = computeTeamHealth({
131         memberXpContributions: [50, 30, 20],
132         thisWeekXp: 100,
133         lastWeekXp: 100,
134         participationRate: 0.5,
135         overlapRate: 0.5,
136     });
137     expect(result.healthStatus).toBe(
138         classifyHealthStatus(result.healthScore)
139     );
140 });
141 });
142 });
143

```

FILE: src/__tests__/unit/teamHealthReportPage.test.tsx

TYPE: .tsx | LINES: 1-111 | SHA256: 458904ffd9b6b5c0ebefdc09053b346b1127871fe2a7b597a4634ec73a46f7f3

[illegible]

```

59 |
60 | vi.mock("@/design-system/patterns/Shimmer", () => ({
61 |   default: ({ className }: { className?: string }) => (
62 |     <div data-testid="shimmer" className={className} />
63 |   ),
64 | }));
65 |
66 | import TeamHealthReportPage from "@/pages/teacher/teams/TeamHealthReportPage";
67 |
68 | describe("TeamHealthReportPage", () => {
69 |   it("renders page title", () => {
70 |     render(<TeamHealthReportPage />);
71 |     expect(screen.getByText("Team Health Report")).toBeDefined();
72 |   });
73 |
74 |   it("renders total teams count", () => {
75 |     render(<TeamHealthReportPage />);
76 |     expect(screen.getByText("3")).toBeDefined(); // total teams
77 |   });
78 |
79 |   it("renders healthy count", () => {
80 |     render(<TeamHealthReportPage />);
81 |     // Multiple "1" values exist (healthy=1, needs_attention=1, at_risk=1)
82 |     const ones = screen.getAllByText("1");
83 |     expect(ones.length).toBeGreaterThanOrEqual(1);
84 |   });
85 |
86 |   it("renders flagged teams section", () => {
87 |     render(<TeamHealthReportPage />);
88 |     expect(screen.getByText(/Flagged Teams/)).toBeDefined();
89 |   });
90 |
91 |   it("renders flagged team names", () => {
92 |     render(<TeamHealthReportPage />);
93 |     expect(screen.getByText("Beta")).toBeDefined();
94 |     expect(screen.getByText("Gamma")).toBeDefined();
95 |   });
96 |
97 |   it("renders recommendations for flagged teams", () => {
98 |     render(<TeamHealthReportPage />);
99 |     expect(screen.getByText(/Consider restructuring/)).toBeDefined();
100 |    expect(screen.getByText(/Monitor this team/)).toBeDefined();
101 |  });
102 |
103 |   it("renders KPI cards", () => {
104 |     render(<TeamHealthReportPage />);
105 |     expect(screen.getByText("Total Teams")).toBeDefined();
106 |     expect(screen.getByText("Healthy")).toBeDefined();
107 |     expect(screen.getByText("Needs Attention")).toBeDefined();
108 |     expect(screen.getByText("At Risk")).toBeDefined();
109 |   });
110 | });
111 |

```

FILE: [src/__tests__/unit/teamInvitationCard.test.tsx](#)

TYPE: .tsx | LINES: 1-79 | SHA256: 9706f94e91d1c6b3a4129e92ebbc9f1983a85f9c176e2f8ba6ddedc3fd716600

```

1 | // Unit test: TeamInvitationCard – accept/decline buttons, keyboard operability
2 | import { describe, it, expect, vi } from "vitest";
3 | import { render, screen, fireEvent } from "@testing-library/react";
4 | import TeamInvitationCard from "@/components/shared/TeamInvitationCard";
5 |
6 | const defaultProps = {
7 |   teamName: "Alpha Squad",
8 |   invitedBy: "John Doe",
9 |   createdAt: new Date().toISOString(),
10 |   onAccept: vi.fn(),
11 |   onDecline: vi.fn(),
12 | };
13 |
14 | describe("TeamInvitationCard", () => {
15 |   it("renders team name", () => {
16 |     render(<TeamInvitationCard {...defaultProps} />);
17 |     expect(screen.getByText("Alpha Squad")).toBeDefined();
18 |   });
19 |
20 |   it("renders invited by name", () => {
21 |     render(<TeamInvitationCard {...defaultProps} />);
22 |     expect(screen.getByText(/Invited by John Doe/)).toBeDefined();
23 |   });
24 |
25 |   it("renders accept button", () => {
26 |     render(<TeamInvitationCard {...defaultProps} />);
27 |     expect(screen.getByText("Accept")).toBeDefined();
28 |   });
29 |
30 |   it("renders decline button", () => {
31 |     render(<TeamInvitationCard {...defaultProps} />);
32 |     expect(screen.getByText("Decline")).toBeDefined();
33 |   });
34 |
35 |   it("calls onAccept when accept button is clicked", () => {
36 |     const onAccept = vi.fn();
37 |     render(<TeamInvitationCard {...defaultProps} onAccept={onAccept} />);
38 |     fireEvent.click(screen.getByText("Accept"));
39 |     expect(onAccept).toHaveBeenCalledOnce();
40 |   });
41 |
42 |   it("calls onDecline when decline button is clicked", () => {
43 |     const onDecline = vi.fn();
44 |     render(<TeamInvitationCard {...defaultProps} onDecline={onDecline} />);
45 |     fireEvent.click(screen.getByText("Decline"));
46 |     expect(onDecline).toHaveBeenCalledOnce();
47 |   });

```



```

48 |
49 | it("accept button has accessible aria-label", () => {
50 |   render(<TeamInvitationCard {...defaultProps} />);
51 |   const btn = screen.getByLabelText("Accept invitation to Alpha Squad");
52 |   expect(btn).toBeDefined();
53 | });
54 |
55 | it("decline button has accessible aria-label", () => {
56 |   render(<TeamInvitationCard {...defaultProps} />);
57 |   const btn = screen.getByLabelText("Decline invitation to Alpha Squad");
58 |   expect(btn).toBeDefined();
59 | });
60 |
61 | it("buttons are keyboard accessible (not disabled by default)", () => {
62 |   render(<TeamInvitationCard {...defaultProps} />);
63 |   const acceptBtn = screen.getByText("Accept").closest("button");
64 |   const declineBtn = screen.getByText("Decline").closest("button");
65 |   expect(acceptBtn?.disabled).toBe(false);
66 |   expect(declineBtn?.disabled).toBe(false);
67 | });
68 |
69 | it("disables buttons when accepting", () => {
70 |   render(<TeamInvitationCard {...defaultProps} isAccepting />);
71 |   const acceptBtn = screen.getByLabelText("Accept invitation to Alpha Squad");
72 |   const declineBtn = screen.getByLabelText(
73 |     "Decline invitation to Alpha Squad"
74 |   );
75 |   expect(acceptBtn.hasAttribute("disabled")).toBe(true);
76 |   expect(declineBtn.hasAttribute("disabled")).toBe(true);
77 | });
78 | });
79 |

```

FILE: [src/__tests__/unit/teamLeaderboardView.test.tsx](#)

TYPE: .tsx | LINES: 1-136 | SHA256: aee4093ca97d226ed4a9d967f9496b027e4eada7f79fc9e28cc4488dc36ada0c

```

1 | // Unit test: TeamLeaderboardView – renders team rows, medal icons, highlight current team, cooperation score
2 | import { describe, it, expect, vi } from "vitest";
3 | import { render, screen, fireEvent } from "@testing-library/react";
4 | import TeamLeaderboardView from "@components/shared/TeamLeaderboardView";
5 | import type { TeamLeaderboardEntry } from "@hooks/useTeamLeaderboard";
6 |
7 | const mockEntries: TeamLeaderboardEntry[] = [
8 |   {
9 |     team_id: "t1",
10 |    team_name: "Alpha",
11 |    avatar_letter: "A",
12 |    member_count: 4,
13 |    xp_total: 5000,
14 |    xp_this_week: 500,
15 |    streak_count: 7,
16 |    cooperation_score: 90,
17 |    health_score: 85,
18 |    rank: 1,
19 |  },
20 |   {
21 |     team_id: "t2",
22 |     team_name: "Beta",
23 |     avatar_letter: "B",
24 |     member_count: 3,
25 |     xp_total: 4000,
26 |     xp_this_week: 400,
27 |     streak_count: 3,
28 |     cooperation_score: 75,
29 |     health_score: 70,
30 |     rank: 2,
31 |   },
32 |   {
33 |     team_id: "t3",
34 |     team_name: "Gamma",
35 |     avatar_letter: "G",
36 |     member_count: 5,
37 |     xp_total: 3000,
38 |     xp_this_week: 300,
39 |     streak_count: 0,
40 |     cooperation_score: 60,
41 |     health_score: 55,
42 |     rank: 3,
43 |   },
44 |   {
45 |     team_id: "t4",
46 |     team_name: "Delta",
47 |     avatar_letter: "D",
48 |     member_count: 2,
49 |     xp_total: 2000,
50 |     xp_this_week: 200,
51 |     streak_count: 0,
52 |     cooperation_score: 50,
53 |     health_score: 40,
54 |     rank: 4,
55 |   },
56 | ];
57 |
58 | describe("TeamLeaderboardView", () => {
59 |   it("renders all team rows", () => {
60 |     render(
61 |       <TeamLeaderboardView
62 |         entries={mockEntries}
63 |         scope="course"
64 |         sortBy="xp_total"
65 |       />
66 |     );
67 |     expect(screen.getByText("Alpha")).toBeDefined();
68 |     expect(screen.getByText("Beta")).toBeDefined();

```

[illegible]

```

33 |         id: "m2",
34 |         student_id: "user-2",
35 |         role: "member",
36 |         contribution_status: "warning",
37 |     },
38 | ],
39 | badges: [
40 |   { id: "b1", badge_key: "streak_squad", earned_at: "2025-01-08T00:00:00Z" },
41 | ],
42 | activeChallenges: [
43 |   {
44 |     id: "c1",
45 |     title: "Weekly Sprint",
46 |     challenge_type: "academic",
47 |     goal_target: 10,
48 |     current_progress: 5,
49 |     completed_at: null,
50 |   },
51 | ],
52 | teachingMoments: [],
53 | };
54 |
55 | vi.mock("@/hooks/useTeamProfile", () => ({
56 |   useTeamProfile: () => ({ data: mockProfile, isLoading: false }),
57 | }));
58 |
59 | vi.mock("@/hooks/useReplacementVotes", () => ({
60 |   useReplacementVotes: () => ({ data: [] }),
61 |   useInitiateVote: () => ({ mutate: vi.fn(), isPending: false }),
62 |   useCastVote: () => ({ mutate: vi.fn(), isPending: false }),
63 | }));
64 |
65 | vi.mock("@/design-system/patterns/Shimmer", () => ({
66 |   default: ({ className }: { className?: string }) => (
67 |     <div data-testid="shimmer" className={className} />
68 |   ),
69 | }));
70 |
71 | vi.mock("@/components/shared/CooperationScoreDisplay", () => ({
72 |   default: ({ score }: { score: number }) => (
73 |     <div data-testid="cooperation-score">{score}</div>
74 |   ),
75 | }));
76 |
77 | vi.mock("@/components/shared/TeamMemberList", () => ({
78 |   default: ({ members }: { members: unknown[] }) => (
79 |     <div data-testid="team-member-list">{members.length} members</div>
80 |   ),
81 | }));
82 |
83 | vi.mock("@/components/shared/TeamBadgeCollection", () => ({
84 |   default: ({ badges }: { badges: unknown[] }) => (
85 |     <div data-testid="team-badge-collection">{badges.length} badges</div>
86 |   ),
87 | }));
88 |
89 | vi.mock("@/components/shared/ChallengeProgressBar", () => ({
90 |   default: ({ current, goal }: { current: number; goal: number }) => (
91 |     <div
92 |       data-testid="challenge-progress"
93 |       role="progressbar"
94 |       aria-valuenow={current}
95 |       aria-valuemax={goal}
96 |     />
97 |   ),
98 | }));
99 |
100 | vi.mock("@/components/shared/PeerTeachingMomentCard", () => ({
101 |   default: () => <div data-testid="teaching-moment-card" />,
102 | }));
103 |
104 | vi.mock("@/components/shared/ReplacementVoteCard", () => ({
105 |   default: () => <div data-testid="replacement-vote-card" />,
106 | }));
107 |
108 | import TeamProfilePage from "@/pages/student/teams/TeamProfilePage";
109 |
110 | describe("TeamProfilePage", () => {
111 |   it("renders team name", () => {
112 |     render(<TeamProfilePage />);
113 |     expect(screen.getByText("Alpha Squad")).toBeDefined();
114 |   });
115 |
116 |   it("renders team XP", () => {
117 |     render(<TeamProfilePage />);
118 |     expect(screen.getByText("1,500")).toBeDefined();
119 |   });
120 |
121 |   it("renders streak count", () => {
122 |     render(<TeamProfilePage />);
123 |     expect(screen.getByText("7")).toBeDefined();
124 |   });
125 |
126 |   it("renders member count", () => {
127 |     render(<TeamProfilePage />);
128 |     expect(screen.getByText("2")).toBeDefined();
129 |   });
130 |
131 |   it("renders cooperation score", () => {
132 |     render(<TeamProfilePage />);
133 |     expect(screen.getByTestId("cooperation-score")).toBeDefined();
134 |     expect(screen.getByTestId("cooperation-score").textContent).toBe("85");
135 |   });
136 |

```

FILE: src/__tests__/unit/themeProvider.test.ts

83
84

```

85     expect(localStorage.getItem("theme")).toBe("dark");
86     expect(result.current.theme).toBe("dark");
87     expect(result.current.resolvedTheme).toBe("dark");
88   });
89
90   it("setTheme persists to Supabase when profile exists", async () => {
91     vi.useFakeTimers();
92     mockProfile = { id: "user-1", theme_preference: "light" };
93     const { result } = renderHook(() => useTheme(), { wrapper });
94
95     act(() => {
96       result.current.setTheme("dark");
97     });
98
99     // Advance past the 800ms debounce
100    await act(async () => {
101      vi.advanceTimersByTime(900);
102    });
103
104    expect(mockUpdate).toHaveBeenCalledWith({ theme_preference: "dark" });
105    vi.useRealTimers();
106  });
107
108   it("setTheme does not call Supabase when no profile", () => {
109     mockProfile = null;
110     const { result } = renderHook(() => useTheme(), { wrapper });
111
112     act(() => {
113       result.current.setTheme("dark");
114     });
115
116     expect(mockUpdate).not.toHaveBeenCalled();
117   });
118
119   it("syncs theme from profile on first load", () => {
120     mockProfile = { id: "user-1", theme_preference: "dark" };
121     renderHook(() => useTheme(), { wrapper });
122     expect(localStorage.getItem("theme")).toBe("dark");
123   });
124
125   it("ignores invalid stored theme values", () => {
126     localStorage.setItem("theme", "invalid-value");
127     const { result } = renderHook(() => useTheme(), { wrapper });
128     expect(result.current.theme).toBe("system");
129   });
130 });
131

```

FILE: src/__tests__/unit/timetableEmptyState.test.tsx

TYPE: .tsx | LINES: 1-110 | SHA256: ed917a720c00a5f4d4fbc9c62da8a640470bec80ed6dd86c3378668d26c70648

[illegible]

[illegible]

FILE: src/__tests__/unit/timetableNow.test.ts

```
TYPE: .ts | LINES: 1-221 | SHA256: 495caf85d3fef36be36837ae7f1f50e737f32ed100eeb57abac11720cf90ae9b
```

```

1 // Unit tests for current/next-class timetable context (Task 19.3)
2 // Requirements: 12.4, 13.2, 21.3
3
4 import { describe, it, expect } from "vitest";
5 import {
6   computeTimetableNow,
7   formatDurationParts,
8   parseTimeToMinutes,
9 } from "@lib/timetableNow";
10 import type { TimetableSlot } from "@hooks/useTimetable";
11
12 const slot = (overrides: Partial<TimetableSlot>): TimetableSlot => ({
13   id: overrides.id ?? "slot-1",
14   section_id: overrides.section_id ?? "sec-1",
15   day_of_week: overrides.day_of_week ?? 1,
16   start_time: overrides.start_time ?? "09:00",
17   end_time: overrides.end_time ?? "10:00",
18   room: overrides.room ?? null,
19   slot_type: overrides.slot_type ?? "lecture",
20   course_name: overrides.course_name,
21   section_code: overrides.section_code,
22   course_id: overrides.course_id,
23   color: overrides.color,
24 });
25
26 // Reference instants. JS getDay(): 0=Sun ... 6=Sat.
27 // 2025-06-02 is a Monday.
28 const MON_0900 = new Date("2025-06-02T09:00:00");
29 const MON_0930 = new Date("2025-06-02T09:30:00");
30 const MON_1100 = new Date("2025-06-02T11:00:00");
31 const FRI_1800 = new Date("2025-06-06T18:00:00"); // Friday evening
32
33 describe("parseTimeToMinutes", () => {
34   it("parses HH:MM", () => {
35     expect(parseTimeToMinutes("09:30")).toBe(570);
36   });
37
38   it("parses HH:MM:SS", () => {
39     expect(parseTimeToMinutes("09:30:45")).toBe(570);
40   });
41
42   it("returns null for invalid input", () => {
43     expect(parseTimeToMinutes("")).toBeNull();
44   });
45 });

```

```

44 |     expect(parseTimeToMinutes("25:00")).toBeNull();
45 |     expect(parseTimeToMinutes("09:60")).toBeNull();
46 |     expect(parseTimeToMinutes(null)).toBeNull();
47 |     expect(parseTimeToMinutes(undefined)).toBeNull();
48 |     expect(parseTimeToMinutes("abc")).toBeNull();
49 |   });
50 | });
51 |
52 | describe("computeTimetableNow", () => {
53 |   it("reports 'none' when there are no slots (R13.2 empty fallback)", () => {
54 |     expect(computeTimetableNow([], MON_0930)).toEqual({
55 |       status: "none",
56 |       slot: null,
57 |       minutes: null,
58 |     });
59 |   });
60 |
61 |   it("reports 'none' when all slots are unparseable", () => {
62 |     const bad = [
63 |       slot({ start_time: "bad", end_time: "10:00" }),
64 |       slot({ day_of_week: 9 }),
65 |       slot({ start_time: "10:00", end_time: "09:00" }), // non-positive duration
66 |     ];
67 |     expect(computeTimetableNow(bad, MON_0930).status).toBe("none");
68 |   });
69 |
70 |   it("detects an in-progress class and minutes until it ends (R12.4, R21.3)", () => {
71 |     const ctx = computeTimetableNow(
72 |       [slot({ day_of_week: 1, start_time: "09:00", end_time: "10:00" })],
73 |       MON_0930
74 |     );
75 |     expect(ctx.status).toBe("in_class");
76 |     expect(ctx.slot?.id).toBe("slot-1");
77 |     expect(ctx.minutes).toBe(30);
78 |   });
79 |
80 |   it("treats the start boundary as in-class and the end boundary as not in-class", () => {
81 |     const slots = [
82 |       slot({ id: "s", day_of_week: 1, start_time: "09:00", end_time: "10:00" }),
83 |     ];
84 |     // Exactly at start !' in class, full hour remaining.
85 |     expect(
86 |       computeTimetableNow(slots, new Date("2025-06-02T09:00:00"))
87 |     ).toMatchObject({ status: "in_class", minutes: 60 });
88 |     // Exactly at end !' not in class anymore (next occurrence next week).
89 |     expect(
90 |       computeTimetableNow(slots, new Date("2025-06-02T10:00:00")).status
91 |     ).toBe("upcoming");
92 |   });
93 |
94 |   it("prefers the soonest-ending class when slots overlap", () => {
95 |     const ctx = computeTimetableNow(
96 |       [
97 |         slot({
98 |           id: "long",
99 |           day_of_week: 1,
100 |           start_time: "09:00",
101 |           end_time: "12:00",
102 |         }),
103 |         slot({
104 |           id: "short",
105 |           day_of_week: 1,
106 |           start_time: "09:00",
107 |           end_time: "10:00",
108 |         }),
109 |       ],
110 |       MON_0930
111 |     );
112 |     expect(ctx.slot?.id).toBe("short");
113 |     expect(ctx.minutes).toBe(30);
114 |   });
115 |
116 |   it("picks the next upcoming class with minutes until it starts", () => {
117 |     const ctx = computeTimetableNow(
118 |       [
119 |         slot({
120 |           id: "early",
121 |           day_of_week: 1,
122 |           start_time: "08:00",
123 |           end_time: "08:45",
124 |         }),
125 |         slot({
126 |           id: "next",
127 |           day_of_week: 1,
128 |           start_time: "10:00",
129 |           end_time: "11:00",
130 |         }),
131 |       ],
132 |       MON_0900
133 |     );
134 |     expect(ctx.status).toBe("upcoming");
135 |     expect(ctx.slot?.id).toBe("next");
136 |     expect(ctx.minutes).toBe(60);
137 |   });
138 |
139 |   it("chooses the nearest future slot across multiple days", () => {
140 |     const ctx = computeTimetableNow(
141 |       [
142 |         slot({
143 |           id: "tue",
144 |           day_of_week: 2,
145 |           start_time: "09:00",
146 |           end_time: "10:00",
147 |         }),

```


[illegible]

[illegible]

```

235 |     const task = mockTask({ title: "Read Chapter 5" });
236 |
237 |     render(
238 |       <TodayTimeline
239 |         sessions={[]}
240 |         tasks={[task]}
241 |         deadlines={[]}
242 |         habits={defaultHabits}
243 |       />
244 |     );
245 |
246 |     expect(screen.getAllByText("To Do").length).toBeGreaterThan(0);
247 |     expect(screen.getByText("Read Chapter 5")).toBeInTheDocument();
248 |   });
249 |
250 |   it("renders deadline items", () => {
251 |     const deadline = mockDeadline({ title: "Assignment 3" });
252 |
253 |     render(
254 |       <TodayTimeline
255 |         sessions={[]}
256 |         tasks={[]}
257 |         deadlines={[deadline]}
258 |         habits={defaultHabits}
259 |       />
260 |     );
261 |
262 |     expect(screen.getByText("Assignment 3")).toBeInTheDocument();
263 |   });
264 |
265 |   it("renders habit status section", () => {
266 |     render(
267 |       <TodayTimeline
268 |         sessions={[]}
269 |         tasks={[mockTask()]}
270 |         deadlines={[]}
271 |         habits={defaultHabits}
272 |       />
273 |     );
274 |
275 |     expect(screen.getByText("Daily Habits")).toBeInTheDocument();
276 |     expect(screen.getByText("Login")).toBeInTheDocument();
277 |     expect(screen.getByText("Submit")).toBeInTheDocument();
278 |     expect(screen.getByText("Journal")).toBeInTheDocument();
279 |     expect(screen.getByText("Read")).toBeInTheDocument();
280 |   });
281 |
282 |   it("shows completed habit with checkmark", () => {
283 |     render(
284 |       <TodayTimeline
285 |         sessions={[]}
286 |         tasks={[mockTask()]}
287 |         deadlines={[]}
288 |         habits={{ login: true, submit: false, journal: false, read: true }}
289 |       />
290 |     );
291 |
292 |     // Login and Read should show checkmarks
293 |     const checkmarks = screen.getAllByText(" ");
294 |     expect(checkmarks).toHaveLength(2);
295 |   });
296 |
297 |   it("calls onSessionStart when Start button is clicked", async () => {
298 |     const user = userEvent.setup();
299 |     const onStart = vi.fn();
300 |     const session = mockSession({ status: "planned", title: "Study Session" });
301 |
302 |     render(
303 |       <TodayTimeline
304 |         sessions={[session]}
305 |         tasks={[]}
306 |         deadlines={[]}
307 |         habits={defaultHabits}
308 |         onSessionStart={onStart}
309 |       />
310 |     );
311 |
312 |     const startButton = screen.getByRole("button", { name: /start/i });
313 |     await user.click(startButton);
314 |
315 |     expect(onStart).toHaveBeenCalledWith(session);
316 |   });
317 |
318 |   it("calls onTaskToggle when task checkbox is clicked", async () => {
319 |     const user = userEvent.setup();
320 |     const onToggle = vi.fn();
321 |     const task = mockTask({ title: "My Task" });
322 |
323 |     render(
324 |       <TodayTimeline
325 |         sessions={[]}
326 |         tasks={[task]}
327 |         deadlines={[]}
328 |         habits={defaultHabits}
329 |         onTaskToggle={onToggle}
330 |       />
331 |     );
332 |
333 |     const checkbox = screen.getByRole("checkbox");
334 |     await user.click(checkbox);
335 |
336 |     expect(onToggle).toHaveBeenCalledWith(task);
337 |   });
338 |

```

[illegible]

[illegible]

FILE: src/__tests__/unit/transactionHistoryPage.test.tsx

TYPE: .tsx | LINES: 1-171 | SHA256: f671d249a562395eda7b56a0f9f81681bf5a734b97f1413615c23445aea49122

[illegible]

[illegible]

```

130 expect(screen.getByText("Assignment Submission")).toBeInTheDocument();
131 expect(screen.getByText("Streak Freeze")).toBeInTheDocument();
132 });
133
134 it("shows an error panel and refuses to display transactions on query failure (R33.1a)", () => {
135   transactionHookState.current = errorResult;
136   renderPage();
137
138   // An error panel is rendered (role=alert with the localized title/body).
139   const alert = screen.getByRole("alert");
140   expect(alert).toBeInTheDocument();
141   expect(
142     screen.getByText("Couldn't load your transactions")
143   ).toBeInTheDocument();
144
145   // No transactions are shown – not even a truncated/partial list.
146   expect(screen.queryByText("Assignment Submission")).not.toBeInTheDocument();
147   expect(screen.queryByText("Streak Freeze")).not.toBeInTheDocument();
148
149   // The empty-state copy is also not shown – failure is distinct from "no data".
150   expect(
151     screen.queryByText("No transactions found.")
152   ).not.toBeInTheDocument();
153 });
154
155 it("surfaces the failure via a toast notification (R33.1a)", () => {
156   transactionHookState.current = errorResult;
157   renderPage();
158
159   expect(toastErrorMock).toHaveBeenCalledWith(
160     "Failed to load transaction history"
161   );
162 });
163
164 it("does not toast on the success path", () => {
165   transactionHookState.current = successResult;
166   renderPage();
167
168   expect(toastErrorMock).not.toHaveBeenCalled();
169 });
170 });
171

```

FILE: src/__tests__/unit/tutorAnalytics.test.ts

TYPE: .ts | LINES: 1-662 | SHA256: 9acf2345e0adb41d6af773496a51d1cb5c7d278fb5c472ef535bb282e5e4367e

[illegible]

59 | "are",
60 | "be",
61 | "been",
62 | "being",
63 | "have",
64 | "has",
65 | "had",
66 | "do",
67 | "does",
68 | "did",
69 | "will",
70 | "would",
71 | "could",
72 | "should",
73 | "may",
74 | "might",
75 | "shall",
76 | "can",
77 | "not",
78 | "no",
79 | "nor",
80 | "so",
81 | "if",
82 | "then",
83 | "than",
84 | "too",
85 | "very",
86 | "just",
87 | "about",
88 | "above",
89 | "after",
90 | "again",
91 | "all",
92 | "also",
93 | "am",
94 | "any",
95 | "as",
96 | "because",
97 | "before",
98 | "below",
99 | "between",
100 | "both",
101 | "during",
102 | "each",
103 | "few",
104 | "further",
105 | "get",
106 | "got",
107 | "he",
108 | "her",
109 | "here",
110 | "hers",
111 | "herself",
112 | "him",
113 | "himself",
114 | "his",
115 | "how",
116 | "i",
117 | "im",
118 | "into",
119 | "me",
120 | "more",
121 | "most",
122 | "my",
123 | "myself",
124 | "now",
125 | "only",
126 | "other",
127 | "our",
128 | "ours",
129 | "ourselves",
130 | "out",
131 | "over",
132 | "own",
133 | "same",
134 | "she",
135 | "some",
136 | "such",
137 | "them",
138 | "themselves",
139 | "there",
140 | "these",
141 | "they",
142 | "those",
143 | "through",
144 | "under",
145 | "until",
146 | "up",
147 | "us",
148 | "we",
149 | "what",
150 | "when",
151 | "where",
152 | "which",
153 | "while",
154 | "who",
155 | "whom",
156 | "why",
157 | "you",
158 | "your",
159 | "yours",
160 | "yourself",
161 | "yourselves",
162 | "hi",


```

163 | "hello",
164 | "hey",
165 | "thanks",
166 | "thank",
167 | "please",
168 | "okay",
169 | "ok",
170 | "yes",
171 | "yeah",
172 | "no",
173 | "sure",
174 | "like",
175 | "know",
176 | "think",
177 | "want",
178 | "need",
179 | "help",
180 | "question",
181 | "understand",
182 | "explain",
183 | "dont",
184 | "doesnt",
185 | "didnt",
186 | "cant",
187 | "wont",
188 | "isnt",
189 | "arent",
190 | "wasnt",
191 | "werent",
192 | "havent",
193 | "hasnt",
194 | "hadnt",
195 | });
196 |
197 | // % % % Pure Functions (replicated from edge function) % % % % % % % % % % % % % % % % % %
198 |
199 | function extractCommonTopics(
200 |   messages: Array<{ content: string }>,
201 |   topN: number = 15
202 | ): CommonTopic[] {
203 |   const wordFrequency = new Map<string, number>();
204 |
205 |   for (const msg of messages) {
206 |     const words = msg.content
207 |       .toLowerCase()
208 |       .replace(/[\^a-z0-9\s]/g, " ")
209 |       .split(/\s+/)
210 |       .filter((w) => w.length >= 3 && !STOP_WORDS.has(w));
211 |
212 |     const uniqueWords = new Set(words);
213 |     for (const word of uniqueWords) {
214 |       wordFrequency.set(word, (wordFrequency.get(word) ?? 0) + 1);
215 |     }
216 |   }
217 |
218 |   return Array.from(wordFrequency.entries())
219 |     .sort((a, b) => b[1] - a[1])
220 |     .slice(0, topN)
221 |     .map(([topic, frequency]) => ({ topic, frequency }));
222 | }
223 |
224 | function isValidUUID(value: string): boolean {
225 |   return /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i.test(
226 |     value
227 |   );
228 | }
229 |
230 | function isValidDate(value: string): boolean {
231 |   const d = new Date(value);
232 |   return !isNaN(d.getTime());
233 | }
234 |
235 | function validateNoPII(response: AnalyticsResponse): boolean {
236 |   const json = JSON.stringify(response);
237 |   const emailPattern = /[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/;
238 |   if (emailPattern.test(json)) {
239 |     return false;
240 |   }
241 |   return true;
242 | }
243 |
244 | /**
245 |  * Computes aggregate metrics from conversation data.
246 |  */
247 | function computeAggregateMetrics(
248 |   conversations: Array<{ message_count: number }>
249 | ) {
250 |   const totalConversations = conversations.length;
251 |   const totalMessages = conversations.reduce(
252 |     (sum, c) => sum + (c.message_count ?? 0),
253 |     0
254 |   );
255 |   const avgMessagesPerConversation =
256 |     totalConversations > 0
257 |       ? Math.round((totalMessages / totalConversations) * 100) / 100
258 |       : 0;
259 |   return { totalConversations, totalMessages, avgMessagesPerConversation };
260 | }
261 |
262 | /**
263 |  * Computes average satisfaction rating from rated messages.
264 |  */
265 | function computeSatisfactionRating(
266 |   ratedMessages: Array<{ satisfaction_rating: string }>

```

[illegible]

[illegible]

[illegible]

[illegible]

FILE: src/__tests__/unit/tutorAttachmentUpload.test.ts

```
TYPE: .ts | LINES: 1-159 | SHA256: e2e5fc74daf1e9dc5b60dd3f50405e0de8d69992e4db5087d1dbe09573332d85
```

[illegible]

[illegible]

```

121 |     await expect(
122 |       uploadTutorAttachmentFile({ file, userId: "user-1" })
123 |     ).rejects.toThrow(FileValidationError);
124 |     expect(mockUpload).not.toHaveBeenCalled();
125 |   });
126 |
127 |   it("throws when the storage upload fails", async () => {
128 |     mockUpload.mockResolvedValue({ error: { message: "Bucket not found" } });
129 |
130 |     const file = createFile("photo.png", "image/png", 500_000);
131 |     await expect(
132 |       uploadTutorAttachmentFile({ file, userId: "user-1" })
133 |     ).rejects.toThrow("Attachment upload failed: Bucket not found");
134 |     expect(mockCreateSignedUrl).not.toHaveBeenCalled();
135 |   });
136 |
137 |   it("throws when a signed URL cannot be produced", async () => {
138 |     mockUpload.mockResolvedValue({ error: null });
139 |     mockCreateSignedUrl.mockResolvedValue({
140 |       data: null,
141 |       error: { message: "object not found" },
142 |     });
143 |
144 |     const file = createFile("notes.pdf", "application/pdf", 500_000);
145 |     await expect(
146 |       uploadTutorAttachmentFile({ file, userId: "user-1" })
147 |     ).rejects.toThrow(/Failed to sign attachment URL/);
148 |   });
149 |
150 |   it("rejects filenames containing path-traversal sequences", async () => {
151 |     mockUpload.mockResolvedValue({ error: null });
152 |
153 |     const file = createFile("../escape.png", "image/png", 500_000);
154 |     await expect(
155 |       uploadTutorAttachmentFile({ file, userId: "user-1" })
156 |     ).rejects.toThrow(FileValidationError);
157 |   });
158 | });
159 |

```

FILE: [src/__tests__/unit/tutorAutonomy.test.ts](#)

TYPE: .ts | LINES: 1-133 | SHA256: 0aa785a099c174ac120cf16699c82f320d6b70ce78e39db56a0270f1091fe8d1

```

1 | import { describe, it, expect } from "vitest";
2 | import { resolveAutonomyLevel, type AutonomyLevel } from "@lib/tutorAutonomy";
3 |
4 | describe("resolveAutonomyLevel", () => {
5 |   // % % % Base level resolution (no student override) % % % % % % % % % % % % % % % % % %
6 |
7 |   it("uses assignment autonomy when set", () => {
8 |     const result = resolveAutonomyLevel({
9 |       assignmentAutonomy: "L1",
10 |      cloAutonomy: "L3",
11 |      studentOverride: null,
12 |    });
13 |     expect(result).toBe("L1");
14 |   });
15 |
16 |   it("falls back to CLO autonomy when assignment is null", () => {
17 |     const result = resolveAutonomyLevel({
18 |       assignmentAutonomy: null,
19 |       cloAutonomy: "L3",
20 |       studentOverride: null,
21 |     });
22 |     expect(result).toBe("L3");
23 |   });
24 |
25 |   it("defaults to L2 when both assignment and CLO are null", () => {
26 |     const result = resolveAutonomyLevel({
27 |       assignmentAutonomy: null,
28 |       cloAutonomy: null,
29 |       studentOverride: null,
30 |     });
31 |     expect(result).toBe("L2");
32 |   });
33 |
34 |   it("prefers assignment over CLO when both are set", () => {
35 |     const result = resolveAutonomyLevel({
36 |       assignmentAutonomy: "L3",
37 |       cloAutonomy: "L1",
38 |       studentOverride: null,
39 |     });
40 |     expect(result).toBe("L3");
41 |   });
42 |
43 |   // % % % Student override L1 ("Figure it out") % % % % % % % % % % % % % % % % % %
44 |
45 |   it("student L1 override always returns L1 regardless of base", () => {
46 |     const levels: AutonomyLevel[] = ["L1", "L2", "L3"];
47 |     for (const base of levels) {
48 |       const result = resolveAutonomyLevel({
49 |         assignmentAutonomy: base,
50 |         cloAutonomy: null,
51 |         studentOverride: "L1",
52 |       });
53 |       expect(result).toBe("L1");
54 |     }
55 |   });
56 |
57 |   it("student L1 override returns L1 even with no teacher config", () => {
58 |     const result = resolveAutonomyLevel({
59 |       assignmentAutonomy: null,
60 |       cloAutonomy: null,
61 |       studentOverride: "L1",

```

[illegible]

FILE: src/_tests_/unit/tutorAutonomyResolution.test.ts

TYPE: .ts | LINES: 1-144 | SHA256: 6630d247a0b103f08d36f7498c43bf3ba9770de467c3b3fe05d7ae050f232747

[illegible]

[illegible]

```

133 |         const result = resolveAutonomyLevel({
134 |             assignmentAutonomy: assignment,
135 |             cloAutonomy: clo,
136 |             studentOverride: override,
137 |         });
138 |         expect(["L1", "L2", "L3"]).toContain(result);
139 |     }
140 | }
141 | }
142 | });
143 | });
144 |

```

FILE: [src/___tests___/unit/tutorCitationParsing.test.ts](#)

TYPE: .ts | LINES: 1-83 | SHA256: 0715211d692f2328a415d857014c25db831d3d4a562b0b628a42831bfd9765fd

```

1 | // @vitest-environment happy-dom
2 | import { describe, it, expect } from "vitest";
3 | import { extractCitationMarkers } from "@lib/tutorCitationParser";
4 |
5 | describe("extractCitationMarkers", () => {
6 |     it("returns empty array for text with no citations", () => {
7 |         expect(
8 |             extractCitationMarkers("This is a plain response with no references.")
9 |         ).toEqual([]);
10 |     });
11 |
12 |     it("extracts a single citation marker", () => {
13 |         expect(
14 |             extractCitationMarkers("Arrays are stored contiguously [1].")
15 |         ).toEqual([1]);
16 |     });
17 |
18 |     it("extracts multiple citation markers", () => {
19 |         const text =
20 |             "As explained in [1], linked lists [2] differ from arrays [3].";
21 |         expect(extractCitationMarkers(text)).toEqual([1, 2, 3]);
22 |     });
23 |
24 |     it("deduplicates repeated citation numbers", () => {
25 |         const text = "See [1] for details. Also referenced in [1] and [2].";
26 |         expect(extractCitationMarkers(text)).toEqual([1, 2]);
27 |     });
28 |
29 |     it("returns sorted citation numbers", () => {
30 |         const text = "Check [3] first, then [1], and finally [2].";
31 |         expect(extractCitationMarkers(text)).toEqual([1, 2, 3]);
32 |     });
33 |
34 |     it("handles citations embedded in markdown bold", () => {
35 |         const text =
36 |             "***Important concept** [1] is explained in the **lecture notes** [2].";
37 |         expect(extractCitationMarkers(text)).toEqual([1, 2]);
38 |     });
39 |
40 |     it("handles citations embedded in markdown lists", () => {
41 |         const text = [
42 |             "- Point one [1]",
43 |             "- Point two [2]",
44 |             "- Point three [3]",
45 |         ].join("\n");
46 |         expect(extractCitationMarkers(text)).toEqual([1, 2, 3]);
47 |     });
48 |
49 |     it("handles citations in code blocks", () => {
50 |         const text = "`\nconst x = arr[1];\n`\nSee [2] for more.";
51 |         // [1] inside code block is still matched as a citation marker
52 |         expect(extractCitationMarkers(text)).toEqual([1, 2]);
53 |     });
54 |
55 |     it("handles citations at the start and end of text", () => {
56 |         const text = "[1] This starts with a citation and ends with one [2]";
57 |         expect(extractCitationMarkers(text)).toEqual([1, 2]);
58 |     });
59 |
60 |     it("handles double-digit citation numbers", () => {
61 |         const text = "References [10] and [11] are from the appendix.";
62 |         expect(extractCitationMarkers(text)).toEqual([10, 11]);
63 |     });
64 |
65 |     it("returns empty array for empty string", () => {
66 |         expect(extractCitationMarkers("")).toEqual([]);
67 |     });
68 |
69 |     it("returns empty array for whitespace-only string", () => {
70 |         expect(extractCitationMarkers(" \n\t ")).toEqual([]);
71 |     });
72 |
73 |     it("ignores [0] as a citation marker", () => {
74 |         const text = "Index [0] is not a valid citation, but [1] is.";
75 |         expect(extractCitationMarkers(text)).toEqual([1]);
76 |     });
77 |
78 |     it("handles adjacent citations without spaces", () => {
79 |         const text = "Multiple sources[1][2][3] confirm this.";
80 |         expect(extractCitationMarkers(text)).toEqual([1, 2, 3]);
81 |     });
82 | });
83 |

```

FILE: src/__tests__/unit/tutorHandoffTrigger.test.ts

```
TYPE: .ts | LINES: 1-217 | SHA256: 034ecafa363ad4de15bc0e607b9056d327fa716e5ac804ebf37cd26880379abe
```

[illegible]

[illegible]

```
TYPE: .ts | LINES: 1-82 | SHA256: e6a26272ed3a1c6665929cb54014ed5092a89c19ea50dd6253a7f14d4d4512e2
```

[illegible]

TYPE: .tsx | LINES: 1-152 | SHA256: 90b7966d2b2b76c74aaa88a59dbd94e1947513076da00540a1ed54d024e7ca5d

[illegible]

[illegible]

FILE: src/__tests__/unit/tutorPersonaAutoSelect.test.ts

TYPE: .ts | LINES: 1-166 | SHA256: 41da20d8b2252a2c6b9652f9e7fd1183f2ed74606705c5af577e1fe845b3d49e

[illegible]

```

50 |
51 | it("defaults to quick_explainer when neither openness nor conscientiousness is "e70", () => {
52 |   const profile: BigFiveTraits = {
53 |     openness: 60,
54 |     conscientiousness: 65,
55 |     extraversion: 90,
56 |     agreeableness: 80,
57 |     neuroticism: 50,
58 |   };
59 |   const result = autoSelectPersona(profile);
60 |   expect(result).not.toBeNull();
61 |   expect(result!.persona).toBe("quick_explainer");
62 | });
63 |
64 | // % % % Multiple high traits – highest percentile wins % % % % % % % % % % % % % % % % % %
65 |
66 | it("selects socratic_guide when openness > conscientiousness and both "e70", () => {
67 |   const profile: BigFiveTraits = {
68 |     openness: 90,
69 |     conscientiousness: 75,
70 |     extraversion: 50,
71 |     agreeableness: 50,
72 |     neuroticism: 50,
73 |   };
74 |   const result = autoSelectPersona(profile);
75 |   expect(result).not.toBeNull();
76 |   expect(result!.persona).toBe("socratic_guide");
77 | });
78 |
79 | it("selects step_by_step_coach when conscientiousness > openness and both "e70", () => {
80 |   const profile: BigFiveTraits = {
81 |     openness: 72,
82 |     conscientiousness: 88,
83 |     extraversion: 50,
84 |     agreeableness: 50,
85 |     neuroticism: 50,
86 |   };
87 |   const result = autoSelectPersona(profile);
88 |   expect(result).not.toBeNull();
89 |   expect(result!.persona).toBe("step_by_step_coach");
90 | });
91 |
92 | // % % % High Neuroticism tone modifier % % % % % % % % % % % % % % % % % % % % % % % % % %
93 |
94 | it("adds tone modifier when neuroticism "e70", () => {
95 |   const profile: BigFiveTraits = {
96 |     openness: 85,
97 |     conscientiousness: 50,
98 |     extraversion: 50,
99 |     agreeableness: 50,
100 |     neuroticism: 75,
101 |   };
102 |   const result = autoSelectPersona(profile);
103 |   expect(result).not.toBeNull();
104 |   expect(result!.toneModifier).toBeDefined();
105 |   expect(result!.toneModifier).toContain("warm");
106 |   expect(result!.toneModifier).toContain("encouraging");
107 | });
108 |
109 | it("does not add tone modifier when neuroticism < 70", () => {
110 |   const profile: BigFiveTraits = {
111 |     openness: 85,
112 |     conscientiousness: 50,
113 |     extraversion: 50,
114 |     agreeableness: 50,
115 |     neuroticism: 60,
116 |   };
117 |   const result = autoSelectPersona(profile);
118 |   expect(result).not.toBeNull();
119 |   expect(result!.toneModifier).toBeUndefined();
120 | });
121 |
122 | // % % % Boundary values % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
123 |
124 | it("treats exactly 70 as high ("e70 threshold)", () => {
125 |   const profile: BigFiveTraits = {
126 |     openness: 70,
127 |     conscientiousness: 69,
128 |     extraversion: 50,
129 |     agreeableness: 50,
130 |     neuroticism: 50,
131 |   };
132 |   const result = autoSelectPersona(profile);
133 |   expect(result).not.toBeNull();
134 |   expect(result!.persona).toBe("socratic_guide");
135 | });
136 |
137 | it("treats 69 as not high (below threshold)", () => {
138 |   const profile: BigFiveTraits = {
139 |     openness: 69,
140 |     conscientiousness: 69,
141 |     extraversion: 50,
142 |     agreeableness: 50,
143 |     neuroticism: 50,
144 |   };
145 |   const result = autoSelectPersona(profile);
146 |   expect(result).not.toBeNull();
147 |   expect(result!.persona).toBe("quick_explainer");
148 | });
149 |
150 | // % % % All zeros % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
151 |
152 | it("returns quick_explainer for all-zero profile", () => {
153 |   const profile: BigFiveTraits = {

```



```

154 |         openness: 0,
155 |         conscientiousness: 0,
156 |         extraversion: 0,
157 |         agreeableness: 0,
158 |         neuroticism: 0,
159 |     };
160 |     const result = autoSelectPersona(profile);
161 |     expect(result).not.toBeNull();
162 |     expect(result!.persona).toBe("quick_explainer");
163 |     expect(result!.toneModifier).toBeUndefined();
164 | });
165 | });
166 |

```

TYPE: .ts | LINES: 1-167 | SHA256: 768b18f4903cbbdf17511d8cc30cc7666b05b09b529854fcb2996e4b1ad473ba

[illegible]

FILE: src/__tests__/unit/tutorPromptAssembly.test.ts

TYPE: .ts | LINES: 1-239 | SHA256: fa1011149d39c17bbdc70a26a354d37e9e84c08ddf1cff7eaf34236a1f386b26

```

1 // @vitest-environment happy-dom
2 import { describe, it, expect } from "vitest";
3 import {
4   assembleSystemPrompt,
5   PERSONA_PROMPTS,
6   AUTONOMY_PROMPTS,
7   SAFETY_INSTRUCTIONS,
8 } from "@lib/tutorPrompt";
9 import type {
10   CLOAttainment,
11   RetrievedChunk,
12   PromptAssemblyInput,
13 } from "@lib/tutorPrompt";
14
15 // %% Helpers %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %%
16
17 const makeCLO = (overrides: Partial<CLOAttainment> = {}): CLOAttainment => ({
18   clo_id: "clo-1",
19   clo_title: "Understand data structures",
20   bloom_level: "Understanding",

```

[illegible]

```
    attainment_percentage: 45,  
  }},  
};  
  
const prompt = assembleSystemPrompt({ ...baseInput, cloAttainments });  
expect(prompt).toContain("STUDENT CLO PROGRESS");  
expect(prompt).toContain("COMPETENCY GAPS");  
expect(prompt).toContain("Passing CLO");  
expect(prompt).toContain("Failing CLO");  
// Passing CLO should have ', failing should have &  
expect(prompt).toContain("80% ' " ");  
expect(prompt).toContain("45% & NEEDS IMPROVEMENT");  
// Only failing CLO in gaps section  
expect(prompt).toContain("Failing CLO: 45% – focus tutoring here");  
});  
});  
  
// %% Persona text inclusion % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
  
describe("assembleSystemPrompt – persona text", () => {  
  it("includes socratic_guide persona text", () => {  
    const prompt = assembleSystemPrompt({  
      ...baseInput,  
      persona: "socratic_guide",  
    });  
    expect(prompt).toContain(PERSONA_PROMPTS.socratic_guide);  
  });  
  
  it("includes step_by_step_coach persona text", () => {  
    const prompt = assembleSystemPrompt({  
      ...baseInput,  
      persona: "step_by_step_coach",  
    });  
    expect(prompt).toContain(PERSONA_PROMPTS.step_by_step_coach);  
  });  
  
  it("includes quick_explainer persona text", () => {  
    const prompt = assembleSystemPrompt({  
      ...baseInput,  
      persona: "quick_explainer",  
    });  
    expect(prompt).toContain(PERSONA_PROMPTS.quick_explainer);  
  });  
});  
  
// %% Autonomy level instructions % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
  
describe("assembleSystemPrompt – autonomy level instructions", () => {  
  it("includes L1 autonomy instructions", () => {  
    const prompt = assembleSystemPrompt({ ...baseInput, autonomyLevel: "L1" });  
    expect(prompt).toContain(AUTONOMY_PROMPTS.L1);  
  });  
  
  it("includes L2 autonomy instructions", () => {  
    const prompt = assembleSystemPrompt({ ...baseInput, autonomyLevel: "L2" });  
    expect(prompt).toContain(AUTONOMY_PROMPTS.L2);  
  });  
  
  it("includes L3 autonomy instructions", () => {  
    const prompt = assembleSystemPrompt({ ...baseInput, autonomyLevel: "L3" });  
    expect(prompt).toContain(AUTONOMY_PROMPTS.L3);  
  });  
});  
  
// %% Safety instructions % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
  
describe("assembleSystemPrompt – safety instructions", () => {  
  it("always includes safety instructions", () => {  
    const prompt = assembleSystemPrompt(baseInput);  
    expect(prompt).toContain(SAFETY_INSTRUCTIONS);  
  });  
});  
  
// %% Course name % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
  
describe("assembleSystemPrompt – course name", () => {  
  it("includes course name when provided", () => {  
    const prompt = assembleSystemPrompt({  
      ...baseInput,  
      courseName: "Data Structures 101",  
    });  
    expect(prompt).toContain("Data Structures 101");  
  });  
  
  it("uses generic intro when course name is not provided", () => {  
    const prompt = assembleSystemPrompt(baseInput);  
    expect(prompt).toContain("helping a student with their coursework");  
  });  
});  
  
// %% Retrieved chunks in prompt % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
  
describe("assembleSystemPrompt – retrieved chunks", () => {  
  it("includes chunk text and source info", () => {  
    const chunks: RetrievedChunk[] = [  
      makeChunk({  
        chunk_text: "Linked lists use pointers.",  
        source_filename: "ch2.pdf",  
      }),  
      makeChunk({  
        chunk_text: "Trees are hierarchical.",  
        source_filename: "ch3.pdf",  
      })  
    ];  
    const prompt = assembleSystemPrompt({
```

```

229 |         ...baseInput,
230 |         retrievedChunks: chunks,
231 |     });
232 |     expect(prompt).toContain("RELEVANT COURSE MATERIALS");
233 |     expect(prompt).toContain("[1] Source: ch2.pdf");
234 |     expect(prompt).toContain("Linked lists use pointers.");
235 |     expect(prompt).toContain("[2] Source: ch3.pdf");
236 |     expect(prompt).toContain("Trees are hierarchical.");
237 | });
238 | });
239 |

```

TYPE: .ts | LINES: 1-49 | SHA256: 3f5a01cd238f2251eebfff77729c64a241a4b01a027a201d8c07621a0e1832375

FILE: src/__tests__/unit/tutorSchemas.test.ts

```

37 | });
38 |
39 | it("rejects message exceeding 2000 characters", () => {
40 |   const result = sendMessageSchema.safeParse({ message: "a".repeat(2001) });
41 |   expect(result.success).toBe(false);
42 | });
43 |
44 | it("accepts message at exactly 2000 characters", () => {
45 |   const result = sendMessageSchema.safeParse({ message: "a".repeat(2000) });
46 |   expect(result.success).toBe(true);
47 | });
48 |
49 | it("accepts message at exactly 1 character", () => {
50 |   const result = sendMessageSchema.safeParse({ message: "x" });
51 |   expect(result.success).toBe(true);
52 | });
53 |
54 | it("rejects missing message field", () => {
55 |   const result = sendMessageSchema.safeParse({});
56 |   expect(result.success).toBe(false);
57 | });
58 |
59 | it("rejects invalid conversation_id (not UUID)", () => {
60 |   const result = sendMessageSchema.safeParse({
61 |     message: "hi",
62 |     conversation_id: "not-a-uuid",
63 |   });
64 |   expect(result.success).toBe(false);
65 | });
66 |
67 | it("rejects invalid course_id (not UUID)", () => {
68 |   const result = sendMessageSchema.safeParse({
69 |     message: "hi",
70 |     course_id: "12345",
71 |   });
72 |   expect(result.success).toBe(false);
73 | });
74 |
75 | it("rejects invalid persona value", () => {
76 |   const result = sendMessageSchema.safeParse({
77 |     message: "hi",
78 |     persona: "invalid_persona",
79 |   });
80 |   expect(result.success).toBe(false);
81 | });
82 |
83 | it("accepts all valid persona values", () => {
84 |   for (const persona of [
85 |     "socratic_guide",
86 |     "step_by_step_coach",
87 |     "quick_explainer",
88 |   ]) {
89 |     const result = sendMessageSchema.safeParse({ message: "hi", persona });
90 |     expect(result.success).toBe(true);
91 |   }
92 | });
93 |
94 | it("rejects more than 2 image URLs", () => {
95 |   const result = sendMessageSchema.safeParse({
96 |     message: "hi",
97 |     image_urls: [
98 |       "https://example.com/1.png",
99 |       "https://example.com/2.png",
100 |       "https://example.com/3.png",
101 |     ],
102 |   });
103 |   expect(result.success).toBe(false);
104 | });
105 |
106 | it("rejects invalid image URL format", () => {
107 |   const result = sendMessageSchema.safeParse({
108 |     message: "hi",
109 |     image_urls: ["not-a-url"],
110 |   });
111 |   expect(result.success).toBe(false);
112 | });
113 |
114 | it("rejects invalid document_url format", () => {
115 |   const result = sendMessageSchema.safeParse({
116 |     message: "hi",
117 |     document_url: "not-a-url",
118 |   });
119 |   expect(result.success).toBe(false);
120 | });
121 |
122 | it("rejects invalid clo_scope entries (not UUIDs)", () => {
123 |   const result = sendMessageSchema.safeParse({
124 |     message: "hi",
125 |     clo_scope: ["not-uuid"],
126 |   });
127 |   expect(result.success).toBe(false);
128 | });
129 |
130 | it("rejects invalid autonomy_override value", () => {
131 |   const result = sendMessageSchema.safeParse({
132 |     message: "hi",
133 |     autonomy_override: "L2",
134 |   });
135 |   expect(result.success).toBe(false);
136 | });
137 |
138 | it("accepts valid autonomy_override values (L1, L3)", () => {
139 |   for (const level of ["L1", "L3"]) {
140 |     const result = sendMessageSchema.safeParse({

```

[illegible]

```

245 |     const result = tutorAnalyticsRequestSchema.safeParse({
246 |       course_id: "550e8400-e29b-41d4-a716-446655440000",
247 |       start_date: "2024-01-01",
248 |     });
249 |     expect(result.success).toBe(false);
250 |   });
251 |
252 |   it("rejects invalid end_date format", () => {
253 |     const result = tutorAnalyticsRequestSchema.safeParse({
254 |       course_id: "550e8400-e29b-41d4-a716-446655440000",
255 |       end_date: "not-a-date",
256 |     });
257 |     expect(result.success).toBe(false);
258 |   });
259 |
260 |   it("accepts course_id with only start_date", () => {
261 |     const result = tutorAnalyticsRequestSchema.safeParse({
262 |       course_id: "550e8400-e29b-41d4-a716-446655440000",
263 |       start_date: "2024-06-15T12:00:00Z",
264 |     });
265 |     expect(result.success).toBe(true);
266 |   });
267 | });
268 |

```

FILE: [src/__tests__/unit/tutorSseParser.test.ts](#)

TYPE: .ts | LINES: 1-117 | SHA256: 4fa55f11a38cf91f75f3da2adb0f5f5177822e232d9eadcf57be7472521c2970

```

1 | // @vitest-environment happy-dom
2 | import { describe, it, expect } from "vitest";
3 | import { parseSSELine } from "@lib/tutorApi";
4 |
5 | describe("parseSSELine", () => {
6 |   it("parses a token event", () => {
7 |     const line = `data: {"type":"token","data":"Hello"}`;
8 |     const result = parseSSELine(line);
9 |     expect(result).toEqual({ type: "token", data: "Hello" });
10 |   });
11 |
12 |   it("parses a citations event", () => {
13 |     const citations = [
14 |       {
15 |         chunk_id: "chunk-1",
16 |         chunk_text: "Arrays are contiguous.",
17 |         source_filename: "lecture.pdf",
18 |         material_type: "lecture_notes",
19 |         similarity_score: 0.92,
20 |       },
21 |     ];
22 |     const line = `data: ${JSON.stringify({
23 |       type: "citations",
24 |       data: citations,
25 |     })}`;
26 |     const result = parseSSELine(line);
27 |     expect(result).toEqual({ type: "citations", data: citations });
28 |   });
29 |
30 |   it("parses a done event", () => {
31 |     const doneData = { message_id: "msg-123", tokens_used: 450 };
32 |     const line = `data: ${JSON.stringify({ type: "done", data: doneData })}`;
33 |     const result = parseSSELine(line);
34 |     expect(result).toEqual({ type: "done", data: doneData });
35 |   });
36 |
37 |   it("parses an error event", () => {
38 |     const errorData = { code: "rate_limited", message: "Daily limit reached" };
39 |     const line = `data: ${JSON.stringify({ type: "error", data: errorData })}`;
40 |     const result = parseSSELine(line);
41 |     expect(result).toEqual({ type: "error", data: errorData });
42 |   });
43 |
44 |   it("returns null for lines not starting with \"data: \", () => {
45 |     expect(parseSSELine("event: message")).toBeNull();
46 |     expect(parseSSELine("id: 123")).toBeNull();
47 |     expect(parseSSELine(": comment")).toBeNull();
48 |     expect(parseSSELine("")).toBeNull();
49 |   });
50 |
51 |   it("returns null for the [DONE] sentinel", () => {
52 |     expect(parseSSELine("data: [DONE]")).toBeNull();
53 |   });
54 |
55 |   it("returns null for empty data payload", () => {
56 |     expect(parseSSELine("data: ")).toBeNull();
57 |     expect(parseSSELine("data: ")).toBeNull();
58 |   });
59 |
60 |   it("returns null for malformed JSON", () => {
61 |     expect(parseSSELine("data: {invalid json}")).toBeNull();
62 |     expect(parseSSELine("data: not-json-at-all")).toBeNull();
63 |   });
64 |
65 |   it("returns null for partial JSON", () => {
66 |     expect(parseSSELine('data: {"type":"token"}')).toBeNull();
67 |   });
68 |
69 |   it("parses a plan_update event", () => {
70 |     const planData = {
71 |       id: "plan-1",
72 |       clo_id: "clo-1",
73 |       clo_title: "Data Structures",
74 |       study_time_recommendation: "Increase to 3 hours/week",
75 |       recommended_materials: [],
76 |       suggested_planner_sessions: 2,

```



```

77 |         interaction_count: 5,
78 |     };
79 |     const line = `data: ${JSON.stringify({
80 |         type: "plan_update",
81 |         data: planData,
82 |     })}`;
83 |     const result = parseSSELine(line);
84 |     expect(result).toEqual({ type: "plan_update", data: planData });
85 | });
86 |
87 | it("parses an independence_nudge event", () => {
88 |     const nudgeData = { message: "Try solving this on your own first!" };
89 |     const line = `data: ${JSON.stringify({
90 |         type: "independence_nudge",
91 |         data: nudgeData,
92 |     })}`;
93 |     const result = parseSSELine(line);
94 |     expect(result).toEqual({ type: "independence_nudge", data: nudgeData });
95 | });
96 |
97 | it("parses a handoff_suggestion event", () => {
98 |     const handoffData = {
99 |         reason: "low_rag_confidence",
100 |         message: "Consider asking your teacher.",
101 |     };
102 |     const line = `data: ${JSON.stringify({
103 |         type: "handoff_suggestion",
104 |         data: handoffData,
105 |     })}`;
106 |     const result = parseSSELine(line);
107 |     expect(result).toEqual({ type: "handoff_suggestion", data: handoffData });
108 | });
109 |
110 | it('handles data with extra whitespace after "data: "', () => {
111 |     const line = `data:   {"type":"token","data":"Hi"}`;
112 |     // The implementation trims after slicing, so this should still parse
113 |     const result = parseSSELine(line);
114 |     expect(result).toEqual({ type: "token", data: "Hi" });
115 | });
116 | });
117 |

```

FILE: [src/__tests__/unit/tutorStatePanel.test.tsx](#)

TYPE: .tsx | LINES: 1-120 | SHA256: 33645d1297a48919bb4cbe1653575b361f7aabde9971c64afcffa67e0bac7482

```

1 | // =====
2 | // TutorStatePanel - Unit tests
3 | // Feature: student-experience-remediation, Task 12.4
4 | // Validates: Requirements 4.2, 4.3, 4.4
5 | // =====
6 | // Verifies that each non-ready TutorUiState renders its own distinct panel with
7 | // the localized title/message routed through the `ai` i18next namespace, and
8 | // that the `ready` state renders nothing so the live chat surface shows through.
9 | // =====
10 |
11 | import { describe, it, expect, beforeEach } from "vitest";
12 | import { render, screen } from "@testing-library/react";
13 | import { I18nextProvider } from "react-i18next";
14 |
15 | import i18n from "@lib/i18n";
16 | import enAi from "@locales/en/ai.json";
17 | import arAi from "@locales/ar/ai.json";
18 | import TutorStatePanel from "@pages/student/tutor/TutorStatePanel";
19 | import type { TutorUiState } from "@lib/tutorStatus";
20 |
21 | // Pull the canonical copy straight from the locale so the assertions stay in
22 | // sync with the source of truth rather than duplicating literal strings.
23 | const states = enAi.tutor.states;
24 |
25 | const renderPanel = (state: TutorUiState, onRetry?: () => void) => {
26 |     render(
27 |         <I18nextProvider i18n={i18n}>
28 |             <TutorStatePanel state={state} onRetry={onRetry} />
29 |         </I18nextProvider>
30 |     );
31 |
32 | describe("TutorStatePanel", () => {
33 |     beforeEach(async () => {
34 |         await i18n.changeLanguage("en");
35 |     });
36 |
37 |     it("renders the unavailable panel with its localized title and message (R4.2)", () => {
38 |         renderPanel({ kind: "unavailable" });
39 |
40 |         expect(screen.getByText(states.unavailable.title)).toBeInTheDocument();
41 |         expect(screen.getByText(states.unavailable.message)).toBeInTheDocument();
42 |     });
43 |
44 |     it("renders the not-enrolled panel with its localized title and message (R4.3)", () => {
45 |         renderPanel({ kind: "not_enrolled" });
46 |
47 |         expect(screen.getByText(states.notEnrolled.title)).toBeInTheDocument();
48 |         expect(screen.getByText(states.notEnrolled.message)).toBeInTheDocument();
49 |     });
50 |
51 |     it("renders the no-embeddings panel with its localized title and message (R4.4)", () => {
52 |         renderPanel({ kind: "no_embeddings" });
53 |
54 |         expect(screen.getByText(states.noEmbeddings.title)).toBeInTheDocument();
55 |         expect(screen.getByText(states.noEmbeddings.message)).toBeInTheDocument();
56 |     });
57 |
58 |     it("renders the rate-limited panel with its localized title and message (R4.2)", () => {
59 |         renderPanel({ kind: "rate_limited" });

```

```

60 |
61 |     expect(screen.getByText(states.rateLimited.title)).toBeInTheDocument();
62 |     expect(screen.getByText(states.rateLimited.message)).toBeInTheDocument();
63 | });
64 |
65 | it("appends the localized reset hint to the rate-limited message when provided", () => {
66 |     const resetHint = "at midnight";
67 |     renderPanel({ kind: "rate_limited", resetHint });
68 |
69 |     expect(screen.getByText(states.rateLimited.title)).toBeInTheDocument();
70 |     // The reset interpolation embeds the hint; assert the rendered fragment
71 |     // contains it without duplicating the full i18n template literal.
72 |     expect(
73 |         screen.getByText((content) => content.includes(resetHint))
74 |     ).toBeInTheDocument();
75 | });
76 |
77 | it("renders the guaranteed error fallback with localized title and default message (R4.2a)", () => {
78 |     renderPanel({ kind: "error", message: "" });
79 |
80 |     expect(screen.getByText(states.error.title)).toBeInTheDocument();
81 |     expect(screen.getByText(states.error.message)).toBeInTheDocument();
82 | });
83 |
84 | it("renders a backend-provided error message in the fallback panel when present", () => {
85 |     const backendMessage = "Upstream model returned 418";
86 |     renderPanel({ kind: "error", message: backendMessage });
87 |
88 |     expect(screen.getByText(states.error.title)).toBeInTheDocument();
89 |     expect(screen.getByText(backendMessage)).toBeInTheDocument();
90 | });
91 |
92 | it("offers a retry control on recoverable panels (unavailable, error)", () => {
93 |     const { unmount } = renderPanel({ kind: "unavailable" }, () => {});
94 |     expect(
95 |         screen.getByRole("button", { name: states.retry })
96 |     ).toBeInTheDocument();
97 |     unmount();
98 |
99 |     renderPanel({ kind: "error", message: "" }, () => {});
100 |     expect(
101 |         screen.getByRole("button", { name: states.retry })
102 |     ).toBeInTheDocument();
103 | });
104 |
105 | it("renders nothing for the ready state so the live chat surface shows through", () => {
106 |     const { container } = renderPanel({ kind: "ready" });
107 |     expect(container).toBeEmptyDOMElement();
108 | });
109 |
110 | it("localizes panels in Arabic when the language switches", async () => {
111 |     await i18n.changeLanguage("ar");
112 |
113 |     renderPanel({ kind: "unavailable" });
114 |
115 |     expect(
116 |         screen.getByText(arAi.tutor.states.unavailable.title)
117 |     ).toBeInTheDocument();
118 | });
119 | });
120 |

```

FILE: src/_tests_/unit/tutorStatus.test.ts

TYPE: .ts | LINES: 1-173 | SHA256: 401dfad4feb3408780fa66b32f6af1c63c7cdcd47e75959f03e07c56e133fa1

```

1 | import { describe, it, expect } from "vitest";
2 | import { mapTutorError, type TutorUiState } from "@lib/tutorStatus";
3 |
4 | describe("mapTutorError", () => {
5 |     describe("network failures", () => {
6 |         it("maps a network error to unavailable", () => {
7 |             expect(mapTutorError({ networkError: true })).toEqual({
8 |                 kind: "unavailable",
9 |             });
10 |         });
11 |
12 |         it("treats network error as dominant over any status", () => {
13 |             expect(mapTutorError({ networkError: true, httpStatus: 403 })).toEqual({
14 |                 kind: "unavailable",
15 |             });
16 |         });
17 |     });
18 |
19 |     describe("HTTP status mapping", () => {
20 |         it("maps 200 to ready", () => {
21 |             expect(mapTutorError({ httpStatus: 200 })).toEqual({ kind: "ready" });
22 |         });
23 |
24 |         it("maps any 2xx to ready", () => {
25 |             expect(mapTutorError({ httpStatus: 204 })).toEqual({ kind: "ready" });
26 |         });
27 |
28 |         it("maps 403 to not_enrolled", () => {
29 |             expect(mapTutorError({ httpStatus: 403 })).toEqual({
30 |                 kind: "not_enrolled",
31 |             });
32 |         });
33 |
34 |         it("maps 429 to rate_limited", () => {
35 |             expect(mapTutorError({ httpStatus: 429 })).toEqual({
36 |                 kind: "rate_limited",
37 |                 resetHint: undefined,
38 |             });
39 |         });

```

```

40 |
41 | it("maps 500 to unavailable", () => {
42 |   expect(mapTutorError({ httpStatus: 500 })).toEqual({
43 |     kind: "unavailable",
44 |   });
45 | });
46 |
47 | it("maps 502 to unavailable", () => {
48 |   expect(mapTutorError({ httpStatus: 502 })).toEqual({
49 |     kind: "unavailable",
50 |   });
51 | });
52 |
53 | it("maps 503 to unavailable", () => {
54 |   expect(mapTutorError({ httpStatus: 503 })).toEqual({
55 |     kind: "unavailable",
56 |   });
57 | });
58 | });
59 |
60 | describe("structured code mapping", () => {
61 |   it("maps RATE_LIMIT_EXCEEDED to rate_limited", () => {
62 |     expect(
63 |       mapTutorError({ httpStatus: 429, code: "RATE_LIMIT_EXCEEDED" })
64 |     ).toEqual({ kind: "rate_limited", resetHint: undefined });
65 |   });
66 |
67 |   it("maps TOKEN_BUDGET_EXCEEDED to rate_limited", () => {
68 |     expect(mapTutorError({ code: "TOKEN_BUDGET_EXCEEDED" })).toEqual({
69 |       kind: "rate_limited",
70 |       resetHint: undefined,
71 |     });
72 |   });
73 |
74 |   it("threads resetHint into rate_limited", () => {
75 |     expect(
76 |       mapTutorError({ code: "RATE_LIMIT_EXCEEDED", resetHint: "midnight" })
77 |     ).toEqual({ kind: "rate_limited", resetHint: "midnight" });
78 |   });
79 |
80 |   it("maps EMBEDDING_ERROR to unavailable", () => {
81 |     expect(
82 |       mapTutorError({ httpStatus: 502, code: "EMBEDDING_ERROR" })
83 |     ).toEqual({ kind: "unavailable" });
84 |   });
85 |
86 |   it("maps LLM_ERROR to unavailable", () => {
87 |     expect(mapTutorError({ code: "LLM_ERROR" })).toEqual({
88 |       kind: "unavailable",
89 |     });
90 |   });
91 |
92 |   it("maps LLM_UNAVAILABLE to unavailable", () => {
93 |     expect(
94 |       mapTutorError({ httpStatus: 503, code: "LLM_UNAVAILABLE" })
95 |     ).toEqual({ kind: "unavailable" });
96 |   });
97 |
98 |   it("maps no_embeddings code to no_embeddings even on a 200 response", () => {
99 |     expect(mapTutorError({ httpStatus: 200, code: "no_embeddings" })).toEqual(
100 |       {
101 |         kind: "no_embeddings",
102 |       }
103 |     );
104 |   });
105 |
106 |   it("matches codes case-insensitively", () => {
107 |     expect(mapTutorError({ code: "rate_limit_exceeded" })).toEqual({
108 |       kind: "rate_limited",
109 |       resetHint: undefined,
110 |     });
111 |   });
112 | });
113 |
114 | describe("guaranteed fallback (R4.2a)", () => {
115 |   it("falls back to error for an empty signal", () => {
116 |     const result = mapTutorError({});
117 |     expect(result.kind).toBe("error");
118 |   });
119 |
120 |   it("falls back to error for an unrecognized status", () => {
121 |     const result = mapTutorError({ httpStatus: 418 });
122 |     expect(result.kind).toBe("error");
123 |   });
124 |
125 |   it("threads a provided message into the fallback", () => {
126 |     expect(mapTutorError({ httpStatus: 418, message: "Teapot" })).toEqual({
127 |       kind: "error",
128 |       message: "Teapot",
129 |     });
130 |   });
131 |
132 |   it("supplies default copy when no message is provided", () => {
133 |     const result = mapTutorError({});
134 |     if (result.kind !== "error") throw new Error("expected error state");
135 |     expect(result.message.length).toBeGreaterThan(0);
136 |   });
137 |
138 |   it("falls back to error when an unknown code has no usable status", () => {
139 |     const result = mapTutorError({ code: "totally_unknown" });
140 |     expect(result.kind).toBe("error");
141 |   });
142 |
143 |   it("uses status when an unknown code accompanies a known status", () => {

```

```

144 |     expect(
145 |       mapTutorError({ httpStatus: 403, code: "totally_unknown" })
146 |     ).toEqual({ kind: "not_enrolled" });
147 |   });
148 | });
149 |
150 | it("always returns a defined TutorUiState kind", () => {
151 |   const kinds: TutorUiState["kind"][] = [
152 |     "ready",
153 |     "unavailable",
154 |     "not_enrolled",
155 |     "no_embeddings",
156 |     "rate_limited",
157 |     "error",
158 |   ];
159 |   const samples = [
160 |     { httpStatus: 200 },
161 |     { httpStatus: 403 },
162 |     { httpStatus: 429 },
163 |     { httpStatus: 500 },
164 |     { networkError: true },
165 |     { code: "no_embeddings" },
166 |     {},
167 |   ];
168 |   for (const s of samples) {
169 |     expect(kinds).toContain(mapTutorError(s).kind);
170 |   }
171 | });
172 | });
173 |

```

FILE: src/__tests__/unit/updateChallengeProgress.test.ts

TYPE: .ts | LINES: 1-439 | SHA256: e26a325e665b6d7cab9f4279133da7f14bf20e7a90395582ceb382e7ff115a96

[illegible]

```
71 if (!p.course_id || typeof p.course_id !== "string") {  
72   return {  
73     valid: false,  
74     error: "course_id is required and must be a string",  
75   };  
76 }  
77  
78 return {  
79   valid: true,  
80   data: {  
81     event_type: p.event_type as EventType,  
82     student_id: p.student_id as string,  
83     course_id: p.course_id as string,  
84     metadata:  
85       ? (p.metadata && typeof p.metadata === "object"  
86         ? (p.metadata as Record<string, unknown>  
87           : undefined),  
88       },  
89   };  
90 }  
91  
92 // % % % Completion Detection Logic % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
93  
94 function isCompleted(currentProgress: number, goalTarget: number): boolean {  
95   return currentProgress >= goalTarget;  
96 }  
97  
98 // % % % Streak Computation (from sorted unique dates) % % % % % % % % % % % % % % % % % % % % % % % %  
99  
100 function computeStreakFromDates(sortedDatesDesc: string[]): number {  
101   if (sortedDatesDesc.length === 0) return 0;  
102  
103   let streak = 1;  
104   for (let i = 1; i < sortedDatesDesc.length; i++) {  
105     const current = new Date(sortedDatesDesc[i]!);  
106     const previous = new Date(sortedDatesDesc[i - 1]!);  
107     const diffDays = Math.round(  
108       (previous.getTime() - current.getTime()) / (1000 * 60 * 60 * 24)  
109     );  
110     if (diffDays === 1) {  
111       streak++;  
112     } else {  
113       break;  
114     }  
115   }  
116   return streak;  
117 }  
118  
119 // % % % Distinct Bloom's Levels Count % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
120  
121 function countDistinctBloomsLevels(levels: (string | null)[]): number {  
122   return new Set(levels.filter(Boolean)).size;  
123 }  
124  
125 // % % % Tests % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
126  
127 describe("update-challenge-progress Edge Function Logic", () => {  
128   describe("getMatchingChallengeTypes", () => {  
129     it("maps grade events to academic, blooms_climb, and cooperative", () => {  
130       expect(getMatchingChallengeTypes("grade")).toEqual([  
131         "academic",  
132         "blooms_climb",  
133         "cooperative",  
134       ]]);  
135     });  
136     it("maps habit events to habit and cooperative", () => {  
137       expect(getMatchingChallengeTypes("habit")).toEqual([  
138         "habit",  
139         "cooperative",  
140       ]]);  
141     });  
142     it("maps xp events to xp_race and cooperative", () => {  
143       expect(getMatchingChallengeTypes("xp")).toEqual([  
144         "xp_race",  
145         "cooperative",  
146       ]]);  
147     });  
148   });  
149 })  
  
describe("validatePayload", () => {  
150   it("accepts a valid payload with all required fields", () => {  
151     const result = validatePayload({  
152       event_type: "grade",  
153       student_id: "123e4567-e89b-12d3-a456-426614174000",  
154       course_id: "223e4567-e89b-12d3-a456-426614174000",  
155     });  
156     expect(result.valid).toBe(true);  
157     if (result.valid) {  
158       expect(result.data.event_type).toBe("grade");  
159       expect(result.data.student_id).toBe(  
160         "123e4567-e89b-12d3-a456-426614174000"  
161       );  
162       expect(result.data.course_id).toBe(  
163         "223e4567-e89b-12d3-a456-426614174000"  
164       );  
165     }  
166   });  
167   it("accepts a valid payload with optional metadata", () => {  
168     const result = validatePayload({  
169       event_type: "grade",
```

```

175 |         student_id: "abc",
176 |         course_id: "def",
177 |         metadata: { blooms_level: "Analyzing" },
178 |     });
179 |     expect(result.valid).toBe(true);
180 |     if (result.valid) {
181 |         expect(result.data.metadata).toEqual({ blooms_level: "Analyzing" });
182 |     }
183 | });
184 |
185 | it("rejects null payload", () => {
186 |     const result = validatePayload(null);
187 |     expect(result.valid).toBe(false);
188 | });
189 |
190 | it("rejects non-object payload", () => {
191 |     const result = validatePayload("not an object");
192 |     expect(result.valid).toBe(false);
193 | });
194 |
195 | it("rejects missing event_type", () => {
196 |     const result = validatePayload({ student_id: "abc", course_id: "def" });
197 |     expect(result.valid).toBe(false);
198 |     if (!result.valid) {
199 |         expect(result.error).toContain("event_type");
200 |     }
201 | });
202 |
203 | it("rejects invalid event_type", () => {
204 |     const result = validatePayload({
205 |         event_type: "invalid",
206 |         student_id: "abc",
207 |         course_id: "def",
208 |     });
209 |     expect(result.valid).toBe(false);
210 |     if (!result.valid) {
211 |         expect(result.error).toContain("event_type");
212 |     }
213 | });
214 |
215 | it("rejects missing student_id", () => {
216 |     const result = validatePayload({ event_type: "grade", course_id: "def" });
217 |     expect(result.valid).toBe(false);
218 |     if (!result.valid) {
219 |         expect(result.error).toContain("student_id");
220 |     }
221 | });
222 |
223 | it("rejects missing course_id", () => {
224 |     const result = validatePayload({
225 |         event_type: "grade",
226 |         student_id: "abc",
227 |     });
228 |     expect(result.valid).toBe(false);
229 |     if (!result.valid) {
230 |         expect(result.error).toContain("course_id");
231 |     }
232 | });
233 |
234 | it("ignores non-object metadata", () => {
235 |     const result = validatePayload({
236 |         event_type: "xp",
237 |         student_id: "abc",
238 |         course_id: "def",
239 |         metadata: "not an object",
240 |     });
241 |     expect(result.valid).toBe(true);
242 |     if (result.valid) {
243 |         expect(result.data.metadata).toBeUndefined();
244 |     }
245 | });
246 | });
247 |
248 | describe("isCompleted (completion detection)", () => {
249 |     it("returns true when progress equals goal_target", () => {
250 |         expect(isCompleted(10, 10)).toBe(true);
251 |     });
252 |
253 |     it("returns true when progress exceeds goal_target", () => {
254 |         expect(isCompleted(15, 10)).toBe(true);
255 |     });
256 |
257 |     it("returns false when progress is below goal_target", () => {
258 |         expect(isCompleted(5, 10)).toBe(false);
259 |     });
260 |
261 |     it("returns true for zero goal_target with zero progress", () => {
262 |         expect(isCompleted(0, 0)).toBe(true);
263 |     });
264 |
265 |     it("handles large values correctly", () => {
266 |         expect(isCompleted(9999, 10000)).toBe(false);
267 |         expect(isCompleted(10000, 10000)).toBe(true);
268 |     });
269 | });
270 |
271 | describe("computeStreakFromDates (habit progress)", () => {
272 |     it("returns 0 for empty dates", () => {
273 |         expect(computeStreakFromDates([])).toBe(0);
274 |     });
275 |
276 |     it("returns 1 for a single date", () => {
277 |         expect(computeStreakFromDates(["2025-01-15"])).toBe(1);
278 |     });

```

```

279 |
280 | it("returns correct streak for consecutive dates", () => {
281 |     expect(
282 |         computeStreakFromDates(["2025-01-15", "2025-01-14", "2025-01-13"])
283 |     ).toBe(3);
284 | });
285 |
286 | it("stops counting at a gap", () => {
287 |     expect(
288 |         computeStreakFromDates([
289 |             "2025-01-15",
290 |             "2025-01-14",
291 |             "2025-01-12", // gap: missing Jan 13
292 |             "2025-01-11",
293 |         ])
294 |     ).toBe(2);
295 | });
296 |
297 | it("handles a single-day streak followed by a gap", () => {
298 |     expect(computeStreakFromDates(["2025-01-15", "2025-01-10"])).toBe(1);
299 | });
300 |
301 | it("handles long consecutive streaks", () => {
302 |     const dates: string[] = [];
303 |     for (let i = 0; i < 30; i++) {
304 |         const d = new Date("2025-01-30");
305 |         d.setDate(d.getDate() - i);
306 |         dates.push(d.toISOString().slice(0, 10));
307 |     }
308 |     expect(computeStreakFromDates(dates)).toBe(30);
309 | });
310 | });
311 |
312 | describe("countDistinctBloomsLevels (blooms_climb progress)", () => {
313 |     it("returns 0 for empty array", () => {
314 |         expect(countDistinctBloomsLevels([])).toBe(0);
315 |     });
316 |
317 |     it("returns 0 for all null values", () => {
318 |         expect(countDistinctBloomsLevels([null, null])).toBe(0);
319 |     });
320 |
321 |     it("counts distinct non-null levels", () => {
322 |         expect(
323 |             countDistinctBloomsLevels([
324 |                 "Remembering",
325 |                 "Understanding",
326 |                 "Applying",
327 |                 "Analyzing",
328 |                 "Evaluating",
329 |                 "Creating",
330 |             ])
331 |         ).toBe(6);
332 |     });
333 |
334 |     it("deduplicates repeated levels", () => {
335 |         expect(
336 |             countDistinctBloomsLevels([
337 |                 "Remembering",
338 |                 "Remembering",
339 |                 "Understanding",
340 |                 "Understanding",
341 |             ])
342 |         ).toBe(2);
343 |     });
344 |
345 |     it("ignores null values mixed with valid levels", () => {
346 |         expect(
347 |             countDistinctBloomsLevels(["Remembering", null, "Applying", null])
348 |         ).toBe(2);
349 |     });
350 | });
351 |
352 | describe("Idempotency (Task 2.6)", () => {
353 |     it("recomputing progress from same source data yields same result", () => {
354 |         // Simulate: same set of graded assignments !' same count
355 |         const gradedAssignments = ["a1", "a2", "a3"];
356 |         const progress1 = gradedAssignments.length;
357 |         const progress2 = gradedAssignments.length;
358 |         expect(progress1).toBe(progress2);
359 |     });
360 |
361 |     it("recomputing XP total from same transactions yields same result", () => {
362 |         const transactions = [
363 |             { xp_amount: 10 },
364 |             { xp_amount: 25 },
365 |             { xp_amount: 15 },
366 |         ];
367 |         const total1 = transactions.reduce((sum, t) => sum + t.xp_amount, 0);
368 |         const total2 = transactions.reduce((sum, t) => sum + t.xp_amount, 0);
369 |         expect(total1).toBe(total2);
370 |         expect(total1).toBe(50);
371 |     });
372 |
373 |     it("reward_granted flag prevents double-awarding", () => {
374 |         // Simulate: first completion triggers reward, second does not
375 |         const progressRecord = {
376 |             current_progress: 10,
377 |             goal_target: 10,
378 |             completed_at: null as string | null,
379 |             reward_granted: false,
380 |         };
381 |     });
382 |     // First processing: completion detected

```

```

383 |     const firstComplete =
384 |       progressRecord.current_progress >= progressRecord.goal_target &&
385 |       !progressRecord.completed_at;
386 |     expect(firstComplete).toBe(true);
387 |
388 |     // Simulate reward granted
389 |     progressRecord.completed_at = "2025-01-15T00:00:00Z";
390 |     progressRecord.reward_granted = true;
391 |
392 |     // Second processing: already completed and rewarded – skip
393 |     const secondComplete =
394 |       progressRecord.current_progress >= progressRecord.goal_target &&
395 |       !progressRecord.completed_at;
396 |     expect(secondComplete).toBe(false);
397 |
398 |     // Even if completed_at is set but reward_granted is checked
399 |     const shouldReward = !progressRecord.reward_granted;
400 |     expect(shouldReward).toBe(false);
401 |   });
402 | });
403 |
404 | describe("Event type coverage", () => {
405 |   it("all three event types map to at least one challenge type", () => {
406 |     for (const eventType of VALID_EVENT_TYPES) {
407 |       const types = getMatchingChallengeTypes(eventType);
408 |       expect(types.length).toBeGreaterThan(0);
409 |     }
410 |   });
411 |
412 |   it("cooperative challenges are reachable from all event types", () => {
413 |     for (const eventType of VALID_EVENT_TYPES) {
414 |       const types = getMatchingChallengeTypes(eventType);
415 |       expect(types).toContain("cooperative");
416 |     }
417 |   });
418 |
419 |   it("each non-cooperative challenge type is reachable from exactly one event type", () => {
420 |     const typeToEvent: Record<string, EventType[]> = {};
421 |     for (const eventType of VALID_EVENT_TYPES) {
422 |       for (const ct of getMatchingChallengeTypes(eventType)) {
423 |         if (ct === "cooperative") continue;
424 |         if (!typeToEvent[ct]) typeToEvent[ct] = [];
425 |         typeToEvent[ct].push(eventType);
426 |       }
427 |     }
428 |     // academic !' grade only
429 |     expect(typeToEvent["academic"]).toEqual(["grade"]);
430 |     // blooms_climb !' grade only
431 |     expect(typeToEvent["blooms_climb"]).toEqual(["grade"]);
432 |     // habit !' habit only
433 |     expect(typeToEvent["habit"]).toEqual(["habit"]);
434 |     // xp_race !' xp only
435 |     expect(typeToEvent["xp_race"]).toEqual(["xp"]);
436 |   });
437 | });
438 | });
439 |

```

FILE: [src/___tests___/unit/useAdminDashboardAggregate.test.ts](#)

TYPE: .ts | LINES: 1-131 | SHA256: 79eb204cfc606e1f71539c3da10bc653913f0aa1437a334ae3ee3a1a200daf91

```

1 | // @vitest-environment happy-dom
2 | // =====
3 | // useAdminDashboardAggregate.test.ts
4 | // Feature: dashboard-and-ux-performance, Phase 8 Task 35 (admin aggregate)
5 | // -----
6 | // 1. CALL – get_admin_dashboard is called once.
7 | // 2. HYDRATION – the EXACT cache useAdminKPIs reads is populated.
8 | // 3. COLLAPSE – with the section hook gated on `aggregate.isError`, a SUCCESSFUL
9 | // aggregate fires ZERO section `supabase.from` requests; a FAILED aggregate
10 | // makes useAdminKPIs fall back and fetch.
11 | // The `get_admin_dashboard` RPC is mocked; this is a hermetic unit test.
12 | // =====
13 | import { describe, it, expect, vi, beforeEach } from "vitest";
14 | import { renderHook, waitFor } from "@testing-library/react";
15 | import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
16 | import { createElement, type ReactNode } from "react";
17 |
18 | const mockRpc = vi.fn();
19 | const mockFrom = vi.fn();
20 |
21 | vi.mock("@/lib/supabase", () => ({
22 |   supabase: {
23 |     rpc: (...args: unknown[]) => mockRpc(...args),
24 |     from: (...args: unknown[]) => mockFrom(...args),
25 |   },
26 | }));
27 |
28 | const makeFromBuilder = () => {
29 |   const builder: Record<string, unknown> = {
30 |     select: () => builder,
31 |     eq: () => builder,
32 |     in: () => builder,
33 |     then: (resolve: (value: unknown) => unknown) =>
34 |       resolve({ data: [], count: 0, error: null }),
35 |   };
36 |   return builder;
37 | };
38 |
39 | import { queryKeys } from "@/lib/queryKeys";
40 | import { useAdminDashboardAggregate } from "@/hooks/useAdminDashboardAggregate";
41 | import { useAdminKPIs, type AdminKPIData } from "@/hooks/useAdminDashboard";
42 |
43 | const INSTITUTION_ID = "44444444-4444-4444-4444-444444444444";

```



```

44 |
45 | const FIXTURE_KPIS: AdminKPIData = {
46 |   totalUsers: 124,
47 |   activeUsers: 124,
48 |   totalPrograms: 7,
49 |   totalCourses: 7,
50 |   usersByRole: {
51 |     admin: 3,
52 |     parent: 36,
53 |     student: 71,
54 |     teacher: 8,
55 |     coordinator: 6,
56 |   },
57 | };
58 |
59 | const useBlock = () => {
60 |   const aggregate = useAdminDashboardAggregate(INSTITUTION_ID);
61 |   const kpisHook = useAdminKPIS({ enabled: aggregate.isError });
62 |   return { aggregate, kpisHook };
63 | };
64 |
65 | const makeClient = () =>
66 |   new QueryClient({ defaultOptions: { queries: { retry: false } } });
67 |
68 | const makewrapper = (client: QueryClient) => {
69 |   const wrapper = ({ children }: { children: ReactNode }) =>
70 |     createElement(QueryClientProvider, { client }, children);
71 |   return Wrapper;
72 | };
73 |
74 | describe("useAdminDashboardAggregate (Phase 8 Task 35)", () => {
75 |   beforeEach(() => {
76 |     vi.clearAllMocks();
77 |     mockFrom.mockImplementation(() => makeFromBuilder());
78 |   });
79 |
80 |   it("calls get_admin_dashboard once", async () => {
81 |     mockRpc.mockResolvedValue({ data: FIXTURE_KPIS, error: null });
82 |     const client = makeClient();
83 |     const { result } = renderHook(
84 |       () => useAdminDashboardAggregate(INSTITUTION_ID),
85 |       { wrapper: makewrapper(client) }
86 |     );
87 |     await waitFor(() => expect(result.current.isSuccess).toBe(true));
88 |     expect(mockRpc).toHaveBeenCalledTimes(1);
89 |     expect(mockRpc).toHaveBeenCalledWith("get_admin_dashboard", {});
90 |   });
91 |
92 |   it("hydrates the exact KPI cache useAdminKPIS reads", async () => {
93 |     mockRpc.mockResolvedValue({ data: FIXTURE_KPIS, error: null });
94 |     const client = makeClient();
95 |     const { result } = renderHook(
96 |       () => useAdminDashboardAggregate(INSTITUTION_ID),
97 |       { wrapper: makewrapper(client) }
98 |     );
99 |     await waitFor(() => expect(result.current.isSuccess).toBe(true));
100 |     expect(
101 |       client.getQueryData<AdminKPIData>(queryKeys.adminDashboard.list({}))
102 |     ).toEqual(FIXTURE_KPIS);
103 |   });
104 |
105 |   it("collapse: a successful aggregate fires ZERO section requests", async () => {
106 |     mockRpc.mockResolvedValue({ data: FIXTURE_KPIS, error: null });
107 |     const client = makeClient();
108 |     const { result } = renderHook(() => useBlock(), {
109 |       wrapper: makewrapper(client),
110 |     });
111 |     await waitFor(() => expect(result.current.aggregate.isSuccess).toBe(true));
112 |     expect(mockRpc).toHaveBeenCalledTimes(1);
113 |     expect(mockFrom).not.toHaveBeenCalled();
114 |     await waitFor(() =>
115 |       expect(result.current.kpisHook.data).toEqual(FIXTURE_KPIS)
116 |     );
117 |     expect(mockFrom).not.toHaveBeenCalled();
118 |   });
119 |
120 |   it("fallback: a failed aggregate makes useAdminKPIS fetch", async () => {
121 |     mockRpc.mockResolvedValue({ data: null, error: { message: "boom" } });
122 |     const client = makeClient();
123 |     const { result } = renderHook(() => useBlock(), {
124 |       wrapper: makewrapper(client),
125 |     });
126 |     await waitFor(() => expect(result.current.aggregate.isError).toBe(true));
127 |     await waitFor(() => expect(mockFrom).toHaveBeenCalledTimes(1));
128 |     expect(mockFrom.mock.calls.map((c) => c[0])).toContain("profiles");
129 |   });
130 | });
131 |

```

```
FILE: src/  tests  /unit/useAISuggestions.test.ts
```

```
TYPE: .ts | LINES: 1-171 | SHA256: da2e3fb795697acdf31e4fa118048eff9ec57b6058a2bac1285b3438bfe7dd2b
```

[illegible]

```
update: mockUpdate,  
eq: mockEq,  
order: mockOrder,  
then: undefined as ((resolve: (v: unknown) => void) => void) | undefined,  
});  
  
vi.mock("@lib/supabase", () => ({  
  supabase: {  
    from: vi.fn(() => chainObj),  
  },  
}));  
  
import { supabase as _supabase } from "@lib/supabase";  
  
// eslint-disable-next-line @typescript-eslint/no-explicit-any  
const supabase = _supabase as unknown as { from: (table: string) => any };  
  
describe("useAISuggestions hooks - queryFn / mutationFn logic", () => {  
  beforeEach(() => {  
    vi.clearAllMocks();  
    mockSelect.mockReturnValue(chainObj);  
    mockUpdate.mockReturnValue(chainObj);  
    mockEq.mockReturnValue(chainObj);  
    mockOrder.mockReturnValue(chainObj);  
  });  
  
  // % % % useAISuggestions queryFn % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
  
  describe("useAISuggestions queryFn", () => {  
    it("queries ai_feedback table filtered by student_id and suggestion_type", () => {  
      const chain = supabase.from("ai_feedback");  
      chain.select(  
        "id, student_id, suggestion_type, suggestion_text, suggestion_data, feedback, validated_outcome, creat  
      );  
      chain.eq("student_id", "student-1");  
      chain.eq("suggestion_type", "module_suggestion");  
      chain.order("created_at", { ascending: false });  
  
      expect(supabase.from).toHaveBeenCalledWith("ai_feedback");  
      expect(mockSelect).toHaveBeenCalledWith(  
        "id, student_id, suggestion_type, suggestion_text, suggestion_data, feedback, validated_outcome, creat  
      );  
      expect(mockEq).toHaveBeenCalledWith("student_id", "student-1");  
      expect(mockEq).toHaveBeenCalledWith(  
        "suggestion_type",  
        "module_suggestion"  
      );  
      expect(mockOrder).toHaveBeenCalledWith("created_at", {  
        ascending: false,  
      });  
    });  
  
    it("uses correct query key structure", async () => {  
      // Verify the query key factory produces expected shape  
      const { queryKeys } = await import("@lib/queryKeys");  
      const key = queryKeys.aiSuggestions.list({ studentId: "student-1" });  
      expect(key).toEqual([  
        "aiSuggestions",  
        "list",  
        { studentId: "student-1" },  
      ]);  
    });  
  });  
  
  // % % % useSubmitAIFeedback mutationFn % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
  
  describe("useSubmitAIFeedback mutationFn", () => {  
    it("updates feedback column on ai_feedback record", () => {  
      const chain = supabase.from("ai_feedback");  
      chain.update({ feedback: "thumbs_up" });  
      chain.eq("id", "feedback-123");  
  
      expect(supabase.from).toHaveBeenCalledWith("ai_feedback");  
      expect(mockUpdate).toHaveBeenCalledWith({ feedback: "thumbs_up" });  
      expect(mockEq).toHaveBeenCalledWith("id", "feedback-123");  
    });  
  
    it("supports thumbs_down feedback value", () => {  
      const chain = supabase.from("ai_feedback");  
      chain.update({ feedback: "thumbs_down" });  
      chain.eq("id", "feedback-456");  
  
      expect(mockUpdate).toHaveBeenCalledWith({ feedback: "thumbs_down" });  
      expect(mockEq).toHaveBeenCalledWith("id", "feedback-456");  
    });  
  });  
  
  // % % % Error handling % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
  
  describe("error handling", () => {  
    it("propagates supabase errors from query", async () => {  
      mockOrder.mockResolvedValue({  
        data: null,  
        error: { message: "RLS denied" },  
      });  
  
      const chain = supabase.from("ai_feedback");  
      chain.select(  
        "id, student_id, suggestion_type, suggestion_text, suggestion_data, feedback, validated_outcome, creat  
      );  
      chain.eq("student_id", "student-1");  
      chain.eq("suggestion_type", "module_suggestion");  
      const result = await chain.order("created_at", { ascending: false });
```

FILE: src/__tests__/unit/useAnnouncements.test.ts

```
TYPE: .ts | LINES: 1-319 | SHA256: e8e8c603139ebb0beb89df82e94d3a82de617043804b8f95def78b2889f46f2f
```

```

46 |
47 | import { supabase as _supabase } from "@lib/supabase";
48 | import { logAuditEvent } from "@lib/auditLogger";
49 | import { useCreateAnnouncement } from "@hooks/useAnnouncements";
50 |
51 | type SupabaseResult = { data: unknown; error: unknown };
52 | interface MockChain {
53 |   select: (...args: unknown[]) => MockChain;
54 |   insert: (...args: unknown[]) => MockChain;
55 |   update: (...args: unknown[]) => MockChain;
56 |   delete: (...args: unknown[]) => MockChain;
57 |   eq: (...args: unknown[]) => MockChain;
58 |   order: (...args: unknown[]) => MockChain;
59 |   in: (...args: unknown[]) => MockChain;
60 |   limit: (...args: unknown[]) => Promise<SupabaseResult>;
61 |   maybeSingle: () => Promise<SupabaseResult>;
62 |   single: () => Promise<SupabaseResult>;
63 |   then: (cb: (v: unknown) => void) => Promise<void>;
64 | }
65 | const supabase = _supabase as unknown as {
66 |   from: (table: string) => MockChain;
67 |   rpc: (fn: string, args: unknown) => Promise<SupabaseResult>;
68 | };
69 |
70 | const createWrapper = () => {
71 |   const queryClient = new QueryClient({
72 |     defaultOptions: { queries: { retry: false }, mutations: { retry: false } },
73 |   });
74 |   return ({ children }: { children: ReactNode }) =>
75 |     createElement(QueryClientProvider, { client: queryClient }, children);
76 | };
77 |
78 | describe("useAnnouncements hooks – queryFn / mutationFn logic", () => {
79 |   beforeEach(() => {
80 |     vi.clearAllMocks();
81 |     mockSelect.mockReturnValue(chainObj);
82 |     mockInsert.mockReturnValue(chainObj);
83 |     mockUpdate.mockReturnValue(chainObj);
84 |     mockDelete.mockReturnValue(chainObj);
85 |     mockEq.mockReturnValue(chainObj);
86 |     mockOrder.mockReturnValue(chainObj);
87 |     mockIn.mockReturnValue(chainObj);
88 |     mockLimit.mockResolvedValue({ data: [], error: null });
89 |     mockMaybeSingle.mockResolvedValue({ data: null, error: null });
90 |     mockSingle.mockResolvedValue({ data: { id: "ann-1" }, error: null });
91 |     mockRpc.mockResolvedValue({ data: 0, error: null });
92 |     mockThen.mockImplementation((cb: (v: unknown) => void) => {
93 |       cb({ error: null });
94 |       return Promise.resolve();
95 |     });
96 |   });
97 |
98 |   describe("useAnnouncements queryFn", () => {
99 |     it("queries announcements table with course_id filter", () => {
100 |       supabase.from("announcements");
101 |       expect(supabase.from).toHaveBeenCalledWith("announcements");
102 |     });
103 |
104 |     it("orders by is_pinned DESC then created_at DESC", () => {
105 |       const chain = supabase.from("announcements");
106 |       chain.select(
107 |         "id, course_id, author_id, title, content, is_pinned, created_at, updated_at"
108 |       );
109 |       chain.eq("course_id", "course-1");
110 |       chain.order("is_pinned", { ascending: false });
111 |       chain.order("created_at", { ascending: false });
112 |
113 |       expect(mockOrder).toHaveBeenCalledWith("is_pinned", { ascending: false });
114 |       expect(mockOrder).toHaveBeenCalledWith("created_at", {
115 |         ascending: false,
116 |       });
117 |     });
118 |   });
119 |
120 |   describe("useAnnouncement queryFn", () => {
121 |     it("queries single announcement by id with maybeSingle", async () => {
122 |       const mockAnn = {
123 |         id: "ann-1",
124 |         course_id: "c-1",
125 |         author_id: "u-1",
126 |         title: "Test",
127 |         content: "Content",
128 |         is_pinned: false,
129 |         created_at: "2024-01-01",
130 |         updated_at: "2024-01-01",
131 |       };
132 |       mockMaybeSingle.mockResolvedValue({ data: mockAnn, error: null });
133 |
134 |       supabase.from("announcements");
135 |       const chain = supabase.from("announcements");
136 |       chain.select(
137 |         "id, course_id, author_id, title, content, is_pinned, created_at, updated_at"
138 |       );
139 |       chain.eq("id", "ann-1");
140 |       const result = await chain.maybeSingle();
141 |
142 |       expect(result.data).toEqual(mockAnn);
143 |     });
144 |   });
145 |
146 |   describe("useCreateAnnouncement mutationFn", () => {
147 |     it("inserts announcement and logs audit event", async () => {
148 |       const newAnn = {
149 |         id: "ann-new",

```

```

150 |         course_id: "c-1",
151 |         author_id: "u-1",
152 |         title: "New Announcement",
153 |         content: "Hello students",
154 |         is_pinned: true,
155 |     };
156 |     mockSingle.mockResolvedValue({ data: { ...newAnn }, error: null });
157 |
158 |     const chain = supabase.from("announcements");
159 |     chain.insert({
160 |         course_id: newAnn.course_id,
161 |         author_id: newAnn.author_id,
162 |         title: newAnn.title,
163 |         content: newAnn.content,
164 |         is_pinned: newAnn.is_pinned,
165 |     });
166 |     chain.select();
167 |     const result = await chain.single();
168 |
169 |     expect((result.data as { title: string }).title).toBe("New Announcement");
170 |     expect(mockInsert).toHaveBeenCalledTimes(1);
171 | });
172 |
173 | it("calls logAuditEvent on create", async () => {
174 |     await (logAuditEvent as unknown as (...args: unknown[]) => Promise<void>)(
175 |         {
176 |             action: "create",
177 |             entity_type: "announcement",
178 |             entity_id: "ann-1",
179 |             changes: { course_id: "c-1", title: "Test", is_pinned: false },
180 |             performed_by: "user-1",
181 |         }
182 |     );
183 |
184 |     expect(logAuditEvent).toHaveBeenCalledWith(
185 |         expect.objectContaining({
186 |             action: "create",
187 |             entity_type: "announcement",
188 |         })
189 |     );
190 | });
191 |
192 | it("fans out notifications via the RPC and never inserts to notifications (Req 15.1, 15.2)", async () => {
193 |     mockSingle.mockResolvedValue({
194 |         data: {
195 |             id: "ann-new",
196 |             course_id: "c-1",
197 |             author_id: "teacher-1",
198 |             title: "New Announcement",
199 |             content: "Hello students",
200 |             is_pinned: false,
201 |             created_at: "2024-01-01",
202 |             updated_at: "2024-01-01",
203 |         },
204 |         error: null,
205 |     });
206 |
207 |     const { result } = renderHook(() => useCreateAnnouncement(), {
208 |         wrapper: createWrapper(),
209 |     });
210 |
211 |     await result.current.mutateAsync({
212 |         course_id: "c-1",
213 |         author_id: "teacher-1",
214 |         title: "New Announcement",
215 |         content: "Hello students",
216 |         is_pinned: false,
217 |         performedBy: "teacher-1",
218 |     });
219 |
220 |     // Fan-out goes through the authorized RPC with the created announcement id
221 |     expect(mockRpc).toHaveBeenCalledWith(
222 |         "fan_out_announcement_notifications",
223 |         {
224 |             p_announcement_id: "ann-new",
225 |         }
226 |     );
227 |     // The buggy author-targeted notifications insert must be gone
228 |     const fromCalls = (supabase.from as unknown as ReturnType<typeof vi.fn>)
229 |         .mock.calls;
230 |     expect(fromCalls).not.toContainEqual(["notifications"]);
231 | });
232 |
233 | it("does not throw when fan-out fails (non-fatal) so the announcement is preserved", async () => {
234 |     mockSingle.mockResolvedValue({
235 |         data: {
236 |             id: "ann-new",
237 |             course_id: "c-1",
238 |             author_id: "teacher-1",
239 |             title: "New Announcement",
240 |             content: "Hello students",
241 |             is_pinned: false,
242 |             created_at: "2024-01-01",
243 |             updated_at: "2024-01-01",
244 |         },
245 |         error: null,
246 |     });
247 |     mockRpc.mockResolvedValue({ data: null, error: { message: "boom" } });
248 |     const errorSpy = vi.spyOn(console, "error").mockImplementation(() => {});
249 |
250 |     const { result } = renderHook(() => useCreateAnnouncement(), {
251 |         wrapper: createWrapper(),
252 |     });
253 |

```

```

254     const created = await result.current.mutateAsync({
255       course_id: "c-1",
256       author_id: "teacher-1",
257       title: "New Announcement",
258       content: "Hello students",
259       is_pinned: false,
260       performedBy: "teacher-1",
261     });
262
263     await waitFor(() => expect(result.current.isSuccess).toBe(true));
264     expect(created.id).toBe("ann-new");
265     expect(errorSpy).toHaveBeenCalledWith(
266       "Announcement fan-out failed:",
267       "boom"
268     );
269     errorSpy.mockRestore();
270   });
271 });
272
273 describe("useUpdateAnnouncement mutationFn", () => {
274   it("updates announcement by id", async () => {
275     mockSingle.mockResolvedValue({
276       data: { id: "ann-1", title: "Updated", is_pinned: true },
277       error: null,
278     });
279
280     const chain = supabase.from("announcements");
281     chain.update({ title: "Updated", is_pinned: true });
282     chain.eq("id", "ann-1");
283     chain.select();
284     const result = await chain.single();
285
286     expect((result.data as { title: string }).title).toBe("Updated");
287     expect(mockUpdate).toHaveBeenCalledWith({
288       title: "Updated",
289       is_pinned: true,
290     });
291   });
292 });
293
294 describe("useDeleteAnnouncement mutationFn", () => {
295   it("deletes announcement by id", async () => {
296     mockEq.mockResolvedValue({ error: null });
297
298     const chain = supabase.from("announcements");
299     chain.delete();
300     await chain.eq("id", "ann-1");
301
302     expect(mockDelete).toHaveBeenCalled();
303     expect(mockEq).toHaveBeenCalledWith("id", "ann-1");
304   });
305 });
306
307 describe("useStudentAnnouncements queryFn", () => {
308   it("fetches enrolled course IDs then queries announcements", () => {
309     // First call: get enrollments
310     supabase.from("student_courses");
311     expect(supabase.from).toHaveBeenCalledWith("student_courses");
312
313     // Second call: get announcements
314     supabase.from("announcements");
315     expect(supabase.from).toHaveBeenCalledWith("announcements");
316   });
317 });
318 });
319

```

FILE: src/__tests__/unit/useAtRiskPredictions.test.ts

TYPE: .ts | LINES: 1-91 | SHA256: ff86e434cf57ee07c24d2273b3a95e62f6c5ab0077ac09e667f0fbadc000f0d0

FILE: src/__tests__/unit/useAttendance.test.ts

TYPE: .ts | LINES: 1-32 | SHA256: 6126cfdcf05199afa9e3beaae7d4a7fdbfc0e00441b71ee9ce5ac810b55bbae47

```

1 import { describe, it, expect } from "vitest";
2 import { calculateAttendancePercent } from "@/hooks/useAttendance";
3
4 describe("calculateAttendancePercent", () => {
5   it("returns 100 when no sessions exist", () => {
6     expect(calculateAttendancePercent(0, 0, 0)).toBe(100);
7   });
8
9   it("counts present + late as attended", () => {
10    // 5 present + 2 late out of 10 sessions = 70%
11    expect(calculateAttendancePercent(5, 2, 10)).toBe(70);
12  });
13
14  it("returns 100% when all present", () => {
15    expect(calculateAttendancePercent(10, 0, 10)).toBe(100);
16  });
17
18  it("returns 0% when all absent", () => {
19    expect(calculateAttendancePercent(0, 0, 5)).toBe(0);
20  });
21
22  it("rounds to nearest integer", () => {
23    // 1 present + 0 late out of 3 sessions = 33.33... !' 33
24    expect(calculateAttendancePercent(1, 0, 3)).toBe(33);
25  });
26
27  it("handles late-only attendance", () => {
28    // 0 present + 3 late out of 4 sessions = 75%
29    expect(calculateAttendancePercent(0, 3, 4)).toBe(75);
30  });
31 });
32

```

FILE: src/__tests__/unit/useBloomsProgression.test.ts

TYPE: .ts | LINES: 1-270 | SHA256: da05809e47056f786366adb9dd7aaf9d684ea71b310198accec807a33f0a13107


```

216 expect(supabase.from()).toHaveBeenCalledWith("blooms_progression");
217 expect(mockSelect).toHaveBeenCalledWith(
218   "id, student_id, clo_id, course_id, highest_bloom_level, bloom_explorer_awarded, bloom_challenger_awa
219 );
220 expect(mockEq).toHaveBeenCalledWith("student_id", "student-1");
221 expect(mockOr).toHaveBeenCalledWith(
222   "bloom_explorer_awarded.eq.true,bloom_challenger_awarded.eq.true,bloom_pioneer_awarded.eq.true"
223 );
224 });
225
226 it("returns only rows with at least one badge awarded", async () => {
227   const badgeRows = [
228     { ...sampleProgression, bloom_explorer_awarded: true },
229     {
230       ...sampleProgression,
231       id: "prog-3",
232       clo_id: "clo-3",
233       bloom_pioneer_awarded: true,
234       highest_bloom_level: 6,
235     },
236   ];
237   mockOr.mockResolvedValue({ data: badgeRows, error: null });
238
239   const result = await chainObj.or(
240     "bloom_explorer_awarded.eq.true,bloom_challenger_awarded.eq.true,bloom_pioneer_awarded.eq.true"
241   );
242   expect(result.data).toHaveLength(2);
243   expect(result.data[0].bloom_explorer_awarded).toBe(true);
244   expect(result.data[1].bloom_pioneer_awarded).toBe(true);
245 });
246
247 it("returns empty array when no badges awarded", async () => {
248   mockOr.mockResolvedValue({ data: [], error: null });
249
250   const result = await chainObj.or(
251     "bloom_explorer_awarded.eq.true,bloom_challenger_awarded.eq.true,bloom_pioneer_awarded.eq.true"
252   );
253   expect(result.data).toEqual([]);
254 });
255
256 it("throws on supabase error", async () => {
257   mockOr.mockResolvedValue({
258     data: null,
259     error: { message: "Permission denied" },
260   });
261
262   const result = await chainObj.or(
263     "bloom_explorer_awarded.eq.true,bloom_challenger_awarded.eq.true,bloom_pioneer_awarded.eq.true"
264   );
265   expect(result.error).toBeTruthy();
266   expect(result.error.message).toBe("Permission denied");
267 });
268 });
269 });
270

```

FILE: src/__tests__/unit/useCalendar.test.ts

TYPE: .ts | LINES: 1-182 | SHA256: 8f3777e62ef1cd83f368e7fed0731ce3774661ab732c7e1095ab853597bb1e07


```

150 mockSelect.mockReturnValue(chainObj);
151 mockEq.mockReturnValue(chainObj);
152 mockIn.mockReturnValue(chainObj);
153 mockGte.mockReturnValue(chainObj);
154 mockLte.mockReturnValue(chainObj);
155 mockNot.mockReturnValue(chainObj);
156 });
157
158 it("exports CalendarEvent type with required fields", () => {
159 // Type-level check – if this compiles, the interface is correct
160 const event: import("@/hooks/useCalendar").CalendarEvent = {
161   id: "test-1",
162   title: "Test Event",
163   date: "2025-01-15",
164   source: "assignment",
165   color: "#3b82f6",
166 };
167 expect(event.id).toBe("test-1");
168 expect(event.source).toBe("assignment");
169 });
170
171 it("exports all CalendarEventSource types", () => {
172   const sources: Array<import("@/hooks/useCalendar").CalendarEventSource> = [
173     "assignment",
174     "quiz",
175     "class_session",
176     "academic_calendar",
177     "announcement",
178   ];
179   expect(sources).toHaveLength(5);
180 });
181 });
182

```

FILE: src/__tests__/unit/useChallengeDetail.test.ts

```
TYPE: .ts | LINES: 1-152 | SHA256: 4cb49bc64278759e841bc84763ca693801e456ba0497912a782f424ca82905b7
```



```

16 | interface MockChain {
17 |   select: (...args: unknown[]) => MockChain;
18 |   eq: (...args: unknown[]) => MockChain;
19 |   in: (...args: unknown[]) => MockChain;
20 |   order: (...args: unknown[]) => MockChain;
21 | }
22 |
23 | const chainObj: MockChain = {
24 |   select: mockSelect,
25 |   eq: mockEq,
26 |   in: mockIn,
27 |   order: mockOrder,
28 | };
29 |
30 | vi.mock("@/lib/supabase", () => ({
31 |   supabase: {
32 |     from: vi.fn(() => chainObj),
33 |   },
34 | }));
35 |
36 | import { supabase as _supabase } from "@/lib/supabase";
37 |
38 | const supabase = _supabase as unknown as { from: (table: string) => MockChain };
39 |
40 | describe("useCLOEvidence - queryFn logic", () => {
41 |   beforeEach(() => {
42 |     vi.clearAllMocks();
43 |     mockSelect.mockReturnValue(chainObj);
44 |     mockEq.mockReturnValue(chainObj);
45 |     mockIn.mockReturnValue(chainObj);
46 |     mockOrder.mockReturnValue(chainObj);
47 |   });
48 |
49 |   it("queries evidence table filtered by clo_id and student_id", () => {
50 |     const chain = supabase.from("evidence");
51 |     chain.select("id, score_percent, created_at, submission_id");
52 |     chain.eq("clo_id", "clo-1");
53 |     chain.eq("student_id", "student-1");
54 |     chain.order("created_at", { ascending: false });
55 |
56 |     expect(supabase.from).toHaveBeenCalled("evidence");
57 |     expect(mockSelect).toHaveBeenCalled("id, score_percent, created_at, submission_id");
58 |     expect(mockEq).toHaveBeenCalled("clo_id", "clo-1");
59 |     expect(mockEq).toHaveBeenCalled("student_id", "student-1");
60 |     expect(mockOrder).toHaveBeenCalled("created_at", { ascending: false });
61 |   });
62 |
63 |   it("queries submissions table to resolve assignment_id", () => {
64 |     const chain = supabase.from("submissions");
65 |     chain.select("id, assignment_id");
66 |     chain.in("id", ["sub-1", "sub-2"]);
67 |
68 |     expect(supabase.from).toHaveBeenCalled("submissions");
69 |     expect(mockSelect).toHaveBeenCalled("id, assignment_id");
70 |     expect(mockIn).toHaveBeenCalled("id", ["sub-1", "sub-2"]);
71 |   });
72 |
73 |   it("queries assignments table to resolve titles", () => {
74 |     const chain = supabase.from("assignments");
75 |     chain.select("id, title");
76 |     chain.in("id", ["assign-1"]);
77 |
78 |     expect(supabase.from).toHaveBeenCalled("assignments");
79 |     expect(mockSelect).toHaveBeenCalled("id, title");
80 |     expect(mockIn).toHaveBeenCalled("id", ["assign-1"]);
81 |   });
82 |
83 |   it("maps evidence rows to CLOEvidenceRecord shape", () => {
84 |     // Simulate the mapping logic from the hook
85 |     const evidenceRows = [
86 |       {
87 |         id: "ev-1",
88 |         score_percent: 85,
89 |         created_at: "2025-01-15T10:00:00Z",
90 |         submission_id: "sub-1",
91 |       },
92 |       {
93 |         id: "ev-2",
94 |         score_percent: 72,
95 |         created_at: "2025-01-10T10:00:00Z",
96 |         submission_id: "sub-2",
97 |       },
98 |     ];
99 |
100 |   });
101 |
102 |   const submissions = [
103 |     { id: "sub-1", assignment_id: "assign-1" },
104 |     { id: "sub-2", assignment_id: "assign-2" },
105 |   ];
106 |
107 |   const assignments = [
108 |     { id: "assign-1", title: "Midterm Essay" },
109 |     { id: "assign-2", title: "Lab Report 3" },
110 |   ];
111 |
112 |   // Build lookup maps (same logic as the hook)
113 |   const assignmentMap = new Map<string, string>();
114 |   for (const a of assignments) {
115 |     assignmentMap.set(a.id, a.title);
116 |   }
117 |
118 |   const submissionAssignmentMap = new Map<string, string>();
119 |   for (const s of submissions) {

```

```

120 |     submissionAssignmentMap.set(s.id, s.assignment_id);
121 | }
122 |
123 | const result = evidenceRows.map((e) => {
124 |   const assignmentId = submissionAssignmentMap.get(e.submission_id) ?? "";
125 |   return {
126 |     id: e.id,
127 |     assignment_title:
128 |       assignmentMap.get(assignmentId) ?? "Unknown Assignment",
129 |     score_percent: e.score_percent,
130 |     created_at: e.created_at,
131 |   };
132 | });
133 |
134 | expect(result).toEqual([
135 |   {
136 |     id: "ev-1",
137 |     assignment_title: "Midterm Essay",
138 |     score_percent: 85,
139 |     created_at: "2025-01-15T10:00:00Z",
140 |   },
141 |   {
142 |     id: "ev-2",
143 |     assignment_title: "Lab Report 3",
144 |     score_percent: 72,
145 |     created_at: "2025-01-10T10:00:00Z",
146 |   },
147 | ]);
148 | });
149 |
150 | it('returns "Unknown Assignment" when assignment lookup fails', () => {
151 |   const evidenceRows = [
152 |     {
153 |       id: "ev-1",
154 |       score_percent: 60,
155 |       created_at: "2025-01-15T10:00:00Z",
156 |       submission_id: "sub-missing",
157 |     },
158 |   ];
159 |
160 |   const submissionAssignmentMap = new Map<string, string>();
161 |   const assignmentMap = new Map<string, string>();
162 |
163 |   const result = evidenceRows.map((e) => {
164 |     const assignmentId = submissionAssignmentMap.get(e.submission_id) ?? "";
165 |     return {
166 |       id: e.id,
167 |       assignment_title:
168 |         assignmentMap.get(assignmentId) ?? "Unknown Assignment",
169 |       score_percent: e.score_percent,
170 |       created_at: e.created_at,
171 |     };
172 |   });
173 |
174 |   expect(result[0]!.assignment_title).toBe("Unknown Assignment");
175 | });
176 | });
177 |

```

FILE: [src/__tests__/unit/useCoordinatorDashboardAggregate.test.ts](#)

TYPE: .ts | LINES: 1-131 | SHA256: 21e696955cf1739bce5c5725adb233448f8ba99ae495483cfbc37565c5921784

```

1 | // @vitest-environment happy-dom
2 | // =====
3 | // useCoordinatorDashboardAggregate.test.ts
4 | // Feature: dashboard-and-ux-performance, Phase 8 Task 34 (coordinator aggregate)
5 | // =====
6 | // 1. CALL – get_coordinator_dashboard is called once.
7 | // 2. HYDRATION – the EXACT cache useCoordinatorKPIs reads is populated.
8 | // 3. COLLAPSE – with the section hook gated on `aggregate.isError`, a SUCCESSFUL
9 | //    aggregate fires ZERO section `supabase.from` requests; a FAILED aggregate
10 | //    makes useCoordinatorKPIs fall back and fetch.
11 | // The `get_coordinator_dashboard` RPC is mocked; this is a hermetic unit test.
12 | // =====
13 | import { describe, it, expect, vi, beforeEach } from "vitest";
14 | import { renderHook, waitFor } from "@testing-library/react";
15 | import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
16 | import { createElement, type ReactNode } from "react";
17 |
18 | const mockRpc = vi.fn();
19 | const mockFrom = vi.fn();
20 |
21 | vi.mock("@/lib/supabase", () => ({
22 |   supabase: {
23 |     rpc: (...args: unknown[]) => mockRpc(...args),
24 |     from: (...args: unknown[]) => mockFrom(...args),
25 |   },
26 | }));
27 |
28 | const makeFromBuilder = () => {
29 |   const builder: Record<string, unknown> = {
30 |     select: () => builder,
31 |     eq: () => builder,
32 |     in: () => builder,
33 |     then: (resolve: (value: unknown) => unknown) =>
34 |       resolve({ data: [], count: 0, error: null }),
35 |   };
36 |   return builder;
37 | };
38 |
39 | import { queryKeys } from "@/lib/queryKeys";
40 | import { useCoordinatorDashboardAggregate } from "@/hooks/useCoordinatorDashboardAggregate";
41 | import {
42 |   useCoordinatorKPIs,

```



```

22 |     order: mockOrder,
23 |     in: mockIn,
24 |     maybeSingle: mockMaybeSingle,
25 |     single: mockSingle,
26 |   };
27 |
28 |   const mockUpload = vi.fn();
29 |   const mockGetPublicUrl = vi.fn();
30 |
31 |   vi.mock("@/lib/supabase", () => ({
32 |     supabase: {
33 |       from: vi.fn(() => chainObj),
34 |       storage: {
35 |         from: vi.fn(() => ({
36 |           upload: mockUpload,
37 |           getPublicUrl: mockGetPublicUrl,
38 |         })),
39 |       },
40 |     },
41 |   }));
42 |
43 |   vi.mock("@/lib/auditLogger", () => ({
44 |     logAuditEvent: vi.fn().mockResolvedValue(undefined),
45 |   }));
46 |
47 |   import { supabase as _supabase } from "@/lib/supabase";
48 |   import { logAuditEvent } from "@/lib/auditLogger";
49 |   import { uploadMaterialFile } from "@/hooks/useCourseModules";
50 |
51 |   /* eslint-disable @typescript-eslint/no-explicit-any */
52 |   const supabase = _supabase as unknown as {
53 |     from: (table: string) => any;
54 |     storage: { from: (bucket: string) => any };
55 |   };
56 |   /* eslint-enable @typescript-eslint/no-explicit-any */
57 |
58 |   describe("useCourseModules hooks - queryFn / mutationFn logic", () => {
59 |     beforeEach(() => {
60 |       vi.clearAllMocks();
61 |       mockSelect.mockReturnValue(chainObj);
62 |       mockInsert.mockReturnValue(chainObj);
63 |       mockUpdate.mockReturnValue(chainObj);
64 |       mockDelete.mockReturnValue(chainObj);
65 |       mockEq.mockReturnValue(chainObj);
66 |       mockOrder.mockReturnValue(chainObj);
67 |       mockIn.mockReturnValue(chainObj);
68 |       mockMaybeSingle.mockResolvedValue({ data: null, error: null });
69 |       mockSingle.mockResolvedValue({ data: { id: "mod-1" }, error: null });
70 |     });
71 |
72 |     describe("useCourseModules queryFn", () => {
73 |       it("queries course_modules table with course_id filter", () => {
74 |         supabase.from("course_modules");
75 |         expect(supabase.from).toHaveBeenCalledTimes(1);
76 |       });
77 |
78 |       it("orders by sort_order ascending", () => {
79 |         const chain = supabase.from("course_modules");
80 |         chain.select(
81 |           "id, course_id, title, description, sort_order, is_published, created_at"
82 |         );
83 |         chain.eq("course_id", "course-1");
84 |         chain.order("sort_order", { ascending: true });
85 |
86 |         expect(mockOrder).toHaveBeenCalledTimes(1);
87 |       });
88 |     });
89 |
90 |     describe("useCourseMaterials queryFn", () => {
91 |       it("queries course_materials table with module_id filter", () => {
92 |         supabase.from("course_materials");
93 |         expect(supabase.from).toHaveBeenCalledTimes(1);
94 |       });
95 |
96 |       it("orders materials by sort_order ascending", () => {
97 |         const chain = supabase.from("course_materials");
98 |         chain.select(
99 |           "id, module_id, title, type, content_url, file_path, description, sort_order, is_published, clo_ids, c
100 |         );
101 |         chain.eq("module_id", "mod-1");
102 |         chain.order("sort_order", { ascending: true });
103 |
104 |         expect(mockOrder).toHaveBeenCalledTimes(1);
105 |       });
106 |     });
107 |
108 |     describe("useCreateModule mutationFn", () => {
109 |       it("inserts module and logs audit event", async () => {
110 |         mockSingle.mockResolvedValue({
111 |           data: { id: "mod-new", course_id: "c-1", title: "Week 1" },
112 |           error: null,
113 |         });
114 |
115 |         const chain = supabase.from("course_modules");
116 |         chain.insert({
117 |           course_id: "c-1",
118 |           title: "Week 1",
119 |           description: null,
120 |           sort_order: 0,
121 |           is_published: false,
122 |         });
123 |         chain.select();
124 |         const result = await chain.single();
125 |

```

```

126     expect(result.data.title).toBe("Week 1");
127     expect(mockInsert).toHaveBeenCalled();
128   });
129
130   it("calls logAuditEvent on module create", async () => {
131     await (logAuditEvent as unknown as (...args: unknown[]) => Promise<void>)(
132       {
133         action: "create",
134         entity_type: "course_module",
135         entity_id: "mod-1",
136         changes: {
137           course_id: "c-1",
138           title: "Week 1",
139           sort_order: 0,
140           is_published: false,
141         },
142         performed_by: "teacher-1",
143       }
144     );
145
146     expect(logAuditEvent).toHaveBeenCalledWith(
147       expect.objectContaining({
148         action: "create",
149         entity_type: "course_module",
150       })
151     );
152   });
153 });
154
155 describe("useCreateMaterial mutationFn", () => {
156   it("inserts material with clo_ids", async () => {
157     mockSingle.mockResolvedValue({
158       data: {
159         id: "mat-1",
160         module_id: "mod-1",
161         title: "Lecture Slides",
162         type: "file",
163         clo_ids: ["clo-1"],
164       },
165       error: null,
166     });
167
168     const chain = supabase.from("course_materials");
169     chain.insert({
170       module_id: "mod-1",
171       title: "Lecture Slides",
172       type: "file",
173       content_url: null,
174       file_path: "/path/to/file.pdf",
175       description: null,
176       sort_order: 0,
177       is_published: true,
178       clo_ids: ["clo-1"],
179     });
180     chain.select();
181     const result = await chain.single();
182
183     expect(result.data.title).toBe("Lecture Slides");
184     expect(result.data.clo_ids).toEqual(["clo-1"]);
185   });
186 });
187
188 describe("useDeleteModule mutationFn", () => {
189   it("deletes module by id", async () => {
190     mockEq.mockResolvedValue({ error: null });
191
192     const chain = supabase.from("course_modules");
193     chain.delete();
194     await chain.eq("id", "mod-1");
195
196     expect(mockDelete).toHaveBeenCalled();
197     expect(mockEq).toHaveBeenCalledWith("id", "mod-1");
198   });
199 });
200
201 describe("uploadMaterialFile", () => {
202   it("rejects files exceeding 50MB", async () => {
203     const bigFile = new File(["x".repeat(100)], "big.pdf", {
204       type: "application/pdf",
205     });
206     Object.defineProperty(bigFile, "size", { value: 51 * 1024 * 1024 });
207
208     await expect(uploadMaterialFile(bigFile, "course-1")).rejects.toThrow(
209       "50MB"
210     );
211   });
212
213   it("rejects disallowed file extensions", async () => {
214     const badFile = new File(["x"], "script.exe", {
215       type: "application/octet-stream",
216     });
217
218     await expect(uploadMaterialFile(badFile, "course-1")).rejects.toThrow(
219       "not allowed"
220     );
221   });
222
223   it("uploads valid file and returns the storage path", async () => {
224     const validFile = new File(["content"], "slides.pdf", {
225       type: "application/pdf",
226     });
227     Object.defineProperty(validFile, "size", { value: 1024 });
228
229     mockUpload.mockResolvedValue({ error: null });

```

```

230 |         const url = await uploadMaterialFile(validFile, "course-1");
231 |
232 |
233 |         // course-materials is a private bucket; we now return the storage path
234 |         // and consumers must call getSignedUrl() at READ time. The path format
235 |         // is `${courseId}/${timestamp}_${safeName}` per useCourseModules.ts.
236 |         expect(url).toMatch(/^course-1\/\d+_slides\.pdf$/);
237 |         expect(mockUpload).toHaveBeenCalled();
238 |     });
239 | });
240 |
241 |

```

FILE: [src/__tests__/unit/useCourseSections.test.ts](#)

TYPE: .ts | LINES: 1-131 | SHA256: a204615dfd43171fa9375f4e18b6a262ea0cf626839387aaa5577c951c030e2f

```

1 | // @vitest-environment happy-dom
2 | import { describe, it, expect, vi } from "vitest";
3 |
4 | // Mock supabase
5 | const mockSelect = vi.fn();
6 | const mockInsert = vi.fn();
7 | const mockUpdate = vi.fn();
8 | const mockEq = vi.fn();
9 | const mockOrder = vi.fn();
10 | const mockSingle = vi.fn();
11 | const mockMaybeSingle = vi.fn();
12 |
13 | const chainable = () => ({
14 |   select: mockSelect,
15 |   insert: mockInsert,
16 |   update: mockUpdate,
17 |   eq: mockEq,
18 |   order: mockOrder,
19 |   single: mockSingle,
20 |   maybeSingle: mockMaybeSingle,
21 | });
22 |
23 | vi.mock("@/lib/supabase", () => ({
24 |   supabase: {
25 |     from: vi.fn(() => chainable()),
26 |   },
27 | }));
28 |
29 | vi.mock("@/lib/auditLogger", () => ({
30 |   logAuditEvent: vi.fn().mockResolvedValue(undefined),
31 | }));
32 |
33 | vi.mock("@/hooks/useAuth", () => ({
34 |   useAuth: () => ({ user: { id: "user-1" }, institutionId: "inst-1" }),
35 | }));
36 |
37 | import type {
38 |   CourseSection,
39 |   CourseSectionWithTeacher,
40 |   CreateCourseSectionInput,
41 |   UpdateCourseSectionInput,
42 | } from "@/hooks/useCourseSections";
43 |
44 | describe("useCourseSections types", () => {
45 |   it("CourseSection has required fields", () => {
46 |     const section: CourseSection = {
47 |       id: "sec-1",
48 |       course_id: "course-1",
49 |       section_code: "A",
50 |       teacher_id: "teacher-1",
51 |       capacity: 40,
52 |       is_active: true,
53 |       created_at: "2025-01-01T00:00:00Z",
54 |     };
55 |     expect(section.id).toBe("sec-1");
56 |     expect(section.section_code).toBe("A");
57 |     expect(section.capacity).toBe(40);
58 |     expect(section.is_active).toBe(true);
59 |   });
60 |
61 |   it("CourseSectionWithTeacher includes teacher profile", () => {
62 |     const section: CourseSectionWithTeacher = {
63 |       id: "sec-1",
64 |       course_id: "course-1",
65 |       section_code: "B",
66 |       teacher_id: "teacher-2",
67 |       capacity: 35,
68 |       is_active: true,
69 |       created_at: "2025-01-01T00:00:00Z",
70 |       profiles: {
71 |         id: "teacher-2",
72 |         full_name: "Dr. Smith",
73 |         email: "smith@uni.edu",
74 |       },
75 |     };
76 |     expect(section.profiles?.full_name).toBe("Dr. Smith");
77 |   });
78 |
79 |   it("CourseSectionWithTeacher allows null profiles", () => {
80 |     const section: CourseSectionWithTeacher = {
81 |       id: "sec-2",
82 |       course_id: "course-1",
83 |       section_code: "C",
84 |       teacher_id: "teacher-3",
85 |       capacity: 30,
86 |       is_active: false,
87 |       created_at: "2025-01-01T00:00:00Z",
88 |       profiles: null,

```

```

89     };
90     expect(section.profiles).toBeNull();
91   });
92
93   it("CreateCourseSectionInput has required fields", () => {
94     const input: CreateCourseSectionInput = {
95       course_id: "course-1",
96       section_code: "A",
97       teacher_id: "teacher-1",
98       capacity: 40,
99     };
100    expect(input.course_id).toBe("course-1");
101    expect(input.section_code).toBe("A");
102    expect(input.is_active).toBeUndefined();
103  });
104
105   it("CreateCourseSectionInput allows optional is_active", () => {
106     const input: CreateCourseSectionInput = {
107       course_id: "course-1",
108       section_code: "B",
109       teacher_id: "teacher-2",
110       capacity: 35,
111       is_active: false,
112     };
113    expect(input.is_active).toBe(false);
114  });
115
116   it("UpdateCourseSectionInput allows partial fields", () => {
117     const input: UpdateCourseSectionInput = {
118       capacity: 50,
119     };
120    expect(input.capacity).toBe(50);
121    expect(input.section_code).toBeUndefined();
122  });
123
124   it("UpdateCourseSectionInput allows is_active toggle", () => {
125     const input: UpdateCourseSectionInput = {
126       is_active: false,
127     };
128    expect(input.is_active).toBe(false);
129  });
130 });
131

```

FILE: src/__tests__/unit/useCQIPlans.test.ts

```
TYPE: .ts | LINES: 1-336 | SHA256: 64f8abec8c1c219b90ffc5d0136fd61a5de1839db7c3d79d656fc1bd7b732ce3
```

[illegible]

[illegible]

[illegible]

[illegible]

FILE: src/__tests__/unit/useFlowCheckIns.test.ts

```
TYPE: .ts | LINES: 1-270 | SHA256: 06e73623f18ed47a7019489157d7b29dbd4c9cc7591a6554baf78fd35f118d80
```

[illegible]

[illegible]

[illegible]

[illegible]

FILE: src/__tests__/unit/useGradebook.test.ts

```
TYPE: .ts | LINES: 1-212 | SHA256: 616424c4f6994a6e8c76a866876bd59a457ef96f2a0eb23ed6161b6e39cb0a74
```

```

1 import { describe, it, expect, vi } from "vitest";
2
3 // Mock supabase
4 vi.mock("@/lib/supabase", () => ({
5   supabase: {
6     from: vi.fn(() => ({
7       select: vi.fn().mockReturnThis(),
8       insert: vi.fn().mockReturnThis(),
9       update: vi.fn().mockReturnThis(),
10      delete: vi.fn().mockReturnThis(),
11      order: vi.fn().mockReturnThis(),
12      eq: vi.fn().mockReturnThis(),
13      in: vi.fn().mockReturnThis(),
14      not: vi.fn().mockReturnThis(),
15      single: vi.fn().mockResolvedValue({ data: null, error: null }),
16      maybeSingle: vi.fn().mockResolvedValue({ data: null, error: null }),
17    })),
18   },
19 }));
20
21 vi.mock("@/lib/auditLogger", () => ({
22   logAuditEvent: vi.fn().mockResolvedValue(undefined),
23 }));
24
25 vi.mock("@/hooks/useAuth", () => ({
26   useAuth: () => ({ user: { id: "teacher-1" }, institutionId: "inst-1" }),
27 }));
28
29 import type {
30   GradeCategory,
31   CreateGradeCategoryInput,
32   UpdateGradeCategoryInput,
33   GradebookEntry,
34   GradebookCategoryEntry,
35   GradebookAssessment,
36 } from "@/hooks/useGradebook";
37
38 describe("useGradebook types", () => {
39   it("GradeCategory has required fields", () => {
40     const cat: GradeCategory = {
41       id: "cat-1",
42       course_id: "course-1",
43       name: "Assignments",
44       weight_percent: 40,
45       sort_order: 0,
46       created_at: "2025-01-01T00:00:00Z",
47     };
48     expect(cat.name).toBe("Assignments");
49     expect(cat.weight_percent).toBe(40);
50   });
51
52   it("CreateGradeCategoryInput has required fields", () => {
53     const input: CreateGradeCategoryInput = {
54       course_id: "course-1",
55       name: "Quizzes",
56       weight_percent: 30,
57       sort_order: 1,
58     };
59     expect(input.course_id).toBe("course-1");
60     expect(input.weight_percent).toBe(30);
61   });
62
63   it("UpdateGradeCategoryInput allows partial fields", () => {
64     const input: UpdateGradeCategoryInput = {
65       name: "Midterm Exam",
66     };
67     expect(input.name).toBe("Midterm Exam");
68   });
69 }

```

```

68 |     expect(input.weight_percent).toBeUndefined();
69 | });
70 |
71 | it("GradebookAssessment tracks score and max_score", () => {
72 |   const assessment: GradebookAssessment = {
73 |     id: "a-1",
74 |     title: "Assignment 1",
75 |     type: "assignment",
76 |     score: 85,
77 |     max_score: 100,
78 |   };
79 |   expect(assessment.score).toBe(85);
80 |   expect(assessment.max_score).toBe(100);
81 | });
82 |
83 | it("GradebookAssessment allows null score for ungraded", () => {
84 |   const assessment: GradebookAssessment = {
85 |     id: "a-2",
86 |     title: "Assignment 2",
87 |     type: "assignment",
88 |     score: null,
89 |     max_score: 100,
90 |   };
91 |   expect(assessment.score).toBeNull();
92 | });
93 |
94 | it("GradebookCategoryEntry calculates subtotal_percent", () => {
95 |   const entry: GradebookCategoryEntry = {
96 |     category_id: "cat-1",
97 |     category_name: "Assignments",
98 |     weight_percent: 40,
99 |     assessments: [
100 |       {
101 |         id: "a-1",
102 |         title: "A1",
103 |         type: "assignment",
104 |         score: 80,
105 |         max_score: 100,
106 |       },
107 |       {
108 |         id: "a-2",
109 |         title: "A2",
110 |         type: "assignment",
111 |         score: 90,
112 |         max_score: 100,
113 |       },
114 |     ],
115 |     subtotal_percent: 85,
116 |   };
117 |   expect(entry.subtotal_percent).toBe(85);
118 |   expect(entry.assessments).toHaveLength(2);
119 | });
120 |
121 | it("GradebookEntry has final_weighted_grade and letter_grade", () => {
122 |   const entry: GradebookEntry = {
123 |     student_id: "student-1",
124 |     student_name: "Alice",
125 |     categories: [
126 |       {
127 |         category_id: "cat-1",
128 |         category_name: "Assignments",
129 |         weight_percent: 60,
130 |         assessments: [],
131 |         subtotal_percent: 80,
132 |       },
133 |       {
134 |         category_id: "cat-2",
135 |         category_name: "Quizzes",
136 |         weight_percent: 40,
137 |         assessments: [],
138 |         subtotal_percent: 90,
139 |       },
140 |     ],
141 |     final_weighted_grade: 84,
142 |     letter_grade: "B",
143 |   };
144 |   // 60% * 80 + 40% * 90 = 48 + 36 = 84
145 |   expect(entry.final_weighted_grade).toBe(84);
146 |   expect(entry.letter_grade).toBe("B");
147 | });
148 | });
149 |
150 | describe("grade category weight validation", () => {
151 |   it("weights summing to 100 is valid", () => {
152 |     const categories: GradeCategory[] = [
153 |       {
154 |         id: "1",
155 |         course_id: "c1",
156 |         name: "Assignments",
157 |         weight_percent: 40,
158 |         sort_order: 0,
159 |         created_at: "",
160 |       },
161 |       {
162 |         id: "2",
163 |         course_id: "c1",
164 |         name: "Quizzes",
165 |         weight_percent: 30,
166 |         sort_order: 1,
167 |         created_at: "",
168 |       },
169 |       {
170 |         id: "3",
171 |         course_id: "c1",

```

```

172 |         name: "Final",
173 |         weight_percent: 30,
174 |         sort_order: 2,
175 |         created_at: "",
176 |     },
177 | ];
178 | const total = categories.reduce((sum, c) => sum + c.weight_percent, 0);
179 | expect(total).toBe(100);
180 | });
181 |
182 | it("weights not summing to 100 is invalid", () => {
183 |     const categories: GradeCategory[] = [
184 |         {
185 |             id: "1",
186 |             course_id: "c1",
187 |             name: "Assignments",
188 |             weight_percent: 40,
189 |             sort_order: 0,
190 |             created_at: "",
191 |         },
192 |         {
193 |             id: "2",
194 |             course_id: "c1",
195 |             name: "Quizzes",
196 |             weight_percent: 30,
197 |             sort_order: 1,
198 |             created_at: "",
199 |         },
200 |     ];
201 |     const total = categories.reduce((sum, c) => sum + c.weight_percent, 0);
202 |     expect(total).not.toBe(100);
203 |     expect(total).toBe(70);
204 | });
205 |
206 | it("adding a category that exceeds 100 should be rejected", () => {
207 |     const existingTotal = 80;
208 |     const newWeight = 25;
209 |     expect(existingTotal + newWeight).toBeGreaterThan(100);
210 | });
211 | });
212 |

```

FILE: [src/__tests__/unit/useIntentPrefetch.test.ts](#)

TYPE: .ts | LINES: 1-90 | SHA256: 9f7eb71be103926365c08ba796c7f4a708221934b5745102521f9a21c930556d

```

1 | // @vitest-environment happy-dom
2 | // =====
3 | // useIntentPrefetch.test.ts
4 | // Feature: dashboard-and-ux-performance, Req 9 (prefetch on intent).
5 | // -----
6 | // Locks in the guard contract of the hover/focus prefetch handler:
7 | //   - Req 9.1: hover/focus on a hover-capable (desktop) pointer warms the target.
8 | //   - Req 9.2: NO prefetch on touch (matchMedia('(hover: hover)') === false).
9 | //   - Req 9.3: failures are silent no-ops; targets warm at most once (dedupe).
10 | // =====
11 | import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
12 | import { renderHook } from "@testing-library/react";
13 | import { useIntentPrefetch } from "@hooks/useIntentPrefetch";
14 |
15 | const setHover = (matches: boolean) => {
16 |     window.matchMedia = vi.fn().mockImplementation((query: string) => ({
17 |         matches: query === "(hover: hover)" ? matches : false,
18 |         media: query,
19 |         addEventListener: vi.fn(),
20 |         removeEventListener: vi.fn(),
21 |         addListener: vi.fn(),
22 |         removeListener: vi.fn(),
23 |         onchange: null,
24 |         dispatchEvent: vi.fn(),
25 |     }));
26 | };
27 |
28 | describe("useIntentPrefetch (dashboard-and-ux-performance Req 9)", () => {
29 |     beforeEach(() => {
30 |         vi.clearAllMocks();
31 |     });
32 |     afterEach(() => {
33 |         vi.restoreAllMocks();
34 |     });
35 |
36 |     it("warms the target on hover (desktop) exactly once, then dedupes", () => {
37 |         setHover(true);
38 |         const prefetch = vi.fn();
39 |         const { result } = renderHook(() => useIntentPrefetch());
40 |
41 |         const handlers = result.current("/admin/users", prefetch);
42 |         handlers.onMouseEnter();
43 |         handlers.onFocus(); // same key !' already warmed
44 |         result.current("/admin/users", prefetch).onMouseEnter(); // rebuilt handler, same key
45 |
46 |         expect(prefetch).toHaveBeenCalledTimes(1);
47 |     });
48 |
49 |     it("does NOT prefetch on a touch device (no hover capability)", () => {
50 |         setHover(false);
51 |         const prefetch = vi.fn();
52 |         const { result } = renderHook(() => useIntentPrefetch());
53 |
54 |         result.current("/admin/users", prefetch).onMouseEnter();
55 |         result.current("/admin/users", prefetch).onFocus();
56 |
57 |         expect(prefetch).not.toHaveBeenCalled();
58 |     });
59 |

```

FILE: src/__tests__/unit/useJournal.test.ts

[illegible]

```
70 describe("useJournalEntries queryFn", () => {  
71   it("queries journal_entries table scoped to student and ordered by created_at desc", () => {  
72     supabase.from("journal_entries");  
73  
74     expect(supabase.from).toHaveBeenCalledWith("journal_entries");  
75   });  
76  
77   it("calls select with explicit columns and applies student_id filter", () => {  
78     const chain = supabase.from("journal_entries");  
79     chain.select(  
80       "id, student_id, course_id, content, clo_id, is_shared, created_at"  
81     );  
82     chain.eq("student_id", "student-user-id");  
83     chain.order("created_at", { ascending: false });  
84  
85     expect(mockSelect).toHaveBeenCalledWith(  
86       "id, student_id, course_id, content, clo_id, is_shared, created_at"  
87     );  
88     expect(mockEq).toHaveBeenCalledWith("student_id", "student-user-id");  
89     expect(mockOrder).toHaveBeenCalledWith("created_at", {  
90       ascending: false,  
91     });  
92   });  
93  
94   it("applies course_id filter when provided", () => {  
95     const chain = supabase.from("journal_entries");  
96     chain.select(  
97       "id, student_id, course_id, content, clo_id, is_shared, created_at"  
98     );  
99     chain.eq("student_id", "student-user-id");  
100    chain.order("created_at", { ascending: false });  
101    chain.eq("course_id", "course-abc");  
102  
103    expect(mockEq).toHaveBeenCalledWith("course_id", "course-abc");  
104  });  
105});  
106// % % % useJournalEntry queryFn % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
107describe("useJournalEntry queryFn", () => {  
108  it("queries journal_entries by id with maybeSingle scoped to student", async () => {  
109    const id = "entry-abc";  
110    const mockEntry = {  
111      id,  
112      student_id: "student-user-id",  
113      course_id: "course-1",  
114      content: "My reflection...",  
115      clo_id: null,  
116      is_shared: false,  
117      created_at: "2025-01-01T00:00:00Z",  
118    };  
119    mockMaybeSingle.mockResolvedValue({ data: mockEntry, error: null });  
120  
121    const chain = supabase.from("journal_entries");  
122    chain.select(  
123      "id, student_id, course_id, content, clo_id, is_shared, created_at"  
124    );  
125    chain.eq("id", id);  
126    chain.eq("student_id", "student-user-id");  
127    const result = await chain.maybeSingle();  
128  
129    expect(supabase.from).toHaveBeenCalledWith("journal_entries");  
130    expect(mockEq).toHaveBeenCalledWith("id", id);  
131    expect(mockEq).toHaveBeenCalledWith("student_id", "student-user-id");  
132    expect(mockMaybeSingle).toHaveBeenCalled();  
133    expect(result.data).toEqual(mockEntry);  
134  });  
135});  
136// % % % useCreateJournalEntry mutationFn % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
137describe("useCreateJournalEntry mutationFn", () => {  
138  it("inserts into journal_entries and logs audit event", async () => {  
139    const input = {  
140      course_id: "course-1",  
141      content:  
142        "This is a long enough journal entry for testing purposes with enough words.",  
143      clo_id: "clo-1",  
144      is_shared: false,  
145    };  
146  
147    const createdEntry = {  
148      id: "created-entry-id",  
149      student_id: "student-user-id",  
150      ...input,  
151    };  
152    mockSingle.mockResolvedValue({ data: createdEntry, error: null });  
153  
154    const chain = supabase.from("journal_entries");  
155    chain.insert({  
156      student_id: "student-user-id",  
157      course_id: input.course_id,  
158      content: input.content,  
159      clo_id: input.clo_id,  
160      is_shared: false,  
161    });  
162    chain.select();  
163    const { data, error } = await chain.single();  
164  
165    expect(error).toBeNull();  
166    expect(data).toEqual(expect.objectContaining({ id: "created-entry-id" }));  
167    expect(mockInsert).toHaveBeenCalledWith({  
168      student_id: "student-user-id",
```

[illegible]


```

43   order: () => builder,
44   range: (from: number, to: number) => {
45     rangeFrom = from;
46     rangeTo = to;
47     return builder;
48   },
49   then: (
50     resolve: (value: {
51       data: Array<Record<string, unknown>> | null;
52       error: Error | null;
53       count: number | null;
54     }) => void
55   ) =>
56     // Resolve on a microtask (like a real network call) so React Query goes
57     // through proper loading! 'success' transitions and notifies observers;
58     // a synchronous resolve suppresses the re-render renderHook observes.
59     Promise.resolve().then(() => {
60       if (state.itemsError) {
61         resolve({ data: null, error: state.itemsError, count: null });
62         return;
63       }
64       const filtered = categoryFilter
65         ? state.items.filter((i) => i.category === categoryFilter)
66         : state.items;
67       const slice = filtered.slice(rangeFrom, rangeTo + 1);
68       resolve({ data: slice, error: null, count: filtered.length });
69     }),
70   );
71   return builder;
72 };
73
74 // Builder for the `sale_event_items` lookup used to compute discounts.
75 const makeSaleBuilder = () => {
76   const builder = {
77     select: () => builder,
78     in: () => builder,
79     filter: () => builder,
80     then: (
81       resolve: (value: {
82         data: Array<Record<string, unknown>> | null;
83         error: Error | null;
84       }) => void
85     ) =>
86       Promise.resolve().then(() =>
87         resolve({ data: state.saleData, error: state.saleError })
88       ),
89   };
90   return builder;
91 };
92
93 vi.mock("@/lib/supabase", () => ({
94   supabase: {
95     from: vi.fn((table: string) =>
96       table === "sale_event_items" ? makeSaleBuilder() : makeItemsBuilder()
97     ),
98   },
99 }));
100
101 // %% Test harness %%%%%%%%%%%%%%%
102
103 import { useMarketplaceItems } from "@/hooks/useMarketplace";
104 import type { MarketplaceItemsPage } from "@/hooks/useMarketplace";
105
106 const createWrapper = () => {
107   const queryClient = new QueryClient({
108     // `notifyOnChangeProps: "all"` forces the observer to re-render on any
109     // cache change. Without it, React Query's tracked-props optimization skips
110     // re-rendering when only `data` grows (status stays "success") – which
111     // happens on `fetchNextPage` – because `renderHook` stores the whole result
112     // object without deconstructing tracked props during render.
113     defaultOptions: {
114       queries: { retry: false, notifyOnChangeProps: "all" },
115     },
116   });
117   return ({ children }: { children: ReactNode }) =>
118     createElement(QueryClientProvider, { client: queryClient }, children);
119 };
120
121 const makeItem = (
122   i: number,
123   overrides: Record<string, unknown> = {}
124 ): Record<string, unknown> => ({
125   id: `item-${i}`,
126   institution_id: "inst-1",
127   name: `Item ${i}`,
128   description: "desc",
129   category: "cosmetic",
130   sub_category: "avatar",
131   xp_price: 100,
132   level_requirement: 1,
133   stock_type: "unlimited",
134   stock_quantity: null,
135   icon_identifier: "icon",
136   metadata: {},
137   is_active: true,
138   created_at: "2024-01-01T00:00:00Z",
139   updated_at: "2024-01-01T00:00:00Z",
140   ...overrides,
141 });
142
143 beforeEach(() => {
144   state.items = [];
145   state.itemsError = null;
146   state.saleData = [];

```

[illegible]

```

251   await waitFor(() => expect(result.current.isSuccess).toBe(true));
252   const firstPage = pageAt(result.current.data?.pages, 0);
253   expect(firstPage.items).toHaveLength(0);
254   expect(firstPage.total).toBe(0);
255   expect(firstPage.hasMore).toBe(false);
256   expect(result.current.hasNextPage).toBe(false);
257 });
258
259 it("applies the highest active sale discount to effective price", async () => {
260   state.items = [makeItem(0, { id: "item-0", xp_price: 100 })];
261   state.saleData = [
262     {
263       item_id: "item-0",
264       sale_event_id: "sale-1",
265       sale_events: { discount_percentage: 25 },
266     },
267   ];
268   const { result } = renderHook(() => useMarketplaceItems(undefined, 24), {
269     wrapper: createWrapper(),
270   });
271
272   await waitFor(() => expect(result.current.isSuccess).toBe(true));
273   const item = pageAt(result.current.data?.pages, 0).items[0];
274   expect(item?.sale_discount).toBe(25);
275   expect(item?.effective_price).toBe(75);
276 });
277
278 it("surfaces a query error", async () => {
279   state.itemsError = new Error("boom");
280   const { result } = renderHook(() => useMarketplaceItems(undefined, 24), {
281     wrapper: createWrapper(),
282   });
283
284   await waitFor(() => expect(result.current.isError).toBe(true));
285   expect(result.current.error).toBeInstanceOf(Error);
286 });
287 });
288

```

FILE: src/__tests__/unit/useMasteryRecovery.test.ts

```
TYPE: .ts | LINES: 1-399 | SHA256: d468b51bcc8a4b96674465075697e7fc1d5b4d255edaa52858894d720fa394e4
```

[illegible]

```
63 mockEq.mockReturnValue(chainObj);  
64 mockSingle.mockResolvedValue({ data: sampleRecovery, error: null });  
65 mockMaybeSingle.mockResolvedValue({ data: sampleRecovery, error: null });  
66 }));  
67  
68 // % % useMasteryRecoveryStatus queryFn % % % % % % % % % % % % % % % % % % % % % %  
69  
70 describe("useMasteryRecoveryStatus queryFn", () => {  
71   it("queries mastery_recovery_pathways for active status by student and CLO", async () => {  
72     mockMaybeSingle.mockResolvedValue({ data: sampleRecovery, error: null });  
73  
74     supabase.from("mastery_recovery_pathways");  
75     chainObj.select("*");  
76     chainObj.eq("student_id", "student-1");  
77     chainObj.eq("clo_id", "clo-1");  
78     chainObj.eq("status", "active");  
79     const result = await chainObj.maybeSingle();  
80  
81     expect(supabase.from).toHaveBeenCalledWith("mastery_recovery_pathways");  
82     expect(mockSelect).toHaveBeenCalledWith("");  
83     expect(mockEq).toHaveBeenCalledWith("student_id", "student-1");  
84     expect(mockEq).toHaveBeenCalledWith("clo_id", "clo-1");  
85     expect(mockEq).toHaveBeenCalledWith("status", "active");  
86     expect(result.data).toEqual(sampleRecovery);  
87   });  
88  
89   it("returns null when no active recovery exists", async () => {  
90     mockMaybeSingle.mockResolvedValue({ data: null, error: null });  
91  
92     const result = await chainObj.maybeSingle();  
93     expect(result.data).toBeNull();  
94   });  
95  
96   it("returns error on supabase failure", async () => {  
97     mockMaybeSingle.mockResolvedValue(  
98       data: null,  
99       error: { message: "RLS denied" },  
100    );  
101  
102    const result = await chainObj.maybeSingle();  
103    expect(result.error).toBeTruthy();  
104    expect(result.error.message).toBe("RLS denied");  
105  });  
106});  
  
107// % % useRecoveryPathway queryFn % % % % % % % % % % % % % % % % % % % % % % % % % %  
108  
109describe("useRecoveryPathway queryFn", () => {  
110  it("queries a specific recovery pathway by id", async () => {  
111    mockMaybeSingle.mockResolvedValue({ data: sampleRecovery, error: null });  
112  
113    supabase.from("mastery_recovery_pathways");  
114    chainObj.select("*");  
115    chainObj.eq("id", "rec-1");  
116    const result = await chainObj.maybeSingle();  
117  
118    expect(supabase.from).toHaveBeenCalledWith("mastery_recovery_pathways");  
119    expect(mockEq).toHaveBeenCalledWith("id", "rec-1");  
120    expect(result.data).toEqual(sampleRecovery);  
121  });  
122  
123  it("returns null when recovery pathway not found", async () => {  
124    mockMaybeSingle.mockResolvedValue({ data: null, error: null });  
125  
126    const result = await chainObj.maybeSingle();  
127    expect(result.data).toBeNull();  
128  });  
129});  
  
130// % % useActivateRecovery mutationFn % % % % % % % % % % % % % % % % % % % % % %  
131  
132describe("useActivateRecovery mutationFn", () => {  
133  it("inserts a new recovery pathway and returns created row", async () => {  
134    mockSingle.mockResolvedValue({ data: sampleRecovery, error: null });  
135  
136    supabase.from("mastery_recovery_pathways");  
137    chainObj.insert({  
138      institution_id: "inst-1",  
139      student_id: "student-1",  
140      clo_id: "clo-1",  
141      course_id: "course-1",  
142      failure_count: 2,  
143      status: "active",  
144    });  
145    chainObj.select();  
146    const result = await chainObj.single();  
147  
148    expect(supabase.from).toHaveBeenCalledWith("mastery_recovery_pathways");  
149    expect(mockInsert).toHaveBeenCalledWith(  
150      institution_id: "inst-1",  
151      student_id: "student-1",  
152      clo_id: "clo-1",  
153      course_id: "course-1",  
154      failure_count: 2,  
155      status: "active",  
156    );  
157    expect(mockSelect).toHaveBeenCalled();  
158    expect(result.data).toEqual(sampleRecovery);  
159  });  
160  
161  it("throws on unique constraint violation (active session already exists)", async () => {  
162    mockSingle.mockResolvedValue(  
163      data: null,  
164      error: { message: "duplicate key value violates unique constraint" },  
165    )
```

[illegible]

[illegible]

[illegible]

```
FILE: src/  tests  /unit/useOptimisticXP.test.ts
```

TYPE: .ts | LINES: 1-178 | SHA256: 733cfeffc5cb51a8dc095e9273784d83d1d6ae5d575bd83aa13f4f7d321fc0645

```

1 | // @vitest-environment happy-dom
2 | // =====
3 | // useOptimisticXP – Unit tests
4 | // Feature: edeviser-platform, Task 54.7
5 | // =====
6 |
7 | import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
8 | import { renderHook, act } from "@testing-library/react";
9 | import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
10 | import { createElement, type ReactNode } from "react";
11 | import { useOptimisticXP, useOptimisticStreak } from "@/hooks/useOptimisticXP";

```

```

12 | import { queryKeys } from "@lib/queryKeys";
13 | import type { StudentKPIData } from "@hooks/useStudentDashboard";
14 | import type { LevelData } from "@hooks/useLevel";
15 |
16 | // Mock the xpClient
17 | vi.mock("@lib/xpClient", () => ({
18 |   awardXP: vi.fn().mockResolvedValue({
19 |     success: true,
20 |     xp_awarded: 25,
21 |     new_total: 125,
22 |     level_up: false,
23 |     new_level: 2,
24 |   }),
25 | }));
26 |
27 | const STUDENT_ID = "student-1";
28 |
29 | const createWrapper = (queryClient: QueryClient) => {
30 |   return ({ children }: { children: ReactNode }) =>
31 |     createElement(QueryClientProvider, { client: queryClient }, children);
32 | };
33 |
34 | describe("useOptimisticXP", () => {
35 |   let queryClient: QueryClient;
36 |
37 |   beforeEach(() => {
38 |     vi.clearAllMocks();
39 |     queryClient = new QueryClient({
40 |       defaultOptions: { queries: { retry: false } },
41 |     });
42 |   });
43 |
44 |   it("optimistically updates KPI cache with new XP amount", async () => {
45 |     const kpiKey = queryKeys.studentGamification.detail(STUDENT_ID);
46 |     const initialKPI: StudentKPIData = {
47 |       enrolledCourses: 3,
48 |       completedAssignments: 5,
49 |       avgAttainment: 72,
50 |       currentStreak: 7,
51 |       currentLevel: 2,
52 |       totalXP: 100,
53 |       totalActiveDays: 0,
54 |     };
55 |     queryClient.setQueryData(kpiKey, initialKPI);
56 |
57 |     const { result } = renderHook(() => useOptimisticXP(), {
58 |       wrapper: createWrapper(queryClient),
59 |     });
60 |
61 |     await act(async () => {
62 |       await result.current.awardXPOptimistic({
63 |         studentId: STUDENT_ID,
64 |         xpAmount: 25,
65 |         source: "submission",
66 |       });
67 |     });
68 |
69 |     // KPI should have been optimistically updated
70 |     const updatedKPI = queryClient.getQueryData<StudentKPIData>(kpiKey);
71 |     expect(updatedKPI?.totalXP).toBe(125);
72 |   });
73 |
74 |   it("optimistically updates Level cache", async () => {
75 |     const levelKey = queryKeys.studentGamification.list({
76 |       scope: "level",
77 |       studentId: STUDENT_ID,
78 |     });
79 |     const initialLevel: LevelData = {
80 |       level: 2,
81 |       title: "Explorer",
82 |       xpTotal: 100,
83 |       xpForCurrentLevel: 100,
84 |       xpForNextLevel: 250,
85 |       progressPercent: 0,
86 |     };
87 |     queryClient.setQueryData(levelKey, initialLevel);
88 |
89 |     const { result } = renderHook(() => useOptimisticXP(), {
90 |       wrapper: createWrapper(queryClient),
91 |     });
92 |
93 |     await act(async () => {
94 |       await result.current.awardXPOptimistic({
95 |         studentId: STUDENT_ID,
96 |         xpAmount: 50,
97 |         source: "submission",
98 |       });
99 |     });
100 |
101 |     const updatedLevel = queryClient.getQueryData<LevelData>(levelKey);
102 |     expect(updatedLevel?.xpTotal).toBe(150);
103 |   });
104 |
105 |   it("rolls back on failure", async () => {
106 |     const { awardXP } = await import("@lib/xpClient");
107 |     (awardXP as ReturnType<typeof vi.fn>).mockResolvedValueOnce(null);
108 |
109 |     const kpiKey = queryKeys.studentGamification.detail(STUDENT_ID);
110 |     const initialKPI: StudentKPIData = {
111 |       enrolledCourses: 3,
112 |       completedAssignments: 5,
113 |       avgAttainment: 72,
114 |       currentStreak: 7,
115 |       currentLevel: 2,

```



```

38 |     expect(mockLogActivity).not.toHaveBeenCalled();
39 | });
40 |
41 | it("logs page_view with duration_seconds when student navigates away", () => {
42 |   mockProfile = { id: "student-1", role: "student" };
43 |   const { rerender } = renderHook(() => usePageViewLogger());
44 |
45 |   // Navigate to a new page
46 |   mockPathname = "/student/assignments";
47 |   rerender();
48 |
49 |   expect(mockLogActivity).toHaveBeenCalledTimes(1);
50 |   expect(mockLogActivity).toHaveBeenCalledWith(
51 |     expect.objectContaining({
52 |       student_id: "student-1",
53 |       event_type: "page_view",
54 |       metadata: expect.objectContaining({
55 |         path: "/student/dashboard",
56 |         duration_seconds: expect.any(Number),
57 |       }),
58 |     }),
59 |   );
60 | });
61 |
62 | it("does not log when profile is null", () => {
63 |   mockProfile = null;
64 |   renderHook(() => usePageViewLogger());
65 |
66 |   expect(mockLogActivity).not.toHaveBeenCalled();
67 | });
68 |
69 | it("does not log for non-student roles", () => {
70 |   mockProfile = { id: "teacher-1", role: "teacher" };
71 |   const { rerender } = renderHook(() => usePageViewLogger());
72 |
73 |   mockPathname = "/teacher/courses";
74 |   rerender();
75 |
76 |   expect(mockLogActivity).not.toHaveBeenCalled();
77 | });
78 |
79 | it("does not log duplicate for same pathname", () => {
80 |   mockProfile = { id: "student-1", role: "student" };
81 |   const { rerender } = renderHook(() => usePageViewLogger());
82 |
83 |   // Re-render with same pathname
84 |   rerender();
85 |   expect(mockLogActivity).not.toHaveBeenCalled();
86 | });
87 | });
88 |

```

FILE: [src/__tests__/unit/useParentAttendanceOverview.test.ts](#)

TYPE: .ts | LINES: 1-188 | SHA256: 4b20b71dc14282fa712dc18ab7a5629beee0263f672c41911cdc4cf626c5db49

```

1 | import { describe, it, expect } from "vitest";
2 | import {
3 |   buildParentAttendanceOverview,
4 |   type RawAttendanceRecordItem,
5 | } from "@/hooks/useAttendance";
6 |
7 | describe("buildParentAttendanceOverview – Canonical Formula & Calculation Verification", () => {
8 |   it("correctly calculates attendance metrics for the Section 13 primary fixture (115 Present, 1 Late, 4 Absen
9 |     const child = { id: "student-1", name: "Aarav" };
10 |     const courses = [
11 |       { courseId: "math-6", code: "MATH6", name: "Mathematics 6" },
12 |       { courseId: "eng-7", code: "ENG7", name: "English 7" },
13 |       { courseId: "soc-7", code: "SOC7", name: "Social Studies 7" },
14 |       { courseId: "sci-8", code: "SCI8", name: "Science 8" },
15 |     ];
16 |
17 |     const records: RawAttendanceRecordItem[] = [];
18 |
19 |     // Math 6: 26 present, 1 late, 3 absent (30 total)
20 |     for (let i = 0; i < 26; i++) {
21 |       records.push({
22 |         id: `m-p-${i}`,
23 |         session_id: `m-sess-${i}`,
24 |         status: "present",
25 |         created_at: "2026-04-10T08:00:00Z",
26 |         class_sessions: {
27 |           id: `m-sess-${i}`,
28 |           session_date: "2026-04-10",
29 |           session_type: "lecture",
30 |           course_sections: {
31 |             id: "sec-m",
32 |             course_id: "math-6",
33 |             courses: { id: "math-6", code: "MATH6", name: "Mathematics 6" },
34 |           },
35 |         },
36 |       });
37 |     }
38 |     records.push({
39 |       id: "m-l-1",
40 |       session_id: "m-sess-late",
41 |       status: "late",
42 |       created_at: "2026-04-11T08:00:00Z",
43 |       class_sessions: {
44 |         id: "m-sess-late",
45 |         session_date: "2026-04-11",
46 |         session_type: "lecture",
47 |         course_sections: {
48 |           id: "sec-m",
49 |           course_id: "math-6",

```

```

50 |         courses: { id: "math-6", code: "MATH6", name: "Mathematics 6" },
51 |     },
52 | },
53 | });
54 | for (let i = 0; i < 3; i++) {
55 |     records.push({
56 |         id: `m-a-${i}`,
57 |         session_id: `m-sess-abs-${i}`,
58 |         status: "absent",
59 |         created_at: "2026-04-12T08:00:00Z",
60 |         class_sessions: {
61 |             id: `m-sess-abs-${i}`,
62 |             session_date: "2026-04-12",
63 |             session_type: "lecture",
64 |             course_sections: {
65 |                 id: "sec-m",
66 |                 course_id: "math-6",
67 |                 courses: { id: "math-6", code: "MATH6", name: "Mathematics 6" },
68 |             },
69 |         },
70 |     });
71 | }
72 |
73 | // English 7: 30 present (30 total)
74 | for (let i = 0; i < 30; i++) {
75 |     records.push({
76 |         id: `e-p-${i}`,
77 |         session_id: `e-sess-${i}`,
78 |         status: "present",
79 |         created_at: "2026-04-15T08:00:00Z",
80 |         class_sessions: {
81 |             id: `e-sess-${i}`,
82 |             session_date: "2026-04-15",
83 |             session_type: "lecture",
84 |             course_sections: {
85 |                 id: "sec-e",
86 |                 course_id: "eng-7",
87 |                 courses: { id: "eng-7", code: "ENG7", name: "English 7" },
88 |             },
89 |         },
90 |     });
91 | }
92 |
93 | // Social Studies 7: 30 present (30 total)
94 | for (let i = 0; i < 30; i++) {
95 |     records.push({
96 |         id: `soc-p-${i}`,
97 |         session_id: `soc-sess-${i}`,
98 |         status: "present",
99 |         created_at: "2026-04-18T08:00:00Z",
100 |         class_sessions: {
101 |             id: `soc-sess-${i}`,
102 |             session_date: "2026-04-18",
103 |             session_type: "lecture",
104 |             course_sections: {
105 |                 id: "sec-soc",
106 |                 course_id: "soc-7",
107 |                 courses: { id: "soc-7", code: "SOC7", name: "Social Studies 7" },
108 |             },
109 |         },
110 |     });
111 | }
112 |
113 | // Science 8: 29 present, 1 absent (30 total)
114 | for (let i = 0; i < 29; i++) {
115 |     records.push({
116 |         id: `sci-p-${i}`,
117 |         session_id: `sci-sess-${i}`,
118 |         status: "present",
119 |         created_at: "2026-04-20T08:00:00Z",
120 |         class_sessions: {
121 |             id: `sci-sess-${i}`,
122 |             session_date: "2026-04-20",
123 |             session_type: "lab",
124 |             course_sections: {
125 |                 id: "sec-sci",
126 |                 course_id: "sci-8",
127 |                 courses: { id: "sci-8", code: "SCI8", name: "Science 8" },
128 |             },
129 |         },
130 |     });
131 | }
132 | records.push({
133 |     id: "sci-a-1",
134 |     session_id: "sci-sess-abs-1",
135 |     status: "absent",
136 |     created_at: "2026-04-21T08:00:00Z",
137 |     class_sessions: {
138 |         id: "sci-sess-abs-1",
139 |         session_date: "2026-04-21",
140 |         session_type: "lab",
141 |         course_sections: {
142 |             id: "sec-sci",
143 |             course_id: "sci-8",
144 |             courses: { id: "sci-8", code: "SCI8", name: "Science 8" },
145 |         },
146 |     },
147 | });
148 |
149 | const overview = buildParentAttendanceOverview(child, courses, records);
150 |
151 | // Verify overall totals
152 | expect(overview.totals.totalSessions).toBe(120);
153 | expect(overview.totals.present).toBe(115);

```

```

154 |     expect(overview.totals.late).toBe(1);
155 |     expect(overview.totals.absent).toBe(4);
156 |     expect(overview.totals.attended).toBe(116);
157 |
158 |     // Formula: (115 + 1) / 120 * 100 = 96.666... -> 97%
159 |     expect(overview.totals.attendanceRate).toBe(97);
160 |
161 |     // Mathematics course-level metrics
162 |     const math = overview.courses.find((c) => c.courseId === "math-6");
163 |     expect(math).toBeDefined();
164 |     expect(math?.totalSessions).toBe(30);
165 |     expect(math?.present).toBe(26);
166 |     expect(math?.late).toBe(1);
167 |     expect(math?.absent).toBe(3);
168 |     // Formula: (26 + 1) / 30 * 100 = 90%
169 |     expect(math?.attendanceRate).toBe(90);
170 |
171 |     // Verify Attention card targets Mathematics 6 (3 of 4 missed sessions)
172 |     expect(overview.attention).toBeDefined();
173 |     expect(overview.attention?.courseId).toBe("math-6");
174 |     expect(overview.attention?.absenceCount).toBe(3);
175 |   });
176 |
177 |   it("handles zero total sessions gracefully without NaN errors", () => {
178 |     const child = { id: "student-empty", name: "EmptyChild" };
179 |     const overview = buildParentAttendanceOverview(child, [], []);
180 |
181 |     expect(overview.totals.totalSessions).toBe(0);
182 |     expect(overview.totals.attendanceRate).toBe(100);
183 |     expect(overview.totals.punctualityRate).toBe(100);
184 |     expect(overview.totals.absenceRate).toBe(0);
185 |     expect(overview.attention).toBeUndefined();
186 |   });
187 | });
188 |

```

FILE: [src/__tests__/unit/useParentDashboardAggregate.test.ts](#)

TYPE: .ts | LINES: 1-156 | SHA256: 561afcc0d334e93a0f5e2964c4b876147ad57b8d7c24f61fd2e35dcefd9e0894

```

1 | // @vitest-environment happy-dom
2 | // =====
3 | // useParentDashboardAggregate.test.ts
4 | // Feature: dashboard-and-ux-performance, Phase 8 Task 36 (parent aggregate)
5 | // -----
6 | // 1. CALL – get_parent_dashboard is called once.
7 | // 2. HYDRATION – the EXACT caches useParentKPIs AND useLinkedChildren read are
8 | //    both populated (kpis !' parentDashboard.detail; children !' parentStudentLinks.list).
9 | // 3. COLLAPSE – with both section hooks gated on `aggregate.isError`, a SUCCESSFUL
10 | //    aggregate fires ZERO section `supabase.from` requests; a FAILED aggregate
11 | //    makes the section hooks fall back and fetch (resolving verified children).
12 | // The `get_parent_dashboard` RPC is mocked; this is a hermetic unit test.
13 | // =====
14 | import { describe, it, expect, vi, beforeEach } from "vitest";
15 | import { renderHook, waitFor } from "@testing-library/react";
16 | import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
17 | import { createElement, type ReactNode } from "react";
18 |
19 | const mockRpc = vi.fn();
20 | const mockFrom = vi.fn();
21 |
22 | vi.mock("@/lib/supabase", () => ({
23 |   supabase: {
24 |     rpc: (...args: unknown[]) => mockRpc(...args),
25 |     from: (...args: unknown[]) => mockFrom(...args),
26 |   },
27 | }));
28 |
29 | const makeFromBuilder = () => {
30 |   const builder: Record<string, unknown> = {
31 |     select: () => builder,
32 |     eq: () => builder,
33 |     in: () => builder,
34 |     gte: () => builder,
35 |     then: (resolve: (value: unknown) => unknown) =>
36 |       resolve({ data: [], count: 0, error: null }),
37 |   };
38 |   return builder;
39 | };
40 |
41 | import { queryKeys } from "@/lib/queryKeys";
42 | import {
43 |   useParentDashboardAggregate,
44 |   type ParentDashboardAggregate,
45 | } from "@/hooks/useParentDashboardAggregate";
46 | import { useParentKPIs, useLinkedChildren } from "@/hooks/useParentDashboard";
47 |
48 | const PARENT_ID = "66f40d1b-8012-4352-b23e-6a65fbdf846d";
49 |
50 | const FIXTURE: ParentDashboardAggregate = {
51 |   kpis: {
52 |     linkedChildren: 1,
53 |     totalCourses: 3,
54 |     avgAttainment: 92,
55 |     upcomingDeadlines: 0,
56 |   },
57 |   children: [
58 |     {
59 |       student_id: "00699cf9-6ab1-487c-8652-179757eb6462",
60 |       student_name: "Yusuf Ahmadi",
61 |       current_level: 11,
62 |       xp_total: 1986,
63 |       current_streak: 3,
64 |       enrolled_courses: 3,
65 |       avg_attainment: 92,

```

```

66 |     },
67 |   ],
68 | };
69 |
70 | const useBlock = () => {
71 |   const aggregate = useParentDashboardAggregate(PARENT_ID);
72 |   const kpisHook = useParentKPIs(PARENT_ID, { enabled: aggregate.isError });
73 |   const childrenHook = useLinkedChildren(PARENT_ID, {
74 |     enabled: aggregate.isError,
75 |   });
76 |   return { aggregate, kpisHook, childrenHook };
77 | };
78 |
79 | const makeClient = () =>
80 |   new QueryClient({ defaultOptions: { queries: { retry: false } } });
81 |
82 | const makeWrapper = (client: QueryClient) => {
83 |   const wrapper = ({ children }: { children: ReactNode }) =>
84 |     createElement(QueryClientProvider, { client }, children);
85 |   return wrapper;
86 | };
87 |
88 | describe("useParentDashboardAggregate (Phase 8 Task 36)", () => {
89 |   beforeEach(() => {
90 |     vi.clearAllMocks();
91 |     mockFrom.mockImplementation(() => makeFromBuilder());
92 |   });
93 |
94 |   it("calls get_parent_dashboard once", async () => {
95 |     mockRpc.mockResolvedValue({ data: FIXTURE, error: null });
96 |     const client = makeClient();
97 |     const { result } = renderHook(
98 |       () => useParentDashboardAggregate(PARENT_ID),
99 |       { wrapper: makeWrapper(client) }
100 |     );
101 |     await waitFor(() => expect(result.current.isSuccess).toBe(true));
102 |     expect(mockRpc).toHaveBeenCalledTimes(1);
103 |     expect(mockRpc).toHaveBeenCalledWith("get_parent_dashboard", {});
104 |   });
105 |
106 |   it("hydrates the exact caches both parent section hooks read", async () => {
107 |     mockRpc.mockResolvedValue({ data: FIXTURE, error: null });
108 |     const client = makeClient();
109 |     const { result } = renderHook(
110 |       () => useParentDashboardAggregate(PARENT_ID),
111 |       { wrapper: makeWrapper(client) }
112 |     );
113 |     await waitFor(() => expect(result.current.isSuccess).toBe(true));
114 |     expect(
115 |       client.getQueryData(queryKeys.parentDashboard.detail(PARENT_ID))
116 |     ).toEqual(FIXTURE.kpis);
117 |     expect(
118 |       client.getQueryData(
119 |         queryKeys.parentStudentLinks.list({ parentId: PARENT_ID })
120 |       )
121 |     ).toEqual(FIXTURE.children);
122 |   });
123 |
124 |   it("collapse: a successful aggregate fires ZERO section requests", async () => {
125 |     mockRpc.mockResolvedValue({ data: FIXTURE, error: null });
126 |     const client = makeClient();
127 |     const { result } = renderHook(() => useBlock(), {
128 |       wrapper: makeWrapper(client),
129 |     });
130 |     await waitFor(() => expect(result.current.aggregate.isSuccess).toBe(true));
131 |     expect(mockRpc).toHaveBeenCalledTimes(1);
132 |     expect(mockFrom).not.toHaveBeenCalled();
133 |     await waitFor(() =>
134 |       expect(result.current.kpisHook.data).toEqual(FIXTURE.kpis)
135 |     );
136 |     await waitFor(() =>
137 |       expect(result.current.childrenHook.data).toEqual(FIXTURE.children)
138 |     );
139 |     expect(mockFrom).not.toHaveBeenCalled();
140 |   });
141 |
142 |   it("fallback: a failed aggregate makes the section hooks fetch", async () => {
143 |     mockRpc.mockResolvedValue({ data: null, error: { message: "boom" } });
144 |     const client = makeClient();
145 |     const { result } = renderHook(() => useBlock(), {
146 |       wrapper: makeWrapper(client),
147 |     });
148 |     await waitFor(() => expect(result.current.aggregate.isError).toBe(true));
149 |     await waitFor(() => expect(mockFrom).toHaveBeenCalledTimes(1));
150 |     // Both section hooks resolve verified children first via parent_student_links.
151 |     expect(mockFrom.mock.calls.map((c) => c[0])).toContain(
152 |       "parent_student_links"
153 |     );
154 |   });
155 | });
156 |

```

FILE: [src/__tests__/unit/usePortfolioSharing.test.tsx](#)

TYPE: .tsx | LINES: 1-219 | SHA256: 966b21e71d82d6177665f3fdaf682392bc75d84e6acbc2bd5fa77c57d2324524

```

1 | // @vitest-environment happy-dom
2 | // =====
3 | // usePortfolioSharingPermission + useTogglePortfolioPublic - Unit tests
4 | // Validates: Requirements 24.1, 24.2 (gate public sharing on school permission)
5 | // =====
6 |
7 | import { describe, it, expect, vi, beforeEach } from "vitest";
8 | import { renderHook, waitFor } from "@testing-library/react";
9 | import { QueryClient, QueryClientProvider } from "@tanstack/react-query";

```

[illegible]

[illegible]

FILE: src/___tests___/unit/useQuestionBank.test.ts

[illegible]

[illegible]

```
203 |         expect(result.error).toBeTruthy();
204 |         expect(result.error.message).toBe("Not found");
205 |     });
206 | });
207 | });
208 |
```

FILE: src/__tests__/unit/useReadHabitTimer.test.ts

```
TYPE: .ts | LINES: 1-203 | SHA256: 966a9896206118b27da4ff14e16b9e1487718edecde946fd63e3864fdef96942
```

[illegible]

```

95 |     expect(result.current.elapsedSeconds).toBe(0);
96 |     expect(mockMutate).not.toHaveBeenCalled();
97 | });
98 |
99 | it("increments elapsed seconds each tick", () => {
100 |   mockProfile = { id: "student-1", role: "student" };
101 |   const { result } = renderHook(() => useReadHabitTimer(defaultOptions));
102 |
103 |   act(() => {
104 |     vi.advanceTimersByTime(5_000);
105 |   });
106 |
107 |   expect(result.current.elapsedSeconds).toBe(5);
108 |   expect(result.current.isCompleted).toBe(false);
109 | });
110 |
111 | it("marks habit as completed and calls mutation at 30 seconds", () => {
112 |   mockProfile = { id: "student-1", role: "student" };
113 |   const { result } = renderHook(() => useReadHabitTimer(defaultOptions));
114 |
115 |   act(() => {
116 |     vi.advanceTimersByTime(30_000);
117 |   });
118 |
119 |   expect(result.current.isCompleted).toBe(true);
120 |   expect(result.current.elapsedSeconds).toBe(30);
121 |
122 |   // Should call mutation with student_id and habit_date
123 |   expect(mockMutate).toHaveBeenCalledWith(
124 |     expect.objectContaining({
125 |       student_id: "student-1",
126 |       habit_date: expect.stringMatching(/^\d{4}-\d{2}-\d{2}$/),
127 |     })
128 |   );
129 | });
130 |
131 | it("logs activity with duration_seconds when habit completes", () => {
132 |   mockProfile = { id: "student-1", role: "student" };
133 |   renderHook(() => useReadHabitTimer(defaultOptions));
134 |
135 |   act(() => {
136 |     vi.advanceTimersByTime(30_000);
137 |   });
138 |
139 |   expect(mockLogActivity).toHaveBeenCalledWith(
140 |     expect.objectContaining({
141 |       student_id: "student-1",
142 |       event_type: "page_view",
143 |       metadata: expect.objectContaining({
144 |         page_type: "assignment_detail",
145 |         page_id: "assignment-123",
146 |         duration_seconds: 30,
147 |         habit_completed: true,
148 |       })
149 |     })
150 |   );
151 | });
152 |
153 | it("only fires habit completion once even after continued viewing", () => {
154 |   mockProfile = { id: "student-1", role: "student" };
155 |   renderHook(() => useReadHabitTimer(defaultOptions));
156 |
157 |   act(() => {
158 |     vi.advanceTimersByTime(60_000);
159 |   });
160 |
161 |   // Should only call mutation once
162 |   expect(mockMutate).toHaveBeenCalledTimes(1);
163 |   expect(mockLogActivity).toHaveBeenCalledTimes(1);
164 | });
165 |
166 | it("logs partial duration on unmount if habit not completed", () => {
167 |   mockProfile = { id: "student-1", role: "student" };
168 |   const { unmount } = renderHook(() => useReadHabitTimer(defaultOptions));
169 |
170 |   act(() => {
171 |     vi.advanceTimersByTime(10_000);
172 |   });
173 |
174 |   unmount();
175 |
176 |   expect(mockLogActivity).toHaveBeenCalledWith(
177 |     expect.objectContaining({
178 |       student_id: "student-1",
179 |       event_type: "page_view",
180 |       metadata: expect.objectContaining({
181 |         duration_seconds: 10,
182 |         habit_completed: false,
183 |       })
184 |     })
185 |   );
186 | });
187 |
188 | it("does not log partial duration on unmount if habit was completed", () => {
189 |   mockProfile = { id: "student-1", role: "student" };
190 |   const { unmount } = renderHook(() => useReadHabitTimer(defaultOptions));
191 |
192 |   act(() => {
193 |     vi.advanceTimersByTime(30_000);
194 |   });
195 |
196 |   mockLogActivity.mockClear();
197 |   unmount();
198 |

```

```
199 |         // Should not log again on unmount since habit was already completed
200 |         expect(mockLogActivity).not.toHaveBeenCalled();
201 |     });
202 | });
203 |
```

FILE: src/__tests__/unit/useRealtime.test.ts

```
TYPE: .ts | LINES: 1-393 | SHA256: 7f1502c48f85feae82553df15fe7a0252af8b2f63dd823dee32da0b2f2183e43
```

[illegible]

```

96 |     })
97 |   );
98 |
99 |   expect(supabase.channel).toHaveBeenCalledWith("my_table:*:all");
100 | });
101 |
102 | it("starts with isLive = true", () => {
103 |   const onPayload = vi.fn();
104 |   const { result } = renderHook(() =>
105 |     useRealtime({ table: "test", onPayload })
106 |   );
107 |
108 |   expect(result.current.isLive).toBe(true);
109 | });
110 |
111 | it("sets isLive = true when subscription succeeds", () => {
112 |   const onPayload = vi.fn();
113 |   const { result } = renderHook(() =>
114 |     useRealtime({ table: "test", onPayload })
115 |   );
116 |
117 |   act(() => {
118 |     mockState.subscribeCallback?.("SUBSCRIBED");
119 |   });
120 |
121 |   expect(result.current.isLive).toBe(true);
122 | });
123 |
124 | it("sets isLive = false on CHANNEL_ERROR and starts polling", () => {
125 |   const pollingFn = vi.fn();
126 |   const onPayload = vi.fn();
127 |   const { result } = renderHook(() =>
128 |     useRealtime({
129 |       table: "test",
130 |       onPayload,
131 |       pollingFn,
132 |       pollingInterval: 30_000,
133 |     })
134 |   );
135 |
136 |   act(() => {
137 |     mockState.subscribeCallback?.("CHANNEL_ERROR");
138 |   });
139 |
140 |   expect(result.current.isLive).toBe(false);
141 |
142 |   // Advance time to trigger polling
143 |   act(() => {
144 |     vi.advanceTimersByTime(30_000);
145 |   });
146 |
147 |   expect(pollingFn).toHaveBeenCalled();
148 | });
149 |
150 | it("applies exponential backoff on reconnection: 1s, 2s, 4s", () => {
151 |   const onPayload = vi.fn();
152 |   renderHook(() => useRealtime({ table: "test", onPayload }));
153 |
154 |   // First error !' 1s backoff
155 |   act(() => {
156 |     mockState.subscribeCallback?.("CHANNEL_ERROR");
157 |   });
158 |   expect(channelObj.unsubscribe).not.toHaveBeenCalled();
159 |
160 |   act(() => {
161 |     vi.advanceTimersByTime(1000);
162 |   });
163 |   // After 1s, should have unsubscribed and re-subscribed
164 |   expect(channelObj.unsubscribe).toHaveBeenCalled();
165 |
166 |   // Reset for next error
167 |   channelObj.unsubscribe.mockClear();
168 |
169 |   // Second error !' 2s backoff
170 |   act(() => {
171 |     mockState.subscribeCallback?.("CHANNEL_ERROR");
172 |   });
173 |
174 |   act(() => {
175 |     vi.advanceTimersByTime(1999);
176 |   });
177 |   expect(channelObj.unsubscribe).not.toHaveBeenCalled();
178 |
179 |   act(() => {
180 |     vi.advanceTimersByTime(1);
181 |   });
182 |   expect(channelObj.unsubscribe).toHaveBeenCalled();
183 | });
184 |
185 | it("caps backoff at 30s", () => {
186 |   const onPayload = vi.fn();
187 |   renderHook(() => useRealtime({ table: "test", onPayload }));
188 |
189 |   // Simulate many errors to push backoff past 30s
190 |   for (let i = 0; i < 10; i++) {
191 |     act(() => {
192 |       mockState.subscribeCallback?.("CHANNEL_ERROR");
193 |     });
194 |     act(() => {
195 |       vi.advanceTimersByTime(30_000);
196 |     });
197 |   }
198 |
199 |   // The backoff should be capped at 30s (Math.min(1000 * 2^10, 30000) = 30000)

```

```

200 |     channelObj.unsubscribe.mockClear();
201 |     act(() => {
202 |       mockState.subscribeCallback?.("CHANNEL_ERROR");
203 |     });
204 |     act(() => {
205 |       vi.advanceTimersByTime(30_000);
206 |     });
207 |     expect(channelObj.unsubscribe).toHaveBeenCalled();
208 |   });
209 |
210 |   it("resets retry count on successful subscription", () => {
211 |     const onPayload = vi.fn();
212 |     renderHook(() => useRealtime({ table: "test", onPayload }));
213 |
214 |     // Error !' backoff
215 |     act(() => {
216 |       mockState.subscribeCallback?.("CHANNEL_ERROR");
217 |     });
218 |     act(() => {
219 |       vi.advanceTimersByTime(1000);
220 |     });
221 |
222 |     // Successful reconnection
223 |     act(() => {
224 |       mockState.subscribeCallback?.("SUBSCRIBED");
225 |     });
226 |
227 |     // Next error should start at 1s again (retry count reset)
228 |     channelObj.unsubscribe.mockClear();
229 |     act(() => {
230 |       mockState.subscribeCallback?.("CHANNEL_ERROR");
231 |     });
232 |     act(() => {
233 |       vi.advanceTimersByTime(1000);
234 |     });
235 |     expect(channelObj.unsubscribe).toHaveBeenCalled();
236 |   });
237 |
238 |   it("stops polling when subscription succeeds", () => {
239 |     const pollingFn = vi.fn();
240 |     const onPayload = vi.fn();
241 |     const { result } = renderHook(() =>
242 |       useRealtime({
243 |         table: "test",
244 |         onPayload,
245 |         pollingFn,
246 |         pollingInterval: 30_000,
247 |       })
248 |     );
249 |
250 |     // Error !' starts polling
251 |     act(() => {
252 |       mockState.subscribeCallback?.("CHANNEL_ERROR");
253 |     });
254 |     expect(result.current.isLive).toBe(false);
255 |
256 |     // Successful reconnection !' stops polling
257 |     act(() => {
258 |       mockState.subscribeCallback?.("SUBSCRIBED");
259 |     });
260 |     expect(result.current.isLive).toBe(true);
261 |
262 |     // Advance time – polling should NOT fire
263 |     pollingFn.mockClear();
264 |     act(() => {
265 |       vi.advanceTimersByTime(60_000);
266 |     });
267 |     expect(pollingFn).not.toHaveBeenCalled();
268 |   });
269 |
270 |   it("cleans up channel and timers on unmount", () => {
271 |     const onPayload = vi.fn();
272 |     const { unmount } = renderHook(() =>
273 |       useRealtime({ table: "test", onPayload })
274 |     );
275 |
276 |     unmount();
277 |
278 |     expect(channelObj.unsubscribe).toHaveBeenCalled();
279 |   });
280 |
281 |   it("invokes onPayload callback when realtime event fires", () => {
282 |     const onPayload = vi.fn();
283 |     renderHook(() => useRealtime({ table: "test", onPayload }));
284 |
285 |     const fakePayload = { eventType: "UPDATE", new: { id: "1" } };
286 |     act(() => {
287 |       mockState.onPayloadCallback?.(fakePayload);
288 |     });
289 |
290 |     expect(onPayload).toHaveBeenCalledWith(fakePayload);
291 |   });
292 |
293 |   it("handles TIMED_OUT status same as CHANNEL_ERROR", () => {
294 |     const pollingFn = vi.fn();
295 |     const onPayload = vi.fn();
296 |     const { result } = renderHook(() =>
297 |       useRealtime({ table: "test", onPayload, pollingFn })
298 |     );
299 |
300 |     act(() => {
301 |       mockState.subscribeCallback?.("TIMED_OUT");
302 |     });
303 |

```



```

304 expect(result.current.isLive).toBe(false);
305 });
306
307 // % % % Polling strategy (broad / institution-scoped data) % % % % % % % % % % % % % % % % % %
308
309 it("strategy 'poll' does NOT open a realtime channel", () => {
310   const onPayload = vi.fn();
311   const pollingFn = vi.fn();
312   renderHook(() =>
313     useRealtime({
314       table: "student_gamification",
315       event: "UPDATE",
316       onPayload,
317       pollingFn,
318       pollingInterval: 60_000,
319       strategy: "poll",
320     })
321   );
322
323   // No fan-out subscription is opened for inherently-broad data.
324   expect(supabase.channel).not.toHaveBeenCalled();
325 });
326
327 it("strategy 'poll' refreshes via pollingFn on the interval, not on mount", () => {
328   const onPayload = vi.fn();
329   const pollingFn = vi.fn();
330   renderHook(() =>
331     useRealtime({
332       table: "student_gamification",
333       onPayload,
334       pollingFn,
335       pollingInterval: 60_000,
336       strategy: "poll",
337     })
338   );
339
340   // The consuming query already fetches on mount, so the hook must NOT
341   // re-invalidate immediately – it only schedules subsequent refreshes.
342   expect(pollingFn).not.toHaveBeenCalled();
343
344   act(() => {
345     vi.advanceTimersByTime(60_000);
346   });
347   expect(pollingFn).toHaveBeenCalledTimes(1);
348
349   act(() => {
350     vi.advanceTimersByTime(60_000);
351   });
352   expect(pollingFn).toHaveBeenCalledTimes(2);
353 });
354
355 it("strategy 'poll' keeps isLive true and retryCount 0 (no disconnected state)", () => {
356   const onPayload = vi.fn();
357   const pollingFn = vi.fn();
358   const { result } = renderHook(() =>
359     useRealtime({
360       table: "teams",
361       onPayload,
362       pollingFn,
363       pollingInterval: 60_000,
364       strategy: "poll",
365     })
366   );
367
368   expect(result.current.isLive).toBe(true);
369   expect(result.current.retryCount).toBe(0);
370 });
371
372 it("strategy 'poll' clears its interval on unmount", () => {
373   const onPayload = vi.fn();
374   const pollingFn = vi.fn();
375   const { unmount } = renderHook(() =>
376     useRealtime({
377       table: "teams",
378       onPayload,
379       pollingFn,
380       pollingInterval: 60_000,
381       strategy: "poll",
382     })
383   );
384
385   unmount();
386   pollingFn.mockClear();
387   act(() => {
388     vi.advanceTimersByTime(120_000);
389   });
390   expect(pollingFn).not.toHaveBeenCalled();
391 });
392 });
393

```

FILE: src/__tests__/unit/useReviewSchedule.test.ts

TYPE: .ts | LINES: 1-447 | SHA256: d206770ed168a2ecd91c744c910217d6fba2106cec9df6428bf9470b3f79969c

```
1 | // @vitest-environment node  
2 | import { describe, it, expect, vi, beforeEach } from "vitest";  
3 |  
4 | // % % % Supabase mock % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
5 |  
6 | const mockMaybeSingle = vi.fn();  
7 | const mockSingle = vi.fn();  
8 | const mockSelect = vi.fn();  
9 | const mockInsert = vi.fn();  
10| const mockUpdate = vi.fn();
```

[illegible]

[illegible]

[illegible]

[illegible]

```

427 |     expect(weekEndDate).toBe("2025-07-06"); // Crosses into July
428 |   });
429 |
430 |
431 |   it("handles year boundary correctly", () => {
432 |     const weekStartDate = "2025-12-29"; // Monday
433 |     const [year, month, day] = weekStartDate.split("-").map(Number) as [
434 |       number,
435 |       number,
436 |       number
437 |     ];
438 |     const weekEnd = new Date(Date.UTC(year, month - 1, day + 6));
439 |     const weekEndDate = `${weekEnd.getUTCFullYear()}-${String(
440 |       weekEnd.getUTCMonth() + 1
441 |     ).padStart(2, "0")}-${String(weekEnd.getUTCDate()).padStart(2, "0")}`;
442 |     expect(weekEndDate).toBe("2026-01-04"); // Crosses into next year
443 |   });
444 | });
445 |
446 | });
447 |

```

FILE: [src/__tests__/unit/useSemesters.test.ts](#)

TYPE: .ts | LINES: 1-78 | SHA256: 23f5f61a85d9d236b33d019a575e8f35e0f2625c511691ae3b356651e1c6dab2

```

1 | import { describe, it, expect, vi } from "vitest";
2 |
3 | // Mock supabase
4 | const mockSelect = vi.fn();
5 | const mockInsert = vi.fn();
6 | const mockUpdate = vi.fn();
7 | const mockDelete = vi.fn();
8 | const mockOrder = vi.fn();
9 | const mockEq = vi.fn();
10 | const mockNeq = vi.fn();
11 | const mockSingle = vi.fn();
12 | const mockMaybeSingle = vi.fn();
13 |
14 | const chainable = () => ({
15 |   select: mockSelect,
16 |   insert: mockInsert,
17 |   update: mockUpdate,
18 |   delete: mockDelete,
19 |   order: mockOrder,
20 |   eq: mockEq,
21 |   neq: mockNeq,
22 |   single: mockSingle,
23 |   maybeSingle: mockMaybeSingle,
24 | });
25 |
26 | vi.mock("@/lib/supabase", () => ({
27 |   supabase: {
28 |     from: vi.fn(() => chainable()),
29 |   },
30 | }));
31 |
32 | vi.mock("@/lib/auditLogger", () => ({
33 |   logAuditEvent: vi.fn().mockResolvedValue(undefined),
34 | }));
35 |
36 | vi.mock("@/hooks/useAuth", () => ({
37 |   useAuth: () => ({ user: { id: "user-1" }, institutionId: "inst-1" }),
38 | }));
39 |
40 | import type {
41 |   CreateSemesterInput,
42 |   UpdateSemesterInput,
43 | } from "@/hooks/useSemesters";
44 |
45 | describe("useSemesters types", () => {
46 |   it("CreateSemesterInput has required fields", () => {
47 |     const input: CreateSemesterInput = {
48 |       name: "Fall 2025",
49 |       code: "F25",
50 |       start_date: "2025-09-01",
51 |       end_date: "2025-12-31",
52 |       is_active: true,
53 |       institution_id: "inst-1",
54 |     };
55 |     expect(input.name).toBe("Fall 2025");
56 |     expect(input.code).toBe("F25");
57 |     expect(input.start_date).toBe("2025-09-01");
58 |     expect(input.end_date).toBe("2025-12-31");
59 |     expect(input.is_active).toBe(true);
60 |     expect(input.institution_id).toBe("inst-1");
61 |   });
62 |
63 |   it("UpdateSemesterInput allows partial fields", () => {
64 |     const input: UpdateSemesterInput = {
65 |       name: "Spring 2026",
66 |     };
67 |     expect(input.name).toBe("Spring 2026");
68 |     expect(input.is_active).toBeUndefined();
69 |   });
70 |
71 |   it("UpdateSemesterInput allows is_active toggle", () => {
72 |     const input: UpdateSemesterInput = {
73 |       is_active: false,
74 |     };
75 |     expect(input.is_active).toBe(false);
76 |   });
77 | });
78 |

```

FILE: src/__tests__/unit/useSessionIntent.test.ts

[illegible]

[illegible]

[illegible]

FILE: src/__tests__/unit/useStudentDashboardAggregate.test.ts

TYPE: .ts | LINES: 1-775 | SHA256: 409e0a8a53ad5603bcabd555ce93c069ed85a1fc3e8a7c1b75e9da3613e197bf

```

1 // @vitest-environment happy-dom
2 // =====
3 // useStudentDashboardAggregate.test.ts
4 // Feature: dashboard-and-ux-performance, Req 2 (per-role dashboard aggregate RPC)
5 // -----
6 // Proves the student dashboard critical block (KPIs + upcoming deadlines):
7 // 1. PARITY — the aggregate's `kpis` / `deadlines` deep-equal the exact shapes

```

[illegible]

```

112 |     enabled: aggregate.isError,
113 |   });
114 |   const freezeData = aggregate.data?.streakFreeze ?? freezeHook.data;
115 |   return { aggregate, freezeHook, freezeData };
116 | };
117 |
118 | // Profile-completeness-section harness: wires the aggregate +
119 | // `useProfileCompleteness` EXACTLY as `StudentDashboard` now does (completeness
120 | // read from the aggregate, the section hook gated to a fallback-only fetch on
121 | // `aggregate.isError`). Isolating it means the only `supabase.from` reads in
122 | // these tests are the completeness fallback's `student_profiles` +
123 | // `onboarding_progress` reads – making the collapse/fallback assertions
124 | // unambiguous.
125 | const useProfileCompletenessBlock = (studentId: string) => {
126 |   const aggregate = useStudentDashboardAggregate(studentId);
127 |   const completenessHook = useProfileCompleteness(studentId, {
128 |     enabled: aggregate.isError,
129 |   });
130 |   const completenessData =
131 |     aggregate.data?.profileCompleteness ?? completenessHook.data;
132 |   return { aggregate, completenessHook, completenessData };
133 | };
134 |
135 | // Announcements-section harness: wires the aggregate + `useStudentAnnouncements`
136 | // EXACTLY as `StudentDashboard`'s `AnnouncementsSection` now does (announcements
137 | // read from the aggregate, the section hook gated to a fallback-only fetch on
138 | // `aggregate.isError`). Isolating it means the only `supabase.from` reads in
139 | // these tests are the announcements fallback's enrolled-courses query – making
140 | // the collapse/fallback assertions unambiguous.
141 | const useAnnouncementsBlock = (studentId: string) => {
142 |   const aggregate = useStudentDashboardAggregate(studentId);
143 |   const announcementsHook = useStudentAnnouncements(studentId, 5, {
144 |     enabled: aggregate.isError,
145 |   });
146 |   const announcements = aggregate.data?.announcements ?? announcementsHook.data;
147 |   return { aggregate, announcementsHook, announcements };
148 | };
149 |
150 | const STUDENT_ID = "11111111-1111-1111-1111-111111111111";
151 |
152 | // Canonical section shapes. Typed as the section-hook return types, so the
153 | // compiler enforces structural parity and the deep-equals enforce value parity.
154 | const FIXTURE_KPIS: StudentKPIData = {
155 |   enrolledCourses: 3,
156 |   completedAssignments: 12,
157 |   avgAttainment: 78,
158 |   currentStreak: 5,
159 |   currentLevel: 4,
160 |   totalXP: 1340,
161 |   totalActiveDays: 27,
162 | };
163 |
164 | const FIXTURE_DEADLINE_1: UpcomingDeadline = {
165 |   id: "a1",
166 |   title: "Essay 1",
167 |   course_name: "Intro to OBE",
168 |   due_date: "2026-09-01T00:00:00.000Z",
169 | };
170 |
171 | const FIXTURE_DEADLINE_2: UpcomingDeadline = {
172 |   id: "a2",
173 |   title: "Quiz 2",
174 |   course_name: "Data Structures",
175 |   due_date: "2026-09-03T00:00:00.000Z",
176 | };
177 |
178 | const FIXTURE_DEADLINES: UpcomingDeadline[] = [
179 |   FIXTURE_DEADLINE_1,
180 |   FIXTURE_DEADLINE_2,
181 | ];
182 |
183 | // Canonical attendance shape – the exact `StudentCourseAttendance[]` that
184 | // `useStudentAttendance` produces, so structural + value parity are enforced.
185 | const FIXTURE_ATTENDANCE: StudentCourseAttendance[] = [
186 |   {
187 |     courseId: "c1",
188 |     courseName: "Intro to OBE",
189 |     attendancePercent: 92,
190 |     totalSessions: 12,
191 |     attended: 11,
192 |   },
193 |   {
194 |     courseId: "c2",
195 |     courseName: "Data Structures",
196 |     attendancePercent: 60,
197 |     totalSessions: 10,
198 |     attended: 6,
199 |   },
200 | ];
201 |
202 | // Canonical streak-freeze shape – the exact `{ freezes, xpTotal }` that
203 | // `useStreakFreezeInventory` produces, so structural + value parity are
204 | // enforced. `xpTotal` mirrors the KPI `totalXP` because in production both are
205 | // read from the same `student_gamification.xp_total` column.
206 | const FIXTURE_STREAK_FREEZE = { freezes: 2, xpTotal: FIXTURE_KPIS.totalXP };
207 |
208 | // Canonical profile-completeness shape – the exact `ProfileCompletenessData`
209 | // that `useProfileCompleteness` produces, so structural + value parity are
210 | // enforced.
211 | const FIXTURE_PROFILE_COMPLETENESS: ProfileCompletenessData = {
212 |   profile_completeness: 80,
213 |   day1_completed: true,
214 | };
215 |

```

```

216 // Canonical announcements shape – the exact `Announcement[]` that
217 // `useStudentAnnouncements(studentId, 5)` produces, already ordered pinned-desc
218 // then created-desc (limit 5), so structural + value parity are enforced.
219 const FIXTURE_ANNOUNCEMENTS: Announcement[] = [
220   {
221     id: "an1",
222     course_id: "c1",
223     author_id: "t1",
224     title: "Welcome to the semester",
225     content: "Please review the syllabus before our first session.",
226     is_pinned: true,
227     created_at: "2026-08-20T09:00:00.000Z",
228     updated_at: "2026-08-20T09:00:00.000Z",
229   },
230   {
231     id: "an2",
232     course_id: "c2",
233     author_id: "t2",
234     title: "Assignment 1 posted",
235     content: "Your first assignment is now available.",
236     is_pinned: false,
237     created_at: "2026-08-19T15:30:00.000Z",
238     updated_at: "2026-08-19T15:30:00.000Z",
239   },
240 ];
241
242 // Canonical spendable-XP value – the exact number `get_xp_balance` returns and
243 // `useXPBalance` exposes as `{ balance }`. Distinct from `totalXP` (lifetime).
244 const FIXTURE_AVAILABLE_XP = 356;
245
246 // Full success payload mirroring the `get_student_dashboard` jsonb (all
247 // sections present), used by the attendance-, streak-freeze-,
248 // profile-completeness- and announcements-section tests.
249 const FIXTURE_AGGREGATE_DATA = {
250   kpis: FIXTURE_KPIS,
251   deadlines: FIXTURE_DEADLINES,
252   attendance: FIXTURE_ATTENDANCE,
253   streakFreeze: FIXTURE_STREAK_FREEZE,
254   profileCompleteness: FIXTURE_PROFILE_COMPLETENESS,
255   announcements: FIXTURE_ANNOUNCEMENTS,
256   availableXP: FIXTURE_AVAILABLE_XP,
257 };
258
259 const makeClient = () =>
260   new QueryClient({
261     defaultOptions: { queries: { retry: false } },
262   });
263
264 const makeWrapper = (client: QueryClient) => {
265   const wrapper = ({ children }: { children: ReactNode }) =>
266     createElement(QueryClientProvider, { client }, children);
267   return wrapper;
268 };
269
270 describe("useStudentDashboardAggregate (dashboard-and-ux-performance Req 2)", () => {
271   beforeEach(() => {
272     vi.clearAllMocks();
273     // The hook now write-throughs the aggregate to a per-user localStorage cache
274     // and hydrates it as SWR initialData. Clear it between tests so a prior
275     // test's snapshot can't suppress the RPC (fresh initialData !' no refetch).
276     localStorage.clear();
277     mockFrom.mockImplementation(() => makeFromBuilder());
278   });
279
280   it("calls get_student_dashboard once with the documented param name", async () => {
281     mockRpc.mockResolvedValue({
282       data: { kpis: FIXTURE_KPIS, deadlines: FIXTURE_DEADLINES },
283       error: null,
284     });
285
286     const client = makeClient();
287     const { result } = renderHook(
288       () => useStudentDashboardAggregate(STUDENT_ID),
289       { wrapper: makeWrapper(client) }
290     );
291
292     await waitFor(() => expect(result.current.isSuccess).toBe(true));
293
294     expect(mockRpc).toHaveBeenCalledTimes(1);
295     expect(mockRpc).toHaveBeenCalledWith("get_student_dashboard", {
296       p_student_id: STUDENT_ID,
297     });
298   });
299
300   it("parity: aggregate kpis + deadlines deep-equal the section-hook shapes (no data change)", async () => {
301     mockRpc.mockResolvedValue({
302       data: { kpis: FIXTURE_KPIS, deadlines: FIXTURE_DEADLINES },
303       error: null,
304     });
305
306     const client = makeClient();
307     const { result } = renderHook(
308       () => useStudentDashboardAggregate(STUDENT_ID),
309       { wrapper: makeWrapper(client) }
310     );
311
312     await waitFor(() => expect(result.current.isSuccess).toBe(true));
313
314     // Value parity.
315     expect(result.current.data?.kpis).toEqual(FIXTURE_KPIS);
316     expect(result.current.data?.deadlines).toEqual(FIXTURE_DEADLINES);
317
318     // Shape parity: exactly the StudentKPIData keys – no extra/missing fields
319     // (guards against RPC jsonb key drift vs. what useStudentKPIS produces).

```

[illegible]

[illegible]

[illegible]

[illegible]


```

736 describe("available XP section", () => {
737   it("hydration: writes the balance cache under the exact key useXPBalance reads", async () => {
738     mockRpc.mockResolvedValue({ data: FIXTURE_AGGREGATE_DATA, error: null });
739
740     const client = makeClient();
741     const { result } = renderHook(
742       () => useStudentDashboardAggregate(STUDENT_ID),
743       { wrapper: makeWrapper(client) }
744     );
745
746     await waitFor(() => expect(result.current.isSuccess).toBe(true));
747
748     const balanceCache = client.getQueryData<{ balance: number }>({
749       queryKeys: marketplace.balance(STUDENT_ID)
750     });
751     expect(balanceCache).toEqual({ balance: FIXTURE_AVAILABLE_XP });
752   });
753
754   it("no-op: a payload without availableXP leaves the balance cache empty (badge falls back)", async () => {
755     // Pre-migration shape: kpis + deadlines only, no availableXP.
756     mockRpc.mockResolvedValue({
757       data: { kpis: FIXTURE_KPIS, deadlines: FIXTURE_DEADLINES },
758       error: null,
759     });
760
761     const client = makeClient();
762     const { result } = renderHook(
763       () => useStudentDashboardAggregate(STUDENT_ID),
764       { wrapper: makeWrapper(client) }
765     );
766
767     await waitFor(() => expect(result.current.isSuccess).toBe(true));
768
769     expect(
770       client.getQueryData(queryKeys.marketplace.balance(STUDENT_ID))
771     ).toBeUndefined();
772   });
773 });
774
775

```

FILE: src/__tests__/unit/useSurveyAssignmentsCount.test.tsx

TYPE: .tsx | LINES: 1-133 | SHA256: 772916925fa58d794be321642f41482e1b20657f6b35564f24dee7d655b24b48

[illegible]

[illegible]

FILE: src/__tests__/unit/useSurveys.test.ts

```
TYPE: .ts | LINES: 1-386 | SHA256: 6c1188c1cadf6d40b059186651fb4b983055fd733ec80cfa74918cad51e5f92d
```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

FILE: src/__tests__/unit/useTeacherDashboardAggregate.test.ts

TYPE: .ts | LINES: 1-179 | SHA256: 8fbfc5a278698f8bc5f1e73680309f30f039f3cd7f719695f6e12ad38a51590a

```

1 // @vitest-environment happy-dom
2 // =====
3 // useTeacherDashboardAggregate.test.ts
4 // Feature: dashboard-and-ux-performance, Phase 8 Task 33 (teacher aggregate RPC)
5 // =====
6 // Proves the teacher dashboard always-on block (KPIs + Bloom's distribution):
7 //   1. CALL – get_teacher_dashboard is called once with the documented param.
8 //   2. HYDRATION – the EXACT caches useTeacherKPIs / useTeacherBloomsDistribution
9 //     read are populated, so those section hooks become cache hits.
10 //   3. COLLAPSE – with the section hooks gated on `aggregate.isError` (as the page
11 //     wires them), a SUCCESSFUL aggregate fires ZERO section `supabase.from`
12 //     requests; a FAILED aggregate makes them fall back and fetch.
13 // The `get_teacher_dashboard` RPC is mocked; this is a hermetic unit test.
14 // =====
15 import { describe, it, expect, vi, beforeEach } from "vitest";
16 import { renderHook, waitFor } from "@testing-library/react";
17 import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
18 import { createElement, type ReactNode } from "react";
19
20 const mockRpc = vi.fn();
21 const mockFrom = vi.fn();
22
23 vi.mock("@/lib/supabase", () => ({
24   supabase: {
25     rpc: (...args: unknown[]) => mockRpc(...args),
26     from: (...args: unknown[]) => mockFrom(...args),
27   },
28 }));
29
30 // useTeacherKPIs / useTeacherBloomsDistribution read `useAuth().user?.id`; stub it
31 // so the section hooks resolve the same teacherId the aggregate is called with.
32 const TEACHER_ID = "22222222-2222-2222-2222-222222222222";
33 vi.mock("@/hooks/useAuth", () => ({
34   useAuth: () => ({ user: { id: TEACHER_ID } }),
35 }));
36
37 const makeFromBuilder = () => {
38   const builder: Record<string, unknown> = {
39     select: () => builder,
40     eq: () => builder,
41     in: () => builder,
42     gte: () => builder,
43     order: () => builder,
44     limit: () => builder,
45     maybeSingle: () => Promise.resolve({ data: null, error: null }),
46     then: (resolve: (value: unknown) => unknown) =>
47       resolve({ data: [], count: 0, error: null }),
48   };
49   return builder;
50 };
51
52 import { queryKeys } from "@/lib/queryKeys";
53 import { useTeacherDashboardAggregate } from "@/hooks/useTeacherDashboardAggregate";

```

```

54 | import {
55 |   useTeacherKPIs,
56 |   useTeacherBloomsDistribution,
57 |   type TeacherKPIData,
58 |   type BloomsDistributionRow,
59 | } from "@hooks/useTeacherDashboard";
60 |
61 | const FIXTURE_KPIS: TeacherKPIData = {
62 |   pendingSubmissions: 4,
63 |   gradedThisWeek: 7,
64 |   avgAttainment: 75,
65 |   atRiskCount: 3,
66 |   totalStudents: 30,
67 | };
68 |
69 | const FIXTURE_BLOOMS: BloomsDistributionRow[] = [
70 |   { level: "remembering", count: 2 },
71 |   { level: "applying", count: 1 },
72 | ];
73 |
74 | const FIXTURE_AGGREGATE = {
75 |   kpis: FIXTURE_KPIS,
76 |   bloomsDistribution: FIXTURE_BLOOMS,
77 | };
78 |
79 | // Composes the aggregate + section hooks EXACTLY as TeacherDashboard wires them.
80 | const useBlock = () => {
81 |   const aggregate = useTeacherDashboardAggregate(TEACHER_ID);
82 |   const kpisHook = useTeacherKPIs({ enabled: aggregate.isError });
83 |   const bloomsHook = useTeacherBloomsDistribution({
84 |     enabled: aggregate.isError,
85 |   });
86 |   return { aggregate, kpisHook, bloomsHook };
87 | };
88 |
89 | const makeClient = () =>
90 |   new QueryClient({ defaultOptions: { queries: { retry: false } } });
91 |
92 | const makeWrapper = (client: QueryClient) => {
93 |   const wrapper = ({ children }: { children: ReactNode }) =>
94 |     createElement(QueryClientProvider, { client }, children);
95 |   return wrapper;
96 | };
97 |
98 | describe("useTeacherDashboardAggregate (Phase 8 Task 33)", () => {
99 |   beforeEach(() => {
100 |     vi.clearAllMocks();
101 |     mockFrom.mockImplementation(() => makeFromBuilder());
102 |   });
103 |
104 |   it("calls get_teacher_dashboard once with the documented param name", async () => {
105 |     mockRpc.mockResolvedValue({ data: FIXTURE_AGGREGATE, error: null });
106 |
107 |     const client = makeClient();
108 |     const { result } = renderHook(
109 |       () => useTeacherDashboardAggregate(TEACHER_ID),
110 |       { wrapper: makeWrapper(client) }
111 |     );
112 |
113 |     await waitFor(() => expect(result.current.isSuccess).toBe(true));
114 |     expect(mockRpc).toHaveBeenCalledTimes(1);
115 |     expect(mockRpc).toHaveBeenCalledWith("get_teacher_dashboard", {
116 |       p_teacher_id: TEACHER_ID,
117 |     });
118 |   });
119 |
120 |   it("hydrates the exact KPI + Bloom's caches the section hooks read", async () => {
121 |     mockRpc.mockResolvedValue({ data: FIXTURE_AGGREGATE, error: null });
122 |
123 |     const client = makeClient();
124 |     const { result } = renderHook(
125 |       () => useTeacherDashboardAggregate(TEACHER_ID),
126 |       { wrapper: makeWrapper(client) }
127 |     );
128 |
129 |     await waitFor(() => expect(result.current.isSuccess).toBe(true));
130 |
131 |     expect(
132 |       client.getQueryData<TeacherKPIData>(
133 |         queryKeys.teacherDashboard.list({ type: "kpis", teacherId: TEACHER_ID })
134 |       )
135 |     ).toEqual(FIXTURE_KPIS);
136 |     expect(
137 |       client.getQueryData<BloomsDistributionRow[]>(
138 |         queryKeys.teacherDashboard.list({
139 |           type: "bloomsDistribution",
140 |           teacherId: TEACHER_ID,
141 |         })
142 |       )
143 |     ).toEqual(FIXTURE_BLOOMS);
144 |   });
145 |
146 |   it("collapse: a successful aggregate fires ZERO section requests", async () => {
147 |     mockRpc.mockResolvedValue({ data: FIXTURE_AGGREGATE, error: null });
148 |
149 |     const client = makeClient();
150 |     const { result } = renderHook(() => useBlock(), {
151 |       wrapper: makeWrapper(client),
152 |     });
153 |
154 |     await waitFor(() => expect(result.current.aggregate.isSuccess).toBe(true));
155 |     expect(mockRpc).toHaveBeenCalledTimes(1);
156 |     expect(mockFrom).not.toHaveBeenCalled();
157 |     // Section hooks resolve from hydrated cache without fetching.

```


[illegible]

```

183 });
184
185 it("reports no further pages on the last page", async () => {
186   // total_count 42, page 2 (offset 40) !' entries 41-42, no more pages
187   state.rows = [
188     {
189       id: "earn-41",
190       kind: "earning",
191       amount: 10,
192       label: "login",
193       category: "login",
194       occurred_at: "2026-05-01T08:00:00Z",
195       total_count: 42,
196     },
197   ];
198   const { result } = renderHook(
199     () => useTransactionHistory(STUDENT, "all", 2),
200     { wrapper: createWrapper() }
201   );
202   await waitFor(() => expect(result.current.isSuccess).toBe(true));
203   expect(result.current.data?.hasMore).toBe(false);
204 });
205
206 it("returns an empty page with zero count when the offset is beyond the end", async () => {
207   state.rows = [];
208   const { result } = renderHook(
209     () => useTransactionHistory(STUDENT, "all", 99),
210     { wrapper: createWrapper() }
211   );
212   await waitFor(() => expect(result.current.isSuccess).toBe(true));
213   expect(result.current.data?.entries).toEqual([]);
214   expect(result.current.data?.totalCount).toBe(0);
215   expect(result.current.data?.hasMore).toBe(false);
216 });
217
218 it("surfaces an RPC failure as an error and refuses to show transactions (R33.1a)", async () => {
219   state.error = new Error("rpc unavailable");
220   const { result } = renderHook(
221     () => useTransactionHistory(STUDENT, "all", 0),
222     { wrapper: createWrapper() }
223   );
224   await waitFor(() => expect(result.current.isError).toBe(true));
225   expect(result.current.error).toBeInstanceOf(Error);
226   // No truncated/fallback data is exposed when the source query fails.
227   expect(result.current.data).toBeUndefined();
228 });
229
230 it("is disabled until a student id is provided", () => {
231   const { result } = renderHook(
232     () => useTransactionHistory("", "all" as TransactionFilter, 0),
233     { wrapper: createWrapper() }
234   );
235   expect(result.current.fetchStatus).toBe("idle");
236   expect(rpcMock).not.toHaveBeenCalled();
237 });
238 });
239

```

FILE: src/__tests__/unit/useUpdateConversationAutonomy.test.ts

```
TYPE: .ts | LINES: 1-105 | SHA256: a02755d1ee3cef8ec2dea8ce1eca7180a545bf6f2e9c2a1cfba934b83761409b
```

[illegible]

```

44     });
45   });
46
47   it("performs a typed update of autonomy_override scoped by conversation id", async () => {
48     eqMock.mockResolvedValueOnce({ error: null });
49
50     const { result } = renderHook(() => useUpdateConversationAutonomy(), {
51       wrapper: createWrapper(queryClient),
52     });
53
54     result.current.mutate({ conversationId: "conv-1", level: "L1" });
55
56     await waitFor(() => expect(result.current.isSuccess).toBe(true));
57
58     expect(fromMock).toHaveBeenCalledTimes(1);
59     expect(updateMock).toHaveBeenCalledTimes(1);
60     expect(eqMock).toHaveBeenCalledTimes(1);
61   });
62
63   it("clears the override when level is null", async () => {
64     eqMock.mockResolvedValueOnce({ error: null });
65
66     const { result } = renderHook(() => useUpdateConversationAutonomy(), {
67       wrapper: createWrapper(queryClient),
68     });
69
70     result.current.mutate({ conversationId: "conv-2", level: null });
71
72     await waitFor(() => expect(result.current.isSuccess).toBe(true));
73     expect(updateMock).toHaveBeenCalledTimes(1);
74   });
75
76   it("invalidates conversation queries on success", async () => {
77     eqMock.mockResolvedValueOnce({ error: null });
78     const invalidateSpy = vi.spyOn(queryClient, "invalidateQueries");
79
80     const { result } = renderHook(() => useUpdateConversationAutonomy(), {
81       wrapper: createWrapper(queryClient),
82     });
83
84     result.current.mutate({ conversationId: "conv-1", level: "L3" });
85
86     await waitFor(() => expect(result.current.isSuccess).toBe(true));
87     expect(invalidateSpy).toHaveBeenCalledTimes(1);
88     expect(queryKey).toEqual(queryKeys.tutorConversations.all);
89   });
90
91   it("surfaces a Sonner toast when the update fails", async () => {
92     eqMock.mockResolvedValueOnce({ error: new Error("update denied") });
93
94     const { result } = renderHook(() => useUpdateConversationAutonomy(), {
95       wrapper: createWrapper(queryClient),
96     });
97
98     result.current.mutate({ conversationId: "conv-1", level: "L1" });
99
100     await waitFor(() => expect(result.current.isError).toBe(true));
101     expect(toastErrorMock).toHaveBeenCalledTimes(1);
102   });
103 });
104
105

```

FILE: src/_tests_/unit/useUsers.test.ts

```
TYPE: .ts | LINES: 1-296 | SHA256: 83d8aae6cb9c276754848194b8f5f44454d4b0f94ac1005618e5d6702b31ec4f
```

[illegible]

[illegible]

[illegible]

[illegible]

```

155 | it("maps transactions to display format with source labels", () => {
156 |   const rawTx = {
157 |     id: "tx-1",
158 |     source: "submission",
159 |     xp_amount: 25,
160 |     reference_id: "assign-1",
161 |     note: "Data Structures HW1",
162 |     created_at: "2025-01-15T10:30:00Z",
163 |   };
164 |
165 |   const display = {
166 |     ...rawTx,
167 |     source_label: getSourceLabel(rawTx.source),
168 |     reference_description: rawTx.note ?? "",
169 |   };
170 |
171 |   expect(display.source_label).toBe("On-time Submission");
172 |   expect(display.reference_description).toBe("Data Structures HW1");
173 |   expect(display.xp_amount).toBe(25);
174 | });
175 |
176 | it("computes category summary from transaction list", () => {
177 |   const rows = [
178 |     { source: "login", xp_amount: 10 },
179 |     { source: "login", xp_amount: 10 },
180 |     { source: "submission", xp_amount: 25 },
181 |     { source: "journal", xp_amount: 20 },
182 |     { source: "login", xp_amount: 10 },
183 |   ];
184 |
185 |   const runningTotal = rows.reduce((sum, tx) => sum + tx.xp_amount, 0);
186 |   expect(runningTotal).toBe(75);
187 |
188 |   const categoryMap = new Map<string, { total_xp: number; count: number }>();
189 |   for (const tx of rows) {
190 |     const existing = categoryMap.get(tx.source);
191 |     if (existing) {
192 |       existing.total_xp += tx.xp_amount;
193 |       existing.count += 1;
194 |     } else {
195 |       categoryMap.set(tx.source, { total_xp: tx.xp_amount, count: 1 });
196 |     }
197 |   }
198 |
199 |   const categories = Array.from(categoryMap.entries())
200 |     .map(([source, stats]) => ({
201 |       source,
202 |       source_label: getSourceLabel(source),
203 |       ...stats,
204 |     })))
205 |     .sort((a, b) => b.total_xp - a.total_xp);
206 |
207 |   expect(categories).toHaveLength(3);
208 |   expect(categories[0]!.source).toBe("login");
209 |   expect(categories[0]!.total_xp).toBe(30);
210 |   expect(categories[0]!.count).toBe(3);
211 |   expect(categories[1]!.source).toBe("submission");
212 |   expect(categories[1]!.total_xp).toBe(25);
213 |   expect(categories[2]!.source).toBe("journal");
214 |   expect(categories[2]!.total_xp).toBe(20);
215 | });
216 | });
217 |

```

FILE: [src/__tests__/unit/viewTransition.test.ts](#)

TYPE: .ts | LINES: 1-84 | SHA256: b0199084d9abc8593d82684b15897e473526fd97632a54d56cb9870c8efa202b

```

1 | // @vitest-environment happy-dom
2 | // =====
3 | // viewTransition.test.ts
4 | // Feature: dashboard-and-ux-performance, Req 11 (View Transitions).
5 | // -----
6 | // Verifies withViewTransition:
7 | //   - uses document.startViewTransition when supported + motion allowed (11.1)
8 | //   - falls back to a synchronous update when the API is missing (11.2)
9 | //   - runs synchronously (no transition) under prefers-reduced-motion (11.3)
10 | //   - still applies the update if startViewTransition throws (never breaks UI)
11 | // In every case the update runs exactly once.
12 | // =====
13 | import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
14 | import { withViewTransition } from "@lib/viewTransition";
15 |
16 | const setReducedMotion = (reduce: boolean) => {
17 |   window.matchMedia = vi.fn().mockImplementation((query: string) => ({
18 |     matches: query === "(prefers-reduced-motion: reduce)" ? reduce : false,
19 |     media: query,
20 |     addEventListener: vi.fn(),
21 |     removeEventListener: vi.fn(),
22 |     addListener: vi.fn(),
23 |     removeListener: vi.fn(),
24 |     onchange: null,
25 |     dispatchEvent: vi.fn(),
26 |   }));
27 | };
28 |
29 | describe("withViewTransition (dashboard-and-ux-performance Req 11)", () => {
30 |   beforeEach(() => {
31 |     setReducedMotion(false);
32 |     delete (document as { startViewTransition?: unknown }).startViewTransition;
33 |   });
34 |   afterEach(() => {
35 |     vi.restoreAllMocks();
36 |     delete (document as { startViewTransition?: unknown }).startViewTransition;
37 |   });

```



```

39 it("wraps the update in startViewTransition when supported and motion is allowed", () => {
40   const svt = vi.fn((cb: () => void) => {
41     cb();
42     return { finished: Promise.resolve() };
43   });
44   (document as { startViewTransition?: unknown }).startViewTransition = svt;
45
46   const update = vi.fn();
47   withViewTransition(update);
48
49   expect(svt).toHaveBeenCalledTimes(1);
50   expect(update).toHaveBeenCalledTimes(1);
51 });
52
53 it("runs synchronously (no transition) under prefers-reduced-motion", () => {
54   setReducedMotion(true);
55   const svt = vi.fn();
56   (document as { startViewTransition?: unknown }).startViewTransition = svt;
57
58   const update = vi.fn();
59   withViewTransition(update);
60
61   expect(svt).nottoHaveBeenCalled();
62   expect(update).toHaveBeenCalledTimes(1);
63 });
64
65 it("falls back to a direct update when the API is unsupported", () => {
66   // No startViewTransition on document (deleted in beforeEach).
67   const update = vi.fn();
68   withViewTransition(update);
69   expect(update).toHaveBeenCalledTimes(1);
70 });
71
72 it("still applies the update if startViewTransition throws", () => {
73   (document as { startViewTransition?: unknown }).startViewTransition = vi
74     .fn()
75     .mockImplementation(() => {
76       throw new Error("boom");
77     });
78
79   const update = vi.fn();
80   expect(() => withViewTransition(update)).not.toThrow();
81   expect(update).toHaveBeenCalledTimes(1);
82 });
83 });
84

```

FILE: src/__tests__/unit/weeklyCalendarGrid.test.tsx

TYPE: .tsx | LINES: 1-472 | SHA256: c1b4e2d6969af0b0c19798e835d4a622c91f04f168477ccf763051b452c722c4

[illegible]

[illegible]

```

158     title: "High Priority",
159     priority: "high",
160     dueDate: "2025-06-16",
161   });
162   const lowTask = makeTask({
163     id: "task-low",
164     title: "Low Priority",
165     priority: "low",
166     dueDate: "2025-06-16",
167   });
168   const medTask = makeTask({
169     id: "task-med",
170     title: "Med Priority",
171     priority: "medium",
172     dueDate: "2025-06-16",
173   });
174
175   const weekData = buildWeekData({ tasks: [lowTask, highTask, medTask] });
176   render(<WeeklyCalendarGrid weekData={weekData} today="2025-06-18" />);
177
178   // All tasks should be rendered
179   expect(screen.getAllByText("High Priority").length).toBeGreaterThanOrEqual(
180     1
181   );
182   expect(screen.getAllByText("Med Priority").length).toBeGreaterThanOrEqual(
183     1
184   );
185   expect(screen.getAllByText("Low Priority").length).toBeGreaterThanOrEqual(
186     1
187   );
188 });
189
190 it("renders deadline items", () => {
191   const deadline = makeDeadline({
192     title: "Assignment 3",
193     dueDate: "2025-06-18T23:59:00Z",
194     urgency: "yellow",
195   });
196   const weekData = buildWeekData({ deadlines: [deadline] });
197
198   render(<WeeklyCalendarGrid weekData={weekData} today="2025-06-18" />);
199
200   // Deadline appears (desktop + mobile both show it since today=Jun 18)
201   const titles = screen.getAllByText("Assignment 3");
202   expect(titles.length).toBeGreaterThanOrEqual(1);
203 });
204
205 it("shows empty state for days with no items", () => {
206   const weekData = buildWeekData();
207   render(<WeeklyCalendarGrid weekData={weekData} today="2025-06-18" />);
208
209   // Desktop shows 7 "No items", mobile shows 1 "Nothing planned for this day"
210   const emptyDesktop = screen.getAllByText("No items");
211   expect(emptyDesktop.length).toBe(7);
212 });
213
214 it("calls onTaskToggle when checkbox is clicked", async () => {
215   const user = userEvent.setup();
216   const onToggle = vi.fn();
217   // Put task on today so mobile view also shows it
218   const task = makeTask({ dueDate: "2025-06-18" });
219   const weekData = buildWeekData({ tasks: [task] });
220
221   render(
222     <WeeklyCalendarGrid
223       weekData={weekData}
224       today="2025-06-18"
225       onTaskToggle={onToggle}
226     />
227   );
228
229   const checkboxes = screen.getAllByRole("checkbox");
230   expect(checkboxes.length).toBeGreaterThanOrEqual(1);
231   await user.click(checkboxes[0]);
232   expect(onToggle).toHaveBeenCalledOnceWith(task);
233 });
234
235 it("does not show action buttons in readOnly mode", () => {
236   const session = makeSession({ plannedDate: "2025-06-18" });
237   const task = makeTask({ dueDate: "2025-06-18" });
238   const weekData = buildWeekData({ sessions: [session], tasks: [task] });
239
240   render(
241     <WeeklyCalendarGrid weekData={weekData} today="2025-06-18" readOnly />
242   );
243
244   // No Start/Edit buttons should be present
245   const startButtons = screen.queryAllByRole("button", { name: /start/i });
246   expect(startButtons.length).toBe(0);
247 });
248
249 it("renders mobile day view with navigation buttons", () => {
250   const weekData = buildWeekData({ todayDate: "2025-06-18" });
251   render(<WeeklyCalendarGrid weekData={weekData} today="2025-06-18" />);
252
253   // Mobile navigation buttons
254   expect(screen.getByLabelText("Previous day")).toBeTruthy();
255   expect(screen.getByLabelText("Next day")).toBeTruthy();
256 });
257
258 it("mobile view defaults to today", () => {
259   const weekData = buildWeekData({ todayDate: "2025-06-18" });
260   render(<WeeklyCalendarGrid weekData={weekData} today="2025-06-18" />);
261

```

[illegible]

```

366 |     const user = userEvent.setup();
367 |     const onSkipReview = vi.fn();
368 |     const review = makeReview({
369 |       reviewDate: "2025-06-18",
370 |       status: "pending",
371 |       cloId: "clo-skip",
372 |     });
373 |     const weekData = buildWeekData();
374 |
375 |     render(
376 |       <WeeklyCalendarGrid
377 |         weekData={weekData}
378 |         today="2025-06-18"
379 |         reviews={[review]}
380 |         onSkipReview={onSkipReview}
381 |       />
382 |     );
383 |
384 |     const skipButtons = screen.getAllByLabelText(
385 |       /Skip review for CLO clo-skip/
386 |     );
387 |     await user.click(skipButtons[0]!);
388 |     expect(onSkipReview).toHaveBeenCalledOnceWith(review.id);
389 |   });
390 |
391 |   it("shows completed status for completed reviews", () => {
392 |     const review = makeReview({
393 |       reviewDate: "2025-06-18",
394 |       status: "completed",
395 |       cloId: "clo-done",
396 |     });
397 |     const weekData = buildWeekData();
398 |
399 |     render(
400 |       <WeeklyCalendarGrid
401 |         weekData={weekData}
402 |         today="2025-06-18"
403 |         reviews={[review]}
404 |       />
405 |     );
406 |
407 |     const doneLabels = screen.getAllByText("Done");
408 |     expect(doneLabels.length).toBeGreaterThanOrEqual(1);
409 |   });
410 |
411 |   it("shows skipped status for skipped reviews", () => {
412 |     const review = makeReview({
413 |       reviewDate: "2025-06-18",
414 |       status: "skipped",
415 |       cloId: "clo-skipped",
416 |     });
417 |     const weekData = buildWeekData();
418 |
419 |     render(
420 |       <WeeklyCalendarGrid
421 |         weekData={weekData}
422 |         today="2025-06-18"
423 |         reviews={[review]}
424 |       />
425 |     );
426 |
427 |     const skippedLabels = screen.getAllByText("Skipped");
428 |     expect(skippedLabels.length).toBeGreaterThanOrEqual(1);
429 |   });
430 |
431 |   it("hides review action buttons in readOnly mode", () => {
432 |     const review = makeReview({
433 |       reviewDate: "2025-06-18",
434 |       status: "pending",
435 |       cloId: "clo-readonly",
436 |     });
437 |     const weekData = buildWeekData();
438 |
439 |     render(
440 |       <WeeklyCalendarGrid
441 |         weekData={weekData}
442 |         today="2025-06-18"
443 |         reviews={[review]}
444 |         readOnly
445 |       />
446 |     );
447 |
448 |     // No Start/Skip buttons for reviews in readOnly mode
449 |     const startButtons = screen.queryAllByLabelText(/Start review/);
450 |     expect(startButtons.length).toBe(0);
451 |     const skipButtons = screen.queryAllByLabelText(/Skip review/);
452 |     expect(skipButtons.length).toBe(0);
453 |   });
454 |
455 |   it("does not show empty state when only reviews exist for a day", () => {
456 |     const review = makeReview({ reviewDate: "2025-06-18" });
457 |     const weekData = buildWeekData();
458 |
459 |     render(
460 |       <WeeklyCalendarGrid
461 |         weekData={weekData}
462 |         today="2025-06-18"
463 |         reviews={[review]}
464 |       />
465 |     );
466 |
467 |     // Desktop: 6 days with "No items" (the 7th day has a review)
468 |     const emptyDesktop = screen.getAllByText("No items");
469 |     expect(emptyDesktop.length).toBe(6);

```

FILE: src/___tests___/unit/weeklyPlannerContent.test.ts

[illegible]

```

98 |     const wed = days.find((d) => d.date === "2025-06-18")!;
99 |     const thu = days.find((d) => d.date === "2025-06-19")!;
100 |
101 |     expect(tue.sessions).toHaveLength(1);
102 |     expect(wed.tasks).toHaveLength(1);
103 |     expect(thu.deadlines).toHaveLength(1);
104 | });
105 |
106 | it("adds a suggested session to empty future/today days (R19.2)", () => {
107 |     const days = buildPlannerWeek({
108 |         weekStartDate: WEEK_START,
109 |         todayStr: WEEK_START,
110 |         sessions: [],
111 |         tasks: [],
112 |         deadlines: [],
113 |         courses: [{ id: "course-1", name: "Math 101" }],
114 |     });
115 |
116 |     for (const day of days) {
117 |         expect(day.suggestions).toBeUndefined();
118 |         expect(day.suggestions).toHaveLength(1);
119 |         const suggestion = day.suggestions![0]!;
120 |         expect(suggestion.date).toBe(day.date);
121 |         expect(suggestion.durationMinutes).toBe(SUGGESTED_SESSION_MINUTES);
122 |         expect(suggestion.courseId).toBe("course-1");
123 |     }
124 | });
125 |
126 | it("does not suggest on days that already have real items", () => {
127 |     const days = buildPlannerWeek({
128 |         weekStartDate: WEEK_START,
129 |         todayStr: WEEK_START,
130 |         sessions: [makeSession({ plannedDate: "2025-06-16" })],
131 |         tasks: [],
132 |         deadlines: [],
133 |         courses: [{ id: "course-1", name: "Math 101" }],
134 |     });
135 |
136 |     const monday = days.find((d) => d.date === "2025-06-16")!;
137 |     expect(monday.suggestions).toBeUndefined();
138 |
139 |     const tuesday = days.find((d) => d.date === "2025-06-17")!;
140 |     expect(tuesday.suggestions).toHaveLength(1);
141 | });
142 |
143 | it("does not suggest on past days (cannot plan in the past)", () => {
144 |     // today is Wed 2025-06-18 - Mon/Tue are in the past
145 |     const days = buildPlannerWeek({
146 |         weekStartDate: WEEK_START,
147 |         todayStr: "2025-06-18",
148 |         sessions: [],
149 |         tasks: [],
150 |         deadlines: [],
151 |         courses: [{ id: "course-1", name: "Math 101" }],
152 |     });
153 |
154 |     expect(
155 |         days.find((d) => d.date === "2025-06-16")!.suggestions
156 |     ).toBeUndefined();
157 |     expect(
158 |         days.find((d) => d.date === "2025-06-17")!.suggestions
159 |     ).toBeUndefined();
160 |     // today and later get suggestions
161 |     expect(days.find((d) => d.date === "2025-06-18")!.suggestions).toHaveLength(
162 |         1
163 |     );
164 |     expect(days.find((d) => d.date === "2025-06-22")!.suggestions).toHaveLength(
165 |         1
166 |     );
167 | });
168 |
169 | it("does not suggest on empty days that carry a scheduled review", () => {
170 |     const days = buildPlannerWeek({
171 |         weekStartDate: WEEK_START,
172 |         todayStr: WEEK_START,
173 |         sessions: [],
174 |         tasks: [],
175 |         deadlines: [],
176 |         courses: [{ id: "course-1", name: "Math 101" }],
177 |         reviewDates: ["2025-06-20"],
178 |     });
179 |
180 |     expect(
181 |         days.find((d) => d.date === "2025-06-20")!.suggestions
182 |     ).toBeUndefined();
183 | });
184 |
185 | it("rotates suggestions across the student's courses", () => {
186 |     const days = buildPlannerWeek({
187 |         weekStartDate: WEEK_START,
188 |         todayStr: WEEK_START,
189 |         sessions: [],
190 |         tasks: [],
191 |         deadlines: [],
192 |         courses: [
193 |             { id: "c1", name: "Course One" },
194 |             { id: "c2", name: "Course Two" },
195 |         ],
196 |     });
197 |
198 |     const courseIds = days.map((d) => d.suggestions![0]!.courseId);
199 |     expect(courseIds[0]).toBe("c1");
200 |     expect(courseIds[1]).toBe("c2");
201 |     expect(courseIds[2]).toBe("c1");

```

```

202 | });
203 |
204 | it("falls back to course-less suggestions when no courses exist", () => {
205 |   const days = buildPlannerWeek({
206 |     weekStartDate: WEEK_START,
207 |     todayStr: WEEK_START,
208 |     sessions: [],
209 |     tasks: [],
210 |     deadlines: [],
211 |     courses: [],
212 |   });
213 |
214 |   expect(days[0]!.suggestions![0]!.courseId).toBeNull();
215 |   expect(days[0]!.suggestions![0]!.courseName).toBeNull();
216 | });
217 | });
218 |
219 | describe("getExampleGoals", () => {
220 |   it("returns three distinct goal-type examples with positive targets (R19.4)", () => {
221 |     const examples = getExampleGoals();
222 |     expect(examples).toHaveLength(3);
223 |     const types = examples.map((e) => e.goalType);
224 |     expect(new Set(types).size).toBe(3);
225 |     for (const example of examples) {
226 |       expect(example.targetValue).toBeGreaterThan(0);
227 |     }
228 |   });
229 | });
230 |
231 | describe("getPlannerLocalizationState", () => {
232 |   it("is ready when both en and ar are available", () => {
233 |     expect(getPlannerLocalizationState(() => true)).toBe("ready");
234 |   });
235 |
236 |   it("withholds actions when exactly one pack is available (R19.6)", () => {
237 |     expect(getPlannerLocalizationState((lng) => lng === "en")).toBe(
238 |       "actions-withheld"
239 |     );
240 |     expect(getPlannerLocalizationState((lng) => lng === "ar")).toBe(
241 |       "actions-withheld"
242 |     );
243 |   });
244 |
245 |   it("hides content when neither pack is available (R19.7)", () => {
246 |     expect(getPlannerLocalizationState(() => false)).toBe("hidden");
247 |   });
248 | });
249 |

```

FILE: src/__tests__/unit/wellnessGoalInput.test.tsx

TYPE: .tsx | LINES: 1-178 | SHA256: c2c70780a12d8d168d499a787f5576e663f8364dc06c7c5c4e441134ab686df1

```

1 | import { describe, it, expect, vi } from "vitest";
2 | import { render, screen, fireEvent } from "@testing-library/react";
3 | import WellnessGoalInput from "@components/shared/WellnessGoalInput";
4 | import type { WellnessTarget } from "@types/habits";
5 |
6 | const makeTarget = (
7 |   overrides: Partial<WellnessTarget> = {}
8 | ): WellnessTarget => ({
9 |   habitType: "meditation",
10 |   targetValue: 10,
11 |   unit: "min",
12 |   ...overrides,
13 | });
14 |
15 | describe("WellnessGoalInput", () => {
16 |   it("renders the goal container", () => {
17 |     render(
18 |       <WellnessGoalInput
19 |         habitType="meditation"
20 |         target={makeTarget()}
21 |         progress={50}
22 |         currentValue={5}
23 |         onSave={vi.fn()}
24 |       />
25 |     );
26 |     expect(screen.getByTestId("wellness-goal-meditation")).toBeInTheDocument();
27 |   });
28 |
29 |   it("shows progress label with current/target values", () => {
30 |     render(
31 |       <WellnessGoalInput
32 |         habitType="hydration"
33 |         target={makeTarget({
34 |           habitType: "hydration",
35 |           targetValue: 8,
36 |           unit: "glasses",
37 |         })}
38 |         progress={50}
39 |         currentValue={4}
40 |         onSave={vi.fn()}
41 |       />
42 |     );
43 |     expect(
44 |       screen.getByTestId("goal-progress-label-hydration")
45 |     ).toHaveTextContent("4/8 glasses");
46 |   });
47 |
48 |   it("shows progress bar", () => {
49 |     render(
50 |       <WellnessGoalInput
51 |         habitType="exercise"
52 |         target={makeTarget({

```



```

53 |         habitType: "exercise",
54 |         targetValue: 30,
55 |         unit: "min",
56 |     }}}
57 |     progress={60}
58 |     currentValue={18}
59 |     onSave={vi.fn()}
60 | }>
61 | );
62 | expect(
63 |   screen.getByTestId("goal-progress-bar-exercise")
64 | ).toBeInTheDocument();
65 | });
66 |
67 | it("shows target met indicator when progress >= 100", () => {
68 |   render(
69 |     <WellnessGoalInput
70 |       habitType="meditation"
71 |       target={makeTarget()}
72 |       progress={100}
73 |       currentValue={10}
74 |       onSave={vi.fn()}
75 |     />
76 |   );
77 |   expect(screen.getByTestId("goal-met-meditation")).toBeInTheDocument();
78 |   expect(screen.getByText("Target met")).toBeInTheDocument();
79 | });
80 |
81 | it("does not show target met indicator when progress < 100", () => {
82 |   render(
83 |     <WellnessGoalInput
84 |       habitType="meditation"
85 |       target={makeTarget()}
86 |       progress={50}
87 |       currentValue={5}
88 |       onSave={vi.fn()}
89 |     />
90 |   );
91 |   expect(screen.queryByTestId("goal-met-meditation")).not.toBeInTheDocument();
92 | });
93 |
94 | it("shows edit mode when no target is set", () => {
95 |   render(
96 |     <WellnessGoalInput
97 |       habitType="meditation"
98 |       target={null}
99 |       progress={0}
100 |       currentValue={0}
101 |       onSave={vi.fn()}
102 |     />
103 |   );
104 |   expect(screen.getByTestId("goal-input-meditation")).toBeInTheDocument();
105 |   expect(screen.getByTestId("goal-save-meditation")).toBeInTheDocument();
106 | });
107 |
108 | it("switches to edit mode when Edit button is clicked", () => {
109 |   render(
110 |     <WellnessGoalInput
111 |       habitType="meditation"
112 |       target={makeTarget()}
113 |       progress={50}
114 |       currentValue={5}
115 |       onSave={vi.fn()}
116 |     />
117 |   );
118 |
119 |   fireEvent.click(screen.getByTestId("goal-edit-meditation"));
120 |   expect(screen.getByTestId("goal-input-meditation")).toBeInTheDocument();
121 | });
122 |
123 | it("calls onSave with correct target when Save is clicked", () => {
124 |   const onSave = vi.fn();
125 |   render(
126 |     <WellnessGoalInput
127 |       habitType="meditation"
128 |       target={null}
129 |       progress={0}
130 |       currentValue={0}
131 |       onSave={onSave}
132 |     />
133 |   );
134 |
135 |   const input = screen.getByTestId("goal-input-meditation");
136 |   fireEvent.change(input, { target: { value: "15" } });
137 |   fireEvent.click(screen.getByTestId("goal-save-meditation"));
138 |
139 |   expect(onSave).toHaveBeenCalledWith({
140 |     habitType: "meditation",
141 |     targetValue: 15,
142 |     unit: "min",
143 |   });
144 | });
145 |
146 | it("does not call onSave for invalid (zero) value", () => {
147 |   const onSave = vi.fn();
148 |   render(
149 |     <WellnessGoalInput
150 |       habitType="meditation"
151 |       target={null}
152 |       progress={0}
153 |       currentValue={0}
154 |       onSave={onSave}
155 |     />
156 |   );

```

```

157 |
158 |     const input = screen.getByTestId("goal-input-meditation");
159 |     fireEvent.change(input, { target: { value: "0" } });
160 |     fireEvent.click(screen.getByTestId("goal-save-meditation"));
161 |
162 |     expect(onSave).not.toHaveBeenCalled();
163 | });
164 |
165 | it("renders habit label correctly", () => {
166 |     render(
167 |         <WellnessGoalInput
168 |             habitType="sleep"
169 |             target={makeTarget({ habitType: "sleep", unit: "hours" })}
170 |             progress={70}
171 |             currentValue={7}
172 |             onSave={vi.fn()}
173 |         />
174 |     );
175 |     expect(screen.getByText("Sleep Goal")).toBeInTheDocument();
176 | });
177 | });
178 |

```

FILE: src/___tests___/unit/wellnessHabitLogger.test.tsx

TYPE: .tsx | LINES: 1-170 | SHA256: b3d424d2d955ab2c57d43b41120700b7df87bf1ddac0a9d1aa685cb5b581baef

```

1 | import { describe, it, expect, vi } from "vitest";
2 | import { render, screen, fireEvent } from "@testing-library/react";
3 | import WellnessHabitLogger from "@components/shared/WellnessHabitLogger";
4 | import type { WellnessHabitLog, WellnessHabitType } from "@types/habits";
5 |
6 | const makeLog = (
7 |     type: WellnessHabitType,
8 |     value: number | null = null
9 | ): WellnessHabitLog => ({
10 |     id: `log-${type}`,
11 |     studentId: "student-1",
12 |     date: "2024-03-15",
13 |     wellnessType: type,
14 |     value,
15 |     completedAt: "2024-03-15T10:00:00Z",
16 | });
17 |
18 | describe("WellnessHabitLogger", () => {
19 |     it("renders enabled habits", () => {
20 |         render(
21 |             <WellnessHabitLogger
22 |                 enabledHabits={["meditation", "exercise"]}
23 |                 todayLogs=[]
24 |                 onLog={vi.fn()}
25 |             />
26 |         );
27 |
28 |         expect(screen.getByTestId("wellness-habit-meditation")).toBeInTheDocument();
29 |         expect(screen.getByTestId("wellness-habit-exercise")).toBeInTheDocument();
30 |         expect(
31 |             screen.queryByTestId("wellness-habit-hydration")
32 |         ).not.toBeInTheDocument();
33 |         expect(
34 |             screen.queryByTestId("wellness-habit-sleep")
35 |         ).not.toBeInTheDocument();
36 |     });
37 |
38 |     it("renders nothing when no habits are enabled", () => {
39 |         render(
40 |             <WellnessHabitLogger enabledHabits=[] todayLogs=[] onLog={vi.fn()} />
41 |         );
42 |
43 |         expect(
44 |             screen.queryByTestId("wellness-habit-logger")
45 |         ).not.toBeInTheDocument();
46 |     });
47 |
48 |     it("shows checkmark for already-logged habits", () => {
49 |         render(
50 |             <WellnessHabitLogger
51 |                 enabledHabits={["meditation", "exercise"]}
52 |                 todayLogs={[makeLog("meditation")]}
53 |                 onLog={vi.fn()}
54 |             />
55 |         );
56 |
57 |         expect(screen.getByTestId("wellness-check-meditation")).toBeInTheDocument();
58 |         expect(
59 |             screen.queryByTestId("wellness-toggle-meditation")
60 |         ).not.toBeInTheDocument();
61 |     });
62 |
63 |     it("disables toggle for already-logged habits", () => {
64 |         render(
65 |             <WellnessHabitLogger
66 |                 enabledHabits={["meditation", "exercise"]}
67 |                 todayLogs={[makeLog("meditation")]}
68 |                 onLog={vi.fn()}
69 |             />
70 |         );
71 |
72 |         // Meditation should show checkmark, not toggle
73 |         expect(
74 |             screen.queryByTestId("wellness-toggle-meditation")
75 |         ).not.toBeInTheDocument();
76 |         // Exercise should still show toggle
77 |         expect(screen.getByTestId("wellness-toggle-exercise")).toBeInTheDocument();
78 |     });

```

```

79 |
80 | it("calls onLog when toggle is clicked", () => {
81 |   const onLog = vi.fn();
82 |   render(
83 |     <WellnessHabitLogger
84 |       enabledHabits={["meditation"]}
85 |       todayLogs=[]
86 |       onLog={onLog}
87 |     />
88 |   );
89 |
90 |   const toggle = screen.getByTestId("wellness-toggle-meditation");
91 |   fireEvent.click(toggle);
92 |
93 |   expect(onLog).toHaveBeenCalledWith("meditation", undefined);
94 | });
95 |
96 | it("calls onLog with value when value input is filled", () => {
97 |   const onLog = vi.fn();
98 |   render(
99 |     <WellnessHabitLogger
100 |       enabledHabits={["meditation"]}
101 |       todayLogs=[]
102 |       onLog={onLog}
103 |     />
104 |   );
105 |
106 |   const valueInput = screen.getByTestId("wellness-value-meditation");
107 |   fireEvent.change(valueInput, { target: { value: "15" } });
108 |
109 |   const toggle = screen.getByTestId("wellness-toggle-meditation");
110 |   fireEvent.click(toggle);
111 |
112 |   expect(onLog).toHaveBeenCalledWith("meditation", 15);
113 | });
114 |
115 | it("shows value input for each enabled habit", () => {
116 |   render(
117 |     <WellnessHabitLogger
118 |       enabledHabits=["meditation", "hydration", "exercise", "sleep"]
119 |       todayLogs=[]
120 |       onLog={vi.fn()}
121 |     />
122 |   );
123 |
124 |   expect(screen.getByTestId("wellness-value-meditation")).toBeInTheDocument();
125 |   expect(screen.getByTestId("wellness-value-hydration")).toBeInTheDocument();
126 |   expect(screen.getByTestId("wellness-value-exercise")).toBeInTheDocument();
127 |   expect(screen.getByTestId("wellness-value-sleep")).toBeInTheDocument();
128 | });
129 |
130 | it("disables value input for already-logged habits", () => {
131 |   render(
132 |     <WellnessHabitLogger
133 |       enabledHabits=["meditation"]
134 |       todayLogs=[makeLog("meditation", 10)]
135 |       onLog={vi.fn()}
136 |     />
137 |   );
138 |
139 |   const valueInput = screen.getByTestId("wellness-value-meditation");
140 |   expect(valueInput).toBeDisabled();
141 | });
142 |
143 | it("renders the Wellness section label", () => {
144 |   render(
145 |     <WellnessHabitLogger
146 |       enabledHabits=["meditation"]
147 |       todayLogs=[]
148 |       onLog={vi.fn()}
149 |     />
150 |   );
151 |
152 |   expect(screen.getByText("Wellness")).toBeInTheDocument();
153 | });
154 |
155 | it("renders all 4 habits when all enabled", () => {
156 |   render(
157 |     <WellnessHabitLogger
158 |       enabledHabits=["meditation", "hydration", "exercise", "sleep"]
159 |       todayLogs=[]
160 |       onLog={vi.fn()}
161 |     />
162 |   );
163 |
164 |   expect(screen.getByText("Meditation")).toBeInTheDocument();
165 |   expect(screen.getByText("Hydration")).toBeInTheDocument();
166 |   expect(screen.getByText("Exercise")).toBeInTheDocument();
167 |   expect(screen.getByText("Sleep")).toBeInTheDocument();
168 | });
169 | });
170 |

```

FILE: [src/__tests__/unit/wellnessPreferencesOptimistic.test.ts](#)

TYPE: .ts | LINES: 1-136 | SHA256: f1d0a7c7afb082afd0ffe362641e79b596796ff4fda5b9edbc86c7d914042eb

```

1 | // @vitest-environment happy-dom
2 | // =====
3 | // wellnessPreferencesOptimistic.test.ts
4 | // Feature: dashboard-and-ux-performance, Req 7 (extend optimistic UI to
5 | // high-frequency mutations) – useUpdateWellnessPreferences.
6 | // -----
7 | // The enabled-habits / parent-visibility toggles are deterministic settings, so
8 | // the hook applies them optimistically. This locks in the standard contract:

```

```

9 // - APPLY: onMutate updates the wellness.preferences cache immediately.
10 // - ROLLBACK: onError restores the pre-mutation snapshot (no wrong toggle).
11 // - SETTLE: onSettled invalidates the preferences query.
12 // supabase + sonner are mocked; the upsert resolution is controllable so the
13 // optimistic window can be observed before success/error.
14 // =====
15 import { describe, it, expect, vi, beforeEach } from "vitest";
16 import { renderHook, waitFor } from "@testing-library/react";
17 import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
18 import { createElement, type ReactNode } from "react";
19
20 const mockSingle = vi.fn();
21
22 vi.mock("@/lib/supabase", () => ({
23   supabase: {
24     from: () => ({
25       upsert: () => ({
26         select: () => ({
27           single: () => mockSingle(),
28         }),
29       }),
30     },
31   },
32 }));
33
34 vi.mock("sonner", () => ({
35   toast: { success: vi.fn(), error: vi.fn() },
36 }));
37
38 import { queryKeys } from "@/lib/queryKeys";
39 import { useUpdateWellnessPreferences } from "@/hooks/useWellnessPreferences";
40 import type { WellnessPreferences } from "@/types/habits";
41
42 const STUDENT_ID = "student-1";
43 const KEY = queryKeys.wellness.preferences(STUDENT_ID);
44
45 const INITIAL: WellnessPreferences = {
46   id: "pref-1",
47   studentId: STUDENT_ID,
48   enabledHabits: ["sleep"],
49   parentVisibility: false,
50   habitTargets: {},
51   reminderTimes: {},
52   dismissedOnboardingTips: [],
53 };
54
55 const makeClientAndWrapper = () => {
56   const client = new QueryClient({
57     defaultOptions: { mutations: { retry: false }, queries: { retry: false } },
58   });
59   client.setQueryData(KEY, INITIAL);
60   const wrapper = ({ children }: { children: ReactNode }) =>
61     createElement(QueryClientProvider, { client }, children);
62   return { client, wrapper };
63 };
64
65 describe("useUpdateWellnessPreferences optimistic (dashboard-and-ux-performance Req 7)", () => {
66   beforeEach(() => {
67     vi.clearAllMocks();
68   });
69
70   it("applies optimistically then rolls back on error", async () => {
71     // Hold the upsert pending so we can observe the optimistic window, then reject.
72     let rejectUpsert: (reason: unknown) => void = () => {};
73     mockSingle.mockReturnValue(
74       new Promise((_resolve, reject) => {
75         rejectUpsert = reject;
76       })
77     );
78
79     const { client, wrapper } = makeClientAndWrapper();
80     const { result } = renderHook(() => useUpdateWellnessPreferences(), {
81       wrapper,
82     });
83
84     result.current.mutate({
85       studentId: STUDENT_ID,
86       enabledHabits: ["sleep", "hydration"],
87       parentVisibility: true,
88     });
89
90     // APPLY: the cache reflects the new toggle state before the server responds.
91     await waitFor(() => {
92       const cached = client.getQueryData<WellnessPreferences>(KEY);
93       expect(cached?.parentVisibility).toBe(true);
94       expect(cached?.enabledHabits).toEqual(["sleep", "hydration"]);
95     });
96
97     // Now fail the server call !' ROLLBACK to the snapshot.
98     rejectUpsert(new Error("network down"));
99
100    await waitFor(() => {
101      const cached = client.getQueryData<WellnessPreferences>(KEY);
102      expect(cached?.parentVisibility).toBe(false);
103      expect(cached?.enabledHabits).toEqual(["sleep"]);
104    });
105  });
106
107   it("invalidates the preferences query on settle (success path)", async () => {
108     mockSingle.mockResolvedValue({
109       data: { id: "pref-1", student_id: STUDENT_ID },
110       error: null,
111     });
112

```

```

113 |     const { client, wrapper } = makeClientAndWrapper();
114 |     const invalidateSpy = vi.spyOn(client, "invalidateQueries");
115 |
116 |     const { result } = renderHook(() => useUpdateWellnessPreferences(), {
117 |       wrapper: Wrapper,
118 |     });
119 |
120 |     result.current.mutate({
121 |       studentId: STUDENT_ID,
122 |       enabledHabits: ["sleep", "hydration"],
123 |       parentVisibility: true,
124 |     });
125 |
126 |     await waitFor(() => expect(result.current.isSuccess).toBe(true));
127 |
128 |     // SETTLE: the preferences query is invalidated so it revalidates.
129 |     expect(invalidateSpy).toHaveBeenCalledOnceWith({ queryKey: KEY });
130 |     // The optimistic value persisted through success (no wrong rollback).
131 |     expect(
132 |       client.getQueryData<WellnessPreferences>(KEY)?.parentVisibility
133 |     ).toBe(true);
134 |   });
135 | });
136 |

```

FILE: src/__tests__/unit/wellnessReminderSettings.test.tsx

TYPE: .tsx | LINES: 1-119 | SHA256: 8d8af2df41fc162e30c969e67d3cb7913d71d2bf3caf7fd7e5321b6d6e64abf7

```

1 | import { describe, it, expect, vi } from "vitest";
2 | import { render, screen, fireEvent } from "@testing-library/react";
3 | import WellnessReminderSettings from "@components/shared/WellnessReminderSettings";
4 | import type { WellnessReminderConfig } from "@types/habits";
5 |
6 | const makeReminder = (
7 |   overrides: Partial<WellnessReminderConfig> = {}
8 | ): WellnessReminderConfig => ({
9 |   habitType: "meditation",
10 |   reminderTime: "09:00",
11 |   enabled: true,
12 |   ...overrides,
13 | });
14 |
15 | describe("WellnessReminderSettings", () => {
16 |   const defaultProps = {
17 |     reminders: [
18 |       makeReminder({
19 |         habitType: "meditation",
20 |         reminderTime: "08:00",
21 |         enabled: true,
22 |       }),
23 |       makeReminder({
24 |         habitType: "hydration",
25 |         reminderTime: null,
26 |         enabled: false,
27 |       }),
28 |     ],
29 |     onToggle: vi.fn(),
30 |     onTimeChange: vi.fn(),
31 |   };
32 |
33 |   it("renders the reminders container", () => {
34 |     render(<WellnessReminderSettings {...defaultProps} />);
35 |     expect(
36 |       screen.getByTestId("wellness-reminder-settings")
37 |     ).toBeInTheDocument();
38 |   });
39 |
40 |   it("renders a row for each reminder", () => {
41 |     render(<WellnessReminderSettings {...defaultProps} />);
42 |     expect(screen.getByTestId("reminder-meditation")).toBeInTheDocument();
43 |     expect(screen.getByTestId("reminder-hydration")).toBeInTheDocument();
44 |   });
45 |
46 |   it("renders nothing when reminders array is empty", () => {
47 |     const { container } = render(
48 |       <WellnessReminderSettings
49 |         reminders={[]}
50 |         onToggle={vi.fn()}
51 |         onTimeChange={vi.fn()}
52 |       />
53 |     );
54 |     expect(container.firstChild).toBeNull();
55 |   });
56 |
57 |   it("shows toggle in checked state for enabled reminders", () => {
58 |     render(<WellnessReminderSettings {...defaultProps} />);
59 |     const toggle = screen.getByTestId("reminder-toggle-meditation");
60 |     expect(toggle).toHaveAttribute("data-state", "checked");
61 |   });
62 |
63 |   it("shows toggle in unchecked state for disabled reminders", () => {
64 |     render(<WellnessReminderSettings {...defaultProps} />);
65 |     const toggle = screen.getByTestId("reminder-toggle-hydration");
66 |     expect(toggle).toHaveAttribute("data-state", "unchecked");
67 |   });
68 |
69 |   it("calls onToggle when toggle is clicked", () => {
70 |     const onToggle = vi.fn();
71 |     render(<WellnessReminderSettings {...defaultProps} onToggle={onToggle} />);
72 |
73 |     fireEvent.click(screen.getByTestId("reminder-toggle-hydration"));
74 |     expect(onToggle).toHaveBeenCalledOnceWith("hydration", true);
75 |   });
76 |

```

```

77 | it("disables time input when reminder is disabled", () => {
78 |   render(<WellnessReminderSettings {...defaultProps} />);
79 |   const timeInput = screen.getByTestId("reminder-time-hydration");
80 |   expect(timeInput).toBeDisabled();
81 | });
82 |
83 | it("enables time input when reminder is enabled", () => {
84 |   render(<WellnessReminderSettings {...defaultProps} />);
85 |   const timeInput = screen.getByTestId("reminder-time-meditation");
86 |   expect(timeInput).not.toBeDisabled();
87 | });
88 |
89 | it("calls onChange when time input changes", () => {
90 |   const onChange = vi.fn();
91 |   render(
92 |     <WellnessReminderSettings {...defaultProps} onChange={onChange} />
93 |   );
94 |
95 |   const timeInput = screen.getByTestId("reminder-time-meditation");
96 |   fireEvent.change(timeInput, { target: { value: "10:30" } });
97 |   expect(onChange).toHaveBeenCalledWith("meditation", "10:30");
98 | });
99 |
100 | it("shows correct time value for enabled reminder", () => {
101 |   render(<WellnessReminderSettings {...defaultProps} />);
102 |   const timeInput = screen.getByTestId(
103 |     "reminder-time-meditation"
104 |   ) as HTMLInputElement;
105 |   expect(timeInput.value).toBe("08:00");
106 | });
107 |
108 | it("renders the Reminders heading", () => {
109 |   render(<WellnessReminderSettings {...defaultProps} />);
110 |   expect(screen.getByText("Reminders")).toBeInTheDocument();
111 | });
112 |
113 | it("renders habit labels correctly", () => {
114 |   render(<WellnessReminderSettings {...defaultProps} />);
115 |   expect(screen.getByText("Meditation")).toBeInTheDocument();
116 |   expect(screen.getByText("Hydration")).toBeInTheDocument();
117 | });
118 | });
119 |

```

FILE: src/___tests___/unit/wellnessSettings.test.tsx

TYPE: .tsx | LINES: 1-118 | SHA256: b269b8d6ce6aeebe0913a331d33d53fc0274c8c108a92cec37f74556a0afebfff

```

1 | import { describe, it, expect, vi } from "vitest";
2 | import { render, screen, fireEvent } from "@testing-library/react";
3 | import WellnessSettingsPanel from "@components/shared/WellnessSettingsPanel";
4 | import type { WellnessHabitType } from "@types/habits";
5 |
6 | describe("WellnessSettingsPanel", () => {
7 |   const defaultProps = {
8 |     enabledHabits: [] as WellnessHabitType[],
9 |     parentVisibility: false,
10 |     onToggleHabit: vi.fn(),
11 |     onToggleParentVisibility: vi.fn(),
12 |   };
13 |
14 |   it("renders 4 wellness habit toggles", () => {
15 |     render(<WellnessSettingsPanel {...defaultProps} />);
16 |
17 |     expect(
18 |       screen.getByTestId("wellness-setting-meditation")
19 |     ).toBeInTheDocument();
20 |     expect(
21 |       screen.getByTestId("wellness-setting-hydration")
22 |     ).toBeInTheDocument();
23 |     expect(screen.getByTestId("wellness-setting-exercise")).toBeInTheDocument();
24 |     expect(screen.getByTestId("wellness-setting-sleep")).toBeInTheDocument();
25 |   });
26 |
27 |   it("renders parent visibility toggle", () => {
28 |     render(<WellnessSettingsPanel {...defaultProps} />);
29 |
30 |     expect(
31 |       screen.getByTestId("wellness-parent-visibility")
32 |     ).toBeInTheDocument();
33 |     expect(
34 |       screen.getByTestId("wellness-parent-visibility-toggle")
35 |     ).toBeInTheDocument();
36 |   });
37 |
38 |   it("shows correct checked state for enabled habits", () => {
39 |     render(
40 |       <WellnessSettingsPanel
41 |         {...defaultProps}
42 |         enabledHabits={["meditation", "sleep"]}
43 |       />
44 |     );
45 |
46 |     const meditationToggle = screen.getByTestId(
47 |       "wellness-setting-toggle-meditation"
48 |     );
49 |     const hydrationToggle = screen.getByTestId(
50 |       "wellness-setting-toggle-hydration"
51 |     );
52 |     const sleepToggle = screen.getByTestId("wellness-setting-toggle-sleep");
53 |
54 |     expect(meditationToggle).toHaveAttribute("data-state", "checked");
55 |     expect(hydrationToggle).toHaveAttribute("data-state", "unchecked");
56 |     expect(sleepToggle).toHaveAttribute("data-state", "checked");
57 |   });

```

```

58 |
59 | it("calls onToggleHabit when a habit toggle is clicked", () => {
60 |   const onToggleHabit = vi.fn();
61 |   render(
62 |     <WellnessSettingsPanel {...defaultProps} onToggleHabit={onToggleHabit} />
63 |   );
64 |
65 |   const meditationToggle = screen.getByTestId(
66 |     "wellness-setting-toggle-meditation"
67 |   );
68 |   fireEvent.click(meditationToggle);
69 |
70 |   expect(onToggleHabit).toHaveBeenCalledWith("meditation", true);
71 | });
72 |
73 | it("calls onToggleParentVisibility when parent visibility toggle is clicked", () => {
74 |   const onToggleParentVisibility = vi.fn();
75 |   render(
76 |     <WellnessSettingsPanel
77 |       {...defaultProps}
78 |       onToggleParentVisibility={onToggleParentVisibility}
79 |     />
80 |   );
81 |
82 |   const toggle = screen.getByTestId("wellness-parent-visibility-toggle");
83 |   fireEvent.click(toggle);
84 |
85 |   expect(onToggleParentVisibility).toHaveBeenCalledWith(true);
86 | });
87 |
88 | it("shows parent visibility as checked when enabled", () => {
89 |   render(<WellnessSettingsPanel {...defaultProps} parentVisibility={true} />);
90 |
91 |   const toggle = screen.getByTestId("wellness-parent-visibility-toggle");
92 |   expect(toggle).toHaveAttribute("data-state", "checked");
93 | });
94 |
95 | it("renders habit descriptions", () => {
96 |   render(<WellnessSettingsPanel {...defaultProps} />);
97 |
98 |   expect(
99 |     screen.getByText("Track daily meditation sessions (5+ minutes)")
100 |   ).toBeInTheDocument();
101 |   expect(
102 |     screen.getByText("Track daily water intake (8 glasses)")
103 |   ).toBeInTheDocument();
104 |   expect(
105 |     screen.getByText("Track daily physical activity (30+ minutes)")
106 |   ).toBeInTheDocument();
107 |   expect(
108 |     screen.getByText("Track nightly sleep duration (7+ hours)")
109 |   ).toBeInTheDocument();
110 | });
111 |
112 | it("renders the Wellness Preferences heading", () => {
113 |   render(<WellnessSettingsPanel {...defaultProps} />);
114 |
115 |   expect(screen.getByText("Wellness Preferences")).toBeInTheDocument();
116 | });
117 | });
118 |

```

FILE: [src/_tests_/unit/wellnessTipCard.test.tsx](#)

TYPE: .tsx | LINES: 1-104 | SHA256: 0f0431ee9cf4a169c4971fcd24f6101de014f23fe770b42b97380b9c1210acfd

```

1 | import { describe, it, expect, vi } from "vitest";
2 | import { render, screen, fireEvent } from "@testing-library/react";
3 | import WellnessTipCard from "@components/shared/WellnessTipCard";
4 | import type { WellnessTip } from "@types/habits";
5 |
6 | const makeTip = (overrides: Partial<WellnessTip> = {}): WellnessTip => ({
7 |   id: "tip-1",
8 |   habitType: "meditation",
9 |   text: "Start with just 2 minutes of meditation.",
10 |   isOnboarding: false,
11 |   ...overrides,
12 | });
13 |
14 | describe("WellnessTipCard", () => {
15 |   it("renders tip text", () => {
16 |     const tip = makeTip({ text: "Drink more water throughout the day." });
17 |     render(<WellnessTipCard tip={tip} isOnboarding={false} />);
18 |
19 |     expect(screen.getByTestId("wellness-tip-text")).toHaveTextContent(
20 |       "Drink more water throughout the day."
21 |     );
22 |   });
23 |
24 |   it("renders the card container", () => {
25 |     render(<WellnessTipCard tip={makeTip()} isOnboarding={false} />);
26 |     expect(screen.getByTestId("wellness-tip-card")).toBeInTheDocument();
27 |   });
28 |
29 |   it("does not show dismiss button for rotating tips", () => {
30 |     render(<WellnessTipCard tip={makeTip()} isOnboarding={false} />);
31 |     expect(
32 |       screen.queryByTestId("wellness-tip-dismiss")
33 |     ).not.toBeInTheDocument();
34 |   });
35 |
36 |   it("shows dismiss button for onboarding tips", () => {
37 |     render(
38 |       <WellnessTipCard
39 |         tip={makeTip({ isOnboarding: true })}

```

```

40 |         isOnboarding={true}
41 |         onDismiss={vi.fn()}
42 |     />
43 | );
44 | expect(screen.getByTestId("wellness-tip-dismiss")).toBeInTheDocument();
45 | });
46 |
47 | it("calls onDismiss when dismiss button is clicked", () => {
48 |     const onDismiss = vi.fn();
49 |     render(
50 |         <WellnessTipCard
51 |             tip={makeTip({ isOnboarding: true })}
52 |             isOnboarding={true}
53 |             onDismiss={onDismiss}
54 |         />
55 |     );
56 |
57 |     fireEvent.click(screen.getByTestId("wellness-tip-dismiss"));
58 |     expect(onDismiss).toHaveBeenCalledTimes(1);
59 | });
60 |
61 | it("renders resource link when resourceUrl is provided", () => {
62 |     const tip = makeTip({
63 |         resourceUrl: "https://example.com/tips",
64 |         resourceLabel: "Read more",
65 |     });
66 |     render(<WellnessTipCard tip={tip} isOnboarding={false} />);
67 |
68 |     const link = screen.getByTestId("wellness-tip-resource");
69 |     expect(link).toBeInTheDocument();
70 |     expect(link).toHaveAttribute("href", "https://example.com/tips");
71 |     expect(link).toHaveAttribute("target", "_blank");
72 |     expect(link).toHaveAttribute("rel", "noopener noreferrer");
73 |     expect(link).toHaveTextContent("Read more");
74 | });
75 |
76 | it("does not render resource link when resourceUrl is absent", () => {
77 |     render(<WellnessTipCard tip={makeTip()} isOnboarding={false} />);
78 |     expect(
79 |         screen.queryByTestId("wellness-tip-resource")
80 |     ).not.toBeInTheDocument();
81 | });
82 |
83 | it('uses default "Learn more" label when resourceLabel is not provided', () => {
84 |     const tip = makeTip({ resourceUrl: "https://example.com" });
85 |     render(<WellnessTipCard tip={tip} isOnboarding={false} />);
86 |
87 |     expect(screen.getByTestId("wellness-tip-resource")).toHaveTextContent(
88 |         "Learn more"
89 |     );
90 | });
91 |
92 | it("does not show dismiss button for onboarding tip without onDismiss handler", () => {
93 |     render(
94 |         <WellnessTipCard
95 |             tip={makeTip({ isOnboarding: true })}
96 |             isOnboarding={true}
97 |         />
98 |     );
99 |     expect(
100 |         screen.queryByTestId("wellness-tip-dismiss")
101 |     ).not.toBeInTheDocument();
102 | });
103 | });
104 |

```

FILE: [src/__tests__/unit/wellnessXpConfig.test.ts](#)

TYPE: .ts | LINES: 1-45 | SHA256: fd8f833ba735b99ad8a898fcc6556fda4bb324650b49b7e7fbb9f299e2a7ebe8

```

1 | import { describe, it, expect } from "vitest";
2 | import { wellnessXpAmountSchema } from "@lib/schemas/wellnessXpAmount";
3 |
4 | describe("wellnessXpAmountSchema", () => {
5 |     it("accepts 0 (disables XP)", () => {
6 |         expect(wellnessXpAmountSchema.safeParse(0).success).toBe(true);
7 |     });
8 |
9 |     it("accepts the default value of 5", () => {
10 |         expect(wellnessXpAmountSchema.safeParse(5).success).toBe(true);
11 |     });
12 |
13 |     it("accepts the maximum value of 25", () => {
14 |         expect(wellnessXpAmountSchema.safeParse(25).success).toBe(true);
15 |     });
16 |
17 |     it("accepts all integers in the valid range 0-25", () => {
18 |         for (let i = 0; i <= 25; i++) {
19 |             const result = wellnessXpAmountSchema.safeParse(i);
20 |             expect(result.success).toBe(true);
21 |         }
22 |     });
23 |
24 |     it("rejects negative values", () => {
25 |         expect(wellnessXpAmountSchema.safeParse(-1).success).toBe(false);
26 |         expect(wellnessXpAmountSchema.safeParse(-10).success).toBe(false);
27 |     });
28 |
29 |     it("rejects values above 25", () => {
30 |         expect(wellnessXpAmountSchema.safeParse(26).success).toBe(false);
31 |         expect(wellnessXpAmountSchema.safeParse(100).success).toBe(false);
32 |     });
33 |
34 |     it("rejects non-integer numbers", () => {
35 |         expect(wellnessXpAmountSchema.safeParse(5.5).success).toBe(false);

```



```

36 |     expect(wellnessXpAmountSchema.safeParse(0.1).success).toBe(false);
37 |   });
38 |
39 |   it("rejects non-number types", () => {
40 |     expect(wellnessXpAmountSchema.safeParse("5").success).toBe(false);
41 |     expect(wellnessXpAmountSchema.safeParse(null).success).toBe(false);
42 |     expect(wellnessXpAmountSchema.safeParse(undefined).success).toBe(false);
43 |   });
44 | });
45 |

```

FILE: [src/_tests_/unit/xpAwardToast.test.tsx](#)

TYPE: .tsx | LINES: 1-143 | SHA256: e29ed78259cbb52b953599e933403fc71e846da685c183b8dca3180fb3469f11

```

1 | import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
2 | import { render, screen, act } from "@testing-library/react";
3 |
4 | // -----
5 | // Mock framer-motion to avoid animation complexity in tests
6 | // -----
7 | vi.mock("framer-motion", () => ({
8 |   motion: {
9 |     div: ({
10 |       children,
11 |       ...props
12 |     }: React.PropsWithChildren<Record<string, unknown>>) => {
13 |       const { initial, animate, exit, transition, ...rest } = props;
14 |       void initial;
15 |       void animate;
16 |       void exit;
17 |       void transition;
18 |       return <div {...rest}>{children}</div>;
19 |     },
20 |   },
21 |   AnimatePresence: ({
22 |     children,
23 |   }: React.PropsWithChildren<Record<string, unknown>>) => <>{children}</>,
24 |   useReducedMotion: () => false,
25 | }));
26 |
27 | // -----
28 | // Mock canvas-confetti
29 | // -----
30 | vi.mock("canvas-confetti", () => ({
31 |   default: vi.fn(),
32 | }));
33 |
34 | import XPAwardToast from "@components/shared/XPAwardToast";
35 |
36 | // -----
37 | // Tests
38 | // -----
39 | describe("XPAwardToast", () => {
40 |   beforeEach(() => {
41 |     vi.clearAllMocks();
42 |     vi.useFakeTimers();
43 |   });
44 |
45 |   afterEach(() => {
46 |     vi.useRealTimers();
47 |   });
48 |
49 |   it("renders the XP amount", () => {
50 |     render(<XPAwardToast xpAmount={50} source="On-time Submission" />);
51 |
52 |     // The count-up starts at 0 and animates to 50.
53 |     // With fake timers the rAF-based animation won't run, so we see the
54 |     // initial value. The text "+0 XP" or "+50 XP" should be present.
55 |     const xpText = screen.getByRole("status");
56 |     expect(xpText).toBeInTheDocument();
57 |     expect(xpText.textContent).toContain("XP");
58 |   });
59 |
60 |   it("displays the source label", () => {
61 |     render(<XPAwardToast xpAmount={25} source="Daily Login" />);
62 |
63 |     expect(screen.getByText("Daily Login")).toBeInTheDocument();
64 |   });
65 |
66 |   it("shows level-up message when levelUp is true", () => {
67 |     render(
68 |       <XPAwardToast
69 |         xpAmount={100}
70 |         source="Streak Milestone"
71 |         levelUp={true}
72 |         newLevel={5}
73 |       />
74 |     );
75 |
76 |     expect(screen.getByText("Level Up! !' Level 5")).toBeInTheDocument();
77 |   });
78 |
79 |   it("hides level-up message when levelUp is false", () => {
80 |     render(
81 |       <XPAwardToast xpAmount={50} source="On-time Submission" levelUp={false} />
82 |     );
83 |
84 |     expect(screen.queryByText(/Level Up/)).not.toBeInTheDocument();
85 |   });
86 |
87 |   it("hides level-up message when levelUp is not provided", () => {
88 |     render(<XPAwardToast xpAmount={50} source="On-time Submission" />);
89 |
90 |     expect(screen.queryByText(/Level Up/)).not.toBeInTheDocument();

```

```

91   });
92
93   it("auto-dismisses after 3 seconds", () => {
94     const onComplete = vi.fn();
95     render(
96       <XPAwardToast
97         xpAmount={50}
98         source="On-time Submission"
99         onComplete={onComplete}
100       />
101     );
102
103     // Before 3 seconds, the toast should be visible
104     expect(screen.getByRole("status")).toBeInTheDocument();
105
106     // Advance past the 3-second auto-dismiss timer
107     act(() => {
108       vi.advanceTimersByTime(3100);
109     });
110
111     // With mocked AnimatePresence the exit animation is instant,
112     // so the element should be removed from the DOM
113     expect(screen.queryByRole("status")).not.toBeInTheDocument();
114   });
115
116   it("fires confetti on mount", async () => {
117     const confettiMock = await import("canvas-confetti");
118     render(<XPAwardToast xpAmount={50} source="On-time Submission" />);
119
120     // launchConfetti lazy-loads canvas-confetti via dynamic import(); flush the
121     // microtask chain so the (mocked) library resolves and fires before asserting.
122     await act(async () => {
123       for (let i = 0; i < 5; i++) await Promise.resolve();
124     });
125
126     expect(confettiMock.default).toHaveBeenCalledTimes(1);
127     expect(confettiMock.default).toHaveBeenCalledWith(
128       expect.objectContaining({
129         particleCount: 30,
130         spread: 60,
131         colors: ["#f59e0b", "#eab308", "#fbbf24"],
132       })
133     );
134   });
135
136   it('has accessible role="status" and aria-live', () => {
137     render(<XPAwardToast xpAmount={50} source="Test" />);
138
139     const toast = screen.getByRole("status");
140     expect(toast).toHaveAttribute("aria-live", "polite");
141   });
142 });
143

```

FILE: src/__tests__/unit/xpBalanceBadge.test.tsx

```
TYPE: .tsx | LINES: 1-82 | SHA256: 94971391e89c058a1a512ba5789d1f3b0657dd96a6e0ea21e8acb362c90af96b
```

[illegible]

```

48 |     render(<XPBalanceBadge />);
49 |     expect(screen.getByText("0 XP")).toBeTruthy();
50 | });
51 |
52 | it("applies size classes for sm variant", () => {
53 |     mockUseXPBalance.mockReturnValue({
54 |         data: { balance: 100 },
55 |         isLoading: false,
56 |     });
57 |     const { container } = render(<XPBalanceBadge size="sm" />);
58 |     const badge = container.firstChild;
59 |     expect(badge?.className).toContain("text-xs");
60 | });
61 |
62 | it("applies size classes for lg variant", () => {
63 |     mockUseXPBalance.mockReturnValue({
64 |         data: { balance: 100 },
65 |         isLoading: false,
66 |     });
67 |     const { container } = render(<XPBalanceBadge size="lg" />);
68 |     const badge = container.firstChild;
69 |     expect(badge?.className).toContain("text-base");
70 | });
71 |
72 | it("applies custom className", () => {
73 |     mockUseXPBalance.mockReturnValue({
74 |         data: { balance: 100 },
75 |         isLoading: false,
76 |     });
77 |     const { container } = render(<XPBalanceBadge className="custom-class" />);
78 |     const badge = container.firstChild;
79 |     expect(badge?.className).toContain("custom-class");
80 | });
81 | });
82 |

```

FILE: [src/__tests__/unit/xpEconomistDashboard.test.tsx](#)

TYPE: .tsx | LINES: 1-93 | SHA256: c7c96daa55c0c0624a96c98d1d36b66497ef8f8a60efbb4c324918c71c7862ea

```

1 | // =====
2 | // Unit Test: XP Economist Dashboard
3 | // Task 26.1 – Earn/spend ratio display, inflation indicator
4 | // =====
5 |
6 | import { describe, it, expect } from "vitest";
7 | import {
8 |     computeEarnSpendRatio,
9 |     classifyInflation,
10 |     getStatusLabel,
11 | } from "@lib/earnSpendRatioCalculator";
12 |
13 | describe("XP Economist Dashboard", () => {
14 |     describe("computeEarnSpendRatio", () => {
15 |         it("computes correct ratio for healthy economy", () => {
16 |             const result = computeEarnSpendRatio({
17 |                 totalEarned: 3000,
18 |                 totalSpent: 1000,
19 |             });
20 |             expect(result.ratio).toBe(3);
21 |             expect(result.status).toBe("healthy");
22 |             expect(result.statusLabel).toBe("Healthy");
23 |         });
24 |
25 |         it("detects inflationary economy (ratio > 4)", () => {
26 |             const result = computeEarnSpendRatio({
27 |                 totalEarned: 5000,
28 |                 totalSpent: 1000,
29 |             });
30 |             expect(result.ratio).toBe(5);
31 |             expect(result.status).toBe("inflationary");
32 |         });
33 |
34 |         it("detects deflationary economy (ratio < 2)", () => {
35 |             const result = computeEarnSpendRatio({
36 |                 totalEarned: 1000,
37 |                 totalSpent: 1000,
38 |             });
39 |             expect(result.ratio).toBe(1);
40 |             expect(result.status).toBe("deflationary");
41 |         });
42 |
43 |         it("handles no spending", () => {
44 |             const result = computeEarnSpendRatio({
45 |                 totalEarned: 5000,
46 |                 totalSpent: 0,
47 |             });
48 |             expect(result.ratio).toBeNull();
49 |             expect(result.status).toBe("no_spending");
50 |         });
51 |
52 |         it("clamps negative values to 0", () => {
53 |             const result = computeEarnSpendRatio({
54 |                 totalEarned: -100,
55 |                 totalSpent: -50,
56 |             });
57 |             expect(result.totalEarned).toBe(0);
58 |             expect(result.totalSpent).toBe(0);
59 |         });
60 |     });
61 |
62 |     describe("classifyInflation", () => {
63 |         it("returns no_spending for null", () => {
64 |             expect(classifyInflation(null)).toBe("no_spending");
65 |         });

```

```

66 |     it("returns healthy for ratio 2", () => {
67 |         expect(classifyInflation(2)).toBe("healthy");
68 |     });
69 |
70 |     it("returns healthy for ratio 4", () => {
71 |         expect(classifyInflation(4)).toBe("healthy");
72 |     });
73 |
74 |     it("returns inflationary for ratio 4.1", () => {
75 |         expect(classifyInflation(4.1)).toBe("inflationary");
76 |     });
77 |
78 |     it("returns deflationary for ratio 1.9", () => {
79 |         expect(classifyInflation(1.9)).toBe("deflationary");
80 |     });
81 | });
82 |
83 | describe("getStatusLabel", () => {
84 |     it("returns correct labels", () => {
85 |         expect(getStatusLabel("healthy")).toBe("Healthy");
86 |         expect(getStatusLabel("inflationary")).toBe("Inflationary");
87 |         expect(getStatusLabel("deflationary")).toBe("Deflationary");
88 |         expect(getStatusLabel("no_spending")).toBe("No spending yet");
89 |     });
90 | });
91 | });
92 | });
93 |

```

FILE: src/___tests___/unit/xpSchedule.test.ts

TYPE: .ts | LINES: 1-106 | SHA256: cfe577e6b109b8ea84d466902bd7479efe20ab91e6840e238e38ba71a85b12fd

```

1 | import { describe, it, expect } from "vitest";
2 | import {
3 |     XP_SCHEDULE,
4 |     LATE_SUBMISSION_XP,
5 |     LATE_QUIZ_XP,
6 | } from "@lib/xpSchedule";
7 | import type { XPSource } from "@types/app";
8 |
9 | describe("XP Schedule", () => {
10 |     it("defines XP amounts for all XPSource types", () => {
11 |         const expectedSources: XPSource[] = [
12 |             "login",
13 |             "submission",
14 |             "grade",
15 |             "journal",
16 |             "streak_milestone",
17 |             "perfect_day",
18 |             "first_attempt_bonus",
19 |             "perfect_rubric",
20 |             "badge_earned",
21 |             "level_up",
22 |             "streak_freeze_purchase",
23 |             "discussion_question",
24 |             "discussion_answer",
25 |             "survey_completion",
26 |             "quiz_completion",
27 |             "onboarding_personality",
28 |             "onboarding_learning_style",
29 |             "onboarding_baseline",
30 |             "onboarding_complete",
31 |             "onboarding_self_efficacy",
32 |             "onboarding_study_strategy",
33 |             "micro_assessment",
34 |             "profile_complete",
35 |             "starter_session_complete",
36 |             "wellness_habit",
37 |             "practice_quiz",
38 |             "study_session",
39 |             "planner_task",
40 |             "session_reflection",
41 |             "weekly_goal",
42 |             "tutor_engagement",
43 |             "tutor_rating",
44 |         ];
45 |
46 |         for (const source of expectedSources) {
47 |             expect(XP_SCHEDULE).toHaveProperty(source);
48 |             expect(typeof XP_SCHEDULE[source]).toBe("number");
49 |         }
50 |     });
51 |
52 |     it("matches requirement 21.1 XP amounts", () => {
53 |         expect(XP_SCHEDULE.login).toBe(10);
54 |         expect(XP_SCHEDULE.submission).toBe(25);
55 |         expect(XP_SCHEDULE.grade).toBe(15);
56 |         expect(XP_SCHEDULE.journal).toBe(20);
57 |         expect(XP_SCHEDULE.streak_milestone).toBe(50);
58 |         expect(XP_SCHEDULE.perfect_day).toBe(50);
59 |         expect(XP_SCHEDULE.first_attempt_bonus).toBe(10);
60 |         expect(XP_SCHEDULE.perfect_rubric).toBe(75);
61 |         expect(XP_SCHEDULE.discussion_question).toBe(10);
62 |         expect(XP_SCHEDULE.discussion_answer).toBe(15);
63 |         expect(XP_SCHEDULE.survey_completion).toBe(15);
64 |         expect(XP_SCHEDULE.quiz_completion).toBe(50);
65 |     });
66 |
67 |     it("has late submission XP less than or equal to on-time submission XP", () => {
68 |         expect(LATE_SUBMISSION_XP).toBeLessThanOrEqual(XP_SCHEDULE.submission);
69 |         expect(LATE_SUBMISSION_XP).toBe(15);
70 |     });
71 |
72 |     it("has late quiz XP less than on-time quiz XP", () => {

```

```

73 |     expect(LATE_QUIZ_XP).toBeLessThan(XP_SCHEDULE.quiz_completion);
74 |     expect(LATE_QUIZ_XP).toBe(15);
75 | });
76 |
77 | it("has non-negative XP for all sources", () => {
78 |   for (const [, amount] of Object.entries(XP_SCHEDULE)) {
79 |     expect(amount).toBeGreaterThanOrEqual(0);
80 |   }
81 | });
82 |
83 | it("has zero XP for variable-amount sources", () => {
84 |   expect(XP_SCHEDULE.badge_earned).toBe(0);
85 |   expect(XP_SCHEDULE.level_up).toBe(0);
86 |   expect(XP_SCHEDULE.streak_freeze_purchase).toBe(0);
87 | });
88 |
89 | it("matches onboarding XP amounts from spec", () => {
90 |   expect(XP_SCHEDULE.onboarding_personality).toBe(25);
91 |   expect(XP_SCHEDULE.onboarding_learning_style).toBe(25);
92 |   expect(XP_SCHEDULE.onboarding_self_efficacy).toBe(25);
93 |   expect(XP_SCHEDULE.onboarding_study_strategy).toBe(25);
94 |   expect(XP_SCHEDULE.onboarding_baseline).toBe(20);
95 |   expect(XP_SCHEDULE.onboarding_complete).toBe(50);
96 |   expect(XP_SCHEDULE.micro_assessment).toBe(10);
97 |   expect(XP_SCHEDULE.profile_complete).toBe(30);
98 |   expect(XP_SCHEDULE.starter_session_complete).toBe(15);
99 | });
100 |
101 | it("matches tutor engagement XP amounts from Requirement 17", () => {
102 |   expect(XP_SCHEDULE.tutor_engagement).toBe(15);
103 |   expect(XP_SCHEDULE.tutor_rating).toBe(5);
104 | });
105 | });
106 |

```

FILE: src/App.tsx

TYPE: .tsx | LINES: 1-70 | SHA256: 3b4c0f9039f61c541f835b81ace81cbe4d9ad425aedcda538f294c54ddc2d76a

```

1 | import { BrowserRouter } from "react-router-dom";
2 | import { NuqsAdapter } from "nuqs/adapters/react-router/v7";
3 | import { QueryClientProvider } from "@tanstack/react-query";
4 | import { MotionConfig } from "framer-motion";
5 | import * as Sentry from "@sentry/react";
6 | import { SpeedInsights } from "@vercel/speed-insights/react";
7 | import { Analytics } from "@vercel/analytics/react";
8 | import { Toaster } from "@components/ui/sonner";
9 | import GamificationFeedbackHost from "@components/shared/GamificationFeedbackHost";
10 | import { AuthProvider } from "@providers/AuthProvider";
11 | import { ThemeProvider } from "@providers/ThemeProvider";
12 | import { LanguageProvider } from "@providers/LanguageProvider";
13 | import ErrorBoundary from "@components/shared/ErrorBoundary";
14 | import AppRouter from "@router/AppRouter";
15 | import SkipToMain from "@components/shared/SkipToMain";
16 | import { offlineQueue } from "@lib/offlineQueue";
17 | import { queryClient } from "@lib/queryClient";
18 | import { lazy, Suspense } from "react";
19 |
20 | const ReactQueryDevtoolsLazy = lazy(() =>
21 |   import("@tanstack/react-query-devtools").then((m) => ({
22 |     default: m.ReactQueryDevtools,
23 |   }))
24 | );
25 |
26 | // Sentry is initialized only through the consent-gated path
27 | // (initAnalyticsIfConsented in main.tsx / CookieConsentBanner), which guards on
28 | // Sentry.isInitialized(). Do NOT call initSentry() unconditionally here – that
29 | // would start error/replay capture before cookie consent is resolved (FERPA/GDPR).
30 |
31 | // Initialize offline queue – auto-flushes queued events when connectivity returns
32 | const cleanupOfflineQueue = offlineQueue.init();
33 | if (import.meta.hot) {
34 |   import.meta.hot.dispose(() => cleanupOfflineQueue());
35 | }
36 |
37 | const App = () => (
38 |   <Sentry.ErrorBoundary fallback={<p>An unexpected error occurred.</p>}>
39 |     <ErrorBoundary>
40 |       <BrowserRouter>
41 |         <NuqsAdapter>
42 |           <QueryClientProvider client={queryClient}>
43 |             <MotionConfig reducedMotion="user">
44 |               <AuthProvider>
45 |                 <LanguageProvider>
46 |                   <ThemeProvider>
47 |                     <SkipToMain />
48 |                     <AppRouter />
49 |                     <GamificationFeedbackHost />
50 |                     <Toaster richColors position="bottom-center" />
51 |                   </ThemeProvider>
52 |                 </LanguageProvider>
53 |               </AuthProvider>
54 |             </MotionConfig>
55 |             {import.meta.env.DEV && (
56 |               <Suspense>
57 |                 <ReactQueryDevtoolsLazy initialIsOpen={false} />
58 |               </Suspense>
59 |             )}
60 |           </QueryClientProvider>
61 |         </NuqsAdapter>
62 |       </BrowserRouter>
63 |       <SpeedInsights />
64 |       <Analytics />
65 |     </ErrorBoundary>
66 |   </Sentry.ErrorBoundary>

```

```

67 | );
68 |
69 | export default App;
70 |

```

FILE: src/app/index.ts

TYPE: .ts | LINES: 1-15 | SHA256: ec5f96d2cffc6a4025890405de4dab0a5910cd1305949b2c3cf7f4fd4ae8a7ff

```

1 | /**
2 |  * App shell (L-shell) – the new Path-A frame around page content.
3 |  *
4 |  * Lands here at P1 (rebuild tasks.md 1.1-1.4): router integration, providers,
5 |  * the five role layouts, and chrome (GlobalHeader / Sidebar / MobileTabBar) driven
6 |  * by `src/lib/navItems.ts`. Route *paths* and guards are kept verbatim (guardrail
7 |  * G.1); the shell only restyles the frame and swaps the element rendered per route.
8 |  *
9 |  * Shared shell, role-configured – NOT forked per role. Role identity is expressed
10 |  * via `[data-role]` accent scopes (see design-system/tokens.css), not separate shells.
11 |  *
12 |  * Empty until P1 – no screens yet (task 0.8).
13 |  */
14 | export {};
15 |

```

FILE: src/app/RoleAppShell.tsx

TYPE: .tsx | LINES: 1-76 | SHA256: 0700ab90cabe9946b74d817af317ce91c0a0d5f57c8fc6c6724fc9f5cfa675ad

```

1 | import type { ReactNode } from "react";
2 | import GlobalHeader from "@components/shared/GlobalHeader";
3 | import Sidebar from "@components/shared/Sidebar";
4 | import { SidebarProvider } from "@components/shared/SidebarContext";
5 | import GuidedTour from "@components/shared/GuidedTour";
6 | import EmailVerificationBanner from "@components/shared/EmailVerificationBanner";
7 | import type { UserRole } from "@types/app";
8 | import { cn } from "@lib/utlis";
9 | import { usePageViewLogger } from "@hooks/usePageViewLogger";
10 |
11 | interface RoleAppShellProps {
12 |   userRole: UserRole;
13 |   children: ReactNode;
14 |   rail?: ReactNode;
15 | }
16 |
17 | /**
18 |  * Shared role shell for the Path-A frontend rebuild.
19 |  *
20 |  * The layout contract is defined by the `--app-*` tokens: a desktop three-column
21 |  * grid (sidebar, start-aligned content, optional rail), a two-column laptop
22 |  * layout, and a single-column mobile layout with its navigation supplied by the
23 |  * existing bottom bar. Feature routes provide only their content and optional rail.
24 |  */
25 | const RoleAppShell = ({ userRole, children, rail }: RoleAppShellProps) => {
26 |   const hasRail = rail != null;
27 |
28 |   // Keep route analytics attached to the shared shell so every standard role
29 |   // layout follows the same lifecycle. The logger itself scopes persistence to
30 |   // student profiles and therefore does not emit unsupported staff events.
31 |   usePageViewLogger();
32 |
33 |   return (
34 |     <SidebarProvider>
35 |       <div
36 |         data-role={userRole}
37 |         data-norail={hasRail ? undefined : "true"}
38 |         className="min-h-screen bg-slate-50 dark:bg-background"
39 |       >
40 |         <GlobalHeader />
41 |         <div
42 |           className={cn(
43 |             "grid min-h-[calc(100vh-var(--app-header-h))] grid-cols-1 px-[var(--app-gutter-mobile)] pb-[calc(3
44 |               min-[640px]:grid-cols-[var(--app-sidebar-w)_minmax(0,1fr)] min-[640px]:gap-[var(--app-gutter)] mi
45 |               hasRail &&
46 |               "xl:grid-cols-[var(--app-sidebar-w)_minmax(0,1fr)_var(--app-rail-w)]"
47 |             )}
48 |         >
49 |           <Sidebar />
50 |           <div className="min-w-0 min-[640px]:col-start-2 min-[640px]:row-start-1">
51 |             <EmailVerificationBanner />
52 |             <main
53 |               id="main-content"
54 |               tabIndex=-1
55 |               className="min-h-[calc(100vh-var(--app-header-h))] py-5 min-[640px]:ps-[var(--app-gutter)] min-[
56 |             >
57 |               <div
58 |                 className={cn(
59 |                   "w-full min-w-0",
60 |                   hasRail && "max-w-[var(--app-content-max)] mx-auto"
61 |                 )}
62 |             >
63 |               {children}
64 |             </div>
65 |           </main>
66 |         </div>
67 |         {rail}
68 |       </div>
69 |       <GuidedTour />
70 |     </div>
71 |   </SidebarProvider>
72 | );
73 | };
74 |
75 | export default RoleAppShell;
76 |

```

FILE: src/components/shared/__tests__/StudentTeamSection.test.tsx

TYPE: .tsx | LINES: 1-102 | SHA256: 3b81501b95d433244e8b7345d53237c2ccc4d7bab24ebe62fadc6244ef289274

```
1 | import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
2 | import { render, act } from "@testing-library/react";
3 |
4 | // -----
5 | // Mock IntersectionObserver
6 | // -----
7 | let ioCallback: IntersectionObserverCallback | null = null;
8 | class MockIntersectionObserver {
9 |   constructor(cb: IntersectionObserverCallback) {
10 |     ioCallback = cb;
11 |   }
12 |   observe = vi.fn();
13 |   disconnect = vi.fn();
14 |   unobserve = vi.fn();
15 |   takeRecords = () => [];
16 | }
17 |
18 | const enterView = () => {
19 |   act(() => {
20 |     ioCallback?.(
21 |       [{ isIntersecting: true } as IntersectionObserverEntry],
22 |       {} as IntersectionObserver
23 |     );
24 |   });
25 | };
26 |
27 | // -----
28 | // Mocks: i18n, the team card, and the four team hooks
29 | // -----
30 | vi.mock("react-i18next", () => ({
31 |   useTranslation: () => ({ t: (key: string) => key }),
32 | }));
33 |
34 | vi.mock("@/components/shared/TeamDashboardCard", () => ({
35 |   default: () => <div data-testid="team-card" />,
36 | }));
37 |
38 | const useFirstEnrolledCourseId = vi.fn();
39 | const useMyTeamId = vi.fn();
40 | const useTeams = vi.fn();
41 | const useTeamGamification = vi.fn();
42 |
43 | vi.mock("@/hooks/useFirstEnrolledCourse", () => ({
44 |   useFirstEnrolledCourseId: (...args: unknown[]) =>
45 |     useFirstEnrolledCourseId(...args),
46 | }));
47 | vi.mock("@/hooks/useTeamLeaderboard", () => ({
48 |   useMyTeamId: (...args: unknown[]) => useMyTeamId(...args),
49 | }));
50 | vi.mock("@/hooks/useTeams", () => ({
51 |   useTeams: (...args: unknown[]) => useTeams(...args),
52 |   useTeamGamification: (...args: unknown[]) => useTeamGamification(...args),
53 | }));
54 |
55 | import StudentTeamSection from "../StudentTeamSection";
56 |
57 | describe("StudentTeamSection (viewport-gated)", () => {
58 |   beforeEach(() => {
59 |     ioCallback = null;
60 |     // The course hook respects `enabled` so the gating is observable end-to-end.
61 |     useFirstEnrolledCourseId
62 |       .mockReset()
63 |       .mockImplementation(
64 |         (_studentId: unknown, opts?: { enabled?: boolean }) => ({
65 |           data: opts?.enabled ? "course-1" : undefined,
66 |         })
67 |       );
68 |     useMyTeamId.mockReset().mockReturnValue({ data: "team-1" });
69 |     useTeams
70 |       .mockReset()
71 |       .mockReturnValue({ data: [{ id: "team-1", name: "Team One" }] });
72 |     useTeamGamification.mockReset().mockReturnValue({
73 |       data: { id: "team-1", team_id: "team-1", xp_total: 0, streak_count: 0 },
74 |     });
75 |     vi.stubGlobal("IntersectionObserver", MockIntersectionObserver);
76 |   });
77 |
78 |   afterEach(() => {
79 |     vi.unstubAllGlobals();
80 |   });
81 |
82 |   it("does not fire team queries or render the card until scrolled into view", () => {
83 |     const { queryByTestId } = render(<StudentTeamSection studentId="stu-1" />);
84 |
85 |     // Before intersection: course hook disabled, downstream ids undefined, no card.
86 |     expect(useFirstEnrolledCourseId).toHaveBeenLastCalledWith("stu-1", {
87 |       enabled: false,
88 |     });
89 |     expect(useMyTeamId).toHaveBeenLastCalledWith(undefined, undefined);
90 |     expect(queryByTestId("team-card")).toBeNull();
91 |
92 |     enterView();
93 |
94 |     // After intersection: hooks enabled with real ids, card renders.
95 |     expect(useFirstEnrolledCourseId).toHaveBeenLastCalledWith("stu-1", {
96 |       enabled: true,
97 |     });
98 |     expect(useMyTeamId).toHaveBeenLastCalledWith("stu-1", "course-1");
99 |     expect(queryByTestId("team-card")).not.toBeNull();
100 |   });
101 | });
```


FILE: src/components/shared/AccessibilitySettingsPanel.tsx

TYPE: .tsx | LINES: 1-116 | SHA256: 63c63a69cc4b377dfd0c9da6a865a229ac2025c6cdcac60d4244600da80decf0

```

1 | import { useTranslation } from "react-i18next";
2 | import { Card } from "@components/ui/card";
3 | import { Label } from "@components/ui/label";
4 | import { Switch } from "@components/ui/switch";
5 | import {
6 |   Select,
7 |   SelectContent,
8 |   SelectItem,
9 |   SelectTrigger,
10 |   SelectValue,
11 | } from "@components/ui/select";
12 | import {
13 |   useAccessibilityPreferences,
14 |   useUpdateAccessibilityPreferences,
15 | } from "@hooks/useAccessibilityPreferences";
16 | import type { AccessibilityPreferences } from "@lib/accessibilityPreferences";
17 |
18 | export const AccessibilitySettingsPanel = () => {
19 |   const { t } = useTranslation("common");
20 |   const { data: prefs } = useAccessibilityPreferences();
21 |   const updatePrefs = useUpdateAccessibilityPreferences();
22 |
23 |   if (!prefs) return null;
24 |
25 |   const update = (partial: Partial<AccessibilityPreferences>) => {
26 |     updatePrefs.mutate({ ...prefs, ...partial });
27 |   };
28 |
29 |   return (
30 |     <Card className="bg-white border-0 shadow-md rounded-xl p-6 space-y-6">
31 |       <h2 className="text-lg font-bold tracking-tight">
32 |         {t("accessibility.title", { defaultValue: "Accessibility Settings" })}
33 |       </h2>
34 |
35 |       <div className="space-y-4">
36 |         <div className="flex items-center justify-between">
37 |           <Label htmlFor="font-size">
38 |             {t("accessibility.fontSize", { defaultValue: "Font Size" })}
39 |           </Label>
40 |           <Select
41 |             value={prefs.font_size}
42 |             onValueChange={(v) =>
43 |               update({ font_size: v as AccessibilityPreferences["font_size"] })
44 |             }
45 |           >
46 |             <SelectTrigger id="font-size" className="w-32">
47 |               <SelectValue />
48 |             </SelectTrigger>
49 |             <SelectContent>
50 |               <SelectItem value="default">
51 |                 {t("accessibility.fontDefault", { defaultValue: "Default" })}
52 |               </SelectItem>
53 |               <SelectItem value="large">
54 |                 {t("accessibility.fontLarge", { defaultValue: "Large" })}
55 |               </SelectItem>
56 |               <SelectItem value="x-large">
57 |                 {t("accessibility.fontXLarge", { defaultValue: "Extra Large" })}
58 |               </SelectItem>
59 |             </SelectContent>
60 |           </Select>
61 |         </div>
62 |
63 |         <div className="flex items-center justify-between">
64 |           <Label htmlFor="high-contrast">
65 |             {t("accessibility.highContrast", { defaultValue: "High Contrast" })}
66 |           </Label>
67 |           <Switch
68 |             id="high-contrast"
69 |             checked={prefs.high_contrast}
70 |             onCheckedChange={(v) => update({ high_contrast: v })}
71 |           />
72 |         </div>
73 |
74 |         <div className="flex items-center justify-between">
75 |           <Label htmlFor="reduced-animations">
76 |             {t("accessibility.reducedAnimations", {
77 |               defaultValue: "Reduced Animations",
78 |             })}
79 |           </Label>
80 |           <Switch
81 |             id="reduced-animations"
82 |             checked={prefs.reduced_animations}
83 |             onCheckedChange={(v) => update({ reduced_animations: v })}
84 |           />
85 |         </div>
86 |
87 |         <div className="flex items-center justify-between">
88 |           <Label htmlFor="dyslexia-font">
89 |             {t("accessibility.dyslexiaFont", {
90 |               defaultValue: "Dyslexia-Friendly Font",
91 |             })}
92 |           </Label>
93 |           <Switch
94 |             id="dyslexia-font"
95 |             checked={prefs.dyslexia_font}
96 |             onCheckedChange={(v) => update({ dyslexia_font: v })}
97 |           />
98 |         </div>
99 |

```



```

100 |         <div className="flex items-center justify-between">
101 |             <Label htmlFor="simplified-view">
102 |                 {t("accessibility.simplifiedView", {
103 |                     defaultValue: "Simplified View",
104 |                 })}
105 |             </Label>
106 |             <Switch
107 |                 id="simplified-view"
108 |                 checked={prefs.simplified_view}
109 |                 onChange={(v) => update({ simplified_view: v })}
110 |             />
111 |         </div>
112 |     </div>
113 | </Card>
114 | );
115 | };
116 |

```

FILE: src/components/shared/ActiveBoostIndicator.tsx

TYPE: .tsx | LINES: 1-62 | SHA256: 59a6cacfcdd61d08ca0a31cb53a0b7281c5d1cbc3ef05ee127ec90fd5aade1b3

```

1 | // =====
2 | // ActiveBoostIndicator - Boost countdown timer with pulse animation
3 | // =====
4 |
5 | import { useState, useEffect } from "react";
6 | import { Zap } from "lucide-react";
7 | import { useAuth } from "@hooks/useAuth";
8 | import { useActiveBoosts } from "@hooks/useActiveBoosts";
9 | import { cn } from "@lib/utils";
10 |
11 | interface ActiveBoostIndicatorProps {
12 |   className?: string;
13 | }
14 |
15 | const ActiveBoostIndicator = ({ className }: ActiveBoostIndicatorProps) => {
16 |   const { user } = useAuth();
17 |   const { data: boosts } = useActiveBoosts(user?.id ?? "");
18 |   const activeBoost = boosts?.[0];
19 |   const [timeLeft, setTimeLeft] = useState(() => (activeBoost ? "" : ""));
20 |
21 |   useEffect(() => {
22 |     if (!activeBoost) return;
23 |
24 |     const updateTimer = () => {
25 |       const now = Date.now();
26 |       const expiresAt = new Date(activeBoost.expires_at).getTime();
27 |       const diff = expiresAt - now;
28 |
29 |       if (diff <= 0) {
30 |         setTimeLeft("Expired");
31 |         return;
32 |       }
33 |
34 |       const minutes = Math.floor(diff / 60_000);
35 |       const seconds = Math.floor((diff % 60_000) / 1000);
36 |       setTimeLeft(`${minutes}:${seconds.toString().padStart(2, "0")}`);
37 |     };
38 |
39 |     updateTimer();
40 |     const interval = setInterval(updateTimer, 1000);
41 |     return () => clearInterval(interval);
42 |   }, [activeBoost]);
43 |
44 |   if (!activeBoost || timeLeft === "Expired") return null;
45 |
46 |   return (
47 |     <div
48 |       className={cn(
49 |         "inline-flex items-center gap-1.5 rounded-full bg-purple-50 border border-purple-200 px-3 py-1.5 text-
50 |       className
51 |     )}
52 |     >
53 |       <Zap className="h-4 w-4 text-purple-500" />
54 |       <span>{activeBoost.multiplier}x XP</span>
55 |       <span className="text-purple-500">.</span>
56 |       <span className="tabular-nums">{timeLeft}</span>
57 |     </div>
58 |   );
59 | };
60 |
61 | export default ActiveBoostIndicator;
62 |

```

FILE: src/components/shared/AdaptiveXPDisplay.tsx

TYPE: .tsx | LINES: 1-66 | SHA256: cf5a139fa94af5933220c7ce5d19409a599045cd7e976a74229906a2ee930826

```

1 | // Task 138.1: Adaptive XP Display component
2 | // Shows current XP multiplier, level, level multiplier, and diminishing returns indicator
3 | // Requirements: 120.5, 122.3
4 |
5 | import { TrendingUp, AlertTriangle, Zap } from "lucide-react";
6 | import { Badge } from "@components/ui/badge";
7 | import {
8 |   useStudentXPMultiplier,
9 |   useDiminishingReturnsStatus,
10 | } from "@hooks/useAdaptiveXP";
11 |
12 | interface AdaptiveXPDisplayProps {
13 |   studentId: string;
14 |   actionType?: string;
15 | }
16 |
17 | const AdaptiveXPDisplay = ({

```

```

18 |   studentId,
19 |   actionType = "submission",
20 | }: AdaptiveXPDisplayProps) => {
21 |   const { data: multiplierData } = useStudentXPMultiplier(studentId);
22 |   const { data: diminishingData } = useDiminishingReturnsStatus(
23 |     studentId,
24 |     actionType
25 |   );
26 |
27 |   if (!multiplierData) return null;
28 |
29 |   return (
30 |     <div className="space-y-3">
31 |       { /* Level & Multiplier row */ }
32 |       <div className="flex items-center gap-2 flex-wrap">
33 |         <Badge
34 |           variant="outline"
35 |           className="text-xs bg-blue-50 text-blue-700 border-blue-200"
36 |         >
37 |           <Zap className="h-3 w-3 me-1" />
38 |           Level {multiplierData.level}
39 |         </Badge>
40 |         <Badge
41 |           variant="outline"
42 |           className="text-xs bg-amber-50 text-amber-700 border-amber-200"
43 |         >
44 |           <TrendingUp className="h-3 w-3 me-1" />
45 |           {multiplierData.multiplier}x XP Multiplier
46 |         </Badge>
47 |       </div>
48 |
49 |       { /* Diminishing Returns warning */ }
50 |       {diminishingData?.is_diminished && (
51 |         <div className="flex items-center gap-2 p-2 rounded-lg bg-amber-50 border border-amber-200">
52 |           <AlertTriangle className="h-4 w-4 text-amber-600 shrink-0" />
53 |           <p className="text-xs text-amber-700">
54 |             Diminishing Returns – next action earns{" "}
55 |             {diminishingData.next_multiplier}x XP. Try a different activity to
56 |             earn full XP.
57 |           </p>
58 |         </div>
59 |       )}
60 |     </div>
61 |   );
62 | };
63 |
64 | export default AdaptiveXPDisplay;
65 | export type { AdaptiveXPDisplayProps };
66 |

```

FILE: [src/components/shared/AdminDashboardNew.tsx](#)

TYPE: .tsx | LINES: 1-202 | SHA256: 62bbc232d16e7ace73ecf970765128a9167f563fb3d1d9687ce1544c481f4633

```

1 | // =====
2 | // AdminDashboardNew – redesigned admin dashboard (P2, spec task 2.5)
3 | // =====
4 | //
5 | // Institution-overview cockpit gated behind `newUiDashboards` (see the wrapper
6 | // in AdminDashboard.tsx). REUSES the existing `useAdminDashboardAggregate` (one
7 | // round-trip, RLS-scoped `get_admin_dashboard`, no new writes) and is composed
8 | // from the P0 primitives (WelcomeHero, KPICard, SectionHeader, SeverityIcon,
9 | // tactile Button, `.card-elevated`). i18n reuses the existing `admin` namespace
10 | // keys.
11 | //
12 | // This is an incremental, flag-off-by-default build presenting the core value
13 | // (hero, KPIs, users-by-role breakdown, manage-users CTA). The richer
14 | // below-the-fold widgets (recent activity, PLO heatmap, AI co-pilot metrics,
15 | // OBE management quick-access) remain on the legacy dashboard until this reaches
16 | // full parity (task 2.6). Flag-off keeps the current dashboard byte-identical.
17 | // =====
18 |
19 | import { useNavigate } from "react-router-dom";
20 | import { useTranslation } from "react-i18next";
21 | import {
22 |   ArrowRight,
23 |   BookOpen,
24 |   GraduationCap,
25 |   Users,
26 |   UserCheck,
27 | } from "lucide-react";
28 |
29 | import {
30 |   Badge,
31 |   Button,
32 |   Card,
33 |   KPICard,
34 |   SectionHeader,
35 |   SeverityIcon,
36 |   Shimmer,
37 |   WelcomeHero,
38 | } from "@design-system";
39 | import { useAuth } from "@hooks/useAuth";
40 | import { useAdminDashboardAggregate } from "@hooks/useAdminDashboardAggregate";
41 | import { formatNumber } from "@lib/formatNumber";
42 |
43 | // Role badge colors – mirrors the legacy AdminDashboard map (kept local: the
44 | // page file exports a single component, so the constant can't be shared from
45 | // there without tripping react-refresh/only-export-components).
46 | const roleBadgeStyles: Record<string, string> = {
47 |   admin: "bg-red-100 text-red-700 border-red-200",
48 |   coordinator: "bg-blue-100 text-blue-700 border-blue-200",
49 |   teacher: "bg-green-100 text-green-700 border-green-200",
50 |   student: "bg-amber-100 text-amber-700 border-amber-200",
51 |   parent: "bg-purple-100 text-purple-700 border-purple-200",

```

```

52 |   };
53 |
54 |   const AdminDashboardNew = () => {
55 |     const { t } = useTranslation("admin");
56 |     const navigate = useNavigate();
57 |     const { institutionId, profile } = useAuth();
58 |
59 |     // The aggregate returns the `AdminKPIData` shape directly.
60 |     const aggregate = useAdminDashboardAggregate(institutionId);
61 |     const kpis = aggregate.data;
62 |     const loading = aggregate.isPending;
63 |
64 |     const usersByRole = kpis?.usersByRole ?? {};
65 |     const roleEntries = Object.entries(usersByRole);
66 |
67 |     return (
68 |       <div className="space-y-6">
69 |         <WelcomeHero
70 |           name={profile?.full_name ?? "Admin"}
71 |           userRole="admin"
72 |           subtitle={t("dashboard.welcome.subtitle")}
73 |           stats={
74 |             loading ? null : (
75 |               <div className="flex items-center gap-6">
76 |                 <div className="text-right">
77 |                   <p className="text-xs font-semibold text-white/70">
78 |                     {t("dashboard.totalUsers")}
79 |                   </p>
80 |                   <p className="text-xl font-black text-white">
81 |                     {formatNumber(kpis?.totalUsers ?? 0)}
82 |                   </p>
83 |                 </div>
84 |                 <div className="h-8 w-px bg-white/20" />
85 |                 <div className="text-right">
86 |                   <p className="text-xs font-semibold text-white/70">
87 |                     {t("dashboard.courses")}
88 |                   </p>
89 |                   <p className="text-xl font-black text-white">
90 |                     {formatNumber(kpis?.totalCourses ?? 0)}
91 |                   </p>
92 |                 </div>
93 |               </div>
94 |             )
95 |           }
96 |         </div>
97 |
98 |         { /* KPI row */ }
99 |         { loading ? (
100 |           <div className="grid grid-cols-2 gap-4 md:grid-cols-4">
101 |             {Array.from({ length: 4 }).map((_, i) => (
102 |               <Shimmer key={i} className="h-24 rounded-xl" />
103 |             ))}
104 |           </div>
105 |         ) : (
106 |           <div className="grid grid-cols-2 gap-4 md:grid-cols-4">
107 |             <KPICard
108 |               icon={Users}
109 |               label={t("dashboard.totalUsers")}
110 |               value={formatNumber(kpis?.totalUsers ?? 0)}
111 |             />
112 |             <KPICard
113 |               icon={UserCheck}
114 |               label={t("dashboard.activeUsers")}
115 |               value={formatNumber(kpis?.activeUsers ?? 0)}
116 |               iconBgClass="bg-green-50"
117 |               iconColorClass="text-green-600"
118 |             />
119 |             <KPICard
120 |               icon={BookOpen}
121 |               label={t("dashboard.programs")}
122 |               value={formatNumber(kpis?.totalPrograms ?? 0)}
123 |             />
124 |             <KPICard
125 |               icon={GraduationCap}
126 |               label={t("dashboard.courses")}
127 |               value={formatNumber(kpis?.totalCourses ?? 0)}
128 |             />
129 |           </div>
130 |         )}
131 |
132 |         { /* Users by role */ }
133 |         <Card className="card-elevated overflow-hidden border-0 bg-white">
134 |           <div className="p-6">
135 |             <SectionHeader icon={Users} title={t("dashboard.usersByRole")} />
136 |             <div className="mt-4">
137 |               {loading ? (
138 |                 <div className="space-y-3">
139 |                   {Array.from({ length: 5 }).map((_, i) => (
140 |                     <Shimmer key={i} className="h-8 rounded-lg" />
141 |                   ))}
142 |                 </div>
143 |               ) : roleEntries.length > 0 ? (
144 |                 <div className="space-y-3">
145 |                   {roleEntries.map(([role, count]) => (
146 |                     <div key={role} className="flex items-center justify-between">
147 |                       <Badge
148 |                         variant="outline"
149 |                         className={roleBadgeStyles[role] ?? ""}
150 |                       >
151 |                         {role.charAt(0).toUpperCase() + role.slice(1)}
152 |                       </Badge>
153 |                       <span className="text-sm font-semibold text-gray-900">
154 |                         {formatNumber(count)}
155 |                       </span>

```

```

156         </div>
157       )}}
158     </div>
159   ) : (
160     <p className="py-6 text-center text-sm text-gray-500">
161       {t("dashboard.noActiveUsers")}
162     </p>
163   )}
164 </div>
165 </div>
166 </Card>
167
168 {/* Primary CTA – manage users */}
169 <Card className="card-elevated overflow-hidden border-0 bg-white">
170   <div className="flex flex-col gap-3 p-6 sm:flex-row sm:items-center sm:justify-between">
171     <div className="flex items-center gap-3">
172       <SeverityIcon
173         icon={Users}
174         severity="brand"
175         label={t("dashboard.usersByRole")}
176       />
177       <div>
178         <p className="text-sm font-semibold text-gray-900">
179           {t("dashboard.usersByRole")}
180         </p>
181         <p className="text-xs text-gray-500">
182           {t("dashboard.totalUsers")}: {t(" ")}
183           {formatNumber(kpis?.totalUsers ?? 0)}
184         </p>
185       </div>
186     </div>
187     <Button
188       variant="tactile"
189       className="shrink-0"
190       onClick={() => navigate("/admin/users")}
191     >
192       {t("dashboard.manage")}
193       <ArrowRight className="h-4 w-4 aria-hidden=true" />
194     </Button>
195   </div>
196 </Card>
197 </div>
198 );
199 };
200
201 export default AdminDashboardNew;
202

```

FILE: src/components/shared/AdminOnboardingWizard.tsx

TYPE: .tsx | LINES: 1-195 | SHA256: 4655c14f125e18a165f36504f28a210586c3e4c62fb6ead4b63d5713828136f9

```

1 // =====
2 // AdminOnboardingWizard – Guided setup wizard for first-time admins
3 // Steps: Create IL0s !' Create Programs !' Invite Coordinators !' Invite Teachers
4 // =====
5
6 import { useState, useMemo } from "react";
7 import { useNavigate } from "react-router-dom";
8 import { motion, AnimatePresence } from "framer-motion";
9 import {
10   Target,
11   BookOpen,
12   UserPlus,
13   Users,
14   CheckCircle2,
15   ChevronRight,
16   Loader2,
17   type LucideIcon,
18 } from "lucide-react";
19 import { Card } from "@/components/ui/card";
20 import { Button } from "@/components/ui/button";
21 import { useCompleteOnboarding } from "@/hooks/useOnboarding";
22 import { toast } from "sonner";

```

```

24 // %% Types %%%%%%%%%%%%%%%
25
26 interface WizardStep {
27   id: string;
28   title: string;
29   description: string;
30   icon: LucideIcon;
31   route: string;
32   isCompleted: boolean;
33 }
34
35 interface AdminOnboardingWizardProps {
36   hasILOs: boolean;
37   hasPrograms: boolean;
38   hasCoordinators: boolean;
39   hasTeachers: boolean;
40   onDismiss?: () => void;
41 }
42
43 // %% Component %%%%%%%%%%%%%%%
44
45 const AdminOnboardingWizard = ({
46   hasILOs,
47   hasPrograms,
48   hasCoordinators,
49   hasTeachers,
50   onDismiss,
51 }: AdminOnboardingWizardProps) => {
52   const navigate = useNavigate();
53   const completeOnboarding = useCompleteOnboarding();

```

```

54 | const [dismissed, setDismissed] = useState(false);
55 |
56 | const steps: WizardStep[] = useMemo(
57 |   () => [
58 |     {
59 |       id: "ilos",
60 |       title: "Create ILOs",
61 |       description: "Define your institution's learning outcomes",
62 |       icon: Target,
63 |       route: "/admin/outcomes",
64 |       isCompleted: hasILOs,
65 |     },
66 |     {
67 |       id: "programs",
68 |       title: "Create Programs",
69 |       description: "Set up academic programs for your institution",
70 |       icon: BookOpen,
71 |       route: "/admin/programs",
72 |       isCompleted: hasPrograms,
73 |     },
74 |     {
75 |       id: "coordinators",
76 |       title: "Invite Coordinators",
77 |       description: "Add coordinators to manage programs",
78 |       icon: UserPlus,
79 |       route: "/admin/users",
80 |       isCompleted: hasCoordinators,
81 |     },
82 |     {
83 |       id: "teachers",
84 |       title: "Invite Teachers",
85 |       description: "Add teachers to manage courses",
86 |       icon: Users,
87 |       route: "/admin/users",
88 |       isCompleted: hasTeachers,
89 |     },
90 |   ],
91 |   [hasILOs, hasPrograms, hasCoordinators, hasTeachers]
92 | );
93 |
94 | const completedCount = steps.filter((s) => s.isCompleted).length;
95 | const allCompleted = completedCount === steps.length;
96 | const progress = (completedCount / steps.length) * 100;
97 |
98 | const handleStepClick = (route: string) => {
99 |   navigate(route);
100 | };
101 |
102 | const handleComplete = () => {
103 |   completeOnboarding.mutate(undefined, {
104 |     onSuccess: () => {
105 |       toast.success("Onboarding complete! Welcome to Edeviser.");
106 |       setDismissed(true);
107 |       onDismiss?();
108 |     },
109 |     onError: (err) => toast.error(err?.message ?? "An error occurred"),
110 |   });
111 | };
112 |
113 | if (dismissed) return null;
114 |
115 | return (
116 |   <Card className="bg-white border-0 shadow-md rounded-xl overflow-hidden gap-0 py-0">
117 |     <div
118 |       className="px-6 py-4 flex items-center gap-2"
119 |       style={{
120 |         background: "var(--brand-gradient)",
121 |       }}
122 |     >
123 |       <CheckCircle2 className="h-5 w-5 text-white" />
124 |       <h2 className="text-lg font-bold tracking-tight text-white">
125 |         Setup Wizard
126 |       </h2>
127 |       <span className="ml-auto text-sm font-medium text-white/80">
128 |         {completedCount}/{steps.length}
129 |       </span>
130 |     </div>
131 |     <div className="p-6 space-y-4">
132 |       <div /* Progress bar */
133 |         className="h-2 w-full rounded-full bg-slate-100 overflow-hidden">
134 |         <motion.div
135 |           className="h-full rounded-full bg-[linear-gradient(93.65deg,#14b8a6_5.37%,#0382bd_78.89%)]"
136 |           initial={{ width: 0 }}
137 |           animate={{ width: `${progress}%` }}
138 |           transition={{ duration: 0.4, ease: "easeOut" }}
139 |         />
140 |       </div>
141 |
142 |       <div /* Steps */
143 |         <AnimatePresence mode="sync">
144 |           <div className="space-y-2">
145 |             {steps.map((step, index) => (
146 |               <motion.button
147 |                 key={step.id}
148 |                 initial={{ opacity: 0, y: 8 }}
149 |                 animate={{ opacity: 1, y: 0 }}
150 |                 transition={{ delay: index * 0.05 }}
151 |                 onClick={() => handleStepClick(step.route)}
152 |                 className="flex items-center gap-3 w-full text-start rounded-lg px-3 py-3 hover:bg-slate-50"
153 |               >
154 |                 <div className="flex items-center justify-center h-8 w-8 rounded-full shrink-0">
155 |                   {step.isCompleted ? (
156 |                     <CheckCircle2 className="h-5 w-5 text-green-500" />
157 |                   ) : (

```

```

158 |         <div className="h-5 w-5 rounded-full border-2 border-slate-300 flex items-center justify-c
159 |             <span className="text-xs font-bold text-slate-400">
160 |                 {index + 1}
161 |             </span>
162 |         </div>
163 |     )}
164 | </div>
165 |     <div className="flex-1 min-w-0">
166 |         <p className="text-sm font-semibold">{step.title}</p>
167 |         <p className="text-xs text-gray-500">{step.description}</p>
168 |     </div>
169 |     <ChevronRight className="h-4 w-4 text-gray-400 shrink-0" />
170 | </motion.button>
171 | )}
172 | </div>
173 | </AnimatePresence>
174 |
175 | /* Complete button */
176 | {allCompleted && (
177 |   <Button
178 |     onClick={handleComplete}
179 |     disabled={completeOnboarding.isPending}
180 |     variant="tactile"
181 |     className="w-full"
182 |   >
183 |     {completeOnboarding.isPending && (
184 |       <Loader2 className="h-4 w-4 animate-spin me-2" />
185 |     )}
186 |     Complete Setup
187 |   </Button>
188 | )}
189 | </div>
190 | </Card>
191 | );
192 | };
193 |
194 | export default AdminOnboardingWizard;
195 |

```

FILE: src/components/shared/AIAtRiskWidget.tsx

```
TYPE: .tsx | LINES: 1-165 | SHA256: 7e4ee32492af29fc990744ea85b60cb264900c40b72e42acdacc1dd830cf2d80
```

[illegible]

```

63 |     },
64 |   });
65 | };
66 |
67 | return (
68 |   <>
69 |     <PCard className="overflow-hidden">
70 |       <div className="p-5 pb-4">
71 |         <SectionHeader icon={Sparkles} title="AI At-Risk Students" />
72 |       </div>
73 |
74 |       <div className="space-y-4 px-5 pb-5">
75 |         {isLoading ? (
76 |           <div className="space-y-3">
77 |             {Array.from({ length: 3 }).map((_, i) => (
78 |               <Shimmer key={i} className="h-20 rounded-lg" />
79 |             ))}
80 |           </div>
81 |         ) : isError ? (
82 |           <ErrorState
83 |             message="We couldn't load AI at-risk predictions."
84 |             onRetry={() => refetch()}
85 |             className="py-8"
86 |           />
87 |         ) : !predictions || predictions.length === 0 ? (
88 |           <div className="flex flex-col items-center justify-center py-8 text-center">
89 |             <div className="p-3 rounded-full bg-green-50 mb-3">
90 |               <CheckSquare className="h-8 w-8 text-green-500" />
91 |             </div>
92 |             <p className="text-sm text-gray-500">
93 |               No AI at-risk predictions. All students are on track!
94 |             </p>
95 |           </div>
96 |         ) : (
97 |           <div>
98 |             {predictions.map((prediction) => (
99 |               <AtRiskStudentRow
100 |                 key={prediction.id}
101 |                 studentName={prediction.student_name}
102 |                 cloTitle={prediction.suggestion_data.at_risk_clo_title}
103 |                 probabilityScore={
104 |                   prediction.suggestion_data.probability_score
105 |                 }
106 |                 contributingSignals={
107 |                   prediction.suggestion_data.contributing_signals
108 |                 }
109 |                 onSendNudge={() => openNudgeDialog(prediction)}
110 |                 isNudging={
111 |                   nudgeMutation.isPending &&
112 |                   nudgeTarget?.student_id === prediction.student_id
113 |                 }
114 |               </div>
115 |             ))}
116 |           </div>
117 |         )}
118 |       </div>
119 |     </PCard>
120 |
121 |     {/* Nudge Dialog */}
122 |     <Dialog
123 |       open={!!nudgeTarget}
124 |       onOpenChange={(open) => {
125 |         if (!open) setNudgeTarget(null);
126 |       }}
127 |     >
128 |       <DialogContent>
129 |         <DialogHeader>
130 |           <DialogTitle>Send Nudge to {nudgeTarget?.student_name}</DialogTitle>
131 |         </DialogHeader>
132 |         <Textarea
133 |           value={nudgeMessage}
134 |           onChange={(e) => setNudgeMessage(e.target.value)}
135 |           rows={4}
136 |           placeholder="Write a personalized message..."
137 |         />
138 |         <DialogFooter>
139 |           <Button
140 |             variant="outline"
141 |             onClick={() => setNudgeTarget(null)}
142 |             disabled={nudgeMutation.isPending}
143 |           >
144 |             Cancel
145 |           </Button>
146 |           <Button
147 |             onClick={handleSendNudge}
148 |             disabled={nudgeMutation.isPending || !nudgeMessage.trim()}
149 |             variant="tactile"
150 |           >
151 |             {nudgeMutation.isPending && (
152 |               <Loader2 className="h-4 w-4 animate-spin" />
153 |             )}
154 |             Send Nudge
155 |           </Button>
156 |         </DialogFooter>
157 |       </DialogContent>
158 |     </Dialog>
159 |   </>
160 | );
161 | };
162 |
163 | export default AIAtriskWidget;
164 |
165 |

```

FILE: src/components/shared/AIFeedbackThumbs.tsx

```
TYPE: .tsx | LINES: 1-95 | SHA256: 1ff28c05a96fc617d95c6c18c91b4063c24162814374e6a17286c41482f09892
```

[illegible]

FILE: src/components/shared/AISuggestionCard.tsx

```
TYPE: .tsx | LINES: 1-112 | SHA256: af3e9fd0145d57f40a61c34d32892e9e9e1f69cfb1a725fcbd6e33b7791cbeed
```

[illegible]

```

102 |         onFeedback={handleFeedback}
103 |       />
104 |     </div>
105 |   </div>
106 | </PCard>
107 | );
108 | };
109 |
110 | export default AISuggestionCard;
111 | export type { AISuggestionCardProps };
112 |

```

FILE: src/components/shared/AnnouncementAttachmentList.tsx

TYPE: .tsx | LINES: 1-90 | SHA256: 0f759c1749418c6807ab0d7727aac820e648db4cd622e17517778a0b4ab2468e

```

1 | // Feature: qa-partner-review-remediation - Req 15.3 (announcement attachments)
2 | // Read-only attachment list with signed-URL download. Used on the student
3 | // announcement detail view; the private bucket is accessed via signed URLs.
4 | import { useState } from "react";
5 | import {
6 |   useAnnouncementAttachments,
7 |   getAnnouncementAttachmentUrl,
8 |   type AnnouncementAttachment,
9 | } from "@hooks/useAnnouncements";
10 | import { Button } from "@components/ui/button";
11 | import { Loader2, Paperclip, Download } from "lucide-react";
12 | import { toast } from "sonner";
13 |
14 | interface AnnouncementAttachmentListProps {
15 |   announcementId: string;
16 | }
17 |
18 | const formatSize = (bytes: number | null): string => {
19 |   if (!bytes) return "";
20 |   const mb = bytes / (1024 * 1024);
21 |   if (mb >= 1) return `${mb.toFixed(1)} MB`;
22 |   return `${Math.max(1, Math.round(bytes / 1024))} KB`;
23 | };
24 |
25 | export const AnnouncementAttachmentList = ({
26 |   announcementId,
27 | }: AnnouncementAttachmentListProps) => {
28 |   const [downloadingId, setDownloadingId] = useState<string | null>(null);
29 |   const { data: attachments, isLoading } =
30 |     useAnnouncementAttachments(announcementId);
31 |
32 |   const handleDownload = async (attachment: AnnouncementAttachment) => {
33 |     setDownloadingId(attachment.id);
34 |     try {
35 |       const url = await getAnnouncementAttachmentUrl(attachment.storage_path);
36 |       if (url) window.open(url, "_blank", "noopener,noreferrer");
37 |       else toast.error("Could not generate a download link. Try again.");
38 |     } finally {
39 |       setDownloadingId(null);
40 |     }
41 |   };
42 |
43 |   // Nothing to show while loading or when there are no attachments.
44 |   if (isLoading || (attachments ?? []).length === 0) return null;
45 |
46 |   return (
47 |     <div className="mt-6 pt-4 border-t border-slate-100">
48 |       <p className="text-xs font-semibold text-gray-600 flex items-center gap-1.5 mb-2">
49 |         <Paperclip className="h-3.5 w-3.5" /> Attachments
50 |       </p>
51 |       <ul className="space-y-1.5">
52 |         {(attachments ?? []).map((a) => (
53 |           <li
54 |             key={a.id}
55 |             className="flex items-center gap-2 rounded-lg bg-slate-50 px-3 py-2"
56 |           >
57 |             <Paperclip className="h-3.5 w-3.5 text-gray-400 shrink-0" />
58 |             <span className="text-xs font-medium text-gray-700 truncate flex-1">
59 |               {a.file_name}
60 |             </span>
61 |             {a.size_bytes ? (
62 |               <span className="text-[11px] text-gray-400 shrink-0">
63 |                 {formatSize(a.size_bytes)}
64 |               </span>
65 |             ) : null}
66 |             <Button
67 |               type="button"
68 |               variant="ghost"
69 |               size="sm"
70 |               className="h-7 gap-1.5 shrink-0"
71 |               disabled={downloadingId === a.id}
72 |               onClick={() => handleDownload(a)}
73 |               aria-label={`Download ${a.file_name}`}
74 |             >
75 |               {downloadingId === a.id ? (
76 |                 <Loader2 className="h-3.5 w-3.5 animate-spin" />
77 |               ) : (
78 |                 <Download className="h-3.5 w-3.5 text-blue-600" />
79 |               )}
80 |             Download
81 |           </Button>
82 |         </li>
83 |       )]}
84 |     </ul>
85 |   </div>
86 | );
87 | };
88 |
89 | export default AnnouncementAttachmentList;

```

FILE: src/components/shared/AnnouncementAttachmentManager.tsx

TYPE: .tsx | LINES: 1-163 | SHA256: 4a9f256e060681a36578103aaa42d560d4aa795c1163d4667a397026f24fc815

```

1 | // Feature: qa-partner-review-remediation - Req 15.3 (announcement attachments)
2 | // Teacher-facing attachment manager: client-validated upload + signed-URL list
3 | // for a single announcement. Uses Shadcn components and Sonner toasts.
4 | import { useRef, useState } from "react";
5 | import {
6 |   useAnnouncementAttachments,
7 |   useUploadAnnouncementAttachment,
8 |   getAnnouncementAttachmentUrl,
9 |   type AnnouncementAttachment,
10 | } from "@hooks/useAnnouncements";
11 | import {
12 |   validateAnnouncementAttachmentFile,
13 |   FileValidationError,
14 | } from "@lib/fileUpload";
15 | import { ANNOUNCEMENT_ATTACHMENT_ACCEPTED_LABEL } from "@lib/schemas/announcementAttachment";
16 | import { Button } from "@components/ui/button";
17 | import { Label } from "@components/ui/label";
18 | import { Loader2, Paperclip, Upload, Download } from "lucide-react";
19 | import { toast } from "sonner";
20 |
21 | interface AnnouncementAttachmentManagerProps {
22 |   announcementId: string;
23 | }
24 |
25 | const formatSize = (bytes: number | null): string => {
26 |   if (!bytes) return "";
27 |   const mb = bytes / (1024 * 1024);
28 |   if (mb >= 1) return `${mb.toFixed(1)} MB`;
29 |   return `${Math.max(1, Math.round(bytes / 1024))} KB`;
30 | };
31 |
32 | export const AnnouncementAttachmentManager = ({
33 |   announcementId,
34 | }: AnnouncementAttachmentManagerProps) => {
35 |   const inputRef = useRef<HTMLInputElement>(null);
36 |   const [downloadingId, setDownloadingId] = useState<string | null>(null);
37 |
38 |   const { data: attachments, isLoading } =
39 |     useAnnouncementAttachments(announcementId);
40 |   const uploadMutation = useUploadAnnouncementAttachment();
41 |
42 |   const handleFileSelected = (event: React.ChangeEvent<HTMLInputElement>) => {
43 |     const file = event.target.files?.[0];
44 |     // Reset the input so selecting the same file again re-triggers onChange.
45 |     event.target.value = "";
46 |     if (!file) return;
47 |
48 |     // CLIENT-SIDE validation before any upload (engineering-guardrails.md).
49 |     try {
50 |       validateAnnouncementAttachmentFile(file);
51 |     } catch (err) {
52 |       const message =
53 |         err instanceof FileValidationError
54 |           ? err.message
55 |           : "That file could not be attached.";
56 |       toast.error(message);
57 |       return;
58 |     }
59 |
60 |     uploadMutation.mutate(
61 |       { announcementId, file },
62 |       {
63 |         onSuccess: () => toast.success(`Attached "${file.name}"`),
64 |         onError: (err) =>
65 |           toast.error(
66 |             err instanceof Error ? err.message : "Attachment upload failed"
67 |           ),
68 |       },
69 |     );
70 |   };
71 |
72 |   const handleDownload = async (attachment: AnnouncementAttachment) => {
73 |     setDownloadingId(attachment.id);
74 |     try {
75 |       const url = await getAnnouncementAttachmentUrl(attachment.storage_path);
76 |       if (url) window.open(url, "_blank", "noopener,noreferrer");
77 |       else toast.error("Could not generate a download link. Try again.");
78 |     } finally {
79 |       setDownloadingId(null);
80 |     }
81 |   };
82 |
83 |   return (
84 |     <div className="mt-3 border-t border-slate-100 pt-3">
85 |       <div className="flex items-center justify-between gap-2">
86 |         <Label className="text-xs font-semibold text-gray-600 flex items-center gap-1.5">
87 |           <Paperclip className="h-3.5 w-3.5" /> Attachments
88 |         </Label>
89 |         <input
90 |           ref={inputRef}
91 |           type="file"
92 |           accept=".pdf,.doc,.docx,.ppt,.pptx,.xls,.xlsx,.txt,.png,.jpg,.jpeg"
93 |           className="hidden"
94 |           onChange={handleFileSelected}
95 |           aria-hidden="true"
96 |           tabIndex=-1
97 |         />
98 |         <Button
99 |           type="button"

```

```

100 |         variant="outline"
101 |         size="sm"
102 |         className="h-7"
103 |         disabled={uploadMutation.isPending}
104 |         onClick={() => inputRef.current?.click()}
105 |     >
106 |         {uploadMutation.isPending ? (
107 |             <Loader2 className="h-3.5 w-3.5 animate-spin" />
108 |         ) : (
109 |             <Upload className="h-3.5 w-3.5" />
110 |         )}
111 |         Attach file
112 |     </Button>
113 | </div>
114 |
115 | <p className="text-[11px] text-gray-400 mt-1">
116 |     Up to 10MB · {ANNOUNCEMENT_ATTACHMENT_ACCEPTED_LABEL}
117 | </p>
118 |
119 | {isLoading ? (
120 |     <p className="text-xs text-gray-400 mt-2">Loading attachments...</p>
121 | ) : (attachments ?? []).length === 0 ? (
122 |     <p className="text-xs text-gray-400 mt-2">No attachments yet.</p>
123 | ) : (
124 |     <ul className="mt-2 space-y-1.5">
125 |         {(attachments ?? []).map((a) => (
126 |             <li
127 |                 key={a.id}
128 |                 className="flex items-center gap-2 rounded-lg bg-slate-50 px-3 py-2"
129 |             >
130 |                 <Paperclip className="h-3.5 w-3.5 text-gray-400 shrink-0" />
131 |                 <span className="text-xs font-medium text-gray-700 truncate flex-1">
132 |                     {a.file_name}
133 |                 </span>
134 |                 {a.size_bytes ? (
135 |                     <span className="text-[11px] text-gray-400 shrink-0">
136 |                         {formatSize(a.size_bytes)}
137 |                     </span>
138 |                 ) : null}
139 |                 <Button
140 |                     type="button"
141 |                     variant="ghost"
142 |                     size="sm"
143 |                     className="h-7 w-7 p-0 shrink-0"
144 |                     disabled={downloadingId === a.id}
145 |                     onClick={() => handleDownload(a)}
146 |                     aria-label={`Download ${a.file_name}`}
147 |                 >
148 |                     {downloadingId === a.id ? (
149 |                         <Loader2 className="h-3.5 w-3.5 animate-spin" />
150 |                     ) : (
151 |                         <Download className="h-3.5 w-3.5 text-blue-600" />
152 |                     )}
153 |                 </Button>
154 |             </li>
155 |         ))}
156 |     </ul>
157 | )}
158 | </div>
159 | );
160 | };
161 |
162 | export default AnnouncementAttachmentManager;
163 |

```

FILE: src/components/shared/AnnouncementCard.tsx

TYPE: .tsx | LINES: 1-56 | SHA256: 45f99a2944014bd92fa2236222b871367bee36e7888a2c0f19f328e739e55c91

```

1 | // =====
2 | // AnnouncementCard - Announcement display card
3 | // =====
4 |
5 | import { Megaphone } from "lucide-react";
6 | import { cn } from "@lib/utils";
7 |
8 | interface AnnouncementCardProps {
9 |   title: string;
10 |   content: string;
11 |   authorName: string;
12 |   createdAt: string;
13 |   priority?: "normal" | "high";
14 |   className?: string;
15 | }
16 |
17 | const AnnouncementCard = ({
18 |   title,
19 |   content,
20 |   authorName,
21 |   createdAt,
22 |   priority = "normal",
23 |   className,
24 | }: AnnouncementCardProps) => (
25 |   <div
26 |     className={cn(
27 |       "rounded-xl border bg-white p-4 space-y-2",
28 |       priority === "high"
29 |         ? "border-amber-200 bg-amber-50/30"
30 |         : "border-slate-200",
31 |       className
32 |     )}
33 |   >
34 |     <div className="flex items-start gap-2">
35 |       <Megaphone
36 |         className={cn(

```

```

37 |         "h-4 w-4 mt-0.5 shrink-0",
38 |         priority === "high" ? "text-amber-500" : "text-gray-400"
39 |     ))
40 | }
41 | </div>
42 | <div className="flex-1 min-w-0">
43 |   <h4 className="text-sm font-bold">{title}</h4>
44 |   <p className="text-sm text-gray-600 mt-1 line-clamp-3">{content}</p>
45 | </div>
46 | </div>
47 | <div className="flex items-center gap-2 text-xs text-gray-400">
48 |   <span>{authorName}</span>
49 |   <span>.</span>
50 |   <span>{createdAt}</span>
51 | </div>
52 | </div>
53 | );
54 | export default AnnouncementCard;
55 | export type { AnnouncementCardProps };
56 |

```

FILE: src/components/shared/AnonymousToggle.tsx

TYPE: .tsx | LINES: 1-76 | SHA256: c0b0d63990b002c0e1c9e9b52736b459b5ce62008c87b05006cfbef15a22365c

```

1 | // =====
2 | // AnonymousToggle - Toggle leaderboard visibility with status indicator
3 | // =====
4 |
5 | import { useCallback } from "react";
6 | import { Eye, EyeOff, Loader2 } from "lucide-react";
7 | import { toast } from "sonner";
8 | import { useAnonymousStatus, useToggleAnonymous } from "@hooks/useLeaderboard";
9 | import { Button } from "@components/ui/button";
10 | import { cn } from "@lib/utls";
11 |
12 | const AnonymousToggle = () => {
13 |   const { data: status, isLoading: isLoadingStatus } = useAnonymousStatus();
14 |   const toggleMutation = useToggleAnonymous();
15 |
16 |   const isAnonymous = status?.isAnonymous ?? false;
17 |
18 |   const handleToggle = useCallback(() => {
19 |     toggleMutation.mutate(undefined, {
20 |       onSuccess: (isNowAnonymous) => {
21 |         toast.success(
22 |           isNowAnonymous
23 |             ? "You are now anonymous on the leaderboard"
24 |             : "You are now visible on the leaderboard"
25 |         );
26 |       },
27 |       onError: (err: unknown) => {
28 |         const message = err instanceof Error ? err.message : String(err);
29 |         toast.error(message);
30 |       },
31 |     });
32 |   }, [toggleMutation]);
33 |
34 |   const isPending = toggleMutation.isPending || isLoadingStatus;
35 |
36 |   return (
37 |     <div className="flex items-center gap-3">
38 |       <span
39 |         className={cn(
40 |           "text-sm font-medium",
41 |           isAnonymous ? "text-gray-500" : "text-green-600"
42 |         )}
43 |       >
44 |         {isLoadingStatus
45 |           ? "... "
46 |           : isAnonymous
47 |             ? "You are anonymous"
48 |             : "You are visible"}
49 |       </span>
50 |       <Button
51 |         variant="outline"
52 |         size="sm"
53 |         onClick={handleToggle}
54 |         disabled={isPending}
55 |         className={cn(
56 |           "gap-2",
57 |           isAnonymous
58 |             ? "border-gray-300 text-gray-600"
59 |             : "border-blue-300 text-blue-600"
60 |         )}
61 |       >
62 |         {isPending ? (
63 |           <Loader2 className="h-4 w-4 animate-spin" />
64 |         ) : isAnonymous ? (
65 |           <EyeOff className="h-4 w-4" />
66 |         ) : (
67 |           <Eye className="h-4 w-4" />
68 |         )}
69 |         {isAnonymous ? "Go Visible" : "Go Anonymous"}
70 |       </Button>
71 |     </div>
72 |   );
73 | };
74 |
75 | export default AnonymousToggle;
76 |

```

FILE: src/components/shared/AnswerDistributionChart.tsx

TYPE: .tsx | LINES: 1-58 | SHA256: 9fb8d2bb4bbbffa467997b7f0c67150e22321439d25de3c997f0791324dcd923

```
1 | import {
2 |   BarChart,
3 |   Bar,
4 |   XAxis,
5 |   YAxis,
6 |   Tooltip,
7 |   ResponsiveContainer,
8 |   Cell,
9 | } from "recharts";
10 | import { cn } from "@lib/utls";
11 |
12 | export interface AnswerDistributionChartProps {
13 |   data: Array<{
14 |     option: string;
15 |     count: number;
16 |     isCorrect: boolean;
17 |   }>;
18 |   className?: string;
19 | }
20 |
21 | const AnswerDistributionChart = ({
22 |   data,
23 |   className,
24 | }: AnswerDistributionChartProps) => {
25 |   return (
26 |     <div className={cn("w-full", className)}>
27 |       <ResponsiveContainer width="100%" height={200}>
28 |         <BarChart
29 |           data={data}
30 |           margin={{ top: 5, right: 10, left: 0, bottom: 5 }}
31 |         >
32 |           <XAxis dataKey="option" tick={{ fontSize: 12, fill: "#64748b" }} />
33 |           <YAxis
34 |             allowDecimals={false}
35 |             tick={{ fontSize: 11, fill: "#64748b" }}
36 |           />
37 |           <Tooltip
38 |             formatter={(value: number) => [`${value} responses`] as never}
39 |             contentStyle={{
40 |               borderRadius: 8,
41 |               border: "none",
42 |               boxShadow: "0 4px 12px rgba(0,0,0,0.1)",
43 |             }}
44 |           />
45 |           <Bar dataKey="count" radius={[4, 4, 0, 0]}>
46 |             {data.map((entry, idx) => (
47 |               <Cell key={idx} fill={entry.isCorrect ? "#22c55e" : "#94a3b8"} />
48 |             ))}
49 |           </Bar>
50 |         </BarChart>
51 |       </ResponsiveContainer>
52 |     </div>
53 |   );
54 | };
55 |
56 | export { AnswerDistributionChart };
57 | export default AnswerDistributionChart;
58 |
```

FILE: src/components/shared/AssessmentIntro.tsx

TYPE: .tsx | LINES: 1-96 | SHA256: fd7ccf969f0ab130cad203f8b1680c493941fd6b8c6df98feb31db8792625374

```
1 | // =====
2 | // AssessmentIntro – Benefit-oriented framing panel shown before an assessment
3 | // body (R17). Communicates the assessment's benefits and estimated time before
4 | // any responses are requested, and exposes an explicit "begin" action. The
5 | // consuming step keeps its assessment body gated until the student begins, so
6 | // the benefit + estimated time are always displayed first (R17.2a).
7 | // =====
8 |
9 | import type { LucideIcon } from "lucide-react";
10 | import { motion } from "framer-motion";
11 | import { Clock, Check, ArrowRight } from "lucide-react";
12 | import { Card } from "@components/ui/card";
13 | import { Button } from "@components/ui/button";
14 | import Mascot from "@components/shared/Mascot";
15 | import { useGatedMotion } from "@lib/motionGate";
16 | import type { AssessmentIntroContent } from "@lib/assessmentIntro";
17 |
18 | export interface AssessmentIntroProps {
19 |   /** Lead icon for the assessment (matches the step's body header icon). */
20 |   icon: LucideIcon;
21 |   /** Resolved, localized framing content (title, description, time, benefits). */
22 |   content: AssessmentIntroContent;
23 |   /** Label for the begin/start button. */
24 |   beginLabel: string;
25 |   /** Invoked when the student chooses to begin the assessment. */
26 |   onBegin: () => void;
27 | }
28 |
29 | const AssessmentIntro = ({
30 |   icon: Icon,
31 |   content,
32 |   beginLabel,
33 |   onBegin,
34 | }: AssessmentIntroProps) => {
35 |   // R31.2/R31.2a – suppress the icon's entrance animation when the user prefers
36 |   // reduced motion (render it already settled) while leaving any in-flight
37 |   // animation to complete naturally; gating lives in JS, not only CSS (R31.3).
38 |   const motionGate = useGatedMotion();
```

```

39 |
40 | return (
41 |   <div className="flex flex-col items-center text-center">
42 |     <motion.div
43 |       initial={motionGate.enter(
44 |         { scale: 0.8, opacity: 0 },
45 |         { scale: 1, opacity: 1 }
46 |       )}
47 |       animate={{ scale: 1, opacity: 1 }}
48 |       transition={motionGate.transition({ duration: 0.3, ease: "easeOut" })}
49 |       className="mb-6 flex h-16 w-16 items-center justify-center rounded-2xl bg-[linear-gradient(93.65deg,#1
50 |     >
51 |     <Icon className="h-8 w-8 text-white" />
52 |   </motion.div>
53 |
54 |   <h2 className="text-xl font-bold tracking-tight text-gray-900">
55 |     {content.title}
56 |   </h2>
57 |   <p className="mt-2 max-w-md text-sm text-gray-500">
58 |     {content.description}
59 |   </p>
60 |
61 |   {/* Mascot coaching at the assessment-intro moment (R35.1). Renders nothing
62 |      when mascot guidance is disabled or inactive (R35.4, R35.5). */}
63 |   <Mascot moment="assessmentIntro" className="mt-6 w-full max-w-sm" />
64 |
65 |   {/* Estimated time (R17.2) */}
66 |   <Card className="mt-6 flex w-full max-w-sm flex-row items-center gap-3 border-0 bg-blue-50 p-4 shadow-no
67 |     <div className="flex h-10 w-10 shrink-0 items-center justify-center rounded-lg bg-blue-100">
68 |       <Clock className="h-5 w-5 text-blue-600" />
69 |     </div>
70 |     <p className="text-start text-sm font-semibold text-gray-900">
71 |       {content.estimatedTime}
72 |     </p>
73 |   </Card>
74 |
75 |   {/* Concrete benefits (R17.3) */}
76 |   <ul className="mt-6 w-full max-w-sm space-y-3 text-start">
77 |     {content.benefits.map((benefit) => (
78 |       <li key={benefit} className="flex items-start gap-3">
79 |         <span className="mt-0.5 flex h-5 w-5 shrink-0 items-center justify-center rounded-full bg-green-10
80 |           <Check className="h-3 w-3 text-green-600" />
81 |         </span>
82 |         <span className="text-sm font-medium text-gray-700">{benefit}</span>
83 |       </li>
84 |     ))}
85 |   </ul>
86 |
87 |   <Button onClick={onBegin} variant="tactile" className="mt-8 gap-2 px-8">
88 |     {beginLabel}
89 |     <ArrowRight className="h-4 w-4" />
90 |   </Button>
91 | </div>
92 | );
93 | };
94 |
95 | export default AssessmentIntro;
96 |

```

FILE: [src/components/shared/AssessmentTimer.tsx](#)

TYPE: .tsx | LINES: 1-91 | SHA256: e96fd98088e436592d3760ca313b0ffee716ae60ecd470f9517b6a9e8034b

```

1 | import { useEffect, useRef, useCallack, useState } from "react";
2 | import { Clock } from "lucide-react";
3 | import { cn } from "@lib/utills";
4 |
5 | const WARNING_THRESHOLD_SECONDS = 120; // 2 minutes
6 |
7 | export interface AssessmentTimerProps {
8 |   /** Total time in seconds */
9 |   totalSeconds: number;
10 |   onExpire: () => void;
11 |   /** Whether the timer is paused */
12 |   paused?: boolean;
13 | }
14 |
15 | const AssessmentTimer = ({
16 |   totalSeconds,
17 |   onExpire,
18 |   paused = false,
19 | }: AssessmentTimerProps) => {
20 |   const [remaining, setRemaining] = useState(totalSeconds);
21 |   const onExpireRef = useRef(onExpire);
22 |   const expiredRef = useRef(false);
23 |
24 |   useEffect(() => {
25 |     onExpireRef.current = onExpire;
26 |   }, [onExpire]);
27 |
28 |   // Pause on tab hidden via visibility API
29 |   const [tabVisible, setTabVisible] = useState(true);
30 |
31 |   useEffect(() => {
32 |     const handleVisibility = () => {
33 |       setTabVisible(document.visibilityState === "visible");
34 |     };
35 |     document.addEventListener("visibilitychange", handleVisibility);
36 |     return () => {
37 |       document.removeEventListener("visibilitychange", handleVisibility);
38 |     };
39 |   }, []);
40 |
41 |   const tick = useCallack(() => {
42 |     setRemaining((prev) => {
43 |       if (prev <= 1) {

```

```

43     if (!expiredRef.current) {
44       expiredRef.current = true;
45       // Defer callback to avoid setState-in-render
46       queueMicrotask(() => onExpireRef.current());
47     }
48     return 0;
49   }
50   return prev - 1;
51 });
52 }, []));
53
54 useEffect(() => {
55   if (paused || !tabVisible || expiredRef.current) return;
56
57   const id = setInterval(tick, 1000);
58   return () => clearInterval(id);
59 }, [paused, tabVisible, tick]);
60
61 const minutes = Math.floor(remaining / 60);
62 const seconds = remaining % 60;
63 const isWarning = remaining <= WARNING_THRESHOLD_SECONDS && remaining > 0;
64 const isExpired = remaining === 0;
65
66 const formatted = `${String(minutes).padStart(2, "0")}:${String(
67   seconds
68   ).padStart(2, "0")}`;
69
70 return (
71   <div
72     role="timer"
73     aria-live="polite"
74     aria-label={` ${minutes} minutes and ${seconds} seconds remaining` }
75     className={cn(
76       "inline-flex items-center gap-1.5 rounded-lg px-3 py-1.5 text-sm font-bold transition-colors",
77       isExpired
78       ? "bg-red-100 text-red-700"
79       : isWarning
80       ? "bg-red-50 text-red-600 animate-pulse"
81       : "bg-slate-50 text-gray-700"
82     )}
83   >
84     <Clock className={cn("h-4 w-4", isWarning && "text-red-500")} />
85     <span>{formatted}</span>
86   </div>
87 );
88 };
89
90 export default AssessmentTimer;
91

```

FILE: src/components/shared/AtRiskStudentRow.tsx

TYPE: .tsx | LINES: 1-114 | SHA256: 921757f13f5cb732db784dca4d7f9f67f277cf3a7ac088d29db42d3bdd954486

[illegible]


```

38 | },
39 | Not_Yet: {
40 |   bar: "bg-red-600",
41 |   bg: "bg-red-50",
42 |   text: "text-red-600",
43 |   label: "Not Yet",
44 | },
45 | };
46 |
47 | function getAttainmentLevel(percent: number): AttainmentLevel {
48 |   if (percent >= 85) return "Excellent";
49 |   if (percent >= 70) return "Satisfactory";
50 |   if (percent >= 50) return "Developing";
51 |   return "Not_Yet";
52 | }
53 |
54 | const AttainmentBar = ({
55 |   percent,
56 |   label,
57 |   className,
58 |   showLabel = true,
59 | }: AttainmentBarProps) => {
60 |   const clamped = Math.max(0, Math.min(100, percent));
61 |   const level = getAttainmentLevel(clamped);
62 |   const style = ATTAINMENT_STYLES[level];
63 |
64 |   return (
65 |     <div className={cn("space-y-1", className)}>
66 |       <div className="flex items-center justify-between">
67 |         {label} && <span className="text-sm font-medium truncate">{label}</span>
68 |         <div className="flex items-center gap-2 shrink-0">
69 |           <span className="text-sm font-bold tabular-nums">
70 |             {Math.round(clamped)}%
71 |           </span>
72 |           {showLabel && (
73 |             <span className={cn("text-xs font-semibold", style.text)}>
74 |               {style.label}
75 |             </span>
76 |           )}
77 |         </div>
78 |       </div>
79 |       <div
80 |         className={cn("h-2.5 w-full rounded-full overflow-hidden", style.bg)}
81 |       >
82 |         <div
83 |           className={cn(
84 |             "h-full rounded-full transition-all duration-500",
85 |             style.bar
86 |           )}
87 |           style={{ width: `${clamped}%` }}
88 |         </div>
89 |       </div>
90 |     </div>
91 |   );
92 | };
93 |
94 | export default AttainmentBar;
95 | export { getAttainmentLevel, ATTAINMENT_STYLES };
96 | export type { AttainmentBarProps };
97 |

```

FILE: src/components/shared/AttainmentInfo.tsx

TYPE: .tsx | LINES: 1-200 | SHA256: 34f0d05f27b7bba15391b9412d627b705126511a4ca8eb53d016e53c075ed294

```

1 | // =====
2 | // AttainmentInfo – Accessible explanation of attainment mastery and the four
3 | // threshold bands with their colors.
4 | //
5 | // Satisfies Requirement 8:
6 | // - 8.1 accessible explanation that attainment reflects mastery of CLOs
7 | // - 8.2 describes Excellent ("e85%), Satisfactory (70-84%), Developing (50-69%),
8 | //   Not Yet (<50%); 50% !' Developing, <50% !' Not Yet
9 | // - 8.3 each band's color is derived from `attainmentClassifier`, so the legend
10 | //   color always corresponds to the classification band
11 | // - 8.4 available in English and Arabic (i18next, en + ar keys)
12 | // =====
13 |
14 | import { useId } from "react";
15 | import { useTranslation } from "react-i18next";
16 | import { Info } from "lucide-react";
17 |
18 | import {
19 |   Popover,
20 |   PopoverContent,
21 |   PopoverTrigger,
22 | } from "@components/ui/popover";
23 | import { Button } from "@components/ui/button";
24 | import { cn } from "@lib/Utils";
25 | import {
26 |   classifyAttainment,
27 |   getAttainmentColor,
28 | } from "@lib/attainmentClassifier";
29 | import {
30 |   DEFAULT_ATTAINMENT_THRESHOLDS,
31 |   type AttainmentLevel,
32 |   type AttainmentThresholdsConfig,
33 | } from "@types/app";
34 |
35 | interface AttainmentInfoProps {
36 |   /**
37 |    * Optional attainment percentage. When provided (and valid), the band the
38 |    * value falls into is highlighted in the legend. A negative value or omission
39 |    * is treated as "no data" and highlights nothing.
40 |    */

```

```

41 | percent?: number;
42 | /**
43 |  * Institution-configurable thresholds. Defaults to the platform defaults
44 |  * (85 / 70 / 50) so the explanation always matches the active classification.
45 |  */
46 | thresholds?: AttainmentThresholdsConfig;
47 | /** Side the popover opens toward. */
48 | side?: "top" | "right" | "bottom" | "left";
49 | /** Extra classes for the trigger button. */
50 | className?: string;
51 | }
52 |
53 | interface BandRow {
54 |   level: AttainmentLevel;
55 |   name: string;
56 |   range: string;
57 |   /** Hex color sourced from `attainmentClassifier` for this band. */
58 |   color: string;
59 |   active: boolean;
60 | }
61 |
62 | const AttainmentInfo = ({
63 |   percent,
64 |   thresholds = DEFAULT_ATTAINMENT_THRESHOLDS,
65 |   side = "bottom",
66 |   className,
67 | }: AttainmentInfoProps) => {
68 |   const { t } = useTranslation("common");
69 |   const titleId = useId();
70 |   const descId = useId();
71 |
72 |   // Only highlight a band when we have a real, non-negative percentage.
73 |   const activeLevel: AttainmentLevel | null =
74 |     typeof percent === "number" && Number.isFinite(percent) && percent >= 0
75 |       ? classifyAttainment(percent, thresholds)
76 |       : null;
77 |
78 |   // Each band's color is derived by classifying a value known to fall inside
79 |   // that band, guaranteeing the legend swatch matches the classifier (R8.3).
80 |   const bands: BandRow[] = [
81 |     {
82 |       level: "Excellent",
83 |       name: t("attainment.excellent"),
84 |       range: t("attainmentInfo.ranges.excellent", {
85 |         min: thresholds.excellent,
86 |       }),
87 |       color: getAttainmentColor(thresholds.excellent, thresholds),
88 |       active: activeLevel === "Excellent",
89 |     },
90 |     {
91 |       level: "Satisfactory",
92 |       name: t("attainment.satisfactory"),
93 |       range: t("attainmentInfo.ranges.satisfactory", {
94 |         min: thresholds.satisfactory,
95 |         max: thresholds.excellent - 1,
96 |       }),
97 |       color: getAttainmentColor(thresholds.satisfactory, thresholds),
98 |       active: activeLevel === "Satisfactory",
99 |     },
100 |    {
101 |      level: "Developing",
102 |      name: t("attainment.developing"),
103 |      range: t("attainmentInfo.ranges.developing", {
104 |        min: thresholds.developing,
105 |        max: thresholds.satisfactory - 1,
106 |      }),
107 |      color: getAttainmentColor(thresholds.developing, thresholds),
108 |      active: activeLevel === "Developing",
109 |    },
110 |    {
111 |      level: "Not_Yet",
112 |      name: t("attainment.notYet"),
113 |      range: t("attainmentInfo.ranges.notYet", {
114 |        threshold: thresholds.developing,
115 |      }),
116 |      // Any value below `developing` classifies as Not_Yet.
117 |      color: getAttainmentColor(thresholds.developing - 1, thresholds),
118 |      active: activeLevel === "Not_Yet",
119 |    },
120 |  ];
121 |
122 |   return (
123 |     <Popover>
124 |       <PopoverTrigger asChild>
125 |         <Button
126 |           type="button"
127 |           variant="ghost"
128 |           aria-label={t("attainmentInfo.triggerLabel")}
129 |           data-testid="attainment-info-trigger"
130 |           className={cn(
131 |             // 44px touch target while keeping a compact icon.
132 |             "inline-flex h-11 w-11 items-center justify-center rounded-full p-0 text-muted-foreground hover:te
133 |           )}
134 |         >
135 |           <Info className="h-4 w-4" aria-hidden="true" />
136 |         </Button>
137 |       </PopoverTrigger>
138 |       <PopoverContent
139 |         side={side}
140 |         align="start"
141 |         role="dialog"
142 |         aria-labelledby={titleId}
143 |         aria-describedby={descId}
144 |

```

```

145 |         data-testid="attainment-info-content"
146 |     >
147 |         <div className="space-y-3">
148 |             <div>
149 |                 <h3
150 |                     id={titleId}
151 |                     className="text-sm font-bold tracking-tight text-foreground"
152 |                 >
153 |                     {t("attainmentInfo.title")}
154 |                 </h3>
155 |                 <p id={descId} className="mt-1 text-xs text-muted-foreground">
156 |                     {t("attainmentInfo.description")}
157 |                 </p>
158 |             </div>
159 |
160 |             <div>
161 |                 <p className="text-[10px] font-black uppercase tracking-widest text-muted-foreground">
162 |                     {t("attainmentInfo.bandsHeading")}
163 |                 </p>
164 |                 <ul className="mt-2 space-y-1.5">
165 |                     {bands.map((band) => (
166 |                         <li
167 |                             key={band.level}
168 |                             data-testid={ `attainment-band-${band.level}` }
169 |                             data-active={band.active ? "true" : undefined}
170 |                             className="flex items-center gap-2"
171 |                         >
172 |                             <span
173 |                                 className="h-3 w-3 shrink-0 rounded-full"
174 |                                 style={{ backgroundColor: band.color }}
175 |                                 aria-hidden="true"
176 |                             />
177 |                             <span
178 |                                 className={cn(
179 |                                     "text-sm text-foreground",
180 |                                     band.active ? "font-bold" : "font-medium"
181 |                                 )}
182 |                             >
183 |                                 {band.name}
184 |                             </span>
185 |                             <span className="ms-auto text-xs tabular-nums text-muted-foreground">
186 |                                 {band.range}
187 |                             </span>
188 |                         </li>
189 |                     ))}
190 |                 </ul>
191 |             </div>
192 |         </div>
193 |     </PopoverContent>
194 | </Popover>
195 | );
196 | };
197 |
198 | export default AttainmentInfo;
199 | export type { AttainmentInfoProps };
200 |

```

FILE: src/components/shared/AttendanceGrid.tsx

TYPE: .tsx | LINES: 1-89 | SHA256: 4d66a8fa0699fb7ef23a940e18ead64b19d0695fffc7412ff7868c134e2e0793b

```

1 | // =====
2 | // AttendanceGrid – Attendance marking grid
3 | // =====
4 |
5 | import { cn } from "@lib/utils";
6 |
7 | type AttendanceStatus = "present" | "absent" | "late" | "excused";
8 |
9 | interface StudentAttendance {
10 |   studentId: string;
11 |   studentName: string;
12 |   status: AttendanceStatus;
13 | }
14 |
15 | interface AttendanceGridProps {
16 |   students: StudentAttendance[];
17 |   onStatusChange: (studentId: string, status: AttendanceStatus) => void;
18 |   className?: string;
19 | }
20 |
21 | const STATUS_STYLES: Record<
22 |   AttendanceStatus,
23 |   { bg: string; text: string; label: string }
24 | > = {
25 |   present: {
26 |     bg: "bg-green-50 border-green-300",
27 |     text: "text-green-700",
28 |     label: "P",
29 |   },
30 |   absent: { bg: "bg-red-50 border-red-300", text: "text-red-700", label: "A" },
31 |   late: {
32 |     bg: "bg-yellow-50 border-yellow-300",
33 |     text: "text-yellow-700",
34 |     label: "L",
35 |   },
36 |   excused: {
37 |     bg: "bg-blue-50 border-blue-300",
38 |     text: "text-blue-700",
39 |     label: "E",
40 |   },
41 | };
42 |
43 | const STATUSES: AttendanceStatus[] = ["present", "absent", "late", "excused"];
44 |

```

```

45 | const AttendanceGrid = ({
46 |   students,
47 |   onStatusChange,
48 |   className,
49 | }: AttendanceGridProps) => (
50 |   <div className={cn("space-y-2", className)}>
51 |     {students.map((student) => (
52 |       <div
53 |         key={student.studentId}
54 |         className="flex items-center gap-3 py-2 border-b border-slate-100 last:border-0"
55 |       >
56 |         <span className="text-sm font-medium flex-1 min-w-0 truncate">
57 |           {student.studentName}
58 |         </span>
59 |         <div className="flex gap-1">
60 |           {STATUSES.map((status) => {
61 |             const style = STATUS_STYLES[status];
62 |             const isActive = student.status === status;
63 |             return (
64 |               <button
65 |                 key={status}
66 |                 type="button"
67 |                 onClick={() => onStatusChange(student.studentId, status)}
68 |                 className={cn(
69 |                   "h-8 w-8 rounded-lg border text-xs font-bold transition-colors",
70 |                   isActive
71 |                     ? `${style.bg} ${style.text}`
72 |                     : "border-gray-200 text-gray-400 hover:bg-gray-50"
73 |                 )}
74 |                 title={status}
75 |               >
76 |                 {style.label}
77 |               </button>
78 |             );
79 |           })}
80 |         </div>
81 |       </div>
82 |     ))}
83 |   </div>
84 | );
85 |
86 | export default AttendanceGrid;
87 | export { STATUS_STYLES, STATUSES };
88 | export type { AttendanceGridProps, StudentAttendance, AttendanceStatus };
89 |

```

FILE: src/components/shared/AuthBrandPanel.tsx

TYPE: .tsx | LINES: 1-316 | SHA256: 4f90dfcb8360a54233e359c7dce7abdca2f11d8179ae1137474da2fd83983239

```

1 | // =====
2 | // AuthBrandPanel – Redesigned brand & value marketing panel
3 | // =====
4 | //
5 | // Matches reference design pixel-for-pixel:
6 | // - Deep navy background with faint educational doodles & radial glows
7 | // - Brand logo + "E DEVISER" wordmark
8 | // - "Outcome-Based • Habit-Driven • Future-Ready" badge
9 | // - Headline with gradient "Outcomes" accent
10 | // - Side-by-side Dual Engine cards (OBE Engine & Habit Engine) connected by an infinity badge
11 | // - 3D Foxi mascot with learning path graphic & milestones
12 | // - "Your Learning Journey" card with Level 7 XP progress bar
13 | // - "Trusted & Secure" card (FERPA-aligned, Privacy-first, Enterprise security)
14 | // - Social proof bar (avatars, 4.9/5 stars, 1,200+ institutions)
15 | // =====
16 |
17 | import { useTranslation } from "react-i18next";
18 | import {
19 |   GraduationCap,
20 |   Target,
21 |   ShieldCheck,
22 |   Lock,
23 |   Star,
24 |   Flag,
25 |   TrendingUp,
26 | } from "lucide-react";
27 | import foxiSmiling from "@design-system/mascot/assets/characters/foxi/foxi-smiling.png";
28 |
29 | const AuthBrandPanel = () => {
30 |   const { t } = useTranslation("auth");
31 |
32 |   return (
33 |     <div
34 |       className="auth-brand-panel relative flex flex-col justify-between overflow-hidden bg-cover bg-center px
35 |       style={{
36 |         backgroundImage: "url('/auth-assets/01_auth_background_only.png')",
37 |       }}
38 |     >
39 |       {/* Ambient radial glows */}
40 |       <div
41 |         className="pointer-events-none absolute end-4 top-10 h-80 w-80 rounded-full opacity-40 blur-3xl"
42 |         style={{
43 |           background:
44 |             "radial-gradient(circle, rgba(56,189,248,0.4) 0%, rgba(20,184,166,0.2) 60%, transparent 80%)",
45 |         }}
46 |       />
47 |       <div
48 |         className="pointer-events-none absolute -start-10 bottom-16 h-72 w-72 rounded-full opacity-30 blur-3xl"
49 |         style={{
50 |           background:
51 |             "radial-gradient(circle, rgba(14,165,233,0.3) 0%, rgba(99,102,241,0.2) 70%, transparent 90%)",
52 |         }}
53 |       />
54 |
55 |       {/* TOP & CENTER CONTENT BLOCK */}

```

```

56 | <div className="relative z-10 flex flex-col gap-6">
57 |   {/* BRAND HEADER: Transparent Glowing Infinity Logo Mark (Bigger, No Box) */}
58 |   <div className="flex items-center justify-between">
59 |     <div className="flex items-center gap-3">
60 |       <svg
61 |         className="h-12 w-12 text-cyan-400 drop-shadow-[0_0_20px_rgba(34,211,238,0.9)]"
62 |         viewBox="0 0 60 30"
63 |         fill="none"
64 |       >
65 |         <path
66 |           d="M12 15 C12 8, 25 8, 30 15 C35 22, 48 22, 48 15 C48 8, 35 8, 30 15 C25 22, 12 22, 12 15 Z"
67 |           stroke="url(#infinityHeaderGlow)"
68 |           strokeWidth="4"
69 |           strokeLinecap="round"
70 |         />
71 |         <circle cx="15" cy="13" r="1.5" fill="#38bdf8" />
72 |         <circle cx="45" cy="17" r="1.5" fill="#2dd4bf" />
73 |         <defs>
74 |           <linearGradient
75 |             id="infinityHeaderGlow"
76 |             x1="0"
77 |             y1="0"
78 |             x2="1"
79 |             y2="0"
80 |           >
81 |             <stop offset="0%" stopColor="#00d2ff" />
82 |             <stop offset="50%" stopColor="#38bdf8" />
83 |             <stop offset="100%" stopColor="#00f2fe" />
84 |           </linearGradient>
85 |         </defs>
86 |       </svg>
87 |       <span className="text-2xl font-black tracking-wider text-white drop-shadow-[0_2px_10px_rgba(0,0,0,
88 |         E DEVISER
89 |       </span>
90 |     </div>
91 |
92 |     {/* Product Positioning Pill Badge */}
93 |     <div className="inline-flex items-center gap-2 rounded-full border border-teal-400/30 bg-teal-500/10
94 |       <span className="h-2 w-2 rounded-full bg-teal-400 animate-pulse shadow-[0_0_8px_rgba(45,212,191,0.
95 |         {("brand.badge", "Outcome-Based • Habit-Driven • Future-Ready")}
96 |       </div>
97 |     </div>
98 |
99 |     {/* HERO SECTION: HEADLINE + MASCOT & LEARNING PATH */}
100 |    <div className="relative grid grid-cols-1 items-center gap-6 lg:grid-cols-12 mt-2">
101 |      {/* Left Text Column */}
102 |      <div className="lg:col-span-7 space-y-3">
103 |        <h1 className="text-3xl font-black leading-tight tracking-tight text-white sm:text-4xl lg:text-4xl
104 |          Learning that adapts.
105 |        <br />
106 |        <span className="bg-gradient-to-r from-cyan-400 via-teal-300 to-emerald-400 bg-clip-text text-tr
107 |          Outcomes
108 |        </span>{" "}
109 |        that last.
110 |      </h1>
111 |
112 |      <p className="max-w-xl text-sm leading-relaxed text-slate-300/90 font-medium">
113 |        {t(
114 |          "brand.subtitle",
115 |          "E Deviser combines a powerful Outcome-Based Education (OBE) engine with a Habit Formation eng
116 |        )}
117 |      </p>
118 |    </div>
119 |
120 |    {/* Right Mascot Column */}
121 |    <div className="hidden lg:col-span-5 lg:flex relative items-center justify-center min-h-[160px]">
122 |      {/* Milestones / Learning Journey Path behind mascot */}
123 |      <div className="absolute inset-0 pointer-events-none flex items-center justify-center">
124 |        <svg
125 |          className="w-full h-full"
126 |          viewBox="0 0 200 130"
127 |          fill="none"
128 |          preserveAspectRatio="none"
129 |        >
130 |          {/* Curved Hill Surface */}
131 |          <path
132 |            d="M0 130 Q 100 75 200 110"
133 |            fill="none"
134 |            stroke="rgba(30, 58, 138, 0.4)"
135 |            strokeWidth="50"
136 |          />
137 |          {/* Dotted path */}
138 |          <path
139 |            d="M20 105 Q 95 65 175 25"
140 |            stroke="url(#pathGradient)"
141 |            strokeWidth="3"
142 |            strokeDasharray="4 4"
143 |          />
144 |          <defs>
145 |            <linearGradient id="pathGradient" x1="0" y1="0" x2="1" y2="0">
146 |              <stop offset="0%" stopColor="#38bdf8" />
147 |              <stop offset="100%" stopColor="#2dd4bf" />
148 |            </linearGradient>
149 |          </defs>
150 |        </svg>
151 |
152 |        {/* Path Node 1 */}
153 |        <div className="absolute start-[25%] bottom-[30%] flex h-6 w-6 items-center justify-center round
154 |          <Target className="h-3 w-3" />
155 |        </div>
156 |        {/* Path Node 2 */}
157 |        <div className="absolute start-[55%] bottom-[50%] flex h-6 w-6 items-center justify-center round
158 |          <TrendingUp className="h-3 w-3" />
159 |        </div>

```

```

160      { /* Path Node 3 (Star) */ }
161      <div className="absolute start-[78%] bottom-[70%] flex h-6 w-6 items-center justify-center round
162      <Star className="h-3 w-3 fill-amber-300" />
163      </div>
164      { /* Flag at peak */ }
165      <div className="absolute end-[5%] top-[8%] text-teal-300">
166      <Flag className="h-5 w-5 fill-teal-400" />
167      </div>
168    </div>
169
170    { /* Foxi 3D Mascot Robot */ }
171    <div className="relative z-10 hover:scale-105 transition-transform duration-300 drop-shadow-[0_15p
172    <img
173      src={foxiSmiling}
174      alt="E Deviser Foxi Mascot"
175      className="h-44 w-auto object-contain"
176    />
177    </div>
178  </div>
179 </div>
180
181 { /* COMPACT LEFT-ALIGNED CARDS CONTAINER */ }
182 <div className="flex w-full max-w-[520px] flex-col gap-4 mt-1">
183 { /* DUAL ENGINE ARCHITECTURE CARDS */ }
184 <div className="relative grid grid-cols-1 gap-4 sm:grid-cols-2">
185 { /* OBE Engine Card */ }
186 <div className="relative rounded-2xl border border-cyan-500/30 bg-gradient-to-br from-[#071d47]/95
187 <div className="mb-3 flex h-10 w-10 items-center justify-center rounded-full border border-cyan-
188 <GraduationCap className="h-5 w-5" />
189 </div>
190 <h3 className="text-base font-extrabold text-white">
191   OBE Engine
192 </h3>
193 <p className="mt-1 text-xs leading-relaxed text-slate-300/90">
194   Focuses on CLO/PLO/ILO mastery and measurable outcomes.
195 </p>
196 </div>
197
198 { /* 3D Glowing Infinity Ribbon Connector Loop */ }
199 <div className="absolute start-1/2 top-1/2 z-20 hidden -translate-x-1/2 -translate-y-1/2 sm:flex">
200 <div className="flex h-10 w-16 items-center justify-center rounded-full border border-cyan-300/5
201 <svg
202   className="w-full h-full text-white drop-shadow-[0_0_8px_rgba(255,255,255,0.9)]"
203   viewBox="0 0 60 30"
204   fill="none"
205 >
206   <path
207     d="M10 15 C10 7, 25 7, 30 15 C35 23, 50 23, 50 15 C50 7, 35 7, 30 15 C25 23, 10 23, 10 15
208     stroke="#ffffff"
209     strokeWidth="4.5"
210     strokeLinecap="round"
211   />
212 </svg>
213 </div>
214 </div>
215
216 { /* Habit Engine Card */ }
217 <div className="relative rounded-2xl border border-blue-500/30 bg-gradient-to-br from-[#071d47]/95
218 <div className="mb-3 flex h-10 w-10 items-center justify-center rounded-full border border-blue-
219 <Target className="h-5 w-5" />
220 </div>
221 <h3 className="text-base font-extrabold text-white">
222   Habit Engine
223 </h3>
224 <p className="mt-1 text-xs leading-relaxed text-slate-300/90">
225   Focuses on streaks, XP, rewards, and mastery paths.
226 </p>
227 </div>
228 </div>
229
230 { /* YOUR LEARNING JOURNEY PROGRESS CARD */ }
231 <div className="rounded-2xl border border-cyan-500/25 bg-gradient-to-r from-[#071d47]/95 via-[#05163
232 <div className="flex items-center justify-between text-xs font-bold">
233   <span className="text-slate-300">Your Learning Journey</span>
234   <span className="text-cyan-300">
235     <span className="text-white font-extrabold">XP 2,450</span> /
236     3,000
237   </span>
238 </div>
239 <div className="mt-2.5 flex items-center gap-3">
240   <div className="inline-flex items-center gap-1.5 rounded-xl border border-amber-400/40 bg-amber-
241     <Star className="h-3.5 w-3.5 fill-amber-300 text-amber-400" />
242     Level 7
243   </div>
244   <div className="relative h-2.5 flex-1 overflow-hidden rounded-full bg-slate-800/90 border border
245     <div
246       className="h-full rounded-full bg-gradient-to-r from-blue-500 via-cyan-400 to-teal-300 shado
247       style={{ width: "81%" }}
248     />
249   </div>
250 </div>
251 </div>
252 </div>
253 </div>
254
255 { /* BOTTOM TRUST & SECURE FOOTER */ }
256 <div className="relative z-10 mt-6 w-full max-w-[520px]">
257 <div className="rounded-2xl border border-cyan-500/25 bg-gradient-to-r from-[#071d47]/95 via-[#051638]
258 { /* Security Row */ }
259 <div className="flex items-center justify-between">
260   <div className="flex items-center gap-3">
261     <div className="flex h-9 w-9 items-center justify-center rounded-xl border border-cyan-400/40 bg
262     <ShieldCheck className="h-5 w-5" />
263   </div>

```

```

264 |         <div>
265 |             <h3 className="text-sm font-extrabold text-white">
266 |                 Trusted &amp; Secure
267 |             </h3>
268 |             <p className="text-[11px] text-slate-300/80 font-medium">
269 |                 FERPA aligned • Privacy-first • Enterprise-grade security
270 |             </p>
271 |         </div>
272 |     </div>
273 |     <Lock className="h-4 w-4 text-slate-400" />
274 | </div>
275 |
276 |     {/ * Social Proof Row */}
277 |     <div className="flex items-center justify-between rounded-xl border border-white/10 bg-white/5 px-3.
278 |     <div className="flex -space-x-2">
279 |         
284 |         
289 |         
294 |         
299 |     </div>
300 |     <div className="flex items-center gap-1 text-xs font-bold text-amber-400">
301 |         <Star className="h-3.5 w-3.5 fill-amber-400 text-amber-400" />
302 |         <Star className="h-3.5 w-3.5 fill-amber-400 text-amber-400" />
303 |         <Star className="h-3.5 w-3.5 fill-amber-400 text-amber-400" />
304 |         <Star className="h-3.5 w-3.5 fill-amber-400 text-amber-400" />
305 |         <Star className="h-3.5 w-3.5 fill-amber-400 text-amber-400" />
306 |         <span className="text-white font-extrabold ms-1">4.9/5</span>
307 |     </div>
308 | </div>
309 | </div>
310 | </div>
311 | </div>
312 | );
313 | };
314 |
315 | export default AuthBrandPanel;
316 |

```

FILE: src/components/shared/AuthShell.tsx

TYPE: .tsx | LINES: 1-48 | SHA256: e64cf08ed3e40236c250a2841972115d53085b575d46c4524d8bf71746425d6d

```

1 | // =====
2 | // AuthShell – shared split-panel background for secondary auth pages
3 | // =====
4 | //
5 | // Reproduces the prototype `auth.html` layout (a light #f8fafc canvas split
6 | // into a hero-gradient brand/value panel + a form panel) so the secondary auth
7 | // screens (Reset / Update password) share the exact same treatment as the
8 | // rebuilt `LoginPage`. The page's form is slotted into the form panel via
9 | // `children`.
10 | //
11 | // Presentation-only: all auth logic stays in the page components. Light + LTR
12 | // (prototype scope; PARITY SE).
13 | // =====
14 |
15 | import type { ReactNode } from "react";
16 | import { LanguageSwitcher } from "@components/shared/LanguageSwitcher";
17 | import AuthBrandPanel from "@components/shared/AuthBrandPanel";
18 |
19 | interface AuthShellProps {
20 |     children: ReactNode;
21 | }
22 |
23 | const AuthShell = ({ children }: AuthShellProps) => {
24 |     return (
25 |         <div className="auth-page relative grid min-h-[100dvh] w-full grid-cols-1 bg-[#f3f6fc] overflow-x-clip lg:
26 |             {/ * Brand / value panel (Left) */}
27 |             <div className="order-2 lg:order-1 h-full">
28 |                 <AuthBrandPanel />
29 |             </div>
30 |
31 |             {/ * Form panel (Right) */}
32 |             <div className="relative order-1 lg:order-2 flex flex-col justify-between px-6 py-8 sm:px-10 lg:px-12 lg
33 |                 <div className="flex justify-end w-full mb-6">
34 |                     <LanguageSwitcher />
35 |                 </div>
36 |
37 |                 <div className="mx-auto w-full max-w-[500px] my-auto">
38 |                     <div className="rounded-3xl border border-slate-100/90 bg-white p-7 sm:p-9 shadow-[0_20px_50px_rgba(
39 |                         {children}
40 |                     </div>
41 |                 </div>
42 |             </div>
43 |         </div>
44 |     );
45 | };
46 |
47 | export default AuthShell;

```


FILE: src/components/shared/AutonomyToggle.tsx

TYPE: .tsx | LINES: 1-85 | SHA256: fee1fb0a14de2d117d3123673c5c086fa7fffe7be77f377125cf5c4d98297d8d

```

1 | // =====
2 | // AutonomyToggle – Student-facing "Figure it out" / "Just explain it" toggle
3 | // =====
4 |
5 | import { Lightbulb, BookOpen } from "lucide-react";
6 | import { useTranslation } from "react-i18next";
7 | import { Button } from "@components/ui/button";
8 | import { cn } from "@lib/utls";
9 |
10 | interface AutonomyToggleProps {
11 |   /** Current autonomy override: L1, L3, or null (use default) */
12 |   value: "L1" | "L3" | null;
13 |   /** Called when the student toggles the autonomy level */
14 |   onChange: (level: "L1" | "L3" | null) => void;
15 |   /** Disable the toggle (e.g., when rate limit reached) */
16 |   disabled?: boolean;
17 | }
18 |
19 | const AutonomyToggle = ({
20 |   value,
21 |   onChange,
22 |   disabled = false,
23 | }: AutonomyToggleProps) => {
24 |   const { t } = useTranslation("ai");
25 |
26 |   const handleClick = (level: "L1" | "L3") => {
27 |     if (disabled) return;
28 |     // Clicking the active button deselects it (sets to null = use default)
29 |     onChange(value === level ? null : level);
30 |   };
31 |
32 |   return (
33 |     <div
34 |       className="flex items-center gap-1"
35 |       role="radiogroup"
36 |       aria-label={t("tutor.autonomy.groupLabel")}
37 |     >
38 |       <Button
39 |         variant="outline"
40 |         size="sm"
41 |         role="radio"
42 |         aria-checked={value === "L1"}
43 |         aria-label={t("tutor.autonomy.figureItOutLabel")}
44 |         disabled={disabled}
45 |         onClick={() => handleClick("L1")}
46 |         className={cn(
47 |           "gap-1 text-xs h-7 px-2 transition-colors",
48 |           value === "L1"
49 |             ? "bg-blue-600 text-white border-blue-600 hover:bg-blue-700 hover:text-white"
50 |             : "bg-white text-gray-600 border-gray-200 hover:bg-gray-50"
51 |         )}
52 |       >
53 |         <Lightbulb className="h-3.5 w-3.5" />
54 |         <span className="hidden sm:inline">
55 |           {t("tutor.autonomy.figureItOut")}
56 |         </span>
57 |       </Button>
58 |
59 |       <Button
60 |         variant="outline"
61 |         size="sm"
62 |         role="radio"
63 |         aria-checked={value === "L3"}
64 |         aria-label={t("tutor.autonomy.justExplainLabel")}
65 |         disabled={disabled}
66 |         onClick={() => handleClick("L3")}
67 |         className={cn(
68 |           "gap-1 text-xs h-7 px-2 transition-colors",
69 |           value === "L3"
70 |             ? "bg-blue-600 text-white border-blue-600 hover:bg-blue-700 hover:text-white"
71 |             : "bg-white text-gray-600 border-gray-200 hover:bg-gray-50"
72 |         )}
73 |       >
74 |         <BookOpen className="h-3.5 w-3.5" />
75 |         <span className="hidden sm:inline">
76 |           {t("tutor.autonomy.justExplain")}
77 |         </span>
78 |       </Button>
79 |     </div>
80 |   );
81 | };
82 |
83 | export default AutonomyToggle;
84 | export type { AutonomyToggleProps };
85 |

```

FILE: src/components/shared/AvatarUpload.tsx

TYPE: .tsx | LINES: 1-221 | SHA256: 2d413e412d1183c9020a86eae6ce21c5be6c535406ecd5b70888b2070ce7773e

```

1 | import { useRef, useState } from "react";
2 | import { Card } from "@components/ui/card";
3 | import { Button } from "@components/ui/button";
4 | import { useAvatarUpload } from "@hooks/useAvatarUpload";
5 | import { Upload, X } from "lucide-react";
6 | import { toast } from "sonner";
7 | import { useTranslation } from "react-i18next";
8 |
9 | export interface AvatarUploadProps {
10 |   userId: string;

```

```

11 |   currentUrl?: string | null;
12 | }
13 |
14 | /**
15 |  * Avatar upload component with dropzone and preview.
16 |  * Validates type/size, resizes, uploads to Supabase Storage, updates profile.
17 |  *
18 |  * Design: ADR-04
19 |  * Requirements: 2.18
20 |  *
21 |  * @example
22 |  * <AvatarUpload userId={user.id} currentUrl={profile.avatar_url} />
23 |  */
24 | const AvatarUpload = ({ currentUrl }: AvatarUploadProps) => {
25 |   const { t } = useTranslation("common");
26 |   const [preview, setPreview] = useState<string | null>(null);
27 |   const [selectedFile, setSelectedFile] = useState<File | null>(null);
28 |   const { uploadAvatar, deleteAvatar, isPending } = useAvatarUpload();
29 |   const inputRef = useRef<HTMLInputElement>(null);
30 |
31 |   const handleFileSelect = async (file: File) => {
32 |     // Validate file type
33 |     const validTypes = ["image/png", "image/jpeg", "image/webp"];
34 |     if (!validTypes.includes(file.type)) {
35 |       toast.error(t("avatar.invalidType"));
36 |       return;
37 |     }
38 |
39 |     // Validate file size (2 MB max)
40 |     if (file.size > 2 * 1024 * 1024) {
41 |       toast.error(t("avatar.fileTooLarge"));
42 |       return;
43 |     }
44 |
45 |     setSelectedFile(file);
46 |
47 |     // Create preview
48 |     const reader = new FileReader();
49 |     reader.onload = (e) => {
50 |       setPreview(e.target?.result as string);
51 |     };
52 |     reader.readAsDataURL(file);
53 |   };
54 |
55 |   const handleDrop = (e: React.DragEvent<HTMLDivElement>) => {
56 |     e.preventDefault();
57 |     e.stopPropagation();
58 |     const files = e.dataTransfer.files;
59 |     const file = files[0];
60 |     if (file) {
61 |       handleFileSelect(file);
62 |     }
63 |   };
64 |
65 |   const handleUpload = async () => {
66 |     if (!selectedFile) {
67 |       toast.error(t("avatar.selectFile"));
68 |       return;
69 |     }
70 |
71 |     try {
72 |       await uploadAvatar({ file: selectedFile });
73 |       toast.success(t("avatar.uploadSuccess"));
74 |       setPreview(null);
75 |       setSelectedFile(null);
76 |     } catch (err) {
77 |       toast.error(err instanceof Error ? err.message : t("common.error"));
78 |     }
79 |   };
80 |
81 |   const handleCancel = () => {
82 |     setPreview(null);
83 |     setSelectedFile(null);
84 |   };
85 |
86 |   return (
87 |     <Card className="bg-white border-0 shadow-md rounded-xl p-6">
88 |       <div className="space-y-4">
89 |         <div>
90 |           <h3 className="text-lg font-bold tracking-tight">
91 |             {t("avatar.title")}
92 |           </h3>
93 |           <p className="text-sm text-gray-600 mt-1">
94 |             {t("avatar.description")}
95 |           </p>
96 |         </div>
97 |
98 |         {/** Current avatar display */}
99 |         {currentUrl && !preview && (
100 |           <div className="flex items-center justify-between gap-4 rounded-xl border border-slate-100 bg-slate-
101 |             <div className="flex items-center gap-3">
102 |               <img
103 |                 src={currentUrl}
104 |                 alt="Current avatar"
105 |                 loading="lazy"
106 |                 decoding="async"
107 |                 className="h-14 w-14 rounded-xl object-cover border border-slate-200"
108 |               />
109 |             </div>
110 |             <p className="text-xs font-bold text-slate-800">
111 |               {t("avatar.current", "Current photo")}
112 |             </p>
113 |             <p className="text-[11px] text-slate-500">
114 |               {t("avatar.uploaded", "Uploaded avatar active")}

```

```

115         </p>
116     </div>
117 </div>
118 <Button
119   type="button"
120   variant="outline"
121   size="sm"
122   disabled={isPending}
123   onClick={async () => {
124     try {
125       await deleteAvatar();
126     } catch {
127       // toast handled inside hook
128     }
129   }}
130   className="text-xs font-bold text-red-600 hover:bg-red-50 hover:text-red-700"
131 >
132   <X className="h-3.5 w-3.5 me-1" />
133   {t("avatar.remove", "Remove")}
134 </Button>
135 </div>
136 }}
137
138 {/* Preview */}
139 {preview && (
140   <div className="flex items-center gap-4">
141     <img
142       src={preview}
143       alt="Preview"
144       className="h-24 w-24 rounded-lg object-cover"
145     />
146     <div>
147       <p className="text-sm text-gray-600">{t("avatar.preview")}</p>
148       <p className="text-xs text-gray-500 mt-1">{selectedFile?.name}</p>
149     </div>
150   </div>
151 )}
152
153 {/* Dropzone */}
154 {!preview && (
155   <div
156     onDrop={handleDrop}
157     onDragOver={(e) => e.preventDefault()}
158     className="border-2 border-dashed border-slate-300 rounded-lg p-8 text-center hover:border-blue-40"
159   >
160     <Upload className="h-8 w-8 text-gray-400 mx-auto mb-2" />
161     <p className="text-sm font-medium text-gray-700">
162       {t("avatar.dragDrop")}
163     </p>
164     <p className="text-xs text-gray-500 mt-1">{t("avatar.formats")}</p>
165     <input
166       ref={inputRef}
167       type="file"
168       accept="image/png,image/jpeg,image/webp"
169       onChange={(e) => {
170         if (e.target.files?.[0]) {
171           handleFileSelect(e.target.files[0]);
172         }
173         // Reset so re-selecting the same file still fires onChange.
174         e.target.value = "";
175       }}
176     className="hidden"
177     id="avatar-input"
178     aria-hidden="true"
179     tabIndex=-1
180   />
181     <Button
182       type="button"
183       variant="outline"
184       size="sm"
185       className="mt-4 cursor-pointer"
186       onClick={() => inputRef.current?.click()}
187     >
188       {t("avatar.selectFile")}
189     </Button>
190   </div>
191 )}
192
193 {/* Action buttons */}
194 {preview && (
195   <div className="flex gap-3">
196     <Button
197       onClick={handleUpload}
198       disabled={isPending}
199       variant="tactile"
200       className="flex-1"
201     >
202       {isPending ? t("common.uploading") : t("avatar.upload")}
203     </Button>
204     <Button
205       onClick={handleCancel}
206       variant="outline"
207       disabled={isPending}
208       className="flex-1"
209     >
210       <X className="h-4 w-4 me-1" />
211       {t("common.cancel")}
212     </Button>
213   </div>
214 )}
215 </div>
216 </Card>
217 );
218 };

```

```
219 |
220 | export default AvatarUpload;
221 |
```

FILE: src/components/shared/BadgeAwardModal.tsx

TYPE: .tsx | LINES: 1-206 | SHA256: 4037abf48c09c98707ae923971c3a4d7ea96249e50e95e14953058a599a7937f

```
1 | import { useEffect, useRef, useCallback, useSyncExternalStore } from "react";
2 | import { motion, AnimatePresence, useReducedMotion } from "framer-motion";
3 | import { launchConfetti } from "@lib/confetti";
4 | import {
5 |   Dialog,
6 |   DialogContent,
7 |   DialogTitle,
8 |   DialogDescription,
9 | } from "@components/ui/dialog";
10 | import { Button } from "@components/ui/button";
11 | import { Sparkles } from "lucide-react";
12 |
13 | interface BadgeAwardModalProps {
14 |   badge: {
15 |     id: string;
16 |     name: string;
17 |     description: string;
18 |     icon: string;
19 |     isMystery: boolean;
20 |     xpReward: number;
21 |   };
22 |   isOpen: boolean;
23 |   onClose: () => void;
24 | }
25 |
26 | const MYSTERY_REVEAL_DELAY_MS = 1000;
27 |
28 | /**
29 |  * Hook that manages a delayed boolean flag via useSyncExternalStore,
30 |  * avoiding setState-in-effect lint warnings.
31 |  */
32 | function useDelayedFlag(active: boolean, delayMs: number): boolean {
33 |   const listeners = useRef(new Set<() => void>());
34 |   const value = useRef(false);
35 |
36 |   useEffect(() => {
37 |     if (!active) {
38 |       if (value.current) {
39 |         value.current = false;
40 |         listeners.current.forEach((fn) => fn());
41 |       }
42 |       return;
43 |     }
44 |
45 |     // Start unrevealed
46 |     value.current = false;
47 |     listeners.current.forEach((fn) => fn());
48 |
49 |     const timer = setTimeout(() => {
50 |       value.current = true;
51 |       listeners.current.forEach((fn) => fn());
52 |     }, delayMs);
53 |
54 |     return () => clearTimeout(timer);
55 |   }, [active, delayMs]);
56 |
57 |   // useSyncExternalStore for tear-free reads
58 |   const subscribe = useCallback((cb: () => void) => {
59 |     listeners.current.add(cb);
60 |     return () => {
61 |       listeners.current.delete(cb);
62 |     };
63 |   }, []);
64 |
65 |   const getSnapshot = useCallback(() => value.current, []);
66 |
67 |   return useSyncExternalStore(subscribe, getSnapshot);
68 | }
69 |
70 | const BadgeAwardModal = ({ badge, isOpen, onClose }: BadgeAwardModalProps) => {
71 |   const prefersReducedMotion = useReducedMotion();
72 |   const confettiFired = useRef(false);
73 |
74 |   const needsReveal = isOpen && badge.isMystery && !prefersReducedMotion;
75 |   const timerRevealed = useDelayedFlag(needsReveal, MYSTERY_REVEAL_DELAY_MS);
76 |
77 |   const isRevealed =
78 |     !badge.isMystery || (prefersReducedMotion ?? false) || timerRevealed;
79 |
80 |   const fireConfetti = useCallback(
81 |     (opts?: { reveal?: boolean }) => {
82 |       if (prefersReducedMotion) return;
83 |
84 |       const isReveal = opts?.reveal ?? false;
85 |       void launchConfetti({
86 |         particleCount: isReveal ? 100 : 60,
87 |         spread: isReveal ? 120 : 80,
88 |         origin: { y: isReveal ? 0.5 : 0.6 },
89 |         colors: isReveal
90 |           ? ["#f59e0b", "#eab308", "#fbbf24", "#a855f7", "#ec4899"]
91 |           : ["#f59e0b", "#eab308", "#fbbf24", "#3b82f6", "#14b8a6"],
92 |       });
93 |     },
94 |     [prefersReducedMotion]
95 |   );
96 |
97 |   // Fire initial confetti when modal opens
```

```

98 | useEffect(() => {
99 |   if (!isOpen) {
100 |     confettiFired.current = false;
101 |     return;
102 |   }
103 |
104 |   if (!confettiFired.current) {
105 |     confettiFired.current = true;
106 |     fireConfetti();
107 |   }
108 | }, [isOpen, fireConfetti]);
109 |
110 | // Fire extra confetti on mystery reveal
111 | useEffect(() => {
112 |   if (isOpen && badge.isMystery && timerRevealed) {
113 |     fireConfetti({ reveal: true });
114 |   }
115 | }, [isOpen, badge.isMystery, timerRevealed, fireConfetti]);
116 |
117 | const displayIcon = isRevealed ? badge.icon : "'S";
118 | const displayName = isRevealed ? badge.name : "Mystery Badge Unlocked!";
119 | const displayDescription = isRevealed
120 |   ? badge.description
121 |   : "Revealing your achievement...";
122 |
123 | return (
124 |   <Dialog open={isOpen} onOpenChange={({open}) => !open && onClose()}>
125 |     <DialogContent>
126 |       showCloseButton={false}
127 |       className="bg-white border-0 shadow-2xl rounded-2xl max-w-sm p-0 overflow-hidden"
128 |     >
129 |       <div className="flex flex-col items-center text-center p-8 gap-4">
130 |         {/* Badge icon */}
131 |         <AnimatePresence mode="wait">
132 |           <motion.div
133 |             key={isRevealed ? "revealed" : "hidden"}
134 |             initial={
135 |               prefersReducedMotion ? undefined : { scale: 0, opacity: 0 }
136 |             }
137 |             animate={
138 |               prefersReducedMotion ? undefined : { scale: 1, opacity: 1 }
139 |             }
140 |             exit={
141 |               prefersReducedMotion ? undefined : { scale: 0.5, opacity: 0 }
142 |             }
143 |             transition={{ type: "spring", stiffness: 300, damping: 15 }}
144 |             className="text-6xl animate-badge-pop"
145 |             aria-hidden="true"
146 |           >
147 |             {displayIcon}
148 |           </motion.div>
149 |         </AnimatePresence>
150 |
151 |         {/* Title */}
152 |         <DialogTitle className="text-xl font-bold tracking-tight">
153 |           <AnimatePresence mode="wait">
154 |             <motion.span
155 |               key={displayName}
156 |               initial={
157 |                 prefersReducedMotion ? undefined : { opacity: 0, y: 8 }
158 |               }
159 |               animate={
160 |                 prefersReducedMotion ? undefined : { opacity: 1, y: 0 }
161 |               }
162 |               exit={prefersReducedMotion ? undefined : { opacity: 0, y: -8 }}
163 |               transition={{ duration: 0.25 }}
164 |             >
165 |               {displayName}
166 |             </motion.span>
167 |           </AnimatePresence>
168 |         </DialogTitle>
169 |
170 |         {/* Description */}
171 |         <DialogDescription className="text-sm text-gray-500">
172 |           <AnimatePresence mode="wait">
173 |             <motion.span
174 |               key={displayDescription}
175 |               initial={prefersReducedMotion ? undefined : { opacity: 0 }}
176 |               animate={prefersReducedMotion ? undefined : { opacity: 1 }}
177 |               exit={prefersReducedMotion ? undefined : { opacity: 0 }}
178 |               transition={{ duration: 0.2, delay: 0.1 }}
179 |             >
180 |               {displayDescription}
181 |             </motion.span>
182 |           </AnimatePresence>
183 |         </DialogDescription>
184 |
185 |         {/* XP reward */}
186 |         <div className="flex items-center gap-1.5 text-amber-500 font-bold text-lg">
187 |           <Sparkles className="h-5 w-5" />
188 |           <span data-testid="xp-reward">+{badge.xpReward} XP</span>
189 |         </div>
190 |
191 |         {/* Dismiss button */}
192 |         <Button
193 |           onClick={onClose}
194 |           variant="tactile"
195 |           className="mt-2 w-full font-semibold"
196 |         >
197 |           Awesome!
198 |         </Button>
199 |       </div>
200 |     </DialogContent>
201 |   </Dialog>

```

FILE: src/components/shared/BadgeCollection.tsx

[illegible]

[illegible]

[illegible]


```

19 |   silver: "border-gray-400",
20 |   gold: "border-yellow-400",
21 | };
22 |
23 | const BadgeSpotlightCard = ({
24 |   category,
25 |   currentTier,
26 |   progress,
27 |   daysRemaining,
28 | }: BadgeSpotlightCardProps) => {
29 |   const progressPercent = useMemo(
30 |     () => Math.min(Math.round(progress * 100), 100),
31 |     [progress]
32 |   );
33 |
34 |   return (
35 |     <Card
36 |       data-testid="badge-spotlight-card"
37 |       className="bg-gradient-to-r from-purple-50 to-indigo-50 border border-purple-200 shadow-md rounded-xl p-
38 |     >
39 |       <div className="flex items-start gap-3">
40 |         <div className="p-2 rounded-lg bg-purple-100">
41 |           <Sparkles className="h-5 w-5 text-purple-600" />
42 |         </div>
43 |         <div className="flex-1 min-w-0">
44 |           <div className="flex items-center gap-2 flex-wrap">
45 |             <p className="text-sm font-bold text-purple-800">Badge Spotlight</p>
46 |             <Badge
47 |               data-testid="spotlight-bonus-label"
48 |               className="text-[10px] bg-amber-100 text-amber-700 border-amber-200"
49 |             >
50 |               2x XP Bonus this week
51 |             </Badge>
52 |           </div>
53 |           <p className="text-xs text-purple-600 mt-0.5 capitalize">
54 |             {category}
55 |           </p>
56 |
57 |           {currentTier && (
58 |             <div
59 |               data-testid="spotlight-current-tier"
60 |               className={ inline-block mt-2 px-2 py-0.5 rounded border-2 ${TIER_BORDER_COLORS[currentTier]} te
61 |             >
62 |               {currentTier}
63 |             </div>
64 |           )}
65 |
66 |           {/* Progress toward next tier */}
67 |           <div className="mt-2" data-testid="spotlight-progress">
68 |             <div className="flex items-center justify-between text-[10px] text-gray-500">
69 |               <span>Progress to next tier</span>
70 |               <span>{progressPercent}%</span>
71 |             </div>
72 |             <div className="h-1.5 rounded-full bg-gray-100 overflow-hidden mt-1">
73 |               <div
74 |                 className="h-full rounded-full bg-purple-500 transition-all duration-300"
75 |                 style={{ width: `${progressPercent}%` }}
76 |               />
77 |             </div>
78 |           </div>
79 |
80 |           {/* Countdown */}
81 |           <p
82 |             className="text-[10px] text-gray-400 mt-1.5"
83 |             data-testid="spotlight-countdown"
84 |           >
85 |             {daysRemaining} day{daysRemaining !== 1 ? "s" : ""} remaining
86 |           </p>
87 |         </div>
88 |       </div>
89 |     </Card>
90 |   );
91 | };
92 |
93 | export default BadgeSpotlightCard;
94 |

```

FILE: src/components/shared/BadgeTierIndicator.tsx

TYPE: .tsx | LINES: 1-72 | SHA256: 5d241b9737f0c83c6888975d1717dd38e3b8a89f9fb57e757084a4e988c9b469

```

1 | // =====
2 | // BadgeTierIndicator - Bronze/Silver/Gold visual indicator on badges
3 | // Task 24.1
4 | // =====
5 |
6 | import { cn } from "@lib/utils";
7 |
8 | export type BadgeTierLevel = "bronze" | "silver" | "gold";
9 |
10 | interface BadgeTierIndicatorProps {
11 |   tier: BadgeTierLevel | null;
12 |   size?: "sm" | "md" | "lg";
13 |   className?: string;
14 | }
15 |
16 | const TIER_STYLES: Record<
17 |   BadgeTierLevel,
18 |   { bg: string; border: string; text: string; label: string }
19 | > = {
20 |   bronze: {
21 |     bg: "bg-amber-600",
22 |     border: "border-amber-600",
23 |     text: "text-white",
24 |     label: "Bronze",

```

```

25 | },
26 | silver: {
27 |   bg: "bg-gray-400",
28 |   border: "border-gray-400",
29 |   text: "text-white",
30 |   label: "Silver",
31 | },
32 | gold: {
33 |   bg: "bg-yellow-400",
34 |   border: "border-yellow-400",
35 |   text: "text-yellow-900",
36 |   label: "Gold",
37 | },
38 | };
39 |
40 | const SIZE_MAP = {
41 |   sm: "text-[8px] px-1 py-0.5",
42 |   md: "text-[10px] px-1.5 py-0.5",
43 |   lg: "text-xs px-2 py-1",
44 | } as const;
45 |
46 | const BadgeTierIndicator = ({
47 |   tier,
48 |   size = "md",
49 |   className,
50 | }: BadgeTierIndicatorProps) => {
51 |   if (!tier) return null;
52 |
53 |   const style = TIER_STYLES[tier];
54 |
55 |   return (
56 |     <span
57 |       className={cn(
58 |         "inline-flex items-center rounded-full font-bold uppercase tracking-wider",
59 |         style.bg,
60 |         style.text,
61 |         SIZE_MAP[size],
62 |         className
63 |       )}
64 |       data-testid={`badge-tier-${tier}`}
65 |     >
66 |       {style.label}
67 |     </span>
68 |   );
69 | };
70 |
71 | export default BadgeTierIndicator;
72 |

```

FILE: [src/components/shared/BestDayChart.tsx](#)

TYPE: .tsx | LINES: 1-87 | SHA256: 760e422cd912a6c5baf8a284b7b522adfcc878c195ed6504eef638cb05a1b4b7

```

1 | import {
2 |   BarChart,
3 |   Bar,
4 |   XAxis,
5 |   YAxis,
6 |   Tooltip,
7 |   ResponsiveContainer,
8 |   Cell,
9 | } from "recharts";
10 | import { getBestDay } from "@lib/heatmapUtils";
11 | import type { DayOfWeekData } from "@types/habits";
12 |
13 | export interface BestDayChartProps {
14 |   data: DayOfWeekData[];
15 | }
16 |
17 | const BestDayChart = ({ data }: BestDayChartProps) => {
18 |   const bestDay = getBestDay(data);
19 |
20 |   if (data.length === 0) {
21 |     return (
22 |       <p
23 |         className="text-sm text-gray-500 text-center py-8"
24 |         data-testid="best-day-chart-empty"
25 |       >
26 |         No data available
27 |       </p>
28 |     );
29 |   }
30 |
31 |   return (
32 |     <div data-testid="best-day-chart">
33 |       {bestDay && (
34 |         <p className="text-sm text-gray-600 mb-3">
35 |           Your best day is{" "}
36 |           <span className="font-bold text-blue-600">{bestDay.day}</span> with an
37 |           average of {bestDay.avgCompletions} habits
38 |         </p>
39 |       )}
40 |       <ResponsiveContainer width="100%" height={200}>
41 |         <BarChart
42 |           data={data}
43 |           layout="vertical"
44 |           margin={{ top: 4, right: 4, bottom: 4, left: 4 }}
45 |         >
46 |           <XAxis
47 |             type="number"
48 |             tick={{ fontSize: 11 }}
49 |             tickLine={false}
50 |             axisLine={false}
51 |           />
52 |           <YAxis

```

```

53 |         type="category"
54 |         dataKey="day"
55 |         tick={{ fontSize: 11 }}
56 |         tickLine={false}
57 |         axisLine={false}
58 |         width={80}
59 |         tickFormatter={(v: string) => v.slice(0, 3)}
60 |     />
61 |     <Tooltip
62 |       formatter={
63 |         ((value: number | undefined) => [
64 |           (value ?? 0).toFixed(2),
65 |           "Avg Habits",
66 |         ]) as never
67 |       }
68 |       contentStyle={{ borderRadius: 8, fontSize: 12 }}
69 |     />
70 |     <Bar dataKey="avgCompletions" radius={[0, 4, 4, 0]} maxBarSize={20}>
71 |       {data.map((entry) => (
72 |         <Cell
73 |           key={entry.day}
74 |           fill={
75 |             bestDay && entry.day === bestDay.day ? "#2563eb" : "#e2e8f0"
76 |           }
77 |         />
78 |       ))}
79 |     </Bar>
80 |   </BarChart>
81 | </ResponsiveContainer>
82 | </div>
83 | );
84 | };
85 |
86 | export default BestDayChart;
87 |

```

FILE: src/components/shared/BloomsPill.tsx

```
TYPE: .tsx | LINES: 1-46 | SHA256: f2e49baa5e91bbe5a3f838eb7270fdeb26e77c458d81aeaf96a05724b9b61a37
```

```

1 // =====
2 // BloomsPill – Badge/pill showing Bloom's taxonomy level with color coding
3 // Remember purple, Understand blue, Apply green, Analyze yellow, Evaluate orange, Create red
4 // =====
5
6 import { cn } from "@lib/utils";
7 import type { BloomsLevel } from "@types/app";
8
9 interface BloomsPillProps {
10   level: BloomsLevel;
11   className?: string;
12 }
13
14 const BLOOMS_PILL_STYLES: Record<
15   BloomsLevel,
16   { bg: string; text: string; label: string }
17 > = {
18   remembering: { bg: "bg-purple-700", text: "text-white", label: "Remember" },
19   understanding: { bg: "bg-blue-700", text: "text-white", label: "Understand" },
20   applying: { bg: "bg-green-700", text: "text-white", label: "Apply" },
21   analyzing: { bg: "bg-yellow-700", text: "text-white", label: "Analyze" },
22   evaluating: { bg: "bg-orange-700", text: "text-white", label: "Evaluate" },
23   creating: { bg: "bg-red-700", text: "text-white", label: "Create" },
24 };
25
26 const BloomsPill = ({ level, className }: BloomsPillProps) => {
27   const style = BLOOMS_PILL_STYLES[level];
28
29   return (
30     <span
31       className={cn(
32         "inline-flex items-center rounded-full px-2.5 py-0.5 text-xs font-bold tracking-wide uppercase",
33         style.bg,
34         style.text,
35         className
36       )}
37     >
38       {style.label}
39     </span>
40   );
41 };
42
43 export default BloomsPill;
44 export { BLOOMS_PILL_STYLES };
45 export type { BloomsPillProps };
46

```

FILE: src/components/shared/BloomsPioneerBadge.tsx

TYPE: .tsx | LINES: 1-126 | SHA256: 8180b1dfd3d05237b1e119202e86cc69e00325ba88708d50101187c7e0334771

[illegible]

[illegible]

```
120 |         </span>
121 |     </div>
122 | );
123 | };
124 |
125 | export default BloomsPioneerBadge;
126 |
```

FILE: src/components/shared/BloomsProgressionLadder.tsx

TYPE: .tsx | LINES: 1-161 | SHA256: 1621d880da6c7ab70d2c1c13dfdccd626d55dc1e5d6ca1cbb0cdab5f0d3bec91

[illegible]

```

94 |         cloTitle ? ` for ${cloTitle}` : ""
95 |     }: highest level reached is ${
96 |         clampedLevel > 0
97 |         ? BLOOMS_LEVELS.find((l) => l.level === clampedLevel)?.label ?? "None"
98 |         : "None"
99 |     }}
100 | >
101 | {cloTitle && (
102 |     <p
103 |         className={cn(
104 |             "font-bold tracking-tight truncate",
105 |             compact ? "text-xs" : "text-sm"
106 |         )}
107 |     >
108 |         {cloTitle}
109 |     </p>
110 | )}
111 |
112 | <div className={cn("flex flex-col", compact ? "gap-0.5" : "gap-1")}>
113 |     {BLOOMS_LEVELS.map((config) => {
114 |         const isReached = clampedLevel >= config.level;
115 |         const isHighest = clampedLevel === config.level;
116 |
117 |         return (
118 |             <motion.div
119 |                 key={config.level}
120 |                 className={cn(
121 |                     "flex items-center rounded-md font-semibold transition-colors",
122 |                     compact ? "px-2 py-0.5 text-[10px]" : "px-3 py-1.5 text-xs",
123 |                     isReached
124 |                     ? cn(config.bg, config.text)
125 |                     : cn(config.dimBg, config.dimText),
126 |                     isHighest && "ring-2 ring-offset-1 ring-gray-900/20"
127 |                 )}
128 |                 initial={prefersReducedMotion ? false : { opacity: 0, y: 8 }}
129 |                 animate={{ opacity: 1, y: 0 }}
130 |                 transition={{
131 |                     duration: prefersReducedMotion ? 0 : 0.3,
132 |                     delay: prefersReducedMotion ? 0 : (6 - config.level) * 0.05,
133 |                     ease: "easeOut",
134 |                 }}
135 |             >
136 |                 <span
137 |                     className={cn("font-bold", compact ? "w-3 me-1" : "w-4 me-2")}
138 |                 >
139 |                     {config.level}
140 |                 </span>
141 |                 <span className="truncate">{config.label}</span>
142 |                 {isHighest && clampedLevel > 0 && (
143 |                     <span
144 |                         className={cn(
145 |                             "ml-auto",
146 |                             compact ? "text-[9px]" : "text-[10px]"
147 |                         )}
148 |                     >
149 |                         &
150 |                     </span>
151 |                 )}
152 |             </motion.div>
153 |         );
154 |     })}
155 | </div>
156 | </div>
157 | );
158 | };
159 |
160 | export default BloomsProgressionLadder;
161 |

```

FILE: src/components/shared/BloomsVerbGuide.tsx

TYPE: .tsx | LINES: 1-76 | SHA256: ba5c55711eef2b3d74698a0521e4b1bf72468202ad2e9108054c02b183915e0a

```

1 | import { AnimatePresence, motion } from "framer-motion";
2 | import { Info } from "lucide-react";
3 | import { Badge } from "@components/ui/badge";
4 | import { BLOOMS_VERBS, BLOOMS_COLORS } from "@lib/bloomsVerbs";
5 | import { BLOOMS_ARABIC_STANDARD } from "@lib/bloomsArabicTerminology";
6 | import type { BloomsLevel } from "@lib/schemas/clo";
7 | import { cn } from "@lib/utills";
8 | import { useTranslation } from "react-i18next";
9 |
10 | interface BloomsVerbGuideProps {
11 |     selectedLevel?: BloomsLevel;
12 |     onVerbClick: (verb: string) => void;
13 | }
14 |
15 | const BloomsVerbGuide = ({
16 |     selectedLevel,
17 |     onVerbClick,
18 | }: BloomsVerbGuideProps) => {
19 |     const { i18n } = useTranslation();
20 |
21 |     if (!selectedLevel) return null;
22 |
23 |     const isArabic = i18n.language === "ar";
24 |     const verbs = isArabic
25 |         ? BLOOMS_ARABIC_STANDARD[selectedLevel]?.verbs ??
26 |           BLOOMS_VERBS[selectedLevel]
27 |         : BLOOMS_VERBS[selectedLevel];
28 |     const colors = BLOOMS_COLORS[selectedLevel];
29 |
30 |     return (
31 |         <AnimatePresence mode="wait">
32 |             <motion.div

```

```

33 |         key={selectedLevel}
34 |         initial={{ opacity: 0, y: -4 }}
35 |         animate={{ opacity: 1, y: 0 }}
36 |         exit={{ opacity: 0, y: -4 }}
37 |         transition={{ duration: 0.15 }}
38 |         className="rounded-lg border border-slate-200 bg-slate-50 p-3"
39 |     >
40 |         <div className="flex items-center gap-1.5 mb-2">
41 |             <Info className="h-3.5 w-3.5 text-gray-400" />
42 |             <span className="text-xs font-medium text-gray-500">
43 |                 Suggested Verbs
44 |             </span>
45 |         </div>
46 |         <div className="flex flex-wrap gap-1.5">
47 |             {verbs.map((verb) => (
48 |                 <Badge
49 |                     key={verb}
50 |                     className={cn(
51 |                         colors.bg,
52 |                         colors.text,
53 |                         colors.hover,
54 |                         "cursor-pointer select-none border-0 transition-colors"
55 |                     )}
56 |                     onClick={() => onVerbClick(verb)}
57 |                     role="button"
58 |                     tabIndex={0}
59 |                     onKeyDown={(e) => {
60 |                         if (e.key === "Enter" || e.key === " ") {
61 |                             e.preventDefault();
62 |                             onVerbClick(verb);
63 |                         }
64 |                     }}
65 |                 >
66 |                     {verb}
67 |                 </Badge>
68 |             ))}
69 |         </div>
70 |     </motion.div>
71 | </AnimatePresence>
72 | );
73 | };
74 |
75 | export default BloomsVerbGuide;
76 |

```

FILE: [src/components/shared/BonusEventBanner.tsx](#)

TYPE: .tsx | LINES: 1-93 | SHA256: a8724b7031336e69c6ecf48240c9efdf1d299178dabdb96b37da992580b877c4

```

1 | import { useState, useEffect } from "react";
2 | import { motion, useReducedMotion } from "framer-motion";
3 | import { Sparkles, Clock } from "lucide-react";
4 | import { Badge } from "@components/ui/badge";
5 | import { useActiveBonusEvent } from "@hooks/useBonusEvents";
6 |
7 | const useCountdown = (endDate: string): string => {
8 |     const [timeLeft, setTimeLeft] = useState("");
9 |
10 |     useEffect(() => {
11 |         const update = () => {
12 |             const diff = new Date(endDate).getTime() - Date.now();
13 |             if (diff <= 0) {
14 |                 setTimeLeft("Ended");
15 |                 return;
16 |             }
17 |             const hours = Math.floor(diff / 3600000);
18 |             const minutes = Math.floor((diff % 3600000) / 60000);
19 |             const seconds = Math.floor((diff % 60000) / 1000);
20 |             setTimeLeft(
21 |                 `${hours.toString().padStart(2, "0")}:${minutes
22 |                     .toString()
23 |                     .padStart(2, "0")}:${seconds.toString().padStart(2, "0")}`
24 |             );
25 |         };
26 |         update();
27 |         const interval = setInterval(update, 1000);
28 |         return () => clearInterval(interval);
29 |     }, [endDate]);
30 |
31 |     return timeLeft;
32 | };
33 |
34 | const BonusEventBanner = () => {
35 |     const { data: event } = useActiveBonusEvent();
36 |     const prefersReducedMotion = useReducedMotion();
37 |
38 |     if (!event) return null;
39 |
40 |     return (
41 |         <BonusEventBannerContent
42 |             title={event.name}
43 |             multiplier={event.xp_multiplier}
44 |             endsAt={event.ends_at ?? ""}
45 |             prefersReducedMotion={prefersReducedMotion ?? false}
46 |         />
47 |     );
48 | };
49 |
50 | interface BonusEventBannerContentProps {
51 |     title: string;
52 |     multiplier: number;
53 |     endsAt: string;
54 |     prefersReducedMotion: boolean;
55 | }
56 |

```



```

57 | const BonusEventBannerContent = ({
58 |   title,
59 |   multiplier,
60 |   endsAt,
61 |   prefersReducedMotion,
62 | }: BonusEventBannerContentProps) => {
63 |   const timeLeft = useCountdown(endsAt);
64 |
65 |   const motionProps = prefersReducedMotion
66 |     ? {}
67 |     : {
68 |       initial: { opacity: 0, y: -10 },
69 |       animate: { opacity: 1, y: 0 },
70 |       transition: { duration: 0.3 },
71 |     };
72 |
73 |   return (
74 |     <motion.div {...motionProps}>
75 |       <div className="bg-gradient-to-r from-amber-500 to-orange-500 text-white rounded-xl p-4 flex items-center gap-3">
76 |         <div className="flex items-center gap-3">
77 |           <Sparkles className="h-5 w-5 shrink-0" />
78 |           <span className="font-semibold text-sm">{title}</span>
79 |           <Badge className="bg-white/20 text-white border-transparent hover:bg-white/30">
80 |             {multiplier}x XP
81 |           </Badge>
82 |         </div>
83 |         <div className="flex items-center gap-2">
84 |           <Clock className="h-4 w-4 shrink-0" />
85 |           <span className="font-mono font-bold text-sm">{timeLeft}</span>
86 |         </div>
87 |       </div>
88 |     </motion.div>
89 |   );
90 | };
91 |
92 | export default BonusEventBanner;
93 |

```

FILE: [src/components/shared/BonusQuestionPopup.tsx](#)

TYPE: .tsx | LINES: 1-159 | SHA256: f300956f6df35e26e2557d741981191fbfbfb324baefe3492593b27790d899b3

```

1 | // =====
2 | // BonusQuestionPopup – Random bonus question modal with 30s timer
3 | // Task 21.4
4 | // =====
5 |
6 | import { useState, useEffect, useCallback } from "react";
7 | import {
8 |   Dialog,
9 |   DialogContent,
10 |  DialogHeader,
11 |  DialogTitle,
12 | } from "@components/ui/dialog";
13 | import { Button } from "@components/ui/button";
14 | import { Textarea } from "@components/ui/textarea";
15 | import { Loader2, Sparkles, Clock } from "lucide-react";
16 | import { useSubmitBonusAnswer } from "@hooks/useBonusQuestion";
17 |
18 | interface BonusQuestionPopupProps {
19 |   open: boolean;
20 |   onClose: () => void;
21 |   question: {
22 |     text: string;
23 |     type: string;
24 |     clo_id: string | null;
25 |     time_limit_seconds: number;
26 |     xp_reward: number;
27 |   };
28 |   studentId: string;
29 |   institutionId: string;
30 | }
31 |
32 | const BonusQuestionPopup = ({
33 |   open,
34 |   onClose,
35 |   question,
36 |   studentId,
37 |   institutionId,
38 | }: BonusQuestionPopupProps) => {
39 |   const [answer, setAnswer] = useState("");
40 |   const [timeLeft, setTimeLeft] = useState(question.time_limit_seconds);
41 |   const [result, setResult] = useState<{
42 |     correct: boolean;
43 |     xp_awarded: number;
44 |     feedback: string;
45 |   } | null>(null);
46 |   const submitAnswer = useSubmitBonusAnswer();
47 |
48 |   // Reset state when dialog opens/closes
49 |   const prevOpen = open;
50 |   useEffect(() => {
51 |     if (!prevOpen) return;
52 |
53 |     const timer = setInterval(() => {
54 |       setTimeLeft((prev) => {
55 |         if (prev <= 1) {
56 |           clearInterval(timer);
57 |           return 0;
58 |         }
59 |         return prev - 1;
60 |       });
61 |     }, 1000);
62 |
63 |     return () => clearInterval(timer);

```

```

64 | }, [prevOpen, question.time_limit_seconds]);
65 |
66 | const handleSubmit = useCallback(() => {
67 |   submitAnswer.mutate(
68 |     {
69 |       studentId,
70 |       institutionId,
71 |       answer,
72 |       cloId: question.clo_id,
73 |     },
74 |     {
75 |       onSuccess: (data) => setResult(data),
76 |     }
77 |   );
78 | }, [answer, studentId, institutionId, question.clo_id, submitAnswer]);
79 |
80 | const timerColor = timeLeft <= 10 ? "text-red-500" : "text-gray-500";
81 |
82 | return (
83 |   <Dialog open={open} onOpenChange={({isOpen} => !isOpen && onClose())>
84 |     <DialogContent className="sm:max-w-md">
85 |       <DialogHeader>
86 |         <DialogTitle className="flex items-center gap-2">
87 |           <Sparkles className="h-5 w-5 text-amber-500" />
88 |           Bonus Question!
89 |         </DialogTitle>
90 |       </DialogHeader>
91 |
92 |       {!result ? (
93 |         <div className="space-y-4">
94 |           <div className="flex items-center justify-between">
95 |             <span className="text-xs font-bold text-amber-600">
96 |               +{question.xp_reward} XP reward
97 |             </span>
98 |             <div
99 |               className={`flex items-center gap-1 text-sm font-mono ${timerColor}`}
100 |             >
101 |               <Clock className="h-4 w-4" />
102 |               {timeLeft}s
103 |             </div>
104 |           </div>
105 |
106 |           <p className="text-sm">{question.text}</p>
107 |
108 |           <Textarea
109 |             placeholder="Type your answer..."
110 |             value={answer}
111 |             onChange={(e) => setAnswer(e.target.value)}
112 |             rows={3}
113 |             disabled={timeLeft === 0}
114 |           />
115 |
116 |           <div className="flex justify-end gap-2">
117 |             <Button variant="outline" size="sm" onClick={onClose}>
118 |               Skip
119 |             </Button>
120 |             <Button
121 |               size="sm"
122 |               disabled={
123 |                 !answer.trim() || timeLeft === 0 || submitAnswer.isPending
124 |               }
125 |               onClick={handleSubmit}
126 |               variant="tactile"
127 |             >
128 |               {submitAnswer.isPending && (
129 |                 <Loader2 className="h-3 w-3 animate-spin" />
130 |               )}
131 |               Submit
132 |             </Button>
133 |           </div>
134 |
135 |           {timeLeft === 0 && (
136 |             <p className="text-xs text-red-500 text-center">Time's up!</p>
137 |           )}
138 |         </div>
139 |       ) : (
140 |         <div className="text-center space-y-3 py-4">
141 |           <div className={`text-4xl ${result.correct ? "" : "opacity-50"} `}>
142 |             {result.correct ? "0<B%>" : "0=p "}
143 |           </div>
144 |           <p className="text-sm font-bold">
145 |             {result.correct ? `+${result.xp_awaited} XP!` : "No XP this time"}
146 |           </p>
147 |           <p className="text-xs text-gray-500">{result.feedback}</p>
148 |           <Button size="sm" onClick={onClose}>
149 |             Continue
150 |           </Button>
151 |         </div>
152 |       )}
153 |     </DialogContent>
154 |   </Dialog>
155 | );
156 | };
157 |
158 | export default BonusQuestionPopup;
159 |

```

FILE: src/components/shared/CalendarEventCard.tsx

TYPE: .tsx | LINES: 1-76 | SHA256: 3ba91b100265a959b72710157a67cda480b4a5be838069a4bdf38e9a6783282

```
1  | // =====
2  | // CalendarEventCard – Calendar event display card
3  | // =====
4  |
5  | import { Calendar } from "lucide-react";
6  | import { cn } from "@lib/utils";
7  |
8  | type EventType = "assignment" | "quiz" | "exam" | "class" | "holiday" | "other";
9  |
10 | interface CalendarEventCardProps {
11 |   title: string;
12 |   date: string;
13 |   time?: string;
14 |   eventType: EventType;
15 |   courseName?: string;
16 |   className?: string;
17 | }
18 |
19 | const EVENT_STYLES: Record<
20 |   EventType,
21 |   { bg: string; text: string; dot: string }
22 | > = {
23 |   assignment: { bg: "bg-blue-50", text: "text-blue-700", dot: "bg-blue-500" },
24 |   quiz: { bg: "bg-purple-50", text: "text-purple-700", dot: "bg-purple-500" },
25 |   exam: { bg: "bg-red-50", text: "text-red-700", dot: "bg-red-500" },
26 |   class: { bg: "bg-green-50", text: "text-green-700", dot: "bg-green-500" },
27 |   holiday: { bg: "bg-amber-50", text: "text-amber-700", dot: "bg-amber-500" },
28 |   other: { bg: "bg-gray-50", text: "text-gray-700", dot: "bg-gray-500" },
29 | };
30 |
31 | const CalendarEventCard = ({
32 |   title,
33 |   date,
34 |   time,
35 |   eventType,
36 |   courseName,
37 |   className,
38 | }: CalendarEventCardProps) => {
39 |   const style = EVENT_STYLES[eventType];
40 |
41 |   return (
42 |     <div
43 |       className={cn(
44 |         "flex items-start gap-3 rounded-xl border border-slate-200 bg-white p-3",
45 |         className
46 |       )}
47 |     >
48 |       <div
49 |         className={cn("mt-0.5 h-2.5 w-2.5 rounded-full shrink-0", style.dot)}
50 |       />
51 |       <div className="flex-1 min-w-0">
52 |         <p className="text-sm font-medium truncate">{title}</p>
53 |         {courseName && <p className="text-xs text-gray-500">{courseName}</p>}
54 |         <div className="flex items-center gap-1 mt-1 text-xs text-gray-400">
55 |           <Calendar className="h-3 w-3" />
56 |           <span>{date}</span>
57 |           {time && <span>· {time}</span>}
58 |         </div>
59 |       </div>
60 |       <span
61 |         className={cn(
62 |           "text-[10px] font-bold tracking-widest uppercase px-1.5 py-0.5 rounded",
63 |           style.bg,
64 |           style.text
65 |         )}
66 |       >
67 |         {eventType}
68 |       </span>
69 |     </div>
70 |   );
71 | };
72 |
73 | export default CalendarEventCard;
74 | export { EVENT_STYLES };
75 | export type { CalendarEventCardProps, EventType };
76 |
```

FILE: src/components/shared/CaptchaChallenge.tsx

TYPE: .tsx | LINES: 1-185 | SHA256: fa92c5566d29d5c6738824ad0dacae1595b53e44979721d0935a7d342c5b4ed1

```
1  | /**
2  |  * CaptchaChallenge – hCaptcha/Turnstile integration (ADR-16)
3  |  *
4  |  * Renders conditionally on /signup and /accept-invite when the caller's IP
5  |  * has logged "e 3 signup events in the last hour.
6  |  *
7  |  * The captcha token is passed to the Supabase auth call via the `options.captchaToken`
8  |  * field. Supabase natively supports hCaptcha and Turnstile verification.
9  |  *
10 |  * Design: ADR-16
11 |  * Requirements: 2.11
12 |  *
13 |  * @example
14 |  * <CaptchaChallenge onVerify={({token}) => setCaptchaToken(token)} />
15 |  */
16 |
17 | import { useEffect, useRef, useState } from "react";
18 | import { useTranslation } from "react-i18next";
19 | import { Shield } from "lucide-react";
20 |
```

```

21 | export interface CaptchaChallengeProps {
22 |   /** Called when the captcha is successfully verified with the token */
23 |   onVerify: (token: string) => void;
24 |   /** Called when the captcha expires */
25 |   onExpire?: () => void;
26 |   /** Called when the captcha encounters an error */
27 |   onError?: (error: Error) => void;
28 | }
29 |
30 | // Detect which captcha provider is configured
31 | const CAPTCHA_SITE_KEY = import.meta.env.VITE_CAPTCHA_SITE_KEY as
32 |   | string
33 |   | undefined;
34 | const CAPTCHA_PROVIDER = import.meta.env.VITE_CAPTCHA_PROVIDER as
35 |   | "hcaptcha"
36 |   | "turnstile"
37 |   | undefined;
38 |
39 | /**
40 |  * CaptchaChallenge renders an hCaptcha or Cloudflare Turnstile widget.
41 |  *
42 |  * If no VITE_CAPTCHA_SITE_KEY is configured, renders a development-mode
43 |  * bypass button so the form remains usable in local development.
44 |  */
45 | const CaptchaChallenge = ({
46 |   onVerify,
47 |   onExpire,
48 |   onError,
49 | }: CaptchaChallengeProps) => {
50 |   const { t } = useTranslation("common");
51 |   const containerRef = useRef<HTMLDivElement>(null);
52 |   const [isLoading, setIsLoaded] = useState(false);
53 |   const isDev = !CAPTCHA_SITE_KEY;
54 |
55 |   useEffect(() => {
56 |     if (isDev || !CAPTCHA_SITE_KEY) return;
57 |
58 |     const provider = CAPTCHA_PROVIDER ?? "hcaptcha";
59 |     // setIsLoaded is stable (React guarantees setState identity is stable)
60 |     const onLoad = () => {
61 |       setIsLoaded(true);
62 |     };
63 |
64 |     if (provider === "hcaptcha") {
65 |       // Load hCaptcha script
66 |       const scriptId = "hcaptcha-script";
67 |       if (!document.getElementById(scriptId)) {
68 |         const script = document.createElement("script");
69 |         script.id = scriptId;
70 |         script.src = "https://js.hcaptcha.com/1/api.js";
71 |         script.async = true;
72 |         script.defer = true;
73 |         script.onload = onLoad;
74 |         document.head.appendChild(script);
75 |       } else {
76 |         onLoad();
77 |       }
78 |     } else if (provider === "turnstile") {
79 |       // Load Cloudflare Turnstile script
80 |       const scriptId = "turnstile-script";
81 |       if (!document.getElementById(scriptId)) {
82 |         const script = document.createElement("script");
83 |         script.id = scriptId;
84 |         script.src = "https://challenges.cloudflare.com/turnstile/v0/api.js";
85 |         script.async = true;
86 |         script.defer = true;
87 |         script.onload = onLoad;
88 |         document.head.appendChild(script);
89 |       } else {
90 |         onLoad();
91 |       }
92 |     }
93 |   }, [isDev]);
94 |
95 |   useEffect(() => {
96 |     if (!isLoading || !CAPTCHA_SITE_KEY || !containerRef.current) return;
97 |
98 |     const provider = CAPTCHA_PROVIDER ?? "hcaptcha";
99 |     const container = containerRef.current;
100 |
101 |     try {
102 |       if (provider === "hcaptcha" && typeof window.hcaptcha !== "undefined") {
103 |         window.hcaptcha.render(container, {
104 |           sitekey: CAPTCHA_SITE_KEY,
105 |           callback: onVerify,
106 |           "expired-callback": onExpire,
107 |           "error-callback": () => onError?.(new Error("hCaptcha error")),
108 |           theme: "light",
109 |           size: "normal",
110 |         });
111 |       } else if (
112 |         provider === "turnstile" &&
113 |         typeof window.turnstile !== "undefined"
114 |       ) {
115 |         window.turnstile.render(container, {
116 |           sitekey: CAPTCHA_SITE_KEY,
117 |           callback: onVerify,
118 |           "expired-callback": onExpire,
119 |           "error-callback": () => onError?.(new Error("Turnstile error")),
120 |           theme: "light",
121 |           size: "normal",
122 |         });
123 |       }
124 |     } catch (err) {

```

```

125         onError?.(
126             err instanceof Error ? err : new Error("Captcha render failed")
127         );
128     }
129 }, [isLoading, onVerify, onExpire, onError]);
130
131 // Development bypass – renders a button that simulates captcha verification
132 if (isDev) {
133     return (
134         <div className="flex items-center gap-3 p-3 rounded-lg border border-amber-200 bg-amber-50">
135             <Shield className="h-4 w-4 text-amber-600 shrink-0" />
136             <div className="flex-1">
137                 <p className="text-xs font-medium text-amber-800">
138                     {t(
139                         "captcha.devMode",
140                         "Captcha (dev mode – no VITE_CAPTCHA_SITE_KEY set)"
141                     )}
142                 </p>
143             </div>
144             <button
145                 type="button"
146                 onClick={() => onVerify("dev-bypass-token")}
147                 className="text-xs font-semibold text-amber-700 hover:text-amber-900 underline"
148             >
149                 {t("captcha.bypass", "Bypass")}
150             </button>
151         </div>
152     );
153 }
154
155 return (
156     <div className="flex justify-center">
157         <div ref={containerRef} />
158     </div>
159 );
160 };
161
162 export default CaptchaChallenge;
163
164 // Extend Window type for captcha providers
165 declare global {
166     interface Window {
167         hcaptcha?: {
168             render: (
169                 container: HTMLElement,
170                 options: Record<string, unknown>
171             ) => string;
172             reset: (widgetId?: string) => void;
173             getResponse: (widgetId?: string) => string;
174         };
175         turnstile?: {
176             render: (
177                 container: HTMLElement,
178                 options: Record<string, unknown>
179             ) => string;
180             reset: (widgetId?: string) => void;
181             getResponse: (widgetId?: string) => string;
182         };
183     }
184 }
185

```

FILE: src/components/shared/CellDetailSheet.tsx

```
TYPE: .tsx | LINES: 1-113 | SHA256: 13bcb56e846cc53d15a9e7112927805006722253a212cabcd4e5240ef7cdd5cb
```

[illegible]

```

40 |   onOpenChange,
41 | }: CellDetailSheetProps) => {
42 |   const { data, isLoading } = useCellDetail(
43 |     open ? ploId : undefined,
44 |     open ? courseId : undefined
45 |   );
46 |
47 |   return (
48 |     <Sheet open={open} onOpenChange={onOpenChange}>
49 |       <SheetContent side="right" className="overflow-y-auto">
50 |         <SheetHeader>
51 |           {isLoading ? (
52 |             <>
53 |               <Shimmer className="h-5 w-48" />
54 |               <Shimmer className="h-4 w-32" />
55 |             </>
56 |           ) : data ? (
57 |             <>
58 |               <SheetTitle className="flex items-center gap-2">
59 |                 <Layers className="h-4 w-4 text-blue-600" />
60 |                 {data.plo.title}
61 |               </SheetTitle>
62 |               <SheetDescription className="flex items-center gap-1.5">
63 |                 <BookOpen className="h-3.5 w-3.5" />
64 |                 {data.course.code} – {data.course.name}
65 |               </SheetDescription>
66 |             </>
67 |           ) : null}
68 |         </SheetHeader>
69 |
70 |         <div className="px-4 pb-4 space-y-3">
71 |           {isLoading ? (
72 |             <div className="space-y-3">
73 |               <Shimmer className="h-16 w-full rounded-lg" />
74 |               <Shimmer className="h-16 w-full rounded-lg" />
75 |             </div>
76 |           ) : data && data.clos.length > 0 ? (
77 |             <>
78 |               <p className="text-xs font-bold tracking-widest uppercase text-gray-500">
79 |                 Mapped CLOs ({data.clos.length})
80 |               </p>
81 |               <ul className="space-y-2">
82 |                 {data.clos.map((clo) => (
83 |                   <li
84 |                     key={clo.id}
85 |                     className="rounded-lg border border-slate-200 bg-white p-3 space-y-1.5"
86 |                   >
87 |                     <p className="text-sm font-medium text-gray-800">
88 |                       {clo.title}
89 |                     </p>
90 |                     <div className="flex items-center gap-2">
91 |                       {clo.blooms_level && (
92 |                         <Badge className={bloomsStyles[clo.blooms_level]}>
93 |                           {clo.blooms_level}
94 |                         </Badge>
95 |                       )}
96 |                     </div>
97 |                   </li>
98 |                 )}
99 |               </ul>
100 |             </>
101 |           ) : data ? (
102 |             <div className="rounded-lg border border-dashed border-slate-300 bg-slate-50 p-6 text-center text-
103 |               No CLOs mapped to this PLO in this course.
104 |             </div>
105 |           ) : null}
106 |         </div>
107 |       </SheetContent>
108 |     </Sheet>
109 |   );
110 | };
111 |
112 | export default CellDetailSheet;
113 |

```

FILE: [src/components/shared/ChallengeLeaderboard.tsx](#)

TYPE: .tsx | LINES: 1-152 | SHA256: aa110e8a417e27f8a57c0a3f5cd68201b965e9654765143dbccccc3e160f5474

```

1 | // =====
2 | // ChallengeLeaderboard – Per-challenge participant ranking
3 | // Task 4.5: medals, anonymous support, current user highlight
4 | // =====
5 |
6 | import { cn } from "@lib/utils";
7 | import { Badge } from "@components/ui/badge";
8 | import { Trophy, EyeOff } from "lucide-react";
9 |
10 | export interface LeaderboardParticipant {
11 |   participantId: string;
12 |   displayName: string;
13 |   currentProgress: number;
14 |   goalTarget: number;
15 |   completedAt: string | null;
16 |   rank: number;
17 |   isAnonymous?: boolean;
18 | }
19 |
20 | export interface ChallengeLeaderboardProps {
21 |   participants: LeaderboardParticipant[];
22 |   currentUserId?: string;
23 |   goalTarget: number;
24 |   className?: string;
25 | }
26 |

```

```

27 | const MEDAL_STYLES: Record<number, { icon: string; bg: string; text: string }> =
28 | {
29 |   1: { icon: "0>Yg", bg: "bg-yellow-50", text: "text-yellow-700" },
30 |   2: { icon: "0>YH", bg: "bg-gray-50", text: "text-gray-600" },
31 |   3: { icon: "0>YI", bg: "bg-amber-50", text: "text-amber-700" },
32 | };
33 |
34 | const ChallengeLeaderboard = ({
35 |   participants,
36 |   currentUserId,
37 |   goalTarget,
38 |   className,
39 | }: ChallengeLeaderboardProps) => {
40 |   if (participants.length === 0) {
41 |     return (
42 |       <div
43 |         className={cn(
44 |           "flex flex-col items-center justify-center py-8 text-center",
45 |           className
46 |         )}
47 |       >
48 |         <Trophy className="h-8 w-8 text-gray-300 mb-2" />
49 |         <p className="text-sm text-gray-400">No participants yet</p>
50 |       </div>
51 |     );
52 |   }
53 |
54 |   return (
55 |     <div
56 |       className={cn("space-y-1", className)}
57 |       role="list"
58 |       aria-label="Challenge leaderboard"
59 |     >
60 |       {participants.map((p) => {
61 |         const isCurrentUser = p.participantId === currentUserId;
62 |         const medal = MEDAL_STYLES[p.rank];
63 |         const pct =
64 |           goalTarget > 0
65 |             ? Math.round((p.currentProgress / goalTarget) * 100)
66 |             : 0;
67 |         const isComplete = p.completedAt !== null;
68 |
69 |         return (
70 |           <div
71 |             key={p.participantId}
72 |             role="listitem"
73 |             className={cn(
74 |               "flex items-center gap-3 rounded-lg px-3 py-2 transition-colors",
75 |               isCurrentUser && "bg-blue-50 ring-1 ring-blue-200",
76 |               !isCurrentUser && "hover:bg-slate-50"
77 |             )}
78 |           >
79 |             { /* Rank */ }
80 |             <div
81 |               className={cn(
82 |                 "h-8 w-8 rounded-full flex items-center justify-center shrink-0 text-sm font-bold",
83 |                 medal
84 |                 ? ` ${medal.bg} ${medal.text}`
85 |                 : "bg-gray-100 text-gray-500"
86 |               )}
87 |             >
88 |               {medal ? (
89 |                 <span aria-label={`Rank ${p.rank}`}>{medal.icon}</span>
90 |               ) : (
91 |                 <span>{p.rank}</span>
92 |               )}
93 |             </div>
94 |
95 |             { /* Name */ }
96 |             <div className="flex-1 min-w-0">
97 |               <div className="flex items-center gap-1.5">
98 |                 {p.isAnonymous ? (
99 |                   <div>
100 |                     <EyeOff className="h-3.5 w-3.5 text-gray-400" />
101 |                     <span className="text-sm font-medium text-gray-400 italic">
102 |                       Anonymous
103 |                     </span>
104 |                   </div>
105 |                 ) : (
106 |                   <span
107 |                     className={cn(
108 |                       "text-sm font-medium truncate",
109 |                       isCurrentUser && "font-bold text-blue-700"
110 |                     )}
111 |                   >
112 |                     {p.displayName}
113 |                     {isCurrentUser && (
114 |                       <span className="text-xs font-normal text-blue-500 ms-1">
115 |                         (You)
116 |                       </span>
117 |                     )}
118 |                   </span>
119 |                 )}
120 |               </div>
121 |               { /* Mini progress bar */ }
122 |               <div className="flex items-center gap-2 mt-0.5">
123 |                 <div className="flex-1 h-1.5 rounded-full bg-gray-100 overflow-hidden">
124 |                   <div
125 |                     className={cn(
126 |                       "h-full rounded-full transition-all duration-300",
127 |                       isComplete ? "bg-green-500" : "bg-blue-400"
128 |                     )}
129 |                     style={{ width: `${Math.min(pct, 100)}%` }}
130 |                   </div>

```

```

131 |         </div>
132 |         <span className="text-[10px] font-medium text-gray-500 tabular-nums">
133 |             {pct}%
134 |         </span>
135 |     </div>
136 | </div>
137 |
138 |     { /* Completion badge */ }
139 |     { isComplete && (
140 |         <Badge className="bg-green-100 text-green-700 border-green-200 text-[10px]">
141 |             Done
142 |         </Badge>
143 |     ) }
144 | </div>
145 |     );
146 |   });
147 | </div>
148 | );
149 | };
150 |
151 | export default ChallengeLeaderboard;
152 |

```

FILE: [src/components/shared/ChallengeProgressBar.tsx](#)

TYPE: .tsx | LINES: 1-85 | SHA256: e4d7fdec69dba66540c6fd34bd638670d5c595a5c1af939a78be3014a0132b7f

```

1 | // =====
2 | // ChallengeProgressBar - Accessible progress bar with ARIA attributes
3 | // Task 4.4: ARIA progressbar role, aria-valuenow/min/max
4 | // =====
5 |
6 | import { cn } from "@lib/utils";
7 |
8 | export interface ChallengeProgressBarProps {
9 |   current: number;
10 |   goal: number;
11 |   label?: string;
12 |   showPercentage?: boolean;
13 |   showValues?: boolean;
14 |   colorClass?: string;
15 |   className?: string;
16 | }
17 |
18 | const ChallengeProgressBar = ({
19 |   current,
20 |   goal,
21 |   label = "Challenge progress",
22 |   showPercentage = true,
23 |   showValues = true,
24 |   colorClass,
25 |   className,
26 | }: ChallengeProgressBarProps) => {
27 |   const safeGoal = Math.max(goal, 1);
28 |   const clampedCurrent = Math.max(0, Math.min(current, safeGoal));
29 |   const percentage = Math.round((clampedCurrent / safeGoal) * 100);
30 |   const isComplete = clampedCurrent >= safeGoal;
31 |
32 |   const barColor =
33 |     colorClass ??
34 |     (isComplete
35 |       ? "bg-green-500"
36 |       : percentage >= 75
37 |       ? "bg-blue-500"
38 |       : percentage >= 50
39 |       ? "bg-amber-500"
40 |       : "bg-blue-400");
41 |
42 |   return (
43 |     <div className={cn("space-y-1.5", className)}>
44 |       { /* Header row */ }
45 |       { (showPercentage || showValues) && (
46 |         <div className="flex items-center justify-between">
47 |           {showValues && (
48 |             <span className="text-xs font-medium text-gray-600">
49 |               {clampedCurrent.toLocaleString()} / {safeGoal.toLocaleString()}
50 |             </span>
51 |           ) }
52 |           {showPercentage && (
53 |             <span
54 |               className={cn(
55 |                 "text-xs font-bold tabular-nums",
56 |                 isComplete ? "text-green-600" : "text-gray-600"
57 |               )}
58 |             >
59 |               {percentage}%
60 |             </span>
61 |           ) }
62 |         </div>
63 |       ) }
64 |
65 |       { /* Progress bar */ }
66 |       <div className="h-3 rounded-full bg-gray-100 overflow-hidden">
67 |         <div
68 |           role="progressbar"
69 |           aria-valuenow={clampedCurrent}
70 |           aria-valuemin={0}
71 |           aria-valuemax={safeGoal}
72 |           aria-label={label}
73 |           className={cn(
74 |             "h-full rounded-full transition-all duration-500 ease-out",
75 |             barColor
76 |           )}
77 |           style={{ width: `${percentage}%` }}
78 |         />

```



```

79 |         </div>
80 |     </div>
81 | );
82 | };
83 |
84 | export default ChallengeProgressBar;
85 |

```

FILE: src/components/shared/ChatMessage.tsx

TYPE: .tsx | LINES: 1-325 | SHA256: 75d6d9835e9ebd51c0318ade3257e5514f6687e0af629fec82f5e0bec67e0d5c

```

1 | // =====
2 | // ChatMessage – Message bubble with markdown, citations, and rating
3 | // =====
4 |
5 | import { useState, useCallback } from "react";
6 | import { Bot, User } from "lucide-react";
7 | import { useTranslation } from "react-i18next";
8 | import { Badge } from "@components/ui/badge";
9 | import { cn } from "@lib/Utils";
10 | import SatisfactionRating from "@components/shared/SatisfactionRating";
11 | import SourceCitationPanel from "@components/shared/SourceCitationPanel";
12 | import type {
13 |   TutorMessage,
14 |   SatisfactionRating as SatisfactionRatingType,
15 | } from "@lib/tutorSchemas";
16 |
17 | interface ChatMessageProps {
18 |   message: TutorMessage;
19 |   onRate?: (variables: {
20 |     messageId: string;
21 |     conversationId: string;
22 |     rating: SatisfactionRatingType;
23 |   }) => void;
24 |   isRatingPending?: boolean;
25 | }
26 |
27 | type TFN = (key: string, options?: Record<string, unknown>) => string;
28 |
29 | /** Parse markdown-like content into simple rendered elements */
30 | const renderContent = (
31 |   content: string,
32 |   t: TFN,
33 |   onCitationClick?: (index: number) => void
34 | ): React.ReactNode => {
35 |   // Split content into segments, handling citation markers [1], [2], etc.
36 |   const parts = content.split(/\[([^\]]+)\]/g);
37 |
38 |   return parts.map((part, i) => {
39 |     const citationMatch = part.match(/\[([^\]]+)\]/);
40 |     if (citationMatch && citationMatch[1]) {
41 |       const citationIndex = parseInt(citationMatch[1], 10) - 1;
42 |       return (
43 |         <button
44 |           key={i}
45 |           onClick={() => onCitationClick?.(citationIndex)}
46 |           className="inline-flex items-center justify-center h-4 min-w-4 px-1 rounded-full bg-blue-100 text-bl
47 |           aria-label={t("tutor.chat.citationLabel", {
48 |             number: citationMatch[1],
49 |           })}
50 |         >
51 |           {citationMatch[1]}
52 |         </button>
53 |       );
54 |     }
55 |
56 |     // Simple markdown rendering for assistant messages
57 |     return <SimpleMarkdown key={i} text={part} />;
58 |   });
59 | };
60 |
61 | /** Minimal markdown renderer for bold, italic, code blocks, and lists */
62 | const SimpleMarkdown = ({ text }: { text: string }) => {
63 |   // Handle code blocks
64 |   if (text.includes("```")) {
65 |     const segments = text.split(/(```[^\s\S]*?```)/g);
66 |     return (
67 |       <div>
68 |         {segments.map((segment, i) => {
69 |           if (segment.startsWith("```") && segment.endsWith("```")) {
70 |             const codeContent = segment.slice(3, -3);
71 |             // Remove optional language identifier from first line
72 |             const firstNewline = codeContent.indexOf("\n");
73 |             const code =
74 |               firstNewline > -1
75 |                 ? codeContent.slice(firstNewline + 1)
76 |                 : codeContent;
77 |             return (
78 |               <pre
79 |                 key={i}
80 |                 className="my-2 rounded-lg bg-gray-900 text-gray-100 p-3 text-xs overflow-x-auto"
81 |               >
82 |                 <code>{code.trim()}</code>
83 |               </pre>
84 |             );
85 |           }
86 |           return <InlineMarkdown key={i} text={segment} />;
87 |         })}
88 |       </div>
89 |     );
90 |   }
91 |
92 |   return <InlineMarkdown text={text} />;
93 | };

```

```

94 |
95 | /** Handle inline markdown: bold, italic, inline code, line breaks */
96 | const InlineMarkdown = ({ text }: { text: string }) => {
97 |   // Split by lines to handle lists and paragraphs
98 |   const lines = text.split("\n");
99 |
100 |   return (
101 |     <>
102 |     {lines.map((line, i) => {
103 |       const trimmed = line.trim();
104 |
105 |       // Unordered list items
106 |       if (trimmed.startsWith("- ") || trimmed.startsWith("* ")) {
107 |         return (
108 |           <div key={i} className="flex gap-2 ps-2">
109 |             <span className="text-gray-400 shrink-0">*</span>
110 |             <span>{renderInline(trimmed.slice(2))}</span>
111 |           </div>
112 |         );
113 |       }
114 |
115 |       // Ordered list items
116 |       const orderedMatch = trimmed.match(/^(\\d+)\\.\\s/);
117 |       if (orderedMatch) {
118 |         return (
119 |           <div key={i} className="flex gap-2 ps-2">
120 |             <span className="text-gray-400 shrink-0 min-w-4 text-end">
121 |               {orderedMatch[1]}.
122 |             </span>
123 |             <span>{renderInline(trimmed.slice(orderedMatch[0].length))}</span>
124 |           </div>
125 |         );
126 |       }
127 |
128 |       // Empty lines become spacing
129 |       if (!trimmed) {
130 |         return i > 0 && i < lines.length - 1 ? (
131 |           <div key={i} className="h-2" />
132 |         ) : null;
133 |       }
134 |
135 |       // Regular text
136 |       return (
137 |         <span key={i}>
138 |           {renderInline(line)}
139 |           {i < lines.length - 1 ? " " : ""}
140 |         </span>
141 |       );
142 |     })}
143 |   </>
144 | );
145 | };
146 |
147 | /** Render inline formatting: bold, italic, `code` */
148 | const renderInline = (text: string): React.ReactNode => {
149 |   // Bold
150 |   const parts = text.split(/(\\*[\\*]+\\*)/g);
151 |   return parts.map((part, i) => {
152 |     if (part.startsWith("****") && part.endsWith("****")) {
153 |       return (
154 |         <strong key={i} className="font-semibold">
155 |           {part.slice(2, -2)}
156 |         </strong>
157 |       );
158 |     }
159 |     // Inline code
160 |     const codeParts = part.split(/(`[\\`]+`)/g);
161 |     return codeParts.map((cp, j) => {
162 |       if (cp.startsWith("`") && cp.endsWith("`")) {
163 |         return (
164 |           <code
165 |             key={`$${i}-${j}`}
166 |             className="px-1 py-0.5 rounded bg-gray-100 text-gray-800 text-xs font-mono"
167 |           >
168 |             {cp.slice(1, -1)}
169 |           </code>
170 |         );
171 |       }
172 |       // Italic
173 |       const italicParts = cp.split(/(\\*[\\*]+\\*)/g);
174 |       return italicParts.map((ip, k) => {
175 |         if (ip.startsWith("****") && ip.endsWith("****") && !ip.startsWith("****")) {
176 |           return <em key={`$${i}-${j}-${k}`}>{ip.slice(1, -1)}</em>;
177 |         }
178 |         return <span key={`$${i}-${j}-${k}`}>{ip}</span>;
179 |       });
180 |     });
181 |   });
182 | };
183 |
184 | const ChatMessage = ({
185 |   message,
186 |   onRate,
187 |   isRatingPending,
188 | }: ChatMessageProps) => {
189 |   const { t } = useTranslation("ai");
190 |   const [expandedCitation, setExpandedCitation] = useState<number | null>(null);
191 |   const isUser = message.role === "user";
192 |   const isAssistant = message.role === "assistant";
193 |
194 |   const handleCitationClick = useCallback((index: number) => {
195 |     setExpandedCitation((prev) => (prev === index ? null : index));
196 |   }, []);
197 |

```

```

198 | return (
199 |   <div
200 |     className={cn(
201 |       "flex gap-3 max-w-full",
202 |       isUser ? "flex-row-reverse" : "flex-row"
203 |     )}
204 |     role="article"
205 |     aria-label={
206 |       isUser
207 |         ? t("tutor.chat.userMessageLabel")
208 |         : t("tutor.chat.tutorMessageLabel")
209 |     }
210 |   >
211 |     {/* Avatar */}
212 |     <div
213 |       className={cn(
214 |         "shrink-0 h-8 w-8 rounded-full flex items-center justify-center",
215 |         isUser
216 |           ? "bg-blue-100"
217 |           : "bg-[linear-gradient(93.65deg,#14b8a6_5.37%,#0382bd_78.89%)]"
218 |       )}
219 |       aria-hidden="true"
220 |     >
221 |       {isUser ? (
222 |         <User className="h-4 w-4 text-blue-600" />
223 |       ) : (
224 |         <Bot className="h-4 w-4 text-white" />
225 |       )}
226 |     </div>
227 |
228 |     {/* Message content */}
229 |     <div
230 |       className={cn(
231 |         "flex flex-col max-w-[80%] min-w-0",
232 |         isUser ? "items-end" : "items-start"
233 |       )}
234 |     >
235 |       <div
236 |         className={cn(
237 |           "rounded-2xl px-4 py-2.5 text-sm leading-relaxed",
238 |           isUser
239 |             ? "bg-blue-600 text-white rounded-br-md"
240 |             : "bg-white border border-gray-100 shadow-sm text-gray-800 rounded-bl-md"
241 |         )}
242 |       >
243 |         {isUser ? (
244 |           <p className="whitespace-pre-wrap break-words">{message.content}</p>
245 |         ) : (
246 |           <div className="whitespace-pre-wrap break-words">
247 |             {renderContent(message.content, t, handleCitationClick)}
248 |           </div>
249 |         )}
250 |
251 |         {/* Image attachments */}
252 |         {message.image_urls.length > 0 && (
253 |           <div className="mt-2 flex gap-2 flex-wrap">
254 |             {message.image_urls.map((url, i) => (
255 |               <img
256 |                 key={i}
257 |                 src={url}
258 |                 alt={t("tutor.chat.attachmentAlt", { index: i + 1 })}
259 |                 className="rounded-lg max-h-48 max-w-48 object-cover"
260 |                 loading="lazy"
261 |               />
262 |             ))}
263 |           </div>
264 |         )}
265 |
266 |         {/* Document attachment */}
267 |         {message.document_url && (
268 |           <div className="mt-2">
269 |             <Badge variant="outline" className="text-xs gap-1">
270 |               0=UI {t("tutor.chat.documentAttached")}
271 |             </Badge>
272 |           </div>
273 |         )}
274 |       </div>
275 |
276 |       {/* Integrity flag */}
277 |       {message.flagged_integrity && isAssistant && (
278 |         <Badge
279 |           variant="outline"
280 |           className="mt-1 text-[10px] text-amber-600 border-amber-200 bg-amber-50"
281 |         >
282 |           {t("tutor.chat.integrityGuidance")}
283 |         </Badge>
284 |       )}
285 |
286 |       {/* Citation panel + rating for assistant messages */}
287 |       {isAssistant && (
288 |         <div className="flex flex-col items-start w-full">
289 |           {message.source_citations.length > 0 && (
290 |             <SourceCitationPanel
291 |               citations={message.source_citations}
292 |               expandedIndex={expandedCitation}
293 |               onExpandedChange={setExpandedCitation}
294 |             />
295 |           )}
296 |
297 |           {onRate && (
298 |             <div className="mt-1">
299 |               <SatisfactionRating
300 |                 messageId={message.id}
301 |                 conversationId={message.conversation_id}

```

```

302 |         currentRating={message.satisfaction_rating}
303 |         onRate={onRate}
304 |         isPending={isRatingPending}
305 |       />
306 |     </div>
307 |   )}
308 | </div>
309 | )}
310 |
311 |   { /* Timestamp */ }
312 |   <span className="text-[10px] text-gray-400 mt-1 px-1">
313 |     {new Date(message.created_at).toLocaleTimeString([], {
314 |       hour: "2-digit",
315 |       minute: "2-digit",
316 |     })}
317 |   </span>
318 | </div>
319 | </div>
320 | );
321 | };
322 |
323 | export default ChatMessage;
324 | export type { ChatMessageProps };
325 |

```

FILE: src/components/shared/ClassDonationProgress.tsx

TYPE: .tsx | LINES: 1-103 | SHA256: f2e859c55de71010d1da2408260adcac725a84cb05f174f7376ba64915401a12

```

1 | // =====
2 | // ClassDonationProgress – Donation progress bar with goal display
3 | // Task 22.2
4 | // =====
5 |
6 | import { useMemo } from "react";
7 | import { Card } from "@components/ui/card";
8 | import { Button } from "@components/ui/button";
9 | import { Heart, Loader2 } from "lucide-react";
10 | import {
11 |   useContributeToDonation,
12 |   type ClassDonation,
13 | } from "@hooks/useClassDonations";
14 | import { useAuth } from "@hooks/useAuth";
15 |
16 | interface ClassDonationProgressProps {
17 |   donation: ClassDonation;
18 | }
19 |
20 | const ClassDonationProgress = ({ donation }: ClassDonationProgressProps) => {
21 |   const { profile } = useAuth();
22 |   const contribute = useContributeToDonation();
23 |
24 |   const progressPercent = useMemo(
25 |     () =>
26 |     Math.min(
27 |       100,
28 |       Math.round((donation.current_total / donation.goal_amount) * 100)
29 |     ),
30 |     [donation.current_total, donation.goal_amount]
31 |   );
32 |
33 |   const isCompleted = donation.status === "completed" || progressPercent >= 100;
34 |
35 |   const handleContribute = () => {
36 |     if (!profile?.id) return;
37 |     contribute.mutate({
38 |       donationId: donation.id,
39 |       studentId: profile.id,
40 |       xpAmount: 10,
41 |     });
42 |   };
43 |
44 |   return (
45 |     <Card
46 |       className="bg-white border-0 shadow-md rounded-xl p-4"
47 |       data-testid="class-donation-progress"
48 |     >
49 |       <div className="flex items-start gap-3">
50 |         <div className="p-2 rounded-lg bg-pink-50">
51 |           <Heart className="h-5 w-5 text-pink-500" />
52 |         </div>
53 |         <div className="flex-1 min-w-0">
54 |           <h3 className="text-sm font-bold">Class Donation</h3>
55 |           <p className="text-xs text-gray-500 mt-0.5">
56 |             {donation.resource_description}
57 |           </p>
58 |
59 |           <div className="mt-3">
60 |             <div className="flex items-center justify-between text-[10px] text-gray-500">
61 |               <span>{donation.current_total.toLocaleString()} XP raised</span>
62 |               <span>{donation.goal_amount.toLocaleString()} XP goal</span>
63 |             </div>
64 |             <div className="h-2 rounded-full bg-gray-100 overflow-hidden mt-1">
65 |               <div
66 |                 className="h-full rounded-full bg-gradient-to-r from-pink-400 to-rose-500 transition-all durat
67 |                 style={{ width: `${progressPercent}%` }}
68 |                 data-testid="donation-progress-bar"
69 |               />
70 |             </div>
71 |             <p className="text-[10px] text-gray-400 mt-1">
72 |               {progressPercent}% complete
73 |             </p>
74 |           </div>
75 |
76 |           {!isCompleted && (

```

```

77 |         <Button
78 |             size="sm"
79 |             variant="outline"
80 |             className="mt-2 text-pink-600 border-pink-200 hover:bg-pink-50"
81 |             disabled={contribute.isPending}
82 |             onClick={handleContribute}
83 |         >
84 |             {contribute.isPending && (
85 |                 <Loader2 className="h-3 w-3 animate-spin" />
86 |             )}
87 |             <Heart className="h-3 w-3" /> Donate 10 XP
88 |         </Button>
89 |     )}
90 |
91 |     {isCompleted && (
92 |         <p className="text-xs font-bold text-green-600 mt-2">
93 |             0<β% Goal reached!
94 |         </p>
95 |     )}
96 | </div>
97 | </div>
98 | </Card>
99 | );
100 | };
101 |
102 | export default ClassDonationProgress;
103 |

```

FILE: src/components/shared/CL0ProgressBar.tsx

TYPE: .tsx | LINES: 1-161 | SHA256: e12095606075deb7a1a0925ffff24ad96e0cc11cd58f9755272ad0becd61cf86e

```
// =====  
1 // CLOProgressBar - Reusable attainment bar with Bloom's color coding  
2 // =====  
3  
4 import { motion } from "framer-motion";  
5 import { ChevronDown } from "lucide-react";  
6 import { Badge } from "@components/ui/badge";  
7 import { cn } from "@lib/utils";  
8 import type { BloomsLevel, AttainmentLevel } from "@types/app";  
9  
10 // % % % Color Maps % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
11  
12 const BLOOMS_COLORS: Record<  
13   BloomsLevel,  
14   { bg: string; text: string; label: string }  
15 > = {  
16   remembering: { bg: "bg-purple-500", text: "text-white", label: "Remember" },  
17   understanding: { bg: "bg-blue-500", text: "text-white", label: "Understand" },  
18   applying: { bg: "bg-green-500", text: "text-white", label: "Apply" },  
19   analyzing: { bg: "bg-yellow-500", text: "text-gray-900", label: "Analyze" },  
20   evaluating: { bg: "bg-orange-500", text: "text-white", label: "Evaluate" },  
21   creating: { bg: "bg-red-500", text: "text-white", label: "Create" },  
22 };  
23  
24 const ATTAINMENT_STYLES: Record<  
25   AttainmentLevel,  
26   { bar: string; text: string; label: string }  
27 > = {  
28   Excellent: {  
29     bar: "bg-green-500",  
30     text: "text-green-600",  
31     label: "Excellent",  
32   },  
33   Satisfactory: {  
34     bar: "bg-blue-500",  
35     text: "text-blue-600",  
36     label: "Satisfactory",  
37   },  
38   Developing: {  
39     bar: "bg-yellow-500",  
40     text: "text-yellow-600",  
41     label: "Developing",  
42   },  
43   Not_Yet: { bar: "bg-red-500", text: "text-red-600", label: "Not Yet" },  
44 };  
45  
46 // % % % Props % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
47  
48 interface CLOProgressBarProps {  
49   title: string;  
50   bloomsLevel: BloomsLevel;  
51   attainmentPercent: number | null;  
52   attainmentLevel: AttainmentLevel | null;  
53   className?: string;  
54   onClick?: () => void;  
55   isExpanded?: boolean;  
56 }  
57  
58 // % % % Component % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
59  
60 const CLOProgressBar = ({  
61   title,  
62   bloomsLevel,  
63   attainmentPercent,  
64   attainmentLevel,  
65   className,  
66   onClick,  
67   isExpanded,  
68 }: CLOProgressBarProps) => {  
69   const blooms = BLOOMS_COLORS[bloomsLevel];  
70   const attainment = attainmentLevel  
71     ? ATTAINMENT_STYLES[attainmentLevel]  
72     : null;  
73 }
```

```

74 | const percent = attainmentPercent ?? 0;
75 | const isAssessed = attainmentPercent !== null;
76 | const isClickable = !!onClick;
77 |
78 | return (
79 |   <div
80 |     className={cn(
81 |       "space-y-2",
82 |       isClickable &&
83 |       "cursor-pointer rounded-lg p-2 -m-2 transition-colors hover:bg-slate-50",
84 |       className
85 |     )}
86 |     onClick={onClick}
87 |     role={isClickable ? "button" : undefined}
88 |     tabIndex={isClickable ? 0 : undefined}
89 |     onKeyDown={
90 |       isClickable
91 |       ? (e) => {
92 |         if (e.key === "Enter" || e.key === " ") {
93 |           e.preventDefault();
94 |           onClick?.();
95 |         }
96 |       }
97 |       : undefined
98 |     }
99 |     aria-expanded={isClickable ? isExpanded : undefined}
100 |   >
101 |     /* Header row: CLO title + Bloom's pill + attainment label */
102 |     <div className="flex items-center justify-between gap-2">
103 |       <div className="flex items-center gap-2 min-w-0">
104 |         <Badge
105 |           className={cn(
106 |             blooms.bg,
107 |             blooms.text,
108 |             "text-xs font-bold tracking-wide uppercase shrink-0"
109 |           )}
110 |         >
111 |           {blooms.label}
112 |         </Badge>
113 |         <span className="text-sm font-medium truncate">{title}</span>
114 |       </div>
115 |       <div className="flex items-center gap-2 shrink-0">
116 |         {isAssessed ? (
117 |           <>
118 |             <span className="text-sm font-bold tabular-nums">
119 |               {Math.round(percent)}%
120 |             </span>
121 |             <span className={cn("text-xs font-semibold", attainment?.text)}>
122 |               {attainment?.label}
123 |             </span>
124 |           </>
125 |         ) : (
126 |           <span className="text-xs text-gray-400 italic">Not assessed</span>
127 |         )}
128 |         {isClickable && (
129 |           <motion.div
130 |             animate={{ rotate: isExpanded ? 180 : 0 }}
131 |             transition={{ duration: 0.2 }}
132 |           >
133 |             <ChevronDown className="h-4 w-4 text-gray-400" />
134 |           </motion.div>
135 |         )}
136 |       </div>
137 |     </div>
138 |
139 |     /* Progress bar */
140 |     <div className="h-2.5 w-full rounded-full bg-slate-100 overflow-hidden">
141 |       {isAssessed ? (
142 |         <motion.div
143 |           className={cn(
144 |             "h-full rounded-full",
145 |             attainment?.bar ?? "bg-gray-300"
146 |           )}
147 |           initial={{ width: 0 }}
148 |           animate={{ width: `${Math.min(percent, 100)}%` }}
149 |           transition={{ duration: 0.6, ease: "easeOut" }}
150 |         />
151 |       ) : (
152 |         <div className="h-full w-full bg-slate-100" />
153 |       )}
154 |     </div>
155 |   </div>
156 | );
157 | };
158 |
159 | export default CLOProgressBar;
160 | export { BLOOMS_COLORS, ATTAINMENT_STYLES };
161 |

```

FILE: [src/components/shared/CognitiveLoadIndicator.tsx](#)

TYPE: .tsx | **LINES:** 1-100 | **SHA256:** d129fd95e68d258db0d4185c44ffa4bfaafd5909171f1849d1625003707b649bb

```

1 | import { useState, useEffect } from "react";
2 | import { useTranslation } from "react-i18next";
3 | import { Button } from "@components/ui/button";
4 | import { X, Eye } from "lucide-react";
5 | import {
6 |   useAccessibilityPreferences,
7 |   useUpdateAccessibilityPreferences,
8 | } from "@hooks/useAccessibilityPreferences";
9 |
10 | interface CognitiveLoadIndicatorProps {
11 |   sectionCount: number;
12 |   threshold?: number;

```

```

13 |   pageId: string;
14 | }
15 |
16 | const DISMISSED_KEY = "edeviser-cognitive-dismissed";
17 |
18 | export const CognitiveLoadIndicator = ({
19 |   sectionCount,
20 |   threshold = 6,
21 |   pageId,
22 | }: CognitiveLoadIndicatorProps) => {
23 |   const { t } = useTranslation("common");
24 |   const { data: prefs } = useAccessibilityPreferences();
25 |   const updatePrefs = useUpdateAccessibilityPreferences();
26 |   const [dismissed, setDismissed] = useState(() => {
27 |     const dismissedPages: string[] = JSON.parse(
28 |       localStorage.getItem(DISMISSED_KEY) || "[]"
29 |     );
30 |     return dismissedPages.includes(pageId);
31 |   });
32 |
33 |   // Re-check when pageId changes
34 |   useEffect(() => {
35 |     const dismissedPages: string[] = JSON.parse(
36 |       localStorage.getItem(DISMISSED_KEY) || "[]"
37 |     );
38 |     if (dismissedPages.includes(pageId) && !dismissed) {
39 |       // Defer state update to avoid synchronous setState in effect
40 |       queueMicrotask(() => setDismissed(true));
41 |     }
42 |   }, [pageId, dismissed]);
43 |
44 |   if (dismissed || sectionCount <= threshold || prefs?.simplified_view)
45 |     return null;
46 |
47 |   const handleDismiss = () => {
48 |     const dismissedPages: string[] = JSON.parse(
49 |       localStorage.getItem(DISMISSED_KEY) || "[]"
50 |     );
51 |     localStorage.setItem(
52 |       DISMISSED_KEY,
53 |       JSON.stringify([...dismissedPages, pageId])
54 |     );
55 |     setDismissed(true);
56 |   };
57 |
58 |   const handleSimplify = () => {
59 |     if (prefs) {
60 |       updatePrefs.mutate({ ...prefs, simplified_view: true });
61 |     }
62 |     handleDismiss();
63 |   };
64 |
65 |   return (
66 |     <div
67 |       role="status"
68 |       className="flex items-center justify-between gap-4 rounded-lg border border-blue-200 bg-blue-50 px-4 py-
69 |     >
70 |       <p>
71 |         {t("accessibility.cognitiveLoadMessage", {
72 |           defaultValue:
73 |             "This page has a lot of information – would you like a simplified view?",
74 |         })}
75 |       </p>
76 |       <div className="flex items-center gap-2">
77 |         <Button
78 |           variant="outline"
79 |           size="sm"
80 |           onClick={handleSimplify}
81 |           className="gap-1"
82 |         >
83 |           <Eye className="h-4 w-4" />
84 |           {t("accessibility.simplifiedView", {
85 |             defaultValue: "Simplified View",
86 |           })}
87 |         </Button>
88 |         <Button
89 |           variant="ghost"
90 |           size="sm"
91 |           onClick={handleDismiss}
92 |           aria-label={t("buttons.close", { defaultValue: "Close" })}
93 |         >
94 |           <X className="h-4 w-4" />
95 |         </Button>
96 |       </div>
97 |     </div>
98 |   );
99 | };
100 |

```

FILE: [src/components/shared/ComebackChallengeBanner.tsx](#)

TYPE: .tsx | LINES: 1-77 | SHA256: ae0f963ec22cb0a335d1c4cc857b71821c5eea825aa9c0b46ff311c7c53e6212

```

1 | // Task 144.4: Comeback Challenge Banner component
2 |
3 | import { Card } from "@components/ui/card";
4 | import { Button } from "@components/ui/button";
5 | import { Flame, X } from "lucide-react";
6 |
7 | interface ComebackChallengeBannerProps {
8 |   daysCompleted: number;
9 |   streakToRestore: number;
10 |   onDismiss: () => void;
11 | }
12 |

```

```

13 | const ComebackChallengeBanner = ({
14 |   daysCompleted,
15 |   streakToRestore,
16 |   onDismiss,
17 | }: ComebackChallengeBannerProps) => {
18 |   const days = [1, 2, 3];
19 |
20 |   return (
21 |     <Card
22 |       className="bg-gradient-to-r from-orange-50 to-amber-50 border border-orange-200 shadow-md rounded-xl p-4
23 |       data-testid="comeback-challenge-banner"
24 |       role="region"
25 |       aria-label="Comeback Challenge"
26 |     >
27 |       <div className="flex items-start justify-between">
28 |         <div className="flex items-start gap-3">
29 |           <div className="p-2 rounded-lg bg-orange-100">
30 |             <Flame className="h-5 w-5 text-orange-500" />
31 |           </div>
32 |           <div>
33 |             <p className="text-sm font-bold text-orange-800">
34 |               Comeback Challenge
35 |             </p>
36 |             <p className="text-xs text-orange-600 mt-0.5">
37 |               Complete 3 days to restore your {streakToRestore}-day streak
38 |             </p>
39 |             <div
40 |               className="flex gap-2 mt-3"
41 |               aria-label={` ${daysCompleted} of 3 days completed`}
42 |             >
43 |               {days.map((day) => (
44 |                 <div
45 |                   key={day}
46 |                   data-testid={`comeback-day-${day}`}
47 |                   aria-label={`Day ${day}${
48 |                     day <= daysCompleted ? " completed" : ""
49 |                   `}
50 |                 >
51 |                   <div
52 |                     className={`h-8 w-8 rounded-full flex items-center justify-center text-xs font-bold border-2
53 |                       ? "bg-green-500 border-green-500 text-white"
54 |                       : "bg-white border-gray-300 text-gray-400"
55 |                   `}
56 |                   >
57 |                     {day}
58 |                   </div>
59 |                 </div>
60 |               ))}
61 |             </div>
62 |             <Button
63 |               variant="ghost"
64 |               size="sm"
65 |               onClick={onDismiss}
66 |               className="text-gray-400 hover:text-gray-600"
67 |               aria-label="Dismiss comeback challenge"
68 |             >
69 |               <X className="h-4 w-4" />
70 |             </Button>
71 |           </div>
72 |         </Card>
73 |       );
74 |     };
75 |
76 | export default ComebackChallengeBanner;
77 |

```

FILE: [src/components/shared/CompetencyAlignmentMatrix.tsx](#)

TYPE: .tsx | LINES: 1-136 | SHA256: e297e66af5dc66cb376686ae4581851173fecb9882f609d8f0f7dca939b760bc

```

1 | // Task 115.3: Competency Alignment Matrix shared component
2 | // Matrix: Competency Indicators (rows) × Outcomes (columns)
3 |
4 | import { Badge } from "@components/ui/badge";
5 | import { AlertTriangle, CheckCircle } from "lucide-react";
6 | import type {
7 |   CompetencyItem,
8 |   CompetencyOutcomeMapping,
9 | } from "@hooks/useCompetencyFrameworks";
10 |
11 | interface OutcomeInfo {
12 |   id: string;
13 |   title: string;
14 |   code?: string;
15 |   type: string;
16 |   attainment?: number;
17 | }
18 |
19 | interface CompetencyAlignmentMatrixProps {
20 |   indicators: CompetencyItem[];
21 |   outcomes: OutcomeInfo[];
22 |   mappings: CompetencyOutcomeMapping[];
23 | }
24 |
25 | const getAttainmentColor = (score: number | undefined) => {
26 |   if (score === undefined) return "bg-slate-100 text-slate-400";
27 |   if (score >= 85) return "bg-green-50 text-green-600";
28 |   if (score >= 70) return "bg-blue-50 text-blue-600";
29 |   if (score >= 50) return "bg-yellow-50 text-yellow-600";
30 |   return "bg-red-50 text-red-600";
31 | };
32 |
33 | const CompetencyAlignmentMatrix = ({
34 |   indicators,
35 |   outcomes,

```



```

36 | mappings,
37 | }: CompetencyAlignmentMatrixProps) => {
38 |   const mappingSet = new Set(
39 |     mappings.map((m) => `${m.competency_item_id}:${m.outcome_id}`)
40 |   );
41 |
42 |   const mappedIndicatorIds = new Set(mappings.map((m) => m.competency_item_id));
43 |
44 |   if (indicators.length === 0 || outcomes.length === 0) {
45 |     return (
46 |       <p className="text-sm text-slate-400 text-center py-6">
47 |         No data available for alignment matrix.
48 |       </p>
49 |     );
50 |   }
51 |
52 |   return (
53 |     <div className="overflow-x-auto">
54 |       <table
55 |         className="min-w-full text-xs"
56 |         role="grid"
57 |         aria-label="Competency alignment matrix"
58 |       >
59 |         <thead>
60 |           <tr>
61 |             <th className="sticky start-0 bg-white z-10 px-3 py-2 text-start font-semibold text-slate-600 border-b border-slate-200">
62 |               Indicator
63 |             </th>
64 |             {outcomes.map((o) => (
65 |               <th
66 |                 key={o.id}
67 |                 className="px-2 py-2 text-center font-semibold text-slate-600 border-b border-slate-200 min-w-
68 |                 title={o.title}
69 |               >
70 |                 <span className="block truncate max-w-[80px]">
71 |                   {o.code ?? o.title.slice(0, 10)}
72 |                 </span>
73 |                 <span className="block text-[9px] text-slate-400 font-normal">
74 |                   {o.type}
75 |                 </span>
76 |               </th>
77 |             )]}
78 |             <th className="px-2 py-2 text-center font-semibold text-slate-600 border-b border-slate-200">
79 |               Status
80 |             </th>
81 |           </tr>
82 |         </thead>
83 |         <tbody>
84 |           {indicators.map((ind) => {
85 |             const isMapped = mappedIndicatorIds.has(ind.id);
86 |             return (
87 |               <tr key={ind.id} className="hover:bg-slate-50">
88 |                 <td className="sticky start-0 bg-white z-10 px-3 py-2 border-e border-slate-100">
89 |                   <span className="font-mono text-slate-400 me-1">
90 |                     {ind.code}
91 |                   </span>
92 |                   <span className="text-slate-700">{ind.title}</span>
93 |                 </td>
94 |                 {outcomes.map((o) => {
95 |                   const key = `${ind.id}:${o.id}`;
96 |                   const hasMapped = mappingSet.has(key);
97 |                   return (
98 |                     <td
99 |                       key={o.id}
100 |                      className={`px-2 py-2 text-center border-slate-100 ${
101 |                        hasMapped
102 |                          ? getAttainmentColor(o.attainment)
103 |                          : "bg-white"
104 |                      }`}
105 |                     >
106 |                       {hasMapped ? (
107 |                         <CheckCircle className="h-3.5 w-3.5 mx-auto text-current" />
108 |                       ) : (
109 |                         <span className="text-slate-200"></span>
110 |                       )}
111 |                     </td>
112 |                   );
113 |                 )}}
114 |                 <td className="px-2 py-2 text-center">
115 |                   {isMapped ? (
116 |                     <Badge className="text-[9px] bg-green-100 text-green-700">
117 |                       Mapped
118 |                     </Badge>
119 |                   ) : (
120 |                     <Badge className="text-[9px] bg-amber-100 text-amber-700">
121 |                       <AlertTriangle className="h-2.5 w-2.5 me-0.5" />
122 |                       Unmapped
123 |                     </Badge>
124 |                   )}
125 |                 </td>
126 |               </tr>
127 |             );
128 |           })}
129 |         </tbody>
130 |       </table>
131 |     </div>
132 |   );
133 | };
134 |
135 | export default CompetencyAlignmentMatrix;
136 |

```

FILE: src/components/shared/CompetencyTree.tsx

TYPE: .tsx | LINES: 1-126 | SHA256: 0623df0b303d9c0e33f0b6a4008291812f30d7a35171247498aae5e1f817d88b

```
1 // Task 115.2: Competency Tree shared component
2
3 import { useState } from "react";
4 import { Badge } from "@components/ui/badge";
5 import { ChevronRight, ChevronDown, AlertTriangle } from "lucide-react";
6 import type { CompetencyItem } from "@hooks/useCompetencyFrameworks";
7
8 interface CompetencyTreeProps {
9   items: CompetencyItem[];
10   mappedItemIds?: Set<string>;
11 }
12
13 const buildTree = (items: CompetencyItem[]): CompetencyItem[] => {
14   const map = new Map<
15     string,
16     CompetencyItem & { children: CompetencyItem[] }
17   >();
18   const roots: (CompetencyItem & { children: CompetencyItem[] })[] = [];
19
20   items.forEach((item) => {
21     map.set(item.id, { ...item, children: [] });
22   });
23
24   items.forEach((item) => {
25     const node = map.get(item.id)!;
26     if (item.parent_id && map.has(item.parent_id)) {
27       map.get(item.parent_id)!.children.push(node);
28     } else {
29       roots.push(node);
30     }
31   });
32
33   return roots;
34 };
35
36 const TreeNode = ({
37   item,
38   mappedItemIds,
39   depth = 0,
40 }: {
41   item: CompetencyItem & { children?: CompetencyItem[] };
42   mappedItemIds?: Set<string>;
43   depth?: number;
44 }) => {
45   const [expanded, setExpanded] = useState(depth < 1);
46   const hasChildren = (item.children?.length ?? 0) > 0;
47   const isUnmapped =
48     item.level === "indicator" && mappedItemIds && !mappedItemIds.has(item.id);
49
50   return (
51     <div>
52       <div
53         className={`flex items-center gap-2 py-1.5 px-2 rounded-lg hover:bg-slate-50 cursor-pointer ${
54           depth > 0 ? "ml-" + depth * 4 : ""
55         }`}
56         style={{ marginLeft: depth * 16 }}
57         role="button"
58         tabIndex={0}
59         onClick={() => hasChildren && setExpanded(!expanded)}
60         onKeyDown={(e) => {
61           if ((e.key === "Enter" || e.key === " ") && hasChildren) {
62             e.preventDefault();
63             setExpanded(!expanded);
64           }
65         }}
66       >
67         {hasChildren ? (
68           expanded ? (
69             <ChevronDown className="h-3.5 w-3.5 text-slate-400" />
70           ) : (
71             <ChevronRight className="h-3.5 w-3.5 text-slate-400" />
72           )
73         ) : (
74           <span className="w-3.5" />
75         )}
76         <Badge variant="outline" className="text-[9px] px-1">
77           {item.level}
78         </Badge>
79         <span className="text-xs font-mono text-slate-400">{item.code}</span>
80         <span className="text-sm text-slate-700 truncate">{item.title}</span>
81         {isUnmapped && (
82           <Badge className="text-[9px] bg-amber-100 text-amber-700 ml-auto">
83             <AlertTriangle className="h-2.5 w-2.5 me-0.5" />
84             Unmapped
85           </Badge>
86         )}
87       </div>
88       {expanded &&
89         (item.children ?? []).map((child) => (
90           <TreeNode
91             key={child.id}
92             item={child as CompetencyItem & { children?: CompetencyItem[] }}
93             mappedItemIds={mappedItemIds}
94             depth={depth + 1}
95           />
96         ))}
97     </div>
98   );
99 };
100
101 const CompetencyTree = ({ items, mappedItemIds }: CompetencyTreeProps) => {
```

```

102 | const tree = buildTree(items);
103 |
104 | if (tree.length === 0) {
105 |   return (
106 |     <p className="text-sm text-slate-400 text-center py-6">
107 |       No competency items.
108 |     </p>
109 |   );
110 | }
111 |
112 | return (
113 |   <div className="space-y-0.5">
114 |     {tree.map((item) => (
115 |       <TreeNode
116 |         key={item.id}
117 |         item={item as CompetencyItem & { children?: CompetencyItem[] }}
118 |         mappedItemIds={mappedItemIds}
119 |       </>
120 |     ))}
121 |   </div>
122 | );
123 | };
124 |
125 | export default CompetencyTree;
126 |

```

FILE: [src/components/shared/ConfirmDialog.tsx](#)

TYPE: .tsx | LINES: 1-69 | SHA256: b9d753b47e78924a0fba9a3c6a5259336fb0044b593ea1df97cc67e30999c195

```

1 | import { Loader2 } from "lucide-react";
2 | import { useTranslation } from "react-i18next";
3 | import { Button } from "@components/ui/button";
4 | import {
5 |   Dialog,
6 |   DialogContent,
7 |   DialogDescription,
8 |   DialogFooter,
9 |   DialogHeader,
10 |   DialogTitle,
11 | } from "@components/ui/dialog";
12 |
13 | interface ConfirmDialogProps {
14 |   open: boolean;
15 |   onOpenChange: (open: boolean) => void;
16 |   title: string;
17 |   description: string;
18 |   confirmLabel?: string;
19 |   cancelLabel?: string;
20 |   variant?: "default" | "destructive";
21 |   isPending?: boolean;
22 |   onConfirm: () => void;
23 | }
24 |
25 | const ConfirmDialog = ({
26 |   open,
27 |   onOpenChange,
28 |   title,
29 |   description,
30 |   confirmLabel,
31 |   cancelLabel,
32 |   variant = "default",
33 |   isPending = false,
34 |   onConfirm,
35 | }: ConfirmDialogProps) => {
36 |   const { t } = useTranslation("common");
37 |
38 |   return (
39 |     <Dialog open={open} onOpenChange={onOpenChange}>
40 |       <DialogContent showCloseButton={false}>
41 |         <DialogHeader>
42 |           <DialogTitle>{title}</DialogTitle>
43 |           <DialogDescription>{description}</DialogDescription>
44 |         </DialogHeader>
45 |         <DialogFooter>
46 |           <Button
47 |             variant="outline"
48 |             onClick={() => onOpenChange(false)}
49 |             disabled={isPending}
50 |           >
51 |             {cancelLabel ?? t("buttons.cancel")}
52 |           </Button>
53 |           <Button
54 |             variant={variant === "destructive" ? "destructive" : "default"}
55 |             onClick={onConfirm}
56 |             disabled={isPending}
57 |           >
58 |             {isPending && <Loader2 className="h-4 w-4 animate-spin" />}
59 |             {confirmLabel ?? t("buttons.confirm")}
60 |           </Button>
61 |         </DialogFooter>
62 |       </DialogContent>
63 |     </Dialog>
64 |   );
65 | };
66 |
67 | export { ConfirmDialog };
68 | export type { ConfirmDialogProps };
69 |

```

FILE: src/components/shared/ConsistencyScoreRing.tsx

TYPE: .tsx | LINES: 1-85 | SHA256: 48e15a0f14a95feeaa7e1daca21d9caa06ccfce1f95ef07e3825cc4cf76229e4

```
1 | import { cn } from "@lib/utils";
2 |
3 | export interface ConsistencyScoreRingProps {
4 |   score: number; // 0-100
5 |   label: string;
6 |   size?: number;
7 |   strokeWidth?: number;
8 |   className?: string;
9 | }
10 |
11 | const ConsistencyScoreRing = ({
12 |   score,
13 |   label,
14 |   size = 120,
15 |   strokeWidth = 10,
16 |   className,
17 | }: ConsistencyScoreRingProps) => {
18 |   const radius = (size - strokeWidth) / 2;
19 |   const circumference = 2 * Math.PI * radius;
20 |   const clamped = Math.max(0, Math.min(100, score));
21 |   const offset = circumference - (clamped / 100) * circumference;
22 |
23 |   return (
24 |     <div
25 |       className={cn("flex flex-col items-center gap-2", className)}
26 |       data-testid="consistency-score-ring"
27 |     >
28 |       <svg
29 |         width={size}
30 |         height={size}
31 |         viewBox={`0 0 ${size} ${size}`}
32 |         role="img"
33 |         aria-label={` ${label}: ${clamped}%`}
34 |       >
35 |         { /* Background circle */ }
36 |         <circle
37 |           cx={size / 2}
38 |           cy={size / 2}
39 |           r={radius}
40 |           fill="none"
41 |           stroke="currentColor"
42 |           strokeWidth={strokeWidth}
43 |           className="text-slate-200"
44 |         />
45 |         { /* Progress circle */ }
46 |         <circle
47 |           cx={size / 2}
48 |           cy={size / 2}
49 |           r={radius}
50 |           fill="none"
51 |           stroke="url(#scoreGradient)"
52 |           strokeWidth={strokeWidth}
53 |           strokeLinecap="round"
54 |           strokeDasharray={circumference}
55 |           strokeDashoffset={offset}
56 |           transform={`rotate(-90 ${size / 2} ${size / 2})`}
57 |           className="transition-[stroke-dashoffset] duration-500"
58 |         />
59 |         <defs>
60 |           <linearGradient id="scoreGradient" x1="0%" y1="0%" x2="100%" y2="0%">
61 |             <stop offset="0%" stopColor="#14B8A6" />
62 |             <stop offset="100%" stopColor="#0382BD" />
63 |           </linearGradient>
64 |         </defs>
65 |         { /* Percentage text */ }
66 |         <text
67 |           x="50%"
68 |           y="50%"
69 |           textAnchor="middle"
70 |           dominantBaseline="central"
71 |           className="fill-gray-900 text-2xl font-black"
72 |           style={{ fontSize: size * 0.22 }}
73 |         >
74 |           {clamped}%
75 |         </text>
76 |       </svg>
77 |       <span className="text-xs font-bold tracking-widest uppercase text-gray-500">
78 |         {label}
79 |       </span>
80 |     </div>
81 |   );
82 | };
83 |
84 | export default ConsistencyScoreRing;
85 |
```

FILE: src/components/shared/ContributionStatusBadge.tsx

TYPE: .tsx | LINES: 1-50 | SHA256: 96d7d82ec57b55b3a79a78ea0a8c30bd57f8addaa71773e84e79421abaa576fd

```
1 | // =====
2 | // ContributionStatusBadge - Color-coded contribution status badge
3 | // Task 4.8: green=active, yellow=warning, red=inactive
4 | // =====
5 |
6 | import { Badge } from "@components/ui/badge";
7 | import { cn } from "@lib/utils";
8 | import type { ContributionStatus } from "@lib/contributionThresholds";
9 |
10 | export interface ContributionStatusBadgeProps {
11 |   status: ContributionStatus;
```

```

12 |   className?: string;
13 | }
14 |
15 | const STATUS_CONFIG: Record<
16 |   ContributionStatus,
17 |   { label: string; classes: string }
18 | > = {
19 |   active: {
20 |     label: "Active",
21 |     classes: "bg-green-100 text-green-700 border-green-200",
22 |   },
23 |   warning: {
24 |     label: "Warning",
25 |     classes: "bg-yellow-100 text-yellow-700 border-yellow-200",
26 |   },
27 |   inactive: {
28 |     label: "Inactive",
29 |     classes: "bg-red-100 text-red-700 border-red-200",
30 |   },
31 | };
32 |
33 | const ContributionStatusBadge = ({
34 |   status,
35 |   className,
36 | }: ContributionStatusBadgeProps) => {
37 |   const config = STATUS_CONFIG[status];
38 |
39 |   return (
40 |     <Badge
41 |       className={cn("text-[10px] font-bold", config.classes, className)}
42 |       aria-label={`Contribution status: ${config.label}`}
43 |     >
44 |       {config.label}
45 |     </Badge>
46 |   );
47 | };
48 |
49 | export default ContributionStatusBadge;
50 |

```

FILE: [src/components/shared/CookieConsentBanner.tsx](#)

TYPE: .tsx | LINES: 1-64 | SHA256: bf039ae6c9696ac0c481e916df6d9fa08313446f25551cefbafe6ed582a52d24c

```

1 | // Task 86: Cookie consent banner
2 | // Fixed bottom banner, shown on first visit when no consent in localStorage
3 |
4 | import { useState } from "react";
5 | import { Card, CardContent } from "@components/ui/card";
6 | import { Button } from "@components/ui/button";
7 | import {
8 |   getConsent,
9 |   setConsent,
10 |   initAnalyticsIfConsented,
11 | } from "@lib/analyticsConsent";
12 | import { Cookie } from "lucide-react";
13 |
14 | // Initialize analytics on module load if already consented
15 | initAnalyticsIfConsented();
16 |
17 | const CookieConsentBanner = () => {
18 |   const [visible, setVisible] = useState(() => getConsent() === null);
19 |
20 |   const handleAcceptAll = () => {
21 |     setConsent({ essential: true, analytics: true });
22 |     initAnalyticsIfConsented();
23 |     setVisible(false);
24 |   };
25 |
26 |   const handleRejectNonEssential = () => {
27 |     setConsent({ essential: true, analytics: false });
28 |     setVisible(false);
29 |   };
30 |
31 |   if (!visible) return null;
32 |
33 |   return (
34 |     <div className="fixed bottom-0 inset-x-0 z-50 p-4">
35 |       <Card className="bg-white border-0 shadow-lg rounded-xl max-w-2xl mx-auto">
36 |         <CardContent className="flex flex-col sm:flex-row items-start sm:items-center gap-4 p-4">
37 |           <div className="flex items-start gap-3 flex-1">
38 |             <Cookie className="h-5 w-5 text-amber-500 mt-0.5 shrink-0" />
39 |             <p className="text-sm text-gray-700">
40 |               We use cookies to improve your experience. Essential cookies are
41 |               always active. You can choose to enable analytics cookies to help
42 |               us improve the platform.
43 |             </p>
44 |           </div>
45 |           <div className="flex gap-2 shrink-0">
46 |             <Button
47 |               variant="outline"
48 |               size="sm"
49 |               onClick={handleRejectNonEssential}
50 |             >
51 |               Reject Non-Essential
52 |             </Button>
53 |             <Button size="sm" onClick={handleAcceptAll} variant="tactile">
54 |               Accept All
55 |             </Button>
56 |           </div>
57 |         </CardContent>
58 |       </Card>
59 |     </div>
60 |   );
61 | };

```

```
62 |
63 | export default CookieConsentBanner;
64 |
```

FILE: src/components/shared/CooperationScoreDisplay.tsx

TYPE: .tsx | LINES: 1-115 | SHA256: 90d85a36710774837d2e16abb681d0aed9abe299a3e9b59dced5b39291afc7ae

```
1  // =====
2  // CooperationScoreDisplay – Cooperation score gauge with color coding
3  // Task 4.9: visual gauge showing cooperation score 0-100
4  // =====
5
6  import { cn } from "@lib/utils";
7  import { Handshake } from "lucide-react";
8
9  export interface CooperationScoreDisplayProps {
10    score: number;
11    size?: "sm" | "md" | "lg";
12    showLabel?: boolean;
13    className?: string;
14  }
15
16  function getScoreColor(score: number): {
17    text: string;
18    bg: string;
19    ring: string;
20  } {
21    if (score >= 80)
22      return {
23        text: "text-green-600",
24        bg: "bg-green-500",
25        ring: "ring-green-200",
26      };
27    if (score >= 60)
28      return { text: "text-blue-600", bg: "bg-blue-500", ring: "ring-blue-200" };
29    if (score >= 40)
30      return {
31        text: "text-yellow-600",
32        bg: "bg-yellow-500",
33        ring: "ring-yellow-200",
34      };
35    return { text: "text-red-600", bg: "bg-red-500", ring: "ring-red-200" };
36  }
37
38  function getScoreLabel(score: number): string {
39    if (score >= 80) return "Excellent";
40    if (score >= 60) return "Good";
41    if (score >= 40) return "Fair";
42    return "Needs Improvement";
43  }
44
45  const CooperationScoreDisplay = ({
46    score,
47    size = "md",
48    showLabel = true,
49    className,
50  }: CooperationScoreDisplayProps) => {
51    const clampedScore = Math.max(0, Math.min(100, Math.round(score)));
52    const colors = getScoreColor(clampedScore);
53    const label = getScoreLabel(clampedScore);
54
55    const sizeClasses = {
56      sm: {
57        wrapper: "gap-2",
58        icon: "h-4 w-4",
59        value: "text-lg font-black",
60        gauge: "h-1.5",
61      },
62      md: {
63        wrapper: "gap-3",
64        icon: "h-5 w-5",
65        value: "text-2xl font-black",
66        gauge: "h-2",
67      },
68      lg: {
69        wrapper: "gap-4",
70        icon: "h-6 w-6",
71        value: "text-3xl font-black",
72        gauge: "h-2.5",
73      },
74    }[size];
75
76    return (
77      <div
78        className={cn("flex flex-col", sizeClasses.wrapper, className)}
79        aria-label={`Cooperation score: ${clampedScore} out of 100, ${label}`}
80      >
81        <div className="flex items-center gap-2">
82          <Handshake className={cn(sizeClasses.icon, colors.text)} />
83          <span className={cn(sizeClasses.value, colors.text, "tabular-nums")}>
84            {clampedScore}
85          </span>
86          <span className="text-[10px] font-black tracking-widest uppercase text-gray-500">
87            / 100
88          </span>
89        </div>
90
91        <div>
92          <div
93            className={cn(
94              "w-full rounded-full bg-gray-100 overflow-hidden",
95              sizeClasses.gauge
96            )}
97          >
```

```

98 |         <div
99 |             className={cn(
100 |                 "h-full rounded-full transition-all duration-500 ease-out",
101 |                 colors.bg
102 |             )}
103 |             style={{ width: `${clampedScore}%` }}
104 |         />
105 |     </div>
106 |
107 |     {showLabel && (
108 |         <span className={cn("text-xs font-medium", colors.text)}>{label}</span>
109 |     )}
110 | </div>
111 | );
112 | };
113 |
114 | export default CooperationScoreDisplay;
115 |

```

FILE: src/components/shared/CoordinatorDashboardNew.tsx

TYPE: .tsx | LINES: 1-866 | SHA256: 1f5787a4fd46e7b8519bba9a2b684859279f0f6238dc50a1adb66414e6fc01ff

```

1 | // =====
2 | // CoordinatorDashboardNew - redesigned coordinator dashboard (P2, spec task 2.4)
3 | // =====
4 | //
5 | // "Program health, prepared" decision cockpit gated behind `newUiDashboards`
6 | // (wrapper in CoordinatorDashboard.tsx). Matches the prototype reference: an
7 | // action-hub hero, KPI cards that act as filters, enriched attainment alerts,
8 | // and the curriculum-gap / accreditation-evidence / CQI / program-timeline row,
9 | // plus the shared CoordinatorInsightRail.
10 | //
11 | // DATA (Phase A backend wiring - REAL, no new backend/RPC/write):
12 | //   • KPI "Programs"           !' usePrograms
13 | //   • KPI "Avg PLO Attainment"!' useCoordinatorDashboardAggregate (get_coordinator_dashboard)
14 | //   • KPI "Below Target" + Attainment alerts
15 | //                                     !' useCoordinatorOutcomeAttainment (count / list of
16 | //                                     PLOs below the institution success threshold;
17 | //                                     weakest course + affected students derived from
18 | //                                     outcome_attainment). A positive empty state
19 | //                                     renders when every PLO meets target.
20 | //   • "Close the loop (CQI)"    !' useCQIPlans (real cqi_action_plans)
21 | //   • "Program timeline"        !' useAcademicCalendarEvents (real events, windowed
22 | //                                     around today)
23 | //   • Curriculum-gap card + accreditation readiness (evidenceReadiness) remain
24 | //     PRESENTATIONAL - the backend does not expose these yet (arrive in Phase
25 | //     B/C). All navigations use EXISTING coordinator routes only. Flag-off keeps
26 | //     the legacy dashboard byte-identical (see the router wrapper).
27 | //
28 | // Composed from P0 primitives (KPICard, SectionHeader, SeverityIcon) + tokens
29 | // (`.card-elevated`, `--brand-gradient`, tactile Button). i18n via the
30 | // `coordinator` namespace. RTL-safe via logical props; light-surface to match
31 | // the sibling `New` dashboards.
32 | // =====
33 |
34 | import { useMemo, useState, type ComponentType, type ReactNode } from "react";
35 | import { Link } from "react-router-dom";
36 | import { useTranslation } from "react-i18next";
37 | import {
38 |     Activity,
39 |     AlertTriangle,
40 |     ArrowRight,
41 |     CheckCircle2,
42 |     Circle,
43 |     Clock,
44 |     Grid3X3,
45 |     Info,
46 |     ShieldCheck,
47 |     Sparkles,
48 |     TrendingDown,
49 |     TrendingUp,
50 |     Users,
51 | } from "lucide-react";
52 |
53 | import {
54 |     Button,
55 |     Card,
56 |     KPICard,
57 |     SectionHeader,
58 |     SeverityIcon,
59 |     Shimmer,
60 | } from "@design-system";
61 | import CoordinatorInsightRail from "@components/shared/CoordinatorInsightRail";
62 | import { useAuth } from "@hooks/useAuth";
63 | import { useCoordinatorDashboardAggregate } from "@hooks/useCoordinatorDashboardAggregate";
64 | import { usePrograms } from "@hooks/usePrograms";
65 | import {
66 |     useCoordinatorOutcomeAttainment,
67 |     type AttainmentPLO,
68 | } from "@hooks/useCoordinatorOutcomeAttainment";
69 | import { useCoordinatorAiInsights } from "@hooks/useCoordinatorAiInsights";
70 | import { useCoordinatorAccreditationReadiness } from "@hooks/useCoordinatorAccreditation";
71 | import { useCQIPlans, type CQIPlanStatus } from "@hooks/useCQIPlans";
72 | import {
73 |     useAcademicCalendarEvents,
74 |     type AcademicCalendarEvent,
75 | } from "@hooks/useAcademicCalendar";
76 | import { attainmentValueClass } from "@lib/attainmentTone";
77 | import { cn } from "@lib/utils";
78 |
79 | type Tone = "red" | "amber" | "green" | "blue" | "slate";
80 |
81 | const PILL: Record<Tone, string> = {
82 |     red: "bg-red-50 text-red-600",

```

[illegible]

[illegible]

[illegible]

```

395 |
396 | const kpiLoading = aggregate.isPending || attainment.isPending;
397 |
398 | // Real CQI action plans !' "Close the loop" timeline.
399 | const cqQuery = useCQIPlans({});
400 | const cqItems = useMemo(
401 |   () =>
402 |     (cqQuery.data ?? []).slice(0, 4).map((plan) => ({
403 |       id: plan.id,
404 |       date: fmtDate(plan.due_date ?? plan.created_at, lang),
405 |       title: plan.action_description,
406 |       statusLabel: t(
407 |         CQI_STATUS_KEY[plan.status] ?? "dashboard.cqi.statusPlanned"
408 |       ),
409 |       tone: CQI_TONE[plan.status] ?? "slate",
410 |     })),
411 |   [cqQuery.data, lang, t]
412 | );
413 |
414 | // Real academic calendar !' "Program timeline" (windowed around today).
415 | // `now` is captured once via a lazy initializer so the render stays pure.
416 | const [now] = useState(() => Date.now());
417 | const calendarQuery = useAcademicCalendarEvents();
418 | const programItems = useMemo(() => {
419 |   const events = calendarQuery.data ?? [];
420 |   const upcoming = events
421 |     .filter((e) => eventEndMs(e) >= now)
422 |     .sort((a, b) => eventStartMs(a) - eventStartMs(b));
423 |   const past = events
424 |     .filter((e) => eventEndMs(e) < now)
425 |     .sort((a, b) => eventStartMs(b) - eventStartMs(a));
426 |   // Prefer what's next, backfill with the most recent past, then show chrono.
427 |   const chosen = [...upcoming, ...past].slice(0, 4);
428 |   const firstFutureId = upcoming.find((e) => eventStartMs(e) > now)?.id;
429 |   return chosen
430 |     .slice()
431 |     .sort((a, b) => eventStartMs(a) - eventStartMs(b))
432 |     .map((e) => {
433 |       const start = eventStartMs(e);
434 |       const end = eventEndMs(e);
435 |       let tone: Tone;
436 |       let statusLabel: string;
437 |       if (end < now) {
438 |         tone = "slate";
439 |         statusLabel = t("dashboard.timeline.statusCompleted");
440 |       } else if (start <= now && now <= end) {
441 |         tone = "green";
442 |         statusLabel = t("dashboard.timeline.statusNow");
443 |       } else if (e.id === firstFutureId) {
444 |         tone = "blue";
445 |         statusLabel = t("dashboard.timeline.statusNext");
446 |       } else {
447 |         tone = "slate";
448 |         statusLabel = t("dashboard.timeline.statusUpcoming");
449 |       }
450 |       return {
451 |         id: e.id,
452 |         date: fmtDate(e.start_date, lang),
453 |         title: e.title,
454 |         statusLabel,
455 |         tone,
456 |       };
457 |     });
458 | }, [calendarQuery.data, lang, t, now]);
459 |
460 | // Meta line for a below-target alert: weakest course + affected students.
461 | const alertMeta = (plo: AttainmentPLO): string => {
462 |   const parts: string[] = [];
463 |   if (plo.weakestCourse) {
464 |     parts.push(
465 |       t("dashboard.alerts.weakest", {
466 |         code: plo.weakestCourse.code,
467 |         pct: plo.weakestCourse.attainment,
468 |       })
469 |     );
470 |   }
471 |   if (plo.affectedStudents > 0) {
472 |     parts.push(
473 |       t("dashboard.alerts.affected", {
474 |         count: plo.affectedStudents,
475 |         percent: plo.cohortPercent,
476 |       })
477 |     );
478 |   }
479 |   return parts.join(" · ");
480 | };
481 |
482 | // Real accreditation readiness (evidence coverage). Null until the RPC is
483 | // deployed !' rendered as "-".
484 | const accred = useCoordinatorAccreditationReadiness(institutionId);
485 | const evidenceReadiness = accred.data?.readinessPercent ?? null;
486 |
487 | const evidenceItems = [
488 |   {
489 |     label: t("dashboard.evidence.cloMapping"),
490 |     statusLabel: t("dashboard.evidence.completed"),
491 |     state: "done" as const,
492 |   },
493 |   {
494 |     label: t("dashboard.evidence.samples"),
495 |     statusLabel: t("dashboard.evidence.completed"),
496 |     state: "done" as const,
497 |   },
498 | ];

```

```

499     label: t("dashboard.evidence.analysis"),
500     statusLabel: t("dashboard.evidence.completed"),
501     state: "done" as const,
502   },
503   {
504     label: t("dashboard.evidence.reflection"),
505     statusLabel: t("dashboard.evidence.inProgress"),
506     state: "prog" as const,
507   },
508   {
509     label: t("dashboard.evidence.cqi"),
510     statusLabel: t("dashboard.evidence.pending"),
511     state: "pend" as const,
512   },
513 ];
514
515 return (
516   <div className="xl:grid xl:grid-cols-[minmax(0,1fr)_320px] xl:gap-6">
517     <div className="min-w-0 space-y-6">
518       /* %% Action-hub hero %%%%%%%%%%%%%%% */
519       <Card
520         className="overflow-hidden rounded-xl border-0 text-white shadow-lg"
521         style={{ background: "var(--hero-gradient)" }}
522       >
523         <div className="p-6">
524           <div className="flex items-start justify-between gap-3">
525             <div className="flex items-start gap-3">
526               <span className="inline-flex h-10 w-10 shrink-0 items-center justify-center rounded-xl bg-whit
527                 <Activity className="h-5 w-5" aria-hidden="true" />
528               </span>
529               <div className="min-w-0">
530                 <h1 className="text-xl font-bold tracking-tight">
531                   {t("dashboard.hub.title")}
532                 </h1>
533                 <p className="mt-0.5 text-sm text-white/70">
534                   {t("dashboard.hub.subtitle")}
535                 </p>
536               </div>
537             </div>
538             <span className="hidden shrink-0 items-center gap-1 rounded-full bg-white/10 px-2.5 py-1 text-xs
539               <Info className="h-3.5 w-3.5" aria-hidden="true" />
540             {t("dashboard.hub.whyn")}
541           </span>
542         </div>
543
544         <div className="mt-5 grid gap-3 sm:grid-cols-3">
545           <ActionChip
546             icon={belowTargetCount > 0 ? TrendingDown : CheckCircle2}
547             tone={belowTargetCount > 0 ? "red" : "green"}
548             title={
549               belowTargetCount > 0
550                 ? t("dashboard.hub.ploBelow", { count: belowTargetCount })
551                 : t("dashboard.hub.ploTitle")
552             }
553             cta={t("dashboard.hub.review")}
554             to="/coordinator/plos"
555           </>
556           <ActionChip
557             icon={Grid3X3}
558             tone="amber"
559             title={t("dashboard.hub.gapTitle")}
560             cta={t("dashboard.hub.openMatrix")}
561             to="/coordinator/matrix"
562           </>
563           <ActionChip
564             icon={ShieldCheck}
565             tone="green"
566             title={t("dashboard.hub.accredTitle")}
567             cta={t("dashboard.hub.continueEvidence")}
568             to="/coordinator/accreditation"
569           </>
570         </div>
571       </div>
572     </Card>
573
574     /* %% KPI row (filters) %%%%%%%%%%%%%%% */
575     {kpiLoading ? (
576       <div className="grid grid-cols-2 gap-4 md:grid-cols-4">
577         {Array.from({ length: 4 }).map((_, i) => (
578           <Shimmer key={i} className="h-24 rounded-xl" />
579         ))}
580       </div>
581     ) : (
582       <div className="grid grid-cols-2 gap-4 md:grid-cols-4">
583         <Link
584           to="/coordinator/matrix"
585           aria-label={t("dashboard.kpi.programs")}
586           className="block rounded-xl outline-none focus-visible:ring-2 focus-visible:ring-sky-300"
587         >
588           <KPICard
589             icon={Users}
590             label={t("dashboard.kpi.programs")}
591             value={programsCount}
592           </>
593         </Link>
594         <Link
595           to="/coordinator/plos"
596           aria-label={t("dashboard.kpi.avgPlo")}
597           className="block rounded-xl outline-none focus-visible:ring-2 focus-visible:ring-sky-300"
598         >
599           <KPICard
600             icon={TrendingUp}
601             label={t("dashboard.kpi.avgPlo")}
602             value={`$${avgAttainment}%`}

```

```

603         valueClassName={attainmentValueClass(avgAttainment)}
604     </>
605 </Link>
606 <Link
607   to="/coordinator/gap-analysis"
608   aria-label={t("dashboard.kpi.belowTarget")}
609   className="block rounded-xl outline-none focus-visible:ring-2 focus-visible:ring-sky-300"
610 >
611   <KPICard
612     icon={TrendingDown}
613     label={t("dashboard.kpi.belowTarget")}
614     value={belowTargetCount}
615     {...(belowTargetCount > 0
616       ? {
617           iconBgClass: "bg-red-50",
618           iconColorClass: "text-red-600",
619           valueClassName: "text-red-600",
620         }
621       : {})}
622   />
623 </Link>
624 <Link
625   to="/coordinator/accreditation"
626   aria-label={t("dashboard.kpi.accredReady")}
627   className="block rounded-xl outline-none focus-visible:ring-2 focus-visible:ring-sky-300"
628 >
629   <KPICard
630     icon={ShieldCheck}
631     label={t("dashboard.kpi.accredReady")}
632     value={
633       evidenceReadiness !== null ? `${evidenceReadiness}%` : "-"
634     }
635     iconBgClass="bg-green-50"
636     iconColorClass="text-green-600"
637     valueClassName="text-green-600"
638   />
639 </Link>
640 </div>
641 )}
642
643 /* %% Attainment alerts %%%%%%%%%%%%%%% */
644 <Card className="card-elevated gap-0 border-0 bg-white py-0">
645   <div className="p-6">
646     <SectionHeader
647       icon={AlertTriangle}
648       title={t("dashboard.alerts.title")}
649       action={
650         <Link
651           to="/coordinator/plos"
652           className="inline-flex items-center gap-1 rounded text-xs font-bold text-sky-700 outline-non
653         >
654           {t("dashboard.alerts.allOutcomes")}
655           <ArrowRight className="h-3.5 w-3.5" aria-hidden="true" />
656         </Link>
657       }
658     />
659     {ai.data?.narrative && (
660       <div className="mt-3 flex items-start gap-2 rounded-lg bg-teal-50/60 p-3">
661         <Sparkles
662           className="mt-0.5 h-4 w-4 shrink-0 text-teal-600"
663           aria-hidden="true"
664         />
665         <p className="text-xs text-gray-700">{ai.data.narrative}</p>
666       </div>
667     )}
668     <div className="mt-4 space-y-3">
669       {attainment.isPending ? (
670         <>
671           <Shimmer className="h-28 rounded-xl" />
672           <Shimmer className="h-28 rounded-xl" />
673         </>
674       ) : belowTargetCount > 0 ? (
675         belowTargetPlos.slice(0, 3).map((plo) => {
676           const high = plo.status === "belowTarget";
677           const att = plo.attainment as number;
678           return (
679             <AlertRow
680               key={plo.id}
681               severity={high ? "high" : "med"}
682               title={plo.title}
683               attainment={att}
684               priorityLabel={
685                 high
686                   ? t("dashboard.alerts.priorityHigh")
687                   : t("dashboard.alerts.priorityMedium")
688               }
689               priorityTone={high ? "red" : "amber"}
690               meta={alertMeta(plo)}
691               body={t("dashboard.alerts.gapBody", {
692                 pct: att,
693                 gap: threshold - att,
694                 threshold,
695               })}
696               primaryLabel={t("dashboard.alerts.draftCqi")}
697               primaryTo="/coordinator/cqi"
698               secondaryLabel={t("dashboard.alerts.drillDown")}
699               secondaryTo="/coordinator/plos"
700             />
701           );
702         });
703       ) : (
704         <div className="flex items-start gap-3 rounded-xl border border-green-100 bg-green-50/60 p-4">
705           <CheckCircle2
706             className="mt-0.5 h-5 w-5 shrink-0 text-green-600"

```

```

707 |         aria-hidden="true"
708 |     />
709 |     <div className="min-w-0">
710 |         <p className="text-sm font-bold text-gray-900">
711 |             {t("dashboard.alerts.empty")}
712 |         </p>
713 |         <p className="mt-1 text-xs text-gray-600">
714 |             {t("dashboard.alerts.emptyDetail", {
715 |                 count: measuredPlos.length,
716 |                 threshold,
717 |             })}
718 |         </p>
719 |         {lowestPlo?.attainment != null && (
720 |             <p className="mt-1 text-xs text-gray-500">
721 |                 {t("dashboard.alerts.lowest", {
722 |                     title: lowestPlo.title,
723 |                     pct: lowestPlo.attainment,
724 |                 })}
725 |             </p>
726 |         )}
727 |     </div>
728 | </div>
729 | )}
730 | </div>
731 | </div>
732 | </Card>
733 |
734 | {/* %% Curriculum gap / Evidence / CQI / Program timeline % % % % % % % % */}
735 | <div className="grid grid-cols-1 gap-4 sm:grid-cols-2 xl:grid-cols-4">
736 |     {/* Curriculum gap (presentational – Phase B/C) */}
737 |     <MiniCard>
738 |         title={t("dashboard.gap.title")}
739 |         ctaLabel={t("dashboard.gap.cta")}
740 |         ctaTo="/coordinator/matrix"
741 |     >
742 |         <p className="text-sm font-semibold text-gray-800">
743 |             {t("dashboard.gap.body")}
744 |         </p>
745 |         <p className="mt-1.5 text-xs text-gray-500">
746 |             {t("dashboard.gap.detail")}
747 |         </p>
748 |         <span className="mt-3 inline-block">
749 |             <Pill tone="amber">{t("dashboard.gap.impact")}</Pill>
750 |         </span>
751 |     </MiniCard>
752 |
753 |     {/* Accreditation evidence (presentational – Phase B/C) */}
754 |     <MiniCard>
755 |         title={t("dashboard.evidence.title")}
756 |         headerRight={
757 |             <span className="text-sm font-black text-sky-700">
758 |                 {evidenceReadiness != null ? `${evidenceReadiness}%` : "-"}
759 |             </span>
760 |         }
761 |         ctaLabel={t("dashboard.evidence.cta")}
762 |         ctaTo="/coordinator/accreditation"
763 |     >
764 |         <div>
765 |             className="h-2 w-full overflow-hidden rounded-full bg-slate-100"
766 |             role="progressbar"
767 |             aria-valuenow={evidenceReadiness ?? 0}
768 |             aria-valuemin={0}
769 |             aria-valuemax={100}
770 |             aria-label={t("dashboard.evidence.overall")}
771 |         >
772 |             <div>
773 |                 className="h-full rounded-full"
774 |                 style={{
775 |                     width: `${evidenceReadiness ?? 0}%`,
776 |                     background: "var(--brand-gradient)",
777 |                 }}
778 |             </div>
779 |         </div>
780 |         <ul className="mt-3 space-y-2">
781 |             {evidenceItems.map((item) => (
782 |                 <ChecklistRow>
783 |                     key={item.label}
784 |                     label={item.label}
785 |                     statusLabel={item.statusLabel}
786 |                     state={item.state}
787 |                 </ChecklistRow>
788 |             ))}
789 |         </ul>
790 |     </MiniCard>
791 |
792 |     {/* Close the loop (CQI) – real cqi_action_plans */}
793 |     <MiniCard>
794 |         title={t("dashboard.cqi.title")}
795 |         ctaLabel={t("dashboard.cqi.cta")}
796 |         ctaTo="/coordinator/cqi"
797 |     >
798 |         {cqiQuery.isPending ? (
799 |             <div className="space-y-2">
800 |                 <Shimmer className="h-10 rounded-lg" />
801 |                 <Shimmer className="h-10 rounded-lg" />
802 |                 <Shimmer className="h-10 rounded-lg" />
803 |             </div>
804 |         ) : cqiItems.length > 0 ? (
805 |             <ul className="mt-1">
806 |                 {cqiItems.map((item, i) => (
807 |                     <TimelineItem>
808 |                         key={item.id}
809 |                         date={item.date}
810 |                         title={item.title}

```

```

811         statusLabel={item.statusLabel}
812         tone={item.tone}
813         last={i === cqiItems.length - 1}
814         clamp
815       />
816     ))}
817   </ul>
818 ) : (
819   <p className="text-xs text-gray-500">
820     {t("dashboard.cqi.empty")}
821   </p>
822 )}
823 </MiniCard>
824
825 /* Program timeline – real academic_calendar_events */
826 <MiniCard
827   title={t("dashboard.timeline.title")}
828   ctaLabel={t("dashboard.timeline.cta")}
829   ctaTo="/coordinator/cqi"
830 >
831   {calendarQuery.isPending ? (
832     <div className="space-y-2">
833       <Shimmer className="h-10 rounded-lg" />
834       <Shimmer className="h-10 rounded-lg" />
835       <Shimmer className="h-10 rounded-lg" />
836     </div>
837   ) : programItems.length > 0 ? (
838     <ul className="mt-1">
839       {programItems.map((item, i) => (
840         <TimelineItem
841           key={item.id}
842           date={item.date}
843           title={item.title}
844           statusLabel={item.statusLabel}
845           tone={item.tone}
846           last={i === programItems.length - 1}
847         />
848       ))}
849     </ul>
850   ) : (
851     <p className="text-xs text-gray-500">
852       {t("dashboard.timeline.empty")}
853     </p>
854   )}
855 </MiniCard>
856 </div>
857 </div>
858
859 /* %% Right rail %%%%%%%%%%% */
860 <CoordinatorInsightRail className="mt-6 xl:mt-0" />
861 </div>
862 );
863 };
864
865 export default CoordinatorDashboardNew;
866
```

```
TYPE: .tsx | LINES: 1-149 | SHA256: dcdf7ad50007dbdea6b2642ba40e65eef1ac9dbe2290573d40b9f0d9b0ad1277
```

```

45 | }) => (
46 |   <div className="flex items-center gap-2">
47 |     <Icon className="h-3.5 w-3.5 text-gray-400" aria-hidden="true" />
48 |     <span className="text-[10px] font-black uppercase tracking-widest text-gray-500">
49 |       {children}
50 |     </span>
51 |     {typeof count === "number" && (
52 |       <span className="ms-auto inline-flex h-5 min-w-5 items-center justify-center rounded-full bg-red-100 px-1">
53 |         {count}
54 |       </span>
55 |     )}
56 |   </div>
57 | );
58 |
59 | const CoordinatorInsightRail = ({
60 |   evidenceReadiness,
61 |   className,
62 | }: CoordinatorInsightRailProps) => {
63 |   const { t } = useTranslation("coordinator");
64 |   const { institutionId } = useAuth();
65 |   const accred = useCoordinatorAccreditationReadiness(institutionId);
66 |   // Prop override wins; otherwise use the real readiness; "-" until available.
67 |   const readinessPct =
68 |     evidenceReadiness ?? accred.data?.readinessPercent ?? null;
69 |
70 |   return (
71 |     <aside className={className}>
72 |       <div className="space-y-4">
73 |         {/* Attainment alerts */}
74 |         <Card className="card-elevated gap-0 border-0 bg-white py-0">
75 |           <div className="space-y-3 p-4">
76 |             <RailLabel icon={AlertTriangle} count={2}>
77 |               {t("rail.attainmentAlerts")}
78 |             </RailLabel>
79 |             <ul className="space-y-2.5">
80 |               <li className="flex items-start gap-2">
81 |                 <span
82 |                   className="mt-1.5 h-2 w-2 shrink-0 rounded-full bg-red-500"
83 |                   aria-hidden="true"
84 |                 />
85 |                 <span className="text-xs font-medium text-gray-700">
86 |                   {t("rail.alertPlo")}
87 |                 </span>
88 |               </li>
89 |               <li className="flex items-start gap-2">
90 |                 <span
91 |                   className="mt-1.5 h-2 w-2 shrink-0 rounded-full bg-amber-500"
92 |                   aria-hidden="true"
93 |                 />
94 |                 <span className="text-xs font-medium text-gray-700">
95 |                   {t("rail.alertClo")}
96 |                 </span>
97 |               </li>
98 |             </ul>
99 |             <Link
100 |               to="/coordinator/plos"
101 |               className="inline-flex items-center gap-1 rounded text-xs font-bold text-sky-700 outline-none ho"
102 |             >
103 |               {t("rail.viewAllAlerts")}
104 |               <ArrowRight className="h-3.5 w-3.5" aria-hidden="true" />
105 |             </Link>
106 |           </div>
107 |         </Card>
108 |
109 |         {/* Curriculum gap */}
110 |         <Card className="card-elevated gap-0 border-0 bg-amber-50/60 py-0 ring-1 ring-amber-100">
111 |           <div className="space-y-2.5 p-4">
112 |             <RailLabel icon={Grid3X3}>{t("rail.curriculumGap")}</RailLabel>
113 |             <p className="text-xs text-gray-600">{t("rail.gapBody")}</p>
114 |             <Button variant="tactile" size="sm" className="w-full" asChild>
115 |               <Link to="/coordinator/matrix">
116 |                 {t("rail.openMatrix")}
117 |               <ArrowRight className="h-3.5 w-3.5" aria-hidden="true" />
118 |             </Link>
119 |             </Button>
120 |           </div>
121 |         </Card>
122 |
123 |         {/* Accreditation */}
124 |         <Card className="card-elevated gap-0 border-0 bg-white py-0">
125 |           <div className="space-y-2.5 p-4">
126 |             <RailLabel icon={ShieldCheck}>{t("rail.accreditation")}</RailLabel>
127 |             <div className="flex items-center justify-between">
128 |               <span className="text-xs text-gray-600">
129 |                 {t("rail.evidenceReadiness")}
130 |               </span>
131 |               <span className="text-sm font-black text-sky-700">
132 |                 {readinessPct != null ? `${readinessPct}%` : "-"}
133 |               </span>
134 |             </div>
135 |             <Button variant="tactile" size="sm" className="w-full" asChild>
136 |               <Link to="/coordinator/accreditation">
137 |                 {t("rail.reviewDraft")}
138 |               <ArrowRight className="h-3.5 w-3.5" aria-hidden="true" />
139 |             </Link>
140 |             </Button>
141 |           </div>
142 |         </Card>
143 |       </div>
144 |     </aside>
145 |   );
146 | };
147 |
148 | export default CoordinatorInsightRail;

```


FILE: src/components/shared/CoordinatorOutcomeAttainmentNew.tsx

TYPE: .tsx | LINES: 1-534 | SHA256: ba50e0fd3026d038e9140090c2afb528eda255697aaa7e698757560ea21db0ee

```

1  // =====
2  // CoordinatorOutcomeAttainmentNew – Outcome Attainment (real data, Phase A)
3  // =====
4  //
5  // The ILO !' PLO !' CLO attainment rollup, now driven by REAL data via
6  // `useCoordinatorOutcomeAttainment` (learning_outcomes + outcome_attainment +
7  // outcome_mappings). Each PLO expands to its real contributing CLOs with real
8  // attainment, a computed insight (weakest contributing outcome), a Draft-CQI
9  // action, and evidence links. Weakest course + affected-students are derived
10 // from real attainment.
11 //
12 // Per-term trend sparklines and AI-authored narrative insights are deferred to
13 // Phase C (semester snapshots + AI edge function) – they are omitted here rather
14 // than faked. Composed from the new-UI design tokens/primitives; RTL-safe.
15 // =====
16
17 import { useState } from "react";
18 import { Link } from "react-router-dom";
19 import { useTranslation } from "react-i18next";
20 import {
21   ArrowRight,
22   ChevronDown,
23   ClipboardCheck,
24   FileText,
25   Lightbulb,
26   MessageSquare,
27   Sparkles,
28   TrendingDown,
29   TrendingUp,
30   Workflow,
31 } from "lucide-react";
32
33 import { Button, Card, SectionHeader, Shimmer } from "@design-system";
34 import CoordinatorInsightRail from "@components/shared/CoordinatorInsightRail";
35 import EmptyState from "@components/shared/EmptyState";
36 import { useAuth } from "@hooks/useAuth";
37 import {
38   useCoordinatorOutcomeAttainment,
39   type AttainmentStatus,
40   type AttainmentPLO,
41 } from "@hooks/useCoordinatorOutcomeAttainment";
42 import { useCoordinatorAiInsights } from "@hooks/useCoordinatorAiInsights";
43 import { useCoordinatorAttainmentTrends } from "@hooks/useCoordinatorAttainmentTrends";
44 import { cn } from "@lib/utils";
45
46 /** Bar fill color by OBE attainment band (mirrors MasteryRing thresholds). */
47 const barColor = (pct: number): string => {
48   if (pct >= 85) return "#22c55e";
49   if (pct >= 70) return "#3b82f6";
50   if (pct >= 50) return "#f59e0b";
51   return "#ef4444";
52 };
53
54 const STATUS_PILL: Record<AttainmentStatus, string> = {
55   onTrack: "bg-green-50 text-green-600",
56   watch: "bg-amber-50 text-amber-700",
57   belowTarget: "bg-red-50 text-red-600",
58   none: "bg-slate-100 text-slate-500",
59 };
60
61 const Bar = ({ pct, label }: { pct: number; label: string }) => (
62   <div
63     className="h-2 w-full overflow-hidden rounded-full bg-slate-100"
64     role="progressbar"
65     aria-valuenow={pct}
66     aria-valuemin={0}
67     aria-valuemax={100}
68     aria-label={label}
69   >
70     <div
71       className="h-full rounded-full"
72       style={{ width: `${pct}%`, background: barColor(pct) }}
73     />
74   </div>
75 );
76
77 const CoordinatorOutcomeAttainmentNew = () => {
78   const { t } = useTranslation("coordinator");
79   const { institutionId } = useAuth();
80   const { data, isPending, isError } =
81     useCoordinatorOutcomeAttainment(institutionId);
82   // Additive AI insight (null when the edge function isn't deployed – graceful).
83   const ai = useCoordinatorAiInsights(institutionId);
84   // Per-outcome trend history (empty until "e2 terms are snapshotted – graceful).
85   const trends = useCoordinatorAttainmentTrends(institutionId);
86
87   // "vs last term" delta for an outcome; null until "e2 snapshots exist.
88   const trendDelta = (outcomeId: string): number | null => {
89     const series = trends.data?.[outcomeId];
90     if (!series || series.length < 2) return null;
91     const last = series[series.length - 1];
92     const prev = series[series.length - 2];
93     if (!last || !prev) return null;
94     return last.attainment - prev.attainment;
95   };
96
97   const ilos = data?.ilos ?? [];
98   const plos = data?.plos ?? [];
99   const target = data?.successThreshold ?? 70;

```

```

100 // Single-open accordion; the first PLO is open by default (openOverride=null).
101 const [openOverride, setOpenOverride] = useState<string | null>(null);
102 const firstId = plos[0]?.id;
103 const currentOpen =
104   openOverride === null
105     ? firstId
106     : openOverride === "__none__"
107       ? undefined
108       : openOverride;
109 const toggle = (id: string) =>
110   setOpenOverride(currentOpen === id ? "__none__" : id);
111
112
113 const statusLabel = (s: AttainmentStatus) =>
114   s === "onTrack"
115     ? t("attainment.onTrack")
116     : s === "watch"
117       ? t("attainment.watch")
118       : s === "belowTarget"
119         ? t("attainment.belowTarget")
120         : "-";
121
122 const weakestClo = (plo: AttainmentPLO) => {
123   let weak: { title: string; attainment: number } | null = null;
124   for (const c of plo.contributingClos) {
125     if (c.attainment == null) continue;
126     if (!weak || c.attainment < weak.attainment) {
127       weak = { title: c.title, attainment: c.attainment };
128     }
129   }
130   return weak;
131 };
132
133 return (
134   <div className="xl:grid xl:grid-cols-[minmax(0,1fr)_320px] xl:gap-6">
135     <div className="min-w-0 space-y-6">
136       {/* Header */}
137       <div className="min-w-0">
138         <h1 className="text-2xl font-bold tracking-tight">
139           {t("attainment.title")}
140         </h1>
141         <p className="mt-1 text-sm text-slate-500">
142           {t("attainment.subtitle")}
143         </p>
144       </div>
145
146       {/* %% AI insight (additive; hidden when unavailable) %%%%%%%%%%% */}
147       {ai.data && (
148         <Card className="card-elevated gap-0 border-0 bg-white py-0">
149           <div className="p-5">
150             <div className="flex items-center justify-between gap-2">
151               <div className="flex items-center gap-2">
152                 <span className="inline-flex h-8 w-8 items-center justify-center rounded-lg bg-teal-50 text-
153                   <Sparkles className="h-4 w-4" aria-hidden="true" />
154                 </span>
155                 <h2 className="text-sm font-bold text-gray-900">
156                   {t("attainment.aiInsight")}
157                 </h2>
158               </div>
159               <span className="rounded-full bg-slate-100 px-2 py-0.5 text-[10px] font-bold text-slate-500">
160                 {ai.data.source === "ai"
161                   ? t("attainment.sourceAi")
162                   : t("attainment.sourceComputed")}
163               </span>
164             </div>
165             <p className="mt-2 text-sm text-gray-700">{ai.data.narrative}</p>
166             {ai.data.recommendations.length > 0 && (
167               <ul className="mt-3 space-y-1.5">
168                 {ai.data.recommendations.map((rec, i) => (
169                   <li
170                     key={i}
171                     className="flex items-start gap-2 text-xs text-gray-600"
172                   >
173                     <ArrowRight
174                       className="mt-0.5 h-3.5 w-3.5 shrink-0 text-teal-600"
175                       aria-hidden="true"
176                     />
177                     {rec}
178                   </li>
179                 ))}
180               </ul>
181             )}
182           </div>
183         </Card>
184       )}
185
186       {/* %% ILO cards %%%%%%%%%%% */}
187       <Card className="card-elevated gap-0 border-0 bg-white py-0">
188         <div className="p-6">
189           <SectionHeader
190             icon={Workflow}
191             title={t("attainment.iloSection")}
192             action={
193               <Link
194                 to="/coordinator/outcome-chain?view=chain"
195                 className="inline-flex items-center gap-1 rounded text-xs font-bold text-sky-700 outline-non
196               >
197                 {t("attainment.viewIloDetails")}
198               <ArrowRight className="h-3.5 w-3.5" aria-hidden="true" />
199             </Link>
200             </div>
201         </div>
202         {isPending ? (
203           <div className="mt-4 grid gap-4 sm:grid-cols-3">

```

[illegible]

```

308 <p className="mt-0.5 truncate text-xs text-slate-500">
309   {plo.description}
310 </p>
311 })
312 <div className="mt-1 flex flex-wrap items-center gap-2">
313   <span className="text-[11px] font-semibold text-sky-700">
314     {t("attainment.contributingClos", {
315       count: plo.contributingClos.length,
316     })}
317   </span>
318   {(() => {
319     const delta = trendDelta(plo.id);
320     if (delta === null) return null;
321     const up = delta >= 0;
322     return (
323       <span
324         className={cn(
325           "inline-flex items-center gap-0.5 rounded-full px-1.5 py-0.5 text-[10px] f
326           up
327             ? "bg-green-50 text-green-600"
328             : "bg-red-50 text-red-600"
329         )}
330       >
331         {up ? (
332           <TrendingUp
333             className="h-3 w-3"
334             aria-hidden="true"
335           />
336         ) : (
337           <TrendingDown
338             className="h-3 w-3"
339             aria-hidden="true"
340           />
341         )}
342         {up ? "+" : ""}
343         {delta} {t("attainment.vsLastTerm")}
344       </span>
345     );
346   })()}
347 </div>
348 </div>
349
350 {/* Attainment */}
351 <div className="hidden w-20 shrink-0 text-center sm:block">
352   <p
353     className="text-lg font-black"
354     style={{
355       color:
356         plo.attainment == null
357           ? "#94a3b8"
358           : barColor(plo.attainment),
359     }}
360   >
361     {plo.attainment == null
362       ? "-"
363       : `${plo.attainment}%`}
364   </p>
365   <span
366     className={cn(
367       "rounded-full px-1.5 py-0.5 text-[10px] font-bold",
368       STATUS_PILL[plo.status]
369     )}
370   >
371     {statusLabel(plo.status)}
372   </span>
373 </div>
374
375 {/* Weakest course */}
376 <div className="hidden w-32 shrink-0 lg:block">
377   {plo.weakestCourse ? (
378     <
379       <p className="truncate text-xs font-semibold text-gray-700">
380         {plo.weakestCourse.code}
381       </p>
382       <p
383         className="text-[11px] font-bold"
384         style={{
385           color: barColor(plo.weakestCourse.attainment),
386         }}
387       >
388         {plo.weakestCourse.attainment}%
389       </p>
390     </>
391   ) : (
392     <p className="text-[11px] text-slate-400"></p>
393   )}
394 </div>
395
396 {/* Affected students */}
397 <div className="hidden w-20 shrink-0 text-center md:block">
398   <p className="text-sm font-bold text-gray-800">
399     {plo.affectedStudents}
400   </p>
401   <p className="text-[10px] text-slate-400">
402     {t("attainment.ofCohort", {
403       pct: plo.cohortPercent,
404     })}
405   </p>
406 </div>
407
408 <ChevronDown
409   className={cn(
410     "h-4 w-4 shrink-0 text-slate-400 transition-transform",
411     open && "rotate-180"

```

```

412     })
413     aria-hidden="true"
414   />
415 </button>
416
417 {open && (
418   <div className="grid gap-4 border-t border-slate-200 p-4 lg:grid-cols-3">
419     {/* CLO breakdown (real) */}
420     <div>
421       <p className="text-[10px] font-black uppercase tracking-widest text-slate-400">
422         {t("attainment.cloBreakdown")}
423       </p>
424       <div className="mt-2 space-y-2.5">
425         {plo.contributingClos.length === 0 ? (
426           <p className="text-xs text-slate-400"></p>
427         ) : (
428           plo.contributingClos.map((clo) => (
429             <div key={clo.id}>
430               <span className="flex items-center justify-between gap-2">
431                 <span className="truncate text-xs text-gray-700">
432                   {clo.title}
433                 </span>
434                 <span>
435                   className="text-xs font-bold"
436                   style={{
437                     color:
438                       clo.attainment == null
439                         ? "#94a3b8"
440                         : barColor(clo.attainment),
441                   }}
442                 >
443                   {clo.attainment == null
444                     ? "-"
445                     : `${clo.attainment}%`}
446                 </span>
447               </div>
448               <div className="mt-1">
449                 <Bar>
450                   pct={clo.attainment ?? 0}
451                   label={clo.title}
452                 </>
453               </div>
454             </div>
455           ))
456         )}
457       </div>
458     </div>
459
460     {/* Computed insight (real) + recommended action */}
461     <div className="rounded-xl bg-teal-50/70 p-3 ring-1 ring-teal-100">
462       <p className="flex items-center gap-1 text-[10px] font-black uppercase tracking-wi">
463         <Lightbulb>
464           className="h-3.5 w-3.5"
465           aria-hidden="true"
466         </>
467         {t("attainment.insight")}
468       </p>
469       <p className="mt-1.5 text-xs text-gray-700">
470         {weak
471           ? t("attainment.insightText", {
472             clo: weak.title,
473             pct: weak.attainment,
474           })
475           : t("attainment.insightNone")}
476       </p>
477       <Button>
478         variant="tactile"
479         size="sm"
480         className="mt-3 w-full"
481         asChild
482       >
483         <Link to="/coordinator/cqi">
484           {t("attainment.draftCqiPlan")}
485         </Link>
486       </Button>
487     </div>
488
489     {/* Evidence links */}
490     <div>
491       <p className="text-[10px] font-black uppercase tracking-widest text-slate-400">
492         {t("attainment.evidenceLinks")}
493       </p>
494       <div className="mt-2 space-y-1.5">
495         <Link>
496           to="/coordinator/matrix"
497           className="flex items-center gap-2 rounded text-xs font-semibold text-sky-700"
498         >
499           <FileText>
500             className="h-3.5 w-3.5"
501             aria-hidden="true"
502           </>
503           {t("attainment.evAssessment")}
504         </Link>
505         <Link>
506           to="/coordinator/accreditation"
507           className="flex items-center gap-2 rounded text-xs font-semibold text-sky-700"
508         >
509           <MessageSquare>
510             className="h-3.5 w-3.5"
511             aria-hidden="true"
512           </>
513           {t("attainment.evFeedback")}
514         </Link>
515       </div>

```

```

516 |         </div>
517 |     </div>
518 |     }}
519 | </div>
520 | };
521 | }}
522 | </div>
523 | }}
524 | </div>
525 | </Card>
526 | </div>
527 |
528 | <CoordinatorInsightRail className="mt-6 xl:mt-0" />
529 | </div>
530 | );
531 | };
532 |
533 | export default CoordinatorOutcomeAttainmentNew;
534 |

```

FILE: [src/components/shared/CoordinatorProfileNew.tsx](#)

TYPE: .tsx | LINES: 1-899 | SHA256: 4d0ad73241847f115753cb5b80fece6200b710bc142e9fd04cf72edb36116d1f

```

1  // =====
2  // CoordinatorProfileNew - redesigned coordinator "Me" page (P3, spec task 3.6)
3  // =====
4  //
5  // Repositions the coordinator profile from an analytics view into a
6  // professional profile + workspace-configuration page, matching the prototype
7  // reference: profile header, AI Assistance Preferences, Role & Permissions,
8  // Programs I Manage, Connected Faculty, Notification Preferences, Security &
9  // Access, Connected Integrations, and Profile & Academic Information.
10 //
11 // Gated behind `newUiModules` for the COORDINATOR role only (wrapper in
12 // pages/shared/ProfilePage.tsx); the flag-off path keeps the shared
13 // ProfilePage (avatar upload, appearance/theme, email preferences) intact and
14 // reversible.
15 //
16 // DATA: name / email / avatar come from `useAuth` (REAL). The header count
17 // tiles (programs / courses / students / faculty), the "Programs I Manage"
18 // list, and the Connected Faculty active/inactive split are REAL, aggregated
19 // by `useCoordinatorProfileStats` (coordinator-scoped reads of programs /
20 // courses / student_courses / profiles - no writes). The remaining sections
21 // (designation, contact details, AI scope, permissions, notification toggles,
22 // security, integrations, academic info) are PRESENTATIONAL sample content
23 // pending Phase B/C. Notification toggles are local-only (preview). Composed
24 // from tokens + primitives; RTL-safe via logical props.
25 // =====
26
27 import { useState } from "react";
28 import { useTranslation } from "react-i18next";
29 import type { ComponentType } from "react";
30 import { useForm } from "react-hook-form";
31 import { zodResolver } from "@hookform/resolvers/zod";
32 import { toast } from "sonner";
33 import {
34   AlertTriangle,
35   ArrowRight,
36   Award,
37   Bot,
38   Building2,
39   Calendar,
40   Check,
41   ChevronRight,
42   Clock,
43   FileCheck2,
44   GraduationCap,
45   Grid3X3,
46   KeyRound,
47   Loader2,
48   Mail,
49   MapPin,
50   MessageSquare,
51   Monitor,
52   Phone,
53   ShieldCheck,
54   Smartphone,
55   User,
56   UserCog,
57   Users,
58   X,
59 } from "lucide-react";
60
61 import { Card } from "@/components/ui/card";
62 import { Button } from "@/components/ui/button";
63 import {
64   Dialog,
65   DialogContent,
66   DialogFooter,
67   DialogHeader,
68   DialogTitle,
69 } from "@/components/ui/dialog";
70 import {
71   Form,
72   FormControl,
73   FormField,
74   FormItem,
75   FormLabel,
76   FormMessage,
77 } from "@/components/ui/form";
78 import { Input } from "@/components/ui/input";
79 import { Shimmer } from "@/design-system";
80 import { useAuth } from "@/hooks/useAuth";
81 import { useCoordinatorProfileStats } from "@/hooks/useCoordinatorProfileStats";

```

```
82 import {  
83   useCoordinatorAcademicProfile,  
84   useUpdateCoordinatorAcademicProfile,  
85   useUpdateCoordinatorAlertPrefs,  
86   readCoordinatorAlertPrefs,  
87   type CoordinatorAlertPrefs,  
88 } from "@hooks/useCoordinatorProfileSettings";  
89 import { useConnectedIntegrations } from "@hooks/useConnectedIntegrations";  
90 import {  
91   coordinatorAcademicSchema,  
92   type CoordinatorAcademicFormValues,  
93 } from "@lib/schemas/coordinatorAcademic";  
94 import { getDisplayFirstName } from "@lib/displayName";  
95 import { cn } from "@lib/utils";  
96  
97 type IconType = ComponentType<{ className?: string }>;  
98  
99 // % % Section card (white surface + inline header) % % % % % % % % % % % % % % % % % % % % % %  
100 const SectionCard = ({  
101   title,  
102   subtitle,  
103   action,  
104   children,  
105 }): {  
106   title: string;  
107   subtitle?: string;  
108   action?: { label: string; onClick?: () => void };  
109   children: React.ReactNode;  
110 }) => (  
111   <Card className="card-elevated gap-0 border-0 bg-white py-0">  
112     <div className="flex flex-col p-5">  
113       <div className="flex items-start justify-between gap-3">  
114         <div className="min-w-0">  
115           <h2 className="text-base font-bold tracking-tight text-gray-900">  
116             {title}  
117           </h2>  
118           {subtitle && (  
119             <p className="mt-0.5 text-xs text-slate-500">{subtitle}</p>  
120           )}  
121         </div>  
122       </div>  
123       <div className="mt-4 flex-1">{children}</div>  
124       {action && (  
125         <button  
126           type="button"  
127           onClick={action.onClick}  
128           className="mt-4 inline-flex items-center gap-1 self-start rounded text-xs font-bold text-sky-700 out  
129         >  
130           {action.label}  
131           <ArrowRight className="h-3.5 w-3.5" aria-hidden="true" />  
132         </button>  
133       )}  
134     </div>  
135   </Card>  
136 );  
137  
138 // % % Capability row (allowed / restricted) % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
139 const CapabilityRow = ({  
140   label,  
141   allowed,  
142 }): {  
143   label: string;  
144   allowed: boolean;  
145 }) => (  
146   <li className="flex items-center gap-2 text-sm text-gray-700">  
147     {allowed ? (  
148       <Check className="h-4 w-4 shrink-0 text-green-600" aria-hidden="true" />  
149     ) : (  
150       <X className="h-4 w-4 shrink-0 text-red-500" aria-hidden="true" />  
151     )}  
152     <span className={cn(!allowed && "text-slate-400")}>{label}</span>  
153   </li>  
154 );  
155  
156 // % % Contact chip % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
157 const ContactChip = ({  
158   icon: Icon,  
159   text,  
160 }): {  
161   icon: IconType;  
162   text: string;  
163 }) => (  
164   <span className="inline-flex items-center gap-2 text-sm text-slate-600">  
165     <Icon className="h-4 w-4 shrink-0 text-slate-400" />  
166     {text}  
167   </span>  
168 );  
169  
170 // % % Header stat tile % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
171 const StatTile = ({  
172   icon: Icon,  
173   value,  
174   label,  
175 }): {  
176   icon: IconType;  
177   value: number;  
178   label: string;  
179 }) => (  
180   <div className="flex items-center gap-2.5">  
181     <span className="inline-flex h-9 w-9 shrink-0 items-center justify-center rounded-lg bg-blue-50 text-blue-<br>182     <Icon className="h-4 w-4" />  
183   </span>  
184   <div className="min-w-0">  
185     <p className="text-lg font-black leading-none text-sky-700">{value}</p>
```

[illegible]


```

290 // connecting needs provider OAuth (external), so the connect button is a stub.
291 const integrations = useConnectedIntegrations(userId);
292 const integrationStatus = integrations.data ?? {};
293
294 // Real academic profile (migration 20260823000001; graceful "not set").
295 const academicQuery = useCoordinatorAcademicProfile(userId);
296 const academic = academicQuery.data;
297 const academicMutation = useUpdateCoordinatorAcademicProfile();
298 const [editOpen, setEditOpen] = useState(false);
299
300 // Coordinator alert notification prefs - persisted in
301 // profiles.notification_preferences with optimistic local state.
302 const alertMutation = useUpdateCoordinatorAlertPrefs();
303 const [notifis, setNotifis] = useState<CoordinatorAlertPrefs>(() =>
304   readCoordinatorAlertPrefs(profile?.notification_preferences)
305 );
306 const toggle = (key: keyof CoordinatorAlertPrefs) => {
307   if (!userId) return;
308   const next = { ...notifis, [key]: !notifis[key] };
309   setNotifis(next); // optimistic
310   alertMutation.mutate(
311     { userId, prefs: next },
312     {
313       onError: () => {
314         setNotifis((cur) => ({ ...cur, [key]: !cur[key] })); // revert
315         toast.error(t("me.notifSaveError"));
316       },
317     },
318   );
319 };
320
321 // Academic edit form.
322 const form = useForm<CoordinatorAcademicFormValues>({
323   resolver: zodResolver(coordinatorAcademicSchema),
324   defaultValues: {
325     department: "",
326     designation: "",
327     academic_rank: "",
328     highest_degree: "",
329     years_experience: "",
330   },
331 });
332
333 const openEdit = () => {
334   form.reset({
335     department: academic?.department ?? "",
336     designation: academic?.designation ?? "",
337     academic_rank: academic?.academic_rank ?? "",
338     highest_degree: academic?.highest_degree ?? "",
339     years_experience:
340       academic?.years_experience !== null
341         ? String(academic.years_experience)
342         : "",
343   });
344   setEditOpen(true);
345 };
346
347 const onSubmitAcademic = (values: CoordinatorAcademicFormValues) => {
348   if (!userId) return;
349   academicMutation.mutate(
350     {
351       userId,
352       values: {
353         department: values.department.trim() || null,
354         designation: values.designation.trim() || null,
355         academic_rank: values.academic_rank.trim() || null,
356         highest_degree: values.highest_degree.trim() || null,
357         years_experience:
358           values.years_experience.trim() === ""
359             ? null
360             : Number(values.years_experience),
361       },
362     },
363     {
364       onSuccess: () => {
365         toast.success(t("me.academicSaved"));
366         setEditOpen(false);
367       },
368       onError: () => toast.error(t("me.academicSaveError")),
369     },
370   );
371 };
372
373 const aiAllowed = [
374   t("me.aiAllow1"),
375   t("me.aiAllow2"),
376   t("me.aiAllow3"),
377   t("me.aiAllow4"),
378 ];
379 const aiRestricted = [t("me.aiDeny1"), t("me.aiDeny2")];
380 const permissions = [
381   t("me.perm1"),
382   t("me.perm2"),
383   t("me.perm3"),
384   t("me.perm4"),
385   t("me.perm5"),
386 ];
387
388 return (
389   <div className="space-y-6">
390     /* % Profile header % % % % % % % % % % % % % % % % % % % % % % % */
391     <Card className="card-elevated gap-0 border-0 bg-white py-0">
392       <div className="flex flex-col gap-5 p-6 lg:flex-row lg:items-start lg:justify-between">
393         <div className="flex items-start gap-4">

```

[illegible]

```

498         {aiRestricted.map((label) => (
499             <CapabilityRow key={label} label={label} allowed={false} />
500         ))}
501     </ul>
502 </SectionCard>
503
504 {/* Role & Permissions */}
505 <SectionCard
506     title={t("me.roleSectionTitle")}
507     subtitle={t("me.roleSectionSubtitle")}
508     action={{ label: t("me.roleViewAll") }}
509 >
510     <div className="flex items-center gap-2 rounded-xl bg-slate-50 p-3">
511         <ShieldCheck className="h-5 w-5 text-blue-600" />
512         <p className="text-sm font-bold text-gray-900">
513             {t("me.roleName")}
514         </p>
515     </div>
516     <ul className="mt-3 space-y-1.5">
517         {permissions.map((label) => (
518             <CapabilityRow key={label} label={label} allowed />
519         ))}
520     </ul>
521 </SectionCard>
522 </div>
523
524 <div className="grid gap-6 md:grid-cols-2">
525     {/* Programs I Manage */}
526     <SectionCard
527         title={t("me.programsTitle")}
528         subtitle={t("me.programsSubtitle")}
529         action={{ label: t("me.programsViewAll") }}
530 >
531         {stats.isPending ? (
532             <div className="space-y-2">
533                 <Shimmer className="h-16 rounded-xl" />
534                 <Shimmer className="h-16 rounded-xl" />
535             </div>
536         ) : managedPrograms.length > 0 ? (
537             <ul className="space-y-2">
538                 {managedPrograms.map((p) => (
539                     <li key={p.id}>
540                         <button
541                             type="button"
542                             className="flex w-full items-center gap-3 rounded-xl border border-slate-100 p-3 text-
543                         >
544                             <span className="inline-flex h-9 w-9 shrink-0 items-center justify-center rounded-lg b
545                                 <GraduationCap
546                                     className="h-4 w-4"
547                                     aria-hidden="true"
548                                 />
549                             </span>
550                             <span className="min-w-0 flex-1">
551                                 <span className="block truncate text-sm font-semibold text-gray-900">
552                                     {p.name}
553                                 </span>
554                                 <span className="block text-[11px] text-slate-400">
555                                     {p.courseCount} {t("me.courses")} · {p.studentCount} {" "}
556                                     {t("me.students")}
557                                 </span>
558                             </span>
559                             <ChevronRight
560                                 className="h-4 w-4 shrink-0 text-slate-300"
561                                 aria-hidden="true"
562                             />
563                         </button>
564                     </li>
565                 ))}
566             </ul>
567         ) : (
568             <p className="text-sm text-slate-500">
569                 {t("me.programsEmpty")}
570             </p>
571         )}
572     </SectionCard>
573
574 {/* Connected Faculty */}
575 <SectionCard
576     title={t("me.facultyTitle")}
577     subtitle={t("me.facultySubtitle")}
578     action={{ label: t("me.facultyManage") }}
579 >
580     <div className="flex items-center gap-3">
581         <span className="inline-flex h-12 w-12 items-center justify-center rounded-xl bg-blue-50 text-
582             <Users className="h-6 w-6" />
583         </span>
584         <div>
585             <p className="text-2xl font-black leading-none text-sky-700">
586                 {totals?.faculty ?? 0}
587             </p>
588             <p className="mt-0.5 text-xs text-slate-500">
589                 {t("me.facultyMembers")}
590             </p>
591         </div>
592     </div>
593     <div className="mt-4 grid grid-cols-2 gap-3">
594         <div className="rounded-xl border border-slate-100 p-3">
595             <p className="text-lg font-black text-green-600">
596                 {totals?.facultyActive ?? 0}
597             </p>
598             <p className="text-[11px] text-slate-500">
599                 {t("me.facultyActive")}
600             </p>
601         </div>

```

```
602     <div className="rounded-xl border border-slate-100 p-3">  
603       <p className="text-lg font-black text-slate-500">  
604         {totals?.facultyInactive ?? 0}  
605       </p>  
606       <p className="text-[11px] text-slate-500">  
607         {t("me.facultyInactive")}  
608       </p>  
609     </div>  
610   </div>  
611 </SectionCard>  
612 </div>  
613  
614 {/* Profile & Academic Information */}  
615 <SectionCard  
616   title={t("me.academicTitle")}  
617   action={{ label: t("me.academicEdit"), onClick: openEdit }}  
618 >  
619   <div className="grid grid-cols-1 gap-4 sm:grid-cols-2 lg:grid-cols-3">  
620     <InfoItem  
621       icon={Building2}  
622       label={t("me.academicDepartment")}  
623       value={academic?.department ?? t("me.notSet")}  
624     />  
625     <InfoItem  
626       icon={User}  
627       label={t("me.academicDesignation")}  
628       value={academic?.designation ?? t("me.notSet")}  
629     />  
630     <InfoItem  
631       icon={Award}  
632       label={t("me.academicRank")}  
633       value={academic?.academic_rank ?? t("me.notSet")}  
634     />  
635     <InfoItem  
636       icon={GraduationCap}  
637       label={t("me.academicDegree")}  
638       value={academic?.highest_degree ?? t("me.notSet")}  
639     />  
640     <InfoItem  
641       icon={Clock}  
642       label={t("me.academicExperience")}  
643       value={  
644         academic?.years_experience != null  
645           ? t("me.experienceYears", {  
646             count: academic.years_experience,  
647           })  
648           : t("me.notSet")  
649       }  
650     />  
651   </div>  
652 </SectionCard>  
653 </div>  
654  
655 {/* % % Right rail % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % */  
656 <div className="space-y-6">  
657   {/* Notification Preferences */}  
658   <SectionCard  
659     title={t("me.notifyTitle")}  
660     subtitle={t("me.notifySubtitle")}  
661     action={{ label: t("me.notifyManage") }}  
662   >  
663     <div className="space-y-3">  
664       <ToggleRow  
665         icon={AlertTriangle}  
666         label={t("me.notify1")}  
667         on={notifs.ploDrop}  
668         onToggle={() => toggle("ploDrop")}  
669       />  
670       <ToggleRow  
671         icon={Grid3X3}  
672         label={t("me.notify2")}  
673         on={notifs.curriculumGap}  
674         onToggle={() => toggle("curriculumGap")}  
675       />  
676       <ToggleRow  
677         icon={FileCheck2}  
678         label={t("me.notify3")}  
679         on={notifs.evidenceReady}  
680         onToggle={() => toggle("evidenceReady")}  
681       />  
682       <ToggleRow  
683         icon={User}  
684         label={t("me.notify4")}  
685         on={notifs.teacherInactivity}  
686         onToggle={() => toggle("teacherInactivity")}  
687       />  
688       <ToggleRow  
689         icon={Clock}  
690         label={t("me.notify5")}  
691         on={notifs.cqiDeadline}  
692         onToggle={() => toggle("cqiDeadline")}  
693       />  
694     </div>  
695   </SectionCard>  
696  
697   {/* Security & Access */}  
698   <SectionCard  
699     title={t("me.securityTitle")}  
700     action={{ label: t("me.securitySettings") }}  
701   >  
702     <div className="space-y-0.5">  
703       <LinkRow icon={KeyRound} label={t("me.changePassword")} />  
704       <LinkRow  
705         icon={ShieldCheck}
```

```

706         label={t("me.twoFactor")}}
707         trailing={
708             <span className="rounded-full bg-green-50 px-2 py-0.5 text-[11px] font-bold text-green-600">
709                 {t("me.on")}
710             </span>
711         }
712     </>
713     <LinkRow icon={Monitor} label={t("me.activeSessions")} />
714     <LinkRow icon={Smartphone} label={t("me.authorizedDevices")} />
715 </div>
716 </SectionCard>
717
718 {/* Connected Integrations */}
719 <SectionCard
720     title={t("me.integrationsTitle")}
721     action={{ label: t("me.integrationsManage") }}
722 >
723     <div className="space-y-2">
724         <IntegrationRow
725             icon={Calendar}
726             label={t("me.googleCalendar")}
727             connected={integrationStatus["google_calendar"] === "connected"}
728             connectedLabel={t("me.connected")}
729             connectLabel={t("me.connect")}
730             onConnect={() => toast.info(t("me.integrationComingSoon"))}
731         />
732         <IntegrationRow
733             icon={Mail}
734             label={t("me.outlook")}
735             connected={integrationStatus["outlook"] === "connected"}
736             connectedLabel={t("me.connected")}
737             connectLabel={t("me.connect")}
738             onConnect={() => toast.info(t("me.integrationComingSoon"))}
739         />
740         <IntegrationRow
741             icon={MessageSquare}
742             label={t("me.slack")}
743             connected={integrationStatus["slack"] === "connected"}
744             connectedLabel={t("me.connected")}
745             connectLabel={t("me.connect")}
746             onConnect={() => toast.info(t("me.integrationComingSoon"))}
747         />
748     </div>
749 </SectionCard>
750 </div>
751 </div>
752
753 {/* Academic edit dialog */}
754 <Dialog open={editOpen} onOpenChange={setEditOpen}>
755     <DialogContent className="sm:max-w-lg">
756         <DialogHeader>
757             <DialogTitle>{t("me.academicEdit")}</DialogTitle>
758         </DialogHeader>
759         <Form {...form}>
760             <form
761                 onSubmit={form.handleSubmit(onSubmitAcademic)}
762                 className="space-y-4">
763             >
764                 <div className="grid gap-4 sm:grid-cols-2">
765                     <FormField
766                         control={form.control}
767                         name="department"
768                         render={({ field }) => (
769                             <FormItem>
770                                 <FormLabel>{t("me.academicDepartment")}</FormLabel>
771                                 <FormControl>
772                                     <Input {...field} />
773                                 </FormControl>
774                                 <FormMessage />
775                             </FormItem>
776                         )}
777                     />
778                     <FormField
779                         control={form.control}
780                         name="designation"
781                         render={({ field }) => (
782                             <FormItem>
783                                 <FormLabel>{t("me.academicDesignation")}</FormLabel>
784                                 <FormControl>
785                                     <Input {...field} />
786                                 </FormControl>
787                                 <FormMessage />
788                             </FormItem>
789                         )}
790                     />
791                     <FormField
792                         control={form.control}
793                         name="academic_rank"
794                         render={({ field }) => (
795                             <FormItem>
796                                 <FormLabel>{t("me.academicRank")}</FormLabel>
797                                 <FormControl>
798                                     <Input {...field} />
799                                 </FormControl>
800                                 <FormMessage />
801                             </FormItem>
802                         )}
803                     />
804                     <FormField
805                         control={form.control}
806                         name="highest_degree"
807                         render={({ field }) => (
808                             <FormItem>
809                                 <FormLabel>{t("me.academicDegree")}</FormLabel>

```



```

11 | CorrelationConfidenceLevel,
12 | { label: string; className: string }
13 | > = {
14 |   early_pattern: {
15 |     label: "Early Pattern",
16 |     className: "bg-amber-50 text-amber-700 border-amber-200",
17 |   },
18 |   emerging_trend: {
19 |     label: "Emerging Trend",
20 |     className: "bg-blue-50 text-blue-700 border-blue-200",
21 |   },
22 |   strong_pattern: {
23 |     label: "Strong Pattern",
24 |     className: "bg-green-50 text-green-700 border-green-200",
25 |   },
26 | };
27 |
28 | const CorrelationConfidenceBadge = ({
29 |   level,
30 |   dataPointCount,
31 | }: CorrelationConfidenceBadgeProps) => {
32 |   const config = CONFIDENCE_CONFIG[level] ?? {
33 |     label: "Unknown",
34 |     className: "bg-slate-50 text-slate-700 border-slate-200",
35 |   };
36 |
37 |   return (
38 |     <Badge
39 |       variant="outline"
40 |       className={cn(config.className)}
41 |       data-testid={`confidence-badge-${level}`}
42 |     >
43 |       {config.label} ({dataPointCount} days)
44 |     </Badge>
45 |   );
46 | };
47 |
48 | export default CorrelationConfidenceBadge;
49 |

```

FILE: src/components/shared/CorrelationDisclaimer.tsx

TYPE: .tsx | LINES: 1-17 | SHA256: f35bb61356a5003664d4380046f2b86091756d85389f06666e2730a20dd99c54

```

1 | import { Info } from "lucide-react";
2 |
3 | const CorrelationDisclaimer = () => (
4 |   <div
5 |     className="flex items-start gap-3 rounded-xl bg-slate-50 border border-slate-200 p-3"
6 |     data-testid="correlation-disclaimer"
7 |   >
8 |     <Info className="h-4 w-4 text-slate-500 shrink-0 mt-0.5" />
9 |     <p className="text-xs text-slate-600">
10 |       These insights show patterns in your data, not cause-and-effect
11 |       relationships. Many factors influence academic performance.
12 |     </p>
13 |   </div>
14 | );
15 |
16 | export default CorrelationDisclaimer;
17 |

```

FILE: src/components/shared/CorrelationInsightCard.tsx

TYPE: .tsx | LINES: 1-109 | SHA256: b268a02c7ad328faa76b4bd6fe1ad8360153db0e61c837115c16eb7653992247

```

1 | import {
2 |   Brain,
3 |   Droplets,
4 |   Dumbbell,
5 |   Moon,
6 |   Login,
7 |   Send,
8 |   BookOpen,
9 |   BookMarked,
10 |   type LucideIcon,
11 | } from "lucide-react";
12 | import { Card } from "@components/ui/card";
13 | import { cn } from "@lib/utils";
14 | import CorrelationConfidenceBadge from "@components/shared/CorrelationConfidenceBadge";
15 | import type {
16 |   CorrelationInsight,
17 |   CorrelationInsightWithConfidence,
18 |   HabitType,
19 | } from "@types/habits";
20 |
21 | export interface CorrelationInsightCardProps {
22 |   insight: CorrelationInsight | CorrelationInsightWithConfidence;
23 | }
24 |
25 | const HABIT_ICONS: Record<HabitType, LucideIcon> = {
26 |   meditation: Brain,
27 |   hydration: Droplets,
28 |   exercise: Dumbbell,
29 |   sleep: Moon,
30 |   login: Login,
31 |   submit: Send,
32 |   journal: BookOpen,
33 |   read: BookMarked,
34 | };
35 |
36 | const HABIT_COLORS: Record<HabitType, string> = {
37 |   meditation: "bg-purple-50 text-purple-600",
38 |   hydration: "bg-blue-50 text-blue-600",
39 |   exercise: "bg-green-50 text-green-600",
40 |   sleep: "bg-indigo-50 text-indigo-600",

```

```

41 | login: "bg-teal-50 text-teal-600",
42 | submit: "bg-amber-50 text-amber-600",
43 | journal: "bg-rose-50 text-rose-600",
44 | read: "bg-cyan-50 text-cyan-600",
45 | };
46 |
47 | const CorrelationInsightCard = ({ insight }: CorrelationInsightCardProps) => {
48 |   const Icon = HABIT_ICONS[insight.habitType] ?? Brain;
49 |   const colorClasses =
50 |     HABIT_COLORS[insight.habitType] ?? "bg-slate-50 text-slate-600";
51 |   const strengthPercent = Math.round(insight.strength * 100);
52 |
53 |   const hasConfidence =
54 |     "confidenceLevel" in insight &&
55 |     insight.confidenceLevel != null &&
56 |     "dataPointCount" in insight;
57 |
58 |   return (
59 |     <Card
60 |       className="bg-white border-0 shadow-sm rounded-xl p-4"
61 |       data-testid={`correlation-insight-${insight.id}`}
62 |     >
63 |       <div className="flex items-start gap-3">
64 |         <div className={cn("p-2 rounded-lg shrink-0", colorClasses)}>
65 |           <Icon className="h-4 w-4" />
66 |         </div>
67 |         <div className="flex-1 min-w-0 space-y-2">
68 |           <p className="text-sm font-medium text-gray-800">
69 |             {insight.description}
70 |           </p>
71 |           {hasConfidence && (
72 |             <CorrelationConfidenceBadge
73 |               level={
74 |                 (insight as CorrelationInsightWithConfidence).confidenceLevel
75 |               }
76 |               dataPointCount={
77 |                 (insight as CorrelationInsightWithConfidence).dataPointCount
78 |               }
79 |             >
80 |           )}
81 |           {/* Strength indicator */}
82 |           <div className="space-y-1">
83 |             <div className="flex items-center justify-between">
84 |               <span className="text-[10px] font-bold tracking-widest uppercase text-gray-400">
85 |                 Strength
86 |               </span>
87 |               <span className="text-xs font-semibold text-gray-600">
88 |                 {strengthPercent}%
89 |               </span>
90 |             </div>
91 |             <div className="h-1.5 w-full rounded-full bg-slate-100 overflow-hidden">
92 |               <div
93 |                 className="h-full rounded-full transition-all duration-300"
94 |                 style={{
95 |                   width: `${strengthPercent}%`,
96 |                   background: "var(--brand-gradient)",
97 |                 }}
98 |                 data-testid="strength-bar"
99 |               </div>
100 |             </div>
101 |           </div>
102 |         </div>
103 |       </div>
104 |     </Card>
105 |   );
106 | };
107 |
108 | export default CorrelationInsightCard;
109 |

```

FILE: src/components/shared/CosmeticPreview.tsx

TYPE: .tsx | LINES: 1-174 | SHA256: 4e3c158c1abefa7b1db4db77e0a2d2e06e7236e9ca707f16e704bad2561a80b0

```

1 | // =====
2 | // CosmeticPreview - Theme/frame/title preview component
3 | // =====
4 |
5 | import { Palette, Frame, Crown, Sparkles } from "lucide-react";
6 | import { resolveThemeCSS, resolveFrameCSS } from "@lib/themeResolver";
7 | import type { ThemeMetadata, FrameMetadata } from "@lib/themeResolver";
8 | import { cn } from "@lib/Utils";
9 |
10 | interface CosmeticPreviewProps {
11 |   subCategory: string;
12 |   metadata: Record<string, unknown>;
13 |   name: string;
14 |   className?: string;
15 | }
16 |
17 | const CosmeticPreview = ({
18 |   subCategory,
19 |   metadata,
20 |   name,
21 |   className,
22 | }: CosmeticPreviewProps) => {
23 |   if (subCategory === "profile_theme") {
24 |     return (
25 |       <ThemePreview metadata={metadata} name={name} className={className} />
26 |     );
27 |   }
28 |
29 |   if (subCategory === "avatar_frame") {
30 |     return (
31 |       <FramePreview metadata={metadata} name={name} className={className} />

```


[illegible]


```

62 | >
63 | <div className="p-5 pb-4">
64 |   <SectionHeader icon={BookOpen} title="Study Time by Course" />
65 | </div>
66 |
67 | <div className="px-5 pb-5">
68 |   {chartData.length === 0 ? (
69 |     <p className="text-sm text-gray-400 text-center py-8">
70 |       No study data for this week
71 |     </p>
72 |   ) : (
73 |     <ResponsiveContainer
74 |       width="100%"
75 |       height={Math.max(chartData.length * 50, 150)}
76 |     >
77 |       <BarChart
78 |         data={chartData}
79 |         layout="vertical"
80 |         margin={{ top: 4, right: 20, bottom: 4, left: 0 }}
81 |       >
82 |         <CartesianGrid strokeDasharray="3 3" horizontal={false} />
83 |         <XAxis
84 |           type="number"
85 |           tick={{ fontSize: 11 }}
86 |           tickLine={false}
87 |           axisLine={false}
88 |           tickFormatter={(v: number) => `${v}h`}
89 |         />
90 |         <YAxis
91 |           type="category"
92 |           dataKey="courseName"
93 |           tick={{ fontSize: 11 }}
94 |           tickLine={false}
95 |           axisLine={false}
96 |           width={120}
97 |         />
98 |         <Tooltip
99 |           formatter={
100 |             ((value: number | undefined) => [
101 |               `${value ?? 0}h`,
102 |               "Study Time",
103 |             ]) as never
104 |           }
105 |           contentStyle={{ borderRadius: 8, fontSize: 12 }}
106 |         />
107 |         <Bar
108 |           dataKey="hours"
109 |           fill={`url(#${gradientId})`}
110 |           radius={[0, 4, 4, 0]}
111 |           maxBarSize={24}
112 |         />
113 |         <defs>
114 |           <linearGradient id={gradientId} x1="0" y1="0" x2="1" y2="0">
115 |             <stop offset="0%" stopColor="#14B8A6" />
116 |             <stop offset="100%" stopColor="#0382BD" />
117 |           </linearGradient>
118 |         </defs>
119 |       </BarChart>
120 |     </ResponsiveContainer>
121 |   )}
122 | </div>
123 | </PCard>
124 | );
125 | };
126 |
127 | export default CourseStudyBreakdown;
128 | export type { CourseStudyBreakdownProps };
129 |

```

FILE: [src/components/shared/CQIStatusBadge.tsx](#)

TYPE: .tsx | LINES: 1-53 | SHA256: 7fa0ae8bc5dcf6ef24e0c9878ddc0dc46a456c1e8d7dca33682924a47e64ac4a

```

1 | // =====
2 | // CQIStatusBadge - CQI action plan status badge
3 | // planned / in_progress / completed / evaluated
4 | // =====
5 |
6 | import { cn } from "@lib/utils";
7 |
8 | type CQIStatus = "planned" | "in_progress" | "completed" | "evaluated";
9 |
10 | interface CQIStatusBadgeProps {
11 |   status: CQIStatus;
12 |   className?: string;
13 | }
14 |
15 | const CQI_STATUS_STYLES: Record<
16 |   CQIStatus,
17 |   { bg: string; text: string; label: string }
18 | > = {
19 |   planned: { bg: "bg-gray-100", text: "text-gray-700", label: "Planned" },
20 |   in_progress: {
21 |     bg: "bg-blue-100",
22 |     text: "text-blue-700",
23 |     label: "In Progress",
24 |   },
25 |   completed: { bg: "bg-green-100", text: "text-green-700", label: "Completed" },
26 |   evaluated: {
27 |     bg: "bg-purple-100",
28 |     text: "text-purple-700",
29 |     label: "Evaluated",
30 |   },
31 | };
32 |

```

```

33 | const CQIStatusBadge = ({ status, className }: CQIStatusBadgeProps) => {
34 |   const style = CQI_STATUS_STYLES[status];
35 |
36 |   return (
37 |     <span
38 |       className={cn(
39 |         "inline-flex items-center rounded-full px-2.5 py-0.5 text-xs font-bold tracking-wide uppercase",
40 |         style.bg,
41 |         style.text,
42 |         className
43 |       )}
44 |       >
45 |       {style.label}
46 |     </span>
47 |   );
48 | };
49 |
50 | export default CQIStatusBadge;
51 | export { CQI_STATUS_STYLES };
52 | export type { CQIStatusBadgeProps, CQIStatus };
53 |

```

FILE: src/components/shared/CreateSessionDialog.tsx

TYPE: .tsx | LINES: 1-326 | SHA256: 172d735b3720e650ca9e2d519dc1e20a7a1c32e5e459f4db226f365c5030a78e

```

1 | // =====
2 | // CreateSessionDialog – Shadcn Dialog with React Hook Form + Zod for creating
3 | // a new study session: title, date, start time, duration (15-min increments),
4 | // course select, CLO multi-select, description
5 | // =====
6 |
7 | import { useForm } from "react-hook-form";
8 | import { zodResolver } from "@hookform/resolvers/zod";
9 | import {
10 |   createStudySessionSchema,
11 |   type CreateStudySessionInput,
12 | } from "@lib/schemas/planner";
13 | import {
14 |   Dialog,
15 |   DialogContent,
16 |   DialogHeader,
17 |   DialogTitle,
18 |   DialogDescription,
19 |   DialogFooter,
20 | } from "@components/ui/dialog";
21 | import {
22 |   Form,
23 |   FormField,
24 |   FormItem,
25 |   FormLabel,
26 |   FormControl,
27 |   FormMessage,
28 | } from "@components/ui/form";
29 | import { Input } from "@components/ui/input";
30 | import { Button } from "@components/ui/button";
31 | import { Textarea } from "@components/ui/textarea";
32 | import {
33 |   Select,
34 |   SelectContent,
35 |   SelectItem,
36 |   SelectTrigger,
37 |   SelectValue,
38 | } from "@components/ui/select";
39 | import { Checkbox } from "@components/ui/checkbox";
40 | import { Label } from "@components/ui/label";
41 | import { Loader2, CalendarDays } from "lucide-react";
42 |
43 | interface CourseOption {
44 |   id: string;
45 |   name: string;
46 |   clos?: Array<{ id: string; title: string }>;
47 | }
48 |
49 | interface CreateSessionDialogProps {
50 |   open: boolean;
51 |   onOpenChange: (open: boolean) => void;
52 |   defaultDate?: string;
53 |   courses: CourseOption[];
54 |   onSubmit: (data: CreateStudySessionInput) => void;
55 |   isPending?: boolean;
56 | }
57 |
58 | const DURATION_OPTIONS = Array.from({ length: 16 }, (_, i) => (i + 1) * 15);
59 |
60 | const CreateSessionDialog = ({
61 |   open,
62 |   onOpenChange,
63 |   defaultDate,
64 |   courses,
65 |   onSubmit,
66 |   isPending = false,
67 | }: CreateSessionDialogProps) => {
68 |   const form = useForm<CreateStudySessionInput>({
69 |     resolver: zodResolver(createStudySessionSchema),
70 |     defaultValues: {
71 |       title: "",
72 |       plannedDate: defaultDate ?? "",
73 |       plannedStartTime: "09:00",
74 |       plannedDurationMinutes: 25,
75 |       courseId: "",
76 |       timerMode: "pomodoro",
77 |       description: null,
78 |       cloIds: null,
79 |     },

```

```

80 | });
81 |
82 | const selectedCourseId = form.watch("courseId");
83 | const selectedCourse = courses.find((c) => c.id === selectedCourseId);
84 | const availableCLOs = selectedCourse?.clos ?? [];
85 | const selectedCLOids = form.watch("cloIds") ?? [];
86 |
87 | const handleSubmit = (data: CreateStudySessionInput) => {
88 |   onSubmit(data);
89 |   form.reset();
90 | };
91 |
92 | const handleCLOToggle = (cloId: string, checked: boolean) => {
93 |   const current = form.getValues("cloIds") ?? [];
94 |   if (checked) {
95 |     form.setValue("cloIds", [...current, cloId]);
96 |   } else {
97 |     form.setValue(
98 |       "cloIds",
99 |       current.filter((id) => id !== cloId)
100 |     );
101 |   }
102 | };
103 |
104 | return (
105 |   <Dialog open={open} onOpenChange={onOpenChange}>
106 |     <DialogContent className="sm:max-w-lg max-h-[90vh] overflow-y-auto">
107 |       <DialogHeader>
108 |         <DialogTitle className="flex items-center gap-2">
109 |           <CalendarDays className="h-5 w-5 text-blue-600" />
110 |           Create Study Session
111 |         </DialogTitle>
112 |         <DialogDescription>
113 |           Schedule a focused study block for a specific course.
114 |         </DialogDescription>
115 |       </DialogHeader>
116 |
117 |       <Form {...form}>
118 |         <form
119 |           onSubmit={form.handleSubmit(handleSubmit)}
120 |           className="space-y-4"
121 |         >
122 |           { /* Title */ }
123 |           <FormField>
124 |             control={form.control}
125 |             name="title"
126 |             render={({ field }) => (
127 |               <FormItem>
128 |                 <FormLabel>Title</FormLabel>
129 |                 <FormControl>
130 |                   <Input placeholder="e.g. Review Chapter 5" {...field} />
131 |                 </FormControl>
132 |                 <FormMessage />
133 |               </FormItem>
134 |             )}
135 |           />
136 |
137 |           { /* Date + Start Time */ }
138 |           <div className="grid grid-cols-2 gap-3">
139 |             <FormField>
140 |               control={form.control}
141 |               name="plannedDate"
142 |               render={({ field }) => (
143 |                 <FormItem>
144 |                   <FormLabel>Date</FormLabel>
145 |                   <FormControl>
146 |                     <Input type="date" {...field} />
147 |                   </FormControl>
148 |                   <FormMessage />
149 |                 </FormItem>
150 |               )}
151 |             />
152 |             <FormField>
153 |               control={form.control}
154 |               name="plannedStartTime"
155 |               render={({ field }) => (
156 |                 <FormItem>
157 |                   <FormLabel>Start Time</FormLabel>
158 |                   <FormControl>
159 |                     <Input type="time" {...field} />
160 |                   </FormControl>
161 |                   <FormMessage />
162 |                 </FormItem>
163 |               )}
164 |             />
165 |           </div>
166 |
167 |           { /* Duration + Timer Mode */ }
168 |           <div className="grid grid-cols-2 gap-3">
169 |             <FormField>
170 |               control={form.control}
171 |               name="plannedDurationMinutes"
172 |               render={({ field }) => (
173 |                 <FormItem>
174 |                   <FormLabel>Duration</FormLabel>
175 |                   <Select
176 |                     value={String(field.value)}
177 |                     onValueChange={(v) => field.onChange(Number(v))}
178 |                   >
179 |                     <FormControl>
180 |                       <SelectTrigger className="w-full">
181 |                         <SelectValue placeholder="Select duration" />
182 |                       </SelectTrigger>
183 |                     </FormControl>

```

```

184         <SelectContent>
185             {DURATION_OPTIONS.map((mins) => (
186                 <SelectItem key={mins} value={String(mins)}>
187                     {mins < 60
188                         ? `${mins} min`
189                         : `${Math.floor(mins / 60)}h${
190                             mins % 60 > 0 ? ` ${mins % 60}m` : ""
191                         }}
192                 </SelectItem>
193             ))}
194         </SelectContent>
195     </Select>
196     <FormMessage />
197 </FormItem>
198 }}
199 />
200 <FormField
201     control={form.control}
202     name="timerMode"
203     render={({ field }) => (
204         <FormItem>
205             <FormLabel>Timer Mode</FormLabel>
206             <Select value={field.value} onChange={field.onChange}>
207                 <FormControl>
208                     <SelectTrigger className="w-full">
209                         <SelectValue />
210                     </SelectTrigger>
211                 </FormControl>
212                 <SelectContent>
213                     <SelectItem value="pomodoro">Pomodoro</SelectItem>
214                     <SelectItem value="custom">Custom</SelectItem>
215                 </SelectContent>
216             </Select>
217             <FormMessage />
218         </FormItem>
219     )}
220 />
221 </div>
222
223 {/* Course */}
224 <FormField
225     control={form.control}
226     name="courseId"
227     render={({ field }) => (
228         <FormItem>
229             <FormLabel>Course</FormLabel>
230             <Select value={field.value} onChange={field.onChange}>
231                 <FormControl>
232                     <SelectTrigger className="w-full">
233                         <SelectValue placeholder="Select a course" />
234                     </SelectTrigger>
235                 </FormControl>
236                 <SelectContent>
237                     {courses.map((course) => (
238                         <SelectItem key={course.id} value={course.id}>
239                             {course.name}
240                         </SelectItem>
241                     ))}
242                 </SelectContent>
243             </Select>
244             <FormMessage />
245         </FormItem>
246     )}
247 />
248
249 {/* CLO Multi-Select */}
250 {availableCLOs.length > 0 && (
251     <div className="space-y-2">
252         <Label className="text-sm font-medium">
253             CLOs <span className="text-gray-400">(optional)</span>
254         </Label>
255         <div className="max-h-32 space-y-2 overflow-y-auto rounded-md border p-3">
256             {availableCLOs.map((clo) => {
257                 const isChecked = selectedCloIds.includes(clo.id);
258                 return (
259                     <div key={clo.id} className="flex items-center gap-2">
260                         <Checkbox
261                             id={`clo-${clo.id}`}
262                             checked={isChecked}
263                             onChange={() =>
264                                 handleCLOToggle(clo.id, checked === true)
265                             }
266                         </>
267                         <Label
268                             htmlFor={`clo-${clo.id}`}
269                             className="text-xs font-normal text-gray-700 cursor-pointer"
270                         >
271                             {clo.title}
272                         </Label>
273                     </div>
274                 );
275             })}
276         </div>
277     </div>
278 )}
279
280 {/* Description */}
281 <FormField
282     control={form.control}
283     name="description"
284     render={({ field }) => (
285         <FormItem>
286             <FormLabel>
287                 Description{" "}

```

```

288 |         <span className="text-gray-400">(optional)</span>
289 |       </FormLabel>
290 |     </FormControl>
291 |     <Textarea
292 |       placeholder="What will you focus on?"
293 |       rows={3}
294 |       {...field}
295 |       value={field.value ?? ""}
296 |       onChange={(e) => field.onChange(e.target.value || null)}
297 |     />
298 |   </FormControl>
299 |   <FormMessage />
300 | </FormItem>
301 | )}
302 | />
303 |
304 |   <DialogFooter>
305 |     <Button
306 |       type="button"
307 |       variant="outline"
308 |       onClick={() => onOpenChange(false)}
309 |     >
310 |       Cancel
311 |     </Button>
312 |     <Button type="submit" disabled={isPending} variant="tactile">
313 |       {isPending && <Loader2 className="h-4 w-4 animate-spin" />}
314 |       Create Session
315 |     </Button>
316 |   </DialogFooter>
317 | </form>
318 | </Form>
319 | </DialogContent>
320 | </Dialog>
321 | );
322 | };
323 |
324 | export default CreateSessionDialog;
325 | export type { CreateSessionDialogProps, CourseOption };
326 |

```

FILE: src/components/shared/CreateTaskDialog.tsx

TYPE: .tsx | LINES: 1-237 | SHA256: cb1ffd46678b0362ad006fa76dc7cee473d036987596ae55912d799b73ce229a

```

1 | // =====
2 | // CreateTaskDialog – Shadcn Dialog with React Hook Form + Zod for creating
3 | // a new planner task: title, due date, priority select, course select,
4 | // description
5 | // =====
6 |
7 | import { useForm } from "react-hook-form";
8 | import { zodResolver } from "@hookform/resolvers/zod";
9 | import {
10 |   createPlannerTaskSchema,
11 |   type CreatePlannerTaskInput,
12 | } from "@lib/schemas/planner";
13 | import type { z } from "zod";
14 |
15 | type TaskFormValues = z.input<typeof createPlannerTaskSchema>;
16 | import {
17 |   Dialog,
18 |   DialogContent,
19 |   DialogHeader,
20 |   DialogTitle,
21 |   DialogDescription,
22 |   DialogFooter,
23 | } from "@components/ui/dialog";
24 | import {
25 |   Form,
26 |   FormField,
27 |   FormItem,
28 |   FormLabel,
29 |   FormControl,
30 |   FormMessage,
31 | } from "@components/ui/form";
32 | import { Input } from "@components/ui/input";
33 | import { Button } from "@components/ui/button";
34 | import { Textarea } from "@components/ui/textarea";
35 | import {
36 |   Select,
37 |   SelectContent,
38 |   SelectItem,
39 |   SelectTrigger,
40 |   SelectValue,
41 | } from "@components/ui/select";
42 | import { Loader2, ListTodo } from "lucide-react";
43 |
44 | interface CourseOption {
45 |   id: string;
46 |   name: string;
47 | }
48 |
49 | interface CreateTaskDialogProps {
50 |   open: boolean;
51 |   onOpenChange: (open: boolean) => void;
52 |   defaultDate?: string;
53 |   courses: CourseOption[];
54 |   onSubmit: (data: CreatePlannerTaskInput) => void;
55 |   isPending?: boolean;
56 | }
57 |
58 | const CreateTaskDialog = ({
59 |   open,
60 |   onOpenChange,
61 |   defaultDate,

```

```

62 |   courses,
63 |   onSubmit,
64 |   isPending = false,
65 | }: CreateTaskDialogProps) => {
66 |   const form = useForm<TaskFormValues>({
67 |     resolver: zodResolver(createPlannerTaskSchema),
68 |     defaultValues: {
69 |       title: "",
70 |       dueDate: defaultDate ?? "",
71 |       description: null,
72 |       priority: "medium",
73 |       courseId: null,
74 |     },
75 |   });
76 |
77 |   const handleSubmit = (data: TaskFormValues) => {
78 |     onSubmit(data as CreatePlannerTaskInput);
79 |     form.reset();
80 |   };
81 |
82 |   return (
83 |     <Dialog open={open} onOpenChange={onOpenChange}>
84 |       <DialogContent className="sm:max-w-md">
85 |         <DialogHeader>
86 |           <DialogTitle className="flex items-center gap-2">
87 |             <ListTodo className="h-5 w-5 text-blue-600" />
88 |             Create Task
89 |           </DialogTitle>
90 |           <DialogDescription>
91 |             Add a to-do item to your planner.
92 |           </DialogDescription>
93 |         </DialogHeader>
94 |
95 |         <Form {...form}>
96 |           <form
97 |             onSubmit={form.handleSubmit(handleSubmit)}
98 |             className="space-y-4"
99 |           >
100 |            {/* Title */}
101 |            <FormField
102 |              control={form.control}
103 |              name="title"
104 |              render={({ field }) => (
105 |                <FormItem>
106 |                  <FormLabel>Title</FormLabel>
107 |                  <FormControl>
108 |                    <Input placeholder="e.g. Read lecture notes" {...field} />
109 |                  </FormControl>
110 |                  <FormMessage />
111 |                </FormItem>
112 |              )}
113 |            />
114 |
115 |            {/* Due Date + Priority */}
116 |            <div className="grid grid-cols-2 gap-3">
117 |              <FormField
118 |                control={form.control}
119 |                name="dueDate"
120 |                render={({ field }) => (
121 |                  <FormItem>
122 |                    <FormLabel>Due Date</FormLabel>
123 |                    <FormControl>
124 |                      <Input type="date" {...field} />
125 |                    </FormControl>
126 |                    <FormMessage />
127 |                  </FormItem>
128 |                )}
129 |            />
130 |              <FormField
131 |                control={form.control}
132 |                name="priority"
133 |                render={({ field }) => (
134 |                  <FormItem>
135 |                    <FormLabel>Priority</FormLabel>
136 |                    <Select
137 |                      value={field.value ?? "medium"}
138 |                      onValueChange={field.onChange}
139 |                    >
140 |                      <FormControl>
141 |                        <SelectTrigger className="w-full">
142 |                          <SelectValue />
143 |                        </SelectTrigger>
144 |                      </FormControl>
145 |                      <SelectContent>
146 |                        <SelectItem value="high">ðŸŒ™ High</SelectItem>
147 |                        <SelectItem value="medium">ðŸŒŒ Medium</SelectItem>
148 |                        <SelectItem value="low">&a Low</SelectItem>
149 |                      </SelectContent>
150 |                    </Select>
151 |                    <FormMessage />
152 |                  </FormItem>
153 |                )}
154 |            />
155 |            </div>
156 |
157 |            {/* Course (optional) */}
158 |            <FormField
159 |              control={form.control}
160 |              name="courseId"
161 |              render={({ field }) => (
162 |                <FormItem>
163 |                  <FormLabel>
164 |                    Course <span className="text-gray-400">(optional)</span>
165 |                  </FormLabel>

```



```

166         <Select
167           value={field.value ?? ""}
168           onChange={(v) =>
169             field.onChange(v === "none" ? null : v || null)
170           }
171         >
172           <FormControl>
173             <SelectTrigger className="w-full">
174               <SelectValue placeholder="No course" />
175             </SelectTrigger>
176           </FormControl>
177           <SelectContent>
178             <SelectItem value="none">No course</SelectItem>
179             {courses.map((course) => (
180               <SelectItem key={course.id} value={course.id}>
181                 {course.name}
182               </SelectItem>
183             ))}
184           </SelectContent>
185         </Select>
186         <FormMessage />
187       </FormItem>
188     )}
189   />
190
191   {/* Description */}
192   <FormField
193     control={form.control}
194     name="description"
195     render={({ field }) => (
196       <FormItem>
197         <FormLabel>
198           Description{" "}
199           <span className="text-gray-400">(optional)</span>
200         </FormLabel>
201         <FormControl>
202           <Textarea
203             placeholder="Any additional details..."
204             rows={2}
205             {...field}
206             value={field.value ?? ""}
207             onChange={(e) => field.onChange(e.target.value || null)}
208           />
209         </FormControl>
210         <FormMessage />
211       </FormItem>
212     )}
213   />
214
215   <DialogFooter>
216     <Button
217       type="button"
218       variant="outline"
219       onClick={() => onOpenChange(false)}
220     >
221       Cancel
222     </Button>
223     <Button type="submit" disabled={isPending} variant="tactile">
224       {isPending && <Loader2 className="h-4 w-4 animate-spin" />}
225       Create Task
226     </Button>
227   </DialogFooter>
228 </form>
229 </Form>
230 </DialogContent>
231 </Dialog>
232 );
233 };
234
235 export default CreateTaskDialog;
236 export type { CreateTaskDialogProps };
237

```

FILE: src/components/shared/CurriculumMatrix.tsx

TYPE: .tsx | LINES: 1-190 | SHA256: edd5d8cbdded74b278abcc29ab837c69619605d8ff1724dcc432f4b18ace0699

[illegible]

[illegible]

```

133 const label = formatCellLabel(cell);
134 const title =
135   cell.cloCount === 0
136     ? "No CLOs mapped"
137     : cell.attainmentPercent === CELL_ATTAINMENT_UNMEASURED
138       ? `${cell.cloCount} CLO(s) mapped – attainment not yet measured`
139       : `${cell.attainmentPercent}% attainment across ${cell.cloCount} CLO(s)`;
140
141   return (
142     <td key={course.id} className="px-3 py-3 text-center">
143       <button
144         type="button"
145         onClick={() => onCellClick?.(plo.id, course.id)}
146         className={cn(
147           "inline-flex h-9 w-9 items-center justify-center rounded-lg text-sm font-bold transi
148             statusStyles[cell.status],
149             onCellClick && "cursor-pointer",
150             !onCellClick && "cursor-default"
151         )}
152         title={title}
153       >
154         {label}
155       </button>
156     </td>
157   );
158   }}}
159   {coverageLabel && (
160     <td className="border-s border-slate-100 px-4 py-3">
161       <div className="flex items-center justify-end gap-2">
162         <div className="h-1.5 w-16 overflow-hidden rounded-full bg-slate-100">
163           <div
164             className="h-full rounded-full"
165             style={{
166               width: `${cov}%`,
167               background: coverageColor(cov),
168             }}
169           />
170         </div>
171         <span
172           className="text-xs font-bold"
173           style={{ color: coverageColor(cov) }}
174         >
175           {cov}%
176         </span>
177       </div>
178     </td>
179   )}
180 </tr>
181 </tbody>
182 </table>
183 </div>
184 </div>
185 </div>
186 </div>
187 </div>
188 </div>
189 export default CurriculumMatrix;
190

```

FILE: src/components/shared/DailyProgressSummary.tsx

TYPE: .tsx | LINES: 1-94 | SHA256: 6ad987e4455b7646ad0c60bee4c2c270a1db66540c1308d80d321bbe1a1ca951

```
// =====  
2 // DailyProgressSummary - 3 KPI cards (study minutes, tasks completed,  
3 // sessions completed) using project KPI card pattern  
4 // =====  
5  
6 import { PCard } from "@design-system";  
7 import { Clock, CheckCircle2, BookOpen, type LucideIcon } from "lucide-react";  
8  
9 // % % % Types % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
10  
11 interface DailyProgressSummaryProps {  
12   studyMinutes: number;  
13   tasksCompleted: number;  
14   sessionsCompleted: number;  
15 }  
16  
17 // % % % KPI Card % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
18  
19 interface KPICardProps {  
20   icon: LucideIcon;  
21   label: string;  
22   value: string;  
23   accent: string;  
24   iconColor: string;  
25 }  
26  
27 const KPICard = ({  
28   icon: Icon,  
29   label,  
30   value,  
31   accent,  
32   iconColor,  
33 }: KPICardProps) => (  
34   <PCard className="p-4 group">  
35     <div className="flex items-center justify-between">  
36       <div>  
37         <p className="text-[10px] font-black tracking-widest uppercase text-gray-500">  
38           {label}  
39         </p>  
40         <p className="text-2xl font-black mt-1">{value}</p>  
41       </div>  
42     </div>
```

[illegible]

FILE: src/components/shared/DataTable.tsx

TYPE: .tsx | LINES: 1-204 | SHA256: e70fdfe2948df1d60b5032cac01fb37947368a6a27b0dd6babcb7839c5aae9fca

```

1 | import { useState } from "react";
2 | import { useTranslation } from "react-i18next";
3 | import {
4 |   type ColumnDef,
5 |   type SortingState,
6 |   flexRender,
7 |   getCoreRowModel,
8 |   getSortedRowModel,
9 |   getPaginationRowModel,
10 |   useReactTable,
11 | } from "@tanstack/react-table";
12 | import {
13 |   Table,
14 |   TableBody,
15 |   TableCell,
16 |   TableHead,
17 |   TableHeader,
18 |   TableRow,
19 | } from "@components/ui/table";
20 | import { Button } from "@components/ui/button";
21 | import { ArrowUpDown } from "lucide-react";
22 | import { Shimmer } from "@design-system";
23 | import type { ReactNode } from "react";
24 |
25 | interface DataTableProps<TData, TValue> {
26 |   columns: ColumnDef<TData, TValue>[];
27 |   data: TData[];
28 |   isLoading?: boolean;
29 |   /**
30 |    * True while a background refetch is in flight (e.g. a page/filter change with
31 |    * `placeholderData: keepPreviousData`). The previous rows stay visible but are
32 |    * dimmed + marked `aria-busy` so the pending state is perceivable without a
33 |    * skeleton flash (dashboard-and-ux-performance Req 5.1). Distinct from
34 |    * `isLoading`, which is the first-load skeleton.
35 |    */
36 |   isFetching?: boolean;
37 |   page?: number;
38 |   pageSize?: number;
39 |   totalCount?: number;
40 |   onPageChange?: (page: number) => void;
41 |   emptyState?: ReactNode;
42 | }
43 |
44 | function DataTable<TData, TValue>({
45 |   columns,
46 |   data,
47 |   isLoading = false,
48 |   isFetching = false,

```

```

49 | page,
50 | pageSize,
51 | totalCount,
52 | onPageChange,
53 | emptyState,
54 | } : DataTableProps<TData, TValue> ) {
55 |   const { t } = useTranslation("common");
56 |   const [sorting, setSorting] = useState<SortingState>([]);
57 |
58 |   const isServerPaginated =
59 |     page !== undefined &&
60 |     totalCount !== undefined &&
61 |     onPageChange !== undefined;
62 |   const totalPages = isServerPaginated
63 |     ? Math.max(1, Math.ceil(totalCount / (pageSize ?? 25)))
64 |     : undefined;
65 |
66 |   const table = useReactTable({
67 |     data,
68 |     columns,
69 |     state: { sorting },
70 |     onSortingChange: setSorting,
71 |     getCoreRowModel: getCoreRowModel(),
72 |     getSortedRowModel: getSortedRowModel(),
73 |     ...(isServerPaginated
74 |       ? {}
75 |       : { getPaginationRowModel: getPaginationRowModel() }),
76 |   });
77 |
78 |   if (isLoading) {
79 |     return (
80 |       <div className="space-y-3">
81 |         {Array.from({ length: 5 }).map((_, i) => (
82 |           <Shimmer key={i} className="h-12 rounded-lg" />
83 |         ))}
84 |       </div>
85 |     );
86 |   }
87 |
88 |   // Show empty state if no data and emptyState is provided
89 |   if (data.length === 0 && emptyState) {
90 |     return emptyState;
91 |   }
92 |
93 |   return (
94 |     <div
95 |       className={`space-y-4 transition-opacity duration-200 ${
96 |         isFetching ? "opacity-60" : ""
97 |       }`}
98 |     >
99 |       <div className="rounded-lg border bg-background">
100 |         <Table>
101 |           <TableHeader>
102 |             {table.getHeaderGroups().map((headerGroup) => (
103 |               <TableRow key={headerGroup.id}>
104 |                 {headerGroup.headers.map((header) => (
105 |                   <TableHead key={header.id}>
106 |                     {header.isPlaceholder
107 |                       ? null
108 |                       : flexRender(
109 |                         header.column.columnDef.header,
110 |                         header.getContext()
111 |                       )}
112 |                   </TableHead>
113 |                 ))}
114 |             </TableRow>
115 |           </TableHeader>
116 |           <TableBody>
117 |             {table.getRowModel().rows.length ? (
118 |               table.getRowModel().rows.map((row) => (
119 |                 <TableRow key={row.id}>
120 |                   {row.getVisibleCells().map((cell) => (
121 |                     <TableCell key={cell.id}>
122 |                       {flexRender(
123 |                         cell.column.columnDef.cell,
124 |                         cell.getContext()
125 |                       )}
126 |                     </TableCell>
127 |                   ))}
128 |                 </TableRow>
129 |               ))}
130 |             </TableBody>
131 |           </Table>
132 |         ) : (
133 |           <TableRow>
134 |             <TableCell
135 |               colspan={columns.length}
136 |               className="h-24 text-center text-gray-500"
137 |             >
138 |               {t("pagination.noResults", "No results found.")}
139 |             </TableCell>
140 |           </TableRow>
141 |         )}
142 |       </div>
143 |     </div>
144 |
145 |     { /* Pagination */ }
146 |     { isServerPaginated ? (
147 |       <div className="flex items-center justify-between">
148 |         <p className="text-sm text-muted-foreground">
149 |           {t("pagination.pageOf", { page, total: totalPages })}
150 |         </p>
151 |         <div className="flex items-center gap-2">

```

```

153 |         <Button
154 |             variant="outline"
155 |             size="sm"
156 |             onClick={() => onPageChange(page - 1)}
157 |             disabled={page <= 1}
158 |         >
159 |             {t("buttons.back", "Previous")}
160 |         </Button>
161 |         <Button
162 |             variant="outline"
163 |             size="sm"
164 |             onClick={() => onPageChange(page + 1)}
165 |             disabled={page >= (totalPages ?? 1)}
166 |         >
167 |             {t("buttons.next", "Next")}
168 |         </Button>
169 |     </div>
170 | </div>
171 | ) : (
172 |     <div className="flex items-center justify-between">
173 |         <p className="text-sm text-muted-foreground">
174 |             {t("pagination.pageOf", {
175 |                 page: table.getState().pagination.pageIndex + 1,
176 |                 total: table.getPageCount() || 1,
177 |             })}
178 |         </p>
179 |         <div className="flex items-center gap-2">
180 |             <Button
181 |                 variant="outline"
182 |                 size="sm"
183 |                 onClick={() => table.previousPage()}
184 |                 disabled={!table.getCanPreviousPage()}
185 |             >
186 |                 {t("buttons.back", "Previous")}
187 |             </Button>
188 |             <Button
189 |                 variant="outline"
190 |                 size="sm"
191 |                 onClick={() => table.nextPage()}
192 |                 disabled={!table.getCanNextPage()}
193 |             >
194 |                 {t("buttons.next", "Next")}
195 |             </Button>
196 |         </div>
197 |     </div>
198 | ))}
199 | </div>
200 | );
201 | }
202 |
203 | export { DataTable, ArrowUpDown };
204 |

```

FILE: src/components/shared/DeadlineItem.tsx

TYPE: .tsx | LINES: 1-91 | SHA256: b35cdfaffdb31ba34be640bb11592d7c37bf97cfcb0e77c2fdbcb5f401154399

```

1 | // =====
2 | // DeadlineItem - Read-only deadline item with urgency color indicator
3 | // (red/yellow/green), course name, due time
4 | // =====
5 |
6 | import { cn } from "@lib/utils";
7 | import type { UpcomingDeadline } from "@types/planner";
8 | import { AlertTriangle, CalendarClock } from "lucide-react";
9 | import { format, parseISO } from "date-fns";
10 |
11 | interface DeadlineItemProps {
12 |     deadline: UpcomingDeadline;
13 |     compact?: boolean;
14 | }
15 |
16 | const URGENCY_STYLES: Record<
17 |     UpcomingDeadline["urgency"],
18 |     { border: string; bg: string; dot: string; icon: string }
19 | > = {
20 |     red: {
21 |         border: "border-red-300",
22 |         bg: "bg-red-50/60",
23 |         dot: "bg-red-500",
24 |         icon: "text-red-500",
25 |     },
26 |     yellow: {
27 |         border: "border-amber-300",
28 |         bg: "bg-amber-50/60",
29 |         dot: "bg-amber-500",
30 |         icon: "text-amber-500",
31 |     },
32 |     green: {
33 |         border: "border-green-300",
34 |         bg: "bg-green-50/60",
35 |         dot: "bg-green-500",
36 |         icon: "text-green-500",
37 |     },
38 | };
39 |
40 | const DeadlineItem = ({ deadline, compact = false }: DeadlineItemProps) => {
41 |     const urgencyConfig = URGENCY_STYLES[deadline.urgency];
42 |
43 |     const formatDueDate = (dateStr: string): string => {
44 |         try {
45 |             const date = parseISO(dateStr);
46 |             return format(date, "MMM d, h:mm a");
47 |         } catch {
48 |             return dateStr;

```

```

49 |     }
50 |   };
51 |
52 |   return (
53 |     <div
54 |       className={cn(
55 |         "flex items-start gap-2 rounded-lg border p-2.5",
56 |         urgencyConfig.border,
57 |         urgencyConfig.bg,
58 |         compact && "p-2"
59 |       )}
60 |     >
61 |       { /* Urgency dot */ }
62 |       <div className="mt-1 shrink-0">
63 |         {deadline.urgency === "red" ? (
64 |           <AlertTriangle className={cn("h-3.5 w-3.5", urgencyConfig.icon)} />
65 |         ) : (
66 |           <div className={cn("h-2.5 w-2.5 rounded-full", urgencyConfig.dot)} />
67 |         )}
68 |       </div>
69 |
70 |       { /* Content */ }
71 |       <div className="min-w-0 flex-1">
72 |         <h4 className="text-xs font-semibold text-gray-900 line-clamp-1">
73 |           {deadline.title}
74 |         </h4>
75 |         <div className="mt-0.5 flex flex-wrap items-center gap-x-2 gap-y-0.5 text-[11px] text-gray-500">
76 |           <span className="flex items-center gap-1">
77 |             <CalendarClock className="h-3 w-3" />
78 |             {formatDueDate(deadline.dueDate)}
79 |           </span>
80 |           {deadline.courseName && (
81 |             <span className="line-clamp-1">{deadline.courseName}</span>
82 |           )}
83 |         </div>
84 |       </div>
85 |     </div>
86 |   );
87 | };
88 |
89 | export default DeadlineItem;
90 | export type { DeadlineItemProps };
91 |

```

FILE: src/components/shared/DifficultyBadge.tsx

TYPE: .tsx | LINES: 1-34 | SHA256: bf9eb7602890d16b5507cfadc72f743f5c1d29dfdfa2c6e32cd8164741d240cb

```

1 | import { Badge } from "@components/ui/badge";
2 | import { cn } from "@lib/utills";
3 |
4 | export interface DifficultyBadgeProps {
5 |   difficulty: number; // 1.0 to 5.0
6 |   className?: string;
7 | }
8 |
9 | const getDifficultyClasses = (difficulty: number): string => {
10 |   if (difficulty < 2.0) return "bg-green-100 text-green-700";
11 |   if (difficulty < 3.0) return "bg-blue-100 text-blue-700";
12 |   if (difficulty < 3.5) return "bg-yellow-100 text-yellow-700";
13 |   if (difficulty < 4.5) return "bg-orange-100 text-orange-700";
14 |   return "bg-red-100 text-red-700";
15 | };
16 |
17 | const DifficultyBadge = ({ difficulty, className }: DifficultyBadgeProps) => {
18 |   return (
19 |     <Badge
20 |       variant="outline"
21 |       className={cn(
22 |         "text-[10px] font-bold tracking-wide uppercase border-transparent",
23 |         getDifficultyClasses(difficulty),
24 |         className
25 |       )}
26 |     >
27 |       Difficulty: {difficulty.toFixed(1)}
28 |     </Badge>
29 |   );
30 | };
31 |
32 | export { DifficultyBadge };
33 | export default DifficultyBadge;
34 |

```

FILE: src/components/shared/DiscussionThreadCard.tsx

TYPE: .tsx | LINES: 1-72 | SHA256: 85ce921b8b2a568efdbbdc7871789bc6f8d72c81672f897d378e7020a073ac9c

```

1 | // =====
2 | // DiscussionThreadCard - Discussion forum thread card
3 | // =====
4 |
5 | import { MessageSquare, Pin } from "lucide-react";
6 | import { cn } from "@lib/utills";
7 |
8 | interface DiscussionThreadCardProps {
9 |   title: string;
10 |   authorName: string;
11 |   createdAt: string;
12 |   replyCount: number;
13 |   isPinned?: boolean;
14 |   isResolved?: boolean;
15 |   onClick?: () => void;
16 |   className?: string;
17 | }
18 |
19 | const DiscussionThreadCard = ({

```

```

20 | title,
21 | authorName,
22 | createdAt,
23 | replyCount,
24 | isPinned = false,
25 | isResolved = false,
26 | onClick,
27 | className,
28 |}: DiscussionThreadCardProps) => (
29 | <div
30 |   className={cn(
31 |     "rounded-xl border border-slate-200 bg-white p-4 hover:shadow-sm transition-shadow cursor-pointer",
32 |     isPinned && "border-blue-200 bg-blue-50/30",
33 |     className
34 |   )}
35 |   onClick={onClick}
36 |   role="button"
37 |   tabIndex={0}
38 |   onKeyDown={(e) => {
39 |     if (e.key === "Enter" || e.key === " ") {
40 |       e.preventDefault();
41 |       onClick?.();
42 |     }
43 |   }}
44 | >
45 |   <div className="flex items-start gap-2">
46 |     <div className="flex-1 min-w-0">
47 |       <div className="flex items-center gap-2">
48 |         {isPinned && <Pin className="h-3 w-3 text-blue-500 shrink-0" />}
49 |         <h4 className="text-sm font-bold truncate">{title}</h4>
50 |         {isResolved && (
51 |           <span className="text-[10px] font-bold tracking-widest uppercase text-green-600 bg-green-50 px-1.5
52 |             Resolved
53 |           </span>
54 |         )}
55 |       </div>
56 |       <div className="flex items-center gap-2 mt-1 text-xs text-gray-400">
57 |         <span>{authorName}</span>
58 |         <span>.</span>
59 |         <span>{createdAt}</span>
60 |       </div>
61 |     </div>
62 |     <div className="flex items-center gap-1 text-xs text-gray-400 shrink-0">
63 |       <MessageSquare className="h-3.5 w-3.5" />
64 |       <span>{replyCount}</span>
65 |     </div>
66 |   </div>
67 | </div>
68 | );
69 |
70 | export default DiscussionThreadCard;
71 | export type { DiscussionThreadCardProps };
72 |

```

FILE: src/components/shared/EmailPreferencesSection.tsx

TYPE: .tsx | LINES: 1-118 | SHA256: f36146bf8eaa5df70b70e1cbc13c72cab266db23ee3af8c0d6202a25a02d1999

```

1 | import { useCallback } from "react";
2 | import { Switch } from "@components/ui/switch";
3 | import { Label } from "@components/ui/label";
4 | import { Loader2, Mail } from "lucide-react";
5 | import { toast } from "sonner";
6 | import { useEmailPreferences } from "@hooks/useEmailPreferences";
7 | import { PCard, SectionHeader } from "@design-system";
8 | import type { EmailPreferencesFormData } from "@lib/schemas/emailPrefs";
9 |
10 | export interface EmailPreferenceItem {
11 |   key: keyof EmailPreferencesFormData;
12 |   label: string;
13 |   description: string;
14 | }
15 |
16 | const PREFERENCE_ITEMS: EmailPreferenceItem[] = [
17 |   {
18 |     key: "streak_risk",
19 |     label: "Streak Risk Alerts",
20 |     description: "Daily reminder at 8 PM if your streak is at risk",
21 |   },
22 |   {
23 |     key: "weekly_summary",
24 |     label: "Weekly Summary",
25 |     description: "Monday morning recap of your XP, badges, and progress",
26 |   },
27 |   {
28 |     key: "new_assignment",
29 |     label: "New Assignments",
30 |     description: "Notification when a new assignment is posted",
31 |   },
32 |   {
33 |     key: "grade_released",
34 |     label: "Grade Released",
35 |     description: "Notification when your grade is ready to view",
36 |   },
37 |   {
38 |     key: "notification_digest",
39 |     label: "Daily Digest",
40 |     description:
41 |       "Receive a single daily summary at 8 PM instead of individual notifications",
42 |   },
43 | ];
44 |
45 | interface EmailPreferencesSectionProps {
46 |   title?: string;
47 |   description?: string;

```



```

48 | items?: EmailPreferenceItem[];
49 | }
50 |
51 | const EmailPreferencesSection = ({
52 |   title = "Email Notifications",
53 |   description = "Choose which email notifications you'd like to receive.",
54 |   items = PREFERENCE_ITEMS,
55 | }: EmailPreferencesSectionProps) => {
56 |   const { preferences, isLoading, isUpdating, updatePreferencesAsync } =
57 |     useEmailPreferences();
58 |
59 |   const handleToggle = useCallback(
60 |     async (key: keyof EmailPreferencesFormData, checked: boolean) => {
61 |       try {
62 |         await updatePreferencesAsync({ ...preferences, [key]: checked });
63 |         toast.success(
64 |           checked ? "Notification enabled" : "Notification disabled"
65 |         );
66 |       } catch {
67 |         toast.error("Failed to update preference");
68 |       }
69 |     },
70 |     [preferences, updatePreferencesAsync]
71 |   );
72 |
73 |   if (isLoading) {
74 |     return (
75 |       <PCard className="overflow-hidden gap-0 py-0">
76 |         <SectionHeader icon={Mail} title={title} />
77 |         <div className="flex items-center justify-center p-6">
78 |           <Loader2 className="h-5 w-5 animate-spin text-slate-400" />
79 |         </div>
80 |       </PCard>
81 |     );
82 |   }
83 |
84 |   return (
85 |     <PCard className="overflow-hidden gap-0 py-0">
86 |       <SectionHeader icon={Mail} title={title} />
87 |       <div className="p-6 space-y-4">
88 |         <p className="text-sm text-slate-500">{description}</p>
89 |         {items.map((item) => (
90 |           <div
91 |             key={item.key}
92 |             className="flex items-center justify-between py-3 border-b border-slate-100 last:border-0"
93 |           >
94 |             <div className="space-y-0.5">
95 |               <Label
96 |                 htmlFor={`email-pref-${item.key}`}
97 |                 className="text-sm font-medium"
98 |               >
99 |                 {item.label}
100 |               </Label>
101 |               <p className="text-xs text-slate-500">{item.description}</p>
102 |             </div>
103 |             <Switch
104 |               id={`email-pref-${item.key}`}
105 |               checked={preferences[item.key] ?? false}
106 |               onCheckedChange={(checked) => handleToggle(item.key, checked)}
107 |               disabled={isUpdating}
108 |               aria-label={`Toggle ${item.label} email notifications`}
109 |             />
110 |           </div>
111 |         ))}
112 |       </div>
113 |     </PCard>
114 |   );
115 | };
116 |
117 | export default EmailPreferencesSection;
118 |

```

FILE: src/components/shared/EmailVerificationBanner.tsx

TYPE: .tsx | LINES: 1-86 | SHA256: dc6c98a8ea2dc101a6e4e9adb901068f7533d18348ea49411934625867401b89

```

1 | import { useState } from "react";
2 | import { useTranslation } from "react-i18next";
3 | import { toast } from "sonner";
4 | import { X, MailWarning } from "lucide-react";
5 | import { useAuth } from "@/hooks/useAuth";
6 | import { supabase } from "@/lib/supabase";
7 | import { Button } from "@components/ui/button";
8 |
9 | /**
10 |  * EmailVerificationBanner (ADR-14)
11 |  *
12 |  * Renders an amber banner when `profile.status === 'pending_verification'`.
13 |  * Dismissible per-session (state resets on page reload).
14 |  */
15 | const EmailVerificationBanner = () => {
16 |   const { t } = useTranslation("common");
17 |   const { user, profile } = useAuth();
18 |   const [dismissed, setDismissed] = useState(false);
19 |   const [isSending, setIsSending] = useState(false);
20 |
21 |   if (dismissed || profile?.status !== "pending_verification") {
22 |     return null;
23 |   }
24 |
25 |   const handleResend = async () => {
26 |     if (!user?.email) return;
27 |
28 |     setIsSending(true);
29 |     try {

```

```

30 |     const { error } = await supabase.auth.resend({
31 |       type: "signup",
32 |       email: user.email,
33 |     });
34 |
35 |     if (error) {
36 |       console.error(
37 |         "[EmailVerificationBanner] Resend failed:",
38 |         error.message
39 |       );
40 |       toast.error(t("emailVerification.resendError"));
41 |     } else {
42 |       toast.success(t("emailVerification.resendSuccess"));
43 |     }
44 |   } catch (err) {
45 |     console.error("[EmailVerificationBanner] Unexpected error:", err);
46 |     toast.error(t("emailVerification.resendError"));
47 |   } finally {
48 |     setIsSending(false);
49 |   }
50 | };
51 |
52 | return (
53 |   <div
54 |     role="alert"
55 |     className="flex items-center gap-3 bg-amber-50 border-b border-amber-200 px-4 py-2.5 text-amber-800"
56 |   >
57 |     <MailWarning
58 |       className="h-4 w-4 shrink-0 text-amber-600"
59 |       aria-hidden="true"
60 |     />
61 |     <p className="flex-1 text-sm font-medium">
62 |       {t("emailVerification.banner")}
63 |     </p>
64 |     <Button
65 |       variant="outline"
66 |       size="sm"
67 |       disabled={isSending}
68 |       onClick={handleResend}
69 |       className="shrink-0 border-amber-300 bg-amber-50 text-amber-800 hover:bg-amber-100 hover:text-amber-90
70 |     >
71 |       {t("emailVerification.resend")}
72 |     </Button>
73 |     <button
74 |       type="button"
75 |       aria-label={t("emailVerification.dismiss")}
76 |       onClick={() => setDismissed(true)}
77 |       className="shrink-0 rounded p-0.5 text-amber-600 hover:bg-amber-100 hover:text-amber-800 transition-co
78 |     >
79 |       <X className="h-4 w-4" />
80 |     </button>
81 |   </div>
82 | );
83 | };
84 |
85 | export default EmailVerificationBanner;
86 |

```

FILE: src/components/shared/EmptyState.tsx

TYPE: .tsx | LINES: 1-780 | SHA256: 181dd26bbbc59728690aed446a8615c110a7d1f4bee78b6abcc14091805d8c631

```

1 | // =====
2 | // EmptyState - Generic empty state with icon, title, description
3 | // =====
4 |
5 | import {
6 |   Inbox,
7 |   BookOpen,
8 |   TrendingUp,
9 |   Bell,
10 |   Trophy,
11 |   FileText,
12 |   ShoppingCart,
13 |   Users,
14 |   Swords,
15 |   Target,
16 |   HelpCircle,
17 |   BarChart3,
18 |   CalendarDays,
19 |   ClipboardList,
20 |   Clock,
21 |   MessageSquare,
22 |   GraduationCap,
23 |   Ruler,
24 |   CheckSquare,
25 |   ListChecks,
26 | } from "lucide-react";
27 | import { cn } from "@lib/utills";
28 | import { Card } from "@components/ui/card";
29 | import { useTranslation } from "react-i18next";
30 |
31 | interface EmptyStateProps {
32 |   icon?: React.ReactNode;
33 |   title: string;
34 |   description?: string;
35 |   children?: React.ReactNode;
36 |   className?: string;
37 | }
38 |
39 | const EmptyState = ({
40 |   icon,
41 |   title,
42 |   description,
43 |   children,

```

```

44 |   className,
45 | }: EmptyStateProps) => (
46 |   <Card
47 |     className={cn(
48 |       "flex flex-col items-center justify-center border border-dashed border-gray-200 dark:border-gray-700 bg-
49 |       className
50 |     )}
51 |   >
52 |     <div className="mb-4 rounded-full bg-gradient-to-br from-slate-50 to-slate-100 dark:from-slate-800 dark:to
53 |       {icon ?? <Inbox className="h-8 w-8 text-gray-400 dark:text-gray-500" />}
54 |     </div>
55 |     <h3 className="text-lg font-semibold tracking-tight text-gray-900 dark:text-gray-100">
56 |       {title}
57 |     </h3>
58 |     {description && (
59 |       <p className="mt-1.5 text-sm text-gray-500 dark:text-gray-400 max-w-sm leading-relaxed">
60 |         {description}
61 |       </p>
62 |     )}
63 |     {children && <div className="mt-5">{children}</div>}
64 |   </Card>
65 | );
66 |
67 | export default EmptyState;
68 | export type { EmptyStateProps };
69 |
70 | // =====
71 | // InlineEmpty – Lightweight empty for use INSIDE existing Card containers
72 | // =====
73 |
74 | interface InlineEmptyProps {
75 |   icon?: React.ReactNode;
76 |   title: string;
77 |   description?: string;
78 |   className?: string;
79 |   children?: React.ReactNode;
80 | }
81 |
82 | export const InlineEmpty = ({
83 |   icon,
84 |   title,
85 |   description,
86 |   className,
87 |   children,
88 | }: InlineEmptyProps) => (
89 |   <div
90 |     className={cn(
91 |       "flex flex-col items-center justify-center py-10 px-4 text-center",
92 |       className
93 |     )}
94 |   >
95 |     <div className="mb-3 rounded-full bg-gradient-to-br from-slate-50 to-slate-100 dark:from-slate-800 dark:to
96 |       {icon ?? <Inbox className="h-6 w-6 text-gray-400 dark:text-gray-500" />}
97 |     </div>
98 |     <p className="text-sm font-semibold text-gray-600 dark:text-gray-300">
99 |       {title}
100 |     </p>
101 |     {description && (
102 |       <p className="mt-1 text-xs text-gray-400 dark:text-gray-500 max-w-xs leading-relaxed">
103 |         {description}
104 |       </p>
105 |     )}
106 |     {children && <div className="mt-3">{children}</div>}
107 |   </div>
108 | );
109 |
110 | // =====
111 | // Named variants for common empty states
112 | // =====
113 |
114 | interface EmptyStateVariantProps {
115 |   children?: React.ReactNode;
116 |   className?: string;
117 | }
118 |
119 | // Each variant is a proper named component to satisfy react-refresh/only-export-components
120 | export const NoCourses = ({ children, className }: EmptyStateVariantProps) => {
121 |   const { t } = useTranslation("common");
122 |   return (
123 |     <EmptyState
124 |       icon={<BookOpen className="h-8 w-8 text-gray-400" />}
125 |       title={t("empty.noCourses.title")}
126 |       description={t("empty.noCourses.description")}
127 |       className={className}
128 |     >
129 |       {children}
130 |     </EmptyState>
131 |   );
132 | };
133 | export const NoProgress = ({ children, className }: EmptyStateVariantProps) => {
134 |   const { t } = useTranslation("common");
135 |   return (
136 |     <EmptyState
137 |       icon={<TrendingUp className="h-8 w-8 text-gray-400" />}
138 |       title={t("empty.noProgress.title")}
139 |       description={t("empty.noProgress.description")}
140 |       className={className}
141 |     >
142 |       {children}
143 |     </EmptyState>
144 |   );
145 | };
146 | export const NoUsers = ({ children, className }: EmptyStateVariantProps) => {
147 |   const { t } = useTranslation("common");

```

```

148 | return (
149 |   <EmptyState
150 |     icon={<Users className="h-8 w-8 text-gray-400" />}
151 |     title={t("empty.noUsers.title")}
152 |     description={t("empty.noUsers.description")}
153 |     className={className}
154 |   >
155 |     {children}
156 |   </EmptyState>
157 | );
158 | };
159 | export const NoNotifications = ({
160 |   children,
161 |   className,
162 | }: EmptyStateVariantProps) => {
163 |   const { t } = useTranslation("common");
164 |   return (
165 |     <EmptyState
166 |       icon={<Bell className="h-8 w-8 text-gray-400" />}
167 |       title={t("empty.noNotifications.title")}
168 |       description={t("empty.noNotifications.description")}
169 |       className={className}
170 |     >
171 |       {children}
172 |     </EmptyState>
173 |   );
174 | };
175 | export const NoBadges = ({ children, className }: EmptyStateVariantProps) => {
176 |   const { t } = useTranslation("common");
177 |   return (
178 |     <EmptyState
179 |       icon={<Trophy className="h-8 w-8 text-gray-400" />}
180 |       title={t("empty.noBadges.title")}
181 |       description={t("empty.noBadges.description")}
182 |       className={className}
183 |     >
184 |       {children}
185 |     </EmptyState>
186 |   );
187 | };
188 | export const NoEvidence = ({ children, className }: EmptyStateVariantProps) => {
189 |   const { t } = useTranslation("common");
190 |   return (
191 |     <EmptyState
192 |       icon={<FileText className="h-8 w-8 text-gray-400" />}
193 |       title={t("empty.noEvidence.title")}
194 |       description={t("empty.noEvidence.description")}
195 |       className={className}
196 |     >
197 |       {children}
198 |     </EmptyState>
199 |   );
200 | };
201 | export const NoMarketplaceItems = ({
202 |   children,
203 |   className,
204 | }: EmptyStateVariantProps) => {
205 |   const { t } = useTranslation("common");
206 |   return (
207 |     <EmptyState
208 |       icon={<ShoppingCart className="h-8 w-8 text-gray-400" />}
209 |       title={t("empty.noMarketplaceItems.title")}
210 |       description={t("empty.noMarketplaceItems.description")}
211 |       className={className}
212 |     >
213 |       {children}
214 |     </EmptyState>
215 |   );
216 | };
217 | export const NoLinkedStudents = ({
218 |   children,
219 |   className,
220 | }: EmptyStateVariantProps) => {
221 |   const { t } = useTranslation("common");
222 |   return (
223 |     <EmptyState
224 |       icon={<Users className="h-8 w-8 text-gray-400" />}
225 |       title={t("empty.noLinkedStudents.title")}
226 |       description={t("empty.noLinkedStudents.description")}
227 |       className={className}
228 |     >
229 |       {children}
230 |     </EmptyState>
231 |   );
232 | };
233 | export const NoTeams = ({ children, className }: EmptyStateVariantProps) => {
234 |   const { t } = useTranslation("common");
235 |   return (
236 |     <EmptyState
237 |       icon={<Users className="h-8 w-8 text-gray-400" />}
238 |       title={t("empty.noTeams.title")}
239 |       description={t("empty.noTeams.description")}
240 |       className={className}
241 |     >
242 |       {children}
243 |     </EmptyState>
244 |   );
245 | };
246 | export const NoChallenges = ({
247 |   children,
248 |   className,
249 | }: EmptyStateVariantProps) => {
250 |   const { t } = useTranslation("common");
251 |   return (

```

```

252 |     <EmptyState
253 |       icon={<Swords className="h-8 w-8 text-gray-400" />}
254 |       title={t("empty.noChallenges.title")}
255 |       description={t("empty.noChallenges.description")}
256 |       className={className}
257 |     >
258 |       {children}
259 |     </EmptyState>
260 |   );
261 | };
262 | export const NoOutcomes = ({ children, className }: EmptyStateVariantProps) => {
263 |   const { t } = useTranslation("common");
264 |   return (
265 |     <EmptyState
266 |       icon={<Target className="h-8 w-8 text-gray-400" />}
267 |       title={t("empty.noOutcomes.title")}
268 |       description={t("empty.noOutcomes.description")}
269 |       className={className}
270 |     >
271 |       {children}
272 |     </EmptyState>
273 |   );
274 | };
275 | export const NoQuestions = ({
276 |   children,
277 |   className,
278 | }: EmptyStateVariantProps) => {
279 |   const { t } = useTranslation("common");
280 |   return (
281 |     <EmptyState
282 |       icon={<HelpCircle className="h-8 w-8 text-gray-400" />}
283 |       title={t("empty.noQuestions.title")}
284 |       description={t("empty.noQuestions.description")}
285 |       className={className}
286 |     >
287 |       {children}
288 |     </EmptyState>
289 |   );
290 | };
291 | export const NoResults = ({ children, className }: EmptyStateVariantProps) => {
292 |   const { t } = useTranslation("common");
293 |   return (
294 |     <EmptyState
295 |       icon={<BarChart3 className="h-8 w-8 text-gray-400" />}
296 |       title={t("empty.noResults.title")}
297 |       description={t("empty.noResults.description")}
298 |       className={className}
299 |     >
300 |       {children}
301 |     </EmptyState>
302 |   );
303 | };
304 | export const NoData = ({ children, className }: EmptyStateVariantProps) => {
305 |   const { t } = useTranslation("common");
306 |   return (
307 |     <EmptyState
308 |       icon={<BarChart3 className="h-8 w-8 text-gray-400" />}
309 |       title={t("empty.noData.title")}
310 |       description={t("empty.noData.description")}
311 |       className={className}
312 |     >
313 |       {children}
314 |     </EmptyState>
315 |   );
316 | };
317 | export const NoSessions = ({ children, className }: EmptyStateVariantProps) => {
318 |   const { t } = useTranslation("common");
319 |   return (
320 |     <EmptyState
321 |       icon={<Clock className="h-8 w-8 text-gray-400" />}
322 |       title={t("empty.noSessions.title")}
323 |       description={t("empty.noSessions.description")}
324 |       className={className}
325 |     >
326 |       {children}
327 |     </EmptyState>
328 |   );
329 | };
330 | export const NoSurveys = ({ children, className }: EmptyStateVariantProps) => {
331 |   const { t } = useTranslation("common");
332 |   return (
333 |     <EmptyState
334 |       icon={<ClipboardList className="h-8 w-8 text-gray-400" />}
335 |       title={t("empty.noSurveys.title")}
336 |       description={t("empty.noSurveys.description")}
337 |       className={className}
338 |     >
339 |       {children}
340 |     </EmptyState>
341 |   );
342 | };
343 | export const NoSemesters = ({
344 |   children,
345 |   className,
346 | }: EmptyStateVariantProps) => {
347 |   const { t } = useTranslation("common");
348 |   return (
349 |     <EmptyState
350 |       icon={<CalendarDays className="h-8 w-8 text-gray-400" />}
351 |       title={t("empty.noSemesters.title")}
352 |       description={t("empty.noSemesters.description")}
353 |       className={className}
354 |     >
355 |       {children}

```

```

356 |     </EmptyState>
357 |   );
358 | };
359 | export const NoTimetable = ({
360 |   children,
361 |   className,
362 | }: EmptyStateVariantProps) => {
363 |   const { t } = useTranslation("common");
364 |   return (
365 |     <EmptyState
366 |       icon={<CalendarDays className="h-8 w-8 text-gray-400" />}
367 |       title={t("empty.noTimetable.title")}
368 |       description={t("empty.noTimetable.description")}
369 |       className={className}
370 |     >
371 |       {children}
372 |     </EmptyState>
373 |   );
374 | };
375 | export const NoResponses = ({
376 |   children,
377 |   className,
378 | }: EmptyStateVariantProps) => {
379 |   const { t } = useTranslation("common");
380 |   return (
381 |     <EmptyState
382 |       icon={<MessageSquare className="h-8 w-8 text-gray-400" />}
383 |       title={t("empty.noResponses.title")}
384 |       description={t("empty.noResponses.description")}
385 |       className={className}
386 |     >
387 |       {children}
388 |     </EmptyState>
389 |   );
390 | };
391 | export const NoAssignments = ({
392 |   children,
393 |   className,
394 | }: EmptyStateVariantProps) => {
395 |   const { t } = useTranslation("common");
396 |   return (
397 |     <EmptyState
398 |       icon={<FileText className="h-8 w-8 text-gray-400" />}
399 |       title={t("empty.noAssignments.title")}
400 |       description={t("empty.noAssignments.description")}
401 |       className={className}
402 |     >
403 |       {children}
404 |     </EmptyState>
405 |   );
406 | };
407 | export const NoSubmissions = ({
408 |   children,
409 |   className,
410 | }: EmptyStateVariantProps) => {
411 |   const { t } = useTranslation("common");
412 |   return (
413 |     <EmptyState
414 |       icon={<CheckSquare className="h-8 w-8 text-gray-400" />}
415 |       title={t("empty.noSubmissions.title")}
416 |       description={t("empty.noSubmissions.description")}
417 |       className={className}
418 |     >
419 |       {children}
420 |     </EmptyState>
421 |   );
422 | };
423 | export const NoCLOs = ({ children, className }: EmptyStateVariantProps) => {
424 |   const { t } = useTranslation("common");
425 |   return (
426 |     <EmptyState
427 |       icon={<Target className="h-8 w-8 text-gray-400" />}
428 |       title={t("empty.noCLOs.title")}
429 |       description={t("empty.noCLOs.description")}
430 |       className={className}
431 |     >
432 |       {children}
433 |     </EmptyState>
434 |   );
435 | };
436 | export const NoPLOs = ({ children, className }: EmptyStateVariantProps) => {
437 |   const { t } = useTranslation("common");
438 |   return (
439 |     <EmptyState
440 |       icon={<Target className="h-8 w-8 text-gray-400" />}
441 |       title={t("empty.noPLOs.title")}
442 |       description={t("empty.noPLOs.description")}
443 |       className={className}
444 |     >
445 |       {children}
446 |     </EmptyState>
447 |   );
448 | };
449 | export const NoILOs = ({ children, className }: EmptyStateVariantProps) => {
450 |   const { t } = useTranslation("common");
451 |   return (
452 |     <EmptyState
453 |       icon={<GraduationCap className="h-8 w-8 text-gray-400" />}
454 |       title={t("empty.noILOs.title")}
455 |       description={t("empty.noILOs.description")}
456 |       className={className}
457 |     >
458 |       {children}
459 |     </EmptyState>

```

```

460 | });
461 | };
462 | export const NoStudents = ({ children, className }: EmptyStateVariantProps) => {
463 |   const { t } = useTranslation("common");
464 |   return (
465 |     <EmptyState
466 |       icon={<Users className="h-8 w-8 text-gray-400" />}
467 |       title={t("empty.noStudents.title")}
468 |       description={t("empty.noStudents.description")}
469 |       className={className}
470 |     >
471 |       {children}
472 |     </EmptyState>
473 |   );
474 | };
475 | export const NoRubrics = ({ children, className }: EmptyStateVariantProps) => {
476 |   const { t } = useTranslation("common");
477 |   return (
478 |     <EmptyState
479 |       icon={<Ruler className="h-8 w-8 text-gray-400" />}
480 |       title={t("empty.noRubrics.title")}
481 |       description={t("empty.noRubrics.description")}
482 |       className={className}
483 |     >
484 |       {children}
485 |     </EmptyState>
486 |   );
487 | };
488 | export const NoActionPlans = ({
489 |   children,
490 |   className,
491 | }: EmptyStateVariantProps) => {
492 |   const { t } = useTranslation("common");
493 |   return (
494 |     <EmptyState
495 |       icon={<ListChecks className="h-8 w-8 text-gray-400" />}
496 |       title={t("empty.noActionPlans.title")}
497 |       description={t("empty.noActionPlans.description")}
498 |       className={className}
499 |     >
500 |       {children}
501 |     </EmptyState>
502 |   );
503 | };
504 | export const NoAttendance = ({
505 |   children,
506 |   className,
507 | }: EmptyStateVariantProps) => {
508 |   const { t } = useTranslation("common");
509 |   return (
510 |     <EmptyState
511 |       icon={<CheckSquare className="h-8 w-8 text-gray-400" />}
512 |       title={t("empty.noAttendance.title")}
513 |       description={t("empty.noAttendance.description")}
514 |       className={className}
515 |     >
516 |       {children}
517 |     </EmptyState>
518 |   );
519 | };
520 |
521 | // =====
522 | // Inline variants – for use INSIDE existing Card containers (no Card wrapper)
523 | // =====
524 |
525 | export const InlineNoCLOs = ({ className }: { className?: string }) => {
526 |   const { t } = useTranslation("common");
527 |   return (
528 |     <InlineEmpty
529 |       icon={<Target className="h-6 w-6 text-gray-400" />}
530 |       title={t("empty.noCLOs.title")}
531 |       description={t("empty.noCLOs.description")}
532 |       className={className}
533 |     />
534 |   );
535 | };
536 |
537 | export const InlineNoData = ({ className }: { className?: string }) => {
538 |   const { t } = useTranslation("common");
539 |   return (
540 |     <InlineEmpty
541 |       icon={<BarChart3 className="h-6 w-6 text-gray-400" />}
542 |       title={t("empty.noData.title")}
543 |       description={t("empty.noData.description")}
544 |       className={className}
545 |     />
546 |   );
547 | };
548 |
549 | export const InlineNoTeams = ({ className }: { className?: string }) => {
550 |   const { t } = useTranslation("common");
551 |   return (
552 |     <InlineEmpty
553 |       icon={<Users className="h-6 w-6 text-gray-400" />}
554 |       title={t("empty.noTeams.title")}
555 |       description={t("empty.noTeams.description")}
556 |       className={className}
557 |     />
558 |   );
559 | };
560 |
561 | export const InlineNoQuestions = ({ className }: { className?: string }) => {
562 |   const { t } = useTranslation("common");
563 |   return (

```

```

564 |     <InlineEmpty
565 |       icon={<HelpCircle className="h-6 w-6 text-gray-400" />}
566 |       title={t("empty.noQuestions.title")}
567 |       description={t("empty.noQuestions.description")}
568 |       className={className}
569 |     />
570 |   );
571 | };
572 |
573 | export const InlineNoSessions = ({ className }: { className?: string }) => {
574 |   const { t } = useTranslation("common");
575 |   return (
576 |     <InlineEmpty
577 |       icon={<Clock className="h-6 w-6 text-gray-400" />}
578 |       title={t("empty.noSessions.title")}
579 |       description={t("empty.noSessions.description")}
580 |       className={className}
581 |     />
582 |   );
583 | };
584 |
585 | export const InlineNoResponses = ({ className }: { className?: string }) => {
586 |   const { t } = useTranslation("common");
587 |   return (
588 |     <InlineEmpty
589 |       icon={<MessageSquare className="h-6 w-6 text-gray-400" />}
590 |       title={t("empty.noResponses.title")}
591 |       description={t("empty.noResponses.description")}
592 |       className={className}
593 |     />
594 |   );
595 | };
596 |
597 | export const InlineNoTimetable = ({ className }: { className?: string }) => {
598 |   const { t } = useTranslation("common");
599 |   return (
600 |     <InlineEmpty
601 |       icon={<CalendarDays className="h-6 w-6 text-gray-400" />}
602 |       title={t("empty.noTimetable.title")}
603 |       description={t("empty.noTimetable.description")}
604 |       className={className}
605 |     />
606 |   );
607 | };
608 |
609 | export const InlineNoBadges = ({ className }: { className?: string }) => {
610 |   const { t } = useTranslation("common");
611 |   return (
612 |     <InlineEmpty
613 |       icon={<Trophy className="h-6 w-6 text-gray-400" />}
614 |       title={t("empty.noBadges.title")}
615 |       description={t("empty.noBadges.description")}
616 |       className={className}
617 |     />
618 |   );
619 | };
620 |
621 | export const InlineNoXPData = ({ className }: { className?: string }) => {
622 |   const { t } = useTranslation("common");
623 |   return (
624 |     <InlineEmpty
625 |       icon={<TrendingUp className="h-6 w-6 text-gray-400" />}
626 |       title={t("empty.noXPData.title")}
627 |       description={t("empty.noXPData.description")}
628 |       className={className}
629 |     />
630 |   );
631 | };
632 |
633 | export const InlineNoImprovementData = ({
634 |   className,
635 | }: {
636 |   className?: string;
637 | }) => {
638 |   const { t } = useTranslation("common");
639 |   return (
640 |     <InlineEmpty
641 |       icon={<TrendingUp className="h-6 w-6 text-gray-400" />}
642 |       title={t("empty.noImprovementData.title")}
643 |       description={t("empty.noImprovementData.description")}
644 |       className={className}
645 |     />
646 |   );
647 | };
648 |
649 | export const InlineNoLeaderboardData = ({
650 |   className,
651 | }: {
652 |   className?: string;
653 | }) => {
654 |   const { t } = useTranslation("common");
655 |   return (
656 |     <InlineEmpty
657 |       icon={<Users className="h-6 w-6 text-gray-400" />}
658 |       title={t("empty.noLeaderboardData.title")}
659 |       description={t("empty.noLeaderboardData.description")}
660 |       className={className}
661 |     />
662 |   );
663 | };
664 |
665 | export const InlineNoCorrelationData = ({
666 |   className,
667 | }: {

```



```

668 |   className?: string;
669 | }) => {
670 |   const { t } = useTranslation("common");
671 |   return (
672 |     <InlineEmpty
673 |       icon={<BarChart3 className="h-6 w-6 text-gray-400" />}
674 |       title={t("empty.noCorrelationData.title")}
675 |       description={t("empty.noCorrelationData.description")}
676 |       className={className}
677 |     />
678 |   );
679 | };
680 |
681 | export const InlineNoBloomsData = ({ className }: { className?: string }) => {
682 |   const { t } = useTranslation("common");
683 |   return (
684 |     <InlineEmpty
685 |       icon={<BarChart3 className="h-6 w-6 text-gray-400" />}
686 |       title={t("empty.noBloomsData.title")}
687 |       description={t("empty.noBloomsData.description")}
688 |       className={className}
689 |     />
690 |   );
691 | };
692 |
693 | export const InlineNoUsageData = ({ className }: { className?: string }) => {
694 |   const { t } = useTranslation("common");
695 |   return (
696 |     <InlineEmpty
697 |       icon={<CalendarDays className="h-6 w-6 text-gray-400" />}
698 |       title={t("empty.noUsageData.title")}
699 |       description={t("empty.noUsageData.description")}
700 |       className={className}
701 |     />
702 |   );
703 | };
704 |
705 | export const InlineNoCitationData = ({ className }: { className?: string }) => {
706 |   const { t } = useTranslation("common");
707 |   return (
708 |     <InlineEmpty
709 |       icon={<FileText className="h-6 w-6 text-gray-400" />}
710 |       title={t("empty.noCitationData.title")}
711 |       description={t("empty.noCitationData.description")}
712 |       className={className}
713 |     />
714 |   );
715 | };
716 |
717 | export const InlineNoAttainmentData = ({
718 |   className,
719 | }: {
720 |   className?: string;
721 | }) => {
722 |   const { t } = useTranslation("common");
723 |   return (
724 |     <InlineEmpty
725 |       icon={<Target className="h-6 w-6 text-gray-400" />}
726 |       title={t("empty.noAttainmentData.title")}
727 |       description={t("empty.noAttainmentData.description")}
728 |       className={className}
729 |     />
730 |   );
731 | };
732 |
733 | export const InlineNoSemesters = ({ className }: { className?: string }) => {
734 |   const { t } = useTranslation("common");
735 |   return (
736 |     <InlineEmpty
737 |       icon={<CalendarDays className="h-6 w-6 text-gray-400" />}
738 |       title={t("empty.noSemesters.title")}
739 |       description={t("empty.noSemesters.description")}
740 |       className={className}
741 |     />
742 |   );
743 | };
744 |
745 | export const InlineNoPLOs = ({ className }: { className?: string }) => {
746 |   const { t } = useTranslation("common");
747 |   return (
748 |     <InlineEmpty
749 |       icon={<Target className="h-6 w-6 text-gray-400" />}
750 |       title={t("empty.noPLOs.title")}
751 |       description={t("empty.noPLOs.description")}
752 |       className={className}
753 |     />
754 |   );
755 | };
756 |
757 | export const InlineNoILOs = ({ className }: { className?: string }) => {
758 |   const { t } = useTranslation("common");
759 |   return (
760 |     <InlineEmpty
761 |       icon={<GraduationCap className="h-6 w-6 text-gray-400" />}
762 |       title={t("empty.noILOs.title")}
763 |       description={t("empty.noILOs.description")}
764 |       className={className}
765 |     />
766 |   );
767 | };
768 |
769 | export const InlineNoStudents = ({ className }: { className?: string }) => {
770 |   const { t } = useTranslation("common");
771 |   return (

```

```

772 |         <InlineEmpty
773 |             icon={<Users className="h-6 w-6 text-gray-400" />}
774 |             title={t("empty.noStudents.title")}
775 |             description={t("empty.noStudents.description")}
776 |             className={className}
777 |         />
778 |     );
779 | };
780 |

```

FILE: src/components/shared/ErrorBoundary.tsx

TYPE: .tsx | LINES: 1-69 | SHA256: 6af1a523ef2b8b962aace828adb01bb4afca6eb869b13876aa459f6893a2ac38

```

1 | import { Component, type ReactNode, type ErrorInfo } from "react";
2 | import ErrorState from "@components/shared/ErrorState";
3 | import { AlertTriangle } from "lucide-react";
4 |
5 | interface Props {
6 |     children: ReactNode;
7 |     fallback?: ReactNode;
8 | }
9 |
10 | interface State {
11 |     hasError: boolean;
12 |     error: Error | null;
13 | }
14 |
15 | class ErrorBoundary extends Component<Props, State> {
16 |     constructor(props: Props) {
17 |         super(props);
18 |         this.state = { hasError: false, error: null };
19 |     }
20 |
21 |     static getDerivedStateFromError(error: Error): State {
22 |         return { hasError: true, error };
23 |     }
24 |
25 |     componentDidCatch(error: Error, errorInfo: ErrorInfo): void {
26 |         console.error("ErrorBoundary caught:", error, errorInfo);
27 |         // Task 97.2: Report to Sentry if initialized
28 |         import("@sentry/react")
29 |             .then((Sentry) => {
30 |                 if (Sentry.isInitialized()) {
31 |                     Sentry.captureException(error, {
32 |                         extra: { componentStack: errorInfo.componentStack },
33 |                     });
34 |                 }
35 |             })
36 |             .catch(() => {
37 |                 /* Sentry not available */
38 |             });
39 |     }
40 |
41 |     handleRetry = (): void => {
42 |         this.setState({ hasError: false, error: null });
43 |     };
44 |
45 |     render(): ReactNode {
46 |         if (this.state.hasError) {
47 |             if (this.props.fallback) return this.props.fallback;
48 |
49 |             return (
50 |                 <div className="flex items-center justify-center min-h-[200px] p-6">
51 |                     <ErrorState
52 |                         title="Something went wrong"
53 |                         message={
54 |                             this.state.error?.message ?? "An unexpected error occurred."
55 |                         }
56 |                         icon={<AlertTriangle className="h-8 w-8 text-red-500" />}
57 |                         onRetry={this.handleRetry}
58 |                         retryLabel="Try Again"
59 |                     />
60 |                 </div>
61 |             );
62 |         }
63 |
64 |         return this.props.children;
65 |     }
66 | }
67 |
68 | export default ErrorBoundary;
69 |

```

FILE: src/components/shared/ErrorState.tsx

TYPE: .tsx | LINES: 1-51 | SHA256: 9e6ea87a0ca016702f2e53ca9eace1578e02ea571217feb6ee98f195de40ceac

```

1 | // =====
2 | // ErrorState - Error state with retry button
3 | // =====
4 |
5 | import { AlertCircle, RefreshCw } from "lucide-react";
6 | import { Button } from "@components/ui/button";
7 | import { cn } from "@lib/utills";
8 |
9 | interface ErrorStateProps {
10 |     title?: string;
11 |     message: string;
12 |     icon?: React.ReactNode;
13 |     onRetry?: () => void;
14 |     retryLabel?: string;
15 |     children?: React.ReactNode;
16 |     className?: string;
17 | }
18 |

```

```

19 const ErrorState = ({
20   title = "Something went wrong",
21   message,
22   icon,
23   onRetry,
24   retryLabel = "Try Again",
25   children,
26   className,
27 }: ErrorStateProps) => (
28   <div
29     className={cn(
30       "flex flex-col items-center justify-center py-12 text-center",
31       className
32     )}
33   >
34     <div className="mb-4 rounded-full bg-red-50 p-4">
35       {icon ?? <AlertCircle className="h-8 w-8 text-red-500" />}
36     </div>
37     <h3 className="text-lg font-bold tracking-tight text-gray-900">{title}</h3>
38     <p className="mt-1 text-sm text-gray-500 max-w-sm">{message}</p>
39     {onRetry && (
40       <Button onClick={onRetry} variant="outline" className="mt-4">
41         <RefreshCw className="h-4 w-4" />
42         {retryLabel}
43       </Button>
44     )}
45     {children && <div className="mt-4">{children}</div>}
46   </div>
47 );
48
49 export default ErrorState;
50 export type { ErrorStateProps };
51

```

FILE: src/components/shared/EvidenceUploader.tsx

TYPE: .tsx | LINES: 1-318 | SHA256: c28f38fc6e6f5766cbf94db5657e96875401615ea0a787d400ebdf717222bb3f

[illegible]

[illegible]

```

172         setIsDragOver(true);
173     },
174     [disabled, canAddMore]
175 );
176
177
178 const handleDragLeave = useCallback((e: React.DragEvent) => {
179     e.preventDefault();
180     e.stopPropagation();
181     setIsDragOver(false);
182 }, []);
183
184 const handleDrop = useCallback(
185     (e: React.DragEvent) => {
186         e.preventDefault();
187         e.stopPropagation();
188         setIsDragOver(false);
189
190         if (disabled || !canAddMore) return;
191
192         const droppedFiles = e.dataTransfer.files;
193         if (droppedFiles.length > 0) {
194             validateAndAddFiles(droppedFiles);
195         }
196     },
197     [disabled, canAddMore, validateAndAddFiles]
198 );
199
200 const handleFileInputChange = useCallback(
201     (e: React.ChangeEvent<HTMLInputElement>) => {
202         const selectedFiles = e.target.files;
203         if (selectedFiles && selectedFiles.length > 0) {
204             validateAndAddFiles(selectedFiles);
205         }
206         // Reset input so the same file can be selected again
207         if (fileInputRef.current) {
208             fileInputRef.current.value = "";
209         }
210     },
211     [validateAndAddFiles]
212 );
213
214 const handleBrowseClick = useCallback(() => {
215     fileInputRef.current?.click();
216 }, []);
217
218 const handleRemoveFile = useCallback(
219     (index: number) => {
220         setError(null);
221         onChange(files.filter((_, i) => i !== index));
222     },
223     [files, onChange]
224 );
225
226 return (
227     <div className={cn("space-y-3", className)}>
228         {/* Drop Zone */}
229         {/* Hidden file input – placed outside drop zone to avoid click propagation */}
230         <input
231             ref={fileInputRef}
232             type="file"
233             accept={ACCEPTED_EXTENSIONS}
234             multiple
235             className="hidden"
236             onChange={handleFileInputChange}
237             disabled={disabled}
238             aria-hidden="true"
239         />
240
241         {canAddMore && (
242             <div
243                 className={cn(
244                     "relative flex min-h-[120px] cursor-pointer flex-col items-center justify-center rounded-lg border
245                     ? "border-blue-400 bg-blue-50"
246                     : "border-gray-300 bg-gray-50 hover:border-gray-400 hover:bg-gray-100",
247                     disabled && "cursor-not-allowed opacity-50"
248                 )}
249                 onDragOver={handleDragOver}
250                 onDragLeave={handleDragLeave}
251                 onDrop={handleDrop}
252                 onClick={disabled ? undefined : handleBrowseClick}
253                 role="button"
254                 tabIndex={disabled ? -1 : 0}
255                 aria-label="Upload evidence files"
256                 onKeyDown={(e) => {
257                     if (e.key === "Enter" || e.key === " ") {
258                         e.preventDefault();
259                         handleBrowseClick();
260                     }
261                 }}
262             >
263
264                 <div className="flex flex-col items-center gap-2 text-center">
265                     {isDragOver ? (
266                         <ImageIcon className="h-8 w-8 text-blue-500" />
267                     ) : (
268                         <Upload className="h-8 w-8 text-gray-400" />
269                     )}
270                     <div>
271                         <p className="text-sm font-medium text-gray-600">
272                             {isDragOver
273                                 ? "Drop files here"
274                                 : "Drag & drop files or click to browse"}
275                         </p>

```

```

276 |         <p className="mt-1 text-xs text-gray-400">
277 |             JPG, PNG, PDF, DOC, DOCX – max 5MB each – up to {maxFiles} files
278 |         </p>
279 |     </div>
280 | </div>
281 | </div>
282 | })
283 |
284 | { /* Error Message */
285 | {error && (
286 |     <div
287 |         className="flex items-center gap-2 rounded-md bg-red-50 px-3 py-2 text-sm text-red-600"
288 |         role="alert"
289 |     >
290 |         <AlertCircle className="h-4 w-4 shrink-0" />
291 |         <span>{error}</span>
292 |     </div>
293 | )}
294 |
295 | { /* File List */
296 | {files.length > 0 && (
297 |     <div className="space-y-2">
298 |         {files.map((file, index) => (
299 |             <FileThumbnail
300 |                 key={` ${file.name}-${file.size}-${index}`}
301 |                 file={file}
302 |                 onRemove={() => handleRemoveFile(index)}
303 |                 disabled={disabled}
304 |             />
305 |         ))}
306 |         <p className="text-xs text-gray-400">
307 |             {files.length} of {maxFiles} files selected
308 |         </p>
309 |     </div>
310 | )}
311 | </div>
312 | );
313 | };
314 |
315 | export default EvidenceUploader;
316 | export type { EvidenceUploaderProps };
317 | export { MAX_FILES, MAX_FILE_SIZE_BYTES, ACCEPTED_MIME_TYPES };
318 |

```

FILE: src/components/shared/ExplanationConfidenceBadge.tsx

TYPE: .tsx | LINES: 1-71 | SHA256: 294acac1ba74501c19c59486a26fd459cd5deba86ca9b193175f475071e77a38

```

1 | import { Badge } from "@components/ui/badge";
2 | import { cn } from "@lib/utils";
3 | import { CheckCircle2, AlertTriangle, ShieldCheck } from "lucide-react";
4 |
5 | export interface ExplanationConfidenceBadgeProps {
6 |     confidence: number | null;
7 |     isVerified: boolean;
8 | }
9 |
10 | const ExplanationConfidenceBadge = ({
11 |     confidence,
12 |     isVerified,
13 | }: ExplanationConfidenceBadgeProps) => {
14 |     if (isVerified) {
15 |         return (
16 |             <Badge
17 |                 variant="outline"
18 |                 className={cn(
19 |                     "text-xs font-bold tracking-wide uppercase border-transparent",
20 |                     "bg-blue-100 text-blue-700"
21 |                 )}
22 |                 title={
23 |                     confidence !== null
24 |                     ? `Confidence: ${((confidence * 100).toFixed(0))}%`
25 |                     : undefined
26 |                 }
27 |             >
28 |                 <ShieldCheck className="h-3 w-3" />
29 |                 Teacher verified
30 |             </Badge>
31 |         );
32 |     }
33 |
34 |     if (confidence === null) {
35 |         return null;
36 |     }
37 |
38 |     if (confidence >= 0.8) {
39 |         return (
40 |             <Badge
41 |                 variant="outline"
42 |                 className={cn(
43 |                     "text-xs font-bold tracking-wide uppercase border-transparent",
44 |                     "bg-green-100 text-green-700"
45 |                 )}
46 |                 title={`Confidence: ${((confidence * 100).toFixed(0))}%`}
47 |             >
48 |                 <CheckCircle2 className="h-3 w-3" />
49 |                 Verified by course materials
50 |             </Badge>
51 |         );
52 |     }
53 |
54 |     return (
55 |         <Badge
56 |             variant="outline"
57 |             className={cn(

```

```

58 |         "text-xs font-bold tracking-wide uppercase border-transparent",
59 |         "bg-amber-100 text-amber-700"
60 |     ))
61 |     title={`Confidence: ${((confidence * 100).toFixed(0))}%`}
62 |   >
63 |     <AlertTriangle className="h-3 w-3" />
64 |     This explanation may need teacher verification
65 |   </Badge>
66 | );
67 | };
68 |
69 | export { ExplanationConfidenceBadge };
70 | export default ExplanationConfidenceBadge;
71 |

```

FILE: [src/components/shared/ExportDataButton.tsx](#)

TYPE: .tsx | LINES: 1-95 | SHA256: fe7f29ede03bd66d57303cf8cb6fa3a42184e2ed004b2364a0360abdb05b2fab

```

1 | // =====
2 | // ExportDataButton – GDPR data export with format selector + download trigger
3 | // =====
4 |
5 | import { useState } from "react";
6 | import { Download, Loader2 } from "lucide-react";
7 | import { Button } from "@components/ui/button";
8 | import {
9 |   Select,
10 |   SelectContent,
11 |   SelectItem,
12 |   SelectTrigger,
13 |   SelectValue,
14 | } from "@components/ui/select";
15 | import { toast } from "sonner";
16 | import { supabase } from "@lib/supabase";
17 | import { cn } from "@lib/utils";
18 |
19 | type ExportFormat = "json" | "csv";
20 |
21 | interface ExportDataButtonProps {
22 |   studentId: string;
23 |   className?: string;
24 | }
25 |
26 | const ExportDataButton = ({ studentId, className }: ExportDataButtonProps) => {
27 |   const [format, setFormat] = useState<ExportFormat>("json");
28 |   const [isExporting, setIsExporting] = useState(false);
29 |
30 |   const handleExport = async () => {
31 |     setIsExporting(true);
32 |     try {
33 |       const { data, error } = await supabase.functions.invoke(
34 |         "export-student-data",
35 |         {
36 |           body: { student_id: studentId, format },
37 |         }
38 |       );
39 |
40 |       if (error) throw new Error(error.message);
41 |       if (!data?.download_url) throw new Error("No download URL returned");
42 |
43 |       // Open the signed URL to trigger download
44 |       const win = window.open(data.download_url, "_blank");
45 |       if (!win) {
46 |         // Popup blocked – fallback to anchor click
47 |         const a = document.createElement("a");
48 |         a.href = data.download_url;
49 |         a.target = "_blank";
50 |         a.rel = "noopener noreferrer";
51 |         a.click();
52 |       }
53 |       toast.success(`Data exported as ${format.toUpperCase()}`);
54 |     } catch (err) {
55 |       const message = err instanceof Error ? err.message : "Export failed";
56 |       toast.error(message);
57 |     } finally {
58 |       setIsExporting(false);
59 |     }
60 |   };
61 |
62 |   return (
63 |     <div className={cn("flex items-center gap-2", className)}>
64 |       <Select
65 |         value={format}
66 |         onValueChange={(v) => setFormat(v as ExportFormat)}
67 |       >
68 |         <SelectTrigger aria-label="Export format" className="w-24">
69 |           <SelectValue />
70 |         </SelectTrigger>
71 |         <SelectContent>
72 |           <SelectItem value="json">JSON</SelectItem>
73 |           <SelectItem value="csv">CSV</SelectItem>
74 |         </SelectContent>
75 |       </Select>
76 |       <Button
77 |         variant="outline"
78 |         onClick={handleExport}
79 |         disabled={isExporting}
80 |         aria-label="Download my data"
81 |       >
82 |         {isExporting ? (
83 |           <Loader2 className="h-4 w-4 animate-spin" />
84 |         ) : (
85 |           <Download className="h-4 w-4" />
86 |         )}

```

```

87 |         Download My Data
88 |     </Button>
89 | </div>
90 | );
91 | };
92 |
93 | export default ExportDataButton;
94 | export type { ExportDataButtonProps, ExportFormat };
95 |

```

FILE: [src/components/shared/ExtraAttemptUsageTable.tsx](#)

TYPE: .tsx | LINES: 1-88 | SHA256: 264537012154b5ea3a699aed6e8eff7146ce6ca0fc96f4ad62cd53cd8d02a1f2

```

1 | import { Card } from "@components/ui/card";
2 | import { Badge } from "@components/ui/badge";
3 | import { Ticket } from "lucide-react";
4 | import { Shimmer } from "@design-system";
5 | import { formatLocalDate } from "@lib/formatDate";
6 | import { useExtraAttemptUsage } from "@hooks/useExtraAttemptUsage";
7 |
8 | interface ExtraAttemptUsageTableProps {
9 |   quizID: string;
10 | }
11 |
12 | const ExtraAttemptUsageTable = ({ quizID }: ExtraAttemptUsageTableProps) => {
13 |   const { data, isLoading } = useExtraAttemptUsage(quizID);
14 |
15 |   if (isLoading) return <Shimmer className="h-32 rounded-xl" />;
16 |   if (!data || data.length === 0) return null;
17 |
18 |   return (
19 |     <Card className="bg-white border-0 shadow-md rounded-xl overflow-hidden gap-0 py-0">
20 |       <div
21 |         className="px-6 py-4 flex items-center gap-2"
22 |         style={{
23 |           background: "var(--brand-gradient)",
24 |         }}
25 |       >
26 |         <Ticket className="h-5 w-5 text-white" />
27 |         <h2 className="text-lg font-bold tracking-tight text-white">
28 |           Extra Attempt Token Usage
29 |         </h2>
30 |       </div>
31 |       <div className="p-6">
32 |         <div className="overflow-x-auto">
33 |           <table className="w-full text-sm">
34 |             <thead>
35 |               <tr className="border-b border-slate-200">
36 |                 <th className="text-start py-2 pe-4 font-semibold text-gray-600">
37 |                   Student
38 |                 </th>
39 |                 <th className="text-start py-2 pe-4 font-semibold text-gray-600">
40 |                   Attempt #
41 |                 </th>
42 |                 <th className="text-start py-2 pe-4 font-semibold text-gray-600">
43 |                   Date
44 |                 </th>
45 |                 <th className="text-start py-2 font-semibold text-gray-600">
46 |                   Score
47 |                 </th>
48 |               </tr>
49 |             </thead>
50 |             <tbody>
51 |               {data.map((usage, idx) => (
52 |                 <tr
53 |                   key={`_${usage.studentId}-${idx}`}
54 |                   className="border-b border-slate-100 last:border-0"
55 |                 >
56 |                   <td className="py-2 pe-4">
57 |                     <div className="flex items-center gap-2">
58 |                       <span className="font-medium">{usage.studentName}</span>
59 |                       <Badge className="text-xs bg-purple-50 text-purple-700 border-purple-200">
60 |                         Token Used
61 |                       </Badge>
62 |                     </div>
63 |                   </td>
64 |                   <td className="py-2 pe-4 text-gray-600">
65 |                     #{usage.attemptNumber}
66 |                   </td>
67 |                   <td className="py-2 pe-4 text-gray-600">
68 |                     {formatLocalDate(usage.purchaseDate, "MMM d, yyyy")}
69 |                   </td>
70 |                   <td className="py-2">
71 |                     {usage.score !== null ? (
72 |                       <span className="font-semibold">{usage.score}</span>
73 |                     ) : (
74 |                       <span className="text-gray-400">Pending</span>
75 |                     )}
76 |                   </td>
77 |                 </tr>
78 |               )}
79 |             </tbody>
80 |           </table>
81 |         </div>
82 |       </div>
83 |     </Card>
84 |   );
85 | };
86 |
87 | export default ExtraAttemptUsageTable;
88 |

```


FILE: src/components/shared/FeeStatusBadge.tsx

TYPE: .tsx | LINES: 1-43 | SHA256: bb6c913a8f6d2645bfeccecfb6c538329c213eb059378471e9f6b34b2ba32971

```

1 // =====
2 // FeeStatusBadge – Fee payment status badge
3 // =====
4
5 import { cn } from "@lib/utils";
6 import type { PaymentStatus } from "@types/app";
7
8 interface FeeStatusBadgeProps {
9   status: PaymentStatus;
10   className?: string;
11 }
12
13 const FEE_STATUS_STYLES: Record<
14   PaymentStatus,
15   { bg: string; text: string; label: string }
16 > = {
17   pending: { bg: "bg-yellow-100", text: "text-yellow-700", label: "Pending" },
18   paid: { bg: "bg-green-100", text: "text-green-700", label: "Paid" },
19   overdue: { bg: "bg-red-100", text: "text-red-700", label: "Overdue" },
20   waived: { bg: "bg-gray-100", text: "text-gray-700", label: "Waived" },
21 };
22
23 const FeeStatusBadge = ({ status, className }: FeeStatusBadgeProps) => {
24   const style = FEE_STATUS_STYLES[status];
25
26   return (
27     <span
28       className={cn(
29         "inline-flex items-center rounded-full px-2.5 py-0.5 text-xs font-bold tracking-wide uppercase",
30         style.bg,
31         style.text,
32         className
33       )}
34     > {style.label}
35     </span>
36   );
37 };
38
39 export default FeeStatusBadge;
40 export { FEE_STATUS_STYLES };
41 export type { FeeStatusBadgeProps };
42
43

```

FILE: src/components/shared/FlowCheckInDialog.tsx

TYPE: .tsx | LINES: 1-213 | SHA256: 91494025e87599af66e0e05ed896106c49cba7c9307b882d45b07aa99983c785

[illegible]

```

54 |     {
55 |         onSuccess: () => {
56 |             onComplete?.(response);
57 |         },
58 |     }
59 | );
60 | };
61 |
62 | const handleDismiss = () => {
63 |     setSelected(null);
64 |     setOpenChange(false);
65 | };
66 |
67 | return (
68 |     <Dialog
69 |       open={open}
70 |       onOpenChange={(o) => {
71 |         if (!o) {
72 |           setSelected(null);
73 |         }
74 |         setOpenChange(o);
75 |       }}
76 |     >
77 |       <DialogContent className="sm:max-w-md" aria-label="Flow check-in">
78 |         <DialogHeader>
79 |           <DialogTitle className="flex items-center gap-2">
80 |             <Sparkles className="h-5 w-5 text-teal-500" />
81 |             Quick Check-In
82 |           </DialogTitle>
83 |           <DialogDescription>
84 |             Interval {intervalNumber} done – how did that feel?
85 |           </DialogDescription>
86 |         </DialogHeader>
87 |
88 |         {/* Response buttons – shown when no selection yet */}
89 |         {!selected && (
90 |           <div
91 |             className="grid grid-cols-1 gap-3 sm:grid-cols-3"
92 |             role="group"
93 |             aria-label="Flow state options"
94 |           >
95 |             <Button
96 |               variant="outline"
97 |               className={cn(
98 |                 "flex h-auto flex-col items-center gap-2 py-4",
99 |                 "border-green-200 hover:bg-green-50 hover:border-green-400 hover:text-green-700"
100 |               )}
101 |               onClick={() => handleRespond("in_the_zone")}
102 |               disabled={saveCheckIn.isPending}
103 |               aria-label="In the zone"
104 |             >
105 |               <Sparkles className="h-6 w-6 text-green-600" />
106 |               <span className="text-xs font-medium">In the zone</span>
107 |             </Button>
108 |
109 |             <Button
110 |               variant="outline"
111 |               className={cn(
112 |                 "flex h-auto flex-col items-center gap-2 py-4",
113 |                 "border-amber-200 hover:bg-amber-50 hover:border-amber-400 hover:text-amber-700"
114 |               )}
115 |               onClick={() => handleRespond("stuck")}
116 |               disabled={saveCheckIn.isPending}
117 |               aria-label="Stuck"
118 |             >
119 |               <HelpCircle className="h-6 w-6 text-amber-600" />
120 |               <span className="text-xs font-medium">Stuck</span>
121 |             </Button>
122 |
123 |             <Button
124 |               variant="outline"
125 |               className={cn(
126 |                 "flex h-auto flex-col items-center gap-2 py-4",
127 |                 "border-blue-200 hover:bg-blue-50 hover:border-blue-400 hover:text-blue-700"
128 |               )}
129 |               onClick={() => handleRespond("too_easy")}
130 |               disabled={saveCheckIn.isPending}
131 |               aria-label="Too easy"
132 |             >
133 |               <Zap className="h-6 w-6 text-blue-600" />
134 |               <span className="text-xs font-medium">Too easy</span>
135 |             </Button>
136 |           </div>
137 |         )}
138 |
139 |         {/* Loading indicator while saving */}
140 |         {saveCheckIn.isPending && (
141 |           <div className="flex items-center justify-center gap-2 text-xs text-gray-400">
142 |             <Loader2 className="h-3 w-3 animate-spin" />
143 |             Saving...
144 |           </div>
145 |         )}
146 |
147 |         {/* Conditional follow-up: "In the zone" */}
148 |         {selected === "in_the_zone" && (
149 |           <div className="rounded-lg bg-green-50 p-4">
150 |             <p className="text-sm font-medium text-green-900">
151 |               Great flow! Keep it up – we'll check back next break.
152 |             </p>
153 |           </div>
154 |         )}
155 |
156 |         {/* Conditional follow-up: "Stuck" !' AI Tutor link */}
157 |         {selected === "stuck" && (

```

```

158 <div className="rounded-lg bg-amber-50 p-4">
159   <p className="text-sm font-medium text-amber-900">
160     Stuck happens – want a hand?
161   </p>
162   <p className="mt-1 text-xs text-amber-800">
163     The AI Tutor can walk you through the concept step by step.
164   </p>
165   <Link to="/student/tutor">
166     <Button
167       className="mt-3 gap-2 bg-gradient-to-r from-amber-500 to-orange-600 active:scale-95"
168       size="sm"
169     >
170       <HelpCircle className="h-4 w-4" />
171       Open AI Tutor
172     </Button>
173   </Link>
174 </div>
175 )}
176
177 /* Conditional follow-up: "Too easy" !' Bloom's suggestion */
178 {selected === "too_easy" && (
179   <div className="rounded-lg bg-blue-50 p-4">
180     <p className="text-sm font-medium text-blue-900">
181       Ready for more challenge?
182     </p>
183     <p className="mt-1 text-xs text-blue-800 flex items-center gap-1">
184       <ChevronUp className="h-3 w-3" />
185       Try moving up a Bloom's level – go from <em>
186         Understanding
187       </em> !' <em>Applying</em> or <em>Analyzing</em>. Explain the why,
188       or apply the concept to a new scenario.
189     </p>
190   </div>
191 )}
192
193 /* Dismiss / Continue button */
194 <div className="flex justify-end">
195   <Button
196     variant="ghost"
197     onClick={handleDismiss}
198     size="sm"
199     className="gap-2"
200     aria-label={selected ? "Continue to break" : "Dismiss check-in"}
201   >
202     {!selected && <X className="h-4 w-4" />}
203     {selected ? "Continue" : "Dismiss"}
204   </Button>
205 </div>
206 </DialogContent>
207 </Dialog>
208 );
209 };
210
211 export default FlowCheckInDialog;
212 export type { FlowCheckInDialogProps };
213

```

FILE: src/components/shared/FocusTimer.tsx

```
TYPE: .tsx | LINES: 1-491 | SHA256: c2a30fbcabd4a08b84a87a91239d099337b0df84aad318852f5cceb05613f720b
```

[illegible]

[illegible]

[illegible]

```

253     { /* Visual Notification Fallback */
254     {showVisualNotification && (
255         <div
256             className="flex items-center gap-2 rounded-lg bg-green-50 px-4 py-3 text-sm text-green-700"
257             role="alert"
258         >
259             <Bell className="h-4 w-4" />
260             <span className="font-medium">Timer completed!</span>
261             <Button
262                 variant="ghost"
263                 size="sm"
264                 className="ms-2 h-7 text-xs"
265                 onClick={dismissVisualNotification}
266             >
267                 Dismiss
268             </Button>
269         </div>
270     )}
271
272     { /* Session Context */
273     {session && (
274         <div className="flex flex-col items-center gap-1 text-center">
275             <h2 className="text-lg font-bold tracking-tight text-gray-900">
276                 {session.title}
277             </h2>
278             {session.courseName && (
279                 <span className="text-sm text-gray-500">{session.courseName}</span>
280             )}
281             {cloTitles.length > 0 && (
282                 <div className="mt-1 flex flex-wrap justify-center gap-1">
283                     {cloTitles.map((title, i) => (
284                         <Badge
285                             key={i}
286                             className="bg-green-100 text-green-700 text-[10px]"
287                         >
288                             {title}
289                         </Badge>
290                     ))}
291                 </div>
292             )}
293         </div>
294     )}
295
296     { /* Session Intent */
297     {intentConcept && (
298         <div className="flex max-w-sm items-start gap-3 rounded-lg border-teal-200 bg-teal-50 px-4 py-3"
299             <Target className="mt-0.5 h-4 w-4 shrink-0 text-teal-600" />
300             <div className="min-w-0 space-y-0.5">
301                 <p className="text-sm font-medium text-teal-800">{intentConcept}</p>
302                 {intentSuccessCriterion && (
303                     <p className="text-xs text-teal-600">{intentSuccessCriterion}</p>
304                 )}
305             </div>
306         </div>
307     )}
308
309     { /* Pomodoro Indicator */
310     {mode === "pomodoro" && (
311         <PomodoroIndicator
312             currentInterval={pomodoroInterval}
313             intervalType={pomodoroIntervalType}
314         />
315     )}
316
317     { /* Timer Display */
318     <div
319         className={cn(
320             "flex h-48 items-center justify-center rounded-full border-4 transition-colors sm:h-56 sm:w-56"
321             , ringColor
322         )}
323     >
324         <span
325             className="text-5xl font-black tabular-nums tracking-tight text-gray-900 sm:text-6xl"
326             aria-label={ `Time remaining: ${display}` }
327         >
328             {display}
329         </span>
330     </div>
331
332     { /* ARIA Live Region */
333     <div
334         aria-live="polite"
335         aria-atomic="true"
336         className="sr-only"
337         role="timer"
338     >
339         {announcement}
340     </div>
341
342     { /* Controls */
343     <div className="flex items-center gap-3">
344         { /* Idle - initial start */
345         {timerState === "idle" && !isWaitingForNextWork && (
346             <Button
347                 size="lg"
348                 variant="tactile"
349                 className="h-12 min-w-[120px] gap-2 text-base"
350                 onClick={handleStart}
351             >
352                 <Play className="h-5 w-5" />
353                 Start
354             </Button>
355         )}
356

```

```

357 |      { /* Waiting for next Pomodoro work interval */
358 |      {isWaitingForNextWork && (
359 |      <>
360 |          <Button
361 |              size="lg"
362 |              variant="tactile"
363 |              className="h-12 min-w-[120px] gap-2 text-base"
364 |              onClick={handleStartNextWork}
365 |          >
366 |              <Play className="h-5 w-5" />
367 |              Start Next
368 |          </Button>
369 |          <Button
370 |              variant="outline"
371 |              size="lg"
372 |              className="h-12 gap-2 text-base"
373 |              onClick={handleEnd}
374 |          >
375 |              <Square className="h-4 w-4" />
376 |              End Session
377 |          </Button>
378 |      </>
379 |      )}
380 |
381 |      { /* Running – pause + end */
382 |      {timerState === "running" && (
383 |      <>
384 |          <Button
385 |              variant="outline"
386 |              size="lg"
387 |              className="h-12 min-w-[100px] gap-2 text-base"
388 |              onClick={handlePause}
389 |          >
390 |              <Pause className="h-5 w-5" />
391 |              Pause
392 |          </Button>
393 |          <Button
394 |              variant="outline"
395 |              size="lg"
396 |              className="h-12 gap-2 text-base text-red-600 hover:bg-red-50 hover:text-red-700"
397 |              onClick={handleEnd}
398 |          >
399 |              <Square className="h-4 w-4" />
400 |              End
401 |          </Button>
402 |      </>
403 |      )}
404 |
405 |      { /* Paused – resume + end */
406 |      {timerState === "paused" && (
407 |      <>
408 |          <Button
409 |              size="lg"
410 |              variant="tactile"
411 |              className="h-12 min-w-[120px] gap-2 text-base"
412 |              onClick={handleResume}
413 |          >
414 |              <Play className="h-5 w-5" />
415 |              Resume
416 |          </Button>
417 |          <Button
418 |              variant="outline"
419 |              size="lg"
420 |              className="h-12 gap-2 text-base text-red-600 hover:bg-red-50 hover:text-red-700"
421 |              onClick={handleEnd}
422 |          >
423 |              <Square className="h-4 w-4" />
424 |              End
425 |          </Button>
426 |      </>
427 |      )}
428 |
429 |      { /* On break – skip break + end */
430 |      {isOnBreak && (
431 |      <>
432 |          <Button
433 |              variant="outline"
434 |              size="lg"
435 |              className="h-12 gap-2 text-base"
436 |              onClick={handleSkipBreak}
437 |          >
438 |              <SkipForward className="h-4 w-4" />
439 |              Skip Break
440 |          </Button>
441 |          <Button
442 |              variant="outline"
443 |              size="lg"
444 |              className="h-12 gap-2 text-base text-red-600 hover:bg-red-50 hover:text-red-700"
445 |              onClick={handleEnd}
446 |          >
447 |              <Square className="h-4 w-4" />
448 |              End
449 |          </Button>
450 |      </>
451 |      )}
452 |
453 |      { /* Completed – audio icon indicator */
454 |      {timerState === "completed" && (
455 |      <div className="flex items-center gap-2 text-green-600">
456 |          <Volume2 className="h-5 w-5" />
457 |          <span className="text-sm font-medium">Session Complete</span>
458 |      </div>
459 |      )}
460 |  </div>

```

```

461 |
462 |     /* Flow Check-In Dialog (Pomodoro break transitions + custom midpoint) */
463 |     {session && (
464 |         <FlowCheckInDialog
465 |             key={`flow-${session.id}-${flowCheckInInterval}`}
466 |             open={flowDialogOpen}
467 |             onOpenChange={setFlowDialogOpen}
468 |             sessionId={session.id}
469 |             intervalNumber={flowCheckInInterval}
470 |             onComplete={handleFlowComplete}
471 |         />
472 |     )}
473 |
474 |     /* Timer mode badge */
475 |     <Badge
476 |         className={cn(
477 |             "text-xs",
478 |             mode === "pomodoro"
479 |               ? "bg-blue-100 text-blue-700"
480 |               : "bg-gray-100 text-gray-600"
481 |         )}
482 |     >
483 |         {mode === "pomodoro" ? "Pomodoro Mode" : "Custom Timer"}
484 |     </Badge>
485 | </div>
486 | );
487 | };
488 |
489 | export default FocusTimer;
490 | export type { FocusTimerProps };
491 |

```

FILE: src/components/shared/GamificationFeedbackHost.tsx

TYPE: .tsx | LINES: 1-124 | SHA256: 3103203773abaf084a2b645ce80d725acb450decfc0927b12fdd00b040fcc535

```

1 | import { useEffect, useState } from "react";
2 | import { useTranslation } from "react-i18next";
3 | import BadgeAwardModal from "@components/shared/BadgeAwardModal";
4 | import LevelUpOverlay from "@components/shared/LevelUpOverlay";
5 | import MysteryRewardBox from "@components/shared/MysteryRewardBox";
6 | import XPAwardToast from "@components/shared/XPAwardToast";
7 | import { useAuth } from "@hooks/useAuth";
8 | import {
9 |     BADGE_FEEDBACK_EVENT,
10 |     type BadgeFeedbackDetail,
11 | } from "@lib/badgeFeedback";
12 | import { XP_FEEDBACK_EVENT, type XPFeedbackDetail } from "@lib/xpFeedback";
13 |
14 | /**
15 |  * One app-wide host for confirmed XP rewards. Events are queued so rapid,
16 |  * legitimate awards are shown in order instead of replacing one another.
17 |  */
18 | const GamificationFeedbackHost = () => {
19 |     const { t } = useTranslation("gamification");
20 |     const { profile } = useAuth();
21 |     const [queue, setQueue] = useState<XPFeedbackDetail[]>([]);
22 |     const [badgeQueue, setBadgeQueue] = useState<BadgeFeedbackDetail[]>([]);
23 |     const [phase, setPhase] = useState<"xp" | "level-up" | "mystery">("xp");
24 |     const current = queue[0];
25 |     const currentBadge = badgeQueue[0];
26 |
27 |     useEffect(() => {
28 |         const handleFeedback = (event: Event) => {
29 |             const detail = (event as CustomEvent<XPFeedbackDetail>).detail;
30 |             if (!detail || detail.amount <= 0) return;
31 |             setQueue((items) => [...items, detail]);
32 |         };
33 |         const handleBadgeFeedback = (event: Event) => {
34 |             const detail = (event as CustomEvent<BadgeFeedbackDetail>).detail;
35 |             if (!detail) return;
36 |             setBadgeQueue((items) => [...items, detail]);
37 |         };
38 |
39 |         window.addEventListener(XP_FEEDBACK_EVENT, handleFeedback);
40 |         window.addEventListener(BADGE_FEEDBACK_EVENT, handleBadgeFeedback);
41 |         return () => {
42 |             window.removeEventListener(XP_FEEDBACK_EVENT, handleFeedback);
43 |             window.removeEventListener(BADGE_FEEDBACK_EVENT, handleBadgeFeedback);
44 |         };
45 |     }, []);
46 |
47 |     if (!current) {
48 |         if (!currentBadge) return null;
49 |         return (
50 |             <BadgeAwardModal
51 |                 badge={currentBadge}
52 |                 isOpen
53 |                 onClose={() => setBadgeQueue((items) => items.slice(1))}
54 |             />
55 |         );
56 |     }
57 |
58 |     const completeCurrent = () => {
59 |         setQueue((items) => items.slice(1));
60 |         setPhase("xp");
61 |     };
62 |
63 |     const continueAfterLevelUp = () => {
64 |         if (current.mysteryRewardTriggered && profile?.institution_id) {
65 |             setPhase("mystery");
66 |             return;
67 |         }
68 |         completeCurrent();
69 |     };

```



```

70 |
71 | if (phase === "level-up") {
72 |   return (
73 |     <LevelUpOverlay
74 |       key={`level-${current.id}`}
75 |       newLevel={current.newLevel}
76 |       onComplete={continueAfterLevelUp}
77 |     />
78 |   );
79 | }
80 |
81 | if (phase === "mystery" && profile?.institution_id) {
82 |   return (
83 |     <div
84 |       className="fixed inset-0 z-[220] flex items-center justify-center bg-slate-950/60 p-5 backdrop-blur-sm
85 |       role="dialog"
86 |       aria-modal="true"
87 |       aria-label={t("mystery.overlayLabel", "Mystery reward")}
88 |     >
89 |       <div className="w-full max-w-sm rounded-3xl bg-white p-8 shadow-2xl">
90 |         <MysteryRewardBox
91 |           key={`mystery-${current.id}`}
92 |           studentId={current.studentId}
93 |           institutionId={profile.institution_id}
94 |           onComplete={completeCurrent}
95 |         />
96 |       </div>
97 |     </div>
98 |   );
99 | }
100 |
101 | return (
102 |   <div className="pointer-events-none fixed inset-x-0 bottom-20 z-[210] flex justify-center lg:bottom-6 lg:m
103 |   <XPAwardToast
104 |     key={current.id}
105 |     xpAmount={current.amount}
106 |     source={t("xp.rewardEarned")}
107 |     onComplete={() => {
108 |       if (current.levelUp) {
109 |         setPhase("level-up");
110 |         return;
111 |       }
112 |       if (current.mysteryRewardTriggered && profile?.institution_id) {
113 |         setPhase("mystery");
114 |         return;
115 |       }
116 |       completeCurrent();
117 |     }}
118 |   />
119 | </div>
120 | );
121 | };
122 |
123 | export default GamificationFeedbackHost;
124 |

```

FILE: src/components/shared/GapStatusBadge.tsx

TYPE: .tsx | LINES: 1-65 | SHA256: 0e8681081da69bead44462c1243c412e5e9f0dae04a35fd37306b171e98e6770

```

1 | // Task 119.1: Gap Status Badge shared component
2 |
3 | import { Badge } from "@components/ui/badge";
4 | import { CheckCircle, AlertTriangle, XCircle, HelpCircle } from "lucide-react";
5 | import type { GapStatus, GapFlag } from "@lib/gapAnalysis";
6 |
7 | interface GapStatusBadgeProps {
8 |   status: GapStatus;
9 |   flag?: GapFlag;
10 | }
11 |
12 | const STATUS_CONFIG: Record<
13 |   GapStatus,
14 |   { label: string; className: string; icon: typeof CheckCircle }
15 | > = {
16 |   fully_mapped: {
17 |     label: "Fully Mapped",
18 |     className: "bg-green-100 text-green-700",
19 |     icon: CheckCircle,
20 |   },
21 |   partially_mapped: {
22 |     label: "Partially Mapped",
23 |     className: "bg-yellow-100 text-yellow-700",
24 |     icon: AlertTriangle,
25 |   },
26 |   unmapped: {
27 |     label: "Unmapped",
28 |     className: "bg-red-100 text-red-700",
29 |     icon: XCircle,
30 |   },
31 |   no_evidence: {
32 |     label: "No Evidence",
33 |     className: "bg-gray-100 text-gray-700",
34 |     icon: HelpCircle,
35 |   },
36 | };
37 |
38 | const GapStatusBadge = ({ status, flag }: GapStatusBadgeProps) => {
39 |   const config = STATUS_CONFIG[status];
40 |   const Icon = config.icon;
41 |
42 |   return (
43 |     <div className="flex items-center gap-1">
44 |       <Badge className={`text-[10px] ${config.className}`}>
45 |         <Icon className="h-3 w-3 me-0.5" />

```

```

46 |         {config.label}
47 |       </Badge>
48 |     {flag === "under_mapped" && (
49 |       <Badge className="text-[10px] bg-amber-100 text-amber-700">
50 |         <AlertTriangle className="h-3 w-3 me-0.5" />
51 |         Under-Mapped
52 |       </Badge>
53 |     )}
54 |     {flag === "unassessed" && (
55 |       <Badge className="text-[10px] bg-amber-100 text-amber-700">
56 |         <AlertTriangle className="h-3 w-3 me-0.5" />
57 |         Unassessed
58 |       </Badge>
59 |     )}
60 |   </div>
61 | );
62 | };
63 |
64 | export default GapStatusBadge;
65 |

```

FILE: src/components/shared/GibbsReflectionTemplate.tsx

```
TYPE: .tsx | LINES: 1-108 | SHA256: fdaffa6626d09f06ef496042e30f5a737f5c145612767c0feb4fc4a26a462313
```

[illegible]

```

81 |         {FIELDS.map((field) => (
82 |             <div key={field.key} className="space-y-1">
83 |                 <label>
84 |                     htmlFor={`gibbs-${field.key}`}
85 |                     className="text-sm font-medium text-gray-700"
86 |                 >
87 |                     {field.label}
88 |                 </label>
89 |                 <p className="text-xs text-gray-400">{field.hint}</p>
90 |                 <Textarea
91 |                     id={`gibbs-${field.key}`}
92 |                     placeholder={field.placeholder}
93 |                     rows={2}
94 |                     value={values[field.key]}
95 |                     onChange={(e) => onChange(field.key, e.target.value)}
96 |                     disabled={disabled}
97 |                     className="resize-y"
98 |                     data-testid={`gibbs-${field.key}`}
99 |                 />
100 |             </div>
101 |         )]}
102 |     </div>
103 | );
104 | };
105 |
106 | export default GibbsReflectionTemplate;
107 | export type { GibbsReflectionTemplateProps };
108 |

```

FILE: src/components/shared/GlobalHeader.tsx

TYPE: .tsx | LINES: 1-85 | SHA256: a8370d879c6a34221108aff5f98c89127542a7068a0fa4114011dda2adda85aa

```

1 | import { Link } from "react-router-dom";
2 | import { Menu } from "lucide-react";
3 | import { useAuth } from "@hooks/useAuth";
4 | import { LanguageSwitcher } from "@components/shared/LanguageSwitcher";
5 | import NotificationBell from "@components/shared/NotificationBell";
6 | import ProfileDropdown from "@components/shared/ProfileDropdown";
7 | import SearchCommand from "@components/shared/SearchCommand";
8 | import RoleHeaderStats from "@components/shared/RoleHeaderStats";
9 | import StudentHeaderStats from "@components/shared/StudentHeaderStats";
10 | import { useSidebar } from "@components/shared/SidebarContext";
11 | import { Button } from "@components/ui/button";
12 | import type { UserRole } from "@types/app";
13 |
14 | const dashboardRouteByRole: Record<UserRole, string> = {
15 |     admin: "/admin",
16 |     coordinator: "/coordinator",
17 |     teacher: "/teacher",
18 |     student: "/student",
19 |     parent: "/parent",
20 | };
21 |
22 | const GlobalHeader = () => {
23 |     const { profile } = useAuth();
24 |     const { toggle } = useSidebar();
25 |
26 |     const role = profile?.role ?? "student";
27 |     const dashboardRoute = dashboardRouteByRole[role as UserRole] ?? "/student";
28 |
29 |     return (
30 |         <header className="sticky top-0 z-[100] h-[var(--app-header-h)] w-full border-b border-slate-200/80 bg-white">
31 |             <div>
32 |                 data-tour="top-bar"
33 |                 className="mx-auto flex h-full w-full items-center justify-between gap-4 px-4 lg:px-6"
34 |             >
35 |                 { /* Left: Mobile Toggle & Brand Logo */ }
36 |                 <div className="flex items-center gap-3 flex-shrink-0">
37 |                     <Button
38 |                         type="button"
39 |                         variant="ghost"
40 |                         size="icon"
41 |                         onClick={toggle}
42 |                         className="rounded-lg text-gray-600 hover:bg-slate-100 min-[640px]:hidden"
43 |                         aria-label="Toggle navigation menu"
44 |                     >
45 |                         <Menu className="h-5 w-5" />
46 |                     </Button>
47 |
48 |                     <Link
49 |                         to={dashboardRoute}
50 |                         className="flex items-center gap-2.5 transition-opacity hover:opacity-90"
51 |                         aria-label="Edeviser - go to dashboard"
52 |                     >
53 |                         
58 |                         <span className="font-black text-xl tracking-tight text-slate-900 dark:text-white">
59 |                             Edeviser
60 |                         </span>
61 |                     </Link>
62 |                 </div>
63 |
64 |                 { /* Center: Search Command */ }
65 |                 <div className="absolute left-1/2 -translate-x-1/2 hidden min-[1280px]:block w-[360px]">
66 |                     <SearchCommand showTrigger />
67 |                 </div>
68 |
69 |                 { /* Right: Role Stats, Bell & Profile */ }
70 |                 <div className="ms-auto flex min-w-0 items-center gap-2.5">
71 |                     {role === "student" ? <StudentHeaderStats /> : null}
72 |                     <RoleHeaderStats />

```

```

73 |         <div className="hidden min-[1024px]:block">
74 |             <LanguageSwitcher />
75 |         </div>
76 |         <NotificationBell />
77 |         <ProfileDropdown />
78 |     </div>
79 | </div>
80 | </header>
81 | );
82 | };
83 |
84 | export default GlobalHeader;
85 |

```

FILE: src/components/shared/GoalDifficultyBadge.tsx

TYPE: .tsx | LINES: 1-49 | SHA256: 5a0bd2a311f5e09d5b6f972e85184f2af7d76b5df298d4aa616b9e28d238a0a2

```

1 | import { Badge } from "@components/ui/badge";
2 | import { cn } from "@lib/utils";
3 | import type { GoalDifficulty } from "@lib/goalTemplates";
4 |
5 | export interface GoalDifficultyBadgeProps {
6 |     difficulty: GoalDifficulty;
7 |     className?: string;
8 | }
9 |
10 | const DIFFICULTY_CONFIG: Record<
11 |     GoalDifficulty,
12 |     { label: string; classes: string }
13 | > = {
14 |     easy: {
15 |         label: "Easy",
16 |         classes: "bg-green-100 text-green-700 border-green-200",
17 |     },
18 |     moderate: {
19 |         label: "Moderate",
20 |         classes: "bg-amber-100 text-amber-700 border-amber-200",
21 |     },
22 |     ambitious: {
23 |         label: "Ambitious",
24 |         classes: "bg-red-100 text-red-700 border-red-200",
25 |     },
26 | };
27 |
28 | const GoalDifficultyBadge = ({
29 |     difficulty,
30 |     className,
31 | }: GoalDifficultyBadgeProps) => {
32 |     const config = DIFFICULTY_CONFIG[difficulty];
33 |
34 |     return (
35 |         <Badge
36 |             variant="outline"
37 |             className={cn(
38 |                 "text-[10px] font-bold tracking-wide uppercase",
39 |                 config.classes,
40 |                 className
41 |             )}
42 |         >
43 |             {config.label}
44 |         </Badge>
45 |     );
46 | };
47 |
48 | export default GoalDifficultyBadge;
49 |

```

FILE: src/components/shared/GoalSuggestionPanel.tsx

TYPE: .tsx | LINES: 1-123 | SHA256: 4d9b937591c9cde2b1cf964a6f6539a1bd7b3f2b2d6d6817c4b9a1c4233ab281

```

1 | import { Button } from "@components/ui/button";
2 | import { Target, Check, Pencil, X, Sparkles } from "lucide-react";
3 | import GoalDifficultyBadge from "@components/shared/GoalDifficultyBadge";
4 | import type { GoalDifficulty } from "@lib/goalTemplates";
5 | import { PCard, SectionHeader } from "@design-system";
6 |
7 | export interface GoalSuggestion {
8 |     id: string;
9 |     goal_text: string;
10 |     difficulty: GoalDifficulty;
11 |     cohort_completion_rate: number | null;
12 |     status: "suggested" | "accepted" | "modified" | "dismissed";
13 | }
14 |
15 | export interface GoalSuggestionPanelProps {
16 |     suggestions: GoalSuggestion[];
17 |     onAccept: (id: string) => void;
18 |     onEdit: (id: string) => void;
19 |     onDismiss: (id: string) => void;
20 |     isLoading?: boolean;
21 | }
22 |
23 | const GoalSuggestionPanel = ({
24 |     suggestions,
25 |     onAccept,
26 |     onEdit,
27 |     onDismiss,
28 |     isLoading = false,
29 | }: GoalSuggestionPanelProps) => {
30 |     const activeSuggestions = suggestions.filter((s) => s.status === "suggested");
31 |
32 |     if (isLoading) {
33 |         return (
34 |             <PCard className="p-6">

```

```

35 |         <div className="flex items-center gap-2 mb-4">
36 |             <Sparkles className="h-4 w-4 text-blue-500 animate-pulse" />
37 |             <span className="text-sm font-medium text-gray-500">
38 |                 Generating goal suggestions...
39 |             </span>
40 |         </div>
41 |         <div className="space-y-3">
42 |             {[1, 2, 3].map((i) => (
43 |                 <div
44 |                     key={i}
45 |                     className="h-20 rounded-lg animate-shimmer bg-slate-100"
46 |                 />
47 |             ))}
48 |         </div>
49 |     </PCard>
50 |   );
51 | }
52 |
53 | if (activeSuggestions.length === 0) return null;
54 |
55 | return (
56 |   <PCard className="overflow-hidden">
57 |     <div className="p-5 pb-4">
58 |       <SectionHeader icon={Target} title="Suggested Goals" />
59 |     </div>
60 |
61 |     <div className="space-y-3 px-5 pb-5">
62 |       <p className="text-xs text-gray-500 mb-2">
63 |         AI-suggested goals based on your courses and progress. Accept, edit,
64 |         or dismiss.
65 |       </p>
66 |
67 |       {activeSuggestions.map((goal) => (
68 |         <div
69 |           key={goal.id}
70 |           className="rounded-lg border border-slate-200 p-4 space-y-3"
71 |         >
72 |           <div className="flex items-start justify-between gap-2">
73 |             <p className="text-sm font-medium text-gray-900 flex-1">
74 |               {goal.goal_text}
75 |             </p>
76 |             <GoalDifficultyBadge difficulty={goal.difficulty} />
77 |           </div>
78 |
79 |           {goal.cohort_completion_rate != null && (
80 |             <p className="text-[10px] text-gray-400">
81 |               {goal.cohort_completion_rate}% of similar students completed
82 |               this type of goal
83 |             </p>
84 |           )}
85 |
86 |           <div className="flex items-center gap-2">
87 |             <Button
88 |               size="sm"
89 |               onClick={() => onAccept(goal.id)}
90 |               variant="tactile"
91 |               className="text-xs font-semibold"
92 |             >
93 |               <Check className="h-3 w-3" />
94 |               Accept
95 |             </Button>
96 |             <Button
97 |               variant="outline"
98 |               size="sm"
99 |               onClick={() => onEdit(goal.id)}
100 |              className="text-xs"
101 |            >
102 |              <Pencil className="h-3 w-3" />
103 |              Edit
104 |            </Button>
105 |            <Button
106 |              variant="ghost"
107 |              size="sm"
108 |              onClick={() => onDismiss(goal.id)}
109 |              className="text-xs text-gray-400"
110 |            >
111 |              <X className="h-3 w-3" />
112 |              Dismiss
113 |            </Button>
114 |          </div>
115 |        </div>
116 |      ))}
117 |    </div>
118 |  </PCard>
119 | );
120 | };
121 |
122 | export default GoalSuggestionPanel;
123 |

```

FILE: [src/components/shared/GradebookMatrix.tsx](#)

TYPE: .tsx | LINES: 1-111 | SHA256: cb9e7d8cb7937b1f52d7f1b1c5b56fb8e49beea1ae4d476860a882e9a7fe586d

```

1 | // =====
2 | // GradebookMatrix - Student x Assignment grade matrix
3 | // =====
4 | /* eslint-disable react-refresh/only-export-components */
5 |
6 | import { cn } from "@lib/utls";
7 |
8 | interface GradeCell {
9 |   score: number | null;
10 |   maxScore: number;
11 | }

```

```

12 |
13 | interface GradebookStudent {
14 |   studentId: string;
15 |   studentName: string;
16 |   grades: Record<string, GradeCell>;
17 | }
18 |
19 | interface GradebookAssignment {
20 |   id: string;
21 |   title: string;
22 |   maxScore: number;
23 | }
24 |
25 | interface GradebookMatrixProps {
26 |   students: GradebookStudent[];
27 |   assignments: GradebookAssignment[];
28 |   onCellClick?: (studentId: string, assignmentId: string) => void;
29 |   className?: string;
30 | }
31 |
32 | function getCellColor(score: number | null, maxScore: number): string {
33 |   if (score === null) return "bg-gray-50 text-gray-400";
34 |   const pct = (score / maxScore) * 100;
35 |   if (pct >= 85) return "bg-green-50 text-green-700";
36 |   if (pct >= 70) return "bg-blue-50 text-blue-700";
37 |   if (pct >= 50) return "bg-yellow-50 text-yellow-700";
38 |   return "bg-red-50 text-red-700";
39 | }
40 |
41 | const GradebookMatrix = ({
42 |   students,
43 |   assignments,
44 |   onCellClick,
45 |   className,
46 | }: GradebookMatrixProps) => (
47 |   <div
48 |     className={cn(
49 |       "overflow-auto rounded-xl border border-slate-200",
50 |       className
51 |     )}
52 |   >
53 |     <table className="w-full text-sm">
54 |       <thead>
55 |         <tr className="bg-slate-50">
56 |           <th className="sticky start-0 bg-slate-50 px-4 py-2 text-start font-medium text-gray-600 z-10">
57 |             Student
58 |           </th>
59 |           {assignments.map((a) => (
60 |             <th
61 |               key={a.id}
62 |               className="px-3 py-2 text-center font-medium text-gray-600 whitespace-nowrap"
63 |             >
64 |               <div className="truncate max-w-[100px]" title={a.title}>
65 |                 {a.title}
66 |               </div>
67 |               <div className="text-[10px] text-gray-400 font-normal">
68 |                 {a.maxScore}
69 |               </div>
70 |             </th>
71 |           ))}
72 |         </tr>
73 |       </thead>
74 |       <tbody>
75 |         {students.map((student) => (
76 |           <tr key={student.studentId} className="border-t border-slate-100">
77 |             <td className="sticky start-0 bg-white px-4 py-2 font-medium truncate max-w-[160px] z-10">
78 |               {student.studentName}
79 |             </td>
80 |             {assignments.map((a) => {
81 |               const cell = student.grades[a.id];
82 |               const score = cell?.score ?? null;
83 |               return (
84 |                 <td
85 |                   key={a.id}
86 |                   className={cn(
87 |                     "px-3 py-2 text-center font-medium tabular-nums cursor-pointer hover:opacity-80 transition
88 |                     getCellColor(score, a.maxScore)
89 |                   )}
90 |                   onClick={() => onCellClick?.(student.studentId, a.id)}
91 |                 >
92 |                   {score !== null ? score : "-"}
93 |                 </td>
94 |               );
95 |             })}
96 |             </tr>
97 |           ))}
98 |         </tbody>
99 |       </table>
100 |     </div>
101 |   );
102 |
103 | export default GradebookMatrix;
104 | export { getCellColor };
105 | export type {
106 |   GradebookMatrixProps,
107 |   GradebookStudent,
108 |   GradebookAssignment,
109 |   GradeCell,
110 | };
111 |

```

FILE: src/components/shared/GradientCardHeader.ts

TYPE: .ts | LINES: 1-5 | SHA256: 5beadfd2db8ae917a5c31fcee27f476b37eddaa2f85b177040d1542638736ac8

```
1 | // Re-export shim – implementation lives in the design system (single source).
2 | // Legacy direct importers keep working; deleted at P5 once none remain.
3 | export { default } from "@design-system/patterns/GradientCardHeader";
4 | export type { GradientCardHeaderProps } from "@design-system/patterns/GradientCardHeader";
5 |
```

FILE: src/components/shared/GradientCardHeader.tsx

TYPE: .tsx | LINES: 1-59 | SHA256: 896d2579328c5bc063b5f31d8463394446f342567d9ff920bacf2fc2673d3676

```
1 | // =====
2 | // GradientCardHeader – Reusable header for section cards
3 | // =====
4 | //
5 | // Shared section-card header: a brand-gradient icon CHIP + dark title on a
6 | // bordered strip, so every section card across the app coheres with the
7 | // redesigned dashboards. Callers are unchanged ('icon' + 'title' [+ optional
8 | // `children`]).
9 | //
10 | // NOTE: the callers wrap this in cards hardcoded to `bg-white` (light surface in
11 | // both themes), so the title uses a fixed dark color + light border (not theme
12 | // tokens) to stay readable regardless of the active theme.
13 | // =====
14 |
15 | import type { LucideIcon } from "lucide-react";
16 | import { cn } from "@lib/utils";
17 |
18 | interface GradientCardHeaderProps {
19 |   icon?: LucideIcon;
20 |   title: string;
21 |   className?: string;
22 |   children?: React.ReactNode;
23 | }
24 |
25 | const GradientCardHeader = ({
26 |   icon: Icon,
27 |   title,
28 |   className,
29 |   children,
30 | }: GradientCardHeaderProps) => {
31 |   // Brand-gradient icon chip + dark title on a bordered white strip
32 |   // (matches the redesigned dashboards' section-header look).
33 |   return (
34 |     <div
35 |       className={cn(
36 |         "flex items-center gap-3 border-b border-slate-100 px-6 py-4",
37 |         className
38 |       )}
39 |     >
40 |       {Icon && (
41 |         <span
42 |           className="inline-flex size-9 shrink-0 items-center justify-center rounded-lg text-white shadow-sm"
43 |           style={{ background: "var(--brand-gradient)" }}
44 |           aria-hidden="true"
45 |         >
46 |           <Icon className="h-5 w-5" />
47 |         </span>
48 |       )}
49 |       <h2 className="truncate text-lg font-bold tracking-tight text-gray-900">
50 |         {title}
51 |       </h2>
52 |       {children && <div className="ms-auto shrink-0">{children}</div>}
53 |     </div>
54 |   );
55 | };
56 |
57 | export default GradientCardHeader;
58 | export type { GradientCardHeaderProps };
59 |
```

FILE: src/components/shared/GraduateAttributeCard.tsx

TYPE: .tsx | LINES: 1-53 | SHA256: 5437f809c03f5472374b1784c02dffaaf992c27d47dc06aec958dec03ac40463

```
1 | // Task 113.2: Graduate Attribute Card shared component
2 |
3 | import { Card } from "@components/ui/card";
4 | import { Badge } from "@components/ui/badge";
5 |
6 | interface GraduateAttributeCardProps {
7 |   title: string;
8 |   code: string;
9 |   attainment: number;
10 |   mappedILOCount: number;
11 | }
12 |
13 | const getAttainmentStyle = (score: number) => {
14 |   if (score >= 85) return "text-green-600 bg-green-50";
15 |   if (score >= 70) return "text-blue-600 bg-blue-50";
16 |   if (score >= 50) return "text-yellow-600 bg-yellow-50";
17 |   return "text-red-600 bg-red-50";
18 | };
19 |
20 | const getAttainmentLabel = (score: number) => {
21 |   if (score >= 85) return "Excellent";
22 |   if (score >= 70) return "Satisfactory";
23 |   if (score >= 50) return "Developing";
24 |   return "Not Yet";
25 | };
26 |
27 | const GraduateAttributeCard = ({
28 |   title,
```

```

29 |   code,
30 |   attainment,
31 |   mappedILOCount,
32 | }: GraduateAttributeCardProps) => (
33 |   <Card className="bg-white border-0 shadow-md rounded-xl p-4 group">
34 |     <div className="flex items-start justify-between">
35 |       <div className="flex-1 min-w-0">
36 |         <p className="text-xs font-mono text-slate-400">{code}</p>
37 |         <p className="text-sm font-semibold mt-0.5 truncate">{title}</p>
38 |         <p className="text-xs text-slate-500 mt-1">
39 |           {mappedILOCount} mapped ILOs
40 |         </p>
41 |       </div>
42 |       <div className="text-end">
43 |         <p className="text-2xl font-black">{Math.round(attainment)}%</p>
44 |         <Badge className={`text-[10px] ${getAttainmentStyle(attainment)}`}>
45 |           {getAttainmentLabel(attainment)}
46 |         </Badge>
47 |       </div>
48 |     </div>
49 |   </Card>
50 | );
51 |
52 | export default GraduateAttributeCard;
53 |

```

FILE: src/components/shared/GuidedTour.tsx

TYPE: .tsx | LINES: 1-59 | SHA256: 3d60e6a414911a40d557e72ad03851b2297b02e1cdd649b20fbde92f66a0da54

```

1 | import { lazy, Suspense, useEffect, useState } from "react";
2 | import { useAuth } from "@/hooks/useAuth";
3 | import { useGuidedTour } from "@/hooks/useGuidedTour";
4 |
5 | // react-joyride lives entirely in TourRunner so it is code-split out of this
6 | // always-mounted component (every role layout renders <GuidedTour />). The
7 | // chunk is fetched only when a tour actually runs (Task 15.2 – load on first use).
8 | const TourRunner = lazy(() => import("@/components/shared/TourRunner"));
9 |
10 | /**
11 |  * Guided tour wrapper using react-joyride.
12 |  * Mounts in each role layout and auto-launches on first login if tourFeatureFlag is true.
13 |  *
14 |  * Design: ADR-02, §8.4
15 |  * Requirements: 2.15
16 |  *
17 |  * @example
18 |  * <GuidedTour />
19 |  */
20 | const GuidedTour = () => {
21 |   const { profile } = useAuth();
22 |   const { steps, run, onCallback, shouldAutoStart, isCompleted, start } =
23 |     useGuidedTour(profile?.role ?? "student");
24 |
25 |   // Track whether we've already auto-started this session
26 |   const [autoStarted, setAutoStarted] = useState(false);
27 |
28 |   // Auto-launch tour on first login if feature flag is enabled
29 |   useEffect(() => {
30 |     if (shouldAutoStart && !isCompleted && !autoStarted) {
31 |       const timer = setTimeout(() => {
32 |         start();
33 |         setAutoStarted(true);
34 |       }, 800); // Small delay to ensure DOM is ready
35 |       return () => clearTimeout(timer);
36 |     }
37 |   }, [shouldAutoStart, isCompleted, autoStarted, start]);
38 |
39 |   if (!steps.length || !run) {
40 |     return null;
41 |   }
42 |
43 |   // Only mount the react-joyride surface (and fetch its chunk) once a tour is
44 |   // actually running. Suspense fallback is null – the tour overlay simply
45 |   // appears a tick later, with no layout placeholder needed.
46 |   return (
47 |     <Suspense fallback={null}>
48 |       <TourRunner
49 |         steps={steps}
50 |         run={run}
51 |         onCallback={onCallback}
52 |         onFinishOrSkip={() => setAutoStarted(false)}
53 |       />
54 |     </Suspense>
55 |   );
56 | };
57 |
58 | export default GuidedTour;
59 |

```

FILE: src/components/shared/HabitCompletionChart.tsx

TYPE: .tsx | LINES: 1-146 | SHA256: 98e4f198a8d15cb8f91bc1b0447792339859bac87b801b51f9eb2e0752eb1f80

```

1 | import { useId, useState } from "react";
2 | import {
3 |   BarChart,
4 |   Bar,
5 |   XAxis,
6 |   YAxis,
7 |   Tooltip,
8 |   ResponsiveContainer,
9 | } from "recharts";
10 | import { cn } from "@/lib/utils";
11 | import { Button } from "@/components/ui/button";
12 | import type { CompletionRateData } from "@/types/habits";

```



```

13 |
14 | export interface HabitCompletionChartProps {
15 |   weeklyData: CompletionRateData[];
16 |   monthlyData: CompletionRateData[];
17 |   academicOnlyWeeklyData?: CompletionRateData[];
18 |   academicOnlyMonthlyData?: CompletionRateData[];
19 | }
20 |
21 | type Period = "weekly" | "monthly";
22 | type View = "all" | "academic";
23 |
24 | const HabitCompletionChart = ({
25 |   weeklyData,
26 |   monthlyData,
27 |   academicOnlyWeeklyData,
28 |   academicOnlyMonthlyData,
29 | }: HabitCompletionChartProps) => {
30 |   const [period, setPeriod] = useState<Period>("weekly");
31 |   const [view, setView] = useState<View>("all");
32 |   const gradientId = `barGradient-${useId()}`;
33 |
34 |   const getData = (): CompletionRateData[] => {
35 |     if (period === "weekly") {
36 |       return view === "academic" && academicOnlyWeeklyData
37 |         ? academicOnlyWeeklyData
38 |         : weeklyData;
39 |     }
40 |     return view === "academic" && academicOnlyMonthlyData
41 |       ? academicOnlyMonthlyData
42 |       : monthlyData;
43 |   };
44 |
45 |   const data = getData();
46 |
47 |   return (
48 |     <div className="space-y-3" data-testid="habit-completion-chart">
49 |       <div className="flex items-center justify-between gap-2 flex-wrap">
50 |         <div className="flex gap-1">
51 |           {[ "weekly", "monthly" ] as const}.map((p) => (
52 |             <Button
53 |               key={p}
54 |               variant="outline"
55 |               size="sm"
56 |               onClick={() => setPeriod(p)}
57 |               aria-pressed={period === p}
58 |               className={cn(
59 |                 "px-3 py-1 text-xs font-semibold rounded-xl",
60 |                 period === p
61 |                   ? "bg-blue-600 text-white border-blue-600 hover:bg-blue-700 hover:text-white"
62 |                   : "bg-white text-gray-600 border-gray-200"
63 |               )}
64 |               data-testid={`period-toggle-${p}`}
65 |             >
66 |               {p === "weekly" ? "Weekly" : "Monthly"}
67 |             </Button>
68 |           ))
69 |         </div>
70 |         <div className="flex gap-1">
71 |           {[ "all", "academic" ] as const}.map((v) => (
72 |             <Button
73 |               key={v}
74 |               variant="outline"
75 |               size="sm"
76 |               onClick={() => setView(v)}
77 |               aria-pressed={view === v}
78 |               className={cn(
79 |                 "px-3 py-1 text-xs font-semibold rounded-xl",
80 |                 view === v
81 |                   ? "bg-blue-600 text-white border-blue-600 hover:bg-blue-700 hover:text-white"
82 |                   : "bg-white text-gray-600 border-gray-200"
83 |               )}
84 |               data-testid={`view-toggle-${v}`}
85 |             >
86 |               {v === "all" ? "All Habits" : "Academic Only"}
87 |             </Button>
88 |           ))
89 |         </div>
90 |       </div>
91 |
92 |       <div>
93 |         <div>
94 |           <div>
95 |             <p className="text-sm text-gray-500 text-center py-8">
96 |               No data available
97 |             </p>
98 |           ) : (
99 |             <ResponsiveContainer width="100%" height={220}>
100 |               <BarChart
101 |                 data={data}
102 |                 margin={{ top: 4, right: 4, bottom: 4, left: -20 }}
103 |               >
104 |                 <XAxis
105 |                   dataKey="period"
106 |                   tick={{ fontSize: 11 }}
107 |                   tickLine={false}
108 |                   axisLine={false}
109 |                 />
110 |                 <YAxis
111 |                   domain={[0, 100]}
112 |                   tick={{ fontSize: 11 }}
113 |                   tickLine={false}
114 |                   axisLine={false}
115 |                   tickFormatter={(v: number) => `${v}%`}
116 |                 />

```

```

117 |         <Tooltip
118 |           formatter={
119 |             ((value: number | undefined) => [
120 |               `${value ?? 0}%`,
121 |               "Completion",
122 |             ]) as never
123 |           }
124 |           contentStyle={{ borderRadius: 8, fontSize: 12 }}
125 |         />
126 |         <Bar
127 |           dataKey="rate"
128 |           fill={`url(#${gradientId})`}
129 |           radius={[4, 4, 0, 0]}
130 |           maxBarSize={32}
131 |         />
132 |         <defs>
133 |           <linearGradient id={gradientId} x1="0" y1="0" x2="0" y2="1">
134 |             <stop offset="0%" stopColor="#14B8A6" />
135 |             <stop offset="100%" stopColor="#0382BD" />
136 |           </linearGradient>
137 |         </defs>
138 |       </BarChart>
139 |     </ResponsiveContainer>
140 |   )}
141 | </div>
142 | );
143 | };
144 |
145 | export default HabitCompletionChart;
146 |

```

FILE: [src/components/shared/HabitDifficultyIndicator.tsx](#)

TYPE: .tsx | LINES: 1-65 | SHA256: 4cb7cebd2e232eb5f711f24c39798100fcb71b1d2c8d13e382fc27e1b1594a2b

```

1 | // Task 145.6: Habit Difficulty Indicator component
2 |
3 | import { Sprout, TreePine, Leaf } from "lucide-react";
4 |
5 | interface HabitDifficultyIndicatorProps {
6 |   level: number;
7 |   habitLevelStreak: number;
8 | }
9 |
10 | const levelConfig = [
11 |   { icon: Leaf, label: "Seedling", color: "text-green-400", bg: "bg-green-50" },
12 |   { icon: Sprout, label: "Sprout", color: "text-green-500", bg: "bg-green-50" },
13 |   {
14 |     icon: TreePine,
15 |     label: "Tree",
16 |     color: "text-green-700",
17 |     bg: "bg-green-100",
18 |   },
19 | ];
20 |
21 | const HabitDifficultyIndicator = ({
22 |   level,
23 |   habitLevelStreak,
24 | }: HabitDifficultyIndicatorProps) => {
25 |   const idx = Math.min(Math.max(level - 1, 0), 2);
26 |   const config = levelConfig[idx!];
27 |   const Icon = config.icon;
28 |   const daysToNext = level < 3 ? Math.max(7 - habitLevelStreak, 0) : 0;
29 |   const progress =
30 |     level < 3 ? Math.min((habitLevelStreak / 7) * 100, 100) : 100;
31 |
32 |   return (
33 |     <div className="flex items-center gap-3">
34 |       <div className={`p-1.5 rounded-lg ${config.bg}`}>
35 |         <Icon className={`h-4 w-4 ${config.color}`} />
36 |       </div>
37 |       <div className="flex-1 min-w-0">
38 |         <div className="flex items-baseline gap-1.5">
39 |           <span className="text-xs font-bold text-gray-700">Level {level}</span>
40 |           <span className="text-[10px] text-gray-500">{config.label}</span>
41 |         </div>
42 |         {level < 3 ? (
43 |           <div className="mt-1">
44 |             <div className="h-1.5 rounded-full bg-gray-100 overflow-hidden">
45 |               <div
46 |                 className="h-full rounded-full bg-green-500 transition-all duration-300"
47 |                 style={{ width: `${progress}%` }}
48 |               />
49 |             </div>
50 |             <p className="text-[10px] text-gray-400 mt-0.5">
51 |               {daysToNext} day{daysToNext !== 1 ? "s" : ""} to Level {level + 1}
52 |             </p>
53 |           </div>
54 |         ) : (
55 |           <p className="text-[10px] text-green-600 font-medium">
56 |             Max level reached
57 |           </p>
58 |         )}
59 |       </div>
60 |     </div>
61 |   );
62 | };
63 |
64 | export default HabitDifficultyIndicator;
65 |

```

FILE: src/components/shared/HabitGrid.tsx

TYPE: .tsx | LINES: 1-54 | SHA256: 5653e2612f5323ed9ea2b4c0a57fff1035e284fbf8a58033c1ad26627cc39a04

```
1 // =====
2 // HabitGrid - 2x2 grid showing 4 daily habits with completion status
3 // Login, Submit, Journal, Read
4 // =====
5
6 import { LogIn, Upload, BookOpen, Eye } from "lucide-react";
7 import { cn } from "@/lib/utils";
8 import type { LucideIcon } from "lucide-react";
9
10 type HabitType = "login" | "submit" | "journal" | "read";
11
12 interface HabitGridProps {
13   completedHabits: HabitType[];
14   className?: string;
15 }
16
17 interface HabitConfig {
18   type: HabitType;
19   label: string;
20   icon: LucideIcon;
21 }
22
23 const HABITS: HabitConfig[] = [
24   { type: "login", label: "Login", icon: LogIn },
25   { type: "submit", label: "Submit", icon: Upload },
26   { type: "journal", label: "Journal", icon: BookOpen },
27   { type: "read", label: "Read", icon: Eye },
28 ];
29
30 const HabitGrid = ({ completedHabits, className }: HabitGridProps) => (
31   <div className={cn("grid grid-cols-2 gap-3", className)}>
32     {HABITS.map(({ type, label, icon: Icon }) => {
33       const done = completedHabits.includes(type);
34       return (
35         <div
36           key={type}
37           className={cn(
38             "flex items-center gap-2 rounded-lg px-3 py-2 text-sm font-medium transition-colors",
39             done ? "bg-green-50 text-green-700" : "bg-gray-50 text-gray-400"
40           )}
41         >
42           <Icon className="h-4 w-4 shrink-0" />
43           <span>{label}</span>
44           {done && <span className="ml-auto text-xs">'</span>}
45         </div>
46       );
47     })}
48   </div>
49 );
50
51 export default HabitGrid;
52 export { HABITS };
53 export type { HabitGridProps, HabitType };
54
```

FILE: src/components/shared/HabitMobileBottomSheet.tsx

TYPE: .tsx | LINES: 1-168 | SHA256: 55a521924b7b4bcc49b6edc877b9d586391630d5982f5009e26c96dcf13c5de

```
1 import {
2   Sheet,
3   SheetContent,
4   SheetHeader,
5   SheetTitle,
6   SheetDescription,
7 } from "@components/ui/sheet";
8 import type { CompletedHabit } from "@/types/habits";
9
10 // -----
11 // Types
12 // -----
13
14 export interface HabitMobileBottomSheetProps {
15   date: string;
16   habits: CompletedHabit[];
17   xpEarned: number;
18   streakActive: boolean;
19   open: boolean;
20   onClose: () => void;
21   /** Whether this day qualifies as a Perfect Day (all 4 academic habits). */
22   isPerfectDay?: boolean;
23   /** XP awarded for a Perfect Day, shown as reward context (R7.3). */
24   perfectDayXp?: number;
25 }
26
27 // -----
28 // Helpers
29 // -----
30
31 const HABIT_LABELS: Record<string, string> = {
32   login: "Login",
33   submit: "Submit",
34   journal: "Journal",
35   read: "Read",
36   meditation: "Meditation",
37   hydration: "Hydration",
38   exercise: "Exercise",
39   sleep: "Sleep",
40 };
41
42 function formatDate(dateStr: string): string {
```

```

43 | const d = new Date(dateStr + "T00:00:00");
44 | return d.toLocaleDateString("en-US", {
45 |   month: "long",
46 |   day: "numeric",
47 |   year: "numeric",
48 | });
49 | }
50 |
51 | // -----
52 | // Component
53 | // -----
54 |
55 | const HabitMobileBottomSheet = ({
56 |   date,
57 |   habits,
58 |   xpEarned,
59 |   streakActive,
60 |   open,
61 |   onClose,
62 |   isPerfectDay,
63 |   perfectDayXp,
64 | }: HabitMobileBottomSheetProps) => {
65 |   const academic = habits.filter((h) => h.category === "academic");
66 |   const wellness = habits.filter((h) => h.category === "wellness");
67 |   const isEmpty = habits.length === 0;
68 |
69 |   return (
70 |     <Sheet open={open} onOpenChange={({isOpen} => !isOpen && onClose())>
71 |       <SheetContent side="bottom" data-testid="habit-bottom-sheet">
72 |         <SheetHeader>
73 |           <SheetTitle data-testid="bottom-sheet-date">
74 |             {formatDate(date)}
75 |           </SheetTitle>
76 |           <SheetDescription className="sr-only">
77 |             Habit details for {formatDate(date)}
78 |           </SheetDescription>
79 |         </SheetHeader>
80 |
81 |         <div className="px-4 pb-4">
82 |           {isPerfectDay && (
83 |             <p
84 |               className="text-xs font-medium text-amber-600 mb-2"
85 |               data-testid="bottom-sheet-perfect-day-label"
86 |             >
87 |               +P Perfect Day
88 |               {perfectDayXp != null && perfectDayXp > 0
89 |                 ? ` . +${perfectDayXp} XP bonus`
90 |                 : ""}
91 |             </p>
92 |           )}
93 |           {isEmpty ? (
94 |             <p
95 |               className="text-sm text-gray-500"
96 |               data-testid="bottom-sheet-empty"
97 |             >
98 |               No habits completed
99 |             </p>
100 |          ) : (
101 |            <
102 |              {academic.length > 0 && (
103 |                <div data-testid="bottom-sheet-academic-section">
104 |                  <p className="text-[10px] font-black tracking-widest uppercase text-gray-400">
105 |                    Academic
106 |                  </p>
107 |                  <ul className="mt-1 space-y-0.5">
108 |                    {academic.map((h) => (
109 |                      <li
110 |                        key={h.type}
111 |                        className="flex items-center gap-1.5 text-sm text-gray-700"
112 |                        data-testid={`bottom-sheet-habit-${h.type}`}
113 |                      >
114 |                        <span className="text-green-500">' </span>
115 |                        {HABIT_LABELS[h.type] ?? h.type}
116 |                      </li>
117 |                    ))}
118 |                  </ul>
119 |                </div>
120 |              )}
121 |
122 |              {wellness.length > 0 && (
123 |                <div
124 |                  className="mt-3"
125 |                  data-testid="bottom-sheet-wellness-section"
126 |                >
127 |                  <p className="text-[10px] font-black tracking-widest uppercase text-gray-400">
128 |                    Wellness
129 |                  </p>
130 |                  <ul className="mt-1 space-y-0.5">
131 |                    {wellness.map((h) => (
132 |                      <li
133 |                        key={h.type}
134 |                        className="flex items-center gap-1.5 text-sm text-gray-700"
135 |                        data-testid={`bottom-sheet-habit-${h.type}`}
136 |                      >
137 |                        <span className="text-green-500">' </span>
138 |                        {HABIT_LABELS[h.type] ?? h.type}
139 |                      </li>
140 |                    ))}
141 |                  </ul>
142 |                </div>
143 |              )}
144 |
145 |            <div className="mt-4 flex items-center justify-between border-t border-slate-100 pt-3">
146 |              <span

```

```

147 |         className="text-xs text-gray-500"
148 |         data-testid="bottom-sheet-xp"
149 |       >
150 |         {xpEarned} XP earned
151 |       </span>
152 |     <span
153 |       className="text-xs font-medium"
154 |       data-testid="bottom-sheet-streak"
155 |     >
156 |       {streakActive ? "0=Ÿ% Streak active" : "Streak broken"}
157 |     </span>
158 |   </div>
159 | </>
160 | })
161 | </div>
162 | </SheetContent>
163 | </Sheet>
164 | );
165 | };
166 |
167 | export default HabitMobileBottomSheet;
168 |

```

FILE: [src/components/shared/HabitTracker.tsx](#)

TYPE: .tsx | LINES: 1-192 | SHA256: 7a50f54bae6029cb7bbfb227992cb554dc7ea357dbe54c6e1b48b531c46a4be6

```

1 | // Task 127.5 + 141.3: Daily Habit Tracker – 8 habits in 7-day grid
2 |
3 | import { Card } from "@components/ui/card";
4 | import { useQuery } from "@tanstack/react-query";
5 | import { supabase } from "@lib/supabase";
6 | import { queryKeys } from "@lib/queryKeys";
7 | import { Shimmer } from "@design-system";
8 | import {
9 |   Login,
10 |   Send,
11 |   BookOpen,
12 |   Eye,
13 |   MessageSquare,
14 |   Brain,
15 |   HandHelping,
16 |   CheckCircle2,
17 |   type LucideIcon,
18 | } from "lucide-react";
19 | import { format, subDays } from "date-fns";
20 |
21 | type HabitType =
22 |   | "login"
23 |   | "submit"
24 |   | "journal"
25 |   | "read"
26 |   | "collaborate"
27 |   | "practice"
28 |   | "review"
29 |   | "mentor";
30 |
31 | interface HabitConfig {
32 |   type: HabitType;
33 |   label: string;
34 |   icon: LucideIcon;
35 |   color: string;
36 | }
37 |
38 | const HABITS: HabitConfig[] = [
39 |   { type: "login", label: "Login", icon: Login, color: "text-blue-500" },
40 |   { type: "submit", label: "Submit", icon: Send, color: "text-green-500" },
41 |   {
42 |     type: "journal",
43 |     label: "Journal",
44 |     icon: BookOpen,
45 |     color: "text-purple-500",
46 |   },
47 |   { type: "read", label: "Read", icon: Eye, color: "text-amber-500" },
48 |   {
49 |     type: "collaborate",
50 |     label: "Collaborate",
51 |     icon: MessageSquare,
52 |     color: "text-teal-500",
53 |   },
54 |   { type: "practice", label: "Practice", icon: Brain, color: "text-red-500" },
55 |   {
56 |     type: "review",
57 |     label: "Review",
58 |     icon: CheckCircle2,
59 |     color: "text-indigo-500",
60 |   },
61 |   {
62 |     type: "mentor",
63 |     label: "Mentor",
64 |     icon: HandHelping,
65 |     color: "text-orange-500",
66 |   },
67 | ];
68 |
69 | interface HabitTrackerProps {
70 |   studentId: string;
71 |   days?: number;
72 | }
73 |
74 | const useHabitLogs = (studentId: string, days: number) => {
75 |   const today = new Date();
76 |   const startDate = format(subDays(today, days - 1), "yyyy-MM-dd");
77 |   const endDate = format(today, "yyyy-MM-dd");
78 |

```

```

79 | return useQuery({
80 |   queryKey: queryKeys.habitLogs.list({ studentId, startDate, endDate }),
81 |   queryFn: async () => {
82 |     // Read from the canonical long-format `habit_logs` table (one row per
83 |     // completed habit per UTC day, keyed by `habit_type`). This is the same
84 |     // source of truth that every write path (login, submit, journal, read,
85 |     // session-completion, perfectDay) targets and that `useHeatmapData`
86 |     // reads. The legacy wide-format `habit_tracking` table is no longer
87 |     // written by any flow, so reading it here showed stale data.
88 |     const { data, error } = await supabase
89 |       .from("habit_logs")
90 |       .select("date, habit_type, completed_at, created_at")
91 |       .eq("student_id", studentId)
92 |       .gte("date", startDate)
93 |       .lte("date", endDate)
94 |       .order("date", { ascending: true });
95 |     if (error) throw error;
96 |     return (data ?? []).map((row) => ({
97 |       habit_type: row.habit_type,
98 |       date: row.date,
99 |       completed_at: row.completed_at ?? row.created_at ?? null,
100 |     }));
101 |   },
102 |   enabled: !!studentId,
103 | });
104 | };
105 |
106 | const HabitTracker = ({ studentId, days = 7 }: HabitTrackerProps) => {
107 |   const { data: logs, isLoading } = useHabitLogs(studentId, days);
108 |
109 |   const today = new Date();
110 |   const dateColumns = Array.from({ length: days }, (_, i) => {
111 |     const d = subDays(today, days - 1 - i);
112 |     return { date: format(d, "yyyy-MM-dd"), label: format(d, "EEE") };
113 |   });
114 |
115 |   const completedSet = new Set(
116 |     (logs ?? []).
117 |       .filter((l) => l.completed_at !== null)
118 |       .map((l) => `${l.habit_type}:${l.date}`)
119 |   );
120 |
121 |   if (isLoading) return <Shimmer className="h-48 rounded-xl" />;
122 |
123 |   return (
124 |     <Card className="bg-white border-0 shadow-md rounded-xl overflow-hidden gap-0 py-0">
125 |       <div
126 |         className="px-6 py-4 flex items-center gap-2"
127 |         style={{
128 |           background: "var(--brand-gradient)",
129 |         }}
130 |       >
131 |         <CheckCircle2 className="h-5 w-5 text-white" />
132 |         <h2 className="text-lg font-bold tracking-tight text-white">
133 |           Daily Habits
134 |         </h2>
135 |       </div>
136 |       <div className="p-6 overflow-x-auto">
137 |         <table className="w-full text-sm">
138 |           <thead>
139 |             <tr>
140 |               <th className="text-start py-1 px-2 text-xs font-semibold text-gray-500 w-28">
141 |                 Habit
142 |               </th>
143 |               {dateColumns.map((col) => (
144 |                 <th
145 |                   key={col.date}
146 |                   className="py-1 px-1 text-xs font-semibold text-gray-500 text-center w-10"
147 |                 >
148 |                   {col.label}
149 |                 </th>
150 |               ))}
151 |             </tr>
152 |           </thead>
153 |           <tbody>
154 |             {HABITS.map((habit) => {
155 |               const Icon = habit.icon;
156 |               return (
157 |                 <tr key={habit.type} className="border-t border-slate-100">
158 |                   <td className="py-1.5 px-2">
159 |                     <div className="flex items-center gap-1.5">
160 |                       <Icon className={`h-3.5 w-3.5 ${habit.color}`} />
161 |                       <span className="text-xs font-medium text-gray-700">
162 |                         {habit.label}
163 |                       </span>
164 |                     </div>
165 |                   </td>
166 |                   {dateColumns.map((col) => {
167 |                     const done = completedSet.has(`${habit.type}:${col.date}`);
168 |                     return (
169 |                       <td key={col.date} className="py-1.5 px-1 text-center">
170 |                         <div
171 |                           className={`w-6 h-6 mx-auto rounded-md ${
172 |                             done ? "bg-green-500" : "bg-gray-200"
173 |                           }`}
174 |                           title={`${habit.label} - ${col.date}: ${
175 |                             done ? "Done" : "Missed"
176 |                           }`}
177 |                         </div>
178 |                       </td>
179 |                     );
180 |                   })}
181 |                 </tr>
182 |               );

```

```

183 |         }}}
184 |       </tbody>
185 |     </table>
186 |   </div>
187 | </Card>
188 | );
189 | };
190 |
191 | export default HabitTracker;
192 |

```

FILE: src/components/shared/HeatmapFilters.tsx

TYPE: .tsx | LINES: 1-95 | SHA256: 6d064916a1b9212305d36e3f1e9e50c67cc374edc9a55c135a115e0a4b47318e

```

1 | import { parseAsString, useQueryState } from "nuqs";
2 | import { Tabs, TabsList, TabsTrigger } from "@components/ui/tabs";
3 | import { cn } from "@lib/utils";
4 | import { getFilterOptions } from "@lib/heatmapUtils";
5 | import type { WellnessHabitType } from "@types/habits";
6 |
7 | // -----
8 | // Types
9 | // -----
10 |
11 | export interface HeatmapFiltersProps {
12 |   enabledWellnessHabits: WellnessHabitType[];
13 |   onFilterChange?: (filter: string) => void;
14 | }
15 |
16 | // -----
17 | // Helpers
18 | // -----
19 |
20 | /** Map display label back to a URL-safe filter value. */
21 | const FILTER_VALUE_MAP: Record<string, string> = {
22 |   "All Habits": "all",
23 |   Login: "login",
24 |   Submit: "submit",
25 |   Journal: "journal",
26 |   Read: "read",
27 |   Meditation: "meditation",
28 |   Hydration: "hydration",
29 |   Exercise: "exercise",
30 |   Sleep: "sleep",
31 | };
32 |
33 | /** Reverse lookup: filter value !' display label. */
34 | const FILTER_LABEL_MAP: Record<string, string> = Object.fromEntries(
35 |   Object.entries(FILTER_VALUE_MAP).map(([k, v]) => [v, k])
36 | );
37 |
38 | // -----
39 | // Component
40 | // -----
41 |
42 | const HeatmapFilters = ({
43 |   enabledWellnessHabits,
44 |   onFilterChange,
45 | }: HeatmapFiltersProps) => {
46 |   const [filter, setFilter] = useQueryState(
47 |     "habit",
48 |     parseAsString.withDefault("all")
49 |   );
50 |
51 |   const options = getFilterOptions(enabledWellnessHabits);
52 |
53 |   const handleChange = (value: string) => {
54 |     setFilter(value);
55 |     onFilterChange?.(value);
56 |   };
57 |
58 |   return (
59 |     <Tabs
60 |       value={filter}
61 |       onValueChange={handleChange}
62 |       data-testid="heatmap-filters"
63 |     >
64 |       <TabsList
65 |         className="flex flex-wrap gap-2 bg-transparent h-auto p-0"
66 |         data-testid="heatmap-filter-list"
67 |       >
68 |         {options.map((label) => {
69 |           const value = FILTER_VALUE_MAP[label] ?? label.toLowerCase();
70 |           const isActive = filter === value;
71 |
72 |           return (
73 |             <TabsTrigger
74 |               key={value}
75 |               value={value}
76 |               className={cn(
77 |                 "rounded-xl border px-3 py-1.5 text-sm font-medium transition-colors",
78 |                 isActive
79 |                   ? "bg-blue-600 text-white border-blue-600"
80 |                   : "bg-white text-gray-600 border-gray-200 hover:bg-slate-50"
81 |               )}
82 |               data-testid={`filter-${value}`}
83 |             >
84 |               {label}
85 |             </TabsTrigger>
86 |           );
87 |         })}
88 |       </TabsList>
89 |     </Tabs>
90 |   );

```

```

91 | };
92 |
93 | export { FILTER_VALUE_MAP, FILTER_LABEL_MAP };
94 | export default HeatmapFilters;
95 |

```

FILE: src/components/shared/HeatmapGrid.tsx

TYPE: .tsx | LINES: 1-634 | SHA256: 8fd389f25db11e050f0f0d7fed5932451a1f8e256bf2c267d5fa839d8a847743

```

1 | import {
2 |   memo,
3 |   useCallback,
4 |   useMemo,
5 |   useRef,
6 |   useState,
7 |   type KeyboardEvent as ReactKeyboardEvent,
8 | } from "react";
9 | import type {
10 |   HeatmapDay,
11 |   DateRange,
12 |   StudentHabitLevel,
13 |   ComebackChallengeStatus,
14 |   StreakMilestone,
15 | } from "@types/habits";
16 | import {
17 |   getIntensityLevel,
18 |   isDateFuture,
19 |   generateAriaLabel,
20 | } from "@lib/heatmapUtils";
21 | import {
22 |   getLevelAwareIntensityLevel,
23 |   getLevelForDate,
24 | } from "@lib/levelAwareHeatmap";
25 | import { cn } from "@lib/utils";
26 |
27 | // -----
28 | // Constants
29 | // -----
30 |
31 | /** Gap between cells, in px. Kept small and integer so cells stay pixel-crisp. */
32 | const CELL_GAP = 3;
33 | /** Fixed cell size (px). DOM boxes at integer sizes never blur (unlike a
34 |  * CSS-scaled SVG). Cells stay square on every breakpoint; the track scrolls
35 |  * horizontally when the range is wider than the container. */
36 | const CELL_SIZE = 15;
37 | /** Width reserved for the sticky weekday label column. */
38 | const WEEKDAY_COL_W = 30;
39 | /** Height reserved for the month label row above the grid. */
40 | const MONTH_ROW_H = 18;
41 | /** Days-of-week rendered in the label column (Mon=row0, Wed=row2, Fri=row4). */
42 | const WEEKDAY_ROWS = 7;
43 |
44 | const MONTH_NAMES = [
45 |   "Jan",
46 |   "Feb",
47 |   "Mar",
48 |   "Apr",
49 |   "May",
50 |   "Jun",
51 |   "Jul",
52 |   "Aug",
53 |   "Sep",
54 |   "Oct",
55 |   "Nov",
56 |   "Dec",
57 | ];
58 |
59 | /** Weekday label per grid row. Weeks are Monday-aligned (ISO). */
60 | const DAY_LABELS: Array<{ label: string; row: number }> = [
61 |   { label: "Mon", row: 0 },
62 |   { label: "Wed", row: 2 },
63 |   { label: "Fri", row: 4 },
64 | ];
65 |
66 | /** Intensity !' CSS custom property (defined in index.css, dark-mode aware). */
67 | const INTENSITY_COLORS: Record<number, string> = {
68 |   0: "var(--heatmap-empty)",
69 |   1: "var(--heatmap-level-1)",
70 |   2: "var(--heatmap-level-2)",
71 |   3: "var(--heatmap-level-3)",
72 |   4: "var(--heatmap-level-4)",
73 | };
74 |
75 | const LEGEND_LABELS_DEFAULT = ["No activity", "", "", "", "4+ habits"];
76 |
77 | /**
78 |  * Level-relative legend labels. For a level-capped student the top swatch reads
79 |  * "N/N habits"; otherwise it falls back to the absolute "4+ habits" scale.
80 |  */
81 | function getLegendLabels(studentLevel?: StudentHabitLevel): string[] {
82 |   if (!studentLevel) return LEGEND_LABELS_DEFAULT;
83 |   const max = studentLevel.currentLevel;
84 |   return ["No activity", "", "", "", `${max}/${max} habits`];
85 | }
86 |
87 | // -----
88 | // Props
89 | // -----
90 |
91 | export interface HeatmapGridProps {
92 |   data: HeatmapDay[];
93 |   semesterRange: DateRange;
94 |   studentLevel?: StudentHabitLevel;
95 |   comebackChallenge?: ComebackChallengeStatus;

```



```

96 | sabbaticalEnabled?: boolean;
97 | milestones?: StreakMilestone[];
98 | onCellClick?: (date: string) => void;
99 | onCellHover?: (date: string | null) => void;
100 | }
101 |
102 | // -----
103 | // Grid model – Monday-aligned weeks with leading/trailing padding
104 | // -----
105 |
106 | interface DaySlot {
107 |   kind: "day";
108 |   date: string;
109 |   count: number;
110 |   isFuture: boolean;
111 |   /** Index into the sequence of real days (drives roving tabindex + arrows). */
112 |   logicalIndex: number;
113 | }
114 | interface PadSlot {
115 |   kind: "pad";
116 | }
117 | type Slot = DaySlot | PadSlot;
118 |
119 | interface GridModel {
120 |   slots: Slot[];
121 |   numCols: number;
122 |   dayCount: number;
123 | }
124 |
125 | function formatISO(d: Date): string {
126 |   return (
127 |     d.getFullYear() +
128 |     "-" +
129 |     String(d.getMonth() + 1).padStart(2, "0") +
130 |     "-" +
131 |     String(d.getDate()).padStart(2, "0")
132 |   );
133 | }
134 |
135 | /** Monday-based weekday index (Mon=0 ... Sun=6). */
136 | function mondayIndex(d: Date): number {
137 |   return (d.getDay() + 6) % 7;
138 | }
139 |
140 | /**
141 |  * Builds the flat slot list in column-major order so a CSS grid with
142 |  * `grid-auto-flow: column` + 7 rows places each day on its true weekday row.
143 |  * Leading padding aligns the first date to its weekday; trailing padding
144 |  * completes the final week column. Padding slots are decorative (no role).
145 |  */
146 | function buildGridModel(data: HeatmapDay[], range: DateRange): GridModel {
147 |   if (!range.start || !range.end) {
148 |     return { slots: [], numCols: 0, dayCount: 0 };
149 |   }
150 |
151 |   const dataMap = new Map<string, number>();
152 |   for (const d of data) dataMap.set(d.date, d.totalCount);
153 |
154 |   const start = new Date(range.start + "T00:00:00");
155 |   const end = new Date(range.end + "T00:00:00");
156 |
157 |   const slots: Slot[] = [];
158 |   const leadingPad = mondayIndex(start);
159 |   for (let i = 0; i < leadingPad; i++) slots.push({ kind: "pad" });
160 |
161 |   const cursor = new Date(start);
162 |   let logicalIndex = 0;
163 |   while (cursor <= end) {
164 |     const date = formatISO(cursor);
165 |     slots.push({
166 |       kind: "day",
167 |       date,
168 |       count: dataMap.get(date) ?? 0,
169 |       isFuture: isDateFuture(date),
170 |       logicalIndex: logicalIndex++,
171 |     });
172 |     cursor.setDate(cursor.getDate() + 1);
173 |   }
174 |
175 |   while (slots.length % WEEKDAY_ROWS !== 0) slots.push({ kind: "pad" });
176 |
177 |   return {
178 |     slots,
179 |     numCols: slots.length / WEEKDAY_ROWS,
180 |     dayCount: logicalIndex,
181 |   };
182 | }
183 |
184 | /** First column at which each month begins – for month labels above the grid. */
185 | function computeMonthLabels(
186 |   slots: Slot[]
187 | ): Array<{ month: string; col: number }> {
188 |   const labels: Array<{ month: string; col: number }> = [];
189 |   let lastMonth = -1;
190 |   slots.forEach((slot, pos) => {
191 |     if (slot.kind !== "day") return;
192 |     const month = new Date(slot.date + "T00:00:00").getMonth();
193 |     if (month !== lastMonth) {
194 |       lastMonth = month;
195 |       labels.push({
196 |         month: MONTH_NAMES[month] ?? "",
197 |         col: Math.floor(pos / WEEKDAY_ROWS),
198 |       });
199 |     }

```

```

200 | });
201 | return labels;
202 | }
203 |
204 | // -----
205 | // Overlay predicates (Comeback Challenge / Sabbatical / Milestone)
206 | // -----
207 |
208 | function isComebackChallengeDate(
209 |   date: string,
210 |   challenge: ComebackChallengeStatus | undefined
211 | ): boolean {
212 |   if (!challenge?.active || !challenge.startDate) return false;
213 |   const start = new Date(challenge.startDate + "T00:00:00");
214 |   const end = new Date(start);
215 |   end.setDate(end.getDate() + 2);
216 |   const d = new Date(date + "T00:00:00");
217 |   return d >= start && d <= end;
218 | }
219 |
220 | function isSabbaticalRestDay(date: string, enabled: boolean): boolean {
221 |   if (!enabled) return false;
222 |   const dow = new Date(date + "T00:00:00").getDay();
223 |   return dow === 0 || dow === 6;
224 | }
225 |
226 | function getMilestoneForDate(
227 |   date: string,
228 |   milestones: StreakMilestone[] | undefined
229 | ): StreakMilestone | undefined {
230 |   return milestones?.find((m) => m.achievedDate === date);
231 | }
232 |
233 | function comebackDayNumberFor(
234 |   date: string,
235 |   challenge: ComebackChallengeStatus
236 | ): number {
237 |   if (!challenge.startDate) return 0;
238 |   const start = new Date(challenge.startDate + "T00:00:00");
239 |   const d = new Date(date + "T00:00:00");
240 |   return (
241 |     Math.floor((d.getTime() - start.getTime()) / (1000 * 60 * 60 * 24)) + 1
242 |   );
243 | }
244 |
245 | function findDayByLogicalIndex(
246 |   slots: Slot[],
247 |   logicalIndex: number
248 | ): DaySlot | undefined {
249 |   for (const slot of slots) {
250 |     if (slot.kind === "day" && slot.logicalIndex === logicalIndex) return slot;
251 |   }
252 |   return undefined;
253 | }
254 |
255 | // -----
256 | // Cell – memoized so hovering/focusing one cell never re-renders the others
257 | // -----
258 |
259 | interface HeatmapCellProps {
260 |   slot: DaySlot;
261 |   intensity: number;
262 |   size: number;
263 |   isRovingTarget: boolean;
264 |   isComeback: boolean;
265 |   isSabbatical: boolean;
266 |   milestone: StreakMilestone | undefined;
267 |   comebackDayNumber: number;
268 |   onKeyDown: (e: ReactKeyboardEvent, index: number) => void;
269 |   onFocusCell: (index: number, date: string, isFuture: boolean) => void;
270 |   onBlurCell: () => void;
271 |   onHover: (date: string | null) => void;
272 |   onClick: (date: string) => void;
273 |   registerRef: (index: number, el: HTMLDivElement | null) => void;
274 | }
275 |
276 | const HeatmapCell = memo(function HeatmapCell({
277 |   slot,
278 |   intensity,
279 |   size,
280 |   isRovingTarget,
281 |   isComeback,
282 |   isSabbatical,
283 |   milestone,
284 |   comebackDayNumber,
285 |   onKeyDown,
286 |   onFocusCell,
287 |   onBlurCell,
288 |   onHover,
289 |   onClick,
290 |   registerRef,
291 | }: HeatmapCellProps) {
292 |   const { date, count, isFuture, logicalIndex } = slot;
293 |
294 |   return (
295 |     <div
296 |       ref={(el) => registerRef(logicalIndex, el)}
297 |       role="gridcell"
298 |       aria-label={generateAriaLabel(date, count)}
299 |       aria-disabled={isFuture}
300 |       tabIndex={isRovingTarget ? 0 : -1}
301 |       data-date={date}
302 |       data-level={intensity}
303 |       data-future={isFuture ? "true" : "false"}

```

```

304 |     data-testid={`heatmap-cell-${date}`}
305 |     className={cn(
306 |       "relative rounded-[3px] outline-none",
307 |       "shadow-[inset_0_0_1px_var(--heatmap-cell-outline)]",
308 |       "motion-safe:transition-[box-shadow] motion-safe:duration-150",
309 |       isFuture
310 |       ? "opacity-40 cursor-default"
311 |       : "cursor-pointer hover:z-10 hover:shadow-[inset_0_0_2px_var(--heatmap-cell-ring)]",
312 |       "focus-visible:z-10 focus-visible:shadow-[inset_0_0_2px_var(--brand-primary)]"
313 |     )}
314 |     style={{
315 |       width: size,
316 |       height: size,
317 |       backgroundColor: INTENSITY_COLORS[intensity],
318 |     }}
319 |     onKeyDown={(e) => onKeyDown(e, logicalIndex)}
320 |     onFocus={() => onFocusCell(logicalIndex, date, isFuture)}
321 |     onBlur={onBlurCell}
322 |     onMouseEnter={() => !isFuture && onHover(date)}
323 |     onMouseLeave={() => onHover(null)}
324 |     onClick={() => !isFuture && onClick(date)}
325 |   >
326 |     {/* Comeback Challenge – dashed teal ring */}
327 |     {isComeback && (
328 |       <span
329 |         role="img"
330 |         aria-label={`Comeback Day ${comebackDayNumber}/3`}
331 |         data-testid={`comeback-overlay-${date}`}
332 |         data-comeback-day={comebackDayNumber}
333 |         className="pointer-events-none absolute inset-0 rounded-[3px] border-[1.5px] border-dashed border-te
334 |       />
335 |     )}
336 |     {/* Sabbatical rest day – diagonal stripes */}
337 |     {isSabbatical && !isFuture && (
338 |       <span
339 |         role="img"
340 |         aria-label="Rest Day (Sabbatical)"
341 |         data-testid={`sabbatical-overlay-${date}`}
342 |         className="pointer-events-none absolute inset-0 rounded-[3px] opacity-50"
343 |         style={{
344 |           backgroundImage:
345 |             "repeating-linear-gradient(45deg, #94a3b8 0 1px, transparent 1px 4px)",
346 |         }}
347 |       />
348 |     )}
349 |     {/* Milestone – star marker, top-right */}
350 |     {milestone && (
351 |       <span
352 |         role="img"
353 |         aria-label={`${milestone.days}-Day Streak Milestone`}
354 |         data-testid={`milestone-marker-${date}`}
355 |         className="pointer-events-none absolute -right-1 -top-1 text-[9px] leading-none text-amber-500 drop-
356 |       >
357 |         &
358 |       </span>
359 |     )}
360 |   </div>
361 | );
362 | });
363 |
364 | // -----
365 | // Component
366 | // -----
367 |
368 | const HeatmapGrid = ({
369 |   data,
370 |   semesterRange,
371 |   studentLevel,
372 |   comebackChallenge,
373 |   sabbaticalEnabled = false,
374 |   milestones,
375 |   onCellClick,
376 |   onCellHover,
377 | }: HeatmapGridProps) => {
378 |   const [focusedIndex, setFocusedIndex] = useState<number | null>(null);
379 |   const cellRefs = useRef<Map<number, HTMLDivElement>>(new Map());
380 |
381 |   const { slots, numCols, dayCount } = useMemo(
382 |     () => buildGridModel(data, semesterRange),
383 |     [data, semesterRange]
384 |   );
385 |
386 |   const monthLabels = useMemo(() => computeMonthLabels(slots), [slots]);
387 |
388 |   const trackWidth = numCols * (CELL_SIZE + CELL_GAP) - CELL_GAP;
389 |
390 |   // --- Interaction callbacks (stable) -----
391 |
392 |   const registerRef = useCallback(
393 |     (index: number, el: HTMLDivElement | null) => {
394 |       if (el) cellRefs.current.set(index, el);
395 |       else cellRefs.current.delete(index);
396 |     },
397 |     []
398 |   );
399 |
400 |   const handleKeyDown = useCallback(
401 |     (e: ReactKeyboardEvent, index: number) => {
402 |       let next: number | null = null;
403 |       switch (e.key) {
404 |         case "ArrowRight":
405 |           next = index + WEEKDAY_ROWS;
406 |           break; // next week, same weekday
407 |         case "ArrowLeft":

```

```

408         next = index - WEEKDAY_ROWS;
409         break;
410     case "ArrowDown":
411         next = index + 1;
412         break; // next day
413     case "ArrowUp":
414         next = index - 1;
415         break;
416     case "Home":
417         next = 0;
418         break;
419     case "End":
420         next = dayCount - 1;
421         break;
422     case "Enter":
423     case " ": {
424         e.preventDefault();
425         const day = findDayByLogicalIndex(slots, index);
426         if (day && !day.isFuture) onCellClick?.(day.date);
427         return;
428     }
429     default:
430         return;
431 }
432 e.preventDefault();
433 if (next !== null && next >= 0 && next < dayCount) {
434     setFocusedIndex(next);
435     cellRefs.current.get(next)?.focus();
436 }
437 },
438 [slots, dayCount, onCellClick]
439 );
440
441 const handleFocusCell = useCallback(
442     (index: number, date: string, isFuture: boolean) => {
443         setFocusedIndex(index);
444         if (!isFuture) onCellHover?.(date);
445     },
446     [onCellHover]
447 );
448
449 const handleBlurCell = useCallback(() => onCellHover?.(null), [onCellHover]);
450 const handleHover = useCallback(
451     (date: string | null) => onCellHover?.(date),
452     [onCellHover]
453 );
454 const handleClick = useCallback(
455     (date: string) => onCellClick?.(date),
456     [onCellClick]
457 );
458
459 const legendLabels = getLegendLabels(studentLevel);
460
461 return (
462     <div className="w-full" data-testid="heatmap-root">
463         <div className="flex gap-1">
464             {/* Sticky weekday label column */}
465             <div
466                 className="shrink-0 select-none"
467                 style={{ width: WEEKDAY_COL_W, paddingTop: MONTH_ROW_H }}
468                 aria-hidden="true"
469             >
470                 <div
471                     className="grid"
472                     style={{
473                         gridTemplateRows: `repeat(${WEEKDAY_ROWS}, ${CELL_SIZE}px)`,
474                         rowGap: CELL_GAP,
475                     }}
476                 >
477                     {Array.from({ length: WEEKDAY_ROWS }).map((_, row) => {
478                         const label = DAY_LABELS.find((d) => d.row === row)?.label;
479                         return (
480                             <div
481                                 key={row}
482                                 className="flex items-center text-[10px] leading-none text-muted-foreground"
483                                 data-testid={label ? `day-label-${label}` : undefined}
484                             >
485                                 {label ?? ""}
486                             </div>
487                         );
488                     })}
489                 </div>
490             </div>
491             {/* Scrollable track: month labels + grid */}
492             <div
493                 className="min-w-0 flex-1 overflow-x-auto overscroll-x-contain pb-1 [scrollbar-width:thin]"
494                 role="group"
495                 aria-label="Habit activity heatmap. Scroll horizontally to see earlier and later weeks."
496             >
497                 <div style={{ width: trackWidth, minWidth: "100%" }}>
498                     {/* Month labels */}
499                     <div className="relative" style={{ height: MONTH_ROW_H }}>
500                         {monthLabels.map((ml) => (
501                             <span
502                                 key={`month-${ml.month}-${ml.col}`}
503                                 className="absolute top-0 text-[10px] leading-none text-muted-foreground"
504                                 style={{ left: ml.col * (CELL_SIZE + CELL_GAP) }}
505                                 data-testid={`month-label-${ml.month}`}
506                             >
507                                 {ml.month}
508                             </span>
509                         ))}
510                     </div>
511                 </div>

```

```

512 |     /* Grid */
513 |     <div
514 |       role="grid"
515 |       aria-label="Habit heatmap grid"
516 |       aria-rowcount={WEEKDAY_ROWS}
517 |       aria-colcount={numCols}
518 |       className="grid w-max"
519 |       style={{
520 |         gridTemplateRows: `repeat(${WEEKDAY_ROWS}, ${CELL_SIZE}px)`,
521 |         gridAutoColumns: `${CELL_SIZE}px`,
522 |         gridAutoFlow: "column",
523 |         gap: CELL_GAP,
524 |       }}
525 |     >
526 |       {slots.map((slot, pos) => {
527 |         if (slot.kind === "pad") {
528 |           // Out-of-range filler for the partial first/last weeks (weeks
529 |           // are Monday-aligned so the weekday labels stay truthful).
530 |           // Styled like an empty day so the grid reads as a complete
531 |           // rectangle instead of showing transparent corner gaps, but
532 |           // kept aria-hidden + non-interactive so it never affects the
533 |           // gridcell count, tooltip, or keyboard navigation.
534 |           return (
535 |             <div
536 |               key={`pad-${pos}`}
537 |               aria-hidden="true"
538 |               className="rounded-[3px] shadow-[inset_0_0_1px_var(--heatmap-cell-outline)]"
539 |               style={{
540 |                 width: CELL_SIZE,
541 |                 height: CELL_SIZE,
542 |                 backgroundColor: "var(--heatmap-empty)",
543 |               }}
544 |             />
545 |           );
546 |         }
547 |
548 |         let intensity: number;
549 |         if (slot.isFuture) {
550 |           intensity = 0;
551 |         } else if (studentLevel) {
552 |           const levelOnDate = getLevelForDate(
553 |             slot.date,
554 |             studentLevel.levelHistory
555 |           );
556 |           intensity = getLevelAwareIntensityLevel(
557 |             slot.count,
558 |             levelOnDate
559 |           );
560 |         } else {
561 |           intensity = getIntensityLevel(slot.count);
562 |         }
563 |
564 |         const isComeback = isComebackChallengeDate(
565 |           slot.date,
566 |           comebackChallenge
567 |         );
568 |         const isSabbatical = isSabbaticalRestDay(
569 |           slot.date,
570 |           sabbaticalEnabled
571 |         );
572 |         const milestone = getMilestoneForDate(slot.date, milestones);
573 |         const comebackDayNumber = isComeback
574 |           ? comebackDayNumberFor(slot.date, comebackChallenge!)
575 |           : 0;
576 |
577 |         return (
578 |           <HeatmapCell
579 |             key={slot.date}
580 |             slot={slot}
581 |             intensity={intensity}
582 |             size={CELL_SIZE}
583 |             isRovingTarget={
584 |               focusedIndex === slot.logicalIndex ||
585 |               (focusedIndex === null && slot.logicalIndex === 0)
586 |             }
587 |             isComeback={isComeback}
588 |             isSabbatical={isSabbatical}
589 |             milestone={milestone}
590 |             comebackDayNumber={comebackDayNumber}
591 |             onKeyDown={handleKeyDown}
592 |             onFocusCell={handleFocusCell}
593 |             onBlurCell={handleBlurCell}
594 |             onHover={handleHover}
595 |             onClick={handleClick}
596 |             registerRef={registerRef}
597 |           />
598 |         );
599 |       })}
600 |     </div>
601 |   </div>
602 | </div>
603 |
604 |
605 |
606 |   /* Legend – real HTML flow, cannot overlap (fixes the SVG overlap bug) */
607 |   <div
608 |     className="mt-3 flex flex-wrap items-center gap-x-2 gap-y-1 ps-[34px] text-[11px] text-muted-foreground"
609 |     data-testid="heatmap-legend"
610 |   >
611 |     <span data-testid="legend-label-0">{legendLabels[0]}</span>
612 |     <div
613 |       className="flex items-center gap-1"
614 |       role="img"
615 |       aria-label="Activity scale from none to high"

```

```

616 |         >
617 |         {[0, 1, 2, 3, 4].map((level) => (
618 |             <span
619 |                 key={`legend-${level}`}
620 |                 data-testid={`legend-level-${level}`}
621 |                 data-level={level}
622 |                 className="h-3 w-3 rounded-[3px] shadow-[inset_0_0_1px_var(--heatmap-cell-outline)]"
623 |                 style={{ backgroundColor: INTENSITY_COLORS[level] }}
624 |             )/>
625 |         )]}
626 |     </div>
627 |     <span data-testid="legend-label-4">{legendLabels[4]}</span>
628 | </div>
629 | </div>
630 | );
631 | };
632 |
633 | export default memo(HeatmapGrid);
634 |

```

FILE: src/components/shared/HeatmapHabitSummary.tsx

TYPE: .tsx | LINES: 1-117 | SHA256: ee76d2905a11c5382289238ac3353d3996011eb9019ae5dca256603ca47ffeed

```

1 | import { Award, Target } from "lucide-react";
2 | import { Card } from "@components/ui/card";
3 | import { cn } from "@lib/Utils";
4 | import type { HabitSummary } from "@lib/heatmapUtils";
5 | import type { HabitType } from "@types/habits";
6 |
7 | // -----
8 | // Types
9 | // -----
10 |
11 | export interface HeatmapHabitSummaryProps {
12 |     summary: HabitSummary;
13 | }
14 |
15 | // -----
16 | // Helpers
17 | // -----
18 |
19 | const HABIT_LABELS: Record<HabitType, string> = {
20 |     login: "Login",
21 |     submit: "Submit",
22 |     journal: "Journal",
23 |     read: "Read",
24 |     meditation: "Meditation",
25 |     hydration: "Hydration",
26 |     exercise: "Exercise",
27 |     sleep: "Sleep",
28 | };
29 |
30 | interface SummaryCardProps {
31 |     icon: React.ElementType;
32 |     label: string;
33 |     value: string;
34 |     caption?: string;
35 |     iconColor: string;
36 |     iconBg: string;
37 |     testId: string;
38 | }
39 |
40 | const SummaryCard = ({
41 |     icon: Icon,
42 |     label,
43 |     value,
44 |     caption,
45 |     iconColor,
46 |     iconBg,
47 |     testId,
48 | }: SummaryCardProps) => (
49 |     <Card
50 |         className="bg-white border-0 shadow-md rounded-xl p-4 group"
51 |         data-testid={testId}
52 |     >
53 |         <div className="flex items-center justify-between">
54 |             <div>
55 |                 <p className="text-[10px] font-black tracking-widest uppercase text-gray-500">
56 |                     {label}
57 |                 </p>
58 |                 <p className="text-2xl font-black mt-1" data-testid={`-${testId}-value`} >
59 |                     {value}
60 |                 </p>
61 |                 {caption} && <p className="text-xs text-gray-500 mt-0.5">{caption}</p>
62 |             </div>
63 |             <div
64 |                 className={cn(
65 |                     "p-2 rounded-lg group-hover:scale-110 transition-transform",
66 |                     iconBg
67 |                 )}
68 |             >
69 |                 <Icon className={cn("h-5 w-5", iconColor)} />
70 |             </div>
71 |         </div>
72 |     </Card>
73 | );
74 |
75 | // -----
76 | // Component
77 | // -----
78 |
79 | /**
80 |  * Shows the student's best-performing habit and overall completion rate across
81 |  * the heatmap period (R7.2).

```

```

82 | */
83 | const HeatmapHabitSummary = ({ summary }: HeatmapHabitSummaryProps) => {
84 |   const bestHabitLabel =
85 |     summary.bestHabit != null ? HABIT_LABELS[summary.bestHabit] : "-";
86 |   const bestHabitCaption =
87 |     summary.bestHabit != null
88 |       ? `${summary.bestHabitCount} day${
89 |         summary.bestHabitCount === 1 ? "" : "s"
90 |       }`
91 |       : "No habits logged yet";
92 |
93 |   return (
94 |     <div className="grid grid-cols-2 gap-4" data-testid="heatmap-habit-summary">
95 |       <SummaryCard
96 |         icon={Award}
97 |         label="Best Habit"
98 |         value={bestHabitLabel}
99 |         caption={bestHabitCaption}
100 |         iconColor="text-amber-500"
101 |         iconBg="bg-amber-50"
102 |         testId="kpi-best-habit"
103 |       />
104 |       <SummaryCard
105 |         icon={Target}
106 |         label="Completion Rate"
107 |         value={` ${summary.completionRate}%`}
108 |         iconColor="text-green-600"
109 |         iconBg="bg-green-50"
110 |         testId="kpi-completion-rate"
111 |       />
112 |     </div>
113 |   );
114 | };
115 |
116 | export default HeatmapHabitSummary;
117 |

```

FILE: [src/components/shared/HeatmapLegend.tsx](#)

TYPE: .tsx | LINES: 1-62 | SHA256: e8ee9120cb44e8fbae8ab0b866fa795bc98ef340880651ff9d5277aee2fdd54e

```

1 | import { cn } from "@lib/utills";
2 |
3 | // -----
4 | // Shared "key" legend for matrix-style heatmaps (labeled color swatches).
5 | //
6 | // This is the categorical/key legend shared by the attainment-style heatmaps
7 | // (teacher student-performance, coordinator coverage). It is intentionally NOT
8 | // used by the student habits heatmap: that one is a sequential Less!More
9 | // *scale* legend (a different dataviz pattern with its own test contract), and
10 | // folding both shapes into one component would require variant flags – the
11 | // "wrong abstraction" this component deliberately avoids.
12 | // -----
13 |
14 | export interface HeatmapLegendItem {
15 |   /** Swatch background color (any valid CSS color / hex). */
16 |   color: string;
17 |   /** Human-readable meaning of the color. Conveys the info (not color alone). */
18 |   label: string;
19 | }
20 |
21 | export interface HeatmapLegendProps {
22 |   items: HeatmapLegendItem[];
23 |   /** Optional leading caption, e.g. "Attainment" or "Evidence count". */
24 |   title?: string;
25 |   className?: string;
26 |   "data-testid"?: string;
27 | }
28 |
29 | /**
30 |  * Renders a wrap-friendly row of labeled swatches. Swatches are decorative
31 |  * (aria-hidden) – the adjacent text label carries the meaning, so the legend
32 |  * is not color-dependent. A faint border keeps light/white swatches visible.
33 |  */
34 | const HeatmapLegend = ({
35 |   items,
36 |   title,
37 |   className,
38 |   "data-testid": testId,
39 | }: HeatmapLegendProps) => (
40 |   <div
41 |     className={cn(
42 |       "flex flex-wrap items-center gap-x-4 gap-y-2 text-xs text-muted-foreground",
43 |       className
44 |     )}
45 |     data-testid={testId}
46 |   >
47 |     {title && <span className="font-semibold text-foreground/80">{title}</span>}
48 |     {items.map((item) => (
49 |       <span key={item.label} className="inline-flex items-center gap-1.5">
50 |         <span
51 |           aria-hidden="true"
52 |           className="h-3 w-3 shrink-0 rounded-sm border border-black/10"
53 |           style={{ backgroundColor: item.color }}
54 |         />
55 |         {item.label}
56 |       </span>
57 |     ))}
58 |   </div>
59 | );
60 |
61 | export default HeatmapLegend;
62 |

```

FILE: src/components/shared/HeatmapPlainSummary.tsx

TYPE: .tsx | LINES: 1-79 | SHA256: b1ee3826c028874181c8f7d07fcc4efff1b81400d30f2eb6358384c600160892

```
1 | // =====
2 | // HeatmapPlainSummary – plain-language habit summary for younger students (R22.1)
3 | // =====
4 | //
5 | // Conveys the student's habit performance in approachable, jargon-free prose so
6 | // the grid does not have to be interpreted on its own. Wording resolves with a
7 | // runtime cross-language fallback (R22.6) via `useApproachableWording`.
8 |
9 | import { Sparkles } from "lucide-react";
10 | import { Card } from "@components/ui/card";
11 | import { useApproachableWording } from "@hooks/useApproachableWording";
12 | import type { HabitSummary } from "@lib/heatmapUtils";
13 |
14 | export interface HeatmapPlainSummaryProps {
15 |   summary: HabitSummary;
16 |   /** Current academic-habit streak in days. */
17 |   currentStreak: number;
18 | }
19 |
20 | /**
21 |  * Renders a friendly, plain-language summary of habit performance.
22 |  *
23 |  * Shows an encouraging empty message when nothing is logged yet, otherwise a
24 |  * completion-rate sentence plus best-habit and streak context – all in
25 |  * approachable wording suitable for younger students (R22.1).
26 |  */
27 | const HeatmapPlainSummary = ({
28 |   summary,
29 |   currentStreak,
30 | }: HeatmapPlainSummaryProps) => {
31 |   const tw = useApproachableWording("student");
32 |
33 |   const hasData = summary.bestHabit !== null || summary.completionRate > 0;
34 |
35 |   const lines: string[] = [];
36 |   if (!hasData) {
37 |     lines.push(tw("heatmap.plainSummary.empty"));
38 |   } else {
39 |     lines.push(
40 |       tw("heatmap.plainSummary.completion", { rate: summary.completionRate })
41 |     );
42 |     if (summary.bestHabit !== null) {
43 |       lines.push(
44 |         tw("heatmap.plainSummary.bestHabit", {
45 |           habit: tw(`heatmap.habitLabels.${summary.bestHabit}`),
46 |         })
47 |       );
48 |     }
49 |     if (currentStreak > 0) {
50 |       lines.push(tw("heatmap.plainSummary.streak", { count: currentStreak }));
51 |     } else {
52 |       lines.push(tw("heatmap.plainSummary.encourage"));
53 |     }
54 |   }
55 |
56 |   return (
57 |     <Card
58 |       className="bg-white border-0 shadow-md rounded-xl p-4"
59 |       data-testid="heatmap-plain-summary"
60 |     >
61 |       <div className="flex items-start gap-3">
62 |         <div className="p-2 rounded-lg bg-teal-50 shrink-0">
63 |           <Sparkles className="h-5 w-5 text-teal-600" aria-hidden="true" />
64 |         </div>
65 |         <div>
66 |           <p className="text-[10px] font-black tracking-widest uppercase text-gray-500">
67 |             {tw("heatmap.plainSummary.title")}
68 |           </p>
69 |           <p className="text-sm font-medium text-gray-700 mt-1 leading-relaxed">
70 |             {lines.join(" ")}
71 |           </p>
72 |         </div>
73 |       </div>
74 |     </Card>
75 |   );
76 | };
77 |
78 | export default HeatmapPlainSummary;
79 |
```

FILE: src/components/shared/HeatmapSummaryStats.tsx

TYPE: .tsx | LINES: 1-107 | SHA256: afee4d1d020298adceb9e208dc5bbcb7bd393a4c5cfbada93ddd711eaf26a454

```
1 | import { Flame, Trophy, Calendar } from "lucide-react";
2 | import { Card } from "@components/ui/card";
3 | import { cn } from "@lib/Utils";
4 | import type { HeatmapSummary } from "@types/habits";
5 |
6 | // -----
7 | // Types
8 | // -----
9 |
10 | export interface HeatmapSummaryStatsProps extends HeatmapSummary {
11 |   sabbaticalDayCount?: number;
12 | }
13 |
14 | // -----
15 | // Internal KPI Card
16 | // -----
17 |
```



```

18 | interface KPICardProps {
19 |   icon: React.ElementType;
20 |   label: string;
21 |   value: number;
22 |   iconColor: string;
23 |   iconBg: string;
24 |   testId: string;
25 | }
26 |
27 | const KPICard = ({
28 |   icon,
29 |   label,
30 |   value,
31 |   iconColor,
32 |   iconBg,
33 |   testId,
34 | }: KPICardProps) => (
35 |   <Card
36 |     className="bg-white border-0 shadow-md rounded-xl p-4 group"
37 |     data-testid={testId}
38 |   >
39 |     <div className="flex items-center justify-between">
40 |       <div>
41 |         <p className="text-[10px] font-black tracking-widest uppercase text-gray-500">
42 |           {label}
43 |         </p>
44 |         <p className="text-2xl font-black mt-1" data-testid={`_${testId}-value`} >
45 |           {value}
46 |         </p>
47 |       </div>
48 |       <div>
49 |         className={cn(
50 |           "p-2 rounded-lg group-hover:scale-110 transition-transform",
51 |           iconBg
52 |         )}
53 |       >
54 |       <Icon className={cn("h-5 w-5", iconColor)} />
55 |     </div>
56 |   </div>
57 | </Card>
58 | );
59 |
60 | // -----
61 | // Component
62 | // -----
63 |
64 | const HeatmapSummaryStats = ({
65 |   currentStreak,
66 |   longestStreak,
67 |   totalActiveDays,
68 |   sabbaticalDayCount,
69 | }: HeatmapSummaryStatsProps) => {
70 |   // When sabbatical days are provided, display adjusted total active days label
71 |   const activeDaysLabel =
72 |     sabbaticalDayCount !== null && sabbaticalDayCount > 0
73 |     ? "Active Days (excl. rest)"
74 |     : "Total Active Days";
75 |
76 |   return (
77 |     <div className="grid grid-cols-3 gap-4" data-testid="heatmap-summary-stats">
78 |       <KPICard
79 |         icon={Flame}
80 |         label="Current Streak"
81 |         value={currentStreak}
82 |         iconColor="text-orange-500"
83 |         iconBg="bg-orange-50"
84 |         testId="kpi-current-streak"
85 |       />
86 |       <KPICard
87 |         icon={Trophy}
88 |         label="Longest Streak"
89 |         value={longestStreak}
90 |         iconColor="text-amber-500"
91 |         iconBg="bg-amber-50"
92 |         testId="kpi-longest-streak"
93 |       />
94 |       <KPICard
95 |         icon={Calendar}
96 |         label={activeDaysLabel}
97 |         value={totalActiveDays}
98 |         iconColor="text-blue-600"
99 |         iconBg="bg-blue-50"
100 |         testId="kpi-total-active-days"
101 |       />
102 |     </div>
103 |   );
104 | };
105 |
106 | export default HeatmapSummaryStats;
107 |

```

FILE: [src/components/shared/HeatmapTooltip.tsx](#)

TYPE: .tsx | LINES: 1-190 | SHA256: [abbacbe19567a2ac414344a25519a4a1295b8a879428fb39312e741a520b7596](#)

```

1 | import { cn } from "@lib/utls";
2 | import type { CompletedHabit, StreakMilestone } from "@types/habits";
3 |
4 | // -----
5 | // Types
6 | // -----
7 |
8 | export interface HeatmapTooltipProps {
9 |   date: string;
10 |   habits: CompletedHabit[];

```

```

11 |   xpEarned: number;
12 |   streakActive: boolean;
13 |   position?: { x: number; y: number };
14 |   isComebackDay?: boolean;
15 |   comebackDayNumber?: number;
16 |   isSabbaticalRestDay?: boolean;
17 |   milestone?: StreakMilestone;
18 |   /** Whether this day qualifies as a Perfect Day (all 4 academic habits). */
19 |   isPerfectDay?: boolean;
20 |   /** XP awarded for a Perfect Day, shown as reward context (R7.3). */
21 |   perfectDayXp?: number;
22 | }
23 |
24 | // -----
25 | // Helpers
26 | // -----
27 |
28 | const HABIT_LABELS: Record<string, string> = {
29 |   login: "Login",
30 |   submit: "Submit",
31 |   journal: "Journal",
32 |   read: "Read",
33 |   meditation: "Meditation",
34 |   hydration: "Hydration",
35 |   exercise: "Exercise",
36 |   sleep: "Sleep",
37 | };
38 |
39 | function formatDate(dateStr: string): string {
40 |   const d = new Date(dateStr + "T00:00:00");
41 |   return d.toLocaleDateString("en-US", {
42 |     month: "long",
43 |     day: "numeric",
44 |     year: "numeric",
45 |   });
46 | }
47 |
48 | // -----
49 | // Component
50 | // -----
51 |
52 | const HeatmapTooltip = ({
53 |   date,
54 |   habits,
55 |   xpEarned,
56 |   streakActive,
57 |   position,
58 |   isComebackDay,
59 |   comebackDayNumber,
60 |   isSabbaticalRestDay: isSabbatical,
61 |   milestone,
62 |   isPerfectDay,
63 |   perfectDayXp,
64 | }: HeatmapTooltipProps) => {
65 |   const academic = habits.filter((h) => h.category === "academic");
66 |   const wellness = habits.filter((h) => h.category === "wellness");
67 |   const isEmpty = habits.length === 0;
68 |
69 |   return (
70 |     <div
71 |       className={cn(
72 |         "bg-white border border-slate-200 shadow-lg rounded-xl p-4 min-w-[220px] max-w-[280px] z-50",
73 |         position ? "absolute" : "relative"
74 |       )}
75 |       style={position ? { left: position.x, top: position.y } : undefined}
76 |       role="tooltip"
77 |       data-testid="heatmap-tooltip"
78 |     >
79 |       <p
80 |         className="text-sm font-semibold text-gray-900"
81 |         data-testid="tooltip-date"
82 |       >
83 |         {formatDate(date)}
84 |       </p>
85 |
86 |       {/* Comeback Challenge label */}
87 |       {isComebackDay && comebackDayNumber != null && (
88 |         <p
89 |           className="text-xs font-medium text-teal-600 mt-1"
90 |           data-testid="tooltip-comeback-label"
91 |         >
92 |           Comeback Day {comebackDayNumber}/3
93 |         </p>
94 |       )}
95 |
96 |       {/* Sabbatical rest day label */}
97 |       {isSabbatical && (
98 |         <p
99 |           className="text-xs font-medium text-slate-500 mt-1"
100 |           data-testid="tooltip-sabbatical-label"
101 |         >
102 |           Rest Day (Sabbatical)
103 |         </p>
104 |       )}
105 |
106 |       {/* Milestone label */}
107 |       {milestone && (
108 |         <p
109 |           className="text-xs font-medium text-amber-600 mt-1"
110 |           data-testid="tooltip-milestone-label"
111 |         >
112 |           {milestone.days}-Day Streak Milestone 0<R%
113 |         </p>
114 |       )}

```

```

115 |
116 |     { /* Perfect Day reward context */ }
117 |     { isPerfectDay && (
118 |       <p
119 |         className="text-xs font-medium text-amber-600 mt-1"
120 |         data-testid="tooltip-perfect-day-label"
121 |       >
122 |         +P Perfect Day
123 |         { perfectDayXp !== null && perfectDayXp > 0
124 |           ? `+${perfectDayXp} XP bonus`
125 |           : "" }
126 |       </p>
127 |     ) }
128 |
129 |     { isEmpty && !isSabbatical ? (
130 |       <p className="text-sm text-gray-500 mt-2" data-testid="tooltip-empty">
131 |         No habits completed
132 |       </p>
133 |     ) : isEmpty && isSabbatical ? null : (
134 |       <div
135 |         { academic.length > 0 && (
136 |           <div className="mt-2" data-testid="tooltip-academic-section">
137 |             <p className="text-[10px] font-black tracking-widest uppercase text-gray-400">
138 |               Academic
139 |             </p>
140 |             <ul className="mt-1 space-y-0.5">
141 |               { academic.map((h) => (
142 |                 <li
143 |                   key={h.type}
144 |                   className="flex items-center gap-1.5 text-sm text-gray-700"
145 |                   data-testid={`tooltip-habit-${h.type}`}
146 |                 >
147 |                   <span className="text-green-500">' </span>
148 |                   { HABIT_LABELS[h.type] ?? h.type }
149 |                 </li>
150 |               ) ) }
151 |             </ul>
152 |           </div>
153 |         ) }
154 |
155 |         { wellness.length > 0 && (
156 |           <div className="mt-2" data-testid="tooltip-wellness-section">
157 |             <p className="text-[10px] font-black tracking-widest uppercase text-gray-400">
158 |               Wellness
159 |             </p>
160 |             <ul className="mt-1 space-y-0.5">
161 |               { wellness.map((h) => (
162 |                 <li
163 |                   key={h.type}
164 |                   className="flex items-center gap-1.5 text-sm text-gray-700"
165 |                   data-testid={`tooltip-habit-${h.type}`}
166 |                 >
167 |                   <span className="text-green-500">' </span>
168 |                   { HABIT_LABELS[h.type] ?? h.type }
169 |                 </li>
170 |               ) ) }
171 |             </ul>
172 |           </div>
173 |         ) }
174 |
175 |         <div className="mt-3 flex items-center justify-between border-t border-slate-100 pt-2">
176 |           <span className="text-xs text-gray-500" data-testid="tooltip-xp">
177 |             { xpEarned } XP earned
178 |           </span>
179 |           <span className="text-xs font-medium" data-testid="tooltip-streak">
180 |             { streakActive ? "Ø=Y% Streak active" : "Streak broken" }
181 |           </span>
182 |         </div>
183 |       </>
184 |     ) }
185 |   </div>
186 | );
187 | };
188 |
189 | export default HeatmapTooltip;
190 |

```

FILE: src/components/shared/ImpersonationBanner.tsx

TYPE: .tsx | LINES: 1-50 | SHA256: 2907a82fa468e6bbe69ae3946ff3d7ced682dbd57cc07c6d7aa8266a9511b61

```

1 | // Task 88.2: Impersonation banner
2 | // Prominent top banner shown during impersonation with exit action
3 |
4 | import { useImpersonation } from "@/hooks/useImpersonation";
5 | import { Button } from "@/components/ui/button";
6 | import { Eye, X } from "lucide-react";
7 |
8 | const formatTime = (seconds: number): string => {
9 |   const m = Math.floor(seconds / 60);
10 |   const s = seconds % 60;
11 |   return `${m}:${s.toString().padStart(2, "0")}`;
12 | };
13 |
14 | const ImpersonationBanner = () => {
15 |   const {
16 |     isImpersonating,
17 |     impersonatedUser,
18 |     timeRemaining,
19 |     stopImpersonation,
20 |   } = useImpersonation();
21 |
22 |   if (!isImpersonating || !impersonatedUser) return null;
23 |
24 |   return (

```

```

25 | <div className="bg-amber-500 text-white px-4 py-2 flex items-center justify-between z-50">
26 |   <div className="flex items-center gap-2 text-sm font-medium">
27 |     <Eye className="h-4 w-4" />
28 |     <span>
29 |       You are viewing as {impersonatedUser.full_name} -{" "}
30 |       {impersonatedUser.role}
31 |     </span>
32 |     <span className="text-white/80">
33 |       ({formatTime(timeRemaining)} remaining)
34 |     </span>
35 |   </div>
36 |   <Button
37 |     variant="ghost"
38 |     size="sm"
39 |     onClick={stopImpersonation}
40 |     className="text-white hover:bg-amber-600 gap-1"
41 |   >
42 |     <X className="h-4 w-4" />
43 |     Exit
44 |   </Button>
45 | </div>
46 | );
47 | };
48 |
49 | export default ImpersonationBanner;
50 |

```

FILE: src/components/shared/ImprovementBonusCelebration.tsx

TYPE: .tsx | LINES: 1-107 | SHA256: ee7a84558864cd18e332fb2096500cc434d45f8e1635fc8b2303b8b1c230549a

```

1 | // Task 139.4: Improvement Bonus Celebration component
2 | // Uses Framer Motion + canvas-confetti, honors prefers-reduced-motion
3 |
4 | import { useEffect, useState, useRef, useCallback } from "react";
5 | import { motion, AnimatePresence, useReducedMotion } from "framer-motion";
6 | import { launchConfetti } from "@lib/confetti";
7 | import { TrendingUp, Sparkles } from "lucide-react";
8 |
9 | interface ImprovementBonusCelebrationProps {
10 |   cloTitle: string;
11 |   bonusXP: number;
12 |   previousPercent: number;
13 |   currentPercent: number;
14 |   onDismiss: () => void;
15 | }
16 |
17 | const ImprovementBonusCelebration = ({
18 |   cloTitle,
19 |   bonusXP,
20 |   previousPercent,
21 |   currentPercent,
22 |   onDismiss,
23 | }: ImprovementBonusCelebrationProps) => {
24 |   const [visible, setVisible] = useState(true);
25 |   const prefersReducedMotion = useReducedMotion();
26 |   const confettiFired = useRef(false);
27 |
28 |   const fireConfetti = useCallback(() => {
29 |     if (confettiFired.current || prefersReducedMotion) return;
30 |     confettiFired.current = true;
31 |
32 |     void launchConfetti({
33 |       particleCount: 40,
34 |       spread: 60,
35 |       origin: { x: 0.85, y: 0.85 },
36 |       colors: ["#22c55e", "#f59e0b", "#3b82f6"],
37 |       disableForReducedMotion: true,
38 |     });
39 |   }, [prefersReducedMotion]);
40 |
41 |   useEffect(() => {
42 |     fireConfetti();
43 |   }, [fireConfetti]);
44 |
45 |   useEffect(() => {
46 |     const duration = prefersReducedMotion ? 1000 : 3000;
47 |     const timer = setTimeout(() => {
48 |       setVisible(false);
49 |       onDismiss();
50 |     }, duration);
51 |     return () => clearTimeout(timer);
52 |   }, [onDismiss, prefersReducedMotion]);
53 |
54 |   return (
55 |     <AnimatePresence>
56 |       {visible && (
57 |         <motion.div
58 |           role="status"
59 |           aria-live="polite"
60 |           initial={
61 |             prefersReducedMotion
62 |               ? { opacity: 1 }
63 |               : { opacity: 0, scale: 0.8, y: 20 }
64 |           }
65 |           animate={{ opacity: 1, scale: 1, y: 0 }}
66 |           exit={
67 |             prefersReducedMotion
68 |               ? { opacity: 0 }
69 |               : { opacity: 0, scale: 0.8, y: -20 }
70 |           }
71 |           transition={
72 |             prefersReducedMotion
73 |               ? { duration: 0.1 }
74 |               : { type: "spring", stiffness: 300, damping: 20 }

```

```

75 |         }
76 |         className="fixed bottom-6 end-6 z-50 bg-white shadow-xl rounded-xl p-4 border border-green-200 max-w
77 |     >
78 |     <div className="flex items-start gap-3">
79 |         <div className="p-2 rounded-lg bg-green-50">
80 |             <TrendingUp className="h-5 w-5 text-green-600" />
81 |         </div>
82 |         <div>
83 |             <div className="flex items-center gap-1">
84 |                 <Sparkles className="h-4 w-4 text-amber-500" />
85 |                 <p className="text-sm font-bold text-green-700">
86 |                     Great improvement!
87 |                 </p>
88 |             </div>
89 |             <p className="text-xs text-gray-600 mt-0.5">
90 |                 Great improvement on {cloTitle}!
91 |             </p>
92 |             <p className="text-xs text-gray-400 mt-0.5">
93 |                 {Math.round(previousPercent)}% !' {Math.round(currentPercent)}%
94 |             </p>
95 |             <p className="text-sm font-bold text-amber-600 mt-1">
96 |                 +{bonusXP} XP Bonus
97 |             </p>
98 |         </div>
99 |     </div>
100 | </motion.div>
101 |   )}
102 | </AnimatePresence>
103 | );
104 | };
105 |
106 | export default ImprovementBonusCelebration;
107 |

```

FILE: src/components/shared/IndependenceScoreBadge.tsx

TYPE: .tsx | LINES: 1-53 | SHA256: 1e4340c2a983d681216c3b5c12d08fe2c31c612986e791be3ac6b37ebddf0664

```

1 | // =====
2 | // IndependenceScoreBadge – Color-coded independence score chip
3 | // =====
4 |
5 | import { cn } from "@lib/utils";
6 |
7 | interface IndependenceScoreBadgeProps {
8 |   /** Independence score between 0 and 1 */
9 |   score: number;
10 |   /** Badge size variant */
11 |   size?: "sm" | "md";
12 | }
13 |
14 | /**
15 |  * Small badge/chip showing independence score as percentage.
16 |  * Color-coded: green ("e70%), yellow (40-69%), red (<40%)
17 |  */
18 | const IndependenceScoreBadge = ({
19 |   score,
20 |   size = "sm",
21 | }: IndependenceScoreBadgeProps) => {
22 |   const percentage = Math.round(score * 100);
23 |
24 |   const colorClasses =
25 |     percentage >= 70
26 |       ? "bg-green-50 text-green-700 border-green-200"
27 |       : percentage >= 40
28 |       ? "bg-yellow-50 text-yellow-700 border-yellow-200"
29 |       : "bg-red-50 text-red-700 border-red-200";
30 |
31 |   const sizeClasses =
32 |     size === "md" ? "text-xs px-2 py-0.5" : "text-[10px] px-1.5 py-0";
33 |
34 |   return (
35 |     <span
36 |       className={cn(
37 |         "inline-flex items-center gap-1 rounded-full border font-bold whitespace-nowrap",
38 |         colorClasses,
39 |         sizeClasses
40 |       )}
41 |       title={`Independence: ${percentage}% (${
42 |         percentage >= 70 ? "High" : percentage >= 40 ? "Moderate" : "Low"
43 |       })`}
44 |       aria-label={`Independence score: ${percentage}%`}
45 |     >
46 |       {percentage}% ind.
47 |     </span>
48 |   );
49 | };
50 |
51 | export default IndependenceScoreBadge;
52 | export type { IndependenceScoreBadgeProps };
53 |

```

FILE: src/components/shared/KPICard.tsx

TYPE: .tsx | LINES: 1-59 | SHA256: c042b991edd51cc7c599e19fd7361c3972b0a7e767cbf56d5c65d13c5b03a830

```

1 | // =====
2 | // KPICard – Reusable KPI card with icon, label, value, and hover effect
3 | // =====
4 |
5 | import type { LucideIcon } from "lucide-react";
6 | import { PCard } from "@design-system";
7 | import { cn } from "@lib/utils";
8 |
9 | interface KPICardProps {
10 |   icon: LucideIcon;

```

```

11 |   label: string;
12 |   value: string | number;
13 |   iconBgClass?: string;
14 |   iconColorClass?: string;
15 |   /**
16 |    * Classes for the KPI value. Defaults to deep brand blue (#0369a1 / sky-700)
17 |    * instead of near-black, per the design system (harsh black stat numbers were
18 |    * intentionally removed). Pass a semantic color (e.g. "text-green-600") for
19 |    * status-bearing values. No dark variant here because the card surface stays
20 |    * white in both themes; revisit if the card adopts `bg-card`.
21 |    */
22 |   valueClassName?: string;
23 |   className?: string;
24 | }
25 |
26 | const KPICard = ({
27 |   icon: Icon,
28 |   label,
29 |   value,
30 |   iconBgClass = "bg-blue-50",
31 |   iconColorClass = "text-blue-600",
32 |   valueClassName = "text-sky-700",
33 |   className,
34 | }: KPICardProps) => (
35 |   <PCard className={cn("p-4 group", className)}>
36 |     <div className="flex items-center justify-between">
37 |       <div>
38 |         <p className="text-[10px] font-black tracking-widest uppercase text-gray-500">
39 |           {label}
40 |         </p>
41 |         <p className={cn("text-2xl font-black mt-1", valueClassName)}>
42 |           {value}
43 |         </p>
44 |       </div>
45 |       <div
46 |         className={cn(
47 |           "p-2 rounded-lg transition-transform group-hover:scale-110",
48 |           iconBgClass
49 |         )}
50 |       >
51 |         <Icon className={cn("h-5 w-5", iconColorClass)} />
52 |       </div>
53 |     </div>
54 |   </PCard>
55 | );
56 |
57 | export default KPICard;
58 | export type { KPICardProps };
59 |

```

FILE: [src/components/shared/LanguageSelector.tsx](#)

TYPE: .tsx | LINES: 1-45 | SHA256: 3764359a03b6daa8b139b9dd66428b694d25bcfee6e0bee73e86c6e6ec192713

```

1 | // Task 81.5: Language selector dropdown – English, bv1b0H (Urdu), bvDc-1b†Jb' „ & &-2•
2 |
3 | import {
4 |   Select,
5 |   SelectContent,
6 |   SelectItem,
7 |   SelectTrigger,
8 |   SelectValue,
9 | } from "@components/ui/select";
10 | import { useLanguage, type Language } from "@providers/LanguageProvider";
11 | import { Globe } from "lucide-react";
12 |
13 | const LANGUAGES = [
14 |   { value: "en", label: "English" },
15 |   { value: "ur", label: "bv1b0H" },
16 |   { value: "ar", label: "bvDc-1b†Jb' " òÀ
17 | ] as const;
18 |
19 | const LanguageSelector = () => {
20 |   const { language, setLanguage } = useLanguage();
21 |
22 |   return (
23 |     <div className="flex items-center gap-2">
24 |       <Globe className="h-4 w-4 text-gray-500" />
25 |       <Select
26 |         value={language}
27 |         onValueChange={(v) => setLanguage(v as Language)}
28 |       >
29 |         <SelectTrigger className="w-36 bg-white">
30 |           <SelectValue />
31 |         </SelectTrigger>
32 |         <SelectContent>
33 |           {LANGUAGES.map((lang) => (
34 |             <SelectItem key={lang.value} value={lang.value}>
35 |               {lang.label}
36 |             </SelectItem>
37 |           ))}
38 |         </SelectContent>
39 |       </Select>
40 |     </div>
41 |   );
42 | };
43 |
44 | export default LanguageSelector;
45 |

```

FILE: src/components/shared/LanguageSwitcher.tsx

TYPE: .tsx | LINES: 1-62 | SHA256: b1a83e9c856c0ecc8178a900bad5aa92314d5b0bf425efbd092424ae820ac0c0

```
1 | import { useTranslation } from "react-i18next";
2 | import { Button } from "@components/ui/button";
3 | import {
4 |   DropdownMenu,
5 |   DropdownMenuContent,
6 |   DropdownMenuItem,
7 |   DropdownMenuTrigger,
8 | } from "@components/ui/dropdown-menu";
9 | import { Globe } from "lucide-react";
10 | import { useUpdateLanguagePreference } from "@hooks/useLanguagePreference";
11 | import { useAuth } from "@hooks/useAuth";
12 |
13 | const languages = [
14 |   { code: "en", nativeLabel: "English" },
15 |   { code: "ar", nativeLabel: "بvDc-1b†Jb' " òÀ
16 | ] as const;
17 |
18 | export const LanguageSwitcher = () => {
19 |   const { i18n } = useTranslation();
20 |   const { user } = useAuth();
21 |   const updatePreference = useUpdateLanguagePreference();
22 |
23 |   const handleLanguageChange = (langCode: string) => {
24 |     i18n.changeLanguage(langCode);
25 |     localStorage.setItem("edeviser-language", langCode);
26 |     if (user) {
27 |       updatePreference.mutate(langCode);
28 |     }
29 |   };
30 |
31 |   const currentLang =
32 |     languages.find((l) => l.code === i18n.language) || languages[0];
33 |
34 |   return (
35 |     <DropdownMenu>
36 |       <DropdownMenuTrigger asChild>
37 |         <Button
38 |           variant="ghost"
39 |           size="sm"
40 |           className="gap-2"
41 |           aria-label={`Language: ${currentLang.nativeLabel}`}
42 |         >
43 |           <Globe className="h-4 w-4" />
44 |           {currentLang.nativeLabel}
45 |         </Button>
46 |       </DropdownMenuTrigger>
47 |       <DropdownMenuContent align="end">
48 |         {languages.map((lang) => (
49 |           <DropdownMenuItem
50 |             key={lang.code}
51 |             onClick={() => handleLanguageChange(lang.code)}
52 |             className={i18n.language === lang.code ? "bg-accent" : ""}
53 |             aria-current={i18n.language === lang.code ? "true" : undefined}
54 |           >
55 |             {lang.nativeLabel}
56 |           </DropdownMenuItem>
57 |         ))}
58 |       </DropdownMenuContent>
59 |     </DropdownMenu>
60 |   );
61 | };
62 |
```

FILE: src/components/shared/LeaguePromotionCelebration.tsx

TYPE: .tsx | LINES: 1-76 | SHA256: 396e0de503404d5873d99b3a94cdfabbda98b4067106a09bd600982a3af1202e

```
1 | // Task 148.8: League Promotion Celebration component
2 |
3 | import { useEffect, useState } from "react";
4 | import { motion, AnimatePresence } from "framer-motion";
5 | import { type LeagueTierName, TIER_COLORS } from "@lib/leagueTier";
6 | import { Crown, Sparkles } from "lucide-react";
7 |
8 | interface LeaguePromotionCelebrationProps {
9 |   newTier: LeagueTierName;
10 |   onDismiss: () => void;
11 | }
12 |
13 | const LeaguePromotionCelebration = ({
14 |   newTier,
15 |   onDismiss,
16 | }: LeaguePromotionCelebrationProps) => {
17 |   const [visible, setVisible] = useState(true);
18 |
19 |   useEffect(() => {
20 |     const prefersReduced = window.matchMedia(
21 |       "(prefers-reduced-motion: reduce)"
22 |     ).matches;
23 |     const timer = setTimeout(
24 |       () => {
25 |         setVisible(false);
26 |         onDismiss();
27 |       },
28 |       prefersReduced ? 1500 : 4000
29 |     );
30 |     return () => clearTimeout(timer);
31 |   }, [onDismiss]);
32 |
33 |   return (
34 |     <AnimatePresence>
```

```

35 |         {visible && (
36 |             <motion.div
37 |                 initial={{ opacity: 0 }}
38 |                 animate={{ opacity: 1 }}
39 |                 exit={{ opacity: 0 }}
40 |                 className="fixed inset-0 z-50 flex items-center justify-center bg-black/50"
41 |                 onClick={() => {
42 |                     setVisible(false);
43 |                     onDismiss();
44 |                 }}
45 |             >
46 |                 <motion.div
47 |                     initial={{ scale: 0.5, y: 50 }}
48 |                     animate={{ scale: 1, y: 0 }}
49 |                     exit={{ scale: 0.5, y: 50 }}
50 |                     transition={{ type: "spring", stiffness: 200, damping: 15 }}
51 |                     className="bg-white rounded-2xl shadow-2xl p-8 text-center max-w-sm mx-4"
52 |                     onClick={(e) => e.stopPropagation()}
53 |                 >
54 |                     <div className="flex justify-center mb-4">
55 |                         <div className={`p-4 rounded-full ${TIER_COLORS[newTier]} `}>
56 |                             <Crown className="h-8 w-8" />
57 |                         </div>
58 |                         <div>
59 |                             <h2 className="text-xl font-black">League Promotion!</h2>
60 |                             <p className="text-gray-600 mt-1">You've been promoted to</p>
61 |                             <p className="text-2xl font-black mt-2">{newTier} League</p>
62 |                             <div className="flex items-center justify-center gap-1 mt-3">
63 |                                 <Sparkles className="h-4 w-4 text-amber-500" />
64 |                                 <span className="text-sm font-bold text-amber-600">
65 |                                     +100 XP Bonus
66 |                                 </span>
67 |                             </div>
68 |                         </div>
69 |                     </motion.div>
70 |                 </motion.div>
71 |             </AnimatePresence>
72 |         );
73 |     };
74 |
75 |     export default LeaguePromotionCelebration;
76 |

```

FILE: src/components/shared/LeagueTierBadge.tsx

TYPE: .tsx | LINES: 1-44 | SHA256: 44bc315ebe386b02b6ae973ea6bd80b65ec25a5cdf16e5cfb2472e23d672a9b

```

1 | // Task 148.7: League Tier Badge component
2 | // Requirements: 132.3
3 |
4 | import { type LeagueTierName, TIER_COLORS } from "@lib/leagueTier";
5 | import { Shield } from "lucide-react";
6 | import { cn } from "@lib/utils";
7 |
8 | interface LeagueTierBadgeProps {
9 |   tier: LeagueTierName;
10 |   size?: "sm" | "md" | "lg";
11 |   className?: string;
12 | }
13 |
14 | const SIZE_MAP = {
15 |   sm: { badge: "px-2 py-0.5 text-xs", icon: "h-3 w-3" },
16 |   md: { badge: "px-3 py-1 text-sm", icon: "h-4 w-4" },
17 |   lg: { badge: "px-4 py-1.5 text-base", icon: "h-5 w-5" },
18 | } as const;
19 |
20 | const LeagueTierBadge = ({
21 |   tier,
22 |   size = "md",
23 |   className,
24 | }: LeagueTierBadgeProps) => {
25 |   const sizeClasses = SIZE_MAP[size];
26 |
27 |   return (
28 |     <span
29 |       className={cn(
30 |         "inline-flex items-center gap-1 rounded-full font-bold",
31 |         TIER_COLORS[tier],
32 |         sizeClasses.badge,
33 |         className
34 |       )}
35 |     >
36 |       <Shield className={sizeClasses.icon} />
37 |       {tier}
38 |     </span>
39 |   );
40 | };
41 |
42 |
43 | export default LeagueTierBadge;
44 |

```

FILE: src/components/shared/LearningPlanCard.tsx

TYPE: .tsx | LINES: 1-201 | SHA256: 19cfb6e70a85e34a49789a20ab02925dca0ded756dae164a864c2d986e72bc3e

```

1 | // =====
2 | // LearningPlanCard – Gradient-bordered card for AI tutor learning plan updates
3 | // Displays study time recommendation, material links, and planner session count
4 | // with accept/modify/dismiss actions.
5 | // =====
6 |
7 | import { useState } from "react";
8 | import { Button } from "@components/ui/button";
9 | import { Card } from "@components/ui/card";
10 | import { Input } from "@components/ui/input";

```


[illegible]

```

115 |
116 |     {/* Planner sessions */}
117 |     <div className="flex items-center gap-2">
118 |       <Calendar className="h-4 w-4 text-blue-500 shrink-0" />
119 |       <p className="text-sm font-medium text-slate-700">
120 |         {planUpdate.suggested_planner_sessions} planner{" "}
121 |         {planUpdate.suggested_planner_sessions === 1
122 |           ? "session"
123 |           : "sessions"}
124 |       /week suggested
125 |     </p>
126 |   </div>
127 |
128 |   {/* Modify input */}
129 |   {showModifyInput && !responded && (
130 |     <div className="flex items-center gap-2 pt-1">
131 |       <Input
132 |         value={modifications}
133 |         onChange={(e) => setModifications(e.target.value)}
134 |         placeholder="Describe your modifications..."
135 |         className="text-sm"
136 |         onKeyDown={(e) => {
137 |           if (e.key === "Enter") handleModifySubmit();
138 |         }}
139 |       />
140 |       <Button
141 |         size="sm"
142 |         onClick={handleModifySubmit}
143 |         disabled={!modifications.trim() || isResponding}
144 |         className="bg-blue-600 hover:bg-blue-700 text-white"
145 |       >
146 |         Send
147 |       </Button>
148 |     </div>
149 |   )}
150 |
151 |   {/* Action buttons */}
152 |   {!responded && (
153 |     <div className="flex items-center gap-2 pt-1">
154 |       <Button
155 |         size="sm"
156 |         onClick={handleAccept}
157 |         disabled={isResponding}
158 |         className={cn(
159 |           "bg-green-600 hover:bg-green-700 text-white active:scale-95 transition-transform duration-100"
160 |         )}
161 |       >
162 |         <Check className="h-4 w-4" />
163 |         Accept
164 |       </Button>
165 |       <Button
166 |         size="sm"
167 |         variant="outline"
168 |         onClick={() => setShowModifyInput(!showModifyInput)}
169 |         disabled={isResponding}
170 |         className="text-blue-600 border-blue-200 hover:bg-blue-50"
171 |       >
172 |         <Pencil className="h-4 w-4" />
173 |         Modify
174 |       </Button>
175 |       <Button
176 |         size="sm"
177 |         variant="outline"
178 |         onClick={handleDismiss}
179 |         disabled={isResponding}
180 |         className="text-slate-500 border-slate-200 hover:bg-slate-50"
181 |       >
182 |         <X className="h-4 w-4" />
183 |         Dismiss
184 |       </Button>
185 |     </div>
186 |   )}
187 |
188 |   {/* Responded state */}
189 |   {responded && (
190 |     <p className="text-xs text-slate-400 italic pt-1">
191 |       Response recorded. Thank you!
192 |     </p>
193 |   )}
194 | </div>
195 | </Card>
196 | </div>
197 | );
198 | };
199 |
200 | export default LearningPlanCard;
201 |

```

FILE: src/components/shared/LevelProgress.tsx

TYPE: .tsx | LINES: 1-111 | SHA256: 65cff60b936ff8ad10bd056543b0a0ad3c6d8e38d9e4f53ee77bf9229795b4d7

```

1 | // =====
2 | // LevelProgress – XP progress bar with level info
3 | // =====
4 |
5 | import { motion, useReducedMotion } from "framer-motion";
6 | import { TrendingUp } from "lucide-react";
7 | import type { LevelData } from "@hooks/useLevel";
8 |
9 | interface LevelProgressProps {
10 |   levelData: LevelData;
11 |   compact?: boolean;
12 | }
13 |

```

```

14 | const LevelProgress = ({ levelData, compact = false }: LevelProgressProps) => {
15 |   const prefersReducedMotion = useReducedMotion();
16 |   const {
17 |     level,
18 |     title,
19 |     xpTotal,
20 |     xpForCurrentLevel,
21 |     xpForNextLevel,
22 |     progressPercent,
23 |   } = levelData;
24 |   const isMaxLevel = xpForNextLevel <= xpForCurrentLevel;
25 |
26 |   if (compact) {
27 |     return (
28 |       <div className="flex items-center gap-2">
29 |         <div className="flex items-center justify-center h-8 w-8 rounded-lg bg-blue-50">
30 |           <TrendingUp className="h-4 w-4 text-blue-600" />
31 |         </div>
32 |         <div className="flex-1 min-w-0">
33 |           <div className="flex items-center justify-between">
34 |             <span className="text-xs font-bold text-gray-700">Lv {level}</span>
35 |             <span className="text-xs text-gray-500">{title}</span>
36 |           </div>
37 |           <div className="mt-1 h-1.5 w-full rounded-full bg-slate-100 overflow-hidden">
38 |             <motion.div
39 |               className="h-full rounded-full bg-gradient-to-r from-blue-500 to-blue-600"
40 |               initial={
41 |                 prefersReducedMotion
42 |                   ? { width: `${progressPercent}%` }
43 |                   : { width: 0 }
44 |               }
45 |               animate={{ width: `${progressPercent}%` }}
46 |               transition={{ duration: 0.6, ease: "easeOut" }}
47 |             />
48 |           </div>
49 |         </div>
50 |       </div>
51 |     );
52 |   }
53 |
54 |   return (
55 |     <div className="space-y-2">
56 |       <div className="flex items-center justify-between">
57 |         <div className="flex items-center gap-2">
58 |           <div className="flex items-center justify-center h-10 w-10 rounded-xl bg-blue-50">
59 |             <TrendingUp className="h-5 w-5 text-blue-600" />
60 |           </div>
61 |           <div>
62 |             <p className="text-sm font-bold text-gray-900">Level {level}</p>
63 |             <p className="text-xs text-gray-500">{title}</p>
64 |           </div>
65 |         </div>
66 |         <div className="text-end">
67 |           <p className="text-sm font-bold text-amber-600">
68 |             {xpTotal.toLocaleString()} XP
69 |           </p>
70 |           {!isMaxLevel && (
71 |             <p className="text-xs text-gray-400">
72 |               {(xpForNextLevel - xpTotal).toLocaleString()} to next
73 |             </p>
74 |           )}
75 |         </div>
76 |       </div>
77 |
78 |       <div className="h-2.5 w-full rounded-full bg-slate-100 overflow-hidden">
79 |         <motion.div
80 |           className="h-full rounded-full bg-gradient-to-r from-blue-500 to-blue-600"
81 |           initial={
82 |             prefersReducedMotion
83 |               ? { width: `${progressPercent}%` }
84 |               : { width: 0 }
85 |           }
86 |           animate={{ width: `${progressPercent}%` }}
87 |           transition={{ duration: 0.8, ease: "easeOut" }}
88 |           role="progressbar"
89 |           aria-valuenow={progressPercent}
90 |           aria-valuemin={0}
91 |           aria-valuemax={100}
92 |           aria-label={`Level ${level} progress: ${progressPercent}%`}
93 |         />
94 |       </div>
95 |
96 |       {!isMaxLevel && (
97 |         <div className="flex justify-between text-[10px] font-black tracking-widest uppercase text-gray-400">
98 |           <span>
99 |             Lv {level} – {xpForCurrentLevel.toLocaleString()}
100 |           </span>
101 |           <span>
102 |             Lv {level + 1} – {xpForNextLevel.toLocaleString()}
103 |           </span>
104 |         </div>
105 |       )}
106 |     </div>
107 |   );
108 | };
109 |
110 | export default LevelProgress;
111 |

```

FILE: src/components/shared/LevelProgressionChart.tsx

TYPE: .tsx | LINES: 1-141 | SHA256: 5fdfaf905c6f418d64bef0714775da7971f009539625d22a5f65821fa9de4783

```
1 | import {
2 |   LineChart,
3 |   Line,
4 |   XAxis,
5 |   YAxis,
6 |   Tooltip,
7 |   ResponsiveContainer,
8 |   ReferenceDot,
9 | } from "recharts";
10 | import type { LevelProgressionPoint } from "@types/habits";
11 |
12 | export interface LevelProgressionChartProps {
13 |   data: LevelProgressionPoint[];
14 |   currentLevel: 1 | 2 | 3 | 4;
15 | }
16 |
17 | /**
18 |  * Detects level-up points where the level increased from the previous entry.
19 |  */
20 | const getLevelUpMarkers = (
21 |   data: LevelProgressionPoint[]
22 | ): LevelProgressionPoint[] => {
23 |   const markers: LevelProgressionPoint[] = [];
24 |   for (let i = 1; i < data.length; i++) {
25 |     const current = data[i];
26 |     const previous = data[i - 1];
27 |     if (current && previous && current.level > previous.level) {
28 |       markers.push(current);
29 |     }
30 |   }
31 |   return markers;
32 | };
33 |
34 | const LevelProgressionChart = ({
35 |   data,
36 |   currentLevel,
37 | }: LevelProgressionChartProps) => {
38 |   const firstPoint = data[0];
39 |   const isSingleLevel =
40 |     data.length <= 1 || data.every((p) => p.level === firstPoint?.level);
41 |
42 |   if (isSingleLevel) {
43 |     const level = firstPoint ? firstPoint.level : currentLevel;
44 |     return (
45 |       <div
46 |         className="flex flex-col items-center justify-center py-8 gap-2"
47 |         data-testid="level-progression-single"
48 |       >
49 |         <div className="flex items-center gap-2">
50 |           <span className="text-3xl font-black text-blue-600">{level}</span>
51 |           <span className="text-sm text-gray-500">/ 4</span>
52 |         </div>
53 |         <p className="text-sm text-gray-600 text-center">
54 |           You've been consistent at Level {level} this semester
55 |         </p>
56 |       </div>
57 |     );
58 |   }
59 |
60 |   const sorted = [...data].sort((a, b) => a.date.localeCompare(b.date));
61 |   const levelUpMarkers = getLevelUpMarkers(sorted);
62 |
63 |   return (
64 |     <div data-testid="level-progression-chart">
65 |       <ResponsiveContainer width="100%" height={220}>
66 |         <LineChart
67 |           data={sorted}
68 |           margin={{ top: 12, right: 12, bottom: 4, left: -20 }}
69 |         >
70 |           <XAxis
71 |             dataKey="date"
72 |             tick={{ fontSize: 11 }}
73 |             tickLine={false}
74 |             axisLine={false}
75 |             tickFormatter={(v: string) => {
76 |               const d = new Date(v + "T00:00:00");
77 |               return d.toLocaleDateString("en-US", {
78 |                 month: "short",
79 |                 day: "numeric",
80 |               });
81 |             }}
82 |           />
83 |           <YAxis
84 |             domain={[1, 4]}
85 |             ticks={[1, 2, 3, 4]}
86 |             tick={{ fontSize: 11 }}
87 |             tickLine={false}
88 |             axisLine={false}
89 |             tickFormatter={(v: number) => `Lv ${v}`}
90 |           />
91 |           <Tooltip
92 |             formatter={
93 |               ((value: number | undefined) => [
94 |                 `Level ${value ?? 0}`,
95 |                 "Level",
96 |               ]) as never
97 |             }
98 |             labelFormatter={(label: unknown) => {
99 |               const d = new Date(String(label) + "T00:00:00");
100 |               return d.toLocaleDateString("en-US", {
101 |                 month: "long",
```

```

102 |         day: "numeric",
103 |         year: "numeric",
104 |     });
105 |     }}
106 |     contentStyle={{ borderRadius: 8, fontSize: 12 }}
107 | }>
108 | <Line
109 |   type="stepAfter"
110 |   dataKey="level"
111 |   stroke="url(#levelGradient)"
112 |   strokeWidth={3}
113 |   dot={false}
114 |   activeDot={{ r: 5, fill: "#2563eb" }}
115 | />
116 | {levelUpMarkers.map((marker) => (
117 |   <ReferenceDot
118 |     key={marker.date}
119 |     x={marker.date}
120 |     y={marker.level}
121 |     r={6}
122 |     fill="#14b8a6"
123 |     stroke="#fff"
124 |     strokeWidth={2}
125 |     data-testid={`level-up-marker-${marker.date}`}
126 |   />
127 | ))}
128 | <defs>
129 |   <linearGradient id="levelGradient" x1="0" y1="0" x2="1" y2="0">
130 |     <stop offset="0%" stopColor="#14b8a6" />
131 |     <stop offset="100%" stopColor="#0382bd" />
132 |   </linearGradient>
133 | </defs>
134 | </LineChart>
135 | </ResponsiveContainer>
136 | </div>
137 | );
138 | };
139 |
140 | export default LevelProgressionChart;
141 |

```

FILE: src/components/shared/LevelUpOverlay.tsx

TYPE: .tsx | LINES: 1-157 | SHA256: d4ced6794df2d7d5b3ad9608ed3502a5ed909d0930511d43024aaaab587ee366

```

1 | // =====
2 | // LevelUpOverlay - Full-screen level-up celebration animation
3 | // =====
4 |
5 | import { useEffect, useRef, useState, useCallback } from "react";
6 | import { motion, AnimatePresence, useReducedMotion } from "framer-motion";
7 | import { launchConfetti } from "@lib/confetti";
8 | import { TrendingUp, Star } from "lucide-react";
9 | import { LEVEL_THRESHOLDS } from "@lib/xpLevelCalculator";
10 | import { useTranslation } from "react-i18next";
11 |
12 | interface LevelUpOverlayProps {
13 |   newLevel: number;
14 |   onComplete?: () => void;
15 | }
16 |
17 | const LevelUpOverlay = ({ newLevel, onComplete }: LevelUpOverlayProps) => {
18 |   const { t } = useTranslation("gamification");
19 |   const [visible, setVisible] = useState(true);
20 |   const prefersReducedMotion = useReducedMotion();
21 |   const confettiFired = useRef(false);
22 |
23 |   const title =
24 |     LEVEL_THRESHOLDS.find((t) => t.level === newLevel)?.title ?? "Unknown";
25 |
26 |   const fireConfetti = useCallback(() => {
27 |     if (confettiFired.current || prefersReducedMotion) return;
28 |     confettiFired.current = true;
29 |
30 |     void launchConfetti({
31 |       particleCount: 80,
32 |       spread: 100,
33 |       origin: { x: 0.5, y: 0.5 },
34 |       colors: ["#3b82f6", "#14b8a6", "#f59e0b", "#eab308"],
35 |     });
36 |
37 |     setTimeout(() => {
38 |       void launchConfetti({
39 |         particleCount: 40,
40 |         angle: 60,
41 |         spread: 55,
42 |         origin: { x: 0, y: 0.6 },
43 |         colors: ["#3b82f6", "#14b8a6"],
44 |       });
45 |       void launchConfetti({
46 |         particleCount: 40,
47 |         angle: 120,
48 |         spread: 55,
49 |         origin: { x: 1, y: 0.6 },
50 |         colors: ["#f59e0b", "#eab308"],
51 |       });
52 |     }, 200);
53 |   }, [prefersReducedMotion]);
54 |
55 |   useEffect(() => {
56 |     fireConfetti();
57 |   }, [fireConfetti]);
58 |
59 |   useEffect(() => {
60 |     const timer = setTimeout(() => setVisible(false), 4000);

```

```

61 |     return () => clearTimeout(timer);
62 | }, []);
63 |
64 | return (
65 |   <AnimatePresence onExitComplete={() => onComplete?.()}>
66 |     {visible && (
67 |       <motion.div
68 |         className="fixed inset-0 z-50 flex items-center justify-center"
69 |         initial={prefersReducedMotion ? undefined : { opacity: 0 }}
70 |         animate={{ opacity: 1 }}
71 |         exit={{ opacity: 0 }}
72 |         transition={{ duration: 0.3 }}
73 |         onClick={() => setVisible(false)}
74 |         role="dialog"
75 |         aria-modal="true"
76 |         aria-label={t("level.overlayLabel", {
77 |           level: newLevel,
78 |           defaultValue: "Level up to level {{level}}",
79 |         })}
80 |       >
81 |         <div className="absolute inset-0 bg-black/60 backdrop-blur-sm" />
82 |
83 |         <motion.div
84 |           className="relative flex flex-col items-center gap-4 p-8"
85 |           initial={prefersReducedMotion ? undefined : { scale: 0.5, y: 30 }}
86 |           animate={{ scale: 1, y: 0 }}
87 |           exit={
88 |             prefersReducedMotion ? { opacity: 0 } : { scale: 0.8, opacity: 0 }
89 |           }
90 |           transition={{ type: "spring", damping: 12, stiffness: 200 }}
91 |         >
92 |           <motion.div
93 |             className="absolute inset-0 rounded-full bg-blue-500/20 blur-3xl"
94 |             initial={prefersReducedMotion ? undefined : { scale: 0 }}
95 |             animate={{ scale: 2.5 }}
96 |             transition={{ duration: 1.5, ease: "easeOut" }}
97 |           />
98 |
99 |           <motion.div
100 |            className="relative flex items-center justify-center h-28 w-28 rounded-full bg-gradient-to-br fr
101 |            initial={
102 |              prefersReducedMotion ? undefined : { rotate: -180, scale: 0 }
103 |            }
104 |            animate={{ rotate: 0, scale: 1 }}
105 |            transition={{
106 |              type: "spring",
107 |              damping: 10,
108 |              stiffness: 150,
109 |              delay: 0.2,
110 |            }}
111 |          >
112 |            <div className="flex flex-col items-center">
113 |              <TrendingUp className="h-8 w-8 text-white mb-1" />
114 |              <span className="text-3xl font-black text-white">
115 |                {newLevel}
116 |              </span>
117 |            </div>
118 |          </motion.div>
119 |
120 |          <motion.div
121 |            className="relative text-center"
122 |            initial={prefersReducedMotion ? undefined : { opacity: 0, y: 20 }}
123 |            animate={{ opacity: 1, y: 0 }}
124 |            transition={{ delay: 0.5, duration: 0.4 }}
125 |          >
126 |            <div className="flex items-center gap-2 justify-center mb-2">
127 |              <Star className="h-5 w-5 text-amber-400" />
128 |              <p className="text-[10px] font-black tracking-widest uppercase text-amber-400">
129 |                {t("level.levelUpLabel", "Level Up")}
130 |              </p>
131 |              <Star className="h-5 w-5 text-amber-400" />
132 |            </div>
133 |            <h2 className="text-3xl font-black text-white">
134 |              {t("level.current", { level: newLevel })}
135 |            </h2>
136 |            <p className="text-lg font-semibold text-white/70 mt-1">
137 |              {title}
138 |            </p>
139 |          </motion.div>
140 |
141 |          <motion.p
142 |            className="relative text-xs text-white/40 mt-4"
143 |            initial={prefersReducedMotion ? undefined : { opacity: 0 }}
144 |            animate={{ opacity: 1 }}
145 |            transition={{ delay: 1.5 }}
146 |          >
147 |            {t("level.dismiss", "Tap anywhere to dismiss")}
148 |          </motion.p>
149 |        </motion.div>
150 |      </motion.div>
151 |    )}
152 |  </AnimatePresence>
153 | );
154 | };
155 |
156 | export default LevelUpOverlay;
157 |

```

FILE: src/components/shared/LikertScale.tsx

TYPE: .tsx | LINES: 1-105 | SHA256: f9f697142eb86c62bd46c08604fa6c2f35439dbff56e89769ecb7ef3254f1151

```
1 | import { useCallback, useRef } from "react";
2 | import { cn } from "@lib/utlis";
3 |
4 | const LIKERT_LABELS = [
5 |   "Strongly Disagree",
6 |   "Disagree",
7 |   "Neutral",
8 |   "Agree",
9 |   "Strongly Agree",
10 | ] as const;
11 |
12 | export interface LikertScaleProps {
13 |   value: number | null;
14 |   onChange: (value: number) => void;
15 |   questionId: string;
16 |   disabled?: boolean;
17 |   labels?: readonly string[];
18 | }
19 |
20 | const LikertScale = ({
21 |   value,
22 |   onChange,
23 |   questionId,
24 |   disabled = false,
25 |   labels = LIKERT_LABELS,
26 | }: LikertScaleProps) => {
27 |   const groupRef = useRef<HTMLDivElement>(null);
28 |   const groupId = `likert-${questionId}`;
29 |
30 |   const handleKeyDown = useCallback(
31 |     (e: React.KeyboardEvent) => {
32 |       if (disabled) return;
33 |
34 |       const current = value ?? 0;
35 |       let next: number | null = null;
36 |       const max = labels.length;
37 |       const min = 1;
38 |
39 |       if (e.key === "ArrowRight" || e.key === "ArrowDown") {
40 |         e.preventDefault();
41 |         next = current < max ? current + 1 : max;
42 |         if (next < min) next = min;
43 |       } else if (e.key === "ArrowLeft" || e.key === "ArrowUp") {
44 |         e.preventDefault();
45 |         next = current > min ? current - 1 : min;
46 |       }
47 |
48 |       if (next !== null) {
49 |         onChange(next);
50 |         const buttons =
51 |           groupRef.current?.querySelectorAll<HTMLButtonElement>(
52 |             '[role="radio"]'
53 |           );
54 |         buttons?.[next - 1]?.focus();
55 |       }
56 |     },
57 |     [value, onChange, disabled, labels]
58 |   );
59 |
60 |   return (
61 |     <div
62 |       ref={groupRef}
63 |       id={groupId}
64 |       role="radiogroup"
65 |       aria-label="Likert scale rating"
66 |       className="flex items-center gap-2"
67 |       onKeyDown={handleKeyDown}
68 |       tabIndex={-1}
69 |     >
70 |       {labels.map((label, index) => {
71 |         const optionValue = index + 1;
72 |         const isSelected = value === optionValue;
73 |
74 |         return (
75 |           <button
76 |             key={optionValue}
77 |             type="button"
78 |             role="radio"
79 |             aria-checked={isSelected}
80 |             aria-label={label}
81 |             tabIndex={isSelected || (value === null && index === 0) ? 0 : -1}
82 |             disabled={disabled}
83 |             onClick={() => !disabled && onChange(optionValue)}
84 |             className={cn(
85 |               "flex flex-col items-center gap-1.5 rounded-lg px-3 py-2 transition-colors min-w-[72px]",
86 |               "focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-blue-500 focus-visible:ring-
87 |               isSelected
88 |               ? "bg-blue-600 text-white shadow-md"
89 |               : "bg-slate-50 text-gray-600 hover:bg-slate-100",
90 |               disabled && "opacity-50 cursor-not-allowed"
91 |             )}
92 |           >
93 |             <span className="text-sm font-bold">{optionValue}</span>
94 |             <span className="text-[10px] font-medium leading-tight text-center">
95 |               {label}
96 |             </span>
97 |           </button>
98 |         );
99 |       })}
100 |     </div>
101 |   );
```

```

102 | };
103 |
104 | export default LikertScale;
105 |

```

FILE: src/components/shared/LockedNode.tsx

TYPE: .tsx | LINES: 1-50 | SHA256: fab0c1258d7add0812c1bb5d78d18214b96c42112ce2e13e8318ba32d82eff6a

```

1 | // =====
2 | // LockedNode - Locked/prerequisite-gated assignment node for learning path
3 | // =====
4 |
5 | import { Lock } from "lucide-react";
6 | import { cn } from "@lib/Utils";
7 |
8 | interface LockedNodeProps {
9 |   title: string;
10 |   prerequisiteCLO?: string;
11 |   requiredAttainment?: number;
12 |   currentAttainment?: number;
13 |   className?: string;
14 | }
15 |
16 | const LockedNode = ({
17 |   title,
18 |   prerequisiteCLO,
19 |   requiredAttainment,
20 |   currentAttainment,
21 |   className,
22 | }: LockedNodeProps) => (
23 |   <div
24 |     className={cn(
25 |       "relative flex items-center gap-3 rounded-xl border border-dashed border-gray-300 bg-gray-50 p-4 opacity
26 |       className
27 |     )}
28 |   >
29 |     <div className="flex h-10 w-10 items-center justify-center rounded-full bg-gray-200">
30 |       <Lock className="h-4 w-4 text-gray-500" />
31 |     </div>
32 |     <div className="min-w-0 flex-1">
33 |       <p className="text-sm font-medium text-gray-500 truncate">{title}</p>
34 |       {prerequisiteCLO && (
35 |         <p className="text-xs text-gray-400 mt-0.5">
36 |           Requires {requiredAttainment ?? 70}% on {prerequisiteCLO}
37 |           {currentAttainment !== null && (
38 |             <span className="ms-1">
39 |               (current: {Math.round(currentAttainment)}%)
40 |             </span>
41 |           )}
42 |         </p>
43 |       )}
44 |     </div>
45 |   </div>
46 | );
47 |
48 | export default LockedNode;
49 | export type { LockedNodeProps };
50 |

```

FILE: src/components/shared/Mascot.tsx

TYPE: .tsx | LINES: 1-72 | SHA256: 5947d4faf508cc799174aa8430130a552c20ef48949f41233a6227f5060310fe

```

1 | // =====
2 | // Mascot - coaching companion shown at key journey moments (R35)
3 | // =====
4 | //
5 | // A purely additive, non-blocking coaching surface. Given a key moment, it
6 | // resolves bilingual guidance via `useMascotGuidance` and renders an
7 | // encouraging title + message in the student's language. When mascot guidance
8 | // is disabled, or no key moment is active, it renders nothing so the host
9 | // surface is unaffected (R35.4, R35.5).
10 | //
11 | // Copy is routed through i18next (`common` namespace, `mascot.`) so it is
12 | // always available in English and Arabic (R35.3). The entrance animation uses
13 | // the `animate-fade-in-up` utility, which the design system disables globally
14 | // under `prefers-reduced-motion: reduce` (see src/index.css), so motion is
15 | // honored without coupling this additive surface to a motion library.
16 | //
17 | import { useTranslation } from "react-i18next";
18 | import { PartyPopper, Sparkles, Lightbulb } from "lucide-react";
19 | import type { LucideIcon } from "lucide-react";
20 | import { cn } from "@lib/Utils";
21 | import { useMascotGuidance } from "@hooks/useMascotGuidance";
22 | import type { MascotMomentId, MascotTone } from "@lib/mascotGuidance";
23 |
24 | export interface MascotProps {
25 |   /** The active key moment, or `null` when no moment is active. */
26 |   moment: MascotMomentId | null;
27 |   /** Optional extra classes for layout placement by the host surface. */
28 |   className?: string;
29 | }
30 |
31 | // Tone !' icon mapping keeps the visual concern in the presentation layer while
32 | // the pure logic only carries the abstract tone.
33 | const TONE_ICON: Record<MascotTone, LucideIcon> = {
34 |   cheer: PartyPopper,
35 |   guide: Lightbulb,
36 |   encourage: Sparkles,
37 | };
38 |
39 | const Mascot = ({ moment, className }: MascotProps) => {
40 |   const { t } = useTranslation("common");
41 |   const guidance = useMascotGuidance(moment);
42 |

```



```

71 |         ai_tutor_completed: aiTutorCompleted,
72 |         practice_completed: practiceCompleted,
73 |     });
74 |
75 | const steps: RecoveryStep[] = [
76 |     {
77 |         number: 1,
78 |         title: t("recoveryPath.steps.tutor.title", "AI Tutor Session"),
79 |         description: t("recoveryPath.steps.tutor.description", {
80 |             defaultValue:
81 |                 'Review foundational concepts for "{{title}}" with the AI Tutor.',
82 |             title: cloTitle,
83 |         }),
84 |         icon: BrainCircuit,
85 |         completed: aiTutorCompleted,
86 |         required: true,
87 |     },
88 |     {
89 |         number: 2,
90 |         title: t("recoveryPath.steps.practice.title", "Practice Questions"),
91 |         description: t(
92 |             "recoveryPath.steps.practice.description",
93 |             "Complete practice questions at a lower difficulty level to rebuild understanding."
94 |         ),
95 |         icon: BookOpen,
96 |         completed: practiceCompleted,
97 |         required: true,
98 |     },
99 |     {
100 |         number: 3,
101 |         title: t("recoveryPath.steps.peer.title", "Peer Study Group"),
102 |         description: peerApplicable
103 |             ? t(
104 |                 "recoveryPath.steps.peer.description",
105 |                 "Join a peer study group working on the same topic."
106 |             )
107 |             : t(
108 |                 "recoveryPath.steps.peer.unavailable",
109 |                 "No active study groups are available for this topic."
110 |             ),
111 |         icon: Users,
112 |         completed: peerShown || !peerApplicable,
113 |         required: false,
114 |     },
115 | ];
116 |
117 | const handleCompleteAITutor = () => {
118 |     completeStep.mutate(
119 |         { recovery_id: recoveryId, step: "ai_tutor" },
120 |         {
121 |             onSuccess: () =>
122 |                 toast.success(
123 |                     t(
124 |                         "recoveryPath.toast.tutorComplete",
125 |                         "AI Tutor session marked as complete"
126 |                     )
127 |                 ),
128 |             onError: (err) =>
129 |                 toast.error(
130 |                     err instanceof Error
131 |                         ? err.message
132 |                         : t("recoveryPath.toast.updateFailed", "Failed to update step")
133 |                 ),
134 |         }
135 |     );
136 | };
137 |
138 | const handleCompletePractice = () => {
139 |     completeStep.mutate(
140 |         { recovery_id: recoveryId, step: "practice" },
141 |         {
142 |             onSuccess: () =>
143 |                 toast.success(
144 |                     t(
145 |                         "recoveryPath.toast.practiceComplete",
146 |                         "Practice questions marked as complete"
147 |                     )
148 |                 ),
149 |             onError: (err) =>
150 |                 toast.error(
151 |                     err instanceof Error
152 |                         ? err.message
153 |                         : t("recoveryPath.toast.updateFailed", "Failed to update step")
154 |                 ),
155 |         }
156 |     );
157 | };
158 |
159 | const handleRetry = () => {
160 |     onRetryUnlocked?();
161 | };
162 |
163 | if (isLoading) {
164 |     return (
165 |         <Card
166 |             className={cn("bg-white border-0 shadow-md rounded-xl p-6", className)}
167 |         >
168 |             <div className="flex items-center justify-center py-8">
169 |                 <Loader2 className="h-6 w-6 animate-spin text-gray-400" />
170 |             </div>
171 |         </Card>
172 |     );
173 | }
174 |

```

[illegible]

```

279 | onCompletePractice: () => void;
280 | peerApplicable: boolean;
281 | }) => {
282 |   const { t } = useTranslation("student");
283 |   const Icon = step.icon;
284 |
285 |   return (
286 |     <div
287 |       className={cn(
288 |         "flex items-start gap-4 rounded-lg border p-4 transition-colors",
289 |         step.completed
290 |           ? "border-green-200 bg-green-50"
291 |           : "border-slate-200 bg-slate-50"
292 |       )}
293 |     >
294 |       { /* Step number / check */ }
295 |       <div
296 |         className={cn(
297 |           "flex-shrink-0 flex items-center justify-center w-8 h-8 rounded-full text-sm font-bold",
298 |           step.completed
299 |             ? "bg-green-500 text-white"
300 |             : "bg-slate-200 text-gray-600"
301 |         )}
302 |         aria-hidden="true"
303 |       >
304 |         {step.completed ? <CheckCircle2 className="h-5 w-5" /> : step.number}
305 |       </div>
306 |
307 |       { /* Content */ }
308 |       <div className="flex-1 min-w-0">
309 |         <div className="flex items-center gap-2">
310 |           <Icon className="h-4 w-4 text-gray-500" />
311 |           <h3 className="text-sm font-semibold text-gray-900">{step.title}</h3>
312 |           { !step.required && (
313 |             <Badge
314 |               variant="outline"
315 |               className="text-xs text-gray-500 border-gray-300"
316 |             >
317 |               {t("recoveryPath.optional", "Optional")}
318 |             </Badge>
319 |           )}
320 |         </div>
321 |         <p className="text-xs text-gray-500 mt-1">{step.description}</p>
322 |
323 |         { /* Action buttons per step */ }
324 |         { !step.completed && (
325 |           <div className="mt-3">
326 |             {step.number === 1 && (
327 |               <div className="flex items-center gap-2">
328 |                 <Button size="sm" variant="outline" asChild className="text-xs">
329 |                   <a
330 |                     href={` /student/tutor?cloIds=${cloId} `}
331 |                     target="_blank"
332 |                     rel="noopener noreferrer"
333 |                   >
334 |                     <BrainCircuit className="h-3.5 w-3.5" />
335 |                     {t("recoveryPath.openTutor", "Open AI Tutor")}
336 |                     <ExternalLink className="h-3 w-3" />
337 |                   </a>
338 |                 </Button>
339 |                 <Button
340 |                   size="sm"
341 |                   onClick={onCompleteAITutor}
342 |                   disabled={isCompletingStep}
343 |                   className="text-xs bg-green-600 hover:bg-green-700 text-white"
344 |                 >
345 |                   {isCompletingStep ? (
346 |                     <Loader2 className="h-3.5 w-3.5 animate-spin" />
347 |                   ) : (
348 |                     <CheckCircle2 className="h-3.5 w-3.5" />
349 |                   )}
350 |                   {t("recoveryPath.markComplete", "Mark Complete")}
351 |                 </Button>
352 |               </div>
353 |             )}
354 |
355 |             {step.number === 2 && (
356 |               <div className="flex items-center gap-2">
357 |                 <Button size="sm" variant="outline" asChild className="text-xs">
358 |                   <a
359 |                     href={` /student/courses/${courseId}/practice?clo=${cloId} `}
360 |                   >
361 |                     <BookOpen className="h-3.5 w-3.5" />
362 |                     {t("recoveryPath.startPractice", "Start Practice")}
363 |                   </a>
364 |                 </Button>
365 |                 <Button
366 |                   size="sm"
367 |                   onClick={onCompletePractice}
368 |                   disabled={isCompletingStep}
369 |                   className="text-xs bg-green-600 hover:bg-green-700 text-white"
370 |                 >
371 |                   {isCompletingStep ? (
372 |                     <Loader2 className="h-3.5 w-3.5 animate-spin" />
373 |                   ) : (
374 |                     <CheckCircle2 className="h-3.5 w-3.5" />
375 |                   )}
376 |                   {t("recoveryPath.markComplete", "Mark Complete")}
377 |                 </Button>
378 |               </div>
379 |             )}
380 |
381 |             {step.number === 3 && peerApplicable && (
382 |               <Button size="sm" variant="outline" asChild className="text-xs">

```

```

383 |         <a
384 |             href={` /student/courses/${courseId}/team-challenges?clo=${cloId}`}
385 |         >
386 |             <Users className="h-3.5 w-3.5" />
387 |             {t("recoveryPath.browseGroups", "Browse Study Groups")}
388 |         </a>
389 |         </Button>
390 |     )}
391 | </div>
392 | })
393 | </div>
394 | </div>
395 | );
396 | };
397 |
398 | export default MasteryRecoveryPanel;
399 |

```

FILE: src/components/shared/MasteryRing.ts

TYPE: .ts | LINES: 1-5 | SHA256: dc5b9655b60f56afa2f6cb1d3a68e88a23bb76937cda4a3ed3d2345ab28bee83

```

1 | // Re-export shim – implementation lives in the design system (single source).
2 | // Legacy direct importers keep working; deleted at P5 once none remain.
3 | export { default } from "@design-system/patterns/MasteryRing";
4 | export type { MasteryRingProps } from "@design-system/patterns/MasteryRing";
5 |

```

FILE: src/components/shared/MasteryRing.tsx

TYPE: .tsx | LINES: 1-135 | SHA256: 806da22ac9fde494fa1f23173b5e05c61792b6d8c97af6200bdd9bf62c1b6cf9

```

1 | // =====
2 | // MasteryRing – general-purpose SVG progress ring
3 | // =====
4 | //
5 | // Presentation-only primitive introduced by the UI prototype migration
6 | // (spec: .kiro/specs/ui-prototype-migration, design $3.2). Renders a circular
7 | // progress ring for mastery / attainment / completion percentages.
8 | //
9 | // `tone="auto"` colors the ring by the OBE attainment thresholds (Excellent
10 | // "e85 green, Satisfactory "e70 brand-blue, Developing "e50 amber, Not Yet <50
11 | // red – see domain knowledge). `tone="brand"` uses the teal!'blue brand
12 | // gradient. Dark-mode aware (track + center label) and reduced-motion safe.
13 | //
14 | // NOTE: `ConsistencyScoreRing` predates this primitive and implements the same
15 | // SVG mechanics locked to the brand gradient + an external label. It can be
16 | // refactored to consume MasteryRing in a later cleanup; it is intentionally
17 | // left untouched here to avoid changing a working component during P0.
18 | // =====
19 |
20 | import * as React from "react";
21 |
22 | import { cn } from "@lib/utls";
23 |
24 | export interface MasteryRingProps {
25 |     /** Progress value, 0-100 (clamped and rounded). */
26 |     value: number;
27 |     /** Outer diameter in px. */
28 |     size?: number;
29 |     /** Ring stroke width in px. */
30 |     strokeWidth?: number;
31 |     /**
32 |      * "auto" (default) colors the ring by attainment threshold; "brand" uses the
33 |      * teal!'blue brand gradient regardless of value.
34 |      */
35 |     tone?: "auto" | "brand";
36 |     /** Show the numeric percentage in the center. Ignored when `children` is set. */
37 |     showValue?: boolean;
38 |     /** Accessible label; defaults to "<value>%". */
39 |     label?: string;
40 |     /** Optional custom center content (overrides `showValue`). */
41 |     children?: React.ReactNode;
42 |     className?: string;
43 | }
44 |
45 | /** Maps a value to its attainment-level stroke color (domain thresholds). */
46 | const attainmentStroke = (v: number): string => {
47 |     if (v >= 85) return "#22c55e"; // success – Excellent
48 |     if (v >= 70) return "#3b82f6"; // brand blue – Satisfactory
49 |     if (v >= 50) return "#f59e0b"; // warning – Developing
50 |     return "#ef4444"; // destructive – Not Yet
51 | };
52 |
53 | const MasteryRing = ({
54 |     value,
55 |     size = 72,
56 |     strokeWidth = 8,
57 |     tone = "auto",
58 |     showValue = true,
59 |     label,
60 |     children,
61 |     className,
62 | }: MasteryRingProps) => {
63 |     const clamped = Math.max(0, Math.min(100, Math.round(value)));
64 |     const radius = (size - strokeWidth) / 2;
65 |     const circumference = 2 * Math.PI * radius;
66 |     const offset = circumference - (clamped / 100) * circumference;
67 |     const gradientId = React.useId();
68 |     const stroke =
69 |         tone === "brand" ? `url(#${gradientId})` : attainmentStroke(clamped);
70 |
71 |     return (
72 |         <div
73 |             className={cn(
74 |                 "relative inline-flex items-center justify-center",

```

```

75 |         className
76 |       )}
77 |       style={{ width: size, height: size }}
78 |     >
79 |       <svg
80 |         width={size}
81 |         height={size}
82 |         viewBox={`0 0 ${size} ${size}`}
83 |         role="img"
84 |         aria-label={label ?? `${clamped}%`}
85 |       >
86 |         {tone === "brand" && (
87 |           <defs>
88 |             <linearGradient id={gradientId} x1="0%" y1="0%" x2="100%" y2="0%">
89 |               <stop offset="0%" stopColor="#14b8a6" />
90 |               <stop offset="100%" stopColor="#0382bd" />
91 |             </linearGradient>
92 |           </defs>
93 |         )}
94 |         {/* Track */}
95 |         <circle
96 |           cx={size / 2}
97 |           cy={size / 2}
98 |           r={radius}
99 |           fill="none"
100 |           strokeWidth={strokeWidth}
101 |           className="stroke-slate-200 dark:stroke-slate-700"
102 |         />
103 |         {/* Progress */}
104 |         <circle
105 |           cx={size / 2}
106 |           cy={size / 2}
107 |           r={radius}
108 |           fill="none"
109 |           stroke={stroke}
110 |           strokeWidth={strokeWidth}
111 |           strokeLinecap="round"
112 |           strokeDasharray={circumference}
113 |           strokeDashoffset={offset}
114 |           transform={`rotate(-90 ${size / 2} ${size / 2})`}
115 |           className="transition-[stroke-dashoffset] duration-500 motion-reduce:transition-none"
116 |         />
117 |       </svg>
118 |       {children ? (
119 |         <div className="absolute inset-0 flex items-center justify-center">
120 |           {children}
121 |         </div>
122 |       ) : showValue ? (
123 |         <span
124 |           className="absolute inset-0 flex items-center justify-center text-sm font-bold tabular-nums text-for"
125 |           aria-hidden="true"
126 |         >
127 |           {clamped}%
128 |         </span>
129 |       ) : null}
130 |     </div>
131 |   );
132 | };
133 |
134 | export default MasteryRing;
135 |

```

FILE: src/components/shared/MaterialItem.tsx

TYPE: .tsx | LINES: 1-70 | SHA256: fdbe804b3c7d621288e7e798b315198d5a25f2c40fa09697a85fdcf6d3c41ecd

```

1 | // =====
2 | // MaterialItem - Course material list item
3 | // =====
4 |
5 | import { FileText, Link as LinkIcon, Video, File } from "lucide-react";
6 | import { cn } from "@lib/utils";
7 | import type { LucideIcon } from "lucide-react";
8 |
9 | type MaterialType = "file" | "link" | "video" | "text";
10 |
11 | interface MaterialItemProps {
12 |   title: string;
13 |   type: MaterialType;
14 |   url?: string;
15 |   onClick?: () => void;
16 |   className?: string;
17 | }
18 |
19 | const TYPE_ICONS: Record<MaterialType, LucideIcon> = {
20 |   file: FileText,
21 |   link: LinkIcon,
22 |   video: Video,
23 |   text: File,
24 | };
25 |
26 | const TYPE_COLORS: Record<MaterialType, string> = {
27 |   file: "bg-blue-50 text-blue-600",
28 |   link: "bg-green-50 text-green-600",
29 |   video: "bg-purple-50 text-purple-600",
30 |   text: "bg-gray-50 text-gray-600",
31 | };
32 |
33 | const MaterialItem = ({
34 |   title,
35 |   type,
36 |   url,
37 |   onClick,
38 |   className,
39 | }: MaterialItemProps) => {

```

```

40 | const Icon = TYPE_ICONS[type];
41 | const color = TYPE_COLORS[type];
42 |
43 | return (
44 |   <div
45 |     className={cn(
46 |       "flex items-center gap-3 rounded-lg px-3 py-2.5 hover:bg-slate-50 transition-colors cursor-pointer",
47 |       className
48 |     )}
49 |     onClick={onClick}
50 |     role="button"
51 |     tabIndex={0}
52 |     onKeyDown={(e) => {
53 |       if (e.key === "Enter" || e.key === " ") {
54 |         e.preventDefault();
55 |         onClick?.();
56 |       }
57 |     }}
58 |   >
59 |     <div className={cn("p-1.5 rounded-lg", color)}>
60 |       <Icon className="h-4 w-4" />
61 |     </div>
62 |     <span className="text-sm font-medium flex-1 truncate">{title}</span>
63 |     <url && <span className="text-xs text-gray-400 shrink-0">{type}</span>
64 |   </div>
65 | );
66 | };
67 |
68 | export default MaterialItem;
69 | export type { MaterialItemProps, MaterialType };
70 |

```

FILE: [src/components/shared/MicroAssessmentCard.tsx](#)

TYPE: .tsx | LINES: 1-121 | SHA256: 71f14b0016da89312091ead6ec2d900de2d5ea58b497737e44d8cfa083c9da64

```

1 | import { useTranslation } from "react-i18next";
2 | import { Badge } from "@components/ui/badge";
3 | import { Button } from "@components/ui/button";
4 | import { Sparkles, Clock, X } from "lucide-react";
5 | import { ONBOARDING_XP, MAX_MICRO_DISMISSALS } from "@lib/onboardingConstants";
6 | import { PCard } from "@design-system";
7 |
8 | export interface MicroAssessmentCardProps {
9 |   assessmentType: string;
10 |   questionCount: number;
11 |   onComplete: () => void;
12 |   onDismiss: () => void;
13 |   dismissalCount: number;
14 |   /** While a complete/dismiss mutation is in flight, disable both actions so
15 |    * the card reflects progress rather than appearing unresponsive. */
16 |   isPending?: boolean;
17 | }
18 |
19 | /** Assessment types that have a dedicated localized label. */
20 | const KNOWN_TYPES = new Set([
21 |   "personality",
22 |   "self_efficacy",
23 |   "study_strategy",
24 |   "learning_style",
25 |   "baseline_prompt",
26 | ]);
27 |
28 | const MicroAssessmentCard = ({
29 |   assessmentType,
30 |   questionCount,
31 |   onComplete,
32 |   onDismiss,
33 |   dismissalCount,
34 |   isPending = false,
35 | }: MicroAssessmentCardProps) => {
36 |   const { t } = useTranslation(["student", "gamification"]);
37 |
38 |   // Use the localized type label when known; otherwise fall back to the raw
39 |   // type so an unseeded/unexpected type still renders meaningfully.
40 |   const typeLabel = KNOWN_TYPES.has(assessmentType)
41 |     ? t(`onboarding.microAssessment.types.${assessmentType}`)
42 |     : assessmentType;
43 |   const estimatedMinutes = Math.max(1, Math.ceil(questionCount * 0.4));
44 |   const remainingDismissals = MAX_MICRO_DISMISSALS - dismissalCount;
45 |
46 |   return (
47 |     <PCard className="p-4 space-y-3">
48 |       <div className="flex items-start justify-between">
49 |         <div className="flex items-center gap-2">
50 |           <div className="p-1.5 rounded-lg bg-blue-50">
51 |             <Sparkles className="h-4 w-4 text-blue-600" />
52 |           </div>
53 |           <div>
54 |             <p className="text-sm font-bold text-gray-900">
55 |               {t("onboarding.microAssessment.title", { type: typeLabel })}
56 |             </p>
57 |             <div className="flex items-center gap-2 mt-0.5">
58 |               <span className="flex items-center gap-1 text-xs text-gray-500">
59 |                 <Clock className="h-3 w-3" />
60 |                 {t("onboarding.microAssessment.estimatedMinutes", {
61 |                   minutes: estimatedMinutes,
62 |                 })}
63 |               </span>
64 |               <Badge className="bg-amber-100 text-amber-700 border-amber-200 text-[10px]">
65 |                 {t("xp.earned", {
66 |                   ns: "gamification",
67 |                   amount: ONBOARDING_XP.micro_assessment,
68 |                 })}
69 |               </Badge>

```

```

70 |         </div>
71 |     </div>
72 | </div>
73 | <button
74 |   type="button"
75 |   onClick={onDismiss}
76 |   disabled={isPending}
77 |   aria-label={t("onboarding.microAssessment.dismiss")}
78 |   className="p-1 rounded-md text-gray-400 hover:text-gray-600 hover:bg-slate-50 transition-colors disa
79 | >
80 |   <X className="h-4 w-4" />
81 | </button>
82 | </div>
83 |
84 | <p className="text-xs text-gray-500">
85 |   {t("onboarding.microAssessment.description", {
86 |     count: questionCount,
87 |   })}
88 | </p>
89 |
90 | <div className="flex items-center gap-2">
91 |   <Button
92 |     size="sm"
93 |     onClick={onComplete}
94 |     disabled={isPending}
95 |     variant="tactile"
96 |     className="text-xs font-semibold"
97 |   >
98 |     {t("onboarding.microAssessment.completeNow")}
99 |   </Button>
100 |   <Button
101 |     variant="ghost"
102 |     size="sm"
103 |     onClick={onDismiss}
104 |     disabled={isPending}
105 |     className="text-xs text-gray-500"
106 |   >
107 |     {t("onboarding.microAssessment.remindLater")}
108 |   </Button>
109 | </div>
110 |
111 | {remainingDismissals <= 1 && remainingDismissals > 0 && (
112 |   <p className="text-[10px] text-amber-600">
113 |     {t("onboarding.microAssessment.lastChance")}
114 |   </p>
115 | )}
116 | </PCard>
117 | );
118 | };
119 |
120 | export default MicroAssessmentCard;
121 |

```

FILE: src/components/shared/MobileTabBar.tsx

TYPE: .tsx | LINES: 1-85 | SHA256: 459bc14ccaa3bf018b7bac7f042c19a95681d6b68705c7d9a4ec1cea1bc229ab

```

1 | // =====
2 | // MobileTabBar – thumb-reachable bottom navigation for < lg screens
3 | // =====
4 | //
5 | // Responsive navigation for the redesigned chrome (spec: ui-prototype-migration,
6 | // task 1.2a / design §18, R18.5). On small screens the sidebar is an off-canvas
7 | // drawer (opened via the header hamburger); this fixed bottom bar surfaces the
8 | // role's core destinations for one-thumb reach. It is driven by the SAME
9 | // `navItems[role]` single source of truth, is RTL-safe (logical flow), uses
10 | // "e44px touch targets, and respects the safe-area inset.
11 | //
12 | // Rendered only when the `newUiChrome` flag is on (from Sidebar) and only below
13 | // `lg` (`lg:hidden`). The `new-mobile-tabbar` class is a hook for the
14 | // content-padding rule in index.css (so the fixed bar never covers content).
15 | // =====
16 |
17 | import { NavLink, useLocation } from "react-router-dom";
18 | import { useTranslation } from "react-i18next";
19 |
20 | import { useAuth } from "@/hooks/useAuth";
21 | import { getMobileTabItems } from "@/lib/navPresentation";
22 | import { cn } from "@/lib/utls";
23 | import type { UserRole } from "@/types/app";
24 |
25 | const MobileTabBar = () => {
26 |   const { t } = useTranslation("common");
27 |   const { profile } = useAuth();
28 |   const location = useLocation();
29 |
30 |   const role = (profile?.role ?? "student") as UserRole;
31 |
32 |   const items = getMobileTabItems(role);
33 |
34 |   // Active detection mirrors Sidebar.isItemActive so the two chromes agree.
35 |   const isActive = (to: string): boolean =>
36 |     location.pathname === to ||
37 |     (to !== `/${role}/dashboard` &&
38 |       to !== `/${role}` &&
39 |       location.pathname.startsWith(to));
40 |
41 |   if (items.length === 0) return null;
42 |
43 |   return (
44 |     <nav
45 |       className="new-mobile-tabbar fixed inset-x-0 bottom-0 z-40 flex items-stretch justify-around border-t bo
46 |       aria-label={t("header.primaryNav.label")}
47 |     >
48 |       {items.map((item) => {

```



```

49 |         const Icon = item.icon;
50 |         const active = isActive(item.to);
51 |         const isFab = item.raised;
52 |         return (
53 |             <NavLink
54 |                 key={item.to}
55 |                 to={item.to}
56 |                 viewTransition
57 |                 aria-current={active ? "page" : undefined}
58 |                 className={cn(
59 |                     "flex min-h-[3.25rem] flex-1 flex-col items-center justify-center gap-0.5 px-1 py-1.5 text-[10px]
60 |                     isFab &&
61 |                     "-mt-5 min-h-14 max-w-14 rounded-full border-[4px] border-white bg-[image:var(--brand-gradient)]
62 |                     !isFab &&
63 |                     (active
64 |                         ? "text-[#075985]"
65 |                         : "text-gray-500 hover:text-gray-700")
66 |                 )}
67 |             >
68 |                 <Icon
69 |                     className={cn(
70 |                         "h-5 w-5 shrink-0 transition-transform",
71 |                         active && "scale-110",
72 |                         isFab && "h-5 w-5"
73 |                     )}
74 |                     aria-hidden="true"
75 |                 />
76 |                 <span className="max-w-full truncate">{t(item.labelKey)}</span>
77 |             </NavLink>
78 |         );
79 |     }
80 | </nav>
81 | );
82 | };
83 |
84 | export default MobileTabBar;
85 |

```

FILE: [src/components/shared/MondayWeekPicker.tsx](#)

TYPE: .tsx | LINES: 1-76 | SHA256: d2e6332785a14ec82e64125232d757a87ffa9c6b436cfb7c9a506b17ef371ba7

```

1 | import { useState } from "react";
2 | import { Button } from "@components/ui/button";
3 | import {
4 |     Popover,
5 |     PopoverContent,
6 |     PopoverTrigger,
7 | } from "@components/ui/popover";
8 | import { Calendar } from "lucide-react";
9 | import { DayPicker } from "react-day-picker";
10 | import { format, endOfWeek } from "date-fns";
11 | import { useTranslation } from "react-i18next";
12 | import "react-day-picker/src/style.css";
13 |
14 | export interface MondayWeekPickerProps {
15 |     value: Date | null;
16 |     onChange: (date: Date) => void;
17 | }
18 |
19 | /**
20 |  * Week picker that only allows selecting Mondays.
21 |  * Displays "Week of {Mon} - {Sun}" format.
22 |  *
23 |  * Design: ADR-05, §8.9
24 |  * Requirements: 2.19
25 |  *
26 |  * Note: Requires react-day-picker to be installed (Task 35).
27 |  *
28 |  * @example
29 |  * <MondayWeekPicker value={date} onChange={setDate} />
30 |  */
31 | const MondayWeekPicker = ({ value, onChange }: MondayWeekPickerProps) => {
32 |     const { t } = useTranslation("common");
33 |     const [open, setOpen] = useState(false);
34 |
35 |     const handleSelect = (date: Date | undefined) => {
36 |         if (date && date.getDay() === 1) {
37 |             // Monday is day 1
38 |             onChange(date);
39 |             setOpen(false);
40 |         }
41 |     };
42 |
43 |     const displayText = value
44 |         ? `${t("weekPicker.weekOf")} ${format(value, "MMM d")} - ${format(
45 |             endOfWeek(value),
46 |             "MMM d"
47 |         )}`
48 |         : t("weekPicker.selectWeek");
49 |
50 |     return (
51 |         <Popover open={open} onOpenChange={setOpen}>
52 |             <PopoverTrigger asChild>
53 |                 <Button
54 |                     variant="outline"
55 |                     className="w-full justify-start text-left font-normal"
56 |                 >
57 |                     <Calendar className="h-4 w-4 me-2" />
58 |                     {displayText}
59 |                 </Button>
60 |             </PopoverTrigger>
61 |             <PopoverContent className="w-auto p-0" align="start">
62 |                 <DayPicker
63 |                     mode="single"

```

```

64 |         selected={value ?? undefined}
65 |         onSelect={handleSelect}
66 |         disabled={(date) => date.getDay() !== 1} // Only allow Mondays
67 |         initialFocus
68 |     />
69 | </PopoverContent>
70 | </Popover>
71 | );
72 | };
73 |
74 | export { MondayWeekPicker };
75 | export default MondayWeekPicker;
76 |

```

FILE: src/components/shared/MysteryBadge.tsx

TYPE: .tsx | LINES: 1-54 | SHA256: 94330778f7e236b5f83036ee5e61e6279d4ec117ff49cfd9d7620f907b6c5bec

```

1 | // =====
2 | // MysteryBadge – Badge with hidden conditions, reveals on earn
3 | // =====
4 |
5 | import { HelpCircle } from "lucide-react";
6 | import { cn } from "@lib/utils";
7 |
8 | interface MysteryBadgeProps {
9 |   name: string;
10 |   emoji: string;
11 |   isEarned: boolean;
12 |   hint?: string;
13 |   className?: string;
14 | }
15 |
16 | const MysteryBadge = ({
17 |   name,
18 |   emoji,
19 |   isEarned,
20 |   hint,
21 |   className,
22 | }: MysteryBadgeProps) => (
23 |   <div
24 |     className={cn(
25 |       "flex flex-col items-center gap-1 rounded-xl p-3 text-center transition-all",
26 |       isEarned ? "bg-amber-50" : "bg-gray-100",
27 |       className
28 |     )}
29 |   >
30 |     <div
31 |       className={cn(
32 |         "flex h-12 w-12 items-center justify-center rounded-full text-2xl",
33 |         isEarned ? "animate-badge-pop" : "grayscale opacity-40"
34 |       )}
35 |     >
36 |       {isEarned ? emoji : <HelpCircle className="h-6 w-6 text-gray-400" />}
37 |     </div>
38 |     <span
39 |       className={cn(
40 |         "text-xs font-semibold",
41 |         isEarned ? "text-gray-900" : "text-gray-400"
42 |       )}
43 |     >
44 |       {isEarned ? name : "???" }
45 |     </span>
46 |     {!isEarned && hint && (
47 |       <span className="text-[10px] text-gray-400 italic">{hint}</span>
48 |     )}
49 |   </div>
50 | );
51 |
52 | export default MysteryBadge;
53 | export type { MysteryBadgeProps };
54 |

```

FILE: src/components/shared/MysteryRewardBox.tsx

TYPE: .tsx | LINES: 1-151 | SHA256: 11b34eee9a46b942200485070fcd2c61d70123c57b83b13bc6280763e8cb1f3d

```

1 | // =====
2 | // MysteryRewardBox – Unboxing animation with reward reveal
3 | // Task 21.5
4 | // =====
5 |
6 | import { useState, useCallback } from "react";
7 | import { motion, AnimatePresence } from "framer-motion";
8 | import { Button } from "@components/ui/button";
9 | import { Gift, Sparkles, Loader2 } from "lucide-react";
10 | import {
11 |   useResolveMysteryReward,
12 |   type MysteryRewardResult,
13 | } from "@hooks/useMysteryRewardBox";
14 |
15 | const REWARD_LABELS: Record<
16 |   string,
17 |   { emoji: string; label: string; color: string }
18 | > = {
19 |   double_xp: { emoji: "🎁", label: "2x XP Bonus!", color: "text-amber-600" },
20 |   cosmetic: {
21 |     emoji: "🎁",
22 |     label: "Random Cosmetic!",
23 |     color: "text-purple-600",
24 |   },
25 |   boost: { emoji: "📈", label: "Temporary Boost!", color: "text-blue-600" },
26 | };
27 |
28 | interface MysteryRewardBoxProps {
29 |   studentId: string;

```

```

30 | institutionId: string;
31 | onComplete?: () => void;
32 | }
33 |
34 | const MysteryRewardBox = ({
35 |   studentId,
36 |   institutionId,
37 |   onComplete,
38 | }: MysteryRewardBoxProps) => {
39 |   const [phase, setPhase] = useState<"idle" | "opening" | "revealed">("idle");
40 |   const [result, setResult] = useState<MysteryRewardResult | null>(null);
41 |   const resolveReward = useResolveMysteryReward();
42 |
43 |   const handleOpen = useCallback(() => {
44 |     setPhase("opening");
45 |     resolveReward.mutate(
46 |       { studentId, institutionId },
47 |       {
48 |         onSuccess: (data) => {
49 |           setTimeout(() => {
50 |             setResult(data);
51 |             setPhase("revealed");
52 |           }, 1500);
53 |         },
54 |         onError: () => {
55 |           setPhase("idle");
56 |         },
57 |       }
58 |     );
59 |   }, [studentId, institutionId, resolveReward]);
60 |
61 |   const handleDismiss = useCallback(() => {
62 |     setPhase("idle");
63 |     setResult(null);
64 |     onComplete?.();
65 |   }, [onComplete]);
66 |
67 |   const rewardInfo = result ? REWARD_LABELS[result.outcome_type] : null;
68 |
69 |   return (
70 |     <div className="relative" data-testid="mystery-reward-box">
71 |       <AnimatePresence mode="wait">
72 |         {phase === "idle" && (
73 |           <motion.div
74 |             key="idle"
75 |             initial={{ scale: 0.8, opacity: 0 }}
76 |             animate={{ scale: 1, opacity: 1 }}
77 |             exit={{ scale: 0.8, opacity: 0 }}
78 |             className="text-center"
79 |           >
80 |             <motion.div
81 |               animate={{ y: [0, -5, 0] }}
82 |               transition={{ repeat: Infinity, duration: 2 }}
83 |               className="inline-block"
84 |             >
85 |               <Gift className="h-16 w-16 text-purple-500 mx-auto" />
86 |             </motion.div>
87 |             <p className="text-sm font-bold mt-2">Mystery Reward!</p>
88 |             <p className="text-xs text-gray-500 mt-1">
89 |               Tap to reveal your prize
90 |             </p>
91 |             <Button
92 |               size="sm"
93 |               className="mt-3 bg-gradient-to-r from-purple-500 to-pink-500 text-white active:scale-95"
94 |               onClick={handleOpen}
95 |             >
96 |               <Sparkles className="h-4 w-4" /> Open Box
97 |             </Button>
98 |           </motion.div>
99 |         )}
100 |
101 |         {phase === "opening" && (
102 |           <motion.div
103 |             key="opening"
104 |             initial={{ scale: 1 }}
105 |             animate={{
106 |               scale: [1, 1.2, 0.9, 1.1, 1],
107 |               rotate: [0, -5, 5, -3, 0],
108 |             }}
109 |             transition={{ duration: 1.5 }}
110 |             className="text-center"
111 |           >
112 |             <Gift className="h-16 w-16 text-purple-500 mx-auto animate-bounce" />
113 |             <div className="flex items-center justify-center gap-2 mt-3">
114 |               <Loader2 className="h-4 w-4 animate-spin text-purple-500" />
115 |               <p className="text-sm font-bold text-purple-600">Opening...</p>
116 |             </div>
117 |           </motion.div>
118 |         )}
119 |
120 |         {phase === "revealed" && rewardInfo && (
121 |           <motion.div
122 |             key="revealed"
123 |             initial={{ scale: 0.5, opacity: 0 }}
124 |             animate={{ scale: 1, opacity: 1 }}
125 |             transition={{ type: "spring", stiffness: 200, damping: 15 }}
126 |             className="text-center"
127 |           >
128 |             <div className="text-5xl mb-2">{rewardInfo.emoji}</div>
129 |             <p className="text-lg font-black ${rewardInfo.color}">
130 |               {rewardInfo.label}
131 |             </p>
132 |             <p className="text-xs text-gray-500 mt-1">
133 |               {result?.outcome_type === "double_xp" &&

```

```

134         "Your next XP award is doubled!")
135         {result?.outcome_type === "cosmetic" &&
136           "A new cosmetic has been added to your inventory!"}
137         {result?.outcome_type === "boost" &&
138           "A temporary XP boost has been activated!"}
139       </p>
140       <Button size="sm" className="mt-3" onClick={handleDismiss}>
141         Awesome!
142       </Button>
143     </motion.div>
144   )}
145 </AnimatePresence>
146 </div>
147 );
148 };
149
150 export default MysteryRewardBox;
151

```

FILE: src/components/shared/NotificationBell.tsx

```
TYPE: .tsx | LINES: 1-270 | SHA256: 2375e10ad6a1d4a1c97df2cbebbba036fbc0fa67bf9bb210b22020e55b64cabde
```

```
import { useState } from "react";  
import { Bell, CheckCheck } from "lucide-react";  
import { useTranslation } from "react-i18next";  
import {  
    Popover,  
    PopoverContent,  
    PopoverTrigger,  
} from "@components/ui/popover";  
import { Button } from "@components/ui/button";  
import { NoNotifications } from "@components/shared/EmptyState";  
import { useAuth } from "@hooks/useAuth";  
import { useNotificationRealtime } from "@hooks/useNotificationRealtime";  
import {  
    useUnreadCount,  
    useNotifications,  
    useMarkAsRead,  
    useMarkAllAsRead,  
    type Notification,  
} from "@hooks/useNotifications";  
import { formatRelativeTime } from "@lib/i18nHelpers";  
import { formatNotificationTitle } from "@lib/notificationPresentation";  
  
// % % % Date grouping helpers % % % % % % % % % % % % % % % % % % %  
  
type NotificationGroup = "today" | "earlierThisWeek" | "older";  
  
const getGroup = (dateStr: string): NotificationGroup => {  
    const now = new Date();  
    const date = new Date(dateStr);  
  
    const startOfToday = new Date(  
        now.getFullYear(),  
        now.getMonth(),  
        now.getDate()  
    );  
    const startOfWeek = new Date(startOfToday);  
    startOfWeek.setDate(startOfToday.getDate() - startOfToday.getDay());  
  
    if (date >= startOfToday) return "today";  
    if (date >= startOfWeek) return "earlierThisWeek";  
    return "older";  
};  
  
const groupNotifications = (  
    notifications: Notification[]  
): Record<NotificationGroup, Notification[]> => {  
    const groups: Record<NotificationGroup, Notification[]> = {  
        today: [],  
        earlierThisWeek: [],  
        older: [],  
    };  
    for (const n of notifications) {  
        groups[getGroup(n.created_at)].push(n);  
    }  
    return groups;  
};  
  
// % % % Relative time helper % % % % % % % % % % % % % % % % % % %  
  
const getRelativeTime = (dateStr: string, locale: string): string => {  
    const now = new Date();  
    const date = new Date(dateStr);  
    const diffMs = date.getTime() - now.getTime();  
    const diffMinutes = Math.round(diffMs / 60_000);  
    const diffHours = Math.round(diffMs / 3_600_000);  
    const diffDays = Math.round(diffMs / 86_400_000);  
  
    const safeLocale = locale.startsWith("ar") ? "ar-QA" : "en";  
  
    if (Math.abs(diffMinutes) < 60) {  
        return formatRelativeTime(diffMinutes, "minute", safeLocale);  
    }  
    if (Math.abs(diffHours) < 24) {  
        return formatRelativeTime(diffHours, "hour", safeLocale);  
    }  
    return formatRelativeTime(diffDays, "day", safeLocale);  
};  
  
// % % % NotificationRow % % % % % % % % % % % % % % % % % % %  
  
interface NotificationRowProps {  
    notification: Notification;
```

[illegible]

```

187 |         >
188 |         {badgeCount}
189 |       </span>
190 |     )}
191 |   </Button>
192 | </PopoverTrigger>
193 |
194 |   <PopoverContent
195 |     align="end"
196 |     className="w-[380px] max-h-[480px] p-0 flex flex-col overflow-hidden"
197 |   >
198 |     {/* Header */}
199 |     <div className="px-4 py-3 border-b border-border flex items-center justify-between">
200 |       <h3 className="text-sm font-semibold text-foreground">
201 |         {t("header.notificationsLabel")}
202 |       </h3>
203 |       {unreadCount > 0 && user?.id ? (
204 |         <Button
205 |           type="button"
206 |           variant="ghost"
207 |           size="sm"
208 |           className="h-7 px-2 text-xs text-primary"
209 |           onClick={() => markAllAsRead.mutate(user.id)}
210 |           disabled={markAllAsRead.isPending}
211 |         >
212 |           <CheckCheck className="h-3.5 w-3.5" aria-hidden="true" />
213 |           {t("header.markAllRead")}
214 |         </Button>
215 |       ) : null}
216 |     </div>
217 |
218 |     {/* Notification list */}
219 |     <div className="flex-1 overflow-y-auto">
220 |       {notifications.length === 0 ? (
221 |         <NoNotifications className="py-8" />
222 |       ) : (
223 |         groupOrder.map((group) => {
224 |           const items = groups[group];
225 |           if (items.length === 0) return null;
226 |           return (
227 |             <div key={group}>
228 |               <div className="px-4 py-2 bg-muted/50">
229 |                 <p className="text-xs font-semibold text-muted-foreground uppercase tracking-wide">
230 |                   {groupLabels[group]}
231 |                 </p>
232 |               </div>
233 |               {items.map((n) => (
234 |                 <NotificationRow
235 |                   key={n.id}
236 |                   notification={n}
237 |                   locale={i18n.language}
238 |                   onSelect={handleNotificationSelect}
239 |                 />
240 |               ))}
241 |             </div>
242 |           );
243 |         })}
244 |       )}
245 |     </div>
246 |
247 |     {/* Footer */}
248 |     {notifications.length > 0 && (
249 |       <div className="px-4 py-3 border-t border-border">
250 |         <Button
251 |           asChild
252 |           variant="ghost"
253 |           className="h-auto w-full justify-center px-0 py-0 text-xs font-semibold text-primary"
254 |         >
255 |           <a
256 |             href={`/${role ?? "student"}/notifications`}
257 |             onClick={() => setOpen(false)}
258 |           >
259 |             {t("header.seeAllNotifications", "View all !")}
260 |           </a>
261 |         </Button>
262 |       </div>
263 |     )}
264 |   </PopoverContent>
265 | </Popover>
266 | );
267 | };
268 |
269 | export default NotificationBell;
270 |

```

FILE: src/components/shared/NotificationCenter.tsx

TYPE: .tsx | LINES: 1-256 | SHA256: 7b26203e791393269d959241c83886471c9b3250d8796754f76af6d4fa1247cf

```

1 | // =====
2 | // NotificationCenter - Dropdown list of notifications
3 | // Validates: Requirements 31.4
4 | // =====
5 |
6 | import { useCallback, useMemo } from "react";
7 | import { useTranslation } from "react-i18next";
8 | import { useNavigate } from "react-router-dom";
9 | import {
10 |   Bell,
11 |   CheckCheck,
12 |   Trash2,
13 |   GraduationCap,
14 |   FileText,
15 |   Award,
16 |   Flame,

```

```

17 | AlertTriangle,
18 | Users,
19 | Sparkles,
20 | Unlock,
21 | Layers,
22 | Bot,
23 | } from "lucide-react";
24 | import { Button } from "@components/ui/button";
25 | import { useAuth } from "@hooks/useAuth";
26 | import {
27 |   useNotifications,
28 |   useMarkAsRead,
29 |   useMarkAllAsRead,
30 |   useDeleteNotification,
31 | } from "@hooks/useNotifications";
32 | import type { NotificationType } from "@hooks/useNotifications";
33 | import {
34 |   groupNotifications,
35 |   type BatchedNotification,
36 | } from "@lib/notificationBatcher";
37 | import { formatDistanceToNow } from "date-fns";
38 | import { cn } from "@lib/utlis";
39 |
40 | interface NotificationCenterProps {
41 |   onClose: () => void;
42 | }
43 |
44 | const NOTIFICATION_ICONS: Record<NotificationType, React.ReactNode> = {
45 |   grade_released: <GraduationCap className="h-4 w-4 text-blue-500" />,
46 |   new_assignment: <FileText className="h-4 w-4 text-teal-500" />,
47 |   badge_earned: <Award className="h-4 w-4 text-amber-500" />,
48 |   streak_at_risk: <Flame className="h-4 w-4 text-red-500" />,
49 |   at_risk_alert: <AlertTriangle className="h-4 w-4 text-orange-500" />,
50 |   peer_milestone: <Users className="h-4 w-4 text-purple-500" />,
51 |   perfect_day_nudge: <Sparkles className="h-4 w-4 text-green-500" />,
52 |   prerequisite_unlocked: <Unlock className="h-4 w-4 text-indigo-500" />,
53 |   digest: <Layers className="h-4 w-4 text-slate-500" />,
54 | };
55 |
56 | const NotificationCenter = ({ onClose }: NotificationCenterProps) => {
57 |   const { t } = useTranslation("common");
58 |   const { user, role } = useAuth();
59 |   const navigate = useNavigate();
60 |   const { data: notifications = [], isLoading } = useNotifications(user?.id);
61 |   const markAsRead = useMarkAsRead();
62 |   const markAllAsRead = useMarkAllAsRead();
63 |   const deleteNotification = useDeleteNotification();
64 |
65 |   const hasUnread = notifications.some((n) => !n.is_read);
66 |
67 |   // Group notifications by type when >3 of same type (Requirement 65.3)
68 |   const groupedNotifications = useMemo(
69 |     () => groupNotifications(notifications),
70 |     [notifications]
71 |   );
72 |
73 |   const handleNotificationClick = useCallback(
74 |     (item: BatchedNotification) => {
75 |       // Mark all items in the group as read
76 |       for (const n of item.items) {
77 |         if (!n.is_read) {
78 |           markAsRead.mutate(n.id);
79 |         }
80 |       }
81 |     },
82 |     [markAsRead]
83 |   );
84 |
85 |   const handleMarkAllAsRead = useCallback(() => {
86 |     if (user?.id) {
87 |       markAllAsRead.mutate(user.id);
88 |     }
89 |   }, [markAllAsRead, user]);
90 |
91 |   const handleDelete = useCallback(
92 |     (e: React.MouseEvent, item: BatchedNotification) => {
93 |       e.stopPropagation();
94 |       // Delete all items in the group
95 |       for (const n of item.items) {
96 |         deleteNotification.mutate(n.id);
97 |       }
98 |     },
99 |     [deleteNotification]
100 |   );
101 |
102 |   return (
103 |     <div className="flex flex-col max-h-96">
104 |       <div className="flex items-center justify-between px-4 py-3 border-b">
105 |         <h3 className="text-sm font-semibold">
106 |           {t("header.notificationsLabel")}
107 |         </h3>
108 |         {hasUnread && (
109 |           <Button
110 |             variant="ghost"
111 |             size="sm"
112 |             className="h-auto px-2 py-1 text-xs text-blue-600 hover:text-blue-700"
113 |             onClick={handleMarkAllAsRead}
114 |             disabled={markAllAsRead.isPending}
115 |           >
116 |             <CheckCheck className="h-3 w-3 me-1" />
117 |             {t("header.markAllRead")}
118 |           </Button>
119 |         )}
120 |     </div>

```

```

121 | </div>
122 |
123 | /* Notification list */
124 | <div className="overflow-y-auto flex-1">
125 |   {isLoading ? (
126 |     <div className="p-4 text-center text-sm text-gray-500">
127 |       {t("status.loading")}
128 |     </div>
129 |   ) : groupedNotifications.length === 0 ? (
130 |     <div className="flex flex-col items-center justify-center py-8 px-4">
131 |       <Bell className="h-8 w-8 text-gray-300 mb-2" />
132 |       <p className="text-sm text-gray-500">
133 |         {t("header.emptyNotifications.title")}
134 |       </p>
135 |     </div>
136 |   ) : (
137 |     groupedNotifications.map((item, idx) => {
138 |       const hasUnreadItems = item.items.some((n) => !n.is_read);
139 |       const mostRecent =
140 |         item.items.length > 0 ? item.items[0] : undefined;
141 |       const timeAgo = mostRecent
142 |         ? formatDistanceToNow(new Date(mostRecent.created_at), {
143 |             addSuffix: true,
144 |           })
145 |       : "";
146 |
147 |       return (
148 |         <button
149 |           key={
150 |             item.is_grouped
151 |               ? `group-${item.type}-${idx}`
152 |               : mostRecent?.id ?? idx
153 |           }
154 |           type="button"
155 |           className={cn(
156 |             "flex items-start gap-3 px-4 py-3 cursor-pointer transition-colors hover:bg-slate-50 w-full",
157 |             hasUnreadItems && "bg-blue-50/50"
158 |           )}
159 |           onClick={() => handleNotificationClick(item)}
160 |         >
161 |           /* Type icon */
162 |           <div className="mt-0.5 shrink-0">
163 |             {item.is_grouped ? (
164 |               <Layers className="h-4 w-4 text-slate-500" />
165 |             ) : (
166 |               NOTIFICATION_ICONS[item.type as NotificationType] ?? (
167 |                 <Bell className="h-4 w-4 text-gray-400" />
168 |               )
169 |             )}
170 |           </div>
171 |
172 |           /* Content */
173 |           <div className="flex-1 min-w-0">
174 |             <p
175 |               className={`text-sm ${
176 |                 hasUnreadItems ? "font-semibold" : "font-medium"
177 |               } text-gray-900 truncate`}
178 |             >
179 |               {item.title}
180 |             </p>
181 |             <p className="text-xs text-gray-500 line-clamp-2 mt-0.5">
182 |               {item.body}
183 |             </p>
184 |             /* Requirement 10.3: "Ask Tutor" deep link for low-scoring grade notifications */
185 |             {!item.is_grouped &&
186 |               mostRecent?.metadata &&
187 |               typeof mostRecent.metadata === "object" &&
188 |               "tutor_action_url" in mostRecent.metadata &&
189 |               typeof (mostRecent.metadata as Record<string, unknown>).
190 |                 tutor_action_url === "string" && (
191 |                 <button
192 |                   type="button"
193 |                   className="inline-flex items-center gap-1 mt-1 text-xs font-semibold text-teal-600 hov
194 |                   onClick={e => {
195 |                     e.stopPropagation();
196 |                     const url = (
197 |                       mostRecent?.metadata as Record<
198 |                         string,
199 |                         unknown
200 |                       )?.tutor_action_url;
201 |                     if (typeof url === "string") {
202 |                       navigate(url);
203 |                     }
204 |                   }}
205 |                 >
206 |                   <Bot className="h-3 w-3" />
207 |                   {t("notificationsGroup.askTutor", "Ask Tutor")}
208 |                 </button>
209 |               )}
210 |             <p className="text-[10px] text-gray-400 mt-1">{timeAgo}</p>
211 |           </div>
212 |         </div>
213 |
214 |         /* Actions */
215 |         <div className="shrink-0 flex items-center gap-1">
216 |           {hasUnreadItems && (
217 |             <span
218 |               className="h-2 w-2 rounded-full bg-blue-500"
219 |               aria-label="Unread"
220 |             />
221 |           )}
222 |           <Button
223 |             variant="ghost"
224 |             size="icon"

```



```

225 |         className="h-6 w-6 text-gray-400 hover:text-red-500"
226 |         onClick={(e) => handleDelete(e, item)}
227 |         aria-label="Delete notification"
228 |     >
229 |         <Trash2 className="h-3 w-3" />
230 |     </Button>
231 | </div>
232 | </button>
233 |     );
234 |   });
235 | }
236 | </div>
237 |
238 | /* Footer: full-page feed */
239 | {role && (
240 |   <button
241 |     type="button"
242 |     onClick={() => {
243 |       navigate(`/${role}/notifications`);
244 |       onClose();
245 |     }}
246 |     className="border-t px-4 py-2.5 text-center text-xs font-semibold text-blue-600 hover:bg-slate-50"
247 |   >
248 |     {t("header.seeAllNotifications", "See all notifications")}
249 |   </button>
250 | )}
251 | </div>
252 | );
253 | };
254 |
255 | export default NotificationCenter;
256 |

```

FILE: src/components/shared/OutcomeTypeBadge.tsx

TYPE: .tsx | LINES: 1-49 | SHA256: b74cb2ea1b40014039e9e9313a704936abef4da3032bb6056763c4ec5e2f1733

```

1 | // =====
2 | // OutcomeTypeBadge – Badge for ILO/PL0/CLO with color coding
3 | // ILO red, PL0 blue, CLO green
4 | // =====
5 |
6 | import { cn } from "@lib/utills";
7 |
8 | type OutcomeType = "ILO" | "PL0" | "CLO";
9 |
10 | interface OutcomeTypeBadgeProps {
11 |   type: OutcomeType;
12 |   className?: string;
13 | }
14 |
15 | const OUTCOME_STYLES: Record<
16 |   OutcomeType,
17 |   { bg: string; text: string; border: string }
18 | > = {
19 |   ILO: { bg: "bg-red-100", text: "text-red-700", border: "border-red-200" },
20 |   PL0: { bg: "bg-blue-100", text: "text-blue-700", border: "border-blue-200" },
21 |   CLO: {
22 |     bg: "bg-green-100",
23 |     text: "text-green-700",
24 |     border: "border-green-200",
25 |   },
26 | };
27 |
28 | const OutcomeTypeBadge = ({ type, className }: OutcomeTypeBadgeProps) => {
29 |   const style = OUTCOME_STYLES[type];
30 |
31 |   return (
32 |     <span
33 |       className={cn(
34 |         "inline-flex items-center rounded-full border px-2.5 py-0.5 text-xs font-bold tracking-wide uppercase"
35 |         style.bg,
36 |         style.text,
37 |         style.border,
38 |         className
39 |       )}
40 |     >
41 |       {type}
42 |     </span>
43 |   );
44 | };
45 |
46 | export default OutcomeTypeBadge;
47 | export { OUTCOME_STYLES };
48 | export type { OutcomeTypeBadgeProps, OutcomeType };
49 |

```

FILE: src/components/shared/ParentButton.tsx

TYPE: .tsx | LINES: 1-64 | SHA256: 137be1fa789f9749edd78da2cbc99e22b18bd6cce3831f2d2ec62997a130818d

```

1 | import * as React from "react";
2 | import { cn } from "@lib/utills";
3 |
4 | export interface ParentButtonProps
5 |   extends React.ButtonHTMLAttributes<HTMLButtonElement> {
6 |   variant?: "primary" | "secondary" | "ghost" | "danger" | "compactIcon";
7 |   size?: "sm" | "md" | "lg";
8 | }
9 |
10 | /**
11 |  * Reusable Parent button system matching prototype CSS:
12 |  * On parent surfaces, buttons use flat brand blue (#0382bd) for primary actions,
13 |  * soft blue (#eff6ff) for secondary, ghost (#f8faff + border) for quiet actions,
14 |  * and soft red (#fef2f2) for destructive actions.
15 |  */

```

```

16 | export const ParentButton = React.forwardRef<
17 |   HTMLButtonElement,
18 |   ParentButtonProps
19 | >(
20 |   (
21 |     { className, variant = "primary", size = "md", children, ...props },
22 |     ref
23 |   ) => {
24 |     const baseClasses =
25 |       "inline-flex items-center justify-center gap-1.5 font-extrabold transition-colors focus-visible:outline-
26 |
27 |     const variantClasses = {
28 |       primary:
29 |         "bg-[#0382bd] text-white shadow-xs hover:bg-[#026fa3] active:bg-[#025c88]",
30 |       secondary:
31 |         "bg-blue-50 text-blue-700 hover:bg-blue-100 border border-blue-100 dark:bg-blue-950/40 dark:text-blue-
32 |       ghost:
33 |         "bg-slate-50 text-slate-700 border border-slate-200 hover:bg-slate-100 dark:bg-slate-900 dark:text-sla
34 |       danger:
35 |         "bg-red-50 text-red-700 border border-red-100 hover:bg-red-100 dark:bg-red-950/40 dark:text-red-300 da
36 |       compactIcon:
37 |         "bg-white text-slate-600 border border-slate-200 hover:bg-slate-50 dark:bg-slate-900 dark:text-slate-3
38 |     };
39 |
40 |     const sizeClasses = {
41 |       sm: "h-8 px-3 text-xs rounded-xl",
42 |       md: "h-9 px-4 text-xs rounded-xl",
43 |       lg: "h-11 px-5 text-sm rounded-xl",
44 |     };
45 |
46 |     return (
47 |       <button
48 |         ref={ref}
49 |         className={cn(
50 |           baseClasses,
51 |           variantClasses[variant],
52 |           variant !== "compactIcon" && sizeClasses[size],
53 |           className
54 |         )}
55 |         {...props}
56 |       >
57 |         {children}
58 |       </button>
59 |     );
60 |   }
61 | );
62 |
63 | ParentButton.displayName = "ParentButton";
64 |

```

FILE: [src/components/shared/ParentDashboardNew.tsx](#)

TYPE: .tsx | LINES: 1-151 | SHA256: 1cb7ee3d33435687b24124793879b2d274c6b51db3a06183eb137fb9ba57a017

```

1 | // =====
2 | // ParentDashboardNew – redesigned parent dashboard (P2, spec task 2.3)
3 | // =====
4 | //
5 | // Growth / wellbeing framing (no raw grades) gated behind `newUiDashboards`
6 | // (see the wrapper in ParentDashboard.tsx). REUSES the existing
7 | // `useParentDashboardAggregate` (one round-trip: KPIs + linked children, no new
8 | // writes). FIRST CUTOVER: this screen now builds entirely from the prototype
9 | // design system (`@/design-system`) – WelcomeHero, KPICard, SectionCard,
10 | // MasteryRing, Shimmer, Badge – instead of the legacy `@/components/shared/*`.
11 | // =====
12 |
13 | import { useTranslation } from "react-i18next";
14 | import {
15 |   BookOpen,
16 |   CalendarDays,
17 |   Flame,
18 |   GraduationCap,
19 |   TrendingUp,
20 |   Users,
21 | } from "lucide-react";
22 |
23 | import {
24 |   Badge,
25 |   KPICard,
26 |   MasteryRing,
27 |   SectionCard,
28 |   Shimmer,
29 |   WelcomeHero,
30 | } from "@/design-system";
31 | import { useAuth } from "@/hooks/useAuth";
32 | import { useParentDashboardAggregate } from "@/hooks/useParentDashboardAggregate";
33 | import { attainmentValueClass } from "@/lib/attainmentTone";
34 |
35 | const ParentDashboardNew = () => {
36 |   const { t } = useTranslation("common");
37 |   const { user, profile } = useAuth();
38 |
39 |   const aggregate = useParentDashboardAggregate(user?.id);
40 |   const kpis = aggregate.data?.kpis;
41 |   const children = aggregate.data?.children ?? [];
42 |   const loading = aggregate.isPending;
43 |
44 |   return (
45 |     <div className="space-y-6">
46 |       <WelcomeHero
47 |         name={profile?.full_name ?? "Parent"}
48 |         userRole="parent"
49 |         subtitle={t("parentDashboard.subtitle")}
50 |       />
51 |     </div>

```

```

52 |     { /* KPI row */
53 |     {loading ? (
54 |       <div className="grid grid-cols-2 gap-4 md:grid-cols-4">
55 |         {Array.from({ length: 4 }).map((_, i) => (
56 |           <Shimmer key={i} className="h-24 rounded-xl" />
57 |         ))}
58 |       </div>
59 |     ) : (
60 |       <div className="grid grid-cols-2 gap-4 md:grid-cols-4">
61 |         <KPICard
62 |           icon={Users}
63 |           label={t("parentDashboard.children")}
64 |           value={kpis?.linkedChildren ?? 0}
65 |         />
66 |         <KPICard
67 |           icon={BookOpen}
68 |           label={t("parentDashboard.totalCourses")}
69 |           value={kpis?.totalCourses ?? 0}
70 |         />
71 |         <KPICard
72 |           icon={TrendingUp}
73 |           label={t("parentDashboard.avgAttainment")}
74 |           value={`$${kpis?.avgAttainment ?? 0}%`}
75 |           valueClassName={attainmentValueClass(kpis?.avgAttainment ?? 0)}
76 |         />
77 |         <KPICard
78 |           icon={CalendarDays}
79 |           label={t("parentDashboard.deadlines")}
80 |           value={kpis?.upcomingDeadlines ?? 0}
81 |           iconBgClass="bg-amber-50"
82 |           iconColorClass="text-amber-600"
83 |         />
84 |       </div>
85 |     )}
86 |
87 |     { /* Children overview */
88 |     <SectionCard
89 |       icon={GraduationCap}
90 |       title={t("parentDashboard.yourChildren")}
91 |     >
92 |       {loading ? (
93 |         <div className="space-y-3">
94 |           {Array.from({ length: 2 }).map((_, i) => (
95 |             <Shimmer key={i} className="h-20 rounded-xl" />
96 |           ))}
97 |         </div>
98 |       ) : children.length > 0 ? (
99 |         <div className="space-y-3">
100 |           {children.map((child) => (
101 |             <div
102 |               key={child.student_id}
103 |               className="flex items-center justify-between gap-4 rounded-xl border border-slate-100 p-4 tran
104 |             >
105 |               <div className="flex min-w-0 items-center gap-4">
106 |                 <MasteryRing
107 |                   value={child.avg_attainment}
108 |                   size={52}
109 |                   strokeWidth={6}
110 |                   tone="auto"
111 |                 />
112 |                 <div className="min-w-0">
113 |                   <p className="truncate text-sm font-semibold text-gray-900">
114 |                     {child.student_name}
115 |                   </p>
116 |                   <p className="truncate text-xs text-gray-500">
117 |                     {t("parentDashboard.levelLabel")} {child.current_level} .{" "}
118 |                     {child.enrolled_courses}{" "}
119 |                     {t("parentDashboard.coursesLabel")}
120 |                   </p>
121 |                 </div>
122 |               </div>
123 |               <div className="flex shrink-0 items-center gap-3">
124 |                 <Badge variant="outline" className="text-xs">
125 |                   {child.xp_total} {t("parentDashboard.xp")}
126 |                 </Badge>
127 |                 <span className="flex items-center gap-1 text-sm font-medium text-orange-500">
128 |                   <Flame className="h-4 w-4" aria-hidden="true" />
129 |                   {child.current_streak}d
130 |                 </span>
131 |               </div>
132 |             </div>
133 |           ))}
134 |         </div>
135 |       ) : (
136 |         <div className="flex flex-col items-center justify-center py-8 text-center">
137 |           <div className="mb-3 rounded-full bg-blue-50 p-3">
138 |             <Users className="h-8 w-8 text-blue-500" aria-hidden="true" />
139 |           </div>
140 |           <p className="max-w-[260px] text-sm text-gray-500">
141 |             {t("parentDashboard.noChildren")}
142 |           </p>
143 |         </div>
144 |       )}
145 |     </SectionCard>
146 |   </div>
147 | );
148 | };
149 |
150 | export default ParentDashboardNew;
151 |

```

FILE: src/components/shared/ParentSectionIcon.tsx

TYPE: .tsx | LINES: 1-31 | SHA256: 9e1c366997b6fcf2f4ad2f394d59a846bc0a77703312f4259ab6ba88d49eb0eb

```
1 | import * as React from "react";
2 | import { cn } from "@lib/utils";
3 |
4 | export interface ParentSectionIconProps {
5 |   children?: React.ReactNode;
6 |   emoji?: string;
7 |   className?: string;
8 | }
9 |
10 | /**
11 |  * Reusable prototype section icon tile (.sec-h .chip in shared.css).
12 |  * Neumorphic soft-extrude 30x30 tile:
13 |  * white/light surface, thin border, subtle extrude shadow.
14 |  */
15 | export const ParentSectionIcon: React.FC<ParentSectionIconProps> = ({
16 |   children,
17 |   emoji,
18 |   className,
19 | }) => {
20 |   return (
21 |     <span
22 |       className={cn(
23 |         "flex h-[30px] w-[30px] shrink-0 items-center justify-center rounded-[10px] text-[16px] leading-none t
24 |         className
25 |       )}
26 |     >
27 |     {emoji ?? children}
28 |   </span>
29 | );
30 | };
31 |
```

FILE: src/components/shared/ParentStudentCard.tsx

TYPE: .tsx | LINES: 1-81 | SHA256: 8b6ad45bf323b4b8ef8616dca24f17d40c352ddd86db0cb811ea619849a386a

```
1 | // =====
2 | // ParentStudentCard – Parent view of linked student summary
3 | // =====
4 |
5 | import { User, TrendingUp, Flame, BookOpen } from "lucide-react";
6 | import { cn } from "@lib/utils";
7 | import { PCard } from "@design-system";
8 |
9 | interface ParentStudentCardProps {
10 |   studentName: string;
11 |   avatarUrl?: string;
12 |   level: number;
13 |   xpTotal: number;
14 |   streak: number;
15 |   coursesEnrolled: number;
16 |   averageAttainment?: number;
17 |   onClick?: () => void;
18 |   className?: string;
19 | }
20 |
21 | const ParentStudentCard = ({
22 |   studentName,
23 |   avatarUrl,
24 |   level,
25 |   xpTotal,
26 |   streak,
27 |   coursesEnrolled,
28 |   averageAttainment,
29 |   onClick,
30 |   className,
31 | }: ParentStudentCardProps) => {
32 |   <PCard
33 |     className={cn(
34 |       "p-4 cursor-pointer hover:shadow-lg transition-shadow",
35 |       className
36 |     )}
37 |     onClick={onClick}
38 |   >
39 |     <div className="flex items-center gap-3">
40 |       <div className="h-12 w-12 rounded-full bg-slate-100 flex items-center justify-center overflow-hidden shr
41 |         {avatarUrl ? (
42 |           <img
43 |             src={avatarUrl}
44 |             alt={studentName}
45 |             loading="lazy"
46 |             decoding="async"
47 |             className="h-full w-full object-cover"
48 |           />
49 |         ) : (
50 |           <User className="h-6 w-6 text-gray-400" />
51 |         )}
52 |     </div>
53 |     <div className="flex-1 min-w-0">
54 |       <h4 className="text-sm font-bold truncate">{studentName}</h4>
55 |       <p className="text-xs text-gray-500">
56 |         Level {level} · {xpTotal.toLocaleString()} XP
57 |       </p>
58 |     </div>
59 |   </div>
60 |   <div className="grid grid-cols-3 gap-2 mt-3">
61 |     <div className="flex items-center gap-1.5 text-xs">
62 |       <Flame className="h-3.5 w-3.5 text-orange-500" />
63 |       <span className="font-medium">{streak}</span>
64 |     </div>
65 |     <div className="flex items-center gap-1.5 text-xs">
```

```

66 |         <BookOpen className="h-3.5 w-3.5 text-blue-500" />
67 |         <span className="font-medium">{coursesEnrolled} courses</span>
68 |     </div>
69 |     {averageAttainment !== null && (
70 |         <div className="flex items-center gap-1.5 text-xs">
71 |             <TrendingUp className="h-3.5 w-3.5 text-green-500" />
72 |             <span className="font-medium">{Math.round(averageAttainment)}%</span>
73 |         </div>
74 |     )}
75 | </div>
76 | </PCard>
77 | );
78 |
79 | export default ParentStudentCard;
80 | export type { ParentStudentCardProps };
81 |

```

FILE: [src/components/shared/PasswordInput.tsx](#)

TYPE: .tsx | LINES: 1-112 | SHA256: ef16bedec92babf839ec2b7d0e6fe6f282987af7cfc33b333fe4a36c0ef678df

```

1 | // =====
2 | // PasswordInput – Shadcn Input with an accessible show/hide visibility toggle
3 | // =====
4 | //
5 | // Renders a password field with a trailing eye toggle that reveals or masks the
6 | // entered characters (Requirements 5.1-5.4). The toggle:
7 | // - swaps the input `type` between "password" (masked) and "text" (plain),
8 | // - exposes an `aria-label` that reflects the action it will perform next
9 | // ("Show password" while masked, "Hide password" while revealed),
10 | // - meets the 44px minimum mobile touch target.
11 | //
12 | // When more than one password field shares a form, wrap them in a
13 | // `PasswordVisibilityGroup` and pass a `groupId`; the group enforces that at
14 | // most one field is revealed at a time via the pure `passwordVisibility`
15 | // reducer (Requirement 5.5). Outside a group, each field manages its own
16 | // independent visibility.
17 | //
18 | // This is a presentational component only – no Supabase access. It is a drop-in
19 | // replacement for a Shadcn `Input type="password"` and forwards its ref to the
20 | // underlying input so it composes with React Hook Form's `register`/`field`.
21 | //
22 | // _Requirements: 5.1, 5.2, 5.3, 5.4, 5.5_
23 |
24 | import * as React from "react";
25 | import { Eye, EyeOff } from "lucide-react";
26 | import { useTranslation } from "react-i18next";
27 |
28 | import { Input } from "@components/ui/input";
29 | import { Button } from "@components/ui/button";
30 | import { cn } from "@lib/utls";
31 | import { usePasswordVisibilityGroup } from "@components/shared/PasswordVisibilityGroup";
32 |
33 | export interface PasswordInputProps
34 |   extends Omit<React.ComponentProps<typeof Input>, "type"> {
35 |   /**
36 |    * Stable identifier used for mutual-exclusion coordination when the field is
37 |    * rendered inside a `PasswordVisibilityGroup`. Omit it for standalone fields
38 |    * (e.g. the single password field on the login form); a unique fallback id is
39 |    * generated automatically so group membership is opt-in.
40 |    */
41 |   groupId?: string;
42 | }
43 |
44 | const PasswordInput = React.forwardRef<HTMLInputElement, PasswordInputProps>(
45 |   ({ className, groupId, ...props }, ref) => {
46 |     const { t } = useTranslation("auth");
47 |     const group = usePasswordVisibilityGroup();
48 |
49 |     // A generated id keeps standalone fields and ungrouped duplicates distinct
50 |     // within a group without requiring callers to supply one.
51 |     const generatedId = React.useId();
52 |     const fieldId = groupId ?? generatedId;
53 |
54 |     // Local visibility for standalone use; the group (when present) owns the
55 |     // mutual-exclusion state instead so revealing one field masks the others.
56 |     const [localRevealed, setLocalRevealed] = React.useState(false);
57 |     const revealed = group ? group.isRevealed(fieldId) : localRevealed;
58 |
59 |     const toggle = () => {
60 |       if (group) {
61 |         if (revealed) {
62 |           group.hide(fieldId);
63 |         } else {
64 |           group.reveal(fieldId);
65 |         }
66 |       }
67 |       return;
68 |     };
69 |     setLocalRevealed((prev) => !prev);
70 |   };
71 |
72 |   // The accessible name describes the action the control performs next, so it
73 |   // changes as visibility toggles (R5.4).
74 |   const toggleLabel = revealed ? t("password.hide") : t("password.show");
75 |
76 |   return (
77 |     <div className="relative">
78 |       <Input
79 |         {...props}
80 |         ref={ref}
81 |         type={revealed ? "text" : "password"}
82 |         // Reserve inline-end space for the toggle (logical for RTL support).
83 |         className={cn("pe-11", className)}
84 |       />
85 |       <Button

```

```

85 |         type="button"
86 |         variant="ghost"
87 |         size="icon"
88 |         onClick={toggle}
89 |         // Mirror the input's disabled state so the toggle is unreachable when
90 |         // the field itself is disabled.
91 |         disabled={props.disabled}
92 |         aria-label={toggleLabel}
93 |         aria-pressed={revealed}
94 |         title={toggleLabel}
95 |         // 44px minimum touch target (R5.4); centered over the inline-end edge.
96 |         className="absolute end-0 top-1/2 h-11 w-11 -translate-y-1/2 text-gray-500 hover:bg-transparent hove
97 |     >
98 |         {revealed ? (
99 |             <EyeOff className="h-4 w-4" aria-hidden="true" />
100 |         ) : (
101 |             <Eye className="h-4 w-4" aria-hidden="true" />
102 |         )}
103 |     </Button>
104 | </div>
105 | );
106 | }
107 | );
108 |
109 | PasswordInput.displayName = "PasswordInput";
110 |
111 | export { PasswordInput };
112 |

```

FILE: [src/components/shared/PasswordVisibilityGroup.tsx](#)

TYPE: .tsx | LINES: 1-84 | SHA256: 81cc5390d6e864adda6bf881d5fd7c3f6fde81d43468a4d98749c06087d88a55

```

1 | // =====
2 | // PasswordVisibilityGroup – mutual-exclusion provider for password fields
3 | // =====
4 | //
5 | // Wraps a set of `PasswordInput` fields so that at most one of them may reveal
6 | // its characters in plain text at a time (Requirement 5.5). The mutual-exclusion
7 | // logic lives entirely in the pure `passwordVisibility` reducer
8 | // (`src/lib/passwordVisibility.ts`); this provider only adapts that reducer to
9 | // React via `useReducer` and exposes a small context API to descendant fields.
10 | //
11 | // A `PasswordInput` that renders outside any `PasswordVisibilityGroup` manages
12 | // its own independent visibility (see `PasswordInput.tsx`), so the provider is
13 | // only needed on forms that contain more than one password field (sign-up,
14 | // accept-invite, update-password).
15 | //
16 | // _Requirements: 5.5_
17 |
18 | import {
19 |     createContext,
20 |     useContext,
21 |     useMemo,
22 |     useReducer,
23 |     type ReactNode,
24 | } from "react";
25 | import {
26 |     initialPasswordVisibilityState,
27 |     isFieldRevealed,
28 |     passwordVisibilityReducer,
29 | } from "@lib/passwordVisibility";
30 |
31 | /**
32 |  * Context API consumed by `PasswordInput` to coordinate mutual exclusion.
33 |  */
34 | export interface PasswordVisibilityContextValue {
35 |     /** Is the field with this id currently revealed? */
36 |     readonly isRevealed: (id: string) => boolean;
37 |     /** Reveal this field, masking every other field in the group. */
38 |     readonly reveal: (id: string) => void;
39 |     /** Mask this field (no-op if it is already masked). */
40 |     readonly hide: (id: string) => void;
41 | }
42 |
43 | const PasswordVisibilityContext =
44 |     createContext<PasswordVisibilityContextValue | null>(null);
45 |
46 | /**
47 |  * Returns the enclosing group's visibility API, or `null` when the field is
48 |  * rendered outside any `PasswordVisibilityGroup` (standalone, self-managed).
49 |  */
50 | // eslint-disable-next-line react-refresh/only-export-components
51 | export const usePasswordVisibilityGroup =
52 |     ((): PasswordVisibilityContextValue | null =>
53 |         useContext(PasswordVisibilityContext);
54 |     )
55 | /**
56 |  * Provider that enforces "at most one password field revealed at a time"
57 |  * across all descendant `PasswordInput` fields via the pure reducer.
58 |  */
59 | export const PasswordVisibilityGroup = ({
60 |     children,
61 | }: {
62 |     children: ReactNode;
63 | }) => {
64 |     const [state, dispatch] = useReducer(
65 |         passwordVisibilityReducer,
66 |         initialPasswordVisibilityState
67 |     );
68 |
69 |     const value = useMemo<PasswordVisibilityContextValue>(
70 |         () => ({
71 |             isRevealed: (id: string) => isFieldRevealed(state, id),
72 |             reveal: (id: string) => dispatch({ type: "reveal", id }),

```

```

73 |         hide: (id: string) => dispatch({ type: "hide", id }),
74 |     },
75 |     [state]
76 | );
77 |
78 | return (
79 |   <PasswordVisibilityContext.Provider value={value}>
80 |     {children}
81 |   </PasswordVisibilityContext.Provider>
82 | );
83 | };
84 |

```

FILE: src/components/shared/PeerTeachingMomentCard.tsx

TYPE: .tsx | LINES: 1-136 | SHA256: 92063281af3c395a63ad21ab6c7cb6f98b943c6c805bc2c3c37f8bf3bf390cc

```

1 | // =====
2 | // PeerTeachingMomentCard – Teaching moment display
3 | // Task 4.10: title, text, media link, average ratings
4 | // =====
5 |
6 | import { Card } from "@components/ui/card";
7 | import { Badge } from "@components/ui/badge";
8 | import { cn } from "@lib/Utils";
9 | import { BookOpen, ExternalLink, Star, Eye } from "lucide-react";
10 | import { formatDistanceToNow } from "date-fns";
11 |
12 | export interface PeerTeachingMomentCardProps {
13 |   title: string;
14 |   explanationText: string;
15 |   mediaUrl?: string | null;
16 |   authorName: string;
17 |   cloTitle?: string;
18 |   createdAt: string;
19 |   avgClarityRating?: number;
20 |   avgHelpfulnessRating?: number;
21 |   totalRatings?: number;
22 |   viewCount?: number;
23 |   className?: string;
24 |   onView?: () => void;
25 | }
26 |
27 | function renderStars(rating: number): string {
28 |   const rounded = Math.round(rating * 10) / 10;
29 |   return `${rounded.toFixed(1)} `;
30 | }
31 |
32 | const PeerTeachingMomentCard = ({
33 |   title,
34 |   explanationText,
35 |   mediaUrl,
36 |   authorName,
37 |   cloTitle,
38 |   createdAt,
39 |   avgClarityRating,
40 |   avgHelpfulnessRating,
41 |   totalRatings = 0,
42 |   viewCount = 0,
43 |   className,
44 |   onView,
45 | }: PeerTeachingMomentCardProps) => (
46 |   <Card
47 |     className={cn(
48 |       "bg-white border-0 shadow-md rounded-xl p-4",
49 |       onView && "cursor-pointer hover:shadow-lg transition-shadow",
50 |       className
51 |     )}
52 |     onClick={onView}
53 |     role={onView ? "button" : undefined}
54 |     tabIndex={onView ? 0 : undefined}
55 |     onKeyDown={
56 |       onView
57 |       ? (e) => {
58 |         if (e.key === "Enter" || e.key === " ") {
59 |           e.preventDefault();
60 |           onView();
61 |         }
62 |       }
63 |       : undefined
64 |     }
65 |   >
66 |     { /* Header */ }
67 |     <div className="flex items-start gap-3">
68 |       <div className="p-2 rounded-lg bg-blue-50 shrink-0">
69 |         <BookOpen className="h-4 w-4 text-blue-600" />
70 |       </div>
71 |       <div className="flex-1 min-w-0">
72 |         <h4 className="text-sm font-bold truncate">{title}</h4>
73 |         <p className="text-xs text-gray-500 mt-0.5">
74 |           by {authorName} · { " " }
75 |           {formatDistanceToNow(new Date(createdAt), { addSuffix: true })}
76 |         </p>
77 |       </div>
78 |       {cloTitle && (
79 |         <Badge variant="outline" className="text-[10px] shrink-0">
80 |           {cloTitle}
81 |         </Badge>
82 |       )}
83 |     </div>
84 |
85 |     { /* Explanation text */ }
86 |     <p className="text-sm text-gray-700 mt-3 line-clamp-3 leading-relaxed">
87 |       {explanationText}
88 |     </p>

```

```

89 |
90 |     { /* Media link */
91 |     {mediaUrl && (
92 |         <a
93 |             href={mediaUrl}
94 |             target="_blank"
95 |             rel="noopener noreferrer"
96 |             className="inline-flex items-center gap-1 text-xs text-blue-600 hover:underline mt-2"
97 |             onClick={(e) => e.stopPropagation()}
98 |         >
99 |             <ExternalLink className="h-3 w-3" />
100 |             View media
101 |         </a>
102 |     )}
103 |
104 |     { /* Footer: ratings + views */
105 |     <div className="flex items-center gap-4 mt-3 pt-3 border-t border-gray-100">
106 |         {avgClarityRating !== undefined && (
107 |         <div className="flex items-center gap-1 text-xs text-gray-500">
108 |             <Star className="h-3.5 w-3.5 text-amber-400 fill-amber-400" />
109 |             <span className="font-medium">
110 |                 Clarity: {renderStars(avgClarityRating)}
111 |             </span>
112 |         </div>
113 |         )}
114 |         {avgHelpfulnessRating !== undefined && (
115 |         <div className="flex items-center gap-1 text-xs text-gray-500">
116 |             <Star className="h-3.5 w-3.5 text-blue-400 fill-blue-400" />
117 |             <span className="font-medium">
118 |                 Helpful: {renderStars(avgHelpfulnessRating)}
119 |             </span>
120 |         </div>
121 |         )}
122 |         {totalRatings > 0 && (
123 |         <span className="text-[10px] text-gray-400">
124 |             ({totalRatings} rating{totalRatings !== 1 ? "s" : ""})
125 |         </span>
126 |         )}
127 |         <div className="ms-auto flex items-center gap-1 text-xs text-gray-400">
128 |             <Eye className="h-3 w-3" />
129 |             {viewCount}
130 |         </div>
131 |     </div>
132 | </Card>
133 | );
134 |
135 | export default PeerTeachingMomentCard;
136 |

```

FILE: [src/components/shared/PeerTeachingMomentForm.tsx](#)

TYPE: .tsx | LINES: 1-159 | SHA256: aa3feb74483dde3928adff6fdf39c5faf11b28c77c5bea8380c5cd82f072cb52

```

1 | // =====
2 | // PeerTeachingMomentForm – Form to create a teaching moment
3 | // Task 4.11: title, text 50-500 chars, optional media URL, Zod validation
4 | // =====
5 |
6 | import { useForm } from "react-hook-form";
7 | import { zodResolver } from "hookform/resolvers/zod";
8 | import {
9 |     createTeachingMomentSchema,
10 |     type CreateTeachingMomentInput,
11 | } from "@lib/schemas/peerTeaching";
12 | import {
13 |     Form,
14 |     FormField,
15 |     FormItem,
16 |     FormLabel,
17 |     FormControl,
18 |     FormMessage,
19 |     FormDescription,
20 | } from "@components/ui/form";
21 | import { Input } from "@components/ui/input";
22 | import { Textarea } from "@components/ui/textarea";
23 | import { Button } from "@components/ui/button";
24 | import { Loader2, BookOpen } from "lucide-react";
25 |
26 | export interface PeerTeachingMomentFormProps {
27 |     teamId: string;
28 |     cloId: string;
29 |     onSubmit: (data: CreateTeachingMomentInput) => void;
30 |     isPending?: boolean;
31 |     className?: string;
32 | }
33 |
34 | const PeerTeachingMomentForm = ({
35 |     teamId,
36 |     cloId,
37 |     onSubmit,
38 |     isPending = false,
39 |     className,
40 | }: PeerTeachingMomentFormProps) => {
41 |     const form = useForm<CreateTeachingMomentInput>({
42 |         resolver: zodResolver(createTeachingMomentSchema),
43 |         defaultValues: {
44 |             team_id: teamId,
45 |             clo_id: cloId,
46 |             title: "",
47 |             explanation_text: "",
48 |             media_url: null,
49 |         },
50 |     });
51 |
52 |     const explanationText = form.watch("explanation_text");

```



```

53 | const charCount = explanationText?.length ?? 0;
54 |
55 | const handleSubmit = (data: CreateTeachingMomentInput) => {
56 |   // Normalize empty media_url to null
57 |   const normalized = {
58 |     ...data,
59 |     media_url: data.media_url?.trim() || null,
60 |   };
61 |   onSubmit(normalized);
62 | };
63 |
64 | return (
65 |   <div className={className}>
66 |     <Form {...form}>
67 |       <form onSubmit={form.handleSubmit(handleSubmit)} className="space-y-4">
68 |         </* Title */>
69 |         <FormField
70 |           control={form.control}
71 |           name="title"
72 |           render={({ field }) => (
73 |             <FormItem>
74 |               <FormLabel>Title</FormLabel>
75 |               <FormControl>
76 |                 <Input
77 |                   placeholder="e.g., Understanding Binary Search"
78 |                   {...field}
79 |                 />
80 |               </FormControl>
81 |               <FormMessage />
82 |             </FormItem>
83 |           )}
84 |         />
85 |
86 |         </* Explanation Text */>
87 |         <FormField
88 |           control={form.control}
89 |           name="explanation_text"
90 |           render={({ field }) => (
91 |             <FormItem>
92 |               <FormLabel>Explanation</FormLabel>
93 |               <FormControl>
94 |                 <Textarea
95 |                   placeholder="Explain the concept in your own words (50-500 characters)..."
96 |                   rows={5}
97 |                   {...field}
98 |                 />
99 |               </FormControl>
100 |               <div className="flex items-center justify-between">
101 |                 <FormMessage />
102 |                 <FormDescription className="text-end tabular-nums">
103 |                   {charCount}/500
104 |                 </FormDescription>
105 |               </div>
106 |             </FormItem>
107 |           )}
108 |         />
109 |
110 |         </* Media URL (optional) */>
111 |         <FormField
112 |           control={form.control}
113 |           name="media_url"
114 |           render={({ field }) => (
115 |             <FormItem>
116 |               <FormLabel>Media Link (optional)</FormLabel>
117 |               <FormControl>
118 |                 <Input
119 |                   type="url"
120 |                   placeholder="https://..."
121 |                   {...field}
122 |                   value={field.value ?? ""}
123 |                   onChange={(e) => field.onChange(e.target.value || null)}
124 |                 />
125 |               </FormControl>
126 |               <FormDescription>
127 |                 Link to a video, audio, or diagram that supports your
128 |                 explanation
129 |               </FormDescription>
130 |               <FormMessage />
131 |             </FormItem>
132 |           )}
133 |         />
134 |
135 |         </* Hidden fields */>
136 |         <input type="hidden" {...form.register("team_id")} />
137 |         <input type="hidden" {...form.register("clo_id")} />
138 |
139 |         <Button
140 |           type="submit"
141 |           disabled={isPending}
142 |           variant="tactile"
143 |           className="gap-1.5"
144 |         >
145 |           {isPending ? (
146 |             <Loader2 className="h-4 w-4 animate-spin" />
147 |           ) : (
148 |             <BookOpen className="h-4 w-4" />
149 |           )}
150 |           Share Teaching Moment
151 |         </Button>
152 |       </form>
153 |     </Form>
154 |   </div>
155 | );
156 | };

```

```
157 |
158 | export default PeerTeachingMomentForm;
159 |
```

FILE: src/components/shared/PersonalBestCard.tsx

TYPE: .tsx | LINES: 1-89 | SHA256: 5e649c4c95fb5e4ebafab197898e3f6c611d7ec4b40b39bf6b32da6d496f7db4

```
1  // =====
2  // PersonalBestCard – Metric comparison card with delta arrows
3  // Task 23.1
4  // =====
5
6  import { Card } from "@components/ui/card";
7  import { TrendingUp, TrendingDown, Minus } from "lucide-react";
8
9  interface PersonalBestCardProps {
10    label: string;
11    currentValue: number;
12    previousValue: number;
13    unit?: string;
14    isPersonalBest?: boolean;
15  }
16
17  const PersonalBestCard = ({
18    label,
19    currentValue,
20    previousValue,
21    unit = "xp",
22    isPersonalBest = false,
23  }: PersonalBestCardProps) => {
24    const delta = currentValue - previousValue;
25    const deltaPercent =
26      previousValue > 0 ? Math.round((delta / previousValue) * 100) : 0;
27
28    return (
29      <Card
30        className={`bg-white border-0 shadow-md rounded-xl p-4 ${
31          isPersonalBest ? "ring-2 ring-amber-400" : ""
32        }`}
33        data-testid="personal-best-card"
34      >
35        <p className="text-[10px] font-black tracking-widest uppercase text-gray-500">
36          {label}
37        </p>
38        <div className="flex items-end gap-2 mt-1">
39          <p className="text-2xl font-black">{currentValue.toLocaleString()}</p>
40          <span className="text-xs text-gray-400 mb-1">{unit}</span>
41        </div>
42
43        <div className="flex items-center gap-1 mt-2">
44          {delta > 0 ? (
45            <>
46              <TrendingUp
47                className="h-3 w-3 text-green-500"
48                data-testid="delta-up"
49              />
50              <span className="text-xs font-bold text-green-600">
51                +{delta.toLocaleString()} ({deltaPercent}%)
52              </span>
53            </>
54          ) : delta < 0 ? (
55            <>
56              <TrendingDown
57                className="h-3 w-3 text-red-500"
58                data-testid="delta-down"
59              />
60              <span className="text-xs font-bold text-red-600">
61                {delta.toLocaleString()} ({deltaPercent}%)
62              </span>
63            </>
64          ) : (
65            <>
66              <Minus
67                className="h-3 w-3 text-gray-400"
68                data-testid="delta-neutral"
69              />
70              <span className="text-xs font-bold text-gray-500">No change</span>
71            </>
72          )}
73          <span className="text-[10px] text-gray-400 ms-1">vs previous</span>
74        </div>
75
76        {isPersonalBest && (
77          <p
78            className="text-[10px] font-bold text-amber-600 mt-1"
79            data-testid="personal-best-label"
80          >
81            +P Personal Best!
82          </p>
83        )}
84      </Card>
85    );
86  };
87
88  export default PersonalBestCard;
89
```

FILE: src/components/shared/PlagiarismPlaceholder.tsx

TYPE: .tsx | LINES: 1-33 | SHA256: 68a4c21195d06e799bb497d3b9931ae378f9064d1e6acbe007617e7022217e75

```
1 | import { Badge } from "@components/ui/badge";
2 | import { ShieldAlert } from "lucide-react";
3 |
4 | interface PlagiarismPlaceholderProps {
5 |   score?: number | null;
6 | }
7 |
8 | const PlagiarismPlaceholder = ({ score }: PlagiarismPlaceholderProps) => {
9 |   if (score !== null) {
10 |     const color =
11 |       score > 30
12 |         ? "text-red-600 bg-red-50"
13 |         : score > 10
14 |         ? "text-yellow-600 bg-yellow-50"
15 |         : "text-green-600 bg-green-50";
16 |     return (
17 |       <Badge variant="outline" className={` ${color} gap-1`>
18 |         <ShieldAlert className="h-3 w-3" />
19 |         Similarity: {score.toFixed(0)}%
20 |       </Badge>
21 |     );
22 |   }
23 |
24 |   return (
25 |     <Badge variant="outline" className="text-slate-400 bg-slate-50 gap-1">
26 |       <ShieldAlert className="h-3 w-3" />
27 |       Plagiarism check: Not configured
28 |     </Badge>
29 |   );
30 | };
31 |
32 | export default PlagiarismPlaceholder;
33 |
```

FILE: src/components/shared/PlannerTaskItem.tsx

TYPE: .tsx | LINES: 1-157 | SHA256: 20bf65391d1a9b98472b79920877320bba92a3153f822cf0b6cf0d28de8ec23c

```
1 | // =====
2 | // PlannerTaskItem - Task item with checkbox, title, priority badge,
3 | // course tag, edit/delete actions
4 | // =====
5 |
6 | import { Badge } from "@components/ui/badge";
7 | import { Button } from "@components/ui/button";
8 | import { Checkbox } from "@components/ui/checkbox";
9 | import { cn } from "@lib/Utils";
10 | import type { PlannerTask } from "@types/planner";
11 | import { Pencil, Trash2 } from "lucide-react";
12 |
13 | interface PlannerTaskItemProps {
14 |   task: PlannerTask;
15 |   onToggle?: (task: PlannerTask) => void;
16 |   onEdit?: (task: PlannerTask) => void;
17 |   onDelete?: (task: PlannerTask) => void;
18 |   compact?: boolean;
19 | }
20 |
21 | const PRIORITY_STYLES: Record<
22 |   PlannerTask["priority"],
23 |   { label: string; className: string }
24 | > = {
25 |   high: { label: "High", className: "bg-red-100 text-red-700" },
26 |   medium: { label: "Med", className: "bg-amber-100 text-amber-700" },
27 |   low: { label: "Low", className: "bg-gray-100 text-gray-600" },
28 | };
29 |
30 | /**
31 |  * The DB-valid planner task status set (matches the `planner_tasks_status_check`
32 |  * constraint: todo/in_progress/done/deferred). Used as a defensive guard so an
33 |  * unexpected persisted value falls back to "todo" instead of yielding an
34 |  * undefined badge config.
35 |  */
36 | const PLANNER_STATUSES = ["todo", "in_progress", "done", "deferred"] as const;
37 |
38 | const STATUS_STYLES: Record<
39 |   PlannerTask["status"],
40 |   { label: string; className: string }
41 | > = {
42 |   todo: { label: "To Do", className: "bg-slate-100 text-slate-600" },
43 |   in_progress: {
44 |     label: "In Progress",
45 |     className: "bg-blue-100 text-blue-700",
46 |   },
47 |   done: { label: "Done", className: "bg-green-100 text-green-700" },
48 |   deferred: { label: "Deferred", className: "bg-amber-100 text-amber-700" },
49 | };
50 |
51 | const PlannerTaskItem = ({
52 |   task,
53 |   onToggle,
54 |   onEdit,
55 |   onDelete,
56 |   compact = false,
57 | }: PlannerTaskItemProps) => {
58 |   const isCompleted = task.status === "done";
59 |   const priorityConfig = PRIORITY_STYLES[task.priority];
60 |   const safeStatus = PLANNER_STATUSES.includes(task.status)
61 |     ? task.status
62 |     : "todo";
63 |   const statusConfig = STATUS_STYLES[safeStatus];
```

```

64 |
65 | return (
66 |   <div
67 |     className={cn(
68 |       "group flex items-start gap-2 rounded-lg border border-gray-200 bg-white p-2.5 transition-shadow hover
69 |       isCompleted && "opacity-60",
70 |       compact && "p-2"
71 |     )}
72 |   >
73 |     { /* Checkbox */ }
74 |     <div className="pt-0.5">
75 |       <Checkbox
76 |         checked={isCompleted}
77 |         disabled={isCompleted}
78 |         onChange={() => onToggle?.(task)}
79 |         className="min-h-[20px] min-w-[20px]"
80 |         aria-label={ `Mark "${task.title}" as ${
81 |           isCompleted ? "incomplete" : "complete"
82 |         }` }
83 |       />
84 |     </div>
85 |
86 |     { /* Content */ }
87 |     <div className="flex-1 min-w-0">
88 |       <div className="flex items-start justify-between gap-1">
89 |         <span
90 |           className={cn(
91 |             "text-xs font-medium text-gray-900 line-clamp-1",
92 |             isCompleted && "line-through text-gray-500"
93 |           )}
94 |         >
95 |           {task.title}
96 |         </span>
97 |         <Badge
98 |           className={cn(
99 |             "shrink-0 text-[10px] px-1.5 py-0",
100 |            priorityConfig.className
101 |          )}
102 |         >
103 |           {priorityConfig.label}
104 |         </Badge>
105 |       </div>
106 |
107 |       <div className="mt-0.5 flex items-center gap-1">
108 |         <Badge
109 |           className={cn(
110 |             "shrink-0 text-[10px] px-1.5 py-0",
111 |             statusConfig.className
112 |           )}
113 |         >
114 |           {statusConfig.label}
115 |         </Badge>
116 |         {task.courseName && (
117 |           <span className="block text-[11px] text-gray-500 line-clamp-1">
118 |             {task.courseName}
119 |           </span>
120 |         )}
121 |       </div>
122 |     </div>
123 |
124 |     { /* Actions – visible on hover */ }
125 |     { !isCompleted && !compact && (
126 |       <div className="flex shrink-0 items-center gap-0.5 opacity-0 transition-opacity group-hover:opacity-10
127 |         {onEdit && (
128 |           <Button
129 |             variant="ghost"
130 |             size="sm"
131 |             className="h-7 w-7 p-0"
132 |             onClick={() => onEdit(task)}
133 |             aria-label={ `Edit "${task.title}"` }
134 |           >
135 |             <Pencil className="h-3.5 w-3.5 text-gray-500" />
136 |           </Button>
137 |         )}
138 |         {onDelete && (
139 |           <Button
140 |             variant="ghost"
141 |             size="sm"
142 |             className="h-7 w-7 p-0"
143 |             onClick={() => onDelete(task)}
144 |             aria-label={ `Delete "${task.title}"` }
145 |           >
146 |             <Trash2 className="h-3.5 w-3.5 text-red-500" />
147 |           </Button>
148 |         )}
149 |       </div>
150 |     )}
151 |   </div>
152 | );
153 | };
154 |
155 | export default PlannerTaskItem;
156 | export type { PlannerTaskItemProps };
157 |

```

FILE: [src/components/shared/PL0DrillDownDialog.tsx](#)

TYPE: .tsx | LINES: 1-125 | SHA256: d594ac7a139014d81d7b3f3e4eba5a67645777a40c6999da5a2229a687e39316

```

1 | // =====
2 | // PL0DrillDownDialog – read-only drill-down for an Admin PLO heatmap cell
3 | // Feature: qa-partner-review-remediation – Req 7.3
4 | // =====
5 | // Opened when an administrator selects a PLO cell in the heatmap. Lists the
6 | // contributing CLOs/courses that roll up into the selected PLO, each with its

```

```

7 // own attainment value rendered using the platform attainment-level color coding.
8 // =====
9
10 import { useTranslation } from "react-i18next";
11 import {
12   Dialog,
13   DialogContent,
14   DialogDescription,
15   DialogHeader,
16   DialogTitle,
17 } from "@components/ui/dialog";
18 import { Badge } from "@components/ui/badge";
19 import { InlineEmpty } from "@components/shared/EmptyState";
20 import {
21   getAttainmentBadgeStyle,
22   classifyAttainment,
23 } from "@lib/attainmentClassifier";
24 import { Target } from "lucide-react";
25 import {
26   PLO_ATTAINMENT_UNMEASURED,
27   type AdminPLOHeatmapRow,
28 } from "@hooks/useAdminPLOHeatmap";
29
30 interface PLODrillDownDialogProps {
31   plo: AdminPLOHeatmapRow | null;
32   open: boolean;
33   onOpenChange: (open: boolean) => void;
34 }
35
36 const formatPercent = (percent: number): string =>
37   percent === PLO_ATTAINMENT_UNMEASURED ? "-" : `${Math.round(percent)}%`;
38
39 const PLODrillDownDialog = ({
40   plo,
41   open,
42   onOpenChange,
43 }: PLODrillDownDialogProps) => {
44   const { t } = useTranslation("admin");
45
46   return (
47     <Dialog open={open} onOpenChange={onOpenChange}>
48       <DialogContent className="sm:max-w-2xl">
49         <DialogHeader>
50           <DialogTitle className="flex items-center gap-2">
51             <Target className="h-5 w-5 text-blue-600" />
52             {plo?.plo_title ?? t("dashboard.ploDrillDown.title")}
53           </DialogTitle>
54           <DialogDescription>
55             {plo
56               ? t("dashboard.ploDrillDown.subtitle", {
57                 value: formatPercent(plo.attainment_percent),
58                 method: t(`dashboard.ploDrillDown.method.${plo.derivation}`),
59               })
60               : t("dashboard.ploDrillDown.title")}
61           </DialogDescription>
62         </DialogHeader>
63
64         {!plo || plo.contributors.length === 0 ? (
65           <InlineEmpty
66             icon={<Target className="h-6 w-6 text-gray-400" />}
67             title={t("dashboard.ploDrillDown.emptyTitle")}
68             description={t("dashboard.ploDrillDown.emptyDescription")}
69           </InlineEmpty>
70         ) : (
71           <div className="max-h-[60vh] overflow-y-auto">
72             <table className="w-full text-sm border-collapse">
73               <thead>
74                 <tr className="border-b border-slate-200">
75                   <th className="p-2 text-start font-bold text-xs uppercase tracking-wide text-gray-500">
76                     {t("dashboard.ploDrillDown.cloColumn")}
77                   </th>
78                   <th className="p-2 text-start font-bold text-xs uppercase tracking-wide text-gray-500">
79                     {t("dashboard.ploDrillDown.courseColumn")}
80                   </th>
81                   <th className="p-2 text-end font-bold text-xs uppercase tracking-wide text-gray-500">
82                     {t("dashboard.ploDrillDown.attainmentColumn")}
83                   </th>
84                 </tr>
85               </thead>
86               <tbody>
87                 {plo.contributors.map((contributor) => (
88                   <tr>
89                     <td key={contributor.clo_id}
90                       className="border-b border-slate-100 last:border-0">
91                       <td className="p-2 font-medium text-gray-700">
92                         {contributor.clo_title}
93                       </td>
94                       <td className="p-2 text-gray-500">
95                         {contributor.course_name ?? "-"}
96                       </td>
97                       <td className="p-2 text-end">
98                         <Badge
99                           variant="outline"
100                           className={getAttainmentBadgeStyle(
101                             contributor.attainment_percent
102                           )}
103                         >
104                           {Math.round(contributor.attainment_percent)}% {" "}
105                           {t(
106                             "dashboard.ploDrillDown.level.${classifyAttainment(
107                               contributor.attainment_percent
108                             )}"
109                           )}
110                         </td>
111                     </tr>
112                   )}
113                 </tbody>
114             </table>
115           </div>
116         )}
117       </DialogContent>
118     </Dialog>
119   );
120 }

```

```

111 |                 </Badge>
112 |             </td>
113 |         </tr>
114 |     </tbody>
115 | </table>
116 | </div>
117 | </DialogContent>
118 | </Dialog>
119 | );
120 | };
121 | export default PLODrillDownDialog;
122 |
123 |
124 |
125 |

```

FILE: [src/components/shared/PomodoroIndicator.tsx](#)

TYPE: .tsx | LINES: 1-96 | SHA256: a397e35a92249dbca1e9d6a0e312ce92fd22c06136d04a92efe09ba827541986

```

1 | // =====
2 | // PomodoroIndicator – Shows current interval number, type (Work/Break/Long
3 | // Break), and progress dots for completed intervals
4 | // =====
5 |
6 | import { cn } from "@lib/utils";
7 | import type { PomodoroIntervalType } from "@types/planner";
8 |
9 | interface PomodoroIndicatorProps {
10 |   /** Current interval index (0-based) */
11 |   currentInterval: number;
12 |   /** Type of the current interval */
13 |   intervalType: PomodoroIntervalType;
14 |   /** Total completed work intervals in this cycle (max 4 before long break) */
15 |   className?: string;
16 | }
17 |
18 | const INTERVAL_LABELS: Record<PomodoroIntervalType, string> = {
19 |   work: "Work",
20 |   break: "Break",
21 |   long_break: "Long Break",
22 | };
23 |
24 | const INTERVAL_COLORS: Record<PomodoroIntervalType, string> = {
25 |   work: "text-blue-600",
26 |   break: "text-teal-600",
27 |   long_break: "text-amber-600",
28 | };
29 |
30 | /**
31 |  * Compute how many work intervals have been completed in the current
32 |  * Pomodoro cycle (a cycle = 4 work intervals before a long break).
33 |  *
34 |  * Interval sequence: work(0), break(1), work(2), break(3), work(4), break(5), work(6), long_break(7)
35 |  * Work intervals are at even indices: 0, 2, 4, 6
36 |  * So completed work intervals = floor(currentInterval / 2) within the cycle.
37 |  */
38 | function getCompletedWorkIntervals(currentInterval: number): number {
39 |   // Each cycle is 8 intervals (4 work + 3 break + 1 long break)
40 |   const posInCycle = currentInterval % 8;
41 |   // Work intervals are at positions 0, 2, 4, 6
42 |   // Completed work = how many even positions are strictly before current
43 |   return Math.floor(posInCycle / 2);
44 | }
45 |
46 | function getCurrentWorkNumber(currentInterval: number): number {
47 |   return getCompletedWorkIntervals(currentInterval) + 1;
48 | }
49 |
50 | const PomodoroIndicator = ({
51 |   currentInterval,
52 |   intervalType,
53 |   className,
54 | }: PomodoroIndicatorProps) => {
55 |   const completedWork = getCompletedWorkIntervals(currentInterval);
56 |   const currentWorkNum = getCurrentWorkNumber(currentInterval);
57 |   const label = INTERVAL_LABELS[intervalType];
58 |   const colorClass = INTERVAL_COLORS[intervalType];
59 |
60 |   return (
61 |     <div
62 |       className={cn("flex flex-col items-center gap-2", className)}
63 |       role="status"
64 |       aria-label={`Pomodoro ${currentWorkNum} of 4, ${label}`}
65 |     >
66 |       {/** Interval label */}
67 |       <span className={cn("text-sm font-semibold", colorClass)}>
68 |         {intervalType === "work" ? `Pomodoro ${currentWorkNum} of 4` : label}
69 |       </span>
70 |
71 |       {/** Progress dots */}
72 |       <div className="flex items-center gap-1.5" aria-hidden="true">
73 |         {Array.from({ length: 4 }).map((_, i) => {
74 |           const isCompleted = i < completedWork;
75 |           const isCurrent = i === completedWork && intervalType === "work";
76 |
77 |           return (
78 |             <div
79 |               key={i}
80 |               className={cn(
81 |                 "h-2.5 w-2.5 rounded-full transition-colors",
82 |                 isCompleted && "bg-teal-500",
83 |                 isCurrent && "bg-blue-500 ring-2 ring-blue-200",
84 |                 !isCompleted && !isCurrent && "bg-gray-200"
85 |               )}

```

```

86 |         );
87 |     });
88 |     </div>
89 | </div>
90 | );
91 | };
92 | };
93 |
94 | export default PomodoroIndicator;
95 | export type { PomodoroIndicatorProps };
96 |

```

FILE: src/components/shared/PracticeModeBanner.tsx

TYPE: .tsx | LINES: 1-26 | SHA256: f7a675cf873dbf9629eaaeca09f9d3313d59472c4fe737b55a1ee68fb87d4f2f

```

1 | import { Info } from "lucide-react";
2 | import { cn } from "@lib/Utils";
3 |
4 | export interface PracticeModeBannerProps {
5 |   className?: string;
6 | }
7 |
8 | const PracticeModeBanner = ({ className }: PracticeModeBannerProps) => {
9 |   return (
10 |     <div
11 |       className={cn(
12 |         "flex items-center gap-2 rounded-lg border border-blue-200 bg-blue-50 px-4 py-3 text-sm font-medium te
13 |         className
14 |       )}
15 |       role="status"
16 |       aria-label="Practice Mode"
17 |     >
18 |       <Info className="h-4 w-4 shrink-0" />
19 |       Practice Mode – This attempt will not affect your grades
20 |     </div>
21 |   );
22 | };
23 |
24 | export { PracticeModeBanner };
25 | export default PracticeModeBanner;
26 |

```

FILE: src/components/shared/PracticeModeToggle.tsx

TYPE: .tsx | LINES: 1-38 | SHA256: 54453bbfb28cf5649dbf4de69ac077bdf7f1d7016c0ec2119e289168ca6838d1

```

1 | import { Switch } from "@components/ui/switch";
2 | import { Label } from "@components/ui/label";
3 |
4 | export interface PracticeModeToggleProps {
5 |   checked: boolean;
6 |   onCheckedChange: (checked: boolean) => void;
7 |   disabled?: boolean;
8 | }
9 |
10 | const PracticeModeToggle = ({
11 |   checked,
12 |   onCheckedChange,
13 |   disabled,
14 | }: PracticeModeToggleProps) => {
15 |   return (
16 |     <div className="flex items-center justify-between gap-4 rounded-lg border border-slate-200 p-4">
17 |       <div className="space-y-0.5">
18 |         <Label htmlFor="practice-mode" className="text-sm font-medium">
19 |           Allow Practice Mode
20 |         </Label>
21 |         <p className="text-xs text-gray-500">
22 |           Students can take this quiz without grade impact
23 |         </p>
24 |       </div>
25 |       <Switch
26 |         id="practice-mode"
27 |         checked={checked}
28 |         onCheckedChange={onCheckedChange}
29 |         disabled={disabled}
30 |         aria-label="Allow Practice Mode"
31 |       />
32 |     </div>
33 |   );
34 | };
35 |
36 | export { PracticeModeToggle };
37 | export default PracticeModeToggle;
38 |

```

FILE: src/components/shared/PrimaryCTA.tsx

TYPE: .tsx | LINES: 1-224 | SHA256: b19180015a1c55fced3f6043db41977b8bfe086eb4c0a48711fdbcdcb559baba

```

1 | // =====
2 | // PrimaryCTA – single dominant dashboard call-to-action + subordinate secondaries
3 | // =====
4 | //
5 | // Presentational component (no Supabase, no data fetching). It consumes the
6 | // pure `primaryCtaSelector` business logic to choose the single highest-priority
7 | // applicable action and renders it as the one dominant CTA, with the remaining
8 | // applicable actions rendered as a visually subordinate secondary-actions row.
9 | //
10 | // All user-facing copy (labels, descriptions, region label) is supplied by the
11 | // consumer already localized, keeping this component bilingual-ready while the
12 | // owning page (StudentDashboard) performs the i18next lookup. This satisfies the
13 | // label-availability requirement (R16.5) at the composition boundary.
14 | //
15 | // Design: Area K – Dashboard single PrimaryCTA
16 | // Requirements: 16.1, 16.4, 16.5

```

[illegible]

[illegible]

FILE: src/components/shared/ProfileCompletenessBar.tsx

TYPE: .tsx | LINES: 1-55 | SHA256: 2f7b551065e1e2a5f8fdb1b8a10588496ce46f9a63a96a7ac7c3bc65c4a92f08

```
1 | import { useNavigate } from "react-router-dom";
2 | import { Badge } from "@components/ui/badge";
3 | import { CheckCircle, ChevronRight } from "lucide-react";
4 |
5 | export interface ProfileCompletenessBarProps {
6 |   completeness: number;
7 | }
8 |
9 | const ProfileCompletenessBar = ({
10 |   completeness,
11 | }: ProfileCompletenessBarProps) => {
12 |   const navigate = useNavigate();
13 |   const clamped = Math.max(0, Math.min(100, completeness));
14 |
15 |   if (clamped >= 100) {
16 |     return (
17 |       <Badge className="bg-green-100 text-green-700 border-green-200 gap-1">
18 |         <CheckCircle className="h-3 w-3" />
19 |         Profile Complete
20 |       </Badge>
21 |     );
22 |   }
23 |
24 |   return (
25 |     <button
26 |       type="button"
27 |       onClick={() => navigate("/student/onboarding/complete-profile")}
28 |       className="w-full rounded-xl bg-white shadow-md p-4 flex items-center gap-3 hover:shadow-lg transition-s
29 |       aria-label={`Profile ${clamped}% complete. Tap to complete your profile.`}
30 |     >
31 |       <div className="flex-1 min-w-0 space-y-1.5">
32 |         <div className="flex items-center justify-between">
33 |           <span className="text-xs font-bold text-gray-700">
34 |             Profile Completeness
35 |           </span>
36 |           <span className="text-xs font-bold text-blue-600">{clamped}%</span>
37 |         </div>
38 |         <div className="h-2 rounded-full bg-slate-100 overflow-hidden">
39 |           <div
40 |             className="h-full rounded-full bg-[linear-gradient(93.65deg,#14b8a6_5.37%,#0382bd_78.89%)] transit
41 |             style={{ width: `${clamped}%` }}
42 |             role="progressbar"
43 |             aria-valuenow={clamped}
44 |             aria-valuemin={0}
45 |             aria-valuemax={100}
46 |           />
47 |         </div>
48 |       </div>
49 |       <ChevronRight className="h-4 w-4 text-gray-400 group-hover:text-gray-600 transition-colors shrink-0" />
50 |     </button>
51 |   );
52 | };
53 |
54 | export default ProfileCompletenessBar;
55 |
```

FILE: src/components/shared/ProfileDropdown.tsx

TYPE: .tsx | LINES: 1-289 | SHA256: d4979c242ca37009d2e25048d6e581ffab27b5f7eb046cbeaef51bbb95860d27

```
1 | import { useAuth } from "@hooks/useAuth";
2 | import { useTheme } from "@hooks/useTheme";
3 | import { useGuidedTour } from "@hooks/useGuidedTour";
4 | import { Avatar, AvatarFallback, AvatarImage } from "@components/ui/avatar";
5 | import {
6 |   DropdownMenu,
7 |   DropdownMenuContent,
8 |   DropdownMenuItem,
9 |   DropdownMenuLabel,
10 |   DropdownMenuSeparator,
11 |   DropdownMenuSub,
12 |   DropdownMenuSubContent,
13 |   DropdownMenuSubTrigger,
14 |   DropdownMenuTrigger,
15 | } from "@components/ui/dropdown-menu";
16 | import {
17 |   ChevronDown,
18 |   LogOut,
19 |   User,
20 |   RotateCcw,
21 |   Sun,
22 |   Moon,
23 |   Monitor,
24 |   Building2,
25 | } from "lucide-react";
26 | import { useNavigate } from "react-router-dom";
27 | import { toast } from "sonner";
28 | import { useTranslation } from "react-i18next";
29 | import { getDisplayFirstName } from "@lib/displayName";
30 | import { useCourses } from "@hooks/useCourses";
31 | import { useLevel } from "@hooks/useLevel";
32 | import { useLinkedChildren } from "@hooks/useParentDashboard";
33 |
34 | const profileSubtitleByRole: Record<string, string> = {
35 |   admin: "header.profileSubtitle.admin",
36 |   coordinator: "header.profileSubtitle.coordinator",
37 |   teacher: "header.profileSubtitle.teacher",
38 |   student: "header.profileSubtitle.student",
39 |   parent: "header.profileSubtitle.parent",
40 | };
41 |
```

```

42 | /**
43 |  * Profile dropdown menu with avatar, name, and actions.
44 |  * Includes: My Profile, Take the tour, Theme selector, Sign out
45 |  *
46 |  * Design: ADR-02, §8.3
47 |  * Requirements: 2.12, 2.13, 2.15, 2.18
48 |  *
49 |  * @example
50 |  * <ProfileDropdown />
51 |  */
52 | const ProfileDropdown = () => {
53 |   const { user, profile, signOut } = useAuth();
54 |   const navigate = useNavigate();
55 |   const { t } = useTranslation("common");
56 |   const { themeMode, setThemeMode } = useTheme();
57 |   const { start: startTour } = useGuidedTour(profile?.role ?? "student");
58 |   const isStudent = profile?.role === "student";
59 |   const isTeacher = profile?.role === "teacher";
60 |   const isParent = profile?.role === "parent";
61 |   const level = useLevel(isStudent ? user?.id : undefined);
62 |   const teacherCourses = useCourses(
63 |     { page: 1, pageSize: 1, teacherId: isTeacher ? user?.id : undefined },
64 |     { enabled: isTeacher }
65 |   );
66 |   const linkedChildren = useLinkedChildren(isParent ? user?.id : undefined, {
67 |     enabled: isParent,
68 |   });
69 |
70 |   if (!user || !profile) {
71 |     return null;
72 |   }
73 |
74 |   const handleSignOut = async () => {
75 |     try {
76 |       await signOut();
77 |       navigate("/login");
78 |       toast.success(t("auth.signedOut"));
79 |     } catch (err) {
80 |       console.error("[ProfileDropdown] Sign out failed:", err);
81 |       toast.error(err instanceof Error ? err.message : t("common.error"));
82 |     }
83 |   };
84 |
85 |   // Get user initials for avatar fallback
86 |   const initials =
87 |     profile.full_name
88 |       ?.split(" ")
89 |       .map((n: string) => n[0])
90 |       .join("")
91 |       .toUpperCase() ?? "U";
92 |
93 |   const defaultAvatarsByRole: Record<string, string> = {
94 |     student:
95 |       "https://images.unsplash.com/photo-1534528741775-53994a69daeb?auto=format&fit=crop&w=120&q=80",
96 |     teacher:
97 |       "https://images.unsplash.com/photo-1573496359142-b8d87734a5a2?auto=format&fit=crop&w=120&q=80",
98 |     coordinator:
99 |       "https://images.unsplash.com/photo-1560250097-0b93528c311a?auto=format&fit=crop&w=120&q=80",
100 |     admin:
101 |       "https://images.unsplash.com/photo-1580489944761-15a19d654956?auto=format&fit=crop&w=120&q=80",
102 |     parent:
103 |       "https://images.unsplash.com/photo-1544005313-94ddf0286df2?auto=format&fit=crop&w=120&q=80",
104 |   };
105 |
106 |   // Get avatar URL with CDN transformation or default role avatar
107 |   const avatarUrl = profile.avatar_url
108 |     ? `${profile.avatar_url}?width=64&height=64&resize=cover`
109 |     : defaultAvatarsByRole[profile.role] ?? defaultAvatarsByRole.student;
110 |
111 |   const profileSubtitle = (() => {
112 |     if (profile.role === "student") {
113 |       return t("header.profileSubtitle.studentLevel", {
114 |         level: level.data?.level ?? 1,
115 |         xp: (level.data?.xpTotal ?? 0).toLocaleString(),
116 |       });
117 |     }
118 |
119 |     if (profile.role === "teacher") {
120 |       return t("header.profileSubtitle.teacherDetail", {
121 |         department: profile.department ?? t("header.profileSubtitle.teacher"),
122 |         count: teacherCourses.data?.count ?? 0,
123 |       });
124 |     }
125 |
126 |     if (profile.role === "parent") {
127 |       const firstChild = linkedChildren.data?.[0];
128 |       return firstChild
129 |         ? t("header.profileSubtitle.guardianOf", {
130 |             name: getDisplayFirstName(firstChild.student_name),
131 |           })
132 |         : t("header.profileSubtitle.parent");
133 |     }
134 |
135 |     return t(
136 |       profileSubtitleByRole[profile.role] ?? "header.profileSubtitle.student"
137 |     );
138 |   })();
139 |
140 |   return (
141 |     <DropdownMenu>
142 |       <DropdownMenuTrigger asChild>
143 |         <button
144 |           type="button"
145 |           className="hdr-profile flex cursor-pointer items-center gap-2.5 rounded-xl p-1.5 transition-colors h

```

```

146 |     data-tour="profile"
147 | >
148 | <Avatar className="h-9 w-9 border border-slate-200 shadow-2xs dark:border-slate-700">
149 |   <AvatarImage src={avatarUrl} alt={profile.full_name ?? "User"} />
150 |   <AvatarFallback className="bg-gradient-to-br from-teal-500 to-blue-600 font-extrabold text-xs text
151 |     {initials}
152 |   </AvatarFallback>
153 | </Avatar>
154 | <span className="hidden min-[640px]:flex flex-col text-start leading-tight">
155 |   <span className="text-xs font-extrabold text-slate-900 dark:text-white">
156 |     {profile.full_name}
157 |   </span>
158 |   <span className="text-[11px] font-bold text-teal-600 dark:text-teal-400">
159 |     {profile.subtitle}
160 |   </span>
161 |   {isStudent && level.data ? (
162 |     <div
163 |       className="mt-1 h-1.5 w-24 overflow-hidden rounded-full bg-slate-200/80 dark:bg-slate-700/80"
164 |       title={`$
165 |         {level.data.progressPercent
166 |         }% (${level.data.xpTotal.toLocaleString()} / ${level.data.xpForNextLevel.toLocaleString()} XP)
167 |       aria-label={`Level progress ${level.data.progressPercent}%`}
168 |     >
169 |       <div
170 |         className="h-full rounded-full bg-[image:var(--brand-gradient)] transition-all duration-500"
171 |         style={{ width: `${level.data.progressPercent}%` }}
172 |       />
173 |     </div>
174 |   ) : null}
175 |   </span>
176 |   <ChevronDown className="ms-0.5 h-3.5 w-3.5 text-slate-400" />
177 | </button>
178 | </DropdownMenuTrigger>
179 |
180 | <DropdownMenuContent align="end" className="w-56">
181 |   <DropdownMenuLabel className="flex flex-col gap-1">
182 |     <p className="text-sm font-semibold">{profile.full_name ?? "User"}</p>
183 |     <p className="text-xs text-gray-500">{user.email}</p>
184 |   </DropdownMenuLabel>
185 |
186 |   <DropdownMenuSeparator />
187 |
188 |   {/* The dropdown and role navigation share the prototype profile destination. */}
189 |   <DropdownMenuItem
190 |     onClick={() => {
191 |       const routeMap: Record<string, string> = {
192 |         admin: "/admin/settings/profile",
193 |         coordinator: "/coordinator/settings/profile",
194 |         teacher: "/teacher/settings/profile",
195 |         student: "/student/profile",
196 |         parent: "/parent/profile",
197 |       };
198 |       navigate(routeMap[profile.role] ?? "/student/profile");
199 |     }}
200 |     className="cursor-pointer"
201 |     data-tour="settings"
202 |   >
203 |     <User className="h-4 w-4 me-2" />
204 |     <span>{t("myProfile")}</span>
205 |   </DropdownMenuItem>
206 |
207 |   {/* Institution Settings – admin only */}
208 |   {profile.role === "admin" && (
209 |     <DropdownMenuItem
210 |       onClick={() => navigate("/admin/settings/institution")}
211 |       className="cursor-pointer"
212 |     >
213 |       <Building2 className="h-4 w-4 me-2" />
214 |       <span>{t("institutionSettings")}</span>
215 |     </DropdownMenuItem>
216 |   )}
217 |
218 |   {/* Take the tour */}
219 |   <DropdownMenuItem onClick={startTour} className="cursor-pointer">
220 |     <RotateCcw className="h-4 w-4 me-2" />
221 |     <span>{t("tour.takeTour")}</span>
222 |   </DropdownMenuItem>
223 |
224 |   {/* Theme selector */}
225 |   <DropdownMenuSub>
226 |     <DropdownMenuSubTrigger
227 |       className="cursor-pointer"
228 |       aria-label={t("theme.label")}
229 |     >
230 |       <Sun className="h-4 w-4 me-2" />
231 |       <span>{t("theme.label")}</span>
232 |     </DropdownMenuSubTrigger>
233 |     <DropdownMenuSubContent>
234 |       <DropdownMenuItem
235 |         onClick={() => setThemeMode("light")}
236 |         className={`cursor-pointer ${
237 |           themeMode === "light" ? "bg-blue-50" : ""
238 |         }`}
239 |       >
240 |         <Sun className="h-4 w-4 me-2" />
241 |         <span>{t("theme.light")}</span>
242 |         {themeMode === "light" && (
243 |           <span className="ms-auto text-xs">'</span>
244 |         )}
245 |       </DropdownMenuItem>
246 |       <DropdownMenuItem
247 |         onClick={() => setThemeMode("dark")}
248 |         className={`cursor-pointer ${
249 |           themeMode === "dark" ? "bg-blue-50" : ""

```

```

250 |         `}`
251 |     >
252 |         <Moon className="h-4 w-4 me-2" />
253 |         <span>{t("theme.dark")}</span>
254 |         {themeMode === "dark" && (
255 |             <span className="ms-auto text-xs">'</span>
256 |         )}
257 |     </DropdownMenuItem>
258 |     <DropdownMenuItem
259 |         onClick={() => setThemeMode("system")}
260 |         className={`cursor-pointer ${
261 |             themeMode === "system" ? "bg-blue-50" : ""
262 |         }`}
263 |     >
264 |         <Monitor className="h-4 w-4 me-2" />
265 |         <span>{t("theme.system")}</span>
266 |         {themeMode === "system" && (
267 |             <span className="ms-auto text-xs">'</span>
268 |         )}
269 |     </DropdownMenuItem>
270 | </DropdownMenuSubContent>
271 | </DropdownMenuSub>
272 |
273 |     <DropdownMenuSeparator />
274 |
275 |     {/* Sign out */}
276 |     <DropdownMenuItem
277 |         onClick={handleSignOut}
278 |         className="cursor-pointer text-red-600"
279 |     >
280 |         <Logout className="h-4 w-4 me-2" />
281 |         <span>{t("auth.signOut")}</span>
282 |     </DropdownMenuItem>
283 | </DropdownMenuContent>
284 | </DropdownMenu>
285 | );
286 | };
287 |
288 | export default ProfileDropdown;
289 |

```

FILE: [src/components/shared/ProfileSummaryCard.tsx](#)

TYPE: .tsx | LINES: 1-250 | SHA256: aba3a7936e434ed29b48d4383edff6d3fe98ea5fdfdba544928c443f95deec5b

```

1 | import { Card } from "@components/ui/card";
2 | import { Badge } from "@components/ui/badge";
3 | import { Button } from "@components/ui/button";
4 | import { User, AlertTriangle, RefreshCw } from "lucide-react";
5 | import {
6 |     Radar,
7 |     RadarChart,
8 |     PolarGrid,
9 |     PolarAngleAxis,
10 |     PolarRadiusAxis,
11 |     ResponsiveContainer,
12 | } from "recharts";
13 | import { BIG_FIVE_LABELS, VARK_DESCRIPTIONS } from "@lib/onboardingConstants";
14 | import type {
15 |     BigFiveTraits,
16 |     VARKProfile,
17 |     SelfEfficacyProfile,
18 |     StudyStrategyProfile,
19 | } from "@lib/scoreCalculator";
20 |
21 | export interface ProfileSummaryCardProps {
22 |     personalityTraits: BigFiveTraits | null;
23 |     learningStyle: VARKProfile | null;
24 |     selfEfficacy: SelfEfficacyProfile | null;
25 |     studyStrategies: StudyStrategyProfile | null;
26 |     onRetake?: () => void;
27 |     onCompleteRemaining?: () => void;
28 |     hasSkippedSections?: boolean;
29 | }
30 |
31 | const SELF_EFFICACY_LABELS: Record<string, string> = {
32 |     general_academic: "General Academic",
33 |     course_specific: "Course-Specific",
34 |     self_regulated_learning: "Self-Regulated Learning",
35 | };
36 |
37 | const STUDY_STRATEGY_LABELS: Record<string, string> = {
38 |     time_management: "Time Management",
39 |     elaboration: "Elaboration",
40 |     self_testing: "Self-Testing",
41 |     help_seeking: "Help Seeking",
42 | };
43 |
44 | const ScoreBar = ({ label, value }: { label: string; value: number }) => (
45 |     <div className="space-y-1">
46 |         <div className="flex items-center justify-between">
47 |             <span className="text-xs font-medium text-gray-600">{label}</span>
48 |             <span className="text-xs font-bold text-gray-900">{value}%</span>
49 |         </div>
50 |         <div className="h-2 rounded-full bg-slate-100 overflow-hidden">
51 |             <div
52 |                 className="h-full rounded-full bg-gradient-to-r from-blue-500 to-blue-600 transition-all duration-300"
53 |                 style={{ width: `${value}%` }}
54 |                 role="progressbar"
55 |                 aria-valuenow={value}
56 |                 aria-valuemin={0}
57 |                 aria-valuemax={100}
58 |                 aria-label={` ${label}: ${value}%`}
59 |             />
60 |         </div>

```

```

61 | </div>
62 | );
63 |
64 | const ProfileSummaryCard = ({
65 |   personalityTraits,
66 |   learningStyle,
67 |   selfEfficacy,
68 |   studyStrategies,
69 |   onRetake,
70 |   onCompleteRemaining,
71 |   hasSkippedSections = false,
72 | }: ProfileSummaryCardProps) => {
73 |   const radarData = personalityTraits
74 |     ? Object.entries(BIG_FIVE_LABELS).map(([key, label]) => ({
75 |       trait: label,
76 |       score: personalityTraits[key as keyof BigFiveTraits] ?? 0,
77 |     }))
78 |     : [];
79 |
80 |   const varkInfo = learningStyle
81 |     ? VARK_DESCRIPTIONS[learningStyle.dominant_style]
82 |     : null;
83 |
84 |   return (
85 |     <Card className="bg-white border-0 shadow-md rounded-xl overflow-hidden gap-0 py-0">
86 |       <div
87 |         className="px-6 py-4 flex items-center gap-2"
88 |         style={{
89 |           background: "var(--brand-gradient)",
90 |         }}
91 |       >
92 |         <User className="h-5 w-5 text-white" />
93 |         <h2 className="text-lg font-bold tracking-tight text-white">
94 |           My Profile
95 |         </h2>
96 |       </div>
97 |
98 |       <div className="p-6 space-y-6">
99 |         {/* Big Five Radar Chart */}
100 |         {personalityTraits && radarData.length > 0 && (
101 |           <div>
102 |             <h3 className="text-sm font-bold text-gray-900 mb-3">
103 |               Personality Traits
104 |             </h3>
105 |             <div className="h-52">
106 |               <ResponsiveContainer width="100%" height="100%">
107 |                 <RadarChart
108 |                   data={radarData}
109 |                   cx="50%"
110 |                   cy="50%"
111 |                   outerRadius="70%"
112 |                 >
113 |                   <PolarGrid stroke="#e2e8f0" />
114 |                   <PolarAngleAxis
115 |                     dataKey="trait"
116 |                     tick={{ fontSize: 11, fill: "#64748b" }}
117 |                   />
118 |                   <PolarRadiusAxis
119 |                     angle={90}
120 |                     domain={[0, 100]}
121 |                     tick={false}
122 |                     axisLine={false}
123 |                   />
124 |                   <Radar
125 |                     name="Score"
126 |                     dataKey="score"
127 |                     stroke="#3b82f6"
128 |                     fill="#3b82f6"
129 |                     fillOpacity={0.2}
130 |                   />
131 |                 </RadarChart>
132 |               </ResponsiveContainer>
133 |             </div>
134 |           </div>
135 |         )}
136 |
137 |         {/* Self-Efficacy */}
138 |         {selfEfficacy && (
139 |           <div>
140 |             <h3 className="text-sm font-bold text-gray-900 mb-3">
141 |               Self-Efficacy
142 |             </h3>
143 |             <div className="space-y-2">
144 |               {Object.entries(SELF_EFFICACY_LABELS).map(([key, label]) => (
145 |                 <ScoreBar
146 |                   key={key}
147 |                   label={label}
148 |                   value={
149 |                     selfEfficacy[
150 |                       key as keyof Omit<SelfEfficacyProfile, "overall">
151 |                     ] ?? 0
152 |                   }
153 |                 </div>
154 |               ))}
155 |             </div>
156 |           </div>
157 |         )}
158 |
159 |         {/* Study Strategies */}
160 |         {studyStrategies && (
161 |           <div>
162 |             <h3 className="text-sm font-bold text-gray-900 mb-3">
163 |               Study Strategies
164 |             </h3>

```

```

165 |         <div className="space-y-2">
166 |             {Object.entries(STUDY_STRATEGY_LABELS).map(([key, label]) => (
167 |                 <ScoreBar
168 |                     key={key}
169 |                     label={label}
170 |                     value={
171 |                         studyStrategies[key as keyof StudyStrategyProfile] ?? 0
172 |                     }
173 |                 />
174 |             ))}
175 |         </div>
176 |     </div>
177 | )}
178 |
179 | {/* VARK – Self-Awareness Section */}
180 | {learningStyle && varkInfo && (
181 |     <div className="rounded-lg bg-amber-50 p-4 space-y-2">
182 |         <div className="flex items-center gap-2">
183 |             <h3 className="text-sm font-bold text-gray-900">
184 |                 Self-Awareness
185 |             </h3>
186 |             <Badge
187 |                 variant="outline"
188 |                 className="text-[10px] border-amber-300 text-amber-700"
189 |             >
190 |                 For reflection only
191 |             </Badge>
192 |         </div>
193 |         <div className="flex items-center gap-2">
194 |             <span className="text-lg" aria-hidden="true">
195 |                 {varkInfo.icon === "Eye"
196 |                     ? "ðŸ””"
197 |                     : varkInfo.icon === "Headphones"
198 |                     ? "ðŸ”Š"
199 |                     : varkInfo.icon === "BookOpen"
200 |                     ? "ðŸ”Œ"
201 |                     : varkInfo.icon === "Hand"
202 |                     ? "ðŸ”²"
203 |                     : "ðŸ””"}
204 |             </span>
205 |             <div>
206 |                 <p className="text-sm font-bold text-gray-900">
207 |                     {varkInfo.label}
208 |                 </p>
209 |                 <p className="text-xs text-gray-500">{varkInfo.description}</p>
210 |             </div>
211 |         </div>
212 |         <p className="text-[10px] text-amber-600 flex items-start gap-1">
213 |             <AlertTriangle className="h-3 w-3 mt-0.5 shrink-0" />
214 |             Learning style preferences are provided as a self-awareness
215 |             exercise. They are not used for content adaptation.
216 |         </p>
217 |     </div>
218 | )}
219 |
220 | {/* Actions */}
221 | <div className="flex items-center gap-2 pt-2">
222 |     {hasSkippedSections && onCompleteRemaining && (
223 |         <Button
224 |             variant="outline"
225 |             size="sm"
226 |             onClick={onCompleteRemaining}
227 |             className="text-xs"
228 |         >
229 |             Complete Assessment
230 |         </Button>
231 |     )}
232 |     {onRetake && (
233 |         <Button
234 |             variant="ghost"
235 |             size="sm"
236 |             onClick={onRetake}
237 |             className="text-xs text-gray-500"
238 |         >
239 |             <RefreshCw className="h-3 w-3" />
240 |             Retake Assessment
241 |         </Button>
242 |     )}
243 | </div>
244 | </div>
245 | </Card>
246 | );
247 | };
248 |
249 | export default ProfileSummaryCard;
250 |

```

FILE: [src/components/shared/ProgramAccreditationManager.tsx](#)

TYPE: .tsx | LINES: 1-412 | SHA256: 5486723dee42b3a383b3d6e730afa9ed4a829fc080526ec96bbb5069ddc35417

```

1 | import { useState } from "react";
2 | import { useForm } from "react-hook-form";
3 | import { zodResolver } from "@hookform/resolvers/zod";
4 | import { z } from "zod";
5 | import { Card } from "@components/ui/card";
6 | import { Button } from "@components/ui/button";
7 | import { Input } from "@components/ui/input";
8 | import { Badge } from "@components/ui/badge";
9 | import {
10 |     Dialog,
11 |     DialogContent,
12 |     DialogHeader,
13 |     DialogTitle,
14 |     DialogFooter,

```

```

15 | } from "@components/ui/dialog";
16 | import {
17 |   Form,
18 |   FormField,
19 |   FormItem,
20 |   FormLabel,
21 |   FormControl,
22 |   FormMessage,
23 | } from "@components/ui/form";
24 | import {
25 |   Select,
26 |   SelectContent,
27 |   SelectItem,
28 |   SelectTrigger,
29 |   SelectValue,
30 | } from "@components/ui/select";
31 | import { Shimmer } from "@design-system";
32 | import { ConfirmDialog } from "@components/shared/ConfirmDialog";
33 | import {
34 |   useAllProgramAccreditations,
35 |   useCreateProgramAccreditation,
36 |   useDeleteProgramAccreditation,
37 |   type CreateProgramAccreditationInput,
38 | } from "@hooks/useInstitutionSettings";
39 | import { usePrograms } from "@hooks/usePrograms";
40 | import {
41 |   Shield,
42 |   Plus,
43 |   Trash2,
44 |   Loader2,
45 |   AlertTriangle,
46 |   Calendar,
47 | } from "lucide-react";
48 | import { format, differenceInDays } from "date-fns";
49 |
50 | const ACCREDITATION_BODIES = [
51 |   "HEC",
52 |   "QQA",
53 |   "ABET",
54 |   "NCAAA",
55 |   "AACSB",
56 |   "Generic",
57 | ] as const;
58 |
59 | const STATUS_STYLES: Record<string, string> = {
60 |   active: "bg-green-50 text-green-700 border-green-200",
61 |   expired: "bg-red-50 text-red-700 border-red-200",
62 |   pending: "bg-yellow-50 text-yellow-700 border-yellow-200",
63 | };
64 |
65 | const accreditationFormSchema = z.object({
66 |   program_id: z.string().min(1, "Program is required"),
67 |   accreditation_body: z.string().min(1, "Accreditation body is required"),
68 |   framework_version: z.string().optional(),
69 |   accreditation_date: z.string().optional(),
70 |   next_review_date: z.string().optional(),
71 |   status: z.enum(["active", "expired", "pending"]),
72 | });
73 |
74 | type AccreditationFormData = z.infer<typeof accreditationFormSchema>;
75 |
76 | const ProgramAccreditationManager = () => {
77 |   const { data: accreditations, isLoading } = useAllProgramAccreditations();
78 |   const { data: programsResult } = usePrograms({ pageSize: 100 });
79 |   const createMutation = useCreateProgramAccreditation();
80 |   const deleteMutation = useDeleteProgramAccreditation();
81 |
82 |   const [dialogOpen, setDialogOpen] = useState(false);
83 |   const [deleteId, setDeleteId] = useState<string | null>(null);
84 |
85 |   const programs = programsResult?.data ?? [];
86 |
87 |   const form = useForm<AccreditationFormData>({
88 |     resolver: zodResolver(accreditationFormSchema),
89 |     defaultValues: {
90 |       program_id: "",
91 |       accreditation_body: "",
92 |       framework_version: "",
93 |       accreditation_date: "",
94 |       next_review_date: "",
95 |       status: "pending",
96 |     },
97 |   });
98 |
99 |   const onSubmit = (data: AccreditationFormData) => {
100 |     const input: CreateProgramAccreditationInput = {
101 |       program_id: data.program_id,
102 |       accreditation_body: data.accreditation_body,
103 |       framework_version: data.framework_version || undefined,
104 |       accreditation_date: data.accreditation_date || undefined,
105 |       next_review_date: data.next_review_date || undefined,
106 |       status: data.status,
107 |     };
108 |
109 |     createMutation.mutate(input, {
110 |       onSuccess: () => {
111 |         setDialogOpen(false);
112 |         form.reset();
113 |       },
114 |     });
115 |   };
116 |
117 |   const handleDelete = () => {
118 |     if (deleteId) {

```



```

119     deleteMutation.mutate(deleteId, {
120       onSuccess: () => setDeleteId(null),
121     });
122   }
123 };
124
125 // Find accreditations with upcoming reviews (within 90 days)
126 const upcomingReviews = (accreditations ?? []).filter((a) => {
127   if (!a.next_review_date) return false;
128   const days = differenceInDays(new Date(a.next_review_date), new Date());
129   return days >= 0 && days <= 90;
130 });
131
132 if (isLoading) {
133   return (
134     <div className="space-y-4">
135       <Shimmer className="h-8 w-64 rounded-lg" />
136       <Shimmer className="h-48 rounded-xl" />
137     </div>
138   );
139 }
140
141 return (
142   <div className="space-y-6">
143     { /* Upcoming Review Alerts */ }
144     { upcomingReviews.length > 0 && (
145       <div className="rounded-xl bg-amber-50 p-4 space-y-2">
146         <div className="flex items-center gap-2">
147           <AlertTriangle className="h-4 w-4 text-amber-600" />
148           <p className="text-sm font-semibold text-amber-800">
149             Upcoming Accreditation Reviews
150           </p>
151         </div>
152         { upcomingReviews.map((a) => (
153           <div
154             key={a.id}
155             className="flex items-center justify-between text-sm"
156           >
157             <span className="text-amber-700">
158               {a.program_name ?? "Unknown"} – {a.accreditation_body}
159             </span>
160             <span className="text-amber-600 text-xs">
161               Review: { " " }
162               {a.next_review_date
163                 ? format(new Date(a.next_review_date), "MMM d, yyyy")
164                 : "-"} { " " }
165               {differenceInDays(new Date(a.next_review_date!), new Date())} { " " }
166               days
167             </span>
168           </div>
169         ) ) }
170       </div>
171     ) }
172
173     { /* Accreditation List */ }
174     <Card className="bg-white border-0 shadow-md rounded-xl overflow-hidden gap-0 py-0">
175       <div
176         className="px-6 py-4 flex items-center justify-between"
177         style={{
178           background: "var(--brand-gradient)",
179         }}
180       >
181         <div className="flex items-center gap-2">
182           <Shield className="h-5 w-5 text-white" />
183           <h2 className="text-lg font-bold tracking-tight text-white">
184             Program Accreditations
185           </h2>
186         </div>
187         <Button
188           size="sm"
189           variant="secondary"
190           onClick={() => setDialogOpen(true)}
191           className="bg-white/20 text-white hover:bg-white/30 border-0"
192           data-testid="add-accreditation"
193         >
194           <Plus className="h-4 w-4" />
195           Add
196         </Button>
197       </div>
198       <div className="p-6">
199         { (accreditations ?? []).length === 0 ? (
200           <p className="text-sm text-gray-500 text-center py-4">
201             No accreditation records yet. Add one to start tracking.
202           </p>
203         ) : (
204           <div className="space-y-3">
205             { (accreditations ?? []).map((acc) => (
206               <div
207                 key={acc.id}
208                 className="flex items-center justify-between p-3 rounded-lg border border-slate-200 hover:bg
209                 data-testid={`accreditation-row-${acc.id}`}
210               >
211                 <div className="space-y-1">
212                   <div className="flex items-center gap-2">
213                     <span className="text-sm font-semibold">
214                       {acc.program_name ?? "Unknown"} (
215                         {acc.program_code ?? "-"}
216                     </span>
217                     <Badge
218                       variant="outline"
219                       className={STATUS_STYLES[acc.status] ?? ""}
220                     >
221                       {acc.status}
222                     </Badge>

```

```

223     </div>
224     <div className="flex items-center gap-4 text-xs text-gray-500">
225       <span>{acc.accreditation_body}</span>
226       {acc.framework_version && (
227         <span>v{acc.framework_version}</span>
228       )}
229       {acc.accreditation_date && (
230         <span className="flex items-center gap-1">
231           <Calendar className="h-3 w-3" />
232           {format(new Date(acc.accreditation_date), "MMM yyyy")}
233         </span>
234       )}
235       {acc.next_review_date && (
236         <span>
237           Next review:{" "}
238           {format(
239             new Date(acc.next_review_date),
240             "MMM d, yyyy"
241           )}
242         </span>
243       )}
244     </div>
245   </div>
246   <Button
247     variant="ghost"
248     size="sm"
249     onClick={() => setDeleteId(acc.id)}
250     className="text-red-500 hover:text-red-700"
251     data-testid={`delete-accreditation-${acc.id}`}
252   >
253     <Trash2 className="h-4 w-4" />
254   </Button>
255 </div>
256 )}}
257 </div>
258 )}}
259 </div>
260 </Card>
261
262 /* Add Accreditation Dialog */
263 <Dialog open={dialogOpen} onOpenChange={setDialogOpen}>
264   <DialogContent>
265     <DialogHeader>
266       <DialogTitle>Add Program Accreditation</DialogTitle>
267     </DialogHeader>
268     <Form {...form}>
269       <form onSubmit={form.handleSubmit(onSubmit)} className="space-y-4">
270         <FormField
271           control={form.control}
272           name="program_id"
273           render={({ field }) => (
274             <FormItem>
275               <FormLabel>Program</FormLabel>
276               <Select onValueChange={field.onChange} value={field.value}>
277                 <FormControl>
278                   <SelectTrigger className="bg-white">
279                     <SelectValue placeholder="Select program" />
280                   </SelectTrigger>
281                 </FormControl>
282                 <SelectContent>
283                   {programs.map((p) => (
284                     <SelectItem key={p.id} value={p.id}>
285                       {p.code} – {p.name}
286                     </SelectItem>
287                   ))}
288                 </SelectContent>
289               </Select>
290               <FormMessage />
291             </FormItem>
292           )}
293         </FormField>
294         <FormField
295           control={form.control}
296           name="accreditation_body"
297           render={({ field }) => (
298             <FormItem>
299               <FormLabel>Accreditation Body</FormLabel>
300               <Select onValueChange={field.onChange} value={field.value}>
301                 <FormControl>
302                   <SelectTrigger className="bg-white">
303                     <SelectValue placeholder="Select body" />
304                   </SelectTrigger>
305                 </FormControl>
306                 <SelectContent>
307                   {ACCREDITATION_BODIES.map((body) => (
308                     <SelectItem key={body} value={body}>
309                       {body}
310                     </SelectItem>
311                   ))}
312                 </SelectContent>
313               </Select>
314               <FormMessage />
315             </FormItem>
316           )}
317         </FormField>
318         <FormField
319           control={form.control}
320           name="framework_version"
321           render={({ field }) => (
322             <FormItem>
323               <FormLabel>Framework Version (optional)</FormLabel>
324               <FormControl>
325                 <Input {...field} placeholder="e.g., 2024" />
326               </FormControl>

```


[illegible]

```

119 |         <span className="text-xs text-gray-500">
120 |             {gp.goal.goalType === "study_hours"
121 |               ? `${gp.currentValue.toFixed(1)} / ${
122 |                 gp.goal.targetValue
123 |                 }h`
124 |               : `${gp.currentValue} / ${gp.goal.targetValue}`}
125 |         </span>
126 |         {gp.isMet && (
127 |           <CheckCircle2
128 |             className="h-4 w-4 text-green-500"
129 |             data-testid="goal-met-indicator"
130 |           />
131 |         )}
132 |       </div>
133 |     </div>
134 |     { /* Progress bar */ }
135 |     <div className="h-2 w-full overflow-hidden rounded-full bg-gray-100">
136 |       <div
137 |         className={cn(
138 |           "h-full rounded-full transition-all duration-500",
139 |           gp.isMet
140 |             ? "bg-green-500"
141 |             : gp.percentage >= 50
142 |               ? "bg-blue-500"
143 |               : "bg-amber-500"
144 |         )}
145 |         style={{ width: `${Math.min(gp.percentage, 100)}%` }}
146 |         role="progressbar"
147 |         aria-valuenow={gp.percentage}
148 |         aria-valuemin={0}
149 |         aria-valuemax={100}
150 |         aria-label={` ${
151 |           GOAL_TYPE_LABELS[gp.goal.goalType]
152 |         } progress`}
153 |       />
154 |     </div>
155 |   </div>
156 |   ))}
157 | </div>
158 | )}
159 |
160 | { /* Empty goal state */ }
161 | {goals.length === 0 && {
162 |   <p className="text-sm text-gray-400 text-center py-2">
163 |     No goals set for this week
164 |   </p>
165 | }}
166 | </div>
167 | </PCard>
168 | );
169 | };
170 |
171 | export default ProgressSummaryPanel;
172 | export type { ProgressSummaryPanelProps };
173 |

```

FILE: src/components/shared/PWAInstallPrompt.tsx

TYPE: .tsx | LINES: 1-80 | SHA256: 55343cf3a687410b2a2250cbd8942c2bc287092b948a4e34cb2405a7df4778fd

```

1 | // Task 82.4: PWA install prompt – shows install banner on mobile
2 |
3 | import { useState, useEffect } from "react";
4 | import { Card, CardContent } from "@components/ui/card";
5 | import { Button } from "@components/ui/button";
6 | import { Download, X } from "lucide-react";
7 |
8 | const DISMISS_KEY = "edeviser_pwa_dismiss";
9 | const DISMISS_DAYS = 30;
10 |
11 | const PWAInstallPrompt = () => {
12 |   const [deferredPrompt, setDeferredPrompt] =
13 |     useState<BeforeInstallPromptEvent | null>(null);
14 |   const [visible, setVisible] = useState(false);
15 |
16 |   useEffect(() => {
17 |     const dismissed = localStorage.getItem(DISMISS_KEY);
18 |     if (dismissed && Date.now() - Number(dismissed) < DISMISS_DAYS * 86400000)
19 |       return;
20 |
21 |     const handler = (e: Event) => {
22 |       e.preventDefault();
23 |       setDeferredPrompt(e as BeforeInstallPromptEvent);
24 |       setVisible(true);
25 |     };
26 |
27 |     window.addEventListener("beforeinstallprompt", handler);
28 |     return () => window.removeEventListener("beforeinstallprompt", handler);
29 |   }, []);
30 |
31 |   const handleInstall = async () => {
32 |     if (!deferredPrompt) return;
33 |     deferredPrompt.prompt();
34 |     await deferredPrompt.userChoice;
35 |     setDeferredPrompt(null);
36 |     setVisible(false);
37 |   };
38 |
39 |   const handleDismiss = () => {
40 |     localStorage.setItem(DISMISS_KEY, String(Date.now()));
41 |     setVisible(false);
42 |   };
43 |
44 |   if (!visible) return null;
45 |

```

```

46 |     return (
47 |       <div className="fixed bottom-4 inset-x-4 z-50 md:inset-x-auto md:end-4 md:start-auto md:w-80">
48 |         <Card className="bg-white border-0 shadow-lg rounded-xl">
49 |           <CardContent className="flex items-center gap-3 p-4">
50 |             <Download className="h-5 w-5 text-blue-600 shrink-0" />
51 |             <div className="flex-1 min-w-0">
52 |               <p className="text-sm font-medium">Install Edeviser</p>
53 |               <p className="text-xs text-gray-500">
54 |                 Add to your home screen for quick access
55 |               </p>
56 |             </div>
57 |             <Button size="sm" onClick={handleInstall} variant="tactile">
58 |               Install
59 |             </Button>
60 |             <button
61 |               onClick={handleDismiss}
62 |               className="text-gray-400 hover:text-gray-600"
63 |               aria-label="Dismiss"
64 |             >
65 |               <X className="h-4 w-4" />
66 |             </button>
67 |           </CardContent>
68 |         </Card>
69 |       </div>
70 |     );
71 |   };
72 |
73 |   // Type augmentation for BeforeInstallPromptEvent
74 |   interface BeforeInstallPromptEvent extends Event {
75 |     prompt: () => Promise<void>;
76 |     userChoice: Promise<{ outcome: "accepted" | "dismissed" }>;
77 |   }
78 |
79 |   export default PWAIInstallPrompt;
80 |

```

FILE: src/components/shared/QualityFeedbackBanner.tsx

TYPE: .tsx | LINES: 1-109 | SHA256: 81409ed58bd93f5ca48817a17df0c2594f17e68d7697d2f4ec56ba1d412f9cc0

[illegible]

```

66 | const config = CATEGORY_CONFIG[category];
67 | const Icon = config.icon;
68 |
69 | return (
70 |   <div
71 |     className={cn(
72 |       "rounded-lg border p-3",
73 |       config.bgClass,
74 |       config.borderClass,
75 |       className
76 |     )}
77 |     data-testid="quality-feedback-banner"
78 |     role="status"
79 |     aria-label={`Reflection quality: ${config.label}`}
80 |   >
81 |     <div className="flex items-start gap-2">
82 |       <Icon className={cn("h-4 w-4 mt-0.5 shrink-0", config.textClass)} />
83 |       <div className="space-y-1">
84 |         <p className={cn("text-sm font-semibold", config.textClass)}>
85 |           {config.label}
86 |         </p>
87 |         <p className="text-xs text-gray-600">{config.description}</p>
88 |         {suggestions.length > 0 && (
89 |           <ul className="mt-1.5 space-y-0.5">
90 |             {suggestions.map((s, i) => (
91 |               <li
92 |                 key={i}
93 |                 className="text-xs text-gray-500 flex items-start gap-1"
94 |               >
95 |                 <span className="text-gray-400">•</span>
96 |                 {s}
97 |               </li>
98 |             )}}
99 |             </ul>
100 |           )}
101 |         </div>
102 |       </div>
103 |     </div>
104 |   );
105 | };
106 |
107 | export default QualityFeedbackBanner;
108 | export type { QualityFeedbackBannerProps };
109 |

```

FILE: [src/components/shared/QuestionCard.tsx](#)

TYPE: .tsx | **LINES:** 1-94 | **SHA256:** 77b4522716e39844fba8b30d5f4639418c86b261648746dd02f8a7f8d2b847b5

```

1 | import { Card } from "@components/ui/card";
2 | import { cn } from "@lib/utils";
3 | import LikertScale from "@components/shared/LikertScale";
4 |
5 | export interface QuestionCardProps {
6 |   questionText: string;
7 |   questionNumber?: number;
8 |   totalQuestions?: number;
9 |   mode: "likert" | "radio";
10 |   /** Radio options – required when mode is 'radio' */
11 |   options?: string[];
12 |   selectedValue: number | null;
13 |   onSelect: (value: number) => void;
14 |   disabled?: boolean;
15 |   /** Likert scale labels override */
16 |   likertLabels?: readonly string[];
17 |   /** Unique question identifier for ARIA */
18 |   questionId: string;
19 | }
20 |
21 | const QuestionCard = ({
22 |   questionText,
23 |   questionNumber,
24 |   totalQuestions,
25 |   mode,
26 |   options = [],
27 |   selectedValue,
28 |   onSelect,
29 |   disabled = false,
30 |   likertLabels,
31 |   questionId,
32 | }: QuestionCardProps) => {
33 |   return (
34 |     <Card className="bg-white border-0 shadow-md rounded-xl p-6 space-y-4">
35 |       {questionNumber != null && totalQuestions != null && (
36 |         <p className="text-xs font-medium text-gray-400">
37 |           Question {questionNumber} of {totalQuestions}
38 |         </p>
39 |       )}
40 |
41 |       <p
42 |         className="text-sm font-medium text-gray-900 leading-relaxed"
43 |         id={`q-${questionId}`}
44 |       >
45 |         {questionText}
46 |       </p>
47 |
48 |       {mode === "likert" && (
49 |         <LikertScale
50 |           value={selectedValue}
51 |           onChange={onSelect}
52 |           questionId={questionId}
53 |           disabled={disabled}
54 |           labels={likertLabels}
55 |         />
56 |       )}

```

```

57 |
58 |     {mode === "radio" && (
59 |       <div
60 |         role="radiogroup"
61 |         aria-labelledby={`q-${questionId}`}
62 |         className="space-y-2"
63 |       >
64 |         {options.map((option, index) => {
65 |           const isSelected = selectedValue === index;
66 |           return (
67 |             <button
68 |               key={index}
69 |               type="button"
70 |               role="radio"
71 |               aria-checked={isSelected}
72 |               disabled={disabled}
73 |               onClick={() => !disabled && onSelect(index)}
74 |               className={cn(
75 |                 "w-full text-start rounded-lg px-4 py-3 text-sm font-medium transition-colors",
76 |                 "focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-blue-500 focus-visible:r
77 |                 isSelected
78 |                 ? "bg-blue-600 text-white shadow-md"
79 |                 : "bg-slate-50 text-gray-700 hover:bg-slate-100",
80 |                 disabled && "opacity-50 cursor-not-allowed"
81 |               )}
82 |             >
83 |               {option}
84 |             </button>
85 |           );
86 |         })}
87 |       </div>
88 |     )}
89 |   </Card>
90 | );
91 | };
92 |
93 | export default QuestionCard;
94 |

```

FILE: [src/components/shared/QuestionPreview.tsx](#)

TYPE: .tsx | LINES: 1-266 | SHA256: 720e17b0ce14f06000cc381a51236e2d9f20971829f87219936938cb270718ee

```

1 | import { Card } from "@components/ui/card";
2 | import { Input } from "@components/ui/input";
3 | import { Textarea } from "@components/ui/textarea";
4 | import { Label } from "@components/ui/label";
5 | import { cn } from "@lib/utils";
6 |
7 | export interface QuestionPreviewProps {
8 |   questionText: string;
9 |   questionType: "mcq" | "true_false" | "short_answer" | "fill_in_blank";
10 |   options?: { key: string; text: string }[] | null;
11 |   selectedAnswer?: string;
12 |   onAnswerChange?: (answer: string) => void;
13 |   disabled?: boolean;
14 |   showCorrectAnswer?: boolean;
15 |   correctAnswer?: string;
16 | }
17 |
18 | const QuestionPreview = ({
19 |   questionText,
20 |   questionType,
21 |   options,
22 |   selectedAnswer,
23 |   onAnswerChange,
24 |   disabled = false,
25 |   showCorrectAnswer = false,
26 |   correctAnswer,
27 | }: QuestionPreviewProps) => {
28 |   const questionId = `qp-${questionType}-${questionText
29 |     .slice(0, 20)
30 |     .replace(/\s+/g, "-")}`;
31 |
32 |   const isCorrect = showCorrectAnswer && selectedAnswer === correctAnswer;
33 |   const isIncorrect =
34 |     showCorrectAnswer &&
35 |     selectedAnswer != null &&
36 |     selectedAnswer !== "" &&
37 |     selectedAnswer !== correctAnswer;
38 |
39 |   return (
40 |     <Card className="bg-white border-0 shadow-md rounded-xl p-6 space-y-4">
41 |       <p
42 |         className="text-sm font-medium text-gray-900 leading-relaxed"
43 |         id={questionId}
44 |       >
45 |         {questionText}
46 |       </p>
47 |
48 |       {questionType === "mcq" && (
49 |         <MCQOptions
50 |           options={options ?? []}
51 |           selectedAnswer={selectedAnswer}
52 |           onAnswerChange={onAnswerChange}
53 |           disabled={disabled}
54 |           showCorrectAnswer={showCorrectAnswer}
55 |           correctAnswer={correctAnswer}
56 |           questionId={questionId}
57 |         />
58 |       )}
59 |
60 |       {questionType === "true_false" && (
61 |         <TrueFalseOptions
62 |           selectedAnswer={selectedAnswer}

```



```

63 |         onAnswerChange={onAnswerChange}
64 |         disabled={disabled}
65 |         showCorrectAnswer={showCorrectAnswer}
66 |         correctAnswer={correctAnswer}
67 |         questionId={questionId}
68 |     )/>
69 | })
70 |
71 | {questionType === "short_answer" && (
72 |   <div className="space-y-2">
73 |     <Label htmlFor={` ${questionId}-answer`} >Your Answer</Label>
74 |     <Textarea
75 |       id={` ${questionId}-answer`}
76 |       aria-labelledby={questionId}
77 |       placeholder="Type your answer here..."
78 |       value={selectedAnswer ?? ""}
79 |       onChange={(e) => onAnswerChange?.(e.target.value)}
80 |       disabled={disabled}
81 |       rows={4}
82 |       className={cn(
83 |         showCorrectAnswer && isCorrect && "border-green-500 bg-green-50",
84 |         showCorrectAnswer && isIncorrect && "border-red-500 bg-red-50"
85 |       )}
86 |     />
87 |     {showCorrectAnswer && correctAnswer && (
88 |       <CorrectAnswerDisplay correctAnswer={correctAnswer} />
89 |     )}
90 |   </div>
91 | )}
92 |
93 | {questionType === "fill_in_blank" && (
94 |   <div className="space-y-2">
95 |     <Label htmlFor={` ${questionId}-blank`} >Fill in the blank</Label>
96 |     <Input
97 |       id={` ${questionId}-blank`}
98 |       aria-labelledby={questionId}
99 |       placeholder="Type your answer..."
100 |       value={selectedAnswer ?? ""}
101 |       onChange={(e) => onAnswerChange?.(e.target.value)}
102 |       disabled={disabled}
103 |       className={cn(
104 |         "max-w-md",
105 |         showCorrectAnswer && isCorrect && "border-green-500 bg-green-50",
106 |         showCorrectAnswer && isIncorrect && "border-red-500 bg-red-50"
107 |       )}
108 |     />
109 |     {showCorrectAnswer && correctAnswer && (
110 |       <CorrectAnswerDisplay correctAnswer={correctAnswer} />
111 |     )}
112 |   </div>
113 | )}
114 | </Card>
115 | );
116 | };
117 |
118 | const MCQOptions = ({
119 |   options,
120 |   selectedAnswer,
121 |   onAnswerChange,
122 |   disabled,
123 |   showCorrectAnswer,
124 |   correctAnswer,
125 |   questionId,
126 | }): {
127 |   options: { key: string; text: string }[];
128 |   selectedAnswer?: string;
129 |   onAnswerChange?: (answer: string) => void;
130 |   disabled: boolean;
131 |   showCorrectAnswer: boolean;
132 |   correctAnswer?: string;
133 |   questionId: string;
134 | }) => (
135 |   <div role="radiogroup" aria-labelledby={questionId} className="space-y-2">
136 |     {options.map((option) => {
137 |       const isSelected = selectedAnswer === option.key;
138 |       const isOptionCorrect = showCorrectAnswer && option.key === correctAnswer;
139 |       const isOptionIncorrect =
140 |         showCorrectAnswer && isSelected && option.key !== correctAnswer;
141 |
142 |       return (
143 |         <button
144 |           key={option.key}
145 |           type="button"
146 |           role="radio"
147 |           aria-checked={isSelected}
148 |           aria-label={` ${option.key}. ${option.text}`}
149 |           disabled={disabled}
150 |           onClick={() => !disabled && onAnswerChange?.(option.key)}
151 |           className={cn(
152 |             "w-full text-start rounded-lg px-4 py-3 text-sm font-medium transition-colors flex items-center ga
153 |             focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-blue-500 focus-visible:ring-of
154 |             !showCorrectAnswer &&
155 |             isSelected &&
156 |             "bg-blue-600 text-white shadow-md",
157 |             !showCorrectAnswer &&
158 |             !isSelected &&
159 |             "bg-slate-50 text-gray-700 hover:bg-slate-100",
160 |             isOptionCorrect &&
161 |             "bg-green-100 text-green-800 border border-green-300",
162 |             isOptionIncorrect &&
163 |             "bg-red-100 text-red-800 border border-red-300",
164 |             showCorrectAnswer &&
165 |             !isOptionCorrect &&
166 |             !isOptionIncorrect &&

```

```

167 |         "bg-slate-50 text-gray-700",
168 |         disabled && "opacity-50 cursor-not-allowed"
169 |     )}
170 | >
171 | <span
172 |   className={cn(
173 |     "flex-shrink-0 w-7 h-7 rounded-full flex items-center justify-center text-xs font-bold",
174 |     !showCorrectAnswer && isSelected && "bg-white/20 text-white",
175 |     !showCorrectAnswer && !isSelected && "bg-slate-200 text-gray-600",
176 |     isOptionCorrect && "bg-green-500 text-white",
177 |     isOptionIncorrect && "bg-red-500 text-white",
178 |     showCorrectAnswer &&
179 |       !isOptionCorrect &&
180 |       !isOptionIncorrect &&
181 |       "bg-slate-200 text-gray-600"
182 |   )}
183 | >
184 |     {option.key}
185 |   </span>
186 |   <span>{option.text}</span>
187 | </button>
188 | );
189 |   }}
190 | </div>
191 | );
192 |
193 | const TrueFalseOptions = ({
194 |   selectedAnswer,
195 |   onAnswerChange,
196 |   disabled,
197 |   showCorrectAnswer,
198 |   correctAnswer,
199 |   questionId,
200 | }): {
201 |   selectedAnswer?: string;
202 |   onAnswerChange?: (answer: string) => void;
203 |   disabled: boolean;
204 |   showCorrectAnswer: boolean;
205 |   correctAnswer?: string;
206 |   questionId: string;
207 | }) => {
208 |   const tfOptions = [
209 |     { key: "True", text: "True" },
210 |     { key: "False", text: "False" },
211 |   ];
212 |
213 |   return (
214 |     <div role="radiogroup" aria-labelledby={questionId} className="flex gap-3">
215 |       {tfOptions.map((option) => {
216 |         const isSelected = selectedAnswer === option.key;
217 |         const isOptionCorrect =
218 |           showCorrectAnswer && option.key === correctAnswer;
219 |         const isOptionIncorrect =
220 |           showCorrectAnswer && isSelected && option.key !== correctAnswer;
221 |
222 |         return (
223 |           <button
224 |             key={option.key}
225 |             type="button"
226 |             role="radio"
227 |             aria-checked={isSelected}
228 |             aria-label={option.text}
229 |             disabled={disabled}
230 |             onClick={() => !disabled && onAnswerChange?.(option.key)}
231 |             className={cn(
232 |               "flex-1 rounded-lg px-4 py-3 text-sm font-medium transition-colors text-center",
233 |               "focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-blue-500 focus-visible:ring-
234 |               !showCorrectAnswer &&
235 |               isSelected &&
236 |               "bg-blue-600 text-white shadow-md",
237 |               !showCorrectAnswer &&
238 |               !isSelected &&
239 |               "bg-slate-50 text-gray-700 hover:bg-slate-100",
240 |               isOptionCorrect &&
241 |               "bg-green-100 text-green-800 border border-green-300",
242 |               isOptionIncorrect &&
243 |               "bg-red-100 text-red-800 border border-red-300",
244 |               showCorrectAnswer &&
245 |               !isOptionCorrect &&
246 |               !isOptionIncorrect &&
247 |               "bg-slate-50 text-gray-700",
248 |               disabled && "opacity-50 cursor-not-allowed"
249 |             )}
250 |           >
251 |             {option.text}
252 |           </button>
253 |         );
254 |       })}
255 |     </div>
256 |   );
257 | };
258 |
259 | const CorrectAnswerDisplay = ({ correctAnswer }: { correctAnswer: string }) => (
260 |   <p className="text-xs font-medium text-green-700 bg-green-50 rounded-md px-3 py-2">
261 |     Correct answer: {correctAnswer}
262 |   </p>
263 | );
264 |
265 | export default QuestionPreview;
266 |

```

FILE: src/components/shared/QuestionQualityIndicator.tsx

```
TYPE: .tsx | LINES: 1-56 | SHA256: 58bc516584d6c7d599ee3fb3c48f37307d10d1051f9d9c9cf44cffc3a40c7cfe
```

```

1 import { cn } from "@lib/utils";
2
3 export interface QuestionQualityIndicatorProps {
4   qualityFlag: "good" | "low_discrimination" | "too_easy" | "too_hard" | null;
5   className?: string;
6 }
7
8 const getQualityConfig = (
9   flag: QuestionQualityIndicatorProps["qualityFlag"]
10 ) => {
11   switch (flag) {
12     case "too_hard":
13       return {
14         dotColor: "bg-red-500",
15         textColor: "text-red-700",
16         label: "Too Hard",
17       };
18     case "low_discrimination":
19       return {
20         dotColor: "bg-yellow-500",
21         textColor: "text-yellow-700",
22         label: "Low Discrimination",
23       };
24     case "too_easy":
25       return {
26         dotColor: "bg-yellow-500",
27         textColor: "text-yellow-700",
28         label: "Too Easy",
29       };
30     case "good":
31       default:
32         return {
33           dotColor: "bg-green-500",
34           textColor: "text-green-700",
35           label: "Good",
36         };
37   }
38 };
39
40 const QuestionQualityIndicator = ({
41   qualityFlag,
42   className,
43 }: QuestionQualityIndicatorProps) => {
44   const { dotColor, textColor, label } = getQualityConfig(qualityFlag);
45
46   return (
47     <span className={cn("inline-flex items-center gap-1.5", className)}>
48       <span className={cn("h-2 w-2 shrink-0 rounded-full", dotColor)} />
49       <span className={cn("text-xs font-medium", textColor)}>{label}</span>
50     </span>
51   );
52 };
53
54 export { QuestionQualityIndicator };
55 export default QuestionQualityIndicator;
56

```

FILE: src/components/shared/QuickThoughtInput.tsx

TYPE: .tsx | LINES: 1-112 | SHA256: af9804f8db1864b7c408e9b5baa91868952f0dce37d320d4e71955f091243dcc

[illegible]

```

41 |
42 | const charCount = text.length;
43 | const isOverLimit = charCount > MAX_CHARS;
44 | const isEmpty = text.trim().length === 0;
45 | const canSubmit = !isEmpty && !isOverLimit && !isPending && !disabled;
46 |
47 | const handleSubmit = useCallback(
48 |   (e: React.FormEvent) => {
49 |     e.preventDefault();
50 |     if (!canSubmit) return;
51 |     onSubmit(text.trim());
52 |     setText("");
53 |   },
54 |   [canSubmit, onSubmit, text]
55 | );
56 |
57 | const handleChange = useCallback((e: React.ChangeEvent<HTMLInputElement>) => {
58 |   setText(e.target.value);
59 | }, []);
60 |
61 | return (
62 |   <form onSubmit={handleSubmit} className={cn("space-y-2", className)}>
63 |     <div className="flex items-center gap-2">
64 |       <input
65 |         type="text"
66 |         value={text}
67 |         onChange={handleChange}
68 |         placeholder={placeholder}
69 |         disabled={disabled || isPending}
70 |         aria-label="Quick thought"
71 |         aria-describedby="quick-thought-counter"
72 |       />
73 |       <button
74 |         type="submit"
75 |         size="icon"
76 |         disabled={!canSubmit}
77 |         aria-label="Submit thought"
78 |         variant="tactile"
79 |       >
80 |         {isPending ? (
81 |           <Loader2 className="h-4 w-4 animate-spin" />
82 |         ) : (
83 |           <Send className="h-4 w-4" />
84 |         )}
85 |       </button>
86 |     </div>
87 |
88 |     <div className="flex items-center justify-between text-[11px] text-gray-500">
89 |       <span>One sentence is plenty.</span>
90 |       <span
91 |         id="quick-thought-counter"
92 |         aria-live="polite"
93 |         className={cn(
94 |           "tabular-nums",
95 |           isOverLimit
96 |             ? "font-medium text-red-600"
97 |             : charCount > MAX_CHARS - 30
98 |               ? "font-medium text-amber-600"
99 |               : ""
100 |         )}
101 |       >
102 |         {charCount}/{MAX_CHARS}
103 |       </span>
104 |     </div>
105 |   </form>
106 | );
107 | };
108 |
109 | export default QuickThoughtInput;
110 | export type { QuickThoughtInputProps };
111 |
112 |

```

FILE: [src/components/shared/QuizQuestionCard.tsx](#)

TYPE: .tsx | LINES: 1-83 | SHA256: b0eb3cf9e82176ead58f86ee2846a90e27b145e0a3db1ae3f018f1d51bda9d83

```

1 | // =====
2 | // QuizQuestionCard – Quiz question display card
3 | // =====
4 |
5 | import { cn } from "@lib/Utils";
6 |
7 | interface QuizQuestionCardProps {
8 |   questionNumber: number;
9 |   questionText: string;
10 |   questionType: "mcq" | "true_false" | "short_answer" | "fill_blank";
11 |   options?: string[];
12 |   selectedAnswer?: string;
13 |   onAnswer: (answer: string) => void;
14 |   points: number;
15 |   className?: string;
16 | }
17 |
18 | const QuizQuestionCard = ({
19 |   questionNumber,
20 |   questionText,
21 |   questionType,
22 |   options,
23 |   selectedAnswer,
24 |   onAnswer,
25 |   points,
26 |   className,
27 | }: QuizQuestionCardProps) => (
28 |   <div

```

```

29 |     className={cn(
30 |       "rounded-xl border border-slate-200 bg-white p-5 space-y-4",
31 |       className
32 |     )}
33 |   >
34 |     <div className="flex items-start justify-between gap-2">
35 |       <p className="text-sm font-medium">
36 |         <span className="text-gray-400 me-2">Q{questionNumber}</span>
37 |         {questionText}
38 |       </p>
39 |       <span className="text-xs text-gray-400 shrink-0">{points} pts</span>
40 |     </div>
41 |
42 |     {(questionType === "mcq" || questionType === "true_false") && options && (
43 |       <div className="space-y-2">
44 |         {options.map((opt, idx) => (
45 |           <button
46 |             key={idx}
47 |             type="button"
48 |             onClick={() => onAnswer(opt)}
49 |             className={cn(
50 |               "w-full text-start rounded-lg border px-4 py-2.5 text-sm transition-colors",
51 |               selectedAnswer === opt
52 |                 ? "border-blue-500 bg-blue-50 text-blue-700 font-medium"
53 |                 : "border-gray-200 text-gray-700 hover:bg-gray-50"
54 |             )}
55 |           >
56 |             <span className="text-gray-400 me-2 font-medium">
57 |               {String.fromCharCode(65 + idx)}.
58 |             </span>
59 |             {opt}
60 |           </button>
61 |         )})}
62 |       </div>
63 |     )}
64 |
65 |     {(questionType === "short_answer" || questionType === "fill_blank") && (
66 |       <input
67 |         type="text"
68 |         value={selectedAnswer ?? ""}
69 |         onChange={(e) => onAnswer(e.target.value)}
70 |         placeholder={
71 |           questionType === "fill_blank"
72 |             ? "Fill in the blank..."
73 |             : "Your answer..."
74 |         }
75 |         className="w-full rounded-lg border border-gray-200 px-4 py-2.5 text-sm focus:border-blue-500 focus:ou
76 |       >/>
77 |     )}
78 |   </div>
79 | );
80 |
81 | export default QuizQuestionCard;
82 | export type { QuizQuestionCardProps };
83 |

```

FILE: src/components/shared/RealtimeStatusBanner.tsx

TYPE: .tsx | LINES: 1-24 | SHA256: 42dc0c916ae2f445a3adacad8e2ad27ede1570efb1500154d759c171d034341c

```

1 | // =====
2 | // RealtimeStatusBanner - Delegates to ReconnectBanner for "Live updates paused"
3 | // Shown when Supabase Realtime connection fails and system falls back to polling.
4 | // Validates: Requirements 32, 53.5, 66.4
5 | // =====
6 |
7 | import ReconnectBanner from "@components/shared/ReconnectBanner";
8 |
9 | interface RealtimeStatusBannerProps {
10 |   /** Whether the realtime connection is live */
11 |   isLive: boolean;
12 |   /** Number of reconnection attempts */
13 |   retryCount?: number;
14 | }
15 |
16 | const RealtimeStatusBanner = ({
17 |   isLive,
18 |   retryCount = 0,
19 | }: RealtimeStatusBannerProps) => (
20 |   <ReconnectBanner isDisconnected={!isLive} retryCount={retryCount} />
21 | );
22 |
23 | export default RealtimeStatusBanner;
24 |

```

FILE: src/components/shared/ReconnectBanner.tsx

TYPE: .tsx | LINES: 1-45 | SHA256: d6d20a6452d438352ea5b0a4caef29abd4f402640542b97a7c5c3f6fba470303

```

1 | // =====
2 | // ReconnectBanner - "Live updates paused" banner for realtime reconnection
3 | // =====
4 |
5 | import { WifiOff } from "lucide-react";
6 | import { cn } from "@lib/utils";
7 |
8 | interface ReconnectBannerProps {
9 |   isDisconnected: boolean;
10 |   retryCount: number;
11 |   className?: string;
12 | }
13 |
14 | const ReconnectBanner = ({
15 |   isDisconnected,
16 |   retryCount,
17 |   className,

```

FILE: src/components/shared/ReflectionDigestCard.tsx

[illegible]

```

73 |         </h3>
74 |         <div className="flex flex-wrap gap-1.5">
75 |             {digest.themes.map((theme, i) => (
76 |                 <Badge
77 |                     key={i}
78 |                     variant="secondary"
79 |                     className="text-xs bg-purple-50 text-purple-700 border-purple-200"
80 |                 >
81 |                     {theme.topic} ({theme.count})
82 |                 </Badge>
83 |             ))}
84 |         </div>
85 |     </section>
86 | })
87 |
88 |     {/* Growth Patterns */}
89 |     {digest.growthPatterns.length > 0 && (
90 |         <section>
91 |             <h3 className="text-sm font-semibold text-gray-700 flex items-center gap-1.5 mb-2">
92 |                 <TrendingUp className="h-4 w-4 text-green-500" />
93 |                 Growth Patterns
94 |             </h3>
95 |             <ul className="space-y-1">
96 |                 {digest.growthPatterns.map((p, i) => (
97 |                     <li key={i} className="text-xs text-gray-600">
98 |                         <span className="font-medium text-gray-700">{p.area}</span>{" "}
99 |                         {p.description}
100 |                     </li>
101 |                 ))}
102 |             </ul>
103 |         </section>
104 |     )}
105 |
106 |     {/* Emotional Trends */}
107 |     {digest.emotionalTrends.length > 0 && (
108 |         <section>
109 |             <h3 className="text-sm font-semibold text-gray-700 flex items-center gap-1.5 mb-2">
110 |                 <Heart className="h-4 w-4 text-red-400" />
111 |                 Emotional Trends
112 |             </h3>
113 |             <div className="flex flex-wrap gap-1.5">
114 |                 {digest.emotionalTrends.map((t, i) => (
115 |                     <Badge
116 |                         key={i}
117 |                         variant="outline"
118 |                         className="text-xs border-red-200 text-red-600"
119 |                     >
120 |                         {t.label}
121 |                     </Badge>
122 |                 ))}
123 |             </div>
124 |         </section>
125 |     )}
126 |
127 |     {/* Suggested Focus */}
128 |     {digest.suggestedFocus.length > 0 && (
129 |         <section>
130 |             <h3 className="text-sm font-semibold text-gray-700 flex items-center gap-1.5 mb-2">
131 |                 <Target className="h-4 w-4 text-blue-500" />
132 |                 Suggested Focus
133 |             </h3>
134 |             <ul className="space-y-1">
135 |                 {digest.suggestedFocus.map((f, i) => (
136 |                     <li key={i} className="text-xs text-gray-600">
137 |                         <span className="font-medium text-gray-700">{f.area}</span>{" "}
138 |                         {f.reason}
139 |                     </li>
140 |                 ))}
141 |             </ul>
142 |         </section>
143 |     )}
144 |
145 |     {/* Share Controls */}
146 |     <div className="pt-2 border-t border-gray-100">
147 |         {!showShareOptions ? (
148 |             <Button
149 |                 variant="outline"
150 |                 size="sm"
151 |                 className="gap-1.5 text-xs"
152 |                 onClick={() => setShowShareOptions(true)}
153 |                 data-testid="share-digest-button"
154 |             >
155 |                 <Share2 className="h-3.5 w-3.5" />
156 |                 Share Insights
157 |             </Button>
158 |         ) : (
159 |             <div className="space-y-2">
160 |                 <p className="text-xs text-gray-500">Share with:</p>
161 |                 <div className="flex flex-wrap gap-2">
162 |                     {shareRoles.map((role) => {
163 |                         const isShared = sharedRoles.has(role);
164 |                         return (
165 |                             <Button
166 |                                 key={role}
167 |                                 variant={isShared ? "default" : "outline"}
168 |                                 size="sm"
169 |                                 className={cn(
170 |                                     "gap-1 text-xs capitalize",
171 |                                     isShared && "bg-blue-600 hover:bg-blue-700 text-white"
172 |                                 )}
173 |                                 onClick={() => {
174 |                                     if (isShared) {
175 |                                         onRevokeShare?.(digest.id, role);
176 |                                     } else {

```

```

177         onShare?.(digest.id, role);
178     }
179     }}
180     disabled={isSharing}
181     data-testid={`share-${role}-button`}
182   >
183     {isSharing ? (
184       <Loader2 className="h-3 w-3 animate-spin" />
185     ) : isShared ? (
186       <X className="h-3 w-3" />
187     ) : null}
188     {role}
189     {isShared && " (shared)"}
190   </Button>
191 );
192   }}
193 </div>
194 </div>
195   )}
196 </div>
197 </div>
198 </PCard>
199 );
200 };
201
202 export default ReflectionDigestCard;
203 export type { ReflectionDigestCardProps };
204

```

FILE: src/components/shared/ReflectionStreakIndicator.tsx

TYPE: .tsx | LINES: 1-43 | SHA256: 23cd0f74b464f70e4d5427e86f3d99017e68d6e9b261f2b7f364daca2d6aa001

```
// =====  
2 // ReflectionStreakIndicator - Shows consecutive weeks with completed reflections  
3 //  
4  
import { Flame } from "lucide-react";  
6 import { cn } from "@lib/utils";  
7  
8 // % % % Types % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
9  
10 interface ReflectionStreakIndicatorProps {  
11   streakWeeks: number;  
12   className?: string;  
13 }  
14  
15 // % % % Component % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
16  
const ReflectionStreakIndicator = ({  
17   streakWeeks,  
18   className,  
19 }: ReflectionStreakIndicatorProps) => {  
20   if (streakWeeks <= 0) return null;  
21  
22   return (  
23     <div  
24       className={cn(  
25         "inline-flex items-center gap-1.5 rounded-full px-3 py-1",  
26         "bg-gradient-to-r from-orange-50 to-red-50 border border-orange-200",  
27         className  
28       )}  
29       data-testid="reflection-streak-indicator"  
30       aria-label={` ${streakWeeks} week reflection streak`}  
31     >  
32       <Flame className="h-4 w-4 text-orange-500 animate-streak-flame" />  
33       <span className="text-xs font-bold text-orange-700">  
34         {streakWeeks} week{streakWeeks !== 1 ? "s" : ""}  
35       </span>  
36     </div>  
37   );  
38 };  
39  
export default ReflectionStreakIndicator;  
41 export type { ReflectionStreakIndicatorProps };  
42  
43
```

FILE: src/components/shared/ReflectionTemplateSelector.tsx

```
TYPE: .tsx | LINES: 1-105 | SHA256: 26092ade002912d390ba5a3642697b7cd3e7ff18509e9bffb6bea898b2503e8d7
```

[illegible]

[illegible]

TYPE: .tsx | LINES: 1-339 | SHA256: dec913a3a3bb059a671dd750db4507f7f548fe7f3854ab931817dd99a1d27cb4

```

21 | import type { VoteStatus } from "@hooks/useReplacementVotes";
22 |
23 | export interface ReplacementVoteCardProps {
24 |   /** Vote data – null means no active vote (show initiation UI) */
25 |   vote?: {
26 |     id: string;
27 |     targetMemberName: string;
28 |     targetMemberId: string;
29 |     status: VoteStatus;
30 |     votesFor: number;
31 |     votesAgainst: number;
32 |     createdAt: string;
33 |     resolvedAt: string | null;
34 |     teacherOverride: boolean;
35 |   } | null;
36 |   /** Whether the current user is the team captain */
37 |   isCaptain: boolean;
38 |   /** Whether the current user is a teacher */
39 |   isTeacher: boolean;
40 |   /** Whether the current user has already voted */
41 |   hasVoted?: boolean;
42 |   /** Total team members (for majority calculation) */
43 |   totalMembers: number;
44 |   /** Vote expiry duration in hours (default 48) */
45 |   expiryHours?: number;
46 |   /** Callbacks */
47 |   onInitiateVote?: (targetMemberId: string) => void;
48 |   onCastVote?: (voteId: string, voteFor: boolean) => void;
49 |   onTeacherOverride?: (voteId: string, approve: boolean) => void;
50 |   /** Loading states */
51 |   isInitiating?: boolean;
52 |   isCasting?: boolean;
53 |   isOverriding?: boolean;
54 |   /** Inactive members available for vote initiation */
55 |   inactiveMembers?: Array<{ id: string; name: string }>;
56 |   className?: string;
57 | }
58 |
59 | /** Calculate remaining time until vote expires */
60 | function getTimeRemaining(
61 |   createdAt: string,
62 |   expiryHours: number
63 | ): { hours: number; minutes: number; expired: boolean } {
64 |   const expiryMs = expiryHours * 60 * 60 * 1000;
65 |   const expiresAt = new Date(createdAt).getTime() + expiryMs;
66 |   const remaining = expiresAt - Date.now();
67 |
68 |   if (remaining <= 0) {
69 |     return { hours: 0, minutes: 0, expired: true };
70 |   }
71 |
72 |   return {
73 |     hours: Math.floor(remaining / (60 * 60 * 1000)),
74 |     minutes: Math.floor((remaining % (60 * 60 * 1000)) / (60 * 1000)),
75 |     expired: false,
76 |   };
77 | }
78 |
79 | const ReplacementVoteCard = ({
80 |   vote,
81 |   isCaptain,
82 |   isTeacher,
83 |   hasVoted = false,
84 |   totalMembers,
85 |   expiryHours = 48,
86 |   onInitiateVote,
87 |   onCastVote,
88 |   onTeacherOverride,
89 |   isInitiating = false,
90 |   isCasting = false,
91 |   isOverriding = false,
92 |   inactiveMembers = [],
93 |   className,
94 | }: ReplacementVoteCardProps) => {
95 |   const [timeRemaining, setTimeRemaining] = useState(
96 |     vote?.status === "open"
97 |       ? getTimeRemaining(vote.createdAt, expiryHours)
98 |       : null
99 |   );
100 |   const [selectedMember, setSelectedMember] = useState("");
101 |
102 |   // Update countdown every minute
103 |   useEffect(() => {
104 |     if (!vote || vote.status !== "open") return;
105 |
106 |     const interval = setInterval(() => {
107 |       setTimeRemaining(getTimeRemaining(vote.createdAt, expiryHours));
108 |     }, 60_000);
109 |
110 |     return () => clearInterval(interval);
111 |   }, [vote, expiryHours]);
112 |
113 |   // %% No active vote: show initiation UI (captain only) % % % % % % % % % % % % % % % %
114 |   if (!vote) {
115 |     if (!isCaptain || inactiveMembers.length === 0) return null;
116 |
117 |     return (
118 |       <Card
119 |         className={cn("bg-white border-0 shadow-md rounded-xl p-4", className)}
120 |       >
121 |         <div className="flex items-center gap-2 mb-3">
122 |           <AlertTriangle className="h-4 w-4 text-amber-500" />
123 |           <h4 className="text-sm font-bold">Replacement Vote</h4>
124 |         </div>

```

[illegible]

```

229 | </div>
230 |
231 | { /* Countdown */ }
232 | {isOpen && timeRemaining && !timeRemaining.expired && (
233 |   <div className="flex items-center gap-1.5 mt-2 text-xs text-gray-500">
234 |     <Clock className="h-3.5 w-3.5" />
235 |     <span>
236 |       Expires in {timeRemaining.hours}h {timeRemaining.minutes}m
237 |     </span>
238 |   </div>
239 | )}
240 |
241 | {isOpen && timeRemaining?.expired && (
242 |   <div className="flex items-center gap-1.5 mt-2 text-xs text-red-500">
243 |     <Clock className="h-3.5 w-3.5" />
244 |     <span>Vote has expired</span>
245 |   </div>
246 | )}
247 |
248 | { /* Teacher override badge */ }
249 | {vote.teacherOverride && (
250 |   <div className="flex items-center gap-1.5 mt-2">
251 |     <Shield className="h-3.5 w-3.5 text-blue-500" />
252 |     <span className="text-xs text-blue-600 font-medium">
253 |       Teacher override applied
254 |     </span>
255 |   </div>
256 | )}
257 |
258 | { /* Actions */ }
259 | {isOpen && (
260 |   <div className="flex items-center gap-2 mt-4 pt-3 border-t border-gray-100">
261 |     { /* Member voting */ }
262 |     { !isTeacher && !hasVoted && (
263 |       <>
264 |         <Button
265 |           size="sm"
266 |           onClick={() => onCastVote?.(vote.id, true)}
267 |           disabled={isPending}
268 |           className="bg-green-600 hover:bg-green-700 text-white active:scale-95 transition-transform dur
269 |         >
270 |           {isCasting ? (
271 |             <Loader2 className="h-4 w-4 animate-spin" />
272 |           ) : (
273 |             <ThumbsUp className="h-4 w-4" />
274 |           )}
275 |           Approve
276 |         </Button>
277 |         <Button
278 |           size="sm"
279 |           variant="outline"
280 |           onClick={() => onCastVote?.(vote.id, false)}
281 |           disabled={isPending}
282 |           className="active:scale-95 transition-transform duration-100 gap-1.5"
283 |         >
284 |           {isCasting ? (
285 |             <Loader2 className="h-4 w-4 animate-spin" />
286 |           ) : (
287 |             <ThumbsDown className="h-4 w-4" />
288 |           )}
289 |           Reject
290 |         </Button>
291 |       </>
292 |     )}
293 |
294 |     { !isTeacher && hasVoted && (
295 |       <span className="text-xs text-gray-500 italic">
296 |         You have already voted
297 |       </span>
298 |     )}
299 |
300 |   { /* Teacher override */ }
301 |   {isTeacher && (
302 |     <>
303 |       <Button
304 |         size="sm"
305 |         onClick={() => onTeacherOverride?.(vote.id, true)}
306 |         disabled={isPending}
307 |         className="gap-1.5 active:scale-95 transition-transform duration-100"
308 |       >
309 |         {isOverriding ? (
310 |           <Loader2 className="h-4 w-4 animate-spin" />
311 |         ) : (
312 |           <Shield className="h-4 w-4" />
313 |         )}
314 |         Override: Approve
315 |       </Button>
316 |       <Button
317 |         size="sm"
318 |         variant="outline"
319 |         onClick={() => onTeacherOverride?.(vote.id, false)}
320 |         disabled={isPending}
321 |         className="gap-1.5 active:scale-95 transition-transform duration-100"
322 |       >
323 |         {isOverriding ? (
324 |           <Loader2 className="h-4 w-4 animate-spin" />
325 |         ) : (
326 |           <Shield className="h-4 w-4" />
327 |         )}
328 |         Override: Reject
329 |       </Button>
330 |     </>
331 |   )}
332 | </div>

```

```

333 |     })
334 |   </Card>
335 | );
336 | };
337 |
338 | export default ReplacementVoteCard;
339 |

```

FILE: src/components/shared/ReviewScheduleList.tsx

TYPE: .tsx | LINES: 1-136 | SHA256: 1c4aa22cfcc8b99af33f5d2725be38f6dd27469d39251481d95cfd1150430907

```

1 | // =====
2 | // ReviewScheduleList - List of pending spaced-repetition reviews with
3 | // Start and Skip actions per item. Shows completed/skipped reviews in a
4 | // muted state. Empty state when no reviews exist.
5 | // =====
6 |
7 | import { Button } from "@components/ui/button";
8 | import { cn } from "@lib/utils";
9 | import type { ReviewSchedule } from "@types/planner";
10 | import { ReviewSessionBadge } from "@components/shared/ReviewSessionBadge";
11 | import { Play, SkipForward, CheckCircle2, CalendarDays } from "lucide-react";
12 |
13 | interface ReviewScheduleListProps {
14 |   reviews: ReviewSchedule[];
15 |   onStartReview: (review: ReviewSchedule) => void;
16 |   onSkipReview: (reviewId: string) => void;
17 |   isSkipping?: boolean;
18 | }
19 |
20 | const ReviewScheduleList = ({
21 |   reviews,
22 |   onStartReview,
23 |   onSkipReview,
24 |   isSkipping = false,
25 | }: ReviewScheduleListProps) => {
26 |   if (reviews.length === 0) {
27 |     return (
28 |       <div className="flex flex-col items-center justify-center py-8 text-center">
29 |         <CalendarDays className="h-8 w-8 text-slate-300 mb-2" />
30 |         <p className="text-sm font-medium text-slate-500">
31 |           No reviews scheduled
32 |         </p>
33 |         <p className="text-xs text-slate-400 mt-1">
34 |           Complete study sessions linked to CLOs to generate review schedules.
35 |         </p>
36 |       </div>
37 |     );
38 |   }
39 |
40 |   return (
41 |     <div className="space-y-2" role="list" aria-label="Review schedule">
42 |       {reviews.map((review) => {
43 |         const isPending = review.status === "pending";
44 |         const isCompleted = review.status === "completed";
45 |         const isSkipped = review.status === "skipped";
46 |
47 |         return (
48 |           <div
49 |             key={review.id}
50 |             role="listitem"
51 |             className={cn(
52 |               "bg-white border border-slate-200 rounded-lg p-3 transition-shadow",
53 |               isPending && "hover:shadow-sm",
54 |               (isCompleted || isSkipped) && "opacity-60"
55 |             )}
56 |           >
57 |             <div className="flex items-start justify-between gap-2">
58 |               { /* Left: Badge + Info */ }
59 |               <div className="flex-1 min-w-0 space-y-1.5">
60 |                 <ReviewSessionBadge
61 |                   intervalDays={review.intervalDays}
62 |                   status={review.status}
63 |                 />
64 |
65 |                 <p
66 |                   className={cn(
67 |                     "text-xs font-medium text-slate-700 line-clamp-1",
68 |                     isSkipped && "line-through text-slate-400"
69 |                   )}
70 |                 >
71 |                   CLO: {review.cloId}
72 |                 </p>
73 |
74 |                 {review.courseId && (
75 |                   <p className="text-[11px] text-slate-500 line-clamp-1">
76 |                     Course: {review.courseId}
77 |                   </p>
78 |                 )}
79 |
80 |                 <p className="text-[11px] text-slate-400">
81 |                   Review date: {review.reviewDate}
82 |                 </p>
83 |               </div>
84 |
85 |               { /* Right: Actions or Status */ }
86 |               <div className="flex items-center gap-1.5 shrink-0">
87 |                 {isPending && (
88 |                   <
89 |                     <Button
90 |                       size="sm"
91 |                       variant="tactile"
92 |                       className="h-7 gap-1 text-[11px]"
93 |                       onClick={() => onStartReview(review)}

```

```

94 |         >
95 |             <Play className="h-3 w-3" />
96 |             Start
97 |         </Button>
98 |         <Button
99 |             variant="outline"
100 |             size="sm"
101 |             className="h-7 gap-1 text-[11px]"
102 |             onClick={() => onSkipReview(review.id)}
103 |             disabled={isSkipping}
104 |         >
105 |             <SkipForward className="h-3 w-3" />
106 |             Skip
107 |         </Button>
108 |     </>
109 | })
110 |
111 |     {isCompleted && (
112 |         <span className="flex items-center gap-1 text-[11px] text-green-600 font-medium">
113 |             <CheckCircle2 className="h-3.5 w-3.5" />
114 |             Completed
115 |         </span>
116 |     )}
117 |
118 |     {isSkipped && (
119 |         <span className="flex items-center gap-1 text-[11px] text-slate-400 font-medium">
120 |             <SkipForward className="h-3.5 w-3.5" />
121 |             Skipped
122 |         </span>
123 |     )}
124 | </div>
125 | </div>
126 | </div>
127 | );
128 | }}}
129 | </div>
130 | );
131 | };
132 |
133 | export default ReviewScheduleList;
134 | export { ReviewScheduleList };
135 | export type { ReviewScheduleListProps };
136 |

```

FILE: src/components/shared/ReviewSessionBadge.tsx

TYPE: .tsx | LINES: 1-63 | SHA256: 713974163f8ad7868b116cf45e2732ae3a3ea5615a841d4a2945a8379ea1c7d4

```

1 | // =====
2 | // ReviewSessionBadge – Badge showing spaced repetition review interval label
3 | // ("Day 1 Review", "Day 3 Review", "Day 7 Review") with distinct color styling
4 | // per interval and a status indicator (pending, completed, skipped).
5 | // =====
6 |
7 | import { Badge } from "@components/ui/badge";
8 | import { cn } from "@lib/Utils";
9 | import type { ReviewStatus } from "@types/planner";
10 | import { CheckCircle2, SkipForward } from "lucide-react";
11 |
12 | interface ReviewSessionBadgeProps {
13 |     intervalDays: 1 | 3 | 7;
14 |     status: ReviewStatus;
15 |     className?: string;
16 | }
17 |
18 | const INTERVAL_CONFIG: Record<1 | 3 | 7, { label: string; className: string }> =
19 | {
20 |     1: {
21 |         label: "Day 1 Review",
22 |         className: "bg-purple-100 text-purple-700 border-purple-200",
23 |     },
24 |     3: {
25 |         label: "Day 3 Review",
26 |         className: "bg-blue-100 text-blue-700 border-blue-200",
27 |     },
28 |     7: {
29 |         label: "Day 7 Review",
30 |         className: "bg-green-100 text-green-700 border-green-200",
31 |     },
32 | };
33 |
34 | const ReviewSessionBadge = ({
35 |     intervalDays,
36 |     status,
37 |     className,
38 | }: ReviewSessionBadgeProps) => {
39 |     const config = INTERVAL_CONFIG[intervalDays];
40 |     const isSkipped = status === "skipped";
41 |     const isCompleted = status === "completed";
42 |
43 |     return (
44 |         <Badge
45 |             variant="outline"
46 |             className={cn(
47 |                 "text-[10px] font-bold tracking-wide uppercase gap-1",
48 |                 config.className,
49 |                 isSkipped && "opacity-60 line-through",
50 |                 className
51 |             )}
52 |         >
53 |             {isCompleted && <CheckCircle2 className="h-3 w-3" aria-hidden="true" />}
54 |             {isSkipped && <SkipForward className="h-3 w-3" aria-hidden="true" />}
55 |             {config.label}
56 |         </Badge>
57 |     );

```

```

58 | };
59 |
60 | export default ReviewSessionBadge;
61 | export { ReviewSessionBadge };
62 | export type { ReviewSessionBadgeProps };
63 |

```

FILE: src/components/shared/RoleHeaderStats.tsx

TYPE: .tsx | LINES: 1-151 | SHA256: 659c312cd7ed6630838c6449e0b00f9f15a1dbdbfaebd793252a45349837436b

```

1 | import { useTranslation } from "react-i18next";
2 | import { useAuth } from "@hooks/useAuth";
3 | import { useAdminKPIs } from "@hooks/useAdminDashboard";
4 | import { useCoordinatorKPIs } from "@hooks/useCoordinatorDashboard";
5 | import { useCourses } from "@hooks/useCourses";
6 | import { useParentKPIs } from "@hooks/useParentDashboard";
7 | import { usePrograms } from "@hooks/usePrograms";
8 | import { useTeacherKPIs } from "@hooks/useTeacherDashboard";
9 |
10 | interface HeaderStatChipProps {
11 |   emoji: string;
12 |   value: string | number;
13 |   label?: string;
14 |   color: string;
15 | }
16 |
17 | const HeaderStatChip = ({
18 |   emoji,
19 |   value,
20 |   label,
21 |   color,
22 | }: HeaderStatChipProps) => (
23 |   <span
24 |     className="stat-chip"
25 |     style={{ color }}
26 |     aria-label={label ? `${value} ${label}` : String(value)}
27 |   >
28 |     <span aria-hidden="true" style={{ fontSize: "14px", lineHeight: 1 }}>
29 |       {emoji}
30 |     </span>
31 |     <span style={{ fontWeight: 800 }}>{value}</span>
32 |     {label && <span style={{ fontWeight: 600, opacity: 0.8 }}>{label}</span>}
33 |   </span>
34 | );
35 |
36 | /**
37 |  * Prototype `.top-stats` for non-student roles.
38 |  * Emoji icons + per-chip colors match shared.js ROLE_STATS exactly.
39 |  */
40 | const RoleHeaderStats = () => {
41 |   const { t } = useTranslation("common");
42 |   const { user, role } = useAuth();
43 |   const isTeacher = role === "teacher";
44 |   const isCoordinator = role === "coordinator";
45 |   const isAdmin = role === "admin";
46 |   const isParent = role === "parent";
47 |
48 |   const teacherKPIs = useTeacherKPIs({ enabled: isTeacher });
49 |   const teacherCourses = useCourses(
50 |     { page: 1, pageSize: 1, teacherId: user?.id },
51 |     { enabled: isTeacher }
52 |   );
53 |   const coordinatorKPIs = useCoordinatorKPIs({ enabled: isCoordinator });
54 |   const programs = usePrograms(
55 |     { page: 1, pageSize: 1 },
56 |     { enabled: isCoordinator }
57 |   );
58 |   const adminKPIs = useAdminKPIs({ enabled: isAdmin });
59 |   const parentKPIs = useParentKPIs(user?.id, { enabled: isParent });
60 |
61 |   if (!role || role === "student") return null;
62 |
63 |   const containerClassName = "top-stats";
64 |
65 |   // Teacher: 0<3" N classes (teal) + ' p N to grade (amber-brown)
66 |   if (isTeacher) {
67 |     const classes = teacherCourses.data?.count ?? 0;
68 |     const toGrade = teacherKPIs.data?.pendingSubmissions ?? 0;
69 |     return (
70 |       <div
71 |         className={containerClassName}
72 |         aria-label={t("header.stats.teacher")}
73 |       >
74 |         <HeaderStatChip
75 |           emoji="0<3"
76 |           value={` ${classes} ${t("header.stats.classes")}` }
77 |           color="#0f766e"
78 |         />
79 |         <HeaderStatChip
80 |           emoji="' p "
81 |           value={` ${toGrade} ${t("header.stats.toGrade")}` }
82 |           color="#b45309"
83 |         />
84 |       </div>
85 |     );
86 |   }
87 |
88 |   // Coordinator: 0<3" N programs (blue) + & p N gaps (amber-brown)
89 |   if (isCoordinator) {
90 |     const programCount = programs.data?.count ?? 0;
91 |     const atRisk = coordinatorKPIs.data?.atRiskStudents ?? 0;
92 |     return (
93 |       <div
94 |         className={containerClassName}

```

```

95 |         aria-label={t("header.stats.coordinator")}
96 |     >
97 |     <HeaderStatChip
98 |       emoji="👤"
99 |       value={` ${programCount} ${t("header.stats.programs")}`}
100 |       color="#2563eb"
101 |     />
102 |     <HeaderStatChip
103 |       emoji="⚠️"
104 |       value={` ${atRisk} ${t("header.stats.belowTarget")}`}
105 |       color="#b45309"
106 |     />
107 |   </div>
108 | );
109 | }
110 |
111 | // Admin: 👤 N learners (blue) + green active%
112 | if (isAdmin) {
113 |   const totalUsers = adminKPIs.data?.totalUsers ?? 0;
114 |   const activePercent = totalUsers
115 |     ? Math.round(((adminKPIs.data?.activeUsers ?? 0) / totalUsers) * 100)
116 |     : 0;
117 |   const learners = adminKPIs.data?.usersByRole.student ?? 0;
118 |
119 |   return (
120 |     <div className={containerClassName} aria-label={t("header.stats.admin")}>
121 |       <HeaderStatChip
122 |         emoji="👤"
123 |         value={` ${learners.toLocaleString()} ${t("header.stats.learners")}`}
124 |         color="#2563eb"
125 |       />
126 |       <HeaderStatChip
127 |         emoji="📊"
128 |         value={` ${activePercent}% ${t("header.stats.activeAccounts")}`}
129 |         color="#16a34a"
130 |       />
131 |     </div>
132 |   );
133 | }
134 |
135 | // Parent: 📊 On track / ⚠️ Needs attention
136 | const onTrack = (parentKPIs.data?.avgAttainment ?? 0) >= 50;
137 | return (
138 |   <div className={containerClassName} aria-label={t("header.stats.parent")}>
139 |     <HeaderStatChip
140 |       emoji={onTrack ? "📊" : "⚠️"}
141 |       value={
142 |         onTrack ? t("header.stats.onTrack") : t("header.stats.needsAttention")
143 |       }
144 |       color={onTrack ? "#16a34a" : "#b45309"}
145 |     />
146 |   </div>
147 | );
148 | };
149 |
150 | export default RoleHeaderStats;
151 |

```

FILE: src/components/shared/RoleProfileAccountPanels.tsx

TYPE: .tsx | LINES: 1-95 | SHA256: 3c6e1d60f3707e1841b343659ae9301631a745575c127aaf415b363a6c5bd829

```

1 | import { KeyRound, Monitor, Moon, Sun } from "lucide-react";
2 | import { Link } from "react-router-dom";
3 | import { useTranslation } from "react-i18next";
4 | import { useAuth } from "@hooks/useAuth";
5 | import { useTheme, type ThemePreference } from "@providers/ThemeProvider";
6 | import { Button } from "@components/ui/button";
7 | import AvatarUpload from "@components/shared/AvatarUpload";
8 | import {
9 |   ProfileSectionCard,
10 |   ProfileSettingRow,
11 | } from "@components/shared/RoleProfileSurface";
12 |
13 | interface RoleProfileAccountPanelsProps {
14 |   appearanceTitle?: string;
15 |   includeAvatar?: boolean;
16 | }
17 |
18 | const RoleProfileAccountPanels = ({
19 |   appearanceTitle,
20 |   includeAvatar = true,
21 | }: RoleProfileAccountPanelsProps) => {
22 |   const { t } = useTranslation("common");
23 |   const { profile } = useAuth();
24 |   const { theme, setTheme } = useTheme();
25 |
26 |   return (
27 |     <
28 |       <ProfileSectionCard
29 |         emoji="👤"
30 |         title={appearanceTitle ?? t("roleProfile.appearance")}
31 |       >
32 |         <div
33 |           className="grid grid-cols-3 gap-2 p-4"
34 |           role="radiogroup"
35 |           aria-label={appearanceTitle ?? t("roleProfile.appearance")}
36 |         >
37 |           {[
38 |             ["light", t("theme.light"), Sun],
39 |             ["dark", t("theme.dark"), Moon],
40 |             ["system", t("theme.system"), Monitor],
41 |           ] as const}
42 |             .map(([value, label, Icon]) => (
43 |

```



```

44 |         <Button
45 |             key={value}
46 |             type="button"
47 |             role="radio"
48 |             aria-checked={theme === value}
49 |             variant={theme === value ? "default" : "outline"}
50 |             className="h-auto flex-col gap-1.5 rounded-xl py-3"
51 |             onClick={() => setTheme(value as ThemePreference)}
52 |         >
53 |             <Icon className="size-4" aria-hidden="true" />
54 |             <span className="text-xs">{label}</span>
55 |         </Button>
56 |     )}}
57 | </div>
58 | </ProfileSectionCard>
59 |
60 | <ProfileSectionCard emoji="ðŸ" title={t("roleProfile.security")}>
61 |     <ProfileSettingRow
62 |         title={t("roleProfile.changePassword")}
63 |         description={t("roleProfile.securityDescription")}
64 |         trailing={
65 |             <Button asChild variant="outline" size="sm">
66 |                 <Link to="/update-password">
67 |                     <KeyRound className="size-4" aria-hidden="true" />
68 |                     {t("roleProfile.changePassword")}
69 |                 </Link>
70 |             </Button>
71 |         }
72 |     />
73 |     <ProfileSettingRow
74 |         title={t("roleProfile.notifications")}
75 |         trailing={
76 |             <Button asChild variant="outline" size="sm">
77 |                 <Link to={`/${profile?.role ?? "student"}/notifications`}>
78 |                     {t("roleProfile.open")}
79 |                 </Link>
80 |             </Button>
81 |         }
82 |     />
83 | </ProfileSectionCard>
84 |
85 | {includeAvatar && profile?.id ? (
86 |     <div id="profile-photo">
87 |         <AvatarUpload userId={profile.id} currentUrl={profile.avatar_url} />
88 |     </div>
89 | ) : null}
90 | </>
91 | );
92 | };
93 |
94 | export default RoleProfileAccountPanels;
95 |

```

FILE: [src/components/shared/RoleProfileSurface.tsx](#)

TYPE: .tsx | LINES: 1-428 | SHA256: 176482180ba47ed7dda652c44a0af259f06f3e65477c9888c786d123b2d1ad13

```

1 | import type { ReactNode } from "react";
2 | import type { LucideIcon } from "lucide-react";
3 | import {
4 |     Bell,
5 |     KeyRound,
6 |     Mail,
7 |     Monitor,
8 |     Moon,
9 |     ShieldCheck,
10 |     Sun,
11 |     UserRound,
12 | } from "lucide-react";
13 | import { Link } from "react-router-dom";
14 | import { useTranslation } from "react-i18next";
15 | import { useAuth } from "@/hooks/useAuth";
16 | import { useTheme, type ThemePreference } from "@/providers/ThemeProvider";
17 | import { Avatar, AvatarFallback, AvatarImage } from "@/components/ui/avatar";
18 | import { Button } from "@/components/ui/button";
19 | import AvatarUpload from "@/components/shared/AvatarUpload";
20 | import EmailPreferencesSection from "@/components/shared/EmailPreferencesSection";
21 | import { PCard, SectionHeader, Shimmer } from "@/design-system";
22 | import { cn } from "@/lib/utils";
23 |
24 | export interface RoleProfileStat {
25 |     label: string;
26 |     value: string | number;
27 |     icon?: LucideIcon;
28 |     emoji?: string;
29 |     tone?: "blue" | "green" | "amber" | "teal";
30 | }
31 |
32 | export interface RoleProfileRow {
33 |     label: string;
34 |     value: string | number;
35 | }
36 |
37 | export interface RoleProfileLink {
38 |     label: string;
39 |     to: string;
40 |     icon: LucideIcon;
41 | }
42 |
43 | interface RoleProfileSurfaceProps {
44 |     roleLabel: string;
45 |     scopeLabel: string;
46 |     stats: RoleProfileStat[];
47 |     isLoading?: boolean;
48 |     overviewTitle: string;

```

```

49 | overviewDescription: string;
50 | overviewRows: RoleProfileRow[];
51 | links: RoleProfileLink[];
52 | primaryActionLabel?: string;
53 | primaryActionHref?: string;
54 | contactRows?: Array<{
55 |     emoji: string;
56 |     label: string;
57 |     href?: string;
58 | }>;
59 | children?: ReactNode;
60 | }
61 |
62 | interface ProfileSectionCardProps {
63 |     emoji: string;
64 |     title: string;
65 |     action?: ReactNode;
66 |     children: ReactNode;
67 |     className?: string;
68 | }
69 |
70 | export const ProfileSectionCard = ({
71 |     emoji,
72 |     title,
73 |     action,
74 |     children,
75 |     className,
76 | }: ProfileSectionCardProps) => (
77 |     <PCard className={cn("overflow-hidden p-0", className)}>
78 |         <div className="flex min-h-14 items-center gap-2 border-b border-slate-100 px-4 py-3">
79 |             <span
80 |                 className="inline-flex size-8 shrink-0 items-center justify-center rounded-xl border border-slate-100
81 |                 aria-hidden="true"
82 |             >
83 |                 {emoji}
84 |             </span>
85 |             <h2 className="min-w-0 flex-1 text-sm font-black text-slate-950">
86 |                 {title}
87 |             </h2>
88 |             {action}
89 |         </div>
90 |         {children}
91 |     </PCard>
92 | );
93 |
94 | interface ProfileSettingRowProps {
95 |     title: string;
96 |     description?: string;
97 |     trailing: ReactNode;
98 |     danger?: boolean;
99 | }
100 |
101 | export const ProfileSettingRow = ({
102 |     title,
103 |     description,
104 |     trailing,
105 |     danger = false,
106 | }: ProfileSettingRowProps) => (
107 |     <div className="flex min-h-14 items-center justify-between gap-4 border-b border-slate-100 px-4 py-3 last:bo
108 |     <div className="min-w-0">
109 |         <p
110 |             className={cn(
111 |                 "text-sm font-semibold",
112 |                 danger ? "text-red-600" : "text-slate-800"
113 |             )}
114 |         >
115 |             {title}
116 |         </p>
117 |         {description ? (
118 |             <p className="mt-0.5 text-xs leading-4 text-slate-400">{description}</p>
119 |         ) : null}
120 |     </div>
121 |     <div className="shrink-0">{trailing}</div>
122 | </div>
123 | );
124 |
125 | const toneClasses: Record<NonNullable<RoleProfileStat["tone"]>, string> = {
126 |     blue: "bg-blue-50 text-blue-700",
127 |     green: "bg-emerald-50 text-emerald-700",
128 |     amber: "bg-amber-50 text-amber-700",
129 |     teal: "bg-teal-50 text-teal-700",
130 | };
131 |
132 | const RoleProfileSurface = ({
133 |     roleLabel,
134 |     scopeLabel,
135 |     stats,
136 |     isLoading = false,
137 |     overviewTitle,
138 |     overviewDescription,
139 |     overviewRows,
140 |     links,
141 |     primaryActionLabel,
142 |     primaryActionHref,
143 |     contactRows,
144 |     children,
145 | }: RoleProfileSurfaceProps) => {
146 |     const { t } = useTranslation("common");
147 |     const { profile } = useAuth();
148 |     const { theme, setTheme } = useTheme();
149 |
150 |     const fullName = profile?.full_name || t("roleProfile.fallbackName");
151 |     const initials = fullName
152 |         .split(/\s+/)

```

```

153 | .filter(Boolean)
154 | .slice(0, 2)
155 | .map((part) => part[0])
156 | .join("")
157 | .toUpperCase();
158 |
159 | return (
160 |   <div className="space-y-5">
161 |     <PCard className="overflow-hidden p-5">
162 |       <div className="flex flex-col gap-5 lg:flex-row lg:items-start lg:justify-between">
163 |         <div className="min-w-0 flex-1">
164 |           <div className="flex items-center gap-4">
165 |             <Avatar className="size-16 rounded-2xl">
166 |               <AvatarImage
167 |                 src={
168 |                   profile?.avatar_url
169 |                     ? `${profile.avatar_url}?width=128&height=128&resize=cover`
170 |                     : undefined
171 |                 }
172 |                 alt={fullName}
173 |                 className="object-cover"
174 |               />
175 |               <AvatarFallback className="rounded-2xl bg-[image:var(--brand-gradient)] text-xl font-black tex
176 |                 {initials || "U"}
177 |               />
178 |             </Avatar>
179 |             <div className="min-w-0">
180 |               <h1 className="truncate text-[22px] font-black tracking-tight text-slate-900">
181 |                 {fullName}
182 |               </h1>
183 |               <p className="truncate text-sm font-semibold text-slate-600">
184 |                 {roleLabel}
185 |                 <span className="font-medium text-slate-400">
186 |                   {" "}
187 |                   · {scopeLabel}
188 |                 </span>
189 |               </p>
190 |               <p className="mt-1 truncate text-xs text-slate-500">
191 |                 {profile?.email || t("roleProfile.noEmail")}
192 |               </p>
193 |             </div>
194 |           </div>
195 |
196 |           <div className="mt-4 flex flex-wrap gap-x-5 gap-y-3">
197 |             {isLoading
198 |               ? stats.map((stat) => (
199 |                 <Shimmer
200 |                   key={stat.label}
201 |                   className="h-[48px] w-[116px] rounded-xl"
202 |                 />
203 |               ))
204 |               : stats.map((stat) => {
205 |                 const Icon = stat.icon;
206 |                 return (
207 |                   <div
208 |                     key={stat.label}
209 |                     className="flex min-w-[104px] items-center gap-2"
210 |                   >
211 |                     <span
212 |                       className={cn(
213 |                         "inline-flex size-8 shrink-0 items-center justify-center rounded-lg",
214 |                         toneClasses[stat.tone ?? "blue"]
215 |                       )}
216 |                     >
217 |                       {stat.emoji ? (
218 |                         <span aria-hidden="true">{stat.emoji}</span>
219 |                       ) : Icon ? (
220 |                         <Icon className="size-4" aria-hidden="true" />
221 |                       ) : null}
222 |                     </span>
223 |                     <span className="min-w-0">
224 |                       <strong className="block truncate text-sm font-black text-slate-900">
225 |                         {typeof stat.value === "number"
226 |                           ? stat.value.toLocaleString()
227 |                           : stat.value}
228 |                       </strong>
229 |                       <span className="block truncate text-[10px] font-bold uppercase tracking-[0.06em] te
230 |                         {stat.label}
231 |                       </span>
232 |                     </span>
233 |                   </div>
234 |                 );
235 |               }
236 |             </div>
237 |           </div>
238 |
239 |           <div className="grid shrink-0 grid-cols-2 gap-2 lg:w-[288px] lg:grid-cols-1">
240 |             <Button asChild variant="outline" size="sm">
241 |               <a href="#profile-photo">{t("roleProfile.editProfile")}</a>
242 |             </Button>
243 |             <Button asChild variant="tactile" size="sm">
244 |               {primaryActionHref?.startsWith("/") ? (
245 |                 <Link to={primaryActionHref}>
246 |                   <Mail className="size-4" aria-hidden="true" />
247 |                   {primaryActionLabel ?? t("roleProfile.contact")}
248 |                 </Link>
249 |               ) : (
250 |                 <a href={primaryActionHref ?? `mailto:${profile?.email ?? ""}`}>
251 |                   <Mail className="size-4" aria-hidden="true" />
252 |                   {primaryActionLabel ?? t("roleProfile.contact")}
253 |                 </a>
254 |               )}
255 |             </Button>
256 |           </div>

```

```

257 |     {(
258 |       contactRows ?? [
259 |         {
260 |           emoji: " " b " ",
261 |           label: profile?.email || t("roleProfile.noEmail"),
262 |           href: profile?.email
263 |             ? `mailto:${profile.email}`
264 |             : undefined,
265 |         },
266 |       ],
267 |     ).map((row) => (
268 |       <div key={` ${row.emoji}-${row.label}`} className="flex gap-2">
269 |         <span className="w-5 text-center" aria-hidden="true">
270 |           {row.emoji}
271 |         </span>
272 |         {row.href ? (
273 |           <a
274 |             href={row.href}
275 |             className="min-w-0 truncate text-blue-600 hover:underline"
276 |           >
277 |             {row.label}
278 |           </a>
279 |         ) : (
280 |           <span className="min-w-0 truncate">{row.label}</span>
281 |         )}
282 |       </div>
283 |     ))}
284 |   </div>
285 | </div>
286 | </div>
287 | </PCard>
288 |
289 | {children ? (
290 |   children
291 | ) : (
292 |   <div className="grid gap-5 xl:grid-cols-[minmax(0,1.45fr)_minmax(280px,0.75fr)]">
293 |     <div className="space-y-5">
294 |       <PCard className="overflow-hidden p-0">
295 |         <SectionHeader
296 |           icon={ShieldCheck}
297 |           title={overviewTitle}
298 |           description={overviewDescription}
299 |           className="border-b border-slate-100 px-5 py-4"
300 |         />
301 |         <dl className="divide-y divide-slate-100 px-5">
302 |           {overviewRows.map((row) => (
303 |             <div
304 |               key={row.label}
305 |               className="flex items-center justify-between gap-4 py-3"
306 |             >
307 |               <dt className="text-sm font-medium text-slate-500">
308 |                 {row.label}
309 |               </dt>
310 |               <dd className="text-end text-sm font-bold text-slate-900">
311 |                 {row.value}
312 |               </dd>
313 |             </div>
314 |           ))}
315 |         </dl>
316 |       </PCard>
317 |
318 |       <PCard className="overflow-hidden p-0">
319 |         <SectionHeader
320 |           icon={UserRound}
321 |           title={t("roleProfile.workspace")}
322 |           description={t("roleProfile.workspaceDescription")}
323 |           className="border-b border-slate-100 px-5 py-4"
324 |         />
325 |         <nav
326 |           className="grid gap-2 p-4 sm:grid-cols-2"
327 |           aria-label={t("roleProfile.workspace")}
328 |         >
329 |           {links.map((link) => {
330 |             const Icon = link.icon;
331 |             return (
332 |               <Button
333 |                 key={link.to}
334 |                 asChild
335 |                 variant="outline"
336 |                 className="h-auto justify-start gap-3 rounded-xl px-3 py-3"
337 |               >
338 |                 <Link to={link.to}>
339 |                   <span className="inline-flex size-9 items-center justify-center rounded-lg bg-blue-50"
340 |                     <Icon className="size-4" aria-hidden="true" />
341 |                   </span>
342 |                   <span className="truncate">{link.label}</span>
343 |                 </Link>
344 |               </Button>
345 |             );
346 |           })}
347 |         </nav>
348 |       </PCard>
349 |
350 |       <EmailPreferencesSection />
351 |     </div>
352 |
353 |     <div className="space-y-5">
354 |       <PCard className="overflow-hidden p-0">
355 |         <SectionHeader
356 |           icon={Sun}
357 |           title={t("roleProfile.appearance")}
358 |           description={t("roleProfile.appearanceDescription")}
359 |           className="border-b border-slate-100 px-5 py-4"
360 |         />

```

```

361 |         <div
362 |             className="grid grid-cols-3 gap-2 p-4"
363 |             role="radiogroup"
364 |             aria-label={t("roleProfile.appearance")}
365 |         >
366 |             {() => {
367 |                 [
368 |                     ["light", t("theme.light"), Sun],
369 |                     ["dark", t("theme.dark"), Moon],
370 |                     ["system", t("theme.system"), Monitor],
371 |                 ] as const
372 |             }).map(([value, label, Icon]) => {
373 |                 <Button
374 |                     key={value}
375 |                     type="button"
376 |                     role="radio"
377 |                     aria-checked={theme === value}
378 |                     variant={theme === value ? "default" : "outline"}
379 |                     className="h-auto flex-col gap-1.5 rounded-xl py-3"
380 |                     onClick={() => setTheme(value as ThemePreference)}
381 |                 >
382 |                     <Icon className="size-4" aria-hidden="true" />
383 |                     <span className="text-xs">{label}</span>
384 |                 </Button>
385 |             )}
386 |         </div>
387 |     </PCard>
388 |
389 |     <div id="profile-photo">
390 |         {profile?.id ? (
391 |             <AvatarUpload
392 |                 userId={profile.id}
393 |                 currentUrl={profile.avatar_url}
394 |             />
395 |         ) : null}
396 |     </div>
397 |
398 |     <PCard className="overflow-hidden p-0">
399 |         <SectionHeader
400 |             icon={KeyRound}
401 |             title={t("roleProfile.security")}
402 |             description={t("roleProfile.securityDescription")}
403 |             className="border-b border-slate-100 px-5 py-4"
404 |         />
405 |         <div className="grid gap-2 p-4">
406 |             <Button asChild variant="outline" className="justify-start">
407 |                 <Link to="/update-password">
408 |                     <KeyRound className="size-4" aria-hidden="true" />
409 |                     {t("roleProfile.changePassword")}
410 |                 </Link>
411 |             </Button>
412 |             <Button asChild variant="outline" className="justify-start">
413 |                 <Link to={`/${profile?.role ?? "student"}/notifications`} >
414 |                     <Bell className="size-4" aria-hidden="true" />
415 |                     {t("roleProfile.notifications")}
416 |                 </Link>
417 |             </Button>
418 |         </div>
419 |     </PCard>
420 | </div>
421 | </div>
422 | )}
423 | </div>
424 | );
425 | };
426 |
427 | export default RoleProfileSurface;
428 |

```

FILE: src/components/shared/RubricPreview.tsx

TYPE: .tsx | LINES: 1-148 | SHA256: 04a8770f1ac7f0a2b341656f1f87e6e7ec924f98e911df8f04de6f4b4f653ba7

```

1 | import { useMemo } from "react";
2 | import { cn } from "@lib/utills";
3 | import { Card } from "@components/ui/card";
4 | import { Badge } from "@components/ui/badge";
5 | import type { RubricWithCriteria } from "@hooks/useRubrics";
6 |
7 | interface RubricPreviewProps {
8 |   rubric: RubricWithCriteria;
9 |   selectedCells?: Record<string, number>; // criterion_id !' level_index
10 |   onCellClick?: (criterionId: string, levelIndex: number) => void;
11 | }
12 |
13 | const RubricPreview = ({
14 |   rubric,
15 |   selectedCells,
16 |   onCellClick,
17 | }: RubricPreviewProps) => {
18 |   const sortedCriteria = useMemo(
19 |     () => [...rubric.criteria].sort((a, b) => a.sort_order - b.sort_order),
20 |     [rubric.criteria]
21 |   );
22 |
23 |   const totalMaxScore = useMemo(
24 |     () => sortedCriteria.reduce((sum, c) => sum + c.max_points, 0),
25 |     [sortedCriteria]
26 |   );
27 |
28 |   const levelLabels = useMemo(() => {
29 |     const first = sortedCriteria[0];
30 |     if (!first) return [];
31 |     return first.levels.map((l) => l.label);
32 |   }, [sortedCriteria]);

```

```

33 |
34 | if (sortedCriteria.length === 0) {
35 |   return (
36 |     <Card className="bg-white border-0 shadow-md rounded-xl p-6">
37 |       <p className="text-sm text-gray-500">
38 |         No criteria defined for this rubric.
39 |       </p>
40 |     </Card>
41 |   );
42 | }
43 |
44 | if (levelLabels.length === 0) {
45 |   return (
46 |     <Card className="bg-white border-0 shadow-md rounded-xl p-6">
47 |       <p className="text-sm text-gray-500">No performance levels defined.</p>
48 |     </Card>
49 |   );
50 | }
51 |
52 | const isInteractive = !!onCellClick;
53 |
54 | return (
55 |   <Card className="bg-white border-0 shadow-md rounded-xl p-6 space-y-4">
56 |     <h3 className="text-lg font-bold tracking-tight">{rubric.title}</h3>
57 |
58 |     <div className="overflow-x-auto">
59 |       <table className="w-full border-collapse">
60 |         <thead>
61 |           <tr>
62 |             <th className="p-3 text-start text-xs font-bold uppercase tracking-wider text-gray-500 border bo
63 |               Criterion
64 |             </th>
65 |             {levelLabels.map((label, idx) => (
66 |               <th
67 |                 key={idx}
68 |                 className="p-3 text-center text-xs font-bold uppercase tracking-wider text-gray-500 border b
69 |               >
70 |                 {label}
71 |               </th>
72 |             ))}
73 |           </tr>
74 |         </thead>
75 |         <tbody>
76 |           {sortedCriteria.map((criterion) => (
77 |             <tr key={criterion.id}>
78 |               <td className="p-3 border border-slate-200 align-top">
79 |                 <span className="font-medium text-gray-700 text-sm">
80 |                   {criterion.criterion_name}
81 |                 </span>
82 |               </td>
83 |               {criterion.levels.map((level, levelIdx) => {
84 |                 const isSelected = selectedCells?.[criterion.id] === levelIdx;
85 |
86 |                 return (
87 |                   <td
88 |                     key={levelIdx}
89 |                     className={cn(
90 |                       "p-3 border align-top transition-colors",
91 |                       isSelected
92 |                         ? "bg-blue-100 border-blue-500 border-2"
93 |                         : "bg-white border-slate-200",
94 |                       isInteractive && "cursor-pointer hover:bg-blue-50"
95 |                     )}
96 |                     onClick={
97 |                       isInteractive
98 |                         ? () => onCellClick(criterion.id, levelIdx)
99 |                         : undefined
100 |                     }
101 |                     role={isInteractive ? "button" : undefined}
102 |                     tabIndex={isInteractive ? 0 : undefined}
103 |                     onKeyDown={
104 |                       isInteractive
105 |                         ? (e) => {
106 |                             if (e.key === "Enter" || e.key === " ") {
107 |                               e.preventDefault();
108 |                               onCellClick(criterion.id, levelIdx);
109 |                             }
110 |                           }
111 |                         : undefined
112 |                     }
113 |                   >
114 |                     <div className="space-y-2">
115 |                       <p className="text-xs text-gray-600">
116 |                         {level.description || "-"}
117 |                       </p>
118 |                       <Badge variant="secondary" className="text-xs">
119 |                         {level.points} pt{level.points !== 1 ? "s" : ""}
120 |                       </Badge>
121 |                     </div>
122 |                   </td>
123 |                 );
124 |               })}
125 |             </tr>
126 |           )}
127 |         </tbody>
128 |         <tfoot>
129 |           <tr className="border-t-2 border-slate-300">
130 |             <td
131 |               className="p-3 text-end text-sm font-bold text-gray-700"
132 |               colSpan={levelLabels.length + 1}
133 |             >
134 |               Total Score:{" "}
135 |               <span className="text-lg font-black text-blue-600">
136 |                 {totalMaxScore}

```

```

137 |         </span>
138 |       </td>
139 |     </tr>
140 |   </tfoot>
141 | </table>
142 | </div>
143 | </Card>
144 | );
145 | };
146 |
147 | export default RubricPreview;
148 |

```

FILE: src/components/shared/RubricPreviewDialog.tsx

TYPE: .tsx | LINES: 1-77 | SHA256: 181beedea8e134980c88e1d77b740ff2e60ba534fb81528a3dfba75c79519a28

```

1 | import { Ruler } from "lucide-react";
2 | import {
3 |   Dialog,
4 |   DialogContent,
5 |   DialogDescription,
6 |   DialogHeader,
7 |   DialogTitle,
8 | } from "@components/ui/dialog";
9 | import { Shimmer } from "@design-system";
10 | import { InlineEmpty } from "@components/shared/EmptyState";
11 | import RubricPreview from "@components/shared/RubricPreview";
12 | import { useRubric } from "@hooks/useRubrics";
13 |
14 | interface RubricPreviewDialogProps {
15 |   /**
16 |    * The id of the rubric to preview. When `null` the query stays disabled and
17 |    * the dialog content renders nothing meaningful – pair this with `open` so
18 |    * the dialog only mounts content for a real rubric.
19 |    */
20 |   rubricId: string | null;
21 |   /** Controlled open state. */
22 |   open: boolean;
23 |   /** Controlled open-change handler (also wired to the dialog close button). */
24 |   onOpenChange: (open: boolean) => void;
25 | }
26 |
27 | /**
28 |  * Read-only rubric preview surfaced from the rubric list "Preview" action
29 |  * (Req 14.2). It loads the rubric's criteria and levels via `useRubric` and
30 |  * renders them through the presentational `RubricPreview` table with **no edit
31 |  * controls** (Req 14.3). Loading shows a shimmer; a missing rubric and a query
32 |  * error each get a distinct empty state.
33 |  */
34 | const RubricPreviewDialog = ({
35 |   rubricId,
36 |   open,
37 |   onOpenChange,
38 | }: RubricPreviewDialogProps) => {
39 |   const { data: rubric, isLoading, isError } = useRubric(rubricId ?? undefined);
40 |
41 |   return (
42 |     <Dialog open={open} onOpenChange={onOpenChange}>
43 |       <DialogContent className="max-w-3xl max-h-[85vh] overflow-y-auto">
44 |         <DialogHeader>
45 |           <DialogTitle>Rubric Preview</DialogTitle>
46 |           <DialogDescription>
47 |             Read-only view of the rubric criteria and performance levels.
48 |           </DialogDescription>
49 |         </DialogHeader>
50 |
51 |         {isLoading ? (
52 |           <div className="space-y-3" data-testid="rubric-preview-loading">
53 |             <Shimmer className="h-6 w-1/3" />
54 |             <Shimmer className="h-40 w-full" />
55 |           </div>
56 |         ) : isError ? (
57 |           <InlineEmpty
58 |             icon={<Ruler className="h-6 w-6 text-gray-400" />}
59 |             title="Unable to load rubric"
60 |             description="Something went wrong while loading this rubric. Please try again."
61 |           />
62 |         ) : rubric ? (
63 |           <RubricPreview rubric={rubric} />
64 |         ) : (
65 |           <InlineEmpty
66 |             icon={<Ruler className="h-6 w-6 text-gray-400" />}
67 |             title="Rubric not found"
68 |             description="This rubric may have been deleted."
69 |           />
70 |         )}
71 |       </DialogContent>
72 |     </Dialog>
73 |   );
74 | };
75 |
76 | export default RubricPreviewDialog;
77 |

```

FILE: src/components/shared/SaleBadge.tsx

TYPE: .tsx | LINES: 1-30 | SHA256: 461cbd3d23cc73fd24122ed5485d57a1aecb2152aa8427fd4c1fa15f291b196f

```

1 | // =====
2 | // SaleBadge – Sale discount badge component
3 | // =====
4 |
5 | import { Tag } from "lucide-react";
6 | import { cn } from "@lib/utills";
7 |

```

```

8 | interface SaleBadgeProps {
9 |   discountPercentage: number;
10 |   className?: string;
11 | }
12 |
13 | const SaleBadge = ({ discountPercentage, className }: SaleBadgeProps) => {
14 |   if (discountPercentage <= 0) return null;
15 |
16 |   return (
17 |     <span
18 |       className={cn(
19 |         "inline-flex items-center gap-1 rounded-full bg-red-500 px-2 py-0.5 text-xs font-bold text-white",
20 |         className
21 |       )}
22 |     >
23 |       <Tag className="h-3 w-3" />
24 |       {discountPercentage}% OFF
25 |     </span>
26 |   );
27 | };
28 |
29 | export default SaleBadge;
30 |

```

FILE: [src/components/shared/SatisfactionRating.tsx](#)

TYPE: .tsx | LINES: 1-78 | SHA256: 93babc59de8cfcc0493ef1ea6ba3e9d49dc199986ac1f1c87a7403c6fee8609f

```

1 | // =====
2 | // SatisfactionRating – Thumbs up/down rating component for tutor messages
3 | // =====
4 |
5 | import { ThumbsUp, ThumbsDown } from "lucide-react";
6 | import { Button } from "@components/ui/button";
7 | import { cn } from "@lib/utils";
8 | import type { SatisfactionRating as SatisfactionRatingType } from "@lib/tutorSchemas";
9 |
10 | interface SatisfactionRatingProps {
11 |   messageId: string;
12 |   conversationId: string;
13 |   currentRating: SatisfactionRatingType | null;
14 |   onRate: (variables: {
15 |     messageId: string;
16 |     conversationId: string;
17 |     rating: SatisfactionRatingType;
18 |   }) => void;
19 |   isPending?: boolean;
20 | }
21 |
22 | const SatisfactionRating = ({
23 |   messageId,
24 |   conversationId,
25 |   currentRating,
26 |   onRate,
27 |   isPending = false,
28 | }: SatisfactionRatingProps) => {
29 |   const handleRate = (rating: SatisfactionRatingType) => {
30 |     if (isPending) return;
31 |     onRate({ messageId, conversationId, rating });
32 |   };
33 |
34 |   return (
35 |     <div
36 |       className="flex items-center gap-1"
37 |       role="group"
38 |       aria-label="Rate this response"
39 |     >
40 |       <Button
41 |         variant="ghost"
42 |         size="icon-xs"
43 |         onClick={() => handleRate("thumbs_up")}
44 |         disabled={isPending}
45 |         aria-label="Thumbs up"
46 |         aria-pressed={currentRating === "thumbs_up"}
47 |         className={cn(
48 |           "transition-colors",
49 |           currentRating === "thumbs_up"
50 |             ? "text-green-600 bg-green-50 hover:bg-green-100"
51 |             : "text-gray-400 hover:text-green-600 hover:bg-green-50"
52 |         )}
53 |       >
54 |         <ThumbsUp className="h-3.5 w-3.5" />
55 |       </Button>
56 |       <Button
57 |         variant="ghost"
58 |         size="icon-xs"
59 |         onClick={() => handleRate("thumbs_down")}
60 |         disabled={isPending}
61 |         aria-label="Thumbs down"
62 |         aria-pressed={currentRating === "thumbs_down"}
63 |         className={cn(
64 |           "transition-colors",
65 |           currentRating === "thumbs_down"
66 |             ? "text-red-600 bg-red-50 hover:bg-red-100"
67 |             : "text-gray-400 hover:text-red-600 hover:bg-red-50"
68 |         )}
69 |       >
70 |         <ThumbsDown className="h-3.5 w-3.5" />
71 |       </Button>
72 |     </div>
73 |   );
74 | };
75 |
76 | export default SatisfactionRating;
77 | export type { SatisfactionRatingProps };

```


FILE: src/components/shared/SearchCommand.tsx

TYPE: .tsx | LINES: 1-403 | SHA256: 405dcee32b9cf4958251cb06285e6a5e8ff95463ff01304f91a1bf6a989fb90b

```

1 | // Task 92.2: Global search command palette (Cmd+K / Ctrl+K)
2 | // Debounced input, results grouped by category
3 |
4 | import { useState, useEffect, useCallback, useRef } from "react";
5 | import { useTranslation } from "react-i18next";
6 | import {
7 |   Dialog,
8 |   DialogContent,
9 |   DialogHeader,
10 |  DialogTitle,
11 | } from "@components/ui/dialog";
12 | import { Input } from "@components/ui/input";
13 | import { Button } from "@components/ui/button";
14 | import { useGlobalSearch, type SearchResult } from "@hooks/useGlobalSearch";
15 | import { useAuth } from "@hooks/useAuth";
16 | import {
17 |   BarChart3,
18 |   BookOpen,
19 |   Bot,
20 |   FileText,
21 |   GraduationCap,
22 |   LayoutDashboard,
23 |   Loader2,
24 |   Megaphone,
25 |   ShieldAlert,
26 |   Sparkles,
27 |   Target,
28 |   Users,
29 |   Search,
30 | } from "lucide-react";
31 | import { useNavigate } from "react-router-dom";
32 | import type { UserRole } from "@types/app";
33 |
34 | const DEBOUNCE_MS = 300;
35 |
36 | const categoryConfig: Record<
37 |   SearchResult["type"],
38 |   { label: string; icon: typeof BookOpen }
39 | > = {
40 |   course: { label: "Courses", icon: BookOpen },
41 |   assignment: { label: "Assignments", icon: FileText },
42 |   announcement: { label: "Announcements", icon: Megaphone },
43 | };
44 |
45 | interface SearchCommandProps {
46 |   showTrigger?: boolean;
47 | }
48 |
49 | interface RoleCommand {
50 |   section: "goTo" | "actions";
51 |   labelKey: string;
52 |   to: string;
53 |   icon: typeof Search;
54 | }
55 |
56 | const commandsByRole: Record<UserRole, RoleCommand[]> = {
57 |   student: [
58 |     {
59 |       section: "goTo",
60 |       labelKey: "header.commands.today",
61 |       to: "/student/dashboard",
62 |       icon: LayoutDashboard,
63 |     },
64 |     {
65 |       section: "goTo",
66 |       labelKey: "header.commands.learningPath",
67 |       to: "/student/courses",
68 |       icon: BookOpen,
69 |     },
70 |     {
71 |       section: "goTo",
72 |       labelKey: "header.commands.tutor",
73 |       to: "/student/tutor",
74 |       icon: Bot,
75 |     },
76 |     {
77 |       section: "goTo",
78 |       labelKey: "header.commands.progress",
79 |       to: "/student/progress",
80 |       icon: BarChart3,
81 |     },
82 |     {
83 |       section: "actions",
84 |       labelKey: "header.commands.fixWeakest",
85 |       to: "/student/progress",
86 |       icon: Target,
87 |     },
88 |     {
89 |       section: "actions",
90 |       labelKey: "header.commands.askTutor",
91 |       to: "/student/tutor",
92 |       icon: Sparkles,
93 |     },
94 |   ],
95 |   teacher: [
96 |     {
97 |       section: "goTo",
98 |       labelKey: "header.commands.home",
99 |       to: "/teacher/dashboard",

```

```

100 |         icon: LayoutDashboard,
101 |     },
102 |     {
103 |         section: "goTo",
104 |         labelKey: "header.commands.studentTriage",
105 |         to: "/teacher/students",
106 |         icon: Users,
107 |     },
108 |     {
109 |         section: "goTo",
110 |         labelKey: "header.commands.curriculumStudio",
111 |         to: "/teacher/modules",
112 |         icon: BookOpen,
113 |     },
114 |     {
115 |         section: "goTo",
116 |         labelKey: "header.commands.gradingQueue",
117 |         to: "/teacher/grading",
118 |         icon: GraduationCap,
119 |     },
120 |     {
121 |         section: "actions",
122 |         labelKey: "header.commands.draftFeedback",
123 |         to: "/teacher/grading",
124 |         icon: Sparkles,
125 |     },
126 |     {
127 |         section: "actions",
128 |         labelKey: "header.commands.uploadLessons",
129 |         to: "/teacher/modules",
130 |         icon: Target,
131 |     },
132 | ],
133 | parent: [
134 |     {
135 |         section: "goTo",
136 |         labelKey: "header.commands.home",
137 |         to: "/parent/dashboard",
138 |         icon: LayoutDashboard,
139 |     },
140 |     {
141 |         section: "goTo",
142 |         labelKey: "header.commands.growth",
143 |         to: "/parent/progress",
144 |         icon: BarChart3,
145 |     },
146 |     {
147 |         section: "goTo",
148 |         labelKey: "header.commands.support",
149 |         to: "/parent/support",
150 |         icon: Megaphone,
151 |     },
152 | ],
153 | coordinator: [
154 |     {
155 |         section: "goTo",
156 |         labelKey: "header.commands.home",
157 |         to: "/coordinator/dashboard",
158 |         icon: LayoutDashboard,
159 |     },
160 |     {
161 |         section: "goTo",
162 |         labelKey: "header.commands.outcomes",
163 |         to: "/coordinator/plos",
164 |         icon: Target,
165 |     },
166 |     {
167 |         section: "goTo",
168 |         labelKey: "header.commands.curriculumMatrix",
169 |         to: "/coordinator/matrix",
170 |         icon: BookOpen,
171 |     },
172 |     {
173 |         section: "goTo",
174 |         labelKey: "header.commands.accreditation",
175 |         to: "/coordinator/accreditation",
176 |         icon: FileText,
177 |     },
178 | ],
179 | admin: [
180 |     {
181 |         section: "goTo",
182 |         labelKey: "header.commands.home",
183 |         to: "/admin/dashboard",
184 |         icon: LayoutDashboard,
185 |     },
186 |     {
187 |         section: "goTo",
188 |         labelKey: "header.commands.analytics",
189 |         to: "/admin/reports",
190 |         icon: BarChart3,
191 |     },
192 |     {
193 |         section: "goTo",
194 |         labelKey: "header.commands.aiGovernance",
195 |         to: "/admin/security",
196 |         icon: ShieldAlert,
197 |     },
198 |     {
199 |         section: "goTo",
200 |         labelKey: "header.commands.people",
201 |         to: "/admin/users",
202 |         icon: Users,
203 |     },

```

```

204 |   ],
205 | };
206 |
207 | const SearchCommand = ({ showTrigger = false }: SearchCommandProps) => {
208 |   const { t } = useTranslation("common");
209 |   const [open, setOpen] = useState(false);
210 |   const [input, setInput] = useState("");
211 |   const [debouncedQuery, setDebouncedQuery] = useState("");
212 |   const { role } = useAuth();
213 |   const navigate = useNavigate();
214 |   const inputRef = useRef<HTMLInputElement>(null);
215 |
216 |   const { data: results, isLoading } = useGlobalSearch(debouncedQuery, role);
217 |   const roleCommands = commandsByRole[role ?? "student"];
218 |   const matchingCommands = roleCommands.filter((command) =>
219 |     t(command.labelKey).toLowerCase().includes(input.trim().toLowerCase())
220 |   );
221 |
222 |   // Debounce input
223 |   useEffect(() => {
224 |     const timer = setTimeout(() => setDebouncedQuery(input), DEBOUNCE_MS);
225 |     return () => clearTimeout(timer);
226 |   }, [input]);
227 |
228 |   // Keyboard shortcut: Cmd+K / Ctrl+K
229 |   const handleKeyDown = useCallback((e: KeyboardEvent) => {
230 |     if ((e.metaKey || e.ctrlKey) && e.key === "k") {
231 |       e.preventDefault();
232 |       setOpen((prev) => !prev);
233 |     }
234 |   }, []);
235 |
236 |   useEffect(() => {
237 |     document.addEventListener("keydown", handleKeyDown);
238 |     return () => document.removeEventListener("keydown", handleKeyDown);
239 |   }, [handleKeyDown]);
240 |
241 |   // Focus input when dialog opens; reset state on close via onOpenChange
242 |   useEffect(() => {
243 |     if (open) {
244 |       setTimeout(() => inputRef.current?.focus(), 50);
245 |     }
246 |   }, [open]);
247 |
248 |   const handleOpenChange = (nextOpen: boolean) => {
249 |     setOpen(nextOpen);
250 |     if (!nextOpen) {
251 |       setInput("");
252 |       setDebouncedQuery("");
253 |     }
254 |   };
255 |
256 |   const handleSelect = (result: SearchResult) => {
257 |     setOpen(false);
258 |     navigate(result.url);
259 |   };
260 |
261 |   const handleCommandSelect = (command: RoleCommand) => {
262 |     setOpen(false);
263 |     navigate(command.to);
264 |   };
265 |
266 |   // Group results by type
267 |   const grouped: Record<string, SearchResult[]> = {};
268 |   for (const r of results ?? []) {
269 |     const list = grouped[r.type] ?? [];
270 |     list.push(r);
271 |     grouped[r.type] = list;
272 |   }
273 |
274 |   return (
275 |     <>
276 |       {showTrigger ? (
277 |         <Button
278 |           type="button"
279 |           variant="ghost"
280 |           onClick={() => setOpen(true)}
281 |           className="top-search h-auto justify-start"
282 |           aria-label={t("header.openGlobalSearch")}
283 |         >
284 |           <span className="flex items-center gap-2">
285 |             <Search aria-hidden="true" />
286 |             {t("header.search")}
287 |           </span>
288 |           <kbd className="kbd"># K</kbd>
289 |         </Button>
290 |       ) : null}
291 |       <Dialog open={open} onOpenChange={handleOpenChange}>
292 |         <DialogContent className="sm:max-w-lg p-0 gap-0">
293 |           <DialogHeader className="sr-only">
294 |             <DialogTitle>{t("header.search")}</DialogTitle>
295 |           </DialogHeader>
296 |
297 |           <div className="flex items-center gap-2 border-b px-4 py-3">
298 |             <Search className="h-4 w-4 text-gray-400 shrink-0" />
299 |             <Input
300 |               ref={inputRef}
301 |               placeholder={t("header.searchPlaceholder")}
302 |               value={input}
303 |               onChange={(e) => setInput(e.target.value)}
304 |               className="border-0 shadow-none focus-visible:ring-0 p-0 h-auto text-sm"
305 |             />
306 |             {isLoading && (
307 |               <Loader2 className="h-4 w-4 animate-spin text-gray-400" />

```

```

308     })
309   </div>
310
311   <div className="max-h-80 overflow-y-auto p-2">
312     {input.trim().length > 0 &&
313       !isLoading &&
314       Object.keys(grouped).length === 0 &&
315       matchingCommands.length === 0 && (
316         <p className="text-sm text-gray-500 text-center py-6">
317           {t("header.searchEmpty", { q: input })}
318         </p>
319       )}
320
321     {[("goTo", "actions") as const].map((section) => {
322       const commands = matchingCommands.filter(
323         (command) => command.section === section
324       );
325       if (commands.length === 0) return null;
326       return (
327         <div key={section} className="mb-2">
328           <p className="px-2 py-1 text-[10px] font-black uppercase tracking-widest text-gray-400">
329             {t(`header.commandSection.${section}`)}
330           </p>
331           {commands.map((command) => {
332             const Icon = command.icon;
333             return (
334               <Button
335                 key={command.labelKey}
336                 type="button"
337                 variant="ghost"
338                 className="h-auto w-full justify-start gap-3 px-3 py-2 text-start text-sm"
339                 onClick={() => handleCommandSelect(command)}
340               >
341                 <Icon
342                   className="h-4 w-4 shrink-0 text-muted-foreground"
343                   aria-hidden="true"
344                 />
345                 {t(command.labelKey)}
346               </Button>
347             );
348           })}
349         </div>
350       );
351     })}
352
353     {Object.entries(grouped).map(([type, items]) => {
354       const config = categoryConfig[type as SearchResult["type"]];
355       if (!config || !items) return null;
356       const Icon = config.icon;
357       return (
358         <div key={type} className="mb-2">
359           <p className="text-[10px] font-black tracking-widest uppercase text-gray-400 px-2 py-1">
360             {config.label}
361           </p>
362           {items.map((item) => (
363             <Button
364               key={item.id}
365               type="button"
366               variant="ghost"
367               onClick={() => handleSelect(item)}
368               className="h-auto w-full justify-start gap-3 px-3 py-2 text-start text-sm"
369             >
370               <Icon className="h-4 w-4 text-gray-400 shrink-0" />
371               <div className="min-w-0">
372                 <p className="font-medium text-gray-900 truncate">
373                   {item.title}
374                 </p>
375                 {item.description && (
376                   <p className="text-xs text-gray-500 truncate">
377                     {item.description}
378                   </p>
379                 )}
380               </div>
381             </Button>
382           ))}
383         </div>
384       );
385     })}
386
387     {!input && (
388       <p className="text-sm text-gray-400 text-center py-6">
389         {t("header.searchPrompt")}{" "}
390         <kbd className="px-1.5 py-0.5 bg-slate-100 rounded text-xs">
391           # K
392         </kbd>
393       </p>
394     )}
395   </div>
396 </DialogContent>
397 </Dialog>
398 </>
399 );
400 };
401
402 export default SearchCommand;
403

```

FILE: src/components/shared/SectionComparisonChart.tsx

TYPE: .tsx | LINES: 1-123 | SHA256: c722fa8b8a2e40a184a044a35a02d73c9a13585ece1b53d0a6bb4a6601a25971

```
1  // =====
2  // SectionComparisonChart – Compare sections on attainment metrics
3  // =====
4  // Feature: qa-partner-review-remediation – Req 10.3, 10.4, 10.5
5  //
6  // • Bars are clickable when `onSectionClick` is provided !' opens a drill-down.
7  // • A section with no attainment evidence (`sampleCount === 0`) renders an
8  //   inline empty state instead of a misleading 0% bar (Req 10.4).
9  // • Bar color is driven by the platform attainment thresholds via
10 //   getAttainmentColor (Req 10.5).
11 // =====
12
13 import { cn } from "@lib/utils";
14 import { getAttainmentColor } from "@lib/attainmentClassifier";
15
16 interface SectionData {
17   /** Stable section identifier (used for drill-down + React keys). */
18   sectionId?: string;
19   sectionCode: string;
20   attainmentPercent: number;
21   studentCount: number;
22   /**
23    * Number of attainment records backing this section. When 0, the section has
24    * no evidence yet and an inline empty state is shown instead of a 0% bar.
25    * Defaults to a truthy value so existing callers keep rendering a bar.
26    */
27   sampleCount?: number;
28 }
29
30 interface SectionComparisonChartProps {
31   sections: SectionData[];
32   className?: string;
33   /** When provided, each section row becomes a clickable button. */
34   onSectionClick?: (section: SectionData) => void;
35 }
36
37 const SectionComparisonChart = ({
38   sections,
39   className,
40   onSectionClick,
41 }: SectionComparisonChartProps) => {
42   const hasEvidence = (section: SectionData): boolean =>
43     (section.sampleCount ?? 1) > 0;
44
45   return (
46     <div className={cn("space-y-3", className)}>
47       {sections.map((section) => {
48         const showBar = hasEvidence(section);
49         const clickable = !!onSectionClick;
50
51         const RowContent = (
52           <>
53             <div className="flex items-center justify-between text-sm">
54               <span className="font-medium">Section {section.sectionCode}</span>
55               <div className="flex items-center gap-2">
56                 <span className="text-xs text-gray-400">
57                   {section.studentCount} students
58                 </span>
59                 {showBar ? (
60                   <span
61                     className="font-bold tabular-nums"
62                     style={{
63                       color: getAttainmentColor(section.attainmentPercent),
64                     }}
65                   >
66                     {Math.round(section.attainmentPercent)}%
67                   </span>
68                 ) : (
69                   <span className="text-xs italic text-gray-400">
70                     No evidence yet
71                   </span>
72                 )}
73             </div>
74           </div>
75           {showBar ? (
76             <div className="h-2.5 w-full rounded-full bg-slate-100 overflow-hidden">
77               <div
78                 className="h-full rounded-full transition-all duration-500"
79                 style={{
80                   width: `${Math.min(
81                     Math.max(section.attainmentPercent, 0),
82                     100
83                   )}%`,
84                   backgroundColor: getAttainmentColor(
85                     section.attainmentPercent
86                   )
87                 }}
88             </div>
89           ) : (
90             <div className="rounded-md border border-dashed border-slate-300 bg-slate-50 px-3 py-2 text-xs">
91               No attainment evidence recorded for this section yet.
92             </div>
93           )}
94         </div>
95       )}
96     </div>
97   );
98
99   return clickable ? (
100     <button
101       key={section.sectionId ?? section.sectionCode}
102       type="button"
```

```

102 |         onClick={() => onSectionClick(section)}
103 |         className="w-full space-y-1 rounded-lg p-1.5 text-left transition-colors hover:bg-slate-50 focus:o
104 |         aria-label={`View details for section ${section.sectionCode}`}
105 |     >
106 |         {RowContent}
107 |     </button>
108 | ) : (
109 |     <div
110 |       key={section.sectionId ?? section.sectionCode}
111 |       className="space-y-1 p-1.5"
112 |     >
113 |       {RowContent}
114 |     </div>
115 |   );
116 |   }}}
117 | </div>
118 | );
119 | };
120 |
121 | export default SectionComparisonChart;
122 | export type { SectionComparisonChartProps, SectionData };
123 |

```

FILE: src/components/shared/SectionDrillDownDialog.tsx
TYPE: .tsx | LINES: 1-160 | SHA256: b2088e15d8ed7a931b58dccf7424fb2b7d9e76f8a35b1afed4c53cfa4543ac1a

```

2 //
3 // SectionDrillDownDialog – Coordinator section drill-down (Req 10.3)
4 //
5 // Feature: qa-partner-review-remediation – Req 10.3, 10.5
6 //
7 // Opened from a clickable bar in SectionComparisonChart. Shows the section's
8 // teacher plus per-CLO attainment (with evidence sample counts), color-coded
9 // via getAttainmentColor.
10 // =====
11 import {
12   Dialog,
13   DialogContent,
14   DialogHeader,
15   DialogTitle,
16   DialogDescription,
17 } from "@components/ui/dialog";
18 import { Badge } from "@components/ui/badge";
19 import { Shimmer } from "@design-system";
20 import { useSectionDrillDown } from "@hooks/useSectionAttainment";
21 import { getAttainmentColor } from "@lib/attainmentClassifier";
22 import { GraduationCap, Users, Target } from "lucide-react";
23 import type { BloomsLevel } from "@types/app";
24
25 // % % % Bloom's badge color map (from design-system steering) % % % % % % % % % % % % % % % %
26
27 const bloomsStyles: Record<BloomsLevel, string> = {
28   remembering: "bg-purple-500 text-white",
29   understanding: "bg-blue-500 text-white",
30   applying: "bg-green-500 text-white",
31   analyzing: "bg-yellow-500 text-gray-900",
32   evaluating: "bg-orange-500 text-white",
33   creating: "bg-red-500 text-white",
34 };
35
36 interface SectionDrillDownDialogProps {
37   courseId: string | undefined;
38   sectionId: string | undefined;
39   open: boolean;
40   onOpenChange: (open: boolean) => void;
41 }
42
43 const SectionDrillDownDialog = ({
44   courseId,
45   sectionId,
46   open,
47   onOpenChange,
48 }: SectionDrillDownDialogProps) => {
49   const { data, isLoading } = useSectionDrillDown(
50     open ? courseId : undefined,
51     open ? sectionId : undefined
52   );
53
54   return (
55     <Dialog open={open} onOpenChange={onOpenChange}>
56       <DialogContent className="max-h-[80vh] overflow-y-auto sm:max-w-lg">
57         <DialogHeader>
58           {isLoading ? (
59             <>
60               <Shimmer className="h-5 w-48" />
61               <Shimmer className="h-4 w-32" />
62             </>
63           ) : data ? (
64             <>
65               <DialogTitle className="flex items-center gap-2">
66                 <Target className="h-4 w-4 text-blue-600" />
67                 Section {data.section_code}
68               </DialogTitle>
69               <DialogDescription className="flex flex-wrap items-center gap-x-4 gap-y-1">
70                 <span className="inline-flex items-center gap-1.5">
71                   <GraduationCap className="h-3.5 w-3.5" />
72                   {data.teacher_name ?? "Unassigned teacher"}
73                 </span>
74                 <span className="inline-flex items-center gap-1.5">
75                   <Users className="h-3.5 w-3.5" />
76                   {data.student_count} students
77                 </span>
78               </DialogDescription>

```

```

79 |         </>
80 |       ) : (
81 |         <DialogTitle>Section detail</DialogTitle>
82 |       )}
83 |     </DialogHeader>
84 |
85 |     <div className="space-y-3">
86 |       {isLoading ? (
87 |         <div className="space-y-2">
88 |           <Shimmer className="h-14 w-full rounded-lg" />
89 |           <Shimmer className="h-14 w-full rounded-lg" />
90 |         </div>
91 |       ) : data && data.clos.length > 0 ? (
92 |         <>
93 |           <p className="text-xs font-bold tracking-widest uppercase text-gray-500">
94 |             CLO Attainment ({data.clos.length})
95 |           </p>
96 |           <ul className="space-y-2">
97 |             {data.clos.map((clo) => (
98 |               <li
99 |                 key={clo.clo_id}
100 |                 className="rounded-lg border border-slate-200 bg-white p-3 space-y-2"
101 |               >
102 |                 <div className="flex items-start justify-between gap-3">
103 |                   <p className="text-sm font-medium text-gray-800">
104 |                     {clo.clo_title}
105 |                   </p>
106 |                   {clo.sample_count > 0 ? (
107 |                     <span
108 |                       className="text-sm font-bold tabular-nums shrink-0"
109 |                       style={{
110 |                         color: getAttainmentColor(clo.attainment_percent),
111 |                       }}
112 |                     >
113 |                       {clo.attainment_percent}%
114 |                     </span>
115 |                   ) : (
116 |                     <span className="text-xs text-gray-400 shrink-0">
117 |                       No evidence
118 |                     </span>
119 |                   )}
120 |                 </div>
121 |                 <div className="flex items-center justify-between gap-2">
122 |                   {clo.blooms_level ? (
123 |                     <Badge className={bloomsStyles[clo.blooms_level]}>
124 |                       {clo.blooms_level}
125 |                     </Badge>
126 |                   ) : (
127 |                     <span />
128 |                   )}
129 |                   {clo.sample_count > 0 && (
130 |                     <div className="h-2 flex-1 max-w-[140px] rounded-full bg-slate-100 overflow-hidden">
131 |                       <div
132 |                         className="h-full rounded-full transition-all duration-500"
133 |                         style={{
134 |                           width: `${clo.attainment_percent}%`,
135 |                           backgroundColor: getAttainmentColor(
136 |                             clo.attainment_percent
137 |                           ),
138 |                         }}
139 |                       />
140 |                     </div>
141 |                   )}
142 |                 </div>
143 |               </li>
144 |             )}
145 |           </ul>
146 |         </>
147 |       ) : (
148 |         <div className="rounded-lg border border-dashed border-slate-300 bg-slate-50 p-6 text-center text-
149 |           No course learning outcomes defined for this section yet.
150 |         </div>
151 |       )}
152 |     </div>
153 |   </DialogContent>
154 | </Dialog>
155 | );
156 | };
157 |
158 | export default SectionDrillDownDialog;
159 | export type { SectionDrillDownDialogProps };
160 |

```

FILE: src/components/shared/SectionHeader.tsx

TYPE: .tsx | LINES: 1-69 | SHA256: 0183a984514dacc131f93d2b4722be1242b531ddb04d27c5ad763a4c4ab7de27

```

1 | // =====
2 | // SectionHeader - section heading with a gradient icon chip + optional action
3 | // =====
4 | //
5 | // Presentation-only primitive introduced by the UI prototype migration
6 | // (spec: .kiro/specs/ui-prototype-migration, design §3.2). Renders a section
7 | // heading as a small brand-gradient icon CHIP + title (+ optional description)
8 | // with an optional trailing action.
9 | //
10 | // This is DISTINCT from `GradientCardHeader`, which is a full-width gradient
11 | // BAR placed at the top of a section card (white text on a gradient band).
12 | // Use SectionHeader for in-page section titles; use GradientCardHeader for the
13 | // colored header strip of a section card.
14 | //
15 | // RTL: uses logical `ms-auto` so the action sits at the inline-end in both LTR
16 | // and RTL. Dark-mode aware (title/description use theme tokens).
17 | // =====
18 |

```

```

19 import type { LucideIcon } from "lucide-react";
20 import type { ReactNode } from "react";
21
22 import { cn } from "@lib/utils";
23
24 export interface SectionHeaderProps {
25   /** Optional leading icon, shown inside the brand-gradient chip. */
26   icon?: LucideIcon;
27   /** Section title. */
28   title: string;
29   /** Optional supporting copy under the title. */
30   description?: string;
31   /** Optional trailing action(s), rendered at the inline-end. */
32   action?: ReactNode;
33   /** Heading level for the title. Default "h2". */
34   as?: "h2" | "h3";
35   className?: string;
36 }
37
38 const SectionHeader = ({
39   icon: Icon,
40   title,
41   description,
42   action,
43   as: Heading = "h2",
44   className,
45 }: SectionHeaderProps) => (
46   <div className={cn("flex items-center gap-3", className)}>
47     {Icon && (
48       <span
49         className="inline-flex size-9 shrink-0 items-center justify-center rounded-lg text-white shadow-sm"
50         style={{ background: "var(--brand-gradient)" }}
51         aria-hidden="true"
52       >
53         <Icon className="h-5 w-5" />
54       </span>
55     )}
56     <div className="min-w-0">
57       <Heading className="text-lg font-bold tracking-tight text-foreground truncate">
58         {title}
59       </Heading>
60       {description && (
61         <p className="text-sm text-muted-foreground">{description}</p>
62       )}
63     </div>
64     {action && <div className="ms-auto shrink-0">{action}</div>}
65   </div>
66 );
67
68 export default SectionHeader;
69

```

FILE: src/components/shared/SectionManager.tsx

TYPE: .tsx | LINES: 1-409 | SHA256: 3e9db7f8afc44f6824d4bc2c6d773f2b3a2646c433be70d9994c6c2e908dd0e6

[illegible]

[illegible]

[illegible]

[illegible]

```

362 |
363 |         <FormField
364 |             control={form.control}
365 |             name="capacity"
366 |             render={({ field }) => (
367 |                 <FormItem>
368 |                     <FormLabel>Capacity</FormLabel>
369 |                     <FormControl>
370 |                         <Input
371 |                             type="number"
372 |                             min={1}
373 |                             max={500}
374 |                             value={field.value ?? ""}
375 |                             onChange={(e) => field.onChange(e.target.valueAsNumber)}
376 |                             onBlur={field.onBlur}
377 |                             ref={field.ref}
378 |                             name={field.name}
379 |                         />
380 |                     </FormControl>
381 |                     <FormMessage />
382 |                 </FormItem>
383 |             )}
384 |         />
385 |
386 |         <DialogFooter>
387 |             <Button
388 |                 variant="outline"
389 |                 type="button"
390 |                 onClick={() => onOpenChange(false)}
391 |                 disabled={pending}
392 |             >
393 |                 Cancel
394 |             </Button>
395 |             <Button type="submit" disabled={pending} variant="tactile">
396 |                 {pending && <Loader2 className="h-4 w-4 animate-spin" />}
397 |                 {isEditing ? "Update" : "Create"}
398 |             </Button>
399 |         </DialogFooter>
400 |     </form>
401 | </Form>
402 | </DialogContent>
403 | </Dialog>
404 | );
405 | };
406 |
407 | export default SectionManager;
408 | export type { SectionManagerProps };
409 |

```

FILE: src/components/shared/SectionState.tsx

TYPE: .tsx | LINES: 1-95 | SHA256: 78ea9aff4b6a60800fd49cadb178c9596d354c203547b953a15827146011937b

```

1 | // =====
2 | // SectionState – declarative loading / error / empty / data wrapper for a
3 | // dashboard or page section backed by a TanStack Query.
4 | //
5 | // Spec: dashboard-and-ux-performance – Appendix B §B.3.5, Phase 7 Task 24.
6 | // Problem it solves: several sections render `null` when their query errors, so a
7 | // cancelled/timed-out request (the 8 s statement_timeout, see Appendix A) makes the
8 | // section SILENTLY VANISH – indistinguishable from "no data". This surfaces a
9 | // compact, retryable error instead, while still rendering nothing for a genuinely
10 | // empty (but successful) result.
11 | //
12 | // Composes the existing Shimmer + ErrorState primitives (no duplicated UI) and is
13 | // bilingual via the shared `common` namespace (Arabic/English) by default.
14 | // =====
15 | import type { ReactNode } from "react";
16 | import { useTranslation } from "react-i18next";
17 | import { Shimmer } from "@design-system";
18 | import ErrorState from "@components/shared/ErrorState";
19 | import { cn } from "@lib/utills";
20 |
21 | export interface SectionStateProps<T> {
22 |     /** TanStack Query `isLoading` (fetching with no data yet). */
23 |     isLoading: boolean;
24 |     /** TanStack Query `isError`. */
25 |     isError: boolean;
26 |     /** The query data (or a value hydrated from an aggregate). */
27 |     data: T | null | undefined;
28 |     /** Renders the populated section. Called only when data is present + non-empty. */
29 |     children: (data: T) => ReactNode;
30 |     /** Invoked by the error state's retry button (e.g. `query.refetch`). */
31 |     onRetry?: () => void;
32 |     /** Custom emptiness check. Defaults to: an array of length 0. */
33 |     isEmpty?: (data: T) => boolean;
34 |     /** Skeleton shown while loading with no data yet. Defaults to a Shimmer. */
35 |     loadingFallback?: ReactNode;
36 |     /** Rendered when the query SUCCEEDED but the result is empty. Defaults to null. */
37 |     emptyFallback?: ReactNode;
38 |     /** Optional error-copy overrides (default to the bilingual `common` namespace). */
39 |     errorTitle?: string;
40 |     errorMessage?: string;
41 |     /** Utility classes applied to the default Shimmer / error block. */
42 |     className?: string;
43 | }
44 |
45 | const defaultIsEmpty = (data: unknown): boolean =>
46 |     Array.isArray(data) ? data.length === 0 : data == null;
47 |
48 | export function SectionState<T>({
49 |     isLoading,
50 |     isError,
51 |     data,
52 |     children,

```

```

53 | onRetry,
54 | isEmpty = defaultIsEmpty,
55 | loadingFallback,
56 | emptyFallback = null,
57 | errorTitle,
58 | errorMessage,
59 | className,
60 | }: SectionStateProps<T>) {
61 |   const { t } = useTranslation("common");
62 |
63 |   // No data yet !' distinguish "still loading" from "failed before any success".
64 |   if (data == null) {
65 |     if (isError) {
66 |       return (
67 |         <ErrorState
68 |           title={errorTitle ?? t("errorBoundary.title")}
69 |           message={errorMessage ?? t("errors.generic")}
70 |           onRetry={onRetry}
71 |           retryLabel={t("actions.retry")}
72 |           className={cn("py-6", className)}
73 |         />
74 |       );
75 |     }
76 |     if (isLoading) {
77 |       return (
78 |         <
79 |           {loadingFallback ?? (
80 |             <Shimmer className={cn("h-32 rounded-xl", className)} />
81 |           )}
82 |         </>
83 |       );
84 |     }
85 |     // Idle / disabled with no data !' render nothing (matches prior behavior).
86 |     return <>{emptyFallback}</>;
87 |   }
88 |
89 |   // We have data (possibly stale while a background refetch errors) !' prefer it.
90 |   if (isEmpty(data)) return <>{emptyFallback}</>;
91 |   return <>{children(data)}</>;
92 | }
93 |
94 | export default SectionState;
95 |

```

FILE: src/components/shared/SessionCompletionForm.tsx

TYPE: .tsx | LINES: 1-323 | SHA256: 116f6d8f5f47fe0ef7bc5f1ae74e28500c11ecfb18f6ee31e871c6fdb8936aca

[illegible]

[illegible]

[illegible]

```

266 |         </button>
267 |
268 |         {showFullEvidence && (
269 |           <EvidenceUploader
270 |             files={evidenceFiles}
271 |             onChange={setEvidenceFiles}
272 |             disabled={isSubmitting}
273 |           />
274 |         )}
275 |       </div>
276 |
277 |       <Separator />
278 |
279 |       {/* Session Reflection */}
280 |       {reflectionSaved ? (
281 |         <div className="flex items-center gap-2 rounded-lg bg-green-50 px-4 py-3 text-sm text-green-700">
282 |           <CheckCircle2 className="h-4 w-4" />
283 |           <span className="font-medium">Reflection saved!</span>
284 |         </div>
285 |       ) : (
286 |         <SessionReflectionInput
287 |           onSave={handleReflectionSave}
288 |           isPending={saveReflection.isPending}
289 |           disabled={isSubmitting}
290 |         />
291 |       )}
292 |
293 |       <Separator />
294 |
295 |       {/* Actions */}
296 |       <div className="flex items-center gap-3">
297 |         <Button
298 |           variant="tactile"
299 |           className="flex-1"
300 |           onClick={handleSubmit}
301 |           disabled={isSubmitting}
302 |         >
303 |           {isSubmitting ? (
304 |             <Loader2 className="h-4 w-4 animate-spin me-2" />
305 |           ) : null}
306 |           Submit & Finish
307 |         </Button>
308 |         <Button
309 |           variant="outline"
310 |           onClick={handleSkip}
311 |           disabled={isSubmitting}
312 |         >
313 |           Skip
314 |         </Button>
315 |       </div>
316 |     </div>
317 |   </Card>
318 | );
319 | };
320 |
321 | export default SessionCompletionForm;
322 | export type { SessionCompletionFormProps };
323 |

```

FILE: [src/components/shared/SessionIntentDialog.tsx](#)

TYPE: .tsx | LINES: 1-259 | SHA256: 883b4e1a2c43888e9d993e6a1a273251ed2dc156418ad6b99ecd425aa9482a6f

```

1 | // =====
2 | // SessionIntentDialog – Pre-learning intent dialog shown before Focus Mode.
3 | // Collects a concept + success criterion, with auto-suggested intents as
4 | // clickable chips. Supports Skip and Submit actions.
5 | // =====
6 |
7 | import { useEffect } from "react";
8 | import { useForm } from "react-hook-form";
9 | import { zodResolver } from "@hookform/resolvers/zod";
10 | import { z } from "zod";
11 | import {
12 |   Dialog,
13 |   DialogContent,
14 |   DialogHeader,
15 |   DialogTitle,
16 |   DialogDescription,
17 |   DialogFooter,
18 | } from "@components/ui/dialog";
19 | import {
20 |   Form,
21 |   FormField,
22 |   FormItem,
23 |   FormLabel,
24 |   FormControl,
25 |   FormMessage,
26 | } from "@components/ui/form";
27 | import { Input } from "@components/ui/input";
28 | import { Button } from "@components/ui/button";
29 | import { Badge } from "@components/ui/badge";
30 | import { Loader2, Target, Lightbulb, SkipForward, Send } from "lucide-react";
31 | import { cn } from "@lib/utills";
32 | import {
33 |   useSaveSessionIntent,
34 |   useSuggestedIntents,
35 | } from "@hooks/useSessionIntent";
36 | import type { SessionIntent, SuggestedIntent } from "@types/planner";
37 |
38 | // % % % Local form schema (sessionId injected at submit time) % % % % % % % % % % % % % %
39 |
40 | const intentFormSchema = z.object({
41 |   concept: z
42 |     .string()

```


[illegible]

```

147 |     <div
148 |       className="flex flex-wrap gap-2"
149 |       role="group"
150 |       aria-label="Suggested intents"
151 |     >
152 |       {suggestions.map((suggestion, idx) => (
153 |         <button
154 |           key={idx}
155 |           type="button"
156 |           onClick={() => handleChipClick(suggestion)}
157 |           className={cn("cursor-pointer transition-colors")}
158 |           aria-label={`Use suggestion: ${suggestion.concept}`}
159 |         >
160 |           <Badge
161 |             variant="outline"
162 |             className={cn(
163 |               "px-3 py-1.5 text-xs hover:bg-teal-50 hover:border-teal-300 hover:text-teal-700 cursor-p
164 |             )}
165 |             form.watch("concept") === suggestion.concept &&
166 |             form.watch("successCriterion") ===
167 |               suggestion.successCriterion
168 |               ? "bg-teal-50 border-teal-400 text-teal-700"
169 |               : ""
170 |           >
171 |             {suggestion.concept}
172 |           </Badge>
173 |         </button>
174 |       ))}
175 |     </div>
176 |   </div>
177 | )}
178 |
179 | {suggestionsLoading && (
180 |   <div className="flex items-center gap-2 text-xs text-gray-400">
181 |     <Loader2 className="h-3 w-3 animate-spin" />
182 |     Loading suggestions...
183 |   </div>
184 | )}
185 |
186 | <Form {...form}>
187 |   <form
188 |     onSubmit={form.handleSubmit(handleSubmit)}
189 |     className="space-y-4"
190 |   >
191 |     <FormField
192 |       control={form.control}
193 |       name="concept"
194 |       render={({ field }) => (
195 |         <FormItem>
196 |           <FormLabel>What specific concept will you work on?</FormLabel>
197 |           <FormControl>
198 |             <Input
199 |               placeholder="e.g. Quadratic equations, Chapter 3 vocabulary"
200 |               {...field}
201 |             />
202 |           </FormControl>
203 |           <FormMessage />
204 |         </FormItem>
205 |       )}
206 |     />
207 |
208 |     <FormField
209 |       control={form.control}
210 |       name="successCriterion"
211 |       render={({ field }) => (
212 |         <FormItem>
213 |           <FormLabel>What does success look like?</FormLabel>
214 |           <FormControl>
215 |             <Input
216 |               placeholder="e.g. Solve 5 practice problems correctly"
217 |               {...field}
218 |             />
219 |           </FormControl>
220 |           <FormMessage />
221 |         </FormItem>
222 |       )}
223 |     />
224 |
225 |     <DialogFooter>
226 |       <Button
227 |         type="button"
228 |         variant="outline"
229 |         onClick={handleSkip}
230 |         className="gap-2"
231 |       >
232 |         <SkipForward className="h-4 w-4" />
233 |         Skip
234 |       </Button>
235 |       <Button
236 |         type="submit"
237 |         disabled={saveIntent.isPending}
238 |         variant="tactile"
239 |         className="gap-2"
240 |       >
241 |         {saveIntent.isPending ? (
242 |           <Loader2 className="h-4 w-4 animate-spin" />
243 |         ) : (
244 |           <Send className="h-4 w-4" />
245 |         )}
246 |       <Submit>
247 |     </Button>
248 |   </DialogFooter>
249 | </form>
250 |

```

```
251 |         </Form>
252 |     </DialogContent>
253 | </Dialog>
254 | });
255 | };
256 |
257 | export default SessionIntentDialog;
258 | export type { SessionIntentDialogProps };
259 |
```

FILE: `src/components/shared/SessionReflectionInput.tsx`

TYPE: .tsx | LINES: 1-162 | SHA256: ba11ce05f67edbc8d5d200d16c47a531574a06bdebf40dfd57ae9ce82abd271f

[illegible]

```

92 |         rows={4}
93 |         value={templates.freeFormContent}
94 |         onChange={(e) => templates.setFreeFormContent(e.target.value)}
95 |         disabled={disabled || isPending}
96 |         className="resize-y"
97 |         aria-describedby="reflection-word-count"
98 |       />
99 |     )}
100 |
101 |     {templates.templateType === "simple" && (
102 |       <SimpleReflectionTemplate
103 |         values={templates.simpleValues}
104 |         onChange={templates.updateSimpleField}
105 |         disabled={disabled || isPending}
106 |       />
107 |     )}
108 |
109 |     {templates.templateType === "gibbs" && (
110 |       <GibbsReflectionTemplate
111 |         values={templates.gibbsValues}
112 |         onChange={templates.updateGibbsField}
113 |         disabled={disabled || isPending}
114 |       />
115 |     )}
116 |
117 |     { /* Word Count + Save */ }
118 |     <div className="flex items-center justify-between">
119 |       { /* Word Count Indicator */ }
120 |       <div
121 |         id="reflection-word-count"
122 |         className={cn(
123 |           "text-xs font-medium transition-colors",
124 |           !hasContent
125 |             ? "text-gray-400"
126 |             : meetsMinimum
127 |               ? "text-green-600"
128 |               : "text-red-500"
129 |         )}
130 |         aria-live="polite"
131 |       >
132 |         {wordCount} / {minWords} words
133 |         {hasContent && !meetsMinimum && (
134 |           <span className="ms-1">({minWords - wordCount} more needed)</span>
135 |         )}
136 |         {meetsMinimum && <span className="ms-1">'</span>}
137 |       </div>
138 |
139 |       { /* Save Button */ }
140 |       <Button
141 |         type="button"
142 |         size="sm"
143 |         variant="tactile"
144 |         className="h-9 gap-1.5"
145 |         onClick={handleSave}
146 |         disabled={!meetsMinimum || isPending || disabled}
147 |       >
148 |         {isPending ? (
149 |           <Loader2 className="h-3.5 w-3.5 animate-spin" />
150 |         ) : (
151 |           <Save className="h-3.5 w-3.5" />
152 |         )}
153 |         Save Reflection
154 |       </Button>
155 |     </div>
156 |   </div>
157 | );
158 | };
159 |
160 | export default SessionReflectionInput;
161 | export type { SessionReflectionInputProps };
162 |

```

FILE: src/components/shared/SeverityIcon.ts

TYPE: .ts | LINES: 1-5 | SHA256: 6437d39bcac291815acc35fd1b21869e0763415f2498f287cec19cf72be0f4e6

```

1 | // Re-export shim – implementation lives in the design system (single source).
2 | // Legacy direct importers keep working; deleted at P5 once none remain.
3 | export { SeverityIcon } from "@design-system/patterns/SeverityIcon";
4 | export type { SeverityIconProps } from "@design-system/patterns/SeverityIcon";
5 |

```

FILE: src/components/shared/SeverityIcon.tsx

TYPE: .tsx | LINES: 1-88 | SHA256: 6058bc4632efee8d8622f56d78ca99c30cf4fca3a3bbcd8c706f1069fe0aada1

```

1 | // =====
2 | // SeverityIcon – leading status icon in a tinted tile with a soft halo ring
3 | // =====
4 | //
5 | // Presentation-only primitive introduced by the UI prototype migration
6 | // (spec: .kiro/specs/ui-prototype-migration, design §3.2). Replaces the old
7 | // colored "status spine" with a leading severity icon. Status is conveyed by
8 | // color + icon, and callers MUST pair it with a text label/badge nearby – never
9 | // rely on color alone (WCAG 1.4.1 / R10.3). When `label` is provided the tile is
10 | // announced to assistive tech; otherwise it is decorative (aria-hidden).
11 | //
12 | // Tokens: uses the Tailwind palette that mirrors the design-system semantic
13 | // hues (high=red/destructive, med=amber/warning, low=green/success, info=blue,
14 | // brand=teal, neutral=slate). Dark-mode aware.
15 | // =====
16 |
17 | import * as React from "react";
18 | import { cva, type VariantProps } from "class-variance-authority";
19 | import type { LucideIcon } from "lucide-react";
20 |

```

```

21 | import { cn } from "@lib/utils";
22 |
23 | const severityIconVariants = cva(
24 |   "inline-flex items-center justify-center rounded-xl ring-4 shrink-0 [&svg]:shrink-0",
25 |   {
26 |     variants: {
27 |       severity: {
28 |         high: "bg-red-50 text-red-600 ring-red-500/10 dark:bg-red-950/40 dark:text-red-400 dark:ring-red-400/10",
29 |         med: "bg-amber-50 text-amber-600 ring-amber-500/10 dark:bg-amber-950/40 dark:text-amber-400 dark:ring-amber-400/10",
30 |         low: "bg-green-50 text-green-600 ring-green-500/10 dark:bg-green-950/40 dark:text-green-400 dark:ring-green-400/10",
31 |         info: "bg-blue-50 text-blue-600 ring-blue-500/10 dark:bg-blue-950/40 dark:text-blue-400 dark:ring-blue-400/10",
32 |         brand: "bg-teal-50 text-teal-600 ring-teal-500/10 dark:bg-teal-950/40 dark:text-teal-300 dark:ring-teal-400",
33 |         neutral: "bg-slate-100 text-slate-600 ring-slate-400/10 dark:bg-slate-800 dark:text-slate-300 dark:ring-slate-400",
34 |       },
35 |     },
36 |     size: {
37 |       sm: "size-8 [&svg]:size-4",
38 |       md: "size-10 [&svg]:size-5",
39 |       lg: "size-12 [&svg]:size-6",
40 |     },
41 |   },
42 |   {
43 |     defaultVariants: {
44 |       severity: "brand",
45 |       size: "md",
46 |     },
47 |   },
48 | );
49 |
50 | export interface SeverityIconProps
51 |   extends React.ComponentProps<"span">,
52 |   VariantProps<typeof severityIconVariants> {
53 |   /** The Lucide icon to render inside the tile. */
54 |   icon: LucideIcon;
55 |   /**
56 |    * Accessible label. When provided, the tile is announced (role="img").
57 |    * When omitted, the tile is treated as decorative (aria-hidden) – in that
58 |    * case the surrounding UI MUST carry the status in text.
59 |    */
60 |   label?: string;
61 | }
62 |
63 | const SeverityIcon = ({
64 |   icon: Icon,
65 |   severity,
66 |   size,
67 |   label,
68 |   className,
69 |   ...props
70 | }: SeverityIconProps) => (
71 |   <span
72 |     {...props}
73 |     className={cn(severityIconVariants({ severity, size }), className)}
74 |     role={label ? "img" : undefined}
75 |     aria-label={label}
76 |     aria-hidden={label ? undefined : true}
77 |   >
78 |     <Icon />
79 |   </span>
80 | );
81 | SeverityIcon.displayName = "SeverityIcon";
82 |
83 | // `severityIconVariants` is intentionally NOT exported: `src/components/shared`
84 | // is linted with `react-refresh/only-export-components`, which forbids mixing a
85 | // component export with a constant export. If the variants are needed elsewhere
86 | // later, move them to a sibling `*.variants.ts` file and import from there.
87 | export { SeverityIcon };
88 |

```

FILE: src/components/shared/Shimmer.tsx

TYPE: .tsx | LINES: 1-12 | SHA256: fac1239c8ac1ebfbefc884e859e807d471652e9303750d073d9c71a4b1a2f81f

```

1 | import { cn } from "@lib/utils";
2 |
3 | interface ShimmerProps {
4 |   className?: string;
5 | }
6 |
7 | const Shimmer = ({ className }: ShimmerProps) => (
8 |   <div className={cn("animate-shimmer rounded-xl", className)} />
9 | );
10 |
11 | export default Shimmer;
12 |

```

FILE: src/components/shared/Sidebar.tsx

TYPE: .tsx | LINES: 1-279 | SHA256: 1373c5e4ec886f8b6b971c07990a62e6804c679440b5b00bb02df6b3e1eab989

```

1 | import { Link, NavLink, useLocation } from "react-router-dom";
2 | import { useTranslation } from "react-i18next";
3 | import { X } from "lucide-react";
4 | import { useAuth } from "@hooks/useAuth";
5 | import { useSurveyAssignmentsCount } from "@hooks/useSurveyAssignmentsCount";
6 | import { navItems } from "@lib/navItems";
7 | import {
8 |   getMoreNavItems,
9 |   getPrimaryNavItems,
10 |   type PresentedNavItem,
11 | } from "@lib/navPresentation";
12 | import { useIntentPrefetch } from "@hooks/useIntentPrefetch";
13 | import { prefetchRoute } from "@lib/routePrefetch";
14 | import { useSidebar } from "../SidebarContext";
15 | import MobileTabBar from "../MobileTabBar";
16 | import StudentSidebarExtras from "../StudentSidebarExtras";

```

```

17 | import { Button } from "@components/ui/button";
18 | import { cn } from "@lib/utils";
19 | import type { UserRole } from "@types/app";
20 |
21 | /** Route of the conditional Surveys item, hidden when the student has none. */
22 | const SURVEYS_ROUTE = "/student/surveys";
23 |
24 | const activeNavStyleByRole: Record<
25 |   UserRole,
26 |   { background: string; color: string }
27 | > = {
28 |   student: { background: "#e6f7ff", color: "#075985" },
29 |   teacher: { background: "#eff6ff", color: "#2563eb" },
30 |   coordinator: { background: "#eff6ff", color: "#2563eb" },
31 |   admin: { background: "#eff6ff", color: "#2563eb" },
32 |   parent: { background: "#eff6ff", color: "#2563eb" },
33 | };
34 |
35 | const Sidebar = () => {
36 |   const { t } = useTranslation("common");
37 |   const { profile } = useAuth();
38 |   const { mobileOpen, close } = useSidebar();
39 |   const location = useLocation();
40 |   // Prefetch-on-intent (Req 9): warm a route's chunk on hover/focus (desktop
41 |   // only, once per target, failures swallowed) so navigation feels instant.
42 |   const getIntentHandlers = useIntentPrefetch();
43 |
44 |   const role = (profile?.role ?? "student") as UserRole;
45 |
46 |   // R23.1/2/2a: the Surveys item is shown only when the student has at least one
47 |   // assigned (active) survey. The query is scoped to students; for other roles
48 |   // it stays disabled and the item is filtered out below regardless. When the
49 |   // last survey is unassigned, an existing survey mutation invalidates this
50 |   // query's key prefix, so the count refetches and the item hides immediately.
51 |   const { data: surveyCount } = useSurveyAssignmentsCount({
52 |     enabled: role === "student",
53 |   });
54 |   const showSurveys = (surveyCount ?? 0) > 0;
55 |
56 |   // Filter conditionally-hidden items before sectioning so no gap or placeholder
57 |   // is left where a hidden item would have been (R23.4).
58 |   const items = (navItems[role] ?? []).filter(
59 |     (item) => item.to !== SURVEYS_ROUTE || showSurveys
60 |   );
61 |   const visiblePaths = new Set(items.map((item) => item.to));
62 |   const primaryItems = getPrimaryNavItems(role).filter((item) =>
63 |     visiblePaths.has(item.to)
64 |   );
65 |   const moreItems = getMoreNavItems(role).filter((item) =>
66 |     visiblePaths.has(item.to)
67 |   );
68 |
69 |   const isItemActive = (to: string): boolean =>
70 |     location.pathname === to ||
71 |     (to !== `/${role}/dashboard` &&
72 |       to !== `/${role}` &&
73 |       location.pathname.startsWith(to));
74 |
75 |   // The prototype companion item route per role: the one primary item that keeps
76 |   // a colorful circular icon even when inactive (Tutor for student, Studio for
77 |   // teacher, etc.).
78 |   const companionRoute: Record<UserRole, string> = {
79 |     student: "/student/tutor",
80 |     teacher: "/teacher/modules",
81 |     parent: "/parent/support",
82 |     coordinator: "/coordinator/matrix",
83 |     admin: "/admin/governance",
84 |   };
85 |   const isCompanion = (to: string) => companionRoute[role] === to;
86 |
87 |   const renderItem = (item: PresentedNavItem, section: "primary" | "more") => {
88 |     const isActive = section === "primary" && isItemActive(item.to);
89 |     const itemIsCompanion = section === "primary" && isCompanion(item.to);
90 |     const group = section === "primary" ? "primary" : "secondary";
91 |     const variant = itemIsCompanion ? "companion" : "standard";
92 |
93 |     // --- Primary section styling ---
94 |     // Active: full-width rounded light-blue pill, dark blue text
95 |     // Inactive primary: transparent bg, lighter muted label (#94a3b8)
96 |     // Secondary (below MORE): transparent bg, stronger slate-blue label (#64748b)
97 |     const itemClassName = cn(
98 |       "sidebar-item flex items-center transition-colors duration-150",
99 |       section === "primary"
100 |        ? "rounded-[13px] px-[16px] gap-[12px] min-h-[47px]"
101 |        : "rounded-[10px] px-[18px] gap-3 py-[8px]",
102 |       isActive ? "font-semibold" : "hover:bg-slate-50 dark:hover:bg-slate-800"
103 |     );
104 |
105 |     const IconComponent = item.icon;
106 |
107 |     const content = (
108 |       <>
109 |         {isActive ? <span className="sr-only">(current page)</span> : null}
110 |
111 |         {/* Icon rendering: 3 distinct treatments */}
112 |         {itemIsCompanion ? (
113 |           /* Companion: prominent colorful circular icon badge (always visible) */
114 |           <div className="sidebar-icon-wrapper flex h-[34px] w-[34px] shrink-0 items-center justify-center rou
115 |             <span className="sidebar-icon text-[16px] leading-none">
116 |               {item.emoji}
117 |             </span>
118 |           </div>
119 |         ) : section === "more" && item.emoji ? (
120 |           /* MORE section: small colorful emoji */

```

```

121 |         <span
122 |             className="sidebar-icon flex size-5 shrink-0 items-center justify-center bg-transparent text-[16px
123 |             aria-hidden="true"
124 |         >
125 |             {item.emoji}
126 |         </span>
127 |     ) : IconComponent ? (
128 |         /* Primary section: Lucide outline icon (20px standard) */
129 |         <IconComponent
130 |             className={cn(
131 |                 "sidebar-icon h-[20px] w-[20px] shrink-0 transition-colors",
132 |                 isActive ? "stroke-[2.2]" : "stroke-[1.6]"
133 |             )}
134 |             aria-hidden="true"
135 |         />
136 |     ) : item.emoji ? (
137 |         /* Fallback: emoji */
138 |         <span
139 |             className="sidebar-icon flex size-[20px] shrink-0 items-center justify-center bg-transparent text-
140 |             aria-hidden="true"
141 |         >
142 |             {item.emoji}
143 |         </span>
144 |     ) : null}
145 |
146 |     <span className="sidebar-nav-label truncate">
147 |         {(() => {
148 |             const raw = t(item.labelKey);
149 |             if (typeof raw === "string" && raw.startsWith("nav.")) {
150 |                 return raw
151 |                     .replace("nav.", "")
152 |                     .replace(/([A-Z])/g, " $1")
153 |                     .replace(/^[^./, (s) => s.toUpperCase());
154 |             }
155 |             return raw;
156 |         })()}
157 |     </span>
158 | </>
159 | );
160 |
161 | const sharedProps = getIntentHandlers(item.to, () =>
162 |     prefetchRoute(item.to)
163 | );
164 |
165 | return section === "primary" ? (
166 |     <NavLink
167 |         key={`${section}:${item.to}:${item.labelKey}`}
168 |         to={item.to}
169 |         onClick={close}
170 |         viewTransition
171 |         data-group={group}
172 |         data-variant={variant}
173 |         data-active={isActive ? "true" : "false"}
174 |         {...sharedProps}
175 |         style={isActive ? activeNavStyleByRole[role] : undefined}
176 |         className={itemClassName}
177 |     >
178 |         {content}
179 |     </NavLink>
180 | ) : (
181 |     <Link
182 |         key={`${section}:${item.to}:${item.labelKey}`}
183 |         to={item.to}
184 |         onClick={close}
185 |         viewTransition
186 |         data-group={group}
187 |         data-variant={variant}
188 |         data-active="false"
189 |         {...sharedProps}
190 |         className={itemClassName}
191 |     >
192 |         {content}
193 |     </Link>
194 | );
195 | };
196 |
197 | return (
198 |     <>
199 |         /* Mobile overlay */
200 |         {mobileOpen && (
201 |             <Button
202 |                 type="button"
203 |                 variant="ghost"
204 |                 className="fixed inset-0 z-[98] h-auto w-auto rounded-none bg-black/30 p-0 hover:bg-black/30 min-[64
205 |                 onClick={close}
206 |                 aria-label="Close navigation"
207 |             />
208 |         )}
209 |
210 |         /* Sidebar */
211 |         <aside
212 |             data-tour="primary-nav"
213 |             className={cn(
214 |                 "app-sidebar fixed start-0 top-0 z-[99] flex h-screen w-[var(--app-sidebar-w)] overflow-y-auto overf
215 |                 role === "student"
216 |                 ? "bg-[linear-gradient(180deg,#ffffff_0%,#f5fdff_100%)]"
217 |                 : "bg-white",
218 |                 "min-[640px]:z-[110] min-[640px]:col-start-1 min-[640px]:row-start-1 min-[640px]:translate-x-0",
219 |                 mobileOpen
220 |                 ? "translate-x-0"
221 |                 : "max-[639px]:-translate-x-full max-[639px]:rtl:translate-x-full"
222 |             )}
223 |         >
224 |         <div className="relative flex h-full min-h-0 flex-1 flex-col px-[14px] pb-[20px] pt-4 min-[640px]:pt-[

```

```

225 |         <span className="hidden min-[640px]:block absolute start-[22px] top-[22px] text-[18px] font-black tr
226 |             Edeviser
227 |         </span>
228 |
229 |         { /* Mobile close button */ }
230 |         <div className="flex items-center justify-end p-2 min-[640px]:hidden">
231 |             <Button
232 |                 type="button"
233 |                 variant="ghost"
234 |                 size="icon"
235 |                 onClick={close}
236 |                 className="rounded-lg text-gray-500 hover:bg-slate-100"
237 |                 aria-label="Close navigation"
238 |             >
239 |                 <X className="h-5 w-5" />
240 |             </Button>
241 |         </div>
242 |
243 |         <div className="flex min-h-0 flex-1 flex-col">
244 |             { /* Nav items */ }
245 |             <nav role="navigation" aria-label={t("header.primaryNav.label")}>
246 |                 <div className="space-y-1">
247 |                     {primaryItems.map((item) => renderItem(item, "primary"))}
248 |                 </div>
249 |             </nav>
250 |
251 |             {moreItems.length > 0 ? (
252 |                 <div className="sidebar-extra mt-[10px] flex min-h-0 flex-1 flex-col">
253 |                     <div
254 |                         className="side-sep mx-3 mb-2 mt-3 h-px bg-[#eef2f6]"
255 |                         aria-hidden="true"
256 |                     />
257 |                     <p className="side-label px-[18px] pb-[6px] pt-[4px] text-[10px] font-[800] uppercase tracking
258 |                         {t("nav.more")}>
259 |                         </p>
260 |                     <div className="space-y-[2px]">
261 |                         {moreItems.map((item) => renderItem(item, "more"))}
262 |                     </div>
263 |                     {role === "student" ? <StudentSidebarExtras /> : null}
264 |                 </div>
265 |             ) : null}
266 |         </div>
267 |     </div>
268 | </aside>
269 |
270 |     { /* Responsive nav (design S18 / R18.5): a thumb-reachable bottom tab bar
271 |         on < lg, driven by the same navItems[role]. The bar itself is
272 |         lg:hidden so it never shows alongside the desktop sidebar. */ }
273 |     <MobileTabBar />
274 | </>
275 | );
276 | };
277 |
278 | export default Sidebar;
279 |

```

FILE: src/components/shared/SidebarContext.tsx

TYPE: .tsx | LINES: 1-35 | SHA256: acf5bb6916c5d5515d660fab43ab9070694b53b57cc265f7cceb38d3840a3c5f

```

1 | import {
2 |     createContext,
3 |     useContext,
4 |     useState,
5 |     useCallback,
6 |     type ReactNode,
7 | } from "react";
8 |
9 | interface SidebarState {
10 |     mobileOpen: boolean;
11 |     toggle: () => void;
12 |     close: () => void;
13 | }
14 |
15 | const SidebarContext = createContext<SidebarState>({
16 |     mobileOpen: false,
17 |     toggle: () => {},
18 |     close: () => {},
19 | });
20 |
21 | // eslint-disable-next-line react-refresh/only-export-components
22 | export const useSidebar = () => useContext(SidebarContext);
23 |
24 | export const SidebarProvider = ({ children }: { children: ReactNode }) => {
25 |     const [mobileOpen, setMobileOpen] = useState(false);
26 |     const toggle = useCallback(() => setMobileOpen(v => !v), []);
27 |     const close = useCallback(() => setMobileOpen(false), []);
28 |
29 |     return (
30 |         <SidebarContext.Provider value={{ mobileOpen, toggle, close }}>
31 |             {children}
32 |         </SidebarContext.Provider>
33 |     );
34 | };
35 |

```

FILE: src/components/shared/SimpleReflectionTemplate.tsx

TYPE: .tsx | LINES: 1-81 | SHA256: eb3ade54cd4cbc66bb2108f30f27e982e654c32acc8145f83b38c5d45252be55

```

1 | // =====
2 | // SimpleReflectionTemplate - 3 guided text areas for structured reflection
3 | // =====
4 |
5 | import { Textarea } from "@components/ui/textarea";
6 | import { cn } from "@lib/utills";

```


[illegible]

FILE: src/components/shared/SkipToMain.tsx

TYPE: .tsx | LINES: 1-21 | SHA256: d1ab8f1e3af40c83cc8b75f15ea88e7958e4a4d0ec513f6a22aa7c3a4bcfd66d

```

1 import { useTranslation } from "react-i18next";
2
3 /**
4  * Skip-to-main-content link for keyboard navigation.
5  * Visually hidden until focused, then appears at the top of the viewport.
6  */
7 const SkipToMain = () => {
8   const { t } = useTranslation("common");
9
10  return (
11    <a
12      href="#main-content"
13      className="sr-only focus:not-sr-only focus:fixed focus:top-2 focus:start-2 focus:z-[9999] focus:rounded-
14    >
15      {t("skipToMain", "Skip to main content")}
16    </a>
17  );
18 };
19
20 export default SkipToMain;
21

```

FILE: src/components/shared/SmartGoalForm.tsx

TYPE: .tsx | LINES: 1-192 | SHA256: 11af2e2114028143996c9666ff9d655a964470d200fe3cad5ea48af19f05fda2

```
1 | import { useForm } from "react-hook-form";
2 | import { zodResolver } from "@hookform/resolvers/zod";
3 | import {
4 |   smartGoalTemplateSchema,
5 |   type SmartGoalTemplate,
6 | } from "@lib/onboardingSchemas";
7 | import { composeGoalText } from "@lib/goalTemplates";
8 | import {
9 |   Form,
10 |   FormField,
11 |   FormItem,
12 |   FormLabel,
13 |   FormControl,
14 |   FormMessage,
15 | } from "@components/ui/form";
16 | import { Input } from "@components/ui/input";
17 | import {
18 |   Select,
19 |   SelectContent,
20 |   SelectItem,
21 |   SelectTrigger,
22 |   SelectValue,
23 | } from "@components/ui/select";
24 | import { Button } from "@components/ui/button";
25 | import { Card } from "@components/ui/card";
26 | import { Loader2 } from "lucide-react";
27 |
28 | export interface CourseOption {
29 |   id: string;
30 |   name: string;
31 | }
32 |
33 | export interface SmartGoalFormProps {
34 |   courses: CourseOption[];
35 |   /** Pre-fill the Relevant field with a course name */
36 |   defaultRelevant?: string;
37 |   /** Pre-fill the Time-bound field with an ISO date */
38 |   defaultTimebound?: string;
39 |   onSubmit: (data: SmartGoalTemplate & { composedText: string }) => void;
40 |   isPending?: boolean;
41 | }
42 |
43 | const SmartGoalForm = ({
44 |   courses,
45 |   defaultRelevant = "",
46 |   defaultTimebound = "",
47 |   onSubmit,
48 |   isPending = false,
49 | }: SmartGoalFormProps) => {
50 |   const form = useForm<SmartGoalTemplate>({
51 |     resolver: zodResolver(smartGoalTemplateSchema),
52 |     defaultValues: {
53 |       specific: "",
54 |       measurable: "",
55 |       achievable: "",
56 |       relevant: defaultRelevant,
57 |       timebound: defaultTimebound,
58 |     },
59 |   });
60 |
61 |   const handleSubmit = (data: SmartGoalTemplate) => {
62 |     const composedText = composeGoalText(data);
63 |     onSubmit({ ...data, composedText });
64 |   };
65 |
66 |   return (
67 |     <Card className="bg-white border-0 shadow-md rounded-xl p-6">
68 |       <h3 className="text-sm font-bold text-gray-900 mb-4">
69 |         SMART Goal Template
70 |       </h3>
71 |       <Form {...form}>
72 |         <form onSubmit={form.handleSubmit(handleSubmit)} className="space-y-4">
73 |           <FormField
74 |             control={form.control}
75 |             name="specific"
76 |             render={({ field }) => (
77 |               <FormItem>
78 |                 <FormLabel className="text-xs font-bold text-gray-700">
79 |                   Specific – What will you accomplish?
80 |                 </FormLabel>
81 |                 <FormControl>
82 |                   <Input
83 |                     placeholder="e.g., Complete all practice problems for Chapter 3"
84 |                     {...field}
85 |                   />
86 |                 </FormControl>
87 |                 <FormMessage />
88 |               </FormItem>
89 |             )}
90 |           />
91 |           <FormField
92 |             control={form.control}
93 |             name="measurable"
94 |             render={({ field }) => (
95 |               <FormItem>
96 |                 <FormLabel className="text-xs font-bold text-gray-700">
97 |                   Measurable – How will you track progress?
98 |                 </FormLabel>
99 |                 <FormControl>
100 |                   <Input
```

```

102 |         placeholder="e.g., Score at least 80% on the practice quiz"
103 |         {...field}
104 |     />
105 |     </FormControl>
106 |     <FormMessage />
107 | </FormItem>
108 | })
109 | />
110 |
111 | <FormField
112 |   control={form.control}
113 |   name="achievable"
114 |   render={({ field }) => (
115 |     <FormItem>
116 |       <FormLabel className="text-xs font-bold text-gray-700">
117 |         Achievable – Why is this realistic?
118 |       </FormLabel>
119 |       <FormControl>
120 |         <Input
121 |           placeholder="e.g., Based on my current progress and available study time"
122 |           {...field}
123 |         />
124 |       </FormControl>
125 |       <FormMessage />
126 |     </FormItem>
127 |   )}
128 | />
129 |
130 | <FormField
131 |   control={form.control}
132 |   name="relevant"
133 |   render={({ field }) => (
134 |     <FormItem>
135 |       <FormLabel className="text-xs font-bold text-gray-700">
136 |         Relevant – Which course or outcome?
137 |       </FormLabel>
138 |       <Select
139 |         onChange={field.onChange}
140 |         defaultValue={field.value}
141 |       >
142 |         <FormControl>
143 |           <SelectTrigger className="bg-white">
144 |             <SelectValue placeholder="Select a course" />
145 |           </SelectTrigger>
146 |         </FormControl>
147 |         <SelectContent>
148 |           {courses.map((course) => (
149 |             <SelectItem key={course.id} value={course.name}>
150 |               {course.name}
151 |             </SelectItem>
152 |           ))}
153 |         </SelectContent>
154 |       </Select>
155 |       <FormMessage />
156 |     </FormItem>
157 |   )}
158 | />
159 |
160 | <FormField
161 |   control={form.control}
162 |   name="timebound"
163 |   render={({ field }) => (
164 |     <FormItem>
165 |       <FormLabel className="text-xs font-bold text-gray-700">
166 |         Time-bound – By when?
167 |       </FormLabel>
168 |       <FormControl>
169 |         <Input type="date" {...field} />
170 |       </FormControl>
171 |       <FormMessage />
172 |     </FormItem>
173 |   )}
174 | />
175 |
176 | <Button
177 |   type="submit"
178 |   disabled={isPending}
179 |   variant="tactile"
180 |   className="w-full font-semibold"
181 | >
182 |   {isPending && <Loader2 className="h-4 w-4 animate-spin" />}
183 |   Create Goal
184 | </Button>
185 | </form>
186 | </Form>
187 | </Card>
188 | );
189 | };
190 |
191 | export default SmartGoalForm;
192 |

```

FILE: `src/components/shared/SourceCitationPanel.tsx`

TYPE: `.tsx` | LINES: 1-143 | SHA256: `b2d7bc33b43ce8579dbf98ba367feaf6bc1862dbe6061e83511a77fb5e930f56`

```

1 | // =====
2 | // SourceCitationPanel – Expandable citation detail panel for tutor messages
3 | // =====
4 |
5 | import { useState } from "react";
6 | import { ChevronDown, ChevronUp, FileText, BookOpen } from "lucide-react";
7 | import { Button } from "@components/ui/button";
8 | import { Badge } from "@components/ui/badge";
9 | import { cn } from "@lib/utils";

```

```

10 | import type { SourceCitation } from "@lib/tutorSchemas";
11 |
12 | interface SourceCitationPanelProps {
13 |   citations: SourceCitation[];
14 |   /** Index of the citation to auto-expand (from clicking inline marker) */
15 |   expandedIndex?: number | null;
16 |   onExpandedChange?: (index: number | null) => void;
17 | }
18 |
19 | const materialTypeLabels: Record<string, string> = {
20 |   lecture_notes: "Lecture Notes",
21 |   slides: "Slides",
22 |   assignment_description: "Assignment",
23 |   rubric_criteria: "Rubric",
24 |   other: "Material",
25 | };
26 |
27 | const materialTypeIcons: Record<string, typeof FileText> = {
28 |   lecture_notes: BookOpen,
29 |   slides: FileText,
30 |   assignment_description: FileText,
31 |   rubric_criteria: FileText,
32 |   other: FileText,
33 | };
34 |
35 | const SourceCitationPanel = ({
36 |   citations,
37 |   expandedIndex: controlledExpanded,
38 |   onExpandedChange,
39 | }: SourceCitationPanelProps) => {
40 |   const [internalExpanded, setInternalExpanded] = useState<number | null>(null);
41 |   const [isOpen, setIsOpen] = useState(false);
42 |
43 |   const expandedIndex = controlledExpanded ?? internalExpanded;
44 |   const setExpandedIndex = onExpandedChange ?? setInternalExpanded;
45 |
46 |   if (citations.length === 0) return null;
47 |
48 |   const toggleCitation = (index: number) => {
49 |     setExpandedIndex(expandedIndex === index ? null : index);
50 |   };
51 |
52 |   return (
53 |     <div className="mt-2">
54 |       <Button
55 |         variant="ghost"
56 |         size="sm"
57 |         onClick={() => setIsOpen(!isOpen)}
58 |         className="text-xs text-gray-500 hover:text-gray-700 gap-1 px-2 h-7"
59 |         aria-expanded={isOpen}
60 |         aria-label={` ${citations.length} source${
61 |           citations.length !== 1 ? "s" : ""
62 |         } referenced`}
63 |       >
64 |         <FileText className="h-3.5 w-3.5" />
65 |         {citations.length} source{citations.length !== 1 ? "s" : ""}
66 |         {isOpen ? (
67 |           <ChevronUp className="h-3 w-3" />
68 |         ) : (
69 |           <ChevronDown className="h-3 w-3" />
70 |         )}
71 |       </Button>
72 |
73 |       {isOpen && (
74 |         <div
75 |           className="mt-1.5 space-y-1.5"
76 |           role="list"
77 |           aria-label="Source citations"
78 |         >
79 |           {citations.map((citation, index) => {
80 |             const Icon = materialTypeIcons[citation.material_type] ?? FileText;
81 |             const isExpanded = expandedIndex === index;
82 |
83 |             return (
84 |               <div
85 |                 key={citation.chunk_id}
86 |                 role="listitem"
87 |                 className={cn(
88 |                   "rounded-lg border transition-colors",
89 |                   isExpanded
90 |                     ? "border-blue-200 bg-blue-50/50"
91 |                     : "border-gray-100 bg-gray-50/50"
92 |                 )}
93 |               >
94 |                 <button
95 |                   onClick={() => toggleCitation(index)}
96 |                   className="flex items-center gap-2 w-full px-3 py-2 text-start"
97 |                   aria-expanded={isExpanded}
98 |                 >
99 |                   <Badge
100 |                     variant="secondary"
101 |                     className="text-[10px] font-bold shrink-0 h-5 w-5 p-0 justify-center"
102 |                   >
103 |                     {index + 1}
104 |                   </Badge>
105 |                   <Icon className="h-3.5 w-3.5 text-gray-400 shrink-0" />
106 |                   <span className="text-xs font-medium text-gray-700 truncate flex-1">
107 |                     {citation.source_filename}
108 |                   </span>
109 |                   <Badge variant="outline" className="text-[10px] shrink-0">
110 |                     {materialTypeLabels[citation.material_type] ?? "Material"}
111 |                   </Badge>
112 |                   {isExpanded ? (
113 |                     <ChevronUp className="h-3 w-3 text-gray-400 shrink-0" />

```

```

114 |         ) : (
115 |             <ChevronDown className="h-3 w-3 text-gray-400 shrink-0" />
116 |         )}
117 |     </button>
118 |
119 |     {isExpanded && (
120 |         <div className="px-3 pb-3 pt-1 border-t border-gray-100">
121 |             <p className="text-xs text-gray-600 leading-relaxed whitespace-pre-wrap">
122 |                 {citation.chunk_text}
123 |             </p>
124 |             <div className="mt-2 flex items-center gap-2">
125 |                 <span className="text-[10px] text-gray-400">
126 |                     Relevance: {Math.round(citation.similarity_score * 100)}
127 |                     %
128 |                 </span>
129 |             </div>
130 |         </div>
131 |     )}
132 | </div>
133 | );
134 | }}
135 | </div>
136 | }}
137 | </div>
138 | );
139 | };
140 |
141 | export default SourceCitationPanel;
142 | export type { SourceCitationPanelProps };
143 |

```

FILE: [src/components/shared/StarterWeekHeroCard.tsx](#)

TYPE: .tsx | LINES: 1-129 | SHA256: d9323d2c11276e5b1f6ebac79d82f1e238fe62158a507b0979aea3054d24b8da

```

1 | import { Card } from "@components/ui/card";
2 | import { Button } from "@components/ui/button";
3 | import { CalendarDays, Clock, CheckCircle, Trophy } from "lucide-react";
4 | import type { StarterWeekSession } from "@hooks/useStarterWeekPlan";
5 |
6 | export interface StarterWeekHeroCardProps {
7 |   sessions: StarterWeekSession[];
8 |   onViewPlan: () => void;
9 |   /** Whether the starter week has ended (7+ days since first session) */
10 |  isPostWeek?: boolean;
11 | }
12 |
13 | const StarterWeekHeroCard = ({
14 |   sessions,
15 |   onViewPlan,
16 |   isPostWeek = false,
17 | }: StarterWeekHeroCardProps) => {
18 |   const totalSessions = sessions.length;
19 |   const completedSessions = sessions.filter(
20 |     (s) => s.status === "completed"
21 |   ).length;
22 |   const totalMinutes = sessions.reduce((sum, s) => sum + s.duration_minutes, 0);
23 |   const completedMinutes = sessions
24 |     .filter((s) => s.status === "completed")
25 |     .reduce((sum, s) => sum + s.duration_minutes, 0);
26 |
27 |   if (isPostWeek) {
28 |     const completionRate =
29 |       totalSessions > 0
30 |       ? Math.round((completedSessions / totalSessions) * 100)
31 |       : 0;
32 |
33 |     return (
34 |       <Card
35 |         className="border-0 shadow-lg rounded-xl overflow-hidden text-white"
36 |         style={{
37 |           background:
38 |             "linear-gradient(135deg, #0f172a 0%, #1e3a8a 50%, #312e81 100%)",
39 |         }}
40 |       >
41 |         <div className="p-6 space-y-3">
42 |           <div className="flex items-center gap-2">
43 |             <Trophy className="h-5 w-5 text-amber-400" />
44 |             <h2 className="text-lg font-bold tracking-tight">
45 |               Starter Week Complete!
46 |             </h2>
47 |           </div>
48 |           <p className="text-sm text-white/70">
49 |             You completed {completedSessions} of {totalSessions} sessions (
50 |               {completionRate}%
51 |             ) for a total of {completedMinutes} minutes of
52 |             study time.
53 |           </p>
54 |           {completionRate >= 80 && (
55 |             <p className="text-xs text-amber-300 font-medium">
56 |               Great start – keep the momentum going!
57 |             </p>
58 |           )}
59 |         </div>
60 |       </Card>
61 |     );
62 |   }
63 |
64 |   const hours = Math.floor(totalMinutes / 60);
65 |   const mins = totalMinutes % 60;
66 |   const timeDisplay = hours > 0 ? `${hours}h ${mins}m` : `${mins}m`;
67 |
68 |   return (
69 |     <Card
70 |       className="border-0 shadow-lg rounded-xl overflow-hidden text-white"
71 |       style={{

```

```

71 |         background:
72 |             "linear-gradient(135deg, #0f172a 0%, #1e3a8a 50%, #312e81 100%)",
73 |     }}
74 | >
75 | <div className="p-6 space-y-4">
76 |   <div>
77 |     <h2 className="text-lg font-bold tracking-tight">
78 |       Your Starter Week Plan
79 |     </h2>
80 |     <p className="text-sm text-white/70 mt-1">
81 |       AI-generated study sessions to kick off your semester
82 |     </p>
83 |   </div>
84 |
85 |   <div className="grid grid-cols-3 gap-3">
86 |     <div className="space-y-0.5">
87 |       <div className="flex items-center gap-1 text-white/60">
88 |         <CalendarDays className="h-3 w-3" />
89 |         <span className="text-[10px] font-black tracking-widest uppercase">
90 |           Sessions
91 |         </span>
92 |       </div>
93 |       <p className="text-xl font-black">{totalSessions}</p>
94 |     </div>
95 |     <div className="space-y-0.5">
96 |       <div className="flex items-center gap-1 text-white/60">
97 |         <Clock className="h-3 w-3" />
98 |         <span className="text-[10px] font-black tracking-widest uppercase">
99 |           Total Time
100 |         </span>
101 |       </div>
102 |       <p className="text-xl font-black">{timeDisplay}</p>
103 |     </div>
104 |     <div className="space-y-0.5">
105 |       <div className="flex items-center gap-1 text-white/60">
106 |         <CheckCircle className="h-3 w-3" />
107 |         <span className="text-[10px] font-black tracking-widest uppercase">
108 |           Done
109 |         </span>
110 |       </div>
111 |       <p className="text-xl font-black">
112 |         {completedSessions}/{totalSessions}
113 |       </p>
114 |     </div>
115 |   </div>
116 |
117 |   <Button
118 |     onClick={onViewPlan}
119 |     className="bg-white/10 hover:bg-white/20 text-white border border-white/20 text-sm font-semibold act
120 |   >
121 |     View Plan
122 |   </Button>
123 | </div>
124 | </Card>
125 | );
126 | };
127 |
128 | export default StarterWeekHeroCard;
129 |

```

FILE: src/components/shared/StreakDisplay.tsx

TYPE: .tsx | LINES: 1-168 | SHA256: dd6809a1d3a7c019e45612bc71252b4f1fd5f6c09c61f5b357698f8da815cd1c

```

1 | import { Flame, Snowflake, Trophy, RefreshCw } from "lucide-react";
2 | import { getNextMilestone, getMilestoneProgress } from "@lib/streakMilestones";
3 | import { PCard } from "@design-system";
4 |
5 | const TOTAL_ACTIVE_MILESTONES = [30, 60, 100, 200, 365] as const;
6 |
7 | export interface StreakDisplayProps {
8 |   streakCount: number;
9 |   lastLoginDate?: string;
10 |   compact?: boolean;
11 |   streakFreezesAvailable?: number;
12 |   /** Weekend rest days within current streak period (sabbatical) */
13 |   restDays?: number;
14 |   /** Total active days since account creation */
15 |   totalActiveDays?: number;
16 |   /** Whether Streak Sabbatical is enabled for the institution */
17 |   streakSabbaticalEnabled?: boolean;
18 |   /** Whether the streak was just reset (show motivational message) */
19 |   streakJustReset?: boolean;
20 | }
21 |
22 | function isTotalActiveMilestone(days: number): boolean {
23 |   return (TOTAL_ACTIVE_MILESTONES as readonly number[]).includes(days);
24 | }
25 |
26 | const StreakDisplay = ({
27 |   streakCount,
28 |   compact = false,
29 |   streakFreezesAvailable = 0,
30 |   restDays = 0,
31 |   totalActiveDays = 0,
32 |   streakSabbaticalEnabled = false,
33 |   streakJustReset = false,
34 | }: StreakDisplayProps) => {
35 |   if (compact) {
36 |     return (
37 |       <div
38 |         className="flex items-center gap-1.5"
39 |         aria-label={`{streakCount} day streak`}
40 |       >
41 |         <Flame className="h-4 w-4 text-orange-500 animate-streak-flame" />

```

```

42 |         <span className="text-sm font-bold text-orange-600">{streakCount}</span>
43 |     </div>
44 | );
45 | }
46 |
47 | const nextMilestone = getNextMilestone(streakCount);
48 | const progress = getMilestoneProgress(streakCount);
49 | const showRangeFormat = streakSabbaticalEnabled && restDays > 0;
50 | const showMilestoneCelebration =
51 |     totalActiveDays > 0 && isTotalActiveMilestone(totalActiveDays);
52 |
53 | return (
54 |     <PCard className="p-4" data-focus-hide="true">
55 |         <div className="flex items-center gap-3">
56 |             <div className="p-2 rounded-lg bg-orange-50">
57 |                 <Flame className="h-5 w-5 text-orange-500 animate-streak-flame" />
58 |             </div>
59 |             <div className="flex-1 min-w-0">
60 |                 <div className="flex items-baseline gap-1.5">
61 |                     <span className="text-2xl font-black bg-gradient-to-r from-red-500 to-orange-500 bg-clip-text text
62 |                         {streakCount}
63 |                     </span>
64 |                     {showRangeFormat ? (
65 |                         <span
66 |                             className="text-sm font-medium text-gray-500"
67 |                             data-testid="streak-range-label"
68 |                         >
69 |                             day streak, {restDays} rest day{restDays !== 1 ? "s" : ""}
70 |                         </span>
71 |                     ) : (
72 |                         <span className="text-sm font-medium text-gray-500">
73 |                             day streak
74 |                         </span>
75 |                     )}
76 |                 </div>
77 |             </div>
78 |
79 |             {streakFreezesAvailable > 0 && (
80 |                 <div
81 |                     className="flex items-center gap-0.5"
82 |                     aria-label={` ${streakFreezesAvailable} streak freeze${
83 |                         streakFreezesAvailable > 1 ? "s" : ""
84 |                     } available`}
85 |                 >
86 |                     {Array.from({ length: Math.min(streakFreezesAvailable, 2) }).map(
87 |                         (_, i) => (
88 |                             <Snowflake key={i} className="h-4 w-4 text-blue-400" />
89 |                         )
90 |                     )}
91 |                 </div>
92 |             )}
93 |         </div>
94 |
95 |         {/* Total Active Days counter */}
96 |         {totalActiveDays > 0 && (
97 |             <div
98 |                 className="mt-3 flex items-center gap-2"
99 |                 data-testid="total-active-days"
100 |             >
101 |                 {showMilestoneCelebration ? (
102 |                     <Trophy className="h-4 w-4 text-amber-500 animate-badge-pop" />
103 |                 ) : (
104 |                     <Trophy className="h-4 w-4 text-gray-400" />
105 |                 )}
106 |                 <span className="text-xs font-medium text-gray-600">
107 |                     {totalActiveDays} Total Active Day{totalActiveDays !== 1 ? "s" : ""}
108 |                 </span>
109 |                 {showMilestoneCelebration && (
110 |                     <span
111 |                         className="text-xs font-bold text-amber-600 animate-xp-pulse"
112 |                         data-testid="active-days-milestone"
113 |                     >
114 |                         Milestone!
115 |                     </span>
116 |                 )}
117 |             </div>
118 |         )}
119 |
120 |         {/* Motivational message on streak reset */}
121 |         {streakJustReset && totalActiveDays > 0 && (
122 |             <div
123 |                 className="mt-3 flex items-center gap-2 bg-blue-50 rounded-lg p-2.5"
124 |                 data-testid="streak-reset-message"
125 |             >
126 |                 <RefreshCw className="h-4 w-4 text-blue-500 shrink-0" />
127 |                 <p className="text-xs text-blue-700 font-medium">
128 |                     Your {totalActiveDays} total active days of learning are still an
129 |                     achievement
130 |                 </p>
131 |             </div>
132 |         )}
133 |
134 |         {nextMilestone !== null && (
135 |             <div className="mt-3 space-y-1.5">
136 |                 <div className="flex items-center justify-between">
137 |                     <span className="text-xs text-gray-500">
138 |                         Next milestone: {nextMilestone} days
139 |                     </span>
140 |                     <span className="text-xs font-medium text-gray-600">
141 |                         {progress}%
142 |                     </span>
143 |                 </div>
144 |                 <div className="h-2 rounded-full bg-gray-100 overflow-hidden">
145 |                     <div

```

```

146 |         className="h-full rounded-full bg-gradient-to-r from-red-500 to-orange-500 transition-all durati
147 |         style={{ width: `${progress}%` }}
148 |         role="progressbar"
149 |         aria-valuenow={progress}
150 |         aria-valuemin={0}
151 |         aria-valuemax={100}
152 |         aria-label={` ${progress}% progress toward ${nextMilestone} day milestone`}
153 |     />
154 | </div>
155 | </div>
156 | )}
157 |
158 | {nextMilestone === null && (
159 |   <p className="mt-2 text-xs text-green-600 font-medium">
160 |     All milestones reached!
161 |   </p>
162 | )}
163 | </PCard>
164 | );
165 | };
166 |
167 | export default StreakDisplay;
168 |

```

FILE: src/components/shared/StreakFreezeShop.tsx

TYPE: .tsx | LINES: 1-101 | SHA256: b4af32d0a366ac59e77d0a4a1c30851815f37707b7fb8ddf09bf70e5f762e3b0

```

1 | // =====
2 | // StreakFreezeShop – Streak freeze purchase UI (costs 200 XP)
3 | // =====
4 |
5 | import { Snowflake, Loader2 } from "lucide-react";
6 | import { useState } from "react";
7 | import { Button } from "@components/ui/button";
8 | import { ConfirmDialog } from "@components/shared/ConfirmDialog";
9 | import { cn } from "@lib/utlis";
10 |
11 | interface StreakFreezeShopProps {
12 |   currentXP: number;
13 |   freezesAvailable: number;
14 |   onPurchase: () => Promise<void>;
15 |   className?: string;
16 | }
17 |
18 | const FREEZE_COST = 200;
19 | const MAX_FREEZES = 3;
20 |
21 | const StreakFreezeShop = ({
22 |   currentXP,
23 |   freezesAvailable,
24 |   onPurchase,
25 |   className,
26 | }: StreakFreezeShopProps) => {
27 |   const [isPurchasing, setIsPurchasing] = useState(false);
28 |   const [showConfirm, setShowConfirm] = useState(false);
29 |   const canPurchase =
30 |     currentXP >= FREEZE_COST && freezesAvailable < MAX_FREEZES;
31 |
32 |   const handleConfirmedPurchase = async () => {
33 |     setIsPurchasing(true);
34 |     try {
35 |       await onPurchase();
36 |       setShowConfirm(false);
37 |     } finally {
38 |       setIsPurchasing(false);
39 |     }
40 |   };
41 |
42 |   return (
43 |     <div
44 |       className={cn(
45 |         "rounded-xl border border-slate-200 bg-white p-4 space-y-3",
46 |         className
47 |       )}
48 |     >
49 |       <div className="flex items-center gap-2">
50 |         <Snowflake className="h-5 w-5 text-blue-500" />
51 |         <span className="text-sm font-bold">Streak Freeze</span>
52 |       </div>
53 |       <p className="text-xs text-gray-500">
54 |         Protect your streak for one missed day. Max {MAX_FREEZES} in inventory.
55 |       </p>
56 |       <div className="flex items-center gap-2">
57 |         {Array.from({ length: MAX_FREEZES }).map((_, i) => (
58 |           <Snowflake
59 |             key={i}
60 |             className={cn(
61 |               "h-6 w-6",
62 |               i < freezesAvailable ? "text-blue-500" : "text-gray-200"
63 |             )}
64 |           />
65 |         ))}
66 |         <span className="text-xs text-gray-400 ms-1">
67 |           {freezesAvailable}/{MAX_FREEZES}
68 |         </span>
69 |       </div>
70 |       <Button
71 |         onClick={() => setShowConfirm(true)}
72 |         disabled={!canPurchase || isPurchasing}
73 |         variant="outline"
74 |         className="w-full"
75 |       >
76 |         {isPurchasing && <Loader2 className="h-4 w-4 animate-spin" />}
77 |         Buy for {FREEZE_COST} XP

```



```

78 |         </Button>
79 |         {currentXP < FREEZE_COST && (
80 |           <p className="text-xs text-red-500">
81 |             Not enough XP ({currentXP}/{FREEZE_COST})
82 |           </p>
83 |         )}
84 |       )}
85 |       <ConfirmDialog
86 |         open={showConfirm}
87 |         onOpenChange={setShowConfirm}
88 |         title="Purchase Streak Freeze?"
89 |         description={`This will deduct ${FREEZE_COST} XP from your balance. You currently have ${currentXP} XP
90 |         confirmLabel="Buy Freeze"
91 |         isPending={isPurchasing}
92 |         onConfirm={handleConfirmedPurchase}
93 |       />
94 |     </div>
95 |   );
96 | };
97 |
98 | export default StreakFreezeShop;
99 | export { FREEZE_COST, MAX_FREEZES };
100 | export type { StreakFreezeShopProps };
101 |

```

FILE: src/components/shared/StudentDashboardNew.tsx

TYPE: .tsx | LINES: 1-313 | SHA256: 20c33c3e15ae448bdca9740b9761c2caf328093293349e1b73df9704d2fc60ba

```

1 | // =====
2 | // StudentDashboardNew – redesigned student dashboard (P2, spec task 2.1)
3 | // =====
4 | //
5 | // The "Today / gap !" action" student dashboard, gated behind the
6 | // `newUiDashboards` feature flag (see StudentDashboard.tsx wrapper). It REUSES
7 | // the existing data hook `useStudentDashboardAggregate` (one round-trip, no new
8 | // writes) and is composed entirely from the P0 primitives – WelcomeHero,
9 | // KPICard, SectionHeader, MasteryRing, SeverityIcon, the tactile Button, and
10 | // the `card-elevated` surface.
11 | //
12 | // This is an incremental, flag-off-by-default build: it presents the core
13 | // value (hero, next-step, KPIS, mastery, deadlines, tutor) with the new design.
14 | // Richer gamification widgets remain on the legacy dashboard until this reaches
15 | // full parity (task 2.6). i18n reuses existing `student` namespace keys.
16 | // =====
17 |
18 | import { useNavigate } from "react-router-dom";
19 | import { useTranslation } from "react-i18next";
20 | import {
21 |   ArrowRight,
22 |   BookOpen,
23 |   Bot,
24 |   CalendarClock,
25 |   CheckCircle2,
26 |   FileCheck,
27 |   Flame,
28 |   Target,
29 |   TrendingUp,
30 | } from "lucide-react";
31 |
32 | import {
33 |   Button,
34 |   KPICard,
35 |   MasteryRing,
36 |   PCard,
37 |   SectionHeader,
38 |   SeverityIcon,
39 |   Shimmer,
40 |   WelcomeHero,
41 | } from "@design-system";
42 | import { useAuth } from "@hooks/useAuth";
43 | import { useStudentDashboardAggregate } from "@hooks/useStudentDashboardAggregate";
44 | import { formatNumber, formatPercent } from "@lib/formatNumber";
45 | import { formatLocalDate } from "@lib/formatDate";
46 | import { attainmentValueClass } from "@lib/attainmentTone";
47 |
48 | const StudentDashboardNew = () => {
49 |   const { t } = useTranslation("student");
50 |   const navigate = useNavigate();
51 |   const { user, profile } = useAuth();
52 |   const studentId = user?.id ?? "";
53 |
54 |   const aggregate = useStudentDashboardAggregate(studentId);
55 |   const kpis = aggregate.data?.kpis;
56 |   const deadlines = aggregate.data?.deadlines ?? [];
57 |   const loading = aggregate.isPending;
58 |
59 |   const avgAttainment = kpis?.avgAttainment ?? 0;
60 |   const nextDeadline = deadlines[0];
61 |
62 |   return (
63 |     <div className="space-y-6">
64 |       { /* Welcome hero with XP / Level / Streak */ }
65 |       <WelcomeHero
66 |         name={profile?.full_name ?? "Student"}
67 |         userRole="student"
68 |         subtitle={t("dashboard.momentum")}
69 |         stats={
70 |           <div className="flex items-center gap-4">
71 |             <div className="text-center">
72 |               <p className="text-2xl font-black">{kpis?.totalXP ?? 0}</p>
73 |               <p className="mt-1 text-[10px] font-black uppercase tracking-widest text-white/60">
74 |                 {t("dashboard.totalXP")}
75 |               </p>
76 |             </div>

```

```

77 |         <div className="h-10 w-px bg-white/20" />
78 |         <div className="text-center">
79 |           <p className="text-2xl font-black">
80 |             {t("dashboard.level", { level: kpis?.currentLevel ?? 1 })}
81 |           </p>
82 |           <p className="mt-1 text-[10px] font-black uppercase tracking-widest text-white/60">
83 |             {t("dashboard.levelLabel")}
84 |           </p>
85 |         </div>
86 |         <div className="h-10 w-px bg-white/20" />
87 |         <div className="text-center">
88 |           <p className="text-2xl font-black">
89 |             {kpis?.currentStreak ?? 0} = %
90 |           </p>
91 |           <p className="mt-1 text-[10px] font-black uppercase tracking-widest text-white/60">
92 |             {t("dashboard.streak")}
93 |           </p>
94 |         </div>
95 |       </div>
96 |     }
97 |   />
98 |
99 |   { /* Your next step (gap !' action) */ }
100 |   <PCard>
101 |     <div className="p-6">
102 |       <SectionHeader
103 |         icon={Target}
104 |         title={t("dashboard.primaryCta.heading")}
105 |       />
106 |       <div className="mt-4 flex flex-col gap-3 sm:flex-row sm:items-center sm:justify-between">
107 |         <div className="flex items-center gap-3">
108 |           <SeverityIcon
109 |             icon={nextDeadline ? FileCheck : BookOpen}
110 |             severity={nextDeadline ? "med" : "brand"}
111 |             label={
112 |               nextDeadline
113 |                 ? t("dashboard.primaryCta.submitAssignment.label")
114 |                 : t("dashboard.primaryCta.continueCourse.label")
115 |             }
116 |           />
117 |           <div className="min-w-0">
118 |             <p className="truncate text-sm font-semibold text-gray-900">
119 |               {nextDeadline
120 |                 ? nextDeadline.title
121 |                 : t("dashboard.primaryCta.continueCourse.label")}
122 |             </p>
123 |             <p className="truncate text-xs text-gray-500">
124 |               {nextDeadline
125 |                 ? `${nextDeadline.course_name} · ${formatLocalDate(
126 |                     nextDeadline.due_date,
127 |                     "MMM d"
128 |                   )}`
129 |                 : t("dashboard.primaryCta.continueCourse.description")}
130 |             </p>
131 |           </div>
132 |         </div>
133 |         <Button
134 |           variant="tactile"
135 |           className="shrink-0"
136 |           onClick={() =>
137 |             navigate(
138 |               nextDeadline
139 |                 ? ` /student/assignments/${nextDeadline.id}`
140 |                 : "/student/courses"
141 |             )
142 |           }
143 |         >
144 |           {nextDeadline
145 |             ? t("dashboard.primaryCta.submitAssignment.cta")
146 |             : t("dashboard.primaryCta.continueCourse.cta")}
147 |           <ArrowRight className="h-4 w-4" aria-hidden="true" />
148 |         </Button>
149 |       </div>
150 |     </div>
151 |   </PCard>
152 |
153 |   { /* KPI row */ }
154 |   { loading ? (
155 |     <div className="grid grid-cols-2 gap-4 md:grid-cols-4">
156 |       {Array.from({ length: 4 }).map((_, i) => (
157 |         <Shimmer key={i} className="h-24 rounded-xl" />
158 |       ))}
159 |     </div>
160 |   ) : (
161 |     <div className="grid grid-cols-2 gap-4 md:grid-cols-4">
162 |       <KPICard
163 |         icon={BookOpen}
164 |         label={t("dashboard.courses")}
165 |         value={formatNumber(kpis?.enrolledCourses ?? 0)}
166 |       />
167 |       <KPICard
168 |         icon={CheckCircle2}
169 |         label={t("dashboard.completed")}
170 |         value={formatNumber(kpis?.completedAssignments ?? 0)}
171 |         iconBgClass="bg-green-50"
172 |         iconColorClass="text-green-600"
173 |       />
174 |       <KPICard
175 |         icon={TrendingUp}
176 |         label={t("dashboard.avgAttainment")}
177 |         value={formatPercent(avgAttainment)}
178 |         valueClassName={attainmentValueClass(avgAttainment)}
179 |       />
180 |       <KPICard

```

```

181         icon={Flame}
182         label={t("dashboard.streak")}
183         value={ ${kpis?.currentStreak ?? 0}d` }
184         iconBgClass="bg-orange-50"
185         iconColorClass="text-orange-500"
186     </div>
187 </div>
188 ))
189
190 { /* Mastery snapshot + Upcoming deadlines */ }
191 <div className="grid grid-cols-1 gap-6 lg:grid-cols-2">
192 { /* Mastery snapshot */ }
193 <PCard>
194     <div className="p-6">
195         <SectionHeader>
196             icon={TrendingUp}
197             title={t("dashboard.avgAttainment")}
198         </SectionHeader>
199         <div className="mt-4 flex items-center gap-5">
200             {loading ? (
201                 <Shimmer className="h-24 w-24 rounded-full" />
202             ) : (
203                 <MasteryRing value={avgAttainment} size={96} tone="auto" />
204             )}
205             <div className="min-w-0">
206                 <p className="text-sm text-gray-600">
207                     {t("dashboard.totalActiveDays", "Active days")}: { " " }
208                     <span className="font-semibold text-gray-900">
209                         {kpis?.totalActiveDays ?? 0}
210                     </span>
211                 </p>
212                 <Button>
213                     variant="link"
214                     className="mt-1 h-auto p-0 text-sky-700"
215                     onClick={() => navigate("/student/progress")}
216                 >
217                     {t("dashboard.primaryCta.reviewFeedback.cta")}
218                     <ArrowRight className="h-3.5 w-3.5" aria-hidden="true" />
219                 </Button>
220             </div>
221         </div>
222     </div>
223 </PCard>
224
225 { /* Upcoming deadlines */ }
226 <PCard>
227     <div className="p-6">
228         <SectionHeader>
229             icon={CalendarClock}
230             title={t("dashboard.upcomingDeadlines")}
231         </SectionHeader>
232         <div className="mt-4">
233             {loading ? (
234                 <div className="space-y-3">
235                     {Array.from({ length: 3 }).map((_, i) => (
236                         <Shimmer key={i} className="h-12 rounded-lg" />
237                     ))}
238                 </div>
239             ) : deadlines.length > 0 ? (
240                 <ul className="space-y-2">
241                     {deadlines.slice(0, 5).map((d) => (
242                         <li key={d.id}>
243                             <button>
244                                 type="button"
245                                 onClick={() => navigate(`/student/assignments/${d.id}`)}
246                                 className="flex w-full items-center gap-3 rounded-lg px-2 py-2 text-start transition-c
247                             >
248                             <CalendarClock>
249                                 className="h-4 w-4 shrink-0 text-gray-400"
250                                 aria-hidden="true"
251                             </CalendarClock>
252                             <span className="min-w-0 flex-1">
253                                 <span className="block truncate text-sm font-medium text-gray-900">
254                                     {d.title}
255                                 </span>
256                                 <span className="block truncate text-xs text-gray-500">
257                                     {d.course_name}
258                                 </span>
259                                 </span>
260                                 <span className="shrink-0 whitespace-nowrap text-xs text-gray-400">
261                                     {formatLocalDate(d.due_date, "MMM d")}
262                                 </span>
263                             </button>
264                         </li>
265                     ))}
266                 </ul>
267             ) : (
268                 <p className="py-6 text-center text-sm text-gray-500">
269                     {t("dashboard.noDeadlines", "No upcoming deadlines")}
270                 </p>
271             )}
272         </div>
273     </div>
274 </PCard>
275 </div>
276
277 { /* AI Tutor entry */ }
278 <PCard>
279     <div className="flex flex-col gap-3 p-6 sm:flex-row sm:items-center sm:justify-between">
280         <div className="flex items-center gap-3">
281             <SeverityIcon>
282                 icon={Bot}
283                 severity="brand"
284                 label={t("nav.aiTutor", { ns: "common" })}

```

```

285 |         />
286 |     <div>
287 |         <p className="text-sm font-semibold text-gray-900">
288 |             {t("nav.aiTutor", { ns: "common" })}
289 |         </p>
290 |         <p className="text-xs text-gray-500">
291 |             {t(
292 |                 "dashboard.tutorPrompt",
293 |                 "Get unstuck with your AI study partner"
294 |             )}
295 |         </p>
296 |     </div>
297 | </div>
298 | <Button
299 |     variant="tactile"
300 |     className="shrink-0"
301 |     onClick={() => navigate("/student/tutor")}
302 | >
303 |     {t("nav.aiTutor", { ns: "common" })}
304 |     <ArrowRight className="h-4 w-4" aria-hidden="true" />
305 | </Button>
306 | </div>
307 | </PCard>
308 | </div>
309 | );
310 | };
311 |
312 | export default StudentDashboardNew;
313 |

```

FILE: src/components/shared/StudentHeaderStats.tsx

TYPE: .tsx | LINES: 1-32 | SHA256: 15fac22169adf1fa3c83a9e3c460b8b056ef4b9cf1353a11642ca5e3e82e0fdc

```

1 | import { useAuth } from "@/hooks/useAuth";
2 | import { useStreak } from "@/hooks/useStreak";
3 | import { useXPBalance } from "@/hooks/useXPBalance";
4 |
5 | /** Prototype `top-stats` treatment for students - 0=¥% streak (orange) + 0=ÜŽ gems (amber). */
6 | const StudentHeaderStats = () => {
7 |     const { user } = useAuth();
8 |     const streak = useStreak();
9 |     const xp = useXPBalance(user?.id ?? "");
10 |
11 |     if (!user) return null;
12 |
13 |     return (
14 |         <div className="top-stats" aria-label="Student progress">
15 |             <span className="stat-chip" style={{ color: "#ea580c" }}>
16 |                 <span aria-hidden="true" style={{ fontSize: "14px", lineHeight: 1 }}>
17 |                     0=¥%
18 |                 </span>
19 |                 {streak.data?.streak_count ?? 0}
20 |             </span>
21 |             <span className="stat-chip" style={{ color: "#d97706" }}>
22 |                 <span aria-hidden="true" style={{ fontSize: "14px", lineHeight: 1 }}>
23 |                     0=ÜŽ
24 |                 </span>
25 |                 {(xp.data?.balance ?? 0).toLocaleString()}
26 |             </span>
27 |         </div>
28 |     );
29 | };
30 |
31 | export default StudentHeaderStats;
32 |

```

FILE: src/components/shared/StudentSidebarExtras.tsx

TYPE: .tsx | LINES: 1-33 | SHA256: 5d49fc615c9ae59eefa94616422257c824391eb43dbc2f5d34152870a619040a

```

1 | import { useTranslation } from "react-i18next";
2 | import { Link } from "react-router-dom";
3 | import { Crown } from "lucide-react";
4 |
5 | /** Prototype student-only upgrade card. */
6 | const StudentSidebarExtras = () => {
7 |     const { t } = useTranslation("common");
8 |
9 |     return (
10 |         <div className="mt-auto space-y-2">
11 |             <Link
12 |                 to="/student/marketplace"
13 |                 className="side-upgrade block rounded-[16px] border border-[#bfe6f5] bg-[linear-gradient(135deg,#f0fdf
14 |             >
15 |                 <span className="su-ic mx-auto mb-2 flex size-[36px] items-center justify-center rounded-[11px] bg-[im
16 |                 <Crown className="size-4" aria-hidden="true" />
17 |             </span>
18 |             <p className="su-t mt-0 text-[13px] font-black text-[#075985]">
19 |                 {t("nav.upgradeToPremium")}
20 |             </p>
21 |             <p className="su-sub mx-0 mt-[5px] mb-[11px] text-[11px] font-medium leading-[1.4] text-[#0e7490] opac
22 |                 {t("nav.upgradeDescription")}
23 |             </p>
24 |             <span className="su-btn block rounded-[11px] bg-[image:var(--brand-gradient)] px-[9px] py-[9px] text-[
25 |                 {t("nav.exploreRewards")}
26 |             </span>
27 |             </Link>
28 |         </div>
29 |     );
30 | };
31 |
32 | export default StudentSidebarExtras;
33 |

```

FILE: src/components/shared/StudentTeamSection.tsx

TYPE: .tsx | LINES: 1-69 | SHA256: b4a7e1b7a5928f8d074ae704c784cc6eb7315c0676f8a5262753450c605fd47f

```
1 | import { useTranslation } from "react-i18next";
2 | import { Users } from "lucide-react";
3 | import { Card } from "@components/ui/card";
4 | import TeamDashboardCard from "@components/shared/TeamDashboardCard";
5 | import { useInViewPort } from "@hooks/useInViewPort";
6 | import { useFirstEnrolledCourseId } from "@hooks/useFirstEnrolledCourse";
7 | import { useMyTeamId } from "@hooks/useTeamLeaderboard";
8 | import { useTeams, useTeamGamification } from "@hooks/useTeams";
9 |
10 | /**
11 |  * Viewport-gated Team dashboard section (Option J, Phase 1).
12 |  *
13 |  * Extracted out of `StudentDashboard` so its four team-related queries
14 |  * (`useFirstEnrolledCourseId` -> `useMyTeamId` / `useTeams` -> `useTeamGamification`)
15 |  * fire ONLY when the section scrolls into view, instead of firing on the blind
16 |  * post-paint timer along with the rest of the dashboard's deferred fan-out. This
17 |  * removes up to four requests from the initial mount burst on a constrained
18 |  * database. Behavior and rendering are otherwise identical to the previous
19 |  * inline block: the card shows only when the student belongs to a team.
20 |  */
21 | const StudentTeamSection = ({ studentId }: { studentId: string }) => {
22 |   const { t } = useTranslation("student");
23 |   const { ref, inView } = useInViewPort<HTMLDivElement>({
24 |     rootMargin: "200px",
25 |   });
26 |
27 |   // Gate every query on visibility: pass undefined ids / enabled:false until the
28 |   // section is in view, mirroring the prior `deferredReady` gating but driven by
29 |   // scroll position rather than a fixed timer.
30 |   const gatedStudentId = inView ? studentId : undefined;
31 |
32 |   const { data: firstCourseId } = useFirstEnrolledCourseId(
33 |     studentId || undefined,
34 |     { enabled: inView }
35 |   );
36 |   const { data: myTeamId } = useMyTeamId(
37 |     gatedStudentId,
38 |     firstCourseId ?? undefined
39 |   );
40 |   const { data: teamsData } = useTeams(firstCourseId ?? undefined);
41 |   const myTeam = (teamsData ?? []).find((team) => team.id === myTeamId);
42 |   const { data: teamGamification } = useTeamGamification(myTeamId ?? undefined);
43 |
44 |   return (
45 |     <div ref={ref}>
46 |       {inView && myTeam && (
47 |         <Card className="bg-white border-0 shadow-md rounded-xl overflow-hidden gap-0 py-0">
48 |           <div
49 |             className="px-6 py-4 flex items-center gap-2"
50 |             style={{
51 |               background: "var(--brand-gradient)",
52 |             }}
53 |           >
54 |             <Users className="h-5 w-5 text-white" />
55 |             <h2 className="text-lg font-bold tracking-tight text-white">
56 |               {t("dashboard.myTeam")}
57 |             </h2>
58 |           </div>
59 |           <div className="p-6">
60 |             <TeamDashboardCard team={myTeam} gamification={teamGamification} />
61 |           </div>
62 |         </Card>
63 |       )}
64 |     </div>
65 |   );
66 | };
67 |
68 | export default StudentTeamSection;
69 |
```

FILE: src/components/shared/StudySessionCard.tsx

TYPE: .tsx | LINES: 1-121 | SHA256: 0ef896eec42ca69a516c62b698d87a3bc67b0b63fb0343702508948b0d139358

```
1 | // =====
2 | // StudySessionCard – Card showing session title, time, course, duration,
3 | // status badge, and Start/Edit actions
4 | // =====
5 |
6 | import { Badge } from "@components/ui/badge";
7 | import { Button } from "@components/ui/button";
8 | import { cn } from "@lib/utills";
9 | import type { StudySession } from "@types/planner";
10 | import { Clock, BookOpen, Play, Pencil } from "lucide-react";
11 |
12 | interface StudySessionCardProps {
13 |   session: StudySession;
14 |   onStart?: (session: StudySession) => void;
15 |   onEdit?: (session: StudySession) => void;
16 |   compact?: boolean;
17 | }
18 |
19 | const STATUS_STYLES: Record<
20 |   StudySession["status"],
21 |   { label: string; className: string }
22 | > = {
23 |   planned: { label: "Planned", className: "bg-blue-100 text-blue-700" },
24 |   in_progress: {
25 |     label: "In Progress",
26 |     className: "bg-amber-100 text-amber-700",
27 |   },

```

```

28 |   completed: { label: "Completed", className: "bg-green-100 text-green-700" },
29 |   cancelled: { label: "Cancelled", className: "bg-gray-100 text-gray-500" },
30 | };
31 |
32 | const StudySessionCard = ({
33 |   session,
34 |   onStart,
35 |   onEdit,
36 |   compact = false,
37 | }: StudySessionCardProps) => {
38 |   const statusConfig = STATUS_STYLES[session.status];
39 |   const canStart = session.status === "planned";
40 |   const canEdit = session.status === "planned";
41 |
42 |   const formatDuration = (minutes: number): string => {
43 |     if (minutes < 60) return `${minutes}m`;
44 |     const h = Math.floor(minutes / 60);
45 |     const m = minutes % 60;
46 |     return m > 0 ? `${h}h ${m}m` : `${h}h`;
47 |   };
48 |
49 |   return (
50 |     <div
51 |       className={cn(
52 |         "rounded-lg border border-blue-200 bg-blue-50/50 p-2.5 transition-shadow hover:shadow-sm",
53 |         compact && "p-2"
54 |       )}
55 |     >
56 |       {/* Title + Status */}
57 |       <div className="flex items-start justify-between gap-1">
58 |         <h4 className="text-xs font-semibold text-gray-900 line-clamp-1">
59 |           {session.title}
60 |         </h4>
61 |         <Badge
62 |           className={cn(
63 |             "shrink-0 text-[10px] px-1.5 py-0",
64 |             statusConfig.className
65 |           )}
66 |         >
67 |           {statusConfig.label}
68 |         </Badge>
69 |       </div>
70 |
71 |       {/* Time + Duration + Course */}
72 |       <div className="mt-1.5 flex flex-wrap items-center gap-x-3 gap-y-1 text-[11px] text-gray-500">
73 |         <span className="flex items-center gap-1">
74 |           <Clock className="h-3 w-3" />
75 |           {session.plannedStartTime}
76 |         </span>
77 |         <span className="flex items-center gap-1">
78 |           {formatDuration(session.plannedDurationMinutes)}
79 |         </span>
80 |         {session.courseName && (
81 |           <span className="flex items-center gap-1">
82 |             <BookOpen className="h-3 w-3" />
83 |             <span className="line-clamp-1">{session.courseName}</span>
84 |           </span>
85 |         )}
86 |       </div>
87 |
88 |       {/* Actions */}
89 |       {(canStart || canEdit) && !compact && (
90 |         <div className="mt-2 flex items-center gap-1.5">
91 |           {canStart && onStart && (
92 |             <Button
93 |               size="sm"
94 |               variant="tactile"
95 |               className="h-7 gap-1 text-[11px]"
96 |               onClick={() => onStart(session)}
97 |             >
98 |               <Play className="h-3 w-3" />
99 |               Start
100 |            </Button>
101 |          )}
102 |           {canEdit && onEdit && (
103 |             <Button
104 |               variant="outline"
105 |               size="sm"
106 |               className="h-7 gap-1 text-[11px]"
107 |               onClick={() => onEdit(session)}
108 |             >
109 |               <Pencil className="h-3 w-3" />
110 |               Edit
111 |            </Button>
112 |          )}
113 |         </div>
114 |       )}
115 |     </div>
116 |   );
117 | };
118 |
119 | export default StudySessionCard;
120 | export type { StudySessionCardProps };
121 |

```

FILE: [src/components/shared/StudyTimeChart.tsx](#)

TYPE: .tsx | LINES: 1-230 | SHA256: dda722f3d354f64614b9d3910f932efae0b257c40c3e1aef550287fa736d7ee9

```

1 | // =====
2 | // StudyTimeChart – Recharts BarChart showing weekly study hours for last 8
3 | // weeks with average line, course filter toggle
4 | // =====
5 |
6 | import { useState, useMemo, useId } from "react";

```

[illegible]

```

111 | >
112 |   <Button
113 |     variant="outline"
114 |     size="sm"
115 |     onClick={() => handleFilterChange(null)}
116 |     aria-pressed={localFilter === null}
117 |     className={cn(
118 |       "px-3 py-1 text-xs font-semibold rounded-xl",
119 |       localFilter === null
120 |       ? "bg-blue-600 text-white border-blue-600 hover:bg-blue-700 hover:text-white"
121 |       : "bg-white text-gray-600 border-gray-200"
122 |     )}
123 |   >
124 |     All Courses
125 |   </Button>
126 |   {courseOptions.map((course) => (
127 |     <Button
128 |       key={course.id}
129 |       variant="outline"
130 |       size="sm"
131 |       onClick={() => handleFilterChange(course.id)}
132 |       aria-pressed={localFilter === course.id}
133 |       className={cn(
134 |         "px-3 py-1 text-xs font-semibold rounded-xl",
135 |         localFilter === course.id
136 |         ? "bg-blue-600 text-white border-blue-600 hover:bg-blue-700 hover:text-white"
137 |         : "bg-white text-gray-600 border-gray-200"
138 |       )}
139 |     >
140 |       {course.name}
141 |     </Button>
142 |   ))}
143 | </div>
144 | )}
145 |
146 | {/* Chart */}
147 | {chartData.length === 0 ? (
148 |   <p className="text-sm text-gray-400 text-center py-8">
149 |     No study data available
150 |   </p>
151 | ) : (
152 |   <
153 |     <ResponsiveContainer width="100%" height={300}>
154 |       <BarChart
155 |         data={chartData}
156 |         margin={{ top: 10, right: 10, bottom: 4, left: -10 }}
157 |         >
158 |           <CartesianGrid strokeDasharray="3 3" vertical={false} />
159 |           <XAxis
160 |             dataKey="weekLabel"
161 |             tick={{ fontSize: 11 }}
162 |             tickLine={false}
163 |             axisLine={false}
164 |           />
165 |           <YAxis
166 |             tick={{ fontSize: 11 }}
167 |             tickLine={false}
168 |             axisLine={false}
169 |             tickFormatter={(v: number) => `${v}h`}
170 |           />
171 |           <Tooltip
172 |             formatter={
173 |               ((value: number | undefined) => [
174 |                 `${value ?? 0}h`,
175 |                 "Study Time",
176 |               ]) as never
177 |             }
178 |             contentStyle={{ borderRadius: 8, fontSize: 12 }}
179 |           />
180 |           {averageHours > 0 && (
181 |             <ReferenceLine
182 |               y={averageHours}
183 |               stroke="#9ca3af"
184 |               strokeDasharray="6 4"
185 |               label={{
186 |                 value: `Avg: ${averageHours}h`,
187 |                 position: "insideTopRight",
188 |                 fill: "#9ca3af",
189 |                 fontSize: 11,
190 |               }}
191 |             />
192 |           )}
193 |           <Bar
194 |             dataKey="hours"
195 |             fill={`url(#${gradientId})`}
196 |             radius={[4, 4, 0, 0]}
197 |             maxBarSize={40}
198 |           />
199 |           <defs>
200 |             <linearGradient id={gradientId} x1="0" y1="0" x2="0" y2="1">
201 |               <stop offset="0%" stopColor="#3b82f6" />
202 |               <stop offset="100%" stopColor="#2563eb" />
203 |             </linearGradient>
204 |           </defs>
205 |         </BarChart>
206 |       </ResponsiveContainer>
207 |
208 |       {/* Average summary */}
209 |       <div className="flex items-center justify-center gap-4 text-xs text-gray-500">
210 |         <div className="flex items-center gap-1.5">
211 |           <div className="h-3 w-3 rounded-sm bg-blue-500" />
212 |           <span>Weekly Hours</span>
213 |         </div>
214 |         {averageHours > 0 && (

```



```

215 |         <div className="flex items-center gap-1.5">
216 |             <div className="h-0.5 w-4 border-t-2 border-dashed border-gray-400" />
217 |             <span>Average ({averageHours}h/week)</span>
218 |         </div>
219 |     )}
220 | </div>
221 | </>
222 | })
223 | </div>
224 | </PCard>
225 | );
226 | };
227 |
228 | export default StudyTimeChart;
229 | export type { StudyTimeChartProps };
230 |

```

FILE: [src/components/shared/SubCLORow.tsx](#)

TYPE: .tsx | LINES: 1-99 | SHA256: 5a2e6bf5895b80132b2aa677c8747015595f5f65a8a944e8f9421b4733f8e05a

```

1 | import { useState } from "react";
2 | import { ChevronRight, Trash2 } from "lucide-react";
3 | import { cn } from "@lib/utills";
4 | import { Button } from "@components/ui/button";
5 | import { Badge } from "@components/ui/badge";
6 | import CLOProgressBar from "@components/shared/CLOProgressBar";
7 | import type { LearningOutcome } from "@types/app";
8 |
9 | interface SubCLORowProps {
10 |   subCLO: LearningOutcome & { weight?: number };
11 |   attainment?: number;
12 |   onDelete?: (id: string) => void;
13 |   deleteDisabled?: boolean;
14 |   deleteTooltip?: string;
15 | }
16 |
17 | const SubCLORow = ({
18 |   subCLO,
19 |   attainment,
20 |   onDelete,
21 |   deleteDisabled,
22 |   deleteTooltip,
23 | }: SubCLORowProps) => {
24 |   const [expanded, setExpanded] = useState(false);
25 |
26 |   return (
27 |     <div className="border-s-2 border-slate-200 ms-6 ps-4 py-2">
28 |       <div className="flex items-center gap-3">
29 |         <button
30 |           type="button"
31 |           onClick={() => setExpanded(!expanded)}
32 |           className="p-1 rounded hover:bg-slate-100 transition-colors"
33 |           aria-expanded={expanded}
34 |           aria-label={`Toggle details for ${subCLO.title}`}
35 |         >
36 |           <ChevronRight
37 |             className={cn(
38 |               "h-4 w-4 text-slate-400 transition-transform",
39 |               expanded && "rotate-90"
40 |             )}
41 |           />
42 |         </button>
43 |
44 |         <Badge variant="outline" className="text-xs font-mono">
45 |           {"SC"}
46 |         </Badge>
47 |
48 |         <span className="text-sm font-medium flex-1 truncate">
49 |           {subCLO.title}
50 |         </span>
51 |
52 |         {subCLO.weight !== undefined && (
53 |           <span className="text-xs text-slate-500 tabular-nums">
54 |             w: {(subCLO.weight * 100).toFixed(0)}%
55 |           </span>
56 |         )}
57 |
58 |         {attainment !== undefined && (
59 |           <div className="w-24">
60 |             <CLOProgressBar
61 |               title={subCLO.title}
62 |               bloomsLevel={subCLO.blooms_level ?? "remembering"}
63 |               attainmentPercent={attainment}
64 |               attainmentLevel={
65 |                 attainment >= 85
66 |                   ? "Excellent"
67 |                   : attainment >= 70
68 |                     ? "Satisfactory"
69 |                     : attainment >= 50
70 |                       ? "Developing"
71 |                       : "Not_Yet"
72 |               }
73 |             </div>
74 |           </div>
75 |         )}
76 |
77 |         {onDelete && (
78 |           <Button
79 |             variant="ghost"
80 |             size="sm"
81 |             onClick={() => onDelete(subCLO.id)}
82 |             disabled={deleteDisabled}
83 |             title={deleteTooltip}
84 |             className="h-7 w-7 p-0 text-slate-400 hover:text-red-500"

```

```

85 |         >
86 |         <Trash2 className="h-3.5 w-3.5" />
87 |       </Button>
88 |     )}
89 |   </div>
90 |
91 |   {expanded && subCLO.description && (
92 |     <p className="text-xs text-slate-500 mt-1 ms-8">{subCLO.description}</p>
93 |   )}
94 | </div>
95 | );
96 | };
97 |
98 | export default SubCLORow;
99 |

```

FILE: src/components/shared/SuggestedSessionItem.tsx

TYPE: .tsx | LINES: 1-73 | SHA256: 226e76e5f468c63ac8c55510a0c0fb92eb5287562e4273737b48b7ab998a4d7e

```

1 | // =====
2 | // SuggestedSessionItem - Derived, non-persisted study-session suggestion shown
3 | // on otherwise-empty planner days (R19.2). Renders a dashed "suggested" card
4 | // with an optional "Plan session" action. All copy is routed through i18next
5 | // (en + ar) via the `student` namespace.
6 | // =====
7 |
8 | import { useTranslation } from "react-i18next";
9 | import { Button } from "@components/ui/button";
10 | import { cn } from "@lib/Utils";
11 | import type { SuggestedStudySession } from "@types/planner";
12 | import { Lightbulb, Plus } from "lucide-react";
13 |
14 | interface SuggestedSessionItemProps {
15 |   suggestion: SuggestedStudySession;
16 |   onPlan?: (suggestion: SuggestedStudySession) => void;
17 |   compact?: boolean;
18 | }
19 |
20 | const SuggestedSessionItem = ({
21 |   suggestion,
22 |   onPlan,
23 |   compact = false,
24 | }: SuggestedSessionItemProps) => {
25 |   const { t } = useTranslation("student");
26 |
27 |   const heading = suggestion.courseName
28 |     ? t("planner.suggestion.withCourse", { course: suggestion.courseName })
29 |     : t("planner.suggestion.label");
30 |
31 |   return (
32 |     <div
33 |       className={cn(
34 |         "rounded-lg border border-dashed border-teal-300 bg-teal-50/40 p-2.5",
35 |         compact && "p-2"
36 |       )}
37 |       data-testid="suggested-session"
38 |     >
39 |       <div className="flex items-start gap-2">
40 |         <Lightbulb className="mt-0.5 h-3.5 w-3.5 shrink-0 text-teal-500" />
41 |         <div className="min-w-0 flex-1">
42 |           <h4 className="text-xs font-semibold text-gray-900 line-clamp-1">
43 |             {heading}
44 |           </h4>
45 |           <p className="mt-0.5 text-[11px] text-gray-500">
46 |             {t("planner.suggestion.duration", {
47 |               minutes: suggestion.durationMinutes,
48 |             })}
49 |           </p>
50 |         </div>
51 |       </div>
52 |
53 |       {onPlan && (
54 |         <Button
55 |           variant="outline"
56 |           size="sm"
57 |           className="mt-2 h-7 w-full gap-1 border-teal-300 text-[11px] text-teal-700 hover:bg-teal-100/60"
58 |           onClick={() => onPlan(suggestion)}
59 |           aria-label={t("planner.suggestion.ariaPlan", {
60 |             minutes: suggestion.durationMinutes,
61 |           })}
62 |         >
63 |           <Plus className="h-3 w-3" />
64 |           {t("planner.suggestion.plan")}
65 |         </Button>
66 |       )}
67 |     </div>
68 |   );
69 | };
70 |
71 | export default SuggestedSessionItem;
72 | export type { SuggestedSessionItemProps };
73 |

```

FILE: src/components/shared/SurveyForm.tsx

TYPE: .tsx | LINES: 1-126 | SHA256: a64c860224ed051417f70c9871e21eae4603ee573ea1f580ad9784b30a464c8a

```

1 | // =====
2 | // SurveyForm - Generic survey/questionnaire form
3 | // =====
4 |
5 | import { useState } from "react";
6 | import { Loader2 } from "lucide-react";
7 | import { Button } from "@components/ui/button";
8 | import { Input } from "@components/ui/input";

```

```

9 | import { Textarea } from "@components/ui/textarea";
10 | import { cn } from "@lib/utlis";
11 |
12 | type QuestionType = "likert" | "mcq" | "text";
13 |
14 | interface SurveyQuestion {
15 |   id: string;
16 |   questionText: string;
17 |   questionType: QuestionType;
18 |   options?: string[];
19 | }
20 |
21 | interface SurveyFormProps {
22 |   title: string;
23 |   questions: SurveyQuestion[];
24 |   onSubmit: (responses: Record<string, string>) => Promise<void>;
25 |   className?: string;
26 | }
27 |
28 | const LIKERT_OPTIONS = [
29 |   "Strongly Disagree",
30 |   "Disagree",
31 |   "Neutral",
32 |   "Agree",
33 |   "Strongly Agree",
34 | ];
35 |
36 | const SurveyForm = ({
37 |   title,
38 |   questions,
39 |   onSubmit,
40 |   className,
41 | }: SurveyFormProps) => {
42 |   const [responses, setResponses] = useState<Record<string, string>>({});
43 |   const [isSubmitting, setIsSubmitting] = useState(false);
44 |
45 |   const setResponse = (questionId: string, value: string) => {
46 |     setResponses((prev) => ({ ...prev, [questionId]: value }));
47 |   };
48 |
49 |   const handleSubmit = async (e: React.FormEvent) => {
50 |     e.preventDefault();
51 |     setIsSubmitting(true);
52 |     try {
53 |       await onSubmit(responses);
54 |     } finally {
55 |       setIsSubmitting(false);
56 |     }
57 |   };
58 |
59 |   return (
60 |     <Form onSubmit={handleSubmit} className={cn("space-y-6", className)}>
61 |       <h2 className="text-lg font-bold tracking-tight">{title}</h2>
62 |       {questions.map((q, idx) => (
63 |         <div key={q.id} className="space-y-2">
64 |           <p className="text-sm font-medium">
65 |             {idx + 1}. {q.questionText}
66 |           </p>
67 |           {q.questionType === "likert" && (
68 |             <div className="flex flex-wrap gap-2">
69 |               {LIKERT_OPTIONS.map((opt) => (
70 |                 <button
71 |                   key={opt}
72 |                   type="button"
73 |                   onClick={() => setResponse(q.id, opt)}
74 |                   className={cn(
75 |                     "rounded-lg border px-3 py-1.5 text-xs font-medium transition-colors",
76 |                     responses[q.id] === opt
77 |                       ? "border-blue-500 bg-blue-50 text-blue-700"
78 |                       : "border-gray-200 text-gray-600 hover:bg-gray-50"
79 |                   )}
80 |                 >
81 |                   {opt}
82 |                 </button>
83 |               ))}
84 |             </div>
85 |           )}
86 |           {q.questionType === "mcq" && q.options && (
87 |             <div className="space-y-1">
88 |               {q.options.map((opt) => (
89 |                 <label
90 |                   key={opt}
91 |                   className="flex items-center gap-2 text-sm cursor-pointer"
92 |                 >
93 |                   <Input
94 |                     type="radio"
95 |                     name={q.id}
96 |                     value={opt}
97 |                     checked={responses[q.id] === opt}
98 |                     onChange={() => setResponse(q.id, opt)}
99 |                     className="h-4 w-4"
100 |                   />
101 |                   {opt}
102 |                 </label>
103 |               ))}
104 |             </div>
105 |           )}
106 |           {q.questionType === "text" && (
107 |             <Textarea
108 |               value={responses[q.id] ?? ""}
109 |               onChange={(e) => setResponse(q.id, e.target.value)}
110 |               placeholder="Your answer..."
111 |               rows={3}
112 |             />

```

```

113 |         }}
114 |       </div>
115 |     ))}
116 |     <Button type="submit" disabled={isSubmitting} variant="tactile">
117 |       {isSubmitting && <Loader2 className="h-4 w-4 animate-spin" />}
118 |       Submit
119 |     </Button>
120 |   </form>
121 | );
122 | };
123 |
124 | export default SurveyForm;
125 | export type { SurveyFormProps, SurveyQuestion, QuestionType };
126 |

```

FILE: [src/components/shared/TeacherDashboardNew.tsx](#)

TYPE: .tsx | LINES: 1-215 | SHA256: 1892eda9ad6fd1d25dffbbe4bacc742f15abd736e2fe3ee67763119629e5ffc8

```

1 | // =====
2 | // TeacherDashboardNew – redesigned teacher dashboard (P2, spec task 2.2)
3 | // =====
4 | //
5 | // At-a-glance teaching cockpit gated behind `newUiDashboards` (see the wrapper
6 | // in TeacherDashboard.tsx). REUSES the existing `useTeacherDashboardAggregate`
7 | // (one round-trip: KPIs + Bloom's distribution, no new writes) and is composed
8 | // from the P0 primitives (WelcomeHero, KPICard, SectionHeader, MasteryRing,
9 | // SeverityIcon, tactile Button, `card-elevated`). i18n reuses the existing
10 | // `teacher` namespace keys. Flag-off keeps the current dashboard unchanged.
11 | // =====
12 |
13 | import { useNavigate } from "react-router-dom";
14 | import { useTranslation } from "react-i18next";
15 | import {
16 |   AlertTriangle,
17 |   ArrowRight,
18 |   CheckSquare,
19 |   ClipboardList,
20 |   Layers,
21 |   TrendingUp,
22 | } from "lucide-react";
23 |
24 | import {
25 |   Button,
26 |   Card,
27 |   KPICard,
28 |   MasteryRing,
29 |   SectionHeader,
30 |   SeverityIcon,
31 |   Shimmer,
32 |   WelcomeHero,
33 | } from "@design-system";
34 | import { useAuth } from "@hooks/useAuth";
35 | import { useTeacherDashboardAggregate } from "@hooks/useTeacherDashboardAggregate";
36 | import { attainmentValueClass } from "@lib/attainmentTone";
37 |
38 | /** Canonical Bloom's-level dot colors (design system domain coding). */
39 | const BLOOM_DOT: Record<string, string> = {
40 |   remembering: "bg-purple-500",
41 |   understanding: "bg-blue-500",
42 |   applying: "bg-green-500",
43 |   analyzing: "bg-yellow-500",
44 |   evaluating: "bg-orange-500",
45 |   creating: "bg-red-500",
46 | };
47 |
48 | const TeacherDashboardNew = () => {
49 |   const { t } = useTranslation("teacher");
50 |   const navigate = useNavigate();
51 |   const { user, profile } = useAuth();
52 |
53 |   const aggregate = useTeacherDashboardAggregate(user?.id);
54 |   const kpis = aggregate.data?.kpis;
55 |   const blooms = aggregate.data?.bloomsDistribution ?? [];
56 |   const loading = aggregate.isPending;
57 |
58 |   const avgAttainment = kpis?.avgAttainment ?? 0;
59 |   const atRisk = kpis?.atRiskCount ?? 0;
60 |
61 |   return (
62 |     <div className="space-y-6">
63 |       <WelcomeHero
64 |         name={profile?.full_name ?? "Teacher"}
65 |         userRole="teacher"
66 |         subtitle={t("dashboard.welcome.subtitle")}
67 |       />
68 |
69 |       { /* KPI row */ }
70 |       { loading ? (
71 |         <div className="grid grid-cols-2 gap-4 md:grid-cols-4">
72 |           {Array.from({ length: 4 }).map((_, i) => (
73 |             <Shimmer key={i} className="h-24 rounded-xl" />
74 |           ))}
75 |         </div>
76 |       ) : (
77 |         <div className="grid grid-cols-2 gap-4 md:grid-cols-4">
78 |           <KPICard
79 |             icon={ClipboardList}
80 |             label={t("dashboard.pendingSubmissions")}
81 |             value={kpis?.pendingSubmissions ?? 0}
82 |           />
83 |           <KPICard
84 |             icon={CheckSquare}
85 |             label={t("dashboard.gradedThisWeek")}
86 |             value={kpis?.gradedThisWeek ?? 0}

```

```

87 |         iconBgClass="bg-green-50"
88 |         iconColorClass="text-green-600"
89 |     />
90 |     <KPICard
91 |         icon={TrendingUp}
92 |         label={t("dashboard.avgAttainment")}
93 |         value={ ${avgAttainment}% }
94 |         valueClassName={attainmentValueClass(avgAttainment)}
95 |     />
96 |     <KPICard
97 |         icon={AlertTriangle}
98 |         label={t("dashboard.atRiskStudents")}
99 |         value={atRisk}
100 |         valueClassName={atRisk > 0 ? "text-red-600" : "text-sky-700"}
101 |         iconBgClass={atRisk > 0 ? "bg-red-50" : "bg-blue-50"}
102 |         iconColorClass={atRisk > 0 ? "text-red-600" : "text-blue-600"}
103 |     />
104 | </div>
105 | })
106 |
107 | {/* Mastery snapshot + Bloom's coverage */}
108 | <div className="grid grid-cols-1 gap-6 lg:grid-cols-2">
109 |     <Card className="card-elevated overflow-hidden border-0 bg-white">
110 |         <div className="p-6">
111 |             <SectionHeader
112 |                 icon={TrendingUp}
113 |                 title={t("dashboard.avgAttainment")}
114 |             />
115 |             <div className="mt-4 flex items-center gap-5">
116 |                 {loading ? (
117 |                     <Shimmer className="h-24 w-24 rounded-full" />
118 |                 ) : (
119 |                     <MasteryRing value={avgAttainment} size={96} tone="auto" />
120 |                 )}
121 |                 <div className="min-w-0">
122 |                     <p className="text-sm text-gray-600">
123 |                         {t("dashboard.totalStudents", "Students")}:{" "}
124 |                         <span className="font-semibold text-gray-900">
125 |                             {kpis?.totalStudents ?? 0}
126 |                         </span>
127 |                     </p>
128 |                     <Button
129 |                         variant="link"
130 |                         className="mt-1 h-auto p-0 text-sky-700"
131 |                         onClick={() => navigate("/teacher/grading")}
132 |                     >
133 |                         {t("dashboard.pendingSubmissions")}
134 |                     <ArrowRight className="h-3.5 w-3.5" aria-hidden="true" />
135 |                 </Button>
136 |             </div>
137 |         </div>
138 |     </div>
139 | </Card>
140 |
141 | <Card className="card-elevated overflow-hidden border-0 bg-white">
142 |     <div className="p-6">
143 |         <SectionHeader
144 |             icon={Layers}
145 |             title={t("dashboard.bloomsDistribution")}
146 |         />
147 |         <div className="mt-4">
148 |             {loading ? (
149 |                 <Shimmer className="h-24 rounded-xl" />
150 |             ) : blooms.length > 0 ? (
151 |                 <ul className="space-y-2">
152 |                     {blooms.map((row) => (
153 |                         <li
154 |                             key={row.level}
155 |                             className="flex items-center gap-3 text-sm"
156 |                         >
157 |                             <span
158 |                                 className={`h-2.5 w-2.5 shrink-0 rounded-full ${
159 |                                     BLOOM_DOT[row.level] ?? "bg-slate-400"
160 |                                 }`}
161 |                                 aria-hidden="true"
162 |                             />
163 |                             <span className="flex-1 capitalize text-gray-700">
164 |                                 {row.level}
165 |                             </span>
166 |                             <span className="font-semibold text-gray-900">
167 |                                 {row.count}
168 |                             </span>
169 |                         </li>
170 |                     ))}
171 |                 </ul>
172 |             ) : (
173 |                 <p className="py-6 text-center text-sm text-gray-500">
174 |                     {t("dashboard.noClosDefined", "No CLOs defined yet")}
175 |                 </p>
176 |             )}
177 |         </div>
178 |     </div>
179 | </Card>
180 | </div>
181 |
182 | {/* Triage CTA */}
183 | <Card className="card-elevated overflow-hidden border-0 bg-white">
184 |     <div className="flex flex-col gap-3 p-6 sm:flex-row sm:items-center sm:justify-between">
185 |         <div className="flex items-center gap-3">
186 |             <SeverityIcon
187 |                 icon={AlertTriangle}
188 |                 severity={atRisk > 0 ? "high" : "low"}
189 |                 label={t("dashboard.atRiskStudents")}
190 |             />

```

```

191 |         <div>
192 |             <p className="text-sm font-semibold text-gray-900">
193 |                 {atRisk} · {t("dashboard.atRiskStudents")}
194 |             </p>
195 |             <p className="text-xs text-gray-500">
196 |                 {t("dashboard.gradedThisWeek")}: {kpis?.gradedThisWeek ?? 0}
197 |             </p>
198 |         </div>
199 |     </div>
200 |     <Button
201 |         variant="tactile"
202 |         className="shrink-0"
203 |         onClick={() => navigate("/teacher/grading")}
204 |     >
205 |         {t("dashboard.pendingSubmissions")}
206 |         <ArrowRight className="h-4 w-4" aria-hidden="true" />
207 |     </Button>
208 | </div>
209 | </Card>
210 | </div>
211 | );
212 | };
213 |
214 | export default TeacherDashboardNew;
215 |

```

FILE: [src/components/shared/TeacherHandoffCard.tsx](#)

TYPE: .tsx | LINES: 1-109 | SHA256: 5103ca25582485891c723c94b4b488dea885d73a9d60ee88e3ab8c9073ec3e88

```

1 | // =====
2 | // TeacherHandoffCard - In-conversation handoff suggestion with consent
3 | // Task 18.3
4 | // =====
5 |
6 | import { useState } from "react";
7 | import { Card } from "@components/ui/card";
8 | import { Button } from "@components/ui/button";
9 | import { Checkbox } from "@components/ui/checkbox";
10 | import { Label } from "@components/ui/label";
11 | import { Handshake, X, Loader2 } from "lucide-react";
12 |
13 | interface TeacherHandoffCardProps {
14 |     /** Why the handoff was suggested */
15 |     reason: string;
16 |     /** Human-readable message explaining the suggestion */
17 |     message: string;
18 |     /** Whether the create-handoff mutation is in progress */
19 |     isPending?: boolean;
20 |     /** Called when the student accepts with consent */
21 |     onAccept: () => void;
22 |     /** Called when the student dismisses the suggestion */
23 |     onDismiss: () => void;
24 | }
25 |
26 | const REASON_LABELS: Record<string, string> = {
27 |     low_rag_confidence:
28 |         "I'm having trouble finding relevant course materials for this topic.",
29 |     repeated_question:
30 |         "It looks like we've been going back and forth on this topic.",
31 |     low_satisfaction:
32 |         "It seems my responses haven't been as helpful as they could be.",
33 | };
34 |
35 | const TeacherHandoffCard = ({
36 |     reason,
37 |     message,
38 |     isPending = false,
39 |     onAccept,
40 |     onDismiss,
41 | }: TeacherHandoffCardProps) => {
42 |     const [consentChecked, setConsentChecked] = useState(false);
43 |
44 |     const reasonLabel = REASON_LABELS[reason] ?? message;
45 |
46 |     return (
47 |         <Card className="bg-white border border-amber-200 shadow-md rounded-xl overflow-hidden my-3">
48 |             </* Header */>
49 |             <div className="px-4 py-3 bg-amber-50 flex items-center justify-between">
50 |                 <div className="flex items-center gap-2">
51 |                     <Handshake className="h-5 w-5 text-amber-600" />
52 |                     <span className="text-sm font-bold text-amber-800">
53 |                         Connect with Your Teacher?
54 |                     </span>
55 |                 </div>
56 |                 <button
57 |                     type="button"
58 |                     onClick={onDismiss}
59 |                     className="text-gray-400 hover:text-gray-600 transition-colors"
60 |                     aria-label="Dismiss handoff suggestion"
61 |                 >
62 |                     <X className="h-4 w-4" />
63 |                 </button>
64 |             </div>
65 |
66 |             </* Body */>
67 |             <div className="p-4 space-y-3">
68 |                 <p className="text-sm text-gray-700">{reasonLabel}</p>
69 |                 <p className="text-sm text-gray-600">
70 |                     Your teacher may be able to help. Would you like to connect?
71 |                 </p>
72 |
73 |                 </* Consent checkbox */>
74 |                 <div className="flex items-start gap-2">
75 |                     <Checkbox

```

```

76 |         id="handoff-consent"
77 |         checked={consentChecked}
78 |         onCheckedChange={(checked) => setConsentChecked(checked === true)}
79 |       />
80 |       <Label
81 |         htmlFor="handoff-consent"
82 |         className="text-xs text-gray-600 leading-tight cursor-pointer"
83 |       >
84 |         I agree to share a summary of this conversation with my teacher
85 |       </Label>
86 |     </div>
87 |
88 |     { /* Actions */ }
89 |     <div className="flex items-center gap-2 pt-1">
90 |       <Button
91 |         size="sm"
92 |         disabled={!consentChecked || isPending}
93 |         onClick={onAccept}
94 |         variant="tactile"
95 |       >
96 |         {isPending && <Loader2 className="h-4 w-4 animate-spin" />}
97 |         Connect with Teacher
98 |       </Button>
99 |       <Button size="sm" variant="ghost" onClick={onDismiss}>
100 |         No thanks
101 |       </Button>
102 |     </div>
103 |   </div>
104 | </Card>
105 | );
106 | };
107 |
108 | export default TeacherHandoffCard;
109 |

```

FILE: src/components/shared/TeachingMomentRating.tsx

TYPE: .tsx | LINES: 1-142 | SHA256: 73733e71c098e4aa4b6fd7760c16ff6db57b9c1aaa3e0d4a4569f9ad02fcf642

```

1 | // =====
2 | // TeachingMomentRating – Star rating for clarity (1-5) and helpfulness (1-5)
3 | // Task 4.12: interactive star rating component
4 | // =====
5 |
6 | import { useState, useCallback } from "react";
7 | import { cn } from "@lib/Utils";
8 | import { Button } from "@components/ui/button";
9 | import { Star, Loader2 } from "lucide-react";
10 |
11 | export interface TeachingMomentRatingProps {
12 |   onSubmit: (clarity: number, helpfulness: number) => void;
13 |   isPending?: boolean;
14 |   initialClarity?: number;
15 |   initialHelpfulness?: number;
16 |   readOnly?: boolean;
17 |   className?: string;
18 | }
19 |
20 | interface StarRatingProps {
21 |   label: string;
22 |   value: number;
23 |   onChange: (value: number) => void;
24 |   readOnly?: boolean;
25 | }
26 |
27 | const StarRating = ({
28 |   label,
29 |   value,
30 |   onChange,
31 |   readOnly = false,
32 | }: StarRatingProps) => {
33 |   const [hovered, setHovered] = useState(0);
34 |
35 |   return (
36 |     <div className="space-y-1">
37 |       <span className="text-xs font-medium text-gray-600">{label}</span>
38 |       <div
39 |         className="flex items-center gap-0.5"
40 |         role="radiogroup"
41 |         aria-label={` ${label} rating` }
42 |         tabIndex={0}
43 |         onMouseLeave={() => setHovered(0)}
44 |       >
45 |         {[1, 2, 3, 4, 5].map((star) => {
46 |           const isFilled = star <= (hovered || value);
47 |           return (
48 |             <button
49 |               key={star}
50 |               type="button"
51 |               role="radio"
52 |               aria-checked={star === value}
53 |               aria-label={` ${star} star${star !== 1 ? "s" : ""} ` }
54 |               disabled={readOnly}
55 |               className={cn(
56 |                 "p-0.5 rounded transition-colors focus-visible:ring-2 focus-visible:ring-blue-400",
57 |                 !readOnly &&
58 |                 "cursor-pointer hover:scale-110 transition-transform",
59 |                 readOnly && "cursor-default"
60 |               )}
61 |               onClick={() => !readOnly && onChange(star)}
62 |               onMouseEnter={() => !readOnly && setHovered(star)}
63 |               onKeyDown={(e) => {
64 |                 if (readOnly) return;
65 |                 if (e.key === "ArrowRight" && value < 5) onChange(value + 1);
66 |                 if (e.key === "ArrowLeft" && value > 1) onChange(value - 1);

```

```

67 |         }}
68 |         >
69 |         <Star
70 |           className={cn(
71 |             "h-5 w-5 transition-colors",
72 |             isFilled ? "text-amber-400 fill-amber-400" : "text-gray-300"
73 |           )}
74 |         />
75 |       </button>
76 |     );
77 |   }}
78 |   {value > 0 && (
79 |     <span className="text-xs text-gray-500 ms-1 tabular-nums">
80 |       {value}/5
81 |     </span>
82 |   )}
83 | </div>
84 | </div>
85 | );
86 | };
87 |
88 | const TeachingMomentRating = ({
89 |   onSubmit,
90 |   isPending = false,
91 |   initialClarity = 0,
92 |   initialHelpfulness = 0,
93 |   readOnly = false,
94 |   className,
95 | }: TeachingMomentRatingProps) => {
96 |   const [clarity, setClarity] = useState(initialClarity);
97 |   const [helpfulness, setHelpfulness] = useState(initialHelpfulness);
98 |
99 |   const handleSubmit = useCallback(() => {
100 |     if (clarity > 0 && helpfulness > 0) {
101 |       onSubmit(clarity, helpfulness);
102 |     }
103 |   }, [clarity, helpfulness, onSubmit]);
104 |
105 |   const canSubmit = clarity > 0 && helpfulness > 0 && !isPending && !readOnly;
106 |
107 |   return (
108 |     <div className={cn("space-y-3", className)}>
109 |       <StarRating
110 |         label="Clarity"
111 |         value={clarity}
112 |         onChange={setClarity}
113 |         readOnly={readOnly}
114 |       />
115 |       <StarRating
116 |         label="Helpfulness"
117 |         value={helpfulness}
118 |         onChange={setHelpfulness}
119 |         readOnly={readOnly}
120 |       />
121 |
122 |       {!readOnly && (
123 |         <Button
124 |           size="sm"
125 |           onClick={handleSubmit}
126 |           disabled={!canSubmit}
127 |           className="active:scale-95 transition-transform duration-100 gap-1.5"
128 |         >
129 |           {isPending ? (
130 |             <Loader2 className="h-4 w-4 animate-spin" />
131 |           ) : (
132 |             <Star className="h-4 w-4" />
133 |           )}
134 |           Submit Rating
135 |         </Button>
136 |       )}
137 |     </div>
138 |   );
139 | };
140 |
141 | export default TeachingMomentRating;
142 |

```

FILE: src/components/shared/TeamBadgeCollection.tsx

TYPE: .tsx | LINES: 1-83 | SHA256: 052f21418f6687db0fbff33c74257bb2aa827ab8aecf53ebb6cd6552180d62c5

```

1 | // =====
2 | // TeamBadgeCollection – Grid of earned team badges with badge-pop animation
3 | // Task 4.3: grid display with animation, honors prefers-reduced-motion
4 | // =====
5 |
6 | import { Card } from "@components/ui/card";
7 | import { cn } from "@lib/utils";
8 | import { Award } from "lucide-react";
9 | import { format } from "date-fns";
10 |
11 | export interface TeamBadgeItem {
12 |   id: string;
13 |   badgeKey: string;
14 |   badgeName: string;
15 |   emoji: string;
16 |   description?: string;
17 |   earnedAt: string;
18 | }
19 |
20 | export interface TeamBadgeCollectionProps {
21 |   badges: TeamBadgeItem[];
22 |   className?: string;
23 | }
24 |

```



```

25 const TeamBadgeCollection = ({
26   badges,
27   className,
28 }: TeamBadgeCollectionProps) => {
29   if (badges.length === 0) {
30     return (
31       <div
32         className={cn(
33           "flex flex-col items-center justify-center py-8 text-center",
34           className
35         )}
36     >
37       <Award className="h-8 w-8 text-gray-300 mb-2" />
38       <p className="text-sm text-gray-400">No team badges earned yet</p>
39       <p className="text-xs text-gray-400 mt-0.5">
40         Keep working together to unlock badges!
41       </p>
42     </div>
43   );
44 }
45
46 return (
47   <div
48     className={cn(
49       "grid grid-cols-2 sm:grid-cols-3 md:grid-cols-4 gap-3",
50       className
51     )}
52     role="list"
53     aria-label="Team badges"
54   >
55     {badges.map((badge) => (
56       <Card
57         key={badge.id}
58         role="listitem"
59         className="bg-white border-0 shadow-md rounded-xl p-3 flex flex-col items-center text-center gap-1.5"
60         data-testid={`team-badge-${badge.badgeKey}`}
61       >
62         <span className="text-3xl" aria-hidden="true">
63           {badge.emoji}
64         </span>
65         <span className="text-xs font-bold tracking-wide">
66           {badge.badgeName}
67         </span>
68         {badge.description && (
69           <span className="text-[10px] text-gray-500 leading-tight line-clamp-2">
70             {badge.description}
71           </span>
72         )}
73         <span className="text-[10px] text-gray-400">
74           {format(new Date(badge.earnedAt), "MMM d, yyyy")}
75         </span>
76       </Card>
77     ))}
78   </div>
79 );
80 };
81
82 export default TeamBadgeCollection;
83

```

FILE: src/components/shared/TeamBadgeDisplay.tsx

```
TYPE: .tsx | LINES: 1-125 | SHA256: 109fe67df371ba2d57ca04761202b7f6dc0300e04ea21e1db42718d901dabb76
```

[illegible]

```

42 |     />
43 |   };
44 | }
45 |
46 | if (compact) {
47 |   return (
48 |     <div
49 |       data-testid="team-badge-display-compact"
50 |       className="flex gap-2 flex-wrap"
51 |     >
52 |       {badgeList.length === 0 && (
53 |         <span className="text-xs text-gray-400">No team badges yet</span>
54 |       )}
55 |       <AnimatePresence>
56 |         {badgeList.map((badge) => (
57 |           <motion.div
58 |             key={badge.id}
59 |             initial={{ scale: 0, opacity: 0 }}
60 |             animate={{ scale: 1, opacity: 1 }}
61 |             exit={{ scale: 0, opacity: 0 }}
62 |             transition={{ type: "spring", stiffness: 300, damping: 20 }}
63 |             className="flex items-center gap-1 px-2 py-1 rounded-lg bg-amber-50 border border-amber-200"
64 |             title={` ${badge.badge_name} – ${
65 |               TEAM_BADGE_DESCRIPTIONS[badge.badge_key] ?? ""
66 |             }`}
67 |           >
68 |             <span className="text-sm" aria-hidden="true">
69 |               {badge.emoji}
70 |             </span>
71 |             <span className="text-[10px] font-bold text-amber-700">
72 |               {badge.badge_name}
73 |             </span>
74 |           </motion.div>
75 |         ))}
76 |       </AnimatePresence>
77 |     </div>
78 |   );
79 | }
80 |
81 | return (
82 |   <div data-testid="team-badge-display" className="space-y-3">
83 |     <h3 className="text-sm font-bold text-gray-700">Team Badges</h3>
84 |     {badgeList.length === 0 && (
85 |       <p className="text-xs text-gray-400">
86 |         No team badges earned yet. Keep working together!
87 |       </p>
88 |     )}
89 |     <div className="grid grid-cols-2 sm:grid-cols-4 gap-3">
90 |       <AnimatePresence>
91 |         {badgeList.map((badge) => (
92 |           <motion.div
93 |             key={badge.id}
94 |             initial={{ scale: 0, opacity: 0 }}
95 |             animate={{ scale: 1, opacity: 1 }}
96 |             exit={{ scale: 0, opacity: 0 }}
97 |             transition={{ type: "spring", stiffness: 300, damping: 20 }}
98 |           >
99 |             <Card
100 |               data-testid={`team-badge-${badge.badge_key}`}
101 |               className="bg-white border-0 shadow-md rounded-xl p-3 flex flex-col items-center text-center g
102 |             >
103 |               <span className="text-2xl animate-badge-pop" aria-hidden="true">
104 |                 {badge.emoji}
105 |               </span>
106 |               <span className="text-xs font-bold tracking-wide">
107 |                 {badge.badge_name}
108 |               </span>
109 |               <span className="text-[10px] text-gray-500 leading-tight">
110 |                 {TEAM_BADGE_DESCRIPTIONS[badge.badge_key] ?? ""}
111 |               </span>
112 |               <span className="text-[10px] text-gray-400">
113 |                 {format(new Date(badge.awarded_at), "MMM d, yyyy")}
114 |               </span>
115 |             </Card>
116 |           </motion.div>
117 |         ))}
118 |       </AnimatePresence>
119 |     </div>
120 |   </div>
121 | );
122 | };
123 |
124 | export default TeamBadgeDisplay;
125 |

```

FILE: src/components/shared/TeamCard.tsx

TYPE: .tsx | LINES: 1-134 | SHA256: adc5978cf6b476d70acdd2fef5950aa14cdf1f8ba96d7647b4087ef0f902e674

```

1 | // =====
2 | // TeamCard – Compact team display with gradient header
3 | // Task 4.1: name, XP, streak, member count, health score badge
4 | // =====
5 |
6 | import { PCard } from "@design-system";
7 | import { cn } from "@lib/utils";
8 | import { Users, Flame, Trophy, Zap } from "lucide-react";
9 | import TeamHealthBadge from "@components/shared/TeamHealthBadge";
10 |
11 | export interface TeamCardProps {
12 |   name: string;
13 |   avatarLetter?: string;
14 |   xpTotal: number;
15 |   streakCount: number;
16 |   memberCount: number;

```

```

17 | healthScore: number;
18 | cooperationScore?: number;
19 | className?: string;
20 | onClick?: () => void;
21 | }
22 |
23 | const TeamCard = ({
24 |   name,
25 |   avatarLetter,
26 |   xpTotal,
27 |   streakCount,
28 |   memberCount,
29 |   healthScore,
30 |   cooperationScore,
31 |   className,
32 |   onClick,
33 | }: TeamCardProps) => (
34 |   <PCard
35 |     className={cn(
36 |       "overflow-hidden",
37 |       onClick && "cursor-pointer hover:shadow-lg transition-shadow",
38 |       className
39 |     )}
40 |     onClick={onClick}
41 |     role={onClick ? "button" : undefined}
42 |     tabIndex={onClick ? 0 : undefined}
43 |     onKeyDown={
44 |       onClick
45 |       ? (e) => {
46 |         if (e.key === "Enter" || e.key === " ") {
47 |           e.preventDefault();
48 |           onClick();
49 |         }
50 |       }
51 |       : undefined
52 |     }
53 |   >
54 |     {/* Gradient Header */}
55 |     <div
56 |       className="px-4 py-3 flex items-center gap-3"
57 |       style={{
58 |         background: "var(--brand-gradient)",
59 |       }}
60 |     >
61 |       <div className="h-9 w-9 rounded-full bg-white/20 flex items-center justify-center text-sm font-bold text-white" >
62 |         {avatarLetter ?? name.charAt(0).toUpperCase()}
63 |       </div>
64 |       <h3 className="text-sm font-bold text-white truncate">{name}</h3>
65 |       <div className="ms-auto">
66 |         <TeamHealthBadge score={healthScore} size="sm" />
67 |       </div>
68 |     </div>
69 |
70 |     {/* Body */}
71 |     <div className="p-4">
72 |       <div className="grid grid-cols-2 gap-3">
73 |         {/* XP */}
74 |         <div className="flex items-center gap-2">
75 |           <div className="p-1.5 rounded-lg bg-amber-50">
76 |             <Zap className="h-4 w-4 text-amber-500" />
77 |           </div>
78 |           <div>
79 |             <p className="text-[10px] font-black tracking-widest uppercase text-gray-500">
80 |               Team XP
81 |             </p>
82 |             <p className="text-lg font-black">{xpTotal.toLocaleString()}</p>
83 |           </div>
84 |         </div>
85 |
86 |         {/* Streak */}
87 |         <div className="flex items-center gap-2">
88 |           <div className="p-1.5 rounded-lg bg-orange-50">
89 |             <Flame className="h-4 w-4 text-orange-500 animate-streak-flame" />
90 |           </div>
91 |           <div>
92 |             <p className="text-[10px] font-black tracking-widest uppercase text-gray-500">
93 |               Streak
94 |             </p>
95 |             <p className="text-lg font-black bg-gradient-to-r from-red-500 to-orange-500 bg-clip-text text-transparent">
96 |               {streakCount}
97 |             </p>
98 |           </div>
99 |         </div>
100 |
101 |         {/* Members */}
102 |         <div className="flex items-center gap-2">
103 |           <div className="p-1.5 rounded-lg bg-blue-50">
104 |             <Users className="h-4 w-4 text-blue-500" />
105 |           </div>
106 |           <div>
107 |             <p className="text-[10px] font-black tracking-widest uppercase text-gray-500">
108 |               Members
109 |             </p>
110 |             <p className="text-lg font-black">{memberCount}</p>
111 |           </div>
112 |         </div>
113 |
114 |         {/* Cooperation Score */}
115 |         {cooperationScore !== undefined && (
116 |           <div className="flex items-center gap-2">
117 |             <div className="p-1.5 rounded-lg bg-green-50">
118 |               <Trophy className="h-4 w-4 text-green-500" />
119 |             </div>
120 |             <div>

```

```

121 |         <p className="text-[10px] font-black tracking-widest uppercase text-gray-500">
122 |             Co-op
123 |         </p>
124 |         <p className="text-lg font-black">{cooperationScore}</p>
125 |     </div>
126 | </div>
127 |     })
128 | </div>
129 | </div>
130 | </PCard>
131 | );
132 |
133 | export default TeamCard;
134 |

```

FILE: [src/components/shared/TeamDashboardCard.tsx](#)

TYPE: .tsx | LINES: 1-78 | SHA256: e47dfc05818b68864b48532499a9f24d8f4ce3518dd7516cb93056c64fcc7d36

```

1 | // Task 129.2 + 132.6: Team Dashboard Card with team streak display and team badges
2 |
3 | import { Badge } from "@components/ui/badge";
4 | import { Flame, Users } from "lucide-react";
5 | import type { Team, TeamGamification } from "@hooks/useTeams";
6 | import TeamBadgeDisplay from "@components/shared/TeamBadgeDisplay";
7 | import { PCard } from "@design-system";
8 |
9 | interface TeamDashboardCardProps {
10 |     team: Team;
11 |     gamification?: TeamGamification | null;
12 |     showBadges?: boolean;
13 | }
14 |
15 | const TeamDashboardCard = ({
16 |     team,
17 |     gamification,
18 |     showBadges = true,
19 | }: TeamDashboardCardProps) => (
20 |     <PCard className="p-4 group hover:shadow-lg transition-shadow">
21 |         <div className="flex items-center gap-3">
22 |             <div className="h-10 w-10 rounded-full bg-blue-100 flex items-center justify-center text-lg font-bold te
23 |                 {team.avatar_letter}
24 |             </div>
25 |             <div className="flex-1 min-w-0">
26 |                 <p className="text-sm font-semibold truncate">{team.name}</p>
27 |                 <div className="flex items-center gap-2 mt-0.5">
28 |                     <span className="text-xs text-slate-500 flex items-center gap-1">
29 |                         <Users className="h-3 w-3" />
30 |                         {team.member_count ?? 0}
31 |                     </span>
32 |                     {gamification && (
33 |                         <>
34 |                             <Badge variant="outline" className="text-[10px]">
35 |                                 {gamification.xp_total} XP
36 |                             </Badge>
37 |                         </>
38 |                     )}
39 |                 </div>
40 |             </div>
41 |         </div>
42 |
43 |         {/* Team Streak Display */}
44 |         {gamification && gamification.streak_count > 0 && (
45 |             <div
46 |                 className="mt-3 flex items-center gap-3 px-1"
47 |                 data-testid="team-streak-display"
48 |             >
49 |                 <div className="flex items-center gap-1.5">
50 |                     <div className="p-1 rounded-md bg-orange-50">
51 |                         <Flame className="h-4 w-4 text-orange-500 animate-streak-flame" />
52 |                     </div>
53 |                     <div className="flex items-baseline gap-1">
54 |                         <span
55 |                             className="text-lg font-black bg-gradient-to-r from-red-500 to-orange-500 bg-clip-text text-tran
56 |                             data-testid="team-streak-current"
57 |                         >
58 |                             {gamification.streak_count}
59 |                         </span>
60 |                         <span className="text-[10px] font-medium text-gray-500">
61 |                             day streak
62 |                         </span>
63 |                     </div>
64 |                 </div>
65 |             </div>
66 |         )}
67 |
68 |         {/* Team Badges */}
69 |         {showBadges && (
70 |             <div className="mt-3">
71 |                 <TeamBadgeDisplay teamId={team.id} compact />
72 |             </div>
73 |         )}
74 |     </PCard>
75 | );
76 |
77 | export default TeamDashboardCard;
78 |

```

FILE: src/components/shared/TeamHealthBadge.tsx

TYPE: .tsx | LINES: 1-73 | SHA256: a60a89ba0f3c9b06b9766da09062f529b4183c45d755693cc8cad8720c1cbf9a

```
1 | // =====
2 | // TeamHealthBadge – Color-coded health score badge
3 | // Task 4.13: green "e70, yellow 40-69, red <40
4 | // =====
5 |
6 | import { Badge } from "@components/ui/badge";
7 | import { cn } from "@lib/utils";
8 | import { Heart } from "lucide-react";
9 |
10 | export interface TeamHealthBadgeProps {
11 |   score: number;
12 |   size?: "sm" | "md";
13 |   showIcon?: boolean;
14 |   className?: string;
15 | }
16 |
17 | function getHealthConfig(score: number): {
18 |   label: string;
19 |   classes: string;
20 |   status: string;
21 | } {
22 |   if (score >= 70) {
23 |     return {
24 |       label: "Healthy",
25 |       classes: "bg-green-100 text-green-700 border-green-200",
26 |       status: "healthy",
27 |     };
28 |   }
29 |   if (score >= 40) {
30 |     return {
31 |       label: "Needs Attention",
32 |       classes: "bg-yellow-100 text-yellow-700 border-yellow-200",
33 |       status: "needs_attention",
34 |     };
35 |   }
36 |   return {
37 |     label: "At Risk",
38 |     classes: "bg-red-100 text-red-700 border-red-200",
39 |     status: "at_risk",
40 |   };
41 | }
42 |
43 | const TeamHealthBadge = ({
44 |   score,
45 |   size = "md",
46 |   showIcon = true,
47 |   className,
48 | }: TeamHealthBadgeProps) => {
49 |   const clampedScore = Math.max(0, Math.min(100, Math.round(score)));
50 |   const config = getHealthConfig(clampedScore);
51 |
52 |   return (
53 |     <Badge
54 |       className={cn(
55 |         "font-bold gap-1",
56 |         size === "sm" ? "text-[10px] px-1.5 py-0" : "text-xs px-2 py-0.5",
57 |         config.classes,
58 |         className
59 |       )}
60 |       aria-label={`Health score: ${clampedScore}, ${config.label}`}
61 |       data-testid="team-health-badge"
62 |       data-status={config.status}
63 |     >
64 |       {showIcon && (
65 |         <Heart className={cn(size === "sm" ? "h-3 w-3" : "h-3.5 w-3.5")} />
66 |       )}
67 |       {clampedScore}
68 |     </Badge>
69 |   );
70 | };
71 |
72 | export default TeamHealthBadge;
73 |
```

FILE: src/components/shared/TeamHealthChart.tsx

TYPE: .tsx | LINES: 1-137 | SHA256: 7b062e4a125d346f44a8fb77ff1dae3b6d1ae8cd2a4b84ca26c5e7026fdfcbb5

```
1 | // =====
2 | // TeamHealthChart – Recharts line chart showing health score trend over time
3 | // Task 4.14: line chart with health score history
4 | // =====
5 |
6 | import { useMemo } from "react";
7 | import {
8 |   LineChart,
9 |   Line,
10 |   XAxis,
11 |   YAxis,
12 |   CartesianGrid,
13 |   Tooltip,
14 |   ResponsiveContainer,
15 |   ReferenceLine,
16 | } from "recharts";
17 | import { cn } from "@lib/utils";
18 | import { format } from "date-fns";
19 |
20 | export interface HealthDataPoint {
21 |   date: string;
22 |   healthScore: number;
23 |   giniCoefficient?: number;
```

```

24 | }
25 |
26 | export interface TeamHealthChartProps {
27 |   data: HealthDataPoint[];
28 |   height?: number;
29 |   className?: string;
30 | }
31 |
32 | const TeamHealthChart = ({
33 |   data,
34 |   height = 250,
35 |   className,
36 | }: TeamHealthChartProps) => {
37 |   const chartData = useMemo(
38 |     () =>
39 |     data.map((d) => ({
40 |       ...d,
41 |       label: format(new Date(d.date), "MMM d"),
42 |     })),
43 |     [data]
44 |   );
45 |
46 |   if (data.length === 0) {
47 |     return (
48 |       <div
49 |         className={cn(
50 |           "flex items-center justify-center text-sm text-gray-400",
51 |           className
52 |         )}
53 |         style={{ height }}
54 |       >
55 |         No health data available yet
56 |       </div>
57 |     );
58 |   }
59 |
60 |   return (
61 |     <div className={cn("w-full", className)}>
62 |       <ResponsiveContainer width="100%" height={height}>
63 |         <LineChart
64 |           data={chartData}
65 |           margin={{ top: 5, right: 20, left: 0, bottom: 5 }}
66 |         >
67 |           <CartesianGrid strokeDasharray="3 3" stroke="#e2e8f0" />
68 |           <XAxis
69 |             dataKey="label"
70 |             tick={{ fontSize: 11, fill: "#94a3b8" }}
71 |             tickLine={false}
72 |             axisLine={{ stroke: "#e2e8f0" }}
73 |           />
74 |           <YAxis
75 |             domain={[0, 100]}
76 |             tick={{ fontSize: 11, fill: "#94a3b8" }}
77 |             tickLine={false}
78 |             axisLine={{ stroke: "#e2e8f0" }}
79 |             width={35}
80 |           />
81 |           <Tooltip
82 |             contentStyle={{
83 |               borderRadius: "0.75rem",
84 |               border: "none",
85 |               boxShadow: "0 4px 6px -1px rgb(0 0 0 / 0.1)",
86 |               fontSize: "12px",
87 |             }}
88 |             formatter={(value) => [`${value}`, "Health Score"]}
89 |             labelFormatter={(label) => String(label)}
90 |           />
91 |           {/* Threshold lines */}
92 |           <ReferenceLine
93 |             y={70}
94 |             stroke="#22c55e"
95 |             strokeDasharray="4 4"
96 |             strokeOpacity={0.5}
97 |             label={{
98 |               value: "Healthy",
99 |               position: "insideTopRight",
100 |               fontSize: 10,
101 |               fill: "#22c55e",
102 |             }}
103 |           />
104 |           <ReferenceLine
105 |             y={40}
106 |             stroke="#ef4444"
107 |             strokeDasharray="4 4"
108 |             strokeOpacity={0.5}
109 |             label={{
110 |               value: "At Risk",
111 |               position: "insideBottomRight",
112 |               fontSize: 10,
113 |               fill: "#ef4444",
114 |             }}
115 |           />
116 |           <Line
117 |             type="monotone"
118 |             dataKey="healthScore"
119 |             stroke="#3b82f6"
120 |             strokeWidth={2.5}
121 |             dot={{ r: 4, fill: "#3b82f6", strokeWidth: 2, stroke: "#fff" }}
122 |             activeDot={{
123 |               r: 6,
124 |               fill: "#3b82f6",
125 |               strokeWidth: 2,
126 |               stroke: "#fff",
127 |             }}

```

```

128 |         animationDuration={600}
129 |       />
130 |     </LineChart>
131 |   </ResponsiveContainer>
132 | </div>
133 | );
134 | };
135 |
136 | export default TeamHealthChart;
137 |

```

FILE: src/components/shared/TeamInvitationCard.tsx

TYPE: .tsx | LINES: 1-94 | SHA256: 79a4c3dad97ca99717bbb5eddb947d89b9862563eff782ff17a595bdf9d1414d

```

1 | // =====
2 | // TeamInvitationCard – Invitation accept/decline with keyboard operability
3 | // Task 4.7: keyboard accessible accept/decline buttons
4 | // =====
5 |
6 | import { Button } from "@components/ui/button";
7 | import { cn } from "@lib/utils";
8 | import { Check, X, Users, Clock, Loader2 } from "lucide-react";
9 | import { formatDistanceToNow } from "date-fns";
10 | import { PCard } from "@design-system";
11 |
12 | export interface TeamInvitationCardProps {
13 |   teamName: string;
14 |   invitedBy: string;
15 |   createdAt: string;
16 |   onAccept: () => void;
17 |   onDecline: () => void;
18 |   isAccepting?: boolean;
19 |   isDeclining?: boolean;
20 |   className?: string;
21 | }
22 |
23 | const TeamInvitationCard = ({
24 |   teamName,
25 |   invitedBy,
26 |   createdAt,
27 |   onAccept,
28 |   onDecline,
29 |   isAccepting = false,
30 |   isDeclining = false,
31 |   className,
32 | }: TeamInvitationCardProps) => {
33 |   const isPending = isAccepting || isDeclining;
34 |
35 |   return (
36 |     <PCard className={cn("p-4", className)}>
37 |       <div className="flex items-start gap-3">
38 |         {/* Team icon */}
39 |         <div className="h-10 w-10 rounded-full bg-blue-100 flex items-center justify-center shrink-0">
40 |           <Users className="h-5 w-5 text-blue-600" />
41 |         </div>
42 |
43 |         <div className="flex-1 min-w-0">
44 |           {/* Title */}
45 |           <p className="text-sm font-semibold">
46 |             Team Invitation: <span className="text-blue-600">{teamName}</span>
47 |           </p>
48 |           <p className="text-xs text-gray-500 mt-0.5">Invited by {invitedBy}</p>
49 |           <div className="flex items-center gap-1 mt-1 text-[10px] text-gray-400">
50 |             <Clock className="h-3 w-3" />
51 |             {formatDistanceToNow(new Date(createdAt), { addSuffix: true })}
52 |           </div>
53 |
54 |           {/* Action buttons */}
55 |           <div className="flex items-center gap-2 mt-3">
56 |             <Button
57 |               size="sm"
58 |               onClick={onAccept}
59 |               disabled={isPending}
60 |               variant="tactile"
61 |               className="gap-1.5"
62 |               aria-label={`Accept invitation to ${teamName}`}
63 |             >
64 |               {isAccepting ? (
65 |                 <Loader2 className="h-4 w-4 animate-spin" />
66 |               ) : (
67 |                 <Check className="h-4 w-4" />
68 |               )}
69 |             <Accept
70 |             </Button>
71 |             <Button
72 |               size="sm"
73 |               variant="outline"
74 |               onClick={onDecline}
75 |               disabled={isPending}
76 |               className="active:scale-95 transition-transform duration-100 gap-1.5"
77 |               aria-label={`Decline invitation to ${teamName}`}
78 |             >
79 |               {isDeclining ? (
80 |                 <Loader2 className="h-4 w-4 animate-spin" />
81 |               ) : (
82 |                 <X className="h-4 w-4" />
83 |               )}
84 |             <Decline
85 |             </Button>
86 |           </div>
87 |         </div>
88 |       </div>
89 |     </PCard>
90 |   );

```

```

91 | };
92 |
93 | export default TeamInvitationCard;
94 |

```

FILE: src/components/shared/TeamLeaderboardView.tsx

TYPE: .tsx | LINES: 1-169 | SHA256: 1a11fc059f86f3f8338976f0eedf5332874587d0f8f2cd1e6b9c6e32cbe528dc

```

1 | // =====
2 | // TeamLeaderboardView – Team leaderboard tab content
3 | // Task 4.6: medals, streak display, course/program scope, Cooperation Score sort
4 | // =====
5 |
6 | import { cn } from "@lib/utils";
7 | import { Badge } from "@components/ui/badge";
8 | import { Button } from "@components/ui/button";
9 | import { Flame, Users, ArrowUpDown } from "lucide-react";
10 | import type {
11 |   TeamLeaderboardEntry,
12 |   TeamLeaderboardSort,
13 |   TeamLeaderboardScope,
14 | } from "@hooks/useTeamLeaderboard";
15 |
16 | export interface TeamLeaderboardViewProps {
17 |   entries: TeamLeaderboardEntry[];
18 |   currentTeamId?: string;
19 |   scope: TeamLeaderboardScope;
20 |   sortBy: TeamLeaderboardSort;
21 |   onSortChange?: (sort: TeamLeaderboardSort) => void;
22 |   onScopeChange?: (scope: TeamLeaderboardScope) => void;
23 |   isLive?: boolean;
24 |   className?: string;
25 | }
26 |
27 | const MEDAL_ICONS: Record<number, string> = {
28 |   1: "🥇",
29 |   2: "🥈",
30 |   3: "🥉",
31 | };
32 |
33 | const TeamLeaderboardView = ({
34 |   entries,
35 |   currentTeamId,
36 |   sortBy,
37 |   onSortChange,
38 |   isLive,
39 |   className,
40 | }: TeamLeaderboardViewProps) => {
41 |   const toggleSort = () => {
42 |     onSortChange?.(sortBy === "xp_total" ? "cooperation_score" : "xp_total");
43 |   };
44 |
45 |   if (entries.length === 0) {
46 |     return (
47 |       <div
48 |         className={cn(
49 |           "flex flex-col items-center justify-center py-12 text-center",
50 |           className
51 |         )}
52 |       >
53 |         <Users className="h-10 w-10 text-gray-300 mb-3" />
54 |         <p className="text-sm font-medium text-gray-500">No teams found</p>
55 |         <p className="text-xs text-gray-400 mt-1">
56 |           Teams will appear here once created
57 |         </p>
58 |       </div>
59 |     );
60 |   }
61 |
62 |   return (
63 |     <div className={cn("space-y-3", className)}>
64 |       <div className="flex items-center justify-between">
65 |         <div className="flex items-center gap-2">
66 |           {isLive && (
67 |             <Badge
68 |               variant="outline"
69 |               className="text-[10px] text-green-600 border-green-300 bg-green-50"
70 |             >
71 |               <span className="h-1.5 w-1.5 rounded-full bg-green-500 animate-pulse me-1" />
72 |               Live
73 |             </Badge>
74 |           )}
75 |         </div>
76 |         <Button
77 |           variant="outline"
78 |           size="sm"
79 |           onClick={toggleSort}
80 |           className="text-xs gap-1.5 active:scale-95 transition-transform duration-100"
81 |         >
82 |           <ArrowUpDown className="h-3.5 w-3.5" />
83 |           {sortBy === "xp_total" ? "Sort by Co-op Score" : "Sort by XP"}
84 |         </Button>
85 |       </div>
86 |
87 |       <div role="list" aria-label="Team leaderboard">
88 |         {/* Leaderboard list */}
89 |         <div role="list" aria-label="Team leaderboard">
90 |           {entries.map((entry) => {
91 |             const isCurrent = entry.team_id === currentTeamId;
92 |             const medal = MEDAL_ICONS[entry.rank];
93 |
94 |             return (
95 |               <div
96 |                 key={entry.team_id}

```



```

97 |         role="listitem"
98 |         className={cn(
99 |           "flex items-center gap-3 rounded-lg px-3 py-2.5 transition-colors",
100 |           isCurrent && "bg-blue-50 ring-1 ring-blue-200",
101 |           !isCurrent && "hover:bg-slate-50"
102 |         )}
103 |       >
104 |         { /* Rank */ }
105 |         <div
106 |           className={cn(
107 |             "h-8 w-8 rounded-full flex items-center justify-center shrink-0 text-sm font-bold",
108 |             entry.rank === 1 && "bg-yellow-50 text-yellow-700",
109 |             entry.rank === 2 && "bg-gray-50 text-gray-600",
110 |             entry.rank === 3 && "bg-amber-50 text-amber-700",
111 |             entry.rank > 3 && "bg-gray-100 text-gray-500"
112 |           )}
113 |         >
114 |           {medal ?? entry.rank}
115 |         </div>
116 |
117 |         { /* Avatar + Name */ }
118 |         <div className="h-9 w-9 rounded-full bg-blue-100 flex items-center justify-center text-sm font-b
119 |           {entry.avatar_letter}
120 |         </div>
121 |         <div className="flex-1 min-w-0">
122 |           <div className="flex items-center gap-1.5">
123 |             <span
124 |               className={cn(
125 |                 "text-sm font-medium truncate",
126 |                 isCurrent && "font-bold text-blue-700"
127 |               )}
128 |             >
129 |               {entry.team_name}
130 |             </span>
131 |             {isCurrent && (
132 |               <span className="text-[10px] text-blue-500">
133 |                 (Your Team)
134 |               </span>
135 |             )}
136 |           </div>
137 |           <div className="flex items-center gap-3 mt-0.5">
138 |             <span className="text-xs text-gray-500 flex items-center gap-0.5">
139 |               <Users className="h-3 w-3" /> {entry.member_count}
140 |             </span>
141 |             {entry.streak_count > 0 && (
142 |               <span className="text-xs text-orange-500 flex items-center gap-0.5">
143 |                 <Flame className="h-3 w-3" /> {entry.streak_count}d
144 |               </span>
145 |             )}
146 |           </div>
147 |         </div>
148 |
149 |         { /* Stats */ }
150 |         <div className="text-end shrink-0">
151 |           <p className="text-sm font-bold tabular-nums">
152 |             {sortBy === "xp_total"
153 |               ? `${entry.xp_total.toLocaleString()} XP`
154 |               : `${entry.cooperation_score}`}
155 |           </p>
156 |           <p className="text-[10px] text-gray-400">
157 |             {sortBy === "xp_total" ? "Team XP" : "Co-op Score"}
158 |           </p>
159 |         </div>
160 |       </div>
161 |     );
162 |   }
163 | }
164 | </div>
165 | );
166 | };
167 |
168 | export default TeamLeaderboardView;
169 |

```

FILE: [src/components/shared/TeamMemberList.tsx](#)

TYPE: .tsx | LINES: 1-118 | SHA256: ad5434bd21f9f72786e879759aa9a58b9e3d714b0187fb543dd91da88a8234eb

```

1 | // =====
2 | // TeamMemberList - Member list with roles, XP contributions, contribution status
3 | // Task 4.2: captain badge, XP contributions, contribution status indicators
4 | // =====
5 |
6 | import { cn } from "@lib/utills";
7 | import { Badge } from "@components/ui/badge";
8 | import { Crown, User } from "lucide-react";
9 | import ContributionStatusBadge from "@components/shared/ContributionStatusBadge";
10 | import type { ContributionStatus } from "@lib/contributionThresholds";
11 |
12 | export interface TeamMemberItem {
13 |   id: string;
14 |   studentId: string;
15 |   displayName: string;
16 |   avatarUrl?: string | null;
17 |   role: "captain" | "member";
18 |   xpContribution: number;
19 |   contributionStatus: ContributionStatus;
20 | }
21 |
22 | export interface TeamMemberListProps {
23 |   members: TeamMemberItem[];
24 |   totalTeamXp?: number;
25 |   className?: string;
26 | }
27 |

```

```

28 | const TeamMemberList = ({
29 |   members,
30 |   totalTeamXp = 0,
31 |   className,
32 | }: TeamMemberListProps) => {
33 |   const sorted = [...members].sort((a, b) => {
34 |     // Captain first, then by XP descending
35 |     if (a.role === "captain" && b.role !== "captain") return -1;
36 |     if (a.role !== "captain" && b.role === "captain") return 1;
37 |     return b.xpContribution - a.xpContribution;
38 |   });
39 |
40 |   return (
41 |     <div
42 |       className={cn("space-y-2", className)}
43 |       role="list"
44 |       aria-label="Team members"
45 |     >
46 |       {sorted.map((member) => {
47 |         const pct =
48 |           totalTeamXp > 0
49 |             ? Math.round((member.xpContribution / totalTeamXp) * 100)
50 |             : 0;
51 |
52 |         return (
53 |           <div
54 |             key={member.id}
55 |             role="listitem"
56 |             className="flex items-center gap-3 rounded-lg p-2 hover:bg-slate-50 transition-colors"
57 |           >
58 |             {/* Avatar */}
59 |             <div className="h-8 w-8 rounded-full bg-blue-100 flex items-center justify-center shrink-0">
60 |               {member.avatarUrl ? (
61 |                 <img
62 |                   src={member.avatarUrl}
63 |                   alt=""
64 |                   loading="lazy"
65 |                   decoding="async"
66 |                   className="h-8 w-8 rounded-full object-cover"
67 |                 />
68 |               ) : (
69 |                 <User className="h-4 w-4 text-blue-600" />
70 |               )}
71 |             </div>
72 |
73 |             {/* Name + Role */}
74 |             <div className="flex-1 min-w-0">
75 |               <div className="flex items-center gap-1.5">
76 |                 <span className="text-sm font-medium truncate">
77 |                   {member.displayName}
78 |                 </span>
79 |                 {member.role === "captain" && (
80 |                   <Badge
81 |                     variant="outline"
82 |                     className="text-[10px] font-bold text-amber-600 border-amber-300 bg-amber-50 gap-0.5 px-1."
83 |                     aria-label="Team captain"
84 |                   >
85 |                     <Crown className="h-3 w-3" />
86 |                     Captain
87 |                   </Badge>
88 |                 )}
89 |               </div>
90 |             {/* XP contribution bar */}
91 |             <div className="flex items-center gap-2 mt-1">
92 |               <div className="flex-1 h-1.5 rounded-full bg-gray-100 overflow-hidden">
93 |                 <div
94 |                   className="h-full rounded-full bg-amber-400 transition-all duration-300"
95 |                   style={{ width: `${Math.min(pct, 100)}%` }}
96 |                 />
97 |               </div>
98 |               <span className="text-[10px] font-medium text-gray-500 tabular-nums w-12 text-end">
99 |                 {member.xpContribution} XP
100 |               </span>
101 |             </div>
102 |           </div>
103 |
104 |           {/* Contribution Status */}
105 |           <ContributionStatusBadge status={member.contributionStatus} />
106 |         </div>
107 |       )};
108 |     </div>
109 |   );
110 |   {members.length === 0 && (
111 |     <p className="text-xs text-gray-400 text-center py-4">No members yet</p>
112 |   )}
113 | </div>
114 | );
115 | };
116 |
117 | export default TeamMemberList;
118 |

```